

ডুহু সএমএস Flutter অ্যাপ্লিকেশন — সম্পূর্ণ কোড ডকুমেন্টেশন

ZuhooCMS Flutter Application — Complete Technical Reference
(বাংলা)

প্রস্তুত করে তারিখ: 30 August 2026
প্রতিটি ফাইল, ক্লাস এবং মেথড বাংলা ভাষায় ব্যাখ্যা করা হয়েছে।
মোট ফাইল কভার করা হয়েছে: 276 (আনুমানিক)

সূচিপত্র (Table of Contents)

1. lib/app/app.dart
2. lib/app/app_drawer.dart
3. lib/app/demo_banner.dart
4. lib/app/impersonation_banner.dart
5. lib/app/nav_registry.dart
6. lib/app/portal_shell.dart
7. lib/app/router.dart
8. lib/app/shell.dart
9. lib/app/splash_screen.dart
10. lib/core/auth/auth_controller.dart
11. lib/core/auth/auth_models.dart
12. lib/core/auth/auth_repository.dart
13. lib/core/auth/impersonation_controller.dart
14. lib/core/auth/permission_controller.dart
15. lib/core/config/env.dart
16. lib/core/network/api_client.dart
17. lib/core/network/api_exception.dart
18. lib/core/network/auth_interceptor.dart
19. lib/core/network/paged_response.dart
20. lib/core/network/uploaded_file.dart
21. lib/core/providers.dart
22. lib/core/push/push_service.dart
23. lib/core/storage/secure_store.dart
24. lib/core/theme/accents.dart
25. lib/core/theme/app_theme.dart
26. lib/core/theme/bos_tokens.dart
27. lib/core/theme/theme_controller.dart
28. lib/features/accounting/accounting_models.dart
29. lib/features/accounting/accounting_repository.dart
30. lib/features/accounting/accounting_screen.dart
31. lib/features/accounting/accounting_sheets.dart
32. lib/features/accounting/reconciliation_models.dart
33. lib/features/accounting/reconciliation_screen.dart
34. lib/features/admin/accounts_screen.dart
35. lib/features/admin/admin_form_sheets.dart
36. lib/features/admin/admin_models.dart
37. lib/features/admin/admin_repository.dart
38. lib/features/admin/admin_screen.dart
39. lib/features/admin/role_permissions_screen.dart
40. lib/features/ai/ai_admin_models.dart
41. lib/features/ai/ai_chat_screen.dart
42. lib/features/ai/ai_drafts_screen.dart

43. lib/features/ai/ai_models.dart
44. lib/features/ai/ai_repository.dart
45. lib/features/ai/ai_screen.dart
46. lib/features/ai/ai_settings_screen.dart
47. lib/features/alerts/alerts_screen.dart
48. lib/features/alerts/notification_preferences_screen.dart
49. lib/features/alerts/notification_repository.dart
50. lib/features/assets_periods/assets_periods_models.dart
51. lib/features/assets_periods/assets_periods_repository.dart
52. lib/features/assets_periods/assets_periods_screen.dart
53. lib/features/attendance/assign_shift_sheet.dart
54. lib/features/attendance/attendance_admin_repository.dart
55. lib/features/attendance/attendance_controller.dart
56. lib/features/attendance/attendance_models.dart
57. lib/features/attendance/attendance_repository.dart
58. lib/features/attendance/attendance_screen.dart
59. lib/features/attendance/employee_attendance_screen.dart
60. lib/features/attendance/punch_card.dart
61. lib/features/attendance/shift_roster_screen.dart
62. lib/features/attendance/submit_timesheet_sheet.dart
63. lib/features/attendance/team_attendance_screen.dart
64. lib/features/attendance/timesheets_tab.dart
65. lib/features/auth/forgot_password_screen.dart
66. lib/features/auth/login_screen.dart
67. lib/features/auth/register_screen.dart
68. lib/features/auth/verify_email_screen.dart
69. lib/features/biometric/biometric_models.dart
70. lib/features/biometric/biometric_repository.dart
71. lib/features/biometric/biometric_screen.dart
72. lib/features/biometric/biometric_sheets.dart
73. lib/features/budgets/budget_models.dart
74. lib/features/budgets/budget_repository.dart
75. lib/features/budgets/budget_screen.dart
76. lib/features/catalogue/catalogue_models.dart
77. lib/features/catalogue/catalogue_repository.dart
78. lib/features/catalogue/catalogue_form_sheets.dart
79. lib/features/catalogue/catalogue_screen.dart
80. lib/features/company/company_models.dart
81. lib/features/company/company_repository.dart
82. lib/features/company/company_screen.dart
83. lib/features/company/public_company_screen.dart
84. lib/features/company/subscription_plan_screen.dart
85. lib/features/crm/activity_timeline.dart
86. lib/features/crm/client_form_sheet.dart
87. lib/features/crm/clients_tab.dart

88. lib/features/crm/contacts_screen.dart
89. lib/features/crm/crm_controllers.dart
90. lib/features/crm/crm_models.dart
91. lib/features/crm/crm_reports_screen.dart
92. lib/features/crm/crm_repository.dart
93. lib/features/crm/crm_screen.dart
94. lib/features/crm/edit_lead_sheet.dart
95. lib/features/crm/lead_detail_screen.dart
96. lib/features/crm/lead_filter_sheet.dart
97. lib/features/crm/leads_tab.dart
98. lib/features/crm/opportunity_detail_screen.dart
99. lib/features/crm/opportunity_form_sheet.dart
100. lib/features/crm/pipeline_tab.dart
101. lib/features/dashboard/dashboard_models.dart
102. lib/features/dashboard/dashboard_repository.dart
103. lib/features/dashboard/dashboard_screen.dart
104. lib/features/dashboard/platform_dashboard_screen.dart
105. lib/features/directory/directory_models.dart
106. lib/features/directory/directory_repository.dart
107. lib/features/directory/directory_screen.dart
108. lib/features/directory/education_section.dart
109. lib/features/directory/employee_form_sheet.dart
110. lib/features/directory/person_detail_screen.dart
111. lib/features/finance/edit_expense_sheet.dart
112. lib/features/finance/expenses_tab.dart
113. lib/features/finance/finance_models.dart
114. lib/features/finance/finance_overview_tab.dart
115. lib/features/finance/finance_repository.dart
116. lib/features/finance/finance_screen.dart
117. lib/features/finance/invoice_extras.dart
118. lib/features/finance/invoice_form_sheet.dart
119. lib/features/finance/record_payment_sheet.dart
120. lib/features/finance/submit_expense_sheet.dart
121. lib/features/finance/vendor_expenses_screen.dart
122. lib/features/home/home_repository.dart
123. lib/features/home/home_screen.dart
124. lib/features/hrpolicy/hrpolicy_models.dart
125. lib/features/hrpolicy/hrpolicy_repository.dart
126. lib/features/hrpolicy/hrpolicy_screen.dart
127. lib/features/hrpolicy/hrpolicy_sheets.dart
128. lib/features/itam/asset_detail_screen.dart
129. lib/features/itam/asset_form_sheet.dart
130. lib/features/itam/asset_history_screen.dart
131. lib/features/itam/itam_models.dart
132. lib/features/itam/itam_repository.dart

133. lib/features/itam/itam_screen.dart
134. lib/features/itam/license_form_sheet.dart
135. lib/features/itam/offboarding_detail_screen.dart
136. lib/features/itam/software_detail_screen.dart
137. lib/features/itam/start_offboarding_sheet.dart
138. lib/features/kb/kb_authoring_screen.dart
139. lib/features/kb/kb_models.dart
140. lib/features/kb/kb_repository.dart
141. lib/features/kb/kb_screen.dart
142. lib/features/leave/apply_leave_sheet.dart
143. lib/features/leave/entitlements_screen.dart
144. lib/features/leave/grant_leave_balance_sheet.dart
145. lib/features/leave/leave_models.dart
146. lib/features/leave/leave_repository.dart
147. lib/features/leave/leave_approvals_tab.dart
148. lib/features/leave/leave_screen.dart
149. lib/features/payables/payables_models.dart
150. lib/features/payables/payables_repository.dart
151. lib/features/payables/payables_screen.dart
152. lib/features/payables/payables_sheets.dart
153. lib/features/payroll/payroll_models.dart
154. lib/features/payroll/payroll_repository.dart
155. lib/features/payroll/payroll_screen.dart
156. lib/features/payroll/payroll_sheets.dart
157. lib/features/payroll/salary_sheet_models.dart
158. lib/features/payroll/salary_sheet_screen.dart
159. lib/features/payslips/create_payroll_sheet.dart
160. lib/features/payslips/payslip_models.dart
161. lib/features/payslips/payslip_repository.dart
162. lib/features/payslips/payslips_screen.dart
163. lib/features/performance/performance_models.dart
164. lib/features/performance/performance_repository.dart
165. lib/features/performance/performance_screen.dart
166. lib/features/performance/review_detail_screen.dart
167. lib/features/performance/review_form_sheet.dart
168. lib/features/platform/context_models.dart
169. lib/features/platform/context_repository.dart
170. lib/features/platform/context_screen.dart
171. lib/features/platform/context_switch_detail.dart
172. lib/features/platform/location_models.dart
173. lib/features/platform/location_repository.dart
174. lib/features/platform/locations_screen.dart
175. lib/features/platform/plans_screen.dart
176. lib/features/platform/platform_create_sheets.dart
177. lib/features/platform/platform_edit_sheets.dart

178. lib/features/platform/platform_expenses_tab.dart
179. lib/features/platform/platform_models.dart
180. lib/features/platform/platform_repository.dart
181. lib/features/platform/platform_screen.dart
182. lib/features/portal/edit_portal_profile_sheet.dart
183. lib/features/portal/portal_billing_screen.dart
184. lib/features/portal/portal_home_screen.dart
185. lib/features/portal/portal_models.dart
186. lib/features/portal/portal_profile_screen.dart
187. lib/features/portal/portal_repository.dart
188. lib/features/portal/portal_requests_screen.dart
189. lib/features/portal/portal_tickets_screen.dart
190. lib/features/profile/appearance_screen.dart
191. lib/features/profile/change_email_screen.dart
192. lib/features/profile/change_password_screen.dart
193. lib/features/profile/edit_profile_screen.dart
194. lib/features/profile/employee_models.dart
195. lib/features/profile/employee_repository.dart
196. lib/features/profile/profile_screen.dart
197. lib/features/receivables/receivables_models.dart
198. lib/features/receivables/receivables_repository.dart
199. lib/features/receivables/receivables_screen.dart
200. lib/features/receivables/receivables_sheets.dart
201. lib/features/recruitment/application_detail_screen.dart
202. lib/features/recruitment/candidate_detail_screen.dart
203. lib/features/recruitment/candidate_form_sheet.dart
204. lib/features/recruitment/career_page_screen.dart
205. lib/features/recruitment/create_offer_sheet.dart
206. lib/features/recruitment/interview_feedback_sheet.dart
207. lib/features/recruitment/job_posting_form_sheet.dart
208. lib/features/recruitment/recruitment_kpi_screen.dart
209. lib/features/recruitment/recruitment_models.dart
210. lib/features/recruitment/recruitment_repository.dart
211. lib/features/recruitment/recruitment_screen.dart
212. lib/features/recruitment/refer_candidate_sheet.dart
213. lib/features/recruitment/schedule_interview_sheet.dart
214. lib/features/reports/report_models.dart
215. lib/features/reports/report_repository.dart
216. lib/features/reports/report_screen.dart
217. lib/features/requests/approvals_tab.dart
218. lib/features/requests/edit_request_sheet.dart
219. lib/features/requests/new_package_sheet.dart
220. lib/features/requests/raise_request_sheet.dart
221. lib/features/requests/rate_request_sheet.dart
222. lib/features/requests/request_card.dart

223. lib/features/requests/request_controllers.dart
224. lib/features/requests/request_detail_screen.dart
225. lib/features/requests/request_models.dart
226. lib/features/requests/request_repository.dart
227. lib/features/requests/request_workflow_models.dart
228. lib/features/requests/request_workflow_screen.dart
229. lib/features/requests/requests_screen.dart
230. lib/features/requests/reviews_screen.dart
231. lib/features/salary/salary_models.dart
232. lib/features/salary/salary_repository.dart
233. lib/features/salary/salary_screen.dart
234. lib/features/salary/salary_sheets.dart
235. lib/features/salary/structure_extras_screen.dart
236. lib/features/salary/template_models.dart
237. lib/features/salary/template_tab.dart
238. lib/features/search/search_models.dart
239. lib/features/search/search_repository.dart
240. lib/features/search/search_screen.dart
241. lib/features/support/chat_thread.dart
242. lib/features/support/edit_ticket_sheet.dart
243. lib/features/support/internal_notes_screen.dart
244. lib/features/support/my_agent_card.dart
245. lib/features/support/new_ticket_sheet.dart
246. lib/features/support/support_controllers.dart
247. lib/features/support/support_models.dart
248. lib/features/support/support_queue_screen.dart
249. lib/features/support/support_repository.dart
250. lib/features/support/support_screen.dart
251. lib/features/support/ticket_card.dart
252. lib/features/support/ticket_detail_screen.dart
253. lib/features/support_admin/audit_detail_screen.dart
254. lib/features/support_admin/support_admin_models.dart
255. lib/features/support_admin/support_admin_repository.dart
256. lib/features/support_admin/support_admin_screen.dart
257. lib/features/support_admin/support_admin_sheets.dart
258. lib/features/workflow/workflow_models.dart
259. lib/features/workflow/workflow_repository.dart
260. lib/features/workflow/workflow_screen.dart
261. lib/features/workflow/workflow_sheets.dart
262. lib/main.dart
263. lib/shared/paged_controller.dart
264. lib/shared/util/formatters.dart
265. lib/shared/widgets/attachment_picker.dart
266. lib/shared/widgets/brand_mark.dart
267. lib/shared/widgets/config_list.dart

268. lib/shared/widgets/date_field.dart
269. lib/shared/widgets/employee_picker.dart
270. lib/shared/widgets/form_sheet.dart
271. lib/shared/widgets/paged_list_view.dart
272. lib/shared/widgets/primitives.dart
273. lib/shared/widgets/prompts.dart
274. lib/shared/widgets/search_field.dart
275. lib/shared/widgets/skeleton.dart
276. lib/shared/widgets/stat_card.dart

ফাইল: lib/app/app.dart

সংক্ষিপ্ত ববিরণ: এই ফাইলটি পুরো অ্যাপ্লিকেশনের রুট widget **ZuhooApp** ডিফাইন করে, যা থিম স্টেআপ, রাউটার কনফিগারেশন, পুশ নোটিফিকেশন ইনশিয়ালাইজেশন এবং ডেমেও/ইমপারসোনেশন ব্যানার স্কোপ একসাথে যুক্ত করে **MaterialApp.router** তৈরি করে।

ক্লাস/উইজটে: ZuhooApp (extends ConsumerWidget)

এই ক্লাসটি অ্যাপের সবচেয়ে উপরে স্তরে widget, যা Riverpod-এর মাধ্যমে থিম স্টেইংস ও রাউটার প্রোভাইডার watch করে এবং পুরো অ্যাপের **MaterialApp.router** ইনস্ট্যান্স তৈরিকার দায়িত্ব পালন করে। এটি এমনভাবে সাজানো যাত ডেমেও সেশন ও ইমপারসোনেশন সেশনের রমাইন্ডার ব্যানারগুলো রাউটারের উপরে বসে, ফলে নভিগেশন করলেও সেগুলো সরে না যায়।

Widget build(BuildContext context, WidgetRef ref)

এই মথোডটি অ্যাপের প্রধান widget ট্রি তৈরি করে। প্রথমে এটি **themeControllerProvider** watch করে বর্তমান থিম স্টেইংস (accent color, theme mode) নিয়ে। এরপর **pushInitProvider** কে watch করে (কিন্তু await না করে) পুশ নোটিফিকেশন সার্ভিস চালু করে — এটি ইচ্ছাকৃতভাবে ব্লকিং না করে শুরু করা হয়েছে, কারণ প্রথম ফ্রমে রেন্ডার হওয়া পারমিশন ডায়ালগের জন্য আটকে থাকা উচিত নয়। এরপর **MaterialApp.router** রিটার্ন করা হয়, যেখানে **theme** ও **darkTheme** উভয়ই accent কালার দিয়ে তৈরি করা হয় (**AppTheme.light/AppTheme.dark**) এবং **themeMode** অনুযায়ী Flutter নজিই সদিধানত নিয়ে কোনটা দেখাবে — এভাবে "System" মোড এবং সূর্যাস্তের সময় থিম পরিবর্তনও রপিইন্ট হিসেবেই কাজ করে, রিস্টার্ট ছাড়াই। **routerConfig** হিসেবে **routerProvider** watch করা হয়। সবশেষে **builder** প্যারামিটারে child widget-কে **ImpersonationScope** এর ভেতরে **DemoScope** দিয়ে মোড়ানো হয় — যেহেতু একটি সেশন একই সাথে ডেমেও ও ইমপারসোনেশন উভয়ই হতে পারে না, তাই এই দুইয়ের মধ্যে সবসময় শুধু একটিই সক্রিয়ভাবে কিছু আঁকে।

ফাইল: lib/app/app_drawer.dart

সংক্ষিপ্ত ববিরণ: এই ফাইলটি অ্যাপের সাইড ড্রয়ার (**AppDrawer**) ডিফাইন করে, যা ওয়েব অ্যাপের সাইডবারের সমতুল্য — সব মডউলারে নভিগেশন লিংক পারমিশন অনুযায়ী দেখায়, সার্চযোগ্য, এবং ব্যবহারকারীর ভূমিকা অনুযায়ী গ্রুপড অ্যাকর্ডয়িন বা ফ্ল্যাট লিস্ট হিসেবে রেন্ডার করে।

ক্লাস/উইজটে: AppDrawer (extends ConsumerStatefulWidget)

এই ক্লাসটি **nav_registry.dart**-এর **buildNavGroups** থেকে ডেটা নিয়ে অ্যাপের সম্পূর্ণ নভিগেশন ড্রয়ার তৈরি করে, যাত ড্রয়ার এবং হোম স্ক্রিনের Quick Actions একই সোর্স থেকে ডেটা পায় এবং কখনো একে অপর থেকে আলাদা হয়ে না যায়। কোম্পানিওনার বা প্ল্যাটফর্ম স্টাফের জন্য গ্রুপড অ্যাকর্ডয়িন, অন্যদরে জন্য ফ্ল্যাট লিস্ট দেখায় এবং সার্চ করলে গ্রুপিং বাদ দিয়ে ম্যাচিং আইটেমগুলো ফ্ল্যাটভাবে দেখায়।

ConsumerState<AppDrawer> createState()

এই মথোডটি **AppDrawer** widget-এর জন্য **_AppDrawerState** স্টেটে অবজেক্ট তৈরি করে রিটার্ন করে — এটি Flutter-এর **StatefulWidget** প্যাটার্নের আদর্শ ওভাররাইড, কোনো কাস্টম লজিক নেই।

ক্লাস: _AppDrawerState (extends ConsumerState<AppDrawer>)

এটি **AppDrawer**-এর স্টেটে ক্লাস, যা সার্চ কোয়েরি স্টেটে (**_query**) ধরে রাখে এবং ড্রয়ারের UI তৈরি, সার্চ ফিল্টারিং, নভিগেশন ও গ্রুপ/আইটেম রেন্ডারিং-এর সব লজিক ধারণ করে।

`void _open(NavDestination item)`

এই মেথডটি ব্যবহারকারী কোনো নির্দিষ্ট নভেগেশন আইটেমে ট্যাপ করলে ডাকা হয়। এটি প্রথম `Navigator.of(context).pop()` দিয়ে ড্রয়ার বন্ধ করে, তারপর আইটেমে `push` ফ্ল্যাগ অনুযায়ী `context.push` (নতুন স্ট্যাক) বা `context.go` (ব্রাউচ পরিবর্তন) কল করে, এবং `item.initialTabLabel` কে `extra` হিসেবে পাঠায় যাত্রে গন্তব্য স্ক্রিনি সঠিকি ট্যাবে খোলে।

`void _go(String path)`

এই মেথডটি একটি সরল পাথে নভেগেটে করার জন্য ব্যবহৃত হয় (যেমন অ্যাকাউন্ট বার বা আপগ্রুডে বাটন থেকে)। এটি ড্রয়ার বন্ধ করে এবং `context.push(path)` কল করে — এখানে কোনো `initialTabLabel` বা `push/go` সন্ধিধান্ত নই কারণ এই কলগুলো সবসময় `push`-ভিত্তিক গন্তব্য।

`Widget build(BuildContext context)`

এই মেথডটি পুরো ড্রয়ারের উইজেটে ট্রাইবেরিকর। এটি `currentUserProvider` ও `permissionControllerProvider` watch করে বর্তমান ইউজার ও পারমিশন স্টেটে নেয়, তারপর `buildNavGroups(user, perms)` কল করে গেটেডে গ্রুপ লিস্ট বানায় এবং `showGroupedNav(user)` দিয়ে ঠিকি করে অ্যাকর্ডিয়ন দেখাবে নাকি ফ্ল্যাট লিস্ট। বর্তমান রুট লোকেশন `GoRouterState.of(context).matchedLocation` থাকে নেওয়া হয়, যা হাইলাইটিং-এর জন্য ব্যবহৃত হয়। যদি `_query` খালি না থাকে তবে `_search` কল করে ম্যাচিং আইটেমে বের করা হয়। রিটার্ন করা `Drawer` widget-এ থাকে: হেডার (`_Header`), সার্চ ফিল্ড (৮০ মিলিসেকেন্ড `debounce` সহ, কারণ এখানে কোনো নটেওয়ার্ক কল নই), তারপর সার্চ রিজোল্ট অথবা গ্রুপড/ফ্ল্যাট লিস্ট, কোম্পানি ওনারদের জন্য একটি "Upgrade plan" বাটন, এবং সবশেষে অ্যাকাউন্ট অ্যাকশন বার (`_AccountBar`)।

`List<NavDestination> _search(List<NavGroup> groups, String query)`

এই মেথডটি সব গ্রুপের সব আইটেমে মধ্য দিয়ে লুপ করে এবং যেকোন আইটেমে লবেলে বা তার গ্রুপের লবেলে কোয়েরি স্ট্রিং (case-insensitive) কনটইন করে, সেগুলোর একটি ফ্ল্যাট লিস্ট রিটার্ন করে। গ্রুপ লবেলের বর্নিত্বও ম্যাচ করার কারণ হলে, যেমন "finance" টাইপ করলে Finance গ্রুপের ভেতরে আইটেমগুলো পাওয়া উচিত, এমনকি সেগুলোর নামে "finance" শব্দ না থাকলেও।

`Widget _results(BosPalette bos, List<NavDestination> matches, String location)`

এই মেথডটি সার্চ রিজোল্ট রেন্ডার করে। যদি `matches` খালি হয়, তাহলে একটি `EmptyState` widget দেখানো হয় যাত্রে বলা থাকে কিছু মিলেনি এবং একটি পিরামিডমূলক বার্তা থাকে। অন্যথায়, প্রতিটি ম্যাচড আইটেমে জন্য `_item` কল করে একটি ফ্ল্যাট `ListView` তৈরি করা হয় (কোনো নস্টেটি ছাড়া)।

`Widget _group(BosPalette bos, NavGroup group, String location)`

এই মেথডটি একটি নভেগেশন গ্রুপ রেন্ডার করে। যদি গ্রুপে শুধু একটি আইটেম থাকে, তাহলে সেটিকে সরাসরি লিংক হিসেবে (`_item`) দেখানো হয়, একটি এক-আইটেমের অ্যাকর্ডিয়ন হিসেবে না — এটি Angular সাইডবারের একই নথি অনুসরণ করে। অন্যথায় একটি `ExpansionTile` তৈরি হয়, যেখানে `holdsCurrent` (বর্তমান লোকেশন এই গ্রুপের কোনো আইটেমের পাথে সাথে মিলে কনি) নির্ধারণ করে গ্রুপটি প্রাথমিকভাবে প্রসারিত থাকবে কনি এবং আইকন/লবেলের রঙ কমন হবে। ভেতরে প্রতিটি আইটেমে `_item` দিয়ে `nested: true` সহ রেন্ডার করা হয়।

Widget _item(BosPalette bos, NavDestination item, String location, {required bool nested})

এই মথডটি একটি একক নভেগেশন আইটেম রেন্ডার করে — আইকন, লবেল, এবং বর্তমান নর্বিচি অবস্থার হাইলাইট সহ। **current** নর্বিধারি হয় `item.path == location && item.initialTabLabel == null` শর্তে — অর্থাৎ, একই পথ শেয়ার করা একাধিক ট্যাব-ভিত্তিক আইটেমে (যেমন ITAM-এর Hardware/Software/Offboarding, যগুলো সবই `/assets`) কখনোই হাইলাইট হয় না, কারণ বুট কোন ট্যাব দেখানো হচ্ছে তা রেকর্ড করে না — শুধু গ্রুপ হডোর হাইলাইট হয়। **Material** ও **InkWell** ব্যবহার করে ট্যাপযোগ্য এলাকা তৈরি হয়, যার `onTap` কলব্যাক `_open(item)` ডাকে। **nested** প্রারামিটার আইটেমে ইনডেন্টেশন নর্বিধারণ করে।

Widget _upgrade(BosPalette bos)

এই মথডটি একটি "Upgrade plan" আউটলাইনড বাটন রিটার্ন করে, যা শুধুমাত্র কোম্পানি ওনারদের জন্য দেখানো হয়। ট্যাপ করলে `_go(Routes.subscriptionPlan)` কল হয়।

ক্লাস: **_Header (extends StatelessWidget)**

এই ক্লাসটি ড্রয়ারে একদম উপরে ব্যবহারকারীর পরিচিতি দেখানোর widget — কে সাইন-ইন করা আছে এবং কী হিসেবে।

Widget build(BuildContext context)

এই মথডটি ব্যবহারকারীর অ্যাকাউন্ট (বা কোনো user না থাকলে **BrandMark**), নাম, ভূমিকা লবেল এবং ইমেইল দেখায়। যদি **user** null হয়, তাহলে নামের বদলে "Zuhoo" দেখানো হয় এবং শুধু ব্র্যান্ড আইকন থাকে। যদি ইউজার থাকে, তাহলে তার `initials/profileImageUrl` দিয়ে **Avatar** widget তৈরি হয় এবং পূর্ণ নাম, `roleLabel`, ও `email` টেক্সট আকারে দেখানো হয় — প্রতিটি লাইন এক লাইনে সীমাবদ্ধ ও ellipsis দিয়ে overflow হ্যান্ডল করা হয়।

ক্লাস: **_AccountBar (extends StatelessWidget)**

এই ক্লাসটি ব্যক্তিগত অ্যাকাউন্ট-সম্পর্কিত অ্যাকশনগুলো (প্রোফাইল, অ্যালার্ট, পাসওয়ার্ড, ইমেইল পরিবর্তন) দেখায়, যগুলো ইচ্ছাকৃতভাবে **buildNavGroups**-এর মডুিল টবেলি রাখা হয়নি, কারণ সেগুলো মডুিল-লিস্ট, সেশন-অ্যাকশন লিস্ট নয়।

Widget build(BuildContext context)

এই মথডটি চারটি অ্যাকশন আইকন (Profile, Alerts, Password, Email) একটি সমানভাবে বন্ডিত সারিতে (Row with `spaceAround`) সাজায়, প্রতিটি `_action` হলেপার দিয়ে তৈরি করে এবং সংশ্লিষ্ট বুটে (`Routes.profile`, `Routes.alerts`, `Routes.changePassword`, `Routes.changeEmail`) নভেগিটে করার জন্য `onGo` কলব্যাক ব্যবহার করে।

Widget _action(BosPalette bos, IconData icon, String label, VoidCallback onTap)

এই হলেপার মথডটি একটি একক অ্যাকশন বাটন তৈরি করে — একটি আইকন, তার নচি একটি লবেল, **InkWell** দিয়ে মোড়ানো যাতে ট্যাপ করা যায়। উল্লম্ব প্রাঙ্গণ এমনভাবে সেট করা হয়েছে যাতে মোট উচ্চতা ৪৮ পিক্সলে হয়, যা ট্যাপযোগ্য এলাকার সর্বনম্ন মান হিসেবে নেওয়া হয়েছে।

ফাইল: **lib/app/demo_banner.dart**

সংক্ষিপ্ত ববিরণ: এই ফাইলটি রিডি-অনলি ডিমে সেশন চলাকালীন স্ক্রিনিরে উপরে একটি স্থায়ী রমাইন্ডার স্ট্রপি দেখায়, যাতে ব্যবহারকারী বুঝতে পারে সে ডিমে মোডে আছে (যদিও অ্যাকাউন্ট, টোকেন ও UI সবকিছুই স্বাভাবিক অ্যাকাউন্টের মতোই দেখায়, শুধু সবে কাজ করে না)।

ক্লাস/উইজটে: DemoScope (extends ConsumerWidget)

এই ক্লাসটি ইমপারসোনেশন ব্যানারের মতোই একই স্লট ব্যবহার করে — রাউটারের উপরে বসে, যাতনে ভেগিশেনের পরও এটি টিকে থাকে। এটি পুরো অ্যাপকে মোড়ায় এবং ডেমো সেশন চলাকালীন একটি ব্যানার স্ট্রুপি দেখায়।

Widget build(BuildContext context, WidgetRef ref)

এই মথডটি demoSessionProvider watch করে ডেমো সেশন সক্রিয় কনি যাচাই করে। যদি ডেমো সেশন না থাকে, তাহলে সরাসরি child রটাইন করা হয় (কোনো ব্যানার আঁকা হয় না)। যদি ডেমো সেশন সক্রিয় থাকে, তাহলে একটি Material widget-এ _Banner ও child-কে একটি Column-এ সাজানো হয়।

MediaQuery.removePadding(removeTop: true) ব্যবহার করা হয় কারণ ব্যানার স্ট্রুপি ইতিমধ্যে স্ট্যাটাস-বার ইনস্টে নথি নথিছে, তাই এটি না করলে নচির Scaffold আবার সেই ইনস্টে যোগ করে একটি অব্যবহৃত ফাঁকা ব্যান্ড তৈরিকরত।

ক্লাস: _Banner (extends ConsumerStatefulWidget)

এই ক্লাসটি ডেমো ব্যানারের স্টেটেফুল অংশ — এর ভেতরে "Exit" বাটনের লোডিং স্টেটে থাকার প্রয়োজন বলে এটি স্টেটেফুল।

ConsumerState<_Banner> createState()

_BannerState তৈরিকরে রটাইন করে, স্ট্যান্ডার্ড Flutter ওভাররাইড।

ক্লাস: _BannerState (extends ConsumerState<_Banner>)

এই ক্লাসটি ডেমো ব্যানারের UI ও "Exit" অ্যাকশনের লজিক ধারণ করে, _leaving বুলিয়ান ফ্ল্যাগ দয়ি বাটনের লোডিং অবস্থা ট্র্যাক করে।

Future<void> _exit()

এই async মথডটি "Exit" বাটনে ট্যাপ করলে ডাকা হয়। যদি ইতিমধ্যে _leaving true থাকে তবে কিছু করে না (ডাবল-ট্যাপ প্রতিরোধ)। অন্যথায় _leaving কে true সেটে করে, authControllerProvider.notifier.logout() কল করে সাধারণ সাইন-আউট করে, এবং finally ব্লকে (widget এখনও mounted থাকলে) _leaving false-এ ফরিয়ি দেয়। এখানে কোনো "আসল অ্যাকাউন্টে ফরি যোগা" নই কারণ ডেমো নজিই সেশন — তাই সরাসরি লগআউট যথেষ্ট।

Widget build(BuildContext context)

এই মথডটি ডেমো ব্যানারের UI তৈরিকরে। bos.infoSoft ব্যাকগ্রাউন্ড হিসেবে এবং bos.info টেক্সট/আইকন কালার হিসেবে ব্যবহৃত হয় (সলডি কালার দয়ি ফলি করলে ডার্ক মোডে খুব উজ্জ্বল দেখাত, তাই সফট-টোনে ফলি ব্যবহার করা হয়ছে)। এটি একটি Container তৈরিকরে যাতনে নচি হালকা বর্ডার থাকে, ভেতরে SafeArea(bottom: false) এবং একটি Row — যাতনে একটি "visibility" আইকন, "Demo — everything is real, nothing is saved." বার্তা, এবং একটি "Exit"/"Leaving..." TextButton থাকে (যা _leaving অনুযায়ী ডিজিবেল ও টেক্সট পরিবর্তন হয়, এবং ট্যাপে _exit ডাকে)।

ফাইল: lib/app/impersonation_banner.dart

সংক্ষিপ্ত ববিরণ: এই ফাইলটি প্ল্যাটফর্ম স্টাফ কোনো কোম্পানিকে ইমপারসোনেটে করার সময় স্ক্রিনিরে উপরে একটি সিতরকতামূলক ব্যানার দেখায়, যাতনে কে কোনো অ্যাকাউন্টে কাজ করছে তা কখনো ভুলে না যায় এবং কাউন্টডাউন টাইমার সহ সেশন শেষে করার অপশন দেয়।

ক্লাস/উইজটে: ImpersonationScope (extends ConsumerWidget)

এই ক্লাসটি পুরো অ্যাপকে মোড়ায় এবং ইমপারসোনেশন সেশন চলাকালীন উপরে একটিবার পনি করে রাখবে। এটি প্রতিস্ক্রিনিং বদলে অ্যাপ-ওয়াইড রাখা হয়েছে যাত ডিপি-লিংক, নভেগেশন বা যেকোনো ট্যাব থেকেই এই রিমাইন্ডার দেখা যায় — যাত স্টাফ কোনো কাস্টমারের লাইভ ডটোকে টেস্ট টেন্যান্ট ভাবে ভুল না করে।

Widget build(BuildContext context, WidgetRef ref)

এই মথডটি impersonationControllerProvider watch করে বর্তমান ImpersonationSession (session) আছে কিনা যাচাই করে। session null হলে সরাসরি child রিটার্ন হয়। অন্যথায় _Banner(session: session) ও child-কে একটি Column-এ একটি Material widget-এর ভেতরে সাজানো হয়, এবং MediaQuery.removePadding(removeTop: true) ব্যবহার করা হয় কারণ বার ইতিমধ্যে স্ট্যাটাস-বার ইনস্টে নথি নেই।

ক্লাস: _Banner (extends ConsumerStatefulWidget)

এই ক্লাসটি ইমপারসোনেশন ব্যানারের স্টেটফুল UI, যা একটি session প্রারামিটার নিয়ে এবং কাউন্টডাউন টাইমার সহ "Leave" বাটনের লজিক ধারণ করে।

ConsumerState<_Banner> createState()

_BannerState তৈরি করে রিটার্ন করে।

ক্লাস: _BannerState (extends ConsumerState<_Banner>)

এই ক্লাসটি ইমপারসোনেশন সেশনের অবশিষ্ট সময় প্রতিসেকেন্ডে রিফ্রেশ করা এবং সেশন শেষ করার লজিক ধারণ করে।

void initState()

এই মথডটি widget তৈরি হওয়ার সময় একবার চলে। এটি একটি Timer.periodic তৈরি করে যা প্রতি সেকেন্ডে (widget mounted থাকলে) setState(() {}) কল করে, যাত কাউন্টডাউন লবেলে প্রতিসেকেন্ডে রিফ্রেশ হয়। সেশনের প্রকৃত ম্যোদ শেষ হওয়া কন্ট্রোলারের দায়িত্ব — এই টাইমার শুধু সংখ্যাটি সঠিক রাখবে।

void dispose()

এই মথডটি widget ধ্বংস হওয়ার আগে _ticker টাইমার বাতিল (cancel()) করে, যাত মেমরালিকি বা widget dispose হওয়ার পরও টাইমার চলতে না থাকে।

Future<void> _end()

এই async মথডটি "Leave" বাটনে ট্যাপ করলে ডাকা হয়। যদি ইতিমধ্যে _ending true থাকে তবে কিছু করে না। এটি _ending কে true সেট করে, বর্তমান ScaffoldMessenger ও companyName ক্যাপচার করে, তারপর authControllerProvider.notifier.endImpersonation() কল করে ইমপারসোনেশন শেষ করে এবং একটি SnackBar দেখায় ("Left [companyName]. You are yourself again.")। finally ব্লকে, widget তখনও mounted থাকলে _ending false করা হয় — যদিও কমনেন্টে বলা আছে সাধারণত এই widget ততক্ষণে অদৃশ্য হয়ে যায় কারণ যত সেশন এটি আঁকছিল তা শেষ হয়ে গেছে।

Widget build(BuildContext context)

এই মেথডটি বিঘানাররে UI তৈরিকরে। `bos.warning` ব্যাকগ্রাউন্ড হিসেবে ব্যবহৃত হয়, এবং টেক্সট কালার (`foreground`) ডাইনামিকভাবে গণনা করা হয় — যদি ব্যাকগ্রাউন্ডের `luminance ০.৫`-এর বেশি হয় (লাইট মোডে `amber` প্রায় কালো) তাহলে প্রায়-কালো টেক্সট, নয়তো (ডার্ক মোডে `amber` প্রায় হলুদ) সাদা টেক্সট ব্যবহার করা হয়। `expiringSoon` নির্ধারণ করে `session.remaining.inMinutes < 2` কনি, যা টেক্সটের ফন্ট-ওয়েটে ও অপাসটি বাড়িয়ে জরুরিভাব তৈরিকরে। UI-তে থাকে: একটি `visibility` আইকন, "Viewing as [companyName]" শিরোনাম, একটি সাব-লাইন "Platform session · [adminEmail] · ends in [remainingLabel]" (আসল ব্যক্তির ইমেইল দেখানো হয় কারণ বাকি অ্যাপ তখন অন্য কাউকে দাবি করছে), এবং ডানপাশে হয় একটি ছোট `CircularProgressIndicator` (`_ending` true হলে) অথবা একটি "Leave" `TextButton` যা `_end` কল করে।

ফাইল: lib/app/nav_registry.dart

সংক্ষিপ্ত ববিরণ: এই ফাইলটি অ্যাপের সমস্ত নভিগেশন ডেস্টিনেশন (মেডউল/স্ক্রিনি/ট্যাব) এবং তাদের পারমিশন/রোল-ভিত্তিক দৃশ্যমানতার নিয়ম একটি কেন্দ্রীয় টবেলি আকারে সংজ্ঞায়িত করে, যা `AppDrawer` ও হোম স্ক্রিনিরে Quick Actions উভয়ই ব্যবহার করে যাতে দুটোর মধ্যে অসামঞ্জস্য না হয়।

enum: _NavGateKind

এই enum-টি `NavGate`-এর অভ্যন্তরীণ প্রকার নির্দেশে করে — চারটি সম্ভাব্য মান: `always` (সবসময় দৃশ্যমান), `anyPermission` (একটি বা একাধিক পারমিশন কোডের যেকোনো একটি থাকলে), `role` (নির্দিষ্ট রোলের তালিকার মধ্যে থাকলে), এবং `custom` (একটি হাতে-লেখো টেস্ট ফাংশনের ফলাফল অনুযায়ী)।

ক্লাস: NavGate

এই ক্লাসটি নির্ধারণ করে কে একটি `NavDestination` বা `NavGroup` দেখতে পাবে। এটি Angular সাইডবারের `roles/requiredPermission` চেকের মতোই আচরণ করে যাতে ওয়েব ও ফ্লাটার অ্যাপে একই অ্যাকাউন্টের জন্য একই দৃশ্যমানতা বজায় থাকে।

const NavGate.always()

এই কনস্ট্রাক্টরটি এমন একটি গেটে তৈরিকরে যা সবসময় `true` রটার্ন করে (`_kind = always`), অর্থাৎ সংশ্লিষ্ট আইটেমে সবসময় দৃশ্যমান।

const NavGate.permission(String code)

এই কনস্ট্রাক্টরটি একটি একক পারমিশন কোড দিয়ে গেটে তৈরিকরে (`_kind = anyPermission`, `_singleCode = code`) — ব্যবহারকারীর কাছে ওই একটি নির্দিষ্ট পারমিশন থাকলেই আইটেমটি দৃশ্যমান হয়।

const NavGate.anyPermission(List<String> codes)

এই কনস্ট্রাক্টরটি একাধিক পারমিশন কোডের তালিকা দিয়ে গেটে তৈরিকরে (`_kind = anyPermission`, `_codes = codes`) — তালিকার যেকোনো একটি পারমিশন থাকলেই যথেষ্ট।

const NavGate.role(List<String> roles)

এই কনস্ট্রাক্টরটি রোলের তালিকা দিয়ে গেটে তৈরিকরে (`_kind = role`) — ব্যবহারকারীর এই তালিকার যেকোনো একটি রোল থাকলে আইটেমটি দৃশ্যমান হয়।

`const NavGate.custom(bool Function(AppUser? user, PermissionState perms) test)`

এই কনস্ট্রাক্টরটি এমন গটেরে জন্য একটি এস্কেপে হ্যাচ, যা সাধারণ পারমিশন-বা-রোল চকে নয় — যমেন "owner অথবা যে কেউ শুধু plain employee নয়"। `_test` ফাংশনটি সংরক্ষণ করা হয় এবং `allows()`-এ কল করা হয়।

`bool allows(AppUser? user, PermissionState perms)`

এই মেথডটি `_kind`-এর মান অনুযায়ী উপযুক্ত চকে চালিয়ে একটি বুলিয়ান রিটার্ন করে: `always` হলে সবসময় `true`; `anyPermission` হলে `_singleCode` থাকলে `perms.has(_singleCode)`, নয়তো `perms.hasAny(_codes)`; `role` হলে `user?.hasAnyRole(_roles) ?? false`; এবং `custom` হলে `_test!(user, perms)` কল করে ফলাফল রিটার্ন করে। এটিই আসল জায়গা যেখানে সন্ধানিত নেওয়া হয় একটি নিভেগেশন আইটেমে দৃশ্যমান হবে কিনা।

ক্লাস: NavDestination

এই ক্লাসটি একটি `NavGroup`-এর ভেতরে একটি একক ডেস্টিনেশন প্রতিনিধিত্ব করে — একটি স্ক্রিনি, অথবা কোনও স্ক্রিনির একটি নির্দিষ্ট ট্যাব। এটি একটি ইমিউটেবল ডটো ক্লাস, যাতে `label`, `icon`, `path`, `gate` (ডিফল্ট `NavGate.always()`), `push` (ডিফল্ট `true`), এবং `initialTabLabel` ফিল্ড থাকে। `push` ফিল্ডটি নির্ধারণ করে ডেস্টিনেশনটি বর্তমান ট্যাবের উপরে স্ট্যাক হবে (`push`) নাকি বিটম-নেভ ব্রাঞ্জ পরবর্তন করবে (`go`)। `initialTabLabel` একই পথ শায়ের করা একাধিক আইটেমকে একটি নির্দিষ্ট ট্যাবে নামানোর জন্য ব্যবহৃত হয়, যেখানে সংশ্লিষ্ট স্ক্রিনিকে অবশ্যই একটি `initialTabLabel` কনস্ট্রাক্টর প্যারামিটার গ্রহণ করতে হবে (যেসব স্ক্রিনি এখনও তা করে না, সেগুলো এটি উপেক্ষা করে ডিফল্ট ট্যাবে খোলে)।

ক্লাস: NavGroup

এই ক্লাসটি একটি সাইডবার/ড্রয়ার সেকশন প্রতিনিধিত্ব করে — Angular-এর অ্যাকর্ডিয়ন গ্রুপগুলোর সাথে এক-এক সম্পর্ক। এটি একটি ইমিউটেবল ডটো ক্লাস, যাতে `label`, `icon`, `items` (একটি `NavDestination` তালিকা), এবং `gate` (ডিফল্ট `NavGate.always()`) ফিল্ড থাকে।

টপ-লেভেলে ফাংশন: bool showGroupedNav(AppUser? user)

এই ফাংশনটি Angular-এর `showGroupLabels` নথি অনুসরণ করে নির্ধারণ করে ড্রয়ার গ্রুপড অ্যাকর্ডিয়ন (গ্রুপ হেডারসহ) হিসেবে দেখানো হবে নাকি ফ্ল্যাট লিস্ট। এটি `true` রিটার্ন করে যদি ব্যবহারকারী কোম্পানি গুনার হয় অথবা `SUPER_ADMIN/SYSTEM_ADMIN` রোলের যেকোনো একটি থাকে। উল্লেখযোগ্য, এখানে `AppUser.isPlatformStaff` ইচ্ছাকৃতভাবে ব্যবহার করা হয়নি — কারণ সঠিক একটি বৃহত্তর সটে যা নচি সম্পূর্ণ গ্রুপ বাদ দেওয়ার সন্ধানিত ব্যবহৃত হয়, এই সংকীর্ণ অ্যাকর্ডিয়ন-নির্ধারণী চকে নয়।

টপ-লভেলে ফাংশন: List<NavGroup> buildNavGroups(AppUser? user, PermissionState perms)

এই ফাংশনটি সম্পূর্ণ নভেগেশন টবেলি তৈরি করে, প্রতিটি আইটেমকে `user/perms` অনুযায়ী গাট (`itemVisible/filterItems` হেল্পার দিয়ে) দিয়ে ফিল্টার করে, এবং শেষে খালি হয়ে যাওয়া গ্রুপগুলো (`g.items.isEmpty` শর্তে) বাদ দিয়ে রিটার্ন করে। এটিই `AppDrawer` ও হোম স্ক্রিনের Quick Actions উভয়ই একক ডটো উৎস। ফাংশনের ভেতরে `isPlatformOnly (user?.isPlatformStaff ?? false)` নির্ধারণ করে কতটি গ্রুপ (CRM, Company Services, Human Resources, Recruitment, Time & Leave, Payroll, Billing, Accounting, Banking, IT Assets) শুধু প্ল্যাটফর্ম-স্টাফ-নয় এমন ব্যবহারকারীদের জন্য অন্তর্ভুক্ত করা হবে কিনা (`if (!isPlatformOnly)` কন্ডিশনাল স্প্রডে ব্যবহার করে)। গ্রুপগুলোর মধ্যে রয়েছে: Dashboards (ড্যাশবোর্ড ভিউ, প্ল্যাটফর্ম অ্যাডমিন আইটেম যেমন Companies, Platform Users, Feature Flags, Locations, Subscription Management), CRM, Company Services (Categories, Services, Requests, Workflows, Knowledge Base, Support ইত্যাদি), Human Resources, Recruitment, Time & Leave, Payroll, Billing, Accounting, Banking, IT Assets, এবং সবসময় থাকা Administration (Users, Roles & Permissions, Company Profile, Announcements, Plans & Upgrade, AI Settings, Notifications, Appearance, Change Password, Change Email, Audit Logs, Platform Support)। বেশ কতটি আইটেম মনত্ব্যে ব্যাখ্যা করা আছে কেনে সেগুলো একই পথ/ট্যাবে ডুপ্লিকেট লিংক করে (যেমন Employee Salaries ও Salary Structures একই ট্যাবে যায়, Offers ও Pipeline আসলে Applicants ট্যাবেই যায়, কারণ আলাদা কোনো ব্যাকিং স্ক্রিন/প্রোভাইডার নেই)।

টপ-লভেলে ফাংশন: List<NavDestination> flattenGroups(List<NavGroup> groups)

এই ফাংশনটি সব গ্রুপের `items` একত্র করে (`expand`) একটি একক ফ্ল্যাট তালিকায় রূপান্তর করে, কোনো সেকশন হডোর ছাড়াই — এটি Angular-এর non-owner রেন্ডারিং-এর সমতুল্য। যেহেতু একটি মাত্র দৃশ্যমান আইটেমযুক্ত গ্রুপ ইতিমধ্যেই সরাসরি লিংকে পরণিত হয়ে যায় (এমনকি গ্রুপড ইউজারের জন্যও), তাই একটি গ্রুপের একক আইটেম ফ্ল্যাট করা এবং সব গ্রুপ ফ্ল্যাট করার মধ্যে কোনো পার্থক্য থাকে না।

ফাইল: lib/app/portal_shell.dart

সংক্ষিপ্ত বিবরণ: এই ফাইলটি ক্লায়েন্ট (কাস্টমার) ব্যবহারকারীদের জন্য একটি আলাদা সাইন-ইন ফ্রমে `PortalShell` সংজ্ঞায়িত করে, যা স্টাফ শেলের থেকে সম্পূর্ণ ভিন্ন — কারণ ক্লায়েন্ট ও কর্মচারী মূলত একই ব্যাকএন্ড শেয়ার করা দুটি ভিন্ন পণ্য।

ক্লাস/উইজটে: PortalShell (extends ConsumerWidget)

এই ক্লাসটি সাইন-ইন করা ক্লায়েন্টের জন্য একটি স্বতন্ত্র শে — স্টাফ শেলের চারটি ট্যাব লুকিয়ে রাখার বদলে সম্পূর্ণ আলাদা একটি শেলে তৈরি করা হয়েছে, কারণ একজন ক্লায়েন্ট জানতে চায় তার কাজ কোথায় আছে ও কত বকযো, আর একজন কর্মচারী চেক-ইন করতে ও ছুটি বুক করতে চায় — এই দুই ভিন্ন প্রয়োজনের জন্য একই শেপে ব্যবহার করা ঠিকি হতো না।

Widget build(BuildContext context, WidgetRef ref)

এই মথোডটি ক্লায়েন্ট পোর্টালরে মূল Scaffold তৈরিকরে। এর body হলো navigationShell (go_router-এর StatefulNavigationShell, যা বর্তমান ব্রাঞ্চে কন্টেন্ট ধারণ করে)। bottomNavigationBar-এ একটি NavigationBar থাকে যার উপরে একটি সূক্ষ্ম বর্ডার আঁকা হয়েছে (DecoratedBox)। selectedIndex হিসেবে navigationShell.currentIndex ব্যবহৃত হয়, এবং onDestinationSelected কলব্যাকের navigationShell.goBranch(index, initialLocation: ...) কল করা হয় — initialLocation তখনই true হয় যখন নির্বাচতি ইনডেক্স বর্তমান ইনডেক্সের সমান, অর্থাৎ একই ট্যাবে আবার ট্যাপ করলে সেই ব্রাঞ্চে তার প্রথমকি অবস্থানে রসিটে হয়। পাঁচটি NavigationDestination সংজ্ঞায়িত আছে: Home, Requests, Billing, Help, এবং Account — প্রতিটি একটি সাধারণ (outlined) ও নির্বাচতি (filled/rounded) আইকন ভ্যারিয়েন্ট আছে।

ফাইল: lib/app/router.dart

সংক্ষিপ্ত বিবরণ: পুরো অ্যাপের GoRouter কনফিগারেশন এবং সব রুট-পাথের কনস্ট্যান্ট এই ফাইলে সংজ্ঞায়িত। এটি স্টাফ শেলে আর ক্লায়েন্ট পোর্টাল—দুই ধরনের ব্যবহারকারীর জন্য আলাদা নভিগেশন গাছ তৈরিকরে এবং redirect লজিক দিয়ে অথেন্টিকেশন/অথরাইজেশন অনুযায়ী পাথ নিয়ন্ত্রণ করে।

ক্লাস: Routes

abstract final class — স্ট্যাটিক const String ফিল্ডের সংগ্রহ, যা অ্যাপের প্রতিটি স্টাফ-সাইড রুটের পাথ স্ট্রিং ধরে রাখে (যেমন splash, login, home, attendance, payroll, accounting ইত্যাদি)। কোনো মথোড নেই, শুধু কনস্ট্যান্ট ফিল্ড, যাকে কোনো স্ক্রিনকে সরাসরি পাথ স্ট্রিং হার্ডকোড করতে না হয় এবং টাইপো এড়ানো যায়।

ক্লাস: PortalRoutes

abstract final class — ক্লায়েন্ট পোর্টালরে রুট পাথগুলোর কনস্ট্যান্ট সংগ্রহ (home, requests, billing, help, account), সবগুলো /client প্রফিক্সের অধীনে। এই প্রফিক্স ব্যবহার করে রডাইরেক্ট লজিক শুধু পাথ দেখেই বুঝতে পারে ব্যবহারকারী স্টাফ অ্যাপে আছেন নাকি পোর্টালে।

ক্লাস/উইজেট: _AuthRefresh

ChangeNotifier-এর একটি প্রাইভেট সাবক্লাস, যা Riverpod-এর auth state-কে GoRouter-এর refreshListenable-এর সাথে সংযুক্ত করার সত্বে হিসেবে কাজ করে, কারণ GoRouter একটি Listenable আশা করে কিন্তু Riverpod প্রোভাইডার সরাসরি নয়।

void ping()

notifyListeners() কল করে GoRouter-কে জানায় যে redirect লজিক আবার চালানো দরকার। সাইন-ইন বা সাইন-আউট হলে এটি ট্রিগার হয়, যাকে ইউজার লগইন স্ক্রিনে আটকে না থাকে।

উপ-লভেলে প্রোভাইডার: _routerRefreshProvider

Provider<_AuthRefresh> — একটি নতুন _AuthRefresh ইনস্ট্যান্স তৈরিকরে এবং ref.onDispose-এ তার dispose রেজিস্টার করে, যাকে প্রোভাইডার ধ্বংস হলে লিসিনোরও পরিস্কার হয়।

টপ-লভেলে প্রোভাইডার: routerProvider

Provider<GoRouter> — পুরো অ্যাপের GoRouter ইনস্ট্যান্স তৈরি করে। এটি `_routerRefreshProvider` কে watch করে এবং `authControllerProvider`-এর পরিবর্তনে `refresh.ping()` কল করার জন্য `ref.listen` ব্যবহার করে। `initialLocation` হলো `Routes.splash`।

`redirect` ফাংশন লজিক: - `auth.isLoading` সত্য হলে (সংরক্ষণ সেশন এখনও পড়া হচ্ছে): স্পল্যাশে না থাকলে `Routes.splash`-এ পাঠায়, স্পল্যাশে থাকলে `null` (কোনো পরিবর্তন নয়)। - ব্যবহারকারী `null` হলে (সাইন-আউট): অথ স্ক্রিনে না থাকলে `Routes.login`-এ পাঠায়। - ব্যবহারকারী লগইন থাকলে: `user.isClient` দিয়ে ঠিকি করে সে পোর্টাল চায় (`wantsPortal`) কনি, আর বর্তমান লোকেশন `PortalRoutes.prefix` দিয়ে শুরু কনি (`inPortal`) তা মেলোয়। - `splash/auth` স্ক্রিনে থাকলে যথাযথ হোমে (পোর্টাল হোম বা স্টাফ হোম) পাঠায়। - পোর্টাল-ইউজার স্টাফ এলাকায় থাকলে পোর্টাল হোমে, আর স্টাফ-ইউজার পোর্টালে থাকলে স্টাফ হোমে ফেরত পাঠায়। - অন্য সব ক্ষেত্রে `null` (রুট পরিবর্তন দরকার নই)।

`routes` তালিকা: স্পল্যাশ, লগইন, ফরগট-পাসওয়ার্ড, রজিস্টার, ভেরিফাই-ইমেইল — এই পাঁচটি প্লইন `GoRoute`। এরপর অনেকেগুলো "ফুল-স্ক্রিন পুশ" রুট (`payslips`, `requests`, `support`, `crm`, `finance` ইত্যাদি) যগুলো ট্যাব-বারের ওপরে বসে; কিছু রুট `state.extra as String?` দিয়ে `initialTabLabel` প্যারামিটার নেয়। সবশেষে দুটি `StatefulShellRoute.indexedStack`: একটি `AppShell` দিয়ে পাঁচটি স্টাফ ট্যাব (`home`, `attendance`, `leave`, `alerts`, `profile`) মোড়ানো, অন্যটি `PortalShell` দিয়ে পাঁচটি পোর্টাল ট্যাব (`home`, `requests`, `billing`, `help`, `account`) মোড়ানো — যা `StatefulNavigationShell` ব্যবহার করে প্রতিটি ব্রাউজার নজিস্ব নভিগেশন স্টেটে রক্ষা করে।

ফাইল: lib/app/shell.dart

সংক্ষিপ্ত বিবরণ: স্টাফ অ্যাপের সাইন-ইন হওয়া মূল ফরমে — নচি পাঁচটি ট্যাবসহ একটি বিটম নভিগেশন বার এবং একটি ড্রয়ার। প্রতিটি ট্যাব `StatefulShellBranch` হিসেবে নজিস্ব নভিগেশন স্টেটে আলাদাভাবে ধরে রাখা।

টপ-লভেলে ভ্যারিয়েবল: appShellScaffoldKey

`GlobalKey<ScaffoldState>` — `AppShell`-এর `Scaffold`-এর কী। যেকোনো ট্যাবের ভেতরে স্ক্রিন থেকে (যেগুলোর নজিস্ব `Scaffold` আছে) এই কী ব্যবহার করে বাইরের ড্রয়ার-ধারী `Scaffold`-এ পৌঁছানো যায়, কারণ প্লইন `Scaffold.of(context)` ভেতরে `Scaffold` খুঁজে পাবে যার কোনো ড্রয়ার নই।

ক্লাস/উইজটে: AppDrawerButton

`StatelessWidget` — প্রতিটি ট্যাবের নজিস্ব অ্যাপবারে বসানো হ্যামবার্গার মেনু বাটন। আগে এটি আলাদা একটি স্ট্রুপি ছিল যা প্রতিটি স্ক্রিনে অতিরিক্ত উচ্চতা নতিও এবং দুটি স্তূপীকৃত হডার দেখাতো; এখন এটি সরাসরি ট্যাবের বার-এ বসে।

`Widget build(BuildContext context)`

একটি `IconButton` রটরন করে (`Icons.menu_rounded` আইকন, টুলটপ "All modules")। প্রসেস করলে `appShellScaffoldKey.currentState?.openDrawer()` কল করে বাইরের `Scaffold`-এর ড্রয়ার খোলে।

ক্লাস/উইজটে: AppShell

ConsumerWidget — সাইন-ইন হওয়া অবস্থায় দেখানো মূল ফ্রমে। কনস্ট্রাক্টরে **navigationShell** (একটি **StatefulNavigationShell**) নিয়ে। পাঁচটি ট্যাব বটম-বারের উর্ধ্বসীমা হিসেবে বছে নেওয়া হয়েছে কারণ এর বশেইলে লেবেল আর ফটি করে না এবং টারগটে ছোট হয়ে যায়; ওয়বে অ্যাপরে বারোটটি ন্যাভ-গ্রুপ এখানে খাটে না — তার বদলে প্রতদিনি যা লাগে তা ট্যাবে, বাকটি ভতের থেকে অ্যাক্সেসযোগ্য।

Widget build(BuildContext context, WidgetRef ref)

থমি টোকনে (**bos**) এবং **unreadCountProvider** থেকে অপঠতি নোটিফিকেশন সংখ্যা watch করে। একটি **Scaffold** রটিরন করে যার **key** হলো **appShellScaffoldKey**, **drawer** হলো **AppDrawer**, **body** হলো **navigationShell** (বর্তমান ব্রাঞ্চে কনটেন্ট), এবং **bottomNavigationBar** একটি **DecoratedBox**-এ মোড়ানো **NavigationBar** (উপরে বর্ডার-লাইনসহ)। পাঁচটি **NavigationDestination**: Home, Attendance, Leave, Alerts (**Badge.count** দিয়ে অপঠতি সংখ্যা দেখায়), **Me**। **selectedIndex** হলো **navigationShell.currentIndex**, আর **onDestinationSelected** হলো **_onTap**।

void _onTap(int index)

navigationShell.goBranch(index, initialLocation: index == **navigationShell.currentIndex**) কল করে ব্রাঞ্চে বদলায়। ইতিমধ্যে সক্রিয় ট্যাবেই আবার ট্যাপ করলে সেই ব্রাঞ্চে রুট স্ক্রিনি ফরিয়ে আনবে (ভতেরে ড্রলি করা স্ট্যাক থেকে বেরিয়ে আসার স্ট্যান্ডার্ড উপায়)।

ফাইল: lib/app/splash_screen.dart

সংক্ষিপ্ত ববিরণ: সংরক্ষতি সশেন পড়ে ফরত আনার সময় (সাধারণত সেকেন্ডরে একটি ভিগ্নাংশ) দেখানো একটি সাধারণ লোডিং স্ক্রিনি, যাতে ব্যবহারকারী লগইন স্ক্রিনি ঝলক দেখে সরাসরি ড্যাশবোর্ডে চলে যাওয়ার অভিজ্ঞতা পায়।

ক্লাস/উইজটে: SplashScreen

StatelessWidget — কেবল রেন্ডারিং, কোনো লজিকি নই। ব্র্যান্ড লোগো, "Zuhoo" টেক্সট এবং একটি প্রগ্রেসে স্পিনার প্রদর্শন করে।

Widget build(BuildContext context)

থমি টোকনে **bos** নিয়ে একটি **Scaffold** রটিরন করে যার ব্যাকগ্রাউন্ড **bos.bgPage**। মাঝখানে **Column** (কেন্দ্রীভূত, **mainAxisSize: min**) যাতে আছে: **BrandMark** (size 66), 18px গ্যাপ, "Zuhoo" টেক্সট (**bos.text** রঙ, 24px, bold, letterSpacing -0.5), 28px গ্যাপ, এবং 22x22 আকারে **CircularProgressIndicator** (strokeWidth 2.4, রঙ **bos.brand**)।

ফাইল: lib/core/auth/auth_controller.dart

সংক্ষিপ্ত ববিরণ: কে সাইন-ইন আছে তার একক উৎস-সত্য (**AuthController**), এবং তার সাথে সংশ্লিষ্ট দুটি ছোট প্রোভাইডার (**DemoSessionController**, **currentUserProvider**)। এখানে সশেন restore, লগইন, ডেমো-লগইন, লগআউট, ইমপারসোনেশন শুরু/শেষ করা, প্রোফাইল রফ্রেশে ইত্যাদি সব স্টেট-পরিবর্তনকারী অপারেশন থাকে।

ক্লাস: AuthController

`AsyncNotifier<AppUser?>` — এর স্টেটে `AsyncValue<AppUser?>`, যা তিনটি অবস্থা প্রকাশ করে: স্টোরেজে থাকে সেশন এখনো লোড হচ্ছে (loading), সাইন-ইন (data non-null user), সাইন-আউট (data null)। লগইন ব্যর্থতা এই স্টেটে রাখা হয় না — লগইন স্ক্রিনি সরাসরি থ্রোন এক্সপেশন থেকে বার্তা দেখায়, যাত্রে একটি ব্যর্থ চেষ্টা পুরো অ্যাপকে error state-এ না ফেলে।

`Future<AppUser?> build()`

`AsyncNotifier`-এর ওভাররাইড। `_restoreSession()` কল করে তার রজোল্ট রটার্ন করে — এটাই প্রোভাইডার প্রথম তরৈ হওয়ার সময় চালানো এন্টরি-পয়েন্ট।

`Future<AppUser?> _restoreSession()`

কোল্ড-স্টার্টে পূর্বের সেশন ফরিয়ে আনো। `secureStoreProvider` থেকে `store.readUser()` কল করে ইউজার JSON পড়ে; না পলে `null` রটার্ন করে। তারপর অ্যাক্সেস/রফ্রেশে টোকেনে পড়ে ও `JwtPayload.tryParse` দিয়ে পলেড পার্স করে। যদি স্টোরেজে একটি বৈধ ইমপারসোনেশন সেশন (`ImpersonationSession.tryFromJson`) পাওয়া যায়: টোকেনে এক্সপায়ার্ড বা ইমপারসোনেশন এক্সপায়ার্ড হলে `_restoreAdminSession(assignState: false)` কল করে অ্যাডমিনের নজিরে সেশনে ফেরত যায় (কারণ ধার-করা টোকেনে রফ্রেশে করা যায় না); অন্যথায় `impersonationControllerProvider`-এ সেশন স্টে করে, পারমিশন ক্যাশ থেকে হাইড্রটে করে, ব্যাকগ্রাউন্ডে রফ্রেশে চালায়, এবং ইমপারসোনটেডে ইউজার রটার্ন করে। ইমপারসোনেশন না থাকলে: টোকেনে অকজে (null পলেড, বা এক্সপায়ার্ড ও রফ্রেশে-টোকেনে-বহীন) হলে `store.clear()` করে `null` রটার্ন করে। অন্যথায় ডেমো-ফ্ল্যাগ সঙ্কি করে, পারমিশন ক্যাশ থেকে হাইড্রটে করে, `_warmSession()` ব্যাকগ্রাউন্ডে চালায় (নেটেওয়ার্ক রাউন্ড-ট্রপে স্পল্যাশ ব্লক না করত্রে), এবং `AppUser.fromJson(userJson)` রটার্ন করে।

`Future<void> _warmSession()`

পুনরুদ্ধারের পরে ব্যাকগ্রাউন্ড রভিয়ালডিশেন। `pushServiceProvider().registerToken()` (অ্যাপ বন্ধ থাকাকালীন টোকেনে ঘোরানো বা নতুন পুশ-সাপোর্ট কভার করত্রে), তারপর `permissionControllerProvider.notifier.refreshQuietly()` await করে, এরপর `_applyProfile()` চেষ্টা করে — ব্যর্থ হলে নীরবে উপেক্ষা করে (ক্যাশড ইউজার এমনতিই ব্যবহারযোগ্য)।

`Future<AppUser> login(String email, String password)`

সাইন-ইন করে এবং প্রথম স্ক্রিনিরে প্রয়োজনীয় সবকিছু লোড করে রটার্ন করে। ইচ্ছাকৃতভাবে `AsyncValue.loading()` স্টে করে না — কারণ এই প্রোভাইডারের loading state রাউটারের কাছে "সঞ্চিত সেশন এখনো পড়া হচ্ছে" অর্থ বহন করে, তাই লগইন স্ক্রিনি নজিই তার প্রগ্রসে ট্র্যাক করে। `authRepositoryProvider.login()` কল করে `LoginResponse` পায়, `AppUser.fromLogin` দিয়ে ইউজার বানায়, `secureStoreProvider.writeUser` দিয়ে সংরক্ষণ করে, `permissionControllerProvider.notifier.load()` await করে (router guard synchronous permission পড়ে বলে navigate-এর আগেই cache হতে হবে), পুশ-টোকেনে রজিস্টার করে ব্যাকগ্রাউন্ডে। এরপর `repo.getProfile()` চেষ্টা করে `fullName/profileImageUrl` আপডেটে করে (ব্যর্থ হলে cosmetic বলে উপেক্ষা)। সবশেষে `state = AsyncValue.data(user)` স্টে করে user রটার্ন করে। এরর হলে: store clear, permission clear, state data-null-এ (যদি আগের কিছু ছিল), এবং stacktrace-সহ error রি-থ্রো করে।

Future<AppUser> startDemo()

ক্রেডেনশিয়াল ছাড়া পাবলিক রিডি-ওনলি ডেমো শুরু করে। `authRepositoryProvider.startDemo()` কল করে `LoginResponse` পায়, ইউজার লেখে, `store.writeIsDemo()` করে, `demoSessionProvider.notifier.set(true)`, `permissionControllerProvider.notifier.load()` await করে, state সেট করে, ইউজার রিটার্ন করে। ইচ্ছাকৃতভাবে পুশ-টোকেনে রজিস্টার করে না (একক শ্যোরড ডেমো অ্যাকাউন্ট বলে) এবং প্রোফাইল ফেচও করে না। এরর হলে `login()`-এর মতোই কলনিআপ করে state null-এ ফরোয় এবং rethrow করে।

Future<void> logout()

প্রথমত যদি ইমপারসোনেশন সক্রিয় থাকে (`impersonationControllerProvider` non-null) তাহলে `endImpersonation()` কল করে সেই সেশন প্রথমত বন্ধ করে (audit row স্ট্র্যাম্প হয়, ও পরের রফ্রেশ-টোকেনে-রিডি অ্যাডমিনের নজিরেটাই হয়)। রফ্রেশ টোকেনে পড়ে, তারপর টোকেনে এখনো Authorization হডরে থাকা অবস্থাতেই `pushServiceProvider().unregisterToken()` কল করে। স্থানীয়ভাবে প্রথমত `clearSession()` করে (নেটেওয়ার্ক ছাড়াও instant হওয়া উচিত), তারপর `authRepositoryProvider.logout(refreshToken)` কল করে সার্ভার-সাইড রিভোক করে (best-effort)।

Future<ImpersonationSession> startImpersonation({required int companyId, required String reason})

কোনো tenant-এর ভেতরে সেশন খোলে এবং সেই tenant হয়ে যায়। বর্তমান স্টেটে থাকে `admin` না পলে এরর থ্রো করে; ইতিমধ্যে ইমপারসোনেশন চলমান থাকলেও এরর থ্রো করে। `authRepositoryProvider.impersonate(companyId, reason)` কল করে `ImpersonationStart` পায়; থার্মি `accessToken/sessionId` হলে এরর থ্রো করে (এখনো কিছু swap হয়নি বলে অ্যাডমিন নরিপদ থাকে)। `ImpersonationSession.fromStart` দিয়ে session বানায়, একটি `tenantUser (AppUser)` বানায় যার roles `['COMPANY_OWNER']` (কারণ ব্যাকএন্ড টোকেনে এই role দিয়েই মনিট করে)। `secureStoreProvider.beginImpersonation(...)` কল করে টোকেন/সেশন/ইউজার সংরক্ষণ করে, permission clear করে, `impersonationControllerProvider.notifier.set(session)`, state-কে `tenantUser`-এ সেট করে। এরপর `permissionControllerProvider.notifier.load()` চেষ্টা করে (ব্যর্থ হলে নীরবে উপেক্ষা, পরের স্ক্রিনি আবার লোড হবে)। session রিটার্ন করে।

Future<void> endImpersonation({bool expired = false})

সেশন শেষে করে অ্যাডমিনিকে তার নজিরে পরচিহ্ন ফরিয়ে দেয়। `impersonationControllerProvider` থাকে সেশন পড়ে, না থাকলে সরাসরি রিটার্ন। থাকলে impersonation controller-কে null সেট করে, `_restoreAdminSession()` কল করে অ্যাডমিন সেশন ফরিয়ে আনে (এটি প্রথমত হয় কারণ end-session এন্ডপয়েন্ট platform-admin role চায়, কিন্তু বর্তমান টোকেনে COMPANY_OWNER — swap-এর আগে কল করলে 403 হতো)। `admin` non-null হলে `authRepositoryProvider.endImpersonation(session.impersonationSessionId)` কল করে (best-effort, ব্যর্থতা উপেক্ষিত) — audit রেকর্ড বন্ধ করার জন্য, টোকেনে এমনতিই ডিভাইস থেকে বাদ পড়ে গেছে।

Future<AppUser?> _restoreAdminSession({bool assignState = true})

পার্ক করা অ্যাডমিন সেশন পুনরায় ইনস্টল করে। `secureStoreProvider.restoreAdmin()` থেকে JSON পড়ে, impersonation controller ও permission controller clear করে। `adminJson` null হলে (`assignState` true থাকলে state null সেট করে) null রিটার্ন করে। অন্যথায় `AppUser.fromJson` দিয়ে অ্যাডমিন বানায়, (`assignState` true হলে) state-এ সেট করে, পারমিশন ক্যাশ থেকে হাইড্রটে করে ও ব্যাকগ্রাউন্ডে রফ্রেশ চালায়, অ্যাডমিন রিটার্ন করে।

Future<void> handleSessionExpired()

HTTP লয়ের কল করে যখন টোকেনে নশ্চিতিভাবে মৃত। ইমপারসোনেশন চলমান থাকলে **endImpersonation(expired: true)** কল করে (কারণ অ্যাডমিনের নজিরে সেশন এখনো বৈধ, লগইন স্ক্রিনি পাঠানো উচিত নয়); অন্যথায় **clearSession()** কল করে।

Future<void> clearSession()

নভেগিটে বা ব্যাকএন্ড কল ছাড়াই সেশন মুছে দেয়। HTTP লয়ের রফ্রেশে নশ্চিতিভাবে প্রত্যাখ্যাত হলেও এটিকল হয়। impersonation controller null সেটে করে, demo session provider false সেটে করে (ডেমো শেষে হওয়ার দুটি পথ কভার করে: "Exit demo" ও অ্যাক্সেস টোকেনে এক্সপায়ার হওয়া), **secureStoreProvider.clear()** await করে, permission clear করে, state-কে **AsyncValue.data(null)** সেটে করে।

Future<void> refreshProfile()

/users/profile পুনরায় পড়ে ক্যাশড নাম ও অ্যাভাটার আপডেট করে। **state.value == null** হলে কচ্ছই করে না; অন্যথায় **_applyProfile()** কল করে।

Future<void> setAvatar(String? imagePath)

/users/profile ব্যতীত অন্য কোথাও (যেমন **PATCH /employees/me**) অ্যাভাটার পরিবর্তিত হওয়ার পর ক্যাশড অ্যাভাটার আপডেট করে। বর্তমান ইউজার null হলে কচ্ছই করে না।

Env.resolveImageUrl(imagePath) দিয়ে রজিলভ করে, **copyWith** দিয়ে ইউজার আপডেট করে (resolved null হলে **clearImage: true**), স্টোরে লেখে, state আপডেট করে।

Future<void> _applyProfile()

/users/profile কল করে **fullName** ও **profileImageUrl** আপডেট করে। ইমপারসোনেশন চলমান থাকলে সরাসরি রিটার্ন করে (কারণ প্রোফাইল এন্ডপয়েন্ট টোকেনের subject অনুযায়ী রজেলভ হয়, যা impersonation-এর সময়ও অ্যাডমিনের — কল করলে ভুলভাবে অ্যাডমিনের নাম/অ্যাভাটার tenant identity-এর উপর বসে যেতে)। বর্তমান ইউজার null হলেও রিটার্ন করে। অন্যথায় **authRepositoryProvider.getProfile()** কল করে, **copyWith** দিয়ে আপডেট করে, স্টোরে লেখে, state সেটে করে।

টপ-লেভেলে প্রোভাইডার: authControllerProvider

AsyncNotifierProvider<AuthController, AppUser?> — **AuthController.new** দিয়ে তৈরি। অ্যাপের সবচেয়ে গুরুত্বপূর্ণ auth-state প্রোভাইডার, যা রাউটার ও UI-এর বাকি অংশ watch করে।

ক্লাস: DemoSessionController

Notifier<bool> — এই সেশনটি পাবলিক রিডি-ওনলি ডেমো কনি তা আলাদাভাবে ধরে রাখে, কারণ অ্যাকাউন্ট বা টোকেনে এই তথ্য নেই (ডেমো owner একটি সাধারণ COMPANY_OWNER)।

AuthController স্টোর্ড ফ্ল্যাগের সাথে এটিকে সঙ্কে রাখতে রসিটার্টে নীরবে normal সেশন মনে না হয়।

bool build()

প্রাথমিক স্টেটে **false** রিটার্ন করে।

void set(bool value)

state != value হলে state আপডেট করে (অহতুক রবিল্ড এড়াতো)।

টপ-লেভেলে প্রোভাইডার: demoSessionProvider

NotifierProvider<DemoSessionController, bool> — **DemoSessionController.new** দিয়ে তৈরি।

টপ-লেভেলে প্রোভাইডার: currentUserProvider

Provider<AppUser?> — authControllerProvider-এর .value watch করে সাইন-ইন থাকা ইউজার (বা loading/signed-out অবস্থায় null) রিটার্ন করে। অথ-রডিইরক্টরে পছন্দে স্ক্রিনিগুলো নিশ্চিতিভাবে এটিকে non-null ধরে নেতি পারবে।

ফাইল: lib/core/auth/auth_models.dart

সংক্ষিপ্ত ববিরণ: অথনেটিকেশন ও ইমপারসোনেশন সম্পর্কিত সমস্ত রকি়োয়স্ট/রসেপন্স ডটো-মডেলে এই ফাইলে সংজ্ঞায়িত — লগইন, রজিস্ট্রেশন, পাসওয়ার্ড রকিভারি, প্রোফাইল, JWT পার্সিং এবং ইমপারসোনেশন সেশন/অডি।

টপ-লেভেলে কনস্ট্যান্ট: platformRoles

const Set<String> — SaaS-প্রোভাইডার স্টাফ রোলের তালিকা (SUPER_ADMIN, SYSTEM_ADMIN, SUPPORT_AGENT, SUPPORT_MANAGER, MARKETING_MANAGER, PLATFORM_ACCOUNTANT, SALES_MANAGER), যা company-scoped রোল (COMPANY_OWNER, EMPLOYEE, CLIENT) থেকে আলাদা।
ব্র্যাকেন্ডরে User.isPlatformUser() ও Angular-এর AuthService-এর তালিকা মরির করে।

ক্লাস: LoginRequest

POST /auth/login-এর রকি়োয়স্ট বডি ফিল্ড: email, password।

Map<String, dynamic> toJson()

{'email': email, 'password': password} রিটার্ন করে।

ক্লাস: LoginResponse

POST /auth/login-এর রসেপন্স। ফিল্ড: userId, firstName, email, role, companyId (nullable), accessToken, refreshToken (nullable)।

factory LoginResponse.fromJson(Map<String, dynamic> json)

JSON থেকে পার্স করে; সব সংখ্যাসূচক/স্ট্রিং ফিল্ডে null-safe ডফিল্ট (0, "") ব্যবহার করে।

ক্লাস: AppUser

অ্যাপে ধরে রাখা সাইন-ইন ইউজার — Angular-এর User মরির করে। ফিল্ড: id, email, fullName, roles (List<String>), companyId (nullable), profileImageUrl (nullable)।

bool get isPlatformStaff

roles.any(platformRoles.contains) — কোনো role platformRoles সেটে থাকলে true।

bool get isClient

roles.contains('CLIENT')।

bool get isCompanyOwner

roles.contains('COMPANY_OWNER')।

bool hasRole(String role)

roles.contains(role)।

`bool hasAnyRole(Iterable<String> candidates)`

`candidates.any(roles.contains)` — যেকোনো একটি ম্যাচ করলে `true`।

`String get displayFirstName`

গ্রটিং-এর জন্য প্রথম নাম দিয়ে। `fullName` ট্রিম করে খালি হলে ইমেইলের `@`-এর আগের অংশ রিটার্ন করে; অন্যথায় হোয়াইটস্পেস দিয়ে স্প্লিট করে প্রথম টুকরো রিটার্ন করে (কারণ প্রোফাইল-কল আসার আগ পর্যন্ত `fullName`-এ শুধু প্রথম নামই থাকে)।

`String get initials`

`fullName` স্প্লিট করে non-empty অংশগুলো নিয়ে। খালি হলে ইমেইলের প্রথম অক্ষর (বা ?) আপারকসেসে; একটি অংশ হলে তার প্রথম অক্ষর; একাধিক হলে প্রথম ও শেষ অংশের প্রথম অক্ষর জোড়া দিয়ে আপারকসেসে রিটার্ন করে।

`String get roleLabel`

ইউজারকে তার নিজের সম্পর্কে দেখানোর `role`, শব্দ আকারে (যেমন "Company owner", `COMPANY_OWNER` নয়)। `roles` খালি হলে "Member"। শুধু প্রথম `role` নিয়ে (ধরে নেওয়া হয় একটি অ্যাকাউন্টে একটি `role`-ই সর্বাধিক নেবে, `_`-এ স্প্লিট করে প্রতিটি শব্দে প্রথম অক্ষর আপার ও বাকি লোয়ার করে স্পেস দিয়ে জোড়া দিয়ে।

`AppUser copyWith({String? fullName, List<String>? roles, String? profileImageUrl, bool clearImage = false})`

নতুন `AppUser` তৈরি করে; `id/email/companyId` অপরিবর্তিত থাকে; `clearImage true` হলে `profileImageUrl` জোরপূর্বক `null` করে, অন্যথায় দেওয়া মান বা বর্তমান মান রাখে।

`factory AppUser.fromLogin(LoginResponse res)`

`LoginResponse` থেকে `AppUser` বানায়ে: `fullName` হয় `res.firstName`, `roles` হয় `[res.role]` (একটি single-element লিস্ট)।

`Map<String, dynamic> toJson()`

সব ফিল্ড ম্যাপে সিরিয়লাইজ করে।

`factory AppUser.fromJson(Map<String, dynamic> json)`

JSON থেকে পার্স করে; `roles` লিস্ট না থাকলে বা টাইপ না মিললে খালি `const` লিস্ট ব্যবহার করে।

ক্লাস: UserProfile

GET `/api/users/profile`-এর রেসপন্স। ফিল্ড: `id`, `firstName`, `lastName`, `email`, `phone` (nullable), `role` (nullable), `image` (nullable)।

`String get fullName`

`'$firstName $lastName'.trim()`।

`String? get imageUrl`

`Env.resolveImageUrl(image)` কল করে পূর্ণ URL রিটার্ন করে।

`factory UserProfile.fromJson(Map<String, dynamic> json)`

JSON থেকে null-safe ডিফল্টসহ পার্স করে।

ক্লাস: RegisterRequest

POST /api/auth/register-এর রকি়োয়েস্ট। ফিল্ড: `firstName`, `lastName`, `email`, `password`, `companyName`, `subdomain`, `companyPhone` (nullable)। ব্যাকএন্ডের `RegisterRequest` ভ্যালিডেশন হুবহু মলে (পাসওয়ার্ডে uppercase/lowercase/digit/special char; subdomain lowercase letters/digits/hyphens)।

`Map<String, dynamic> toJson()`

সব ফিল্ড ম্যাপে রূপান্তর করে; `companyPhone` non-null ও non-empty (trim করে) হলই তা যোগ করে।

ক্লাস: RegisteredUser

POST /api/auth/register-এর রসেপন্স (ব্যাকএন্ডের `UserResponse`)। ফিল্ড: `email`, `firstName`, `emailVerified`। রজিস্ট্রেশনের পরপরই `emailVerified` সবসময় false থাকে — ভেরিফাই কোড দিয়ে সাইন-ইন সক্রিয় করতে হয়।

`factory RegisteredUser.fromJson(Map<String, dynamic> json)`

JSON থেকে null-safe ডফিল্টসহ পার্স করে।

ক্লাস: VerifyEmailRequest

ফিল্ড: `email`, `code`।

`Map<String, dynamic> toJson()`

`{'email': email, 'code': code}`।

ক্লাস: ResendVerificationRequest

ফিল্ড: `email` (পজিশিনাল কনস্ট্রাক্টর)।

`Map<String, dynamic> toJson()`

`{'email': email}`।

ক্লাস: ForgotPasswordRequest

ফিল্ড: `email` (পজিশিনাল কনস্ট্রাক্টর)।

`Map<String, dynamic> toJson()`

`{'email': email}`।

ক্লাস: VerifyResetCodeRequest

ফিল্ড: `email`, `code`।

`Map<String, dynamic> toJson()`

`{'email': email, 'code': code}`।

ক্লাস: ResetPasswordRequest

রসিটে অথরাইজ করার দুটি পিরস্পর-এক্সক্লুসিভ উপায়: `email+code` (forgot-password ইমেইল থেকে), অথবা লম্বা-স্থায়ী `token` (ক্লায়েন্ট-পোর্টাল ইনভাইট লিংক থেকে)। ফিল্ড: `email`, `code`, `token` (সব nullable), `newPassword`, `confirmPassword` (required)।

Map<String, dynamic> toJson()

শুধু non-null ফিল্ডগুলো যোগ করে (email/code/token), সাথে newPassword ও confirmPassword সবসময়।

ক্লাস: ChangePasswordRequest

ফিল্ড: currentPassword, newPassword, confirmPassword।

Map<String, dynamic> toJson()

তনিটি ফিল্ড ম্যাপে রূপান্তর করে।

ক্লাস: ChangeEmailRequest

ইমেইল sign-in identifier বলে বর্তমান পাসওয়ার্ড দিয়ে পুনরায়-প্রমাণীকরণ প্রয়োজন। ফিল্ড: email, currentPassword।

Map<String, dynamic> toJson()

{'email': email, 'currentPassword': currentPassword}।

ক্লাস: JwtPayload

JWT থেকে exp (ভেরিফাই না করাই) পড়ে। ভেরিফিকেশন সার্ভারের কাজ — এটি শুধু "পাঠানোর যোগ্য কনি" উত্তর দিয়ে, যাতা সংরক্ষণ অনেকে-আগে-মৃত টোকেনে সরাসরি লিগইন স্ক্রিনি পাঠায়, ড্যাশবোর্ড ব্লক দেখিয়ে 401 না করে। ফিল্ড: expiresAt, role (nullable), companyId (nullable), impersonatedBy (nullable — শুধু ইমপারসোনেশন টোকেনে থাকে), impersonationSessionId (nullable)।

bool get isImpersonation

impersonatedBy != null।

bool get isExpired

DateTime.now().isAfter(expiresAt)।

static JwtPayload? tryParse(String? token)

টোকেনে null/খালি হলে null। টোকেনে . দিয়ে তনি অংশে স্পলিট না হলে null। দ্বিতীয় অংশ (payload) base64Url.normalize দিয়ে normalize করে ডিকোড ও jsonDecode করে; ফলাফল Map না হলে বা exp num না হলে null। সফল হলে JwtPayload তৈরি করে রিটার্ন করে (expiresAt মলিসিকেন্ডে এক থেকে, role/impersonatedBy/impersonationSessionId/companyId ঐচ্ছিকভাবে)। যেকোনো এক্সপেশনে null রিটার্ন করে।

ক্লাস: ImpersonationStart

POST /api/platform-admin/companies/{id}/impersonate-এর রেসপন্স। কোনো রিফ্রেশে টোকেনে নেই — ইমপারসোনেশনে সেশন ঠিক এই একটি টোকেনের মেয়াদ (ডিফল্ট ৩০ মিনিট) পর্যন্তই টকি থাকে।

ফিল্ড: accessToken, companyId, companyName, impersonationSessionId, expiresInSeconds।

factory ImpersonationStart.fromJson(Map<String, dynamic> json)

JSON থেকে পার্স করে; expiresInSeconds অনুপস্থিতি হলে ব্যাকএন্ডের নজিস্ব ডিফল্ট 1800-এ ফলব্যাক করে (0 করলে "already expired" পড়া যতে এবং প্রথম টকিই সেশন শেষ হয়ে যতে)।

ক্লাস: ImpersonationSession

চলমান ইমপারসোনেশন সেশন যভাবে অ্যাপ ট্র্যাক করে। ফিল্ড: `companyId`, `companyName`, `impersonationSessionId`, `expiresAt`, `adminEmail` (ব্যানারে নাম দেখাতে, কারণ সাইন-ইন ইউজার ততক্ষণে tenant identity দিয়ে প্রতস্থাপতি), `reason` (nullable)।

`Duration` get remaining

`expiresAt.difference(DateTime.now());` ঋণাত্মক হলে `Duration.zero`।

`bool` get hasExpired

`remaining == Duration.zero`।

`String` get remainingLabel

ব্যানারের কাউন্টডাউনের জন্য `mm:ss` ফরম্যাট। মিলিসেকেন্ডকে সেকেন্ডে রূপান্তর করে `ceil()` (রাউন্ড-আপ) করে, যেহেতু `4m05.9s` বাকি থাকলে `04:06` দেখায় এবং সদ্য-খোলা সেশন `29:59` নয় `30:00` দেখায়।

factory `ImpersonationSession.fromStart(ImpersonationStart start, {required String adminEmail, String? reason})`

`start`-এর ফিল্ড কপিকরে, `expiresAt` হিসাব করে `DateTime.now().add(Duration(seconds: start.expiresInSeconds))` দিয়ে।

`Map<String, dynamic>` toJson()

সব ফিল্ড সেরিয়ালাইজ করে; `expiresAt` ISO8601 স্ট্রিং হিসাবে।

static `ImpersonationSession? tryFromJson(Map<String, dynamic>? json)`

`json` null হলে null। `expiresAt` পার্স ব্যর্থ হলে বা `impersonationSessionId` null/খালি হলে null। অন্যথায় বৈধ `ImpersonationSession` বানায় (`companyName` ডিফল্ট "this company")।

ক্লাস: ImpersonationAuditEntry

GET /api/platform-admin/impersonate/history-এর একটি রিকর্ড। ফিল্ড: `id`, `companyName`, `adminName`, `startedAt`, `reason` (nullable), `endedAt` (nullable), `companyId` (nullable)।

`bool` get openEnded

`endedAt == null || endedAt.isEmpty` — এই দুটি ক্ষেত্রই টবেলি একই রকম দেখায় (এখনো চলমান, অথবা অ্যাপ বন্ধ হয়ে/টোকেনে ম্যোদোত্তীর্ণ হয়ে কখনো রিপোর্ট না করে শেষে হওয়া) তাই লবেলে "active" নয়, "not ended" বলে।

factory `ImpersonationAuditEntry.fromJson(Map<String, dynamic> json)`

JSON থেকে null-safe ডিফল্টসহ পার্স করে (`companyName` ডিফল্ট "Unknown company", `adminName` ডিফল্ট "Unknown admin")।

ফাইল: lib/core/auth/auth_repository.dart

সংক্ষিপ্ত ববিরণ: `/api/auth` ও `/api/users` এন্ডপয়েন্টের ববির্দধে অ্যাপরে প্ৰতিটি কল এখানে সংজ্ঞায়িত। ইচ্ছাকৃতভাবে "নবিবোধ": শুধু ব্যাকএন্ড ও স্টোরজেরে সাথে কথা বলে, নজিে কনো স্টেটে রাখে না — কখন কল করবে এবং তারপর অ্যাপ কমনে দেখাবে তা ঠকি করা `AuthController`-এর কাজ।

ক্লাস: AuthRepository

কনস্ট্রাক্টর `AuthRepository(this._api, this._store)` — একটি `ApiClient` ও একটি `SecureStore` নেয়।

`Future<LoginResponse> login(LoginRequest request)`

`POST /auth/login` কল করে `LoginResponse` পাবস করে, `_store.writeTokens(accessToken, refreshToken)` দিয়ে টোকেনে সংরক্ষণ করে, রসেপন্স রিটার্ন করে। `ApiException` ধরে: 401 হলে এবং `!e.fromServer` (অর্থ্যাৎ ব্যাকএন্ড নজিে থেকে কনো ব্যবহারযোগ্য বার্তা পাঠায়নি) হলে সাধারণ "session expired"-জাতীয় বার্তার বদলে "Invalid email or password." ছুড়ে দিয়ে (কারণ এখানে 401-এর অর্থ ভিন্ন — ক্রডেনেশিয়াল প্ৰত্যাখ্যাত), `statusCode` ও `cause` বজায় রাখে। সারভার থেকে আসা প্ৰকৃত বার্তা থাকলে (locked/unverified অ্যাকাউন্ট ইত্যাদি) তা রথিরো করে।

`Future<LoginResponse> startDemo()`

`POST /public/demo/session` কল করে (কনো বডি ছাড়া) ও টোকেনে সংরক্ষণ করে। `app.demo.enabled=false` বা সডি না-করা tenant-এর ক্ষেত্রে 404 আসে যা ডকুমেন্টে "ডমেো বন্ধ" উত্তর; সক্ষেত্রে (`!e.fromServer`) হলে "The demo is not available right now." বার্তা দিয়ে থরো করে।

`Future<void> logout(String? refreshToken)`

`refreshToken null`/খালি হলে সরাসরি রিটার্ন। অন্যথায় `POST /auth/logout` কল করে `{'refreshToken': refreshToken}` দিয়ে; যেকনো এরর নীরবে ধরে ফলে (ইতিমধ্যে লোকালি সাইন-আউট হয়ে গেছে, ডেডে নটেওয়ারকে কিছু পাল্টায় না)।

`Future<RegisteredUser> register(RegisterRequest request)`

`POST /auth/register` কল করে `RegisteredUser` পাবস করে রিটার্ন করে। নতুন tenant company ও owner অ্যাকাউন্ট তরৈ কিরে, কনিতু কাউকে সাইন-ইন করায় না — ইমইলে পাঠানো কোড এন্টার না করা প্ৰযন্ত অ্যাকাউন্ট অকজেো।

`Future<String> verifyEmail(String email, String code)`

`_api.postText('/auth/verify-email', VerifyEmailRequest(...).toJson())` কল করে প্লাইন-টক্সট রসেপন্স রিটার্ন করে।

`Future<String> resendVerification(String email)`

`_api.postText('/auth/resend-verification', ...)`। ইমইল রজিস্টার্ড থাক বা না থাক সবসময় 200 রিটার্ন করে (অ্যান্টি-এনুমারশেন)।

`Future<String> forgotPassword(String email)`

`_api.postText('/auth/forgot-password', ...)`। অ্যাকাউন্ট থাক বা না থাক সবসময় 200।

`Future<String> verifyResetCode(String email, String code)`

`_api.postText('/auth/verify-reset-code', ...)`।

`Future<String> resetPassword(ResetPasswordRequest request)`

`_api.postText('/auth/reset-password', request.toJson())`।

`Future<String> changePassword(ChangePasswordRequest request)`

`_api.postText('/auth/change-password', request.toJson())`। সঠিক current password-এই সফল হয়, এবং সফল হলে সার্ভার-সাইডে সব রফির্শে টোকেনে revoke করে দেয় — তাই caller-কে অবশ্যই ব্যবহারকারীকে সাইন-আউট করে লগইনে পাঠাতে হবে, মৃতপ্রায় টোকেনে নিয়ে চালিয়ে যাওয়া চলবে না।

`Future<ImpersonationStart> impersonate(int companyId, String reason)`

`reason.trim()` করে খালি হিলে "A reason is required to access a company." থরো করে (ব্যাকএন্ডে পাঠানোর আগেই — কারণ `reason` সার্ভার-সাইডে `@NotBlank` এবং `audit log`-এ লখো হয়)। অন্যথায় `POST /platform-admin/companies/$companyId/impersonate` কল করে `{'reason': trimmed}` দিয়ে, `ImpersonationStart.fromJson` পার্স করে রিটার্ন করে।

`Future<void> endImpersonation(String impersonationSessionId)`

`POST /platform-admin/impersonate/end` কল করে `{'impersonationSessionId': impersonationSessionId}` দিয়ে। এটি টোকেনে revoke করে না — ব্যাকএন্ডের `endImpersonation` শুধু `endedAt` সেটে করে; `tenant` টোকেনে নজি থেকে ময়োদোত্তীর্ণ না হওয়া পর্যন্ত বধৈ থাকে। তাই সশেন শেষে করা মূলত ক্লায়েন্টের কাজ (টোকেনে ধ্বংস করে), আর এই কলটি শুধু সহগামী বুককপিং — যে কারণে caller এই কল সফল হোক বা না হোক টোকেনে বাতলি করে।

`Future<UserProfile> getProfile()`

`GET /users/profile` কল করে `UserProfile.fromJson` পার্স করে রিটার্ন করে।

`Future<UserProfile> updateProfile(Map<String, dynamic> patch)`

`PATCH /users/profile` কল করে `patch` ম্যাপ দিয়ে, `UserProfile` পার্স করে রিটার্ন করে।

`Future<UserProfile> changeEmail(ChangeEmailRequest request)`

`PATCH /users/profile` কল করে `request.toJson()` দিয়ে (ইমেইল সাইন-ইন-এ টাইপ হয় বলে, `changePassword`-এর মতোই বর্তমান পাসওয়ার্ড দিয়ে ব্যাকএন্ড পুনরায়-যাচাই করে), `UserProfile` পার্স করে রিটার্ন করে।

`Future<List<String>> loadPermissions()`

`GET /users/permissions` কল করে `List<dynamic>` পায়, `whereType<String>()` দিয়ে ফিল্টার করে `unmodifiable (growable: false)` লিস্ট রিটার্ন করে — এই ইউজার প্রকৃতপক্ষে যসেব permission code ধারণ করে।

`Future<List<String>> loadPermissionCatalog()`

`GET /permissions` কল করে অসত্তিবশীল সব permission code রিটার্ন করে (প্রতিটি আইটেমে থেকে 'code' ফল্ডি নয়)। শুধু "এই ইউজার কিসবগুলো ধারণ করে" প্রশ্নের উত্তর দিতে দরকার — যাতে সব বক্স টকি-করা একটি কাস্টম role UI-তে owner-এর মতো আচরণ করে।

ফাইল: `lib/core/auth/impersonation_controller.dart`

সংক্ষিপ্ত ববিরণ: এই ফাইলটি একজন অ্যাডমিনি যখন কোনো টেন্যান্টের পরিচয় ধার করে (impersonation) অ্যাপ ব্যবহার করেন, সেই সশেনটি ট্র্যাক করে। এটি একটি Riverpod `Notifier` হিসেবে বাস্তবায়িত, যা টাইমার দিয়ে টোকেনের ময়োদ (৩০ মিনিট) স্বয়ংক্রিয়ভাবে পরবক্ষণ করে এবং ময়োদ শেষে হলে অ্যাডমিনিকে তার নজিরে সশেনে ফরিযি আনে। `AuthController`-এর থেকে এটি ইচ্ছাকৃতভাবে আলাদা রাখা হয়েছে, কারণ `AuthController` বলে অ্যাপ কার হয়ে কাজ করছে, আর এটি বলে সেই পরিচয়টি ধার করা কনি।

ক্লাস/উইজটে: ImpersonationController

এই ক্লাসটি `Notifier<ImpersonationSession?>` এক্সটেন্ড করে চলমান impersonation সেশন (থাকলে) ধরে রাখবে এবং একটি `Timer` দিয়ে তার ময়্যোদ শেষে হওয়ার মুহূর্তটি পর্যবেক্ষণ করে। কারণ impersonation টোকেনের কোনো রফ্রেশে টোকেনে থাকবে না, তাই ময়্যোদ শেষে হলে কউে না কউে সক্রিয়ভাবে ব্যবস্থা না নলি অ্যাপ তমেন টেন্যোন্ট দখোতে থাকবে যার সাথে সে আর যোগাযোগ করতে পারবে না, ফলে প্রতিটি অনুরোধে ৪০১ ত্রুটি আসবে।

`ImpersonationSession? build()`

`Notifier`-এর লাইফসাইকেলে মথেড, যা প্রোভাইডার তরৈ হওয়ার সময় একবার চলে। এটি `ref.onDispose(_cancel)` রেজিস্টার করে যাতে প্রোভাইডার নষ্ট হওয়ার সময় চলমান টাইমার বাতলি হয়। প্রাথমিক অবস্থা হিসেবে `null` রটরিন করে, অর্থাৎ শুরুতে কোনো impersonation সেশন সক্রিয় থাকে না।

`void set(ImpersonationSession? session)`

নতুন `session` ইনস্টল করে এবং ময়্যোদ শেষে হওয়ার টাইমার সেটে করে দেয়; `null` পাস করলে বর্তমান সেশন ও টাইমার দুটোই মুছে যায়। প্রথমতে এটি `_cancel()` কল করে আগরে যেকোনো টাইমার বাতলি করে, তারপর `state` আপডেট করে। যদি `session` ইতমিধ্য ময়্যোদোত্তীর্ণ হয়ে থাকে (যেমন স্টোরজে থেকে পুনরুদ্ধার করা একটি পুরনো সেশন), তাহলে `scheduleMicrotask(_expire)` দিয়ে অবলিম্বে ময়্যোদ শেষে করার প্রক্রিয়া শুরু করা হয়; অন্যথায় `session.remaining` সময় পরে `_expire` চালানোর জন্য একটি `Timer` বসানো হয়।

`void _cancel()`

প্রাইভটে হলেপার মথেড, যা চলমান `_timer` থাকলে তা বাতলি (`cancel()`) করে এবং `_timer`-কে `null` করে দেয়। এটি `build()`, `set()` এবং `_expire()` — সব জায়গা থেকেই ডাকা হয়, যাতে পুরনো টাইমার কখনো ফাঁকা অবস্থায় চলতে না থাকে।

`void _expire()`

টোকেনের ময়্যোদ শেষে হওয়ার মুহূর্তে টাইমার দ্বারা ডাকা প্রাইভটে মথেড। প্রথমতে `_cancel()` কল করে, তারপর `state` ইতমিধ্য `null` কনি যাচাই করে (যদি হয়, কছুই করে না)। এরপর `ref.read` (watch নয়, কারণ এটি নিরিমাণরে অনকে পরে চলে) দিয়ে `authControllerProvider`-এর `endImpersonation(expired: true)` মথেড অ্যাসঙ্ক্রোনাসভাবে (`unawaited`) কল করে, যা অ্যাডমিনিকে তার আসল সেশনে ফরিযি আনার দায়তিব পালন করে।

টপ-লভেলে ভ্যারযিবেল

`final impersonationControllerProvider`

`NotifierProvider<ImpersonationController, ImpersonationSession?>` — অ্যাপজুড়ে `ImpersonationController`-এর একক ইনস্ট্যান্স অ্যাক্সেস করার জন্য প্রোভাইডার।

`final isImpersonatingProvider`

একটি `Provider<bool>` যা `impersonationControllerProvider`-কে watch করে সটো `null` নয় কনি যাচাই করে `true/false` রটরিন করে। যেকোনো উইজটে থেকে সস্তায় (cheap) জানার উপায় যে বর্তমান পরচিঠিধার করা কনি, যাতে `UI` ভনিভাবে (যেমন ব্যানার দেখি়ে) আঁকা যায়।

ফাইল: lib/core/auth/permission_controller.dart

সংক্ষিপ্ত ববিরণ: ব্যবহারকারীর অনুমতি(permission) স্টে পরচালনা করে, যা ক্যাশ করা থাকে যাতো রাউট গার্ডরে মতো সঙ্ক্রোনাস চকেগুলো তাৎক্ষণিকভাবে উত্তর দিতে পারে। এই কারণেই সাইন-ইন প্রক্রিয়া নভেগিটে করার আগে অনুমতিলোড করে নেয়, প্রতটি স্ক্রনিকে lazily fetch করতে দেয় না।

ক্লাস/উইজটে: PermissionState

এই ইমডিটবেল ক্লাসটি ব্যবহারকারীর বর্তমান অনুমতিকোডসমূহ (codes), সিস্টেমের সম্পূর্ণ অনুমতি ক্যাটালগ (catalog), এবং একটি loaded ফ্ল্যাগ ধারণ করে যা বোঝায় বর্তমান অ্যাপ-সেশনে সার্বভার থেকে সত্যিকারের একটি fetch সম্পন্ন হয়েছে কিনা।

```
const PermissionState({this.codes = const {}, this.catalog = const {}, this.loaded = false})
```

কনস্ট্রাক্টর, যখনে তিনটি ফিল্ডেরই ডিফল্ট মান ফাঁকা স্টে এবং false — অর্থাৎ শুরুতে কোনো অনুমতিলোড করা নেই।

```
bool has(String? permission)
```

নির্দিষ্ট একটি permission কোড ব্যবহারকারীর কাছে আছে কিনা যাচাই করে। যদি permission null বা খালি স্ট্রিং হয়, তাহলে true রটার্ন করে (কোনো অনুমতির প্রয়োজন নেই ধরে), যা Angular সার্বভাসিরে hasPermission(null) === true আচরণের সাথে মিলে যায়। অন্যথায় codes স্টে permission আছে কিনা চকে করে।

```
bool hasAny(Iterable<String>? permissions)
```

একটি তালিকার যেকোনো একটি অনুমতি ব্যবহারকারীর কাছে আছে কিনা যাচাই করে। permissions null বা খালি হলে true রটার্ন করে (কোনো শর্ত নেই ধরে); অন্যথায় permissions.any(codes.contains) দিয়ে অন্তত একটি মিলি আছে কিনা পরীক্ষা করে।

```
bool get hasAll
```

একটি গিটোর, যা true রটার্ন করে শুধুমাত্র তখনই যখন ক্যাটালগ লোড হয়ে গেছে (catalog.isNotEmpty) এবং ব্যবহারকারী ক্যাটালগের প্রতটি কোড কভার করে (catalog.every(codes.contains)) — অর্থাৎ ব্যবহারকারীর কাছে সিস্টেমের সব অনুমতি আছে কিনা।

```
PermissionState copyWith({Set<String>? codes, Set<String>? catalog, bool? loaded})
```

একটি নতুন PermissionState ইনস্ট্যান্স রটার্ন করে, যখনে দেওয়া প্যারামিটারগুলো ওভাররাইট হয় এবং বাকিগুলো বর্তমান মান বজায় রাখে। ইমডিটবেল স্টেটে আপডেটের জন্য প্রমতি প্যাটার্ন।

ক্লাস/উইজটে: PermissionController

Notifier<PermissionState> এক্সটেন্ড করা এই ক্লাসটি অনুমতি স্টে ক্যাশ থেকে হাইড্রটে করা, সার্বভার থেকে লোড করা এবং সেভে করার সম্পূর্ণ জীবনচক্র নিয়ন্ত্রণ করে।

```
PermissionState build()
```

লাইফসাইকেলে মেথড, যা একটি ফাঁকা ডিফল্ট PermissionState() রটার্ন করে প্রোভাইডারের প্রাথমিক অবস্থা হিসেবে।

```
Future<void> hydrateFromCache()
```

নিরাপদ স্টোরেজে (secureStoreProvider) থেকে আগে সেভে করা codes ও catalog পড়ে state পূরণ করে, যাতো কোল্ড স্টার্টের পরে প্রথম ফ্রমেই নটেওয়ার্কের অপেক্ষা না করে গেটেডে UI দেখানো যায়। এটি loaded ফ্ল্যাগ স্টে করে না, কারণ এটি স্টোরেজের একটি অনুমান মাত্র যতক্ষণ না সার্বভার নিশ্চিতি করে।

`Future<void> load()`

`authRepositoryProvider` থেকে `loadPermissions()` ও `loadPermissionCatalog()` একসাথে (`Future.wait`) কল করে সারভার থেকে প্রকৃত ডটো আনবে, তারপর সেগুলো নরিপদ স্টোরজে লিখে রাখবে এবং `state`-কে `loaded: true` সহ নতুন `PermissionState` দিয়ে প্রতিস্থাপন করে। এখানে কোনো `error handling` নেই — ত্রুটি প্রোপাগেটে হয়ে যায়, কারণ সাইন-ইন প্রক্রিয়া এই কলরে জন্য অপেক্ষা করে এবং নীরবে খালি অনুমতিসিটে পলে ব্যবহারকারী এমন একটি অ্যাপে ঢুকে পড়বে যেখানে কিছুই দৃশ্যমান নয় এবং তার কারণ ব্যাখ্যা করা নেই।

`Future<void> refreshQuietly()`

`load()`-এর মতোই কাজ করে কিন্তু `try/catch` দিয়ে যেকোনো ব্যর্থতা নীরবে ধামাচাপা দেয়। পুনরুদ্ধার করা সেশনের ব্যাকগ্রাউন্ড ওয়ার্ম-আপে ব্যবহৃত হয়, যেখানে ক্যাশ করা স্টে ইতিমধ্যে যথেষ্ট ভালো এবং একটি সাময়িকি ব্যর্থতার কারণে কাউকে সাইন আউট করা উচিত নয়।

`void clear()`

`state`-কে একটি ফাঁকা ডিফল্ট `PermissionState()` দিয়ে রিসিটে করে দেয়। সাইন-আউটের সময় ব্যবহৃত হয় বলে অনুমান করা যায়।

টপ-লেভেলে ভ্যারিয়েবল

`final permissionControllerProvider`

`NotifierProvider<PermissionController, PermissionState>` — `PermissionController`-এর একক ইনস্ট্যান্স অ্যাক্সেস করার প্রোভাইডার।

`final hasPermissionProvider`

`Provider.family<bool, String>` — একটি নির্দিষ্ট অনুমতি `code`-এর জন্য `ref.watch(hasPermissionProvider("LEAVE_VIEW"))` আকারে সহজে ব্যবহারের সুবিধা দেয়, ভিতরে `permissionControllerProvider` `watch` করে `has(code)` কল করে।

ফাইল: `lib/core/config/env.dart`

সংক্ষিপ্ত বিবরণ: `BusinessOS` ব্যাকএন্ডের ঠিকানা কোথায় তা প্ল্যাটফর্ম অনুযায়ী নির্ধারণ করে (Android emulator, iOS simulator, রলিজি ব্লিড ইত্যাদি ভিন্নভাবে)। Angular অ্যাপের `environment.ts`-এর হার্ডকোড করা URL-এর বিপরীতে, এটি রানটাইমে সঠিক হোস্ট বের করে এবং ব্লিড-টাইম ওভাররাইডের সুযোগ দেয়।

ক্লাস/উইজটে: `Env`

একটি স্ট্যাটিক-ওনলি ইউটিলিটি ক্লাস (প্রাইভেটে কনস্ট্রাক্টর দিয়ে ইনস্ট্যান্স তৈরি রোধ করা) যা API হোস্ট, URL, টাইমআউট এবং ছবির URL রজিলাভ করার লজিক ধারণ করে।

`Env._()`

প্রাইভেটে নামহীন কনস্ট্রাক্টর, যা এই ক্লাসকে ইনস্ট্যান্সটি হওয়া থেকে আটকায — এটি শুধুমাত্র স্ট্যাটিক মেম্বার ধরে রাখার একটি নিমেষ্পেসের মতো কাজ করে।

static String get host

ব্যাকএন্ড হোস্টের ঠিকানা (/api ছাড়া) রিটার্ন করে। প্রথমে `_override` (কম্পাইল-টাইম `--dart-define=API_BASE_URL=...`) খালি নয় কনি দিতে; খালি না হলে তার শেষে থাকা `/api` বা `/api/` সাফক্স রগেক্স দিয়ে বাদ দিয়ে রিটার্ন করে। ওভাররাইড না থাকলে এবং `kReleaseMode` সত্য হলে একটি `StateError` ত্রুটি করে, কারণ রলিজি ব্লিডে ওভাররাইড ছাড়া চুপচাপ এমুলেটর/লোকালহোস্ট ঠিকানা পড়ে যাওয়া একটি বিভ্রান্তিকর "connect করা যাচ্ছে না" সমস্যা তৈরি করবে যা প্রডাকশনে ধরা পড়বে। অন্যথায়, Android হলে (!`kIsWeb && Platform.isAndroid`) `http://10.0.2.2:8085` রিটার্ন করে (এমুলেটরে হোস্ট-লুপব্যাক অ্যালাইস), নয়তো `http://localhost:8085` (iOS সিমুলেটর হোস্টের নেটেওয়ার্ক শয়ের করে বলে localhost-ই ঠিক আছে)।

static String get apiUrl

'\$host/api' রিটার্ন করে — প্রতিটি API কলরে বসে URL, Angular-এর `environment.apiUrl`-এর সমতুল্য।

static String get imgUrl

host-ই সরাসরি রিটার্ন করে (কোনো `/api` সাফক্স ছাড়া) — সার্ভ করা ছবি (avatar, attachment) লোড করার বসে, Angular-এর `resolveImageUrl()`-এর মতো যা `apiUrl` থেকে `/api` স্ট্রপি করে।

static const Duration connectTimeout / static const Duration receiveTimeout

যথাক্রমে ৬০ সেকেন্ড ও ৯০ সেকেন্ডের ধ্রুবক `Duration`, যা HTTP কানেকশন ও রসেপন্স টাইমআউট নির্ধারণ করে।

static String? resolveImageUrl(String? url)

ব্যাকএন্ড থেকে ফেরত পাওয়া সম্ভাব্য আপেক্ষিক (relative) ছবির পাথকে `Image.network` দিয়ে লোডযোগ্য পূর্ণ URL-এ রূপান্তর করে। `url` null বা খালি হলে null রিটার্ন করে। যদি `url` ইতিমধ্যে `http://`, `https://` বা `data:` দিয়ে শুরু হয়, তাহলে অপরিবর্তিত রিটার্ন করে। অন্যথায়, `url` দিয়ে শুরু হলে `'$imgUrl$url'`, নাহলে `'$imgUrl/$url'` রিটার্ন করে (একটি / জুড়ে দিয়ে)। এটি Angular-এর `AuthService.resolveImageUrl`-এর পোর্ট।

ফাইল: lib/core/network/api_client.dart

সংক্ষিপ্ত ববিরণ: অ্যাপরে একমাত্র HTTP এন্ট্রি পয়েন্ট — Angular-এর `ApiService` ও তার দুটি ইন্টারফেসের ফ্লাটার সমতুল্য। প্রতিটি মিথড ব্যর্থতা `ApiException.from`-এর মধ্য দিয়ে পাঠায়, ফলে কলারদের শুধু একটি error টাইপ নিয়ে কাজ করতে হয়, Dio ইমপোর্ট করতে হয় না।

ক্লাস/উইজটে: ApiClient

Dio ইনস্ট্যান্স মোডানো এই ক্লাস JSON, প্লাইন-টেক্সট, বাইনারি, ও মাল্টিপার্ট আপলোড রকিওয়েস্টের জন্য এককেন্দ্রিক মিথড সরবরাহ করে, এবং সব রকিওয়েস্টের সাথে `AuthInterceptor` যুক্ত থাকে যা টোকেন সংযুক্তিও রিফ্রেশে পরিচালনা করে।

`ApiClient({required SecureStore store, required Future<void> Function() onSessionExpired, void Function()? onTokensRefreshed})`

কনস্ট্রাক্টর, যা প্রধান `dio` ইনস্ট্যান্স ও একটি ইন্টারসপেটরবাহীন `refreshDio` তৈরি করে (রিফ্রেশ কলটিকে নজিই ৪০১ হয়ে রিফ্রেশে শুরু করা থাকে আটকাত, অর্থাৎ রিকারিশন এড়াত)। তারপর প্রধান `dio`-তে একটি `AuthInterceptor` যুক্ত করে, যাকে `store`, `dio` (একটি ফাংশন হিসেবে, কারণ ইন্টারসপেটরটি তৈরি হয় সেই ক্লায়েন্ট দ্বারা যার `Dio` তার পরে দরকার হয় রকি়োয়েস্ট রপিলে করার জন্য), `refreshDio`, `onSessionExpired`, এবং `onTokensRefreshed` পাস করা হয়।

late final Dio dio

মূল `Dio` ইনস্ট্যান্স, যা প্রতিটি HTTP মেথড ব্যবহার করে।

`static BaseOptions _baseOptions()`

`Env.apiUrl`, `Env.connectTimeout`, `Env.receiveTimeout` সহ একটি `BaseOptions` তৈরি করে, এবং `validateStatus` কে এমনভাবে সেট করে যাতে শুধু ২০০-২৯৯ রঞ্জে স্ট্যাটাসকে সফল ধরা হয় — বাকি সব নন-২xx রেসপন্স `DioException` হিসেবে ইন্টারসপেটরে পৌঁছায়, যাতে ৪০১ পরচালনার একটমাত্র জায়গা থাকে।

`Future<T> get<T>(String path, {Map<String, dynamic>? query})`

একটি GET রকি়োয়েস্ট `_guard` রূপারের ভেতরে পাঠায়, `query` প্যারামিটারগুলো `_clean` দিয়ে পরিশ্কার করে পাঠানো হয়, এবং রেসপন্স ডটো `T` টাইপে কাস্ট করে রিটার্ন করে।

`Future<T> post<T>(String path, [Object? body])`

একটি POST রকি়োয়েস্ট পাঠায়, `body` না দিলে একটি খালি ম্যাপ (`const {}`) পাঠানো হয়; রেসপন্স `T`-তে কাস্ট করে রিটার্ন করে।

`Future<T> put<T>(String path, [Object? body])`

একটি PUT রকি়োয়েস্ট, `post`-এর মতোই আচরণ করে।

`Future<T> patch<T>(String path, [Object? body])`

একটি PATCH রকি়োয়েস্ট, একইরকম ডিফল্ট-বডি লিজকি সহ।

`Future<T> delete<T>(String path)`

একটি DELETE রকি়োয়েস্ট পাঠায় এবং রেসপন্স `T`-তে কাস্ট করে রিটার্ন করে।

`Future<String> postText(String path, [Object? body])`

এমন এন্ডপয়েন্টের জন্য যা `ResponseEntity<String>` ঘোষণা করে — JSON হিসেবে ডিকোড করতে বললে সফল ২০০ রেসপন্সও পার্স-ফাইলগিরে পরণিত হতে পারে, তাই `ResponseType.plain` ব্যবহার করে POST পাঠায় এবং কাঁচা স্ট্রিং রিটার্ন করে (`null` হলে খালি স্ট্রিং)।

`Future<String> patchText(String path, [Object? body])`

`postText`-এর মতোই, কিন্তু PATCH মেথড ব্যবহার করে।

`Future<String> deleteText(String path)`

`postText`-এর মতোই, কিন্তু DELETE মেথড ব্যবহার করে, কোনো বডি ছাড়া।

`Future<List<int>> getBytes(String path, {Map<String, dynamic>? query})`

বাইনারি ডটো (যেমন জনোরেটে করা পেস-স্লপি PDF, রপিোর্ট এক্সপোর্ট) আনার জন্য `ResponseType.bytes` ব্যবহার করে GET রকি়োয়েস্ট পাঠায় এবং বাইটের তালিকা রিটার্ন করে (`null` হলে খালি তালিকা)।

`Future<UploadedFile> uploadDocument(String filePath, String fileName)`

`POST /api/upload` এন্ডপয়েন্টে একটি ফাইল মাল্টিপার্ট ফর্ম হিসেবে আপলোড করে — এটি একমাত্র সাধারণ ডকুমেন্ট এন্ডপয়েন্ট যার মধ্য দিয়ে প্রতিটি ফিচারে অ্যাটাচমেন্ট সুবিধা যায়, যাত ১০MB সীমা ও কন্টেন্ট-টাইপ যাচাই শুধু ব্যাকএন্ডে একজায়গায় এবং ফ্রন্টএন্ডে একটি কিলে থাকে।

`MultipartFile.fromFile` দিয়ে ফাইল প্রস্তুত করে, রেসপন্স `UploadedFile.fromJson` দিয়ে পার্স করে রিটার্ন করে।

`Future<UploadedFile> uploadAvatar(String filePath, String fileName)`

`POST /api/upload/avatar` এন্ডপয়েন্টে আলাদা নথিমে (শুধু ছবি, এক্সটেনশন নয় বরং প্রকৃত ইমেজে ডটো যাচাই করে, ৫MB সীমা) ফাইল আপলোড করে। শুধু ফাইল সংরক্ষণ করে একটি URL ফেরত দেয়; কোনো রেকর্ডে সেই URL সবে না করা পর্যন্ত কিছুই কারো সাথে সংযুক্ত হয় না।

`Future<T> postFile<T>(String path, String filePath, String fileName)`

যেকোনো এন্ডপয়েন্টে একটি `file` পার্ট সহ মাল্টিপার্ট `POST` পাঠায় যা আপলোড রসদিরে বদলে অন্য কিছু ফেরত দেয় (যেমন একটি স্টেটমেন্ট ইম্পোর্ট, যা ফাইল থেকে কী তৈরি হলে তা রিটার্ন করে, কোথায় সংরক্ষিত হলে তা নয়)। রেসপন্স `T`-তে কাস্ট করে রিটার্ন করে।

`Future<PagedResponse<T>> getPaged<T>(String path, T Function(Map<String, dynamic>) fromItem, {int page = 0, int size = 20, Map<String, dynamic>? query})`

পজেনিটেড ডটো আনার জন্য `GET` রকিওয়েস্ট পাঠায়, `page` (নেগেটিভি হলে ০-এ কল্যাম্প) ও `size` (০ বা কম হলে ২০-এ কল্যাম্প) কোয়েরে প্যারামিটার হিসেবে যুক্ত করে, সাথে `_clean(query)`-এর ফলাফল। রেসপন্স `PagedResponse<T>.fromJson`-এ পাস করে পার্স করে, `fromItem` কনভার্টার ও ফলব্যাক পজে/সাইজ সহ।

`static Map<String, dynamic>? _clean(Map<String, dynamic>? query)`

`null` বা খালি স্ট্রিং মানসম্পন্ন কোয়েরে প্যারামিটারগুলো বাদ দেয়, যাত সে না করা ফিল্টার `?status=null`-এ পরণিত না হয় (যা ব্যাকএন্ড একটি প্রকৃত মান হিসেবে পার্স করার চেষ্টা করবে)। ফলাফল খালি হলে `null` রিটার্ন করে, নাহলে পরিস্কার করা ম্যাপ।

`static Future<T> _guard<T>(Future<T> Function() request)`

একটি সাধারণ রুপার, যা `request` চালায় এবং যেকোনো ব্যতক্রম ধরে `ApiException.from(error, stackTrace)`-এ রূপান্তরিত করে থরো করে। প্রতিটি পাবলিক মেথডে এই রুপার ব্যবহার হয়, যাত সে সব `error` একটমাত্র আকারে কলারের কাছে পৌঁছায়।

ফাইল: `lib/core/network/api_exception.dart`

সংক্ষিপ্ত বিবরণ: ব্যাকএন্ড বা ট্রান্সপোর্ট ব্যর্থতাকে একটি প্রদর্শনযোগ্য ফর্মে রূপান্তরিত করে। স্ক্রিনিগুলো `DioException`-এর বদলে এই ক্লাস ধরে, যাত কোনো উইজটেকে জানতে হয় না API কীভাবে `error` রিপোর্ট করে।

ক্লাস/উইজটে: `ApiException`

`Exception` ইন্টারফেসে বাস্তবায়নকারী একটি ইমিউটেবল ক্লাস, যা একটি মানুষের-পড়ার-উপযোগী বার্তা, ঐচ্ছিক স্ট্যাটাস কোড, মূল কারণ (`cause`), এবং বার্তাটি সারভার থেকে এসছে কিনা তা ধারণ করে।

`const ApiException(this.message, {this.statusCode, this.cause, this.fromServer = false})`

কনস্ট্রাক্টর, যেখানে `message` আবশ্যিক, বাকি সব ঐচ্ছকি এবং `fromServer` ডিফল্টভাবে `false`।

`bool get isUnauthorized`

`statusCode == 401` কনি রটার্ন করে — সেশন ময়োদোতীরণ নরিদশে করে প্রায় সর্বত্র।

`bool get isForbidden`

`statusCode == 403` কনি রটার্ন করে — অনুমতি নেই নরিদশে করে।

`bool get isNotFound`

`statusCode == 404` কনি রটার্ন করে।

`bool get isNetwork`

`statusCode == null` কনি রটার্ন করে — অর্থাৎ কোনো রসেপন্সই আসেনি (DNS, সংযোগ প্রত্যাখ্যাত, টাইমআউট, এয়ারপ্লেনে মোড ইত্যাদি)। কয়েকটি স্ক্রিনি ৪০১/৪০৩-কে খালি স্টেটে হিসেবে দেখায় `error`-এর বদলে, তার জন্য `isUnauthorized/isForbidden` গটোর আলাদাভাবে দরকার হয়।

`String toString()`

`'ApiException($statusCode): $message'` ফরম্যাটে একটি স্ট্রিং রটার্ন করে, ডিবাগিং/লগিং-এর জন্য উপযোগী।

`static ApiException from(Object error, [StackTrace? _])`

যেকোনো `error`-কে একটি `ApiException`-এ রূপান্তর করে এই ক্লাসের ফ্যাক্টরি মেথড। `error` ইতিমধ্যে `ApiException` হলে তা যমেন আছে তমেন রটার্ন করে। `error DioException` না হলে একটি সাধারণ বার্তা ('Something went wrong...') সহ `ApiException` তৈরি করে। `DioException` হলে, রসেপন্সের `statusCode` বের করে এবং `_messageFromBody` দিয়ে রসেপন্স বডি থেকে বার্তা বের করার চেষ্টা করে; পাওয়া গেলে সটে `fromServer: true` সহ ব্যবহার করে, না পাওয়া গেলে `_fallbackFor` দিয়ে একটি জেনেরিক বার্তা তৈরি করে।

`static String? _messageFromBody(Object? data)`

Angular-এর `extractErrorMessage`-এর পোর্ট। কছি এন্ডপয়ন্টে `ResponseEntity<String>` ঘোষণা করে এবং প্লাইন রসেপন্স-টাইপে রকিওয়েস্ট করা হয়, তাই তাদের `error` বডি একটি ডিকোড করা ম্যাপ না হয়ে JSON ধারণকারী কাঁচা স্ট্রিং হিসেবে আসে। `data` একটি স্ট্রিং হলে, ট্রমি করে খালি কনি দেখে; { দিয়ে শুরু হলে JSON হিসেবে ডিকোড করার চেষ্টা করে এবং সফল হলে ম্যাপ হিসেবে `_pick` কল করে, ব্যর্থ হলে কাঁচা ট্রমি করা স্ট্রিং রটার্ন করে। `data` সরাসরি একটি `Map` হলে `_pick(data)` কল করে। অন্য কোনো টাইপ হলে `null` রটার্ন করে।

`static String? _pick(Map<dynamic, dynamic> map)`

`'message', 'error', 'detail'` — এই কী গুলো ক্রমানুসারে ম্যাপে খুঁজে প্রথম খালি-না-হওয়া স্ট্রিং মান পলে (ট্রমি করে) রটার্ন করে; কছি না পলে `null`।

`static String _fallbackFor(DioException error, int? status)`

সারভার থেকে কোনো ব্যবহারযোগ্য বার্তা পাওয়া না গেলে, HTTP স্ট্যাটাস কোড (৪০০, ৪০১, ৪০৩, ৪০৪, ৪০৯, ৫০০/৫০২/৫০৩) অনুযায়ী একটি প্রাক-নরিধারতি বার্তা রটার্ন করে। স্ট্যাটাসের সাথে কোনো মলি না পলে, `error.type` (কানেকশন টাইমআউট, বাতলি, কানেকশন এরর, খারাপ সার্টিফিকেট ইত্যাদি) অনুযায়ী আরেকটি সুইচে যায়; `connectionError/unknown` টাইপে `error.error` একটি `SocketException` কনি যাচাই করে সংযোগ-সংক্রান্ত নরিদষ্টি বার্তা দেয়, নাহলে জেনেরিক সংযোগ-ত্রুটি বার্তা।

ফাইল: lib/core/network/auth_interceptor.dart

সংক্ষিপ্ত ববিরণ: প্রতটি রিকি়োয়েস্টে বয়োরার টোকনে সংযুক্ত করে, এবং ব্যাকএন্ড ৪০১ (ময়োদোতীর্ণ) ফরেত দলিে স্বচ্ছভাবে একবার টোকনে রফিরশে করে। Angular-এর `authInterceptor`-এর পোর্ট, যার সবচেয়ে গুরুত্বপূর্ণ অংশ হলো — একসাথে শুধুমাত্র একট রফিরশে চলতে পারে, যাতে একসাথে একাধিক রিকি়োয়েস্ট ৪০১ হলে একাধিক রফিরশে শুরু না হয়ে ব্যবহারকারী মাঝপথে সাইন-আউট না হয়ে যায়।

ক্লাস/উইজটে: AuthInterceptor

`Interceptor` এক্সটেন্ড করা এই ক্লাস প্রতটি আউটগোয়িং রিকি়োয়েস্টে টোকনে যুক্ত করা এবং ৪০১ রসেপন্সে রফিরশে-ও-রট্রাই লজিকি পরচালনা করার দায়িত্ব পালন করে, একট শিয়োর্ড `Completer` দয়িে সমান্তরাল রফিরশে প্রতরিোধ করে।

`AuthInterceptor({required SecureStore store, required Dio Function() dio, required Dio refreshDio, required Future<void> Function() onSessionExpired, this.onTokensRefreshed})`

কনস্ট্রাক্টর, যা `store`, `dio` (লেটে-বাউন্ড ফাংশন, কারণ ইন্টারসপেটরটি যে ক্লায়েন্ট তরৈকিরে তার `Dio` পরে দরকার হয় রিকি়োয়েস্ট রপিলে করার জন্য), `refreshDio` (কোনো ইন্টারসপেটর নই এমন একট বয়োর `Dio`, যাতে রফিরশে কল নজিহে ৪০১ হয়ে আবার রফিরশে শুরু করার রকি়ারশন এড়ানো যায়), `onSessionExpired`, এবং ঐচ্ছকি `onTokensRefreshed` কলব্যাক গ্রহণ করে।

`Future<void> onRequest(RequestOptions options, RequestInterceptorHandler handler)`

প্রতটি আউটগোয়িং রিকি়োয়েস্টে আগে চলে। যদি রিকি়োয়েস্টে পাথ অথ-এন্ডপয়ন্ট না হয় (`!_isAuthEndpoint`), তাহলে `_store.readAccessToken()` দয়িে টোকনে পড়ে এবং তা খালি না হলে `Authorization: Bearer $token` হডোর যুক্ত করে। শেষে `handler.next(options)` কল করে রিকি়োয়েস্ট চালয়িে যায়।

`Future<void> onError(DioException err, ErrorInterceptorHandler handler)`

কোনো রিকি়োয়েস্ট ব্যর্থ হলে চলে। প্রথমে যাচাই করে রিকি়োয়েস্টটি "রট্রাইযোগ্য" কনি — স্ট্যাটাস ৪০১, অথ-এন্ডপয়ন্ট নয়, এবং আগে একবার রট্রাই না হয়ে থাকা (`options.extra[retryKey] != true`)। রট্রাইযোগ্য না হলে `handler.next(err)` দয়িে এগয়িে যায়। রট্রাইযোগ্য হলে, প্রথমে impersonation সশেন সক্রয়ি কনি (`_store.readImpersonation()`) যাচাই করে — থাকলে রফিরশে না করই `_onSessionExpired()` কল করে বরেয়িে যায় (কারণ impersonation টোকনেরে নজিস্ব রফিরশে টোকনে নই, আর অ্যাডমিনেরে রফিরশে টোকনে ব্যবহার করলে অজান্তেই অ্যাডমিনেরে আসল পরচয়িে ফরয়িে দবে অথচ `UI` টেন্যান্ট দেখাতই থাকবে)। এরপর `refreshToken` পড়ে; না থাকলে বা খালি হলে `_onSessionExpired()` কল করে বরেয়িে যায়। থাকলে `_refresh(refreshToken)` কল করে নতুন টোকনে পাওয়ার চেষ্টা করে; ব্যর্থ হলে (`_performRefresh` ইতিমধ্যে সদিধান্ত নয়িছে লগআউট দরকার কনি) `handler.next(err)` দয়িে বরেয়িে যায়। সফল হলে, মূল রিকি়োয়েস্টে `extra[retryKey] = true` সটে করে (দ্বিতীয়বার রট্রাই এড়াতে), নতুন টোকনে দয়িে হডোর আপডটে করে, `_dio().fetch(options)` দয়িে রিকি়োয়েস্টটি পুনরায় পাঠায়, এবং সফল হলে `handler.resolve(response)` দয়িে সমাধান করে; আবার ব্যর্থ হলে `handler.next(retryError)` কল করে।

`Future<String> _refresh(String refreshToken)`

রফ্রেশে অনুরোধ সমন্বয় করে — যদি ইতিমধ্যে `_inFlight` একটি `Completer` চলমান থাকে, তাহলে সেই একই `future` রিটার্ন করে (নতুন রফ্রেশে শুরু না করে)। নাহলে একটি নতুন `Completer<String>` তৈরি করে `_inFlight`-এ বসায়, `_performRefresh(refreshToken)` কল করে, সফল হলে `completer` সম্পন্ন করে ও `_inFlight`-কে `null` করে, ব্যর্থ হলে `completer`-এ `error` পাঠায় ও একইভাবে `_inFlight` পরিস্কার করে। শেষে `completer.future` রিটার্ন করে।

`Future<String> _performRefresh(String refreshToken)`

প্রকৃত রফ্রেশে নটেওয়ার্ক কল সম্পন্ন করে। `_refreshDio.post('/auth/refresh', data: {'refreshToken': refreshToken})` কল করে; রসেপন্স `Map` না হলে বা `accessToken` স্ট্রিং না হলে একটি `DioException` ত্রুটি করে। বৈধ রসেপন্স পলে, `_store.writeTokens` দিয়ে নতুন `accessToken` ও `refreshToken` সংরক্ষণ করে (মন্তব্যে উল্লেখ আছে যে `/auth/refresh` শুধু টোকেনে ফেরত দেয়, ব্যবহারকারীর তথ্য নয়, তাই ক্যাশ করা ইউজার ডটো অপরিবর্তিত থাকে), `onTokensRefreshed?.call()` কল করে, এবং নতুন `accessToken` রিটার্ন করে। `DioException` ধরা পড়লে, স্ট্যাটাস কোড পরীক্ষা করে — ৪০০, ৪০১, বা ৪০৩ হলে নিশ্চিতভাবে টোকেনে মৃত ধরে নিয়ে `_onSessionExpired()` কল করে, নাহলে (যেমন সার্ভার সাময়িকভাবে পুনরায় চালু হচ্ছে) সেশন শেষ না করাই শুধু `error rethrow` করে।

`static bool _isAuthEndpoint(String path)`

`path`-এ `/auth/login`, `/auth/register`, বা `/auth/refresh` থাকলে `true` রিটার্ন করে — এই এন্ডপয়েন্টগুলোতে বোয়ারার টোকেনে যুক্ত করা বা রফ্রেশে-রিট্রাই লজিক প্রযোজ্য নয়।

টপ-লেভেলে ধ্রুবক

`static const _retriedKey`

`'zuhoo.retried'` — `RequestOptions.extra` ম্যাপে একটি ফিল্ডাগ কী হিসেবে ব্যবহৃত হয়, যা নির্দেশ করে একটি রিকোয়েস্ট ইতিমধ্যে একবার রফ্রেশের পর রিট্রাই হয়েছে কিনা, যাতে অসীম রিট্রাই লুপ এড়ানো যায়।

ফাইল: `lib/core/network/paged_response.dart`

সংক্ষিপ্ত বিবরণ: Spring Data-র `Page<T>` এন্ডপয়েন্ট থেকে আসা রসেপন্স পার্স করার জনৈকিক ক্লাস। ব্যাকএন্ড কখনো `number/size` ফিল্ড ফলিড হিসেবে, কখনো `page` অবজেক্টের নচি নেস্টেডে করে পজেনিশন তথ্য পাঠায়—এই ক্লাস উভয় বানানকেই একটি একক ফিল্ড আকারে স্বাভাবিকীকরণ (`normalise`) করে, যাতে কোনো লস্ট স্ক্রনিকে এই পার্থক্য নিয়ে ভাবতে না হয়।

ক্লাস/উইজটে: `PagedResponse<T>`

একটি পজের ফলাফল (`content`), মোট আইটেম সংখ্যা, মোট পজে, বর্তমান পজে ও পজের আকার ধারণ করে এমন একটি ইমডিটবেল জনৈকিক ডটো ক্লাস। এটি Angular অ্যাপের `ApiService.getPaged()`-এর সমতুল্য একই স্বাভাবিকীকরণ যুক্ত অনুসরণ করে, যাতে ব্যাকএন্ডের বিভিন্ন সিরিয়লাইজেশন ফরম্যাট (`flat`, `nested page`, অথবা `bare list`) সব ক্ষেত্রেই একটিমাত্র সামঞ্জস্যপূর্ণ ইন্টারফেস পাওয়া যায়।

`const PagedResponse({required this.content, required this.totalElements, required this.totalPages, required this.currentPage, required this.pageSize})`

প্রধান কনস্ট্রাক্টর, যা সব ফিল্ড সরাসরি প্রয়োজনীয় (`required`) প্যারামিটার হিসেবে গ্রহণ করে একটি ইমডিটবেল ইনস্ট্যান্স তৈরি করে। এটি সাধারণত `fromJson` ফ্যাক্টরির ভেতর থেকে ব্যবহৃত হয়, তবে সরাসরি টেস্ট বা ডিফল্ট মান তৈরি জন্যও ব্যবহারযোগ্য।

`bool get isEmpty`

`content` লিস্টটি খালি কিনা তা যাচাই করে একটি সুবিধাজনক গটোর, যা UI স্তরে "কোনো ফলাফল নেই" অবস্থা দেখানোর জন্য ব্যবহৃত হয়।

`bool get hasMore`

`currentPage + 1 < totalPages` শর্ত পরীক্ষা করে বলে দেয় আরও পজে আনা বাকি আছে কিনা, যা ইনফিনিটি-স্ক্রল বা "আরও লোড করুন" বাটনের যুক্তির ভিত্তি।

`const PagedResponse.empty()`

সব ফল্ড শূন্য/খালি মান দিয়ে সেটে করা একটি `const` কনস্ট্রাক্টর, যা একটি সম্পূর্ণ খালি পজে প্রতিনিধিত্ব করে—লোডিং শুরুর আগের প্রাথমিক স্টেটে বা এরর অবস্থায় ফলব্যাক হিসেবে ব্যবহারযোগ্য।

`factory PagedResponse.fromJson(Object? json, T Function(Map<String, dynamic>) fromItem, {int fallbackPage = 0, int fallbackSize = 20})`

API থেকে আসা কাঁচা JSON (যা `List` অথবা `Map` হতে পারে) থেকে `PagedResponse<T>` তৈরিকরার মূল ফ্যাক্টরি মেথড। যদি `json` সরাসরি একটি `List` হয়, তাহলে সেটিকে একক সম্পূর্ণ পজে হিসেবে বিবেচনা করা হয় (কিছু এন্ডপয়েন্ট পজের বদলে বয়োর লিস্ট রিটার্ন করে, এক্ষেত্রে স্ক্রিনি ফাইল না করিয়ে তা সামলানো হয়)। `Map` হলে, এটি প্রথমে `page` কী-এর নচি নেস্টেডে অবজেক্ট আছে কিনা দেখে, তারপর অভ্যন্তরীণ `pick` হলেপার ফাংশনের মাধ্যমে ফ্ল্যাট কী (`totalElements`, `totalPages` ইত্যাদি) বা Spring-এর কী (`number`, `size`) থেকে মান বের করে, কোনোটিই না পলে `fallbackPage/fallbackSize` ব্যবহার করে। `content` লিস্টের প্রতিটি `Map<String, dynamic>` আইটেমকে `fromItem` কলব্যাক দিয়ে `T` টাইপে রূপান্তর করা হয়।

ফাইল: `lib/core/network/uploaded_file.dart`

সংক্ষিপ্ত বিবরণ: `POST /api/upload` এন্ডপয়েন্টের (ব্যাকএন্ডের `FileUploadController`, ১০MB কার্যকর সীমা, এক্সটেনশনের বিপরীতে সার্ভার-সাইড কনটেন্ট-টাইপ যাচাইসহ) রেসপন্স মডলে করার একটি ছোট ইমিউটবেল ক্লাস।

ক্লাস/উইজটে: `UploadedFile`

একটি আপলোড করা ফাইলের নাম (`fileName`) ও URL (`fileUrl`) ধারণ করে এমন একটি সাধারণ ডটো ক্লাস, যা ফাইল আপলোড প্রবাহে সার্ভারের রেসপন্স ক্লায়েন্ট কোডে ব্যবহারযোগ্য আকারে ধরে রাখে।

`const UploadedFile({required this.fileName, required this.fileUrl})`

উভয় ফল্ড সরাসরি সেটে করে একটি ইমিউটবেল ইনস্ট্যান্স তৈরিকরার প্রধান কনস্ট্রাক্টর।

`factory UploadedFile.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে `UploadedFile` তৈরিকরে; `fileName` ও `fileUrl` কী দুটো থাকা বা `String` না হলে নরিপদে খালি স্ট্রিং (") ফলব্যাক হিসেবে ব্যবহার করে, ফলে ম্যালফর্মড রেসপন্সে অ্যাপ ক্র্যাশ করে না।

ফাইল: lib/core/providers.dart

সংক্ষিপ্ত ববিরণ: অ্যাপ-ব্যাপী ক্রস-কাটিং সিঙ্গেলটনগুলোর (Riverpod প্রোভাইডার) সংজ্ঞা—Angular-এর DI কন্টইনার যত্নে অ্যাপ-ওয়াইড সার্ভিস সরবরাহ করে, এটি তার Flutter সমতুল্য। এখানে কোনো ক্লাস নেই, শুধু উপ-লেভেলে প্রোভাইডার ভ্যারিয়েবল।

উপ-লেভেলে প্রোভাইডার: sharedPreferencesProvider

`dart final sharedPreferencesProvider = Provider<SharedPreferences>(...) ` একটি Provider<SharedPreferences> যার ডিফল্ট বডি UnimplementedError থরো করে। main() ফাংশনে আসল ইনস্ট্যান্স async-এ await করার পর এটি ওভাররাইড করা হয়, যাত অ্যাপের বাকি অংশকে async প্রফরেন্সেসে হ্যান্ডলে নিয়ে মাথা ঘামাতে না হয়।`

উপ-লেভেলে প্রোভাইডার: secureStorageProvider

`dart final secureStorageProvider = Provider<FlutterSecureStorage>((ref) => const FlutterSecureStorage()); ` ডিফল্ট কনস্ট্রাক্টর ব্যবহার করে FlutterSecureStorage-এর একটি ইনস্ট্যান্স সরবরাহ করে। Android-এ v11 থেকে ডিফল্ট ইমপ্লিমেন্টেশন ইতিমধ্যেই AES-GCM ডটো-কী প্ল্যাটফর্ম KeyStore-এর RSA কী দিয়ে রাপ করা থাকে, তাই পুরনো encryptedSharedPreferences ফ্ল্যাগের মতো কোনো দুর্বল মোড এড়ানোর প্রয়োজন নেই।`

উপ-লেভেলে প্রোভাইডার: secureStoreProvider

`dart final secureStoreProvider = Provider<SecureStore>((ref) => SecureStore(ref.watch(secureStorageProvider))); ` SecureStore ক্লাসের ইনস্ট্যান্স তৈরি করে, যা secureStorageProvider-এর উপর নির্ভরশীল (watch করে)।`

উপ-লেভেলে প্রোভাইডার: apiClientProvider

`dart final apiClientProvider = Provider<ApiClient>((ref) { ... }); ` ApiClient তৈরি করে, যাকে secureStoreProvider থেকে স্টোর ও একটি onSessionExpired কলব্যাক দেওয়া হয়। এই কলব্যাক authControllerProvider-কে watch না করে read করে ডাকা হয়, কারণ এটি রিকিওয়েস্টের ভেতর থেকে, কনস্ট্রাকশনের অনেকে পরে ফায়ার হয়—watch করলে প্রতিটি সাইন-ইনে HTTP ক্লায়েন্ট রিবিল্ড হয়ে যেত।`

উপ-লেভেলে প্রোভাইডার: authRepositoryProvider

`dart final authRepositoryProvider = Provider<AuthRepository>((ref) => AuthRepository(ref.watch(apiClientProvider), ref.watch(secureStoreProvider))); ` AuthRepository-র ইনস্ট্যান্স তৈরি করে, যা apiClientProvider ও secureStoreProvider উভয়ের উপর নির্ভরশীল।`

ফাইল: lib/core/push/push_service.dart

সংক্ষিপ্ত ববিরণ: পুশ নোটিফিকেশনের সম্পূর্ণ জীবনচক্র পরিচালনা করে—অনুমতি চাওয়া, FCM টোকেন রজিস্ট্রেশন/আনরজিস্ট্রেশন, ফোরগ্রাউন্ড/ব্যাকগ্রাউন্ড ডটো-মসেজে থেকে লোকাল নোটিফিকেশন তৈরি, এবং ট্যাপ করা নোটিফিকেশনকে সঠিক স্ক্রিনে নেভিগেট করা। ব্যাকগ্রাউন্ড সবসময় ডটো-অনলি FCM পলোড পাঠায় (notification: নয়), তাই ক্লায়েন্টকেই সিদ্ধান্ত নিতে হয় কী দেখানো হবে।

টপ-লভেলে কনস্ট্যান্ট: `_androidChannel`

``dart const _androidChannel = AndroidNotificationChannel('zuho_default', 'Notifications', description: ..., importance: Importance.high);`` Android নোটিফিকেশন চ্যানেলের সংজ্ঞা, যা `PushService.init()` এবং ব্যাকগ্রাউন্ড হ্যান্ডলার উভয় জায়গা থেকে চ্যানেলে তৈরি জন্য ব্যবহৃত হয়।

টপ-লভেলে ফাংশন: `Future<void>`

`firebaseMessagingBackgroundHandler(RemoteMessage message)`

`@pragma('vm:entry-point')` চহিনতি একটি টপ-লভেলে ফাংশন, যা অ্যাপ চলমান না থাকা অবস্থায় ডটো মসেজে আসলে প্ল্যাটফর্ম একটি আলাদা আইসোলটে চালায়। যহেতু এই আইসোলটে চলমান অ্যাপরে কোনো স্টেটে (কোনো `ProviderScope`, সাইন-ইন সেশন) নই, তাই এটি `Firebase.initializeApp()` দিয়ে নজি থেকেই যথেষ্ট প্রাথমিকীকরণ করে এবং `_showDataMessage` কল করে একটি লোকাল নোটিফিকেশন দেখায়।

টপ-লভেলে ফাংশন: `Future<void> _showDataMessage(Map<String, dynamic> data, FlutterLocalNotificationsPlugin plugin)`

একটি ডটো-অনলি FCM পলোডক লোকাল নোটিফিকেশন হিসাবে রেন্ডার করে। `title` ও `body` উভয়ই খালি হলে কিছু দেখায় না এবং সরাসরি রিটার্ন করে। অন্যথায় নোটিফিকেশন চ্যানেলে তৈরি নিশ্চিতি করে, `notificationId` থেকে বা টাইমস্ট্যাম্প থেকে একটি আইডি বিরে করে, এবং `plugin.show()` কল করে Android নোটিফিকেশন ডাটাইলসসহ নোটিফিকেশন দেখায়, যখন `payload`-এ পুরো ডটো ম্যাপ JSON-এনকোড করে সংযুক্ত থাকে যাতে ট্যাপ করলে তা পড়া যায়।

ক্লাস/উইজটে: `PushService`

পুশ নোটিফিকেশনের প্রান্ত-থেকে-প্রান্ত (end to end) যুক্তিধারণকারী প্রধান ক্লাস—অনুমতি চাওয়া, টোকেনের জীবনচক্র এবং ট্যাপ করা নোটিফিকেশনকে স্ক্রিনি রূপান্তর করা। টোকেনে রজিস্ট্রেশন ইচ্ছাকৃতভাবে অ্যাপ লঞ্চারে বদলে সাইন-ইন/সাইন-আউটের সাথে যুক্ত করা হয়েছে, কারণ `RegisterDeviceTokenRequest`-এ কোনো "কার জন্য এই টোকেন" ফিল্ড নই—রজিস্ট্রেশনের সময়কার JWT থেকেই মালিকি নির্ধারণিত হয়—তাই অ্যাকাউন্ট পরিবর্তনের পরও ডিভাইস সাইন-ইন করা থাকলে আগের ব্যবহারকারীর নোটিফিকেশন পতে থাকত।

`PushService(this._ref)`

কনস্ট্রাক্টর, যা একটি Riverpod `Ref` গ্রহণ করে `_ref` ফিল্ডে সংরক্ষণ করে, যা পরবর্তীতে অন্যান্য প্রোভাইডার অ্যাক্সেস করতে ব্যবহৃত হয়।

`ApiClient get _api`

``dart ApiClient get _api => _ref.read(apiClientProvider);`` `apiClientProvider` থেকে বর্তমান `ApiClient` ইনস্ট্যান্স পড়ে আনে (প্রতাবির কল হলে সাম্প্রতিকতম মান পড়ে)।

`bool get _supported`

``dart bool get _supported => !kIsWeb && Platform.isAndroid;`` বর্তমান প্ল্যাটফর্মে পুশ নোটিফিকেশন সমর্থিত কিনা যাচাই করে। শুধুমাত্র Android-এ `google-services.json` রজিস্টার করা আছে, তাই অন্য যেকোনো প্ল্যাটফর্মে Firebase কল করলে সরাসরি throw করত—এই গটোর সেই সমস্যা এড়িয়ে নিঃশব্দে ফিচার বন্ধ রাখা।

Future<void> init()

পুশ সার্ভিস প্রাথমিকীকরণের মূল এন্ট্রি পয়েন্ট। প্রথমে `_supported` না হলে সরাসরি রিটার্ন করে। এরপর ধারাবাহিকভাবে: Android নোটিফিকেশন চ্যানেলে তৈরি করে, `_local.initialize()` কল করে `onDidReceiveNotificationResponse` হ্যান্ডলার সেট করে, FCM পারমিশন চায়, ফোরগ্রাউন্ড মসেজের জন্য `FirebaseMessaging.onMessage` লিসেন করে, টোকেনে রিফ্রেশ ইভেন্টে (ব্যবহারকারী লগইন থাকলে) নতুন টোকেনে রজিস্টার করে, এবং সবশেষে `getNotificationAppLaunchDetails()` চেকে করে দেখে অ্যাপটিনোটিফিকেশন ট্যাপ করে সম্পূর্ণ বন্ধ অবস্থা থেকে লঞ্চে হয়েছে কিনা, হলে সংশ্লিষ্ট পলোড হ্যান্ডলে করে।

void _onNotificationTapped(NotificationResponse response)

`FlutterLocalNotificationsPlugin.initialize()`-এর `onDidReceiveNotificationResponse` কলব্যাক হিসেবে ব্যবহৃত হয়; সরাসরি `response.payload` কে `_handlePayload`-এ পাঠিয়ে দেয়।

void _handlePayload(String? payload)

পলোড স্ট্রিংকে JSON ডিকোড করে `_navigateFor`-এ পাঠায়। পলোড `null` বা খালি হলে কিছু করে না, এবং ডিকোডিং ব্যর্থ হলে (ম্যালফর্মড পলোড) নীরবে ব্যর্থতা গ্রহণ করে যাতে ট্যাপ হ্যান্ডলার ক্র্যাশ না করে।

Future<void> registerToken()

সাইন-ইনের পর বা সেশন পুনরুদ্ধারের পর একবার কল করার উদ্দেশ্যে তৈরি একটি পাবলিক মেথড, যাতে সাইন-আউট অবস্থায় নতুন তৈরি বা রোটটে হওয়া টোকেনও রজিস্টার হয়ে যায়। `_supported` না হলে কিছু করে না। `FirebaseMessaging.instance.getToken()` থেকে টোকেন এনে `_register` কল করে; ব্যর্থ হলে (Google Play services নেই, পারমিশন প্রত্যাখ্যাত, নেটওয়ার্ক নেই) ত্রুটি গ্রাস করে, কারণ এসব কারণে সাইন-ইন প্রক্রিয়া ব্লক হওয়া উচিত নয়।

Future<void> _register(String token)

``dart Future<void> _register(String token) => _api.postText('/notifications/device-tokens', {'token': token, 'platform': 'ANDROID'})``; `_api.postText` কল করে টোকেন ও প্ল্যাটফর্ম ব্যাকএন্ডে পাঠায়, যাতে ডভাইস টোকেনে সার্বভারে রজিস্টার হয়।

Future<void> unregisterToken()

সেশন ক্লয়ার করার আগে কল করার উদ্দেশ্যে তৈরি, কারণ এন্ডপয়েন্টের জন্য বর্তমান `Authorization` হডোর এখনও দরকার। `_supported` না হলে কিছু করে না। বর্তমান টোকেনে থাকলে সার্বভার থেকে স্টেটি ডিলিট করে (`DELETE /notifications/device-tokens/{token}`), তারপর `FirebaseMessaging.instance.deleteToken()` কল করে যাতে পরের সাইন-ইনে নতুন টোকেনে তৈরি হয় (শেয়ার করা ডভাইসে পুরনো ব্যবহারকারীর সাথে টোকেনে যুক্ত থাকা এড়াতে)। পুরো প্রক্রিয়া try-catch-এ মোড়ানো, কারণ এটি best-effort—স্থানীয় সেশন যাই হোক মুছে যাচ্ছে।

Future<void> _navigateFor(Map<String, dynamic> data)

নোটিফিকেশন ট্যাপ থেকে সঠিক মডুলে (নর্দিষ্ট রেকর্ডে নয়, কারণ কোনো স্ক্রিনিই রুট থেকে রেকর্ড আইডি নিয়ে না) নভেগিটে করে। প্রথমে `authControllerProvider.future` অপেক্ষা করে প্রাথমিক সেশন-রিসিটোর সম্পূর্ণ হওয়া নিশ্চিতি করে (নাহলে GoRouter স্প্ল্যাশ স্ক্রিনি রিডাইরেক্ট করে দিতে পারে)। ব্যবহারকারী `null` হলে রিটার্ন করে। এরপর `user.isClient` অনুযায়ী `_clientRouteFor` বা `_staffRouteFor` থেকে পথ নির্ণয় করে, এবং `WidgetsBinding.instance.addPostFrameCallback` ব্যবহার করে বর্তমান ফ্রেমের পরে `routerProvider`-এর মাধ্যমে পুশ করে—কারণ কোল্ড-স্টার্ট লঞ্চে ক্রমশে GoRouter-এর নভেগিটের তখনো অ্যাটাচড নাও থাকতে পারে।

String _staffRouteFor(String? type)

নোটিফিকেশনের type স্ট্রিং-এর উপর ভিত্তি করে স্টাফ সাইডের রুট নির্ধারণ করে একটি switch এক্সপ্রেশন। রকিওয়েস্ট-সম্পর্কিত টাইপ Routes.requests, পমেন্ট/ইনভয়েন্স/রফান্ড টাইপ Routes.finance, লাইসেন্স/ওয়ারেন্ট/অ্যাসটে টাইপ Routes.itam, ছুটি টাইপ Routes.leave, PAYSLIP_READY Routes.payslips, পারফরম্যান্স রিভিউ টাইপ Routes.performance, CRM টাইপ Routes.crm, অন্যথায় ডফিল্ট Routes.alerts।

String _clientRouteFor(String? type)

ক্লায়েন্ট পোর্টালের জন্য রুট নির্ধারণ করে, যখন কোন alerts ট্যাব বা স্টাফ-অনলিমিডউল নাই। রকিওয়েস্ট-সম্পর্কিত টাইপ PortalRoutes.requests, পমেন্ট/ইনভয়েন্স/রফান্ড টাইপ PortalRoutes.billing, অন্যথায় PortalRoutes.home—কারণ অমলিতি টাইপ পোর্টালের কোনো নষিদ্ধ রুটে না গিয়ে হোমে ফিরে যাওয়া উচিত (router.dart-এর রডাইরেক্ট যা তাৎক্ষণিকভাবে সরিয়ে দি়ে)।

void dispose()

``dart void dispose() => \_refreshSub?.cancel();`` টোকেন-রিফ্রেশ স্ট্রিম সাবস্ক্রিপশন বাতলি করে, প্রোভাইডার নষিপ্ততার (dispose) সময় রসিওরস লকি এডাত।

টপ-লেভেলে প্রোভাইডার: pushServiceProvider

``dart final pushServiceProvider = Provider<PushService>((ref) { final service = PushService(ref); ref.onDispose(service.dispose); return service; });`` PushService-এর একক ইনস্ট্যান্স তরৈকিরে এবং প্রোভাইডার নষিপ্ততার সময় dispose() কল হওয়া নষিচতি করে।

টপ-লেভেলে প্রোভাইডার: pushInitProvider

``dart final pushInitProvider = FutureProvider<void>((ref) => ref.read(pushServiceProvider).init());`` অ্যাপের জীবনকালে ঠকি একবার PushService.init() চালানোর জন্য একটি FutureProvider। কোনো widget-এর initState-এর বদলে প্রোভাইডার হিসিবে রাখা হযছে, যাতে এটি প্রথমযে য়ে স্ক্রনি দখোনো হোক না কনে তা থকে বঁচে যায় এবং অসম্পর্কিত রবিলিডে পুনরায় না চলে—sharedPreferencesProvider-এর মতোই একই কারণে।

ফাইল: lib/core/storage/secure_store.dart

সংক্ষিপ্ত ববিরণ: সশেন সংক্রান্ত তথ্য (টোকেন, ব্যবহারকারী, পারমশিন) নরিপদে সংরক্ষণকারী ক্লাস, যা Angular অ্যাপের AuthService/PermissionService-এর localStorage কী-গুলোর সমতুল্য কন্টি ডটো রাখে প্ল্যাটফর্মের কীস্টোর। এছাড়া এটি টিন্যেন্ট ইমপার্সোনেশন (impersonation) সশেন ও ডমো-সশেন ফ্ল্যাগও পরচালনা করে।

ক্লাস/উইজটে: SecureStore

সমস্ত সশেন-সম্পর্কিত স্টোরেজে অপারেশনের জন্য একটি একক গটেওয়ে ক্লাস, যা FlutterSecureStorage-কে মোড়ায়। এটি সাধারণ টোকেন/ইউজার/পারমশিন CRUD, ডমো-সশেন ফ্ল্যাগ, এবং একটি দুই-নমেস্পেস (admin_* বনাম লাইভ কী) ভিত্তিক ইমপার্সোনেশন প্রক্রিয়া সরবরাহ করে যাতে প্ল্যাটফর্ম অ্যাডমনি এক টিন্যেন্টের মধ্যে ঢুকে আবার নজি পরচিয়ে নরিপদে ফিরে আসতে পারনে।

SecureStore(this._storage)

কনস্ট্রাক্টর, একটি FlutterSecureStorage ইনস্ট্যান্স গ্রহণ করে _storage ফিল্ডে সংরক্ষণ করে।

Future<String?> readAccessToken()

``dart Future<String?> readAccessToken() => \_storage.read(key: \_accessToken); `` সংরক্ষিত অ্যাক্সেস টোকেনে পড়ে আনবে, না থাকলে null রিটার্ন করে।

Future<String?> readRefreshToken()

``dart Future<String?> readRefreshToken() => \_storage.read(key: \_refreshToken); `` সংরক্ষিত রিফ্রেশ টোকেনে পড়ে আনবে।

Future<void> writeTokens(String accessToken, String? refreshToken)

অ্যাক্সেস টোকেন লেখে, এবং refreshToken non-null হলে সেটিও লেখে। /auth/refresh সবসময় দুটোই রিটার্ন করে, তবে ডেমো সেশন ইচ্ছাকৃতভাবে কোনো রিফ্রেশ টোকেন ইস্যু করে না—অ্যাক্সেস টোকেনে ময়োদোত্তীর্ণ হলে ডেমো শেষ হয়ে যায়, তাই null হলে পুরনো মান স্পর্শ করা হয় না।

Future<bool> readIsDemo()

``dart Future<bool> readIsDemo() async => await \_storage.read(key: \_demo) == '1'; `` ডেমো ফ্ল্যাগ '1' কনি যাচাই করে বর্তমান সেশন ডেমো সেশন কনি তা রিটার্ন করে।

Future<void> writeIsDemo()

``dart Future<void> writeIsDemo() => \_storage.write(key: \_demo, value: '1'); `` বর্তমান সেশনকে ডেমো হিসেবে চিহ্নিত করে।

Future<Map<String, dynamic>?> readUser()

``dart Future<Map<String, dynamic>?> readUser() => \_readJson(\_user); `` সংরক্ষিত ব্যবহারকারী JSON রেকর্ড _readJson হলেপার দিয়ে পড়ে আনবে।

Future<void> writeUser(Map<String, dynamic> user)

ব্যবহারকারী ম্যাপকে JSON-এনকোড করে _user কী-তে লেখে।

Future<List<String>> readPermissions()

``dart Future<List<String>> readPermissions() => \_readStringList(\_permissions); `` সংরক্ষিত পারমিশন কোডের তালিকা _readStringList হলেপার দিয়ে পড়ে আনবে।

Future<List<String>> readPermissionCatalog()

``dart Future<List<String>> readPermissionCatalog() => \_readStringList(\_permissionCatalog); `` সম্পূর্ণ পারমিশন ক্যাটালগ (সিস্টেমে বর্তমান সব পারমিশন কোড) পড়ে আনবে।

Future<void> writePermissions(List<String> codes)

ব্যবহারকারীর নিজস্ব পারমিশন কোডের তালিকা JSON-এনকোড করে সংরক্ষণ করে।

Future<void> writePermissionCatalog(List<String> codes)

সম্পূর্ণ পারমিশন ক্যাটালগ JSON-এনকোড করে সংরক্ষণ করে।

Future<Map<String, dynamic>?> readImpersonation()

``dart Future<Map<String, dynamic>?> readImpersonation() => \_readJson(\_impersonation); `` বর্তমান ইমপার্সোনেশন সেশন তথ্য (থাকলে) পড়ে আনবে।

`Future<void> beginImpersonation({required String accessToken, required Map<String, dynamic> session, required Map<String, dynamic> impersonatedUser})`

প্ল্যাটফর্ম অ্যাডমিনের নজিরে সেশন `admin_*` কী-গুলোর নচি "পারক" করে এবং টেনেয়ান্টের ইমপার্সোনটেডে সেশন সক্রিয় করে। প্রথমে বর্তমান অ্যাক্সেস/রফ্রেশ টোকেনে, ইউজার, পারমিশন ও ক্যাটালগ পড়ে `admin_*` কী-তে কপি করে। এরপর লাইভ কী-গুলোতে নতুন অ্যাক্সেস টোকেনে, ইমপার্সোনটেডে ইউজার ও সেশন লেখে, এবং ইচ্ছাকৃতভাবে রফ্রেশ টোকেনে, পারমিশন ও ক্যাটালগ ডলিটি করে—কারণ ইমপার্সোনেশন এন্ডপয়েন্ট শুধু একটি অ্যাক্সেস টোকেনে দেয় যা রফ্রেশযোগ্য নয়; অ্যাডমিনের রফ্রেশ টোকেনে রয়েছে দলি মডি-সেশন ৪০১ এরর হলে ইন্টারসপেটর নীরবে নতুন *অ্যাডমিন* টোকেনে মনিট করে ফলেতে পারত, যা UI-তে কিছু না দেখিয়ে ব্যবহারকারীকে টেনেয়ান্ট থেকে সরিয়ে দিত।

`Future<Map<String, dynamic>?> restoreAdmin()`

পারক করা অ্যাডমিন সেশন আবার লাইভ কী-তে ফরিয়ে আনে এবং অ্যাডমিনের ইউজার রকের্ড রটার্ন করে। প্রথমে সব `admin_*` মান পড়ে, তারপর `clearImpersonation()` কল করে ইমপার্সোনেশন কী-গুলো মুছে দেয় (সাফল্য/ব্যর্থতা নরিবশিষে)। `adminAccess` বা `adminUserRaw` না থাকলে (কিছুই পারক করা নহে) `clear()` কল করে পুরো সেশন মুছে `null` রটার্ন করে—যার অর্থ অ্যাপ কোনো প্রকৃত পরচিয়ে ফরিতে পারবে না, একমাত্র নরিপদ গন্তব্য লগইন সক্রনি। অন্যথায় অ্যাডমিনের অ্যাক্সেস টোকেনে, রফ্রেশ টোকেনে (থাকলে, নাহলে ডলিটি), ইউজার, পারমিশন ও ক্যাটালগ লাইভ কী-তে ফরিয়ে লেখে।

`Future<void> clearImpersonation()`

ইমপার্সোনেশন সেশন এবং সব `admin_*` কী মুছে ফলে, `Future.wait` ব্যবহার করে সমান্তরালে।

`Future<void> clear()`

সম্পূর্ণ সাইন-আউটরে জন্ম সব কী মুছে ফলে—অ্যাক্সেস/রফ্রেশ টোকেনে, ইউজার, পারমিশন, ক্যাটালগ, পারক করা অ্যাডমিন সেশনের সব কী, ইমপার্সোনেশন কী এবং ডমে ফল্যাগ—যাতে প্রকৃত সাইন-আউটরে পর অন্য কারও পরচিয়ে "রজিডিম" করা যায় এমন কিছু অবশিষ্ট না থাকে।

`Future<Map<String, dynamic>?> _readJson(String key)`

নরিদর্শি কী থেকে স্ট্রিং পড়ে JSON ডকিড করে `Map<String, dynamic>` রটার্ন করার প্রাইভটে হলেপার। মান `null`, খালি বা `'undefined'` হলে `null` রটার্ন করে; ডকিডিং ব্যর্থ হলে (পুরনো ব্লিডরে লখো ডটো যা আর পার্স হয় না) ক্র্যাশ না করে `null` রটার্ন করে এবং পরে পুনরায় লখো হওয়ার সুযোগ দেয়।

`Future<List<String>> _readStringList(String key)`

নরিদর্শি কী থেকে স্ট্রিং পড়ে JSON ডকিড করে `List<String>` রটার্ন করার প্রাইভটে হলেপার। মান খালি বা ডকিডিং ব্যর্থ হলে খালি `const` লিস্ট (`const []`) রটার্ন করে।

ফাইল: `lib/core/theme/accents.dart`

সংক্ষিপ্ত ববিরণ: নরিবাচনযোগ্য ব্র্যান্ড অ্যাকসেন্ট রঙের সংজ্ঞা। শুধুমাত্র ব্র্যান্ড রঙ পরবির্ততি হয়—পৃষ্ঠতল (`surface`), লখো ও সমিান্টিক রঙগুলো `BosPalette` দ্বারা নরিদর্শি থাকে, যাত থমি পকির সম্পূর্ণ `chrome`-কে বদলে না ফলে বরং একটি ডিজাইন করা নরিপক্ষে ভিত্তির উপর শুধু অ্যাকসেন্ট বদলায়।

টপ-লেভেলে enum: AppAccent

সাতটি নির্বাচনযোগ্য ব্র্যান্ড অ্যাকসেন্ট (emerald, teal, blue, indigo, violet, rose, amber) ধারণকারী enum, যখন প্রতীতি ভিয়ারিয়েন্ট চারটি রঙের মান বহন করে: হালকা মোডের fill (বোতামের মতো ভরাট ব্যাকগ্রাউন্ড, যার উপর সাদা লেখা বসে বলে গাঢ় থাকতে হয়) ও ink (পেজে ব্যাকগ্রাউন্ডের উপর টেক্সট/আইকন রঙ), এবং তাদের ডার্ক-মোড প্রতীক fillDark/inkDark। হালকা মোডে fill ও ink একই হতে পারে, কিন্তু ডার্ক সারফেসে fillDark এখনও সাদা লেখা বহন করতে হয় বলে গাঢ় থাকে, অথচ inkDark-কে প্রায়-কালোর বর্ণিত পড়ার জন্য যথেষ্ট হালকা হতে হয়—যা _dark.scss-এর --bos-brand (#047857, বোতামের জন্য) বনাম --bos-link (#6EE7B7, লেখার জন্য) বিভাজনের সমতুল্য।

```
const AppAccent({required this.id, required this.label, required this.fill, required this.ink, required this.fillDark, required this.inkDark})
```

enum-এর প্রাইভেট কনস্ট্রাক্টর, প্রতীতি ভিয়ারিয়েন্টের ছয়টি ফিল্ড (id, label, fill, ink, fillDark, inkDark) সেট করে।

final String id

প্রফারেন্সে লেখা স্থায়ী কী। এনামের name ব্যবহার করা যতে, কিন্তু কডে পরে ভিয়ারিয়েন্ট রিউন্ডার বা রনিমে করলে প্রতীতি ব্যবহারকারীর সংরক্ষিত পছন্দ নীরবে অন্য কিছুতে পরিণত হয়ে যতে—তাই আলাদা স্থিতিশীল id রাখা হয়েছে।

final String label

UI-তে দেখানোর জন্য অ্যাকসেন্টের মানুষ-পাঠযোগ্য নাম (যেমন "Emerald")।

final Color fill, final Color ink, final Color fillDark, final Color inkDark

যথাক্রমে হালকা-মোড ভরাট রঙ, হালকা-মোড টেক্সট/আইকন রঙ, ডার্ক-মোড ভরাট রঙ ও ডার্ক-মোড টেক্সট/আইকন রঙ ধারণকারী স্থিতি ফিল্ড।

Color fillFor(Brightness brightness)

dart Color fillFor(Brightness brightness) => brightness == Brightness.dark ? fillDark : fill; বর্তমান ব্রাইটনেস অনুযায়ী সঠিক fill রঙ (ডার্ক হলে fillDark, নাহলে fill) রিটার্ন করে।

Color inkFor(Brightness brightness)

dart Color inkFor(Brightness brightness) => brightness == Brightness.dark ? inkDark : ink; বর্তমান ব্রাইটনেস অনুযায়ী সঠিক ink রঙ রিটার্ন করে।

Color softFor(Brightness brightness)

নির্বাচতি সারি ও আইকন টাইলের পছন্দের হালকা টিন্ট রঙ রিটার্ন করে। ফ্ল্যাট হেক্স-এর বদলে আলফা-ভিত্তিক রঙ ব্যবহার করা হয় যতে একই টোকেনে কার্ড ও ইতিমধ্যে টিন্টে সারি উভয় ক্ষেত্রে কাজ করে। ডার্ক মোডে inkDark-এর উপর ০.১৮ আলফা ব্যবহার করা হয় (কারণ প্রায়-কালোর উপর ০.১০ আলফা অদৃশ্য থাকত), হালকা মোডে fill-এর উপর ০.১০ আলফা ব্যবহৃত হয়।

static const defaultAccent

dart static const defaultAccent = AppAccent.emerald; ডিফল্ট অ্যাকসেন্ট হিসেবে emerald নির্ধারণ করে।

static AppAccent fromId(String? id)

সংরক্ষিত id স্ট্রিং থেকে সংশ্লিষ্ট AppAccent ভিয়ারিয়েন্ট খুঁজে বের করে। id null হলে সরাসরি defaultAccent রিটার্ন করে; অন্যথায় সব ভিয়ারিয়েন্টের মধ্যে মিলিত id খুঁজে সেটি রিটার্ন করে, না পলে (অজানা/পুরনো মান) defaultAccent-এ ফলব্যাক করে।

ফাইল: lib/core/theme/app_theme.dart

সংক্ষিপ্ত ববিরণ: **BosPalette** টোকেনে থেকে সম্পূর্ণ **Material ThemeData** তৈরিকরে। এই একটি জায়গায়ই টোকেনকে **Material**-এর নরিদর্ষিট কম্পোনেন্ট স্লটে (**AppBar**, **Card**, **Button**, **Chip** ইত্যাদি) রূপান্তর করা হয়, যাতো কোনো ফচার উইজটে সরাসরি **Colors.something** ব্যবহার না করে সবসময় প্যালটে থেকেই রঙ নিয়ে।

ক্লাস: **AppTheme**

একটি প্রাইভেট কনস্ট্রাক্টরসহ ফাইনাল ক্লাস, যা শুধু স্ট্যাটিক ফ্যাক্টর/হেল্পার মেথড ধারণ করে। এর দায়িত্ব হলো **AppAccent** থেকে হালকা/গাঢ় **ThemeData** তৈরিকরা এবং **TextTheme** ও প্রতটি কম্পোনেন্ট থিম কনফিগার করা।

AppTheme._()

প্রাইভেট, বডাইন কনস্ট্রাক্টর — ক্লাসটিকে ইনস্ট্যান্স করা থেকে আটকাই, কারণ এর সব মেথবারই স্ট্যাটিক।

static ThemeData light(AppAccent accent)

BosPalette.light(accent) দিয়ে **_build** কল করে হালকা থিম রিটার্ন করে।

static ThemeData dark(AppAccent accent)

BosPalette.dark(accent) দিয়ে **_build** কল করে গাঢ় থিম রিটার্ন করে।

static ThemeData _build(BosPalette bos)

মূল থিম-বল্ডিং লজিক। প্রথম **bos**-এর মান থেকে একটি সম্পূর্ণ **ColorScheme** তৈরিকরে (**primary/secondary/error/surface** ইত্যাদি প্রতটি স্লট একটি **BosPalette** ফল্ডারে সাথে ম্যাপ করা), তারপর **useMaterial3: true** সহ বসে **ThemeData** তৈরিকরে। এরপর **bos**-কে **extensions** হিসেবে যুক্ত করে এবং **appBarTheme**, **cardTheme**, **dividerTheme**, বাটন থিমগুলো (**elevatedButtonTheme**, **filledButtonTheme**, **outlinedButtonTheme**, **textButtonTheme**), **inputDecorationTheme**, **chipTheme**, **navigationBarTheme**, **bottomSheetTheme**, **dialogTheme**, **snackBarTheme**, **listTileTheme**, **switchTheme**, **progressIndicatorTheme**, **iconTheme**, ও **tabBarTheme** স্টে করে **base.copyWith(...)** রিটার্ন করে। **AppBar**-এ শ্যাডোর বদলে একটি হিয়ারলাইন বর্ডার ব্যবহার করা হয়েছে যেহেতু পজে টনিটেডে আর কার্ড সাদা — শ্যাডো তৃতীয় একটি সিরফসেরে মতো দেখাত। **chipTheme**-এ **selectedColor/secondarySelectedColor/secondaryLabelStyle** স্পষ্টভাবে ব্র্যান্ড রঙে স্টে করা হয়েছে, কারণ **Material** ডিফল্টে সলিক্টেডে চপি **secondary** (info নীল) রঙ ব্যবহার করত, যা অ্যাপরে অন্য কাজে ব্যবহৃত রঙের সাথে সাংঘর্ষিক ছিল।

static OutlineInputBorder _inputBorder(Color color, {double width = 1})

একটি নিরিদর্ষিট রঙ ও প্রস্থসহ **_radiusSm** (8.0) কোণ-ব্যাসার্ধবিশিষ্ট **OutlineInputBorder** তৈরিকরে। **inputDecorationTheme**-এর সব বর্ডার ভ্যারিয়েন্ট (**normal/enabled/focused/error/focusedError**) এই একই হেল্পার দিয়ে তৈরি হয়, শুধু রঙ ও প্রস্থ পাল্টে।

static TextTheme _textTheme(TextTheme base, BosPalette bos)

Material-এর ডিফল্ট **TextTheme**-এর প্রতটি স্টাইল (**displaySmall** থেকে **labelSmall** পর্যন্ত) **copyWith** করে **bos**-এর টেক্সট রঙ (**text**, **textSecondary**, **muted**) ও নিরিদর্ষিট **fontWeight/letterSpacing** বসিয়ে দিয়ে। হডেলাইন ও টাইটলে স্টাইলগুলোতে বাড়তি **fontWeight: FontWeight.w700** ও নগেটেভি **letterSpacing** যোগ করা হয়েছে টাইট, ভারী শরিরোনামের জন্য।

ফাইল: lib/core/theme/bos_tokens.dart

সংক্ষিপ্ত ববিরণ: ওয়বে অ্যাপরে স্টাইলশাটি থেকে পোর্ট করা --bos-* ডিজাইন টোকেনগুলো ধারণকারী ThemeExtension। অ্যাপ যত রঙ আঁকে সবই এখান থেকে আসে — একটি দ্বিতীয় প্যালেটে শুধু এই ফাইলের মানগুলো পাল্টালেই ডার্ক মোড কাজ করে।

ক্লাস/উইজটে: BosPalette

ThemeExtension<BosPalette>-এর ইমিউটেবল সাবক্লাস, যা ব্র্যান্ড/সমিন্টিকি (success/danger/warning/info)/নডিট্রাল/টেক্সট/ব্রডার/ব্যাকগ্রাউন্ড রঙের সম্পূর্ণ সেট ধারণ করে। Theme.of(context).bos দিয়ে অ্যাক্সেস করা হয়। প্রতটি সমিন্টিকি জোড়া (যেমন success/successSoft) একটি ইউনিটি: -Soft হলো ফলি রঙ, সলডি রঙটিটির উপর আঁকা টেক্সট/আইকনের জন্য।

bool get isDark

brightness == Brightness.dark কনি তা রটার্ন করে — গাঢ় মোডের শাখা-নরিভর লজকিরে জন্য সংক্ষিপ্ত গটোর।

factory BosPalette.light(AppAccent accent)

_variables.scss/global.scss থেকে নেওয়া হালকা থমিরে প্রতটি রঙ হার্ডকোড করে একটি BosPalette তরৈকিরে। ব্র্যান্ড রঙ তনিটি (brand, brandInk, brandSoft) দেওয়া accent-এর Brightness.light ভ্যারিয়েন্ট থেকে আসে, বাকি সব রঙ ধুবক (const Color)।

factory BosPalette.dark(AppAccent accent)

_dark.scss থেকে নেওয়া গাঢ় থমিরে রঙ সেট করে একইভাবে BosPalette তরৈকিরে। টেক্সট র্যাম্প ইচ্ছাকৃতভাবে সর্বোচ্চ কন্ট্রাস্টেরে নচি রাখা হয়েছে — প্রায়-কালো ব্যাকগ্রাউন্ডে ১৫:১ কন্ট্রাস্টেরে হালকা টেক্সট "halate" করে (অক্ষরেরে প্রান্ত ঝলমল করে), তাই বডি টেক্সট প্রায় ১১:১-এ রাখা হয়েছে, ৪.৫:১ ন্যূনতম মানেরে অনেক উপরে কনিতু ঝলকরে বাইরে; হায়ারার্কি বিজায় রাখা হয়েছে পেজে ও কার্ডেরে লুমিনিেন্স আলাদা রেখে।

((Color fg, Color bg)) statusColors(String status)

ব্যাকগ্রাউন্ডেরে একটি স্ট্যাটাস স্ট্রিং (যেমন 'APPROVED', 'REJECTED', 'PENDING') নিয়ে তার জন্য উপযুক্ত foreground/background রঙেরে জোড়া রটার্ন করে একটি switch স্টেটমেন্টেরে মাধ্যমে। স্ট্যাটাসগুলো চারটি সমিন্টিকি দলে ভাগ করা — সফল/সবুজ (PRESENT, APPROVED, PAID, COMPLETED ইত্যাদি), ব্যর্থ/লাল (ABSENT, REJECTED, CANCELLED, FAILED ইত্যাদি), অপেক্ষমাণ/হলুদ (LATE, PENDING, WAITING ইত্যাদি), এবং চলমান/তথ্যমূলক/নীল (ON_LEAVE, IN_PROGRESS, SENT ইত্যাদি) — এবং কিছু বিশেষ ক্ষতের (HOLIDAY, NEGOTIATION) ব্র্যান্ড রঙে। অচনো যকোনো মান ডিফল্টভাবে নডিট্রাল (muted/neutralSoft) রঙে পড়ে, যাতে একটি অশ্রণীবদ্ধ মান কখনো ব্র্যান্ড রঙ হিসেবে ভুলভাবে না দেখায়।

@override BosPalette copyWith({...})

ThemeExtension চুক্তরি অংশ; দেওয়া প্যারামটারগুলো নতুন মান দিয়ে প্রতস্থাপন করে বাকি সব ফলিড অপরিবর্তিত রেখে একটি নতুন BosPalette তরৈকিরে (প্রতটি প্যারামটার nullable, ?? দিয়ে বদিয়মান মান ফলব্যাক করে)।

@override BosPalette lerp(ThemeExtension<BosPalette>? other, double t)

ThemeExtension চুক্তরি অংশ; থমি পরিবর্তনেরে সময় অ্যানিমেশনেরে জন্য দুটি BosPalette-এর মধ্যে t অনুপাতে ইন্টারপোলেটে করে। other BosPalette না হলে this রটার্ন করে। প্রতটি Color ফলিড একটি স্থানীয় mix হলেপার দিয়ে Color.lerp করা হয়; brightness t < 0.5 হলে বর্তমান, নাহলে other-এর মান নেয়।

এক্সটেনশন: BosThemeAccess (on ThemeData)

BosPalette get bos

ThemeData-তে `extension<BosPalette>()` খুঁজে রিটার্ন করে; কোনো `BosPalette` সংযুক্ত না থাকলে ডিফল্ট অ্যাকসেন্টসহ `BosPalette.light(AppAccent.defaultAccent)`-এ ফলব্যাক করে। এটিই প্যালটে পৌঁছানোর একমাত্র প্রবশেপথ — `Theme.of(context).bos.brand`।

টপ-লভেলে ধ্রুবক/enum

এই ফাইলে কোনো টপ-লভেলে ফাংশন বা enum নেই।

ফাইল: lib/core/theme/theme_controller.dart

সংক্ষিপ্ত বিবরণ: ব্যবহারকারীর থিম মোড (system/light/dark) ও অ্যাকসেন্ট রঙের পছন্দ পরিচালনা করে, `SharedPreferences`-এ স্থায়ীভাবে সংরক্ষণ করে। Riverpod `Notifier` হিসেবে বাস্তবায়িত।

ক্লাস/উইজটে: ThemeSettings

একটি ইমিউটেবল ডটো ক্লাস যা বর্তমান `ThemeMode` (ডিফল্ট `system`) ও `AppAccent` (ডিফল্ট `AppAccent.defaultAccent`) ধারণ করে। `system` মানে "OS অনুসরণ করো" — এবং ব্যবহারকারী স্পষ্টভাবে বেছে না নেওয়া পর্যন্ত এটিই ডিফল্ট থাকে; একবার বেছে নলি তা লগে থাকে, যাত সূর্যাস্তরে সময় ফোনের শডিউল ট্রগার হওয়ায় অ্যাপ হ্যাং পাল্টে না যায়।

`ThemeSettings copyWith({ThemeMode? mode, AppAccent? accent})`

দেওয়া প্যারামিটার নতুন মান দিয়ে প্রতস্থাপন করে (?? দিয়ে) একটি নতুন `ThemeSettings` তৈরি করে রিটার্ন করে।

ক্লাস/উইজটে: ThemeController (extends Notifier<ThemeSettings>)

থিম মোড ও অ্যাকসেন্ট স্টেটে এবং তাদের `SharedPreferences`-এ পড়া/লিখো পরিচালনা করে।

`@override ThemeSettings build()`

`sharedPreferencesProvider` থেকে `prefs` পড়ে, সংরক্ষিত `_modeKey` ও `_accentKey` স্ট্রিং থেকে যথাক্রমে `_modeFromId` ও `AppAccent.fromId` দিয়ে প্রাথমিক `ThemeSettings` তৈরি করে রিটার্ন করে।

`Future<void> setMode(ThemeMode mode)`

মোড অপরিবর্তিত থাকলে সরাসরি রিটার্ন করে। নাহলে স্টেটে আপডেট করে এবং যদি নতুন মোড `ThemeMode.system` হয় তবে `_modeKey`-টি `prefs` থেকে সরিয়ে ফেলে (সেভে করার বদলে) — যাত "system অনুসরণ করা" একটি তৃতীয় সঙ্ক-করা মান না হয়ে পছন্দে অনুপস্থিতি হিসেবেই থাকে; নাহলে `_idFromMode(mode)` স্ট্রিং হিসেবে সংরক্ষণ করে।

`Future<void> setAccent(AppAccent accent)`

অ্যাকসেন্ট অপরিবর্তিত থাকলে সরাসরি রিটার্ন করে। নাহলে স্টেটে আপডেট করে এবং `accent.id` `_accentKey`-তে `prefs`-এ সংরক্ষণ করে।

Future<void> toggle(Brightness current)

বর্তমান **Brightness** (যা "system অনুসরণ করলে সিস্টেমে আপাতত কী দেখাচ্ছে" তা প্রতিনিধিত্ব করে) নিয়ে হালকা ও গাঢ় মध्ये সুইচ করে — **current dark** হলে **setMode(ThemeMode.light)**, নাহলে **setMode(ThemeMode.dark)** কল করে।

static ThemeMode _modeFromId(String? id)

একটি সংরক্ষিত স্ট্রিং আইডি ('light'/'dark'/অন্য কিছু বা null) কে ThemeMode-এ রূপান্তর করে, না মিললে ThemeMode.system-এ ফলব্যাক করে।

static String _idFromMode(ThemeMode mode)

ThemeMode-কে সংরক্ষণযোগ্য স্ট্রিং আইডিতে ('light'/'dark'/'system') রূপান্তর করে।

টপ-লেভেলে প্রোভাইডার

final themeControllerProvider = NotifierProvider<ThemeController, ThemeSettings>(ThemeController.new); — অ্যাপজুড়ে ThemeController অ্যাক্সেস করার Riverpod প্রোভাইডার।

ফাইল: lib/features/accounting/accounting_models.dart

সংক্ষিপ্ত ববিরণ: ডাবল-এন্ট্রি বুককপিংয়ের ডটো মডেলে — অ্যাকাউন্ট, জার্নাল এন্ট্রি (ও তার লাইন), এবং জনোরলে লজোর লাইন — সব JSON (de)সিরিয়লাইজেশনসহ। এই মডউলে একটি পোস্ট করা এন্ট্রি কিখনো এডটি করে "আনডু" করা যায় না, শুধু একটি বিপরীত এন্ট্রি দিয়ে "রভার্স" করা যায়, তাই ক্লাসগুলো প্রতটি প্রিয়ায় কোন অ্যাকশন এখনো সম্ভব তা স্পষ্টভাবে প্রকাশ করে।

ক্লাস/উইজটে: AccountingPermissions

abstract final class — শুধু স্ট্যাটিক String ধ্রুবক ধারণকারী পারমিশন-কোড ক্যাটালগ (accountView, accountCreate, accountUpdate, accountDelete, entryView, entryCreate, entryApprove, entryPost, entryDelete, ledgerView, ledgerReconcile)। কোনো মথেড নই — শুধু কনস্ট্যান্ট গ্রুপি।

টপ-লেভেলে ধ্রুবক ও ফাংশন

const accountTypes = <String>[...]

AccountType-এর মরি — অ্যাকাউন্টের ধরনের তালিকা (ASSET, CONTRA_ASSET, LIABILITY, CONTRA LIABILITY, EQUITY, REVENUE, CONTRA_REVENUE, EXPENSE), বর্ণানুক্রমিকভাবে নয় বরং ব্যালান্স শিট ও তারপর ইনকাম স্টেটমেন্ট পড়ার ক্রমে সাজানো — একজন অ্যাকাউন্ট্যান্ট যতভাবে আশা করে।

String accountGroup(String type)

একটি অ্যাকাউন্ট টাইপ স্ট্রিং নিয়ে চার্ট-লিস্ট গ্রুপিংয়ের জন্য একটি বিভাগ নাম ('Assets', 'Liabilities', 'Equity', 'Revenue', 'Expenses', অথবা 'Other') রটার্ন করে switch এক্সপ্রেশন দিয়ে।

ক্লাস: Account

চার্ট-অব-অ্যাকাউন্টসের একটি একক অ্যাকাউন্ট, তার কোড, নাম, টাইপ, চার্ট ফ্ল্যাগ (isHeaderAccount, isBankAccount, allowDirectPosting, active), ঐচ্ছিক ব্যালান্স/ববিরণ/নোটসহ।

`bool get canPostTo`

`!isHeaderAccount && allowDirectPosting && active` — একটি জার্নাল-এন্ট্রি লাইন এই অ্যাকাউন্টের নাম উল্লেখ করতে পারে কিনা তা জানায়।

`String get group`

`accountGroup(type)` কল করে এই অ্যাকাউন্টের বিভাগ নাম রিটার্ন করে।

`factory Account.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে একটি `Account` তৈরি করে, প্রতিটি ফিল্ডের জন্য নরিপদ ডিফল্ট ব্যবহার করে (যেমন `id` না থাকলে `0`, `type` না থাকলে `'ASSET'`)। `isHeaderAccount` ও `isBankAccount` দুটি বানানই (`isHeaderAccount/headerAccount`, `isBankAccount/bankAccount`) পড়ে, কারণ ব্যাকএন্ড `@JsonProperty` দিয়ে এই দুটো কী জোর করে সেট করে (নাহলে Jackson Lombok-এর গটোর থেকে `"is"` ছুঁটে ফলেত), আর দুটি বানানই পড়া হয় যাতে ভবিষ্যতে সেই অ্যানোটেশন পরিবর্তন হলেও ফ্ল্যাগটি নীরবে ভেঙে না যায়।

ক্লাস: AccountRequest

`POST/PATCH /company/finance/chart-of-accounts`-এর জন্য পলোড। আপডেট নাম, টাইপ, বিবরণ, চারটি ফ্ল্যাগ ও নোটস — সবকিছু শরতহীনভাবে পাঠায়, কারণ চারটি ফ্ল্যাগ ব্যাকএন্ডে প্রমিটিভি বুলিয়ান হিসেবে ফিল্ড ইনশিয়ালাইজারসহ থাকে (`allowDirectPosting` ও `active` অনুপস্থিতি থাকলে ডিফল্ট `true` হয়ে যায়), তাই একটি ফ্ল্যাগ বাদ দেওয়া তাকে `false` পাঠানোর চেষ্টাও খারাপ — নীরবে একটি বন্ধ করা অ্যাকাউন্ট পুনরায় চালু করে দিতে পারে।

`factory AccountRequest.from(Account account)`

একটি বিদ্যমান `Account` থেকে (ওপেনিং ব্যালান্স বাদে) একটি `AccountRequest` তৈরি করে — এডটি ফর্ম প্র-ফিলি করার জন্য ব্যবহৃত হয়।

`Map<String, dynamic> toJson()`

রকিওয়েস্টকে JSON ম্যাপে রূপান্তর করে। ভেতরে একটি লোকাল `clean` হেল্পার আছে যা স্ট্রিং ট্রিম করে খালি হলে `null` করে দেয় (`description/notes`-এর জন্য)। `accountCode/accountName` ট্রিম করে পাঠানো হয়; `isHeaderAccount/isBankAccount` কী নাম দুটো স্পষ্টভাবে ব্যাকএন্ডের `@JsonProperty` পনিং মেলোতে ব্যবহৃত হয়েছে; `openingBalance/openingBalanceDate` শুধু `non-null` হলে অন্তর্ভুক্ত হয়।

ক্লাস: JournalEntry

একটি জার্নাল এন্ট্রি ও তার লাইনগুলো। এন্ট্রি নম্বর, তিনটি অবস্থা-ফ্ল্যাগ (`approved`, `posted`, `reversed`), মোট পরিমাণ (`amount` — যহেতু ডেবিট ও ক্রেডিট সমান হতে বাধ্য, একটি সংখ্যাই এন্ট্রির আকার বলে দেয়), তারিখ, তৈরিকারী/অনুমোদনকারী, ও রিভার্সাল-সম্পর্কিত আইডি ধারণ করে।

`bool get isReversal`

`reversedFromEntryId != null` — এই এন্ট্রিটি অন্য একটির রিভার্সাল কিনা।

`String get status`

`reversed` `'REVERSED'`, নাহলে `posted` `'POSTED'`, নাহলে `approved` `'APPROVED'`, নাহলে `'DRAFT'` — স্ট্যাটাস চপিরে জন্য একটি শব্দে অবস্থা।

`bool get canApprove`

`!approved` — এখনো অনুমোদিত না হলে অনুমোদন করা যায়।

`bool get canPost`

`approved && !posted` — অনুমোদিত কিন্তু এখনো পোস্ট না হলে পোস্ট করা যায়। এই তিনটি গটোর সার্বভসি-লয়োরের নজিস্ব গার্ডরে সাথে হুবহু মলে।

`bool get canReverse`

`posted && !reversed` — পোস্ট করা এবং এখনো রভির্স না হলে রভির্স করা যায়।

`bool get canDelete`

`!posted` — পোস্ট হয়ে গেলে ডলিটি করা প্রত্যাখ্যাত হয়, কারণ তখন লজোর নড়ে গেছে ও শুধু একটি রভির্সালই তা আনডু করতে পারে।

`double get totalDebits`

`lines`-এর প্রতটিলাইনরে `debitAmount` যোগ করে মোট রটার্ন করে।

`double get totalCredits`

`lines`-এর প্রতটিলাইনরে `creditAmount` যোগ করে মোট রটার্ন করে।

`factory JournalEntry.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে একটি `JournalEntry` তৈরি করে, নরিপদ ডফিল্টসহ, এবং `json['lines']`-এর প্রতটি ম্যাপ এন্ট্রিকে `JournalLine.fromJson` দিয়ে পার্স করে একটি তালিকা তৈরি করে।

ক্লাস: `JournalLine`

একটি পঠতি জার্নাল লাইন — `accountId`, `debitAmount`, `creditAmount`, ঐচ্ছকি `id/accountName/accountCode/lineDescription`।

`bool get isDebit`

`debitAmount > 0` — লাইনটি ডিবেটি সাইডে কনি।

`factory JournalLine.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে একটি `JournalLine` তৈরি করে, সংখ্যাসূচক ফলিডে নরিপদ ডফিল্ট (0) ব্যবহার করে।

ক্লাস: `JournalEntryRequest`

`POST /company/finance/journal-entries`-এর পলেড, সবসময় একটি লাইনরে তালিকা হসিবে পাঠানো হয় (DTO একটি লিগিয়াসি `debitAccountId/creditAccountId/amount` ট্রপিলও গ্রহণ করে যা ব্যাকএন্ড দুটি লাইনে প্রসারতি করে, কিন্তু সেই ফর্ম তনি-পায়রে এন্ট্রি প্রকাশ করতে পারে না)। `balances` ও `problem` গটোর ব্যাকএন্ডরে নয়িমগুলো স্থানীয়ভাবে যাচাই করে, যাতে ফর্মটি পাঠানোর আগাই সমস্যা বলতে পারে।

`double get totalDebits`

`lines`-এর প্রতটিলাইনরে `debitAmount` যোগ করে মোট রটার্ন করে।

`double get totalCredits`

`lines`-এর প্রতটিলাইনরে `creditAmount` যোগ করে মোট রটার্ন করে।

`bool get balances`

`(totalDebits - totalCredits).abs() < 0.005` — সমতা নয়, একটি সহনশীলতার মধ্যে তুলনা, কারণ এগুলো হাতে টাইপ করা ডেসিমাল আর ফ্লোটিং পয়ন্টে একটি নিরিভুল তুলনাকে মথিয়া করে তোল।

String? get problem

এন্ট্রির সাথে প্রথম যা ভুল আছে তা ব্যাকএন্ডের ভাষায় বর্ণনা করে রিটার্ন করে, অথবা কোনো সমস্যা না থাকলে null। ক্রমানুসারে যাচাই করে: অন্তত দুটি লাইন আছে কনি, কোনো নগেটেভি পরমাণ নই, কোনো লাইন একসাথে ডেবিট ও ক্রেডিট উভয়ই নয়, প্রতিটি লাইনে ডেবিট বা ক্রেডিট আছে, প্রতিটি লাইনে একটি অ্যাকাউন্ট নির্বাচন আছে, এবং শেষে balances — না মিললে পার্থক্যসহ বার্তা।

Map<String, dynamic> toJson()

রকিওয়েস্টকে JSON ম্যাপে রূপান্তর করে। ভেতরে একটি লোকাল clean হেল্পার (স্ট্রিং ট্রিম করে খালি হলে null) আছে। entryDate ও lines (প্রতিটি লাইনের toJson() কল করে) সবসময় পাঠানো হয়; description/notes শুধু clean করার পর অ-null হলে অন্তর্ভুক্ত হয়।

ক্লাস: JournalLineRequest

একটি এন্ট্রির একটি পা (leg)। accountId এখানে nullable কিন্তু ওয়ারে @NotNull — ফর্মটিকিউ এখানে অ্যাকাউন্ট হচ্ছে না নেওয়া পর্যন্ত একটি অর্ধ-তৈরি লাইন ধরে রাখে, এবং JournalEntryRequest.problem পাঠানোর আগে সেটি ধরে ফলে।

JournalLineRequest copyWith({int? accountId, double? debitAmount, double? creditAmount, String? lineDescription})

দেওয়া ফিল্ডগুলো নতুন মান দিয়ে প্রতিস্থাপন করে (?? দিয়ে) একটি নতুন JournalLineRequest তৈরি করে রিটার্ন করে — ফর্মে লাইন এডিট করার জন্য ব্যবহৃত।

Map<String, dynamic> toJson()

accountId, debitAmount, creditAmount — উভয় ডেবিট/ক্রেডিট সবসময় পাঠানো হয় (একটি শূন্য থাকলেও), যাতে ডেবিট-বা-ক্রেডিট আকৃতিটি ওয়ারে স্পষ্ট থাকে; lineDescription শুধু ট্রিম করার পর অ-খালি হলে অন্তর্ভুক্ত হয়।

ক্লাস: LedgerLine

জেনারেল লজোরের একটি পোস্ট করা লাইন। লজোর অ্যাপেন্ড-অনলি — এখানে কিছুই এডিট করা যায় না, শুধু ব্যাংক স্টেটমেন্টের বিপরীতে রকিনসাইল হিসাবে চহিনতি করা যায়।

bool get isDebit

debitAmount > 0 — লাইনটি ডেবিট সাইডে কনি।

double get amount

isDebit হলে debitAmount, নাহলে creditAmount — যে সাইডে পড়ছে সেটাই পরমাণ।

factory LedgerLine.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে একটি LedgerLine তৈরি করে। isReconciled উভয় বানানই (isReconciled/reconciled) পড়ে — Account-এর মতো একই "is"-প্রফিক্স প্রশ্ন, তবে এটির কোনো @JsonProperty পনিং নই, তাই উভয় বানানই পড়া নিরাপত্তার জন্য।

ফাইল: lib/features/accounting/accounting_repository.dart

সংক্ষিপ্ত ববিরণ: হিসাবরে বই — তনিটিস্তরে সংগঠতি API কল ও Riverpod প্রোভাইডার/কন্ট্রোলার: অ্যাকাউন্ট (একটি বাকটে), জার্নাল এন্ট্রি (সিই বাকটেগুলোর বপিরাতে ডেবিট/ক্রেডিটে একটি প্রস্তাব), এবং পোস্ট করার পর তরৈইওয়া লজোর লাইন (যা পরে আর এডিটি করা যায় না)। এছাড়াও ব্যাংক রকিনসলিযিশেন-সম্পর্কতি কল।

ক্লাস: AccountingRepository

`ApiClient`-কে রূপা করুে চার্ট-অব-অ্যাকাউন্টস, জার্নাল এন্ট্রি, জনোরলে লজোর, ও ব্যাংক রকিনসলিযিশেনরে সব HTTP কল প্রদান করুে।

`AccountingRepository(this._api)`

কনস্ট্রাক্টর — একটি `ApiClient` ইনস্ট্যান্স গ্রহণ করুে।

`Future<PagedResponse<Account>> accounts({String? type, int page = 0, int size = 100})`

পুরো চার্ট, বা `type` দেওয়া থাকলে একটি নির্দিষ্ট ধরনের অ্যাকাউন্ট (path segment হিসেবে) পৃষ্ঠাযুক্তভাবে ফরেত দেয়। সার্ভার-সাইডে কোড অনুসারে সাজানো থাকে।

`Future<Account> createAccount(AccountRequest request)`

`_accounts`-এ POST করুে নতুন অ্যাকাউন্ট তরৈকিরে এবং প্রতিক্রিয়া `Account.fromJson` দিয়ে পার্স করুে রটার্ন করুে।

`Future<Account> updateAccount(int id, AccountRequest request)`

`$_accounts/$id`-এ PATCH করুে (পুরো অ্যাকাউন্ট শরতহীনভাবে পাঠিয়ে — কনে, তার কারণ `AccountRequest`-এর ডকুমেন্টেশনে) অ্যাকাউন্ট আপডেটে করুে রটার্ন করুে।

`Future<void> deleteAccount(int id)`

`$_accounts/$id`-এ DELETE কল করুে। শুধু কোম্পানির মালকি করুতে পারে এবং এতে কখনো কিছু পোস্ট হয়ুে থাকলে সরাসরি প্রত্যাখ্যাত হয়।

`Future<PagedResponse<JournalEntry>> entries({int page = 0, int size = 25})`

`_entries`-এ পৃষ্ঠাযুক্ত জার্নাল এন্ট্রির তালকি আনে।

`Future<JournalEntry> entry(int id)`

`$_entries/$id`-এ একটি নির্দিষ্ট এন্ট্রি (তার লাইনসহ) আনে। লস্ট এন্ডপয়ন্টে কিছু ব্যাকএন্ডে লাইন বাদ দেয়, কনিতু ডিটাইল এন্ডপয়ন্টে সবসময় লাইন বহন করুে — তাই লাইন দেখানো একটি স্ক্রিনি সারতিে ভরসা না করুে পুনরায় পড়ুে।

`Future<JournalEntry> createEntry(JournalEntryRequest request)`

`_entries`-এ POST করুে নতুন জার্নাল এন্ট্রি তরৈকিরে রটার্ন করুে।

`Future<void> approveEntry(int id)`

`$_entries/$id/approve`-এ POST করুে এন্ট্রি অনুমোদন করুে। যুে ব্যবহারকারী এটি তরৈকিরছে সে অনুমোদন করুতে পারে না (segregation of duties) — ব্যাকএন্ড ভিন্ন বার্তায় প্রত্যাখ্যান করুে। খালি বডি ফরেত দেয়, তাই কলার পুনরায় পড়ুে।

`Future<void> postEntry(int id)`

`$_entries/$id/post`-এ POST করুে এন্ট্রি লিজোর লখে। আগুে অনুমোদতি হতে হয়। খালি বডি।

Future<JournalEntry> reverseEntry(int id)

\$_entries/\$id/reverse-এ POST করে একই অ্যাকাউন্টের বিপরীতে একটি বিপরীত এন্ট্রি পোস্ট করে এবং তা রিটার্ন করে। শুধু একটি পোস্ট করা, এখনো রিভার্স না হওয়া এন্ট্রি রিভার্স করা যায়।

Future<JournalEntry> deleteEntry(int id)

\$_entries/\$id-এ DELETE কল করে এবং প্রতিক্রিয়া পার্স করে রিটার্ন করে। শুধু কোম্পানির মালিকি এবং শুধু পোস্ট হওয়ার আগে করতে পারে।

Future<PagedResponse<LedgerLine>> ledger(({int? accountId, String? start, String? end, int page = 0, int size = 30}))

পোস্ট করা সব লেজার লাইন, একটি অ্যাকাউন্টের লাইন, অথবা একটি তারিখ-পরসির — তিনটি এন্ডপয়েন্ট একটি মিথেডের পছন্দে, কারণ তারা একই প্রশ্নের বিভিন্ন ফিল্টার দিয়ে উত্তর দেয় এবং একবারে শুধু একটি ফিল্টার প্রযোজ্য হয়।

Future<double> accountBalance(int accountId)

\$_ledger/account/\$accountId/balance-এ GET করে একটি অ্যাকাউন্টের বর্তমান ব্যালান্স আনবে। এন্ডপয়েন্ট ResponseEntity<BigDecimal> ঘোষণা করে, তাই বডি একটি খালি সংখ্যা (যেমন 1234.56) — কোনো অবজেক্ট আনুষঙ্গ্য করার প্রয়োজন নেই।

Future<void> reconcileLine(int id, String? notes)

\$_ledger/\$id/reconcile-এ POST করে একটি লেজার লাইনকে ব্যাংক স্টেটমেন্টের বিপরীতে টকি দেয়। notes একটি ঐচ্ছকি কোয়েরি প্যারামিটার (ট্রিমি করে খালি হলে বাদ) এবং প্রতিক্রিয়া খালি।

Future<PagedResponse<BankReconciliation>> reconciliations(({int page = 0, int size = 20}))

_reconciliation-এ পৃষ্ঠাযুক্ত রকিনসলিগেশনের তালিকা আনবে।

Future<List<BankReconciliation>> pendingReconciliations()

\$_reconciliation/pending-এ GET করে একটি খালি তালিকা (bare list) হিসেবে এখনো খোলা রকিনসলিগেশনগুলো আনবে এবং Map<String, dynamic> এন্ট্রিগুলো BankReconciliation.fromJson দিয়ে ম্যাপ করে অপরবর্তনীয় তালিকা রিটার্ন করে।

Future<BankReconciliation> reconciliation(int id)

\$_reconciliation/\$id-এ একটি নির্দিষ্ট রকিনসলিগেশন আনবে।

Future<BankReconciliation> openReconciliation(BankReconciliationRequest request)

_reconciliation-এ POST করে একটি ব্যাংক অ্যাকাউন্টের বিপরীতে নতুন রকিনসলিগেশন খোলে। লেজার ব্যালান্স ও তারিখ সার্ভার-সাইডে নির্ধারণিত হয়।

Future<List<LedgerLine>> unclearedTransactions(int id)

\$_reconciliation/\$id/uncleared-transactions-এ GET করে এই রকিনসলিগেশনের অ্যাকাউন্টের যসেব লেজার লাইন এখনো ব্যাংক দেখায়নি তার তালিকা আনবে — এগুলোই টকি দেওয়ার জন্য।

Future<BankReconciliation> toggleTransaction(int id, int glEntryId, {required bool cleared})

\$_reconciliation/\$id/transactions/\$glEntryId/toggle?cleared=\$cleared-এ POST করে একটি এন্ট্রিকি ক্লিয়ারড হিসেবে টকি দেয় বা তোলবে; রকিনসলিগেশন তার হিসাব পুনরায় গণনা করে ফেরত আসবে।

`Future<StatementImportResult> importStatement(int id, String filePath, String fileName)`
\$_reconciliation/\$id/import-statement-এ একটি ফাইল আপলোড করে (postFile) — একটি বি্যাংক স্টেটমেন্ট CSV পড়ে যতটা সম্ভব ম্যাচ করে টকি দিয়ে; যা ম্যাচ করনেতি কারণসহ লাইন-বাই-লাইন ফরমের আসে।

`Future<BankReconciliation> attachStatement(int id, String fileName, String fileUrl)`
\$_reconciliation/\$id/statement-এ POST করে একটি নাম ও URL (আগরে আপলোড থেকে, ফাইল নজি নয়) রকিনসলিগিশেনের সাথে সংযুক্ত করে, যা পরে অডিটের জন্য স্টেটমেন্টটি রিখে দিয়ে।

`Future<BankReconciliation> attachUploadedStatement(int id, PickedAttachment file)`
একটি ফাইল আপলোড করে (_api.uploadDocument) এবং তারপর attachStatement কল করে এক ধাপে রকিনসলিগিশেনের সাথে সংযুক্ত করে। দুই কল লাগে কারণ ব্যাকএন্ড এগুলো আলাদা রাখা — জনেরেকি আপলোড এন্ডপয়েন্ট ফাইল সংরক্ষণ করে ও একটি URL দিয়ে, তারপরই রকিনসলিগিশেনকে জানানো হয়; এই বিভাজন রপিজিটিবিলি রাখা হয়েছে যাত স্ক্রনিকে তা জানতে না হয়।

`Future<void> markReconciled(int id, {String? notes})`
\$_reconciliation/\$id/reconcile-এ POST করে রকিনসলিগিশেন সাইন-অফ করে। দুই পাশে এখনো পার্থক্য থাকলে প্রত্যাখ্যাত হয়। notes ঐচ্ছকি কোয়েরি প্যারামিটার, প্রতিক্রিয়া খালি।

টপ-লেভেলে প্রোভাইডার ও কন্ট্রোলার

`final accountingRepositoryProvider = Provider<AccountingRepository>(...)`
apiClientProvider থেকে ApiClient নিয়ে একটি AccountingRepository তৈরি করে সরবরাহ করে।

ক্লাস: AccountTypeFilterController (extends Notifier<String?>)

চার্ট কোন ধরনের অ্যাকাউন্টে সীমাবদ্ধ তা ধারণ করে; null মান পুরো চার্ট।

@override String? build()

প্রাথমিক অবস্থা null (কোনো ফিল্টার নেই) রিটার্ন করে।

void set(String? type)

নতুন টাইপ বর্তমানেরটির সমান হলে কিছু করে না, নাহলে স্টেটে আপডেট করে।

`final accountTypeFilterProvider = NotifierProvider<AccountTypeFilterController, String?>(...)` — ফিল্টার কন্ট্রোলার সরবরাহকারী প্রোভাইডার।

ক্লাস: AccountsController (extends AsyncNotifier<List<Account>>)

বর্তমান ফিল্টার অনুযায়ী অ্যাকাউন্টের তালিকা লোড ও কনসিসটেন্ট রাখে।

@override Future<List<Account>> build()

currentUserProvider ও accountTypeFilterProvider watch করে (ব্যবহারকারী বা ফিল্টার বদলালে পুনর্নির্মাণের জন্য) এবং _load() রিটার্ন করে।

Future<List<Account>> _load()

বর্তমান টাইপ-ফিল্টার দিয়ে accountingRepositoryProvider.accounts(...) কল করে এবং page.content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে স্টেটে পুনরায় লোড করে, এরর হলে সেটোও ধরে।

`void apply(Account updated)`

বর্তমান তালিকায় `id` মিলে যাওয়া অ্যাকাউন্টটি `updated` দিয়ে প্রতিস্থাপন করে; বর্তমান স্টেটে `null` (লোডে না) হলে কিছু করে না।

`void remove(int id)`

বর্তমান তালিকা থেকে `id` মিলে অ্যাকাউন্ট বাদ দিয়ে নতুন স্টেটে সেট করে; বর্তমান স্টেটে `null` হলে কিছু করে না।

`final accountsProvider = AsyncNotifierProvider<AccountsController, List<Account>>(...) —`
অ্যাকাউন্ট কন্ট্রোলারের প্রোভাইডার।

`final postableAccountsProvider = FutureProvider<List<Account>>(...) —` যসেব অ্যাকাউন্টে জার্নাল লাইন পোস্ট করা যায় (`canPostTo`) শুধু তাদের তালিকা, চার্ট স্ক্রিনিংর বর্তমান ফিল্টার থেকে স্বাধীনভাবে — একটি লাইন পিকার কখনো হডের অ্যাকাউন্ট দেখাবে না।

ক্লাস: JournalEntriesController (extends AsyncNotifier<List<JournalEntry>>)

জার্নাল এন্ট্রির তালিকা লোড ও কনসিসটেন্ট রাখা।

`@override Future<List<JournalEntry>> build()`

`currentUserProvider` watch করে এবং `_load()` রিটার্ন করে।

`Future<List<JournalEntry>> _load()`

`accountingRepositoryProvider.entries()` কল করে `page.content` রিটার্ন করে।

`Future<void> refresh()`

`AsyncValue.guard(_load)` দিয়ে পুনরায় লোড করে।

`void apply(JournalEntry updated)`

বর্তমান তালিকায় `id` মিলে যাওয়া এন্ট্রিটি `updated` দিয়ে প্রতিস্থাপন করে।

`void remove(int id)`

বর্তমান তালিকা থেকে `id` মিলে এন্ট্রি বাদ দিয়ে।

`final journalEntriesProvider = AsyncNotifierProvider<JournalEntriesController, List<JournalEntry>>(...) —`
এন্ট্রি কন্ট্রোলারের প্রোভাইডার।

`final journalEntryProvider = FutureProvider.autoDispose.family<JournalEntry, int>(...) —` একটি নির্দিষ্ট `id`-র এন্ট্রি তার লাইনসহ আনবে; প্রতিটি অ্যাকশনের পর পুনরায় পড়া হয়।

ক্লাস: LedgerAccountFilterController (extends Notifier<int?>)

লজার কোন অ্যাকাউন্টে সীমাবদ্ধতা ধারণ করে; `null` মানে সব।

`@override int? build()`

প্রাথমিক অবস্থা `null` রিটার্ন করে।

`void set(int? accountId)`

নতুন `accountId` বর্তমানটির সমান হলে কিছু করে না, নাহলে স্টেটে আপডেট করে।

`final ledgerAccountFilterProvider = NotifierProvider<LedgerAccountFilterController, int?>(...) —` এই কন্ট্রোলারের প্রোভাইডার।

ক্লাস: LedgerController (extends AsyncNotifier<List<LedgerLine>>)

বর্তমান অ্যাকাউন্ট-ফিল্টার অনুযায়ী লজোর লাইনরে তালিকা লোড ও কনসিসটেন্ট রাখে।

@override Future<List<LedgerLine>> build()

currentUserProvider ও ledgerAccountFilterProvider watch করে এবং _load() রটাইন করে।

Future<List<LedgerLine>> _load()

বর্তমান accountId ফিল্টার দিয়ে accountingRepositoryProvider.ledger(...) কল করে page.content রটাইন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে পুনরায় লোড করে।

void apply(LedgerLine updated)

বর্তমান তালিকায় id মিলে যাওয়া লাইনটি updated দিয়ে প্রতিস্থাপন করে — যখন রকিনসাইল করার পর প্রতিক্রিয়া না থাকায় স্থানীয়ভাবে লাইন পুনর্নির্মাণ করে প্রয়োগ করা হয়।

final ledgerProvider = AsyncNotifierProvider<LedgerController, List<LedgerLine>>(...) — লজোর কন্ট্রোলারের প্রোভাইডার।

final accountBalanceProvider = FutureProvider.autoDispose.family<double, int>(...) — একটি নির্দিষ্ট অ্যাকাউন্টের বর্তমান ব্যালান্স আনবে।

final pendingReconciliationsProvider = FutureProvider.autoDispose<List<BankReconciliation>>(...) — এখনো খোলা রকিনসলিযিশনের তালিকা; currentUserProvider watch করে।

final allReconciliationsProvider = FutureProvider.autoDispose<List<BankReconciliation>>(...) — বন্ধসহ সব রকিনসলিযিশন (size: 50) আনবে; currentUserProvider watch করে।

final reconciliationProvider = FutureProvider.autoDispose.family<BankReconciliation, int>(...) — একটি নির্দিষ্ট id-র রকিনসলিযিশন আনবে।

final unclearedTransactionsProvider = FutureProvider.autoDispose.family<List<LedgerLine>, int>(...) — একটি নির্দিষ্ট রকিনসলিযিশনের এখনো ক্লিয়ার না হওয়া লেনদেন আনবে।

ফাইল: lib/features/accounting/accounting_screen.dart

সংক্ষিপ্ত ববিরণ: হিসাবের বইয়ের প্রধান স্ক্রিনি — তিনটি ট্যাব (Accounts, Entries, Ledger) যথোভাবে জনিসিগুলো চলে সেই ক্রমে, প্রতিটি পারমশিন-নির্ভরভাবে দেখানো হয়। লজোর ট্যাবে কিছুই এডিট করা যায় না (একটি লাইন ব্যাংক স্টেটমেন্টের বিপরীতে টেকি দেওয়া ছাড়া) — এটাই লজোরের উদ্দেশ্য, তাই এটি অ্যাক্ট করার বদলে পড়ে।

ক্লাস/উইজটে: AccountingScreen (extends ConsumerWidget)

পারমশিন অনুযায়ী কোন ট্যাব (ও কোন FAB "New" অ্যাকশন) দেখানো হবে তা নির্ধারণ করে এবং DefaultTabController দিয়ে তিন-ট্যাব Scaffold তৈরি করে।

`AccountingScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর — ঐচ্ছকি `initialTabLabel` নষে, যা দযি়ে নর্দিষ্ট একটট্টিযাব থোলা যায় (যমেন অ্যাকাউন্ট থকে "Ledger for this account" নভেগিটে করার সময)।

`@override Widget build(BuildContext context, WidgetRef ref)`

`permissionControllerProvider` watch করে। পারমশিন এখনো লোড না হলে এবং কোড খালি থাকলে একটিলোডার-সহ `Scaffold` দেখায়। নাহলে পারমশিন অনুযায়ী `tabs` তালিকা তরৈকিরে (`Accounts` যদি `accountView` থাকে, `Entries` যদি `entryView`, `Ledger` যদি `ledgerView`) — প্রতটিরি নিজস্ব ভডি ও ঐচ্ছকি `create`-অ্যাকশন (`showAccountSheet/showJournalEntrySheet`, শুধু সংশ্লিষ্ট `create`-পারমশিন থাকলে)। `tabs` খালি হলে একটি "not available" `EmptyState` দেখায়। নাহলে `initialTabLabel` অনুযায়ী `initialIndex` নর্গয় করে (না মলিলে `clamp` দযি়ে সীমার ভেতরে রাখে) এবং `DefaultTabController`-এর ভেতরে `AnimatedBuilder` দযি়ে বর্তমান ট্যাবরে `create` অ্যাকশন অনুযায়ী `FloatingActionButton.extended` দেখায় বা লুকায, সাথে `TabBar/TabBarView`।

ক্লাস/উইজটে: `_AccountsTab` (extends `ConsumerStatefulWidget`)

"Accounts" ট্যাবরে জন্য একটিস্টেফুল শলে যা `_AccountsTabState`-এ কাজ ডলেগিটে করে।

`@override ConsumerState<_AccountsTab> createState()`

`_AccountsTabState`-এর একটাইনস্ট্যান্স তরৈকিরে রটার্ন করে।

ক্লাস: `_AccountsTabState` (extends `ConsumerState<_AccountsTab>`)

চার্ট-অব-অ্যাকাউন্টসরে তালিকা রন্ডার করে ও ডলিটি-অ্যাকশন পরচালনা করে। `_busyId` একটাই একক অ্যাকাউন্ট আইডি ধরে রাখে যা বর্তমানে ডলিটি প্রক্রিয়াধীন।

`Future<void> _delete(Account account)`

একটিনশ্চিতিকরণ ডায়ালগ (`confirmAction`) দেখায়; বাতলি হলে বা widget আনমাউন্ট হলে ফরেত যায়। নশ্চিতি হলে `_busyId` সেটে করে, `deleteAccount` কল করে, সফল হলে `accountsProvider.notifier.remove` ও `postableAccountsProvider` ইনভ্যালডিটে করে একটিসাফল্য সন্ধ্যাকবার দেখায়। `ApiException` হলে সরাসরি ব্যাকএন্ডরে বার্তা (যমেন "Cannot delete account with existing ledger entries") দেখায়; অন্য এরর হলে জনেরেকি বার্তা দেখায়। সবশষে `_busyId` null করে দেয়।

`@override Widget build(BuildContext context)`

পারমশিন অনুযায়ী `canEdit/canDelete` নর্গয় করে, বর্তমান টাইপ-ফল্টিার watch করে, এবং একটি `ConfigList<Account>` রটার্ন করে যার `header` একটি `FilterBar` (সব টাইপ + "All")। প্রতটিসিারি (`itemBuilder`) একটি `ConfigRow` — কোড+নাম শরিনোম, সাবটাইটলে টাইপ/হডোর/no-direct-posting/bank ট্যাগ, ট্রইলিং ব্যালান্স, এডিটি (থাকলে) ও অ্যাকশন তালিকা: "Ledger for this account" (`ledger` ট্যাব থাকলে — এটি সবসময় শেষে ট্যাব ধরে নযি়ে সরাসরি সখোনে যায়) এবং "Remove" (`canDelete` থাকলে)।

ক্লাস/উইজটে: `_EntriesTab` (extends `ConsumerWidget`)

"Entries" ট্যাবরে ভডি।

`@override Widget build(BuildContext context, WidgetRef ref)`

`journalEntriesProvider` watch করে একটি `ConfigList<JournalEntry>` রটার্ন করে, যার প্রতটিসিারি `_EntryRow` দযি়ে রন্ডার হয়।

ক্লাস/উইজটে: `_EntryRow` (extends `StatelessWidget`)

একটি জার্নাল এন্ট্রির তালিকা-সারি

`@override Widget build(BuildContext context)`

একটি ট্যাপযোগ্য `AppCard` তৈরি করে যা `JournalEntryScreen.open` কল করে, এন্ট্রি নম্বর (রভার্সাল হলে "reversal" লবেল সহ), বিবরণ, তারিখ বাম দিকে দেখায়, এবং পরিমাণ (`Fmt.money`) ও `StatusChip(entry.status, dense: true)` ডান দিকে দেখায়।

ক্লাস/উইজটে: `JournalEntryScreen` (extends `ConsumerStatefulWidget`)

একটি নির্দিষ্ট জার্নাল এন্ট্রির বিস্তারিত স্ক্রিন — তার লাইন, টোটাল, এবং এতে এখনো কী করা সম্ভব।

`JournalEntryScreen({super.key, required this.id})`

কনস্ট্রাক্টর — যে এন্ট্রি দেখানো হবে তার `id` নম্বর।

`static void open(BuildContext context, {required int id})`

`Navigator.of(context).push` দিয়ে একটি `MaterialPageRoute`-এ `JournalEntryScreen(id: id)` পুশ করে — এই স্ক্রিনিং নভিগেটে করার একমাত্র প্রবেশপথ।

`@override ConsumerState<JournalEntryScreen> createState()`

`_JournalEntryScreenState`-এর একটি ইনস্ট্যান্স তৈরি করে রিটার্ন করে।

ক্লাস: `_JournalEntryScreenState` (extends `ConsumerState<JournalEntryScreen>`)

এন্ট্রির অ্যাকশনগুলো (`approve/post/reverse/delete`) পরিচালনা করে, প্রতিটি অ্যাকশনের পরে সংশ্লিষ্ট সব প্রোভাইডার রিফ্রেশ/ইনভ্যালিডেট করে।

`Future<void> _act(Future<void> Function(AccountingRepository repo) action, String success, String failure, {bool pop = false})`

`_busy` সেট করে, দেওয়া `action` চালায়, সফল হলে `journalEntryProvider(widget.id)` ইনভ্যালিডেট করে, `journalEntriesProvider.refresh()` করে, `ledgerProvider` ইনভ্যালিডেট করে ও `accountsProvider.refresh()` করে (কারণ পোস্টিং ও রভার্সিং উভয়ই লেজার আর প্রতিটি অ্যাকাউন্ট ব্যালান্স নাড়ায়), একটি সাফল্য সূচকবার দেখায় এবং `pop: true` হলে ও `widget` এখনো মাউন্টে থাকলে পপ করে।

`ApiException` হলে ব্যাকএন্ডের সঠিক বার্তা (যেমন "must be approved before posting") দেখায়; অন্য এরর হলে দেওয়া `failure` বার্তা দেখায়। সবশেষে `_busy false` করে দেয়।

`@override Widget build(BuildContext context)`

`journalEntryProvider(widget.id)` ও `permissionControllerProvider` watch করে। লোডিং/এরর/ডাটা অবস্থা `async.when` দিয়ে হ্যান্ডলে করে। ডাটা অবস্থায় একটি `ListView` তৈরি করে:

বিবরণ+স্ট্যাটাসচিপ সহ একটি হিডার কার্ড (রভার্সড/রভার্সাল হলে ব্যানার সহ), "Lines" সেকশন যা প্রতিটি `_LineRow` ও টোটাল রো দেখায়, নোটস থাকলে একটি আলাদা কার্ড, এবং `_busy` হলে একটি লোডার নাহলে একটি `Wrap` — যাতে `canApprove/canPost/canReverse/canDelete` ও সংশ্লিষ্ট পারমিশন অনুযায়ী `_Action` বাটনগুলো দেখানো হয় (Post ও Reverse ও Delete-এর আগে প্রতিটির নিজস্ব `confirmAction` নিশ্চিতকরণ ডায়ালগ)। শেষে `canApprove` হলে একটি বিখ্যামূলক লাইন দেখায় যে যদি এন্ট্রি তৈরি করেছেন তিনি নিজেকে তা অনুমোদন করতে পারবেন না।

ক্লাস/উইজটে: `_LineRow` (extends `StatelessWidget`)

একটি জিওর্নাল লাইনের সারি— অ্যাকাউন্ট কোড+নাম, ঐচ্ছকি ববিরণ, এবং ডেবিট/ক্রেডিট কলাম আলাদা রয়েছে।

@override Widget build(BuildContext context)

অ্যাকাউন্ট কোড+নাম ও লাইন-ববিরণ বাম দিকে দেখায়, এবং দুটি নিরিদর্শিত-প্রস্থের কলামে (`debitAmount > 0/creditAmount > 0` হলে `Fmt.money`, নাহলে খালি) ডান দিকে দেখায় — কাগজে যত্নে লাইন মিলিয়ে দেখা হয় ঠিকি সত্নে দুটি স্থির কলাম রাখা হয়েছে।

ক্লাস/উইজটে: `_Action` (extends `StatelessWidget`)

একটি লবেলেড, আইকনসহ অ্যাকশন বাটন (`OutlinedButton.icon`), ঐচ্ছকিভাবে "destructive" স্টাইলে (লাল রঙে) রেন্ডার হয়।

`_Action({required this.label, required this.icon, required this.onPressed, this.destructive = false})`

কনস্ট্রাক্টর — লবেলে, আইকন, কলব্যাক, ও destructive ফ্ল্যাগ নিয়ে।

@override Widget build(BuildContext context)

destructive অনুযায়ী `bos.danger` বা `bos.brand` রঙে ফোরগ্রাউন্ড ও বর্ডার সেট করে একটি `OutlinedButton.icon` রিটার্ন করে।

ক্লাস/উইজটে: `_LedgerTab` (extends `ConsumerStatefulWidget`)

"Ledger" ট্যাবের জন্য একটি স্টেটফুল শেলে যা `_LedgerTabState`-এ কাজ ডেলিগেটে করে।

@override ConsumerState<`_LedgerTab`> createState()

`_LedgerTabState`-এর একটি ইনস্ট্যান্স তৈরি করে রিটার্ন করে।

ক্লাস: `_LedgerTabState` (extends `ConsumerState<_LedgerTab>`)

জেনারেল লেজারের তালিকা রেন্ডার করে ও রকিনসাইল-অ্যাকশন পরিচালনা করে। `_busyId` একটি একক লাইন আইডি ধরে রাখে যা বর্তমানে রকিনসাইল প্রক্রিয়াজ্ঞ।

`Future<void> _reconcile(LedgerLine line)`

`askForText` দিয়ে একটি ঐচ্ছকি নোট চায়; বাতলি হলে বা widget আনমাউন্ট হলে ফেরত যায়। নিশ্চিতি হলে `_busyId` সেট করে, `reconcileLine` কল করে, এবং যহেতু কোনও প্রতিক্রিয়া-বডি নিই তাই `ledgerProvider.notifier.apply` কল করে একটি নতুন `LedgerLine` (একই ডটো কনিতু `isReconciled: true` ও `reconciliationNotes` সেট করে) দিয়ে স্থানীয়ভাবে সারি পুনরনির্মাণ করে প্রয়োগ করে। সফল হলে একটি সন্ধ্যাকবার দেখায়; `ApiException` বা অন্য এরর হলে যথাযথ বার্তা দেখায়। সবশেষে `_busyId` null করে দেয়।

@override Widget build(BuildContext context)

বর্তমান `ledgerAccountFilterProvider` (`accountId`) ও `ledgerReconcile` পারমিশন watch করে। ফিল্টার করা থাকলে `accountsProvider`-এর ক্যাশ থেকে (আবার `fetch` না করে) অ্যাকাউন্টের নাম খুঁজে বের করে। একটি `ConfigList<LedgerLine>` রিটার্ন করে, যার `header` (ফিল্টার করা থাকলে) অ্যাকাউন্টের নাম ও একটি "Everything" বাটন দেখায় যা ফিল্টার সাফ করে। প্রতিটি সারি `_LedgerRow` দিয়ে রেন্ডার হয়, `onReconcile` শুধু `canReconcile && !line.isReconciled` হলে দেওয়া হয়।

ক্লাস/উইজটে: `_LedgerRow` (extends `StatelessWidget`)

একটি লেজার লাইনের সারি— অ্যাকাউন্ট নাম, ববিরণ, তারিখ+রফোরেন্স নম্বর, রকিনসাইল-আইকন (থাকলে), পরিমাণ, ডেবিট/ক্রেডিট লবেলে, এবং `busy/reconcile`-আইকন।

`_LedgerRow({required this.line, required this.busy, this.onReconcile})`

কনস্ট্রাক্টর — লজোর লাইন, busy-ফ্ল্যাগ, ও ঐচ্ছকি রকিনসাইল-কলব্যাক নিয়ে।

`@override Widget build(BuildContext context)`

একটি `AppCard`-এর ভেতরে অ্যাকাউন্ট নাম/ববিরণ/তারিখ+রফোরেন্স (রকিনসাইল হলে একটি চকেমার্ক আইকনসহ) বাম দিকে, পরমাণ (Fmt.money) ও "debit"/"credit" লবেলে ডান দিকে দেখায়। `busy` হলে একটি ছোট `CircularProgressIndicator`, নাহলে `onReconcile` দেওয়া থাকলে একটি `IconButton` ("Mark as reconciled" টুলটীপসহ) দেখায়।

`done`

ফাইল: lib/features/accounting/accounting_sheets.dart

সংক্ষিপ্ত ববিরণ: অ্যাকাউন্ট মডউলরে দুটি ফর্ম শিট নিয়ে গঠিত — একাউন্ট তৈরি/এডিট করার শিট এবং জার্নাল এন্ট্রি রচনা করার শিট। জার্নাল এন্ট্রি শিটে একাধিক লাইন (ডেবিট/ক্রেডিট) যোগ করা যায় এবং টাইপ করার সময়ই ব্যালেন্স যাচাই দেখানো হয়।

টপ-লেভেলে ফাংশন: showAccountSheet

`Future<void> showAccountSheet(BuildContext context, {Account? existing})`

একটি মডাল বটম শিট খোলে যাতো নতুন অ্যাকাউন্ট তৈরি বা বদ্যমান অ্যাকাউন্ট (`existing`) এডিট করা যায়। `isScrollControlled: true` দয়ি শিট পুরো উচ্চতা নতিে পারে। ভেতরে `_AccountSheet` উইজেটে বন্ডি করে।

ক্লাস/উইজেটে: _AccountSheet

`ConsumerStatefulWidget` — অ্যাকাউন্ট ফর্মে স্টেটেফুল কন্টইনার, `existing` নাল হলে ক্রয়িটে মোড, নাহলে এডিট মোড।

`_AccountSheetState createState()`

`_AccountSheetState` তৈরি করে, `override` মখেড।

ক্লাস: _AccountSheetState

অ্যাকাউন্ট ফর্মে স্টেটে ম্যানজেমেন্ট, ভ্যালিডেশন এবং সাবমশিন লজকি ধারণ করে। কোড, নাম, ববিরণ, নোটস — এই সব `TextEditingController` দয়িে চালতি, আর

টাইপ/হডোর/ব্যাক/ডাইরেক্ট-পোস্টিং/একটিভি — এসব বুলয়ান/ড্রপডাউন স্টেটে হসিবে রাখা।

`bool get _isEdit`

`widget.existing` নাল না হলে `true` রটার্ন করে — অর্থাৎ ফর্মটি এডিট মোডে আছে কনি তা বলে দয়ে।

`void dispose()`

সব `TextEditingController` (`_code`, `_name`, `_description`, `_notes`, `_openingBalance`) ডসিপোজ করে মেমেরি লকি এড়ায়, তারপর `super.dispose()` কল করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে, `AccountRequest` তৈরি করে সাবমিট করে। উল্লেখযোগ্য: হডোর অ্যাকাউন্ট হলে `allowDirectPosting` জোর করে `false` করে দেয় (যাতে ব্যাকএন্ডের সাথে সাংঘর্ষিক তথ্য না যায়); ওপেনিং ব্যালেন্স শুধু ক্রয়িটে মোডে পাঠানো হয়, কারণ এটি একটি প্রকৃত ব্যালেন্সিং জার্নাল এন্ট্রি পোস্ট করে দেয়। সফল হলে `updateAccount/createAccount` কল করে, প্রোভাইডার রফ্রেশ করে, শিট বন্ধ করে সন্ধ্যাকবার দেখায়। `ApiException` এলে সারভার-বার্তা দেখায়, অন্য এরর হলে জনৈকি বার্তা দেখায়। `finally`-তে `_submitting` `false` করে।

Widget build(BuildContext context)

`FormSheetFrame` দিয়ে ফর্মের কাঠামো বানায় — কোড ও নাম (পাশাপাশি), টাইপ ড্রপডাউন, ববিরণ, তনিট সুইচ (হডোর/ডাইরেক্ট-পোস্টিং/ব্যাক অ্যাকাউন্ট), একটি সুইচ, এবং ক্রয়িটে মোডে ওপেনিং ব্যালেন্স + তারখি ফিল্ড। কোড ভ্যালিডেশনে ৩-১০ অক্ষরের সীমা ব্যাকএন্ডের `@Size(min=3,max=10)` মতো।

টপ-লেভেলে ফাংশন: `showJournalEntrySheet`

Future<void> `showJournalEntrySheet`(BuildContext context)

জার্নাল এন্ট্রি রিচনার মডাল বটম শিট খোলে — `_JournalEntrySheet` উইজেটে রেন্ডার করে। ডকুমেন্টেশন কমেন্টে বলা আছে এটি অ্যাপের সবচেয়ে চওড়া ফর্ম: লাইনের তালিকা, প্রত্যাতিতে একাউন্ট ও পরিমাণ, এবং টাইপ করার সময়ই ব্যালেন্স-অসামঞ্জস্য দেখানো হয়।

ক্লাস/উইজেট: `_JournalEntrySheet`

জার্নাল এন্ট্রি ফর্মের স্টেটেফুল উইজেট শেলে।

`_JournalEntrySheetState createState()`

`_JournalEntrySheetState` তৈরি করে।

ক্লাস: `_JournalEntrySheetState`

জার্নাল এন্ট্রি লাইনসমূহ, তারখি, ববিরণ ও সাবমিশন লজিকি পরিচালনা করে। `_lines` একটি `List<JournalLineRequest>` যা শুরুতেই দুটি খালি লাইন দিয়ে ইনিশিয়ালাইজ হয় — কারণ ব্যালেন্সড এন্ট্রি ন্যূনতম দুই লাইন লাগে।

`void dispose()`

`_description` কন্ট্রোলার ডিসপোজ করে।

`JournalEntryRequest` `get _request`

বর্তমান ফর্ম স্টেটে থেকে একটি `JournalEntryRequest` তৈরি করে — এন্ট্রি তারখি (নাল হলে আজকের তারখি), লাইনগুলো, ববিরণ।

Future<void> `_submit()`

ফর্ম ভ্যালিডেট করে, `_request.problem` চেক করে (ব্যালেন্স না মিললে সেখান থেকেই এরর দেখিয়ে থামে)। এরপর `createEntry` কল করে, জার্নাল প্রোভাইডার রফ্রেশ করে, শিট বন্ধ করে জানায় যে এন্ট্রিটি অনুমোদনের অপেক্ষায় আছে। এরর হ্যান্ডলিং আগের মতোই — `ApiException` মসেজে বা জনৈকি বার্তা।

Widget build(BuildContext context)

তারিখ ফিল্ড, ববিরণ ফিল্ড, এবং `postableAccountsProvider`-এর `AsyncValue` অনুযায়ী লোডিং/এরর/ডটো স্টেটে রেন্ডার করে। ডটো এলে প্রতিটি লাইনরে জন্য `_LineEditor` বন্ডি করে ("আরকেটিলাইন" বাটনসহ) এবং সবশেষে একটি রানিং টোটাল ব্যানার দেখায় — ব্যালেন্সড হল সবিজ, না হল লাল রঙে কত টাকার পার্থক্য তা দেখায়।

ক্লাস/উইজেট: `_LineEditor`

`StatefulWidget` — একটি জিার্নাল লাইন (অ্যাকাউন্ট, ডেবিট/ক্রেডিট সাইড, পরিমাণ) এডিট করার উইজেট। ডেবিট/ক্রেডিট একটি সগেমেন্টেডে বাটন দিয়ে একসাথে নম্নিত্রতি — দুটি আলাদা ফিল্ড রাখলে একসাথে দুটোতে মান দেওয়ার ভুল হতে পারে বলে এভাবে ডিজাইন করা হয়েছে।

`_LineEditorState createState()`

`_LineEditorState` তৈরি করে।

ক্লাস: `_LineEditorState`

`_amount` কন্ট্রোলার এবং `_isDebit` বুলিয়ান দিয়ে একটি লাইনরে ইনপুট স্টেটে রাখে।

`static String _plain(double value)`

একটি ডাবল সংখ্যাকে প্রদর্শনযোগ্য স্ট্রিং-এ রূপান্তর করে — শূন্য হল খালি স্ট্রিং, পূর্ণসংখ্যা হল দশমিকি ছাড়া, নাহলে যমেন আছে তমেন।

`void dispose()`

`_amount` কন্ট্রোলার ডিসপোজ করে।

`void _emit()`

`_amount` টেক্সট পার্স করে বর্তমান `_isDebit` অনুযায়ী ডেবিট বা ক্রেডিট ফিল্ডে বসিয়ে `widget.onChangeed` কলব্যাক ডাকে — প্যারেন্টকে নতুন `JournalLineRequest` জানায়।

Widget build(BuildContext context)

অ্যাকাউন্ট নরিবাচনের ড্রপডাউন (কোড ও নাম দেখায়), সরানোর আইকন বাটন (যদি `onRemove` দেওয়া থাকে), ডেবিট/ক্রেডিট সগেমেন্টেডে বাটন এবং পরিমাণ ইনপুট ফিল্ড রেন্ডার করে।

ফাইল: `lib/features/accounting/reconciliation_models.dart`

সংক্ষিপ্ত ববিরণ: ব্যাংক রকিনসলিযিশেন সম্পর্কতি মডলে ও পারমশিন কোডরে সংগ্রহ। একটি রকিনসলিযিশেন একটি ব্যাংক অ্যাকাউন্টরে বপিরীতে খোলা হয়, স্টেটেমেন্ট ব্যালেন্সরে সাথে জনোরলে লজোররে ব্যালেন্সরে পার্থক্য মলোনো হয় লজোর এন্ট্রিটিকি করে, এবং পার্থক্য শূন্য হলই তা সাইন-অফ করা যায়।

ক্লাস: `ReconciliationPermissions`

`abstract final class` — রকিনসলিযিশেন সম্পর্কতি পারমশিন কোডগুলোর ধারক। `view`, `create` এবং `reconcile` সাইন-অফ করার জন্য আলাদা কোড, যাতো একজন রকিনসলিযিশেন খুলতে পারে কন্টি সাইন-অফ না করতে পারে) — তনিটি স্ট্যাটিক কনস্ট্যান্ট ধারণ করে। কোনো মেথড নেই।

ক্লাস: BankReconciliation

একটি ব্যাংক রকিনসলিযিশেনের সম্পূর্ণ অবস্থা — জট্রিল ব্যালেন্স, স্টেটমেন্ট ব্যালেন্স, পার্থক্য, সমন্বতি ব্যাংক ব্যালেন্স, সাইন-অফ স্ট্যাটাস, এবং সংযুক্ত স্টেটমেন্ট ফাইলের তথ্য।

`bool get balances`

পার্থক্য (*difference*) ০.০০৫-এর কম কনি যাচাই করে বলে দেয় রকিনসলিযিশেনটি সাইন-অফের জন্য প্রস্তুত কনি — রাউন্ডিং ত্রুটির জন্য সামান্য পার্থক্যকে গ্রহণযোগ্য ধরা হয়।

`factory BankReconciliation.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে `BankReconciliation` অবজেক্ট তৈরি করে; ভেতরে একটি লোকাল হেল্পার `number(String key)` সংখ্যাসূচক ফিল্ডগুলো নরিপদে `double`-এ রূপান্তর করে, অনুপস্থিতি থাকলে 0 ধরে।

ক্লাস: BankReconciliationRequest

নতুন রকিনসলিযিশেন খোলার রকি়োস্ট বডি — শুধু ব্যাংক অ্যাকাউন্ট আইডি ও স্টেটমেন্ট ব্যালেন্স পাঠানো হয়; তারিখ ও লজোর ব্যালেন্স সারভার-সাইডে নরিধারতি হয় বলে এখানে পাঠানো হয় না।

`Map<String, dynamic> toJson()`

`bankAccountId` ও `bankStatementBalance` নিয়ে একটি JSON ম্যাপ তৈরি করে।

ক্লাস: StatementImportResult

ব্যাংক স্টেটমেন্ট আমদানিকরার পরণিতি — মোট লাইন, মলি যাওয়া সংখ্যা, না-মলো লাইনের তালিকা, এবং আমদানির পরে হালনাগাদ `BankReconciliation` (যদি থাকে, যাত পর্দার পছনের স্ক্রনিকে আলাদাভাবে আবার কল করতে না হয়)।

`factory StatementImportResult.fromJson(Map<String, dynamic> json)`

JSON থেকে গণনা ও তালিকা পার্স করে `StatementImportResult` তৈরি করে; `unmatchedLines` তালিকাটি `UnmatchedStatementLine.fromJson` ম্যাপ করে বানানো হয়, এবং `reconciliation` ফিল্ডটি থাকলে সেটিও পার্স করে।

ক্লাস: UnmatchedStatementLine

একটি স্টেটমেন্ট লাইন যা আমদানির সময় লজোরের কোনো এন্ট্রির সাথে মলোনো যায়নি, এবং কনে যায়নিতির কারণ।

`factory UnmatchedStatementLine.fromJson(Map<String, dynamic> json)`

JSON থেকে পরমিণ, তারিখ, ববিরণ ও কারণ পড়ে `UnmatchedStatementLine` অবজেক্ট তৈরি করে।

ফাইল: lib/features/accounting/reconciliation_screen.dart

সংকষপিত ববিরণ: ব্যাংক রকিনসলিযিশেনের তালিকা স্ক্রনি এবং বসিতারতি স্ক্রনি — নতুন রকিনসলিযিশেন খোলা, স্টেটমেন্ট ফাইল আমদানি/সংযুক্ত করা, লজোর এন্ট্রি টিক করা এবং সাইন-অফ করার সম্পূর্ণ ফলো এখানে বাসতবায়তি। প্রতিটি অ্যাকশন পারমশিন-গটেডে।

ক্লাস: ReconciliationScopeController

Notifier<bool> — তালিকা "শুধু খোলা" নাকি "সব" দেখাচ্ছে তার স্টেটে রাখে।

bool build()

প্রাথমিক স্টেটে true রিটার্ন করে — অর্থাৎ, ডিফল্টভাবে শুধু খোলা রকিনসলিযিশেনগুলোই দেখানো হয়।

void set(bool openOnly)

স্টেটে আপডেট করে openOnly মান দিয়ে, যা তালিকার ফিল্টার নম্বিন্ত্রণ করে।

ক্লাস: ReconciliationScreen

ConsumerWidget — রকিনসলিযিশেনগুলোর তালিকা দেখায়, "শুধু খোলা"/"সব" ফিল্টার বার সহ, এবং তৈরি পারমিশন থাকলে একটি ফিল্টারিং অ্যাকশন বাটন দেখায়।

Widget build(BuildContext context, WidgetRef ref)

openOnly স্কোপ অনুযায়ী pendingReconciliationsProvider বা allReconciliationsProvider থেকে ডেটা দেখায়। ConfigList উইজেটে দিয়ে লোডিং/এরর/খালি ডেটা স্টেটে হ্যান্ডল করে, প্রতিটি আইটেমের জন্য _ReconciliationRow বন্ডি করে।

ক্লাস: _ReconciliationRow

StatelessWidget — একটি রকিনসলিযিশেনের সারাংশ কার্ড হিসেবে দেখায়: ব্যাংক অ্যাকাউন্টের নাম, স্ট্যাটাস চপি, তারিখ, ব্যাংক ও বইয়ের ব্যালেন্স, এবং না মলো থাকলে "আউট বাই" বার্তা।

Widget build(BuildContext context)

AppCard রেন্ডার করে যা ট্যাপ করলে ReconciliationDetailScreen.open ডাকে; কার্ডের ভেতরে নাম, স্ট্যাটাস, তারিখ+ব্যালেন্স তথ্য, এবং সাইন-অফ না হলে ব্যালেন্স/অসামঞ্জস্যের বার্তা দেখায়।

টপ-লেভেলে ফাংশন: showOpenReconciliationSheet

Future<void> showOpenReconciliationSheet(BuildContext context)

রকিনসলিযিশেন খোলার _OpenSheet মডাল বটম শিট প্রদর্শন করে।

ক্লাস: _OpenSheet

ConsumerStatefulWidget — নতুন রকিনসলিযিশেন খোলার ফর্ম শিট।

_OpenSheetState createState()

_OpenSheetState তৈরি করে।

ক্লাস: _OpenSheetState

ব্যাংক অ্যাকাউন্ট নির্বাচন ও স্টেটেমেন্ট ব্যালেন্স ইনপুট নিয়ে রকিনসলিযিশেন খোলার লজিক ধারণ করে।

void dispose()

_balance কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে, অ্যাকাউন্ট নরিবাচতি না থাকলে এরর দেখায়। openReconciliation কল করে, তালিকার প্রোভাইডার ইনভ্যালিডেট করে, শিট বন্ধ করে সরাসরিনতুন রকিনসলিযিশেনরে ReconciliationDetailScreen-এ নভেগিটে করে। এরর হলে যথাযথ বার্তা দেখায়।

Widget build(BuildContext context)

শুধু isBankAccount == true চহিনতি অ্যাকাউন্টগুলো ফলিটার করে ড্রপডাউনে দেখায় (নন-ব্যাংক অ্যাকাউন্ট রকিনসাইল করা অর্থহীন বলে); অ্যাকাউন্ট না থাকলে সতর্কবার্তা দেখায়। স্টেটেমেন্ট ব্যালেন্স ইনপুট ফলিড এবং একটি বিখ্যামূলক বার্তা (বইয়ের ব্যালেন্স লজোর থেকে পড়া হয়, তারখি আজকরে) রনেডার করে।

ক্লাস: ReconciliationDetailScreen

ConsumerWidget — একটি নির্দিষ্ট id-এর রকিনসলিযিশেনরে বসিতারতি পৃষ্ঠা: হিসাব-সারাংশ, স্টেটেমেন্ট ফাইল ম্যানজেমেন্ট, এবং না-মলো লজোর লাইনরে তালিকা।

static void open(BuildContext context, {required int id})

MaterialPageRoute দিয়ে নভেগিটরে ReconciliationDetailScreen(id: id) পুশ করে বসিতারতি পৃষ্ঠা খোলো।

Widget build(BuildContext context, WidgetRef ref)

reconciliationProvider(id)-এর AsyncValue অনুযায়ী লোডিং/এরর/ডটো দেখায়; ডটো এলে RefreshIndicator-এর ভতেরে _Figures, _Statement, _Uncleared — তনিট সিকেশন সাজিয়ে দেখায়।

ক্লাস: _Figures

ConsumerStatefulWidget — রকিনসলিযিশেনরে সাংখ্যিক সারাংশ (ব্যাংক ব্যালেন্স, সমন্বয়, বইয়ের ব্যালেন্স, পার্থক্য) এবং সাইন-অফ বাটন ধারণ করে।

_FiguresState createState()

_FiguresState তরৈকিরে।

ক্লাস: _FiguresState

সাইন-অফ অ্যাকশনের _busy স্টেটে রাখো।

Future<void> _signOff()

askForText ডায়ালগ দিয়ে ঐচ্ছকি নোট চায়, তারপর markReconciled কল করে রকিনসলিযিশেনটা সাইন-অফ করে এবং সংশ্লিষ্ট প্রোভাইডারগুলো ইনভ্যালিডেট করে। এরর হলে সন্ধ্যাকবারে বার্তা দেখায়।

Widget build(BuildContext context)

AppCard-এ ব্যাংক অ্যাকাউন্টরে নাম, একাধকি _Line (ব্যাংক ব্যালেন্স, না-দখোনো ডপিোজটি, উপস্থাপন না-হওয়া চকে, সমন্বতি ব্যাংক ব্যালেন্স, বইয়ের ব্যালেন্স, পার্থক্য) দেখায়। ইতমিধ্যে সাইন-অফ হলে কে ও কবে করছে তা দেখায়, না হলে (পারমশিন থাকলে) সাইন-অফ বাটন দেখায় যা পার্থক্য শূন্য না হওয়া পর্যন্ত নষিক্রয়ি থাকো।

ক্লাস: _Line

StatelessWidget — একটি লিবেলে-মূল্য জোড়া সারহিসিবে দেখোনোর সাধারণ উইজটে, emphasis ও tone দিয়ে গুরুত্ব ও রঙ নযিন্তরণ করা যায়।

Widget build(BuildContext context)

লবেলে বাম দিকে, টাকার পরিমাণ (Fmt.money) ডান দিকে — emphasis সত্য হলে ভারী ফন্ট ও বড় সাইজ, tone দেওয়া থাকলে সেই রঙে মূল্য দেখায়।

ক্লাস: _Statement

ConsumerStatefulWidget — স্টেটমেন্ট ফাইল আমদানি (CSV থেকে স্বয়ংক্রিয় ম্যাচিং) এবং শুধু ফাইল সংযুক্ত করা (কপি রাখা) — দুই ভিন্ন অ্যাকশনের UI ধারণ করে।

_StatementState createState()

_StatementState তৈরি করে।

ক্লাস: _StatementState

_busy স্টেটে এবং দুটি ফাইল-সম্পর্কিত অ্যাকশন মথড রাখো।

Future<void> _import()

ফাইল বাছাই করিয়ে (pickAttachment) importStatement কল করে, ফলাফল দিয়ে প্রোভাইডার রিফ্রেশ করে এবং _ImportResultSheet একটি নতুন মডাল বটম শিটে দেখায় — কারণ কী মলেনেসিটোই ফলাফলের গুরুত্বপূর্ণ অংশ, তাই সন্ধ্যাকবারের বদলে শিট ব্যবহার করা হয়েছে।

Future<void> _attach()

ফাইল বাছাই করিয়ে attachUploadedStatement কল করে — এটি দুই ধাপের প্রক্রিয়া: ফাইলটি জিনেরেকি আপলোড এন্ডপয়েন্টে জমা হয়, তারপর তার URL রকিনসলিফিশনের সাথে ফাইল করা হয়। সফল হলে ফাইলের নাম জানিয়ে সন্ধ্যাকবার দেখায়।

Widget build(BuildContext context)

সংযুক্ত ফাইলের নাম (থাকলে) দেখায়, এবং সাইন-অফ না হওয়া পর্যন্ত "Import a CSV" ও "File/Replace the filed statement" বাটন দুটি দেখায়, সাথে দুই অ্যাকশনের পার্থক্য ব্যাখ্যা করা টেক্সট।

ক্লাস: _ImportResultSheet

StatelessWidget — আমদানির ফলাফল (মোট লাইন, মোলো সংখ্যা, না-মোলো লাইনের তালিকা কারণসহ) দেখানোর শিট।

Widget build(BuildContext context)

শরীোনামে "X of Y matched" দেখায়, তারপর না-মোলো প্রতিটি লাইনের বিবরণ, তারিখ, কারণ ও পরিমাণ তালিকাভুক্ত করে দেখায়।

ক্লাস: _Uncleared

ConsumerStatefulWidget — এখনও ব্যাংকে না-দেখানো লজোর এন্ট্রিগুলোর তালিকা ও টকি-অফ ইন্টারঅ্যাকশন ধারণ করে।

_UnclearedState createState()

_UnclearedState তৈরি করে।

ক্লাস: _UnclearedState

_busyId দিয়ে কোন লাইনটি বর্তমানে টগল হচ্ছে তা ট্র্যাক করে।

Future<void> _toggle(LedgerLine line, bool cleared)

toggleTransaction কল করে একটি লিজার লাইনকে ক্লিয়ার/আনক্লিয়ার হিসেবে চিহ্নিত করে, তারপর রকিনসলিগেশন ও না-মলো তালিকার প্রোভাইডার ইনভ্যালিডেট করে। ব্যর্থ হলে সন্ধ্যাকবারে বার্তা দেখায়।

Widget build(BuildContext context)

unclearedTransactionsProvider(id)-এর অ্যাসঙ্কি স্টেটে অনুযায়ী লোডিং/এরর/খালি-বার্তা/তালিকা দেখায়; প্রতটি লাইনরে জন্য _LineRow বন্ডি করে।

ক্লাস: _LineRow

StatelessWidget — একটি না-মলো লিজার লাইনরে চকেবক্স, বিবরণ ও পরিমাণ দেখানোর সারাি

Widget build(BuildContext context)

busy হলে স্পিনার, না হলে চকেবক্স দেখায় (যা enabled না থাকলে নষ্টকরয়ি); ডেবিটি বয়োগ ক্রেডিট হিসাব করে পরিমাণ দেখায় — ঋণাত্মক হলে লাল রঙে।

ফাইল: lib/features/admin/accounts_screen.dart

সংক্ষিপ্ত বিবরণ: কোম্পানির লগইনধারী ব্যবহারকারীদের ("কে সাইন-ইন করতে পারে") তালিকা দেখানোর স্ক্রিনি, সংশ্লিষ্ট মডলে, রপিজিটিরি ও প্রোভাইডার। এটি এমপ্লয়মেন্ট-ভিত্তিক পপিল ডিরেক্টরি থেকে ভিন্ন — এটি অ্যাক্সেসে (লগইন সক্ষমতা ও অনুমতি) নষি কাজ করে।

ক্লাস: CompanyUser

একজন ব্যক্তির কোম্পানিতে লগইন আছে — মালিক, কর্মচারী বা পোর্টাল ক্লায়েন্ট হতে পারে; তালিকায় প্রত ব্যক্তির শুধুমাত্র সবচেয়ে শক্তিশালী মেম্বারশপি দেখানো হয়, দুইবার নয়।

String get fullName

firstName ও lastName জোড়া দিয়ে ট্রিম করে সম্পূর্ণ নাম রটার্ন করে।

String get initials

নাম থেকে আদ্যক্ষর তৈরি করে — উভয় নাম খালি হলে '?', শুধু lastName খালি হলে firstName-এর প্রথম অক্ষর, নাহলে দুটোর প্রথম অক্ষর মলিয়ে বড় হাতের অক্ষরে রটার্ন করে।

factory CompanyUser.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে CompanyUser তৈরি করে, অনুপস্থিতি ফিল্ডে যথাযথ ডিফল্ট বসায়।

ক্লাস: AccountsRepository

ব্যবহারকারীদের তালিকা আনার জন্য API কলের রূপার ক্লাস।

Future<PagedResponse<CompanyUser>> users({String? keyword, String? membership, int page = 0, int size = 20})

/users এন্ডপয়েন্ট থেকে পেজেড ব্যবহারকারী তালিকা আনে, কীওয়ার্ড ও মেম্বারশপি ফিল্ডার সহ। কোম্পানি-বহীন অ্যাকাউন্টরে (প্ল্যাটফর্ম স্টাফ) জন্য এটি প্রত্যাখ্যাত হয়, কারণ তাদের নিজস্ব আলাদা স্ক্রিনি আছে।

ক্লাস: AccountSearchController

Notifier<String?> — অনুসন্ধান কীওয়ার্ডের স্টেটে রাখবে।

String? build()

প্রাথমিক স্টেটে null রিটার্ন করে।

void set(String? keyword)

প্রদত্ত কীওয়ার্ড ট্রিম করে; খালি হলে null হিসেবে সংরক্ষণ করে, নতুন মান আগের সমান হলে কিছু করে না (অপ্রয়োজনীয় রিবিল্ড এড়াতে)।

ক্লাস: AccountMembershipController

Notifier<String?> — নির্বাচনী মেম্বারশিপ ফিল্টারের স্টেটে রাখবে।

String? build()

প্রাথমিক স্টেটে null (অর্থাৎ "সবাই") রিটার্ন করে।

void set(String? membership)

নতুন মেম্বারশিপ মান আগের থেকে ভিন্ন হলে স্টেটে আপডেট করে।

ক্লাস: AccountsController

AsyncNotifier<PagedState<CompanyUser>> যা PagedLoader<CompanyUser> মকসইন ব্যবহার করে — পজেনিটেডে অ্যাকাউন্ট তালিকা লোড ও পজিং পরিচালনা করে।

Future<PagedState<CompanyUser>> build()

currentUserProvider, accountSearchProvider, accountMembershipProvider ওয়াচ করে (পরিবর্তন হলে পুনর্গঠনের জন্য) এবং loadFirstPage() কল করে প্রথম পজে লোড করে।

Future<PagedResponse<CompanyUser>> fetchPage(int page)

AccountsRepository.users কল করে বর্তমান কীওয়ার্ড ও মেম্বারশিপ ফিল্টার দিয়ে নির্দিষ্ট পজে আনে — PagedLoader মকসইনের জন্য প্রয়োজনীয় override।

ক্লাস: AccountsScreen

ConsumerStatefulWidget — অনুসন্ধান ফিল্ড, মেম্বারশিপ ফিল্টার বার এবং পজেনিটেডে ব্যবহারকারী তালিকা সমন্বিত মূল স্ক্রিনি।

static void open(BuildContext context)

MaterialPageRoute দিয়ে AccountsScreen নভিগেটরে পুশ করে।

_AccountsScreenState createState()

_AccountsScreenState তৈরি করে।

ক্লাস: _AccountsScreenState

স্ক্রিনির UI বিল্ড করে — কোনো অতিরিক্ত স্টেটে নেই, শুধু build।

Widget build(BuildContext context)

AppSearchField, FilterBar (Everyone/Owner/Staff/Clients) এবং PagedListview<CompanyUser> সাজিয়ে দেখায়, প্রতিটি আইটেমের জন্য _AccountRow বিল্ড করে।

ক্লাস: `_AccountRow`

`StatelessWidget` — একজন ব্যবহারকারীর তথ্য (অ্যাভাটার, নাম, ইমেইল, মেম্বারশিপ/রোল, একটি স্ট্যাটাস, যাচাই সতর্কতা) কার্ড আকারে দেখায়।

`Widget build(BuildContext context)`

`Avatar`, নাম, ইমেইল (থাকলে), মেম্বারশিপ ও কাস্টম-রোল/বলিট-ইন-রোল (কাস্টম রোলটি অগ্রাধিকার পায় কারণ সেটাই আসল অনুমতি নির্ধারণ করে) দেখায়। ডান পাশে `StatusChip` (Active/Disabled) এবং ইমেইল যাচাই না হলে "not verified" সতর্কতা দেখায়।

ফাইল: `lib/features/admin/admin_form_sheets.dart`

সংক্ষিপ্ত বিবরণ: অ্যাডমিন মডুলের একাধিক ফর্ম শিট একত্রে ধারণ করে — ডিপার্টমেন্ট, ডেজিগনেশন (গ্রুপে), সার্ভিস ক্যাটাগরি, অ্যানাউন্সমেন্ট (AI-ড্রাফটিং সহ), এবং কাস্টম রোল তৈরি/এডিট করার শিট। প্রতিটি শিট একই প্যাটার্ন অনুসরণ করে: ফর্ম ভ্যালিডেশন, রকিওয়েস্ট তৈরি, রপিজিটরিকল, এরর হ্যান্ডলিং।

টপ-লেভেলে ফাংশন: `showDepartmentSheet`

`Future<void> showDepartmentSheet(BuildContext context, {Department? existing})`

ডিপার্টমেন্ট তৈরি/এডিট করার `_DepartmentSheet` মডাল বটম শিট প্রদর্শন করে।

ক্লাস: `_DepartmentSheet`

`ConsumerStatefulWidget` — ডিপার্টমেন্ট ফর্মের শেলে।

`_DepartmentSheetState createState()`

`_DepartmentSheetState` তৈরি করে।

ক্লাস: `_DepartmentSheetState`

নাম, কোড, বিবরণ, বাজেটে ইনপুট নিয়ে ডিপার্টমেন্ট ফর্মের স্টেটে ও সাবমিশন লজিক ধারণ করে।

`bool get _isEdit`

`widget.existing` নাল না হলে `true` রিটার্ন করে।

`void dispose()`

`_name, _code, _description, _budget` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে `DepartmentRequest` তৈরি করে, এডিট মোডে `updateDepartment` নাহলে `createDepartment` কল করে, প্রোভাইডার আপডেট/রিফ্রেশ করে, শিট বন্ধ করে স্ন্যাকবার দেখায়। ডুপ্লিকেট নাম হলে `ApiException`-এর বার্তা দেখায়।

`Widget build(BuildContext context)`

নাম (২+ অক্ষর ভ্যালিডেশন, ব্যাকএন্ডের `@Size(min=2)` মতো), কোড (ঐচ্ছিক), বাজেটে (ঐচ্ছিক, ঋণাত্মক নষিধে), বিবরণ (ঐচ্ছিক) ফিল্ড দিয়ে `FormSheetFrame` বন্ডি করে।

টপ-লভেলে ফাংশন: showDesignationSheet

Future<void> showDesignationSheet(BuildContext context, {Designation? existing})
ডেজিগনেশন (জব-গ্রুডে) তৈরি/এডিট করার _DesignationSheet মডাল বটম শিট প্রদর্শন করে।

ক্লাস: _DesignationSheet

ConsumerStatefulWidget — ডেজিগনেশন ফর্মের শেলে।

_DesignationSheetState createState()

_DesignationSheetState তৈরি করে।

ক্লাস: _DesignationSheetState

নাম, কোড, লভেলে, এমপ্লয়মেন্ট ক্যাটাগরি, বিবরণ ও ডিপার্টমেন্ট নির্বাচন নিয়ে ডেজিগনেশন ফর্মের স্টেটে ধারণ করে।

bool get _isEdit

এডিট মোডে আছে কিনা যাচাই করে।

void dispose()

_name, _code, _level, _category, _description কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে DesignationRequest তৈরি করে (level স্ট্রিং থেকে int.parse), এডিট/ক্রিয়েটে অনুযায়ী updateDesignation/createDesignation কল করে, প্রোভাইডার আপডেট করে, শিট বন্ধ করে সন্ধ্যাকবার দেখায়।

Widget build(BuildContext context)

নাম, কোড (স্টোর হওয়ার আগে আপার-কসে রূপান্তরিত হবে জানিয়ে হেল্পার টেক্সট), লভেলে (হুইল-সংখ্যা ভ্যালিডেশন, নমিনমান মানবে বেশি সিনিয়র), ডিপার্টমেন্ট ড্রপডাউন (থাকলে), এমপ্লয়মেন্ট ক্যাটাগরি ও বিবরণ ফিল্ড দিয়ে ফর্ম বন্ডি করে।

টপ-লভেলে ফাংশন: showServiceCategorySheet

Future<void> showServiceCategorySheet(BuildContext context, {ServiceCategory? existing})
সার্ভিস ক্যাটাগরি তৈরি/এডিট করার _ServiceCategorySheet মডাল বটম শিট প্রদর্শন করে।

ক্লাস: _ServiceCategorySheet

ConsumerStatefulWidget — সার্ভিস ক্যাটাগরি ফর্মের শেলে।

_ServiceCategorySheetState createState()

_ServiceCategorySheetState তৈরি করে।

ক্লাস: _ServiceCategorySheetState

নাম, সর্ব-অর্ডার, বিবরণ নিয়ে সার্ভিস ক্যাটাগরি ফর্মের স্টেটে ধারণ করে।

bool get _isEdit

এডিট মোডে আছে কিনা যাচাই করে।

`void dispose()`

`_name`, `_sortOrder`, `_description` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে `ServiceCategoryRequest` তৈরি করে (`sortOrder` প্যারামিটার বর্ধিত হলে 0 ধরে), এডিট/ক্রিয়েটে অনুযায়ী রপিকর্ষের ক্রিয়াকলাপ করে, প্রোভাইডার আপডেট করে, শিট বন্ধ করে সন্যাকবার দেখায়।

`Widget build(BuildContext context)`

নাম, সর্ট-অর্ডার (হুইল-সংখ্যা, নমিনমান আগা দেখায়), ববিরণ (ঐচ্ছিক) ফলিড দিয়ে ফর্ম বলিড করে।

টপ-লেভেলে ফাংশন: `showAnnouncementSheet`

`Future<void> showAnnouncementSheet(BuildContext context, {AdminAnnouncement? existing})`

অ্যানাউন্সমেন্ট তৈরি/এডিট করার `_AnnouncementSheet` মডাল বটম শিট প্রদর্শন করে।

ক্লাস: `_AnnouncementSheet`

`ConsumerStatefulWidget` — অ্যানাউন্সমেন্ট ফর্মের শেলে, যাতায়ে AI দিয়ে বার্তা ড্রাফট করার সুবিধাও আছে।

`_AnnouncementSheetState createState()`

`_AnnouncementSheetState` তৈরি করে।

ক্লাস: `_AnnouncementSheetState`

শিরোনাম, বার্তা, দর্শক, শিডিউল/মোয়াদ-উত্তীর্ণ তারিখ, নোটিফিকেশন-সুইচ নিয়ে অ্যানাউন্সমেন্ট ফর্মের স্টেটে ধারণ করে। AI ড্রাফটিংয়ের জন্য আলাদা `_drafting` স্টেটে।

`Future<void> _draftWithAi()`

`_InstructionsDialog` দিয়ে নির্দেশনা চায়, তারপর `draftAnnouncement` কল করে শিরোনাম ও বার্তা ফলিডে খসড়া বসায় — এই ফাংশন কীভাবে সবে করে না, শুধু ফলিডে টেক্সট বসিয়ে দেয়, যা সবে ও পাবলিশ না হওয়া পর্যন্ত ড্রাফট থেকে যায়। বর্ধিত হলে সন্যাকবার বার্তা দেখায়।

`bool get _isEdit`

এডিট মোডে আছে কিনা যাচাই করে।

`void dispose()`

`_title` ও `_body` কন্ট্রোলার ডিসপোজ করে।

`static String? _at(DateTime? date)`

একটি `DateTime`-কে `LocalDateTime` ফর্ম্যাটের স্ট্রিং (`yyyy-MM-ddT00:00:00`, মধ্যরাত, কোনো জোন ছাড়া) এ রূপান্তর করে; `date` নাল হলে `null` রিটার্ন করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে `AnnouncementRequest` তৈরি করে (`scheduledAt/expiresAt _at()` দিয়ে ফর্ম্যাট করা), এডিট/ক্রিয়েটে অনুযায়ী রপিকর্ষের ক্রিয়াকলাপ করে, প্রোভাইডার আপডেট করে, শিট বন্ধ করে সন্যাকবার দেখায়। "Cannot edit a published announcement" এর মতো ব্যাকএন্ড এরর এখানে ধরা পড়ে।

Widget build(BuildContext context)

শরিরোনাম, বার্তা (AI চ্যাট পারমশিন থাকলে সাফক্স আইকন হিসেবে "ড্রাফট" বাটন), দর্শক ড্রপডাউন, দুটি `DateField` (প্রকাশ তারিখ ও ময়োদ-উত্তীর্ণ তারিখ), এবং নোটফিকেশন-পাঠানোর সুইচ দিয়ে ফর্ম বন্ডি করো।

টপ-লেভেলে ফাংশন: `showRoleSheet`

Future<void> showRoleSheet(BuildContext context, {CustomRole? existing})

কাস্টম রোল তৈরি/এডিট করার `_RoleSheet` মডাল বটম শিট প্রদর্শন করে।

ক্লাস: `_RoleSheet`

ConsumerStatefulWidget — কাস্টম রোল ফর্মের শেলে।

`_RoleSheetState createState()`

`_RoleSheetState` তৈরি করে।

ক্লাস: `_RoleSheetState`

নাম ও বিবরণ নিয়ে কাস্টম রোল ফর্মের স্টেটে ধারণ করে।

bool get `_isEdit`

এডিট মোডে আছে কিনা যাচাই করে।

void dispose()

`_name` ও `_description` কন্ট্রোলার ডিসপোজ করে।

Future<void> `_submit()`

ফর্ম ভ্যালিডেট করে `CustomRoleRequest` তৈরি করে, এডিট/ক্রিয়েটে অনুযায়ী `updateRole/createRole` কল করে, `customRolesProvider` ইনভ্যালিডেট করে, শিট বন্ধ করে স্ন্যাকবার দেখায় (ক্রিয়েটে মোডে মনে করিয়ে দেয় যে পরবর্তীতে পারমশিন দিতে হবে)।

Widget build(BuildContext context)

নাম ও বিবরণ ফিল্ড দিয়ে ফর্ম বন্ডি করে; নতুন রোল তৈরির সময় একটি তিথ্যসূচক বার্তা দেখায় যে নতুন রোলে পারমশিন টকি না করা পর্যন্ত কিছুই দেয় না।

ক্লাস: `_InstructionsDialog`

StatefulWidget — অ্যানাউন্সমেন্ট ড্রাফট করার জন্য AI-কে দেওয়া নির্দেশনা লেখার ডায়ালগ। ফর্মের একটি ফিল্ড না বানিয়ে আলাদা ডায়ালগ হিসেবে রাখা হয়েছে, কারণ নির্দেশনাটি AI-এর ইনপুট, নোটসের অংশ নয়।

`_InstructionsDialogState createState()`

`_InstructionsDialogState` তৈরি করে।

ক্লাস: `_InstructionsDialogState`

`_instructions` টেক্সট কন্ট্রোলার নিয়ে ডায়ালগের স্টেটে ধারণ করে।

void dispose()

`_instructions` কন্ট্রোলার ডিসপোজ করে।

Widget build(BuildContext context)

AlertDialog — একটি বিহু-লাইন টেক্সটফিল্ড, "Cancel" (কিছু না ফরিয়ি বন্ধ করে) এবং "Draft" (টেক্সট ফরিয়ি বন্ধ করে) বাটন দেখায়।

ফাইল: lib/features/admin/admin_models.dart

সংক্ষিপ্ত ববিরণ: অ্যাডমিনি মডেলের পারমিশন কোড, কনস্ট্যান্ট তালিকা এবং সব রকি়োস্ট/রসেপন্স মডেলে ক্লাস (ডিপার্টমেন্ট, ডেজিগনেশন, অ্যানাউন্সমেন্ট, সার্ভিস ক্যাটাগরি, কাস্টম রোল, পারমিশন অপশন) একত্রে ধারণ করে। প্রতটি সারফেসে একই কাঠামো অনুসরণ করে — তালিকা, তৈরি, এডিট, একটিভি টগল — এবং নিজস্ব পারমিশনে গটেডে।

ক্লাস: AdminPermissions

abstract final class — অ্যাডমিনি স্ক্রিনিং সব পারমিশন কোড (ইউজার-ভিডি, ডিপার্টমেন্ট ভিডি/ক্রিয়েট/আপডেট, ডেজিগনেশন ভিডি/ক্রিয়েট/আপডেট, অ্যানাউন্সমেন্ট ভিডি/ক্রিয়েট/আপডেট, সার্ভিস ক্যাটাগরি ভিডি/ক্রিয়েট/আপডেট) স্ট্যাটিক কনস্ট্যান্ট হিসাবে ধারণ করে। কোনো মথড নেই।

টপ-লেভেলে কনস্ট্যান্ট: customRoleRoles

const List<String> — কারা কাস্টম রোল ম্যানজে করতে পারে (COMPANY_OWNER, SUPER_ADMIN, SYSTEM_ADMIN) তার তালিকা। রোল পারমিশন-কোড দিয়ে নয়, বরং সরাসরি রোল দিয়ে গটেডে — কারণ এই স্ক্রিনিংই পারমিশন কোডের অর্থ নির্ধারণ করে।

টপ-লেভেলে কনস্ট্যান্ট: customRoleAssignRoles

const List<String> — শুধুমাত্র COMPANY_OWNER একজন কর্মচারীকে রোলে নিয়োগ করতে পারে।

টপ-লেভেলে কনস্ট্যান্ট: announcementAudiences

const List<String> — অ্যানাউন্সমেন্টের সম্ভাব্য দর্শক তালিকা (ALL, DEPARTMENT, MANAGERS, CLIENTS), ব্যাকএন্ডের AnnouncementAudience enum মলোয়।

ক্লাস: DepartmentRequest

POST/PUT /departments-এর রকি়োস্ট বডি name আবশ্যক এবং সর্বদা পাঠানো হয়; বাকি ফিল্ডগুলো সার্ভার-সাইডে নাল-গার্ডে বলে অনুপস্থিতি ফিল্ড অপরিবর্তিত থাকে।

Map<String, dynamic> toJson()

নাম ট্রিম করে সবসময় পাঠায়; ভেতরের clean(String? value) হলেপার দিয়ে কোড ও ববিরণ ট্রিম-করে-খালি-হলে-বাদ দিয়ে; বাকি ফিল্ড (headEmployeeId, parentDepartmentId, budget) নাল না হলেই যুক্ত হয়।

ক্লাস: Designation

একটি জিব-গ্রুপে — নাম, কোড, লেভেলে (নিম্নমান বশে সিনিয়র, তালিকা এই অনুযায়ী সর্ট হয়), সক্রিয়তা, ঐচ্ছিক ববিরণ/এমপ্লয়মেন্ট-ক্যাটাগরি/ডিপার্টমেন্ট।

factory Designation.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে Designation তৈরিকরে, অনুপস্থিতি ফিল্ডে ডিফল্ট বসায় (level0, activetrue)।

ক্লাস: DesignationRequest

POST/PUT /hr/designations-এর রকি়োস্ট বডি। employmentCategory ও departmentId সবসময় পাঠানো হয় — কারণ ব্যাকএন্ডের DesignationServiceImpl.update দুটোই শর্তহীনভাবে বসায়, তাই বাদ দিলে সগেলো কলিয়ার হয়ে যাবে। code সার্ভার-সাইডে আপার-কসে রূপান্তরিত হওয়ার আগেই যাচাই হয় বলে এখানেও আপার-কসে করা হয়।

factory DesignationRequest.from(Designation designation)

একটি বিদ্যমান Designation অবজেক্ট থেকে সমতুল্য DesignationRequest তৈরিকরে (এডটি ফর্মের প্র-ফিলি করার জন্য ব্যবহৃত)।

Map<String, dynamic> toJson()

নাম ও কোড (আপার-কসে) সবসময় পাঠায়; level সবসময় পাঠায় (প্রমিটিভি int বলে); বিবরণ/active শর্তসাপেক্ষে; employmentCategory ও departmentId শর্তহীনভাবে (ক্লাস-কমেন্টে ব্যাখ্যা করা কারণে) পাঠায়।

ক্লাস: AdminAnnouncement

অ্যাডমিন তালিকায় দেখানো একটি কোম্পানি অ্যানাউন্সমেন্ট — হোম স্ক্রিনিং রডি-ওনলি Announcement-এর থেকে ভিন্ন, কারণ এটি খসড়া, দর্শক ও প্রকাশের সময়ও দেখাতে পারে।

bool get isEditable

published না হলে true রিটার্ন করে — প্রকাশিত অ্যানাউন্সমেন্ট এডটি করা যায় না ("Cannot edit a published announcement"), তাই অ্যাকশনটলিকানো থাকে বরং প্রত্যাখ্যাত হওয়ার বদলে।

factory AdminAnnouncement.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে তৈরিকরে; published ফিল্ডটি স্পষ্ট বুলিয়ান না থাকলে publishedAt নাল কনি তা দিয়ে অনুমান করে (দুটির যেকোনো একটি সিতে থাকলে প্রকাশিত ধরা হয়)।

ক্লাস: AnnouncementRequest

POST/PUT /announcements-এর রকি়োস্ট বডি। expiresAt, scheduledAt ও notifyAll সবসময় পাঠানো হয় — কারণ ব্যাকএন্ডের AnnouncementServiceImpl.update তিনটিই শর্তহীনভাবে বসায়, তাই একটি বাদ দিলে চুপসিারে মেয়াদ-উত্তীর্ণ তারিখ মুছে যায় বা শডিউল বাতলি হয়ে যায়।

factory AnnouncementRequest.from(AdminAnnouncement announcement)

একটি বিদ্যমান AdminAnnouncement থেকে সমতুল্য AnnouncementRequest তৈরিকরে (এডটি ফর্মের প্র-ফিলির জন্য)।

Map<String, dynamic> toJson()

শরিনোম ও বডি ট্রিমি করে সবসময় পাঠায়; দর্শক/টারগেট-ডপার্টমেন্ট/প্রায়োরিটি শর্তসাপেক্ষে; scheduledAt, expiresAt, notifyAll শর্তহীনভাবে (ক্লাস-কমেন্টে ব্যাখ্যা করা কারণে) পাঠায়।

ক্লাস: ServiceCategory

একটি সার্ভিস-ডেস্ক ক্যাটাগরি — নাম, সর্ট-অর্ডার, সক্রিয়তা, ঐচ্ছকি বিবরণ।

factory ServiceCategory.fromJson(Map<String, dynamic> json)

JSON থেকে ফিল্ড পার্স করে তৈরিকরে, active ডফিল্ট true।

ক্লাস: ServiceCategoryRequest

POST/PUT /service-categories-এর রকি়োস্ট বডি। sortOrder ব্যাকএন্ডে প্রমিটিভি int বলে সবসময় মান থাকে — অনুপস্থিতি পাঠালে 0 হিসেবে ধরা হয়ে ক্যাটাগরিটি চুপচাপ শীর্ষে চলে যতে, তাই সবসময় পাঠানো হয়।

Map<String, dynamic> toJson()

নাম ও sortOrder সবসময় পাঠায়; ববিরণ ট্রমি-করে-খাল-না-হলে যুক্ত করে।

ক্লাস: CustomRole

কোম্পানি নিজেরে জন্য য়ে রোল সংজ্ঞায়িত করেছে — নাম, সক্রিয়তা, systemRole ফ্ল্যাগ, ঐচ্ছকি ববিরণ।

factory CustomRole.fromJson(Map<String, dynamic> json)

JSON থেকে ফিল্ড পার্স করে তৈরিকরে; active ডফিল্ট true, systemRole ডফিল্ট false।

ক্লাস: CustomRoleRequest

POST/PUT /custom-roles-এর রকি়োস্ট বডি — নাম ও ঐচ্ছকি ববিরণ ধারণ করে।

Map<String, dynamic> toJson()

নাম ট্রমি করে সবসময় পাঠায়; ববিরণ ট্রমি-করে-খাল-না-হলে যুক্ত করে।

ক্লাস: PermissionOption

GET /permissions-এর রটিব্রন করা একটি অনুমতি অপশন — কোড, নাম, ঐচ্ছকি মডউল ও ববিরণ।

String get group

প্রদর্শনের জন্য গ্রুপিং-কী রটিব্রন করে — module থাকলে সেটে ট্রমি করে, না থাকলে 'Other'-এ সংগ্রহ করে (বাদ না দিয়ে)।

factory PermissionOption.fromJson(Map<String, dynamic> json)

JSON থেকে ফিল্ড পার্স করে তৈরিকরে; name না থাকলে code-কে ফলব্যাক হিসেবে ব্যবহার করে।

ফাইল: lib/features/admin/admin_repository.dart

সংক্ষিপ্ত ববিরণ: কোম্পানি-সিটেআপরে পাঁচটি তালিকা (departments, designations, announcements, service categories, custom roles) নিয়ে কাজ করার রপিজিটরি এবং সংশ্লিষ্ট Riverpod provider/controller-গুলো। প্রায় সব লস্টিই এক ছাঁচে গড়া — list, create, edit, আর delete-এর বদলে toggle। শুধু custom role-এর সাথে permission set আর employee assignment-এর অতিরিক্ত কাজ জড়িত।

ক্লাস: AdminRepository

ApiClient মুডিয়ে departments, designations, announcements, service categories আর custom roles-এর সব API কল সরবরাহ করে। কোনো তালিকা কখনো "delete" হয় না (custom role বাদে) — বরং টগল করে সক্রিয়/নস্ক্রিয় করা হয়, কারণ অন্য কোথাও সেই রকর্ডের রফোরেন্স থাকতে পারে।

`AdminRepository(this._api)`

কনস্ট্রাক্টর; `ApiClient` ইনস্ট্যান্স গ্রহণ করে অভ্যন্তরীণ ফিল্ড `_api`-তে সংরক্ষণ করে।

`Future<PagedResponse<Department>> departments({int page = 0, int size = 50})`

`/departments` এন্ডপয়েন্ট থেকে পজেনিটেডে ডিপার্টমেন্ট তালিকা আনে। `page` ও `size` দিয়ে পজেনিশন নিয়ন্ত্রণ হয় এবং `Department.fromJson` দিয়ে প্রতিটি আইটেম পার্স করা হয়।

`Future<Department> createDepartment(DepartmentRequest request)`

`POST /departments`-এ `request.toJson()` পাঠিয়ে নতুন ডিপার্টমেন্ট তৈরি করে এবং সার্ভারের রসপন্স থেকে `Department` অবজেক্ট রিটার্ন করে।

`Future<Department> updateDepartment(int id, DepartmentRequest request)`

`PUT /departments/$id`-এ রকিওয়েস্ট পাঠিয়ে বর্তমান ডিপার্টমেন্ট আপডেট করে এবং হালনাগাদ অবজেক্ট ফেরত দেয়।

`Future<Department> toggleDepartment(int id)`

`PATCH /departments/$id/toggle` কল করে ডিপার্টমেন্টকে সক্রিয়/নিষ্ক্রিয় অবস্থার মধ্যে টগল করে। ডলিটি নেই — কারণ কর্মী থাকা ডিপার্টমেন্ট হুট করে হারিয়ে যেতে পারে না, তাই ব্যাকএন্ড কবেল একটি ফ্ল্যাগ পাল্টায়।

`Future<List<Designation>> activeDesignations()`

`/hr/designations/active` থেকে শুধুমাত্র সক্রিয়ভাবে ব্যবহৃত (in-use) ডিজিগনেশনের তালিকা আনে — এটি এখনও কারো পদবিসিবে দেওয়া যায় এমন তালিকা, তৈরি হওয়া সব ডিজিগনেশন নয়। ফর্ম দেখনোর জন্য উপযুক্ত।

`Future<PagedResponse<Designation>> designations({int page = 0, int size = 50})`

`/hr/designations` থেকে পজেনিটেডে আকারে সব ডিজিগনেশন (সক্রিয়+নিষ্ক্রিয়) আনে, প্রশাসনিক তালিকার জন্য।

`Future<Designation> createDesignation(DesignationRequest request)`

`POST /hr/designations`-এ রকিওয়েস্ট পাঠিয়ে নতুন ডিজিগনেশন তৈরি করে।

`Future<Designation> updateDesignation(int id, DesignationRequest request)`

`PUT /hr/designations/$id`-এ পাঠিয়ে বর্তমান ডিজিগনেশন হালনাগাদ করে।

`Future<Designation> toggleDesignation(int id)`

`PATCH /hr/designations/$id/toggle` কল করে ডিজিগনেশনের সক্রিয়/নিষ্ক্রিয় অবস্থা পাল্টায়।

`Future<PagedResponse<AdminAnnouncement>> announcements({int page = 0, int size = 20})`

`/announcements` থেকে পজেনিটেডে ঘোষণা তালিকা আনে।

`Future<AdminAnnouncement> createAnnouncement(AnnouncementRequest request)`

`POST /announcements`-এ রকিওয়েস্ট পাঠিয়ে নতুন ঘোষণা (খসড়া) তৈরি করে।

`Future<AdminAnnouncement> updateAnnouncement(int id, AnnouncementRequest request)`

`PUT /announcements/$id`-এ পাঠিয়ে ঘোষণা হালনাগাদ করে। একবার প্রকাশিত হয়ে গেলে ব্যাকএন্ড এটি প্রত্যাখ্যান করে — দেখুন `AdminAnnouncement.isEditable`।

Future<({String title, String body})> draftAnnouncement(String instructions)

/announcements/ai-draft-এ নরিদশেনা পাঠিয়ে AI দিয়ে শরিনোনা ও বডি তিরৈকিয়া। কছুই সংরক্ষতি হয় না — শুধু খসড়া ফরৈত আসে, লখেক সম্পাদনা করে নজিে সংরক্ষণ করে। ঐটি ঐ AI_CHAT পারমশিনরে অধীনে গটেডে।

Future<AdminAnnouncement> publishAnnouncement(int id)

PATCH /announcements/\$id/publish কল করে ঘোষণা প্রকাশ করে। ঐটি ঐকমুখী কাজ — আনপাবলিশ করার কোনো উপায় নই, তাই U। আগে নশ্চিতি করে নেয়।

Future<PagedResponse<ServiceCategory>> serviceCategories({int page = 0, int size = 50})

/service-categories/all (ডিফল্ট লস্টিং নয়) থকে সব সার্ভিসি ক্যাটাগরি (সক্রিয়+নশ্ক্রিয়) আনে, যাত অ্যাডমনি স্ক্রিনি থকে নশ্ক্রিয় ক্যাটাগরি ফরিয়ে আনা যায়।

Future<ServiceCategory> createServiceCategory(ServiceCategoryRequest request)

POST /service-categories-এ পাঠিয়ে নতুন সার্ভিসি ক্যাটাগরি তিরৈকিয়া।

Future<ServiceCategory> updateServiceCategory(int id, ServiceCategoryRequest request)

PUT /service-categories/\$id-এ পাঠিয়ে বদিযমান ক্যাটাগরি হালনাগাদ করে।

Future<ServiceCategory> toggleServiceCategory(int id)

PATCH /service-categories/\$id/toggle কল করে ক্যাটাগরি সক্রিয়/নশ্ক্রিয় অবস্থা পালায়।

Future<List<CustomRole>> customRoles()

/custom-roles থকে সাধারণ (non-paged) List<CustomRoleResponse> আনে — কোম্পানরি কাছ মুষ্টমিয়ে রোল থাকে, পজে লাগে না।

Future<CustomRole> createRole(CustomRoleRequest request)

POST /custom-roles-এ পাঠিয়ে নতুন কাস্টম রোল তিরৈকিয়া।

Future<CustomRole> updateRole(int id, CustomRoleRequest request)

PUT /custom-roles/\$id-এ পাঠিয়ে বদিযমান রোল (যমেন নাম) হালনাগাদ করে।

Future<void> deleteRole(int id)

DELETE /custom-roles/\$id কল করে রোলটি স্থায়ীভাবে মুছে দেয়। ব্যাকএন্ড ঐটি প্রত্যাখ্যান করবে যদি ঐতে এখনও কটে থাকে।

Future<List<String>> rolePermissions(int id)

/custom-roles/\$id/permissions থকে ঐ রোলটি বর্তমান যসেব পারমশিন কোড দেয় তার তালিকা আনে।

Future<List<String>> setRolePermissions(int id, List<String> codes)

PUT /custom-roles/\$id/permissions-এ কোডরে ঐকটি খালি JSON অ্যারে (অবজেক্ট-মোডানো নয়) পাঠিয়ে রোলের সম্পূর্ণ পারমশিন সেটে প্রতস্থাপন করে — merge নয়, replace। ফলে caller-কে যা রাখতে চায় তার সবগুলো কোডই পাঠাতে হয়, শুধু নতুনগুলো নয়।

Future<void> assignRole(int roleId, int employeeId)

POST /custom-roles/\$roleId/employees/\$employeeId কল করে ঐকজন কর্মীকে ঐ রোলে বসায়। ঐটি শুধু কোম্পানি-owner-এর জন্য উপলব্ধ, বাকি স্ক্রিনিরে চয়ে সংকীরণ (platform admin ঐতে প্রবশে করতে পারে না)।

```
Future<void> unassignRole(int employeeId)
```

DELETE /custom-roles/employees/\$employeeId কল করে কর্মীকে তার বর্তমান রোল থেকে সরিয়ে দেয়। এটি রোলের নয়, কর্মীর ওপর ভিত্তি করে কাজ করে — যহেতু একজনকে বশে একটি কাস্টম রোল থাকতে পারে না।

টপ-লেভেলে প্রোভাইডার ও কন্ট্রোলার

```
final adminRepositoryProvider = Provider<AdminRepository>(...)
```

ApiClient থেকে AdminRepository-এর একটি ইনস্ট্যান্স তৈরি করে সরবরাহকারী Riverpod প্রোভাইডার।

```
final permissionCatalogueProvider = FutureProvider<List<PermissionOption>>(...)
```

/permissions এন্ডপয়েন্ট থেকে সিস্টেমে বর্তমান সব পারমিশন কোডের ক্যাটালগ একবার আনবে ও ধরে রাখবে (role editor-এর চেকলিস্টের জন্য)। ব্যাকএন্ডের enum দ্বারা নির্ধারিত বলে অ্যাপ চালু থাকা অবস্থায় এটি পাল্টায় না, তাই একবার আনলেই যথেষ্ট।

ক্লাস: DepartmentsAdminController extends AsyncNotifier<List<Department>>

ডিপার্টমেন্ট তালিকার প্রশাসনিক অবস্থা পরিচালনা করে — লোড, রিফ্রেশ, এবং একক সার্চ আপডেটের জন্য দায়ী।

```
Future<List<Department>> build()
```

currentUserProvider watch করে (ইউজার পাল্টালে পুনরায় লোড হওয়ার জন্য) এবং _load() কল করে প্রাথমিক অবস্থা তৈরি করে।

```
Future<List<Department>> _load()
```

adminRepositoryProvider-এর departments() কল করে পেজের content অংশ রিটার্ন করে।

```
Future<void> refresh()
```

AsyncValue.guard দিয়ে _load() পুনরায় চালায় এবং ফলাফল/এরর অনুযায়ী state আপডেট করে।

```
void apply(Department updated)
```

তালিকায় হালনাগাদ হওয়া রেকর্ডটি খুঁজে সেই সারটিকু বদলে দেয়, পুরো তালিকা রিলোড না করে — কারণ তালিকা বর্ণানুক্রমিক এবং এডিট করতে ক্রম বদলায় না।

```
final departmentsAdminProvider =
```

```
AsyncNotifierProvider<DepartmentsAdminController, List<Department>>(...)
```

উপরের কন্ট্রোলারকে অ্যাপে ব্যবহারযোগ্য করে তোলে।

ক্লাস: DesignationsAdminController extends AsyncNotifier<List<Designation>>

ডিজিগনেশন তালিকার প্রশাসনিক অবস্থা পরিচালনা করে; গঠন DepartmentsAdminController-এর অনুরূপ।

```
Future<List<Designation>> build()
```

currentUserProvider watch করে ও _load() কল করে প্রাথমিক তালিকা তৈরি করে।

```
Future<List<Designation>> _load()
```

adminRepositoryProvider-এর designations() কল করে পেজের content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard দিয়ে _load() পুনরায় চালিয়ে state হালনাগাদ করে।

void apply(Designation updated)

আইডি মিলিয়ে হালনাগাদ সারটি প্রতিস্থাপন করে।

final designationsAdminProvider =

AsyncNotifierProvider<DesignationsAdminController, List<Designation>>(...)

ডিজিগনেশন কন্ট্রোলার সরবরাহ করে।

ক্লাস: AnnouncementsAdminController extends

AsyncNotifier<List<AdminAnnouncement>>

ঘোষণা তালিকার প্রশাসনিক অবস্থা পরিচালনা করে।

Future<List<AdminAnnouncement>> build()

currentUserProvider watch করে ও _load() কল করে।

Future<List<AdminAnnouncement>> _load()

adminRepositoryProvider-এর announcements() কল করে content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard দিয়ে পুনরায় লোড করে state হালনাগাদ করে।

void apply(AdminAnnouncement updated)

আইডি মিলিয়ে হালনাগাদ ঘোষণাটি তালিকায় প্রতিস্থাপন করে।

final announcementsAdminProvider =

AsyncNotifierProvider<AnnouncementsAdminController,

List<AdminAnnouncement>>(...)

ঘোষণা কন্ট্রোলার সরবরাহ করে।

ক্লাস: ServiceCategoriesAdminController extends

AsyncNotifier<List<ServiceCategory>>

সার্ভিস ক্যাটাগরি তালিকার প্রশাসনিক অবস্থা পরিচালনা করে।

Future<List<ServiceCategory>> build()

currentUserProvider watch করে ও _load() কল করে।

Future<List<ServiceCategory>> _load()

adminRepositoryProvider-এর serviceCategories() কল করে content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard দিয়ে পুনরায় লোড করে।

void apply(ServiceCategory updated)

আইডি মিলিয়ে হালনাগাদ সারটি প্রতিস্থাপন করে।

```
final serviceCategoriesAdminProvider =
AsyncNotifierProvider<ServiceCategoriesAdminController,
List<ServiceCategory>>(...)
```

সার্ভিস ক্যাটাগরি কন্ট্রোলার সরবরাহ করে।

অন্যান্য top-level providers

```
final customRolesProvider = FutureProvider<List<CustomRole>>(...)
```

`currentUserProvider` watch করে এবং `adminRepositoryProvider.customRoles()` কল করে কাস্টম রোলের তালিকা আনে।

```
final rolePermissionsProvider = FutureProvider.autoDispose.family<List<String>, int>(...)
```

নর্দিস্ট `roleId`-এর জন্য `rolePermissions(roleId)` কল করে সেই রোলের বর্তমান পারমিশন কোডগুলো আনে; ব্যবহার শেষে অটো-ডিসপোজড হয়।

```
final activeDesignationsProvider = FutureProvider.autoDispose<List<Designation>>(...)
```

`activeDesignations()` কল করে ফর্মের উপযোগী (in-use) ডিজিগনেশনের তালিকা আনে।

ফাইল: lib/features/admin/admin_screen.dart

সংক্ষিপ্ত বিবরণ: "Setup" নামের কোম্পানি-কনফিগারেশন স্ক্রিন — Departments, Grades, Notices, Categories ও Roles ট্যাব নিয়ে গঠিত, যেখানে প্রতিটি ট্যাব নর্দিস্ট পারমিশনের ভিত্তিতে দেখা যায় বা লুকানো থাকে। কোনো পারমিশন না থাকলে পুরো স্ক্রিনটাই লক আইকনসহ "Not available to you" দেখায়।

ক্লাস/উইজেট: AdminScreen

কোম্পানি-সিটেআপের মূল স্ক্রিন। ইউজারের পারমিশন অনুযায়ী গতিশীলভাবে ট্যাব তৈরি করে, একটি নর্দিস্ট ট্যাবে শুরু করার সুবিধা দেয়, এবং প্রতিটি স্ক্রিন ট্যাবের জন্য `FloatingActionButton` দিয়ে "New" তৈরি অ্যাকশন যুক্ত করে।

```
const AdminScreen({super.key, this.initialTabLabel})
```

কনস্ট্রাক্টর; ঐচ্ছক `initialTabLabel` দিয়ে নর্দিস্ট ট্যাবে খোলা যায় (যেমন "Announcements" নভিগেশন থেকে সরাসরি Notices ট্যাব)।

`Widget build(BuildContext context, WidgetRef ref)`

বর্তমান ইউজারের পারমিশন (`permissionControllerProvider`) ও ইউজার (`currentUserProvider`) watch করে। পারমিশন এখনো লোড না হলে `Loader` দেখায়। `canRoles` চেক করে ইউজার `customRoleRoles`-এর কোনো একটির অধিকারী কিনা। এরপর প্রতিটি সাব-স্ক্রিনের ভিউ-পারমিশন অনুযায়ী `tabs` তালিকা (label, view, create) তৈরি করে — Departments, Grades, Notices, Categories, এবং role-ভিত্তিক Roles ট্যাব। কোনো ট্যাব না থাকলে "Not available to you" `EmptyState` দেখায়। অন্যথায় `DefaultTabController` দিয়ে `TabBar+TabBarView` সহ `Scaffold` রটিন করে, যেখানে বর্তমান ট্যাবের `create` কলব্যাক থাকলে `FloatingActionButton` দেখায় এবং `userView` পারমিশন থাকলে `AppBar`-এ "Who can sign in" আইকন (`AccountsScreen` খোলার জন্য) যুক্ত থাকে।

ক্লাস/উইজটে: _DepartmentsTab ও _DepartmentsTabState

Departments ট্যাবের কনটেন্ট — ConfigList-এর মাধ্যমে ডিপার্টমেন্ট তালিকা দেখায়, প্রতিটি সারিতে এডটি ও টগল (active/inactive) অ্যাকশন থাকে।

Future<void> _toggle(Department row)

নর্দিস্ট row-এর আইডি _busyId-তে সেটে করে বাটন disable দেখায়, adminRepositoryProvider.toggleDepartment কল করে ফলাফল departmentsAdminProvider.notifier.apply(...) দিয়ে তালিকায় প্রতিফলিত করে। ApiException হলে তার message সন্ধ্যাকবারে দেখায়, অন্য এরর হলে জনৈকি বার্তা দেখায়, শেষে _busyId রসিটে করে।

Widget build(BuildContext context)

departmentUpdate পারমিশন চেক করে canUpdate নির্ধারণ করে, departmentsAdminProvider watch করে ConfigList<Department> রেন্ডার করে — প্রতিটি সারিতে নাম, কোড, কর্মী সংখ্যা (থাকলে), busy অবস্থা এবং edit/toggle কলব্যাক (পারমিশন থাকলে) দেখায়।

ক্লাস: DesignationScopeController extends Notifier<bool>

Grades তালিকা শুধু "in-use" ডিজিগনেশন দেখাচ্ছে কনি তার বুলিয়ান অবস্থা রাখে।

bool build()

প্রাথমিক অবস্থা false (সব ডিজিগনেশন দেখানো) রিটার্ন করে।

void set(bool inUseOnly)

state-কে দেওয়া মান দিয়ে সেটে করে।

final designationScopeProvider = NotifierProvider<DesignationScopeController, bool>(...)

উপরের কন্ট্রোলার সরবরাহকারী প্রোভাইডার।

ক্লাস/উইজটে: _DesignationsTab ও _DesignationsTabState

Grades ট্যাবের কনটেন্ট — সব বা শুধু "in-use" ডিজিগনেশন ফিল্টার করে দেখানোর সুবিধাসহ ConfigList রেন্ডার করে।

Future<void> _toggle(Designation row)

_DepartmentsTabState._toggle-এর অনুরূপ প্যাটার্নে toggleDesignation কল করে এবং designationsAdminProvider.notifier.apply(...) দিয়ে তালিকা হালনাগাদ করে; এরর হ্যান্ডলিং ও _busyId ব্যবস্থাপনা অভিন্ন।

Widget build(BuildContext context)

designationUpdate পারমিশন থেকে canUpdate নির্ধারণ করে, designationScopeProvider থেকে inUseOnly পড়ে — সত্য হলে activeDesignationsProvider, নাহলে designationsAdminProvider watch করে ভিন্ন উৎস থেকে ডটো দেখায় (কারণ "in use" একটি পৃথক এন্ডপয়েন্ট, ফিল্টার নয়)। onRefresh-এ উভয় প্রোভাইডার রিফ্রেশ/ইনভ্যালিডেট করে। শীর্ষে FilterBar (All grades / In use) হডার হিসেবে যুক্ত করে এবং প্রতিটি সারিতে নাম, কোড, বিভাগের নাম, লভেলে ও edit/toggle কলব্যাক দেখায়।

ক্লাস/উইজটে: _CategoriesTab ও _CategoriesTabState

Categories ট্যাবের কনটেন্ট — সার্ভিস ক্যাটাগরি তালিকা দেখায়।

Future<void> _toggle(ServiceCategory row)

একই প্যাটার্নে toggleServiceCategory কল করে serviceCategoriesAdminProvider.notifier.apply(...) দিয়ে হালনাগাদ করে; এরর হ্যান্ডলিং অভিন্ন।

Widget build(BuildContext context)

serviceCategoryUpdate পারমশিন থেকে canUpdate নরিধারণ করে, serviceCategoriesAdminProvider watch করে প্রতটি সারিতে নাম, বর্ণনা, sortOrder ও edit/toggle কলব্যাকসহ ConfigList<ServiceCategory> রন্ডার করে।

ক্লাস/উইজটে: _AnnouncementsTab ও _AnnouncementsTabState

Notices ট্যাবের কন্টেন্ট — ঘোষণার কার্ড আকারে তালিকা, প্রতটির অবস্থা (Draft/Published), audience, সময়সূচী এবং প্রকাশযোগ্য হলে Edit/Publish বাটন দেখায়।

Future<void> _publish(AdminAnnouncement row)

প্রথমে একটি নিশ্চিতিকরণ ডায়ালগ দেখায় (কারণ প্রকাশ একমুখী ও প্রকাশের পর সম্পাদনাযোগ্য নয়)। নিশ্চিতি হলে _busyId সটে করে publishAnnouncement(row.id) কল করে, ফলাফল announcementsAdminProvider.notifier.apply(...) দিয়ে তালিকায় প্রতফিলতি করে এবং সাফল্যের সন্ধ্যাকবার দেখায়। ব্যর্থ হলে যথার্থ এরর বার্তা দেখায়, শেষে _busyId রসিটে করে।

Widget build(BuildContext context)

announcementUpdate পারমশিন থেকে canUpdate নরিধারণ করে, announcementsAdminProvider watch করে প্রতটি ঘোষণাকে AppCard-এ রন্ডার করে — শরিনোম, স্ট্যাটাসচপি (Draft/Published), সংক্ষিপ্ত বডি, audience/scheduledAt তথ্য, এবং সম্পাদনাযোগ্য (canUpdate && row.isEditable) হলে Edit ও Publish বাটন দেখায় (অথবা busy হলে Loader)।

Future<void> _deleteRole(BuildContext context, WidgetRef ref, CustomRole role)

কাস্টম রোল মুছে ফেলার টপ-লভেলে ফাংশন। প্রথমে নিশ্চিতিকরণ ডায়ালগ দেখায় (কারণ রোলে থাকা কাউকে আগে সরাতে হবে), নিশ্চিতি হলে deleteRole(role.id) কল করে, customRolesProvider ইনভ্যালডিটে করে তালিকা রফির্শে করে, এবং সাফল্য বা এরর বার্তা সন্ধ্যাকবারে দেখায়।

ক্লাস/উইজটে: _RolesTab

Roles ট্যাবের কন্টেন্ট — কাস্টম ও সিস্টমে রোলের তালিকা RefreshIndicator মোড়ানো অবস্থায় দেখায়।

Widget build(BuildContext context, WidgetRef ref)

customRolesProvider watch করে; লোডিং/এরর/ফাঁকা/ডটো অবস্থার জন্য যথাক্রমে Loader, ErrorState, EmptyState, এবং ListView.builder রন্ডার করে। প্রতটির রোল একটি AppCard-এ দেখানো হয় — সিস্টমে রোলের জন্য lock আইকন ও "Built in" লবেলে (কোনো এডটির নই, কারণ এর অনুমতগুলো ব্যাকএন্ড দ্বারা নরিধারতি), কাস্টম রোলের জন্য শীল্ড আইকন, ট্যাপে RolePermissionsScreen.open, এবং একটি PopupMenuButton যা Permissions/Rename/Delete বকিল্প দেয়।

ফাইল: lib/features/admin/role_permissions_screen.dart

সংক্ষিপ্ত ববিরণ: একটি নির্দিষ্ট কাস্টম রোলরে সব পারমিশন কোড গ্রুপ করে চেকবক্স আকারে দেখানো ও সংরক্ষণ করার স্ক্রিনি, পাশাপাশি এই রোলে কর্মী নিয়োগ/অপসারণের সুবিধা।

ক্লাস/উইজটে: RolePermissionsScreen

নির্দিষ্ট `role`-এর জন্য পারমিশন এডটির দেখায়। পুরো নির্বাচতি স্টে সংরক্ষণের সময় পাঠানো হয় (diff নয়) — তাই সম্ভে বাটন এক্সপ্লসিটি, প্রত্যেকটি অটো-সম্ভে নয়।

```
const RolePermissionsScreen({super.key, required this.role})
```

কনস্ট্রাক্টর; যে `CustomRole` সম্পাদনা করা হবে তা গ্রহণ করে।

```
static void open(BuildContext context, {required CustomRole role})
```

`MaterialPageRoute` দিয়ে `RolePermissionsScreen`-এ নভেগিটে করে।

```
ConsumerState<RolePermissionsScreen> createState()
```

`_RolePermissionsScreenState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: _RolePermissionsScreenState

স্ক্রিনির অভ্যন্তরীণ অবস্থা পরিচালনা করে — নির্বাচতি কোড স্টে, সংরক্ষণের `busy` অবস্থা এবং এরর বার্তা।

```
Future<void> _assign()
```

`EmployeePicker.show` দিয়ে একজন কর্মী বেছে নতি দেয়; বাছাই করা হলে `_saving` স্টে করে `assignRole(widget.role.id, person.id)` কল করে এবং সাফল্য/এরর সন্ধ্যাকবার দেখায়। এটি শিখু কোম্পানি-owner-এর জন্য উপলব্ধ। একজন কর্মীর একটিই কাস্টম রোল থাকতে পারে, তাই এটি তাকে আগের রোল থেকে সরিয়ে নতুনটিতে বসায়।

```
Future<void> _unassign()
```

`EmployeePicker.show` দিয়ে কর্মী বেছে `unassignRole(person.id)` কল করে তাকে তার বর্তমান রোল (যেটাই হোক) থেকে সরিয়ে দেয় — এন্ডপয়েন্ট কর্মী-কন্ট্রোল বলে রোল আইডি লাগে না। সাফল্য/এরর সন্ধ্যাকবার দেখায়।

```
Future<void> _save()
```

`_selected` null বা ইতিমধ্যে `_saving` চললে কিছু করে না। অন্যথায় `_saving=true` স্টে করে `setRolePermissions(widget.role.id, selected.toList())` কল করে, `rolePermissionsProvider(widget.role.id)` ইনভ্যালিডেটে করে (যাতে স্ক্রিনি সদ্য-সংরক্ষতি স্টে পুনরায় পড়ে), এবং সাফল্য সন্ধ্যাকবার বা `_error`-এ এরর বার্তা স্টে করে।

```
Widget build(BuildContext context)
```

`permissionCatalogueProvider` (সব পারমিশন কোড) এবং `rolePermissionsProvider(widget.role.id)` (রোলরে বর্তমান স্টে) watch করে। `_selected` null থাকলে ও `current` লোড হলে এটিকে সার্বভারের উত্তর থেকে একবার সডি করে (পুনরায় পড়া হয় না, যাতো অসংরক্ষতি টকি হারিয়ে না যায়)। `AppBar`-এ owner হলো "People" পপআপ মেনু (Assign/Unassign) এবং নির্বাচতি সংখ্যা দেখায়। বডতি ক্যাটালগ লোডিং/এরর/ডটো অবস্থা হ্যান্ডল করে; ডটো এলে পারমিশনগুলোকে `group` অনুযায়ী `ExpansionTile`-এ ভাগ করে, প্রত্যেকটি ভেতরে `CheckboxListTile` দিয়ে টকি/আনটকি করা যায় (সেরাসরি `local selected` স্টে পরিবর্তিত হয়), এবং নচি "Save permissions" `LoadingButton` থাকে।

ফাইল: lib/features/ai/ai_admin_models.dart

সংক্ষিপ্ত ববিরণ: AI সহকারী কনফিগার করার (কোন প্রোভাইডার উত্তর দবে, কী বলতে বলা হয়েছে, এবং এখন পর্যন্ত খরচ কত) মডলে ও enum-সদৃশ ক্লাসগুলো। সবকিছু **AI_ADMIN** পারমিশনের অধীনে, যা **AI_CHAT**-এর চেয়ে সংকীর্ণ — সহকারীর সাথে কথা বলা মানুষ সাধারণত এটুকিনফিগার করতে পারে না।

ক্লাস: AiFeature

abstract final class — ব্যাকএন্ড যসেব ফিচারের জন্য কনটেন্ট জনোরটে করতে পারে তার সম্পূর্ণ তালিকা (স্ট্যাটিক কনস্ট্যান্ট স্ট্রিং)। এতে **all** (সম্পূর্ণ enum) ও **draftable** (হাতে-লখো প্রম্পট দিয়ে চাওয়ার যোগ্য উপসেট) — দুই ধরনের তালিকা আছে।

কোনো ইনস্ট্যান্স মথড নেই — শুধু স্ট্যাটিক কনস্ট্যান্ট ফিল্ড (**employmentLetter**, **leavePolicy**, **performanceReview**, **crmLeadSummary**, **crmActivitySummary**, **invoiceSummary**, **serviceRequestSummary**, **announcementDraft**, **holidayDraft**, **workflowSuggestion**, **searchAnswer**, **businessInsights**, **general**, **timesheetEntry**, **expenseEntry**, **dailyBriefing**, **agentTask**) এবং দুটি তালিকা **all**, **draftable**।

ক্লাস: AiProviderType

abstract final class — সমর্থিত AI প্রোভাইডারের কোড (**gemini**, **claude**, **openai**, **groq**, **mock**) এবং তাদের লবেল/মডলে ম্যাপিং।

static String label(String provider)

প্রোভাইডার কোডকে মানুষ-পঠনযোগ্য নামে রূপান্তর করে (যেমন **claude** "Anthropic Claude", **mock** "Mock (no provider)")। অজানা কোড হলে স্টেই ফরেনত দিয়ে।

static const models = <String, List<String>>{...}

কোন প্রোভাইডারের সাথে কোন মডেলগুলো যুক্ত তার ম্যাপিং। এটি অ্যাপের নিজস্ব দায়িত্ব — সারভারের **AiModel** enum-এ প্রোভাইডারের সাথে কোনো বাঁধন নেই বলে ভুল জোড়া (যেমন **CLAUDE** + **GPT** মডলে) সংরক্ষণের সময় গৃহীত হলেও জনোরশনের সময় ব্যর্থ হবো।

static List<String> modelsFor(String provider)

models ম্যাপ থেকে দেওয়া প্রোভাইডারের মডলে তালিকা ফরেনত দিয়ে; না পলে ডিফল্ট [**'GEMINI_2_5_FLASH'**] রটার্ন করে।

ক্লাস: AiProviderConfig

একটি সংরক্ষিত AI প্রোভাইডার কনফিগারেশন। প্রত্যেকোম্পানতি প্রতপ্রোভাইডার-টাইপের জন্য সর্বোচ্চ একটি, এবং একই সময়ে সর্বোচ্চ একটি সক্রিয় থাকতে পারে।

const AiProviderConfig({required this.id, ...})

কনস্ট্রাক্টর; **id**, **provider**, **model**, **active**, **maxTokens** আবশ্যিক, **temperature** ও **createdAt** ঐচ্ছিক।

factory AiProviderConfig.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে নিরাপদভাবে (নাল-সেফ ডিফল্টসহ) **AiProviderConfig** তৈরি করে।

ক্লাস: AiProviderConfigRequest

প্রোভাইডার সংরক্ষণের রকি়োস্ট বডি প্রোভাইডার-টাইপ অনুযায়ী upsert হয় (id দি়ে নয়) — দুবার GEMINI পাঠালে একই সারি বিদলে যায়, নতুন তৈরি হয় না। সংরক্ষণ করলে সেই প্রোভাইডারই সক্রিয় হয়ে যায়, তাই এটি কখনো নীরব সম্পাদনা নয়। **apiKey** write-only — এটি এনক্রিপ্ট হয়ে ঢোকে, কখনো ফেরত আসে না; খালি রাখলে বদ্যমান কী অক্ষুণ্ণ থাকে।

```
const AiProviderConfigRequest({required this.provider, required this.model, ...})
```

কনস্ট্রাক্টর; **provider**, **model** আবশ্যিক, **apiKey**, **temperature**, **maxTokens** ঐচ্ছিক (null মান বদ্যমান মান অপরিবর্তিত থাকবে)।

```
Map<String, dynamic> toJson()
```

রকি়োস্ট বডি তৈরি করে — **apiKey** শুধু তখনই যোগ করে যখন এটি null নয় এবং trim করার পর খালি নয়; **temperature**/**maxTokens** শুধু null না হলে যুক্ত হয়।

ক্লাস: AiPromptTemplate

একটি নির্দিষ্ট ফিচারের জন্য AI-কে দেওয়া প্রম্পট। একটি ফিচারের জন্য নতুন টেমপ্লেটে সংরক্ষণ করলে পুরনো টেমপ্লেটের আইডি হয় না, বরং **version** বড়ে যায় এবং পুরনো টেমপ্লেট রেকর্ডে থেকে যায়।

```
const AiPromptTemplate({required this.id, ...})
```

কনস্ট্রাক্টর; **id**, **feature**, **name**, **template**, **version**, **active** আবশ্যিক; **changeNotes**, **updatedAt** ঐচ্ছিক।

```
factory AiPromptTemplate.fromJson(Map<String, dynamic> json)
```

JSON থেকে নাল-সফেভাবে **AiPromptTemplate** তৈরি করে (**version** ডিফল্ট ১)।

ক্লাস: AiPromptTemplateRequest

টেমপ্লেটে সংরক্ষণের রকি়োস্ট বডি

```
const AiPromptTemplateRequest({required this.feature, required this.name, required this.template, this.changeNotes})
```

কনস্ট্রাক্টর; **name** সর্বোচ্চ ১০০ অক্ষর (এর বেশি হলে ব্যাকএন্ড প্রত্যাখ্যান করে), **changeNotes** সর্বোচ্চ ৫০০ অক্ষর (কেনে এই ভারসন তৈরি হলে তার একমাত্র রেকর্ড)।

```
Map<String, dynamic> toJson()
```

feature, **name**, **template** সহ রকি়োস্ট বডি তৈরি করে; **changeNotes** শুধু non-null এবং trim-এর পর অখালি হলে যুক্ত হয়।

ক্লাস: AiUsageSummary

একটি নির্দিষ্ট দিনে সহকারী কী করেছে তার সারাংশ।

```
const AiUsageSummary({required this.totalRequests, ...})
```

কনস্ট্রাক্টর; **totalRequests**, **totalTokens**, **avgResponseTimeMs**, **requestsByFeature**, **tokensByFeature** আবশ্যিক, **date** ঐচ্ছিক।

```
static Map<String, int> _counts(Object? value)
```

একটি dynamic JSON ভ্যালুক নিরীপদে **Map<String, int>**-এ রূপান্তর করে — শুধু **String** কী ও **num** মান সহ এন্ট্রিগুলো রাখে; **value** Map না হলে খালি ম্যাপ ফেরত দেয়।

factory AiUsageSummary.fromJson(Map<String, dynamic> json)
JSON থেকে `_counts` হলেপার ব্যবহার করে `AiUsageSummary` তৈরি করে।

ক্লাস: AiExchange

সহকারীর একটি উত্তর, `AiGenerateResponse`-এর মরির। এতে কোনো `id`, `timestamp` বা প্রম্পটের কপি নেই — কথোপকথনের ইতিহাসও একই DTO ফেরত দেয়, তাই অতীতের বনিমিষ পড়া যায় কিন্তু তারখি জানা যায় না, আর যে প্রশ্নে এই উত্তর এসেছিল তা রসেপন্সে নেই। শুধু `conversationUuid` একে অন্যদরে থেকে আলাদা করে।

const AiExchange({required this.result, ...})

কনস্ট্রাক্টর; `result` আবশ্যিক; `conversationUuid`, `feature`, `provider`, `model`, `threadId` ঐচ্ছক; `executionTimeMs` ডফিল্ট 0; `awaitingConfirmation` ডফিল্ট false।

factory AiExchange.fromJson(Map<String, dynamic> json)

JSON থেকে নাল-সফেভাবে `AiExchange` তৈরি করে। `awaitingConfirmation` সত্য হলে বোঝায় এই উত্তরটি একটি প্রস্তাবতি অ্যাকশন যা নিশ্চিতকরণে অপেক্ষায় আছে — শুধু agent loop এটি স্টে করে, এক-দফা খসড়া কখনো করে না।

ফাইল: lib/features/ai/ai_chat_screen.dart

সংক্ষিপ্ত ববিরণ: একটি বিদ্যমান AI থ্রেডে চ্যাট-স্টাইল কথোপকথনের স্ক্রিনি। থ্রেডে তৈরি করা caller-এর দায়িত্ব বলে এই স্ক্রনিকে কখনো "খালি অবস্থা যা আসলে লোডিং" রেন্ডার করতে হয় না।

ক্লাস/উইজটে: AiChatScreen

প্রদত্ত `AiThread`-এর সাথে চ্যাট UI দেখায়।

const AiChatScreen({super.key, required this.thread})

কনস্ট্রাক্টর; যে থ্রেডে চ্যাট চলবে তা গ্রহণ করে।

static Future<void> open(BuildContext context, {required AiThread thread})

`MaterialPageRoute` দিয়ে `AiChatScreen`-এ নভেগিটে করে।

ConsumerState<AiChatScreen> createState()

`_AiChatScreenState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: _AiChatScreenState

থ্রেডেরে বার্তা তালিকা, ইনপুট নিয়ন্ত্রণ এবং স্ক্রলিং পরিচালনা করে।

void initState()

`super.initState()` কল করে `_load()` চালু করে।

void dispose()

`_input` ও `_scroll` কন্ট্রোলার ডিসপোজ করে, তারপর `super.dispose()` কল করে।

Future<void> _load()

aiRepositoryProvider.threadMessages(widget.thread.id) কল করে থ্রেডেরে ইতিহাস আনবে, _messages লিস্ট পূরণ করে, _loading=false সটে করে এবং _scrollToEnd() কল করে। এরর হলে _error সটে করে।

void _scrollToEnd()

WidgetsBinding.instance.addPostFrameCallback ব্যবহার করে — ফ্রমে আঁকা শেষ হওয়ার পর (যাতে নতুন আইটেমসহ তালিকা মাপা যায়) — স্ক্রল কন্ট্রোলারকে সর্বোচ্চ স্ক্রল extent পর্যন্ত অ্যানিমিটে করে নিয়ে যায়।

Future<void> _send()

_input-এর টেক্সট trim করে, খালি হলে বা ইতিমধ্যে _sending হলে কিছু করে না। অন্যথায় local _messages-এ ব্যবহারকারীর বার্তা (AiMessage.user(text)) যোগ করে, ইনপুট খালি করে, aiRepositoryProvider.agentTurn(threadId:..., message:...) কল করে, সহকারীর উত্তর _messages-এ যোগ করে এবং স্ক্রল করে নচি নামায়। এরর হলে _error সটে করে, শেষে _sending=false করে।

Widget build(BuildContext context)

_error থাকলে ব্যানার দেখায়। মূল বডিতে লোডিং/খালি/বার্তা-তালিকা অবস্থা হ্যান্ডল করে — খালি অবস্থায় agent থ্রেডে কনি অনুযায়ী ভিন্ন বার্তা দেখায় ("Ask it to do something" বনাম "Ask it something")। _sending সত্য হলে "Thinking..." ইন্ডিকেটর দেখায়। নচি _Composer উইজেটে দিয়ে ইনপুট বার রেন্ডার করে।

ক্লাস/উইজেট: _Bubble

একটি একক চ্যাট বাবল রেন্ডার করে।

const _Bubble({required this.message})

কনস্ট্রাক্টর; যে AiMessage দেখানো হবে তা গ্রহণ করে।

Widget build(BuildContext context)

message.fromUser অনুযায়ী বাবলটি ডানে (ইউজার) বা বামে (সহকারী) সারবিদ্ধ করে, ভিন্ন ব্যাকগ্রাউন্ড রঙ ব্যবহার করে। message.awaitingConfirmation সত্য হলে বর্ডার হলুদ/warning রঙে দেখায় এবং "Nothing has changed yet — reply to confirm." সতর্কবার্তা যোগ করে — কারণ এখনো কিছু ঘটেনি এবং পরবর্তী রপ্তাই সন্ধানিত হবে।

ক্লাস/উইজেট: _Composer

বার্তা টাইপ ও পাঠানোর জন্য নচি ইনপুট বার।

const _Composer({required this.controller, required this.enabled, required this.onSend})

কনস্ট্রাক্টর; টেক্সট কন্ট্রোলার, সক্রিয়/নিষ্ক্রিয় ফ্ল্যাগ এবং সেন্ড কলব্যাক গ্রহণ করে।

Widget build(BuildContext context)

একটি TextField (মাল্টিলাইন, enabled অনুযায়ী সক্রিয়/নিষ্ক্রিয়, Enter চাপলে onSend কল করে) এবং পাশে একটি সেন্ড IconButton দেখায়, যা কীবোর্ড ইনস্টেরে জন্য padding সমন্বয় করে।

ফাইল: lib/features/ai/ai_drafts_screen.dart

সংক্ষিপ্ত ববিরণ: সহকারীর কাছে এক-দফা টেক্সট খসড়া চাওয়ার এবং আগের কী জিজ্ঞাসা করা হয়েছে তা দেখার স্ক্রিনি। চ্যাটের থেকে আলাদা — এখানে কিছুই মনে রাখা হয় না, প্রতিটি উত্তর একটি নির্দিষ্ট ফিচারের বর্ণনায় একক প্রচেষ্টা।

ক্লাস: AiHistoryFilterController extends Notifier<String?>

ইতিহাস তালিকা কোন ফিচারে সীমাবদ্ধ তা রাখতে; null মান সব।

String? build()

প্রাথমিক অবস্থা null রিটার্ন করে।

void set(String? feature)

state-কে দেওয়া feature-এ সেট করে।

final aiHistoryFilterProvider = NotifierProvider<AiHistoryFilterController, String?>(...)

উপরের কন্ট্রোলার সরবরাহ করে।

final aiHistoryProvider = FutureProvider.autoDispose<List<AiExchange>>(...)

aiHistoryFilterProvider-এর বর্তমান ফিচার watch করে, aiRepositoryProvider.conversations(feature:..., size: 30) কল করে পেজের content রিটার্ন করে।

ক্লাস/উইজটে: AiDraftsScreen

"Drafting" স্ক্রিনির রুট — দুটি ট্যাব: Draft (নতুন খসড়া চাওয়া) ও Earlier (ইতিহাস)।

const AiDraftsScreen({super.key})

কনস্ট্রাক্টর; কোনো প্রারামিটার নেই।

Widget build(BuildContext context)

DefaultTabController(length: 2) দিয়ে TabBar (Draft/Earlier) সহ Scaffold তৈরি করে এবং TabBarView-এ _DraftTab() ও _HistoryTab() রাখে।

ক্লাস/উইজটে: _DraftTab ও _DraftTabState

খসড়া তৈরি ফর্ম — ফিচার বাছাই, প্রম্পট লেখা, এবং ফলাফল দেখানো।

void dispose()

_prompt কন্ট্রোলার ডিসপোজ করে।

Future<void> _draft()

_prompt-এর টেক্সট trim করে খালি হলে থামে। অন্যথায় _busy=true সেট করে

aiRepositoryProvider.generate(feature: _feature, prompt: prompt) কল করে, ফলাফল _result-এ রাখে, এবং aiHistoryProvider ইনভ্যালিডেট করে কারণ উত্তরটি সার্ভার-সাইডে রেকর্ড হয়েছে, তাই ইতিহাস ট্যাব একটি সারি পিছিয়ে আছে। এরর হলে _error সেট করে, শেষে _busy=false করে।

Widget build(BuildContext context)

`_error` থাকলে ব্যানার দেখায়। একটি `AppCard`-এ ফচার বাছাইয়ের `DropDownButtonFormField` (`AiFeature.draftable` থেকে অপশন), প্রম্পট `TextField`, এবং "Draft it" `LoadingButton` দেখায়। `_result` থাকলে নিচি ফলাফল টেক্সট, কপি বাটন (`Clipboard.setData`) এবং সতর্কবার্তা ("Read it before you use it...") দেখায়।

ক্লাস/উইজটে: `_HistoryTab`

পূর্ববরে খসড়ার ইতিহাস, ফচার অনুযায়ী ফিল্টারযোগ্য।

Widget build(BuildContext context, WidgetRef ref)

`aiHistoryFilterProvider`-এর বর্তমান ফচার পড়ে, `aiHistoryProvider` watch করে `ConfigList<AiExchange>` রেন্ডার করে — শীর্ষে `FilterBar` (Everything + প্রতটি draftable ফচার) যা `aiHistoryFilterProvider.notifier.set` কল করে। প্রতটি এন্ট্রি একটি `AppCard`-এ ফচারের নাম, মডলে (থাকলে), ফলাফলের সংক্ষিপ্তসার (৬ লাইন পর্যন্ত), এবং `executionTimeMs > 0` হলে "answered in Xs" দেখায় — কারণ রসেপন্সে টাইমস্ট্যাম্প বা প্রম্পটের কপি নাই, তাই সময়টুকুই দেখানোর মতো একমাত্র বাড়তি তথ্য।

done

ফাইল: `lib/features/ai/ai_models.dart`

সংক্ষিপ্ত বিবরণ: AI ফচারের মূল ডটো মডলে ও পারমিশন কোড রাখে — অনুমতি গিটে, "ask" রসেপন্স, থ্রুডে এবং একটি থ্রুডের এককেটি মিসেজে। এখানে কোনো রপিজিটির বা UI কোড নাই, শুধু JSON থেকে/এ রূপান্তরযোগ্য ডটো ক্লাস।

ক্লাস: `AiPermissions`

`abstract final class` — শুধু দুইটি স্ট্যাটিক কনস্ট্যান্ট রাখার একটি নিমেষ্পসে। পুরো AI ফচারটি এই দুই পারমিশন কোডের মধ্য দিয়ে গটে করা হয়: সাধারণ চ্যাট ব্যবহারের জন্য `chat` ('AI_CHAT') এবং প্রোভাইডার/প্রম্পট কনফিগারেশনের জন্য `admin` ('AI_ADMIN')। ব্যাকএন্ডে

`AiServiceImpl.generateRaw` মেথড কোন কন্ট্রোলার থেকে কল হচ্ছে তা নির্বিশেষে এই একই কোড চকে করে।

`static const chat`

সাধারণ ব্যবহারকারীদের AI চ্যাট/জিজ্ঞাসা করার অনুমতি কোড, মান 'AI_CHAT'।

`static const admin`

কোন প্রোভাইডার উত্তর দবে, তাকে কী নির্দেশে দেওয়া হবে এবং তার খরচ কমে — এসব কনফিগার করার অনুমতি, মান 'AI_ADMIN'। এটি অনেক ছোট একটি গ্রুপের কাছে থাকে।

ক্লাস: `AskAnswer`

`POST /api/search/ask` এন্ডপয়েন্টের রসেপন্স মডলে করে। এতে ব্যবহারকারীর প্রশ্ন, AI-এর উত্তর, এবং সার্ভিস রেকর্ডের উপর ভিত্তি করে উত্তরটি তৈরি হয়েছে সেই উৎসের (`SearchHit`) তালিকা থাকে — এই উৎসগুলো গ্লোবাল সার্চের রিজাল্টের মতোই শেপে থাকে যাতে একই উপায়ে ওপেন করা যায়।

`const AskAnswer({required this.question, required this.answer, required this.sources})`

সব ফিল্ড আবশ্যিক রেখে একটি `AskAnswer` তৈরি করে এমন একটি কনস্ট্রাক্টর।

factory AskAnswer.fromJson(Map<String, dynamic> json)

সারভার JSON থেকে AskAnswer তৈরিকরে। question ও answer স্ট্রিং হিসেবে (না পলে খালি স্ট্রিং) এবং sources-কে SearchHit.fromJson দিয়ে ম্যাপ করে অপরিবর্তনীয় (growable: false) লিস্টে রূপান্তর করে; sources না থাকলে খালি Const লিস্ট ব্যবহার হয়।

ক্লাস: AiThreadFeature

abstract final class নমেন্সপেসে যা কথোপকথনের থ্রেডে খোলার সময় কোন ধরনের থ্রেডে খুলতে হবে তা নির্দিষ্ট করে। ব্যাকএন্ডের AiFeature-এ বশিরে অধিক মান আছে, বশেরিভাগই এক-বার-ব্যবহারযোগ্য ড্রাফট হলেপার; কনিতু থ্রেডে হিসেবে শুধু এই দুটোই অর্থপূর্ণ।

static const agent

মান 'AGENT_TASK' — এমন একটি সহায়ক যে সত্যিই কাজ করতে পারে (ছুটি, উপস্থিতি, টাইমশিট)। এর টুল-কলিং লুপে যেকোনো "লখো" (write) অ্যাকশন করার আগে নিশ্চিতকরণের জন্য ফিরে আসে।

static const general

মান 'GENERAL' — কোনো টুল ছাড়া শুধু প্রশ্নোত্তর।

ক্লাস: AiThread

একটি কথোপকথন (থ্রেডে) উপস্থাপন করে — আইডি, ফিচার টাইপ, শিরোনাম (ঐচ্ছিক) ও সর্বশেষ আপডেটের সময়।

const AiThread({required this.id, required this.feature, this.title, this.updatedAt})

সব ফিল্ড দিয়ে একটি AiThread তৈরিকরে; title ও updatedAt ঐচ্ছিক।

bool get isAgent

feature ফিল্ডটি AiThreadFeature.agent-এর সমান কনি তা চকে করে বুলিয়ান রিটার্ন করে — এটি এজেন্ট-টাইপ থ্রেডে কনি তা নির্ধারণ করে।

String get label

থ্রেডের প্রদর্শনযোগ্য শিরোনাম রিটার্ন করে। title ট্রমি করার পর খালি না হলে সটাইই ব্যবহার হয়; নতুবা এজেন্ট থ্রেডের জন্য 'New task' এবং সাধারণ থ্রেডের জন্য 'New conversation' ফিরিয়ে দেয় — কারণ থ্রেডে কিছু বলার আগেই খালি অবস্থায় তৈরি হয় এবং শিরোনাম ব্যাকএন্ড পরে বের করে।

factory AiThread.fromJson(Map<String, dynamic> json)

JSON থেকে AiThread তৈরিকরে। id-কে num-থেকে int রূপান্তর করে (না থাকলে 0), feature-এর ডিফল্ট AiThreadFeature.general, এবং updatedAt-এর জন্য প্রথমতে updatedAt না থাকলে createdAt ফিল্ড থেকে মান নেয়।

ক্লাস: AiMessage

কথোপকথনের একটি টার্ন উপস্থাপন করে, যতাবো অ্যাপ সটো দেখায়। গুরুত্বপূর্ণ সীমাবদ্ধতা:

ব্যাকএন্ড ব্যবহারকারীর নজিরে প্রশ্ন ফেরত দেয় না — getThreadMessages শুধু responsePayload-কে result-এ ম্যাপ করে; প্রশ্নটি requestPayload হিসেবে সংরক্ষিত থাকে কনিতু কখনো এক্সপোজ হয় না। তাই পরে পুনরায় খোলা থ্রেডে শুধু AI-এর উত্তরগুলোই দেখা যায়, যখনে চলতি সেশনের কথোপকথনে অ্যাপ নজিরে ব্যবহারকারীর টাইপ করা প্রশ্ন মনে রাখবে বলে সঠিকভাবে দেখায়।

const AiMessage({required this.text, required this.fromUser, this.awaitingConfirmation = false})

সাধারণ কনস্ট্রাক্টর — টেক্সট, প্রেরক (ইউজার কনি), এবং কনফার্মেশনের অপেক্ষায় আছে কনি তা সটে করে।

```
const AiMessage.user(this.text)
```

ব্যবহারকারীর পাঠানো একটি মैसेজের জন্য সংক্ষিপ্ত কনস্ট্রাক্টর — `fromUser` সবসময় `true` এবং `awaitingConfirmation` সবসময় `false` স্টেট করে।

```
factory AiMessage.fromJson(Map<String, dynamic> json)
```

সার্বভার JSON থেকে একটি AI রপ্লাই তৈরি করে। `result`-কে `text` হিসেবে নেয় (না থাকলে খালি স্ট্রিং), `fromUser` সবসময় `false` (কারণ সার্বভার শুধু রপ্লাই ফেরত দেয়), এবং `awaitingConfirmation` বুলিয়ান হিসেবে পড়ে — `true` হলে বোঝায় এজেন্ট একটি `write` অ্যাকশনের প্রস্তাব দিয়েছে কিন্তু এখনো কিছু ঘটেনি, পরবর্তী টার্নে "হ্যাঁ" বললে তা কার্যকর হবে।

ফাইল: `lib/features/ai/ai_repository.dart`

সংক্ষিপ্ত বিবরণ: AI ফিচারের সব ব্যাকএন্ড এন্ডপয়েন্টের জন্য একটি রিপিজিটরি ক্লাস — জিজ্ঞাসা, থ্রেডে ম্যানজেমেন্ট, এক-বার-ব্যবহারযোগ্য জেনারেশন, এবং অ্যাডমিন কনফিগারেশন (প্রোভাইডার, প্রম্পট টেমপ্লেট, ব্যবহারের পরসিংখ্যান)। এছাড়া Riverpod প্রোভাইডারগুলোও এখানে সংজ্ঞায়িত।

ক্লাস: `AiRepository`

`ApiClient`-এর উপর একটি পাতলা মোড়ক (wrapper) যা AI সম্পর্কিত সব HTTP কল একত্রিত করে। মথডগুলো তিনটি ভাগে বিভক্ত: থ্রেডে (কথোপকথন), এক-বার ড্রাফটিং, এবং `AI_ADMIN`-গেটেড কনফিগারেশন।

```
AiRepository(this._api)
```

একটি `ApiClient` ইনজেক্ট করে রিপিজিটরি তৈরি করে; এই ক্লায়েন্ট দিয়েই সব HTTP কল হয়।

```
Future<AskAnswer> ask(String question)
```

`/search/ask`-এ প্রশ্ন (ট্রিম করা) POST করে এবং রসেপন্স `AskAnswer.fromJson` দিয়ে পার্স করে ফিরিয়ে দেয়। এটি এক-বার প্রশ্নোত্তরের মূল এন্ট্রি পয়েন্ট, কোনো থ্রেডে বা মমেন্টের ছিড়াই।

```
Future<AiThread> createThread(String feature)
```

`/ai/threads`-এ `feature` পাঠিয়ে একটি নতুন খালি থ্রেডে তৈরি করে। থ্রেডে কিছু বলার আগ পর্যন্ত কোনো শিরোনাম থাকে না — সটো পরে ব্যাকএন্ড বের করে।

```
Future<PagedResponse<AiThread>> threads({int page = 0, int size = 20})
```

বর্তমান ব্যবহারকারীর নজিস্ব থ্রেডগুলোর পজেড তালিকা আনে `/ai/threads` থেকে। সার্বভার-সাইডে ব্যবহারকারী অনুযায়ী স্কোপ করা, শুধু কোম্পানি অনুযায়ী নয় — তাই অন্য কারো থ্রেডে দেখার কোনো উপায় নেই।

```
Future<List<AiMessage>> threadMessages(int threadId)
```

`/ai/threads/$threadId/messages` থেকে (page=0, size=100) পজেড রসেপন্স আনে এবং শুধু `content` লিস্টটি ফেরত দেয় — পুরনো থেকে নতুন ক্রমে সংরক্ষিত রপ্লাইগুলো (শুধু AI-এর, ইউজারের প্রশ্ন নয়)।

Future<AiMessage> agentTurn({required int threadId, required String message})

/ai/agent/turn-এ threadId ও টরমি করা message পাঠিয়ে এজেন্টের উত্তর পায়। যদি রিপ্লাইতে awaitingConfirmation স্টে থাকে, তার মানে এজেন্ট এখনো কোনো write অ্যাকশন সম্পন্ন করেনি — শুধু একটি প্রস্তাব দিয়েছে এবং একই থ্রেডে পরের টার্নে অনুমোদনের অপেক্ষায় আছে।

Future<void> deleteThread(int id)

/ai/threads/\$id DELETE করে একটি থ্রেডে মুছে ফেলে।

Future<String> dailyBriefing()

/ai/daily-briefing থেকে content স্ট্রিং আনে (না থাকলে খালি স্ট্রিং)। এই ব্রিফিং প্রতি কোম্পানিতে দিনে একবার তৈরি হয়ে সবাইকে দেওয়া হয় (DailyBriefingService.getOrBuildToday), তাই স্ক্রিনি প্রতিবার ভিজিটে কল করা সস্তা।

Future<AiExchange> generate({required String feature, required String prompt})

/ai/generate-এ feature ও টরমি করা prompt পাঠিয়ে একটি টেক্সট জেনারেট করে। কোনো কথোপকথন বা মমোরিনিই — সেসবের জন্য থ্রেডে এন্ডপয়েন্টগুলো আছে।

Future<PagedResponse<AiExchange>> conversations({String? feature, int page = 0, int size = 20})

/ai/conversations থেকে পূর্ববর্তী জিজ্ঞাসা করা হয়েছে তার পজেড তালিকা আনে, নতুন থেকে পুরনো ক্রমে, ঐচ্ছকিভাবে একটি feature দিয়ে ফিল্টার করা যায় (query প্যারামিটার ?feature — null হলে বাদ পড়ে)।

Future<List<AiProviderConfig>> configs()

/ai/configs থেকে কোম্পানির সংরক্ষিত সব AI প্রোভাইডার কনফিগারেশনের লিস্ট আনে এবং AiProviderConfig.fromJson দিয়ে ম্যাপ করে।

Future<AiProviderConfig> saveConfig(AiProviderConfigRequest request)

/ai/config-এ POST করে একটি প্রোভাইডার সংরক্ষণ করে এবং সটেকে সক্রিয় করে দেয়। এটি প্রোভাইডার টাইপের উপর আপসার্ট (upsert), আইডির উপর নয়।

Future<AiProviderConfig> activateConfig(int id)

/ai/config/\$id/activate PATCH করে একটি কনফিগারেশনকে সক্রিয় প্রোভাইডার বানায়।

Future<void> deleteConfig(int id)

/ai/config/\$id DELETE করে একটি প্রোভাইডার কনফিগারেশন মুছে ফেলে।

Future<PagedResponse<AiPromptTemplate>> templates({int page = 0, int size = 50})

/ai/templates থেকে ফচার অনুযায়ী সাজানো প্রম্পট টেমপ্লেটগুলোর পজেড তালিকা আনে।

Future<AiPromptTemplate> saveTemplate(AiPromptTemplateRequest request)

/ai/templates-এ POST করে একটি প্রম্পট সংরক্ষণ করে। এটি সেই ফচারের আগে প্রম্পটকে প্রতিস্থাপন না করে সুপারসেড (supersede) করে, ফলে ভার্সন নম্বর বাড়তে থাকে।

Future<void> deleteTemplate(int id)

/ai/templates/\$id DELETE করে একটি প্রম্পট টেমপ্লেট মুছে ফেলে।

Future<AiUsageSummary> usage({String? date})

/ai/usage থেকে একটি নির্দিষ্ট দিনের (query date, না দিলে ব্যাকএন্ড আজকের তারিখ ধরে নেয়) AI ব্যবহারের সারসংক্ষেপে আনবে।

টপ-লেভেলে প্রোভাইডার: aiRepositoryProvider

Provider<AiRepository> — apiClientProvider থেকে ApiClient নিয়ে একটি AiRepository তৈরি করে দেয়।

টপ-লেভেলে প্রোভাইডার: dailyBriefingProvider

FutureProvider.autoDispose<String> — aiRepositoryProvider.dailyBriefing() কল করে দিনের ব্রিফিং আনবে; স্ক্রিনি থেকে বেরিয়ে গেলে অটো-ডিসপোজ হয়।

টপ-লেভেলে প্রোভাইডার: aiThreadsProvider

FutureProvider<List<AiThread>> — ব্যবহারকারীর থ্রেডে (size 50 পর্যন্ত) আনবে সাম্প্রতিক কার্যকলাপের ক্রম। সাধারণ future ব্যবহার করা হয়েছে, পজেড কন্ট্রোলার নয়, কারণ থ্রেডে সংখ্যা যথেষ্ট বেশি হয় না যে দ্বিতীয় পজে দরকার পড়বে, আর থ্রেডে থোলা/তৈরি/মুছে ফেলার পর প্রত্যাশিত এটি পুনরায় আনা হয়।

ফাইল: lib/features/ai/ai_screen.dart

সংক্ষিপ্ত ববিরণ: মোবাইল অ্যাপের মূল AI স্ক্রিনি — একটি হিলকা সহায়ক যখনই দিনের ব্রিফিং, এক-বার প্রশ্নোত্তর বক্স, এবং চলমান কথোপকথন (থ্রেডে) তালিকা দেখা যায়। ইচ্ছাকৃতভাবেই এটি পুরো AI মডেল নয় — কথোপকথনের ইতিহাস, প্রোভাইডার কনফিগারেশন, ব্যবহারের রিপোর্টিং একটি আলাদা অ্যাডমিনি কনসোল, ফোনের জন্য উপযুক্ত নয়। এখানে সব কিছু AI_CHAT-গেটেডে দুইটি এন্ডপয়েন্টে ম্যাপ হয়, AI_ADMIN-এ কখনো নয়।

ক্লাস/উইজটে: AiScreen

ConsumerStatefulWidget — পুরো AI স্ক্রিনির রুট উইজটে।

AiScreen({super.key})

কী দিয়ে উইজটে তৈরির কনস্ট্রাক্টর।

ConsumerState<AiScreen> createState()

_AiScreenState স্টেটে অবজেক্ট তৈরি করে।

ক্লাস: _AiScreenState

AiScreen-এর স্টেটে — একটি টিক্সট কন্ট্রোলার, "জিজ্ঞাসা করা হচ্ছে" বুলিয়ান, বর্তমান উত্তর, এবং সম্ভাব্য এরর ধরে রাখবে।

void dispose()

_controller ডিসপোজ করে এবং সুপারক্লাসের dispose কল করে — মেমোরালিকি এড়াতো।

Future<void> _ask()

টেক্সট ফিল্ডের প্রশ্ন ট্রিম করে; খালি হলে বা ইতিমধ্যে একটি অনুরোধ চলমান থাকলে কিছু করে না। ফোকাস সরিয়ে নিয়ে `_asking = true` সেট করে, `aiRepositoryProvider.ask()` কল করে; সফল হলে `_answer` আপডেট করে, ব্যর্থ হলে `_error` সেট করে, শেষে `_asking = false` করে দেয়।

Widget build(BuildContext context)

পারমিশন লোড না হলে একটি লোডার দেখায়; `AiPermissions.chat` না থাকলে "Not available to you" এম্পটি স্টেটে দেখায়। অন্যথায় একটি `AppBar` (ড্রাফটিং ও, শর্তসাপেক্ষে `AI_ADMIN` থাকলে, স্টেটিস বাটনসহ) এবং একটি `ListView`-এ ব্রিফিং, প্রশ্ন করার টেক্সট ফিল্ড ও বাটন, এরর ব্যানার (থাকলে), উত্তর (থাকলে), এবং `_Threads` উইজেটে সাজিয়ে দেখায়।

ক্লাস/উইজেট: `_Threads`

`ConsumerStatefulWidget` — চলমান কথোপকথনগুলোর তালিকা এবং নতুন থ্রেডে শুরু করার বাটন দেখায়। এক-বার প্রশ্নোত্তরে বন্ধিত, একটি থ্রেডে চলতেই থাকে এবং একটি এজেন্ট-থ্রেডে কাজ করতে পারে — তাই এটি শুরু করা একটি সিচেনে পছন্দ, টেক্সট বক্সে টাইপ করলেই যা হয়ে যায় তা নয়।

`_Threads()`

কনস্ট্রাক্টর, কোনো প্রারামিটার নেই।

`ConsumerState<_Threads> createState()`

`_ThreadsState` তৈরি করে।

ক্লাস: `_ThreadsState`

`_Threads`-এর স্টেটে, একটি `_starting` বুলিয়ান দিয়ে থ্রেডে তৈরি হচ্ছে কনি ট্র্যাক করে।

Future<void> _start(String feature)

`_starting = true` সেট করে `createThread(feature)` কল করে, `aiThreadsProvider` ইনভ্যালিডেট করে, তারপর `AiChatScreen.open` দিয়ে নতুন থ্রেডে খোলে এবং ফরোর পর আবার থ্রেডে তালিকা ইনভ্যালিডেট করে (কারণ শিরোনাম এক্ষণে যা বলা হয়েছে তা থেকে বের হয়ে থাকতে পারে)। এরর হলে `ApiException` বা সাধারণ এরর অনুযায়ী সন্ধ্যাকবার দেখায়; শেষে `_starting = false` করে।

Future<void> _delete(AiThread thread)

একটি নিশ্চিতকরণ ডায়ালগ দেখায়; নিশ্চিতি হলে `deleteThread` কল করে এবং `aiThreadsProvider` ইনভ্যালিডেট করে। এরর হলে যথাযথ সন্ধ্যাকবার বার্তা দেখায়।

Widget build(BuildContext context)

`aiThreadsProvider` watch করে। `_starting` হলে লোডার দেখায়, নয়তো "New task" ও "New chat" দুটি বাটন দেখায় যা `_start` কল করে। এরপর থ্রেডে তালিকার `async` স্টেটে অনুযায়ী: লোডিং, এরর কার্ড, খালি হলে বার্তা, অথবা প্রতিটি থ্রেডের জন্য একটি `ListTile` (আইকন, শিরোনাম, সর্বশেষ আপডেট সময়, মুছে ফেলার বাটন, ট্যাপ করলে চ্যাট স্ক্রিনি খোলে) দেখায়।

ক্লাস/উইজেট: `_Briefing`

`ConsumerWidget` — দিনের ব্রিফিং কার্ড দেখায়।

`_Briefing()`

কনস্ট্রাক্টর, কোনো প্রারামিটার নেই।

Widget build(BuildContext context, WidgetRef ref)

dailyBriefingProvider watch করে। লোডিং অবস্থায় একটি ছোট লোডার-সহ কার্ড দেখায়; এরর হলে কিছুই দেখায় না (ব্রফিং লোড না হলেও নচিহ্নে প্রশ্ন বক্স ঠিকভাবে কাজ করে বলে পুরো এরর স্ক্রিনি দেখানোর দরকার নাই); কন্টেন্ট খালি হলে কিছুই দেখায় না; নয়তো আইকন, "Today's briefing" লবেলে ও কন্টেন্ট টেক্সট সহ একটি কার্ড রেন্ডার করে।

ক্লাস/উইজটে: _Answer

StatelessWidget — একটি AskAnswer (প্রশ্ন, উত্তর, উৎস) প্রদর্শন করে।

_Answer({required this.answer})

answer (একটি AskAnswer) দিয়ে উইজটে তৈরি করে।

Widget build(BuildContext context)

প্রথমে প্রশ্ন (ইটালিকি, মডিউটে রঙে) ও উত্তরের টেক্সট সহ একটি কার্ড দেখায়। এরপর answer.sources খালি না হলে "Sources" শিরোনামসহ প্রতিটি উৎসের জন্য একটি ট্র্যাপযোজ্য কার্ড (শিরোনাম, সাবটাইটলে, চিত্র আইকন) দেখায় — ট্র্যাপ করলে openSearchHit কল হয়ে সংশ্লিষ্ট সার্চ রজাল্টে নিয়ে যায়।

ফাইল: lib/features/ai/ai_settings_screen.dart

সংক্ষিপ্ত বিবরণ: AI সহায়ককে কনফিগার করার অ্যাডমিনি কনসোল — প্রোভাইডার, প্রম্পট টেমপ্লেটে এবং ব্যবহারের পরিসংখ্যান, তিনটি ট্যাবে বিভক্ত। এই স্ক্রিনিংরে সব কিছু AI_ADMIN পারমিশনের আড়ালে থাকে, তাই এটি ইচ্ছাকৃতভাবে চ্যাট স্ক্রিনি থেকে আলাদা রাখা হয়েছে।

টপ-লেভেলে প্রোভাইডার: aiConfigsProvider

FutureProvider.autoDispose<List<AiProviderConfig>> — aiRepositoryProvider.configs() কল করে সব সংরক্ষিত প্রোভাইডার কনফিগারেশন আনবে।

টপ-লেভেলে প্রোভাইডার: aiTemplatesProvider

FutureProvider.autoDispose<List<AiPromptTemplate>> — templates() কল করে পজেড রেসপন্সের content অংশ ফেরত দেয়।

টপ-লেভেলে প্রোভাইডার: aiUsageProvider

FutureProvider.autoDispose<AiUsageSummary> — usage() কল করে আজকের ব্যবহারের সারসংক্ষেপে আনবে।

ক্লাস/উইজটে: AiSettingsScreen

StatelessWidget — তিনটি ট্যাব (Providers, Prompts, Usage) নিয়ে DefaultTabController সহ একটি Scaffold তৈরি করে।

AiSettingsScreen({super.key})

কনস্ট্রাক্টর।

Widget build(BuildContext context)

একটি TabBar (৩টি ট্যাব) সহ AppBar এবং TabBarView-এ _ProvidersTab, _TemplatesTab, _UsageTab তিনটি রিন্ডার করে একটি DefaultTabController-এ মোডানো Scaffold রটার্ন করে।

ক্লাস/উইজটে: _ProvidersTab

ConsumerWidget — প্রোভাইডার তালিকা এবং নতুন প্রোভাইডার যোগ করার ফ্লোটিং বাটন দেখায়।

_ProvidersTab()

কনস্ট্রাক্টর।

Widget build(BuildContext context, WidgetRef ref)

aiConfigsProvider-কে ConfigList<AiProviderConfig>-এ পাস করে রেফ্রেশে, খালি/এরর বার্তা, _ConfigRow আইটেমে বন্ডার সহ) এবং একটি Positioned FloatingActionButton.extended দেখায় যা showProviderSheet খোলে।

ক্লাস/উইজটে: _ConfigRow

ConsumerStatefulWidget — একটি একক প্রোভাইডার কনফিগারেশনের সারি প্রদর্শন করে।

_ConfigRow({required this.config})

config (একটি AiProviderConfig) দিয়ে সারি তৈরি করে।

ConsumerState<_ConfigRow> createState()

_ConfigRowState তৈরি করে।

ক্লাস: _ConfigRowState

সারি স্টেটে — _busy ফ্ল্যাগ দিয়ে কোনো অ্যাকশন চলছে কিনা ট্র্যাক করে।

Future<void> _run(Future<void> Function(AiRepository) action, String done)

_busy = true সেট করে দেওয়া action চালায়; সফল হলে aiConfigsProvider ইনভ্যালিডেট করে ও done বার্তা স্ন্যাকবারে দেখায়; ApiException বা সাধারণ এরর হলে যথাযথ বার্তা দেখায়; শেষে _busy = false করে।

Future<void> _delete()

একটি নিশ্চিতকরণ ডায়ালগ (confirmAction) দেখায় — সক্রিয় প্রোভাইডার মুছলে ভিন্ন সতর্কবার্তা দেখায় (সহায়ক প্রশ্ন পাঠানোর জায়গাহীন হয়ে যাবে)। নিশ্চিতি হলে _run দিয়ে deleteConfig কল করে।

Widget build(BuildContext context)

মডলে, তাপমাত্রা (থাকলে), ম্যাক্স টোকেন (থাকলে) নিয়ে একটি তথ্য-স্ট্রিং তৈরি করে এবং একটি ConfigRow রটার্ন করে যেখানে সম্পাদনার জন্য showProviderSheet এবং অ্যাকশন হিসাবে (নেস্ক্রয়িং হলে) "Use this one" ও "Remove" থাকে।

উপ-লভেলে ফাংশন: showProviderSheet

Future<void> showProviderSheet(BuildContext context, {AiProviderConfig? config})`

showModalBottomSheet দিয়ে _ProviderSheet(config: config) খোলে — প্রোভাইডার তৈরি বা সম্পাদনার ফর্ম। এডটি মোডে কী (API key) ফলিড ফাঁকা থাকে ও শুধু কতটাইপ করলেই পাঠানো হয়, কারণ সংরক্ষিত কী এনক্রিপ্টেডে এবং কখনো ফরেন আসে না — ফাঁকা মানে "যা আছে তা রাখো"।

ক্লাস/উইজটে: _ProviderSheet

ConsumerStatefulWidget — প্রোভাইডার তৈরিসম্পাদনার ফর্ম শীট।

_ProviderSheet({this.config})

ঐচ্ছিক config দিয়ে তৈরিকরে — null হলে নতুন প্রোভাইডার, নয়তো এডটি মোড।

ConsumerState<_ProviderSheet> createState()

_ProviderSheetState তৈরিকরে।

ক্লাস: _ProviderSheetState

ফর্মের স্টেটে — ফর্ম কী, API-কী কন্ট্রোলার, তাপমাত্রা ও ম্যাক্স-টোকেনে কন্ট্রোলার (বদ্যমান কনফিগ থেকে প্র-ফলি করা), নরিবাচতি প্রোভাইডার ও মডলে, এরর, এবং busy ফ্ল্যাগ রাখা।

void dispose()

_apiKey, _temperature, _maxTokens কন্ট্রোলার ডিসপোজ করে।

void _setProvider(String? provider)

নতুন প্রোভাইডার null না হলে এবং আগেরটার থেকে ভিন্ন হলে স্টেটে আপডেট করে — প্রোভাইডার পাল্টালে মডলে তালিকাও প্রোভাইডার-নরিদৃষ্টি বলে _model-কে নতুন প্রোভাইডারের প্রথম মডলে রসিটে করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে; নতুন প্রোভাইডার (কনফিগ নই) এবং API-কী ফাঁকা হলে (মক প্রোভাইডার ছাড়া) এরর দেখায় — যাত কী ছাড়া সবে হয়ে প্রথম প্রশ্নইে ব্রথ না হয়। _busy = true করে saveConfig কল করে AiProviderConfigRequest দিয়ে (প্রোভাইডার, মডলে, কী, পার্স করা তাপমাত্রা ও ম্যাক্স টোকেনে সহ); সফল হলে aiConfigsProvider ইনভ্যালিডেট করে শীট বন্ধ করে; ব্রথ হলে এরর স্টেট করে।

Widget build(BuildContext context)

FormSheetFrame-এ প্রোভাইডার ও মডলে ড্রপডাউন, API-কী টেক্সটফিল্ড (obscured), তাপমাত্রা (0-2 রেঞ্জ ভ্যালিডেশন) ও ম্যাক্স-টোকেনে (ধনাত্মক পূর্ণসংখ্যা ভ্যালিডেশন) ফিল্ড, এবং একটি সাহায্যকারী নোট টেক্সট সাজায়।

ক্লাস/উইজটে: _TemplatesTab

ConsumerWidget — প্রম্পট টেমপ্লেটে তালিকা এবং নতুন লখোর ফ্লোটিং বাটন দেখায়।

_TemplatesTab()

কনস্ট্রাক্টর।

Widget build(BuildContext context, WidgetRef ref)

aiTemplatesProvider-কে ConfigList<AiPromptTemplate>-এ পাস করে এবং showTemplateSheet খোলার ফ্লোটিং বাটন দেখায়।

ক্লাস/উইজটে: _TemplateRow

ConsumerStatefulWidget — একটি প্রম্পট টেমপ্লেটে সারি প্রদর্শন করে।

_TemplateRow({required this.template})

template দিয়ে তৈরিকরে।

ConsumerState<_TemplateRow> createState()

_TemplateRowState তৈরি করে।

ক্লাস: _TemplateRowState

_busy ফ্ল্যাগ রাখে।

Future<void> _delete()

নশিচতিকরণ ডায়ালগ দেখিয়ে deleteTemplate কল করে; সফল/ব্যর্থ অনুযায়ী সন্ধ্যাকবার বার্তা দেখায় এবং aiTemplatesProvider ইনভ্যালিডিটে করে।

Widget build(BuildContext context)

ফচার, ভার্সন, আপডেটকারী নাম, সময় দিয়ে একটি তথ্য-স্ট্রিং তৈরি করে একটি ConfigRow রটাইন করে — এডটি showTemplateSheet খোলে, অ্যাকশনে "Delete" থাকে।

টপ-লেভেলে ফাংশন: showTemplateSheet

Future<void> showTemplateSheet(BuildContext context, {AiPromptTemplate? template})`_TemplateSheet'(template: template) একটি মিডাল বটম শীটে খোলে। কোনো আপডেট এন্ডপয়েন্ট নেই — একই ফচারে সেভে করলে আগেরটা সুপারসডি হয়ে নতুন ভার্সন শুরু হয়, তাই এডটি মানবে আসলে পরের ভার্সন লখো, শীটে সেটাই বলা হয়।

ক্লাস/উইজটে: _TemplateSheet

ConsumerStatefulWidget — প্রম্পট টেমপ্লেটে তৈরি/নতুন-ভার্সন লেখার ফর্ম।

_TemplateSheet({this.template})

ঐচ্ছিক template দিয়ে তৈরি করে।

ConsumerState<_TemplateSheet> createState()

_TemplateSheetState তৈরি করে।

ক্লাস: _TemplateSheetState

ফর্ম কী, নাম/টেমপ্লেট-টেক্সট/চ্যেঞ্জ-নোট কন্ট্রোলার, নরিবাচতি ফচার, এরর ও busy ফ্ল্যাগ রাখে।

void dispose()

_name, _template, _changeNotes কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ফর্ম ভ্যালিডিটে করে _busy = true করে saveTemplate কল করে AiPromptTemplateRequest দিয়ে (ফচার, নাম, টেমপ্লেট টেক্সট, চ্যেঞ্জ নোট, সব ট্রিম করা); সফল হলে aiTemplatesProvider ইনভ্যালিডিটে করে শীট বন্ধ করে; ব্যর্থ হলে এরর দেখায়।

Widget build(BuildContext context)

FormSheetFrame-এ ফচার ড্রপডাউন, নাম টেক্সটফিল্ড (ভ্যালিডেশন সহ), প্রম্পট টেক্সটফিল্ড (মাল্টিলাইন, ভ্যালিডেশন সহ), চ্যেঞ্জ-নোট টেক্সটফিল্ড, এবং প্রসঙ্গ অনুযায়ী (নতুন বনাম ভার্সন) ভিন্ন সাহায্যকারী টেক্সট দেখায়।

ক্লাস/উইজটে: _UsageTab

ConsumerWidget — একটি নির্দিষ্ট দিনের AI ব্যবহারের পরিসংখ্যান দেখায়।

_UsageTab()

কনস্ট্রাক্টর।

Widget build(BuildContext context, WidgetRef ref)

aiUsageProvider watch করে; একটি RefreshIndicator-এ মোড়ানো এবং async স্টেটে অনুযায়ী লোডার, এরর স্টেটে, অথবা তারিখ, প্রশ্ন সংখ্যা ও টোকেনে কাউন্টের StatCard, গড় উত্তরের সময়, এবং (থাকলে) ফচার-অনুযায়ী অনুরোধ/টোকেনে ব্রেকেডাউন দেখায়।

ফাইল: lib/features/alerts/alerts_screen.dart

সংক্ষিপ্ত বিবরণ: ব্যবহারকারীর নোটিফিকেশন তালিকা দেখানোর স্ক্রিনি — অপঠতি/পঠতি পার্থক্য, "সব পঠতি করুন" অ্যাকশন, এবং লোড-মোর পজেনিশন সহ। খালি অবস্থার জন্য একটি বিশিষ্ট বার্তাও দেখায়।

ক্লাস/উইজটে: AlertsScreen

ConsumerWidget — নোটিফিকেশন স্ক্রিনির রুট উইজটে।

AlertsScreen({super.key})

কনস্ট্রাক্টর।

Widget build(BuildContext context, WidgetRef ref)

notificationsControllerProvider watch করে অপঠতি আছে কিনা বের করে। AppBar-এ অপঠতি থাকলে "Mark all read" বাটন দেখায়। মূল বডিতে async স্টেটে অনুযায়ী লোডার, এরর স্টেটে, খালি হলে EmptyState, অথবা RefreshIndicator-এ মোড়ানো একটি ListView.separated দেখায় যেখানে প্রতিটি আইটেমে _NotificationTile এবং hasMore হলো শেষে একটি "Load older" বাটন/লোডার থাকে।

Future<void> _markAll(BuildContext context, WidgetRef ref)

notificationsControllerProvider.notifier.markAllRead() কল করে সব নোটিফিকেশন পঠতি হিসেবে চিহ্নিত করে; ApiException হলো সন্ধ্যাকবারে বার্তা দেখায়।

ক্লাস/উইজটে: _NotificationTile

ConsumerWidget — একটি একক নোটিফিকেশন কার্ড দেখায়।

_NotificationTile({required this.notification})

notification (একটি AppNotification) দিয়ে তৈরি করে।

Widget build(BuildContext context, WidgetRef ref)

অপঠতি হলো কার্ডের ব্যাকগ্রাউন্ড ব্র্যান্ড-টিন্ট রঙে দেখায় (একটি ছোট ডট একা ফোনে খোঁজ না-ও হতে পারে, তাই পুরো সারি রঙ পাল্টায়) এবং ট্যাপ করলে markRead কল করে। কার্ডে একটি রিগুনি ডট (অপঠতি হলো), শিরোনাম, বার্তা (থাকলে), এবং আপেক্ষিক সময় (Fmt.relative) দেখায়।

ফাইল: lib/features/alerts/notification_preferences_screen.dart

সংক্ষিপ্ত ববিরণ: ব্যবহারকারীর নোটিফিকেশন পছন্দ (ইমেইল, ইন-অ্যাপ, মার্কটেং) টগল করার স্ক্রিনি, এবং সবকিছু ডিফল্টে রসিটে করার অপশন সহ।

ক্লাস/উইজটে: NotificationPreferencesScreen

ConsumerWidget — নোটিফিকেশন প্রফোরেন্স স্ক্রিনিরে বুট উইজটে।

NotificationPreferencesScreen({super.key})

কনস্ট্রাক্টর।

Future<void> _reset(BuildContext context, WidgetRef ref)

একটি নিশ্চিতকরণ ডায়ালগ দেখায় ("প্রতিটি সুইচ নতুন অ্যাকাউন্টের ডিফল্টে ফরি যাবে")। নিশ্চিতি হলে notificationPreferencesProvider.notifier.resetToDefaults() কল করে; ApiException বা সাধারণ এরর হলে যথাযথ সন্ধ্যাকবার বার্তা দেখায়।

Future<void> _toggle(BuildContext context, WidgetRef ref, NotificationPreferences updated)

notificationPreferencesProvider.notifier.save(updated) কল করে একটি পিরবির্ততি প্রফোরেন্স অবজেক্ট সংরক্ষণ করে; ব্যর্থ হলে এরর বার্তা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

notificationPreferencesProvider watch করে; AppBar-এ ডটো লোড হলে সক্রিয় "Reset" বাটন থাকে। বডতিে async স্টেটে অনুযায়ী লোডার, এরর স্টেটে, অথবা তনিটসিকেশনে ভাগ করা ListView দেখায়: "Email" (৬টি সুইচ — সারভিস রকি়োস্টে, স্ট্যাটাস পিরবির্তন, ইনভয়সে, পমেন্টে, টাস্ক অ্যাসাইনমেন্টে, লভি আপডটে), "In-app" (২টি সুইচ), এবং "Other" (মার্কটেং সুইচ) — প্রতিটি সুইচ পাল্টালে _toggle কল হয়ে prefs.copyWith(...) দিয়ে আপডটে পাঠায়।

ক্লাস/উইজটে: _Switch

StatelessWidget — একটি লবেলেসহ সুইচ-লস্টি-টাইল এবং ঐচ্ছকি ডিভাইডার দেখায়।

_Switch({required this.label, required this.value, required this.onChanged, this.isLast = false})

লবেলে, বর্তমান মান, কলব্যাক এবং এটি তালকির শেষে আইটেমে কনি (ডিভাইডার দেখানো/না-দেখানো নির্ধারণ করত) দিয়ে তৈরি করে।

Widget build(BuildContext context)

একটি SwitchListTile (সাদা থাম্ব রঙ সহ) দেখায়, এবং isLast না হলে নচি একটি হিলকা ডিভাইডার যোগ করে।

done

ফাইল: lib/features/alerts/notification_repository.dart

সংক্ষিপ্ত ববিরণ: এই ফাইলটি অ্যাপ-নোটিফিকেশন, নোটিফিকেশন প্রফোরেন্স এবং আনরডি কাউন্টের জন্ম মডলে, রপি়োজিটরি এবং Riverpod কন্ট্রোলার সংজ্ঞায়তি করে। এটি /notifications ও /notification-preferences API এন্ডপয়ন্টে মোড়ক (wrap) করে এবং লস্টি, রডি-মার্কিং ও ব্যাজ-কাউন্ট পোলিংয়ের অপ্টমিস্টিকি UI স্টেটে ম্যানেজে করে।

ক্লাস: AppNotification

একটি একক নোটিফিকেশন আইটেম উপস্থাপন করে — id, title, message, read-স্ট্যাটাস, ঐচ্ছিক type, link ও createdAt। ব্যাকএন্ড DTO-তে message/body এবং read/isRead উভয় বানানই ব্যবহৃত হয়েছে বলে fromJson উভয়টিই গ্রহণ করে, যাতে ভুল বানানরে কারণে সবকিছু "unread" না দেখায়।

AppNotification({required id, required title, required message, required read, type, link, createdAt})

কনস্ট্রাক্টর; সব ফিল্ড final এবং const-উপযোগী।

AppNotification asRead()

একই ডটোসহ কনিন্তু read: true সেটে করা একটি নিন্তুন AppNotification ইনস্ট্যান্স রিটার্ন করে — অপ্টমিস্টিক UI আপডেটে ব্যবহৃত হয়, যখনে একটি আইটেমকে সঙ্গে সঙ্গে read দেখাতে হয়।

factory AppNotification.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে ইনস্ট্যান্স তৈরিকরে। id নাম্বার থেকে int-এ কনভার্ট হয়, message না থাকলে body ফলব্যাক হিসেবে ব্যবহৃত হয়, এবং read না থাকলে isRead চেকে করা হয় — দুই ধরনের ব্যাকএন্ড বানান-অসঙ্গতি সামলাতে।

ক্লাস: NotificationPreferences

ব্যবহারকারীর নোটিফিকেশন সেটিংস (ইমেইল/ইন-অ্যাপ টগল) ধারণ করে। GET/PUT/DELETE /api/notification-preferences এন্ডপয়েন্টের সাথে ম্যাপ হয়; PUT একটি সম্পূর্ণ প্রতিস্থাপন (full replacement), আংশিক প্যাচ নয়, তাই UI-কে সবসময় সম্পূর্ণ অবজেক্ট পাঠাতে হয়।

NotificationPreferences({...})

নয়টি বুলিয়ান ফিল্ড নিয়ে কনস্ট্রাক্টর, প্রতিটির ডিফল্ট মান আছে (বিশেষিভাগ true, emailMarketing false)।

factory NotificationPreferences.fromJson(Map<String, dynamic> json)

JSON থেকে প্রতিটি বুলিয়ান ফিল্ড পড়ে, অনুপস্থিতি থাকলে কনস্ট্রাক্টরের ডিফল্ট মান ব্যবহার করে ইনস্ট্যান্স তৈরিকরে।

Map<String, dynamic> toJson()

সব নয়টি ফিল্ডকে একটি JSON ম্যাপে সিরিয়লাইজ করে — PUT কলে সম্পূর্ণ অবজেক্ট পাঠানোর জন্য প্রয়োজন।

NotificationPreferences copyWith({...})

নরিদৃষ্ট কিছু ফিল্ড পরিবর্তন করে বাকি সব বর্তমান মান রেখে একটি নিন্তুন ইনস্ট্যান্স তৈরিকরে।

ক্লাস: NotificationRepository

নোটিফিকেশন ও প্রফোরেন্স সংক্রান্ত সব API কল কনেন্দ্রীভূত করে।

NotificationRepository(this._api)

ApiClient ইনজেক্ট করে কনস্ট্রাক্ট করে।

Future<NotificationPreferences> getPreferences()

/notification-preferences থেকে GET করে ব্যবহারকারীর বর্তমান প্রফোরেন্স ফেরত দেয়।

`Future<NotificationPreferences> updatePreferences(NotificationPreferences preferences)`

সম্পূর্ণ প্রফোরেন্স অবজেক্ট PUT করে আপডেট করা মান ফেরত দেয়।

`Future<NotificationPreferences> resetPreferences()`

DELETE কল করে প্রফোরেন্স ডিফল্টে রসিটে করে এবং নতুন মান ফেরত দেয়।

`Future<PagedResponse<AppNotification>> list({bool unreadOnly = false, int page = 0, int size = 20})`

পজেনিটেড নোটিফিকেশন লিস্ট আনে, `unreadOnly` ফ্ল্যাগ দিয়ে শুধু আনরডি ফিল্টার করা যায়।

`Future<int> unreadCount()`

`/notifications/count` থেকে বর্তমান আনরডি সংখ্যা ফেরত দেয়; মান না পলে ০ রটার্ন করে।

`Future<void> markRead(int id)`

নর্দিস্ট নোটিফিকেশনকে read হিসেবে চিহ্নিত করতে PATCH কল করে।

`Future<void> markAllRead()`

সব নোটিফিকেশনকে read হিসেবে চিহ্নিত করতে PATCH কল করে।

টপ-লেভেলে ভেরিফিকেশন: notificationRepositoryProvider

NotificationRepository-এর একক ইনস্ট্যান্স সরবরাহকারী Riverpod Provider, যা apiClientProvider থেকে ApiClient নেয়।

ক্লাস/কন্ট্রোলার: NotificationPreferencesController

`AsyncNotifier<NotificationPreferences>` — ব্যবহারকারীর প্রফোরেন্স স্টেটে ও তা সবে/রসিটে করার লজিক ধারণ করে।

`Future<NotificationPreferences> build()`

`currentUserProvider` ওয়াচ করে (সাইন-আউটে রিবিল্ড নিশ্চিত করতে) এবং প্রাথমিক প্রফোরেন্স লোড করে।

`Future<void> save(NotificationPreferences updated)`

অপ্টিমিস্টিকভাবে স্টেটে আপডেট করে, তারপর সার্বভারে পাঠায়; ব্যর্থ হলে আগের স্টেটে ফিরে যায় এবং এর রথিরো করে। নাম `save` — কারণ `AsyncNotifier`-এর নিজস্ব `update` মেথড আগে থেকেই আছে, যা ওভাররাইড করলে কম্পাইল হবে না।

`Future<void> resetToDefaults()`

রসিটে কল করে, ফলাফল স্টেটে বসায়; ব্যর্থ হলে আগের স্টেটে ফিরে যায় ও রথিরো করে।

টপ-লেভেলে ভেরিফিকেশন: notificationPreferencesProvider

NotificationPreferencesController-এর AsyncNotifierProvider।

ক্লাস/কন্ট্রোলার: UnreadCountController

`Notifier<int>` — আনরডি ব্যাজ কাউন্ট প্রতি ৬০ সেকেন্ডে পোল করে রাখে, কারণ ব্যাকগ্রাউন্ডে এখনো কোনো পুশ চ্যানেল যুক্ত নাই।

`int build()`

`currentUserProvider` ওয়াচ করে; সাইন-ইন না থাকলে টাইমার বাতলি করে ০ রিটার্ন করে। সাইন-ইন থাকলে সঙ্কে সঙ্কে একবার `_tick()` চালায় এবং ৬০ সেকেন্ড পরপর পুনরাবৃত্ত টাইমার সেট করে; ডিসপোজে টাইমার বাতলি করে।

`Future<void> _tick()`

সাব্ভার থেকে আনরডি কাউন্ট আনার চেষ্টা করে; ব্যর্থ হলে আগের কাউন্ট রেখে চুপচাপ পরের টিকে জন্ম অপেক্ষা করে — ব্যাজের জন্ম এরর-স্টেটে প্রয়োজন নই বলে।

`Future<void> refresh()`

`_tick()` তাৎক্ষণিকভাবে কল করার পাবলিক শর্টকাট, সাধারণত কছু `read` করার পরপরই ডাকা হয় যাত ব্যাজ এক মিনিটি পছিয়ে না থাকে।

টপ-লেভেলে ভরেয়িবেল: `unreadCountProvider`

`UnreadCountController`-এর `NotifierProvider`।

ক্লাস: `NotificationsState`

`@immutable` স্টেটে ক্লাস — নোটিফিকেশন আইটেমের লিস্ট, বর্তমান পেজে, মোট পেজে এবং "আরও লোড হচ্ছে" ফ্ল্যাগ ধারণ করে।

`NotificationsState({items, page, totalPages, loadingMore})`

ডিফল্ট মানসহ কনস্ট্রাক্টর (খালি লিস্ট, পেজে ০)।

`bool get hasMore`

বর্তমান পেজের পরে আরও পেজে আছে কনি যাচাই করে।

`bool get hasUnread`

লিস্টে কোনো আনরডি আইটেমে আছে কনি যাচাই করে।

`NotificationsState copyWith({items, page, totalPages, loadingMore})`

নরিদৃষ্টি ফল্ড পরবর্তন করে নতুন ইনস্ট্যান্স তৈরি করে।

ক্লাস/কন্ট্রোলার: `NotificationsController`

`AsyncNotifier<NotificationsState>` — নোটিফিকেশন লিস্ট স্ক্রিনিং সম্পূর্ণ স্টেটে, পেজেনিশন এবং `read`-মার্কিং লজিক পরিচালনা করে।

`Future<NotificationsState> build()`

`currentUserProvider` ওয়াচ করে (কারণ নোটিফিকেশন প্রত-ব্যবহারকারী, এবং প্রোভাইডার সাইন-আউট টিকে থাকে) এবং `_load()` কল করে।

`Future<NotificationsState> _load()`

প্রথম পেজে ফেচ করে একটি নতুন `NotificationsState` তৈরি করে।

`Future<void> refresh()`

`_load()` পুনরায় চালিয়ে `AsyncValue.guard` দিয়ে স্টেটে রিফ্রেশ করে।

Future<void> loadMore()

আরও পজে থাকলে ও ইতিমধ্যে লোড না চলাকালীন `loadingMore: true` স্টেটে করে পরে পজে ফেলে করে বর্তমান আইটেমের সাথে যুক্ত করে; ব্যর্থ হলে `loadingMore` false করে রাখবে।

Future<void> markRead(int id)

সংশ্লিষ্ট আইটেমে ইতিমধ্যে read না হলে, তা অপ্টিমিস্টিকভাবে read হিসেবে দেখিয়ে সার্ভারে কল করে এবং `unreadCountProvider.notifier.refresh()` ট্রিগার করে; ব্যর্থ হলে আগের স্টেটে ফিরে যায় ও রাখবে।

Future<void> markAllRead()

সব আইটেমে অপ্টিমিস্টিকভাবে read দেখায়, সার্ভারে কল করে ও আনরডি কাউন্ট রিফ্রেশ করে; ব্যর্থ হলে পূর্বাবস্থায় ফিরে যায় ও রাখবে।

টপ-লেভেলে ভেরিফিকেশন: notificationsControllerProvider

NotificationsController-এর AsyncNotifierProvider।

ফাইল: lib/features/assets_periods/assets_periods_models.dart

সংক্ষিপ্ত ববিরণ: অ্যাকাউন্টিং পরিয়িড কলোজিং এবং ফিক্সড অ্যাসেটে (স্থায়ী সম্পদ) ব্যবস্থাপনার জন্য মডেলে ও পারমিশন কনস্ট্যান্ট সংজ্ঞায়িত করে। এতে `AccountingPeriod`, `FiscalYear`, `FixedAsset`, `FixedAssetRequest`, `DepreciationRun` ক্লাস এবং সংশ্লিষ্ট স্ট্যাটাস enum-সদৃশ কনস্ট্যান্ট রয়েছে।

ক্লাস: ClosingPermissions

abstract final class — অ্যাকাউন্টিং পরিয়িড ও ফিক্সড অ্যাসেটে সংক্রান্ত পারমিশন কোডগুলোর স্ট্যাটিক কনস্ট্যান্ট ধারণ করে (`periodView`, `periodClose`, `assetView`, `assetManage`)। এতে কোনো মেথড নেই, শুধু স্ট্যাটিক ফিল্ড।

ক্লাস: PeriodStatus

abstract final class — ব্যাকএন্ডের `PeriodStatus` enum-এর প্রতিনিধিত্ব, `open` ও `closed` স্ট্যাটিক স্ট্রিং কনস্ট্যান্ট।

ক্লাস: AssetStatus

abstract final class — ব্যাকএন্ডের `FixedAssetStatus` enum-এর প্রতিনিধিত্ব, `active`, `fullyDepreciated`, `disposed` স্ট্যাটিক স্ট্রিং কনস্ট্যান্ট।

ক্লাস: AccountingPeriod

একটি অ্যাকাউন্টিং পরিয়িড (সাধারণত এক মাস) উপস্থাপন করে, তার স্ট্যাটাস, তারিখ-সীমা এবং কলোজ/রপ্তানে ইতিহাসসহ।

`AccountingPeriod`({required id, required fiscalYear, required periodNumber, required status, label, startDate, endDate, closedBy, closedAt, reopenedBy, reopenedAt})

কনস্ট্রাক্টর, সব ফিল্ড final।

bool get isOpen

status == `PeriodStatus.open` কনি যাচাই করে।

`bool get wasReopened`

`reopenedAt` নন-নাল কনি যাচাই করে — অর্থাৎ, পরিষিডটি একবার ক্লোজ হয়ে আবার খোলা হয়েছিল কনি, যা রিপোর্ট করা তথ্য বদলে যাওয়ার ইঙ্গিত দিয়ে।

`String get title`

`label` থাকলে তা, নাহলে `'Period $periodNumber'` ফরমত দিয়ে।

`factory AccountingPeriod.fromJson(Map<String, dynamic> json)`

JSON থেকে ইনস্ট্যান্স তৈরিকরে, সংখ্যাসূচক ফিল্ড `int`-এ কনভার্ট করে ও `status`-এর ডিফল্ট `PeriodStatus.open`।

ক্লাস: FiscalYear

একটি অর্থবছর এবং তার ক্লোজিং অগ্রগতি (কতগুলো পরিষিড খোলা/বন্ধ) উপস্থাপন করে।

`FiscalYear({required fiscalYear, required totalPeriods, required openPeriods, required closedPeriods, name, startDate, endDate})`

কনস্ট্রাক্টর।

`bool get fullyClosed`

সব পরিষিড বন্ধ কনি (`totalPeriods > 0 && openPeriods == 0`) যাচাই করে — বছরটি ক্লোজ করার পূর্বশর্ত।

`double? get progress`

কতগুলো পরিষিড ক্লোজ হয়েছে তার অনুপাত (০-১ এর মধ্যে ক্ল্যাম্প করা) রিটার্ন করে; পরিষিড না থাকলে `null`।

`String get title`

`name` থাকলে তা, নাহলে `'FY $fiscalYear'` ফরমত দিয়ে।

`factory FiscalYear.fromJson(Map<String, dynamic> json)`

JSON থেকে ইনস্ট্যান্স তৈরিকরে।

ক্লাস: FixedAsset

কোম্পানির মালিকানাধীন একটি স্থায়ী সম্পদ, যার মূল্য সময়ের সাথে হ্রাস পায়।

`FixedAsset({required id, required name, required cost, required usefulLifeMonths, required status, assetTag, category, salvageValue, acquisitionDate, accumulatedDepreciation, bookValue, monthlyDepreciation, notes})`

কনস্ট্রাক্টর।

`bool get isDisposed`

`status == AssetStatus.disposed` কনি যাচাই করে।

`bool get canDispose`

`!isDisposed` — অর্থাৎ এখনো ধরে রাখা সম্পদই কেবল ডিসপোজ করা যায়।

`double? get depreciated`

`accumulatedDepreciation / cost`-এর অনুপাত (০-১ ক্ল্যাম্প করা) রিটার্ন করে; `cost <= 0` হলে `null`।

`int? get monthsRemaining`

বর্তমান মাসকি অবচয়রে হারে বাকি পূর্ণ মাসরে সংখ্যা হিসাব করে; ডিসিপোজড হলে, মাসকি হার শূন্য/অজানা হলে, বা `bookValue` অজানা থাকলে `null` রিটার্ন করে। বাকি অবচয়যোগ্য মূল্য শূন্য বা কম হলে `0` রিটার্ন করে।

`factory FixedAsset.fromJson(Map<String, dynamic> json)`

JSON থেকে ইনস্ট্যান্স তৈরি করে, সংখ্যাসূচক ফিল্ডগুলো `int/double`-এ কনভার্ট করে।

ক্লাস: FixedAssetRequest

নতুন ফক্সড অ্যাসেটে তৈরি জন্য `POST /company/finance/fixed-assets` রিকোয়েস্ট বডি। `name`, `cost`, `usefulLifeMonths`, `acquisitionDate` আবশ্যিক; `postPurchaseToLedger` একটিনালবেল বুলিয়ান — না দিলে ব্যাকএন্ড নজিে সন্ধিধান্ত নেয়, তাই ফর্মে স্পষ্টভাবে জিজ্ঞাসা করা হয়।

`FixedAssetRequest({required name, required cost, required usefulLifeMonths, required acquisitionDate, assetTag, category, salvageValue, notes, postPurchaseToLedger})`

কনস্ট্রাক্টর।

`double get monthlyDepreciation`

স্ট্রেট-লাইন পদ্ধতিতে মাসকি অবচয় হিসাব করে: $(cost - salvageValue) / usefulLifeMonths$; `usefulLifeMonths <= 0` হলে `0` রিটার্ন করে। ফর্ম পূরণ করার সময় প্রভিডি দখোতে ব্যবহৃত হয়।

`Map<String, dynamic> toJson()`

রিকোয়েস্ট বডি তৈরি করে; ভেতরে হলেপার `clean()` ফাংশন ফাঁকা/হোয়াইটস্পেস-ওনলি স্ট্রিং ফিল্ডকে `null` করে বাদ দেয়, যাত খালি অপশনাল ফিল্ড না পাঠানো হয়।

ক্লাস: DepreciationRun

একবারে সব অ্যাসেটের জন্য চালানো এক মাসরে অবচয় হিসাবের ফলাফল উপস্থাপন করে।

`DepreciationRun({required id, required year, required month, required totalAmount, required assetsDepreciated, runBy, runAt})`

কনস্ট্রাক্টর।

`factory DepreciationRun.fromJson(Map<String, dynamic> json)`

JSON থেকে ইনস্ট্যান্স তৈরি করে, `month`-এর ডিফল্ট `1`।

ফাইল: `lib/features/assets_periods/assets_periods_repository.dart`

সংক্ষিপ্ত ববিরণ: অ্যাকাউন্টিং পরিষিড ক্লোজিং ও ফক্সড অ্যাসেটে সংক্রান্ত সব API কল ও Riverpod স্টেটে কন্ট্রোলার এই ফাইলে আছে। `ClosingRepository` ব্যাকএন্ডের `/company/finance/accounting-periods` ও `/company/finance/fixed-assets` এন্ডপয়েন্ট মোডক করে।

ক্লাস: ClosingRepository

পরিষিড ক্লোজিং ও ফক্সড অ্যাসেটে ব্যবস্থাপনার সব API কল কন্ট্রোলার করে।

`ClosingRepository(this._api)`

`ApiClient` ইনজেক্ট করে কনস্ট্রাক্ট করে।

Future<List<FiscalYear>> fiscalYears()

কোম্পানির সব অর্থবছরে তালিকা (বয়ের লস্ট, পজেনিশেন ছাড়া) আনবে, প্রতিটি কলোজিং অগ্রগতিসহ।

Future<List<AccountingPeriod>> periods(int fiscalYear)

নির্দিষ্ট এক অর্থবছরে সব পেরিয়ড আনবে; **fiscalYear** আবশ্যিক প্যারামিটার — "সব পেরিয়ড" পাওয়ার কোনো কল নেই।

Future<AccountingPeriod> closePeriod(int id)

একটি পেরিয়ড বন্ধ করতে **POST** কল করে, যাতে তার পরে কোনো এন্ট্রিপোস্ট করা না যায়।

Future<AccountingPeriod> reopenPeriod(int id)

বন্ধ পেরিয়ড আবার খুলতে **POST** কল করে।

Future<void> closeFiscalYear(int fiscalYear)

নটে ইনকাম রটাইনড আর্নহিসে রোল করে বছর বন্ধ করে; **fiscalYear** একটি কৌশলে প্যারামিটার এবং রসেপন্স খালি। এই ধাপ না করলে পরের বছরে ব্যালান্স শীট মিলে না।

Future<PagedResponse<FixedAsset>> assets({int page = 0, int size = 30})

ফক্সড অ্যাসেটের পজেনিটেডে তালিকা আনবে।

Future<FixedAsset> createAsset(FixedAssetRequest request)

নতুন ফক্সড অ্যাসেট তৈরি করতে **POST** কল করে।

Future<FixedAsset> disposeAsset(int id)

একটি অ্যাসেটকে হিসাব থেকে বাদ দিতে (write off) **POST** কল করে; এটি পূর্বাবস্থায় ফেরানো যায় না।

Future<DepreciationRun> runDepreciation(int year, int month)

একবারে সব অ্যাসেটের এক মাসের অবচয় চালাতে **POST** কল করে; **year** ও **month** উভয়ই আবশ্যিক কৌশলে প্যারামিটার।

Future<List<DepreciationRun>> depreciationRuns()

এ পর্যন্ত চালানো সব ডেপ্রেসিয়েশন রানের তালিকা (বয়ের লস্ট) আনবে।

টপ-লেভেলে ভেরিবেল: closingRepositoryProvider

ClosingRepository-এর Provider।

টপ-লেভেলে ভেরিবেল: fiscalYearsProvider

currentUserProvider ওয়াচ করে ClosingRepository.fiscalYears() কল করার FutureProvider।

ক্লাস/কন্ট্রোলার: FiscalYearController

Notifier<int> — বর্তমানে UI-তে কোন অর্থবছর দেখানো হচ্ছে তা ধারণ করে, ডিফল্ট বর্তমান ক্যালেন্ডার বছর।

int build()

DateTime.now().year রটার্ন করে।

void set(int year)

নির্বাচনী বছর বদলায়, একই মান হলে কিছু করে না।

টপ-লভেলে ভরেয়িবেল: selectedFiscalYearProvider

FiscalYearController-এর NotifierProvider।

ক্লাস/কন্ট্রোলার: PeriodsController

AsyncNotifier<List<AccountingPeriod>> — নির্বাচতি অর্থবছরে পরিষিড তালিকার স্টেটে পরচালনা করে।

Future<List<AccountingPeriod>> build()

currentUserProvider ও selectedFiscalYearProvider ওয়াচ করে _load() কল করে।

Future<List<AccountingPeriod>> _load()

বর্তমান নির্বাচতি অর্থবছরে পরিষিড আনে।

Future<void> refresh()

_load() পুনরায় চালিয়ে স্টেটে রফ্রিশে করে।

void apply(AccountingPeriod updated)

লস্টে থাকা মলি যাওয়া id-এর পরিষিডকে আপডেটে করা ভার্সন দ্বি়ে প্রতস্থাপন করে — ক্লোজ/রওপনে-এর পর তাৎক্ষণিক U। আপডেটে করে জন্ম।

টপ-লভেলে ভরেয়িবেল: periodsProvider

PeriodsController-এর AsyncNotifierProvider।

ক্লাস/কন্ট্রোলার: FixedAssetsController

AsyncNotifier<List<FixedAsset>> — ফক্সিড অ্যাসটে রেজিস্টারের স্টেটে পরচালনা করে।

Future<List<FixedAsset>> build()

currentUserProvider ওয়াচ করে _load() কল করে।

Future<List<FixedAsset>> _load()

প্রথম পেজের ফক্সিড অ্যাসটে আনে (.content মাত্র নিয়ে)।

Future<void> refresh()

_load() পুনরায় চালিয়ে স্টেটে রফ্রিশে করে।

void apply(FixedAsset updated)

লস্টে মলি যাওয়া id-এর অ্যাসটে আপডেটে করা ভার্সন দ্বি়ে প্রতস্থাপন করে।

টপ-লভেলে ভরেয়িবেল: fixedAssetsProvider

FixedAssetsController-এর AsyncNotifierProvider।

টপ-লভেলে ভরেয়িবেল: depreciationRunsProvider

ClosingRepository.depreciationRuns() কল করা FutureProvider.autoDispose।

ফাইল: lib/features/assets_periods/assets_periods_screen.dart

সংক্ষিপ্ত ববিরণ: "মাস শেষ" (Month end) স্ক্রিনি — পরিষিড ক্লোজিং ও ফক্সিড অ্যাসটে ব্যবস্থাপনার জন্য দুটি ট্যাব একসাথে দেখায়, কারণ দুটোই কবেল মাস শেষে প্রয়োজন হয়। এতে পরিষিড খোলা/বন্ধ করা, বছর বন্ধ করা, অ্যাসটে যোগ করা, ডিসপোজ করা এবং মাসকি অবচয় চালানোর UI আছে।

ক্লাস/উইজটে: ClosingScreen

ConsumerWidget — মূল স্ক্রিনি উইজটে, পারমিশন অনুযায়ী "Periods" ও/অথবা "Fixed assets" ট্যাব দেখায়, প্রতটিরি নিজস্ব ফ্লোটিং অ্যাকশন বাটন (create) থাকতে পারে।

ClosingScreen({super.key, this.initialTabLabel})

কনস্ট্রাক্টর; **initialTabLabel** দিয়ে নির্দিষ্ট ট্যাব খুলে শুরু করা যায়।

Widget build(BuildContext context, WidgetRef ref)

পারমিশন লোড না হওয়া পর্যন্ত **Loader** দেখায়। পারমিশন অনুযায়ী উপলব্ধ ট্যাবের তালিকা তৈরি করে (**periodView** থাকলে **Periods** ট্যাব, **assetView** থাকলে **Fixed assets** ট্যাব, **assetManage** থাকলে তার সাথে create বাটন)। কোনো ট্যাব উপলব্ধ না থাকলে "Not available to you" এম্পটি স্টেটে দেখায়। অন্যথায় **DefaultTabController** ও **TabBar/TabBarView** সহ স্ক্র্যাফোল্ড তৈরি করে, যখনে সক্রিয় ট্যাব বদলালে ফ্লোটিং অ্যাকশন বাটনও বদলায় (**AnimatedBuilder** দিয়ে)।

ক্লাস/উইজটে: _PeriodsTab

ConsumerStatefulWidget — **Periods** ট্যাবের কন্টইনার।

ConsumerState<_PeriodsTab> createState()

_PeriodsTabState তৈরি করে।

ক্লাস: _PeriodsTabState

ConsumerState<_PeriodsTab> — পরিষিড টগল (ক্লোজ/রপ্তিপনে) ও বছর ক্লোজ করার লজিক এবং লোডিং স্টেটে (**_busyId**, **_closingYear**) ধারণ করে।

Future<void> _toggle(AccountingPeriod period)

পরিষিড বন্ধ করার সময় নশ্চিতিকরণ ডায়ালগ দেখায় (রপ্তিপনের সময় দেখায় না)। নশ্চিতি হলে **_busyId** সটে করে **closePeriod/reopenPeriod** কল করে, **periodsProvider.notifier.apply()** দিয়ে লিস্ট আপডেট করে এবং **fiscalYearsProvider** ইনভ্যালিডেট করে (কারণ বছরে খোলা/বন্ধ কাউন্ট বদলে গেছে)। সফলতা/ব্যর্থতায় সন্ধ্যাকবার দেখায়।

Future<void> _closeYear(FiscalYear year)

বছর বন্ধ করার আগে নশ্চিতিকরণ ডায়ালগ দেখায় (নেটে ইনকাম রোল করার সতর্কবার্তাসহ)। নশ্চিতি হলে **closeFiscalYear** কল করে, **fiscalYearsProvider** ইনভ্যালিডেট করে ও **periodsProvider.notifier.refresh()** কল করে।

Widget build(BuildContext context)

বর্তমান নির্বাচতি অর্থবছর, তার তথ্য (ক্লোজড/টোটাল পরিষিড, অগ্রগতি বার) এবং **canClose** পারমিশন থাকলে "Close year" বাটন হডোরে দেখিয়ে **ConfigList<AccountingPeriod>** রেন্ডার করে। প্রতটি পরিষিড সারতি (**ConfigRow**) স্ট্র্যাটাস, তারখি, ক্লোজড-বাই, রপ্তিপনেড ট্যাগ এবং ক্লোজ/রপ্তিপনে অ্যাকশন দেখায়।

ক্লাস/উইজটে: _AssetsTab

ConsumerStatefulWidget — Fixed assets ট্যাবের কন্ট্রোলার।

ConsumerState<_AssetsTab> createState()

_AssetsTabState তৈরি করে।

ক্লাস: _AssetsTabState

ConsumerState<_AssetsTab> — অ্যাসেটে ডিসপোজ ও মাসিক ডেপ্রেসিয়েশন রান করার লজিক এবং লোডিং স্টেটে (_busyId, _running) ধারণ করে।

Future<void> _dispose(FixedAsset asset)

অ্যাসেটের বুক ভ্যালু অনুযায়ী ভিন্ন সতর্কবার্তাসহ নশ্চিতিকরণ ডায়ালগ দেখায়। নশ্চিতি হলে disposeAsset কল করে, fixedAssetsProvider.notifier.apply() দিয়ে লিস্ট আপডেট করে ও সন্ধ্যাকবার দেখায়।

Future<void> _runDepreciation()

আগের মাসের জন্য (ডিফল্ট) নশ্চিতিকরণ ডায়ালগ দেখায়। নশ্চিতি হলে runDepreciation কল করে, fixedAssetsProvider.notifier.refresh() করে, depreciationRunsProvider ইনভ্যালিডিটে করে এবং কতগুলো অ্যাসেটে কত টাকা লখো হলো তা সন্ধ্যাকবারে দেখায়।

Widget build(BuildContext context)

canManage পারমিশন থাকলে হডোরের সর্বশেষ ডেপ্রেসিয়েশন রানের সারাংশ ও "Run monthly depreciation" বাটন দেখিয়ে ConfigList<FixedAsset> রেন্ডার করে, প্রতিটি আইটেমের জন্য _AssetCard ব্যবহার করে।

ক্লাস/উইজটে: _AssetCard

StatelessWidget — একটি ফিক্সড অ্যাসেটে কার্ড আকারে দেখায় — নাম, ট্যাগ/ক্যাটাগরি/অধিগ্রহণ তারিখ, বুক ভ্যালু, স্ট্যাটাস চপি, অবশ্য অগ্রগতিবার এবং বাকি মাস, প্লাস ডিসপোজ করার পপআপ মেনু (যদি অনুমোদিত হয়)।

_AssetCard({required this.asset, required this.busy, this.onDispose})

কনস্ট্রাক্টর।

Widget build(BuildContext context)

অ্যাসেটের নাম, সাবটাইটলে (ট্যাগ/ক্যাটাগরি/তারিখ), মূল্য, স্ট্যাটাস চপি, busy হলে স্পনির নাহলে (onDispose থাকলে) পপআপ মেনু রেন্ডার করে। ডিসপোজ না হলে অবশ্য অগ্রগতিবার ও "কত মাস বাকি" টেক্সট দেখায়।

টপ-লেভেলে ফাংশন: Future<void> showFixedAssetSheet(BuildContext context)

_FixedAssetSheet-কে স্ক্রল-কন্ট্রোলড মডাল বটম শিট হিসেবে খোলো।

ক্লাস/উইজটে: _FixedAssetSheet

ConsumerStatefulWidget — নতুন ফিক্সড অ্যাসেটে যোগ করার ফর্ম শিট।

ConsumerState<_FixedAssetSheet> createState()

_FixedAssetSheetState তৈরি করে।

ক্লাস: `_FixedAssetSheetState`

`ConsumerState<_FixedAssetSheet>` — ফর্মের সব টেক্সট কন্ট্রোলার, তারখি, লজোর-পোস্ট টগল এবং সাবমিটি লজিকি ধারণ করে।

`void dispose()`

সব `TextEditingController` (`_name`, `_tag`, `_category`, `_cost`, `_salvage`, `_life`, `_notes`) ডিসপোজ করে।

`FixedAssetRequest? get _request`

ফর্মের বর্তমান ইনপুট থেকে `FixedAssetRequest` তৈরিকরার চেষ্টা করে; `cost`, `life`, বা `acquired` পার্স করা না গেলে `null` রিটার্ন করে। লাইভ প্রভিডি (যেমন মাসকি অবচয়ের অনুমান) দেখাতে ব্যবহৃত হয়।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, `_request` তৈরিকরে (না পারলে এরর দেখায়); সাবমিটিং স্টেটে সেটে করে `createAsset` কল করে, `fixedAssetsProvider.notifier.refresh()` করে এবং সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়; ব্যর্থ হলে এরর মেসেজে দেখায়।

`Widget build(BuildContext context)`

`FormSheetFrame` ব্যবহার করে নাম, ট্যাগ, ক্যাটাগরি, খরচ, স্ক্রুয়াপ ভ্যালু, কার্যকর জীবনকাল (মাসে), অধগ্রহণ তারখি, "লজোর পোস্ট করুন" সুইচ এবং নোটসের ফর্ম ফিল্ড তৈরিকরে; সংশ্লিষ্ট ভ্যালিডিটরসহ।

ফাইল: `lib/features/attendance/assign_shift_sheet.dart`

সংক্ষিপ্ত বিবরণ: একজন কর্মীকে একটি শিফটে অ্যাসাইন করার জন্য বটম শিট UI প্রদান করে। এতে সক্রিয় শিফট তালিকা আনার একটি প্রোভাইডার এবং কর্মী নির্বাচন, শিফট নির্বাচন, শুরু/শেষ তারখি ও কারণসহ একটি ফর্ম শিট রয়েছে।

টপ-লেভেলে ভেরিয়েবল: `activeShiftsProvider`

`FutureProvider.autoDispose<List<Shift>>` — শিট খোলার সময়ই কেবল সক্রিয় শিফট তালিকা আনে (স্থায়ীভাবে রাখা হয় না, কারণ এটি একটি ছোট, কম-পরিবর্তনশীল তালিকা এবং পুরনো ডটো এমন শিফট দেখাতে পারে যা ইতিমধ্যে বাতলি হয়ে গেছে)।

টপ-লেভেলে ফাংশন: `Future<void> showAssignShiftSheet(BuildContext context)`

`_AssignShiftSheet`-কে স্ক্রল-কন্ট্রোলড মডাল বটম শিট হিসেবে খোলে।

ক্লাস/উইজটে: `_AssignShiftSheet`

`ConsumerStatefulWidget` — শিফট অ্যাসাইনমেন্ট শিটের কন্ট্রোলার।

`ConsumerState<_AssignShiftSheet> createState()`

`_AssignShiftSheetState` তৈরিকরে।

ক্লাস: `_AssignShiftSheetState`

`ConsumerState<_AssignShiftSheet>` — ফর্মের স্টেটে (নির্বাচতি কর্মী, শিফট, তারখিসীমা, কারণ) ও সাবমিটি লজিকি ধারণ করে।

`void dispose()`

`_reason` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _pickEmployee()`

`EmployeePicker.show()` খুলে নির্বাচন বস্তুকি `_employee` স্টেটে সেট করে।

`Future<void> _submit()`

কর্মী ও শফট নির্বাচন না হলে বা শেষে তারিখ শুরুর আগে হলে ভ্যালিডেশন এরর দেখায়। বৈধ হলে সাবমিট স্টেটে সেট করে `assignShift` কল করে `ShiftAssignmentRequest` পাঠায়; সফল হলে শিফট বন্ধ করে সন্ধ্যাকবার দেখায়। `ApiException` (যেমন ওভারল্যাপিং অ্যাসাইনমেন্ট প্রত্যাখ্যান হলে) সরাসরি সার্ভার-মসেজ দেখায়, অন্য এরর হলে জনেরেকি মসেজ দেখায়।

`Widget build(BuildContext context)`

`activeShiftsProvider` ওয়াচ করে; লোডিং/এরর/খালি অবস্থার জন্য যথাক্রমে `Loader`, `MessageBanner.error`, `MessageBanner.info` দেখায়। ডটো এলে কর্মী নির্বাচক (`InkWell` + `InputDecorator`), শফট ড্রপডাউন, শুরুর শেষে `DateField`, কারণ `TextField` এবং সাবমিট `LoadingButton` রেন্ডার করে।

ফাইল: `lib/features/attendance/attendance_admin_repository.dart`

সংক্ষিপ্ত ববিরণ: প্রশাসকের দৃষ্টিকোণ থেকে অন্যদের অ্যাটেন্ডেন্স দেখা, সংশোধন করা এবং রিপোর্ট করার জন্য পৃথক রিপোজিটরি — নিজের অ্যাটেন্ডেন্স-সংক্রান্ত রিপোজিটরি থেকে আলাদা রাখা হয়েছে, কারণ দুটোর পারমিশন-সেট আলাদা। এতে রেকর্ড, ম্যানুয়াল এন্ট্রি, ব্যাকফলি, রিপোর্ট, শফট অ্যাসাইনমেন্ট ও টাইমশিট সংক্রান্ত মেথড এবং সংশ্লিষ্ট `Riverpod` কন্ট্রোলার রয়েছে।

ক্লাস: `AttendanceAdminRepository`

টমিরে অ্যাটেন্ডেন্স পড়া, সংশোধন করা ও রিপোর্ট করার সব API কল কন্ট্রোলিত করে।

`AttendanceAdminRepository(this._api)`

`ApiClient` ইনজেক্ট করে কনস্ট্রাক্ট করে।

`Future<PagedResponse<AttendanceRecord>> records({String? status, String? date, String? search, int page = 0, int size = 30})`

সবার অ্যাটেন্ডেন্স রেকর্ড আনবে, `status`, `date`, `search` — একাধিক ফিল্টার একসাথে কোয়েরি প্যারামিটার হিসেবে প্রয়োগ করা যায় (এই কৌশল অস্বাভাবিক)।

`Future<PagedResponse<AttendanceRecord>> recordsFor(int employeeId, {String? date, int page = 0, int size = 30})`

নির্দিষ্ট এক কর্মীর রেকর্ড আনবে, ঐচ্ছিকভাবে একটি নির্দিষ্ট দিনের জন্য ফিল্টার করা।

`Future<AttendanceRecord> createManual(ManualAttendanceRequest request)`

হাতে অ্যাটেন্ডেন্স রেকর্ড তৈরি করে — মসিড চকে-ইন, টার্মিনাল ডাউন থাকা, বা পরে লভি হিসেবে চিহ্নিত করার জন্য।

Future<void> approveRecord(int id)

একটরিকের্ড অনুমোদন করতে PATCH কল করে; রসেপন্স খালি বিল করে কলার নজি পুনরায় পড়ে নেয়।

Future<int> backfillAbsentees(String startDate, String endDate)

নাইটলমারকার মসি করা ABSENT দিনগুলো পূরণ করে (কোম্পানি-মালিক-মাত্র)। উভয় তারখিই আবশ্যক কোয়েরেপ্যারামটির। ব্যাকএন্ড স্প্যান সর্বোচ্চ ৩৬৬ দিনে সীমাবদ্ধ রাখতে, ভবিষ্যতের তারখি প্রক্রিয়া করে না, এবং একই রেঞ্জের দুইবার চালানো নিরাপদ (idempotent)। JSON রসেপন্সে created বা count — উভয় কী চকে করে, কারণ ডকুমেন্টে না এবং আগের আকার বদলেছে।

Future<DailyAttendanceReport> dailyReport({String? date})

একদিনের কোম্পানি-ব্যাপী রপোর্ট আনতে; তারখি না দিলে ব্যাকএন্ড আজকের দিন ব্যবহার করে।

Future<MonthlyAttendanceReport> monthlyReport(int month, int year)

এক মাসের রপোর্ট আনতে; উভয় প্যারামটির আবশ্যক।

Future<EmployeeAttendanceSummary> employeeSummary(int employeeId, String startDate, String endDate)

একজন কর্মীর নির্দিষ্ট সময়সীমার সারাংশ আনতে; উভয় তারখিই আবশ্যক।

Future<({int lateCount, int absentCount})> lateAndAbsent({String? date})

নির্দিষ্ট দিনে কতজন দেরি করেছেন ও কতজন অনুপস্থিতি ছিলেন তা একটরিকের্ড টাইপ হিসেবে ফেরত দেয়।

Future<PagedResponse<ShiftAssignment>> assignmentsForShift(int shiftId, {int page = 0, int size = 30})

একটি নির্দিষ্ট শফিটে থাকা সবার অ্যাসাইনমেন্ট আনতে।

Future<ShiftAssignment> updateAssignment(int id, ShiftAssignmentRequest request)

একটি অ্যাসাইনমেন্ট পরিবর্তন করে (ভিন্ন শফিট বা ভিন্ন তারখি)।

Future<void> endAssignment(int id)

অ্যাসাইনমেন্ট রেকর্ড মুছে না ফলে বরং এখনই শেষ হিসেবে চিহ্নিত করে — বড ছাড়া PUT কল, খালি রসেপন্স।

Future<void> deleteAssignment(int id)

একটি অ্যাসাইনমেন্ট সম্পূর্ণ মুছে ফলে।

Future<List<Timesheet>> timesheetRange(int employeeId, String from, String to)

একজন কর্মীর নির্দিষ্ট রেঞ্জের টাইমশিট (বয়ের লস্ট) আনতে।

Future<Timesheet> approveTimesheet(int id)

একটি টাইমশিট অনুমোদন করতে PATCH কল করে।

টপ-লেভেলে ভরেয়িবেল: attendanceAdminRepositoryProvider

AttendanceAdminRepository-এর Provider।

ক্লাস/কন্ট্রোলার: AttendanceDayController

Notifier<DateTime> — টমি-ভডি কোন দিনের অ্যাটনেডেন্স দেখাচ্ছে তা ধারণ করে, ডফিল্ট আজকের দিন।

`DateTime build()`

`DateTime.now()` রটাইন করে।

`void set(DateTime day)`

নর্বাচতি দিন বদলায় (সময় অংশ বাদ দিয়ে শুধু তারিখ রাখতে)।

`void shift(int days)`

নর্দিষ্ট দিন-সংখ্যা যোগ/বিয়োগ করে সরে যায়, তবে আজকের চেয়ে ভবিষ্যতে সরে গেলে তা প্রত্যাখ্যান করে — ভবিষ্যতের দিনের অ্যাটেন্ডেন্স অস্বাভাবিক বলবে।

উপ-লেভেলে ভরেয়েবেল: attendanceDayProvider

AttendanceDayController-এর NotifierProvider।

ক্লাস/কন্ট্রোলার: TeamAttendanceSearchController

`Notifier<String>` — টিম লিস্ট কোন সার্চ টার্ম দিয়ে সংকীর্ণ করা হয়েছে তা ধারণ করে; খালি মানে সবাই।

`String build()`

খালি স্ট্রিং রটাইন করে।

`void set(String value)`

ট্রিম করা মান একই না হলে স্টেটে আপডেট করে।

উপ-লেভেলে ভরেয়েবেল: teamAttendanceSearchProvider

TeamAttendanceSearchController-এর NotifierProvider।

ক্লাস/কন্ট্রোলার: TeamAttendanceController

`AsyncNotifier<List<AttendanceRecord>>` — বর্তমানে দেখানো দিনের সবার অ্যাটেন্ডেন্স রেকর্ড পরচালনা করে।

`Future<List<AttendanceRecord>> build()`

`currentUserProvider`, `attendanceDayProvider`, `teamAttendanceSearchProvider` ওয়াচ করে `_load()` কল করে।

`Future<List<AttendanceRecord>> _load()`

বর্তমান দিন ও সার্চ টার্ম ব্যবহার করে রেকর্ড আনবে।

`Future<void> refresh()`

`_load()` পুনরায় চালিয়ে স্টেটে রিফ্রেশ করে।

`void apply(AttendanceRecord updated)`

লিস্টে মিলে যাওয়া id-এর রেকর্ড আপডেট করা ভার্সন দিয়ে প্রতস্থাপন করে।

উপ-লেভেলে ভরেয়েবেল: teamAttendanceProvider

TeamAttendanceController-এর AsyncNotifierProvider।

উপ-লেভেলে ভরেয়েবেল: dailyReportProvider

বর্তমান দিনের `dailyReport()` কল করা `FutureProvider.autoDispose<DailyAttendanceReport>`।

টপ-লেভেলে ভরেয়িবেল: `monthlyReportProvider`

বর্তমান দিনি যে মাসে পড়ে তার `monthlyReport()` কল করা
`FutureProvider.autoDispose<MonthlyAttendanceReport> |`

টপ-লেভেলে ভরেয়িবেল: `lateAndAbsentProvider`

বর্তমান দিনের `lateAndAbsent()` কল করা `FutureProvider.autoDispose<({int lateCount, int absentCount})> |`

টপ-লেভেলে ফাংশন: `String _iso(DateTime day)`

একটি `DateTime`-কে `yyyy-MM-dd` ফরম্যাটে রূপান্তর করে — এটি বিয়াকএন্ডরে
`@DateTimeFormat(ISO.DATE)`-এর প্রত্যাশিত ফরম্যাট।

ফাইল: `lib/features/attendance/attendance_controller.dart`

সংক্ষিপ্ত বিবরণ: নজিরে অ্যাটেন্ডেন্স (আজকের রেকর্ড, মাসিকি সামারি, ইতিহাস) এবং টাইমশিট
ম্যানেজ করার জন্য Riverpod `AsyncNotifier` কন্ট্রোলার। চকে-ইন/চকে-আউট, রফ্রেশ, এবং একই
দিনের একাধিক রেকর্ডকে একটিকে মার্জ করার লজিকি এখানে আছে।

ক্লাস: `AttendanceState`

`@immutable` স্টেটে ক্লাস যা আজকের রেকর্ড (`today`), মাসিকি সামারি (`summary`), সাম্প্রতিক
দিনগুলোর ইতিহাস (`history`), এবং একটি `busy` ফ্ল্যাগ ধারণ করে — এই `busy` ফ্ল্যাগটি পিএইচ-ইন/আউট
চলাকালীন ব্যবহৃত হয়, যাতে বাইরের `AsyncValue`-এর লোডিং স্টেটে থেকে আলাদাভাবে বোঝা যায় যে
স্ক্রিনি ব্ল্যাংক না করাই একটি পিএইচ চলছে।

`AttendanceState({today, summary, history = const [], busy = false})`

কনস্ট্রাক্টর — ডিফল্টভাবে খালি ইতিহাস ও `busy = false` সহ একটি স্টেটে তৈরি করে।

`bool get canCheckIn`

`true` হয় যদি আজকের কোনো রেকর্ড না থাকে অথবা থাকলেও এখনো চকে-ইন করা না হয়ে থাকে।

`bool get canCheckOut`

`true` হয় যদি আজকের রেকর্ড থাকে, চকে-ইন করা হয়ে থাকে, কিন্তু চকে-আউট এখনো করা না হয়।

`AttendanceState copyWith({today, clearToday = false, summary, history, busy})`

একটি নতুন স্টেটে তৈরি করে, বর্তমান ফিল্ডগুলো বজায় রেখে যেগুলো নতুনভাবে দেওয়া হয়নি
`clearToday true` দিলে `today`-কে জোর করে `null` করে দেয়, এমনকি নতুন `today` প্যারামিটার দেওয়া
থাকলেও।

ক্লাস: `AttendanceController`

`AsyncNotifier<AttendanceState>` — সাইন-ইন করা ব্যবহারকারীর নজিরে অ্যাটেন্ডেন্স ডটো লোড,
রফ্রেশ এবং চকে-ইন/আউট পরিচালনা করে।

`Future<AttendanceState> build()`

`currentUserProvider` ওয়াচ করে (যাতে দ্বিতীয় ব্যবহারকারী সাইন-ইন করলে আগের ব্যবহারকারীর
অ্যাটেন্ডেন্স ডটো উত্তরাধিকার সূত্রে না পায়) এবং `_load()` কল করে প্রাথমিক স্টেটে তৈরি করে।

`Future<AttendanceState> _load()`

তনিটি API কল (আজকের রকের্ড, মাসকি সামারি, ৩০টি ইতিহাস রকের্ড) একসাথে শুরু করে (sequential await না করে) যাতে মোবাইল কানেকশনে স্পনির কম সময় দেখাতে হয়। প্রতিটি Future-এ `.ignore()` কল করা হয় শুরুতেই, কারণ প্রথম await ব্যর্থ হলে ফাংশন রিটার্ন করে ফলেবে, আর বাকি দুটি Future-এর ফলাফল কটে await না করলে সেগুলো unhandled exception হিসেবে রিপোর্ট হতে পারে। ইতিহাস `_consolidate()` দিয়ে প্রসেস করা হয়।

`Future<void> refresh()`

`_load()` কে `AsyncValue.guard` দিয়ে মুড়ে স্টেটে আপডেট করে — এরর হলে তা `AsyncValue.error` হিসেবে ধরা পড়ে।

`Future<void> checkIn({String? notes, String? location})`

`_punch()` কে রিপোজিটরির `checkIn` মেথড দিয়ে কল করে।

`Future<void> checkOut({String? location})`

বর্তমান স্টেটে `today` রকের্ড না থাকলে `StateError` থ্রো করে (চেক-ইন ছাড়া চেক-আউট করা যায় না)। থাকলে `_punch()`-কে রিপোজিটরির `checkOut(record.id, ...)` দিয়ে কল করে।

`Future<void> _punch(Future<AttendanceRecord> Function(AssessmentRepository) action)`

একটি পাঞ্চ (চেক-ইন/আউট) চালায়, তারপর মাসকি সামারি ও ইতিহাস পুনরায় ফেচ করে। প্রথম স্টেটকে `busy: true` করে। পাঞ্চ সফল হলে নতুন `today`, `summary`, `history` দিয়ে স্টেটে আপডেট হয় (রিটার্নকৃত রকের্ডটি যিভাবে এসছে সেভাবেই ব্যবহার করা হয়, কারণ লটে/ওভারটাইম ইত্যাদি সার্ভার নরিধারণ করে, ফোন থেকে নয়)। ব্যর্থ হলে `busy` ফ্ল্যাগ ফরিয়ে `false` করে আগের ডটো রেখে দেয় (এরর স্টেটে না গিয়ে) এবং এররটি `rethrow` করে।

`static List<AttendanceRecord> _consolidate(List<AttendanceRecord> records)`

একই দিনের একাধিক রকের্ডকে একটি রো-তে মার্জ করে (কারণ নাইটল শিডিউলার একটি ABSENT প্লসেহোল্ডার লিখে রাখতে, আর পরে একটি প্রকৃত পাঞ্চ যোগ হয়)। `employeeId_attendanceDate` কী ব্যবহার করে গ্রুপ করে, `mergedWith()` দিয়ে মার্জ করে, তারপর তারখি অনুসারে নতুন থেকে পুরনো ক্রমে সাজায়।

টপ-লেভেলে ফাংশন/প্রোভাইডার

`final attendanceControllerProvider`

`AsyncNotifierProvider<AttendanceController, AttendanceState> |`

`final myShiftAssignmentProvider`

`FutureProvider<ShiftAssignment?>` — সাইন-ইন করা কর্মীর বর্তমান শফিট রিটার্ন করে; কর্মীর কোনো এমপ্লয়ীর রকের্ড না থাকলে (যেমন COMPANY_OWNER) বা শফিট না থাকলে `null`।

ক্লাস: TimesheetsController

`AsyncNotifier<PagedState<Timesheet>>` (`PagedLoader<Timesheet>` মক্সিনি সহ) — নিজের টাইমশিট পজেনিটেডে লস্ট, লগিং, আপডেট, ডিলিট এবং রিভিউ-এর জন্য সাবমিশন পরিচালনা করে।

`Future<PagedState<Timesheet>> build()`

`currentUserProvider` ওয়াচ করে এবং `loadFirstPage()` কল করে।

Future<PagedResponse<Timesheet>> fetchPage(int page)

PagedLoader মক্সিনিরে জন্য প্রয়োজনীয় override — রপিয়োজটিররি myTimesheets(page: page) কল করে।

Future<void> log({required DateTime workDate, required double hoursWorked, double? billableHours, String? projectName, String? description})

রপিয়োজটিররি logTimesheet কল করে নতুন এন্ট্রি লিগ করে, তারপর refresh() কল করে।

Future<void> updateItem(int id, {required DateTime workDate, required double hoursWorked, double? billableHours, String? projectName, String? description})

রপিয়োজটিররি updateTimesheet কল করে, তারপর replaceItem() দিয়ে লিস্টে থাকা আইটেমটি জায়গায় বদলে দেয় (আইটেমের অবস্থান পরিবর্তন হয় না, কারণ প্রতিটি এমপ্লয়ীর জন্য workDate ইউনিক)।

Future<void> delete(int id)

রপিয়োজটিররি deleteTimesheet কল করে এবং removeItem() দিয়ে লিস্ট থেকে সরায়।

Future<int> submitForReview()

সব "not-yet-submitted" এন্ট্রি একসাথে সাবমিট করে; কতগুলো এন্ট্রি সাবমিটেডে হলো তার সংখ্যা রিটার্ন করে (কনফার্মেশন মসেজে শব্দ চয়নের জন্য)। কাউন্ট শূন্যের বশেই হলে refresh() কল করে।

টপ-লেভেলে প্রোভাইডার

final timesheetsProvider

AsyncNotifierProvider<TimesheetsController, PagedState<Timesheet>> |

ফাইল: lib/features/attendance/attendance_models.dart

সংক্ষিপ্ত ববিরণ: অ্যাটেন্ডেন্স, টাইমশিট, শফিট অ্যাসাইনমেন্ট, এবং অ্যাডমনি রপিয়োর্টের জন্য সমস্ত ডটো মডেলে, স্ট্যাটাস কনস্ট্যান্ট এবং পারমিশন কোড। কোনো UI বা নটেওয়ার্ক লজিক নই — শুধু JSON প্রারসং/সিরিয়লাইজেশন ও ডেরাইভড গটোর।

ক্লাস: AttendanceStatus (abstract final)

দিনের স্ট্যাটাসের জন্য প্লেইন স্ট্রিং কনস্ট্যান্ট (Dart enum না — কারণ সোর্স অফ ট্রুথ হলো ব্যাকএন্ডের Java enum, ফলে অজানা ভ্যালু এলও ক্র্যাশ না করে রেন্ডার করা যায়)।

স্ট্যাটিক কনস্ট্যান্ট: present, late, absent, onLeave, halfDay, workFromHome, weekend, holiday, partialDay, unmarked

প্রতিটি একটি নির্দিষ্ট স্ট্যাটাস স্ট্রিং ধারণ করে (যেমন 'PRESENT', 'LATE')।

static const manuallySettable

যসেব স্ট্যাটাস মানুষ হাতে রেকর্ড করতে পারে তাদের তালিকা (present, late, absent, onLeave, halfDay, workFromHome) — WEEKEND/HOLIDAY/PARTIAL_DAY/UNMARKED সিস্টেমে নির্ধারিত বলে বাদ।

ক্লাস: AttendanceRecord

একটি নির্দিষ্ট দিনের একজন কর্মীর অ্যাটেন্ডেন্স রেকর্ড ধারণ করে — চকে-ইন/আউট সময়, লেট/ওভারটাইম তথ্য, লোকেশন ইত্যাদি।

`bool get isCheckedIn`

`checkInTime` নন-নাল ও নন-এম্পটি হলে `true`।

`bool get isCheckedOut`

`checkOutTime` নন-নাল ও নন-এম্পটি হলে `true`।

`bool get isComplete`

`isCheckedIn && isCheckedOut` — উভয় স্ট্যাম্প থাকলে দনিটি সম্পূর্ণ।

`factory AttendanceRecord.fromJson(Map<String, dynamic> json)`

JSON থেকে অবজেক্ট তৈরি করে; প্রতিটি ফিল্ডের জন্য নাল-সফে ডিফল্ট মান রাখা (যেমন `id` না থাকলে `0`, `status` না থাকলে `unmarked`)।

`AttendanceRecord mergedWith(AttendanceRecord other)`

`this`-কে `other` দিয়ে মার্জ করে একটি নতুন রেকর্ড রিটার্ন করে — একই দিনের দুটি রেকর্ড (যেমন প্লসেহোল্ডার ও প্রকৃত পাঞ্চে) একত্রিত করার জন্য ব্যবহৃত। `status: this` ABSENT হলে এবং `other` ABSENT না হলে `other`-এর স্ট্যাটাস জেতে (প্লসেহোল্ডার বাতিল)। বাকি ফিল্ডগুলো `null-coalescing` বা OR লজিক দিয়ে মার্জ হয়; `lateMinutes` শুধু `other.isLate` হলে `other`-এরটা নেয়।

ক্লাস: MonthlySummary

একটি মাসের অ্যাটেন্ডেন্স সংক্ষিপ্তসার — প্রজেন্ট/অ্যাবসেন্ট/হাফ-ডে/লভি/হলডি/উইকঅফ দিনের সংখ্যা এবং কাজ করা ঘণ্টা।

`double? get attendancePercent`

রেকর্ডকৃত ওয়ার্কিং ডে-র মধ্যে উপস্থিতির শতাংশ রিটার্ন করে; কোনো ওয়ার্কিং ডে রেকর্ড না থাকলে `null` (০% না, কারণ ফাঁকা মাসকে ০% উপস্থিতি বিলা একটি ছোট মথিয়া)।

`factory MonthlySummary.fromJson(Map<String, dynamic> json)`

JSON থেকে তৈরি করে; `year/month` না থাকলে বর্তমান তারিখ থেকে ডিফল্ট নেয়, বাকি সংখ্যাসূচক ফিল্ড `0`।

ক্লাস: TimesheetStatus (abstract final)

টাইমশিটে স্ট্যাটাস কনস্ট্যান্ট: `notSubmitted`, `submitted`, `approved` — এগুলো সার্ভার-সাইডে `submitted/approved` বুলিয়ান থেকে ডেরাইভ হয়।

ক্লাস: Timesheet

একটি টাইমশিট এন্ট্রি — কাজের তারিখ, কাজ করা ঘণ্টা, বলিযোগ্য ঘণ্টা, প্রজেক্ট নাম, বিবরণ ইত্যাদি।

`bool get isEditable`

`status == TimesheetStatus.notSubmitted` হলেই `true` — শুধু ড্রাফট অবস্থায় থাকা এন্ট্রি পরিবর্তনযোগ্য।

`factory Timesheet.fromJson(Map<String, dynamic> json)`

JSON থেকে অবজেক্ট তৈরি করে, নাল-সফে ডিফল্ট সহ।

ক্লাস: ShiftAssignment

একজন কর্মী বর্তমানে যে শফিটে আছে তার রডি-অনলভিউ (শফিট অ্যাসাইন/এন্ড/রঅ্যাসাইন করা ওয়বে হয়)।

factory ShiftAssignment.fromJson(Map<String, dynamic> json)

JSON থেকে তৈরি করে; shiftName না থাকলে 'Unnamed shift' ডফিল্ট।

ক্লাস: AttendancePermissions (abstract final)

শফিট অ্যাসাইনমেন্ট সংক্রান্ত পারমিশন কোড কনস্ট্যান্ট: shiftAssignmentView, shiftAssignmentCreate, shiftAssignmentUpdate, shiftAssignmentDelete, shiftView।

ক্লাস: Shift

কোম্পানির একটি শফিট, অ্যাসাইনমেন্ট পকারের জন্য প্রয়োজনীয় ন্যূনতম তথ্য সহ (ইচ্ছাকৃতভাবে পাতলা — গ্রসে পরিষিড, নাইট-শফিট ফ্ল্যাগ ইত্যাদি বাদ)।

String get hoursLabel

"09:00 – 17:30" ফরম্যাটে সময় রটার্ন করে; startTime/endTime না থাকলে শুধু নাম রটার্ন করে।

static String _hhmm(String time)

HH:mm:ss স্ট্রিং থেকে সেকেন্ড বাদ দিয়ে HH:mm অংশ রটার্ন করে।

factory Shift.fromJson(Map<String, dynamic> json)

JSON থেকে অবজেক্ট তৈরি করে, নাল-সফে ডফিল্ট সহ।

ক্লাস: ShiftAssignmentRequest

POST /hrm/attendance/shift-assignments-এর রকি়োস্ট বডি — কর্মী ও শফিট আবশ্যক, তারিখ ও কারণ ঐচ্ছিক।

Map<String, dynamic> toJson()

রকি়োস্ট বডি তৈরি করে; ভতিররে clean() হলেপার ফাংশন ট্রমি করে খালি স্ট্রিং-কে null করে দেয়, শুধু নন-নাল/নন-এম্পটি ফিল্ড JSON-এ যোগ করে।

ক্লাস: AttendanceAdminPermissions (abstract final)

অন্যদের অ্যাটনেডেন্স দখো/সংশোধনের পারমিশন কোড: view, mark, update, approve, timesheetView, timesheetApprove।

ক্লাস: DailyAttendanceReport

পুরো কোম্পানির একদিনের অ্যাটনেডেন্স রপি়োর্ট।

int get unaccounted

মোট এমপ্লয়িং সংখ্যা থেকে present+late+absent+onLeave বাদ দিয়ে হিসাবের বাইরে থাকা সংখ্যা রটার্ন করে (ফলাফল ঋণাত্মক হলে 0)।

factory DailyAttendanceReport.fromJson(Map<String, dynamic> json)

JSON থেকে অবজেক্ট তৈরি করে, নাল-সফে ডফিল্ট সহ।

ক্লাস: MonthlyAttendanceReport

পুরো কোম্পানির এক মাসের অ্যাটেন্ডেন্স রিপোর্ট।

factory MonthlyAttendanceReport.fromJson(Map<String, dynamic> json)

JSON থেকে অবজেক্ট তৈরি করে, নাল-সেফ ডিফল্ট সহ।

ক্লাস: EmployeeAttendanceSummary

একজন কর্মীর নির্দিষ্ট সময়সীমার অ্যাটেন্ডেন্স সংক্ষিপ্তসার।

factory EmployeeAttendanceSummary.fromJson(Map<String, dynamic> json)

JSON থেকে অবজেক্ট তৈরি করে, নাল-সেফ ডিফল্ট সহ।

ক্লাস: ManualAttendanceRequest

POST /company/attendance/manual-এর রিকোয়েস্ট বডি — HR কর্তৃক ম্যানুয়াল অ্যাটেন্ডেন্স সংশোধনের জন্য (যেমন কটে-ইন করতে ভুলে গেলে)।

Map<String, dynamic> toJson()

রিকোয়েস্ট বডি তৈরি করে। isLate, lateMinutes, isOvertime সবসময় পাঠানো হয় (DTO-তে প্রমিটিভ)। checkInTime/checkOutTime থাকলে সংশ্লিষ্ট মথড ফলিডও যোগ হয়। ভিতরের clean() হলেপার খালি স্ট্রিং ফলিডকে বাদ দেয়।

ফাইল: lib/features/attendance/attendance_repository.dart

সংক্ষিপ্ত বিবরণ: নিজের অ্যাটেন্ডেন্স, টাইমশিট এবং শফিট অ্যাসাইনমেন্ট (রেডি-অনলি) সংক্রান্ত সব HTTP কল এই ক্লাসে কেন্দ্রীভূত। ApiClient রূপ করে ডোমেইন-নির্দিষ্ট মথড এক্সপোজ করে।

ক্লাস: AttendanceRepository

সার্ভারের /company/attendance, /hr/timesheets, এবং /hrm/attendance/shift-assignments এন্ডপয়েন্টের সাথে যোগাযোগ করে।

Future<AttendanceRecord?> myToday()

আজকের রেকর্ড আনবে, না থাকলে null রিটার্ন করে। ব্যাকএন্ড খালি বিডসিহ ২০০ রিটার্ন করে যখন এখনো পাঞ্চ করা হয়নি — তাই Map না হলে বা খালি হলে null ধরা হয়। ApiException ধরে isNotFound, isForbidden (কোনো এমপ্লয়ীর রেকর্ড না থাকলে ৪০৩ আসতে পারে), বা স্ট্যাটাস কোড ২০৪ হলেও null রিটার্ন করে; অন্য এরর rethrow হয়।

Future<PagedResponse<AttendanceRecord>> myRecords({int page = 0, int size = 20})

\$_base/my থেকে পজেনিটেড অ্যাটেন্ডেন্স রেকর্ড আনবে।

Future<MonthlySummary> myMonthlySummary({int? year, int? month})

\$_base/my/monthly-summary থেকে নির্দিষ্ট (বা বর্তমান) মাসের সামারি আনবে।

`Future<AttendanceRecord> checkIn({String? notes, String? location})`

আজকরে দনি খোলে। `checkInTime` স্থানীয় ওয়াল-ক্লক (`HH:mm:ss`, `Fmt.wallClockNow()`) হিসেবে পাঠানো হয় — ইউটিসি ইনস্ট্যান্ট না, কারণ সার্ভার এটাকে শফিটের স্টার্ট টাইমের (যা নজিও লোকাল ওয়াল-ক্লক) সাথে তুলনা করে।

`Future<AttendanceRecord> checkOut(int id, {String? location})`

নরিদষ্টি রকের্ড আইডি জন্য চকে-আউট রকের্ড করে, `checkOutTime` হিসেবে বর্তমান ওয়াল-ক্লক পাঠায়।

`Future<PagedResponse<Timesheet>> myTimesheets({int page = 0, int size = 31})`

`$_timesheets/my` থেকে নজিরে টাইমশিট পজেনিটেডে আকারে আনে।

`Future<Timesheet> logTimesheet({required DateTime workDate, required double hoursWorked, double? billableHours, String? projectName, String? description})`

নতুন টাইমশিট এন্ট্রি লিগ করে।

`Future<String> composeTimesheetEntry({String? projectName, required String roughNotes})`

রুক্ষ নোট থেকে AI-জনোরটেডে টাইমশিট বিবরণ ড্রাফট করে (`/ai-compose` এন্ডপয়েন্ট); রসেপন্সে সরাসরি কিছু লিখো হয় না, শুধু ব্যবহারকারীর সম্পাদনার জন্য টেক্সট রিটার্ন করে। রসেপন্সে `taskDescription` বা `description` — উভয় কী থেকেই পড়ার চেষ্টা করে।

`Future<Timesheet> updateTimesheet(int id, {required DateTime workDate, required double hoursWorked, double? billableHours, String? projectName, String? description})`

সাবমিট না হওয়া এন্ট্রি সংশোধন করে (`Timesheet.isEditable`-এর একই নিয়ম ব্যাকএন্ডেও প্রযোজ্য)। `workDate` ও `hoursWorked` সবসময় পাঠানো হয়; বাকিগুলো স্পার্স প্যাচ — খালি করলে ক্লয়ার হয় না, কারণ সার্ভিস শুধু নন-নাল মান অ্যাসাইন করে।

`Future<void> deleteTimesheet(int id)`

নরিদষ্টি টাইমশিট এন্ট্রি ডিলিট করে।

`Future<int> submitTimesheetsForReview()`

সব "not-yet-submitted, not-yet-approved" এন্ট্রি বি্যাচে সাবমিট করে (প্রতি-এন্ট্রি সাবমিট এন্ডপয়েন্ট নই); কতগুলো মুভ হয়েছে তা রিটার্ন করে।

`Future<List<Shift>> activeShifts()`

`/hr/shifts/active` থেকে সক্রিয় শফিটের পুরো তালিকা আনে (পজেনিটেডে না) — শফিট অ্যাসাইনমেন্ট পকিয়ারে জন্য।

`Future<ShiftAssignment> assignShift(ShiftAssignmentRequest request)`

একজন কর্মীকে একটাই শফিটে অ্যাসাইন করে।

`Future<ShiftAssignment?> shiftAssignmentFor(int employeeId)`

নরিদষ্টি কর্মীর বর্তমান শফিট অ্যাসাইনমেন্ট আনে; না পাওয়া গেলে (`isNotFound`) `null` রিটার্ন করে, অন্য এরর `rethrow` হয়।

টপ-লভেলে প্রোভাইডার

final attendanceRepositoryProvider

Provider<AttendanceRepository> — apiClientProvider থেকে ApiClient নিয়ে রপিজিটরি ইনস্ট্যান্স তৈরিকরে।

ফাইল: lib/features/attendance/attendance_screen.dart

সংক্ষিপ্ত ববিরণ: নিজের অ্যাটনেডেন্স দেখার মূল স্ক্রিনি, দুটি ট্যাবে বিভক্ত — Overview (পাঞ্জ কার্ড, শফিট, মাসকি সামারি, ইতহাস) এবং Timesheets।

ক্লাস/উইজটে: AttendanceScreen

ConsumerWidget — অ্যাপবার সহ দুই-ট্যাবের স্ক্রিনি। যাদের shiftAssignmentCreate পারমিশন আছে তাদের জন্য অ্যাপবারে "শফিট অ্যাসাইন" বাটন দেখায় (এটি একটি HR অ্যাকশন বলে ট্যাবের বদলে অ্যাপবারে রাখা হয়েছে)।

Widget build(BuildContext context, WidgetRef ref)

permissionControllerProvider থেকে canAssignShift চেক করে, initialTabLabel থেকে প্রাথমিক ট্যাব ইনডেক্স নির্ণয় করে (ক্ল্যাম্প করা), তারপর DefaultTabController সহ Scaffold রটাইন্ করবে যখনে TabBarView-এ _OverviewTab ও TimesheetsTab আছে।

ক্লাস/উইজটে: _OverviewTab

ConsumerWidget — পাঞ্জ কার্ড, শফিট কার্ড (থাকলে), মাসকি সামারি এবং সাম্প্রতিক দিনগুলোর ইতহাস দেখায়, পুল-টু-রিফ্রেশ সহ।

Widget build(BuildContext context, WidgetRef ref)

attendanceControllerProvider ও myShiftAssignmentProvider ওয়াচ করে। শফিট প্রোভাইডারে .value ব্যবহার করে (পূর্ণ .when() না) — কারণ শফিট কার্ড সাপ্লিমেন্টারি থ্য, এর জন্য আলাদা এরর ব্যানার দরকার নাই। RefreshIndicator-এর onRefresh উভয় প্রোভাইডার ইনভ্যালিডেট/রিফ্রেশ করে। ListView-এ PunchCard, শর্তসাপেক্ষ _ShiftCard, _MonthSummary (সামারি থাকলে), এবং _History সাজায়।

ক্লাস/উইজটে: _ShiftCard

StatelessWidget — কর্মীর বর্তমান শফিটের নাম ও শুরুর তারখি দেখানো একটি ছোট কার্ড।

Widget build(BuildContext context)

একটি আইকন, শফিটের নাম, এবং (থাকলে) "Since <date>" টেক্সট সহ একটি AppCard রো তৈরিকরে।

ক্লাস/উইজটে: _MonthSummary

StatelessWidget — মাসের নাম, কাজ করা ঘণ্টা, উপস্থিতির শতাংশ, এবং একটি ব্রকেডাউন গ্রুপি (প্রজেন্ট/অ্যাবসেন্ট/হাফ-ডে/লভি/হলডি/উইকঅফ) দেখায়।

Widget build(BuildContext context)

summary.attendancePercent null হলে ড্যাশ দেখায়, নাহলে ১ দশমিক স্থান পর্যন্ত পার্সেন্টেজে ফর্ম্যাট করে। rows লিস্টে প্রতিটি বিভাগের লবেলে, মান ও রং জোড়া তৈরি করে, তারপর Wrap উইজেটে প্রতিটি রো দেখায় (প্রস্থ স্ক্রিনি সাইজের ওপর ভিত্তি করে গণনা করা)।

ক্লাস/উইজেট: _History

ConsumerWidget — অ্যাটেন্ডেন্স স্টেটে AsyncValue নিয়ে লোডিং/এরর/ডটো/খালি স্টেটে রেন্ডার করে।

Widget build(BuildContext context, WidgetRef ref)

async.when() ব্যবহার করে: লোডিং-এ Loader, এরর-এ ErrorState (রিট্রাই বাটন সহ refresh() কল করে), এবং ডটো খালি হলে EmptyState; নাহলে প্রতিটি AttendanceRecord-এর জন্য একটি _HistoryRow সহ AppCard রিটার্ন করে, রো-গুলোর মাঝে ডিভাইডার সহ।

ক্লাস/উইজেট: _HistoryRow

StatelessWidget — একদমের ইতিহাসের একটি রো: তারিখ, চকে-ইন/আউট সময়, লেটে মিনিটি (থাকলে), স্ট্যাটাস চপি এবং মোট কাজের ঘণ্টা।

Widget build(BuildContext context)

একটি Row তৈরি করে — বাম দিকে তারিখ ও ইন/আউট সময় (আইকন সহ), লেটে থাকলে অতিরিক্ত টেক্সট; ডান দিকে StatusChip ও (থাকলে) মোট কাজের ঘণ্টা।

ফাইল: lib/features/attendance/employee_attendance_screen.dart

সংক্ষিপ্ত বিবরণ: একজন নির্দিষ্ট কর্মীর অ্যাটেন্ডেন্স, টাইমশিট এবং সামারি দেখার অ্যাডমিন/ম্যানেজার স্ক্রিনি, একটি পরিবর্তনযোগ্য তারিখ-সীমা সহ। টমি ভিডিও বা প্রোফাইল থেকে খোলা হয়।

টপ-লেভেলে ফাংশন

((DateTime from, DateTime to)) _defaultRange()

ডিফল্ট সময়সীমা রিটার্ন করে — মাসের শুরু থেকে আজ পর্যন্ত।

String _iso(DateTime day)

DateTime-কে yyyy-MM-dd স্ট্রিং-এ রূপান্তর করে (জেরিও-প্যাডেড)।

ক্লাস: AttendanceRangeController

Notifier<((DateTime from, DateTime to))> — স্ক্রিনি দেখানো তারিখ-সীমা ধারণ করে।

((DateTime from, DateTime to)) build()

_defaultRange() রিটার্ন করে প্রাথমিক স্টেটে হিসেবে।

void set(DateTime from, DateTime to)

স্টেটকে নতুন (from, to) টাপল দিয়ে আপডেট করে।

টপ-লভেলে প্রোভাইডার

`final attendanceRangeProvider`

`NotifierProvider<AttendanceRangeController, ({DateTime from, DateTime to})> |`

`final employeeSummaryProvider`

`FutureProvider.autoDispose.family<EmployeeAttendanceSummary, int>` — নর্দিদষ্টি কর্মীর বর্তমান রঞ্জে জন্ম সামারি আনে (`attendanceRangeProvider` ওয়াচ করে)।

`final employeeRecordsProvider`

`FutureProvider.autoDispose.family<List<AttendanceRecord>, int>` — নর্দিদষ্টি কর্মীর রকের্ড আনে (`recordsFor`)।

`final employeeTimesheetsProvider`

`FutureProvider.autoDispose.family<List<Timesheet>, int>` — নর্দিদষ্টি কর্মীর রঞ্জে-ভিত্তিক টাইমশিট আনে; পজেনিটেডে এন্ডপয়ন্টের বদলে রঞ্জে এন্ডপয়ন্ট ব্যবহার করা হয় যাতে লস্ট ও উপরে ফগির সবসময় একই সময়সীমার সাথে মিলে যায়।

ক্লাস/উইজটে: EmployeeAttendanceScreen

`ConsumerWidget` — একজন কর্মীর অ্যাটেন্ডেন্স/টাইমশিট/সামারি দেখানোর স্ক্রিনি, তারখি-সীমা পরিবর্তনযোগ্য।

`static void open(BuildContext context, {required int employeeId, required String name})`

`MaterialPageRoute` দিয়ে এই স্ক্রিনিটি পুশ করে নেভিগেটে করে।

`Widget build(BuildContext context, WidgetRef ref)`

অ্যাপবারে একটি "Change the period" আইকন বাটন থাকে যা `showDateRangePicker` খুলে নতুন রঞ্জে সেট করে (`attendanceRangeProvider.notifier.set`)। `RefreshIndicator`-এর `onRefresh` তনিটি প্রোভাইডার ইনভ্যালিডেট করে। `ListView`-এ রঞ্জে টেক্সট, `_Summary`, "Timesheets" সেকশন (`_Timesheets`), এবং "Recent days" সেকশন (`_Records`) দেখায়।

ক্লাস/উইজটে: _Summary

`ConsumerWidget` — কর্মীর প্রজেন্ট/অ্যাবসেন্ট/লেটে দিনের সংখ্যা এবং উপস্থিতির শতাংশ `StatCard` গ্রুপে দেখায়।

`Widget build(BuildContext context, WidgetRef ref)`

`employeeSummaryProvider(employeeId)` ওয়াচ করে `async.when()` দিয়ে লোডিং/এরর/ডটো হ্যান্ডলে করে। ডটো এলে দুই সারিতে চারটি `StatCard` দেখায় (Present, Absent, Late, Turned up%) — শতাংশটি ব্যাকএন্ডের নজিস্ব ফগির, কারণ এটি ক্যালেন্ডারের বপিরীতে না, শুধু রকের্ড থাকা দিনের বপিরীতে গণনা করে।

ক্লাস/উইজটে: _Timesheets

`ConsumerStatefulWidget` — কর্মীর টাইমশিট তালিকা দেখায়, অনুমোদনের (`approve`) বাটন সহ যাদের পারমিশন আছে।

`ConsumerState<_Timesheets> createState()`

`_TimesheetsState` ইনস্ট্যান্স রটার্ন করে।

ক্লাস: _TimesheetsState

ConsumerState<_Timesheets> — _busyId দিয়ে কোন এন্ট্রি অনুমোদন প্রক্রিয়াধীন তা ট্র্যাক করে।

Future<void> _approve(Timesheet timesheet)

attendanceAdminRepositoryProvider.approveTimesheet(timesheet.id) কল করে; সফল হলে employeeTimesheetsProvider ইনভ্যালিডেট করে ও সন্ধ্যাকবার দেখায়; ApiException বা অন্য এরর হলে যথাযথ মैसेজে সহ সন্ধ্যাকবার দেখায়; শেষে _busyId null করে।

Widget build(BuildContext context)

employeeTimesheetsProvider ও permissionControllerProvider ওয়াচ করে। প্রতিটি টাইমশিটে জন্ম তারিখ, ঘণ্টা, প্রজেক্ট নাম, বিবরণ দেখায়; ডান দিকে — প্রক্রিয়াধীন হলে স্পিনার, canApprove ও স্ট্যাটাস submitted হলে "Approve" বাটন, নাহলে StatusChip।

ক্লাস/উইজটে: _Records

ConsumerWidget — কর্মীর সাম্প্রতিক (সর্বোচ্চ ৩০টি) অ্যাটেন্ডেন্স রেকর্ড তালিকা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

employeeRecordsProvider(employeeId) ওয়াচ করে async.when() দিয়ে লোডিং/এরর/খালি/ডেটো হ্যান্ডলে করে। প্রতিটি রেকর্ডের জন্ম তারিখ, ইন/আউট সময় (থাকলে), এবং StatusChip দেখায়।

ফাইল: lib/features/attendance/punch_card.dart

সংক্ষিপ্ত বিবরণ: আজকের অ্যাটেন্ডেন্স স্টেটে ও একমাত্র উপলব্ধ অ্যাকশন (চেক-ইন/আউট) দেখানো কার্ড উইজটে। ড্যাশবোর্ড ও অ্যাটেন্ডেন্স ট্যাব উভয় জায়গায় শোয়ার করা হয়।

ক্লাস/উইজটে: PunchCard

ConsumerWidget — আজকের অ্যাটেন্ডেন্স স্টেটে নথি লোডিং/এরর/ডেটো রেন্ডার করে।

Widget build(BuildContext context, WidgetRef ref)

attendanceControllerProvider ওয়াচ করে, async.when() দিয়ে Loader, ErrorState (রিট্রাই সহ), অথবা _Body-কে AppCard-এর ভেতরে রেন্ডার করে।

ক্লাস/উইজটে: _Body

ConsumerWidget — আজকের তারিখ, স্ট্যাটাস লাইন, চেক-ইন/আউট স্ট্যাম্প, লেটে নোটিশ, অ্যাকশন বাটন এবং (কমপ্যাক্ট না হলে) মোট কাজের ঘণ্টা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

তারিখ (Fmt.dayDate), _statusLine(today) টেক্সট, এবং today থাকলে StatusChip দেখায়। দুটি _Stamp উইজটে (চেক-ইন ও চেক-আউট) পাশাপাশি একটি বিভাজক রেখা সহ দেখায়। today লেটে হলে (এবং lateMinutes > 0) একটি ইনফো ব্যানার দেখায়। _Action উইজটে দেখায়, এবং কমপ্যাক্ট না হয়ে দনি সম্পূর্ণ হলে "Worked X today" টেক্সট দেখায়।

static String _statusLine(AttendanceRecord? today)

today null হলে বা এখনো চেক-ইন করা না হলে "You have not checked in yet." রিটার্ন করে; দনি সম্পূর্ণ হলে "Your day is recorded."; চেক-ইন করা থাকলে "You are checked in."।

ক্লাস/উইজটে: _Stamp

StatelessWidget — একটি একক টাইম-স্ট্যাম্প (চকে-ইন বা চকে-আউট) লবেলে, আইকন ও সময় সহ দেখায়।

Widget build(BuildContext context)

time নন-নাল/নন-এম্পটি হিলে (has) আইকন ও টেক্সটের রং tone/bos.text ব্যবহার করে, নাহলে মডিউটে রং ও Fmt.dash দেখায়।

ক্লাস/উইজটে: _Action

ConsumerWidget — বর্তমান স্টেটে অনুযায়ী "Check in", "Check out", অথবা "All done for today" দেখায়।

Future<void> _punch(BuildContext context, WidgetRef ref, bool checkingIn)

attendanceControllerProvider.notifier-এর checkIn() বা checkOut() কল করে। সফল হলে সংশ্লিষ্ট সন্ধ্যাকবার দেখায়; ApiException হলে তার message দেখায়; অন্য এরর হলে জেনেরিক ব্যর্থতার বার্তা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

state.canCheckIn হলে "Check in" LoadingButton (busy স্টেটে সহ) রটাইরন করে; state.canCheckOut হলে "Check out" বাটন; নাহলে (উভয় স্ট্যাম্প উপস্থিতি) একটি স্ট্যাটিক "All done for today" ব্যানার দেখায় — ডিজিবেল বাটনের বদলে, কারণ ডিজিবেল বাটন ট্যাপ করতে প্রলুব্ধ করতে পারে।

done

ফাইল: lib/features/attendance/shift_roster_screen.dart

সংক্ষিপ্ত বিবরণ: কোন কর্মী কোন শফিটে আছে তা দেখানোর স্ক্রিনি — একবারে একটি শফিট বছে নযি়ে তার অ্যাসাইনমেন্ট তালিকা দেখায়। এখানই অ্যাসাইনমেন্ট এন্ড করা, ডলিটি করা এবং এডটি করার সুবধিও আছে।

ক্লাস: RosterShiftController

Notifier<int?> — বর্তমানে কোন শফিট আইডি নির্বাচতি আছে তা রাখে। প্রাথমিক অবস্থায় null।

int? build()

স্টেটে শুরু করে null দয়ি — কোনো শফিট প্রথমতে নির্বাচতি থাকে না।

void set(int? shiftId)

নির্বাচতি শফিট আইডি পরিবর্তন করে স্টেটকে নতুন shiftId-তে সেট করে।

টপ-লভেলে প্রোভাইডার: rosterShiftProvider, rosterProvider

rosterShiftProvider হলো RosterShiftController-এর NotifierProvider। rosterProvider একটি

FutureProvider.autoDispose যা নির্বাচতি শফিটের জন্য

attendanceAdminRepositoryProvider.assignmentsForShift(shiftId) কল করে অ্যাসাইনমেন্ট তালিকা আন; শফিট আইডি null হলে খালি লিস্ট রটাইরন করে। কমেন্টে ব্যাখ্যা করা আছে কেনে একবারে একটি শফিট দেখানো হয় — ব্যাকএন্ডে "সব অ্যাসাইনমেন্ট" নামে কোনো এন্ডপয়েন্ট নই।

ক্লাস/উইজটে: ShiftRosterScreen

ConsumerWidget যা "কে কোন শফিটে আছে" স্ক্রিনি রেন্ডার করে, উপরে একটি FilterBar দিয়ে শফিট বাছাই করার সুযোগ দেয় এবং পারমিশন থাকলে নতুন অ্যাসাইনমেন্ট যোগ করার FAB দেখায়।

static void open(BuildContext context)

MaterialPageRoute দিয়ে ShiftRosterScreen-কে নতুন রুটে পুশ করে খোলে।

Widget build(BuildContext context, WidgetRef ref)

সক্রিয় শফিট তালিকা, নরিবাচতি শফিট এবং অ্যাসাইনমেন্ট তৈরি পারমিশন পড়ে। শফিট তালিকা এলে এবং কিছু নরিবাচতি না থাকলে প্রথম শফিটটি পোস্ট-ফরমে কলব্যাক অটো-সলিক্ট করে। শফিট না থাকলে EmptyState দেখায়; থাকলে ConfigList-এ FilterBar হডোর ও প্রতটি অ্যাসাইনমেন্টের জন্য _AssignmentRow রেন্ডার করে। পারমিশন থাকলে FAB চাপলে showAssignShiftSheet খুলে rosterProvider ইনভ্যালিডেট করে।

ক্লাস/উইজটে: _AssignmentRow

ConsumerStatefulWidget — একটি একক শফিট-অ্যাসাইনমেন্ট সারি দেখায়, সাথে "টকে অফ" ও "ডলিটি" অ্যাকশন প্রদান করে।

ক্লাস: _AssignmentRowState

_AssignmentRow-এর স্টেটে, ব্যস্ত-অবস্থা (_busy) এবং লগিন-অ্যাকশন পরিচালনা করে।

Future<void> _run(Future<void> Function(AssignmentRepository) action, String done)

_busy সত্য করে দেওয়া action চালায়; সফল হলে rosterProvider ইনভ্যালিডেট করে ও done মসেজে সন্যাকবারে দেখায়; ApiException এলে সেই মসেজে দেখায়, অন্য ত্রুটিতে জনেরেকি বার্তা দেখায়; শেষে _busy মথিয়া করে।

Future<void> _end()

অ্যাসাইনমেন্টের আইডি না থাকলে থেমে যায়; নিশ্চিতকরণ ডায়ালগ দেখিয়ে হ্যাঁ হলে endAssignment কল করে কর্মীকে শফিট থেকে বাদ দেয়।

Future<void> _delete()

আইডি না থাকলে থেমে যায়; নিশ্চিতকরণ (destructive) ডায়ালগ দেখিয়ে হ্যাঁ হলে deleteAssignment কল করে অ্যাসাইনমেন্ট রেকর্ড স্থায়ীভাবে মুছে দেয়।

Widget build(BuildContext context)

পারমিশন পড়ে canUpdate/canDelete নির্ণয় করে; শুরু ও শেষ তারখির সংক্ষিপ্ত টেক্সট বানায়; ConfigRow রেন্ডার করে যাতা কর্মীর নাম, তারখি, কারণ, এডটি বাটন এবং শর্তসাপেক্ষে "টকে অফ"/"ডলিটি" অ্যাকশন থাকে।

উপ-লভেলে ফাংশন: showEditAssignmentSheet

Future<void> showEditAssignmentSheet(BuildContext context, {required ShiftAssignment assignment})

_EditAssignmentSheet-কে মডাল বটম শিটে খোলে, যাতা বদ্যমান একটি অ্যাসাইনমেন্টের তারখি বা শফিট পরিবর্তন করা যায়।

ক্লাস/উইজটে: `_EditAssignmentSheet`

`ConsumerStatefulWidget` — একটি অ্যাসাইনমেন্ট এডিট করার ফর্ম শিট, শিফট, তারিখ, কারণ ও নোট ফিল্ড ধারণ করে।

ক্লাস: `_EditAssignmentSheetState`

ফর্মের স্টেট, কন্ট্রোলার ও লোকাল ভ্যালু পরিচালনা করে।

`void dispose()`

`_reason` ও `_notes` টেক্সট কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম বৈধ কিনা যাচাই করে; কর্মী/শিফট আইডি অনুপস্থিতি থাকলে ত্রুটি দেখায়; শেষে তারিখ শুরুর আগে হলে ত্রুটি দেখায়; বৈধ হলে `updateAssignment` কল করে সাবমিট করে, সফল হলে `rosterProvider` ইনভ্যালিডেট করে শিট বন্ধ করে; `ApiException`-এর বার্তা (যেমন ওভারল্যাপিং অ্যাসাইনমেন্ট প্রত্যাখ্যান) সরাসরি দেখায়, অন্য ত্রুটিতে জনৈকি বার্তা দেখায়।

`Widget build(BuildContext context)`

`FormSheetFrame`-এর ভেতরে কর্মীর নাম, শিফট ড্রপডাউন, "শুরু"/"শেষ" `DateField` জোড়া, কারণ ও নোট টেক্সট ফিল্ড রেন্ডার করে এবং সাবমিট হ্যান্ডলার হিসেবে `_submit` বাঁধে।

ফাইল: `lib/features/attendance/submit_timesheet_sheet.dart`

সংক্ষিপ্ত বিবরণ: একটি দিনের কর্মঘণ্টা লগ করা বা বদ্যমান একটি টাইমশিট এন্ট্রি এডিট করার জন্য বটম শিট ফর্ম। এতে AI দিয়ে বর্ণনা "গুছিয়ে" নেওয়ার সুবিধাও আছে।

উপ-লেভেলে ফাংশন: `showLogTimesheetSheet`, `showEditTimesheetSheet`

`Future<void> showLogTimesheetSheet(BuildContext context)`

`_LogTimesheetSheet`-কে (কোনো `existing` ছাড়া, অর্থাৎ, নতুন এন্ট্রিমোডে) মডাল বটম শিটে খোলে।

`Future<void> showEditTimesheetSheet(BuildContext context, Timesheet timesheet)`

`_LogTimesheetSheet`-কে `existing: timesheet` দিয়ে খোলে, যাতে বদ্যমান এন্ট্রি এডিট করা যায়। শুধুমাত্র `Timesheet.isEditable` সত্য থাকলেই এটি ডাকা হয়, কারণ সাবমিট বা অনুমোদিত এন্ট্রি ব্যাকএন্ড প্রত্যাখ্যান করে।

ক্লাস/উইজটে: `_LogTimesheetSheet`

`ConsumerStatefulWidget` — একটি কাজের দিনের ঘণ্টা লগ/এডিট করার ফর্ম শিট। `existing` দেওয়া থাকলে এডিট মোড, না থাকলে নতুন এন্ট্রিমোড।

ক্লাস: `_LogTimesheetSheetState`

ফর্মের সব কন্ট্রোলার, তারিখ, সাবমিট/টাইড-আপ স্টেটে রাখে।

`bool get _isEdit`

`widget.existing` `null` না হলে সত্য — এডিট মোডে আছে কিনা তা বলে।

`static String _plain(double value)`

একটি সংখ্যাকে মানুষ যত্নে টাইপ করবে তমেন স্ট্রিং বানায় — পূর্ণসংখ্যা হলে দশমিকি ছাড়া (যমেন "8"), না হলে যমেন আছে তমেন (যমেন "7.5")।

`void initState()`

বদ্যমান এন্ট্রিথিকে `_hours`, `_billable`, `_project`, `_description` কন্ট্রোলার এবং `_date` প্রাথমিকভাবে পূরণ করে; ঘণ্টা `_plain` দ্যি়ে সাদামাটা সংখ্যা হসিবে বসানো হয় যাতে আবার সাবমটি করার সময় পার্স করা যায়।

`void dispose()`

চারটি টেক্সট কন্ট্রোলার ডসিপোজ করে।

`Future<void> _pickDate()`

`showDatePicker` থোলে (গত এক বছর থেকে আজ পর্যন্ত সীমাবদ্ধ) এবং নির্বাচতি তারখি `_date`-এ সটে করে।

`Future<void> _submit()`

ফর্ম বধৈতা যাচাই করে, ফোকাস সরয়ি়ে সাবমটি-স্টেটে চালু করে; `_isEdit` অনুযায়ী `timesheetsProvider.notifier`-এর `updateItem` বা `log` কল করে ঘণ্টা, বলিবেল ঘণ্টা, প্রজেক্ট ও বর্ণনা পাঠায়; সফল হলে শটি বন্ধ করে সংশ্লিষ্ট স্ন্যাকবার মসেজে দথোয়; `ApiException`-এ সই বার্তা, অন্যথায় জনেরেকি এডিটি/লগ ব্যর্থতার বার্তা দথোয়।

`Future<void> _tidy()`

বর্ণনা বাক্সের কাঁচা নোট নয়ি়ে `attendanceRepositoryProvider.composeTimesheetEntry` কল করে AI-জেনেরটেডে পরষিকার বর্ণনা আনে ও বাক্সে বসয়ি়ে দয়ে; নোট খালি থাকলে আগে জানয়ি়ে দয়ে; কিছু না লখো থাকলে বাক্স যমেন ছিল তমেনই থাকে। এটি কিছু লগ করে না — শুধু খসড়া তরৈকিরে, সদ্ধান্ত ব্যবহারকারীর।

`Widget build(BuildContext context)`

তারখি পকার, ঘণ্টা/বলিবেল-ঘণ্টা/প্রজেক্ট/বর্ণনা ফল্ড, বর্ণনার পাশে "Tidy it up" আইকন বাটন (বা লোডিং স্পিনার) এবং সাবমটি বাটনসহ পুরো ফর্ম শটি রন্ডার করে।

ফাইল: `lib/features/attendance/team_attendance_screen.dart`

সংক্ষিপ্ত ববিরণ: টিম-লভেলে অ্যাটনেডেন্স স্ক্রনি — একটি নির্দিষ্ট দিনেরে সবার উপস্থতি দথোয়, ম্যানুয়ালি রেকর্ড করা, রেকর্ড অনুমোদন করা এবং অনুপস্থতি ব্যাকফলি করার সুবধি দয়ে।

ক্লাস/উইজটে: `TeamAttendanceScreen`

`ConsumerWidget` — পারমশিন অনুযায়ী লোডিং, "নট অ্যাভইলবেল" বা মূল স্ক্রনি দথোয়। মূল স্ক্রনি অ্যাপবার অ্যাকশন হসিবে ব্যাকফলি (শুধু কোম্পানিওনার) ও শফিট রোস্টার বাটন, এবং মার্ক-পারমশিন থাকলে "রেকর্ড আ ডে" FAB থাকে।

`Widget build(BuildContext context, WidgetRef ref)`

পারমশিন লোড হয়নইলে `Loader` দথোয়; `AttendanceAdminPermissions.view` না থাকলে `EmptyState` দয়ি়ে বলে দয়ে এই স্ক্রনি উপলব্ধ নয়; অন্যথায় `attendanceDayProvider`, মার্ক-পারমশিন ও `isOwner` পড়ে অ্যাপবার/FAB সহ `Scaffold` বানয়ি়ে `_Body` রন্ডার করে।

ক্লাস/উইজটে: _Body

`ConsumerStatefulWidget` — মূল তালিকা এলাকা, দিনের নভেগেশন, সার্চ ও পরসিংখ্যান হডোর ধারণ করে।

ক্লাস: _BodyState

`_busyId` দিয়ে ট্র্যাক করে কোন রকর্ড অনুমোদন-প্রক্রিয়ায় বসত।

`Future<void> _approve(AttendanceRecord record)`

`_busyId` স্টে করে `approveRecord` কল করে; রসেপন্স বডিনা থাকায় ফলাফল প্যাচ না করে পুরো `teamAttendanceProvider` রিফ্রেশ করে; সফল/ব্যর্থ বার্তা সন্ধ্যাকবারে দেখায়; শেষে `_busyId` null করে।

`Widget build(BuildContext context)`

বর্তমান দিন `today`-র সমান কনি দেখে; `ConfigList`-এ দিন-নভেগেশন কার্ড (আগরে/পররে দিন, ভবিষ্যৎ দিনে "পররে" বাটন নথিক্রয়), সার্চ ফল্ড, এবং `daily` রপোর্ট থাকলে `_DayFigures` হডোর হিসেবে দেখায়; প্রতিটি রকর্ডের জন্য `_RecordRow` তৈরি করে, অনুমোদন-পারমিশন থাকলে `onApprove` পাস করে।

ক্লাস/উইজটে: _DayFigures

`ConsumerWidget` — দৈনিক রপোর্টের সংখ্যাগুলো (উপস্থিতি+দরে, দরে, অনুপস্থিতি, ছুটিতে) স্ট্যাট কার্ডে এবং মাসিক উপস্থিতি শতাংশ কার্ডে দেখায়।

`Widget build(BuildContext context, WidgetRef ref)`

`report`-এর সংখ্যা দিয়ে দুই সারিতে চারটি `StatCard` বানায় (দরে/অনুপস্থিতি সংখ্যা থাকলে সংশ্লিষ্ট রঙ দিয়ে); `report.unaccounted > 0` হলে সতর্কীকরণ ব্যানার দেখায় (নাইটলি মারকার না চলার ইঙ্গিত); `monthly` রপোর্ট থাকলে মাসিক সারসংক্ষেপে কার্ড যোগ করে।

ক্লাস/উইজটে: _RecordRow

`StatelessWidget` — একজন কর্মীর দিনের অ্যাটেন্ডেন্স রকর্ড দেখায়, ট্যাপ করলে সেই কর্মীর `EmployeeAttendanceScreen` খোলে।

`Widget build(BuildContext context)`

নাম, চকে-ইন/আউট সময় (বা "no times recorded"), `StatusChip`, এবং `busy` হলে স্পনির নয়তো `onApprove` থাকলে অনুমোদন আইকন বাটন দেখায়।

টপ-লেভেলে ফাংশন: showManualAttendanceSheet

`Future<void> showManualAttendanceSheet(BuildContext context, {required DateTime day})`

`_ManualSheet`-কে নির্দিষ্ট `day` দিয়ে মডাল বটম শিটে খোলে, ম্যানুয়ালি একটি দিনের অ্যাটেন্ডেন্স রকর্ড করার জন্য।

ক্লাস/উইজটে: _ManualSheet

`ConsumerStatefulWidget` — কর্মী বাছাই, তারিখ, স্ট্যাটাস, চকে-ইন/আউট সময়, দরের কারণ ও নোট নিয়ে ম্যানুয়াল অ্যাটেন্ডেন্স এন্ট্রিফর্ম।

ক্লাস: _ManualSheetState

ফর্মের স্টেটে রাখে।

`bool get_hasTimes`

স্ট্যাটাস PRESENT বা LATE হলই সত্য — শুধু এই দুই অবস্থায় সময় ধরা হয়, বাকগুলো পুরো-দনিরে অবস্থা।

`void dispose()`

`_notes` ও `_lateReason` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _pickEmployee()`

`EmployeePicker.show` খুলে বাছাই করা কর্মীর আইডিও নাম সেটে করে, ত্রুটি রিসিটে করে।

`Future<void> _pickTime({required bool start})`

`showTimePicker` খুলে `start` অনুযায়ী `_checkIn` বা `_checkOut` আপডেট করে।

`static String _iso(TimeOfDay time)`

একটি `TimeOfDay`-কে "HH:mm:00" ফরম্যাটেরে স্ট্রিংয়ে রূপান্তর করে।

`Future<void> _submit()`

কর্মী ও তারখি না থাকলে ত্রুটি দেখায়; বৈধ হলে `createManual` কল করে (ধরা সময় শুধু `_hasTimes` হলে পাঠায়; `isLate` স্ট্যাটাস থাকে নরিণয় করা হয়, ব্যবহারকারীকে জিজ্ঞাসা না করে); সফল হলে `attendanceDayProvider` সহি তারখি সেটে করে, তালিকা রফির্শে করে, দৈনিক রপোর্ট ইনভ্যালডিটে করে ও শটি বন্ধ করে; `ApiException`-এর বার্তা (যেমন ডুপ্লিকিটে রকোর্ড প্রত্যাখ্যান) সরাসরি দেখায়।

`Widget build(BuildContext context)`

`FormSheetFrame`-এ কর্মী নরিবাচক, তারখি, স্ট্যাটাস ড্রপডাউন, শর্তসাপেক্ষে ইন/আউট সময় বাটন ও দেরি কারণ ফল্ড, নোট ফল্ড এবং তথ্যভিত্তিকি ব্যানার রন্ডার করে।

টপ-লেভেলে ফাংশন: `showBackfillSheet`

`Future<void> showBackfillSheet(BuildContext context)`

`_BackfillSheet`-কে মডাল বটম শটি থোলো — নাইটলি মার্কার মসি করা অনুপস্থিতি একটি তারখি-পরসিরে পূরণ করার জন্য, শুধুমাত্র কোম্পানি ওনারের জন্য এবং ইডমেন্টে।

ক্লাস/উইজটে: `_BackfillSheet`

`ConsumerStatefulWidget` — শুরু ও শেষে তারখি নযিে ব্যাকফিলি অনুরোধ পাঠানোর ফর্ম শটি।

ক্লাস: `_BackfillSheetState`

তারখি পরসির, সাবমটি ও ত্রুটি স্টেটে রাখো।

`Future<void> _submit()`

শুরু/শেষে তারখি অনুপস্থিতি, শুরু শেষের পরে, অথবা পরসির ৩৬৬ দিনের বেশিহলে স্থানীয়ভাবে ত্রুটি দেখায়; নশ্চিতিকরণ ডায়ালগে হ্যাঁ হলে `backfillAbsentees` কল করে; সফল হলে তালিকা ও রপোর্ট রফির্শে/ইনভ্যালডিটে করে, শটি বন্ধ করে এবং কতগুলো অনুপস্থিতিরেকোর্ড হয়েছে তা জানযিে সন্ধ্যাকবার দেখায় (০, ১, বা একাধিকেরে জন্য আলাদা বার্তা)।

`Widget build(BuildContext context)`

`FormSheetFrame`-এ ব্যাখ্যামূলক টেক্সট, "শুরু"/"শেষ" `DateField` জোড়া এবং একটি তথ্যভিত্তিকি ব্যানার (ভবষিৎ তারখি উপেক্ষতি, পরসির এক বছরের বেশি নয়) রন্ডার করে।

ফাইল: lib/features/attendance/timesheets_tab.dart

সংক্ষিপ্ত ববিরণ: ব্যবহারকারীর নিজের টাইমশিট এন্ট্রি তালিকার ট্যাব — নতুন এন্ট্রি লিগ করা, রিভিউয়ের জন্য সাবমিট করা এবং প্রত্যাগী এন্ট্রি এডিট/ডিলিট করার সুবিধা দিয়ে।

ক্লাস/উইজটে: TimesheetsTab

ConsumerWidget — **PagedListView** দিয়ে ব্যবহারকারীর টাইমশিট এন্ট্রি দেখায়, ড্রাফট এন্ট্রি থাকলে "Submit for review" FAB ও সবসময় "Log time" FAB দেখায়।

Future<void> **_submitForReview(BuildContext context, WidgetRef ref)**

timesheetsProvider.notifier.submitForReview() কল করে; কতগুলো এন্ট্রি সাবমিট হলো তার ওপর ভিত্তি করে আলাদা বার্তা (কিছু না থাকলে, ১টি বা একাধিক) সন্ধ্যাকবারে দেখায়; **ApiException**-এ সেই বার্তা, অন্যথায় জনৈকি ব্যর্থতার বার্তা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

timesheetsProvider থেকে ডটো পড়ে অন্তত একটি "notSubmitted" এন্ট্রি আছে কিনা (**hasDraft**) নরিণয় করে; **PagedListView** দিয়ে তালিকা রেন্ডার করে (রিফ্রেশ/লোড-মোর হ্যান্ডলারসহ), প্রত্যাগী আইটেমে জনৈকি **_TimesheetCard** তৈরি করে; নচি শর্তসাপেক্ষ "Submit for review" এবং সবসময়-দৃশ্যমান "Log time" FAB দেখায়।

ক্লাস/উইজটে: _TimesheetCard

ConsumerStatefulWidget — একটি টাইমশিট এন্ট্রির কার্ড, তারখি, প্রজেক্ট, বর্ণনা, ঘণ্টা, অনুমোদনকারীর নাম এবং (এডিটযোগ্য হলে) এডিট/ডিলিট বাটন দেখায়।

ক্লাস: _TimesheetCardState

_deleting দিয়ে ডিলিট-প্রক্রিয়ার ব্যস্ততা ট্র্যাক করে।

Future<void> **_delete()**

AlertDialog দিয়ে নশ্চিতিরূপ চায় ("This entry is removed for good"); নশ্চিতি হলে **_deleting** সত্য করে **timesheetsProvider.notifier.delete** কল করে; **ApiException**-এ সেই বার্তা, অন্যথায় জনৈকি ব্যর্থতার বার্তা সন্ধ্যাকবারে দেখায়; শেষে **_deleting** মিথ্যা করে।

Widget build(BuildContext context)

AppCard-এ তারখি, প্রজেক্ট নাম (থাকলে), স্ট্যাটাস চপি, বর্ণনা (থাকলে), কাজ করা/বলিবেল ঘণ্টা আইকন-সহ টেক্সট, অনুমোদনকারীর নাম (থাকলে) এবং **timesheet.isEditable** হলে এডিট/ডিলিট বাটন (অথবা ডিলিটের সময় লোডার) রেন্ডার করে।

ফাইল: lib/features/auth/forgot_password_screen.dart

সংক্ষিপ্ত ববিরণ: তনি-ধাপের পাসওয়ার্ড রসিটে স্ক্রিনি — ইমেইল দিয়ে কোড চাওয়া, কোড যাচাই করা এবং নতুন পাসওয়ার্ড সেট করা। মাঝের ধাপটি যাচাইকৃত কোড খরচ করে না; শেষে কলই আসল যাচাই হয়।

enum: _Step

email, code, password — তনিটি ধাপ চহ্নিতি করে, যা স্ক্রিনিরে বর্তমান অবস্থা নর্ধারণ করে।

ক্লাস/উইজটে: ForgotPasswordScreen

ConsumerStatefulWidget — পাসওয়ার্ড রসিটে ফ্লো-র মূল স্ক্রিনি।

ক্লাস: _ForgotPasswordScreenState

ধাপ, ফর্ম কন্ট্রোলার, লোডিং/ত্রুট/নোটিশ স্টেটে রাখো।

void dispose()

_email, _code, _password, _confirm কন্ট্রোলার ডিসপোজ করে।

Future<void> _run(Future<void> Function() action)

ইতিমধ্যে লোডিং চললে বা ফর্ম অবধৈ হলে থেমে যায; ফোকাস সরিয়ে লোডিং চালু করে action চালায়; `ApiException` বা অন্য ত্রুটিতে যথাযথ বার্তা স্টে করে; শেষে লোডিং বন্ধ করে।

void _sendCode()

_run-এর মাধ্যমে `authRepositoryProvider.forgotPassword` কল করে; সফল হলে ধাপ `code`-এ নিয়ে যায এবং ইচ্ছাকৃতভাবে অ্যাকাউন্ট আছে কিনা তা প্রকাশ না করা একটি নিরিপক্শ নোটিশ দেখায় (ব্যাকএন্ড উভয় ক্ষেত্রেই ২০০ রটাইন করে)।

void _verifyCode()

_run-এর মাধ্যমে `verifyResetCode` কল করে; সফল হলে ধাপ `password`-এ নিয়ে যায এবং নোটিশ মুছে দেয়।

void _resetPassword()

_run-এর মাধ্যমে `resetPassword` কল করে (ইমেইল, কোড, নতুন ও নিশ্চিতকরণ পাসওয়ার্ডসহ); সফল হলে সফলতার সন্ধ্যাকবার দেখিয়ে `context.pop()` করে আগরে স্ক্রিনি ফরিে যায।

Widget build(BuildContext context)

বর্তমান ধাপ অনুযায়ী শরিনোম, ত্রুট/নোটিশ ব্যানার, `_fieldsFor(_step)` থেকে ফল্ডি এবং ধাপ-নির্দিষ্ট লবেলে ও হ্যান্ডলারসহ `LoadingButton` রেন্ডার করে; `code` ধাপে "Send another code" বাটনও দেখায়; অ্যাপবাররে ব্যাক বাটন প্রথম ধাপে স্ক্রিনি থেকে বরে করে দেয়, নয়তো আগরে ধাপে ফরোয়।

List<Widget> _fieldsFor(_Step step)

ধাপ অনুযায়ী ফর্ম ফল্ডি তরৈকিরে: `email` ধাপে ইমেইল ফল্ডি (বধৈতা যাচাইসহ); `code` ধাপে ৬-অঙ্করে সংখ্যা-শুধু কোড ফল্ডি (ফরম্যাটার ও বধৈতাসহ); `password` ধাপে নতুন পাসওয়ার্ড ও নিশ্চিতকরণ পাসওয়ার্ড ফল্ডি (দরৈঘ্য ও মলি যাচাইসহ, দেখো/লুকানো টগল)।

ফাইল: lib/features/auth/login_screen.dart

সংক্ষিপ্ত ববিরণ: অ্যাপরে লগইন স্ক্রিনি — ইমেইল/পাসওয়ার্ড দিয়ে সাইন-ইন, "ফরগট পাসওয়ার্ড", রডি-অনলি ডিমো তনোন্ট শুরু করা এবং রজিস্ট্রেশনে যাওয়ার লংক ধারণ করে।

ক্লাস/উইজটে: LoginScreen

ConsumerStatefulWidget — লগইন ফর্মের মূল স্ক্রিনি।

ক্লাস: _LoginScreenState

ফর্ম কন্ট্রোলার, লোডিং, ত্রুটি, "unverified" ও ডেমো-শুরু স্টেটে রাখে।

void dispose()

_email ও _password কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ইতিমধ্যে লোডিং চললে বা ফর্ম অবধি হলে থেমে যায়; ফোকাস সরিয়ে লোডিং চালু করে `authControllerProvider.notifier.login` কল করে; নভিগেশন এখানে করা হয় না — রাউটারের রডাইরেক্ট অথ-স্টেটে পরিবর্তন দেখে স্বয়ংক্রিয়ভাবে সরিয়ে দেয়, তাই এখানে পুশ করলে রেসের সম্ভাবনা থাকত; `ApiException`-এ বার্তা স্টেট করে, এবং বার্তায় "not been verified" থাকলে `_unverified` সত্য করে (ব্যাকএন্ডের নজিস্ব শব্দচয়নের সাথে চলিতোলাভাবে মিলিয়ে); অন্য ত্রুটিতে জনৈকি বার্তা দেখায়; শেষে লোডিং বন্ধ করে।

Future<void> _startDemo()

ইতিমধ্যে লোডিং/ডেমো-শুরু চললে থেমে যায়; ফোকাস সরিয়ে `_startingDemo` চালু করে `authControllerProvider.notifier.startDemo()` কল করে (কোনো ক্রডেনশিয়াল ছাড়াই ব্যাকএন্ড সংক্ষিপ্ত সেশন তৈরি করে); নভিগেশন এখানেও রাউটারের রডাইরেক্টে ছেড়ে দেওয়া হয়; `ApiException` বা অন্য ত্রুটিতে যথাযথ বার্তা দেখায়; শেষে `_startingDemo` বন্ধ করে।

Widget build(BuildContext context)

`_Header`, ত্রুটি ব্য়ানার (unverified হলে "Verify my email" বাটনসহ), ইমেইল ও পাসওয়ার্ড ফিল্ড (বৈধতা, অটোফলি হিন্ট, দেখো/লুকানো টগলসহ), "Forgot password?" লিংক, সাইন-ইন বাটন, একটি ডিভাইডার, "Explore the demo" আউটলাইনড বাটন, ডেমো সম্পর্কে ব্যাখ্যা টেক্সট এবং রজিস্ট্রেশনে যাওয়ার লিংকসহ পুরো লগইন ফর্ম রেন্ডার করে।

ক্লাস/উইজটে: _Header

StatelessWidget — লোগো (BrandMark), অ্যাপের নাম "Zuhoo" এবং "Sign in to your workspace" সাবটাইটলে দেখায়।

Widget build(BuildContext context)

`box` প্যালাটে ব্যবহার করে ব্র্যান্ড মার্ক ও দুটি টেক্সট উপাদান নিয়ে একটি কলাম রেন্ডার করে।

ফাইল: lib/features/auth/register_screen.dart

সংক্ষিপ্ত ববিরণ: নতুন কোম্পানি (টেন্ড্যান্ট) ও তার মালিক অ্যাকাউন্ট একসাথে সাইন-আপ করানোর স্ক্রিনি — ব্যাকএন্ডের `POST /api/auth/register` এর মোবাইল প্রতরুপ। শুধুমাত্র নতুন কোম্পানি তৈরি জন্ব; কর্মী বা ক্লায়েন্টরা এই স্ক্রিনি দেখে না, তাদের অ্যাকাউন্ট কোম্পানির মালিক তৈরি করে দেন।

টপ-লভেলে ভ্যারিয়েবল

- `_subdomainPattern`, `_passwordUpper`, `_passwordLower`, `_passwordDigit`, `_passwordSpecial` — সাবডোমেইন ও পাসওয়ার্ড ভ্যালিডেশনের জন্য ব্যবহৃত `RegExp` প্যাটার্নসমূহ, ব্যাকএন্ডের `RegisterRequest` ভ্যালিডেশন নিয়মের সাথে মিলিয়ে রাখা হয়েছে।

ক্লাস/উইজটে: `RegisterScreen`

`ConsumerStatefulWidget`, স্টেটে `_RegisterScreenState`-এ রাখা। এটি শুধু একটি এন্ট্রি পয়েন্ট — মূল কাজ স্টেটে ক্লাসে।

`createState()`

`_RegisterScreenState`-এর একটি নতুন ইনস্ট্যান্স তৈরি করে রিটার্ন করে, স্ট্যান্ডার্ড Flutter override।

ক্লাস: `_RegisterScreenState`

রেজিস্ট্রেশন ফর্মের সমস্ত টেক্সট কন্ট্রোলার, ভ্যালিডেশন, লোডিং/এর স্টেটে এবং সাবমিশন লজিক পরিচালনা করে। ফর্মে ব্যবহারকারীর নাম, ইমেইল, পাসওয়ার্ড এবং কোম্পানির তথ্য (নাম, সাবডোমেইন, ফোন) নেওয়া হয়।

`dispose()`

সবগুলো `TextEditingController` (`_firstName`, `_lastName`, `_email`, `_password`, `_confirm`, `_companyName`, `_subdomain`, `_companyPhone`) ডিসপোজ করে এবং সুপারক্লাসের `dispose()` কল করে মেমরালিকি রোধ করে।

`_submit()`

ফর্ম ভ্যালিডেশন কনি যাচাই করে, `_loading` টগল করে, ফোকাস সরিয়ে `authRepositoryProvider`-এর `register()` মেথড কল করে `RegisterRequest` পাঠায়। সফল হলে নতুন ইউজারের ইমেইলসহ `Routes.verifyEmail`-এ `pushReplacement` করে নেভিগেটে করে। `ApiException` এলে সার্ভার-প্রদত্ত বার্তা দেখায়, অন্য যেকোনো এক্সপেশনে জনেরেকি বার্তা দেখায়, এবং শেষে `_loading` ফলস করে দেয়।

`_passwordError(String? v)`

পাসওয়ার্ডের দৈর্ঘ্য (৮+), বড় হাতের অক্ষর, ছোট হাতের অক্ষর, সংখ্যা এবং বিশেষ চিহ্ন — এই পাঁচটি শর্ত ক্রমান্বয়ে যাচাই করে, প্রথম যেটি ব্যর্থ হয় তার জন্য একটি নির্দিষ্ট এরর বার্তা রিটার্ন করে; সব শর্ত পূরণ হলে `null` রিটার্ন করে।

`build(BuildContext context)`

পুরো রেজিস্ট্রেশন ফর্মের UI তৈরি করে — নাম, ইমেইল, পাসওয়ার্ড, পাসওয়ার্ড নিশ্চিতকরণ, কোম্পানির নাম, সাবডোমেইন (ইনপুট নেওয়ার সময় স্বয়ংক্রিয়ভাবে ছোট হাতের অক্ষরে রূপান্তর করে), কোম্পানির ফোন (ঐচ্ছিক), এবং একটি `LoadingButton`। এররকে `MessageBanner.error` দিয়ে দেখায় এবং লগইন স্ক্রিনে ফিরে যাওয়ার একটি লিংক রাখা।

ফাইল: `lib/features/auth/verify_email_screen.dart`

সংক্ষিপ্ত বিবরণ: রেজিস্ট্রেশনের সময় ইমেইলে পাঠানো ৬-সংখ্যার কোড দিয়ে ইমেইল যাচাই করার স্ক্রিনি। এটি রেজিস্ট্রেশনের পরপর (ইমেইল প্র-ভ্যালিডেশন করা থাকে) অথবা লগইন স্ক্রিনির "not verified" বার্তা থেকেও (ইমেইল খালি থাকে) পৌঁছানো যায়।

ক্লাস/উইজটে: VerifyEmailScreen

ConsumerStatefulWidget। কনস্ট্রাক্টরে একটি ঐচ্ছক initialEmail নেয়, যা ইমইল ফল্ডি প্র-ফলি করতে ব্যবহৃত হয়।

createState()

_VerifyEmailScreenState-এর নতুন ইনস্ট্যান্স তৈরি করে রিটার্ন করে।

ক্লাস: _VerifyEmailScreenState

ইমইল ও কোড ইনপুট ফল্ডি, ভেরিফিকেশন সাবমিশন এবং কোড পুনরায় পাঠানোর (resend) লজিক নিয়ন্ত্রণ করে।

dispose()

_email ও _code কন্ট্রোলার ডিসপোজ করে এবং সুপারক্লাসের dispose() কল করে।

_verify()

ফর্ম ভ্যালিডেশন যাচাই করে, _loading স্টেট করে, authRepositoryProvider.verifyEmail() কল করে ট্রিম করা ইমইল ও কোড পাঠায়। সফল হলে একটি সিন্থাকবার দেখায় এবং Routes.login-এ নভিগেট করে দেয়। ApiException-এর বার্তা বা জনেরেকি এর বার্তা _error-এ স্টেট করে, শেষে _loading ফলস করে।

_resend()

প্রথমে ইমইল ফল্ডি খালি বা অবধি যাচাই করে (এক্ষেত্রে সরাসরি এর দেখিয়ে থেমে যায়); বধি হলে _resending স্টেট করে authRepositoryProvider.resendVerification() কল করে। সফল হলে একটি সাধারণ নরিপক্ষে, অ্যাকাউন্ট থাকা/না-থাকা ফাঁস না করে এমন) নোটশি বার্তা দেখায়; ব্যর্থ হলে এর বার্তা দেখায়।

build(BuildContext context)

শরিনোম, ব্যাখ্যা টেক্সট, এরর/নোটশি ব্যানার, ইমইল ফল্ডি, ৬-সংখ্যার কোড ইনপুট ফল্ডি (কেবল সংখ্যা, সর্বোচ্চ ৬ অক্ষর, মাঝে ফাঁকা জায়গাসহ বড় ফন্টে দেখানো), একটি "Verify email" বাটন এবং একটি "Resend code" টেক্সট বাটন সাজিয়ে UI তৈরি করে।

ফাইল: lib/features/biometric/biometric_models.dart

সংক্ষিপ্ত ববিরণ: বায়োমেট্রিক অ্যাটনেডন্স টার্মিনাল (ডভাইস) এবং তাত কর্মীদরে এনরোলমেন্ট সংক্রান্ত মডলে ক্লাস ও ধ্রুবক সংজ্ঞায়িত করে। ইচ্ছাকৃতভাবে এনরোল/ভেরিফাই করার মডলে এখন নেই, কারণ সেগুলো টার্মিনালের নজিস্ব SDK থেকে বায়োমেট্রিক টেমপ্লেটে পাঠায়, ফোন থেকে নয়।

ক্লাস: BiometricPermissions

abstract final class, শুধু দুটি স্ট্যাটিক পারমিশন কোড ধ্রুবক ধারণ করে — কোনো ইনস্ট্যান্স মেথড নেই। - view = 'BIOMETRIC_VIEW' - manage = 'BIOMETRIC_MANAGE'

টপ-লেভেলে ধ্রুবক

- biometricDeviceTypes — ব্যাকএন্ডের BiometricDeviceType enum-এর মরি, ডভাইসের ধরনের তালিকা (ফাঙিগারপ্রিন্ট, ফসেয়াল রকিগনশিন, RFID ইত্যাদি)। - biometricDeviceStatuses — ব্যাকএন্ডের BiometricDeviceStatus enum-এর মরি (ACTIVE, INACTIVE, MAINTENANCE, OFFLINE); এটি অনলাইন/অফলাইন থেকে ভিন্ন ধারণা।

ক্লাস: BiometricDevice

একটি অ্যাটনেডেন্স টার্মিনালের সম্পূর্ণ তথ্য ধারণ করে — নাম, স্ট্যাটাস, নটেওয়ার্ক ঠিকানা, ক্যাপাসিটি, এনরোলমেন্ট সংখ্যা ইত্যাদি।

whereAndHow (getter)

location এবং (থাকলে) ipAddress:portNumber জোড়া করে একটি একক বর্ণনামূলক স্ট্রিং তৈরি করে, যখনে থাকলে তা কীভাবে পৌঁছানো যায় তা বোঝায়।

usage (getter)

enabledForCheckIn ও enabledForCheckOut অনুযায়ী "In and out", "In only", "Out only" অথবা "Records nothing" রিটার্ন করে।

capacityUsed (getter)

maxEnrollments না থাকলে বা শূন্য/নগেটেভি হলে null রিটার্ন করে (যাতে "কোনো লিমিট নেই" আর "কোনো এনরোলমেন্ট নেই" গুলিয়ে না যায়); অন্যথায় totalEnrollments / maxEnrollments অনুপাত ০ থেকে ১-এর মধ্যে ক্ল্যাম্প করে রিটার্ন করে।

BiometricDevice.fromJson(Map<String, dynamic> json) (factory)

JSON থেকে BiometricDevice তৈরি করে, প্রতিটি ফিল্ডের জন্য যুক্তসিঙিত ডিফল্ট বসায়। isOnline ফিল্ডের জন্য বিশেষভাবে json['isOnline'] অথবা json['online'] — দুটোই চেষ্টা করে, কারণ ব্যাকএন্ডের Jackson সিরিয়লাইজেশন বুলিয়ান গটোরকে online নামে সিরিয়লাইজ করতে পারে।

ক্লাস: BiometricDeviceRequest

ডভাইস তৈরি/আপডেট করার (POST/PATCH /company/biometric/devices) রকিওয়েস্ট বডি মডলে করে। আপডেট এন্ডপয়েন্ট সাতটি ফিল্ড শর্তহীনভাবে সেট করে দেয়, তাই সবগুলো সবসময় পাঠাতে হয়; portNumber একটি প্রমিটিভি int বলে অনুপস্থিতি থাকলে শূন্য হয়ে যায় এবং ডভাইসের পোর্ট হারিয়ে যেতে পারে।

BiometricDeviceRequest.from(BiometricDevice device) (factory)

একটি বিদ্যমান BiometricDevice থেকে এডটি-ফর্মের প্রাথমিক মান হিসেবে একটি BiometricDeviceRequest তৈরি করে, deviceType না থাকলে biometricDeviceTypes তালিকার প্রথম মান ব্যবহার করে।

toJson()

একটি Map<String, dynamic> তৈরি করে যেখানে portNumber সবসময় থাকে (প্রমিটিভি int বলে), আর ipAddress/location/departments/notes একটি স্থানীয় clean() হলেপার দিয়ে ট্রিম করে খালি হলে null করে দেয় — কনিতু কী হিসেবে বাদ না দিয়ে স্পষ্ট null হিসেবে পাঠায়, যাতে সার্ভার-সাইড আনগার্ডেডে অ্যাসাইনমেন্টে ভুলবশত পুরনো মান রয়ে না যায়। matchThreshold conditional spread (?matchThreshold) দিয়ে শুধু থাকলেই যোগ হয়।

ক্লাস: BiometricEnrollment

একজন কর্মীর একটি নির্দিষ্ট ডভাইসে এনরোলমেন্টের তথ্য ধারণ করে। বায়োমট্রিক টেমপ্লেট নজি এখানে মডলে করা হয়নি — এটি ফ্রোনের মমেরিতে রাখার কোনো প্রয়োজন নেই।

isUsable (getter)

enrolled && active — এনরোলমেন্টটি প্রকৃতপক্ষে চেক-ইনের জন্য ব্যবহারযোগ্য কনি তা বলে।

failureRate (getter)

successfulMatches + failedMatches শূন্য হলে (অর্থাৎ এখনও কখনো ব্যবহৃত হয়নি) **null** রিটার্ন করে; অন্যথায় ব্যর্থতার হার (**failedMatches / total**) রিটার্ন করে।

BiometricEnrollment.fromJson(Map<String, dynamic> json) (factory)

JSON থেকে **BiometricEnrollment** তৈরি করে, প্রতিটি সংখ্যাসূচক ফিল্ড নিরাপদে **int/double**-এ রূপান্তর করে এবং ঐচ্ছকি ফিল্ডগুলো **null**-নিরাপদভাবে পার্স করে।

ফাইল: **lib/features/biometric/biometric_repository.dart**

সংক্ষিপ্ত বিবরণ: বায়োমেট্রিকি টার্মিনাল ও এনরোলমেন্ট সংক্রান্ত সব API কল এবং তাদের সংশ্লিষ্ট Riverpod প্রোভাইডার/কন্ট্রোলার ধারণ করে। এনরোল/ভেরিফাই এন্ডপয়েন্ট ইচ্ছাকৃতভাবে এখানে নেই।

ক্লাস: **BiometricRepository**

ApiClient মুড়িয়ে বায়োমেট্রিকি-সম্পর্কিত সব HTTP কল প্রকাশ করে।

devices({String? status, int page = 0, int size = 30})

সব ডিভাইস, নির্দিষ্ট স্ট্যাটাসের ডিভাইস, অথবা শুধু এখন-অনলাইন ডিভাইস আনে। **status** অনুযায়ী পাথ ঠিক করে: **null** হলে বেসে পাথ, **onlineDevicesFilter** হলে **/online** সাব-পাথ, অন্যথায় **/status/\$status**। মন্তব্য অনুযায়ী switch-এর বদলে if-else ব্যবহার করা হয়েছে কারণ প্যাটার্ন পজিশনে বয়োর আইডেন্টিফায়ার ধ্রুবককে সাথে তুলনা না করে ভ্যারিয়েবল বাইন্ড করে ফলেত। রজোল্ট **getPaged** দিয়ে আনা হয়, যা পজেড ও বয়োর-লিস্ট উভয় রেসপন্স শেপে পড়তে পারে।

createDevice(BiometricDeviceRequest request)

POST /company/biometric/devices কল করে নতুন ডিভাইস তৈরি করে এবং JSON থেকে **BiometricDevice** পার্স করে রিটার্ন করে।

updateDevice(int id, BiometricDeviceRequest request)

PATCH /company/biometric/devices/\$id কল করে পুরো রকিওয়েস্ট বডি পাঠায় (কোন ফিল্ড আনগার্ডেডে অ্যাসাইন হয় তা **BiometricDeviceRequest** ডকুমেন্টেশনে বলা আছে) এবং আপডেট হওয়া **BiometricDevice** রিটার্ন করে।

setDeviceStatus(int id, String status)

PATCH /company/biometric/devices/\$id/status?status=\$status কল করে, খালি রেসপন্স আসে — কলার নজিই স্টেটে আপডেট করে নেয়।

setDeviceOnline(int id, bool online)

PATCH .../\$id/online-status?online=\$online কল করে ডিভাইসকে ম্যানুয়ালি অনলাইন/অফলাইন চহ্নিতি করে, সাধারণত ডিভাইস নজিই এটা রপিোর্ট করে, কন্িতু হলেথ-চকেরে অপক্েষা না করে একটি ডাউন টার্মিনালকে রোটেশন থেকে বাদ দতিে এটি কাজে লাগে।

recordSync(int id)

POST .../\$id/sync কল করে ডিভাইসটি সিবমোত্ৰ সঙ্ক্ৰোনাইজ হয়েছে তা রকের্ড করে, খালি রেসপন্স আসে।

deleteDevice(int id)

DELETE /company/biometric/devices/\$id কল করে ডিভাইস মুছে ফলে।

enrollmentsFor(int employeeId)

GET /company/biometric/employee/\$employeeId কল করে একজন কর্মীর সব এনরোলমেন্ট (বেয়োর লস্ট আকারে) আনে ও BiometricEnrollment.fromJson দিয়ে ম্যাপ করে অপরিবর্তনীয় লস্ট রটার্ন করে।

deleteEnrollment(int id)

DELETE /company/biometric/\$id কল করে একটি এনরোলমেন্ট বাতিল করে; সংশ্লিষ্ট ব্যক্তি টার্মিনাল গিয়ে আবার এনরোল করতে পারবেন।

টপ-লেভেলে ধ্রুবক ও প্রোভাইডার

onlineDevicesFilter

'ONLINE_NOW' — BiometricDeviceStatus-এর প্রকৃত মান নয়, বরং /online ফিল্টারের জন্য একটি ছদ্ম-স্ট্যাটাস, যা "এখন কী রচিবেল" জিজ্ঞাসা করে, "কী অবস্থায় আছে" নয়।

biometricRepositoryProvider

ApiClient (via apiClientProvider) দিয়ে একটি BiometricRepository তৈরি করে দেয়।

ক্লাস: DeviceFilterController

Notifier<String?> — বর্তমানে কোন স্ট্যাটাস ফিল্টার সক্রিয় তা ধরে রাখে (null মানে সবকিছু)।

build()

প্রাথমিক স্টেটে হিসেবে null রটার্ন করে।

set(String? status)

নতুন ফিল্টার মান আগের মতোই হলে কিছু করে না; ভিন্ন হলে state আপডেট করে।

ক্লাস: BiometricDevicesController

AsyncNotifier<List<BiometricDevice>> — ডভাইস তালিকা লোড, রিফ্রেশ এবং লোকাল স্টেটে আপডেট (optimistic) পরিচালনা করে।

build()

currentUserProvider ও deviceFilterProvider watch করে (যাতে ইউজার বদল বা ফিল্টার বদলে অটো-রিবিল্ড হয়) এবং _load() কল করে প্রাথমিক ডভাইস তালিকা রটার্ন করে।

_load()

বর্তমান ফিল্টার অনুযায়ী biometricRepositoryProvider.devices() কল করে পজেবল content রটার্ন করে।

refresh()

AsyncValue.guard(_load) ব্যবহার করে স্টেটে আবার লোড করে, এরর ঘটলেও তা AsyncValue-তে নরিপদে ধরা পড়ে।

apply(BiometricDevice updated)

বর্তমান তালিকায় (যদি লোড থাকে) মিলে যাওয়া id-এর ডভাইসটি updated দিয়ে প্রতিস্থাপন করে নতুন AsyncValue.data স্টেটে বসায় — সার্বভারে রাউন্ড-ট্রপি ছাড়াই optimistic আপডেট।

remove(int id)

বর্তমান তালিকা থেকে নির্দিষ্ট id-এর ডভাইস বাদ দিয়ে নতুন স্টেটে বসায়।

প্রোভাইডার

- deviceFilterProvider — DeviceFilterController-এর NotifierProvider। - biometricDevicesProvider — BiometricDevicesController-এর AsyncNotifierProvider। - enrollmentsProvider — employeeId দিয়ে ফ্যামলিকিরা FutureProvider.autoDispose, একজন কর্মীর এনরোলমেন্ট আনবে এবং স্ক্রিনি বন্ধ হলে অটো-ডিসপোজ হয়।

ফাইল: lib/features/biometric/biometric_screen.dart

সংক্ষিপ্ত ববিরণ: অ্যাটনেডনেস টার্মিনালরে তালিকা ও ব্যবস্থাপনা স্ক্রিনি (BiometricScreen), এবং একজন নরিদষ্টি কর্মীর এনরোলমেন্ট দেখা ও বাতলি করার স্ক্রিনি (EnrollmentsScreen)।

ক্লাস/উইজটে: BiometricScreen

ConsumerStatefulWidget, স্টেটে _BiometricScreenState-এ রাখা।

createState()

_BiometricScreenState-এর নতুন ইনস্ট্যান্স রিটার্ন করে।

ক্লাস: _BiometricScreenState

টার্মিনাল তালিকার প্রতটি অ্যাকশন (স্ট্যাটাস বদল, অনলাইন টগল, সঙ্কি রকের্ড, ডলিটি) এবং একজন কর্মী খুঁজে তার এনরোলমেন্ট দেখার লজিকি ধারণ করে।

_run(int id, Future<void> Function() action, String failure, {BiometricDevice? optimistic, String? success})

একটি সাধারণীকৃত হেল্পার — _busyId স্টেটে করে অ্যাকশন চালায়; optimistic দেওয়া থাকলে (স্ট্যাটাস/অনলাইন/সঙ্কি এন্ডপয়েন্ট খালিবিডি রিটার্ন করে বলে) BiometricDevicesController.apply() দিয়ে লোকাল স্টেটে আপডেটে করে, নাহলে পুরো তালিকা refresh() করে। সফল হলে ঐচ্ছকি success বার্তা স্ন্যাকবারে দেখায়; ApiException বা অন্য এক্সপেশন যথাক্রমে সার্তার বার্তা বা failure বার্তা দেখায়; শেষে _busyId ক্লয়ির করে।

_setStatus(BiometricDevice device)

pickOne() দিয়ে একটি স্ট্যাটাস পকির দেখায়, বাছাই বাতলি বা অপরিবর্তিতি হলে কিছু করে না; নতুন স্ট্যাটাস বাছা হলে _run() দিয়ে repo.setDeviceStatus() কল করে, _withStatus() দিয়ে তরৈ optimistic ডিভাইস পাঠায়।

_toggleOnline(BiometricDevice device)

বর্তমান isOnline-এর উল্টো মান নিয়ে repo.setDeviceOnline() কল করে, _run()-এর মাধ্যমে optimistic আপডেটেসহ।

_sync(BiometricDevice device)

repo.recordSync() কল করে, কন্ট্রি optimistic ডিভাইস না পাঠিয়ে পুরো তালিকা রফ্রিশে করে (কারণ সঙ্কি-টাইম সার্তার থেকে আসে), সফল হলে ডিভাইসের নাম উল্লেখ করে সাফল্য বার্তা দেখায়।

_delete(BiometricDevice device)

একটি নিশ্চিতকরণ ডায়ালগ দেখায় — এনরোলমেন্ট থাকলে সতর্কবার্তা ভিন্ন হয়। নিশ্চিত হলে _busyId স্টেটে করে repo.deleteDevice() কল করে, সফল হলে BiometricDevicesController.remove() দিয়ে তালিকা থেকে বাদ দেয় ও স্ন্যাকবার দেখায়; ব্যর্থতায় যথায়থ এরর বার্তা দেখায়।

`_lookupPerson()`

`EmployeePicker.show()` দ্বিধি ংকজন কৰ্মী বাছাই কৰাৰ শীট খোলো; বাছাই হলে `EnrollmentsScreen.open()` কল কৰে সেই কৰ্মীৰ ংনৰোলমেন্ট স্ক্ৰিনি খোলো।

`build(BuildContext context)`

পারমশিন লোড না হলে লোডাৰ দখোয়; `BiometricPermissions.view` না থাকলে "Not available to you" ংম্পট-স্টটে দখোয়। ংন্যথায় ংযাপবার (কৰ্মী খোঁজাৰ আইকনসহ), ম্যানজে পারমশিন থাকলে "New" ফ্লোটিং ংযাকশন বাটন (`showDeviceSheet()` খোলো), ংবং `ConfigList<BiometricDevice>` দ্বিধি ফল্টিটারবারসহ ডভাইস তালকি রনুডাৰ কৰে; প্ৰতটি আইটমেরে জন্থ `_DeviceCard` তরৈ কৰে, ম্যানজে পারমশিন ংনুযায়ী কলব্যাকগুলো শরুতসাপক্ষে দয়ে।

টপ-লভেলে ফাংশন

`_withStatus(BiometricDevice device, {String? status, bool? online})`

`device`-ংর ংকট কিপতিরৈ কৰে যখনে শুধু `status` ংবং/ংথবা `isOnline` ফল্ডি বদলানো হয়, বাকসিব ফল্ডি ংপৰবিরুতি থাকে — কাৰণ স্ট্যাটাস/ংনলাইন ংন্ডপয়ন্ট খালি বিডরিটারন কৰে, তাই সারুভাৰ রসেপন্স থেকে পারস কৰাৰ কছি নই।

ক্লাস/ংইজটে: `_DeviceCard`

`StatelessWidget` — ংকট টার্মিনিালরে কারুড রনুডাৰ কৰে: ংনলাইন-সূচক ডট, নাম, স্ট্যাটাস চপি, ংকট পিপংপ মনু (ংডিটি, স্ট্যাটাস বদল, ংনলাইন/ংফলাইন টগল, সঙ্কি রকেৰুড, রম্ভ — যগেলোর কলব্যাক `null` নয় শুধু সগেলোই দখোয়), ডভাইসরে ধরন/ব্যবহার/ংবস্থান, ক্যাপাসটি প্ৰোগ্ৰসে বার (থাকলে, ৯০%+ হলে সতরুতা রঙে) ংবং ংনৰোলমেন্ট/সঙ্কি সারাংশ।

`build(BuildContext context)`

ংপরেরে সব ংপাদান ংকত্ৰ কৰে `AppCard`-ংর ভতেরে সাজয়িে রটারন কৰে; `busy` `true` হলে ংযাকশন মনুর বদলে ংকট ছোট লোডিং ংন্ডকিটের দখোয়।

ক্লাস/ংইজটে: `EnrollmentsScreen`

`ConsumerWidget` — ংকজন নরিদষ্টি কৰ্মীর (`employeeId`, `name`) সব বায়োমটেরকি ংনৰোলমেন্ট দখোয় ংবং প্ৰতটি বাতলি কৰাৰ সুবধি দয়ে।

`EnrollmentsScreen.open(BuildContext context, {required int employeeId, required String name})`
(**static**)

`MaterialPageRoute` দ্বিধি `EnrollmentsScreen` পুশ কৰে নতুন স্ক্ৰিনি খোলো।

`_revoke(BuildContext context, WidgetRef ref, BiometricEnrollment enrollment)`

ংকট নিশ্চিতকরণ ডায়াগ দখোয় (কৰ্মীর নাম ও ডভাইসরে নাম ংল্লখেসহ); নিশ্চিতি হলে `repo.deleteEnrollment()` কল কৰে, সফল হলে `enrollmentsProvider(employeeId)` ংন্ডযালডিটে কৰে তালকি রফি়রশে কৰে ংবং ংকট "Revoked." সন্যাকবার দখোয়; ব্যরুথতায় সংশ্লষ্টি ংরর বারুতা দখোয়।

`build(BuildContext context, WidgetRef ref)`

`ConfigList<BiometricEnrollment>` দ্বিধি কৰ্মীর ংনৰোলমেন্ট তালকি রনুডাৰ কৰে; প্ৰতটি সারতিে ডভাইসরে নাম, সক্রয়িতা (`isUsable`), ংনৰোলমেন্টেরে ধরন/তারথি/ব্যরুথতার হার, শেষে ব্যবহারেরে সময় ংবং (ম্যানজে পারমশিন থাকলে) ংকট "Revoke" ংযাকশন দখোয়।

ফাইল: lib/features/biometric/biometric_sheets.dart

সংক্ষিপ্ত ববিরণ: বায়োমেট্রিক টার্মিনাল তৈরি বা এডটি করার জন্য একটি বিটম শীট ফর্ম প্রদান করে।

টপ-লেভেলে ফাংশন

`showDeviceSheet(BuildContext context, {BiometricDevice? existing})`

`showModalBottomSheet` দিয়ে `_DeviceSheet` (স্ক্রল-কন্ট্রোলড) খোলবে; `existing` দিলে এডটি মোড, না দিলে নতুন-তৈরি মোড।

ক্লাস/উইজটে: `_DeviceSheet`

`ConsumerStatefulWidget` — ডিভাইস তৈরি/এডটি ফর্মের জন্য এন্ট্রি পয়েন্ট, `existing` ঐচ্ছকি ফিল্ডে বদ্যমান ডিভাইস রাখবে।

`createState()`

`_DeviceSheetState`-এর নতুন ইনস্ট্যান্স রিটার্ন করে।

ক্লাস: `_DeviceSheetState`

টার্মিনাল ফর্মের সব কন্ট্রোলার (নাম, ডিভাইস আইডি, IP, পোর্ট, লোকেশন, ডিপার্টমেন্ট, নোটস, ম্যাচ থ্রেশহোল্ড), ড্রপডাউন টাইপ, চেক-ইন/চেক-আউট সুইচ এবং সাবমিশন লজিক পরিচালনা করে। পোর্ট ও থ্রেশহোল্ড ফিল্ড ফাঁকা রাখা যায় না কারণ এগুলো ব্যাকএন্ডে প্রমিটিভি `int` হিসেবে ডিফল্ট শূন্য হয়ে যায়।

`_isEdit` (**getter**)

`widget.existing != null` — ফর্মটি এডটি মোডে আছে কিনা তা বলে।

`dispose()`

সবগুলো `TextEditingController` (`_name`, `_deviceId`, `_ip`, `_port`, `_location`, `_department`, `_notes`, `_threshold`) ডিসপোজ করে এবং সুপারক্লাসের `dispose()` কল করে।

`_submit()`

ফর্ম ভ্যালিডেশন করা যাচাই করে, `_submitting` সেট করে, ফিল্ড থেকে একটি `BiometricDeviceRequest` তৈরি করে (`portNumber/matchThreshold int.tryParse` দিয়ে পার্স করে)। এডটি মোডে `repo.updateDevice()` কল করে ফলাফল `BiometricDevicesController.apply()`-এ পাঠায়; নতুন-তৈরি মোডে `repo.createDevice()` কল করে পুরো তালিকা `refresh()` করে। সফল হলে শীট বন্ধ করে একটি সফলতা সন্ধ্যাকবার দেখায় ("Terminal updated." বা "Terminal added.")। `ApiException`-এ (যেমন ডুপ্লিকিটে ডিভাইস আইডি) সার্বভার বার্তা `_error`-এ বসায়, অন্য এক্সপেশনে জেনেরিক বার্তা দেখায়, শেষে `_submitting` ফলস করে।

`build(BuildContext context)`

`FormSheetFrame`-এর ভেতরে সব ফিল্ড সাজায়: নাম, ডিভাইস আইডি (এডটি মোডে `readOnly`, কারণ সার্বভার আপডেটে সেট উপকেষ্টা করে), ধরন ড্রপডাউন, IP ও পোর্ট (পাশাপাশি, পোর্ট ভ্যালিডেটেডে ০-৬৫৫৩৫), লোকেশন, ডিপার্টমেন্ট, ম্যাচ থ্রেশহোল্ড (১-১০০, ঐচ্ছকি), নোটস, এবং চেক-ইন/চেক-আউট রেকর্ডিং সুইচ। দুটি সুইচই বন্ধ থাকলে একটি সিতরকবার্তা ("With both off the terminal records nothing at all.") দেখায়।

ফাইল: lib/features/budgets/budget_models.dart

সংক্ষিপ্ত ববিরণ: বাজেটে (Budget) ফচারের ডটো মডলে এবং পারমিশন কনস্ট্র্যান্ট ধারণ করে। এখানে ফসিক্যাল ইয়ার-ভিত্তিক ক্যাটাগরি বাজেটে ও তার বপিরাতে প্রকৃত খরচের হিসাব রাখার জন্য ক্লাস আছে। প্রতিটি বাজেটে এন্ট্রিসারভার-সাইড থেকে হিসাব করা `actualSpend`, `remaining`, `usedPercent`, `overBudget` মান নিয়ে আসে।

ক্লাস/উইজটে: BudgetPermissions

`abstract final class` — বাজেটে ফচারের দুটি পারমিশন কোড (`view`, `manage`) স্ট্যাটিক কনস্ট্র্যান্ট হিসেবে ধরে রাখে, যা UI-তে অ্যাক্সেস কন্ট্রোলের জন্য ব্যবহৃত হয়।

`static const view`

'BUDGET_VIEW' — বাজেটে স্ক্রিনি দেখার অনুমতির কোড।

`static const manage`

'BUDGET_MANAGE' — বাজেটে তৈরি/সম্পাদনা/মুছে ফেলার অনুমতির কোড।

ক্লাস/উইজটে: Budget

একটি নির্দিষ্ট ক্যাটাগরি ও ফসিক্যাল ইয়ারের জন্য নির্ধারণিত ব্যয়সীমা এবং তার বপিরাতে প্রকৃত খরচের লাইভ রোলআপ ধারণ করে। `actualSpend`, `remaining`, `usedPercent`, `overBudget` — এই মানগুলো ব্যবহারকারী নজরে টাইপ করে না, বরং সারভার এগুলো অনুমোদিত/পরিশোধিত খরচ এবং পোস্ট করা ভেন্ডর বলি যোগ করে হিসাব করে পাঠায়।

`Budget({required this.id, ...})`

কনস্ট্রাক্টর — `id`, `category`, `fiscalYear`, `amount`, `actualSpend`, `remaining`, `usedPercent`, `overBudget` আবশ্যিক প্যারামিটার হিসেবে নেয়, আর `notes` ঐচ্ছিক।

`double get progress`

প্রোগ্রামে বার আঁকার জন্য ০ থেকে ১-এর মধ্যে ক্যালকুলেশন করা মান দেয় (`actualSpend / amount`)। `usedPercent` ১০০ ছাড়িয়ে যেতে পারে ওভারবাজেটের ক্ষেত্রে, কিন্তু সরাসরি স্টো বার ব্যবহার করলে বার ওভারফ্লো করে যেতে বলই আলাদা করে ক্যালকুলেশন করা এই গটোর আছে। `amount` শূন্য বা তার কম হলে ০ রিটার্ন করে।

`factory Budget.fromJson(Map<String, dynamic> json)`

সারভার থেকে আসা JSON ম্যাপকে `Budget` অবজেক্টে রূপান্তর করে। প্রতিটি ফিল্ডের জন্য null-safe ডিফল্ট মান আছে (যেমন সংখ্যার জন্য ০, স্ট্রিং-এর জন্য খালি স্ট্রিং, বুলিয়ানের জন্য `false`), যাত্রে অসম্পূর্ণ রেসপন্সও ক্র্যাশ না করে।

ক্লাস/উইজটে: BudgetRequest

POST এবং PUT `/company/finance/budgets` কল করার জন্য ব্যবহৃত রিকোয়েস্ট DTO। প্রতিটি `category+fiscalYear` জোড়ার জন্য একটিমাত্র বাজেটে থাকতে পারে — ব্যাকএন্ড ডুপ্লিকেট তৈরি চেষ্টায় উভয় নাম উল্লেখ করে প্রত্যাখ্যান করে।

`BudgetRequest({required this.category, required this.fiscalYear, required this.amount, this.notes})`

কনস্ট্রাক্টর — একটি নতুন বা আপডেট হওয়া বাজেটে রিকোয়েস্ট তৈরি করে।

Map<String, dynamic> toJson()

রকি়োস্ট বডি তৈরি করে সার্ভারে পাঠানোর জন্য। `category` ও `notes` ট্রিম করা হয়, আর `notes` খালি হলে JSON থেকে বাদ দেওয়া হয় (if কন্ডিশনাল কী)।

ফাইল: lib/features/budgets/budget_repository.dart

সংক্ষিপ্ত ববিরণ: বাজটে ফচারের জন্য নটেওয়ার্ক লয়ার এবং Riverpod স্টেটে ম্যানেজমেন্ট প্রোভাইডার সংজ্ঞায়িত করে। `BudgetRepository` API কল হ্যান্ডলে করে, আর `BudgetsController/BudgetYearController` UI-এর জন্য স্টেটে রাখে ও সঙ্ক্রোনাইজ করে।

ক্লাস/উইজটে: BudgetRepository

`ApiClient`-এর উপর মোড়ক (wrapper) হিসেবে বাজটে সংক্রান্ত সব HTTP অপারেশন (লিস্ট, ক্যাটাগরি, তৈরি, আপডেট, মুছে ফেলো) প্রকাশ করে।

`BudgetRepository(this._api)`

কনস্ট্রাক্টর — একটি `ApiClient` ইনস্ট্যান্স নিয়ে রপিজিটরি তৈরি করে।

`Future<List<Budget>> listForYear(int fiscalYear)`

নরিদর্শিত ফসিক্যাল ইয়ারের সব বাজটে এন্ট্রি `GET /company/finance/budgets?fiscalYear=...` কল করে আনে এবং `Budget.fromJson` দিয়ে পার্স করে তালিকা রিটার্ন করে।

`Future<List<String>> categories()`

`GET /company/finance/budgets/categories` কল করে ইতিমধ্যে বাজটে করা ক্যাটাগরি নামগুলোর ডিস্টিংক্ট তালিকা আনে, যা এই ফর্ম এবং এক্সপেন্স-ক্লেইম ফর্ম সাজেশন হিসেবে ব্যবহৃত হয়।

`Future<Budget> create(BudgetRequest request)`

`POST /company/finance/budgets` কল করে নতুন বাজটে তৈরি করে এবং তৈরি হওয়া `Budget` অবজেক্ট রিটার্ন করে।

`Future<Budget> update(int id, BudgetRequest request)`

`PUT /company/finance/budgets/$id` কল করে বর্ধমান বাজটে আপডেট করে এবং আপডেটেড `Budget` রিটার্ন করে।

`Future<void> delete(int id)`

`DELETE /company/finance/budgets/$id` কল করে নরিদর্শিত বাজটে মুছে ফেলে।

টপ-লেভেলে: budgetRepositoryProvider

`Provider<BudgetRepository>` — `apiClientProvider`-এর উপর নির্ভর করে একটি `BudgetRepository` ইনস্ট্যান্স সরবরাহ করে।

ক্লাস/উইজটে: BudgetYearController

`Notifier<int>` — বর্তমানে বাজটে স্ক্রিনি কৌন ফসিক্যাল ইয়ার দেখানো হচ্ছে তা ধরে রাখে। ডিফল্ট হিসেবে বর্তমান বছর ব্যবহার করে, কারণ বেশিরভাগ ক্ষেত্রেই সটোই সঠিক।

`int build()`

প্রাথমিক স্টেটে হিসেবে `DateTime.now().year` রিটার্ন করে।

`void set(int year)`

প্রদত্ত বছরে স্টেটে পরিবর্তন করে। বছর একই হলে কোনো কাজ করে না (অপ্রয়োজনীয় রিবিল্ড এড়াতে)।

টপ-লেভেল: `BudgetYearProvider`

`NotifierProvider<BudgetYearController, int>` — `BudgetYearController` এর প্রোভাইডার ইনস্ট্যান্স।

ক্লাস/উইজটে: `BudgetsController`

`AsyncNotifier<List<Budget>>` — নির্বাচতি ফসিক্যাল ইয়ারের বাজেটে তালিকা লোড, রিফ্রেশ এবং স্থানীয়ভাবে আপডেট/মুছে ফেলার লজিক রাখবে।

`Future<List<Budget>> build()`

`currentUserProvider` এবং `budgetYearProvider` ওয়াচ করে (যাতে ইউজার বা বছর পরিবর্তন হলে অটো-রিবিল্ড হয়) এবং `_load()` কল করে প্রাথমিক ডেটা লোড করে।

`Future<List<Budget>> _load()`

`budgetRepositoryProvider` থেকে বর্তমান নির্বাচতি ফসিক্যাল ইয়ারের বাজেটে তালিকা আনবে।

`Future<void> refresh()`

`AsyncValue.guard` দিয়ে `_load()` পুনরায় চালিয়ে স্টেটে রিফ্রেশ করে, এরর হলে সেটোও `AsyncValue`-তে ধরা পড়ে।

`void apply(Budget updated)`

বর্তমান তালিকায় প্রদত্ত `id` এর বাজেটে এন্ট্রিটি নতুন `updated` মান দিয়ে প্রতিস্থাপন করে (সম্পাদনার পর সার্ভার-রাউন্ডট্রপি ছাড়াই)। আপডেট করতে ব্যবহৃত হয়।

`void remove(int id)`

বর্তমান তালিকা থেকে প্রদত্ত `id` এর বাজেটে এন্ট্রি বাদ দিয়ে (মুছে ফেলার পর)। আপডেট করতে ব্যবহৃত হয়।

টপ-লেভেল: `budgetsProvider`

`AsyncNotifierProvider<BudgetsController, List<Budget>>` — `BudgetsController`-এর প্রোভাইডার।

টপ-লেভেল: `budgetCategoriesProvider`

`FutureProvider<List<String>>` — `budgetRepositoryProvider.categories()` কল করে ক্যাটাগরি সাজেশন তালিকা সরবরাহ করে।

ফাইল: `lib/features/budgets/budget_screen.dart`

সংক্ষিপ্ত ববিরণ: বাজেটে স্ক্রিনিং U। বাস্তবায়ন করে — ফসিক্যাল ইয়ার নভিগেশন, বাজেটে তালিকা প্রদর্শন (প্রোগ্রামে বার-সহ কার্ড আকারে), এবং তৈরি/সম্পাদনার জন্য বটম শিট। পারমিশন-ভিত্তিক অ্যাক্সেসে কন্ট্রোল অন্তর্ভুক্ত।

ক্লাস/উইজটে: BudgetScreen

ConsumerWidget — বাজটে ফচারের মূল স্ক্রিনি। পারমিশন যাচাই করে (লোডিং/নট-অথরাইজড অবস্থা হ্যান্ডলে করে), অ্যাপবারে বছর নভিগেশন দেখায়, এবং **_BudgetList** উইজটে বডিসিবে রেন্ডার করে। **canManage** থাকলে "New" ফ্লোটিং অ্যাকশন বাটন দেখায়।

Widget build(BuildContext context, WidgetRef ref)

পারমিশন স্টেটে চেক করে তিনটি সম্ভাব্য অবস্থা রেন্ডার করে: (১) পারমিশন এখনো লোড হয়নি — লোডার দেখায়, (২) **BUDGET_VIEW** পারমিশন নেই — এম্পটি স্টেটে বার্তা দেখায়, (৩) স্বাভাবিক — অ্যাপবার (বছর প্রভিডার/নক্সট বাটনসহ), ফ্লোটিং অ্যাকশন বাটন (ম্যানজে পারমিশন থাকলে) এবং **_BudgetList** বডিসহ স্ক্র্যাফোল্ড রিটার্ন করে।

ক্লাস/উইজটে: _BudgetList

ConsumerStatefulWidget — বাজটে তালিকা লোড ও প্রদর্শন করার জন্য প্রাইভেট উইজটে, মুছে ফেলার লজিক এখনো থাকে।

_BudgetList({required this.canManage})

কনস্ট্রাক্টর — ব্যবহারকারীর ম্যানজে-পারমিশন আছে কিনা তা নেয়।

ConsumerState<_BudgetList> createState()

_BudgetListState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _BudgetListState

ConsumerState<_BudgetList> — মুছে ফেলার সময় কোন আইটেমে "busy" (লোডিং) দেখাতে হবে তার স্টেটে রাখে এবং মুছে ফেলার confirm-ও-execute ফ্লো পরিচালনা করে।

Future<void> _delete(Budget budget)

confirmAction ডায়ালগে নিশ্চিতকরণ চায়; নিশ্চিত হলে **_busyId** স্টেটে করে রপিজিটরি থেকে **delete** কল করে, সফল হলে **budgetsProvider.notifier.remove** দিয়ে লোকাল স্টেটে আপডেট করে ও সন্ধ্যাকবার দেখায়। **ApiException** বা অন্য এরর হলে আলাদা এরর বার্তা দেখায়, শেষে **_busyId** রিসেটে করে।

Widget build(BuildContext context)

budgetsProvider ওয়াচ করে **ConfigList<Budget>** উইজটে রিটার্ন করে, যখনে রফ্রেশে, খালি/এরর স্টেটে বার্তা এবং প্রতিটি বাজটেইর জন্য **_BudgetCard** আইটেমে বন্ডার সংজ্ঞায়িত করা আছে।

ক্লাস/উইজটে: _BudgetCard

StatelessWidget — একটি একক বাজটে এন্ট্রির কার্ড রেন্ডার করে: ক্যাটাগরিনাম, ওভারবাজটে চপি (যদি প্রয়োজ্য), পরিমাণ, এডিট/ডিলিট অ্যাকশন মেনু, প্রোগ্রসে বার (রঙ থ্রেশহোল্ড-ভিত্তিক: সবুজ/হলুদ/লাল), এবং খরচ/অবশিষ্ট/শতাংশের সারাংশ লাইন।

_BudgetCard({required this.budget, required this.busy, this.onEdit, this.onDelete})

কনস্ট্রাক্টর — বাজটে ডটো, **busy** ফ্ল্যাগ এবং ঐচ্ছিক এডিট/ডিলিট কলব্যাক নেয়।

Widget build(BuildContext context)

usedPercent অনুযায়ী বার রঙ নির্ধারণ করে (>১০০ = danger, ৮০ = warning, নাহলে success), এডিট/ডিলিট অ্যাকশন তালিকা তৈরি করে, এবং কার্ডের পূরণ লেআউট (হেডার, প্রোগ্রসে বার, সারাংশ টেক্সট) রেন্ডার করে। **busy** হলে অ্যাকশন মেনুর জায়গায় স্পনির দেখায়।

টপ-লভেলে ফাংশন: showBudgetSheet

```
`dart Future<void> showBudgetSheet(BuildContext context, {required int fiscalYear, Budget? existing})`  
`মডাল বটম শিট হিসেবে _BudgetSheet খুলে দেয়। existing` দেওয়া হলে এডটি মোড, না দিলে ক্রয়টি মোড।
```

ক্লাস/উইজটে: _BudgetSheet

ConsumerStatefulWidget — বাজটে তৈরি বা সম্পাদনার জন্য ফর্ম শিট।

```
_BudgetSheet({required this.fiscalYear, this.existing})
```

কনস্ট্রাক্টর — বর্তমান ফসিক্যাল ইয়ার ও ঐচ্ছকি বদ্যমান বাজটে (এডটিরে জন্য) নয়।

```
ConsumerState<_BudgetSheet> createState()
```

_BudgetSheetState ইনস্ট্যান্স তৈরিকরে।

ক্লাস/উইজটে: _BudgetSheetState

ConsumerState<_BudgetSheet> — ফর্মের টেক্সট কন্ট্রোলার, ভ্যালিডেশন এবং সাবমিট লজিকি ধারণ করে।

```
bool get _isEdit
```

widget.existing != null হলে true — এডটি নাকি ক্রয়টি মোড তা নির্ধারণ করে।

```
void dispose()
```

সব TextEditingController (_category, _fiscalYear, _amount, _notes) ডিসপোজ করে।

```
Future<void> _submit()
```

ফর্ম ভ্যালিডে করে, BudgetRequest তৈরিকরে, এডটি মোডে repo.update কল করে

budgetsProvider.notifier.apply দিয়ে স্টেটে আপডেট করে, নতুন তৈরি ক্ষেত্রে repo.create কল করে পুরো তালিকা refresh করে। সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়; ApiException এলে সারভারের এরর বার্তা সরাসরি দেখায় (যেমন ডুপ্লিকিটে ক্যাটাগরি/বছরের বার্তা)।

```
Widget build(BuildContext context)
```

budgetCategoriesProvider থেকে ক্যাটাগরিসাজেশন নিয়ে Autocomplete<String> ফিল্ড (ক্যাটাগরি নাম), ফসিক্যাল ইয়ার ও পরিমাণের জন্য পাশাপাশি দুটি TextFormField, এবং নোটসের জন্য মাল্টিলাইন ফিল্ড সহ FormSheetFrame রটাইন করে। প্রতিটি ফিল্ডে ইনলাইন ভ্যালিডেটর আছে (যেমন ফসিক্যাল ইয়ার ২০০০-২১০০ এর মধ্যে, পরিমাণ শূন্যের বেশি হতে হবে)।

ফাইল: lib/features/catalogue/catalogue_models.dart

সংক্ষিপ্ত বিবরণ: সারভিসি ক্যাটালগ ফচারের ডটো মডেলে — সারভিসি, টেমপ্লেট, প্যাকজে এবং সাবস্ক্রিপশন — এবং সংশ্লিষ্ট পারমিশন/এনাম কনস্ট্যান্ট ধারণ করে। কোম্পানিকী বক্রিকরে (সারভিসি), কী দিয়ে সারভিসি তৈরি হয় (টেমপ্লেট), কীভাবে সারভিসি বান্ডল হয় (প্যাকজে), এবং ক্লায়েন্ট কীসে সাবস্করাইব করছে তার সব মডেলে এখান।

ক্লাস/উইজটে: CataloguePermissions

abstract final class — সার্ভিস, টেমপ্লেটে ও প্যাকজে রে জন্ম আটটি পারমিশন কোড স্ট্যাটিক কনস্ট্যান্ট হিসেবে ধরে রাখবে।

static const serviceView / serviceCreate / serviceUpdate / serviceDelete

সার্ভিস দেখো/তৈরি/আপডেট/মুছে ফেলার পারমিশন কোড (SERVICE_CATALOG_*)।

static const templateView / templateCreate / templateUpdate / templateDelete

টেমপ্লেটে দেখো/তৈরি/আপডেট/মুছে ফেলার পারমিশন কোড (SERVICE_TEMPLATE_*)।

static const packageView

'SERVICE_PACKAGE_VIEW' — প্যাকজে দেখার পারমিশন কোড।

টপ-লেভেলে কনস্ট্যান্ট: servicePriceTypes

const List<String> — সার্ভিসের মূল্য নির্ধারণের ধরন (FIXED, HOURLY, DAILY, MONTHLY, YEARLY, CUSTOM)। ব্যাকএন্ডের ServicePriceType এনাম প্রতফিলতি করে।

ক্লাস/উইজটে: SubscriptionStatus

abstract final class — সাবস্ক্রিপশনের সম্ভাব্য স্ট্যাটাস কোড এবং একটি settled স্টে স্ট্যাটিক কনস্ট্যান্ট হিসেবে রাখবে। ব্যাকএন্ডের SubscriptionStatus এনাম প্রতফিলতি করে।

static const active / pendingPayment / expired / suspended / cancelled

সাবস্ক্রিপশনের পাঁচটি সম্ভাব্য স্ট্যাটাস মান।

static const settled

{expired, cancelled} — এই দুটি স্ট্যাটাসে থাকলে সাবস্ক্রিপশনে আর কিছু ঘটবে না (চূড়ান্ত অবস্থা)।

ক্লাস/উইজটে: ServiceListing

কোম্পানির ক্যাটালগে থাকা একটি সার্ভিস, যমেনটা অ্যাডমিনি তালিকায় দেখা যায়। রকিওয়েস্ট মডউলের CatalogService-এর চেয়ে আলাদা ক্লাস — এটিতে অ্যাডমিনি-সম্পাদনযোগ্য ফ্ল্যাগ ও প্রাইসিং তথ্য থাকবে।

ServiceListing({required this.id, required this.name, required this.active, ...})

কনস্ট্রাক্টর — id, name, active আবশ্যিক; বাকি সব (বর্ণনা, মূল্য, নয়টি বিলিয়ান ফ্ল্যাগ ইত্যাদি) ঐচ্ছিক বা ডিফল্ট false।

String get deliveryLabel

remote, onSite, online ফ্ল্যাগের ভিত্তিতে ডেলিভারির ধরন জোড়া টেক্সট (যমেন "Remote · On site") তৈরি করে সাবটাইটলে রে জন্ম; কোনোটিই স্টে না থাকলে "Delivery not set" রটার্ন করে।

factory ServiceListing.fromJson(Map<String, dynamic> json)

JSON থেকে ServiceListing তৈরি করে। একটি ইনার হলেপার b(key) ব্যবহার করে নয়টি বিলিয়ান ফ্ল্যাগ পার্স করে — এই ফ্ল্যাগগুলো ডিটিওতে প্রমিটিভি boolean, তাই কী অনুপস্থিতি থাকলে সার্ভার এমনতিই false ধরে নেয় — একই আচরণ এখানও অনুসরণ করা হয়েছে।

ক্লাস/উইজটে: ServiceListingRequest

POST/PUT /services-এর জন্য রকি়োয়েস্টে DTO। নয়টি বুলিয়ান ফ্ল্যাগ প্রত্যাশিত রকি়োয়েস্টে অবশ্যই পাঠাতে হয় — কারণ এগুলো DTO-তে প্রমিটিভি এবং `CompanyServiceServiceImpl.update` দ্বারা শর্তহীনভাবে অ্যাসাইন করা হয়; বাদ পড়া কী মান ফ্ল্যাগ বন্ধ হয়ে যাওয়া। এডটি ফর্ম বর্ধমান সার্বভাসি থেকে ফ্ল্যাগগুলো সীড করে এবং স্পর্শ করা হোক বা না হোক নয়টিই ফরম পাঠায়।

`ServiceListingRequest({required this.name, required this.featured, ..., this.description, ...})`

কনস্ট্রাক্টর — নাম ও নয়টি ফ্ল্যাগ আবশ্যিক, বাকি স্কালের ফিল্ড ঐচ্ছকি (এগুলো সার্বভার-সাইডে null-guard করা হয়, তাই খালি থাকলে বাদ দেওয়া হয়)।

factory `ServiceListingRequest.from(ServiceListing service)`

একটি বর্ধমান `ServiceListing` থেকে সব ফিল্ড কপি করে সমতুল্য রকি়োয়েস্ট তৈরি করে — এডটি ফর্মে প্রাথমিক অবস্থা সীড করতে ব্যবহৃত হয়।

`ServiceListingRequest.copyWith({String? name, bool? featured, ...})`

নর্দিশ্টি ফিল্ড পরিবর্তন করে নতুন `ServiceListingRequest` ইনস্ট্যান্স রটার্ন করে, বাকি ফিল্ড অপরিবর্তিত রাখে। ইমিউটেবল রকি়োয়েস্টে অবজেক্টে ফর্মে প্রত্যাশিত পরিবর্তন প্রত্যাফলিত করতে ব্যবহৃত হয়।

`Map<String, dynamic> toJson()`

রকি়োয়েস্টে বডি তৈরি করে। নয়টি ফ্ল্যাগ শর্তহীনভাবে অন্তর্ভুক্ত থাকে; `description`, `price`, `priceType`, `estimatedDays`, `categoryId` শুধু নন-নাল/নন-এম্পটি হলে যোগ হয়। একটি ইনার হেল্পার `clean(value)` স্ট্রিং ট্রিম করে খালি হলে null রটার্ন করে।

ক্লাস/উইজটে: ServiceTemplate

একটি পুনর্ব্যবহারযোগ্য সার্বভাসি সংজ্ঞা — ফর্ম ফিল্ড, ডকুমেন্ট এবং ওয়ার্কফ্লো স্টেজে যা দিয়ে একটি সার্বভাসি তৈরি হয়। নস্টেডে তালিকাগুলোর শুধু সংখ্যা রাখা হয় (পূর্ণ তালিকা ওয়বে সম্পাদনা হয়)।

`ServiceTemplate({required this.id, required this.name, required this.active, ...})`

কনস্ট্রাক্টর — মূল ফিল্ড ও তিনটি কাউন্ট (`formFieldCount`, `requiredDocumentCount`, `workflowStageCount`) নিয়ে, ডফিল্ট ০।

factory `ServiceTemplate.fromJson(Map<String, dynamic> json)`

JSON থেকে টেমপ্লেটে তৈরি করে; একটি ইনার হেল্পার `count(key)` নর্দিশ্টি কী-এর লিস্টের দৈর্ঘ্য বের করে তিনটি কাউন্ট ফিল্ড পূরণ করে।

ক্লাস/উইজটে: ServiceTemplateRequest

POST/PUT /v1/service-templates-এর জন্য রকি়োয়েস্টে DTO। `categoryId` আবশ্যিক নয় বলে চহ্নিতি না হলেও `create` এন্ডপয়েন্ট null-চকে ছাড়াই ক্যাটাগরি লুকআপ করে, তাই ফর্মে এটি আবশ্যিক হিসেবে গণ্য করা হয়। ফর্ম ফিল্ড/ডকুমেন্ট/ওয়ার্কফ্লো স্টেজে তালিকা ইচ্ছাকৃতভাবে কখনো পাঠানো হয় না, কারণ `update` সগুলো উপস্থিতি না থাকলে স্পর্শ করে না।

`ServiceTemplateRequest({required this.name, required this.active, this.description, ...})`

কনস্ট্রাক্টর — নাম ও `active` আবশ্যিক, বাকি ঐচ্ছকি।

`Map<String, dynamic> toJson()`

রকি়োয়েস্টে বডি তৈরি করে। `active` শর্তহীনভাবে পাঠানো হয় (প্রমিটিভি বলে); বর্ণনা ট্রিম করে খালি না হলে যোগ হয়; `defaultPrice`, `estimatedDays`, `categoryId` নন-নাল হলে যোগ হয়।

ক্লাস/উইজটে: ServicePackage

একটি সাইকেলে (billing cycle) বক্রি হওয়া সার্ভিসের বান্ডল।

ServicePackage({required this.id, required this.name, required this.active, ...})

কনস্ট্রাক্টর — packagePrice (ম্যানুয়াল স্টে করা মূল্য) এবং effectivePrice (ক্লায়েন্ট প্রকৃতপক্ষে যা দিয়ে, সার্ভার-সাইডে হিসাব করা) সহ প্যাকেজের সব ফিল্ড নিয়ে।

factory ServicePackage.fromJson(Map<String, dynamic> json)

JSON থেকে ServicePackage তৈরি করে, সব ফিল্ডে null-safe ডিফল্ট প্রয়োগ করে।

ক্লাস/উইজটে: PackageSubscription

একটি নির্দিষ্ট ক্লায়েন্টের কোনো প্যাকেজের প্রতি সাবস্ক্রিপশনের অবস্থা।

PackageSubscription({required this.id, required this.status, required this.requestsUsed, required this.remainingRequests, ...})

কনস্ট্রাক্টর — স্ট্যাটাস, ব্যবহৃত/অবশিষ্ট রকোয়েস্ট সংখ্যা আবশ্যিক; বাকি (প্যাকেজ নাম, ক্লায়েন্ট নাম, বিলিং তারিখসমূহ ইত্যাদি) ঐচ্ছিক।

bool get isActive

status == SubscriptionStatus.active কনি যাচাই করে।

bool get isSuspended

status == SubscriptionStatus.suspended কনি যাচাই করে।

bool get isSettled

SubscriptionStatus.settled (expired বা cancelled) স্টে স্ট্যাটাস আছে কনি যাচাই করে — থাকলে সাবস্ক্রিপশন চূড়ান্ত অবস্থায়, তাই রো-তে কোনো অ্যাকশন দেখানো হয় না।

double? get usage

requestsUsed / requestQuota অনুপাত (০ থেকে ১-এর মধ্যে ক্ল্যাম্প করা) রটার্ন করে। requestQuota null বা ০ বা তার কম হলে null রটার্ন করে (কোটা সীমাহীন বোঝাতে) — কারণ খালি বার "কিছু ব্যবহার হয়নি" বোঝাতে, "কোনো সীমা নেই" নয়।

factory PackageSubscription.fromJson(Map<String, dynamic> json)

JSON থেকে PackageSubscription তৈরি করে; status অনুপস্থিতি থাকলে ডিফল্ট হিসেবে

SubscriptionStatus.pendingPayment ব্যবহার করে।

ফাইল: lib/features/catalogue/catalogue_repository.dart

সংক্ষিপ্ত ববিরণ: ক্যাটালগ ফচারের সম্পূর্ণ নটেওয়ার্ক লয়ের — সার্ভিস, টেমপ্লেট, প্যাকেজ এবং সাবস্ক্রিপশনের জন্য API কল, এবং প্রতিটির জন্য Riverpod কন্ট্রোলার/প্রোভাইডার। সার্ভিস ও টেমপ্লেটে মুছে ফেলা মানে সফট-ডিলিটি — ইতিহাস অক্ষুণ্ণ থাকে।

ক্লাস/উইজটে: CatalogueRepository

ApiClient-এর উপর মডক হিসেবে সার্ভিস, টেমপ্লেট, প্যাকেজ ও সাবস্ক্রিপশন সংক্রান্ত সব HTTP অপারেশন প্রকাশ করে।

CatalogueRepository(this._api)

কনস্ট্রাক্টর — ApiClient ইনস্ট্যান্স নিয়ে রপিজিটরি তৈরি করে।

Future<PagedResponse<ServiceListing>> services({int? categoryId, int page = 0, int size = 30})

GET /services কল করে সব সার্ভিস (রটিয়ার্ড সহ) পৃষ্ঠায়তি (paged) আকারে আনে — এটি অ্যাডমিন তালিকা; পাবলিক /services/active তালিকা রকি়োস্ট ফ্লো ব্যবহার করে।

Future<ServiceListing> createService(ServiceListingRequest request)

POST /services কল করে নতুন সার্ভিস তৈরি করে।

Future<ServiceListing> updateService(int id, ServiceListingRequest request)

PUT /services/\$id কল করে সার্ভিস আপডেট করে। নয়টি ফ্ল্যাগ পরিবর্তন হোক বা না হোক সব সময় পাঠানো হয় — দেখুন ServiceListingRequest।

Future<ServiceListing> toggleService(int id)

PATCH /services/\$id/toggle কল করে সার্ভিসের active/inactive টগল করে।

Future<void> deleteService(int id)

_api.deleteText('/services/\$id') কল করে সফট-ডেলিটি করে। deleteText ব্যবহার করা হয় কারণ এই এন্ডপয়েন্ট ResponseEntity<String> রটার্ন করে (JSON নয়), যা সাধারণ delete মেথডে ডিকোড ব্যর্থ হতো।

Future<PagedResponse<ServiceTemplate>> templates({int page = 0, int size = 30})

GET /v1/service-templates কল করে টেমপ্লেটে তালিকা আনে, activeOnly: false স্পষ্টভাবে পাঠিয়ে (কারণ সার্ভার-সাইডে ডিফল্ট true, যা রটিয়ার্ড টেমপ্লেটে লুকিয়ে ফলেত)।

Future<ServiceTemplate> createTemplate(ServiceTemplateRequest request)

POST /v1/service-templates কল করে নতুন টেমপ্লেটে তৈরি করে।

Future<ServiceTemplate> updateTemplate(int id, ServiceTemplateRequest request)

PUT /v1/service-templates/\$id কল করে টেমপ্লেটে আপডেট করে।

Future<void> deleteTemplate(int id)

DELETE /v1/service-templates/\$id কল করে টেমপ্লেটে মুছে ফেলে (সফট-ডেলিটি)।

Future<List<ServicePackage>> activePackages()

GET /packages/active কল করে বর্তমানে বিক্রয়যোগ্য প্যাকেজে তালিকা আনে — এটি আলাদা এন্ডপয়েন্ট, তালিকার ফিল্টার নয়, কারণ বন্ধ করা প্যাকেজের বদ্যমান সাবস্ক্রিপশন থেকে যায়।

Future<List<ServiceTemplate>> templatesInCategory(int categoryId)

GET /v1/service-templates/category/\$categoryId কল করে (পৃষ্ঠা ০, সাইজ ৫০) নির্দিষ্ট ক্যাটাগরির অধীনে থাকা টেমপ্লেটে তালিকা রটার্ন করে।

Future<PagedResponse<ServicePackage>> packages({int page = 0, int size = 30})

GET /packages কল করে সব প্যাকেজে পৃষ্ঠায়তি আকারে আনে।

Future<ServicePackage> updatePackage(int id, CreateServicePackageRequest request)

PUT /packages/\$id কল করে প্যাকেজে আপডেট করে। ক্রিয়েটে DTO পুনরায় ব্যবহার করা হয়, কারণ এন্ডপয়েন্ট সটোই গ্রহণ করে।

Future<ServicePackage> togglePackage(int id)

PATCH /packages/\$id/toggle কল করে প্যাকেজে active/inactive টগল করে।

Future<PagedResponse<PackageSubscription>> subscriptions({String? status, int page = 0, int size = 30})

GET /packages/subscriptions কল করে সাবস্ক্রিপশন তালিকা পৃষ্ঠায়িত আকারে আনে, ঐচ্ছকি স্ট্যাটাস ফিল্টারসহ।

Future<PackageSubscription> subscribe({required int packageId, int? clientId, bool? autoRenew})

POST /packages/subscribe কল করে একটি ক্লায়েন্টকে প্যাকেজে সাবস্ক্রাইব করে। স্টাফ clientId পাঠায়; ক্লায়েন্ট নজি সাবস্ক্রাইব করলে স্টো বাদ দিয়ে এবং ব্যাকএন্ড টোকেন থেকে পড়ে নেয়।

Future<PackageSubscription> activateSubscription(int id)

PATCH /packages/subscriptions/\$id/activate কল করে PENDING_PAYMENT সাবস্ক্রিপশনের পেমেন্ট নশ্চিতি করে। প্রোডাকশনে সাধারণত পেমেন্ট ওয়েবেহুক এটা করে; ম্যানুয়ালি নিওয়া সাবস্ক্রিপশন ক্লয়ার করার অন্য কোনো উপায় না থাকায় অ্যাপ এই অপশন দিয়ে।

Future<PackageSubscription> suspendSubscription(int id, String? reason)

PATCH /packages/subscriptions/\$id/suspend কল করে সাবস্ক্রিপশন সাময়িকি স্থগতি করে। reason কোয়েরি প্যারামিটার হিসেবে যায় (JSON বডিনয়), কারণ কন্ট্রোলার @RequestParam String reason নেয়।

Future<PackageSubscription> reactivateSubscription(int id)

PATCH /packages/subscriptions/\$id/reactivate কল করে স্থগতি সাবস্ক্রিপশন পুনরায় সক্রিয় করে।

Future<PackageSubscription> cancelSubscription(int id, String? reason)

PATCH /packages/subscriptions/\$id/cancel কল করে সাবস্ক্রিপশন স্থায়ীভাবে বাতলি করে। suspend-এর মতোই কোয়েরি-প্যারামিটার আকারে reason পাঠায়।

String _reason(String? reason)

প্রাইভেট হেল্পার — reason ট্রিম করে খালি না হলে ?reason=... কোয়েরি স্ট্রিং (URL-এনকোডেডে) রিটার্ন করে, নাহলে খালি স্ট্রিং রিটার্ন করে।

উপ-লভেলে: catalogueRepositoryProvider

Provider<CatalogueRepository> — apiClientProvider-এর উপর নির্ভর করে CatalogueRepository ইনস্ট্যান্স সরবরাহ করে।

ক্লাস/উইজটে: ServicesController

AsyncNotifier<List<ServiceListing>> — সার্ভিস তালিকার স্টেটে (লোড, রিফ্রেশ, লোকাল আপডেট/মুছে ফেলা) পরিচালনা করে। পৃষ্ঠায়িত লোডারের বদলে সাধারণ তালিকা ব্যবহার করে কারণ একটুকোম্পানির সার্ভিস সংখ্যা কয়কে ডজন মাত্র, প্রথম পৃষ্ঠাতেই সব ধরে।

Future<List<ServiceListing>> build()

currentUserProvider ওয়াচ করে _load() কল করে প্রাথমিক তালিকা লোড করে।

Future<List<ServiceListing>> _load()

catalogueRepositoryProvider.services() কল করে প্রথম পৃষ্ঠার কন্টেন্ট রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard দিয়ে _load() পুনরায় চালিয়ে স্টেটে রফ্রিশে করে।

void apply(ServiceListing updated)

তালিকায় নরিদষ্টি id-র সার্ভিসকে updated দিয়ে প্রতস্থাপন করে।

void remove(int id)

তালিকা থেকে নরিদষ্টি id-র সার্ভিস বাদ দিয়ে।

টপ-লেভেল: serviceProvider

AsyncNotifierProvider<ServicesController, List<ServiceListing>> — ServicesController-এর প্রোভাইডার।

ক্লাস/উইজটে: TemplatesController

AsyncNotifier<List<ServiceTemplate>> — টেমপ্লেটে তালিকার স্টেটে পরিচালনা করে, ServicesController-এর মতোই কার্যমো।

Future<List<ServiceTemplate>> build()

currentUserProvider ওয়াচ করে _load() কল করে।

Future<List<ServiceTemplate>> _load()

catalogueRepositoryProvider.templates() কল করে প্রথম পৃষ্ঠার কন্টেন্ট রিটার্ন করে।

Future<void> refresh()

_load() পুনরায় চালিয়ে স্টেটে রফ্রিশে করে।

void apply(ServiceTemplate updated)

তালিকায় নরিদষ্টি id-র টেমপ্লেটে প্রতস্থাপন করে।

void remove(int id)

তালিকা থেকে নরিদষ্টি id-র টেমপ্লেটে বাদ দিয়ে।

টপ-লেভেল: templatesProvider

AsyncNotifierProvider<TemplatesController, List<ServiceTemplate>> — TemplatesController-এর প্রোভাইডার।

ক্লাস/উইজটে: PackagesController

AsyncNotifier<List<ServicePackage>> — প্যাকেজে তালিকার স্টেটে পরিচালনা করে।

Future<List<ServicePackage>> build()

currentUserProvider ওয়াচ করে _load() কল করে।

Future<List<ServicePackage>> _load()

catalogueRepositoryProvider.packages() কল করে প্রথম পৃষ্ঠার কন্টেন্ট রিটার্ন করে।

Future<void> refresh()

_load() পুনরায় চালিয়ে স্টেটে রফ্রিশে করে।

`void apply(ServicePackage updated)`

তালকায় নরিদষ্টি `id`-র প্যাকেজে প্রতস্থাপন করে।

টপ-লেভেলে: `packagesProvider`

`AsyncNotifierProvider<PackagesController, List<ServicePackage>>` — `PackagesController`-এর প্রোভাইডার।

ক্লাস/উইজটে: `SubscriptionsController`

`AsyncNotifier<List<PackageSubscription>>` — সাবস্ক্রিপশন তালিকার স্টেটে পরিচালনা করে।

`Future<List<PackageSubscription>> build()`

`currentUserProvider` ওয়াচ করে `_load()` কল করে।

`Future<List<PackageSubscription>> _load()`

`catalogueRepositoryProvider.subscriptions()` কল করে প্রথম পৃষ্ঠার কন্টেন্ট রিটার্ন করে।

`Future<void> refresh()`

`_load()` পুনরায় চালিয়ে স্টেটে রিফ্রেশ করে।

`void apply(PackageSubscription updated)`

তালকায় নরিদষ্টি `id`-র সাবস্ক্রিপশন প্রতস্থাপন করে।

টপ-লেভেলে: `subscriptionsProvider`

`AsyncNotifierProvider<SubscriptionsController, List<PackageSubscription>>` — `SubscriptionsController`-এর প্রোভাইডার।

টপ-লেভেলে: `activePackagesProvider`

`FutureProvider.autoDispose<List<ServicePackage>>` — একজন ক্লায়েন্ট যে প্যাকেজগুলোতে প্রকৃতপক্ষে সাইন-আপ করতে পারে সেগুলোর তালিকা সরবরাহ করে।

টপ-লেভেলে: `templatesInCategoryProvider`

`FutureProvider.autoDispose.family<List<ServiceTemplate>, int>` — নরিদষ্টি ক্যাটাগরি আইডি অনুযায়ী টেমপ্লেটে তালিকা সরবরাহ করে।

ফাইল: `lib/features/catalogue/catalogue_form_sheets.dart`

সংক্ষিপ্ত ববিরণ: সার্ভিস, টেমপ্লেটে ও প্যাকেজে তৈরি/সম্পাদনার জন্য বটম-শীট ফর্ম UI বাস্তবায়ন করে, সাথে দুটি পুনর্ব্যবহারযোগ্য ফর্ম উইজটে (ক্যাটাগরি ড্রপডাউন, ফ্ল্যাগ সুইচ)। প্যাকেজে তৈরি এখান থেকে নেই (তা রকিওয়েস্ট স্ক্রিনিং হয়), শুধু সম্পাদনা আছে।

ক্লাস/উইজটে: _CategoryField

`ConsumerWidget` — একটি ক্যাটাগরি বিচ্ছেদে নেওয়ার ড্রপডাউন ফিল্ড।

`serviceCategoriesAdminProvider` থেকে অ্যাডমিনি ক্যাটাগরি তালিকা পড়ে। সার্ভিসের ক্ষেত্রে ক্যাটাগরি ঐচ্ছকি, টেমপ্লেটে কক্ষে আবশ্যিক — `required` প্যারামিটার দিয়ে এই পার্থক্য কভার করা হয়। "No category" অপশন শুধু তখনই দেখানো হয় যখন কিছুই স্টেটে নেই, কারণ কোনো আপডেট এন্ডপয়েন্টই একবার স্টেটে হওয়া ক্যাটাগরি ক্লিয়ার করতে পারে না।

```
_CategoryField({required this.value, required this.onChanged, this.required = false, this.allowNone = true})
```

কনস্ট্রাক্টর — বর্তমান মান, পরিবর্তন কলব্যাক, এবং আবশ্যিক/none-অনুমতি ফ্ল্যাগ নেয়।

`Widget build(BuildContext context, WidgetRef ref)`

`serviceCategoriesAdminProvider` ওয়াচ করে ক্যাটাগরি তালিকা পায়। তালিকা খালি হলে — `required` হলে ও লোডিং শেষ হলে এরর বার্তা দেখায় (একটি টেমপ্লেটে বাঁচানো যাবে না), নাহলে কিছু দেখায় না। রিটার্নড ক্যাটাগরি লুকানো হয় যদি না স্টোই বর্তমান নির্বাচনী মান হয়। `DropDownButtonFormField<int?>` রিটার্ন করে, `required` হলে ভ্যালিডিটর যুক্ত থাকে।

ক্লাস/উইজটে: _FlagSwitch

`StatelessWidget` — একটি স্বাধীন বুলিয়ান স্টেটিং টগল করার সুইচ (চেকেবক্সের বদলে সুইচ, কারণ প্রতিটি ফ্ল্যাগ স্বাধীন স্টেটিং, স্টেটে অংশ নেয়)।

```
_FlagSwitch({required this.label, required this.hint, required this.value, required this.onChanged})
```

কনস্ট্রাক্টর — লবেলে, হিন্ট টেক্সট, বর্তমান মান, পরিবর্তন কলব্যাক নেয়।

`Widget build(BuildContext context)`

লবেলে ও হিন্টসহ `SwitchListTile` রিটার্ন করে (dense, zero padding)।

উপ-লভেলে ফাংশন: showServiceSheet

```
`dart Future<void> showServiceSheet(BuildContext context, {ServiceListing? existing})` মডাল বটম শাটে _ServiceSheet থোল্ডে — existing দিলে এডটি মোড, না দিলে ক্রয়িটে মোড।
```

ক্লাস/উইজটে: _ServiceSheet

`ConsumerStatefulWidget` — সার্ভিস তৈরি/সম্পাদনার ফর্ম শাট।

```
_ServiceSheet({this.existing})
```

কনস্ট্রাক্টর — ঐচ্ছকি বর্ধমান সার্ভিস (এডটিরে জন্ম) নেয়।

```
ConsumerState<_ServiceSheet> createState()
```

`_ServiceSheetState` তৈরি করে।

ক্লাস/উইজটে: _ServiceSheetState

`ConsumerState<_ServiceSheet>` — সার্ভিস ফর্মের সব টেক্সট কন্ট্রোলার, নয়টি ফ্ল্যাগ (একটি `ServiceListingRequest` অবজেক্টে একসাথে ধরে রাখা, যাতে দশম ফ্ল্যাগ যোগ করলে পাঠাতে ভুলে যাওয়া অসম্ভব হয়), ভ্যালিডেশন ও সাবমিট লজিক রাখা।

`bool get _isEdit`

`widget.existing != null` কনি যাচাই করে।


```
void dispose()
```

`_name, _description, _price, _days` কন্ট্রোলার ডিসপোজ করে।

```
Future<void> _submit()
```

ফর্ম ভ্যালিডেট করে, `_flags.copyWith(...)` দিয়ে নাম/বর্ণনা/মূল্য/টাইপ/দনি/ক্যাটাগরি বসিয়ে চূড়ান্ত `ServiceListingRequest` তৈরি করে। এডটি মোডে `repo.updateService` কল করে `servicesProvider.notifier.apply` দিয়ে আপডেট করে; নতুন তৈরি ক্ষেত্রে `repo.createService` কল করে পুরো তালিকা `refresh` করে। সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়; `ApiException` এলে সার্ভার বার্তা দেখায়।

```
Widget build(BuildContext context)
```

নাম, বর্ণনা, `_CategoryField`, মূল্য+মূল্যের ধরন (পাশাপাশি), আনুমানিক দনি — এই ফিল্ডগুলোর পর "How it works" শিরোনামের নিচে নয়টি `_FlagSwitch` (featured, remote, onSite, online, autoApproval, requiresQuotation, requiresDocuments, supportsCustomWorkflow, aiAssisted) দেখায়। প্রতিটি সুইচ পরিবর্তনে `_flags = _flags.copyWith(...)` দিয়ে স্টেটে আপডেট হয়। মন্তব্যে স্পষ্ট করা আছে যে নয়টিই সব সময় সতে করার সময় পাঠানো হয়।

টপ-লেভেলে ফাংশন: `showTemplateSheet`

```
`dart Future<void> showTemplateSheet(BuildContext context, {ServiceTemplate? existing})` মডাল বটম শিটে _TemplateSheet খোলে — existing দিলে এডটি মোড।
```

ক্লাস/উইজটে: `_TemplateSheet`

`ConsumerStatefulWidget` — টেমপ্লেটে তৈরি/সম্পাদনার ফর্ম শিট।

```
_TemplateSheet({this.existing})
```

কনস্ট্রাক্টর — ঐচ্ছকি বদ্যমান টেমপ্লেটে নেই।

```
ConsumerState<_TemplateSheet> createState()
```

`_TemplateSheetState` তৈরি করে।

ক্লাস/উইজটে: `_TemplateSheetState`

`ConsumerState<_TemplateSheet>` — টেমপ্লেটে ফর্মের কন্ট্রোলার, ভ্যালিডেশন ও সাবমিট লজিক রাখা।

```
bool get _isEdit
```

`widget.existing != null` কনি যাচাই করে।

```
void dispose()
```

`_name, _description, _price, _days` কন্ট্রোলার ডিসপোজ করে।

```
Future<void> _submit()
```

ফর্ম ভ্যালিডেট করে `ServiceTemplateRequest` তৈরি করে। এডটি মোডে `repo.updateTemplate` কল করে `templatesProvider.notifier.apply` দিয়ে আপডেট করে; নতুন তৈরি ক্ষেত্রে `repo.createTemplate` কল করে তালিকা `refresh` করে। এরর হ্যান্ডলিং `_ServiceSheet`-এর মতোই।

Widget build(BuildContext context)

নাম, বর্ণনা, `_CategoryField` (এখানে `required: true`, `allowNone: false`), ডিফল্ট মূল্য+দনি (পাশাপাশি), এবং "Available" `_FlagSwitch` (active টগল) দেখায়। এডটি মোডে যদি টেমপ্লেটের ফর্ম ফল্ড/ডকুমেন্ট/ওয়ার্কফল্ড স্টেজে সংখ্যা শূন্যের বেশি হয়, তাহলে একটি তথ্যমূলক নোট দেখায় যে সেগুলো অপরবর্তী থাকবে এবং ওয়বে সম্পাদনা করতে হবে।

টপ-লেভেলে ফাংশন: showPackageEditSheet

`dart Future<void> showPackageEditSheet(BuildContext context, {required ServicePackage existing})`
মডাল বটম শিটে _PackageEditSheet খোলে — শুধু সম্পাদনার জন্য (তৈরিকিওয়েস্ট স্ক্রিনিং হয়, একই এন্ডপয়েন্ট ও DTO ব্যবহার করে)।`

ক্লাস/উইজেট: _PackageEditSheet

`ConsumerStatefulWidget` — বর্তমান প্যাকেজে সম্পাদনার ফর্ম শিট।

`_PackageEditSheet({required this.existing})`

কনস্ট্রাক্টর — সম্পাদনাযোগ্য প্যাকেজে নিয়ে (আবশ্যিক, কারণ এই শিট শুধু এডটিরে জন্য)।

`ConsumerState<_PackageEditSheet> createState()`

`_PackageEditSheetState` তৈরি করে।

ক্লাস/উইজেট: _PackageEditSheetState

`ConsumerState<_PackageEditSheet>` — প্যাকেজে ফর্মের কন্ট্রোলার (নাম, বর্ণনা, মূল্য, ছাড়, কোট, দনি), বলিং সাইকেল ও ফ্ল্যাগ স্টেটে, ভ্যালিডেশন ও সাবমিট লজিক রাখা।

`void dispose()`

`_name, _description, _price, _discount, _quota, _days` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে `CreateServicePackageRequest` তৈরি করে (বান্ডলড সার্ভিস তালিকা ইচ্ছাকৃতভাবে বাদ দেওয়া হয়, কারণ খালি তালিকা পাঠালে তা "অপরবর্তী রাখা" হিসেবে গণ্য হয় এবং অ্যাপে সার্ভিস বাছাইয়ের কোনো উপায়ও নেই)। `repo.updatePackage` কল করে `packagesProvider.notifier.apply` দিয়ে স্টেটে আপডেট করে। সফল/ব্যর্থ হ্যান্ডলিং অন্য শিটগুলোর মতোই।

Widget build(BuildContext context)

নাম, বর্ণনা, বলিং সাইকেল ড্রপডাউন, নব্বিশটি মূল্য (খালি রাখলে বান্ডলড সার্ভিস থেকে হিসাব হয় বলে হেল্পার টেক্সট), ছাড়ের শতাংশ+অনুরোধ কোট (পাশাপাশি), ডেলিভারি দনি, এবং "Featured"/"Popular" দুটি `_FlagSwitch` দেখায়। প্যাকেজের নিজস্ব মূল্য না থাকলে (`packagePrice == null`) কনিন্তু `effectivePrice` থাকলে, বর্তমান হিসাবকৃত মূল্য জানানো একটি তথ্যমূলক নোট দেখায়।

ফাইল: lib/features/catalogue/catalogue_screen.dart

সংক্ষিপ্ত বিবরণ: কোম্পানী বক্রিকরে তার একটি চার-ট্যাবের স্ক্রিনি — সার্ভিস, টেমপ্লেট, প্যাকেজ এবং সাবস্ক্রিপশন। প্রতিটি ট্যাব তার নিজস্ব পারমশিনের ওপর গটেডে, এবং প্রতিটি ট্যাব নিজস্ব লস্ট, তৈরি/সম্পাদনা/মুছে ফলো কার্যক্রম পরিচালনা করে।

ক্লাস/উইজটে: CatalogueScreen

`ConsumerWidget`, ক্যাটালগ স্ক্রিনিংর মূল প্রবশেদ্বার। ব্যবহারকারীর পারমিশন অনুযায়ী কোন কোন ট্যাব (Services, Templates, Packages, Subscriptions) দেখানো হবে তা ঠিকি করে, ট্যাববহীন অবস্থায় লক-স্টেটে দেখায়, এবং সক্রিয় ট্যাবের জন্য একটি ফ্লোটিং অ্যাকশন বাটন (নতুন তৈরিকার) প্রদর্শন করে যদা সেই ট্যাবের `create` কলব্যাক থাকে।

`CatalogueScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর। ঐচ্ছকি `initialTabLabel` প্যারামিটার দিয়ে নির্দিষ্ট করা যায় কোন ট্যাব দিয়ে স্ক্রিনিং শুরু হবে।

`Widget build(BuildContext context, WidgetRef ref)`

পারমিশন কন্ট্রোলার লোড না হওয়া পর্যন্ত একটি লোডার দেখায়। এরপর `CataloguePermissions` অনুযায়ী চারটি সম্ভাব্য ট্যাব (Services, Templates, Packages, Subscriptions) থেকে যেকোনো ব্যবহারকারীর কাছে দৃশ্যমান তার একটি লিস্ট তৈরিকরে — কোনো পারমিশন না থাকলে "Not available to you" এম্পটি-স্টেটে দেখায়। `initialTabLabel` মালিগে প্রাথমিক ট্যাব ইনডেক্স নির্ধারণ করে এবং `DefaultTabController + AnimatedBuilder` দিয়ে একটি `Scaffold` তৈরিকরে যখনো সক্রিয় ট্যাবের `create` কলব্যাক থাকলে একটি ফ্লোটিং "New" বাটন দেখায়।

ক্লাস/উইজটে: _ServicesTab

`ConsumerStatefulWidget`। সারভিস তালিকা ট্যাবের প্রবশেবিন্দু, প্রকৃত স্টেটে `_ServicesTabState`-এ থাকে।

`ConsumerState<_ServicesTab> createState()`

`_ServicesTabState`-এর একটি নতুন ইনস্ট্যান্স তৈরিকরে।

ক্লাস/উইজটে: _ServicesTabState

সারভিস তালিকার সক্রিয়/নিষ্ক্রিয় টগল ও মুছে ফেলার কার্যক্রম পরিচালনা করে এবং `busyId`-এর মাধ্যমে কোন সারভিস ব্যস্ত তা ট্র্যাক করে।

`Future<void> _toggle(ServiceListing row)`

প্রদত্ত সারভিসটির সক্রিয়/নিষ্ক্রিয় অবস্থা টগল করতে `catalogueRepositoryProvider.toggleService` কল করে, সফল হলে `servicesProvider`-এ আপডেটে প্রতফিলতি করে; ব্যর্থ হলে সন্ধ্যাকবারে ত্রুটির বার্তা দেখায়।

`Future<void> _delete(ServiceListing row)`

প্রথম একটি নিশ্চিতিকরণ ডায়ালগ দেখায়, নিশ্চিত হলে `deleteService` কল করে এবং সফল হলে `servicesProvider` থেকে সারভিস সরিয়ে দেয়।

`Widget build(BuildContext context)`

পারমিশন অনুযায়ী `canEdit/canDelete` নির্ধারণ করে এবং `ConfigList<ServiceListing>` তৈরিকরে যা প্রতিটি সারভিসকে `ConfigRow` হিসেবে রেন্ডার করে — নাম, ক্যাটাগরি, ডিলেভারিবেলে, দাম এবং এডিট/টগল/মুছে ফেলার অ্যাকশনসহ।

ক্লাস/উইজটে: _TemplatesTab

`ConsumerStatefulWidget`। টেমপ্লেটে তালিকা ট্যাবের প্রবশেবিন্দু।

`ConsumerState<_TemplatesTab> createState()`

`_TemplatesTabState`-এর একটি নতুন ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: `_TemplatesTabState`

টেমপ্লেটে তালিকা, ক্যাটাগরি অনুযায়ী ফিল্টারিং, এবং মুছে ফেলার কার্যক্রম পরিচালনা করে।

`Future<void> _delete(ServiceTemplate row)`

নিশ্চিতকরণ ডায়ালগ দেখায় এবং নিশ্চিতি হলে `deleteTemplate` কল করে, সফল হলে `templatesProvider` থেকে টেমপ্লেটটি সরিয়ে দেয়।

`Widget build(BuildContext context)`

`templateCategoryProvider`-এর ভিত্তিতে টেমপ্লেটে লিস্টকে ফিল্টার করে (নির্দিষ্ট ক্যাটাগরিতে হলে `templatesInCategoryProvider` ব্যবহার করে), টেমপ্লেটগুলো থেকে ডাইনামিকভাবে ক্যাটাগরি তালিকা বের করে একটি `FilterBar` হডার দেখায়, এবং প্রতিটি টেমপ্লেটকে ফর্ম-ফিল্ড/স্টেজে কাউন্ট ও দামসহ `ConfigRow`-এ রেন্ডার করে।

ক্লাস/উইজটে: `_PackagesTab`

`ConsumerStatefulWidget`। প্যাকেজে তালিকা ট্যাবের প্রবেশবিন্দু।

`ConsumerState<_PackagesTab> createState()`

`_PackagesTabState`-এর একটি নতুন ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: `_PackagesTabState`

প্যাকেজে তালিকা, "শুধু বক্রিয়যোগ্য" ফিল্টার, টগল, এবং একজন ক্লায়েন্টকে প্যাকেজে সাবস্ক্রাইব করার কার্যক্রম পরিচালনা করে।

`Future<void> _toggle(ServicePackage row)`

প্যাকেজের সক্রিয়/নিষ্ক্রিয় অবস্থা `togglePackage` দিয়ে পরিবর্তন করে এবং `packagesProvider`-এ আপডেট প্রত্যাফলিত করে।

`Future<void> _subscribe(ServicePackage row)`

একটি বিটম-শিট থেকে ক্লায়েন্ট বাছাই করতে `_pickClient` কল করে, তারপর `catalogueRepositoryProvider.subscribe` দিয়ে সেই ক্লায়েন্টকে প্যাকেজে সাবস্ক্রাইব করে এবং `subscriptionsProvider` রিফ্রেশ করে। নতুন সাবস্ক্রিপশন `PENDING_PAYMENT` স্টেটে শুরু হয় বলে সাবস্ক্রিপশন ট্যাবটিও একইসাথে রিফ্রেশ করা হয়।

`Widget build(BuildContext context)`

রোল-ভিত্তিক `canSubscribe` চেক করে (এই ট্যাবে পৌঁছানো মানই ইতিমধ্যে অনুমতিপ্রাপ্ত), `packageScopeProvider` অনুযায়ী "সব প্যাকেজে" বা "শুধু বক্রিয়যোগ্য" এর মধ্যে টগল করে `activePackagesProvider/packagesProvider` থেকে ডেটা লোড করে, এবং প্রতিটি প্যাকেজকে বলিং সাইকেলে, অনুরোধ কতোটা, ছাড়, দামসহ `ConfigRow`-এ রেন্ডার করে — সাথে "Subscribe a client" অ্যাকশন।

ক্লাস/উইজটে: `_SubscriptionsTab`

`ConsumerStatefulWidget`। সাবস্ক্রিপশন তালিকা ট্যাবের প্রবেশবিন্দু।

`ConsumerState<_SubscriptionsTab> createState()`

`_SubscriptionsTabState`-এর একটি নতুন ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _SubscriptionsTabState

সাবস্ক্রিপশনের চারটি সম্ভাব্য স্টেটে-পরিবর্তন (activate, suspend, reactivate, cancel) পরিচালনা করে।

`Future<void> _act(PackageSubscription row, Future<PackageSubscription> Function() action, String failure)`

একটি সাধারণ হেল্পার যা প্রদত্ত `action` চালায়, সফল হলে ফলাফল `subscriptionsProvider`-এ প্রত্যাফলিত করে, ব্যর্থ হলে `failure` বার্তা বা API-এর নজিস্ব ত্রুটি বার্তা দেখায়।

`Future<void> _suspend(PackageSubscription row)`

একটি কারণ জিজ্ঞাসা করার জন্য `askForText` ডায়ালগ দেখায়, তারপর `_act` দিয়ে `suspendSubscription` কল করে।

`Future<void> _cancel(PackageSubscription row)`

একটি ধ্বংসাত্মক (destructive) নিশ্চিতিকরণ সহ কারণ জিজ্ঞাসা করে, তারপর `_act` দিয়ে `cancelSubscription` কল করে — এটি চূড়ান্ত এবং পূর্বাবস্থায় ফেরানো যায় না।

`Widget build(BuildContext context)`

`subscriptionsProvider` থেকে ডটো লোড করে `ConfigList<PackageSubscription>` তৈরি করে, প্রতিটি সারি `_SubscriptionRow` হিসাবে রেন্ডার করে এবং `activate/suspend/reactivate/cancel` কলব্যাক পাস করে।

ক্লাস/উইজটে: _SubscriptionRow

`StatelessWidget`। একটি সাবস্ক্রিপশনের জন্য কাস্টম রো — নাম, প্যাকেজ, স্ট্যাটাস চিপি, ব্যবহারের প্রগ্রসে বার এবং প্রক্বেপট-নির্ভর অ্যাকশন মেনু দেখায়, কারণ এখানে স্ট্যাটাস ও অ্যালাউয়েন্সই মূল বিষয়, শুধু নাম নয়।

`Widget build(BuildContext context)`

সাবস্ক্রিপশনের স্ট্যাটাস অনুযায়ী উপলব্ধ অ্যাকশনগুলোর তালিকা তৈরি করে (`pendingPayment` Confirm payment, active Pause, suspended Resume, settled না হলে End it), ক্লায়েন্ট/প্যাকেজে নাম ও `StatusChip` দেখায়, ব্যবহারের অনুপাত থাকলে একটি `LinearProgressIndicator` দেখায় (কোটা শেষে হলে বর্গাকৃতির, এবং ব্যবহারের সংখ্যা ও পরবর্তী বলিৎ/শেষ তারিখ প্রদর্শন করে)।

উপ-লভেলে ফাংশন: Future<Client?> _pickClient(BuildContext context, WidgetRef ref)

একটি মোডাল বটম-শিট খোলে যেখানে সক্রিয় ক্লায়েন্টদের (`activeClientsProvider`) একটি তালিকা দেখানো হয় (বন্ধ হয়ে যাওয়া ক্লায়েন্ট বাদ দিয়ে), ব্যবহারকারী একজনকে ট্যাপ করলে সেই `Client`টি রিটার্ন করে। খালি বা লোড-ব্যর্থ অবস্থার জন্যও এম্পটি/এর স্টেটে দেখায়।

ক্লাস: TemplateCategoryController

`Notifier<int?>`। টেমপ্লেটে ট্যাবটি কোন ক্যাটাগরিতে সংকীর্ণ করা হয়েছে তা রাখে; `null` মানে সব ক্যাটাগরি।

`int? build()`

প্রাথমিক স্টেটে `null` (কোনো ফিল্টার নেই) রিটার্ন করে।

`void set(int? categoryId)`

সক্রিয় ক্যাটাগরি ফিল্টার আপডেট করে।

প্রোভাইডার: templateCategoryProvider

NotifierProvider<TemplateCategoryController, int?> — টেমপ্লেটে ট্যাবের সক্রিয় ক্যাটাগরি ফিল্টার এক্সপোজ করে।

ক্লাস: PackageScopeController

Notifier<bool>। প্যাকেজে ট্যাবে শুধু বক্রিয়যোগ্য প্যাকেজে দেখানো হচ্ছে কনি তা রাখবে।

bool build()

প্রাথমিক স্টেটে false (সব প্যাকেজে) রিটার্ন করে।

void set(bool onSaleOnly)

"শুধু বক্রিয়যোগ্য" ফ্ল্যাগ আপডেট করে।

প্রোভাইডার: packageScopeProvider

NotifierProvider<PackageScopeController, bool> — প্যাকেজে স্কোপ ফিল্টার এক্সপোজ করে।

ফাইল: lib/features/company/company_models.dart

সংক্ষিপ্ত বিবরণ: কোম্পানির নিজের রেকর্ড (প্রোফাইল, ব্যাংকিং, পোর্টাল কনফিগারেশন) এবং সংশ্লিষ্ট রকিওয়েস্ট/রসেপন্স মডেলগুলোর সংজ্ঞা দেয়। প্ল্যাটফর্ম মডুলের Company-এর থেকে ভিন্ন — এটি টেন্যান্ট নজি নজিরে রেকর্ডকে যত্নে দেখে।

ক্লাস: CompanyPermissions

abstract final class, কোম্পানির প্রোফাইল সম্পাদনার জন্য পারমিশন কোডগুলোর ধরুর সংজ্ঞা রাখে — ব্যাকএন্ড তিনটি যেকোনো একটি গ্রহণযোগ্য বলে সবগুলোই এখানে নামটি এবং UI একই স্টে অনুমোদন করে। কোনো মথড নেই, শুধু স্ট্যাটিক কনস্ট্যান্ট (update, settings, branding, editAny)।

ক্লাস/উইজটে: CompanyProfile

কোম্পানির নিজের প্রোফাইল রেকর্ড উপস্থাপন করে — যোগাযোগের তথ্য, ব্যাংকিং, অর্থবহর, ব্র্যান্ডিং এবং প্ল্যাটফর্ম-নির্ধারিত সাবস্ক্রিপশন তথ্য (শুধু প্রদর্শনের জন্য, এখান থেকে সম্পাদনযোগ্য নয়)।

CompanyProfile({required this.id, required this.companyName, ...})

কনস্ট্রাক্টর যা সকল ফিল্ড (আইডি, নাম, সাবডোমেইন, যোগাযোগ, ব্র্যান্ডিং, ব্যাংকিং, সাবস্ক্রিপশন তথ্য ইত্যাদি) স্টে করে।

factory CompanyProfile.fromJson(Map<String, dynamic> json)

JSON রসেপন্স থেকে একটি CompanyProfile তৈরি করে; প্রতিটি ফিল্ড নাল-নিরাপদে প্যার্স করে এবং অনুপস্থিতি id/companyName-এর জন্য যথাক্রমে 0 ও খালি স্ট্রিং ডিফল্ট ব্যবহার করে।

ক্লাস/উইজটে: CompanyProfileRequest

কোম্পানির নিজস্ব রেকর্ড পরিবর্তনের রকিওয়েস্টে বহন করে — একটি প্রকৃত প্যাচ, কারণ ব্যাকএন্ড প্রতিটি ফিল্ড নাল-চকে করে, তাই শুধু পরিবর্তিত ফিল্ডই পাঠানো প্রয়োজন। সাবডোমেইন ও নবিন্ধতি ইমেইল এই DTO-তেই নই বলে এখান থেকে পরিবর্তনযোগ্য নয়।

`CompanyProfileRequest({this.companyName, this.companyPhone, ...})`

কনস্ট্রাক্টর, সকল ফিল্ড ঐচ্ছক — শুধু যা পরিবর্তন করতে হবে তাই সরবরাহ করা হয়।

`Map<String, dynamic> toJson()`

শুধু নন-নাল ফিল্ডগুলোকে JSON ম্যাপে রূপান্তর করে, যাতে যা পরিবর্তিত হয়নি তা রকিওয়েস্টে অন্তর্ভুক্ত না হয়।

ক্লাস/উইজটে: SubscriptionUpgradeRequest

প্ল্যাটফর্ম সাবস্ক্রিপশন প্ল্যান আপগ্রেডের জন্য SSLCommerz চেকআউট শুরু করার রকিওয়েস্টে মডলে করে। এতে থাকা `amount` শুধু ভ্যালিডেশন শেপের জন্য পাঠানো হয় — ব্যাকএন্ড কখনো ক্লায়েন্ট-প্রদত্ত দামকে বিশ্বাস করে না, বরং প্ল্যানের নিজস্ব ক্যাটালগ রেকর্ড থেকে দাম পুনরায় নির্ধারণ করে।

`SubscriptionUpgradeRequest({required this.planId, required this.amount})`

কনস্ট্রাক্টর, প্ল্যান আইডি ও পরিমাণ স্টে করে।

`Map<String, dynamic> toJson()`

`purpose: 'PLATFORM_SUBSCRIPTION', targetId`, এবং `amount` সহ একটি JSON ম্যাপ তৈরি করে।

ক্লাস/উইজটে: PublicCompany

অপ্রমাণীকৃত (unauthenticated) দর্শকরা একটি কোম্পানির পাবলিক পোর্টাল পেজে যা দেখে তার সংক্ষিপ্ত মডলে — শুধু কোম্পানি নিজে যা প্রকাশ করতে বছে নথিছে তাই থাকে।

`PublicCompany({required this.companyName, this.subdomain, ...})`

কনস্ট্রাক্টর, পাবলিক প্রোফাইলের ফিল্ডগুলো স্টে করে।

`factory PublicCompany.fromJson(Map<String, dynamic> json)`

JSON থেকে একটি `PublicCompany` তৈরি করে; `logo/logoUrl` এবং `portalAbout/about` — উভয় বকলিপ কী-নাম থেকে ফলব্যাক করে ভ্যালু নেয়।

ফাইল: lib/features/company/company_repository.dart

সংক্ষিপ্ত বিবরণ: কোম্পানির নিজস্ব প্রোফাইল রডি/আপডেট করার এবং দুটি পাবলিক (অপ্রমাণীকৃত) এন্ডপয়েন্ট কল করার জন্য রপিজিটরি। এছাড়া সাবস্ক্রিপশন আপগ্রেডে চেকআউট শুরু করার মথড সরবরাহ করে।

ক্লাস/উইজটে: CompanyRepository

`/companies` বেসে পাথরে ওপর নর্মিতি API কল-এর একটি সংগ্রহ — যা সাইন-ইন করা ব্যবহারকারীর নিজস্ব কোম্পানির জন্য কাজ করে, এবং দুটি পাবলিক মথড যো কারো জন্য কাজ করে।

`CompanyRepository(this._api)`

কনস্ট্রাক্টর, একটি `ApiClient` ইনজেক্ট করে।

`Future<CompanyProfile> me()`

`GET /companies/me` কল করে সাইন-ইন করা ব্যবহারকারীর নিজস্ব কোম্পানির প্রোফাইল ফেরত দেয়। কোনো কোম্পানি ছাড়া অ্যাকাউন্টে (যেমন প্ল্যাটফর্ম স্টাফ) জন্য এটাসিরাসরি প্রত্যাখ্যাত হয়।

`Future<CompanyProfile> updateMe(CompanyProfileRequest request)`

`PATCH /companies/me` কল করে কোম্পানির রেকর্ড পরিবর্তন করে; প্রতটি ফিল্ড সারভার-সাইডে নাল-চকে করা হয়, তাই শুধু পরিবর্তিত ফিল্ডই পাঠানো প্রয়োজন।

`Future<PublicCompany> publicProfile(String subdomain)`

`GET /companies/public/{subdomain}` কল করে একটি কোম্পানির পাবলিক পেজের তথ্য ফেরত দেয় — অপ্রমাণীকৃত, তাই শুধু কোম্পানির প্রকাশিত তথ্যই বহন করে।

`Future<List<CatalogService>> publicServices(String subdomain)`

`GET /companies/public/{subdomain}/services` কল করে সেই কোম্পানির পোর্টালে প্রকাশিত সার্ভিসগুলোর তালিকা ফেরত দেয় — এটিও পাবলিক, তাই শুধু পোর্টালে জন্য চহ্নিত সার্ভিসগুলোই তালিকাভুক্ত করে।

`Future<String> initiateSubscriptionUpgrade(SubscriptionUpgradeRequest request)`

`POST /payments/sslcommerz/initiate` কল করে কোম্পানির প্ল্যান আপগ্রেডে করার জন্য অনলাইন চেকআউট শুরু করে এবং রসেপন্স থেকে `gatewayUrl` ফেরত দেয়। ব্যাকএন্ড কলার কোম্পানির মালিকি কনি এবং টারগটে প্ল্যানের দাম বর্তমান প্ল্যানের চয়ে বশে কনি তা পুনরায যাচাই করে।

প্রোভাইডার: `companyRepositoryProvider`

`Provider<CompanyRepository>` — `apiClientProvider` থেকে একটি `CompanyRepository` ইনস্ট্যান্স তৈরিকরে দেয়।

প্রোভাইডার: `myCompanyProvider`

`FutureProvider<CompanyProfile>` — `currentUserProvider`-এর ওপর নির্ভরশীল, সাইন-ইন করা ব্যবহারকারীর নিজস্ব কোম্পানি প্রোফাইল লোড করে।

প্রোভাইডার: `publicCompanyProvider`

`FutureProvider.autoDispose.family<PublicCompany, String>` — সাবডোমেইন কী দিয়ে একটি কোম্পানির পাবলিক পেজ লোড করে।

প্রোভাইডার: `publicServicesProvider`

`FutureProvider.autoDispose.family<List<CatalogService>, String>` — সাবডোমেইন কী দিয়ে একটি কোম্পানি পাবলিকভাবে কী বজ্জ্ঞাপন দেয় তা লোড করে।

ফাইল: lib/features/company/company_screen.dart

সংক্ষিপ্ত ববিরণ: কোম্পানি নিজেকে যভাবে দেখে তার স্ক্রিনি — যোগাযোগের তথ্য, ইনভয়েসে যা থাকে, ক্ল্যায়েন্টরা যা দেখে এবং বর্তমান প্ল্যান বিভাগে বিভক্ত। প্রতিটি বিভাগ সম্পাদনাযোগ্য (পারমশিন থাকলে) একটি বিটম-শিটে মাধ্যমে।

ক্লাস/উইজেট: CompanyScreen

ConsumerWidget। প্রধান কোম্পানি প্রোফাইল স্ক্রিনি — লোগো/নাম হডার এবং চারটি তথ্য বিভাগ (যোগাযোগ, ইনভয়েসে, পোর্টাল, প্ল্যান) দেখায়, ট্রায়াল শেষ হয়ে গেলে একটি সিতরুতা ব্যানার দেখায়, এবং সাবডোমইন থাকলে পাবলিক পেজে দেখার বাটন সরবরাহ করে।

Widget build(BuildContext context, WidgetRef ref)

myCompanyProvider থেকে ডটো লোড করে, লোড/এরর/ডটো অবস্থা হ্যান্ডলে করে, এবং **canEdit** পারমশিন অনুযায়ী প্রতিটি বিভাগে এডটি বাটন যোগ করে। পুল-টু-রিফ্রেশে সাপোর্ট করে।

ক্লাস/উইজেট: _Header

ConsumerStatefulWidget। কোম্পানির লোগো, নাম, ট্যাগলাইন ও লোকেশন প্রদর্শন করে এবং লোগো পরিবর্তনের জন্য একটি ট্যাপযোগ্য অ্যাক্টার দিয়ে।

ConsumerState<_Header> createState()

_HeaderState-এর একটি নতুন ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজেট: _HeaderState

লোগো পরিবর্তনের আপলোড ও প্যাচ কার্যক্রম পরিচালনা করে।

Future<void> _changeLogo()

pickAttachment দিয়ে একটি ফাইল বাছাই করতে দেয়, **uploadAvatar** দিয়ে স্টে আপলোড করে (URL ফরমের পায়ে), তারপর **CompanyProfileRequest(logo: ...)** দিয়ে **updateMe** কল করে কোম্পানির রেকর্ডে লোগো URL স্টে করে এবং **myCompanyProvider** ইনভ্যালিডেট করে।

Widget build(BuildContext context)

লোগো (বা প্রথম অক্ষরের ইনিশিয়াল), কোম্পানির নাম, ট্যাগলাইন ও লোকেশন প্রদর্শন করে একটি **AppCard** তৈরি করে; **canEdit** থাকলে ও ব্যস্ত না থাকলে লোগোতে ট্যাপ করে **_changeLogo** কল করা যায়।

ক্লাস/উইজেট: _Section

StatelessWidget। একটি লিবেলেড তথ্য বিভাগ (যেমন যোগাযোগ, ইনভয়েসে) প্রদর্শন করে — শিরোনাম, আইকন, ঐচ্ছিক এডটি বাটন, এবং কী-ভ্যালু জোড়ার তালিকা যখন খালি ভ্যালুগুলো ফিল্টার করে বাদ দেওয়া হয়।

Widget build(BuildContext context)

facts তালিকা থেকে খালি নয় এমন এন্ট্রিগুলো ফিল্টার করে; কোনো পূরণকৃত এন্ট্রি না থাকলে "Nothing recorded." দেখায়, অন্যথায় প্রতিটি ফ্যাক্টকে লেবেল-ভ্যালু সারহিসিবে রেন্ডার করে।

enum: CompanyPart

কোন অংশ সম্পাদনা করা হচ্ছে তা চিহ্নিত করে — `contact`, `money`, `portal`। একটিমাত্র লম্বা ফর্মের বদলে তিনটি সংক্ষিপ্ত ফর্মে বিভক্ত করা হয়েছে কারণ ভিন্ন ভিন্ন সময়ে ভিন্ন ভিন্ন মানুষ এগুলো সম্পাদনা করে।

টপ-লেভেলে ফাংশন: `Future<void> showCompanySheet(BuildContext context, {required CompanyProfile company, required CompanyPart part})`

নির্দিষ্ট `part`-এর জন্য `_CompanySheet` প্রদর্শনকারী একটি মডাল বটম-শিট খোলে।

ক্লাস/উইজটে: _CompanySheet

`ConsumerStatefulWidget`। কোম্পানির রেকর্ডের একটি অংশ সম্পাদনার জন্য ফর্ম-শিট।

`ConsumerState<_CompanySheet> createState()`

`_CompanySheetState`-এর একটি নতুন ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _CompanySheetState

ফর্ম কন্ট্রোলার তৈরি, প্রাথমিক ভ্যালু সেট এবং সাবমিট পরিচালনা করে।

`Map<String, String?> get _initialValues`

`widget.part` অনুযায়ী প্রাসঙ্গিক ফিল্ডগুলোর প্রাথমিক ভ্যালুর একটি ম্যাপ ফেরত দেয় (`contact` নাম/ফোন/ওয়েবসাইট, `money` ট্যাক্স/ব্যাকিং/মুদ্রা/অর্থবছর, `portal` ট্যাগলাইন/এবোউট)।

`void dispose()`

সকল `TextEditingController` ডিসপোজ করে।

`String? _read(String key)`

প্রদত্ত কী-এর কন্ট্রোলারের টেক্সট ট্রিম করে ফেরত দেয়, খালিহলে `null` রিটার্ন করে।

`Future<void> _submit()`

ফর্ম ভ্যালুডিটে করে, `widget.part` অনুযায়ী উপযুক্ত `CompanyProfileRequest` তৈরি করে, `updateMe` কল করে, সফল হলে `myCompanyProvider` ইনভ্যালুডিটে করে শিট বন্ধ করে; ব্যর্থ হলে ত্রুটির বার্তা সেট করে।

`Widget build(BuildContext context)`

`widget.part` অনুযায়ী একটি `FormSheetFrame` তৈরি করে যাত শিরোনাম এবং সংশ্লিষ্ট ফিল্ডসেট (`contact/money/portal`) থাকে — প্রতটির নিজস্ব ভ্যালুডিশেন ও ব্যাখ্যামূলক টেক্সট।

ফাইল: lib/features/company/public_company_screen.dart

সংক্ষিপ্ত ববিরণ: একটি কোম্পানির পাবলিক পোর্টাল পেজে যত্নে যে কেউ লিংকসহ দেখতে পারে ঠিক সত্বে দেখায়। কোম্পানিস্ক্রিনি থেকে প্রতিটি হিসাবে পৌঁছানো যায় — অপ্ৰমাণীকৃত দর্শক ঠিক এটিই দেখে।

ক্লাস/উইজটে: PublicCompanyScreen

ConsumerWidget। একটি সাবডোমেইন পাবলিক পোর্টাল পেজে রেন্ডার করে — লোগো, নাম, ট্যাগলাইন, এবাউট টেক্সট, এবং পোর্টালে তালিকাভুক্ত সার্ভিসসমূহ।

PublicCompanyScreen({super.key, required this.subdomain})

কনস্ট্রাক্টর, প্রদর্শনিত হবে এমন কোম্পানির সাবডোমেইন স্টেটে করে।

static void open(BuildContext context, {required String subdomain})

Navigator.push দিয়ে একটি MaterialPageRoute খোলে যা প্রদত্ত সাবডোমেইনের জন্য PublicCompanyScreen দেখায়।

Widget build(BuildContext context, WidgetRef ref)

publicCompanyProvider(subdomain) থেকে ডটো লোড করে, লোড/এর অবস্থা হ্যান্ডলে করে, এবং সফল হলে একটি তথ্য ব্যানার ("এটাই সাইন-ইন করার আগে যা দেখা যায়"), কোম্পানির হোম কার্ড (লোগো/নাম/ট্যাগলাইন/এবাউট), এবং _PublicServices উইজটে প্রদর্শন করে।

ক্লাস/উইজটে: _PublicServices

ConsumerWidget। কোম্পানির পাবলিকভাবে বজ্রপতি সার্ভিসগুলোর তালিকা প্রদর্শন করে।

Widget build(BuildContext context, WidgetRef ref)

publicServicesProvider(subdomain) থেকে ডটো লোড করে; লোড/এর অবস্থা হ্যান্ডলে করে, খালি হলে ব্যাখ্যামূলক বার্তা দেখায়, অন্যথায় প্রতীক সার্ভিসের নাম, বিবরণ (দুই লাইন পর্যন্ত) এবং দাম একটি AppCard-এ তালিকাভুক্ত করে।

ফাইল: lib/features/company/subscription_plan_screen.dart

সংক্ষিপ্ত বিবরণ: কোম্পানির প্ল্যানের জন্য অর্থ প্রদান করছে এবং আর কী প্ল্যান উপলব্ধ তা দেখায়। আপগ্রেডে একটি প্রকৃত পেমেন্ট বলে এটি সিস্টেমে ব্রাউজারে SSLCommerz-এর হোস্টেড চেকআউটে পাঠিয়ে দেয়, কারণ পেমেন্ট সফল/ব্যর্থ কলব্যাক শুধু ওয়েব অ্যাপে রিডাইরেক্ট করে।

ক্লাস/উইজটে: SubscriptionPlanScreen

ConsumerWidget। মূল প্ল্যান স্ক্রিনি — কোম্পানির প্রোফাইল ও উপলব্ধ প্ল্যানগুলো লোড করে _Body-তে পাস করে।

Widget build(BuildContext context, WidgetRef ref)

myCompanyProvider ও subscriptionPlansProvider উভয়ের নস্টেডে when হ্যান্ডলিং করে (লোড/এর/ডটো), এবং উভয়ের ডটো প্রস্তুত হলে _Body উইজটে তৈরি করে। পুল-টু-রিফ্রেশে উভয় প্রোভাইডার ইনভ্যালিডেট করে।

ক্লাস/উইজটে: _Body

ConsumerStatefulWidget। কোম্পানির প্রোফাইল ও প্ল্যান লিস্ট নিয়ে প্রকৃত UI বডি রেন্ডার করে।

ConsumerState<_Body> createState()

_BodyState-এর একটি নতুন ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _BodyState

বর্তমান প্ল্যানের দাম নির্ধারণ এবং আপগ্রেডে চেকেআউট শুরু করার কার্যক্রম পরিচালনা করে।

`double get _currentPrice`

বর্তমান সাবস্ক্রিপশন প্ল্যান কন্ডেইশনে সাথে মিলে যাওয়া প্ল্যানের দাম খুঁজে ফেরত দেয়, না পাওয়া গেলে 0।

`Future<void> _upgrade(SubscriptionPlanOption plan)`

`plan.id/plan.price` নাল না হলে `initiateSubscriptionUpgrade` কল করে একটি গিটেওয়ে URL পাঠায়, `launchUrl` দিয়ে সিস্টেমে ব্রাউজারে (external application মোডে) স্টেট খোলে। খোলা না গেলে বা ত্রুটি হলে সন্ধ্যাকবারে বার্তা দেখায়; সফল হলে ব্যবহারকারীকে ব্রাউজারে পেমেন্ট শেষ করে ফিরে এসে রিফ্রেশ করতে বলে।

`Widget build(BuildContext context)`

বর্তমান প্ল্যানের তথ্যসহ একটি কার্ড দেখায়, তারপর প্রতিটি উপলব্ধ প্ল্যানের জন্য একটি `_PlanCard` রেন্ডার করে — `canUpgradeTo` নির্ধারণ করা হয় শুধুমাত্র বর্তমান প্ল্যানের চেয়ে কঠোরভাবে বেশি দামের প্ল্যানের জন্য (ব্যাকএন্ডের নিয়মের সাথে সামঞ্জস্যপূর্ণ)।

ক্লাস/উইজটে: _PlanCard

`StatelessWidget`। একটি একক প্ল্যান বকিল্পের কার্ড — নাম, দাম, ফিচার তালিকা এবং প্রযোজ্য হলে একটি "Upgrade" বাটন প্রদর্শন করে।

`List<String> get _features`

প্ল্যানের বিবরণকে (কমা-বন্ধিত বাক্যাংশ) একটি ফিচার চেকলিস্টে রূপান্তর করে — শেষের বিন্দুচিহ্ন সরিয়ে কমা দিয়ে ভাগ করে, প্রতিটি অংশ ট্রিম ও ক্যাপিটলাইজ করে।

`Widget build(BuildContext context)`

প্ল্যানের নাম, দাম/বলিং সাইকেলে, ফিচার তালিকা (চেকমার্কসহ), এবং `isCurrent/canUpgradeTo` অনুযায়ী একটি স্ট্যাটাস চিহ্ন বা "Upgrade" বাটন প্রদর্শনকারী একটি `AppCard` তৈরি করে।

ফাইল: lib/features/crm/activity_timeline.dart

সংক্ষিপ্ত বিবরণ: এই ফাইলটি একটি ক্লায়েন্ট বা ডলি (opportunity) সাথে সংযুক্ত অ্যাক্টিভিটি টাইমলাইন দেখায় এবং সেগুলো সম্পূর্ণ/মুছে ফেলার সুবিধা দেয়। এছাড়াও এটি AI-জেনারেটেড অ্যাক্টিভিটি সিরসংক্ষেপে দেখানোর একটি বিটম শিট সরবরাহ করে।

টপ-লেভেলে টাইপডেফ ও প্রোভাইডার

`typedef ActivityScope = ({int? clientId, int? opportunityId})`

একটি টাইমলাইন কী কীসের সাথে সংযুক্ত তা বোঝানোর জন্য একটি রিকর্ড টাইপ। এন্ডপয়েন্টটি হয় একটি ক্লায়েন্ট অথবা একটি অপারচুনটিনিয়, দুটোই একসাথে নয়। এই পয়ের আকারে মডেল করার কারণ হলো প্রোভাইডারগুলো এর উপর ভিত্তি করে ক্লনিভাবে কী (key) করতে পারে।

`activitiesProvider`

`FutureProvider.autoDispose.family<List<CrmActivity>, ActivityScope>` — নির্দিষ্ট স্কোপের (client বা opportunity) জন্য `CrmRepository.activities` কল করে অ্যাক্টিভিটি তালিকা লোড করে এবং পজেড রেসপন্স থেকে `content` রিটার্ন করে।

ক্লাস/উইজটে: ActivityTimeline

একটি ক্লায়েন্ট বা ডলিের সাথে সংঘটিত সব কছির তালিকা দেখানো `ConsumerStatefulWidget`। লডি, ক্লায়েন্ট বা ডলি — যেকোনো কছির টাইমলাইন একই আকৃতির, কারণ একটি কিল সবসময় একটি কিলই থাকে, শুধু কীসের সাথে সংযুক্ত তা পাল্টায়। কনস্ট্রাক্টরে `clientId` অথবা `opportunityId` (দুটোই ঐচ্ছকি) নেয়।

`createState()`

`_ActivityTimelineState` ইনস্ট্যান্স তরৈকিরে রটিরন করে — স্ট্যান্ডার্ড Flutter override।

ক্লাস: _ActivityTimelineState

`ActivityTimeline`-এর স্টেটে ক্লাস; কোন অ্যাক্টিভিটি বর্তমানে বজি (loading) অবস্থায় আছে তা ট্র্যাক করে এবং সম্পন্ন/মুছে ফেলার অ্যাকশন চালায়।

`_scope` (getter)

`widget.clientId` ও `widget.opportunityId` থেকে একটি `ActivityScope` রকেড তরৈকিরে রটিরন করে, যা প্রোভাইডার কল করতে ব্যবহৃত হয়।

`Future<void> _run(Future<void> Function(CrmRepository) action, String done)`

একটি রিপোজিটরি-অ্যাকশন সাধারণভাবে চালায়: `CrmRepository`-এর একটি ফাংশন কল করে, সফল হলে `activitiesProvider(_scope)` ইনভ্যালিডিটে করে এবং সাফল্যের বার্তা `SnackBar`-এ দেখায়। `ApiException` ধরা পড়লে তার `message` দেখায়, অন্য যেকোনো ব্যতিক্রমে একটি জিনেরেকি বার্তা দেখায়, এবং শেষে `_busyId` রসিটে করে।

`Future<void> _complete(CrmActivity activity)`

`_busyId` কে `activity.id` স্টেটে করে এবং `repo.completeActivity(activity.id)` কল করে `_run`-এর মাধ্যমে অ্যাক্টিভিটিটি সম্পন্ন হিসেবে চহ্নিত করে, সফল হলে "Marked done." বার্তা দেখায়।

`Future<void> _delete(CrmActivity activity)`

প্রথম `confirmAction` ডায়ালগ দিয়ে ব্যবহারকারীর কাছে নশ্চিতিকরণ চায় ("Remove this entry?")। নশ্চিতি হলে `_busyId` স্টেটে করে `repo.deleteActivity(activity.id)` কল করে টাইমলাইন থেকে এন্ট্রিটি চরিতরে সরিয়ে দেয়।

`Widget build(BuildContext context)`

`activitiesProvider(_scope)`-কে watch করে এবং `currentUserProvider` থেকে বর্তমান ব্যবহারকারী ক্লায়েন্ট (পোর্টাল ব্যবহারকারী) কনি যাচাই করে `canWrite` নরিধারণ করে (রোল-ভিত্তিকি গটেং, পোর্টাল ক্লায়েন্ট ছাড়া বাকি সবাই লখিতে পারে)। একটি `SectionHeader` দেখায় যাত "Summarise" বাটন থাকে যা `showActivitySummarySheet` খোলে, এবং একটি `AppCard`-এ `loading/error/data` অবস্থা অনুযায়ী `_ActivityRow` তালিকা রন্ডার করে। সিস্টেমে-জেনারেটেডে এন্ট্রি (যেমন স্টজে পরিবর্তন) সম্পন্ন বা মুছে ফেলার অপশন পায় না, কারণ সেগুলো অডিটি ট্রাইলের অংশ।

ক্লাস/উইজটে: _ActivityRow

একটি একক অ্যাক্টিভিটি এন্ট্রি প্রদর্শনকারী `StatelessWidget`। বিষয় (subject), বিবরণ, ধরন, সময়, এবং সিস্টেমে-জেনারেটেডে কনি তা দেখায়; এবং একটি বিজি ইন্ডিকিটের অথবা "Mark done"/"Remove" পপআপ মেনু প্রদর্শন করে।

Widget build(BuildContext context)

সম্পন্ন/অসম্পন্ন অনুযায়ী একটি আইকন (চেক/রডেও বাটন) দেখায়, subject ও (থাকলে) description, এরপর type · relative time · (থাকলে) "automatic" জোড়া লাগানো একটি সাবটেক্সট লাইন দেখায়। **busy** হলে একটি ছোট স্পিনার, নাহলে (onComplete/onDelete থাকলে) একটি **PopupMenuButton** দেখায় যা "Mark done" ও লাল রঙের "Remove" আইটেমে দেয়।

টপ-লেভেলে ফাংশন

Future<void> showActivitySummarySheet(BuildContext context, {int? clientId, int? opportunityId})

একটি বিটম শিট খোলে যা একটি ক্লায়েন্ট বা ডিলির সাথে সংঘটিত সব কছির একটি AI-লিখিত সারসংক্ষেপে দেখায়। কছিই সংরক্ষিত হয় না — শুধু চাওয়া হলে তৈরি হয় — এবং যখনই দেখানো হয় সোথানে "মেশিন-লিখিত" হিসেবে লেবেল করা থাকে, যাতে এটুকোনো সহকর্মীর নোট হিসেবে ভুল বোঝা না হয়।

ক্লাস/উইজটে: _ActivitySummarySheet

বিটম শিটের কনটেন্ট, যা clientId/opportunityId নিয়ে সারসংক্ষেপে ফিচ করে দেখায়।

createState()

_ActivitySummarySheetState তৈরিকরে রটার্ন করে।

ক্লাস: _ActivitySummarySheetState

সারসংক্ষেপে ফিচ ও প্রদর্শনের স্টেটে। **_summary** একটি late final Future<String> যা ইনশিয়ালাইজেশনের সময় **crmRepositoryProvider().summariseActivity(...)** কল করে একবারই ফিচ করে।

Widget build(BuildContext context)

শিরোনাম "Where things stand" ও একটি ইটালিকি নোট "Written by the assistant, not by a person." দেখায়। **FutureBuilder<String>** দিয়ে **_summary**-এর অবস্থা অনুযায়ী: লোডিং হলে **Loader**, এরর হলে **MessageBanner.error**, খালি রিজোল্টে "কছি আসনি" বার্তা, এবং ডটো থাকলে **SelectableText**-এ সারসংক্ষেপে দেখায়।

ফাইল: lib/features/crm/client_form_sheet.dart

সংক্ষিপ্ত বিবরণ: ক্লায়েন্ট তৈরি ও সম্পাদনার জন্য দুটি বিটম শিট ফর্ম সরবরাহ করে। উভয় ফর্মই একটি সাধারণ **_SheetFrame** শেয়ার করে, যাতে শুধু নিজ নিজ ফিল্ডগুলোর মধ্যে পার্থক্য থাকে।

টপ-লেভেলে ফাংশন

Future<void> showNewClientSheet(BuildContext context)

_NewClientSheet উইজটেকে একটি **isScrollControlled** মোডাল বিটম শিটে খোলে, নতুন ক্লায়েন্ট তৈরি ফর্ম দেখানোর জন্য।

Future<void> showEditClientSheet(BuildContext context, Client client)

প্রদত্ত client-কে সম্পাদনার জন্য **_EditClientSheet** উইজটেকে একটি বিটম শিটে খোলে।

ক্লাস/উইজটে: _SheetFrame

নডি/এডটি শটিরে জন্য শয়ের করা ক্রোম (title, form, error banner, spacing)। কনস্ট্রাক্টরে title, formKey, error, onDismissError, এবং children নিয়ে।

Widget build(BuildContext context)

কীবোর্ড ইনস্টে মনে চলা একটি SingleChildScrollView-এর ভেতরে একটি Form তৈরি করে, শিরোনাম দেখায়, error নন-নাল হলে MessageBanner.error দেখায়, এবং children রেন্ডার করে।

ক্লাস/উইজটে: _NewClientSheet

নতুন ক্লায়েন্ট তৈরি ফর্ম উইজটে (ConsumerStatefulWidget), কোনো প্রারামিটির নিয়ে না।

createState()

_NewClientSheetState তৈরি করে রিটার্ন করে।

ক্লাস: _NewClientSheetState

নতুন ক্লায়েন্ট ফর্মের স্টেটে: প্রথম নাম, শেষ নাম, ইমেইল, ফোন, কোম্পানি, ইন্ডাস্ট্রি, ওয়েবসাইটে জন্য টেক্সট কন্ট্রোলার, এবং _submitting/_error স্টেটে রাখা।

void dispose()

সবগুলো TextEditingController (_firstName, _lastName, _email, _phone, _company, _industry, _website) dispose করে এবং সুপারের dispose() কল করে।

Future<void> _submit()

ইতিমধ্যে সাবমিট হচ্ছে বা ভ্যালিডেশন ফেইল হলে বেরিয়ে যায়। ফোকাস সরিয়ে _submitting=true স্টেটে করে, clientsProvider.notifier.create(CreateClientRequest(...)) কল করে নতুন ক্লায়েন্ট তৈরি করে, সফল হলে শিট বন্ধ করে "Client added." সন্যাকবার দেখায়। ApiException বা অন্য এরর হলে _error স্টেটে করে, শেষে _submitting=false করে।

Widget build(BuildContext context)

_SheetFrame-এর ভেতরে প্রথম নাম, শেষ নাম, ইমেইল, ফোন (ঐচ্ছিক), কোম্পানি (ঐচ্ছিক), ইন্ডাস্ট্রি (ঐচ্ছিক), ওয়েবসাইট (ঐচ্ছিক) ফিল্ড দেখায়, প্রতিটি নজিস্ব ভ্যালিডেটর সহ (নাম কমপক্ষে ২ অক্ষর, ইমেইলে "@" থাকা আবশ্যিক)। এরপর একটি _Note দেখায় যে এটি শুধু ক্লায়েন্ট রেকর্ড তৈরি করে, পোর্টাল লগইন আলাদা পদক্ষেপে, এবং একটি LoadingButton ("Add client") দেখায়।

ক্লাস/উইজটে: _EditClientSheet

বর্তমান ক্লায়েন্ট সম্পাদনার ফর্ম, কনস্ট্রাক্টরে client নিয়ে।

createState()

_EditClientSheetState তৈরি করে রিটার্ন করে।

ক্লাস: _EditClientSheetState

এডটি ফর্মের স্টেটে: কোম্পানি, ইন্ডাস্ট্রি, ওয়েবসাইট, কর্মচারী সংখ্যা কন্ট্রোলার এবং _status স্ট্রিং, _submitting/_error রাখা।

void initState()

widget.client-এর বর্তমান মান থেকে সব কন্ট্রোলার (_company, _industry, _website, _employees) ও _status ইনশিয়ালাইজ করে।

`void dispose()`

`_company, _industry, _website, _employees` কন্ট্রোলার `dispose` করে সুপার কল করে।

`Future<void> _submit()`

ভ্যালিডেশন পাস হলে `clientsProvider.notifier.updateItem(widget.client.id, UpdateClientRequest(...))` কল করে ক্লায়েন্ট আপডেট করে, ফোন সংখ্যা `int.tryParse` দিয়ে পার্স করে। সফল হলে শিট বন্ধ করে "Client updated." দেখায়; এরর হ্যান্ডলিং `_NewClientSheetState._submit`-এর মতোই।

`Widget build(BuildContext context)`

`clientStatuses`-এ বর্তমান `_status` না থাকলে সটেকে তালিকার শুরুতে যোগ করে (যাতে ড্রপডাউন কখনো অজানা মান নিয়ে ক্র্যাশ না করে)। কোম্পানি, ইন্ডাস্ট্রি, ওয়েবসাইট, কর্মচারী (নন-নগেটেভি পূর্ণসংখ্যা ভ্যালিডেশন সহ), এবং স্ট্যাটাস ড্রপডাউন দেখায়। এরপর একটি `_Note` দেখায় যে নাম ও ইমেল এখানে সম্পাদনাযোগ্য নয় (সেগুলো ক্লায়েন্টের নিজস্ব অ্যাকাউন্টের অংশ), এবং একটি `LoadingButton` ("Save changes") দেখায়।

ক্লাস/উইজটে: `_Note`

তথ্যমূলক নোট প্রদর্শনকারী ছোট `StatelessWidget`, একটি `info` আইকন ও ধূসর টেক্সট নিয়ে গঠিত। কনস্ট্রাক্টরে শুধু `text` নেয়।

`Widget build(BuildContext context)`

একটি `info` আইকন ও প্রদত্ত `text`-কে `Expanded` টেক্সটে সারবিদ্ধভাবে দেখায়।

ফাইল: `lib/features/crm/clients_tab.dart`

সংক্ষিপ্ত বিবরণ: ক্লায়েন্টদের তালিকা ট্যাব ও একজন ক্লায়েন্টের বিস্তারিত স্ক্রিনি দেখায়, যখন ক্লায়েন্টের তথ্য, ট্যাগ, অ্যাকটিভিটি টাইমলাইন এবং এডটি/ডলিটি/পোর্টাল-ইনভাইট/নতুন-ডলি অ্যাকশনসমূহ থাকে।

ক্লাস/উইজটে: `ClientsTab`

ক্লায়েন্টদের পেজড তালিকা দেখানো `ConsumerWidget`।

`Widget build(BuildContext context, WidgetRef ref)`

`clientsProvider`-কে `watch` করে একটি `PagedListView<Client>` তৈরি করে, রিফ্রেশ/লোড-মোর হ্যান্ডলার, খালি/এরর অবস্থার বার্তা, এবং প্রতিটি আইটেমের জন্য `_ClientCard` দেখায় যা ট্যাপ করলে `_openClient` কল করে।

`static void _openClient(BuildContext context, int id)`

`ClientDetailScreen(id: id)`-কে একটি `MaterialPageRoute`-এর মাধ্যমে নভিগেটে করে।

ক্লাস/উইজটে: `_ClientCard`

তালিকায় একজন ক্লায়েন্টের সংক্ষিপ্ত কার্ড, কনস্ট্রাক্টরে `client` ও ঐচ্ছিক `onTap` নেয়।

`Widget build(BuildContext context)`

একটি `Avatar`, ক্লায়েন্টের `headline/subline`, `StatusChip`, লাইফটাইম ভ্যালু, রকিওয়েস্ট সংখ্যা, পোর্টাল-অ্যাক্সেস আইকন (থাকলে), এবং সর্বোচ্চ ৩টি `TagChip` দেখায়।

ক্লাস/উইজটে: ClientDetailScreen

একজন ক্লায়েন্টের সম্পূর্ণ বিস্তারিত স্ক্রিনি, কনস্ট্রাক্টরে id নিয়ে (ConsumerWidget)।

Widget build(BuildContext context, WidgetRef ref)

clientDetailProvider(id) watch করে; AppBar-এর অ্যাকশনে ডটো লোড হলে _ClientMenu দেখায় (এডটি/ডলিটি/ডলি তৈরি অনুমতিগুলো permissionControllerProvider থেকে চেকে করে)। বডিতে loading/error/data হ্যান্ডল করে; ডটো থাকলে একটি RefreshIndicator-এ অ্যাভাটার-কার্ড, ট্যাগ, _ClientFacts, এবং ActivityTimeline(clientId: client.id) দেখায়।

ক্লাস/উইজটে: _ClientFacts

ক্লায়েন্টের ফ্যাক্ট শিট (ইমেইল, ফোন, ইন্ডাস্ট্রি, ওয়েবসাইট ইত্যাদি) দেখানো StatelessWidget, কনস্ট্রাক্টরে client নিয়ে।

Widget build(BuildContext context)

একটি রেকর্ড তালিকা (label, value) তৈরি করে (ইমেইল, ফোন, ইন্ডাস্ট্রি, ওয়েবসাইট, অ্যাকাউন্ট ম্যানজোর, কর্মচারী, বার্ষিকি আয়, লাইফটাইম ভ্যালু, রকিওয়েস্ট, পোর্টাল অ্যাক্সেস, অনবোর্ডে তারিখ, তৈরি তারিখ), খালি মান বাদ দিয়ে ফিল্টার করে, এবং একটি SectionHeader ও AppCard-এর ভেতরে প্রতিটি সারি label/value আকারে ডিভাইডার সহ দেখায়।

ক্লাস/উইজটে: _ClientMenu

AppBar-এর ওভারফ্লো মেনুতে ক্লায়েন্ট-সম্পর্কিত অ্যাকশন (ConsumerWidget)। "New deal" এখানে থাকে কারণ ব্যাকএন্ড ডিরেক্ট-ক্রিয়েটে পাথে একটি ক্লায়েন্ট আবশ্যিক করে। কনস্ট্রাক্টরে client, canEdit, canDelete, canOpenDeal নিয়ে।

Future<void> _invite(BuildContext context, WidgetRef ref)

ক্লায়েন্টকে পোর্টালে আমন্ত্রণ জানায় এবং তাদের অ্যাক্সেস চালু করে। প্রথমে confirmAction দিয়ে নিশ্চিতকরণ চায় (কারণ একই ট্যাপে দুটি ফলাফল হয় — ইমেইল পাঠানো ও অ্যাক্সেস চালু করা)।

নিশ্চিতি হলে crmRepositoryProvider().inviteToPortal(client.id) কল করে, clientDetailProvider(client.id) ইনভ্যালিডেট করে, এবং "Invitation sent." দেখায়। এরর হ্যান্ডলিং ApiException-এর বার্তা দেখায় (যেমন ইমেইল না থাকলে ব্যাকএন্ডের প্রত্যাখ্যান বার্তা)।

Future<void> _delete(BuildContext context, WidgetRef ref)

একটি AlertDialog দিয়ে নিশ্চিতকরণ চায় (মুছে ফেললে ক্লায়েন্ট CRM থেকে সরে যাবে, খোলা ডলি থাকলে মুছা যাবে না)। নিশ্চিতি হলে clientsProvider.notifier.delete(client.id) কল করে, নভিগেটর পপ করে, "Client deleted." দেখায়। এরর হলে ব্যাকএন্ডের বার্তা (যেমন খোলা-ডলি প্রত্যাখ্যান) দেখায়।

Widget build(BuildContext context, WidgetRef ref)

কোনো অনুমতি না থাকলে SizedBox.shrink() রিটার্ন করে। নাহলে একটি PopupMenuButton দেখায় যাতে অনুমতি অনুযায়ী "Edit client", "New deal" (showNewOpportunitySheet খোলে), "Invite to the portal" (শুধু পোর্টাল অ্যাক্সেস চালু না থাকলে), ও "Delete" আইটেম থাকে, এবং সলিকেশন অনুযায়ী সংশ্লিষ্ট মথোড কল করে।

ফাইল: lib/features/crm/contacts_screen.dart

সংক্ষিপ্ত ববিরণ: ক্লায়েন্ট কোম্পানির যোগাযোগ ব্যক্তি(contacts) এবং সিস্টেম-ব্যাপী ট্যাগের জন্য দুটি ট্যাব সরবরাহ করে, সাথে সংশ্লিষ্ট সব প্রোভাইডার, সাব-স্ক্রিনি ও ফর্ম শাট।

ক্লাস/উইজটে: ContactsScreen

"People" ও "Tags" — দুটি ট্যাবের কনটইনার (ConsumerWidget), যগুলো অনুমতি অনুযায়ী দেখানো হয়।

Widget build(BuildContext context, WidgetRef ref)

permissionControllerProvider watch করে; অনুমতি এখনো লোড না হলে একটি Loader দেখায়। CrmPermissions.contactView/tagView অনুযায়ী উপলব্ধ ট্যাবের তালিকা তৈরি করে (প্রতিটি ট্যাবে label, view, ঐচ্ছকি create ফাংশন থাকে); কোনো ট্যাব উপলব্ধ না থাকলে একটি EmptyState ("Not available to you") দেখায়। নাহলে DefaultTabController-এর মধ্যে একটি TabBar/TabBarView তৈরি করে, বর্তমান ট্যাব অনুযায়ী ফ্লোটিং অ্যাকশন বাটন (create থাকলে) দেখায়।

ক্লাস: ContactSearchController

Notifier<String> — সব যোগাযোগ ব্যক্তির উপর ফ্রন্টিকেস্ট সার্চ কীওয়ার্ড ধরে রাখে।

String build()

প্রাথমিক অবস্থা হিসেবে খালি স্ট্রিং রিটার্ন করে।

void set(String keyword)

প্রদত্ত কীওয়ার্ড ট্রিম করে বর্তমান স্টেটে সমান হলে কিছু করে না, নাহলে স্টেটে আপডেট করে (যা AllContactsController-কে পুনরায় লোড করতে ট্রিগার করে)।

contactSearchProvider

NotifierProvider<ContactSearchController, String> — উপরের কন্ট্রোলার এক্সপোজ করে।

ক্লাস: AllContactsController

AsyncNotifier<List<ClientContact>> — সব ক্লায়েন্টের সব যোগাযোগ ব্যক্তির ফ্ল্যাট, সার্চযোগ্য তালিকা।

Future<List<ClientContact>> build()

currentUserProvider ও contactSearchProvider watch করে (ব্যবহারকারী বদলালে বা সার্চ কীওয়ার্ড বদলালে পুনরায় লোড হয়) এবং _load() কল করে।

Future<List<ClientContact>> _load()

crmRepositoryProvider().allContacts(keyword: ...) কল করে পেজড রেসপন্সের content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে স্টেটে পুনরায় লোড করে, এরর হলে স্টেটিও স্টেটে ধরে রাখে।

allContactsProvider

AsyncNotifierProvider<AllContactsController, List<ClientContact>> — উপরের কন্ট্রোলার এক্সপোজ করে।

টপ-লভেলে প্রোভাইডার

`contactsForClientProvider`

`FutureProvider.autoDispose.family<List<ClientContact>, int>` — একটি নির্দিষ্ট ক্লায়েন্টে যোগাযোগ ব্যক্তিদের তালিকা `crmRepositoryProvider().contactsFor(clientId)` থেকে ফেটে করে (ক্লায়েন্টের নজিস্ব স্ক্রিনিংর জন্য)।

ক্লাস: TagsController

`AsyncNotifier<List<Tag>>` — সিস্টেমের সব ট্যাগ পরিচালনা করে।

`Future<List<Tag>> build()`

`currentUserProvider` watch করে এবং `crmRepositoryProvider().tags()` কল করে ট্যাগ তালিকা লোড করে।

`Future<void> refresh()`

`AsyncValue.guard`-এর মাধ্যমে `crmRepositoryProvider().tags()` পুনরায় কল করে স্টেটে রিফ্রেশ করে।

`void apply(Tag updated)`

বর্তমান তালিকায় `updated.id`-এর সাথে মিলে যাওয়া ট্যাগটিকে আপডেটেড ভার্সন দিয়ে প্রতিস্থাপন করে, বাকিগুলো অপরিবর্তিত রাখে।

`void remove(int id)`

বর্তমান তালিকা থেকে প্রদত্ত `id`-এর ট্যাগটি বাদ দিয়ে নতুন স্টেটে সেট করে।

`tagsProvider`

`AsyncNotifierProvider<TagsController, List<Tag>>` — উপরের কন্ট্রোলার এক্সপোজ করে।

ক্লাস/উইজটে: _ContactsTab

"People" ট্যাবের কন্টেন্ট — সার্চ-সহ সব যোগাযোগ ব্যক্তির তালিকা (`ConsumerStatefulWidget`)।

`createState()`

`_ContactsTabState` তৈরি করে রিটার্ন করে।

ক্লাস: _ContactsTabState

সার্চ টেক্সট কন্ট্রোলার ধরে রাখে।

`void dispose()`

`_search` কন্ট্রোলার `dispose` করে সুপার কল করে।

`Widget build(BuildContext context)`

`allContactsProvider` watch করে একটি `ConfigList<ClientContact>` তৈরি করে, একটি সার্চ `TextField` হডার সহ যা সাবমিট হলে `contactSearchProvider.notifier.set` কল করে। প্রতিটি সারিতে নাম, কোম্পানি/রোল/ইমেইল সাবটাইটলে, এবং প্রাইমারি হলে "primary" লেবেলে দেখায়।

ক্লাস/উইজটে: ClientContactsScreen

একটি নির্দিষ্ট ক্লায়েন্টের সব যোগাযোগ ব্যক্তির স্ক্রিনিং, যখন

যোগ/সম্পাদনা/প্রাইমারি-করা/মুছে ফেলার অ্যাকশন থাকে। কনস্ট্রাক্টরে `clientId` ও `clientName` নেয়।

static void open(BuildContext context, {required int clientId, required String clientName})

ClientContactsScreen-কে একটি MaterialPageRoute-এর মাধ্যমে পুশ করে নভেগিটে করা হয়।

createState()

_ClientContactsScreenState তৈরি করে রিটার্ন করে।

ক্লাস: _ClientContactsScreenState

কোন কন্ট্র্যাক্ট বজি আছে তা ট্র্যাক করে ও অ্যাকশন চালায়।

Future<void> _act(int id, Future<void> Function() action, String success, String failure)

সাধারণ অ্যাকশন-রানার: _busyId স্টেটে করে action() কল করে, সফল হলে

contactsForClientProvider(widget.clientId) ইনভ্যালিডেট করে এবং allContactsProvider.notifier.refresh()

কল করে (ফ্ল্যাট তালিকাও সঙ্কে থাকে), তারপর success বার্তা দেখায়। এরর হলে ApiException-এর বার্তা বা failure দেখায়, শেষে _busyId রিসেট করে।

Future<void> _delete(ClientContact contact)

confirmAction দিয়ে নিশ্চিতকরণ চায় — বার্তাটি প্রাইমারি কন্ট্র্যাক্ট হলে ভিন্ন (অন্য কাউকে প্রমোট করতে হবে বলে সতর্ক করে)। নিশ্চিতি হলে _act দিয়ে repo.deleteContact(widget.clientId, contact.id) কল করে।

Widget build(BuildContext context)

permissionControllerProvider থেকে canWrite/canDelete নির্ধারণ করে। একটি

FloatingActionButton.extended ("Add", contactCreate অনুমতি থাকলে) দেখায় যা showContactSheet

খোলে। বডিতে contactsForClientProvider(widget.clientId) watch করে ConfigList<ClientContact>

দেখায়, প্রতটি সিরিজে এডটি বাটন (canWrite হলে showContactSheet খোলে), এবং অ্যাকশন হিসেবে

"Make primary" (শুধু নন-প্রাইমারিও canWrite হলে, repo.makePrimaryContact কল করে) এবং "Remove" (canDelete হলে _delete কল করে) দেখায়।

ক্লাস/উইজটে: _TagsTab

"Tags" ট্যাবের কন্টেন্ট (ConsumerStatefulWidget)।

createState()

_TagsTabState তৈরি করে রিটার্ন করে।

ক্লাস: _TagsTabState

কোন ট্যাগ বজি আছে তা ট্র্যাক করে।

Future<void> _delete(Tag tag)

confirmAction দিয়ে নিশ্চিতকরণ চায় (ট্যাগ মুছে ট্যাগ করা রেকর্ডগুলো শুধু ট্যাগটি হারায়ে, রেকর্ড অক্ষত থাকে)। নিশ্চিতি হলে _busyId স্টেটে করে crmRepositoryProvider().deleteTag(tag.id) কল করে, tagsProvider.notifier.remove(tag.id) দিয়ে স্থানীয়ভাবে সরিয়ে দেয় এবং সাফল্য বার্তা দেখায়। এরর হ্যান্ডলিং যথারীতি।

Widget build(BuildContext context)

permissionControllerProvider-এর tagManage থেকে canManage নির্ধারণ করে, tagsProvider watch করে ConfigList<Tag> তৈরি করে, প্রতটি সিরিজে _TagRow দিয়ে রেন্ডার করে (এডটি/ডিলিট শুধু canManage হলে সক্রিয়)।

ক্লাস/উইজটে: _TagRow

একটি একক ট্যাগের সারি (StatelessWidget), কনস্ট্রাক্টরে tag, busy, ঐচ্ছকি onEdit/onDelete নিয়ে।

Widget build(BuildContext context)

ট্যাগের রঙের একটি বৃত্ত (tag.rgb পার্স না হলে মডিউডে রঙে ফলব্যাক করে), ট্যাগের নাম, এবং busy হলে স্পিনিংর নাহলে (মেনু আইটেমে থাকলে) "Edit"/লাল "Remove" সহ PopupMenuButton দেখায়।

উপ-লভেলে ফাংশন ও ফর্ম শাট

Future<void> showContactSheet(BuildContext context, {required int clientId, ClientContact? existing})

_ContactSheet-কে একটি isScrollControlled বটম শাট খোলে, নতুন কন্ট্রাক্ট তৈরি বা (existing দেওয়া থাকলে) সম্পাদনার জন্য।

ক্লাস/উইজটে: _ContactSheet

যোগাযোগ ব্যক্তির তৈরি/সম্পাদনা ফর্ম (ConsumerStatefulWidget), কনস্ট্রাক্টরে clientId ও ঐচ্ছকি existing নিয়ে।

createState()

_ContactSheetState তৈরি করে রিটার্ন করে।

ক্লাস: _ContactSheetState

নাম, ইমেইল, ফোন, পদবি, বিভাগ, নোট-এর টেক্সট কন্ট্রোলার (বর্তমান কন্ট্রাক্টের মান দিয়ে প্রি-ফিল্ড হয়), এবং _makePrimary, _submitting, _error স্টেটে রাখে।

_isEdit (getter)

widget.existing != null কনি রিটার্ন করে — বুঝায় এটি এডিট মোড না নতুন-তৈরি মোড।

void dispose()

সবগুলো কন্ট্রোলার (_fullName, _email, _phone, _jobTitle, _department, _notes) লুপে dispose করে সুপার কল করে।

Future<void> _submit()

ভ্যালিডেশন পাস হলে একটি ClientContactRequest তৈরি করে (_makePrimary সত্য হলেই primaryContact: true পাঠায়)। এডিট মোডে repo.updateContact, নাহলে repo.createContact কল করে; সফল হলে contactsForClientProvider(widget.clientId) ইনভ্যালিডেট করে ও allContactsProvider.notifier.refresh() কল করে, শাট বন্ধ করে সাফল্য বার্তা দেখায়। এরর হ্যান্ডলিং যথারীতি _error-এ স্টেট করে।

String? Function(String?) _max(int limit)

একটি ভ্যালিডেটর ফ্যাক্টরি রিটার্ন করে — ইনপুট ট্রিম করে limit-এর বেশি দৈর্ঘ্য হলে "\$limit characters at most." এরর দেয়, নাহলে null।

Widget build(BuildContext context)

FormSheetFrame-এর ভেতরে নাম (আবশ্যিক, সর্বোচ্চ ১৫০ অক্ষর), ইমেইল (ঐচ্ছকি, সর্বোচ্চ ২৫৫), ফোন (ঐচ্ছকি, সর্বোচ্চ ৩০), পদবি ও বিভাগ (পাশাপাশি সারিতে, সর্বোচ্চ ১০০ করে), নোট (২ লাইন) ফিল্ড দেখায়। ইতিমধ্যে প্রাইমারি হলে একটি তিথ্য টেক্সট দেখায়, নাহলে একটি SwitchListTile ("Make them the primary contact") দেখায় যা _makePrimary টগল করে।

টপ-লেভেলে কনস্ট্রাক্ট

`_tagColours`

৮টি নির্দিষ্ট হেক্স রঙের একটি স্থির তালিকা (`const List<String>`), যা ট্যাগের জন্য ব্যবহৃত হয়। ফর্ম কালার পিকারের পরিবর্তে নির্দিষ্ট সেটে রাখা হয়েছে যাতে ট্যাগগুলো এক নজরে আলাদা করা যায়।

টপ-লেভেলে ফাংশন

`Future<void> showTagSheet(BuildContext context, {Tag? existing})`

`_TagSheet`-কে একটি বিটম শিটে খোলে, নতুন ট্যাগ তৈরি বা (`existing` থাকলে) সম্পাদনার জন্য।

ক্লাস/উইজটে: `_TagSheet`

ট্যাগ তৈরি/সম্পাদনা ফর্ম (`ConsumerStatefulWidget`), কনস্ট্রাক্টরে ঐচ্ছিক `existing` নিয়ে।

`createState()`

`_TagSheetState` তৈরি করে রিটার্ন করে।

ক্লাস: `_TagSheetState`

নাম কন্ট্রোলার ও নির্বাচনী `_colour` স্টেটে রাখে।

`_isEdit` (**getter**)

`widget.existing != null` রিটার্ন করে।

`_seedColour` (**getter**)

বর্তমান ট্যাগের সংরক্ষণ রঙকে `_tagColours`-এর একটির সাথে মেলানোর চেষ্টা করে (হ্যাশ যোগ/আপারকসে নরমলাইজ করে); মিললে সেট, না মিললে বা `existing` না থাকলে প্রথম রঙ রিটার্ন করে — যাতে সম্পাদনা নিরব রঙ না বদলায়।

`void dispose()`

`_name` কন্ট্রোলার `dispose` করে সুপার কল করে।

`Future<void> _submit()`

ভ্যালিডেশন পাস হলে একটি `TagRequest(name, color)` তৈরি করে। এটি মোডে `repo.updateTag` কল করে ফলাফল `tagsProvider.notifier.apply(updated)`-এ পাঠায়; নতুন-তৈরিতে `repo.createTag` কল করে `tagsProvider.notifier.refresh()` কল করে। সফল হলে শিট বন্ধ করে সাফল্য বার্তা দেখায়; এরর হ্যান্ডলিং যথারীতি।

`Widget build(BuildContext context)`

`FormSheetFrame`-এর ভেতরে নাম ফিল্ড এবং `_tagColours`-এর প্রতীতির জন্য একটি বৃত্তাকার `InkWell` সোয়াচ (`Wrap`-এ) দেখায় যা ট্যাপে `_colour` আপডেট করে; নির্বাচনী রঙের বৃত্তে মোটা বর্ডার দেখায়।

ফাইল: `lib/features/crm/crm_controllers.dart`

সংক্ষিপ্ত বিবরণ: লিড, অপরচুনটি (পাইপলাইন) ও ক্লায়েন্টের জন্য Riverpod কন্ট্রোলার ও প্রোভাইডারের সংকলন — প্রতীতি এন্টিটি পজেড লিস্টিং, তৈরি/আপডেট/ডিলিট এবং বিস্তারিত-প্রোভাইডার এখানে সংজ্ঞায়িত।

ক্লাস: LeadViewController

`Notifier<LeadView>` — লডি ট্যাব বর্তমানে কোন ভিউ (mine/সব ইত্যাদি) দেখাচ্ছে তা ধরে রাখবে।
উইজটে স্টেটে বদলে প্লাইন নোটফায়ার ব্যবহার করা হয়েছে যাত্রে একটি লিডের ভেতরে গিয়ে ফিরে আসার পরও ভিউ বজায় থাকে।

`LeadView build()`

প্রাথমিক অবস্থা হিসেবে `LeadView.mine` রটরিন করে।

`void set(LeadView view)`

নতুন ভিউ বর্তমানের সমান না হলে স্টেটে আপডেট করে।

`leadViewProvider`

`NotifierProvider<LeadViewController, LeadView>` — উপরে কন্ট্রোলার এক্সপোজ করে।

ক্লাস: LeadsController

`AsyncNotifier<PagedState<Lead>>` (মক্সইন `PagedLoader<Lead>`) — লিডের পজেড তালিকা পরচালনা করে।

`Future<PagedState<Lead>> build()`

`currentUserProvider` ও `leadViewProvider` watch করে (ভিউ বদলালে তালিকা পুনরায় লোড হয়) এবং `loadFirstPage()` কল করে।

`Future<PagedResponse<Lead>> fetchPage(int page)`

`crmRepositoryProvider().leads(ref.read(leadViewProvider), page: page)` কল করে একটি নির্দিষ্ট পজে ফেচে করে — `PagedLoader` মক্সইনের প্রয়োজনীয় override।

`Future<Lead> create(CreateLeadRequest request)`

`crmRepositoryProvider().createLead(request)` কল করে নতুন লিড তৈরিকরে, `refresh()` কল করে তালিকা রফ্রেশ করে, এবং তৈরি হওয়া লিড রটরিন করে।

`Future<Lead> updateItem(int id, UpdateLeadRequest request)`

`crmRepositoryProvider().updateLead(id, request)` কল করে, ফলাফল দিয়ে `replaceItem` করে সম্পাদতি সারি স্থানে বদলে দেয় (পুরো তালিকা পুনরায় লোড না করে স্ক্রল পজিশন বজায় রাখতে), এবং `leadDetailProvider(id)` ইনভ্যালিডে করে (কারণ একই লিড ডিটেইল স্ক্রিনি খোলা থাকতে পারে)।

`Future<void> delete(int id)`

`crmRepositoryProvider().deleteLead(id)` কল করে, তারপর `removeItem` দিয়ে স্থানীয় তালিকা থেকে সারি বাদ দেয় (রফিচে ছাড়াই, কারণ মোছা লিড পরবর্তী পজে ফিরে আসতে পারে না)।

`leadsProvider`

`AsyncNotifierProvider<LeadsController, PagedState<Lead>>` — উপরে কন্ট্রোলার এক্সপোজ করে।

উপ-লভেলে প্রোভাইডার

`leadDetailProvider`

`FutureProvider.autoDispose.family<Lead, int>` — একটি নির্দিষ্ট id-এর জন্য `crmRepositoryProvider().lead(id)` কল করে একক লিড ফেচে করে।

leadActivitiesProvider

FutureProvider.autoDispose.family<List<CrmActivity>, int> — একটি লিডের id অনুযায়ী
crmRepositoryProvider().leadActivities(id) কল করে অ্যাক্টিভিটি তালিকা ফেচ করে।

ক্লাস: PipelineStageController

Notifier<String?> — পাইপলাইন কোন স্টেজে ফিল্টার করা আছে তা ধরে রাখে; null মানে সব খোলা স্টেজে।

String? build()

প্রাথমিক অবস্থা হিসেবে null রিটার্ন করে।

void set(String? stage)

নতুন স্টেজে বর্তমানের সমান না হলে স্টেটে আপডেট করে।

pipelineStageProvider

NotifierProvider<PipelineStageController, String?> — উপরে কন্ট্রোলার এক্সপোজ করে।

ক্লাস: OpportunitiesController

AsyncNotifier<PagedState<Opportunity>> (মিক্সইন PagedLoader<Opportunity>) — অপরচুনটির (ডলি) পজেড তালিকা পরচালনা করে।

Future<PagedState<Opportunity>> build()

currentUserProvider ও pipelineStageProvider watch করে এবং loadFirstPage() কল করে।

Future<PagedResponse<Opportunity>> fetchPage(int page)

crmRepositoryProvider().opportunities(stage: ref.read(pipelineStageProvider), page: page) কল করে একটি পজে ফেচ করে।

Future<Opportunity> changeStage(int id, ChangeStageRequest request)

crmRepositoryProvider().changeStage(id, request) কল করে একটি ডলিকে নতুন স্টেজে সরায়, pipelineSummaryProvider ও opportunityDetailProvider(id) ইনভ্যালিডেট করে, এবং refresh() কল করে তালিকা রিফ্রেশ করে। সারাংশ পুনরায় ফেচ করা হয় কারণ probability ও weighted forecast সারভার-নরিভর — ফোনে হিসাব করলে ওয়েবে অ্যাপের সংখ্যার সাথে অমলি হতে পারত।

Future<Opportunity> create(OpportunityRequest request)

crmRepositoryProvider().createOpportunity(request) কল করে নতুন ডলি তৈরি করে, pipelineSummaryProvider ইনভ্যালিডেট করে ও refresh() কল করে।

Future<Opportunity> updateItem(int id, OpportunityRequest request)

crmRepositoryProvider().updateOpportunity(id, request) কল করে, replaceItem দিয়ে স্থানীয়ভাবে সারাংশ বদলায়, pipelineSummaryProvider (অ্যামাউন্ট এডটিভোগ্য বলে সারাংশও বদলাতে পারে) ও opportunityDetailProvider(id) ইনভ্যালিডেট করে।

Future<void> delete(int id)

crmRepositoryProvider().deleteOpportunity(id) কল করে, removeItem দিয়ে স্থানীয় তালিকা থেকে বাদ দেয়, এবং pipelineSummaryProvider ইনভ্যালিডেট করে।

opportunitiesProvider

AsyncNotifierProvider<OpportunitiesController, PagedState<Opportunity>> — উপরে কন্ট্রোলার এক্সপোজ করে।

টপ-লেভেলে প্রোভাইডার

opportunityDetailProvider

FutureProvider.autoDispose.family<Opportunity, int> — একটি নির্দিষ্ট id-এর অপরচুনটি বিস্তারিত ফেচ করে।

pipelineSummaryProvider

FutureProvider<PipelineSummary> — currentUserProvider watch করে
crmRepositoryProvider().pipelineSummary() কল করে পাইপলাইনের হডেলাইন পরিসংখ্যান আনবে।

ক্লাস: ClientsController

AsyncNotifier<PagedState<Client>> (মক্সইন PagedLoader<Client>) — ক্লায়েন্টের পজেড তালিকা পরচালনা করে।

Future<PagedState<Client>> build()

currentUserProvider watch করে loadFirstPage() কল করে।

Future<PagedResponse<Client>> fetchPage(int page)

crmRepositoryProvider().clients(page: page) কল করে একটি পজে ফেচ করে।

Future<Client> create(CreateClientRequest request)

crmRepositoryProvider().createClient(request) কল করে নতুন ক্লায়েন্ট তৈরি করে, refresh() কল করে তালিকা রফ্রেশ করে।

Future<Client> updateItem(int id, UpdateClientRequest request)

crmRepositoryProvider().updateClient(id, request) কল করে, replaceItem দিয়ে স্থানীয় সারি বিদলয়, clientDetailProvider(id) ইনভ্যালিডেট করে।

Future<void> delete(int id)

crmRepositoryProvider().deleteClient(id) কল করে, removeItem দিয়ে সারি বিদ দাখ। খোলা অপরচুনটি থাকা অবস্থায় ব্যাকএন্ড প্রত্যাখ্যান করবে, তাই কল সফল হলেই সারি বিদ দাখ হয়।

clientsProvider

AsyncNotifierProvider<ClientsController, PagedState<Client>> — উপরে কন্ট্রোলার এক্সপোজ করে।

টপ-লেভেলে প্রোভাইডার

clientDetailProvider

FutureProvider.autoDispose.family<Client, int> — নির্দিষ্ট id-এর ক্লায়েন্ট বিস্তারিত ফেচ করে।

activeClientsProvider

FutureProvider.autoDispose<List<Client>> — `crmRepositoryProvider().activeClients()` কল করে শুধু এখনো ব্যবসা চলছে এমন ক্লায়েন্টদের একটি খালি (নন-পজেড) তালিকা রিটার্ন করে, যা পক্ষির উইজটে ব্যবহৃত হয়। `clientsProvider`-এর থেকে আলাদা কারণ সটে বিন্দু হয়ে যাওয়া ক্লায়েন্টদেরও অন্তর্ভুক্ত করে।

ফাইল: `lib/features/crm/crm_models.dart`

সংক্ষিপ্ত বিবরণ: CRM মডেলের সব ডটো মডেলে, রকিওয়েস্ট/রসেপন্স DTO, এবং ধ্রুবক (permission code, stage, status, activity type ইত্যাদি) সংজ্ঞায়িত করে। এখানে কোনো UI লজিক নেই — শুধু JSON সিরিয়লাইজেশন ও ডোমহেন হল্‌পার গটোর।

ক্লাস: `CrmPermissions`

abstract final class — CRM-সংশ্লিষ্ট সব পারমিশন কোড ধ্রুবক হিসেবে সংজ্ঞায়িত (`leadView`, `leadCreate`, `leadUpdate`, `leadDelete`, `opportunityView`, `opportunityCreate`, `opportunityUpdate`, `opportunityDelete`, `clientView`, `clientCreate`, `clientUpdate`, `clientDelete`, `contactView`, `contactCreate`, `contactUpdate`, `contactDelete`, `tagView`, `tagManage`)। কোনো মথড নেই — শুধু স্ট্যাটিক স্ট্রিং কনস্ট্যান্ট।

ক্লাস: `Stage`

abstract final class — একটি ডিলি কোন স্টেজে আছে তার ধ্রুবক (`qualification`, `presentation`, `proposal`, `negotiation`, `won`, `lost`), এবং `open` (খোলা স্টেজের তালিকা, ক্রম গুরুত্বপূর্ণ — পাইপলাইনের ক্রম), `all` (সব স্টেজ)।

static bool `isOpen(String stage)`

প্রদত্ত স্টেজে `open` তালিকায় আছে কিনা রিটার্ন করে।

static bool `isClosed(String stage)`

প্রদত্ত স্টেজে `won` বা `lost`-এর সমান কিনা রিটার্ন করে।

ক্লাস: `LeadStatus`

abstract final class — লডি স্ট্যাটাসের ধ্রুবক (`isNew`, `contacted`, `qualified`, `disqualified`) ও `all` তালিকা। কোনো মথড নেই।

টপ-লেভেলে কনস্ট্যান্ট

`leadSources`, `leadPriorities`, `clientStatuses`, `crmActivityTypes`

যথাক্রমে লডি সোর্স, লডি অগ্রাধিকার, ক্লায়েন্ট স্ট্যাটাস (`BLOCKED` একটি প্রকৃত অবস্থা, সফট-ডলিটি নয়), ও CRM অ্যাক্টিভিটি টাইপের স্থির স্ট্রিং তালিকা।

`lostReasons`

(code, label) রকেডের একটি তালিকা, ডলি হারানোর কারণ বোঝাতে। ব্যাকএন্ড LOST ট্রানজিশনে কোড আবশ্যিক করে, এবং কোড OTHER হল্‌ ফ্রি-টেক্সট বিবরণও চায়।

ক্লাস: Tag

একটি ট্যাগ (id, name, color)।

argb (getter)

color-কে হেক্স হিসেবে পার্স করে একটি ARGB int রিটার্ন করে। ৩-অক্ষরে হেক্স হলে প্রতটি অক্ষর দ্বিগুণ করে ৬-অক্ষরে রূপান্তর করে; পার্স ব্যর্থ হলে বা দৈর্ঘ্য ভুল হলে null রিটার্ন করে (যাতে ক্যালার নরিপক্ষে চপি দেখাতে পারে, ক্র্যাশ না করে)।

factory Tag.fromJson(Map<String, dynamic> json)

JSON থেকে id, name, color পড়ে একটি Tag তৈরি করে, প্রতটি ফিল্ডের জন্য নরিপদ ডিফল্ট মান রাখে।

static List<Tag> listFrom(Object? raw)

raw একটি List হলে প্রতটি Map<String, dynamic> এন্ট্রিকে Tag.fromJson দিয়ে ম্যাপ করে তালিকা রিটার্ন করে, নাহলে খালি তালিকা রিটার্ন করে।

ক্লাস: DuplicateMatch

একটি ক্ল্যায়েন্ট যার সাথে ব্যাকএন্ড মনে করে একটি ডিলি ইতিমধ্যে সম্পর্কিত হতে পারে (clientId, clientCompanyName, matchedOn)।

static DuplicateMatch? tryFrom(Object? json)

json একটি Map না হলে বা clientId না থাকলে null রিটার্ন করে; নাহলে একটি DuplicateMatch তৈরি করে রিটার্ন করে।

ক্লাস: Lead

একটি লিড এন্টটির সম্পূর্ণ ডটো মডেলে — অনেকেগুলো ফিল্ড (id, contactName, status, source, createdAt, converted, companyName, email, phone, industry, jobTitle, notes, priority, estimatedValue, expectedCloseDate, assignedToName, lastContactDate, lastActivityAt, convertedClientName, aiSummary, activitiesCount, tags, possibleDuplicate)।

canConvert (getter)

শুধুমাত্র যোগ্যতাপ্রাপ্ত (qualified) ও এখনো রূপান্তরিত না হওয়া লিডের ক্ষেত্রে true রিটার্ন করে — এই দুটি শর্ত ব্যাকএন্ডেও প্রয়োগ করা আছে, তাই UI-তে এই শর্ত না মিললে অ্যাকশন লুকানো হয়।

isUnassigned (getter)

assignedToName == null কনি রিটার্ন করে।

neverContacted (getter)

lastContactDate ও lastActivityAt দুটোই null কনি রিটার্ন করে — কারো সাথে এখনো যোগাযোগ রকের্ড হয়নি বোঝায়।

headline (getter)

companyName থাকলে ও খালি না হলে তা রিটার্ন করে, নাহলে contactName রিটার্ন করে — তালিকায় লিডকে কী নামে ডাকা হবে তা নির্ধারণ করে (কোম্পানি থাকলে রপে অ্যাকাউন্ট হিসেবে চিন্তা করে)।

subline (getter)

companyName থাকলে contactName রিটার্ন করে, নাহলে jobTitle বা email রিটার্ন করে।

factory Lead.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে একটি Lead তৈরি করে; notes ফিল্ড notes কী না থাকলে description কী থেকে ফলব্ধাক নেয়; tags ও possibleDuplicate যথাক্রমে Tag.listFrom/DuplicateMatch.tryFrom দিয়ে পার্স হয়।

ক্লাস: Opportunity

একটি ডলি/অপরচুনটির মডলে (id, name, stage, probability, createdAt, description, source, amount, weightedAmount, expectedCloseDate, actualCloseDate, nextStep, lostReason, lostReasonCode, clientId, clientCompanyName, contactName, ownerName, lastActivityAt, stageChangedAt, sourceLeadId, tags)।

isOpen (getter)

Stage.isOpen(stage) রিটার্ন করে।

isWon (getter)

stage == Stage.won কনি রিটার্ন করে।

isLost (getter)

stage == Stage.lost কনি রিটার্ন করে।

needsClientDecisionOnWin (getter)

clientId == null কনি রিটার্ন করে — একটি ক্লায়েন্ট-বহীন ডলি জতো হলে ডুপ্লিকিটে-চকে ট্রিগার হয় কারণ ব্যাকএন্ডকে সিদ্ধান্ত নতি হয় নতুন ক্লায়েন্ট তৈরি করবে না বদ্যমানটির সাথে লঙ্ক করবে।

isOverdue (getter)

খোলা নয় বা expectedCloseDate না থাকলে false রিটার্ন করে। নাহলে তারখিটি পার্স করে (পার্স ব্যর্থ হলে false) এবং সটে আজকরে তারখিরে আগে কনি তুলনা করে (শুধু বছর/মাস/দনি, সময় বাদে) রিটার্ন করে।

factory Opportunity.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে একটি Opportunity তৈরি করে, প্রতটি নাল-সফে ডফিল্ট সহ।

ক্লাস: PipelineStageSummary

একটি স্টেজেরে সারাংশ পরসিংখ্যান (stage, dealCount, totalAmount, weightedAmount)।

factory PipelineStageSummary.fromJson(Map<String, dynamic> json)

JSON থেকে চারটি ফিল্ড পার্স করে একটি ইনস্ট্যান্স তৈরি করে।

ক্লাস: PipelineSummary

পুরো পাইপলাইনের হডেলাইন পরসিংখ্যান (stages, openPipelineValue, weightedForecast, wonValue, totalOpenDeals)।

PipelineStageSummary? forStage(String stage)

stages-এর মধ্যে প্রদত্ত stage-এর সাথে মলি যাওয়া সারাংশ খুঁজে রিটার্ন করে, না পলে null।

factory PipelineSummary.fromJson(Map<String, dynamic> json)

JSON থেকে stages তালকি (প্রতটি এন্ট্রি PipelineStageSummary.fromJson দিয়ে পার্স) ও চারটি সংখ্যাসূচক ফিল্ড পার্স করে একটি ইনস্ট্যান্স তৈরি করে।

ক্লাস: CrmActivity

একটি অ্যাক্টিভিটি লিগ এন্ট্রি (id, type, subject, activityDate, completed, systemGenerated, description, performedByName, createdAt)। systemGenerated বোঝায় এটি ব্যাকএন্ড লিখেছে (যেমন স্টজে পরিবর্তন), মানুষ নয়।

factory CrmActivity.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে একটি CrmActivity তৈরি করে; activityDate না থাকলে createdAt কী থেকে ফলব্যাক নেয়।

ক্লাস: Client

একজন ক্লায়েন্টের সম্পূর্ণ মডেল (id, status, createdAt, firstName, lastName, email, phone, clientCompanyName, industry, website, portalAccessEnabled, accountManagerName, onboardedAt, employeeCount, annualRevenue, lifetimeValue, totalRequests, tags)।

contactName (getter)

firstName ও lastName (খালনিয় এমন অংশগুলো) যুক্ত করে ট্রিম করে একটি স্ট্রিং রিটার্ন করে।

headline (getter)

কোম্পানির নাম থাকলে ও খালনি হলে তা রিটার্ন করে; নাহলে contactName (খালনি হলে), নাহলে email, নাহলে "Client #id" রিটার্ন করে।

subline (getter)

কোম্পানি থাকলে contactName (খালনি হলে) বা email রিটার্ন করে; কোম্পানি না থাকলে সরাসরি email রিটার্ন করে।

initials (getter)

headline-কে হোয়াইটস্পেস দিয়ে ভেঙে অংশগুলো নেয়; একটি অংশ থাকলে তার প্রথম অক্ষর, একাধিক থাকলে প্রথম ও শেষ অংশের প্রথম অক্ষর যুক্ত করে আপারকসে রিটার্ন করে; কোনো অংশ না থাকলে "?" রিটার্ন করে।

factory Client.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে একটি Client তৈরি করে; tags tagList কী থেকে Tag.listFrom দিয়ে পার্স হয়।

ক্লাস: ConvertLeadRequest

PATCH /crm/leads/{id}/convert-to-opportunity-এর বডি (opportunityName, expectedValue, expectedCloseDate — yyyy-MM-dd ফরম্যাট)।

Map<String, dynamic> toJson()

তনিটি ফিল্ড (নাম ট্রিম করে) সহ একটি JSON ম্যাপ রিটার্ন করে।

ক্লাস: ChangeStageRequest

PATCH /crm/opportunities/{id}/stage-এর বডি (stage, lostReasonCode, lostReason, linkToExistingClientId, forceCreateNewClient)।

Map<String, dynamic> toJson()

stage সবসময় পাঠায়; lostReasonCode নাল-অ্যাওয়ার্ড স্প্রডে অপারটের দ্বি়ে ঐচ্ছকিভাবে; lostReason খালি না হলে; এবং linkToExistingClientId/forceCreateNewClient নন-নাল হলে যুক্ত করে একটি ম্যাপ রটাইরন করে।

ক্লাস: CreateLeadRequest

POST /crm/leads-এর বডি (contactName, source, companyName, email, phone, jobTitle, priority, estimatedValue, notes)।

Map<String, dynamic> toJson()

একটি অভ্যন্তরীণ clean হলেপার দ্বি়ে প্রতটি স্ট্রিং ফল্ড ট্রমি করে খালি হলে বাদ দয়ে; contactName/source সবসময় পাঠায়, বাকি ঐচ্ছকি ফল্ড শুধু মান থাকলে যুক্ত করে JSON ম্যাপ রটাইরন করে।

ক্লাস: LogActivityRequest

POST /crm/leads/{id}/activities-এর বডি (type, subject, description)।

Map<String, dynamic> toJson()

type, ট্রমি করা subject, খালি না হলে description, এবং সবসময় 'completed': true কারণ এটি ইতিমধ্যে ঘটতে যাওয়া কিছু লগ করে, সহ একটি ম্যাপ রটাইরন করে।

ক্লাস: UpdateLeadRequest

PATCH /crm/leads/{id}-এর বডি (contactName, companyName, email, phone, jobTitle, industry, status, source, priority, estimatedValue, notes)। এন্ডপয়েন্টটি create-এর মতোই DTO ব্যবহার করে @Valid দ্বি়ে ভ্যালিডেট করে, তাই contactName সবসময় পাঠাতে হয়; বাকি ফল্ড একটি প্রকৃত sparse patch।

Map<String, dynamic> toJson()

clean হলেপার দ্বি়ে স্ট্রিং ফল্ডগুলো ট্রমি করে, contactName সবসময়, বাকি ফল্ড শুধু মান থাকলে (নাল-অ্যাওয়ার্ড স্প্রডে সহ কিছু) যুক্ত করে JSON ম্যাপ রটাইরন করে।

ক্লাস: CreateClientRequest

POST /clients-এর বডি (firstName, lastName, email, phone, clientCompanyName, industry, website, employeeCount)। provisionPortalLogin সবসময় স্পষ্টভাবে false পাঠানো হয় কারণ সেটি চালু করলে ক্লায়েন্টকে সে-পাসওয়ার্ড লগিক ইমেইল করে — এটি নীরবে ইনহেরিট করার মতো ডিফল্ট নয়।

Map<String, dynamic> toJson()

firstName, lastName, email ট্রমি করে ও 'provisionPortalLogin': false সবসময় পাঠায়; বাকি ঐচ্ছকি ফল্ড ট্রমি করে খালি না হলে যুক্ত করে JSON ম্যাপ রটাইরন করে।

ক্লাস: UpdateClientRequest

PATCH /clients/{id}-এর বডি (clientCompanyName, industry, website, status, employeeCount) — একটি প্রকৃত sparse patch; নাম ও ইমেইল ইচ্ছাকৃতভাবে বাদ দেওয়া হয়েছে কারণ সেগুলো ক্লায়েন্টের ইউজার অ্যাকাউন্টে থাকে, এই এন্ডপয়েন্ট সেগুলো বদলাতে পারে না।

factory UpdateClientRequest.from(Client client)

প্রদত্ত client-এর বর্তমান মান থেকে একটি UpdateClientRequest তৈরি করে রটাইরন করে — এটি ফর্ম প্র-ফিলির জন্য উপযোগী।

Map<String, dynamic> toJson()

clean হলেপার দ্বিগুণে ট্রমি করে খালি স্ট্রিং বাদ দ্বিগুণে, শুধু নন-নাল ফিল্ডগুলো (কোম্পানি, ইন্ডাস্ট্রি, ওয়েবসাইট, স্ট্যাটাস, কর্মচারী) সম্ভলতি JSON ম্যাপ রটরিন করে।

ক্লাস: OpportunityRequest

POST /crm/opportunities ও PATCH /crm/opportunities/{id} উভয়রে জন্ব একই DTO (name, description, nextStep, clientId, amount, expectedCloseDate, source)। description ও nextStep সবসময় পাঠাতে হয় (এমনকি খালিহলও), কারণ ব্যাকএন্ডরে update মথেড এই দুটো নশির্তভাবে অ্যাসাইন করে — বাদ দলিগে সেগুলো মুছে যাবে। stage ইচ্ছাকৃতভাবে অনুপস্থতি কারণ স্টজে পরবির্তন একটি আলাদা এন্ডপয়ন্টে দ্বিগুণে হয়।

factory OpportunityRequest.from(Opportunity opportunity)

বদ্বিমান opportunity-এর মান থেকে একটি OpportunityRequest তরৈকিরে রটরিন করে — এডটি ফর্ম প্র-ফিলিরে জন্ব।

Map<String, dynamic> toJson()

name ট্রমি করে সবসময় পাঠায়; description/nextStep ইচ্ছাকৃতভাবে নশির্তভাবে কলনি করে হলও, null হলও পাঠায়; বাকি ফিল্ড (clientId, amount, expectedCloseDate, source) নন-নাল হলে যুক্ত করে JSON ম্যাপ রটরিন করে।

ক্লাস: ClientContact

একজন ক্লায়ন্টে কোম্পানির নরিদ্বিষ্ট ব্যক্তরি মডলে (id, fullName, primaryContact, clientId, clientCompanyName, email, phone, jobTitle, department, notes)। একটি ক্লায়ন্টে প্রতস্থিঠান, আর কন্ট্যাক্ট সেখনকার মানুষ; একজনই সবসময় প্রাইমারি থাকে।

role (getter)

jobTitle ও departmentকে (নাল-সফে স্প্রডে দ্বিগুণে) " . " দ্বিগুণে জোড়া লাগয়িগে একটি স্ট্রিং রটরিন করে — সাররি সাবটাইটলেরে জন্ব।

factory ClientContact.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে একটি ClientContact তরৈকিরে।

ক্লাস: ClientContactRequest

POST/PATCH /clients/{clientId}/contacts-এর বডটি (fullName, email, phone, jobTitle, department, notes, primaryContact)। আপডটে এন্ডপয়ন্টে ইমেইল/ফোন/পদব/বভিাগ/নোট নশির্তভাবে অ্যাসাইন করে (শুধু নাম নাল-গার্ডডে), তাই এই সব সবসময় পাঠানো হয়। primaryContact এক-মুখী — true সটে করলে আগরে প্রাইমারিসিরে যায়, false সটে করার কোনো অর্থ নাই।

factory ClientContactRequest.from(ClientContact contact)

বদ্বিমান contact-এর মান থেকে একটি রিকি়োয়স্ট তরৈকিরে (primaryContact বাদে) — এডটি ফর্ম প্র-ফিলিরে জন্ব।

Map<String, dynamic> toJson()

fullName ট্রমি করে সবসময় পাঠায়; email, phone, jobTitle, department, notes — এগুলো clean দ্বিগুণে ট্রমি করে, null হলও স্পষ্টভাবে পাঠায় (কারণ ব্যাকএন্ড এভাবেই পডে); primaryContact শুধু true হলে যুক্ত করে JSON ম্যাপ রটরিন করে।

ক্লাস: TagRequest

POST/PATCH /crm/tags-এর বডি (name, color — উভয়ই আবশ্যিক)।

Map<String, dynamic> toJson()

ট্রিম করা name ও color সহ একটি ম্যাপ রিটার্ন করে।

ক্লাস: CrmActivityRequest

POST /crm/activities-এর বডি (type, subject, description, scheduledAt, completed, clientId, opportunityId)। LogActivityRequest-এর চেয়ে পূর্ণাঙ্গ — এটি একটি ফিল্টার-আপও শিডিউল করতে পারে এবং ক্লায়েন্ট বা অপরচুনটির সাথে সংযুক্ত হতে পারে।

Map<String, dynamic> toJson()

description ট্রিম করে খালি না হলে যুক্ত করে, বাকি নন-নাল ফিল্ড (scheduledAt, completed, clientId, opportunityId) সহ, এবং type/ট্রিম করা subject সবসময় সহ JSON ম্যাপ রিটার্ন করে।

ক্লাস: LeadFilter

লিড তালিকা সংকীর্ণ করার জন্য সব ফিল্টার (keyword, status, source, priority, assignedToId, tagId, expectedCloseDateFrom, expectedCloseDateTo, hasActivity, isConverted, isUnassigned, isHighPriority, sortBy ডিফল্ট 'createdAt', sortDirection ডিফল্ট 'DESC')। GET-এর বদলে POST ব্যবহৃত হয় কারণ চৌদ্দটি ফিল্ড থাকায় কুয়েরি স্ট্রিং অপাঠ্য হয়ে যেতে।

sortOptions (static const)

বৈধ sort-by মানের তালিকা (createdAt, lastActivityAt, expectedCloseDate, priority)।

isEmpty (getter)

সব ফিল্টার ফিল্ড ডিফল্ট/নাল/খালি কিনা যাচাই করে true/false রিটার্ন করে — ফিল্টার কয়টি সংকীর্ণ করেছে কিনা তা নির্ধারণ করতে ব্যবহৃত হয়, যাতে তালিকা ফিল্টার এন্ডপয়েন্ট নাকি প্লাইন ভিডিও এন্ডপয়েন্ট ব্যবহার করবে তা ঠিক করা যায়।

LeadFilter copyWith({...})

প্রদত্ত প্যারামিটার দিয়ে নির্বাচিত ফিল্ড ওভাররাইড করে একটি নতুন LeadFilter তৈরি করে রিটার্ন করে; clearStatus/clearSource/clearPriority ফ্ল্যাগ দিয়ে সংশ্লিষ্ট ফিল্ড স্পষ্টভাবে null করা যায়।

Map<String, dynamic> toJson()

keyword ট্রিম করে খালি না হলে যুক্ত করে; বাকি ঐচ্ছিক ফিল্ড নন-নাল হলে যুক্ত করে; sortBy/sortDirection সবসময় পাঠিয়ে JSON ম্যাপ রিটার্ন করে।

ক্লাস: LeadImportResult

CSV থেকে লিড ইমপোর্টের ফলাফল (created, skipped — প্রতিটি এন্ট্রির কোন সারিও কনে বাদ পড়ছে তা বলে)।

factory LeadImportResult.fromJson(Map<String, dynamic> json)

JSON থেকে created সংখ্যা ও skipped স্ট্রিং তালিকা পার্স করে একটি LeadImportResult তৈরি করে।

done

ফাইল: lib/features/crm/crm_reports_screen.dart

সংক্ষিপ্ত ববিরণ: CRM-এর রপিরোর্ট স্ক্রিনি — পাইপলাইন সামারি, উইন/লস ট্রেন্ড, লডি কনভার্সন, রভেনিডি, ফোরকাস্ট, লস-রজিন এবং সলেস পারফরম্যান্স একসাথে দেখায়। Angular ভার্সনরে মতৌ ব্যাকএন্ডে আলাদা রপিরোর্টিং এন্ডপয়ন্টে নই বলে ক্লায়ন্ট-সাইডইে সব হিসাব করা হয়।

টপ-লভেলে টাইপডেফ ও প্রোভাইডার

`typedef _ReportData`

রকোর্ড টাইপ — `summary` (PipelineSummary), `won/lost/all` (Opportunity লিস্ট) এবং `leads` (Lead লিস্ট) একসাথে ধারণ করে। রপিরোর্ট স্ক্রিনিরে একবাররে ফচে যা লাগে তার সবটাই এখানে বান্ডলে করা।

`_crmReportsProvider`

`FutureProvider.autoDispose<_ReportData>`। পাঁচটি রপিরোর্জটিরকিল (`pipelineSummary`, `opportunities(stage: won)`, `opportunities(stage: lost)`, `opportunities()` সব, `leads(all)`) একসাথে `Future.wait` দিয়ে চালায় এবং ফলাফলগুলো `_ReportData` রকোর্ডে সাজিয়ে ফরেত দিয়ে। `won/lost/all`-এর সাইজ ৫০০-১০০০ পর্যন্ত টানা হয় যাতো ক্লায়ন্ট-সাইড অ্যাগ্রগিশেন প্রায় সম্পূর্ণ ডটোসটেরে ওপর হয়।

ক্লাস/উইজটে: CrmReportsScreen

`ConsumerWidget`। `_crmReportsProvider` ওয়াচ করে লোডিং/এরর/ডটো তনি অবস্থা হ্যান্ডল করে, `RefreshIndicator` দিয়ে পুল-টু-রফিরশে সমর্থন করে এবং ডটো এলে `_Report` উইজটে পােস করে।

`Widget build(BuildContext context, WidgetRef ref)`

`_crmReportsProvider` থেকে `async` রজোল্ট নিয়ে `AppBar` টাইটলে "CRM Reports" সহ একটি `Scaffold` বানায়। `async.when` দিয়ে `loading` অবস্থায় `Loader`, `error` অবস্থায় `ErrorState` (`ApiException` হলে বার্তা, নয়তো জনেরেকি বার্তা, রট্রাই বাটনসহ), এবং `data` অবস্থায় `_Report(data: data)` দেখায়। `onRefresh` কলব্যাক প্রোভাইডার ইনভ্যালডিটে করে পুনরায় ফচে করে।

ক্লাস/উইজটে: _Report

`StatelessWidget`, `_ReportData` নিয়ে। বশে কিছু গটোর দিয়ে ডটো থেকে অ্যাগ্রগিটে হিসাব (মোট জতো/হারা টাকা, জয়রে হার, মাসকি বাকটে, ফোরকাস্ট, লস-রজিন, সলেস পারফরম্যান্স) বরে করে এবং `build`-এ সগেলো বার-চার্ট আকৃতির উইজটে সাজিয়ে দেখায়।

`double get _wonAmount`

`data.won`-এর সব অপরচুনটির `amount` (null হলে ০) যোগ করে মোট জতো টাকার পরমাণ দিয়ে।

`double get _lostAmount`

একইভাবে `data.lost`-এর সব `amount` যোগ করে মোট হারা টাকার পরমাণ দিয়ে।

`int get _winRate`

জতো ও হারা ডলিরে মোট সংখ্যার অনুপাতে জতোর শতাংশ রাউন্ড করে দিয়ে; মোট শূন্য হলে ০ ফরেত দিয়ে।

`List<DateTime> get _months`

বর্তমান মাস থেকে পছনের দকিে গুনে গত ৬ ক্যালেন্ডার মাসরে `DateTime` লিস্ট দিয়ে (পুরনোটা আগো) — ট্রেন্ড ও রভেনিডি সেকেশন এই উইন্ডোতই বাকটে করে।

`bool _sameMonth(String? isoDate, DateTime month)`

একটি ISO তারিখ স্ট্রিং পার্স করে দেওয়া `month`-এর একই বছর/মাসে পড়ে কনি যাচাই করে। `null` বা পার্স-অযোগ্য হলে `false` দেয়।

`List<...> get _forecast`

ওপনে অপরচুনটিগুলিকে প্রত্যাশিত কলোজ-মাস অনুযায়ী বাকটে করে — প্রতি বাকটে লবেলে, ডলি সংখ্যা, কাঁচা টাকার যোগফল এবং প্রোবাবলিটি-ওয়েটেডে যোগফল রাখা। কলোজ-ডটে না-থাকা ডলিগুলো "No close date" নামে আলাদা বাকটে রাখা হয়, বাদ দেওয়া হয় না। শেষে ক্রমানুসারে (`order`) সাজানো লিস্ট রটার্ন করে।

`List<...> get _lossReasons`

হারা ডলিগুলোকে `lostReasonCode` অনুযায়ী গ্রুপ করে — কোড না-থাকা ডলিগুলোকে "Not recorded" লবেলে দেওয়া হয় যাতে রিপোর্টেরে কভারজে গোপন না থাকে। প্রতি গ্রুপে সংখ্যা, মোট মূল্য ও শতাংশ হিসাব করে সংখ্যা অনুযায়ী বড় থেকে ছোট সাজিয়ে দেয়। `data.lost` খালি হলে খালি লিস্ট দেয়।

`List<...> get _salesPerformance`

`data.all`-এর প্রতিটি অপরচুনটিকে মালিক (`ownerName`, `null` হলে "unassigned") অনুযায়ী গ্রুপ করে মোট, জতো, হারা সংখ্যা এবং জতো মূল্যের যোগফল বের করে, এরপর জয়ের হার হিসাব করে জতো-মূল্য অনুযায়ী নম্বিনক্রমে সাজায়।

`int get _convertedLeads`

`data.leads`-এর মধ্যে `converted == true` এমন লডি়েরে সংখ্যা গোনো।

`int get _leadConversionRate`

কনভার্টেড লডি়েরে অনুপাতে শতাংশ রাউন্ড করে দেয়; লডি লিস্ট খালি হলে ০।

`Widget build(BuildContext context)`

মাস-ভিত্তিক ট্রেন্ড (`trendByMonth`), রভেনিউ (`revenueByMonth`) ও তাদের ম্যাক্সিমাম মান হিসাব করে, তারপর `ListView`-এ ক্রমান্বয়ে: স্ট্যাট কার্ড রো (Won/Lost/Win rate), স্টজে-ভিত্তিক পাইপলাইন ভ্যালু বার, ৬-মাসের উইন/লস ট্রেন্ড, লডি কনভার্সন সেকশন, মাসিক রভেনিউ, ফোরকাস্ট টবেলি (টোটালসহ), হারার কারণসমূহ ও সেলস পারফরম্যান্স কার্ডেরে তালিকা রন্ডার করে।

ক্লাস/উইজটে: `_MiniStat`

`StatelessWidget`। একটি মান, লবেলে ও ঐচ্ছকি রঙ নিয়ে ছোট উল্লম্ব স্ট্যাট (সংখ্যা উপরে, লবেলে নচি) দেখায়।

`Widget build(BuildContext context)`

`value` বড় বোল্ড টেক্সট (রঙ `color` বা ডফিল্ট `bos.text`) এবং নচি `label` ছোট মডিটেড টেক্সট হিসেবে `Column`-এ সাজায়।

ক্লাস/উইজটে: `_BarRow`

`StatelessWidget`। লবেলে, ভ্যালু-টেক্সট, একটি ভিগ্নাংশ (`fraction`) ও রঙ নিয়ে একটি লবেলে+মান রো এবং তার নচি একটি অনুপাতিকি প্রগ্রসে বার আঁকে — এই কোডবসেরে একমাত্র হাতে-বানানো চার্ট প্রমিটিভি, অ্যাকাউন্টস-পয়েবেল এজিং বাকটেও একই ধরন ব্যবহৃত হয়।

Widget build(BuildContext context)

ওপরে Row-এ লবেলে (ইলিপসিসিসহ) ও ভ্যালু-টেক্সট দেখায়, নচি ClipRect-এ মোড়ানো LinearProgressIndicator দিয়ে fraction (০-১ ক্ল্যাম্প করা) অনুপাতে বার আঁকে। dense true হলে ফন্ট ও বার-উচ্চতা ছোট হয়।

ফাইল: lib/features/crm/crm_repository.dart

সংক্ষিপ্ত ববিরণ: CRM মডউলরে সব API কল একত্রে রাখা রপিয়োজটির ক্লাস — লডি, অপরচুনটিও ক্লায়েন্ট নিয়ে কোর মথেড এবং কন্ট্রাক্ট/ট্যাগ/অ্যাক্টিভিটি নিয়ে এক্সটেনশন মথেড।

ApiClient-এর ওপর একটি পাতলা স্তর হিসেবে কাজ করে।

টপ-লেভেলে enum

```
enum LeadView { mine, all, highPriority, neverContacted, stale, unassigned }
```

লডিরে কোন স্টে দেখানো হবে তা নির্ধারণ করে — প্রতটি সার্ভার-সাইড এন্ডপয়েন্ট (/my, /high-priority, ইত্যাদি), ক্লায়েন্ট-সাইড ফিল্টার নয়, যাতে ওয়েব অ্যাপের সংজ্ঞার সাথে মিল থাকে।

ক্লাস: CrmRepository

ApiClient-কে রূপান্তর করে লডি, অপরচুনটিও ক্লায়েন্ট নিয়ে সব HTTP কল প্রকাশ করে। কনস্ট্রাক্টরে _api ইনস্ট্যান্স নেয়; _leads, _opportunities, _clients স্ট্যাটিক প্যাথ কনস্ট্যান্ট ব্যবহার করে।

```
Future<PagedResponse<Lead>> leads(LeadView view, {int page = 0, int size = 20})
```

view অনুযায়ী switch দিয়ে সঠিক এন্ডপয়েন্ট প্যাথ বছে নিয়ে (/crm/leads, /crm/leads/my ইত্যাদি) পজেড লডি লিস্ট আনে।

```
Future<Lead> lead(int id)
```

একক লডি আইডি দিয়ে GET /crm/leads/\$id কল করে Lead অবজেক্টে পার্স করে ফেরত দেয়।

```
Future<Lead> createLead(CreateLeadRequest request)
```

নতুন লডি তৈরি জন্য POST /crm/leads কল করে তৈরি হওয়া Lead ফেরত দেয়।

```
Future<Lead> updateLead(int id, UpdateLeadRequest request)
```

PATCH /crm/leads/\$id দিয়ে বদ্যমান লডি আপডেট করে আপডেটেড Lead ফেরত দেয়।

```
Future<void> deleteLead(int id)
```

DELETE /crm/leads/\$id কল করে সফট-ডেলিট করে; সার্ভার ২০৪ (বডি-বাহীন) রসপন্স দেয়।

```
Future<Opportunity> convertLead(int id, ConvertLeadRequest request)
```

PATCH /crm/leads/\$id/convert-to-opportunity কল করে যোগ্যতাপ্রাপ্ত লডিকে অপরচুনটিতে রূপান্তর করে। এখানে কোনো ক্লায়েন্ট তৈরি হয় না — ডলি জটিলে পরে তৈরি হয়।

```
Future<List<CrmActivity>> leadActivities(int id)
```

/crm/leads/\$id/activities থেকে পজেড অ্যাক্টিভিটি (সাইজ ৫০) আনে, তারপর activityDate অনুযায়ী নতুন-থেকে-পুরনো সাজিয়ে লিস্ট ফেরত দেয়।

```
Future<CrmActivity> logActivity(int id, LogActivityRequest request)
```

POST /crm/leads/\$id/activities কল করে একটি লডি-অ্যাক্টিভিটি লগ করে।

Future<PagedResponse<Lead>> filterLeads(LeadFilter filter, {int page = 0, int size = 20})

POST /crm/leads/filter?page=&size= — চৌদ্দটি ফিল্টার ক্রাইটেরিয়া থাকায় বড় হিসেবে পাঠানো হয় (GET-এর বদলে POST-দিয়ে-রডি প্যাটার্ন), পজেড লডি রসেপন্স পার্স করে ফেরত দেয়।

Future<int> activeLeadCount()

/crm/leads/stats/active থেকে একটি বিয়ের সংখ্যা (num) পড়ে int-এ কনভার্ট করে দেয়; null হলে ০।

Future<int> myActiveLeadCount()

একই রকম, তবে /crm/leads/stats/my-active — বর্তমান ইউজারের নজিরে সক্রিয় লডি সংখ্যা।

Future<Lead> summariseLead(int id)

GET /crm/leads/\$id/summary — এই রডি-কলই AI-জেনারেটেডে সামারি তৈরি করে (কেছু স্টোর হয় না), আপডেটেডে Lead-এ সামারিসি ফেরত দেয়।

Future<void> deleteLeadActivity(int leadId, int activityId)

DELETE /crm/leads/\$leadId/activities/\$activityId কল করে লডিরে টাইমলাইন থেকে একটি এন্ট্রিসিরায়।

Future<LeadImportResult> importLeads(String filePath, String fileName)

POST /crm/leads/import (মাল্টিপার্ট ফাইল আপলোড) করে CSV থেকে লডি তৈরি করার চেষ্টা করে, ব্যবহারযোগ্য না-এমন রো-গুলোর কারণসহ ফলাফল ফেরত দেয়।

Future<List<int>> leadsPdf({String? status})

লডি লিস্টের PDF বাইট আন; status দিলে সেই স্ট্যাটাসে সীমাবদ্ধ করে, নয়তো সব লডি।

Future<Opportunity> opportunityFromLead(int leadId, OpportunityRequest request)

POST /crm/opportunities/from-lead/\$leadId — যে লডিরে ইতিমধ্যে একটি অপারচুনটি আছে তাকে আরেকটি নতুন অপারচুনটি যোগ করে (convertLead-এর থেকে ভিন্ন — লডিকে "converted" মার্ক করে না, একবারের বদলে বারবার ব্যবহারযোগ্য)।

Future<PagedResponse<Opportunity>> opportunities({String? stage, int page = 0, int size = 20})

পাইপলাইন লিস্ট আন। stage না-দিলে openOnly: true কয়েকটি প্রায়ামটির যোগ হয় যাত শূণ্য ওপনে ডলি আসে — এটা ছাড়া কলোজড ডলিও ফরিত আসত এবং লাইভ ডলিগুলো ইতিহাসের পাতার নচি চাপা পড়ে যেত।

Future<Opportunity> opportunity(int id)

GET /crm/opportunities/\$id — একক অপারচুনটি আন।

Future<Opportunity> createOpportunity(OpportunityRequest request)

POST /crm/opportunities — সরাসরি (লডি-কনভার্সন ছাড়া) নতুন ডলি খোল; ব্যাকএন্ড এই পাথে clientId বাধ্যতামূলক করে।

Future<Opportunity> updateOpportunity(int id, OpportunityRequest request)

PATCH /crm/opportunities/\$id — ডলিরে ববিরণ এডটি করে (স্টেজে নয়, তার জন্য changeStage)। WON/LOST ডলিরে জন্য ব্যাকএন্ড এটি প্রত্যাখ্যান করে বলে UI শূণ্য ওপনে ডলি এই অপশন দেয়।

Future<void> deleteOpportunity(int id)

DELETE /crm/opportunities/\$id — সফট-ডলিটি, ২০৪ বডি-বহীন।

Future<PipelineSummary> pipelineSummary()

GET /crm/opportunities/pipeline-summary — স্টজে-ভিত্তিক পাইপলাইন সামারি আনবে।

Future<Opportunity> changeStage(int id, ChangeStageRequest request)

PATCH /crm/opportunities/\$id/stage — একটি ডিলির স্টজে পরিবর্তন করে।

Future<DuplicateMatch?> wonDuplicateCheck(int id)

GET /crm/opportunities/\$id/won-duplicate-check কল করে দেখে ডলি জটিলে স্টো কোনও বদ্যমান ক্লায়েন্টের সাথে ডুপ্লিকেট কনি। try/catch-এ ApiException ধরে null ফেরত দেয় (fails open) — চকে ব্যর্থ হলেও রপে যনে ডলি বন্ধ করা থেকে আটকে না যায়।

Future<PagedResponse<Client>> clients({int page = 0, int size = 20})

GET /clients — পজেড ক্লায়েন্ট লস্ট আনবে।

Future<Client> client(int id)

GET /clients/\$id — একক ক্লায়েন্ট আনবে।

Future<Client> createClient(CreateClientRequest request)

POST /clients — নতুন ক্লায়েন্ট তৈরি করে।

Future<Client> updateClient(int id, UpdateClientRequest request)

PATCH /clients/\$id — বদ্যমান ক্লায়েন্ট আপডেট করে।

Future<void> deleteClient(int id)

DELETE /clients/\$id — সফট-ডলিটি; ক্লায়েন্টের ওপনে অপরচুনটি থাকলে ব্যাকএন্ড ৪০০ এরর দেয় যা এখানে গলি ফলো হয় না, বার্তা যমেন আসে তমেনই দেখনো হয়।

ক্লাস/এক্সটেনশন: CrmContactsAndTags on CrmRepository

কন্ট্যাক্ট, ট্যাগ ও অ্যাক্টিভিটি সংক্রান্ত মথডগুলো এখানে যোগ করা — মূল ফাইলের গঠন অপরবর্তি রাখতে আলাদা এক্সটেনশন হিসেবে রাখা হয়েছে, রিাইট নয়।

Future<List<ClientContact>> contactsFor(int clientId)

GET /clients/\$clientId/contacts — একটি ক্লায়েন্টের সব কন্ট্যাক্ট বয়ের লস্ট হিসেবে আনবে।

Future<PagedResponse<ClientContact>> allContacts({String? keyword, int page = 0, int size = 30})

/crm/contacts — সব ক্লায়েন্ট জুড়ে সার্চযোগ্য (keyword কোয়েরি) পজেড কন্ট্যাক্ট লস্ট আনবে।

Future<ClientContact> createContact(int clientId, ClientContactRequest request)

POST /clients/\$clientId/contacts — একটি ক্লায়েন্টের জন্য নতুন কন্ট্যাক্ট তৈরি করে।

Future<ClientContact> updateContact(int clientId, int id, ClientContactRequest request)

PATCH /clients/\$clientId/contacts/\$id — পুরো কন্ট্যাক্ট অবজেক্ট পাঠায়; বাদ পড়া ফিল্ড ক্লয়ার হয় যায় (ClientContactRequest-এর ডকুমেন্টেশন অনুসারে)।

Future<ClientContact> makePrimaryContact(int clientId, int id)

PATCH /clients/\$clientId/contacts/\$id/primary — এই কন্ট্যাক্টকে একমাত্র প্রাইমারি বানায়, আগের প্রাইমারি স্বয়ংক্রিয়ভাবে সরে যায়।

Future<void> deleteContact(int clientId, int id)

DELETE /clients/{clientId}/contacts/{id} — একটি কন্ট্যাক্ট মুছে ফেলে।

Future<List<Tag>> tags()

GET /crm/tags — শ্যোরড ট্যাগ ভোকাবুলারি বয়োর লস্ট হসিবো আনো।

Future<Tag> createTag(TagRequest request)

POST /crm/tags — নতুন ট্যাগ তরো করো।

Future<Tag> updateTag(int id, TagRequest request)

PATCH /crm/tags/{id} — বদ্যমান ট্যাগ আপডটে করো।

Future<void> deleteTag(int id)

DELETE /crm/tags/{id} — ট্যাগ ভোকাবুলারি থকো সরায়; ইতমধ্যো ট্যাগ করা রকোরডগুলো ট্যাগটি হারায়, তাই স্ক্রনি আগো নশ্চতি করো নয়ো।

Future<PagedResponse<CrmActivity>> activities({int? clientId, int? opportunityId, int page = 0, int size = 30})

/crm/activities — ঐচ্ছকিভাবে clientId বা opportunityId দয়ো সংকীরণ করা পজোড টাইমলাইন আনো।

Future<CrmActivity> logActivity(CrmActivityRequest request)

POST /crm/activities — সাধারণ (ক্লায়নোট/অপরচুনটি) অ্যাক্টিভিটি লগ করো। এটি একই ক্লাসরো logActivity(int id, LogActivityRequest)-এর থকো ভনিন প্যারামটির সগিনচোররো এক্সটেনশন মথোড ওভারলোড।

Future<CrmActivity> completeActivity(int id)

PATCH /crm/activities/{id}/complete — একটি অ্যাক্টিভিটি সিম্পন্ন হসিবো মার্ক করো।

Future<void> deleteActivity(int id)

DELETE /crm/activities/{id} — একটি সাধারণ অ্যাক্টিভিটি মুছে ফেলে।

Future<String> summariseActivity({int? clientId, int? opportunityId})

GET /crm/activities/summary — একটি ক্লায়নোট বা অপরচুনটির সাথে যা ঘটছো তার একটি প্যারাগ্রাফ সামারি আনো (ড্রাফটডো, সভো হয় না); JSON-এর summary ফলিড থকো স্ট্রিং বরো করো, না থাকলো খালি স্ট্রিং।

Future<List<Client>> activeClients()

GET /clients/active — এখনো লনোদনোরত ক্লায়নোটদরো বয়োর লস্ট আনো; পকিার উইজটে এটি পিডো।

Future<Client> inviteToPortal(int id)

POST /clients/{id}/invite-portal — ক্লায়নোটকো পোর্টাল ইনভটিশোন পাঠায় ও পোর্টাল অ্যাক্সসো চালু করো।

টপ-লভেলো প্রোভাইডার

crmRepositoryProvider

Provider<CrmRepository> — apiClientProvider থকো ApiClient নয়ো CrmRepository ইনস্ট্যান্স তরো ও সরবরাহ করো।

ফাইল: lib/features/crm/crm_screen.dart

সংক্ষিপ্ত ববিরণ: CRM মডউলরে মূল শলে স্ক্রনি — পারমশিন অনুযায়ী Pipeline/Leads/Clients ট্যাব দেখায় এবং প্রতটি ট্যাবরে নজিস্ব ফ্লোটিং অ্যাকশন বাটন সরবরাহ করে। এখানই "New lead" বটম-শটি সংজ্ঞায়িত।

ক্লাস: _TabAction

একটি ট্যাবরে ফ্লোটিং বাটন কী তরৈকিরে তা ধরে রাখা সাধারণ ডটো হোল্ডার — label, icon এবং open (একটি BuildContext নিয়ে Future<void> রটার্নকারী ফাংশন)।

ক্লাস/উইজটে: CrmScreen

ConsumerWidget, ঐচ্ছকি initialTabLabel নিয়ে। পারমশিন কন্ট্রোলার দেখে কোন ট্যাব দেখানো যাবে ঠিক করে — একজন রপেরে কাছে শুধু LEAD_VIEW থাকলে সে শুধু একটি ট্যাব পাবে, দুটি ট্যাব যা 403 দবে তা নয়।

Widget build(BuildContext context, WidgetRef ref)

পারমশিন এখনো লোড না-হলে লোডার দেখায়। permissions.has(...) চকে করে tabs লস্ট তরৈকিরে (Pipeline — অ্যাকশনবাহীন কারণ ডলি একটি ক্লায়েন্ট থেকে খোলা হয়; Leads — leadCreate থাকলে "New lead" অ্যাকশন; Clients — clientCreate থাকলে "New client" অ্যাকশন)। tabs খালি হলে "Not available to you" EmptyState দেখায়। নয়তো DefaultTabController তরৈকিরে, AnimatedBuilder দিয়ে বর্তমান ট্যাব-ইনডেক্স অনুসরণ করে সংশ্লিষ্ট _TabAction-এর FAB দেখায় এবং TabBarView/একক ভডি রন্ডার করে।

টপ-লভেলে ফাংশন

Future<void> showNewLeadSheet(BuildContext context)

showModalBottomSheet দিয়ে _NewLeadSheet খোলে, স্ক্রল-নয়িন্ত্রতি।

ক্লাস/উইজটে: _NewLeadSheet / _NewLeadSheetState

নতুন লডি তরৈরি ফর্ম — কন্ট্রাক্ট নাম, কোম্পানি, ইমেইল, ফোন, উৎস, প্রায়োরটি, প্রত্যাশতি মূল্য নিয়ে।

void dispose()

সব TextEditingController (_contactName, _companyName, _email, _phone, _value) ডসিপোজ করে।

Future<void> _submit()

ফর্ম ভ্যালডিটে করে, _submitting ফ্ল্যাগ সটে করে ফোকাস সরায়, তারপর

leadsProvider.notifier.create(...) কল করে CreateLeadRequest পাঠায়। সফল হলে শটি বন্ধ করে

স্ন্যাকবার দেখায়; ApiException হলে সার্তার-বার্তা, অন্য এরর হলে জনেরেকি বার্তা _error-এ সটে করে; finally-তে _submitting false করে।

Widget build(BuildContext context)

একটি Form-এ কন্ট্রাক্ট নাম (আবশ্যক), কোম্পানি, ইমেইল (থাকলে '@' যাচাই), ফোন, উৎস ও প্রায়োরটি ড্রপডাউন এবং প্রত্যাশতি মূল্য (সংখ্যা যাচাই) ফলিড সাজায়, শেষে LoadingButton দিয়ে সাবমটি করায়।

ফাইল: lib/features/crm/edit_lead_sheet.dart

সংক্ষিপ্ত ববিরণ: একটি বিদ্যমান লডি এডিট করার বটম-শিট ফর্ম। এডিট-যোগ্যতা যাচাইয়ের একটি শয়ার্ড হলেপার ফাংশনও এখানে সংজ্ঞায়িত, যাতে ডিটাইল স্ক্রিনি ও ভবিষ্যতের যেকোনো কলার একই উত্তরে একমত হয়।

টপ-লেভেলে ফাংশন

`bool canEditLead(Lead lead)`

লডিটি `converted` না-হলে এবং স্ট্যাটাস `disqualified` না-হলে `true` দিয়ে — ব্যাকএন্ড কনভার্টেড বা ডিসকোয়ালিফায়ড লডি এডিট প্রত্যাখ্যান করে বলে UI-তে অ্যাকশন লুকিয়ে রাখা হয়, দেখিয়ে-প্রত্যাখ্যান করা হয় না।

`Future<void> showEditLeadSheet(BuildContext context, Lead lead)`

`showModalBottomSheet` দিয়ে দেওয়া `lead` নিয়ে `_EditLeadSheet` খোলো।

ক্লাস/উইজটে: `_EditLeadSheet / _EditLeadSheetState`

বিদ্যমান লডিরে সব ফিল্ড প্র-ফিলি করা এডিট ফর্ম — কন্ট্রাক্ট নাম, কোম্পানি, ইমেইল, ফোন, জব টাইটলে, স্ট্যাটাস, উৎস, প্রায়েরটি, প্রত্যাশিত মূল্য, নোট।

`void initState()`

`widget.lead`-এর মান দিয়ে সব `TextEditingController` ইনশিয়ালাইজ করে। `_status`, `_source`, `_priority` সরাসরি লিডিরে বর্তমান মান থেকে সেটে করে — অজানা এনাম-ভ্যালু থাকলেও তা ফলব্যাক না করে রেখে দিয়ে (নেচি `_optionsFor` দেখুন), যাতে ড্রপডাউন থ্রো না করে এবং অজানা মান নীরবে বদলে না যায়।

`void dispose()`

সব সাত `TextEditingController` (`_contactName`, `_companyName`, `_email`, `_phone`, `_jobTitle`, `_value`, `_notes`) ডিসপোজ করে।

`List<String> _optionsFor(List<String> known, String current)`

পরচিতি অপশন লিস্টে `current` থাকলে সেই লিস্টই ফেরত দিয়ে, না থাকলে `current`-কে সামনে যোগ করে দিয়ে — যাতে ড্রপডাউন লডিরে অজানা বর্তমান অবস্থা দেখাতে পারে ব্যবহারকারীর অনুমতি ছাড়া কোনো পরিবর্তন না করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেটে করে `leadsProvider.notifier.updateItem(widget.lead.id, UpdateLeadRequest(...))` কল করে সব ফিল্ড পাঠায়। সফল হলে শিট বন্ধ করে "Lead updated." সন্ধ্যাকবার দেখায়; ব্যর্থ হলে `ApiException`-এর বার্তা বা জনৈকি বার্তা `_error`-এ সেটে করে; `finally`-তে `_submitting` `false`।

`Widget build(BuildContext context)`

একটি `Form`-এ সব ফিল্ড রেন্ডার করে — স্ট্যাটাস ড্রপডাউনে `DISQUALIFIED` অপশনও থাকে (একবার সেটে হলে ব্যাকএন্ড আর এডিট মেনে নেয় না, তাই শিটটি আর পুনরায় খুলবে না)। মূল্য ফিল্ডের ভ্যালিডেটের ঋণাত্মক মান প্রত্যাখ্যান করে ("A deal cannot be worth less than nothing."), শেষে `LoadingButton` দিয়ে "Save changes"।

ফাইল: lib/features/crm/lead_detail_screen.dart

সংক্ষিপ্ত ববিরণ: একটি একক লডি়ে বসিতারতি স্ক্রিনি — হডোর, তথ্য, অ্যাকশন (অ্যাক্টিভিটি লিগ, কনভার্ট), অ্যাক্টিভিটি টাইমলাইন এবং এডিটি/ডলিটি/সামারাইজ মনেু দখোয়। AI-জনোরটেডে লডি-সামারাই ও কনভারশন ফলো-ও এখানই।

টপ-লভেলে ফাংশন

`void openLead(BuildContext context, int id)`

Navigator.push দিয়ে LeadDetailScreen(id: id)-এ নভেগিটে করে।

`Future<void> showLeadSummarySheet(BuildContext context, {required Lead lead})`

showModalBottomSheet দিয়ে _LeadSummarySheet খোলে যা লডি়ে AI-ড্রাফটে সামারাই দখোয়।

ক্লাস/উইজটে: LeadDetailScreen / _LeadDetailScreenState

ConsumerStatefulWidget, id নিয়ে। leadDetailProvider(id) ওয়াচ করে লডি ডটো লোড করে, ব্যস্ততা (_busy) ট্র্যাক করে যাতো একসাথে একাধিক অ্যাকশন না চলো।

`Future<void> _logActivity()`

_LogActivitySheet খুলে ব্যবহারকারীর কাছ থেকে LogActivityRequest নিয়ে; পাওয়া গেলে crmRepositoryProvider.logActivity(id, entry) কল করে, তারপর leadActivitiesProvider, leadDetailProvider ইনভ্যালিডিটে করে ও leadsProvider.notifier.refresh() চালায় (লডি়ে "last contacted" বদলে যায় বলে হডোর ও তালিকা-কার্ড দুটোই স্টলে হয়ে যায়)। সফল/ব্যর্থ স্ন্যাকবার দখোয়, finally-তে _busy false।

`Future<void> _convert(Lead lead)`

_ConvertLeadSheet খুলে ConvertLeadRequest নিয়ে; পাওয়া গেলে crmRepositoryProvider.convertLead(...) কল করে অপরচুনটি তিরে করে, সংশ্লিষ্ট প্রোভাইডার (leadDetailProvider, leadsProvider, opportunitiesProvider, pipelineSummaryProvider) ইনভ্যালিডিটে/রিফ্রেশ করে, তারপর openOpportunity দিয়ে নতুন ডলি়ে ডটিইল স্ক্রিনি নভেগিটে করে (কনভারশন লডি-কাজরে শেষে নয়, শুরু)।

`Future<void> _delete(Lead lead)`

AlertDialog-এ নশ্চিতিকরণ চায়; নশ্চিতি হলে leadsProvider.notifier.delete(lead.id) কল করে, সফল হলে স্ক্রিনি পপ করে স্ন্যাকবার দখোয়।

`Widget build(BuildContext context, WidgetRef ref)`

leadDetailProvider(id) ও permissionControllerProvider ওয়াচ করে; লডি লোড হয়ে থাকলে canEditLead+ পারমিশন থেকে canEdit/canDelete বরে করে, অ্যাপবারে _LeadMenu (লডি লোড হলে) দখোয়। async.when দিয়ে লোডিং/এরর/ডটো হ্যান্ডল করে, ডটো এলে RefreshIndicator-এর ভতের _Header, _Facts, ব্যস্ত না-থাকলে _Actions, ও _Activities সাজিয়ে দখোয়।

ক্লাস/উইজটে: _LeadMenu

StatelessWidget। এডিটি/সামারাইজ/আরকেটি ডলি/ডলিটি — এই চারটি অ্যাকশনের PopupMenuButton, যা একটি নন-নাল Lead-এর ওপর কলোজ করার জন্য আলাদা উইজটে রাখা হয়েছে।

Widget build(BuildContext context)

onSelected-এ ভ্যালু অনুযায়ী showEditLeadSheet, showLeadSummarySheet, showAnotherDealSheet, বা onDelete(lead) কল করে। মনে আইটেমে শর্তসাপেক্ষ — 'edit' শুধু canEdit হলে, 'summarise' সবসময়, 'another' শুধু canEdit && lead.converted হলে (কনভার্ট হয়ে থাকলেই আরকেট ডিলি বানানো অর্থপূর্ণ), 'delete' শুধু canDelete হলে।

ক্লাস/উইজটে: _Header

StatelessWidget। লডিং হেডলাইন, সাবলাইন, স্ট্যাটাস-চিপ, ট্যাগ, নোট এবং কনভার্টেড/সম্ভাব্য-ডুপ্লিকেট ব্যানার দেখায়।

Widget build(BuildContext context)

AppCard-এ হেডলাইন+সাবলাইন Column ও পাশে StatusChip রাখা; ট্যাগ থাকলে Wrap-এ TagChip দেখায়; নোট থাকলে টেক্সট দেখায়; lead.converted হলে সাফল্য-ব্যানার (কনভার্টেড ক্লায়েন্ট নাম বা জনৈকি বার্তা), নয়তো possibleDuplicate থাকলে তথ্য-ব্যানার দেখায়।

ক্লাস/উইজটে: _Facts

StatelessWidget। লডিংে বস্তিতারতি ফল্ডগুলো লবেলে-ভ্যালু জোড়ায় সাজিয়ে দেখায়, শুধু নন-নাল/নন-খালি রো দেখানো হয়।

Widget build(BuildContext context)

ইমেইল, ফোন, জব টাইটলে, ইন্ডাস্ট্রি, উৎস, প্রায়োরিটি, প্রত্যাশিত মূল্য, প্রত্যাশিত কলোজ, মালিক, সর্বশেষ যোগাযোগ, তৈরি তারিখ — এই রেকর্ড লিস্ট বানিয়ে where-এ ফিল্টার করে, AppCard-এর ভেতর প্রতটি রো Divider সহ রেন্ডার করে।

ক্লাস/উইজটে: _Actions

StatelessWidget। লডি-ডিটাইল স্ক্রিনিংে প্রাইমারি অ্যাকশন বাটন (অ্যাক্টিভিটি লিগ ও কনভার্ট) দেখায়।

Widget build(BuildContext context)

সবসময় "Log a call, meeting or note" আউটলাইনড বাটন দেখায়; lead.canConvert হলে "Convert to opportunity" এলভিটেডে বাটন দেখায়; না-হলে (এখনো কনভার্ট না-হয়ে থাকলে) কারণ বোঝানো একটি টেক্সট দেখায় ("Qualify this lead before converting...") — ডিসবেলড বাটনের বদলে ব্যাখ্যা দেওয়া হয় যাত্রে ট্যাপ করার প্রলোভন না থাকে।

ক্লাস/উইজটে: _Activities

ConsumerWidget, id নিয়ে। leadActivitiesProvider(id) ওয়াচ করে অ্যাক্টিভিটি টাইমলাইন দেখায়।

Widget build(BuildContext context, WidgetRef ref)

async.when-এ লোডিং/এরর/ডটো হ্যান্ডল করে; খালি হলে "Nothing logged yet." দেখায়, নয়তো প্রতিটি অ্যাক্টিভিটির জন্য _ActivityRow রেন্ডার করে — systemGenerated এন্ট্রির জন্য onDelete null রাখা হয় (অডিট ট্রাইল মুছতে দেওয়া হয় না)।

টপ-লেভেলে ফাংশন

`Future<void> _removeActivity(BuildContext context, WidgetRef ref, {required int leadId, required CrmActivity activity})`

`confirmAction`-এ নশ্চিতিকরণ চায়, নশ্চিতি হলে `crmRepositoryProvider.deleteLeadActivity(leadId, activity.id)` কল করে `leadActivitiesProvider(leadId)` ইনভ্যালিডেট করে এবং সন্ধ্যাকবার দেখায়।

ক্লাস/উইজটে: `_ActivityRow`

`StatelessWidget`। একটি অ্যাক্টিভিটি এন্ট্রির আইকন, বিষয়, বর্ণনা, সময় (রলিটেভি), টাইপ ও পারফরমার নাম, এবং ঐচ্ছকি ডলিটি আইকন দেখায়।

`Widget build(BuildContext context)`

`systemGenerated` হলে টোন মডিটেডে, নয়তো `brandInk`। আইকন-বক্স, বিষয়+সময়+(থাকলে) কলোজ-আইকন-রো, বর্ণনা (থাকলে) এবং নচি "টাইপ · পারফরমার" মটোডটো লাইন রন্ডার করে।

`static IconData _iconFor(String type)`

অ্যাক্টিভিটি টাইপ (`CALL`, `MEETING`, `EMAIL`, `TASK`, `FOLLOW_UP`, `STAGE_CHANGE/STATUS_CHANGE`, `DOCUMENT`, ডফিল্ট) অনুযায়ী উপযুক্ত `IconData` ফরত দিয়ে।

ক্লাস/উইজটে: `_LogActivitySheet / _LogActivitySheetState`

`StatefulWidget`। অ্যাক্টিভিটি লিগ করার ফর্ম — টাইপ ড্রপডাউন, সারাংশ, বস্টিারতি।

`void dispose()`

`_subject, _description` কন্ট্রোলার ডসিপোজ করে।

`Widget build(BuildContext context)`

টাইপ ড্রপডাউন (`crmActivityTypes`), আবশ্যক "Summary" ফলিড, ঐচ্ছকি "Detail" ফলিড এবং একটি `ElevatedButton` যা ভ্যালিডেট করে `Navigator.pop`-এ `LogActivityRequest` রটির্ন করে।

ক্লাস/উইজটে: `_ConvertLeadSheet / _ConvertLeadSheetState`

`StatefulWidget`, `lead` নেয়। লডিকে অপরচুনটিতে কনভার্ট করার জন্য ডলি-নাম, প্রত্যাশতি মূল্য ও কলোজ-ডটে নেওয়ার ফর্ম।

`void initState()`

`_name` লডি়ে `headline` দিয়ে, `_value` লডি়ে `estimatedValue` দিয়ে (থাকলে) প্র-ফলি করে; `_closeDate` লডি়ে `expectedCloseDate` পার্স করে সটে করে।

`void dispose()`

`_name, _value` কন্ট্রোলার ডসিপোজ করে।

`Future<void> _pickDate()`

`showDatePicker` খুলে (৩৬৫ দিন আগ থেকে ৩ বছর পরে পর্যন্ত রঞ্জে, ডফিল্ট আজ+৩০দিন) নর্বাচতি তারখি `_closeDate`-এ সটে করে।

Widget build(BuildContext context)

ব্যাখ্যামূলক নোট ("No client is created yet...") এর পর ডলি-নাম, প্রত্যাশিত মূল্য (শূন্যের বশে হতে হবে যাচাই), এবং InkWell-চালতি তারখি পকির ফলিড দেখায়। সাবমটি ফর্ম ও _closeDate != null দুটোই যাচাই করে (তারখি ফর্মের বাইরে থাকায় আলাদাভাবে চকে করতে হয়) এবং ConvertLeadRequest নিয়ে পপ করে।

ক্লাস/উইজটে: _LeadSummarySheet / _LeadSummarySheetState

ConsumerStatefulWidget, lead নিয়ে। AI-ড্রাফটে লডি-সামারি দেখায় — একবার জিজ্ঞাসা করাই সামারি তৈরি করে, কিছু স্টোর হয় না, এবং একই লডি দ্বিতীয়বার জিজ্ঞাসা করলে ভিন্ন ফলাফল আসতে পারে।

late final Future<Lead> _summary

crmRepositoryProvider.summariseLead(widget.lead.id) কল করে ইনশিয়ালাইজ হওয়া ফাউচার — বলিড-টাইমে একবারই তৈরি হয়।

Widget build(BuildContext context)

লডি-নাম, "মেশিনি-লখিত" সতরুতা টেক্সট এবং FutureBuilder<Lead> দেখায় — লোডিং, এরর (ApiException বার্তা বা জনেরেকি), অথবা aiSummary টেক্সট (খালি হলে ব্যাখ্যামূলক বার্তা) SelectableText-এ দেখায়।

ফাইল: lib/features/crm/lead_filter_sheet.dart

সংক্ষিপ্ত ববিরণ: লডি-ফলিটারিং, ফলিটার-প্রোভাইডার, লডি-কাউন্ট প্রোভাইডার, এবং CSV ইমপোর্ট/PDF এক্সপোর্ট ফচার একত্রে রাখা ফাইল।

ক্লাস: LeadFilterController

Notifier<LeadFilter>। বর্তমান সক্রিয় লডি-ফলিটার ধরে রাখে; খালি ফলিটার মানে "কোনো ফলিটার নেই"।

LeadFilter build()

প্রাথমিক অবস্থা হিসেবে খালি const LeadFilter() দিয়ে।

void set(LeadFilter filter)

স্টেটে সরাসরি দেওয়া filter-এ সটে করে।

void clear()

স্টেটে আবার খালি LeadFilter()-এ রসিটে করে।

টপ-লেভেলে প্রোভাইডার

leadFilterProvider

NotifierProvider<LeadFilterController, LeadFilter> — অ্যাপ জুড়ে বর্তমান লডি-ফলিটার সরবরাহ করে।

ক্লাস: FilteredLeadsController

AsyncNotifier<PagedState<Lead>> উইথ PagedLoader<Lead> মক্সিনি। বর্তমান ফলিটার অনুযায়ী পজেড লডি লসিট লোড করে।

Future<PagedState<Lead>> build()

leadFilterProvider ওয়াচ করে (ফিল্টার বদলালে রিবিল্ড হয়) এবং loadFirstPage() কল করে প্রথম পজে লোড করে।

Future<PagedResponse<Lead>> fetchPage(int page)

crmRepositoryProvider.filterLeads(currentFilter, page: page) কল করে একটি পজে ফেচ করে।

টপ-লেভেলে প্রোভাইডার

filteredLeadsProvider

AsyncNotifierProvider<FilteredLeadsController, PagedState<Lead>> |

leadCountsProvider

FutureProvider.autoDispose<({int active, int mine})> — activeLeadCount() ও myActiveLeadCount()

Future.wait-এ একসাথে চালিয়ে দুটি সংখ্যা রেকর্ড হিসাবে ফেরত দিয়ে (ব্যাকএন্ডে কম্বাইনড এন্ডপয়ন্ট নই বলে দুটি আলাদা কল, একসাথে চালানো হয় যাত হডোর দ্বিগুণ সময় না নিয়ে)।

টপ-লেভেলে ফাংশন

Future<void> showLeadFilterSheet(BuildContext context)

showModalBottomSheet দিয়ে _LeadFilterSheet খোলো।

ক্লাস/উইজটে: _LeadFilterSheet / _LeadFilterSheetState

ConsumerStatefulWidget। লডি-লিস্টের ফিল্টার নির্মাণের ফর্ম — কীওয়ার্ড, স্ট্যাটাস, উৎস, প্রায়োরিটি, তারিখ-রেঞ্জ, তনিটি বুলিয়ান ফ্ল্যাগ, সর্ট বাই ও ডিরেকশন।

late final _keyword

বর্তমান leadFilterProvider-এর keyword দিয়ে প্র-ফিলি করা TextEditingController।

late LeadFilter _filter

বর্তমান ফিল্টারের একটি লোকাল মডিউবেল কপি, শিটে সম্পাদনার সময় ব্যবহৃত হয়।

void dispose()

_keyword কন্ট্রোলার ডিসপোজ করে।

void _apply()

leadFilterProvider.notifier.set(...) কল করে _filter-কে (কীওয়ার্ড ট্রিম করে যোগ করে) গ্লোবাল স্টেটে প্রয়োগ করে এবং শিট বন্ধ করে।

Widget build(BuildContext context)

FormSheetFrame-এর ভেতর কীওয়ার্ড টেক্সটফিল্ড, স্ট্যাটাস/উৎস/প্রায়োরিটি _Choice ড্রপডাউন, দুটি DateField (Closing after/before), তনিটি _Flag সুইচ (activity/unassigned/highPriority) এবং একটি isConverted _Flag, sortBy/sortDirection ড্রপডাউন, এবং "Clear the filter" বাটন সাজায়। action: 'Apply' চাপলে _apply() চলে।

static String _sortLabel(String value)

সর্ট-কী (createdAt, lastActivityAt, expectedCloseDate, priority) কে পাঠযোগ্য বাংলা/ইংরেজি লেবেলে ম্যাপ করে; অজানা মান হলে যমেন-তমেন ফেরত দিয়ে।

ক্লাস/উইজটে: _Choice

StatelessWidget। একটি ঐচ্ছকি-স্ট্রিং ড্রপডাউন — "Any" অপশনসহ (null-এর প্রতিনিধিত্ব করে) দেওয়া options লিস্ট থেকে বেছে নেওয়ার UI।

Widget build(BuildContext context)

DropDownButtonFormField<String?> বানায়, প্রথম আইটেমে "Any" (value null), তারপর প্রতিটি option লবেলে-ফর্ম্যাট করে দেখায়।

ক্লাস/উইজটে: _Flag

StatelessWidget। তনি-অবস্থার বুলিয়ান (null/true/false) প্রতিনিধিত্বকারী একটি সুইচ — off মান "অনুপস্থিতি" নয়, বরং সুস্পষ্টভাবে "না"।

Widget build(BuildContext context)

SwitchListTile.adaptive বানায় যার value হলো value ?? false; onChanged-এ সুইচ অন করলে true, অফ করলে null পাঠায় (ফোনে "শুধু অ্যাসাইনড" জাতীয় ঋণাত্মক-নির্দিষ্ট অবস্থা প্রয়োজন হয় না বলে false-অবস্থা কখনো সটে হয় না)।

টপ-লেভেলে ফাংশন (ইমপোর্ট/এক্সপোর্ট)

Future<void> showLeadImportSheet(BuildContext context)

showModalBottomSheet দিয়ে _ImportSheet খোলো।

ক্লাস/উইজটে: _ImportSheet / _ImportSheetState

ConsumerStatefulWidget। CSV ফাইল থেকে লডি ইমপোর্ট করার UI — ফাইল বেছে নেওয়া, ফলাফল (তৈরি সংখ্যা, বাদ পড়া রো) দেখানো।

Future<void> _pick()

pickAttachment(context) দিয়ে ফাইল বেছে নেবে; পাওয়া গেলে crmRepositoryProvider.importLeads(path, name) কল করে ফলাফল _result-এ সটে করে এবং leadCountsProvider ইনভ্যালিডেট করে। ApiException হলে ব্যাকগ্রাউন্ডে নজিরে বার্তা দেখায় (খালি ফাইল/অতিরিক্ত রো ইত্যাদির নির্দিষ্ট কারণসহ), অন্য এরর হলে জনৈকি বার্তা। finally-তে _busy false।

Widget build(BuildContext context)

শরীনাং, ব্যাখ্যা, এরর ব্যানার (থাকলে), ফলাফল থাকলে "N created" এবং বাদ-পড়া লাইনে ListView, শেষে LoadingButton ("Choose a file" / "Import another")।

টপ-লেভেলে ফাংশন

Future<void> exportLeadsPdf(BuildContext context, WidgetRef ref, {String? status})

crmRepositoryProvider.leadsPdf(status: status) দিয়ে বাইট আনে, getTemporaryDirectory()-এ leads.pdf ফাইল লখে, OpenFilex.open দিয়ে খোলার চেষ্টা করে; ওপেন করার মতো অ্যাপ না-থাকলে সন্ধ্যাকবার দেখায়। ApiException বা অন্য এরর হলে যথাক্রমে সার্বভার-বার্তা বা জনৈকি বার্তা দেখায়।

done

ফাইল: lib/features/crm/leads_tab.dart

সংক্ষিপ্ত ববিরণ: CRM মডউলরে "Leads" ট্যাব — লডি তালকা, ভডি ফল্টিার চপি, কাউন্ট স্ট্রপি এবং প্রতটি লিডিরে কার্ড দেখোয়। ফল্টিার সক্রয়ি থাকলে ভন্নি ব্যানার ও ডটো সোর্স ব্যবহার হয়, নাহলে সাধারণ "ভডি" (mine/all/high priority ইত্যাদি) অনুযায়ী তালকা লোড হয়।

ক্লাস/উইজেটে: LeadsTab

ConsumerWidget। এটি মূল প্রবশেবন্দি — লডি ফল্টিার প্রোভাইডার দেখে ঠকি করে ফল্টিার্ড নাকি সাধারণ লডি লস্টি দেখাবে, তারপর কাউন্ট স্ট্রপি, ফল্টিার ব্যানার/ভডি চপি এবং পজেড লস্টি রন্ডার করে।

Widget build(BuildContext context, WidgetRef ref)

leadFilterProvider থেকে বর্তমান ফল্টিার পড়ে, খালি না হলে **filteredLeadsProvider** আর খালি হলে **leadsProvider** ব্যবহার করে। **PagedList<Lead>** দিয়ে তালকা রন্ডার করে, **onRefresh/onLoadMore** কন্ট্রোলারের সাথে বাঁধা, খালি অবস্থায় ভন্নি আইকন/মসেজে, এবং প্রতটি আইটেমেরে জন্ম **_LeadCard** বানায় যা ট্যাপে **openLead** কল করে।

static String _emptyMessageFor(LeadView view)

প্রতটি **LeadView** (mine, all, highPriority, neverContacted, stale, unassigned)-এর জন্ম আলাদা "খালি" বার্তা রটাবন করে একটি **switch** এক্সপ্রেশন দিয়ে, কারণ প্রতটি ভিউয়ের খালি হওয়ার কারণ ও অর্থ ভন্নি।

ক্লাস/উইজেটে: _ViewFilter

ConsumerWidget। অনুভূমিক স্ক্রলযোগ্য চপি তালকা, যখনে ব্যবহারকারী "Mine", "All", "High priority" ইত্যাদি ভিউ বছে নতিে পারে।

static const _views (স্ট্যাটিক ফল্ডি)

(view: LeadView, label: String) রকেডরে একটি তালকা — প্রতটি চপিরে জন্ম কোন **LeadView** এবং কোন লবেলে দেখানো হবে তা সংজ্ঞায়িত করে।

Widget build(BuildContext context, WidgetRef ref)

leadViewProvider থেকে নরিবাচতি ভডি পড়ে, **ListView.separated** দিয়ে প্রতটি ভিউয়ের জন্ম একটি ট্যাপযোগ্য চপি বানায়; নরিবাচতি চপি ব্র্যান্ড কালারে হাইলাইট হয়, ট্যাপে **leadViewProvider.notifier.set(...)** কল হয়।

ক্লাস/উইজেটে: _LeadCard

StatelessWidget। একটি একক লডিরে কার্ড — শরিনোম, স্ট্যাটাস চপি, সাবলাইন, প্রায়োরিটি ডিট, আনুমানিক মূল্য, শেষে যোগাযোগের সময়, এবং কনভার্টেড/আনঅ্যাসাইনড/ট্যাগ ব্যাজ দেখোয়।

Widget build(BuildContext context)

AppCard দিয়ে মোড়ানো একটি **Column** তরৈকিরে: শীর্ষে শরিনোম ও **StatusChip**, তারপর ঐচ্ছকি সাবলাইন, তারপর প্রায়োরিটি/মূল্য/শেষে সক্রয়িতার সময়ের সারি, এবং সবশেষে কনভার্টেড/আনঅ্যাসাইনড ব্যাজ ও প্রথম তনিটি ট্যাগ (**TagChip**) দেখোয় (যদি প্রাসঙ্গিক হয়)।

ক্লাস/উইজেটে: TagChip

StatelessWidget (পাবলিক)। একটি CRM ট্যাগকে ব্যবহারকারীর বছে নেওয়া রঙে রন্ডার করে।

Widget build(BuildContext context)

tag.argb থেকে রঙ পার্স করার চেষ্টা করে; পার্স ব্যর্থ হলে (null) নরিপক্ষে bos.muted রঙে ফলব্যাক করে। একটি গোলাকার-কোণ কনটইনারে ট্যাগের নাম হালকা সবুজ ব্যাকগ্রাউন্ড ও বর্ডার সহ দেখায়; ডার্ক মোডে টেক্সট রঙ bos.text, লাইট মোডে ট্যাগের নজিস্ব রঙ ব্যবহৃত হয়।

ক্লাস/উইজটে: _PriorityDot

StatelessWidget। প্রায়োরিটি নির্দেশক একটি ছোট রঙনি বৃত্ত।

Widget build(BuildContext context)

priority স্ট্রিং (URGENT/HIGH/NORMAL/অন্যান্য) অনুযায়ী switch দিয়ে রঙ নির্বাচন করে (danger/warning/info/muted), এবং ৮x৮ পিক্সলের বৃত্তাকার Container রেন্ডার করে।

ক্লাস/উইজটে: _Counts

ConsumerWidget। লডি কতটাসিক্রয়/নজিরে তা দেখায়, পাশাপাশি ফিল্টার আইকন বাটন এবং import/export মেনু।

Widget build(BuildContext context, WidgetRef ref)

leadCountsProvider থেকে কাউন্ট পড়ে, লোড না হলে শুধু "Leads" শিরোনাম দেখায়, লোড হলে "N live, M yours" দেখায়। ডানপাশে একটি ফিল্টার IconButton (যা showLeadFilterSheet কল করে) এবং একটি PopupMenuButton (import showLeadImportSheet, export exportLeadsPdf) রয়েছে।

ক্লাস/উইজটে: _FilterBanner

ConsumerWidget। ফিল্টার সক্রিয় থাকলে ভিডিও চপিরে জায়গায় দেখানো হয়, যাতো বোঝা যায় তালিকা কনে স্বাভাবিক নয়।

Widget build(BuildContext context, WidgetRef ref)

একটি ফিল্টার আইকন, "Filtered" টেক্সট এবং একটি "Clear" বাটন দেখায়; বাটনের onPressed সরাসরি leadFilterProvider.notifier.clear-এ বাঁধা।

ফাইল: lib/features/crm/opportunity_detail_screen.dart

সংক্ষিপ্ত বিবরণ: একটি একক ডিলারে (Opportunity) বিস্তারিত স্ক্রিনি — হডার (নাম, স্ট্যাটাস, মূল্য), ফ্যাক্টস কার্ড, স্টেজ পরিবর্তনের অ্যাকশন, এবং অ্যাক্টিভিটি টাইমলাইন। LOST বা WON-এ যাওয়ার সময় বাড়তি তথ্য সংগ্রহ করার জন্য বটম শিট/ডায়ালগ দেখায়।

টপ-লেভেলে ফাংশন

void openOpportunity(BuildContext context, int id)

OpportunityDetailScreen-কে MaterialPageRoute দিয়ে পুশ করে নেভিগেটে করে দেয়; দিয়ে দেওয়া id দিয়ে ডিলি লোড করা হয়।

ক্লাস/উইজটে: OpportunityDetailScreen

ConsumerStatefulWidget। id প্রারামিটার নিয়ে একটি ডিলারে সম্পূর্ণ বিবরণ স্ক্রিনি তৈরি করে।

ক্লাস/উইজটে: _OpportunityDetailScreenState

মূল স্টেটে ক্লাস — স্টেজে পরিবর্তন, ডলিটি, এবং সংশ্লিষ্ট ডায়ালগ/শটিরে লজিক ধারণ করে।

Future<void> _moveTo(Opportunity opportunity, String stage)

ডলিকে নতুন স্টেজে সরায়। LOST-এ গেলে _askLostReason() দিয়ে কারণ সংগ্রহ করে (ব্যবহারকারী বাতলি করলে কিছুই হয় না)। WON-এ গেলে এবং ডলিরে ক্লায়েন্ট সন্ধিধান্তের প্রয়োজন হলে (needsClientDecisionOnWin) প্রথমত wonDuplicateCheck কল করে সম্ভাব্য ডুপ্লিকেটে ক্লায়েন্ট খোঁজা, মিলে গেলে _askDuplicateDecision দিয়ে জিজ্ঞাসে করে লিংক করবে নাকি নতুন বানাবে। অন্য সব ক্ষেত্রে সরাসরি ChangeStageRequest(stage: stage) পাঠায়। শেষে opportunitiesProvider.notifier.changeStage কল করে সফলতা/ত্রুটি সন্যাকবারে দেখায়, এবং _busy ফ্ল্যাগ স্টেট/আনস্টেট করে।

Future<({String code, String? detail})?> _askLostReason()

_LostReasonSheet একটি মিডাল বটম শটি হিসেবে দেখিয়ে ব্যবহারকারীর কাছ থেকে LOST কারণের কোড ও বিস্তারিত টেক্সট সংগ্রহ করে রিটার্ন করে; বাতলি করলে null।

Future<bool?> _askDuplicateDecision(DuplicateMatch match)

একটি AlertDialog দেখায় যেখানে বলা হয় সম্ভাব্য মিলে যাওয়া ক্লায়েন্টের নাম ও কীসের ভিত্তিতে মিলেছে। ব্যবহারকারী true (বিদ্যমান ক্লায়েন্টে লিংক করো) বা false (নতুন বানাও) বছে নতি পারে; "Cancel" করলে null। "Link existing" বাটন প্রাইমারি/জোরালো স্টাইলে দেখানো হয়, কারণ এটাই নরিপদ ডফিল্ট।

Future<void> _delete(Opportunity opportunity)

প্রথমত একটি নিশ্চিতকরণ ডায়ালগ দেখায়; নিশ্চিত হলে opportunitiesProvider.notifier.delete(id) কল করে, সফল হলে স্ক্রিনি পপ করে সন্যাকবার দেখায়, ব্যর্থ হলে ApiException-এর মেসেজে বা জনেরেকি ত্রুটি দেখায়।

Widget build(BuildContext context)

opportunityDetailProvider(widget.id) অ্যাসঙ্কি ভ্যালু ও permissionControllerProvider দেখে। canEdit তখনই সত্য যখন ডলি খোলা (Stage.isOpen) এবং আপডেটের অনুমতি আছে; canDelete শুধু ডলিটিরে অনুমতির উপর নির্ভর করে (বন্ধ ডলিও ডলিটি করা যায়)। AppBar-এ প্রয়োজনে _OpportunityMenu দেখায়। বডিতে লোডিং/এরর/ডটো তনি অবস্থা হ্যান্ডল করে; ডটো থাকলে RefreshIndicator-এর ভতের _Header, _Facts, (_busy হলে Loader নাহলে) _StageActions, এবং ActivityTimeline দেখায়।

ক্লাস/উইজটে: _OpportunityMenu

StatelessWidget। অ্যাপবারের ওভারফ্লো মেনুতে "Edit deal" ও "Delete" অপশন দেখায় (অনুমতি অনুযায়ী)।

Widget build(BuildContext context)

PopupMenuButton<String> তৈরিকরে; edit নির্বাচনে showEditOpportunitySheet কল করে, delete নির্বাচনে onDelete(opportunity) কল করে। মেনু আইটেমে শুধুমাত্র canEdit/canDelete সত্য হলে দেখানো হয়।

ক্লাস/উইজটে: _Header

StatelessWidget। ডলিরে নাম, স্ট্যাটাস চপি, পরিমাণ, ওয়েটেডে অ্যামাউন্ট, বর্ণনা এবং প্রয়োজ্য হলে ওভারডি/লস্ট ব্যানার দেখায়।

Widget build(BuildContext context)

একটি AppCard-এ নাম ও StatusChip, তারপর টাকার অংক ও ওয়েটেডে শতাংশ, তারপর বর্ণনা (থাকলে), এবং opportunity.isOverdue/isLost অনুযায়ী MessageBanner.error/MessageBanner.info দেখায়।

static String _lostLabel(String code)

lostReasons তালিকায় প্রদত্ত code-এর সাথে মিলে যাওয়া লবেলে খোঁজ; না পলে Fmt.label(code) ফলব্যাক হিসেবে ব্যবহার করে।

ক্লাস/উইজটে: _Facts

StatelessWidget। ডলি সম্পর্কিত key-value তথ্যের তালিকা (ক্লায়েন্ট, কন্টাক্ট, ওনার, সোর্স, প্রত্যাশিত/প্রকৃত কলোজ তারিখ, নক্সেস্ট স্টেপ ইত্যাদি) কার্ড আকারে দেখায়।

Widget build(BuildContext context)

একটি রিকর্ডের তালিকা (label, value) বানায়, null/খালি ভ্যালুর সারি ফিল্টার করে বাদ দেয়, তারপর SectionHeader ও AppCard-এর ভেতর প্রতিটি সারি divider সহ রেন্ডার করে।

ক্লাস/উইজটে: _StageActions

StatelessWidget। ডলিকে পরবর্তী স্টেজে সরানো, Won/Lost মার্ক করা, বা অন্য যেকোনো স্টেজে সরাসরি সরানোর অ্যাকশন বোতাম দেখায়।

Widget build(BuildContext context)

ডলি বন্ধ (!opportunity.isOpen) হলে শুধু একটি তথ্য ব্যানার দেখায় (রেগিপেনিং ওয়েবে অ্যাপ থাকে করত হয়)। খোলা থাকলে Stage.open তালিকায় পরবর্তী স্টেজে বের করে একটি প্রাইমারি "Advance to..." বোতাম দেখায় (যদি থাকে), তারপর "Lost" ও "Won" বোতাম পাশাপাশি, এবং সবশেষে Wrap-এ বাকসিব স্টেজের জন্য আউটলাইন বোতাম (current/next বাদে)। প্রতিটি বোতাম onMove(stage) কল করে।

ক্লাস/উইজটে: _LostReasonSheet

StatefulWidget। LOST করার সময় কারণ কোড ও ঐচ্ছিক বিস্তারিত টেক্সট সংগ্রহের ফর্ম শীট।

ক্লাস/উইজটে: _LostReasonSheetState

bool get _needsDetail

সত্য হয় যখন নির্বাচিত কোড 'OTHER', অর্থাৎ তখনই বিস্তারিত টেক্সট ফিল্ড বাধ্যতামূলক।

void dispose()

_detail কন্ট্রোলার ডিসপোজ করে, তারপর super.dispose() কল করে।

Widget build(BuildContext context)

একটি Form-এর ভেতর RadioGroup<String> দিয়ে lostReasons তালিকার প্রতিটি কারণের জন্য RadioListTile দেখায়। _needsDetail সত্য হলে একটি TextFormField (validator সহ, খালি হলে এরর) দেখায়। "Mark as lost" বোতাম ফর্ম ভ্যালিডেট করে (প্রয়োজন) এবং (code, detail) রেকর্ড রটির্ন করে Navigator.pop করে।

ফাইল: lib/features/crm/opportunity_form_sheet.dart

সংক্ষিপ্ত ববিরণ: নতুন ডলি তরৈ, লডি থকে আরকেটি ডলি তরৈ, বা বদিযমান ডলি সম্পাদনার জন্য একটি ভাগাভাগি বটম শটি ফর্ম। কনটেক্সট (নতুন/এডিট/লডি-থকে) অনুযায়ী ভিন্ন API কল রুট করে।

উপ-লভেলে ফাংশন

`Future<void> showNewOpportunitySheet(BuildContext context, {required int clientId, required String clientName})`

`_OpportunityFormSheet`-কে `clientId/clientName` সহ মডাল বটম শটি হিসেবে দেখায়। যহেতু ব্যাকএন্ডের সরাসরি-তরৈ পাথে `clientId` আবশ্যক, তাই এই শটি সবসময় একটি ক্লায়েন্ট থকে থোলা হয়, নজি ক্লায়েন্ট পকার দেখায় না।

`Future<void> showAnotherDealSheet(BuildContext context, {required int leadId, required String leadName})`

একই লডি়ের বপি়রীতে আরও একটি ডলি যোগ করার শটি দেখায় (লডি কনভার্ট করা থকে আলাদা — লডিকে converted মার্ক করে না)। `clientId` পাঠানো হয় না; ব্যাকএন্ড লডি়ের সাথে যুক্ত ক্লায়েন্ট ব্যবহার করে।

`Future<void> showEditOpportunitySheet(BuildContext context, Opportunity opportunity)`

বদিযমান `opportunity` সম্পাদনার জন্য শটি দেখায়।

ক্লাস/উইজটে: `_OpportunityFormSheet`

`ConsumerStatefulWidget`। তনিটি কনস্ট্রাক্টর-প্যাটার্নে ব্যবহৃত হয়: নতুন ডলি (`clientId/clientName`), লডি থকে আরকেটি ডলি (`fromLeadId`), অথবা এডিট (`existing`)।

ক্লাস/উইজটে: `_OpportunityFormSheetState`

`bool get _isEdit`

`widget.existing` নন-নাল হলে সত্য — অর্থাৎ এটি একটি এডিট অপারেশন কনি তা নরিদশে করে।

`void initState()`

টেক্সট কন্ট্রোলারগুলো (`_name`, `_amount`, `_description`, `_nextStep`) বদিযমান ডলি়ের মান দয়ি়ে (অথবা খালি) শুরু করে। `_description/_nextStep` খালি হলেও সেটে করা হয়, কারণ আপডেটে পাথে ব্যাকএন্ড এই দুটো ফলিড শর্তহীনভাবে বসয়ি়ে দেয়। `_expectedClose` ও `_source`ও বদিযমান মান থকে সেটে হয়।

`void dispose()`

চারটি টেক্সট কন্ট্রোলার (`_name`, `_amount`, `_description`, `_nextStep`) ডিসপোজ করে, তারপর `super.dispose()` কল করে।

`Future<void> _pickCloseDate()`

`showDatePicker` দেখায়, বর্তমান বছরে এক বছর আগে থকে পাঁচ বছর পরে পর্যন্ত রঞ্জে সহ (অতীত তারিখ অনুমোদতি, কারণ স্লপিড ডেটে রকের্ড করার প্রয়োজন হতে পারে)। বাছাই করা হলে `_expectedClose` সেটে করে।

`OpportunityRequest _buildRequest()`

বর্তমান ফর্ম ফলিড থকে একটি `OpportunityRequest` তরৈকিরে। `clientId` শুধু তখনই পাঠানো হয় যখন এটি এডিট নয় (আপডেটে পাথ এটি উপেক্ষা করে)। `amount` স্ট্রিং থকে পার্স হয়, `expectedCloseDate` ISO ফর্ম্যাটে রূপান্তরিত হয়।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে, ফোকাস সরিয়ে _submitting স্টেটে করে। তনিট পিথ: এডটি হল controller.updateItem, fromLeadId থাকলে crmRepositoryProvider.opportunityFromLead কল করে তারপর controller.refresh(), নাহলে controller.create। সফল হলে শাট পপ করে সন্ধ্যাকবার দেখায়; ApiException বা জনেরেকি ত্রুটি হলে _error স্টেটে করে।

Widget build(BuildContext context)

_source যদি leadSources-এ না থাকে তবু সটোকে তালকির সামনে যোগ করে (যাতে বদ্যমান কাস্টম সোর্স হারিয়ে না যায়)। একটি Form-এ শরিনোম, ঐচ্ছকি এরর ব্যানার, নাম/পরমাণ/প্রত্যাশতি কলোজ তারখি (InkWell + InputDecorator)/সোর্স (DropDownButtonFormField)/নক্সেস্ট স্টেপে/বর্ণনা ফলিড, এডটি মোডে একটি তিথ্য নোট (স্টেজে পরবির্তন এখান থেকে হয় না), এবং সবশেষে LoadingButton (_submit কল করে) দেখায়।

ফাইল: lib/features/crm/pipeline_tab.dart

সংক্ষিপ্ত ববিরণ: CRM মডউলরে "Pipeline" ট্যাব — কানবান বোর্ডরে বদলে একটি সারাংশ স্ট্রপি, স্টেজে ফলিটার চপি এবং একটি ফলিটারযোগ্য তালকা দয়ি ডলিগুলো দেখায়, কারণ ফোন ড্র্যাগ-এন্ড-ড্রপ কার্যকর নয়।

ক্লাস/উইজটে: PipelineTab

ConsumerWidget। ট্যাবরে মূল প্রবশেবিন্দু — সারাংশ স্ট্রপি, স্টেজে ফলিটার, এবং পজেড অপারচুনটি তালকা একত্র করে।

Widget build(BuildContext context, WidgetRef ref)

opportunitiesProvider অ্যাসঙ্কি ভ্যালু দেখে PagedListView<Opportunity> তরৈকিরে। রফিরশে pipelineSummaryProvider ইনভ্যালিডেট করে এবং কন্ট্রোলার রফিরশে করে; প্রতটি আইটেমেরে জন্য _OpportunityCard তরৈকিরে যা ট্যাপে openOpportunity কল করে।

ক্লাস/উইজটে: _SummaryStrip

ConsumerWidget। ওপনে পাইপলাইন ভ্যালু, ওয়টেডে ফোরকাস্ট এবং উওন ভ্যালু — তনিট সিংখ্যা পাশাপাশি দেখায়, সাথে খোলা ডলিরে সংখ্যা।

Widget build(BuildContext context, WidgetRef ref)

pipelineSummaryProvider থেকে মান না পাওয়া পর্যন্ত SizedBox.shrink() রটার্ন করে (ড্যাশরে স্ট্রপিরে বদলে কিছুই না দেখানো পছন্দনীয়)। মান পলে তনিট ফিগাররে রকেড তালকা বানয়ি ডভাইডার-বভিক্ত Row-এ দেখায়, নচি মোট খোলা ডলিরে সংখ্যা।

ক্লাস/উইজটে: _StageFilter

ConsumerWidget। "All open" সহ প্রতটি খোলা স্টেজে, এবং Won/Lost-এর জন্য অনুভূমকি ফলিটার চপি দেখায়।

Widget build(BuildContext context, WidgetRef ref)

stages তালিকা বানায় (null [সব খোলা] + Stage.open + Won + Lost) — Won/Lost এখানে থাকলেও ডিফল্ট ভিউয়ে দেখানো হয় না, যাতো বন্ধ ডলি স্ক্রল করে পার হতে না হয়। প্রতিটি চিপের পাশে pipelineSummaryProvider থেকে গণনা (থাকলে) দেখায়; ট্যাপে pipelineStageProvider.notifier.set(stage) কল করে।

ক্লাস/উইজটে: _OpportunityCard

StatelessWidget। একটি ডিলির কার্ড — নাম, স্ট্যাটাস চিপ, ক্লায়েন্ট/কন্টাক্ট নাম, পরিমাণ, ওয়েটেডে পরিমাণ, সম্ভাবনা শতাংশ, প্রতিশ্রুতি কলোজ তারিখ, ওভারডাউ সতর্কতা এবং নেক্সট স্টেপে দেখায়।

Widget build(BuildContext context)

AppCard-এর ভেতর নাম ও StatusChip উপরে, তারপর ক্লায়েন্ট/কন্টাক্ট নাম, তারপর পরিমাণ/ওয়েটেডে পরিমাণ (বামে) ও সম্ভাবনা%/কলোজ তারিখ (ডানে) একটি সিরিজে। opportunity.isOverdue সত্য হলে একটি লাল সতর্কতা সারি, এবং nextStep থাকলে একটি নেক্সট-স্টেপে সারি দেখায়।

ফাইল: lib/features/dashboard/dashboard_models.dart

সংক্ষিপ্ত বিবরণ: ড্যাশবোর্ড API-এর রেসপন্সগুলো মডেল করে এমন ডটো ক্লাসের সংগ্রহ — কোম্পানিস্তরের সারাংশ (CRM, সার্ভিস ডেস্ক, সাপোর্ট, ফিনিয়ান্স, HR), সুপারিশ, ইনসাইট, এবং প্ল্যাটফর্ম-স্তরের (সুপার অ্যাডমিন) সারাংশ ও মটেরক্স হস্টিং। প্রতিটি ক্লাসের একটি fromJson ফ্যাক্টরি রয়েছে যা null-নিরপদ পার্সিং করে।

ক্লাস/উইজটে: DashboardSummary

কোম্পানি ড্যাশবোর্ডের প্রধান সারাংশ — লডি, ক্লায়েন্ট, অপরচুনটি, সার্ভিস রকিওয়েস্ট, টিকিটে, ইনভয়েস, ওয়ালটে, কর্মচারী সংক্রান্ত সংখ্যা এবং ট্রেন্ড শতাংশ ধারণ করে। ব্যাকএন্ডের পঞ্চাশ-প্লাস ফল্ডারে DTO থেকে শুধু ফোনে দেখানোর উপযোগী অংশটুকু মডেল করে।

static int _int(Object? value)

JSON থেকে আসা num? মানকে নিরপদে int-এ রূপান্তর করে; null হলে 0 রিটার্ন করে।

static double _double(Object? value)

JSON থেকে আসা num? মানকে নিরপদে double-এ রূপান্তর করে; null হলে 0 রিটার্ন করে।

factory DashboardSummary.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে সব ফল্ডার পার্স করে DashboardSummary তৈরি করে; _int/_double হলে পার দ্বি সংখ্যা, এবং overdueInvoices/announcements তালিকা whereType+map দিয়ে যথাক্রমে OverdueInvoice.fromJson/DashboardAnnouncement.fromJson ব্যবহার করে পার্স করে।

ক্লাস/উইজটে: OverdueInvoice

একটি বিকয়ো ইনভয়েসের সংক্ষিপ্ত তথ্য — নম্বর, পরিমাণ, কতদিন ওভারডাউ, ও ঐচ্ছিক ক্লায়েন্ট নাম।

factory OverdueInvoice.fromJson(Map<String, dynamic> json)

JSON থেকে invoiceNumber, amount, daysOverdue, clientName নিরপদে (ডিফল্ট মান সহ) পার্স করে অবজেক্ট তৈরি করে।

ক্লাস/উইজটে: DashboardAnnouncement

ড্যাশবোর্ডে দেখানো একটি ঘোষণার সংক্ষিপ্ত রূপ — শিরোনাম, কনটেন্ট, ও কতক্ষণ আগে (টেক্সট আকারে); কোনো id নেই তাই এটি খোলা যায় না।

factory DashboardAnnouncement.fromJson(Map<String, dynamic> json)

JSON থেকে title, content, timeAgo পার্স করে অবজেক্ট তৈরি করে।

ক্লাস/উইজটে: Recommendation

সিস্টেমে যা করা উচিত মনে করে তার একটি সুপারিশ — টাইপ (FOLLOW_UP/SLA_RISK/LICENSE_EXPIRY/OVERDUE_INVOICE), সতর্কবার্তা (INFO/WARNING/CRITICAL), বার্তা এবং একটি ওয়েব-অ্যাপ লিংক (যা এই অ্যাপে অনুসরণ করা হয় না)।

bool get isCritical

severity == 'CRITICAL' হলে সত্য রিটার্ন করে।

bool get isWarning

severity == 'WARNING' হলে সত্য রিটার্ন করে।

factory Recommendation.fromJson(Map<String, dynamic> json)

JSON থেকে type, severity (ডিফল্ট 'INFO'), message, link পার্স করে অবজেক্ট তৈরি করে।

ক্লাস/উইজটে: DashboardInsights

সহায়ক (assistant/AI) সৃষ্ট ইনসাইট টেক্সট এবং স্টেজিনোরটে করতে কত মলিসিকেন্ড লগেছে তা ধারণ করে।

factory DashboardInsights.fromJson(Map<String, dynamic> json)

JSON থেকে insights (স্ট্রিং) ও generatedInMs (int, ডিফল্ট ০) পার্স করে অবজেক্ট তৈরি করে।

ক্লাস/উইজটে: PlatformSummary

সম্পূর্ণ প্ল্যাটফর্মের (সব টেনেয়ান্ট) বাহ্যিক দৃষ্টিভঙ্গি থেকে সারাংশ — কোম্পানি সংখ্যা (মোট/সক্রিয়/ট্রায়াল/সাসপেন্ডেড/যাচাইয়ের অপেক্ষমাণ), ৭ দিনে শেষে হতে যাওয়া ট্রায়াল, মোট প্ল্যাটফর্ম ব্যবহারকারী, রাজস্ব, এবং প্ল্যান অনুযায়ী কোম্পানি গণনা। শুধু প্ল্যাটফর্ম স্টাফের জন্য।

factory PlatformSummary.fromJson(Map<String, dynamic> json)

JSON থেকে সব সাংখ্যিক ফিল্ড নরিপদে পার্স করে, এবং companiesByPlan তালিকা whereType+map(PlanCount.fromJson) দিয়ে পার্স করে। প্ল্যান তালিকা একটি নির্দিষ্ট স্টে নয় বরং রানটাইমে যা আছে তাই — কারণ সুপার অ্যাডমিনি যেকোনো সময় নতুন প্ল্যান যোগ করতে পারে।

ক্লাস/উইজটে: PlanCount

একটি নির্দিষ্ট প্ল্যানে (কোড ও নাম) কতগুলো কোম্পানি আছে তার গণনা।

factory PlanCount.fromJson(Map<String, dynamic> json)

JSON থেকে code, name, count পার্স করে অবজেক্ট তৈরি করে।

ক্লাস/উইজটে: PlatformMetricsPoint

প্ল্যাটফর্মের ট্রেন্ড লাইনের একটি দিনের সন্ধ্যাপন্ট — কোম্পানি সংখ্যা ও রাজস্ব। সন্ধ্যাপন্ট নেওয়া শুরুর আগে দিনগুলো শূন্য না দেখিয়ে সম্পূর্ণ অনুপস্থিতি থাকে।

factory PlatformMetricsPoint.fromJson(Map<String, dynamic> json)

JSON থেকে date, চারটি কোম্পানি সংখ্যা ফিল্ড এবং revenue পার্স করে অবজেক্ট তৈরি করে।

ফাইল: lib/features/dashboard/dashboard_repository.dart

সংক্ষিপ্ত বিবরণ: ড্যাশবোর্ড API এন্ডপয়েন্টের রপিওজিটিরি এবং সংশ্লিষ্ট Riverpod প্রোভাইডারগুলো। একই কন্ট্রোলারের পছন্দে দুটি ভিন্ন অডিয়েন্স আছে — সাইন-ইন করা কোম্পানির জন্য /summary, /recommendations, /insights, এবং প্ল্যাটফর্ম পরিচালকদের জন্য /platform-summary, /platform-metrics-history।

ক্লাস/উইজটে: DashboardRepository

ড্যাশবোর্ড-সংশ্লিষ্ট সব HTTP কল এই ক্লাসে কেন্দ্রীভূত। কনস্ট্রাক্টরে একটি ApiClient ইনজেক্ট করা হয়; _base ধ্রুবক (/dashboard) সব এন্ডপয়েন্টের পথ প্রফিক্স।

Future<DashboardSummary> summary({String? from, String? to})

GET /dashboard/summary কল করে, ঐচ্ছকি from/to কুয়েরি প্যারামিটার সহ (দেওয়া না হলে ব্যাকএন্ড নজির্ব সময় উইন্ডো বেছে নেয়)। রসেপন্স DashboardSummary.fromJson দিয়ে পার্স করে রিটার্ন করে।

Future<List<Recommendation>> recommendations()

GET /dashboard/recommendations কল করে একটি বিয়ের JSON লিস্ট পায়, whereType+map(Recommendation.fromJson) দিয়ে পার্স করে অপরিবর্তনীয় (growable: false) তালিকা রিটার্ন করে।

Future<DashboardInsights> insights()

GET /dashboard/insights কল করে; এটি একটি AI প্রোভাইডার কল খরচ করে তাই স্ক্রিনি লোডের সময় নয়, চাহিদা অনুযায়ী কল করা হয়। রসেপন্স DashboardInsights.fromJson দিয়ে পার্স করে।

Future<PlatformSummary> platformSummary()

GET /dashboard/platform-summary কল করে রসেপন্স PlatformSummary.fromJson দিয়ে পার্স করে রিটার্ন করে।

Future<List<PlatformMetricsPoint>> platformHistory({int days = 30})

GET /dashboard/platform-metrics-history কল করে days কুয়েরি প্যারামিটার সহ (ডিফল্ট ৩০), রসেপন্স তালিকা whereType+map(PlatformMetricsPoint.fromJson) দিয়ে পার্স করে রিটার্ন করে।

টপ-লেভেলে প্রোভাইডার

dashboardRepositoryProvider

একটি Provider<DashboardRepository> যা apiClientProvider থেকে ApiClient নিয়ে DashboardRepository ইনস্ট্যান্স তৈরি করে।

dashboardSummaryProvider

একটি `FutureProvider.autoDispose<DashboardSummary>` — `currentUserProvider` দেখে (ব্যবহারকারী বদলালে পুনরায় লোড হয়) এবং `DashboardRepository.summary()` কল করে।

recommendationsProvider

একটি `FutureProvider.autoDispose<List<Recommendation>>` — `currentUserProvider` দেখে এবং `DashboardRepository.recommendations()` কল করে।

dashboardInsightsProvider

একটি `FutureProvider.autoDispose<DashboardInsights>` — স্ক্রিনি লোডের সময় স্বয়ংক্রিয়ভাবে watch করা হয় না, শুধু কডে চাইলে (উদাহরণস্বরূপ বাটন চাপলে) পড়া হয়, কারণ প্রতিটি কল একটি প্রোভাইডার রকিওয়েস্ট খরচ করে।

platformSummaryProvider

একটি `FutureProvider.autoDispose<PlatformSummary>` — `currentUserProvider` দেখে এবং `DashboardRepository.platformSummary()` কল করে।

platformHistoryProvider

একটি `FutureProvider.autoDispose<List<PlatformMetricsPoint>>` — `DashboardRepository.platformHistory()` কল করে (ডিফল্ট ৩০ দিন)।

done

ফাইল: lib/features/dashboard/dashboard_screen.dart

সংক্ষিপ্ত বিবরণ: কোম্পানির ড্যাশবোর্ড স্ক্রিনি — ওয়েব অ্যাপের পনেরোটি কনফিগারযোগ্য উইজটে বদলে একই পরিসংখ্যান নব্বিটি ক্রমে ফোনের জন্য দেখায়। মনোযোগ দরকার এমন বিষয় সবার আগে, তারপর সেলস, কাজ, লোকবল, বক্যো ইনভয়েন্স, নোটিশ এবং সবশেষে AI ইনসাইট।

ক্লাস/উইজটে: DashboardScreen

`ConsumerWidget`, মূল স্ক্রিনি। `dashboardSummaryProvider` থেকে ডটো লোড করে `RefreshIndicator`-এর মাধ্যমে পুরো ড্যাশবোর্ড দেখায় — লোডিং, এরর ও ডটো তিন অবস্থাতেই আলাদা UI।

`Widget build(BuildContext context, WidgetRef ref)`

`dashboardSummaryProvider` ওয়াচ করে `async.when` দিয়ে লোডিং/এরর/ডটো হ্যান্ডল করে। পুল-টু-রিফ্রেশে `dashboardSummaryProvider` ও `recommendationsProvider` উভয়ই ইনভ্যালিডেট করে। ডটো এলে `ListView`-এ ক্রমান্বয়ে `_Recommendations`, `_NeedsAttention`, `_Sales`, `_Work`, `_People`, শর্তসাপেক্ষে `_Overdue` ও `_Announcements`, এবং শেষে `_Insights` সাজায়।

ক্লাস/উইজটে: _Recommendations

`ConsumerWidget`। সিস্টেমের সুপারিশগুলো সবচেয়ে জরুরি আগে দেখায়; সুপারিশ খালি থাকলে কিছুই রেন্ডার করে না।

Widget build(BuildContext context, WidgetRef ref)

recommendationsProvider-এর মান পড়ে; null/খালি হলে `SizedBox.shrink()` ফেরত দেয়। বাকগিলোক `isCriticalWarning` বাকি এই ক্রমে সর্ট করে এবং প্রতটি জন্ম `MessageBanner.error/warning/info` দেখায়। মন্তব্যে বলা আছে প্রতটি সুপারিশের সাথে থাকা লিংক একটি Angular রুট, তাই ট্যাপযোগ্য নয়।

ক্লাস/উইজটে: `_NeedsAttention`

`StatelessWidget`। SLA ব্রচি, ছুটি অনুমোদন, নতুন টিকিটি ও বক্যো টাকার কার্ড দেখায়; সব শূন্য থাকলে "সব ঠিক আছে" বার্তা দেখায়।

Widget build(BuildContext context)

`summary`-এর চারটি সংখ্যা (>0 হলে) দিয়ে `StatCard` লিস্ট তৈরি করে। লিস্ট খালি হলে সবুজ চকে আইকন সহ `AppCard` দেখায়; নয়তো `SectionHeader` ও `_Grid`-এ কার্ডগুলো সাজায়।

ক্লাস/উইজটে: `_Sales`

`StatelessWidget`। সেলস-সম্পর্কিত ছয়টি পরিসংখ্যান (লেডি, কোয়ালিফায়ডে, ক্লায়েন্ট, ওপনে ডলি, পাইপলাইন, ফোরকাস্ট) দেখায়।

Widget build(BuildContext context)

`SectionHeader('Sales')` ও `_Grid`-এ ছয়টি `StatCard` সাজায়; কচু কার্ডে `_trend()` দিয়ে ট্রেন্ড শতাংশ suffix হিসেবে দেখায়।

ক্লাস/উইজটে: `_Work`

`StatelessWidget`। চলমান কাজ — অপেক্ষমাণ, চলমান রকোয়েস্ট, ওপনে টিকিটি ও ওয়ালটে ব্যালেন্স দেখায়।

Widget build(BuildContext context)

`SectionHeader('Work in hand')` ও `_Grid`-এ চারটি `StatCard` সাজায়।

ক্লাস/উইজটে: `_People`

`StatelessWidget`। কর্মী সংখ্যা, আজ উপস্থিতি ও ছুটিতে থাকা কর্মীর সংখ্যা দেখায়।

Widget build(BuildContext context)

`SectionHeader('People')` ও `_Grid`-এ তিনটি `StatCard` সাজায়।

ক্লাস/উইজটে: `_Overdue`

`StatelessWidget`। বক্যো ইনভয়সের তালিকা দেখায়, শুধু তখনই রেন্ডার হয় যখন `summary.overdueInvoices` খালি নয়।

Widget build(BuildContext context)

প্রতটি `OverdueInvoice`-এর জন্ম ক্লায়েন্টের নাম, ইনভয়সে নম্বর, কতদিন লেটে এবং লাল রঙে টাকার পরিমাণ দেখায় `AppCard`-এর ভেতরে।

ক্লাস/উইজটে: `_Announcements`

`StatelessWidget`। কোম্পানির নোটিশ/ঘোষণাগুলো দেখায়, খালি থাকলে রেন্ডার হয় না।

Widget build(BuildContext context)

প্রতিটি ঘোষণার শিরোনাম, ঐচ্ছিক কন্টেন্ট (৩ লাইন পর্যন্ত, ellipsis সহ) ও ঐচ্ছিক সময় (timeAgo) দেখায়।

ক্লাস/উইজটে: _Insights

ConsumerStatefulWidget। AI অ্যাসিস্ট্যান্টের পড়া পরিসংখ্যান-বিশ্লেষণ দেখায়। স্ক্রিনি লোড হওয়ার সাথে সাথে না চেয়ে, ব্যবহারকারী বাটনে চাপ দিলে তবই রকিওয়েস্ট পাঠায় — কারণ এটি প্রোভাইডার-নির্ভর একটি খরচযুক্ত কল।

ConsumerState<_Insights> createState()

_InsightsState তৈরি করে।

ক্লাস: _InsightsState

_asked ফ্ল্যাগ দিয়ে ব্যবহারকারী চেয়েছে কিনা তা ট্র্যাক করে।

Widget build(BuildContext context)

_asked মিথ্যা হলে "চান কি অ্যাসিস্ট্যান্ট পড় শোনাক" প্রশ্নসহ LoadingButton দেখায়, চাপলে _asked = true সেট করে। true হলে dashboardInsightsProvider ওয়াচ করে async.when-এ লোডিং/এরর/ডটো দেখায়; ডটো এলে SelectableText দিয়ে বিশ্লেষণ টেক্সট দেখায়।

ক্লাস/উইজটে: _Grid

StatelessWidget। দুই কলামে র‍্যাপ হওয়া স্ট্যাট কার্ডের গ্রিড।

Widget build(BuildContext context)

LayoutBuilder দিয়ে উপলব্ধ প্রস্থ থেকে প্রতিটি কার্ডের প্রস্থ (মোট প্রস্থ - গ্যাপ)/২ হিসাব করে Wrap-এ সাজায়।

টপ-লেভেলে ফাংশন

String? _trend(double value)

value শূন্য হলে null ফেরত দেয়; ন্যূনতম ধনাত্মক হলে + চহিন্সহ ও ঋণাত্মক হলে যমেন আছে তমেন শতাংশ স্ট্রিং (যমেন +12%) তৈরি করে দেয়।

ফাইল: lib/features/dashboard/platform_dashboard_screen.dart

সংক্ষিপ্ত ববিরণ: প্ল্যাটফর্ম-স্তরের ড্যাশবোর্ড, প্রতিটি টেন্যান্ট/কোম্পানিকে বাইরে থেকে দেখানো হয়। শুধুমাত্র প্ল্যাটফর্ম স্টাফ রোলের জন্য গটেডে; একজন টেন্যান্ট ইউজার এই এন্ডপয়েন্ট কল করলে 403 পায়।

ক্লাস/উইজটে: PlatformDashboardScreen

ConsumerWidget, মূল স্ক্রিনি। কোম্পানির সংখ্যা, ট্রায়াল, রাজস্ব ও প্ল্যান অনুযায়ী কোম্পানির বিভাজন দেখায়।

Widget build(BuildContext context, WidgetRef ref)

platformSummaryProvider ওয়াচ করে `async.when` দিয়ে অবস্থা হ্যান্ডল করে। রফিকশে platformSummaryProvider ও platformHistoryProvider ইনভ্যালিডেট করে। ডটো এলে সাত দিনের মধ্যে ট্রায়াল শেষ হওয়ার সতর্কতা, ভেরিফিকেশনের অপেক্ষায় থাকা কোম্পানির তথ্য, "Companies" গ্রুডি (মোট/সক্রিয়/ট্রায়াল/সাসপেন্ড), "Money" গ্রুডি (এই মাস/সর্বকাল), প্ল্যান অনুযায়ী তালিকা, প্ল্যাটফর্ম স্টাফ সংখ্যা এবং `_History` দেখায়।

ক্লাস/উইজটে: `_History`

ConsumerWidget। গত ৩০ দিনের রাতের সন্ধ্যাপশট থেকে ইতিহাস দেখায়, চার্ট নয় বরং সারণী হিসাবে — কারণ সন্ধ্যাপশট কোনো এক সময় থেকে শুরু হয়েছিল, ফলে ফাঁকে রেখে টানলে ভুল তথ্য দেখাত।

Widget build(BuildContext context, WidgetRef ref)

platformHistoryProvider ওয়াচ করে; খালি বা এরর হলে বার্তা দেখায়। ডটো থাকলে সর্বশেষ থেকে শুরু করে ফোনরে পরদায় ধরে এমন সর্বোচ্চ ১৪টি পয়েন্ট (`points.reversed.take(14)`) সারণিতে দেখায় — তারিখ, সক্রিয় কোম্পানি সংখ্যা ও রাজস্ব।

ক্লাস/উইজটে: `_Grid`

StatelessWidget। `dashboard_screen.dart`-এর মতোই, দুই কলামের ব্ল্যাপ-গ্রুডি — এই ফাইলে আলাদাভাবে সংজ্ঞায়িত (প্রাইভেট, তাই পুনঃব্যবহার হয় না)।

Widget build(BuildContext context)

LayoutBuilder দিয়ে উপলব্ধ প্রস্থ অনুযায়ী প্রতিটি চাইল্ডের প্রস্থ গণনা করে `Wrap`-এ সাজায়।

ফাইল: `lib/features/directory/directory_models.dart`

সংক্ষিপ্ত বিবরণ: এমপ্লয় ডিরেক্টরির ডটো মডেলে ও পারমিশন কনস্ট্যান্ট — `Person`, `Department`, এবং কর্মী তৈরি/আপডেটের রকোয়েস্ট ক্লাস। বতেন, ব্যাংক তথ্য ইত্যাদি স্পর্শকাতর ফল্ড ইচ্ছাকৃতভাবে বাদ দেওয়া হয়েছে।

ক্লাস: `DirectoryPermissions`

`abstract final class` — পারমিশন কোডের স্ট্যাটিক কনস্ট্যান্ট রাখার কনটইনার, কোনো ইনস্ট্যান্স তৈরি হয় না।

- `static const employeeView = 'EMPLOYEE_VIEW'` — কর্মী তালিকা দেখার পারমিশন। মন্তব্যে ব্যাখ্যা করা আছে যে API নজি এই চেক করে না, কিন্তু বতেন/ব্যাংক তথ্যও একই এন্ডপয়েন্টে ফেরত আসে বলে অ্যাপ নজি থেকে এই বাধা রেখেছে। - `static const employeeCreate = 'EMPLOYEE_CREATE'` — নতুন কর্মী যোগ করার পারমিশন। - `static const employeeUpdate = 'EMPLOYEE_UPDATE'` — কর্মীর তথ্য সম্পাদনার পারমিশন।

টপ-লেভেলে কনস্ট্যান্ট (তালিকা)

- `const genders` — ব্যাকএন্ডের `Gender enum`-এর মরির: `MALE`, `FEMALE`, `OTHER`, `PREFER_NOT_TO_SAY`। - `const employmentTypes` — `EmploymentType`-এর মরির: `FULL_TIME`, `PART_TIME`, `CONTRACT`, `INTERN`, `CONSULTANT`। - `const employmentStatuses` — `EmploymentStatus`-এর মরির। শেষে চারটি (`SUSPENDED`, `RESIGNED`, `TERMINATED`, `RETIRED`) টার্মিনাল — এসব হচ্ছে নলি `updateEmployeeDetails` কর্মীর `active` ফ্ল্যাগ ও পোর্টাল লগইনও বন্ধ করে দেয়। - `const terminalEmploymentStatuses` — উপরের টার্মিনাল চারটি স্ট্যাটাসের তালিকা।

ক্লাস: Person

একজন সহকর্মীকে ডিরেক্টরতি যেভাবে দেখানো হয় তার মডেল। `GET /api/employees` অনেকে বেশি স্পর্শকাতর ফলিড (বেতন, ব্যাংক অ্যাকাউন্ট, জাতীয় পরিচয়পত্র ইত্যাদি) দেয়, কিন্তু এই ক্লাস ইচ্ছাকৃতভাবে সেগুলো ম্যাপ করে না।

কনস্ট্রাক্টর `Person({required id, required firstName, required lastName, ...})` — বাকসিবি ফলিড ঐচ্ছিক (নাল-যোগ্য), যেমন `employeeNumber`, `jobTitle`, `designationName`, `departmentId`, `departmentName`, `email`, `officialEmail`, `phone`, `workPhone`, `reportingManagerId`, `reportingManagerName`, `shiftName`, `hireDate`, `employmentStatus`, `employmentType`, `officeLocation`, `imageUrl`।

`String get fullName`

`firstName` ও `lastName` স্পেসে দিয়ে যুক্ত করে ট্রিম করে ফেরত দেয়।

`String? get roleLabel`

নামের নচি কী দেখানো হবে তা ঠিক করে: প্রথমে `jobTitle` (ট্রিম করে খালি না হলে), না থাকলে `designationName` ফেরত দেয়; দুটোই খালি হলে `null`।

`String? get bestEmail`

`officialEmail` (কাজের ইমেইল) থাকলে সেটি অগ্রাধিকার পায়, নয়তো ব্যক্তিগত `email` ফেরত দেয়।

`String? get bestPhone`

`workPhone` অগ্রাধিকার পায়, নয়তো ব্যক্তিগত `phone` ফেরত দেয়।

`bool get isFormer`

`employmentStatus` `TERMINATED`, `RESIGNED` বা `INACTIVE` হলে `true` — অর্থাৎ কটে প্রতিষ্ঠান ছড়ে গেছে।

`String get initials`

`firstName` ও `lastName`-এর প্রথম অক্ষর বড় হাতের করে যুক্ত করে দেয়; দুটোই খালি হলে '?', একটি খালি হলে অন্যটির প্রথম অক্ষর ফেরত দেয়।

`factory Person.fromJson(Map<String, dynamic> json)`

JSON থেকে `Person` তৈরি করে; সংখ্যা ফলিডগুলো `num?` থেকে `int/double`-এ কনভার্ট করে, `officeLocation` `officeLocation` অথবা লগিয়াস `location` কী থেকে নেয়, এবং `imageUrl` `profileImageUrl` বা `image` কী থেকে `Env.resolveImageUrl()` দিয়ে রজিলভ করে।

ক্লাস: Department

একটি বিভাগের তথ্য — একই ক্লাস দুটি ভিন্ন পলোড (সংক্ষিপ্ত ফলিটার ও বিস্তারিত অ্যাডমনি) হ্যান্ডেল করে, না-থাকা ফলিডগুলো নাল-যোগ্য।

কনস্ট্রাক্টর `Department({required id, required name, code, employeeCount, description, active = true, budget, parentDepartmentId, parentDepartmentName, headEmployeeId, headEmployeeName})`।

factory `Department.fromJson(Map<String, dynamic> json)`

JSON থেকে `Department` তৈরি করে; `active` ফিল্ড না থাকলে ডিফল্ট `true` ধরে নেবে (কারণ ফিল্ডারে দেখানো বিভাগ মানই তা এখনো বদ্বিমান)।

ক্লাস: `CreateEmployeeRequest`

`POST /employees`-এর জন্য রকি়োস্ট মডলে — একই সাথে ব্যক্তিও তার লগইন তৈরি করে, তাই পাসওয়ার্ড আবশ্যিক।

কনস্ট্রাক্টর: আবশ্যিক `firstName`, `lastName`, `email`, `password`, `employmentType`; ঐচ্ছকি `jobTitle`, `workPhone`, `employmentStatus`, `departmentId`, `reportingManagerId`, `gender`, `dateOfBirth`, `hireDate`, `officeLocation`।

`Map<String, dynamic> toJson()`

আবশ্যিক ফিল্ডগুলো ট্রমি করে (পাসওয়ার্ড বাদে — এটকিখনো ট্রমি হয় না, কারণ স্পসেও পাসওয়ার্ডের অংশ হতে পারে) সবসময় পাঠায়; ঐচ্ছকি ফিল্ডগুলো ভেতরে `clean()` হলেপার দিয়ে খালি স্ট্রিং হলে বাদ দিয়ে শুধু নাল-নয় মান থাকলই যোগ করে।

ক্লাস: `UpdateEmployeeRequest`

`PATCH /employees/{id}`-এর জন্য স্পার্স-প্যাচ রকি়োস্ট — যা পাঠানো হয় না তা অক্সুণ থাকে।

কনস্ট্রাক্টর: সবগুলো ফিল্ড ঐচ্ছকি — `jobTitle`, `employmentType`, `employmentStatus`, `workPhone`, `officeLocation`, `departmentId`, `reportingManagerId`, `hireDate`।

factory `UpdateEmployeeRequest.from(Person person)`

একটি বদ্বিমান `Person`-এর বর্তমান মান থেকে একটি প্রি-ফিল্ড `UpdateEmployeeRequest` তৈরি করে দেয় (এডটি ফর্মের প্রাথমিক মান হিসেবে ব্যবহারের জন্য)।

`Map<String, dynamic> toJson()`

ভেতরে `clean()` হলেপার দিয়ে ট্রমি-খালি স্ট্রিং ফিল্ড বাদ দিয়ে; শুধু নাল-নয় মানগুলোই JSON ম্যাপে যোগ করে, ফলে না-বদলানো ফিল্ড আদৌ পাঠানো হয় না।

ফাইল: `lib/features/directory/directory_repository.dart`

সংক্ষিপ্ত ববিরণ: এমপ্লয়ি ডিরেক্টরির জন্য API রপি়োজিটরিও Riverpod প্রোভাইডারগুলো — তালিকা, PDF এক্সপোর্ট, তৈরি/সম্পাদনা এবং সার্চ/ফিল্টার স্টেটে। সন্ডে করা যায় এমন ফিল্ড ইচ্ছাকৃতভাবে সীমতি রাখা হয়েছে, যনে `Person` মডলে যা দেখতে পারে তার বাইরে কিছু এডটি করার চেষ্টা না হয়।

ক্লাস: `DirectoryRepository`

কোম্পানির কর্মীদের নিয়ে API কল করার মূল রপি়োজিটরিক্লাস।

কনস্ট্রাক্টর `DirectoryRepository(this._api)` — `ApiClient` ইনজেক্ট করে।

Future<PagedResponse<Person>> people({String? search, int? departmentId, int page = 0, int size = 20})

/employees-এ পজেড GET কল করে Person.fromJson দিয়ে ডিসিরিযালাইজ করে। কুয়েরিতে search, departmentId ও সবসময় excludeOwner: true পাঠায় — কোম্পানির মালিককে সাধারণ কর্মী না ধরে তালিকার শীর্ষে খালিসার হিসেবে দেখানো এড়াতো।

Future<List<int>> peoplePdf({String? search, int? departmentId})

একই ফিল্টার (সহ excludeOwner: true) দিয়ে /employees/pdf-এ বাইট রকিওয়েস্ট করে PDF ডটো আনে, যাতো প্রিন্ট করা তালিকা ও স্ক্রিনিং তালিকা মিলে যায়।

Future<Person> person(int id)

/employees/\$id থেকে একটমাত্র কর্মীর সম্পূর্ণ রেকর্ড আনে এবং Person.fromJson দিয়ে পার্স করে।

Future<Person> create(CreateEmployeeRequest request)

/employees-এ POST করে নতুন কর্মী (ও তার লগইন) তৈরি করে; ডুপ্লিকেটে ইমেইল হলে ব্যাকএন্ড এরর মেসেজসহ প্রত্যাখ্যান করে।

Future<Person> update(int id, UpdateEmployeeRequest request)

/employees/\$id-এ PATCH করে UpdateEmployeeRequest-এ থাকা ফিল্ডগুলো আপডেট করে।

Future<List<Department>> departments()

/departments/active থেকে সক্রিয় বিভাগের তালিকা আনে; ApiException হলে খালি তালিকা ফেরত দিয়ে (ফিল্টার অনুপস্থিতি হলে চলবে, তাই ব্যর্থতা মারাত্মক নয়)। খালি নাম থাকা বিভাগও বাদ দিয়ে।

টপ-লেভেলে প্রোভাইডার

final directoryRepositoryProvider

ApiClient থেকে DirectoryRepository-এর একক ইনস্ট্যান্স দিয়ে।

ক্লাস: DirectorySearchController

Notifier<String> — টাইপ করা সার্চ টার্মের স্টেটে আলাদাভাবে রাখা, যাতো লস্ট কন্ট্রোলার টেক্সট ফিল্ডকে না ছুঁয়ে রিবিল্ড করতে পারে।

String build()

প্রাথমিক অবস্থা হিসেবে খালি স্ট্রিং ফেরত দিয়ে।

void set(String value)

মানটি ট্রিম করে; নতুন মান আগের মতোই হলে কিছু করে না, নয়তো স্টেটে আপডেট করে।

টপ-লেভেলে প্রোভাইডার

final directorySearchProvider

DirectorySearchController চালতি NotifierProvider<..., String>।

ক্লাস: DepartmentFilterController

Notifier<int?> — তালিকা কোন বিভাগে সীমাবদ্ধ তা রাখা; null মান সবাই।

int? build()

প্রাথমিক অবস্থা null।

```
void set(int? departmentId)
```

মান আগরে মতোই হলে কিছু করে না, নয়তো আপডেট করে।

টপ-লেভেলে প্রোভাইডার

```
final departmentFilterProvider
```

DepartmentFilterController চালতি প্রোভাইডার।

ক্লাস: DirectoryController

AsyncNotifier<PagedState<Person>> যা PagedLoader<Person> মক্সইন ব্যবহার করে — পজেড ডিরেক্টর তালিকার প্রধান কন্ট্রোলার।

```
Future<PagedState<Person>> build()
```

currentUserProvider, directorySearchProvider ও departmentFilterProvider ওয়াচ করে (এদের যেকোনো পরিবর্তনে রিবিল্ড হয়) এবং loadFirstPage() কল করে প্রথম পজে লোড করে।

```
Future<PagedResponse<Person>> fetchPage(int page)
```

বর্তমান সার্চ টার্ম (খালি হলে null) ও বিভাগ ফিল্টার দিয়ে directoryRepositoryProvider.people() কল করে নির্দিষ্ট পজে আনে।

```
Future<Person> create(CreateEmployeeRequest request)
```

রপিজিটরির create() কল করে, তারপর সম্পূর্ণ refresh() করে (নতুন এন্ট্রি ম্যানুয়ালি insert না করে) — কারণ অ্যালফাবেটিক্যাল/ফিল্টার্ড তালিকায় নতুন কর্মী কোথায় বসবে তা সার্ভারই ঠিকি করে।

```
Future<Person> updateItem(int id, UpdateEmployeeRequest request)
```

রপিজিটরির update() কল করে, তালিকায় সংশ্লিষ্ট আইটেমে replaceItem() দিয়ে প্রতিস্থাপন করে এবং personProvider(id) ইনভ্যালিডেট করে।

টপ-লেভেলে প্রোভাইডার

```
final directoryProvider
```

DirectoryController চালতি AsyncNotifierProvider।

```
final departmentsProvider
```

currentUserProvider ওয়াচ করে departments() কল করা FutureProvider<List<Department>>।

```
final personProvider
```

id অনুযায়ী FutureProvider.autoDispose.family<Person, int> — ডিটেইল স্ক্রিনিংর জন্য একজন কর্মীর সম্পূর্ণ রেকর্ড আনে।

ফাইল: lib/features/directory/directory_screen.dart

সংক্ষিপ্ত বিবরণ: কর্মী ডিরেক্টর স্ক্রিনি — সার্চ, বিভাগ ফিল্টার, পজেড তালিকা, PDF এক্সপোর্ট এবং নতুন কর্মী যোগ করার FAB। EMPLOYEE_VIEW পারমিশনের ওপর গটেডে।

ক্লাস/উইজটে: DirectoryScreen

ConsumerWidget, মূল স্ক্রিনি।

Widget build(BuildContext context, WidgetRef ref)

permissionControllerProvider ওয়াচ করে। পারমিশন এখনো লোড না হলে Loader দেখায়;

EMPLOYEE_VIEW না থাকলে লক আইকনসহ EmptyState দেখায় ("Not available to you")। অন্যথায় পুরো ডিরেক্টরিস্ক্রিনি তৈরিকরে — PDF এক্সপোর্ট বাটন সহ AppBar, EMPLOYEE_CREATE থাকলে "New employee" FAB, এবং বডিতে _SearchField, _DepartmentFilter ও PagedListView<Person>।

ক্লাস/উইজটে: _SearchField

ConsumerWidget। নাম দিয়ে সার্চ করার টেক্সট ফিল্ড।

Widget build(BuildContext context, WidgetRef ref)

AppSearchField দেখায়; onChanged-এ যা নজিই ডিবাউন্সড)

directorySearchProvider.notifier.set(value) কল করে।

ক্লাস/উইজটে: _DepartmentFilter

ConsumerWidget। অনুভূমিক স্ক্রলযোগ্য বিভাগ-চপিরে স্ট্রপি।

Widget build(BuildContext context, WidgetRef ref)

departmentsProvider থেকে বিভাগের তালিকা পড়ে; খালিহলে কিছু দেখায় না। প্রথম আইটেমে হিসিবে "Everyone" চপি (ফিল্টার null করে) এবং বাকগুলোর জন্য প্রতিটি বিভাগের নাম ও কর্মী সংখ্যাসহ চপি দেখায়; ট্যাপে ঐ বিভাগ নির্বাচতি হলে টগল করে null করে দেয়।

ক্লাস/উইজটে: _Chip

StatelessWidget। নির্বাচতি/অনির্বাচতি অবস্থার স্টাইলসহ একটি ফিল্টার চপি।

Widget build(BuildContext context)

active হলে ব্র্যান্ড রঙ ও সাদা টেক্সট, নয়তো কার্ড ব্যাকগ্রাউন্ড ও সকেন্ডারি টেক্সট রঙে InkWell+Container রেন্ডার করে।

ক্লাস/উইজটে: _PersonRow

StatelessWidget। তালিকায় একজন কর্মীর একটি সারি।

Widget build(BuildContext context)

Avatar (ইনশিয়াল/ছবি), নাম, প্রাক্তন কর্মী হলে "Left" StatusChip, roleLabel, departmentName এবং একটি শিভেরন আইকন দেখায়; ট্যাপে PersonDetailScreen.open() কল করে।

টপ-লেভেলে ফাংশন

Future<void> _exportPdf(BuildContext context, WidgetRef ref)

বর্তমান সার্চ/ফিল্টার দিয়ে peoplePdf() কল করে PDF বাইট আন, টেম্পোরারি ডিরেক্টরিতে ফাইল হিসিবে লখে এবং OpenFilex.open() দিয়ে খোলে। কোনো অ্যাপ PDF খুলতে না পারলে বা ApiException/অন্য এরর হলে SnackBar-এ বার্তা দেখায়।

ফাইল: lib/features/directory/education_section.dart

সংক্ষিপ্ত ববিরণ: প্রোফাইল স্ক্রিনিং একটি সিকেশন যা কারও শিক্ষাগত যোগ্যতা (ডিগ্রি) তালিকাভুক্ত, যোগ, সম্পাদনা ও মুছে ফেলার সুবিধা দেয়। নিজের রেকর্ডে যে কডে, অন্যেরটা EMPLOYEE_UPDATE পারমিশন লাগে।

ক্লাস: Qualification

একটি ডিগ্রি/যোগ্যতার তথ্য, EducationQualificationResponse-এর মরি।

কনস্ট্রাক্টর Qualification({required id, required degree, institution, fieldOfStudy, passingYear, result, notes})।

factory Qualification.fromJson(Map<String, dynamic> json)

JSON থেকে Qualification তৈরি করে, সংখ্যা ফিল্ড নিরাপদে কনভার্ট করে।

ক্লাস: QualificationRequest

একটি যোগ্যতা তৈরি/আপডেটের জন্য রকিওয়েস্ট মডেল। প্রতিটি ফিল্ড সবসময় পাঠানো হয়, তাই বাদ দেওয়া মানে মুছে ফেলো।

কনস্ট্রাক্টর QualificationRequest({required employeeId, required degree, institution, fieldOfStudy, passingYear, result, notes})।

Map<String, dynamic> toJson()

সবগুলো ফিল্ড (নাল হলও) সরাসরি JSON ম্যাপে রাখে — কোনো clean()/শর্তসাপেক্ষ বাদ দেওয়া নেই।

ক্লাস: EducationRepository

শিক্ষাগত যোগ্যতা API-এর রপিজিটরি

কনস্ট্রাক্টর EducationRepository(this._api)।

স্ট্যাটিক কনস্ট্যান্ট _base = '/hr/education-qualifications'।

Future<List<Qualification>> forEmployee(int employeeId)

\$_base/employee/\$employeeId-এ GET করে, ব্যাকএন্ডের ক্রম (নতুন আগে) অনুযায়ী Qualification তালিকা ফেরত দেয়।

Future<Qualification> create(QualificationRequest request)

_base-এ POST করে নতুন যোগ্যতা তৈরি করে।

Future<Qualification> update(int id, QualificationRequest request)

\$_base/\$id-এ PUT করে বিদ্যমান যোগ্যতা আপডেট করে।

Future<void> delete(int id)

\$_base/\$id-এ DELETE কল করে।

টপ-লেভেলে প্রোভাইডার

final educationRepositoryProvider

ApiClient থেকে EducationRepository-এর একক ইনস্ট্যান্স দেয়।

final qualificationsProvider

employeeId অনুযায়ী FutureProvider.autoDispose.family<List<Qualification>, int> — নির্দিষ্ট কর্মীর যোগ্যতা তালিকা আনবে।

ক্লাস/উইজটে: EducationSection

ConsumerStatefulWidget। প্রোফাইল স্ক্রিনে বসানো শিক্ষা সেকশন।

কনস্ট্রাক্টর EducationSection({super.key, required this.employeeId})।

ConsumerState<EducationSection> createState()

_EducationSectionState তৈরি করে।

ক্লাস: _EducationSectionState

সেকশন স্টেটে ও লজিক পরিচালনা করে; _busy ফ্ল্যাগ দিয়ে চলমান অপারেশন ট্র্যাক করে।

Future<void> _delete(Qualification qualification)

confirmAction() দিয়ে নিশ্চিতকরণ ডায়ালগ দেখায়; নিশ্চিত হলে _busy = true স্টেটে করে ডিলিট API কল করে, qualificationsProvider ইনভ্যালিডেট করে এবং সফল/ব্যর্থ বার্তা SnackBar-এ দেখায়; finally-তে _busy = false করে।

Widget build(BuildContext context)

qualificationsProvider(employeeId) ওয়াচ করে। canEdit নির্ধারণ করে হয় EMPLOYEE_UPDATE পারমিশন থাকে অথবা এটি নিজের রেকর্ড কনি (myEmployeeProvider) তা থাকে। SectionHeader-এ এডটি অনুমতি থাকলে "Add" বাটন, এবং AppCard-এ লোডিং/এরর/খালি/তালিকা অবস্থায় যথাযথ কন্টেন্ট (_QualificationRow তালিকা) দেখায়।

ক্লাস/উইজটে: _QualificationRow

StatelessWidget। একটি যোগ্যতার সারি — ডিগ্রি, বিষয়, প্রতিষ্ঠান, বছর, ফলাফল, নোট এবং এডটি অনুমতি থাকলে এডটি/রিমুভ মেনু।

Widget build(BuildContext context)

facts তালিকা (প্রতিষ্ঠান, বছর, ফলাফল — যগুলো নাল নয়) তৈরি করে ড্যাশ দিয়ে জোড়া লাগায়। ডিগ্রি ও বিষয় কমা দিয়ে যুক্ত করে দেখায়, নোট থাকলে আলাদা লাইনে দেখায়। canEdit হলে PopupMenuButton দিয়ে "Edit" (শীট খোলে) ও "Remove" (লাল রঙে, onDelete কল করে) অপশন দেয়।

টপ-লেভেলে ফাংশন

Future<void> showQualificationSheet(BuildContext context, {required int employeeId, Qualification? qualification})

showModalBottomSheet দিয়ে _QualificationSheet খোলে — qualification দিলে এডটি মোড, না দিলে নতুন যোগ করার মোড।

ক্লাস/উইজটে: _QualificationSheet

ConsumerStatefulWidget। যোগ্যতা তৈরি/সম্পাদনার ফর্ম শীট।

কনস্ট্রাক্টর _QualificationSheet({required this.employeeId, this.qualification})।

ConsumerState<_QualificationSheet> createState()

_QualificationSheetState তৈরি করে।

ক্লাস: `_QualificationSheetState`

ফর্মের স্টেটে, টেক্সট কন্ট্রোলার ও সাবমিট লজিক ধারণ করে। ছয়টি `TextEditingController` (`_degree`, `_institution`, `_field`, `_year`, `_result`, `_notes`) বদ্যমান `qualification`-এর মান দিয়ে প্রাথমিকভাবে পূরণ হয়।

`void dispose()`

ছয়টি কন্ট্রোলারই লুপে `dispose()` করে, তারপর `super.dispose()` কল করে।

`String? _clean(TextEditingController controller)`

কন্ট্রোলারের টেক্সট ট্রিম করে; খালি হলে `null`, নয়তো ট্রিম করা টেক্সট ফেরত দেয়।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে; ব্যর্থ হলে থমে যায়। `_busy = true`, `_error = null` স্টেটে করে `QualificationRequest` তৈরি করে (বছর `int.tryParse` দিয়ে পার্স করে)। `qualification` নাল হলে `create()`, নয়তো `update()` কল করে; সফল হলে `qualificationsProvider` ইনভ্যালিডেট করে শীট বন্ধ করে। `ApiException`-এ এরর মैसेজ দেখায়, অন্য এরর হলে জনৈকি বার্তা; `finally`-তে `_busy = false` করে।

`Widget build(BuildContext context)`

`FormSheetFrame` দিয়ে শীর্ষক ("Add a qualification"/"Edit qualification"), এরর, সাবমিট-বটন লবেলে "Save" ও `onSubmit`: `_submit` স্টেটে করে। ফর্মে ডিগ্রি (আবশ্যিক, ভ্যালিডেটের সহ), বিষয়, প্রতিষ্ঠান, বছর (৪-সংখ্যার, ১৯০০-২১০০ রেঞ্জ ভ্যালিডেশন সহ) ও ফলাফল (পাশাপাশি সারতি), নোট (২ লাইন) ফিল্ড এবং একটি সিতরুকীকরণ টেক্সট দেখায় যা খালি বক্স ও যা ছিল তা মুছে দেবে।

ফাইল: `lib/features/directory/employee_form_sheet.dart`

সংক্ষিপ্ত বিবরণ: এই ফাইলে একটি বিটম-শিট ফর্ম আছে যা দিয়ে নতুন কর্মচারী যোগ করা যায় বা বদ্যমান কর্মচারীর তথ্য এডিট করা যায়। নতুন তৈরির সময় নাম, ইমেইল, পাসওয়ার্ড ইত্যাদি "identity" ফিল্ড দেখায়, কিন্তু এডিটের সময় সেগুলো লুকিয়ে রাখে কারণ ব্যাকএন্ড এন্ডপয়েন্ট সেগুলো পরবর্তন করতে পারে না।

টপ-লেভেলে ফাংশন

`Future<void> showNewEmployeeSheet(BuildContext context)`

একটি নতুন সহকর্মী যোগ করার জন্য মডাল বিটম-শিট খোলে, যেখানে তার লগইন পাসওয়ার্ডও স্টেট করা হয়। `_EmployeeFormSheet`-কে `existing` ছাড়াই (অর্থাৎ `create` মোডে) বন্ডি করে।

`Future<void> showEditEmployeeSheet(BuildContext context, Person person)`

বদ্যমান একজন কর্মচারীকে এডিট করার শিট খোলে। এটুকিছু রিটার্ন করে না — ডিটেইল স্ক্রিনি `personProvider` ওয়াচ করে, যা সফল আপডেটের পর কন্ট্রোলার ইনভ্যালিডেট করে দেয়, ফলে স্ক্রিনি নজি থেকেই রিফ্রেশ হয়।

ক্লাস/উইজটে: `_EmployeeFormSheet`

`ConsumerStatefulWidget`, কনস্ট্রাক্টরে ঐচ্ছকি `existing` `Person` নেয়। `existing` থাকলে এডিট মোড, না থাকলে ক্রিয়েটে মোড হিসেবে কাজ করে; `createState()` দিয়ে `_EmployeeFormSheetState` তৈরি করে।

ক্লাস: `_EmployeeFormSheetState`

এই স্টেটে ক্লাসটি ফর্মের সব টেক্সট কন্ট্রোলার, ড্রপডাউন সলিকেশন, ভ্যালিডেশন এবং সাবমিট লজিক পরিচালনা করে। নতুন কর্মচারী তৈরির সময় পাসওয়ার্ড/কনফার্ম পাসওয়ার্ড ফিল্ড শুধু তৈরির সময় ব্যবহৃত হয় — কখনও প্র-ফিলি করা হয় না, ফরম পড়া হয় না বা রাখা হয় না। কর্মচারীর স্ট্যাটাস এমন কচ্ছিতে বদলানো হলে যা তাকে "টার্মিনিটে" করে দেয় (যেমন `resigned/terminated`), স্টেট নিশ্চিত করার জন্য একটি ডায়ালগ দেখায়।

`bool get _isEdit`

`widget.existing` নাল না হলে `true` রিটার্ন করে — অর্থাৎ এটি এডটি মোডে আছে কিনা তা বোঝায়।

`void initState()`

কন্ট্রোলারগুলো `widget.existing`-এর মান দিয়ে (থাকলে) প্র-ফিলি করে ইনশিয়ালাইজ করে, অথবা খালি স্ট্রিং দিয়ে শুরু করে। এমপ্লয়মেন্ট টাইপ, স্ট্যাটাস, ডিপার্টমেন্ট আইডি এবং হায়ার-ডেট-ও সেট করে দেয়।

`void dispose()`

সব `TextEditingController` (পাসওয়ার্ড ও কনফার্ম পাসওয়ার্ড সহ) `dispose` করে মেমরালিকি এড়ায়।

`Future<void> _pickManager()`

`EmployeePicker.show` দিয়ে একজন কলিগি বেছে নেওয়ার UI দেখায় ("Reports to" টাইটলেসহ); বাছাই করা ব্যক্তিকে `_manager`-এ সেট করে `setState` কল করে।

`bool get _willDeactivate`

এডটি মোডে থাকা অবস্থায়, নির্বাচতি `_employmentStatus` যদি `terminalEmploymentStatuses`-এর মধ্যে থাকে এবং বর্তমান স্ট্যাটাস থেকে ভিন্ন হয়, তাহলে `true` রিটার্ন করে। ব্যাকএন্ড এমন স্ট্যাটাসে `active` ফ্ল্যাগ ক্লিয়ার করে এবং পোর্টাল লগইন বন্ধ করে দেয় বলে, এটি ব্যবহারকারীকে সতর্ক করার জন্য ব্যবহৃত হয়।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, `_willDeactivate true` হলে একটি নিশ্চিতকরণ ডায়ালগ দেখায় (Cancel/Continue)। এরপর `directoryProvider.notifier` দিয়ে এডটি মোডে `updateItem` অথবা ক্রিয়েটে মোডে `create` কল করে। সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়; `ApiException` এলে সেই মैसेজ দেখায় (যেমন ডুপ্লিকেটে ইমেইল), অন্য এরর হলে জেনেরিক মैसेজ দেখায়। শেষে `_submitting` ফ্ল্যাগ রিসেট করে।

`Widget build(BuildContext context)`

পুরো ফর্মের UI তৈরি করে — শিরোনাম, এরর ব্যানার, (শুধু ক্রিয়েটে মোডে) নাম/ইমেইল/পাসওয়ার্ড/জন্মতারিখ ফিল্ড, এবং (উভয় মোডে) জব টাইটলে, এমপ্লয়মেন্ট টাইপ, স্ট্যাটাস, ডিপার্টমেন্ট, ম্যানজোর, ফোন, অফিস লোকেশন ও হায়ার ডেট। এডটি মোডে পের/ব্যাংক/ইমার্জেন্সি কন্টাক্ট এই অ্যাপ থেকে এডটি করা যায় না তা জানানো নোট দেখায়, এবং শেষে `LoadingButton` দিয়ে সাবমিট বাটন রাখে।

ফাইল: `lib/features/directory/person_detail_screen.dart`

সংক্ষিপ্ত বিবরণ: একজন কর্মচারীর প্রোফাইল দেখানোর স্ক্রিনি। যে row থেকে ওপেন করা হয়েছিল তার সংক্ষিপ্ত ডেটা দিয়ে সাথে সাথেই দেখানো শুরু করে, এবং পূর্ণাঙ্গ রেকর্ড লোড হলে তা আপগ্রেডে করে দেখায়।

ক্লাস/উইজটে: PersonDetailScreen

ConsumerWidget, কনস্ট্রাক্টরে person (Person) নেয়। এটি ডিটেইল স্ক্রিনিংর প্রধান উইজটে যা হডোর, কন্টাক্ট অ্যাকশন, ডিটেইলস, অফবোর্ডিং স্ট্যাটাস, ধারণকৃত অ্যাসেট এবং শিক্ষাগত তথ্য সেকশন একত্রে সাজায়।

static void open(BuildContext context, {required Person person})

Navigator.push দিয়ে MaterialPageRoute-এর মাধ্যমে PersonDetailScreen খোলার স্ট্যাটিক হেল্পার মথড।

Widget build(BuildContext context, WidgetRef ref)

personProvider(person.id) ওয়াচ করে পূর্ণাঙ্গ রেকর্ড আনে; না এলে পুরনো person (row) দেখায়। DirectoryPermissions.employeeUpdate পারমিশন থাকলে এবং পূর্ণ রেকর্ড লোড হয়ে গেলে এডটি বাটন দেখায় (আংশিক রেকর্ড দিয়ে এডটি ফর্ম সডি করা এড়াত)। এরর হলে MessageBanner.info দেখায়, এবং একাধিক সাব-সেকশন (_Header, _ContactActions, _Details, _Offboarding, _HeldAssets, EducationSection) সাজিয়ে দেয়।

ক্লাস: _Header

StatelessWidget, ব্যক্তির অ্যাভাটার, পুরো নাম, রোল লবেলে, ডিপার্টমেন্ট চপি এবং (প্রাক্তন কর্মচারী হলে) সতর্কীকরণ ব্যানার দেখায়।

Widget build(BuildContext context)

Avatar, নাম, রোল লবেলে ও StatusChip (ডিপার্টমেন্ট নামসহ) দেখায়। ব্যক্তি প্রাক্তন কর্মী (isFormer) হলে একটি সতর্কীকরণ ব্যানার দেখায় যাতো বলা হয় তার তথ্য পুরনো হতে পারে।

ক্লাস: _ContactActions

"কল করা" এবং "ইমেইল পাঠানো" — এই দুটি অ্যাকশন বোতাম হিসেবে দেখায়, যা ডিরেক্টরির মূল উদ্দেশ্য।

Future<void> _launch(BuildContext context, Uri uri, String failure)

launchUrl দিয়ে URI (tel: বা mailto:) খোলার চেষ্টা করে; ব্যর্থ হলে বা এক্সপেশন ধরা পড়লে (যেমন এমুলেটরে ডায়ালার/মাইল অ্যাপ না থাকলে) failure মেসেজসহ সন্যাকবার দেখায়।

Widget build(BuildContext context)

person.bestPhone এবং person.bestEmail চকে করে; দুটোই না থাকলে "No contact details" ইনফো কার্ড দেখায়, নাহলে Call/Email বাটন সারি হিসেবে দেখায়।

ক্লাস: _ActionButton

StatelessWidget, একটি আইকন, লবেলে এবং onTap কলব্যাক নিয়ে একটি FilledButton.icon তৈরি করে।

Widget build(BuildContext context)

পূর্ণ প্রস্থের FilledButton.icon রেন্ডার করে যেখানে ন্যূনতম উচ্চতা ৪৬।

ক্লাস: _Details

person এবং loading নিয়ে কাজ করে; ব্যক্তির বিভিন্ন তথ্য (ইমেইল, ফোন, ডিপার্টমেন্ট, ডিজিটাইজেশন, রপোর্টিং ম্যানেজার, অফিস, শফিট, কর্মচারী নম্বর, এমপ্লয়মেন্ট টাইপ, জয়নেং ডটে) সারি আকারে দেখায়।

Widget build(BuildContext context)

একটরিকেরড-টাইপ লিস্ট `rows` তৈরিকরে, শুধু নন-নাল/নন-এম্পটি মানগুলো ফিল্টার করে রাখো। কোনো রো না থাকলে লোডিং অবস্থায় `Loader` অথবা খালি `SizedBox` দেখায়; নাহলে `SectionHeader` ও `AppCard`-এর মধ্যে প্রতটি রো আইকন, লবেল, মান সহ দেখায় (মাঝে ডিভাইডার সহ)। লোডিং চলাকালীন নচি একটি অতিরিক্ত `Loader` দেখায়।

ক্লাস: `_Offboarding`

`ConsumerWidget`, কাউকে কোম্পানি ছেড়ে যাওয়ার প্রক্রিয়া (অফবোর্ডিং চকেলস্টি) চলছে কিনা তা দেখানোর জন্য একটি নীরব নোট দেখায়। বশেরিভাগ কর্মীর ক্ষেত্রে এটি 404 হয় (তারা ছেড়ে যাচ্ছেন না), তাই ব্যর্থতাকে এরর হিসেবে না দেখিয়ে চুপচাপ কিছুই দেখায় না।

Widget build(BuildContext context, WidgetRef ref)

`offboardingForEmployeeProvider(employeeId)` ওয়াচ করে চকেলস্টি আনছে; নাল হলে খালি উইজটে দেয়। থাকলে একটি ট্র্যাপযোগ্য `AppCard` দেখায় যা সম্পন্ন/চলমান স্ট্যাটাস, কমপ্লিশন শতাংশ দেখায় এবং ট্র্যাপে `OffboardingDetailScreen.open` কল করে।

ক্লাস: `_HeldAssets`

`ConsumerWidget`, এই মুহূর্তে ব্যক্তির কাছে থাকা IT অ্যাসেটগুলো দেখায় — ঐতিহাসিকি ডটো নয়, শুধু বর্তমান হোল্ডিং।

Widget build(BuildContext context, WidgetRef ref)

`assetsForEmployeeProvider(employeeId)` ওয়াচ করে; পারমিশন না থাকলে বা তালিকা খালি হলে কিছু দেখায় না। তালিকা থাকলে `SectionHeader` ও `AppCard`-এর মধ্যে প্রতটি অ্যাসেটের নাম ও (থাকলে) ট্র্যাগ দেখায়।

ফাইল: `lib/features/finance/edit_expense_sheet.dart`

সংক্ষিপ্ত বিবরণ: এই ফাইলে দুটি বিটম-শিট আছে — একটি এখনো সন্ধানিত না হওয়া (পেন্ডিং) খরচের দাবি এডটি করার জন্য, আরেকটি অনুমোদিত দাবিকে "রহিম্বার্সড" (পরিশোধিত) হিসেবে চহিনতি করার জন্য।

টপ-লেভেলে ফাংশন

`Future<void> showEditExpenseSheet(BuildContext context, {required Expense expense})`

`_EditExpenseSheet`-কে মডাল বিটম-শিট হিসেবে খোল, যেখানে `expense` পুরোপুরি পাস করা হয় — কারণ ব্যাকএন্ড এন্ডপয়েন্ট যা ফর্ম দেখনো হয় না (chart-of-accounts row, রসিটি, কারন্স) তাও পাঠাতে হয়, নাহলে সেগুলো ক্লিয়ার হয়ে যাবে।

ক্লাস/উইজটে: `_EditExpenseSheet`

`ConsumerStatefulWidget`, কনস্ট্রাক্টরে `required Expense expense` নেয়; `_EditExpenseSheetState` তৈরিকরে।

ক্লাস: `_EditExpenseSheetState`

টাইটলে, বর্ণনা, পরমাণ, ভন্ডর, ক্যাটাগরি, নোট ও তারখিরে কন্ট্রোলার/স্টেটে রাখে এবং এডটি সাবমিট করে।

`void dispose()`

সব টেক্সট কন্ট্রোলার (`_title`, `_description`, `_amount`, `_vendor`, `_category`, `_notes`) dispose করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করার পর `expensesProvider.notifier.edit` কল করে, যখন

`UpdateExpenseRequest.from(widget.expense)`-কে বেসে হিসেবে নিয়ে `copyWith` দিয়ে ফর্মের নতুন মানগুলো বসায় (বাকগুলো আগের মতোই থাকে)। সফল হলে শিট পপ করে; `ApiException` ধরা পড়লে (যেমন কটে ইতিমধ্যে সন্ধানিত নিয়ে ফলেছে) সেই মসেজে দেখায়, অন্যথায় জনেরেকি এরর দেখায়। শেষে `_busy` রসিটে করে।

`Widget build(BuildContext context)`

`FormSheetFrame`-এর মধ্যে টাইটলে, বর্ণনা ("What it was for"), পরিমাণ ও তারিখ (একই সারতি), ভেন্ডর, ক্যাটাগরি, নোট ফিল্ড দেখায়। খরচটি যদি ইতিমধ্যে কোনো অ্যাকাউন্টে পোস্ট হয় বা রসিটি থাকে, তার একটি অপ্রবর্তনীয় নোট দেখায়।

টপ-লেভেলে ফাংশন

`Future<void> showMarkPaidSheet(BuildContext context, {required Expense expense})`

`_MarkPaidSheet`-কে মডাল বটম-শিট হিসেবে খোলে, যা একটি অনুমোদিত দাবিকে পরিশোধিত হিসেবে রেকর্ড করে।

ক্লাস/উইজটে: `_MarkPaidSheet`

`ConsumerStatefulWidget`, কনস্ট্রাক্টরে `required Expense expense` নিয়ে; `_MarkPaidSheetState` তৈরি করে।

ক্লাস: `_MarkPaidSheetState`

পমেন্ট মেথড ও রফোরেন্স নম্বর ইনপুট নিয়ে "মার্ক পাইড" কল করে; এটি অপ্রবর্তনীয় অ্যাকশন — পরে undo করার কোনো এন্ডপয়েন্ট নেই।

`void dispose()`

`_reference` কন্ট্রোলার dispose করে।

`Future<void> _submit()`

`expensesProvider.notifier.markPaid`-কে `method` ও `reference` সহ কল করে। সফল হলে শিট পপ করে; `ApiException`-এর মসেজে বা জনেরেকি এরর দেখায়। শেষে `_busy` রসিটে করে।

`Widget build(BuildContext context)`

`FormSheetFrame`-এর মধ্যে পরিমাণ ও দাবিকারীর নাম দেখায়, পমেন্ট মেথড ড্রপডাউন (`_methods`: `BANK_TRANSFER`, `CASH`, `CHEQUE`, `PAYROLL`, `CARD`), রফোরেন্স ফিল্ড, এবং শেষে সতর্কীকরণ টেক্সট দেখায় যে এই অ্যাকশন পরে undo করা যাবে না।

ফাইল: `lib/features/finance/expenses_tab.dart`

সংক্ষিপ্ত বিবরণ: খরচের দাবি তালিকা দেখানো ও পরচালনার ট্যাব — ফল্টিং (Mine/Awaiting approval/All), প্রতিটি দাবি কার্ড হিসেবে দেখানো, এবং অনুমোদন/প্রত্যাখ্যান/এডিট/মার্ক-পাইড/ডিলিট অ্যাকশন।

ক্লাস/উইজটে: ExpensesTab

ConsumerWidget, খরচের তালিকা দেখানোর মূল ট্যাব উইজটে। ফিল্টার স্ট্রিপি এবং PagedListView একত্রে দেখায়।

Widget build(BuildContext context, WidgetRef ref)

expensesProvider-এর notifier পড়ে এবং expenseViewProvider ওয়াচ করে বর্তমান ভিউ বদলে দেয়।

_ViewFilter এবং একটি PagedListView<Expense> দেখায়, যখন বর্তমান ভিউ অনুযায়ী খালি-অবস্থার টাইটলে/মসেজে বদলায় (ExpenseView.mine/pending/all)।

ক্লাস: _ViewFilter

ConsumerWidget, উপরে অনুভূমিক স্ক্রলযোগ্য চপি-স্ট্রিপি হিসেবে "Mine"/"Awaiting approval"/"All" ফিল্টার দেখায়। অনুমোদন ও দখলের এন্টাইটলেমেন্ট আলাদা — তাই কুইউ-ভিউগুলো শুধু FinancePermissions.expenseView থাকা ব্যবহারকারীদের কাছে দেখা যায়।

Widget build(BuildContext context, WidgetRef ref)

expenseViewProvider থেকে নির্বাচনী ভিউ পড়ে, canSeeOthers পারমিশন চেক করে ভিউ-লিস্ট তৈরি করে। একটি মাত্র ভিউ থাকলে শুধু একটি ফাঁকা স্পেসের দেখায়; নাহলে ListView.separated দিয়ে ট্যাপযোগ্য চপি দেখায়, ট্যাপে expenseViewProvider.notifier.set কল করে।

ক্লাস/উইজটে: _ExpenseCard

ConsumerStatefulWidget, কনস্ট্রাক্টরে required Expense expense নিয়ে; একটি খরচ দাবির কার্ড রেন্ডার করে, _ExpenseCardState তৈরি করে।

ক্লাস: _ExpenseCardState

প্রতিটি কার্ডের নিজস্ব busy-state, decide/delete/notes-prompt লজিক এবং পারমিশন-নির্ভর বাটন দেখায়।

Future<void> _decide({required bool approved})

_promptForNotes-এর মাধ্যমে নোট/কারণ চেয়ে নেয়; পাওয়া গেলে expensesProvider.notifier.decide-কে approved ও notes সহ কল করে। রেসপন্সে বাজটে-ওয়ার্নিং থাকলে (approve response-এর মূল কারণ) তা সন্ধ্যাকবার দেখায়, নাহলে "Approved."/"Rejected." দেখায়। ওয়ার্নিং থাকলে সন্ধ্যাকবার বর্শে সময় (৬ সেকেন্ড) দেখায়। এরর হ্যান্ডলিং করে ও _busy টগল করে।

Future<void> _delete()

confirmAction দিয়ে নিশ্চিতকরণ চায়, নিশ্চিতি হলে expensesProvider.notifier.remove কল করে; সফল হলে "Deleted." সন্ধ্যাকবার দেখায়, ব্যর্থ হলে এরর মসেজে দেখায়।

Future<String?> _promptForNotes({required bool approved})

একটি ছোট মডাল বটম-শিট খুলে অনুমোদন/প্রত্যাখ্যানের জন্য বাধ্যতামূলক নোট/কারণ চায় (approve হলে "Note", reject হলে "Reason"), ভ্যালিডেশন ব্যর্থ হলে খালি রাখতে দেয় না। সাবমিট হলে ট্রিম করা টেক্সট রিটার্ন করে, ডায়ালগ dismiss হলে নাল রিটার্ন করে। শেষে কন্ট্রোলার dispose করে।

Widget build(BuildContext context)

পারমশিন চকে করে (canApprove, canReject, canDecide, canEdit, canMarkPaid, canDelete) একটি `AppCard` তৈরি করে: হডেলাইন ও স্ট্যাটাস চপি, তনি-ডট মনু (Edit/Mark reimbursed/Delete — উপলব্ধ অ্যাকশন অনুযায়ী), খরচ নম্বর/ক্যাটাগরি/ভেন্ডর (ভেন্ডরে ট্যাপ করলে `VendorExpensesScreen` খোলবে), পরমাণ ও তারখি/দাবকিরীর নাম, রহিম্বার্সমেন্ট-স্ট্যাটাস নোট, অনুমোদন-নোট (থাকলে), এবং `canDecide` হলে Reject/Approve বাটন সারি।

ফাইল: lib/features/finance/finance_models.dart

সংক্ষিপ্ত বিবরণ: ফাইন্যান্স মডেলের সব ডটো মডলে, রকি়োস্ট ডিটিও এবং পারমশিন কোড এখানে সংজ্ঞায়িত — খরচ, ইনভয়েস, ওয়ালটে, পমেন্ট রসিটি এবং ফাইন্যান্স ড্যাশবোর্ড ওভারভিউ।

ক্লাস: FinancePermissions

abstract final class, ফাইন্যান্স মডেলের সব পারমশিন কোড কনস্ট্যান্ট হিসেবে ধরে রাখবে (reportView, invoiceView, expenseView, expenseApprove, expenseReject, walletView, invoiceCreate, invoiceUpdate, invoiceDelete, receiptCreate) — এতে কোনো মেথড নেই, শুধু স্ট্যাটিক কনস্ট্যান্ট।

ক্লাস: ExpenseStatus

abstract final class, খরচ দাবরি সম্ভাব্য স্ট্যাটাস কনস্ট্যান্ট (pending, approved, rejected, paid, cancelled) এবং all তালিকা ধরে রাখবে।

ক্লাস: FinanceOverview

GET /finance/dashboard?month&year-এর জন্য একটি মাসের হডেলাইন ফাইন্যান্সিয়াল ফিগার ধরে রাখবে (রেভিনিউ, খরচ, নটি প্রফিট, ক্যাশ কালকেটেডে, আউটস্ট্যান্ডিং, ওভারভিউ, পেয়েবেলস, পরোল খরচ ইত্যাদি)।

bool get isProfitable

netProfit >= 0 হলে true রিটার্ন করে।

double? get overdueShare

outstanding শূন্য বা তার কম হলে নাল রিটার্ন করে (নাহলে ০% "সব সুস্থ" বোঝাবে); নাহলে overdue / outstanding-কে ০ থেকে ১-এর মধ্যে ক্ল্যাম্প করে রিটার্ন করে।

factory FinanceOverview.fromJson(Map<String, dynamic> json)

JSON থেকে অবজেক্ট তৈরি করে; সংখ্যাসূচক ফিল্ডগুলো একটি হিল্পার d() ফাংশন দিয়ে ডাবল-এ কনভার্ট করে (নাল হলে ০), মাস/বছর না থাকলে বর্তমান তারখি ব্যবহার করে, এবং recentInvoices-কে InvoiceLine লিস্টে ম্যাপ করে।

ক্লাস: InvoiceLine

ড্যাশবোর্ড থেকে ফেরত আসা একটি কিম্প্যাক্ট ইনভয়েস রো — পূর্ণাঙ্গ ইনভয়েসের চেয়ে কম ফিল্ড, নিজস্ব overdue ফ্ল্যাগ বহন করে (ক্যালেন্ডার নজি হিসাব করার প্রয়োজন নেই)।

factory InvoiceLine.fromJson(Map<String, dynamic> json)

JSON থেকে InvoiceLine অবজেক্ট তৈরি করে, প্রতটি ফিল্ডের জন্য যথাযথ ডিফল্ট (০, খালি স্ট্রিং, false) ব্যবহার করে।

ক্লাস: Expense

একটি খরচ দাবরি সম্পূর্ণ মডেলে — আইডি, নম্বর, বর্ণনা, পরিমাণ, তারিখ, স্ট্যাটাস, ভেন্ডর, ক্যাটাগরি, অ্যাকাউন্ট, রসিডি, দাবিকারী/অনুমোদনকারীর নাম, নোট এবং রহিম্বার্সমেন্ট তথ্য ধরে রাখবে।

`bool get isPending`

স্ট্যাটাস `ExpenseStatus.pending` হলে `true`।

`bool get isApproved`

স্ট্যাটাস `ExpenseStatus.approved` হলে `true`।

`bool get isPaid`

স্ট্যাটাস `ExpenseStatus.paid` হলে `true`।

`bool get awaitingReimbursement`

অনুমোদিত কিন্তু এখনো রহিম্বার্স করা হয়নি — অর্থাৎ `isApproved && reimbursedDate == null` হলে `true`, যা বোঝায় কডে টাকার জন্য অপেক্ষা করছে।

`bool get hasReceipt`

`receiptUrl` নাল না হয়ে খালি না হলে `true`।

`String get headline`

`title` (AI-কম্পোজড ছোট লাইন) থাকলে সেটি, নাহলে `description` (মানুষের টাইপ করা) রিটার্ন করে — রো-এর শিরোনাম হিসেবে দেখানোর জন্য।

`factory Expense.fromJson(Map<String, dynamic> json)`

JSON থেকে `Expense` অবজেক্ট তৈরি করে, সব ফিল্ডের জন্য নরিপদ ডিফল্ট ব্যবহার করে (স্ট্যাটাস না থাকলে `pending`)।

ক্লাস: CreateExpenseRequest

`POST /company/finance/expenses`-এর জন্য রকি়োস্ট বডি — বর্ণনা, পরিমাণ, তারিখ আবশ্যক; টাইটলে, ভেন্ডর, ক্যাটাগরি, নোট, রসিডি-URL ঐচ্ছকি।

`Map<String, dynamic> toJson()`

একটি অভ্যন্তরীণ `clean()` হলেপার দিয়ে খালি/ট্রিমড-খালি স্ট্রিং ফিল্ডগুলো বাদ দিয়ে এবং শুধু নন-এম্পটি ঐচ্ছকি ফিল্ড JSON-এ যোগ করে।

ক্লাস: Wallet

কোম্পানির ওয়ালেটে ব্যালেন্স মডেলে — ক্যাশ ব্যালেন্স, ক্রেডিট ব্যালেন্স, এবং দুটোর সারভার-কম্পিউটেডে যোগফল `totalAvailable`।

`bool get hasCredit`

`creditBalance > 0` হলে `true`।

`factory Wallet.fromJson(Map<String, dynamic> json)`

JSON থেকে `Wallet` অবজেক্ট তৈরি করে, সংখ্যাসূচক ফিল্ড না থাকলে ০ ব্যবহার করে।

ক্লাস: WalletTransaction

একটি একক ওয়ালেটে লেনদেনে — টাইপ, পরিমাণ, লেনদেনের পরের ব্যালেন্স, রফোরেন্স এবং নোট।

`bool get isCredit`

লেনদেনের `type` একটি নির্দিষ্ট স্টেটে (`TOP_UP`, `CREDIT`, `REFUND`, `DEPOSIT`, `BONUS`, `ADJUSTMENT_CREDIT`) থাকলে `true` — চহিনরে বদলে টাইপ থেকে ঠকি করা হয়, কারণ ব্যাকএন্ড উভয় পাশে পজটিভি পরমাণ সংরক্ষণ করে।

`factory WalletTransaction.fromJson(Map<String, dynamic> json)`

JSON থেকে `WalletTransaction` অবজেক্ট তরৈ করে।

টপ-লভেলে কনস্ট্যান্ট

`paymentTermsOptions` — ক্লায়েন্টের পমেন্টের সময়সীমার বকিল্প (`DUE_ON_RECEIPT`, `NET_15` ইত্যাদি)। `paymentMethods` — পমেন্ট পদ্ধতির তালকি (`BKASH`, `NAGAD`, `ROCKET`, `SSLCOMMERZ`, `BANK_TRANSFER`, `CASH`, `WALLET`, `CHEQUE`)।

ক্লাস: InvoiceRequest

`POST/PATCH /company/finance/invoices`-এর জন্য একই ডটিওি ব্যবহার করে, কারণ ব্যাকএন্ডও তাই করে। `PATCH`-কেও `create`-এর মতো সব ভ্যালডিশেন (ক্লায়েন্ট, দুটিতারখি, নন-এম্পটি আইটেমে) পাস করতে হয়, এবং `update` অনকে ফলিড শর্তহীনভাবে স্টে করে ফলে — তাই এডটিরে একমাত্র নরিপদ উপায় হলো পুরো জনিসিটা ফরেত পাঠানো।

`factory InvoiceRequest.from(Invoice invoice)`

বদ্যমান ইনভয়েস থেকে একটি রিকোয়েস্ট সডি করে, যাতে ফর্মে না দেখানো ফলিডগুলোও ফরেত পাঠানো হয়, খালনি হয় যায়।

`InvoiceRequest copyWith({...})`

নর্দিষ্ট ফলিড পরবির্তন করে একটি নতুন `InvoiceRequest` তরৈ করে, বাকসিব আগরেটির মতো রাখে।

`Map<String, dynamic> toJson()`

JSON বডিতরৈ করে; `taxAmount`, `taxRatePercent`, `discountAmount`, `paymentTerms`, `description`, `notes` — এই ফলিডগুলো শর্তহীনভাবে (ক্লাস কমন্টে বরণতি কারণে) পাঠায়।

ক্লাস: CreatePaymentReceiptRequest

`POST /company/finance/payment-receipts`-এর জন্য — টাকা প্রাপ্তিরকেরড করে। ক্লায়েন্ট, পরমাণ, তারখি ও পদ্ধতি বাধ্যতামূলক; ইনভয়েসের সাথে লংক করা ঐচ্ছকি (না করলে "অন-অ্যাকাউন্ট পমেন্ট" বোঝায়)।

`Map<String, dynamic> toJson()`

`clean()` হলেপার দয়ি খালি স্ট্রিং ফলিড বাদ দয়ি JSON বডিতরৈ করে।

ক্লাস: ExpenseComposeRequest

`POST /company/finance/expenses/ai-compose`-এর জন্য — এলোমলেো নোট থেকে টাইটলে ও বরণনা তরৈ করতে সাহায্য চায়। `roughNotes` বাধ্যতামূলক, বাকি (`vendorName`, `amount`, `category`) কনটেকেস্ট হিসেবে ঐচ্ছকি।

`Map<String, dynamic> toJson()`

JSON বডিতরৈ করে, ঐচ্ছকি ফলিডগুলো খালনি হলই যোগ করে।

ক্লাস: ExpenseDraft

AI-এর তৈরিকরা সাজশেন (টাইটলে ও বর্ণনা) ধরে রাখতে — শুধু সাজশেন, দাবিকারী সাবমিট করার আগে এডিট করতে পারে।

factory ExpenseDraft.fromJson(Map<String, dynamic> json)

JSON থেকে ExpenseDraft তৈরিকরে।

ক্লাস: UpdateExpenseRequest

একটি পিন্ডেডিং খরচ এডিট করার জন্য — এটি মূলত পুরো রিসোর্স রিপ্লসেমেন্ট, কারণ ব্যাকএন্ড update শুধু title/currency/vendorName গার্ড করে, বাকি সব ফিল্ড সরাসরি রিকোয়েস্ট থেকে অ্যাসাইন করে (বাদ পড়লে ক্লয়ার হয়ে যায়)।

factory UpdateExpenseRequest.from(Expense expense)

বর্তমান খরচ থেকে (অ্যাকাউন্ট আইডিসহ, যা কোনো ফর্মে দেখানো হয় না) একটি রিকোয়েস্ট সডি করে।

UpdateExpenseRequest copyWith({...})

নরদৃষ্টি ফিল্ড আপডেট করে নতুন UpdateExpenseRequest রিটার্ন করে, বাকি সব আগেরটির মতো রাখে।

Map<String, dynamic> toJson()

প্রত্যেকটি (নাল থাকলেও) JSON-এ রাখে — এখানে অনুপস্থিতি মানে "অপরিবর্তিত রাখো" নয়।

ফাইল: lib/features/finance/finance_overview_tab.dart

সংক্ষিপ্ত বিবরণ: ফাইন্যান্স ড্যাশবোর্ডের ওভারভিউ ট্যাব — বর্তমান মাসের নটি প্রফিট/লস, রভেনিউ-খরচের পরিবর্তন, মূল ফগার (ক্যাশ কালকেটেড, পরোল, পেয়েবেলস), বক্যো/ওভারডিউ প্রাপ্তি এবং সাম্প্রতিক ইনভয়েস দেখায়।

ক্লাস/উইজটে: FinanceOverviewTab

ConsumerWidget, ফাইন্যান্স ওভারভিউ ট্যাবের রুট উইজটে।

Widget build(BuildContext context, WidgetRef ref)

financeOverviewProvider ওয়াচ করে; লোডিং, এরর (রিট্রাই বোতাম সহ) এবং ডটো—তনি অবস্থা হ্যান্ডল করে। ডটো এলে RefreshIndicator-এর মধ্যে _ProfitCard, _Figures, _Receivables এবং (থাকলে) _RecentInvoices সাজিয়ে দেখায়।

ক্লাস: _ProfitCard

মাসের নটি প্রফিট/লস এবং রভেনিউ/খরচের মাস-ভিত্তিক পরিবর্তন দেখানো কার্ড।

Widget build(BuildContext context)

মাস-বছর, বড় করে নটি প্রফিট/লস (রঙ প্রফিটবেল কনি অনুযায়ী), এবং একটি সারিতে দুটি _Movement উইজটে (রভেনিউ ও খরচ) দেখায়, মাঝে একটি ভার্টিক্যাল ডিভাইডার সহ।

ক্লাস: `_Movement`

একটি একক ফিগারের (রেভিনিউ বা খরচ) মূল্য এবং মাস-ওভার-মাস শতাংশ পরিবর্তন দেখায়, তীর আইকনসহ।

`Widget build(BuildContext context)`

লবেলে ও পরিমাণ দেখায়; `changePercent` নাল হলে ড্যাশ দেখায় (তুলনা করার আগে মাস নই বোঝাতো), নাহলে বৃদ্ধি/হ্রাস অনুযায়ী তীর আইকন ও রঙ (সবুজ/লাল, `upIsGood` অনুযায়ী নির্ধারণ) সহ শতাংশ দেখায়।

ক্লাস: `_Figures`

"এই মাসে" শিরোনামে ক্যাশ কালকেটেডে, পরেোল খরচ, পয়েবেলস, এবং ওভারডিউ পয়েবেলস — এই চারটি ফিগার সারি আকারে দেখায়।

`Widget build(BuildContext context)`

একটি রেকর্ড-টাইপ লিস্ট `rows` তৈরি করে (প্রতিটির লবেলে, মান, রঙ), `SectionHeader` ও `AppCard`-এর মধ্যে প্রতিটি সারি (মাঝে ডিভাইডার সহ) দেখায়।

ক্লাস: `_Receivables`

"Owed to you" শিরোনামে মোট বক্যো, তার মধ্যে ওভারডিউ অংশ, এবং একটি প্রগ্রেসে বার দেখায় কতটা ওভারডিউ তার অনুপাত বোঝাতো।

`Widget build(BuildContext context)`

`overview.outstanding` ও `overview.overdue` দেখায়; `overview.overdueShare` নাল না হলে (কেছু বক্যো থাকলে) একটি `LinearProgressIndicator` এবং শতাংশ টেক্সট দেখায় ("X% of what you are owed is late")।

ক্লাস: `_RecentInvoices`

সাম্প্রতিক সর্বোচ্চ ৫টি ইনভয়েস (ইনভয়েসে নম্বর, ক্লায়েন্ট নাম, ব্যালেন্স/মোট পরিমাণ, স্ট্যাটাস চপি) তালিকা আকারে দেখায়।

`Widget build(BuildContext context)`

`SectionHeader` ও `AppCard`-এর মধ্যে সর্বোচ্চ ৫টি `InvoiceLine` লুপ করে দেখায় (মাঝে ডিভাইডার সহ), প্রতিটির জন্য `balanceAmount` (শূন্যের বশেইল) নাহলে `totalAmount` দেখায়, এবং `overdue` ফ্ল্যাগ অনুযায়ী "OVERDUE" বা আসল স্ট্যাটাস `StatusChip`-এ দেখায়।

done

ফাইল: `lib/features/finance/finance_repository.dart`

সংক্ষিপ্ত বিবরণ: ফিনিয়ান্স মডুলের API রপ্যোজিটরি — ইনভয়েসে, এক্সপেন্সে, রসিটি ও ওয়ালটে সম্প্রকৃতি সব ব্যাকএন্ড কল এখানে সংজ্ঞায়িত। এর পাশাপাশি Riverpod প্রোভাইডার ও কন্ট্রোলার ক্লাসগুলোও এখানে আছে যা UI-কে পজেড লিস্ট ও স্টেটে সরবরাহ করে।

টপ-লেভেলে `enum: ExpenseView`

`mine`, `pending`, `all` — এক্সপেন্স লিস্টের কোন অংশ দেখানো হবে তা নির্ধারণ করে। এগুলো ক্লায়েন্ট-সাইড ফিল্টার নয়, বরং সরাসরি সার্ভার-সাইড এন্ডপয়েন্ট/স্ট্যাটাস অনুযায়ী সংজ্ঞায়িত: `mine` কলকারীর employee রেকর্ড রজিভ করে, আর বাকগুলো ব্যাকএন্ডের নিজস্ব স্ট্যাটাস লিস্ট।

ক্লাস: FinanceRepository

ApiClient ব্যবহার করে /company/finance ও /wallet পাথরে অধীনে সব ফিন্যান্স-সম্পর্কিত REST কল করে। কনস্ট্রাক্টরে ApiClient ইনজেক্ট করা হয়।

FinanceRepository(this._api)

কনস্ট্রাক্টর, যা ApiClient ইনস্ট্যান্স গ্রহণ করে _api ফিল্ডে সংরক্ষণ করে।

Future<FinanceOverview> overview({int? month, int? year})

/finance/dashboard এন্ডপয়েন্ট থেকে মাস ও বছর অনুযায়ী ফিল্টার করা ড্যাশবোর্ড সারাংশ আনে এবং FinanceOverview.fromJson দিয়ে পার্স করে রিটার্ন করে।

Future<PagedResponse<Invoice>> invoices({String? status, int page = 0, int size = 20})

ইনভয়েস লিস্ট আনে। status overdue হলে overdueInvoices() কল করে ফলাফলকে একটি সম্পূর্ণ একক পেজে হিসেবে মোড়ানো হয়, কারণ overdue স্ট্যাটাস রেকর্ডে স্থায়ীভাবে সংরক্ষিত থাকে না বরং due date থেকে গণনা করা হয়। অন্য স্ট্যাটাসের জন্য আলাদা পাথ (/invoices/status/\${status}) ব্যবহৃত হয়, আর status null হলে সাধারণ /invoices পাথ থেকে পেজেড রেসপন্স আনা হয়।

Future<Invoice> createInvoice(InvoiceRequest request)

POST \$_base/invoices কল করে নতুন ইনভয়েস তৈরি করে এবং তৈরি হওয়া Invoice অবজেক্ট রিটার্ন করে।

Future<Invoice> updateInvoice(int id, InvoiceRequest request)

PATCH \$_base/invoices/\$id কল করে একটি ড্রাফট ইনভয়েস এডিট করে। শুধু DRAFT স্ট্যাটাসের ইনভয়েসের জন্য কাজ করে; অন্য যেকোনো স্ট্যাটাসে সার্ভার প্রত্যাখ্যান করে।

Future<void> deleteInvoice(int id)

DELETE \$_base/invoices/\$id কল করে ইনভয়েস মুছে দেয়। শুধুমাত্র COMPANY_OWNER রোলের জন্য অনুমোদিত (কন্ট্রোলার-লভেলে রোল গটে, পারমিশন কোড নয়); সফল হলে ২০৪ স্ট্যাটাস, কোনো বডি নেই।

Future<void> createReceipt(CreatePaymentReceiptRequest request)

POST \$_base/payment-receipts কল করে ক্লায়েন্টের কাছ থেকে প্রাপ্ত অর্থের রেকর্ড তৈরি করে, ঐচ্ছিকভাবে একটি নির্দিষ্ট ইনভয়েসের বিপরীতে।

Future<String> invoicePdf(Invoice invoice)

\$_base/invoices/\${invoice.id}/pdf থেকে বাইট আকারে PDF ডাউনলোড করে, ফাইলের নাম থেকে নরিপদ অক্ষর ছাড়া সব রমিউব করে অস্থায়ী ডিরেক্টরিতে সংরক্ষণ করে এবং লোকাল ফাইল পাথ রিটার্ন করে।

Future<PagedResponse<Expense>> expenses(ExpenseView view, {int page = 0, int size = 20})

view-এর উপর ভিত্তি করে ভিন্ন পাথ থেকে (my-expenses, status/pending, বা সব) এক্সপেন্স লিস্ট আনে। mine ভিউতে যদি ব্যবহারকারীর কোনো employee রেকর্ড না থাকে (400/404/403 এরর) তাহলে খালি পেজেড রেসপন্স রিটার্ন করে, ব্যতীক্রম ছুঁড়ে দেয় না।

Future<Expense> expense(int id)

\$_base/expenses/\$id থেকে একটি এক্সপেন্সের সম্পূর্ণ বিবরণ আনে — সার্ভারে বর্তমানে যা আছে তা নিশ্চিত করতে ডিটেইল ভিউ প্রত্যাখ্যান পুনরাবৃত্তি করে।

Future<PagedResponse<Expense>> expensesByVendor(String vendorName, {int page = 0, int size = 20})

নব্ব্বদশটি ভেন্ডরের নাম (URL-এনকোডেড) দিয়ে ফিল্টার করা এক্সপেন্স লিস্ট আনবে — এখানে ভেন্ডর একটি টাইপ করা নাম, আলাদা রেকর্ড নয়।

Future<Expense> updateExpense(int id, UpdateExpenseRequest request)

PATCH `$_base/expenses/$id` কল করে এক্সপেন্স আপডেট করে। শুধুমাত্র PENDING স্ট্যাটাসের এক্সপেন্সের জন্য কাজ করে, এবং পলোড সম্পূর্ণ (আংশিক নয়)।

Future<void> markExpensePaid(int id, {String? method, String? reference})

POST `$_base/expenses/$id/mark-as-paid` কল করে, যখন `reimbursementMethod` ও `referenceNumber` ঐচ্ছিক কৌশলের প্রারামিটার হিসেবে যোগ করা হয় (থাকলে)। রসপন্স খালি।

Future<void> deleteExpense(int id)

DELETE `$_base/expenses/$id` কল করে এক্সপেন্স মুছে দেয়। শুধু কোম্পানিগণের জন্য অনুমোদিত, শুধুমাত্র EXPENSE_UPDATE পারমিশন থাকলেই যথেষ্ট নয়।

Future<List<Invoice>> overdueInvoices()

`$_base/invoices/overdue` থেকে ডাউনলোড করে পাওয়া ইনভয়েসগুলোর একটি বিবরণ লিস্ট (পেজ নয়) আনবে ও প্যার্স করে।

Future<String> invoiceSummary(int id)

`$_base/invoices/$id/ai-summary` থেকে AI-জেনারেটেড সারাংশ টেক্সট আনবে। কিছুই সংরক্ষণ করা হয় না — শুধু পড়ার/ব্যবহারের জন্য টেক্সট ফেরত আসবে।

Future<void> recordInvoicePayment(int id, double amount)

POST `$_base/invoices/$id/record-payment?amount=$amount` কল করে ইনভয়েসের বিপরীতে পেমেন্ট বসায়। `createReceipt`-এর থেকে আলাদা: এটি শুধু ইনভয়েসের `paid` ফিল্ডের সত্য, ক্লায়েন্টের বিপরীতে কোনো রসিট ডকুমেন্ট তৈরি করে না।

Future<ExpenseDraft> composeExpense(ExpenseComposeRequest request)

POST `$_base/expenses/ai-compose` কল করে বুদ্ধি নোট থেকে AI দিয়ে শিরোনাম ও বিবরণ ড্রাফট করায়।

Future<Expense> submit(CreateExpenseRequest request)

POST `$_base/expenses` কল করে নতুন এক্সপেন্স ক্লাইম সাবমিট করে।

Future<String?> approve(int id, String notes)

POST `$_base/expenses/$id/approve?notes=...` কল করে একটি ক্লাইম অনুমোদন করে। নোট কৌশলের প্রারামিটার হিসেবে পাঠানো হয়, বডি হিসেবে নয়। রসপন্সে `budgetWarning` থাকলে তা রিটার্ন করে (ক্যাটাগরি বাজেটে ছাড়িয়ে গেলে সতর্কবার্তা)।

Future<void> reject(int id, String reason)

POST `$_base/expenses/$id/reject?reason=...` কল করে একটি ক্লাইম প্রত্যাখ্যান করে।

Future<Wallet> wallet()

`/wallet` থেকে বর্তমান ওয়ালেটে ব্যালেন্স তথ্য আনবে।

`Future<PagedResponse<WalletTransaction>> walletTransactions({int page = 0, int size = 20})`
`/wallet/transactions` থেকে ওয়ালটে লেনদেনের পজেড লিস্ট আনবে।

টপ-লেভেলে প্রোভাইডার

`financeRepositoryProvider` (`FinanceRepository` তৈরি করে), `financeOverviewProvider` (ফিনিয়ান্স ওভারভিউ `FutureProvider`, বর্তমান ইউজার পরবর্তন রফ্রেশে হয়), `walletProvider` (ওয়ালটে `FutureProvider`)।

ক্লাস: InvoiceFilterController

`Notifier<String?>` — ইনভয়সে লিস্ট কোন স্ট্যাটাসে ফিল্টার করা আছে তা রাখবে, `null` মানে সব।

`String? build()`

ইনশিয়াল স্টেটে `null` (কোনো ফিল্টার নেই) স্টেট করে।

`void set(String? status)`

নতুন স্ট্যাটাস ফিল্টার স্টেট করে; বর্তমান স্টেটের সাথে একই হলে কিছু করে না।

ক্লাস: InvoicesController

`AsyncNotifier<PagedState<Invoice>>` যা `PagedLoader<Invoice>` মক্সইন ব্যবহার করে ইনভয়সে লিস্টের পজেনিশন, তৈরি, আপডেট ও মুছে ফেলার কাজ পরিচালনা করে।

`Future<PagedState<Invoice>> build()`

`currentUserProvider` ও `invoiceFilterProvider` ওয়াচ করে প্রথম পজে লোড করে।

`Future<PagedResponse<Invoice>> fetchPage(int page)`

বর্তমান স্ট্যাটাস ফিল্টার সহ নরিদ্রিষ্ট পজে আনবে — `PagedLoader` মক্সইনের জন্য `override`।

`Future<Invoice> create(InvoiceRequest request)`

রপিজিটির `createInvoice` কল করে ইনভয়সে তৈরি করে এবং লিস্ট রফ্রেশে করে।

`Future<Invoice> updateItem(int id, InvoiceRequest request)`

রপিজিটির `updateInvoice` কল করে এবং লিস্টের সংশ্লিষ্ট আইটেমকে নতুন ডটো দিয়ে প্রতিস্থাপন করে।

`Future<void> delete(int id)`

রপিজিটির `deleteInvoice` কল করে এবং লিস্ট থেকে আইটেমটি সরিয়ে দেয়।

ক্লাস: ExpenseViewController

`Notifier<ExpenseView>` — বর্তমান এক্সপেন্স ভিউ (`mine/pending/all`) সংরক্ষণ করে, ডিফল্ট `mine`।

`ExpenseView build()`

ইনশিয়াল স্টেটে `ExpenseView.mine` রটার্ন করে।

`void set(ExpenseView view)`

নতুন ভিউ স্টেট করে; একই হলে কিছু করে না।

ক্লাস: ExpensesController

`AsyncNotifier<PagedState<Expense>>` যা `PagedLoader<Expense>` মক্সইন ব্যবহার করে এক্সপেন্স লিস্টের পজেনিশন, সাবমিশন, অনুমোদন/প্রত্যাখ্যান, এডিট, পাইড মার্কিং ও মুছে ফেলা পরিচালনা করে।

`Future<PagedState<Expense>> build()`

`currentUserProvider` ও `expenseViewProvider` ওয়াচ করে প্রথম পজে লোড করে।

`Future<PagedResponse<Expense>> fetchPage(int page)`

বর্তমান `ExpenseView` অনুযায়ী নির্দিষ্ট পজে আনে।

`Future<Expense> submit(CreateExpenseRequest request)`

রপিজিটির `submit` কল করে নতুন ক্লেইম তৈরি করে এবং লিস্ট রিফ্রেশ করে।

`Future<String?> decide(int id, {required bool approved, required String notes})`

`approved` অনুযায়ী `approve` বা `reject` কল করে। লিস্ট `pending` ভিউতে থাকলে আইটেমটি সরিয়ে দিয়ে (কারণ সিদ্ধান্ত নেওয়া ক্লেইম আর `pending queue`-তে থাকে না); অন্যথায় পুরো লিস্ট রিফ্রেশ করে। অনুমোদনের ফলে বাজটে সতর্কতা থাকলে তা রিটার্ন করে।

`Future<void> edit(int id, UpdateExpenseRequest request)`

রপিজিটির `updateExpense` কল করে লিস্টের আইটেম প্রতস্থাপন করে।

`Future<void> markPaid(int id, {String? method, String? reference})`

রপিজিটির `markExpensePaid` কল করে; রসপন্স খালি হলে এরপর `expense(id)` দিয়ে পুনরায় পড়ে আইটেমটি রিফ্রেশ করে।

`Future<void> remove(int id)`

রপিজিটির `deleteExpense` কল করে লিস্ট থেকে আইটেম সরিয়ে দিয়ে।

ক্লাস: WalletTransactionsController

`AsyncNotifier<PagedState<WalletTransaction>>` যা `PagedLoader<WalletTransaction>` মক্সইন ব্যবহার করে ওয়ালটে লেনদেন লিস্ট পজেনিটে করে।

`Future<PagedState<WalletTransaction>> build()`

`currentUserProvider` ওয়াচ করে প্রথম পজে লোড করে।

`Future<PagedResponse<WalletTransaction>> fetchPage(int page)`

নির্দিষ্ট পজে ওয়ালটে লেনদেন আনে।

টপ-লেভেলে প্রোভাইডার (কন্ট্রোলার)

`invoiceFilterProvider`, `invoicesProvider`, `expenseViewProvider`, `expensesProvider`, `walletTransactionsProvider` — যথাক্রমে উপরে কন্ট্রোলার ক্লাসগুলোকে `Riverpod`-এ এক্সপোজ করে।

ফাইল: lib/features/finance/finance_screen.dart

সংক্ষিপ্ত ববিরণ: ফিনিয়ান্স মডউিলরে মূল স্ক্রনি — Overview, Expenses, Invoices ও Wallet ট্যাব সংবলতি, পারমশিন অনুযায়ী কোন ট্যাব দখোনো হবো তা নরিধারণ করে। ওয়বে সংস্করণরে ১৮টি স্ক্রনিরে মধ্যে ১২টি (chart of accounts, general ledger ইত্যাদি) ইচ্ছাকৃতভাবে বাদ দেওয়া হয়েছে কারণ ফোনো সেগুলো ব্যবহারযোগ্য নয়।

ক্লাস/উইজটে: FinanceScreen

মূল ConsumerWidget — পারমশিন অনুযায়ী ট্যাব তালিকা তরৈকিরে, প্রতটি ট্যাবরে জন্য উপযুক্ত FAB (Claim expense বা New invoice) দখোয় এবং TabBar/TabBarView সটেআপ করে।

FinanceScreen({super.key, this.initialTabLabel})

কনস্ট্রাক্টর, ঐচ্ছকি initialTabLabel গ্রহণ করে যা নরিদষ্টি করে কোন ট্যাবে শুরু করতে হবে।

Widget build(BuildContext context, WidgetRef ref)

পারমশিন এখনো লোড না হলে লোডার দখোয়। এরপর পারমশিন অনুযায়ী দৃশ্যমান ট্যাবরে তালিকা তরৈকিরে (Overview, Expenses সবসময়, Invoices, Wallet — শর্তসাপেক্ষে), initialTabLabel থকে প্রারম্ভকি ইনডেক্স নরিণয় করে, এবং DefaultTabController + AnimatedBuilder দখি সক্রয় ট্যাব অনুযায়ী FAB পরবির্তন করে Scaffold রটার্ন করে।

ক্লাস/উইজটে: _InvoicesTab

ইনভয়সে তালিকার ট্যাব — ফল্টার স্ট্রপি ও পজেড ইনভয়সে লস্ট দখোয়।

Widget build(BuildContext context, WidgetRef ref)

_InvoiceFilter ও PagedList<Invoice> কম্বাইন করে একটি Column-এ সাজায়, invoicesProvider থকে ডটো নখি _InvoiceCard দখি প্রতটি আইটেমে রনেডার করে।

ক্লাস/উইজটে: _InvoiceFilter

ইনভয়সে স্ট্যাটাস অনুযায়ী অনুভূমকি চপি ফল্টার দখোয়।

Widget build(BuildContext context, WidgetRef ref)

ব্যাকএন্ড এনামরে স্ট্যাটাসগুলো (গুরুত্ব অনুযায়ী সাজানো — overdue প্রথম) একটি অনুভূমকি ListView এ চপি আকারে দখোয়; নরিবাচতি চপি ব্র্যান্ড কালার প্রয়োগ হয় এবং ট্যাপে invoiceFilterProvider আপডটে হয়।

ক্লাস/উইজটে: _InvoiceCard

ConsumerStatefulWidget — একটি ইনভয়সের কার্ড প্রদর্শন করে, সাথে অ্যাকশন মেনু (PDF থোলা, পমেন্ট রকের্ড, পাঠানো, সারাংশ, এডটি, ডলিটি, ক্যানসলে)।

_InvoiceCard({required this.invoice})

কনস্ট্রাক্টর, প্রদর্শনযোগ্য Invoice গ্রহণ করে।

ConsumerState<_InvoiceCard> createState()

_InvoiceCardState ইনস্ট্যান্স তরৈকিরে।

ক্লাস: _InvoiceCardState

_InvoiceCard-এর স্টটে — ডলিটি/ডাউনলোড/অন্যান্য অ্যাকশনের বজি-স্টটে পরচালনা করে।

Future<void> _delete(Invoice invoice)

কনফার্মেশন ডায়ালগ দেখায়; নশ্চিতি হলে `invoicesProvider.notifier.delete` কল করে এবং সফলতা/ব্যর্থতা সন্ধ্যাকবার দেখায়।

Future<void> _act(Future<void> Function(ReceiveablesRepository repo) action, String success, String failure)

একটি জিনেরেকি অ্যাকশন হ্যান্ডলার — যমেন পাঠানো বা ক্যানসলে করা — যা রসেপন্স খালি বিধায় অ্যাকশনের পর পুরো ইনভয়সে লস্টি রফির্শে করে ও ফলাফল সন্ধ্যাকবার দেখায়।

Future<void> _open()

ইনভয়সের PDF ডাউনলোড করে (`invoicePdf`) এবং `OpenFilex.open` দিয়ে খোলতে; কোনো অ্যাপ না থাকলে বা ব্যর্থ হলে সন্ধ্যাকবার দেখায়।

Widget build(BuildContext context)

পারমশিন অনুযায়ী উপলব্ধ অ্যাকশনগুলোর (`RowAction`) তালিকা তৈরি করে — যমেন PDF দেখা, পমেন্ট রকের্ড, পাঠানো, সারাংশ, `adjust paid`, এডিট, ডলিটি, ক্যানসলে — এবং `PopupMenuButton` দিয়ে দেখায়। কার্ডে ইনভয়সে নম্বর, স্ট্যাটাস চপি, ববিরণ, বাকটাকা/মোট টাকা এবং ডিউ ডটে দেখায়।

ক্লাস/উইজটে: `_WalletTab`

ওয়ালটে ব্যালন্স ও লনেদনে তালিকা প্রদর্শন করে।

Widget build(BuildContext context, WidgetRef ref)

`walletProvider` থেকে ব্যালন্স কার্ড (মোট উপলব্ধ, ক্রেডিট থাকলে ক্যাশ/ক্রেডিট split) ও `walletTransactionsProvider` থেকে `PagedListview<WalletTransaction>` দেখায়। রফির্শে ওয়ালটে ও লনেদনে উভয়ই আপডেটে হয়।

ক্লাস/উইজটে: `_Split`

ওয়ালটে ব্যালন্সের একটি ভাগ (Cash বা Credit) প্রদর্শনকারী ছোট উইজটে।

`_Split({required this.label, required this.amount})`

কনস্ট্রাক্টর, লবেলে ও পরমাণ গ্রহণ করে।

Widget build(BuildContext context)

পরমাণ (মানি ফিরম্যাটে) ও লবেলে উল্লেখ্যভাবে সাজিয়ে দেখায়।

ক্লাস/উইজটে: `_TransactionRow`

একটি ওয়ালটে লনেদনে সার প্রদর্শন করে।

`_TransactionRow({required this.transaction})`

কনস্ট্রাক্টর, `WalletTransaction` গ্রহণ করে।

Widget build(BuildContext context)

ক্রেডিট/ডেবিট অনুযায়ী আইকন ও রঙ (সবুজ/লাল) নির্ধারণ করে, লনেদনের ধরন, সময়, রফোরন্স, পরমাণ (চহিনসহ, টাইপ থেকে নির্ণীত — অ্যামাউন্ট থেকে নয়) এবং লনেদনে-পরবর্তী ব্যালন্স দেখায়।

ফাইল: lib/features/finance/invoice_extras.dart

সংক্ষিপ্ত ববিরণ: ইনভয়সে সম্পরকতি দুটি অতিরিক্ত বটম শাটি সরবরাহ করে — একটি AI-জনোরটেডে সারাংশ দেখানোর জন্য, অন্যটি ইনভয়সে paid ফগিার সরাসরি সম্ন্বয় করার জন্য।

টপ-লভেলে ফাংশন: Future<void> showInvoiceSummarySheet(BuildContext context, {required Invoice invoice})

একটি মিডাল বটম শাটি খোলে যা `_SummarySheet` দেখায় — নির্দিষ্ট ইনভয়সে জন্য AI-জনোরটেডে সারাংশ।

ক্লাস/উইজটে: `_SummarySheet`

`ConsumerStatefulWidget` — ইনভয়সে AI সারাংশ আনে ও প্রদর্শন করে।

`_SummarySheet({required this.invoice})`

কনস্ট্রাক্টর, প্রদর্শনযোগ্য `Invoice` গ্রহণ করে।

`ConsumerState<_SummarySheet> createState()`

`_SummarySheetState` তৈরি করে।

ক্লাস: `_SummarySheetState`

`_SummarySheet`-এর স্টেটে, সারাংশ ফিচ ও কপি-টু-ক্লিপবোর্ড পরিচালনা করে।

`_summary` (লেটে ফল্ড)

`ref.read(financeRepositoryProvider).invoiceSummary(widget.invoice.id)` কল করে সারাংশ আনার Future — একবার তৈরি হলে ক্যাশড থাকে।

`Widget build(BuildContext context)`

`FutureBuilder<String>` দিয়ে সারাংশ লোডিং/এরর/খালি/সফল স্টেটে হ্যান্ডল করে। সফল হলে

`SelectableText` দিয়ে সারাংশ দেখায়, "Copy" বাটন দিয়ে যা ক্লিপবোর্ডে কপি করে, এবং সতর্কতা দেখায় যে এটি কোথাও সংরক্ষিত হয় না।

টপ-লভেলে ফাংশন: Future<void> showAdjustPaidSheet(BuildContext context, {required Invoice invoice})

একটি মিডাল বটম শাটি খোলে যা `_AdjustPaidSheet` দেখায় — ইনভয়সে paid ফগিার সরাসরি সম্ন্বয় করার ফর্ম।

ক্লাস/উইজটে: `_AdjustPaidSheet`

`ConsumerStatefulWidget` — শুধু ইনভয়সে পেইড অ্যামাউন্ট আপডেট করার ফর্ম, কোনো রসিডি তৈরি করে না।

`_AdjustPaidSheet({required this.invoice})`

কনস্ট্রাক্টর, লক্ষ্য `Invoice` গ্রহণ করে।

`ConsumerState<_AdjustPaidSheet> createState()`

`_AdjustPaidSheetState` তৈরি করে।

ক্লাস: `_AdjustPaidSheetState`

`_AdjustPaidSheet`-এর স্টেটে — ফর্ম ভ্যালিডেশন ও সাবমিশন পর্যালোচনা করে।

`void dispose()`

`_amount` কন্ট্রোলার `dispose` করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, `recordInvoicePayment` কল করে, তারপর `invoicesProvider.notifier.refresh()` করে (কারণ রেসপন্স খালি) এবং শিট বন্ধ করে; ত্রুটিতে এরর মैसेজ দেখায়।

`Widget build(BuildContext context)`

`FormSheetFrame` ব্যবহার করে — বর্তমানে কত বাকি আছে তা দেখায়, একটি "Amount received" ইনপুট ফিল্ড (ভ্যালিডেশন: শূন্যের বেশি, বাক্যের বেশি নয়) এবং একটি সিতরুকবার্তা যে এটি শুধু ফগিয়ার সরায়, প্রকৃত রসিটি তৈরি করে না।

ফাইল: `lib/features/finance/invoice_form_sheet.dart`

সংক্ষিপ্ত বিবরণ: নতুন ইনভয়েস তৈরি বা বিদ্যমান ড্রাফট ইনভয়েস এডিট করার জন্য বটম শিট ফর্ম সরবরাহ করে, লাইন আইটেম, ট্যাক্স, ডিসকাউন্ট ও মোট হিসাব সহ।

টপ-লেভেলে ফাংশন: `Future<void> showNewInvoiceSheet(BuildContext context)`

`_InvoiceFormSheet` (কোনো `existing` ছাড়া) দেখানোর জন্য মডাল বটম শিট খোলে — নতুন ইনভয়েস তৈরি করার জন্য।

টপ-লেভেলে ফাংশন: `Future<Invoice?> showEditInvoiceSheet(BuildContext context, Invoice invoice)`

`_InvoiceFormSheet(existing: invoice)` দেখানোর জন্য মডাল বটম শিট খোলে; সারভারের আপডেটেড ভার্সন রিটার্ন করে অথবা বাতিল হলে `null`।

ক্লাস: `_LineControllers`

একটি এডিটিং ইনভয়েস লাইনের জন্য `TextEditingController` গুলো ধারণ করে — ভ্যালুর বদলে কন্ট্রোলার রাখা হয় যাতে প্রতিটি কীস্ট্রোকে টোটাল রকিয়ালকুলটে হলেও ফিল্ডের কার্সর পজিশন বজায় থাকে।

`_LineControllers({String description = "", String quantity = '1', String unitPrice = ""})`

তিনটি `TextEditingController` (`description`, `quantity`, `unitPrice`) প্রাথমিক টেক্সট সহ তৈরি করে।

`double get amount`

`quantity` ও `unitPrice` পার্স করে গুণফল হিসেবে লাইনের মোট পরিমাণ রিটার্ন করে; পার্স ব্যর্থ হলে 0 ধরে নেয়।

`bool get isBlank`

`description` ও `unitPrice` উভয়ই ফাঁকা কিনা তা যাচাই করে — খালি লাইন ফিল্টার করতে ব্যবহৃত।

`InvoiceItem toItem()`

বর্তমান কন্ট্রোলার মানগুলো থেকে একটি `InvoiceItem` মডেলে তৈরি করে।

`void dispose()`

তনিটি কন্ট্রোলার `dispose` করে।

ক্লাস/উইজটে: `_InvoiceFormSheet`

`ConsumerStatefulWidget` — নতুন ইনভয়েসে তৈরি বা বদ্যমান ড্রাফট এডিট করার সম্পূর্ণ ফর্ম।

`_InvoiceFormSheet({this.existing})`

কনস্ট্রাক্টর, ঐচ্ছকি `existing` ইনভয়েসে গ্রহণ করে (এডিট মোডের জন্য)।

`ConsumerState<_InvoiceFormSheet> createState()`

`_InvoiceFormSheetState` তৈরি করে।

ক্লাস: `_InvoiceFormSheetState`

`_InvoiceFormSheet`-এর স্টেটে — ফর্মের সব ফিল্ড, লাইন আইটেম, ভ্যালিডেশন ও সাবমিশন পরিচালনা করে।

`bool get _isEdit`

`widget.existing` null না হলে `true` — বর্তমানে এডিট মোডে আছে কিনা তা নির্দেশ করে।

`void initState()`

`existing` থেকে ক্লায়েন্ট আইডি, তারিখ, পেমেন্ট শর্ত, বিবরণ, নোট, ট্যাক্স রটে, ডিসকাউন্ট ও লাইন আইটেমগুলো প্রিফিলি করে; নতুন ইনভয়েসের জন্য একটি খালি লাইন যোগ করে।

`void dispose()`

সব টেক্সট কন্ট্রোলার ও প্রতিলিইন কন্ট্রোলার `dispose` করে।

`double get _subtotal`

সব লাইনের `amount` যোগ করে সাবটোটাল রটার্ন করে।

`double get _total`

সাবটোটালের সাথে ট্যাক্স রটে প্রয়োগ করে ও ডিসকাউন্ট ব্যয়োগ করে মোট হিসাব করে (এটি শুধু প্রভিডি, প্রকৃত হিসাব সার্বভারে হয়)।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, ক্লায়েন্ট নির্বাচতি কনি, অন্তত একটি লাইন আছে কনি এবং ডিউ ডেই ইনভয়েসে তারিখের পরে কনি যাচাই করে। এরপর `InvoiceRequest` তৈরি করে (ট্যাক্স রটে শতাংশ হিসাবে পাঠানো হয়, নির্দিষ্ট অ্যামাউন্ট হিসাবে নয়) এবং এডিট মোডে `updateItem` অথবা নতুন মোডে `create` কল করে; সফলতা/ব্যর্থতায় সূন্যাকবার/এরর দেখায়।

`Widget build(BuildContext context)`

সম্পূর্ণ ফর্ম UI তৈরি করে — ক্লায়েন্ট নির্বাচন (এডিট মোডে অপরিবর্তনীয়), ইনভয়েস/ডিউ তারিখ, পেমেন্ট শর্ত, লাইন আইটেম তালিকা (`_LineRow` দিয়ে), ট্যাক্স/ডিসকাউন্ট ইনপুট, সাবটোটাল/টোটাল প্রভিডি কার্ড, বিবরণ/নোট এবং সাবমিট বাটন।

ক্লাস/উইজটে: `_LineRow`

একটি একক ইনভয়েস লাইনের UI সারি (description, quantity, unit price, amount, remove বাটন)।

`_LineRow({super.key, required this.line, required this.enabled, required this.onChanged, required this.onRemove})`

কনস্ট্রাক্টর; `onRemove` শেষে অবশিষ্ট লাইনে null থাকে কারণ ইনভয়েসে অন্তত একটি লাইন থাকতেই হবে।

`Widget build(BuildContext context)`

description ফিল্ড (সাথে remove আইকন), quantity ও unit price ফিল্ড এবং গণনাকৃত amount টেক্সট প্রদর্শন করে।

ক্লাস/উইজটে: `_TotalRow`

সাবটোটাল/টোটাল দেখানোর একটি সাধারণ সারি।

`_TotalRow({required this.label, required this.amount, this.emphasised = false})`

কনস্ট্রাক্টর; `emphasised true` হলে বোল্ড/বড় স্টাইল প্রয়োগ হয় (টোটালরে জন্য)।

`Widget build(BuildContext context)`

লবেলে ও মান-ফরম্যাটেডে অ্যামাউন্ট পাশাপাশি দেখায়।

ফাইল: `lib/features/finance/record_payment_sheet.dart`

সংক্ষিপ্ত বিবরণ: ক্লায়েন্টের কাছ থেকে প্রাপ্ত অর্থ রেকর্ড করার জন্য বটম শিট ফর্ম সরবরাহ করে, যা কোনও নির্দিষ্ট ইনভয়েসের বিপরীতে বা সাধারণভাবে খোলা যায়।

উপ-লভেলে ফাংশন: `Future<void> showRecordPaymentSheet(BuildContext context, {Invoice? against})`

`_RecordPaymentSheet` দেখানোর জন্য মডাল বটম শিট খোলতে; `against` দেওয়া থাকলে সেই ইনভয়েসের বিপরীতে পেমেন্ট রেকর্ড করা হয়, নাহলে ক্লায়েন্ট নির্বাচন প্রয়োজন হয়।

ক্লাস/উইজটে: `_RecordPaymentSheet`

`ConsumerStatefulWidget` — পেমেন্ট রসিডি রেকর্ড করার ফর্ম।

`_RecordPaymentSheet({this.against})`

কনস্ট্রাক্টর, ঐচ্ছকি টার্গেটে `Invoice` গ্রহণ করে।

`ConsumerState<_RecordPaymentSheet> createState()`

`_RecordPaymentSheetState` তৈরি করে।

ক্লাস: `_RecordPaymentSheetState`

`_RecordPaymentSheet`-এর স্টেটে — ফর্ম ফিল্ড, ভ্যালিডেশন ও সাবমিশন পরিচালনা করে।

`bool get _againstInvoice`

`widget.against` null না হলে `true` — নরিদৃষ্ট ইনভয়সেরে বপিরীতে পমেন্ট রকের্ড করা হচ্ছে কনি তা নরিদৃশে করে।

`void dispose()`

`_amount`, `_reference`, `_notes` কন্ট্রোলার `dispose` করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, ক্লায়েন্ট নরিবাচতি কনি যাচাই করে, `createReceipt` কল করে

`CreatePaymentReceiptRequest` সহ (ক্লায়েন্ট আইডি, অ্যামাউন্ট, তারিখ, পদ্ধতি, ইনভয়সে আইডি, রফোরন্স, নোট)। সফল হলে `invoicesProvider` ইনভ্যালিডেট করে (ইনভয়সেরে ব্যালন্স পরবির্ততি হয়েছে) এবং শাটি বন্ধ করে স্ন্যাকবার দেখায়।

`Widget build(BuildContext context)`

ফর্ম UI তরৈ করে — ক্লায়েন্ট (ইনভয়সে থেকে এলে অপরবির্তনীয়, নাহলে ড্রপডাউন), অ্যামাউন্ট (ডিফিল্ট বকয়ো পরমাণ), পমেন্ট তারিখ (ভবষিৎ তারিখ অনুমোদতি নয়), পদ্ধতি ড্রপডাউন, রফোরন্স, নোট এবং একটিসিতর্কবার্তা যে রকের্ড করাই নশ্চিতিকরণ নয়।

ক্লাস/উইজেটে: `_Note`

একটি ছোট তথ্য-আইকন সহ সতর্কতা টক্সট প্রদর্শনকারী উইজেটে।

`_Note(this.text)`

কনস্ট্রাক্টর, প্রদর্শনযোগ্য টক্সট গ্রহণ করে।

`Widget build(BuildContext context)`

একটি info আইকন ও টক্সট পাশাপাশি সিরতি দেখায়।

ফাইল: `lib/features/finance/submit_expense_sheet.dart`

সংক্ষিপ্ত ববিরণ: কর্মীদের এক্সপেন্স কলইম সাবমিট করার বটম শাটি ফর্ম — AI দিয়ে ববিরণ ড্রাফট করা ও রসিটি আপলোডের সুবধিসহ। ডকুমেন্টেশনে বলা হয়েছে এটি ফোনে ব্যবহারকারীরা সবচয়ে বশেকিরে থাকেন — খরচ করার সাথে সাথেই দাঁড়িয়ে কলইম করা।

টপ-লেভেলে ফাংশন: `Future<void> showSubmitExpenseSheet(BuildContext context)`

`_SubmitExpenseSheet` দেখানোর জন্য মডাল বটম শাটি খোলে।

ক্লাস/উইজেটে: `_SubmitExpenseSheet`

`ConsumerStatefulWidget` — এক্সপেন্স কলইম সাবমিশনের সম্পূর্ণ ফর্ম।

`_SubmitExpenseSheet()`

কনস্ট্রাক্টর, কোনো প্রারামিটার নাই।

`ConsumerState<_SubmitExpenseSheet> createState()`

`_SubmitExpenseSheetState` তরৈ করে।

ক্লাস: `_SubmitExpenseSheetState`

`_SubmitExpenseSheet`-এর স্টেটে — ফর্ম ফিল্ড, AI ড্রাফটিং, রসিটি আপলোড ও সাবমিশন পরচালনা করে।

`void dispose()`

`_title`, `_description`, `_amount`, `_vendor`, `_category` কন্ট্রোলার `dispose` করে।

`Future<void> _draftWithAi()`

`_RoughNotesDialog` দেখিয়ে ইউজারের কাছ থেকে রুক্স নোট নেয়, তারপর ইতিমধ্যে টাইপ করা ভন্ডের/অ্যামাউন্ট/ক্যাটাগরি প্রসঙ্গ হিসেবে পাঠিয়ে `composeExpense` কল করে। ফলাফলের টাইটলে ও বিবরণ ফিল্ডে বসায় (ফাঁকা হলে বসায় না) — এখানে কিছু সাবমিট হয় না, শুধু ফিল্ড পূরণ হয়।

`Future<void> _pickReceipt()`

`pickAttachment` দিয়ে ফাইল নির্বাচন করায়, তারপর `apiClientProvider.uploadDocument` দিয়ে আপলোড করে এবং ফলে পাওয়া URL সংরক্ষণ করে; ব্যর্থ হলে নির্বাচন বাতিল করে এরর দেখায়।

`Future<void> _pickDate()`

`showDatePicker` দিয়ে তারিখ নির্বাচন করায় — সর্বোচ্চ এক বছর পছিনে, ভবিষ্যতে নয় (কারণ ভবিষ্যতে খরচ করা যায় না)।

`Future<void> _submit()`

রসিটি আপলোড চলমান থাকলে সাবমিট আটকায়, ফর্ম ভ্যালিডেট করে, `expensesProvider.notifier.submit` কল করে `CreateExpenseRequest` সহ (title, description, amount, expenseDate, vendorName, category, receiptUrl)। সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়।

`Widget build(BuildContext context)`

পারমিশন (`AiPermissions.chat`) থাকলে AI ড্রাফট বাটন সহ ফর্ম UI তৈরি করে — অ্যামাউন্ট, বিবরণ (AI ড্রাফট আইকনসহ), টাইটলে (থাকলে দৃশ্যমান), তারিখ, ভন্ডের, ক্যাটাগরি, রসিটি ফিল্ড (`_ReceiptField`) এবং সাবমিট বাটন।

ক্লাস/উইজটে: `_ReceiptField`

সংযুক্ত রসিটি ফাইল প্রদর্শন বা "Attach a receipt" বাটন দেখানোর উইজটে।

`_ReceiptField({required this.receipt, required this.uploading, required this.onPick, required this.onRemove})`

কনস্ট্রাক্টর — বর্তমান নির্বাচতি রসিটি, আপলোড হচ্ছে কনি এবং পিক/রমিভ কলব্যাক গ্রহণ করে।

`Widget build(BuildContext context)`

`receipt` null হলে "Attach a receipt (optional)" বাটন দেখায়; নাহলে ফাইলের নাম, লোডিং ইন্ডিকেটর (আপলোডকালে) বা রমিভ বাটন সহ একটি কার্ড দেখায়।

ক্লাস/উইজটে: `_RoughNotesDialog`

`StatefulWidget` — AI ড্রাফটিংয়ের ইনপুট হিসেবে রুক্স নোট নেওয়ার জন্য একটি ডায়ালগ। মূল ফর্মে আলাদা ফিল্ড না রেখে ডায়ালগ ব্যবহার করা হয়েছে যাতাবে বোঝা যায় এই নোট নজি সাবমিট হবে না, শুধু AI-এর ইনপুট।

`_RoughNotesDialog()`

কনস্ট্রাক্টর, কোনো প্রারামিটার নেই।

State<_RoughNotesDialog> createState()

_RoughNotesDialogState তৈরি করে।

ক্লাস: _RoughNotesDialogState

_RoughNotesDialog-এর স্টেটে — নোট টেক্সট ফিল্ড পরিচালনা করে।

void dispose()

_notes কন্ট্রোলার dispose করে।

Widget build(BuildContext context)

একটি `AlertDialog` দেখায় যখন একটি মাল্টি-লাইন টেক্সট ফিল্ড আছে; "Cancel" চাপলে null রিটার্ন করে ডায়ালগ বন্ধ করে, "Draft" চাপলে টাইপ করা নোট রিটার্ন করে।

ফাইল: lib/features/finance/vendor_expenses_screen.dart

সংক্ষিপ্ত বিবরণ: একজন নির্দিষ্ট সাপ্লায়ার/ভেন্ডরের নামে দাখল করা সব খরচের (Expense) দাবি একত্রে দেখায়। শীর্ষে পরিশোধিত মোট অংক এবং নচি প্রতিটি claim-এর কার্ড তালিকা রেন্ডার করে।

টপ-লেভেলে প্রোভাইডার: vendorExpensesProvider

FutureProvider.autoDispose.family<List<Expense>, String> — vendorName কী হিসেবে নিয়ে

FinanceRepository.expensesByVendor কল করে সর্বোচ্চ ৫০টি খরচ (page.content) ফেরত দেয়।

প্রতিটি ভিন্ন vendorName-এর জন্য আলাদা instance তৈরি হয় এবং widget dispose হলে অটো-ডিসপোজ হয়।

ক্লাস/উইজটে: VendorExpensesScreen

ConsumerWidget। একটি ভেন্ডরের নাম নিয়ে (constructor-এ vendorName আবশ্যিক) তার সব claim প্রদর্শন করে। খরচ ম্যাচিং হয় claim-এ লেখা নামের ভিত্তিতে, ভেন্ডর রেকর্ডের লিংকের ভিত্তিতে নয় — তাই একই ভেন্ডরের নাম দুইভাবে বানান করলে দুইটি আলাদা ভেন্ডর হিসেবে দেখা যাবে, ফলে টোটাল প্রত্যাশার চেয়ে কম দেখাতে পারে।

static void open(BuildContext context, {required String vendorName})

MaterialPageRoute দিয়ে VendorExpensesScreen-কে নতুন রুটে push করে নেভিগেটে করার শর্টকাট। কলকারীকে Navigator ইনস্ট্যান্স নজি তৈরি করতে হয় না।

Widget build(BuildContext context, WidgetRef ref)

vendorExpensesProvider(vendorName)-কে watch করে async অবস্থা (loading/error/data) অনুযায়ী UI দেখায়। RefreshIndicator-এর onRefresh প্রোভাইডারটি invalidate করে পুনরায় লোড করায়। ডটো এলে শুধু ExpenseStatus.paid স্ট্যাটাসের খরচগুলো যোগ করে "settled" (পরিশোধিত) মোট বের করা হয় — pending বা rejected claim দেখানো হলো এই যোগফলে ধরা হয় না, কারণ সেগুলো এখনো কোম্পানি থেকে বের হওয়া টাকা নয়। খালি তালিকার ক্ষেত্রে EmptyState দেখায়, আর প্রতিটি খরচের জন্য headline, expense number, তারিখ, ক্যাটাগরি, amount ও StatusChip সহ একটি AppCard রেন্ডার করে।

ফাইল: lib/features/home/home_repository.dart

সংক্ষিপ্ত ববিরণ: হোম স্ক্রিনিং নোটশি বোর্ড (ঘোষণা ও ছুটির দিন) এবং কছি অতিরিক্ত ড্যাশবোর্ড ফগিার (ওপনে রকি়োস্ট সংখ্যা, রভিডি স্কোর, HR/CRM সন্ধ্যাপশট) সরবরাহ করে। প্রতিটি কিল ঐচ্ছকি — পারমশিন না থাকলে বা কনফগিার না থাকলে ব্য়র্থতাকে গলি ফলে খালি/null রটার্ন করে, যাত ড্যাশবোর্ড এরর স্ক্রিনিং বদলে শুধু ছোট হয়ে যায়।

ক্লাস: Announcement

GET /api/announcements/active থেকে আসা একটি কীম্পান ঘোষণা। id, title, body, priority, publishedAt, createdAt, createdByName ফিল্ড ধারণ করে।

String? get shownAt

publishedAt থাকলে সেটাই, না থাকলে createdAt ফরতে দিয়ে — অর্থাৎ প্রকাশের তারখি না থাকলে লখোর তারখি ফলব্যাক করে।

factory Announcement.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে Announcement অবজেক্ট তরৈকিরে, প্রতিটি ফিল্ডেরে জন্য় null-safe ডফিল্ট (খালি স্ট্রিং/শূন্য) ব্য়বহার করে।

ক্লাস: Holiday

একটি ছুটির দিনেরে তথ্য়: id, name, holidayDate, holidayType।

int get daysAway

আজকের তারখি থেকে ছুটির তারখি পর্যন্ত কত পূরণ দিন বাকতি হিসাব করে; তারখি parse করা না গেলে ০ ফরতে দিয়ে, তারখি পার হয়ে গেলে ঋণাত্মক সংখ্যা আসে।

String get countdownLabel

daysAway-এর ভিত্তিতে "Today", "Tomorrow", "Passed", বা "in \$d days" — এই লবেলেগুলোর একটি রটার্ন করে (switch expression ব্য়বহার করে)।

factory Holiday.fromJson(Map<String, dynamic> json)

JSON থেকে Holiday তরৈকিরে, নাল-সফে ডফিল্ট সহ।

ক্লাস: HomeRepository

হোম স্ক্রিনিং নোটশি বোর্ড ও ঐচ্ছকি ফগিারগুলোর জন্য় API কল সংকলন করে। কনস্ট্রাক্টরে একটি ApiClient নেয়।

Future<List<Announcement>> activeAnnouncements()

_optionalList ব্য়বহার করে /announcements/active থেকে সক্রিয় ঘোষণার তালিকা আনে।

Future<List<Holiday>> currentYearHolidays()

_optionalList ব্য়বহার করে /hr/holidays/current-year থেকে চলতি বছরের ছুটির তালিকা আনে।

Future<int?> openRequestCount()

/service-requests/assigned-to-me থেকে এই কর্মচারীকে অ্যাসাইন করা অনুরোধগুলো আনে এবং যগুলোর স্ট্যাটাস COMPLETED/CANCELLED/REJECTED নয় সগুলোর সংখ্যা গোনো। ApiException ধরে null রটার্ন করে (যেমন পারমশিন না থাকলে)।

Future<double?> latestReviewScore(int employeeId)

নর্দিষ্ট কর্মচারীর জন্য `/hr/performance/employee/$employeeId` থেকে সর্বশেষ একটি (size=1) পারফরম্যান্স রিভিউ এনে তার `overallScore` ফেরত দেয়। এন্ট্রি না থাকলে বা এরর হলে `null`।

Future<List<T>> _optionalList<T>(String path, T Function(Map<String, dynamic>) fromJson)

একটি প্রাইভেট হলেপার — GET করে একটি লিস্ট আনে এবং প্রতিটি আইটেমকে `fromJson`-এর মাধ্যমে মডেলে রূপান্তর করে; `ApiException` হলে খালি লিস্ট (`const []`) রিটার্ন করে।

উপ-লভেলে প্রোভাইডার: homeRepositoryProvider

Provider<HomeRepository> — `ApiClientProvider`-এর উপর নির্ভর করে `HomeRepository` ইনস্ট্যান্স তৈরি করে।

ক্লাস: NoticeBoard (@immutable)

নোটিশ বোর্ডের সম্মিলিত অবস্থা ধারণ করে: `announcements`, `holidays` (আসন্ন, নিকটতমটি আগে), এবং `openRequests` (int? — `null` মানে নির্ধারণ করা যায়নি, শূন্য একটি বৈধ উত্তর)।

উপ-লভেলে প্রোভাইডার: latestReviewScoreProvider

FutureProvider<double?> — প্রথমে `myEmployeeProvider` থেকে `employee` রেকর্ড আনে (না থাকলে `null`), তারপর তার `id` দিয়ে `latestReviewScore` কল করে। এটি আলাদা প্রোভাইডার রাখা হয়েছে কারণ এটি `employee id`-এর উপর নির্ভরশীল, তাই `employee` রেকর্ড না আসা পর্যন্ত চলতে পারে না।

উপ-লভেলে প্রোভাইডার: noticeBoardProvider

FutureProvider<NoticeBoard> — ঘোষণা, ছুটি ও ওপেন-রকর্ড — তিনটি কল একসাথে (concurrently) শুরু করে। প্রতিটি Future-এ আগেই `.ignore()` কল করা হয়, যাত্রে একটি Future ব্যর্থ হয় ফাংশন আগের রিটার্ন করলেও বাকি Future-গুলোর প্রত্যাখ্যান "unhandled exception" হিসেবে রিপোর্ট না হয়। ছুটিগুলো `daysAway >= 0` দিয়ে ফিল্টার করে নিকটতমটি আগের সাজায়; ঘোষণাগুলো `priority` (বিশেষ আগের) তারপর সাম্প্রতিকতা অনুযায়ী সাজায়।

ক্লাস: HrSnapshot

GET `/hr/dashboard/summary`-এর সংক্ষিপ্ত রূপ — ফোনের টাইলে দেখানোর মতো কয়েকটি ফিগার মাত্র (`totalEmployees`, `presentToday`, `onLeaveToday`, `absentToday`, `openPositions`); রসেপন্সে আরও অনেকে কিছু (চার্ট-ভিত্তিক ডটো) থাকলেও তা এখানে পার্স করা হয় না।

factory `HrSnapshot.fromJson(Map<String, dynamic> json)`

একটি লোকাল হলেপার `int i(String key)` ব্যবহার করে প্রতিটি সংখ্যার ফিল্ড `null-safe` ভাবে পড়ে `HrSnapshot` তৈরি করে।

ক্লাস: CrmSnapshot

GET `/crm/dashboard/summary`-এর সংক্ষিপ্ত রূপ — `pipelineValue`, `wonThisMonth`, `qualifiedLeads`, `openOpportunities`।

factory `CrmSnapshot.fromJson(Map<String, dynamic> json)`

JSON থেকে ফিল্ডগুলো (কিছু ভিন্ন কী-নাম যমেন `qualifiedLeadsCount`, `openOpportunitiesCount` সহ) `null-safe` ডিফল্ট দিয়ে পার্স করে।

টপ-লেভেলে প্রোভাইডার: hrSnapshotProvider

FutureProvider<HrSnapshot?> — currentUserProvider watch করে, /hr/dashboard/summary GET করে।
ApiException-এ isForbidden বা isNotFound হলে null রিটার্ন করে (EMPLOYEE_VIEW না থাকলে পুরো
ড্যাশবোর্ড ভাঙে না), অন্য এরর হলে rethrow করে।

টপ-লেভেলে প্রোভাইডার: crmSnapshotProvider

FutureProvider<CrmSnapshot?> — একই প্যাটার্নে /crm/dashboard/summary থেকে CRM সন্ধ্যাপশট
আনে, forbidden/not-found হলে null।

ফাইল: lib/features/home/home_screen.dart

সংক্ষিপ্ত ববিরণ: কর্মচারীর প্রধান ড্যাশবোর্ড স্ক্রিনি — অভিবাদন, পাঞ্চ কার্ড, পরসিংখ্যান,
দলীয় আজকের অবস্থা, পাইপলাইন, দ্রুত অ্যাকশন, ছুটির ব্যালেন্স এবং নোটশি বোর্ড একত্রে
দেখায়। প্রতিটি সেকশন ঐচ্ছকি — ডটো না থাকলে সেকশনটি কেবল উধাও হয়ে যায়, এরর দেখায় না।

ক্লাস/উইজটে: HomeScreen

ConsumerWidget। পুরো হোম ট্যাবের বুট স্ক্রিনি — নচিরে সব প্রাইভেটে সেকশন উইজটে একটি
ListView-এ সাজায়।

Future<void> _refresh(WidgetRef ref)

পুল-টু-রিফ্রেশ হ্যান্ডলার — myEmployeeProvider, noticeBoardProvider, latestReviewScoreProvider
invalidate করে, এবং attendance ও leave কন্ট্রোলারের refresh() একসাথে (Future.wait) চালায়।

Widget build(BuildContext context, WidgetRef ref)

currentUserProvider ও myEmployeeProvider watch করে _Greeting, PunchCard, _Stats, _TeamToday,
_Pipeline, _QuickActions, _LeaveBalances, _NoticeBoard উইজটেগুলোকে একটি স্ক্রলযোগ্য
ListView-এ সাজায়, RefreshIndicator দিয়ে মোড়ানো।

ক্লাস/উইজটে: _Greeting

StatelessWidget। নাম, রোল, ছবি ও ইনশিয়াল নিয়ে সময়ভিত্তিক অভিবাদন ("Good
morning/afternoon/evening"), ড্রয়ার খোলার মেনু বাটন এবং অ্যাভাটার দেখায়। মন্তব্য অনুযায়ী,
হোম-ই একমাত্র ট্যাব যার নজিস্ব app bar নেই, তাই ড্রয়ারে ঢোকান পথ এখানই থাকতে হয়।

Widget build(BuildContext context)

বর্তমান ঘণ্টা অনুযায়ী গ্রিটিং স্ট্রিং বাছাই করে, মেনু আইকন বাটন
(appShellScaffoldKey.currentState?.openDrawer()), নাম/রোল টেক্সট এবং Avatar উইজটে নিয়ে Row
তৈরি করে।

ক্লাস/উইজটে: _Stats

ConsumerWidget। উপস্থিতি শতাংশ, ছুটি ব্যালেন্স, ওপনে রকিওয়েস্ট ও সর্বশেষ রভিউ স্কোর —
এই চারটি StatCard একটি হি-কলাম গ্রিডে দেখায়।

Widget build(BuildContext context, WidgetRef ref)

attendanceControllerProvider, leaveControllerProvider, noticeBoardProvider, latestReviewScoreProvider watch করে চারটি StatCard তৈরি করে। গ্রিডি টাইলারে উচ্চতা fixed (aspect ratio নয়) রাখা হয়েছে যাতে টেক্সট-স্কেলে বড় করলে বা ছোট স্ক্রিনে লেবেলে কাটা না পড়ে — MediaQuery.textScalerOf(context).scale(1) থেকে হিসাব করা extent ব্যবহার করে SliverGridDelegateWithFixedCrossAxisCount-এর mainAxisExtent সেট করে।

ক্লাস/উইজটে: _QuickActions

ConsumerWidget। নভেগেশন রজিস্ট্রি (ড্রয়ারের মতো একই সোর্স) থেকে অ্যাকশনের তালিকা বানিয়ে একটি লিস্ট-টাইল কার্ড দেখায়, যাতে দুটি জায়গায় নভেগেশনের কপি আলাদাভাবে বজায় রাখতে না হয়। "Search everything" ও "My profile" রজিস্ট্রির বাইরে আলাদাভাবে যোগ করা হয়।

Widget build(BuildContext context, WidgetRef ref)

currentUserProvider ও permissionControllerProvider থেকে buildNavGroups/flattenGroups কল করে অ্যাকশনের List<NavDestination> বানায়, তারপর প্রতিটি জন্য একটি ListTile (আইকন, লেবেল, chevron) রেন্ডার করে; ট্যাপে push ফ্ল্যাগ অনুযায়ী context.push বা context.go কল হয়।

ক্লাস/উইজটে: _LeaveBalances

ConsumerWidget। কর্মচারীর ছুটি ব্যালেন্স (সর্বোচ্চ ৪টি) দেখায়; কনফিগার করা না থাকলে কিছুই রেন্ডার করে না।

Widget build(BuildContext context, WidgetRef ref)

leaveControllerProvider-এর balances পড়ে, খালি হলে SizedBox.shrink(); নয়তো SectionHeader ("See all" বাটনসহ, Routes.leave-এ নভেগিটে করে) এবং _BalanceRow-এর একটি কার্ড লিস্ট রেন্ডার করে।

ক্লাস/উইজটে: _BalanceRow

StatelessWidget। এক ধরনের ছুটির ব্যালেন্স — নাম, "X of Y left" টেক্সট এবং একটি প্রোগ্রেস বার (consumedFraction) দেখায়।

Widget build(BuildContext context)

LeaveBalance থেকে লেবেল, বাকি/মোট দিন টেক্সট এবং LinearProgressIndicator তৈরি করে।

static String _n(double value)

সংখ্যাটি পূর্ণসংখ্যা হলে দশমিক ছাড়া, না হলে এক দশমিক স্থান পর্যন্ত ফরম্যাট করে ফেরত দেয়।

ক্লাস/উইজটে: _NoticeBoard

ConsumerWidget। সর্বোচ্চ ৩টি ঘোষণা ও ৩টি আসন্ন ছুটি — উভয়ই খালি হলে কিছুই রেন্ডার করে না।

Widget build(BuildContext context, WidgetRef ref)

noticeBoardProvider থেকে বোর্ড পড়ে, প্রতিটি ঘোষণার জন্য আইকন/শিরোনাম/বডি (২ লাইন পর্যন্ত)/সময়-আপেক্ষিক টেক্সট, আর প্রতিটি ছুটির জন্য নাম ও countdownLabel দেখায়; দুটোর মাঝে একটি Divider।

ক্লাস/উইজটে: _TeamToday

ConsumerWidget। HR সন্ধ্যাপশট থেকে "আজ কতজন উপস্থিতি/ছুটি/অনুপস্থিতি/কতটি পিড খালি" — এই চারটি StatCard (২x২) দেখায়; সন্ধ্যাপশট null হলে (পারমিশন নাই) কিছুই রেন্ডার হয় না।

Widget build(BuildContext context, WidgetRef ref)

hrSnapshotProvider watch করে, presentToday/totalEmployees, onLeaveToday, absentToday (শূন্যের বশেহিলে warning রঙ), openPositions (ট্যাপে Routes.recruitment) দেখায়।

ক্লাস/উইজটে: _Pipeline

ConsumerWidget। CRM সন্ধ্যাপশট থেকে "ওপনে পাইপলাইন" ও "এ মাসে জতো" — দুটি StatCard দেখায়; সন্ধ্যাপশট null হলে কিছুই রেন্ডার হয় না।

Widget build(BuildContext context, WidgetRef ref)

crmSnapshotProvider watch করে, পাইপলাইন মূল্য (ট্যাপে Routes.crm) এবং এ মাসে জতো অংক/যোগ্য লডি সংখ্যা দেখায়।

ফাইল: lib/features/hrpolicy/hrpolicy_models.dart

সংক্ষিপ্ত ববিরণ: HR-এর স্থায়ী নয়িমকানুন — কোন দিনগুলো ছুটি, কী ধরনের ছুটিও কত পাওয়া যায়, কাজের সময়সূচি (শিফট), এবং এসব রকের্ড করা চিঠি (letter) — এই সবের মডেলে ও রকি়োস্টে DTO ধারণ করে।

ক্লাস: HrPolicyPermissions (abstract final class)

হলডি, লভি পলসি, শিফট ও লটোর — চারটি এন্টিটির জন্য view/create/update/delete পারমিশন কোডেরে স্ট্যাটিক কনস্ট্যান্ট সংকলন। মন্তব্য অনুযায়ী, একটি চিঠি "ইস্যু" করা একটি আপডেটে, তার নজিস্ব আলাদা কোড নই (letterUpdate ব্যবহার হয়)।

উপ-লভেলে কনস্ট্যান্ট: holidayTypes

List<String> — HolidayType এনামেরে মরির: NATIONAL, RELIGIOUS, OPTIONAL, COMPANY।

উপ-লভেলে কনস্ট্যান্ট: shiftTypes

List<String> — ShiftType-এর মরির: MORNING, AFTERNOON, EVENING, NIGHT, FULL_DAY, FLEXIBLE।

উপ-লভেলে কনস্ট্যান্ট: letterTypes

List<String> — LetterType-এর মরির, কর্মজীবনের ক্রম অনুযায়ী সাজানো: OFFER থেকে TERMINATION পর্যন্ত ১২টি ধরন।

ক্লাস: Holiday

একটি ছুটির দিন — id, name, holidayType, ঐচ্ছকি holidayDate, description।

bool get isOptional

holidayType == 'OPTIONAL' কনি তা জানায় — ঐচ্ছকি ছুটি (যা নেওয়া বা না নেওয়া যায়) সাধারণ বন্ধ দিন থেকে আলাদা করার জন্য।

factory Holiday.fromJson(Map<String, dynamic> json)

JSON থেকে Holiday তরৈ করে, null-safe ডফিল্ট সহ (holidayType ডফিল্ট 'COMPANY')।

ক্লাস: HolidayRequest

POST/PUT /hr/holidays-এর জন্য রিকোয়েস্ট বডাি name, holidayDate (yyyy-MM-dd), holidayType আবশ্যিক; description ঐচ্ছকি। আপডেটেও তনিটি ফিল্ড শর্তহীনভাবে পাঠানো হয়, শুধু description-এ null-guard আছে।

Map<String, dynamic> toJson()

নাম ও ডসেক্রিপশন ট্রমি করে, ডসেক্রিপশন খালি না হলে তবই সটেকি JSON-এ রাখে।

ক্লাস: LeavePolicy

একটি নির্দিষ্ট ধরনের ছুটির (leave type) নমি — এনটাইটলেমেন্ট, ক্যারি-ফরওয়ার্ড সীমা, অনুমোদন প্রয়োজন কনি, বতেনসহ কনি, কার্যকরী হওয়ার আগে কত মাস অপেক্ষা, সক্রিয় কনি, এবং ঐচ্ছকি employment type ও একটানা সর্বোচ্চ দনি।

String get terms

এনটাইটলেমেন্ট, বতেনসহ/ছাড়া, ক্যারি-ফরওয়ার্ড তথ্য, একটানা সর্বোচ্চ দনি এবং কার্যকর হওয়ার শর্ত — এই অংশগুলোকে " . " দয়ি জোড়া দয়ি একটি বাক্যের মতো সারাংশ তরৈকিরে, তালকির সাবটাইটলে ব্যবহারের জন্য।

factory LeavePolicy.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড null-safe ডফিল্ট (যেমন requiresApproval ও paid ডফিল্ট true) সহ পার্স করে।

ক্লাস: LeavePolicyRequest

POST/PUT /hr/leave-policies-এর রিকোয়েস্ট বডাি requiresApproval, canCarryForward, paid — তনিটি বুলয়ান ফিল্ড শর্তহীনভাবে পাঠানো হয় (কোনোটি বাদ পড়লে false ধরে নেওয়া হবে, বিশেষত paid বাদ গেলে অজান্তে unpaid হয়ে যেতে পারে বলে সতর্কতা)। leaveType ও annualEntitlement @NotNull, entitlement @Min(0)।

factory LeavePolicyRequest.from(LeavePolicy policy)

বদ্যমান LeavePolicy থেকে একটি রিকোয়েস্ট DTO তরৈকিরে — এডিটি ফর্মের বদ্যমান পলসিলিড করার জন্য ব্যবহৃত।

Map<String, dynamic> toJson()

আবশ্যকি ও তনিটি বুলয়ান ফিল্ড সবসময় পাঠায়; ঐচ্ছকি ফিল্ডগুলো (employmentType, maxCarryForward, maxConsecutiveDays, applicableFromMonths) শুধু non-null হলে যুক্ত করে।

ক্লাস: Shift

একটি কর্মপদ্ধতি (working pattern) — id, name, shiftType, গ্রসে পরিয়িড, flexible/nightShift ফ্ল্যাগ, active, workingMinutes, এবং ঐচ্ছকি startTime/endTime/weeklyOffDays/description/notes।

double get hours

workingMinutes ঋণাত্মক হলে (রাতের শফিট মধ্যরাত পার হলে ব্যাকএন্ড থেকে ঋণাত্মক আসে) ২৪ ঘণ্টা যোগ করে সংশোধন করে, তারপর ঘণ্টায় রূপান্তর করে।

String get window

startTime ও endTime-কে প্রথম ৫ অক্ষর (HH:mm) নয়ি "09:00 — 17:30" আকারে ফর্ম্যাট করে; কোনোটি না থাকলে "No hours set" রটার্ন করে। একটি অভ্যন্তরীণ হলেপার short(String? time) টাইম স্ট্রিং ছাঁটাই করে।

factory Shift.fromJson(Map<String, dynamic> json)

JSON থেকে Shift তৈরি করে, null-safe ডফিল্ট সহ (shiftType ডফিল্ট 'FULL_DAY')।

ক্লাস: ShiftRequest

POST/PUT /hr/shifts-এর রকি়োস্ট বডি name, shiftType, startTime, endTime (HH:mm:ss), flexible, nightShift আবশ্যিক। আপডেটে নাম, টাইপ, সময়, উভয় ফ্ল্যাগ, description ও notes শর্তহীনভাবে পাঠানো হয়; শুধু গ্রুপে পরিষিড ও weekly off days-এ null-guard আছে।

factory ShiftRequest.from(Shift shift)

বর্তমান Shift থেকে রকি়োস্ট তৈরি করে; startTime/endTime না থাকলে যথাক্রমে '09:00:00'/'17:00:00' ডফিল্ট ব্যবহার করে।

Map<String, dynamic> toJson()

একটি অভ্যন্তরীণ হেল্পার clean(String? value) স্ট্রিং ট্রিম করে খালি হলে null করে দেয়; description/notes শর্তহীনভাবে (even if null) পাঠানো হয়, আর gracePeriodMinutes/weeklyOffDays শুধু non-null/non-empty হলে যুক্ত হয়।

ক্লাস: HrLetter

কারও সম্পর্কে লেখা একটি চিঠি (অফার, কনফার্মেশন, ওয়ার্নিং) — id, letterType, issued (বুলিয়ান — একমুখী, ইস্যু হলে আর ফেরত যায় না), এবং ঐচ্ছকি রফোরেন্স নম্বর, ইস্যু-তারিখ, কন্টেন্ট, সাক্ষরকারী, ফাইল URL, employee/recipient তথ্য, লেখকের নাম।

String get about

employeeName থাকলে সেটাই, না থাকলে recipientName, নাহলে 'Unnamed' — যহেতু অফার লটার কোনও এখনো-কর্মচারী-নয় প্রার্থীকে যায়, employee এবং recipient আলাদা ফলিড।

factory HrLetter.fromJson(Map<String, dynamic> json)

JSON থেকে HrLetter তৈরি করে, null-safe ডফিল্ট সহ (letterType ডফিল্ট 'OFFER', issued ডফিল্ট false)।

ক্লাস: HrLetterRequest

POST /hr/letters-এর রকি়োস্ট বডি letterType, issueDate (yyyy-MM-dd), content (@NotBlank) আবশ্যিক। employeeId অথবা jobApplicationId — যেকোনো একটি (কোনোটাই একা আবশ্যিক নয়) হতে পারে, যা একই DTO-কে কর্মচারী ও চাকরি-আবেদনকারী উভয়ের জন্য কাজে লাগায়। কোনও আপডেটে এন্ডপয়েন্ট নই — চিঠি একটি ডিকুমেন্ট, সংশোধন মানই নতুন করে লেখা।

Map<String, dynamic> toJson()

একটি অভ্যন্তরীণ clean হেল্পার দিয়ে referenceNumber/signedBy ট্রিম-ও-খালি-চিকে করে, আবশ্যিক ফলিডগুলো সবসময় এবং ঐচ্ছকিগুলো শুধু non-null/non-empty হলে যুক্ত করে।

ফাইল: lib/features/hrpolicy/hrpolicy_repository.dart

সংক্ষিপ্ত ববিরণ: HR-এর স্থায়ী নিয়মের (ছুটি, লভি পলিসি, শফিট, চিঠি) সব API কল সংকলন করে, এবং প্রতিটির জন্য Riverpod প্রোভাইডার/কন্ট্রোলার সরবরাহ করে যগুলো তালিকা লোড, রফ্রেশ ও লোকাল স্টেটে আপডেটে (apply/remove) সামলায়।

ক্লাস: HrPolicyRepository

ApiClient মোড়ানো, চারটি সিকেশনে ভাগ করা (হলিডে, লভি পলিসি, শফিট, লেটার) সব HR-নিয়ম-সম্পর্কিত এন্ডপয়েন্ট কল করে। এন্ডপয়েন্ট পাথগুলো স্ট্যাটিক কনস্ট্যান্ট (`_holidays`, `_policies`, `_shifts`, `_letters`) হিসেবে সংজ্ঞায়িত।

`Future<List<Holiday>> holidaysForYear(int year)`

`$_holidays/year/$year` থেকে একটি বছরে সব ছুটি একটি বিয়ের লিস্ট হিসেবে আনবে (পেজেনিশন ছাড়া, কারণ এক বছরে সর্বোচ্চ কয়েক ডজন থাকবে)।

`Future<Holiday> createHoliday(HolidayRequest request)`

POST `$_holidays` কল করে নতুন ছুটি তৈরি করে ও রেসপন্স পার্স করে রিটার্ন করে।

`Future<Holiday> updateHoliday(int id, HolidayRequest request)`

PUT `$_holidays/$id` কল করে বর্তমান ছুটি আপডেট করে।

`Future<void> deleteHoliday(int id)`

`$_holidays/$id` ডিলিট করে (`_api.deleteText`)।

`Future<({...})> draftHoliday(String instructions)`

`$_holidays/ai-draft`-এ নির্দেশনা (যেমন "Eid al-Fitr next year") পাঠিয়ে AI-জেনারেটেডে ছুটির প্রস্তাব (নাম, তারিখ, ধরন, বিবরণ) একটি রিকর্ড টাইপ হিসেবে ফেরত দেয়; কিছুই সন্তোষ না, শুধু ফর্ম পূরণের জন্য উত্তর।

`Future<PagedResponse<LeavePolicy>> policies({int page = 0, int size = 50})`

`_policies` পাথ থেকে পেজেনিটেডে লভি পলিসি তালিকা আনবে।

`Future<LeavePolicy> createPolicy(LeavePolicyRequest request)`

POST করে নতুন লভি পলিসি তৈরি করে।

`Future<LeavePolicy> updatePolicy(int id, LeavePolicyRequest request)`

PUT `$_policies/$id` করে পলিসি আপডেট করে।

`Future<void> deletePolicy(int id)`

পলিসি ডিলিট করে।

`Future<List<LeavePolicy>> activePolicies()`

`$_policies/active` থেকে শুধু কার্যকর পলিসিগুলোর একটি বিয়ের লিস্ট আনবে — একটি বন্ধ (switched-off) পলিসি এখনো "আছে" কিন্তু কিছু দেয় না, তাই এটি আলাদা এন্ডপয়েন্ট।

`Future<String> draftPolicyDocument({required bool remoteWorkAllowed, String? additionalContext})`

`$_policies/draft`-এ কল করে ইতিমধ্যে কনফিগার করা এনটাইটলেমেন্ট থেকে একটি লভি পলিসি ডকুমেন্ট (prose, রেকর্ড নয়) তৈরি করায়; কিছুই সন্তোষ না।

`Future<PagedResponse<Shift>> shifts({int page = 0, int size = 50})`

`_shifts` পাথ থেকে পেজেনিটেডে শফিট তালিকা আনবে।

`Future<Shift> createShift(ShiftRequest request)`

নতুন শফিট তৈরি করে।

Future<Shift> updateShift(int id, ShiftRequest request)

বর্তমান শফিট আপডেট করে।

Future<Shift> toggleShift(int id)

PATCH `$_shifts/$id/toggle` কল করে শফিটের active/inactive অবস্থা টগল করে।

Future<void> deleteShift(int id)

শফিট ডিলিট করে।

Future<PagedResponse<HrLetter>> letters({int page = 0, int size = 30})

সব চিঠির পজেনিটেডে তালিকা আনে।

Future<PagedResponse<HrLetter>> lettersFor(int employeeId, {int page = 0, int size = 30})

`$_letters/employee/$employeeId` থেকে একজন নির্দিষ্ট কর্মচারী সম্পর্কিত সব চিঠি আনে।

Future<HrLetter> createLetter(HrLetterRequest request)

নতুন চিঠি তৈরি করে।

Future<HrLetter> issueLetter(int id)

PATCH `$_letters/$id/issue` কল করে চিঠিকে ইস্যু করা হিসেবে চিহ্নিত করে — এটি একমুখী, ইতিমধ্যে দেওয়া চিঠি ফিরে "un-issue" করার উপায় নেই।

Future<void> deleteLetter(int id)

চিঠি ডিলিট করে।

Future<String> draftLetter({required String letterType, int? employeeId, int? jobApplicationId})

`$_letters/draft`-এ কল করে একটি ব্যক্তি ও চিঠির ধরনের জন্য বডি খসড়া করায়; কিছুই সবে হয় না।

উপ-লভেলে প্রোভাইডার: hrPolicyRepositoryProvider

Provider<HrPolicyRepository> — apiClientProvider থেকে রপ্যোজিটরি ইনস্ট্যান্স তৈরি করে।

ক্লাস: HolidayYearController

Notifier<int> — কোন বছরে ছুটি দেখানো হচ্ছে তা ধরে রাখে।

int build()

প্রাথমিক অবস্থা হিসেবে চলতি বছর (`DateTime.now().year`) রিটার্ন করে।

void set(int year)

নতুন বছর একই হলে কিছু করে না, ভিন্ন হলে state আপডেট করে।

উপ-লভেলে প্রোভাইডার: holidayYearProvider

NotifierProvider<HolidayYearController, int> |

ক্লাস: HolidaysController

AsyncNotifier<List<Holiday>> — currentUserProvider ও holidayYearProvider watch করে সংশ্লিষ্ট বছরে ছুটির তালিকা লোড রাখে।

Future<List<Holiday>> build()

currentUserProvider ও holidayYearProvider watch করে _load() কল করে (যেকোনো একটি বিদ্যমান পুনরায় বন্ডি হয়)।

Future<List<Holiday>> _load()

holidayYearProvider-এর বর্তমান মান দিয়ে holidaysForYear কল করে।

Future<void> refresh()

AsyncValue.guard(_load)-এর মাধ্যমে state পুনরায় লোড করে, এরর হলে সেটোও state-এ ধরে রাখে।

void apply(Holiday updated)

বর্তমান তালিকায় ম্যাচিং id-এর এন্ট্রি নতুন ভ্যালু দিয়ে প্রতিস্থাপন করে (তালিকা না থাকলে কিছু করে না)।

void remove(int id)

তালিকা থেকে নির্দিষ্ট id-এর এন্ট্রি বাদ দিয়ে।

টপ-লেভেলে প্রোভাইডার: holidaysProvider

AsyncNotifierProvider<HolidaysController, List<Holiday>> |

ক্লাস: LeavePoliciesController

AsyncNotifier<List<LeavePolicy>> — লভি পলিসি তালিকা লোড ও লোকাল আপডেট সামলায়। গঠন HolidaysController-এর মতোই।

Future<List<LeavePolicy>> build()

currentUserProvider watch করে _load() কল করে।

Future<List<LeavePolicy>> _load()

policies() কল করে page.content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে state রিলাড করে।

void apply(LeavePolicy updated)

ম্যাচিং id-এর পলিসি প্রতিস্থাপন করে।

void remove(int id)

নির্দিষ্ট id-এর পলিসি তালিকা থেকে বাদ দিয়ে।

টপ-লেভেলে প্রোভাইডার: leavePoliciesProvider

AsyncNotifierProvider<LeavePoliciesController, List<LeavePolicy>> |

ক্লাস: ShiftsController

AsyncNotifier<List<Shift>> — শফিটরে তালিকা লোড ও লোকাল আপডেট সামলায়। গঠন উপরে দুটির মতো।

Future<List<Shift>> build()

currentUserProvider watch করে _load() কল করে।

Future<List<Shift>> _load()

shifts() কল করে page.content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে state রলিওড করে।

void apply(Shift updated)

ম্যাচিং id-এর শফিট প্রতস্থাপন করে।

void remove(int id)

নর্দিষ্ট id-এর শফিট তালিকা থেকে বাদ দেয়।

উপ-লভেলে প্রোভাইডার: hrShiftsProvider

AsyncNotifierProvider<ShiftsController, List<Shift>> |

ক্লাস: LettersController

AsyncNotifier<List<HrLetter>> — সব চঠির তালিকা লোড ও লোকাল আপডেট সামলায়।

Future<List<HrLetter>> build()

currentUserProvider watch করে _load() কল করে।

Future<List<HrLetter>> _load()

letters() কল করে page.content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে state রলিওড করে।

void apply(HrLetter updated)

ম্যাচিং id-এর চঠি প্রতস্থাপন করে।

void remove(int id)

নর্দিষ্ট id-এর চঠি তালিকা থেকে বাদ দেয়।

উপ-লভেলে প্রোভাইডার: lettersProvider

AsyncNotifierProvider<LettersController, List<HrLetter>> |

উপ-লভেলে প্রোভাইডার: lettersForEmployeeProvider

FutureProvider.autoDispose.family<List<HrLetter>, int> — একজন নর্দিষ্ট কর্মচারীর ফাইলে থাকা সব চঠি আনে (lettersFor-এর page.content)।

উপ-লভেলে প্রোভাইডার: activeLeavePoliciesProvider

FutureProvider.autoDispose<List<LeavePolicy>> — বর্তমানে কার্যকর লভি পলিসিগুলো (activePolicies()) আনে, যা ছুটির আবেদনকারী কী পাওয়ার যোগ্য তা নর্ধারণ করতে ব্যবহৃত হয়।

ফাইল: lib/features/hrpolicy/hrpolicy_screen.dart

সংক্ষিপ্ত ববিরণ: HR-এর স্থায়ী নথিমের মূল স্ক্রিনি — Holidays, Leave, Shifts, Letters — চারটি ট্যাবে (পারমশিন অনুযায়ী দেখানো/লুকানো) ভাগ করা। প্রতটি ট্যাবে একটি `ConfigList` দিয়ে তালিকা দেখায় এবং তৈরি/সম্পাদনা/মুছে ফেলার অ্যাকশন সামলায়।

ক্লাস/উইজটে: HrPolicyScreen

`ConsumerWidget`। ব্যবহারকারীর পারমশিন অনুযায়ী গতিশীলভাবে ট্যাব তৈরি করে (কোনো পারমশিন না থাকলে "Not available to you" এম্পটি-স্টেটে দেখায়), প্রতটি ট্যাবের জন্য ঐচ্ছকি ফ্লোটিং "New" বাটন সহ।

`Widget build(BuildContext context, WidgetRef ref)`

`permissionControllerProvider` watch করে; পারমশিন এখনো লোড না হলে লোডার দেখায়। এরপর `holidayView/policyView/shiftView/letterView` পারমশিন অনুযায়ী চারটি (record টাইপ {label, view, create}) ট্যাব বন্ডি করে — প্রতটির `create` কলব্যাক শুধু সংশ্লিষ্ট create-পারমশিন থাকলে সেটে হয়। কোনো ট্যাব না থাকলে `EmptyState` দেখায়। `initialTabLabel` দিয়ে নির্দিষ্ট ট্যাবে শুরু করা যায় (`indexWhere`-এর ফলাফল `clamp` করে বন্ডি রেঞ্জের রাখা হয়)। `DefaultTabController` ও `AnimatedBuilder` দিয়ে বর্তমান ট্যাব বদলালে ফ্লোটিং অ্যাকশন বাটনও বদলে যায়।

ক্লাস/উইজটে: _HolidaysTab

`ConsumerStatefulWidget` — ছুটির তালিকা ট্যাব।

(state) `Future<void> _delete(Holiday holiday)`

`confirmAction` দিয়ে নিশ্চিতকরণ ডায়ালগ দেখায়; সম্মত হলে `deleteHoliday` কল করে এবং `holidaysProvider.notifier.remove` দিয়ে লোকাল তালিকা থেকে বাদ দেয়, সাফল্য/ব্যর্থতায় সন্ধ্যাকবার দেখায়, চলাকালীন `_busyId` সেটে করে রো-ট বিজি দেখায়।

`Widget build(BuildContext context)`

পারমশিন (`holidayUpdate`, `holidayDelete`) চকে করে `ConfigList<Holiday>` রন্ডার করে — বছর বাছাইয়ের জন্য একটি `FilterBar` হডোর (গত/চলতি/পরবর্তী বছর) সহ, প্রতটি সারিতে নাম, তারিখ, ধরন, ববিরণ, এবং ঐচ্ছকি (`isOptional`) ছুটিকে "Optional" লবেলে ভিন্নভাবে দেখায়।

ক্লাস/উইজটে: _PoliciesTab

`ConsumerStatefulWidget` — লভি পলিসি তালিকা ট্যাব।

(state) `Future<void> _delete(LeavePolicy policy)`

নিশ্চিতকরণের পর `deletePolicy` কল করে ও `leavePoliciesProvider.notifier.remove` দিয়ে তালিকা থেকে বাদ দেয়; একই `busy`/সন্ধ্যাকবার প্যাটার্ন।

`Widget build(BuildContext context)`

পারমশিন চকে করে `ConfigList<LeavePolicy>` রন্ডার করে — হডোরে "Draft the document" বাটন (`showPolicyDocumentSheet` খোলো), প্রতটি সারিতে leave type, terms, এবং employment type (বা "everybody") ট্রাইলিং লবেলে হসিবে।

ক্লাস/উইজটে: _ShiftsTab

`ConsumerStatefulWidget` — শফিট তালিকা ট্যাব।

(state) Future<void> _toggle(Shift shift)

নশ্চতিকরণ ছাড়াই সরাসরি toggleShift কল করে সক্রিয়/নিষ্ক্রিয় অবস্থা বদলায় এবং hrShiftsProvider.notifier.apply দিয়ে ফলাফল প্রত্যাফলিত করে।

(state) Future<void> _delete(Shift shift)

নশ্চতিকরণের পর deleteShift কল করে ও তালিকা থেকে বাদ দেয়; মন্তব্যে বলা আছে শফিট মোছার পরও যাদরে আগে অ্যাসাইন করা হয়েছিল তারা অ্যাসাইনমেন্ট ধরে রাখে — নতুন কাউকে দেওয়া বন্ধ করতে চাইলে বরং "retire" (toggle) করা উচিত।

Widget build(BuildContext context)

পারমশিন চকে করে ConfigList<Shift> রেন্ডার করে — প্রতটি সারিতে নাম, window, ঘণ্টা, flexible/night ট্যাগ, গ্রসে পরিষিড; onToggle ও onEdit উভয় shiftUpdate পারমশিনে সেটে হয়।

ক্লাস/উইজটে: _LettersTab

ConsumerStatefulWidget — চঠিরি তালিকা ট্যাব।

(state) Future<void> _issue(HrLetter letter)

নশ্চতিকরণের পর issueLetter কল করে (মন্তব্যে সতর্ক করা হয়েছে এটি একমুখী — পরে "un-issue" করা যায় না), সফল হলে lettersProvider.notifier.apply দিয়ে আপডেট করে।

(state) Future<void> _delete(HrLetter letter)

নশ্চতিকরণ বার্তা letter.issued অনুযায়ী পরিবর্তিত হয় (ইতিমধ্যে ইস্যু করা চঠি মোছা মানে সেই রেকর্ড হারানো); সম্মততি deleteLetter কল করে ও তালিকা থেকে বাদ দেয়।

Widget build(BuildContext context)

পারমশিন (letterUpdate-কই "canIssue" হিসেবে ব্যবহার করে, letterDelete) চকে করে

ConfigList<HrLetter> রেন্ডার করে — প্রতটি সারিরি টাইটলে letter.about, active ফ্ল্যাগ issued-এর উপর ভিত্তি করে (ড্রাফট হলে "Draft" লবেলে), onEdit সবসময় LetterScreen.open খোলে (এডটি নয়, শুধু পড়া), এবং শর্তসাপেক্ষে "Issue it"/"Delete" অ্যাকশন যোগ করে।

ক্লাস/উইজটে: LetterScreen

StatelessWidget। একটি চঠি সম্পূর্ণ পড়ার স্ক্রিনি — সম্পাদনার কোনো সুযোগ নেই, কারণ ব্যাকএন্ডে চঠিরি জন্ম কোনো আপডেটে এন্ডপয়ন্টে নেই (ভুল হলে মুছে আবার লখো হয়)।

static void open(BuildContext context, {required HrLetter letter})

MaterialPageRoute দিয়ে LetterScreen push করার শর্টকাট।

Widget build(BuildContext context)

চঠিরি ধরন AppBar-এর টাইটলে দেখায়; একটি কার্ডে প্রাপক/সাবজেক্ট নাম, স্ট্যাটাস চপি (Issued/Draft), রফোরন্স নম্বর, তারখি, লখেকরে নাম; আরকেটি কার্ডে SelectableText দিয়ে পূর্ণ কন্টেন্ট (না থাকলে "This letter has no content."); সবশেষে থাকলে সাক্ষরকারীর নাম।

done

ফাইল: lib/features/hrpolicy/hrpolicy_sheets.dart

সংক্ষিপ্ত ববিরণ: এই ফাইলে HR পলিসিসংক্রান্ত চারটি বটম-শিট (Holiday, Leave Policy, Shift, Letter) এবং একটি পলিসি-ডকুমেন্ট ড্রাফটিং শিট সংজ্ঞায়িত আছে। প্রতিটি শিট নতুন রকের্ড তৈরি অথবা বিদ্যমান রকের্ড এডিট করার ফর্ম দেখায়, এবং কিছু ক্ষেত্রে AI দিয়ে অটো-ড্রাফট করার সুবিধাও দেয়। সব শিট `FormSheetFrame` উইজটের মাধ্যমে একই রকম UI কাঠামো ব্যবহার করে।

টপ-লেভেলে ফাংশন

`Future<void> showHolidaySheet(BuildContext context, {Holiday? existing})`

একটি মডাল বটম-শিট খোলে যেখানে নতুন ছুটির দিন (holiday) যোগ করা যায় অথবা `existing` প্যারামিটার দিলে বিদ্যমান ছুটি এডিট করা যায়। `isScrollControlled: true` দিয়ে শিটটি পূর্ণ উচ্চতা পর্যন্ত বাড়তে পারে। এটি `_HolidaySheet` উইজটে বন্ডি করে রিটার্ন করে।

`Future<void> showLeavePolicySheet(BuildContext context, {LeavePolicy? existing})`

লভি পলিসি তৈরি বা এডিট করার জন্য বটম-শিট খোলে। `_LeavePolicySheet` উইজটে রেন্ডার করে, যা এনটাইটলেমেন্ট, ক্যারি-ফরওয়ার্ড ইত্যাদি স্টেইস ধারণ করে।

`Future<void> showShiftSheet(BuildContext context, {Shift? existing})`

শফিট তৈরি বা এডিট করার বটম-শিট খোলে। `_ShiftSheet` উইজটে বন্ডি করে।

`Future<void> showLetterSheet(BuildContext context)`

নতুন HR লেটার (offer letter, warning ইত্যাদি) তৈরির বটম-শিট খোলে। সবসময় নতুন — কোনো `existing` প্যারামিটার নেই, কারণ চিঠি সবসময় নতুন করে ড্রাফট হয়। `_LetterSheet` উইজটে বন্ডি করে।

`Future<void> showPolicyDocumentSheet(BuildContext context)`

ইতোমধ্যে কনফিগার করা এনটাইটলেমেন্টগুলো থেকে AI দিয়ে লভি পলিসি ডকুমেন্ট (প্রোজ ফর্ম্যাটে) ড্রাফট করার শিট খোলে। এটি কিছুই সবে করে না — শুধু দেখায়, যাত কপি করা যায়। `_PolicyDocumentSheet` উইজটে বন্ডি করে।

ক্লাস/উইজটে: `_HolidaySheet`

`ConsumerStatefulWidget`, যা একটি ছুটির দিন তৈরি/এডিট করার ফর্ম উপস্থাপন করে। কনস্ট্রাক্টরে এইচসি `existing` (Holiday?) নেয়; না থাকলে নতুন এন্ট্রি মোডে কাজ করে।

`_HolidaySheetState createState()`

স্টেটে অবজেক্ট তৈরি করে — Flutter এর স্ট্যান্ডার্ড override, নির্দিষ্ট কোনো লজিক নেই।

ক্লাস: `_HolidaySheetState (State of _HolidaySheet)`

ফর্মের সব কন্ট্রোলার, তারিখ/ধরন স্টেটে, সাবমিশন ও ড্রাফটিং এর স্ট্যাটাস পরিচালনা করে।

`bool get _isEdit`

`widget.existing` নাল না হলে `true` রিটার্ন করে — অর্থাৎ এডিট মোডে আছে কিনা তা বলে।

`void dispose()`

`_name` ও `_description` টেক্সট কন্ট্রোলার `dispose` করে, তারপর সুপারক্লাসের `dispose` কল করে — মমোরালিকি এড়াতে।

Future<void> _draft()

একটি ডায়ালগ দেখিয়ে ব্যবহারকারীর কাছ থেকে ছুটির বিবরণী বাক্য (যেমন "Eid al-Fitr next year") নিয়ে, তারপর `hrPolicyRepositoryProvider.draftHoliday()` কল করে AI থেকে নাম, বিবরণ, ধরন ও তারিখ অনুমান করায়। ফলাফল দিয়ে ফর্মের ফিল্ডগুলো পূরণ করে দেয়, কনস্ট্রাক্টর সবে করে না — ব্যবহারকারী চাইলে পরিবর্তন করে সাবমিট করবে। ব্যর্থ হলে `_error` স্টেট করে; `_drafting` লোডিং স্টেটে নিয়ন্ত্রণ করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে, তারিখ না থাকলে এরর দেখায়, তারপর `HolidayRequest` তৈরি করে `hrPolicyRepositoryProvider` এর মাধ্যমে `create` বা `update` কল করে। এডটি মোডে `holidaysProvider.notifier.apply()` দিয়ে লিস্ট আপডেট করে; নতুন তৈরি ক্ষেত্রে ছুটির বছর অনুযায়ী `holidayYearProvider` স্টেট করে ও লিস্ট রিফ্রেশ করে। সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়, ব্যর্থ হলে এরর মেসেজ স্টেট করে।

Widget build(BuildContext context)

`FormSheetFrame` এর ভিতরে নাম, তারিখ, ধরন (dropdown) ও বিবরণ ফিল্ড রেন্ডার করে। AI চ্যাট পারমিশন থাকলে ও এডটি মোড না হলে নাম ফিল্ডের পাশে ড্রাফট বাটন দেখায়।

ক্লাস/উইজটে: `_LeavePolicySheet`

লভি পলসি তৈরি/এডটি করার `ConsumerStatefulWidget`। কনস্ট্রাক্টরে ঐচ্ছিক `existing` (`LeavePolicy?`) নিয়ে।

`_LeavePolicySheetState createState()`

স্টেটে অবজেক্ট তৈরি স্ট্যান্ডার্ড override।

ক্লাস: `_LeavePolicySheetState`

এনটাইটলেমেন্ট, ক্যারি-ফরওয়ার্ড, ছুটির ধরন ইত্যাদি ফিল্ড ও তাদের ভ্যালিডেশন পরিচালনা করে।

bool get `_isEdit`

`widget.existing` নাল না হলে `true` — এডটি মোড কনি নির্ধারণ করে।

void `dispose()`

`_entitlement`, `_carryForward`, `_consecutive`, `_fromMonths` কন্ট্রোলারগুলো `dispose` করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে `LeavePolicyRequest` তৈরি করে — লক্ষণীয় যে `maxCarryForward` শুধু `_canCarryForward` `true` থাকলেই পাঠানো হয়, নাহলে `0`। এডটি মোডে `updatePolicy` কল করে `leavePoliciesProvider.notifier.apply()` করে; নতুন তৈরি ক্ষেত্রে `createPolicy` কল করে লিস্ট রিফ্রেশ করে। সফল/ব্যর্থ ক্ষেত্রে যথাক্রমে শিট বন্ধ/এরর দেখানো হয়।

String? `_wholeDays`(String? value, {bool required = false})

একটি ইনপুট স্ট্রিং পূরণসংখ্যা দিনের সংখ্যা কনি যাচাই করে — খালি হলে `required` অনুযায়ী এরর দেয় বা না দেয়, পার্স ব্যর্থ হলে "Whole days only" এবং ঋণাত্মক হলে "Zero or more" এরর রিটার্ন করে।

Widget build(BuildContext context)

লভি টাইপ ও এমপ্লয়মেন্ট টাইপের ড্রপডাউন, বার্ষিক দিন সংখ্যা, সর্বোচ্চ পরপর দিন, প্রযোজ্য হওয়ার মাস, পাইড/অ্যাপ্রুভাল/ক্যারি-ফরওয়ার্ড সুইচ এবং শর্তসাপেক্ষে ক্যারি-ফরওয়ার্ড সীমার ইনপুট রেন্ডার করে।

ক্লাস/উইজটে: _ShiftSheet

শফিট তৈরি/এডিট করার `ConsumerStatefulWidget`। কনস্ট্রাক্টরে ঐচ্ছিক `existing` (Shift?) নিয়ে।

`_ShiftSheetState createState()`

স্টেটে অবজেক্ট তৈরি স্ট্যান্ডার্ড `override`।

ক্লাস: _ShiftSheetState

শফিটের নাম, ধরন, শুরু/শেষ সময়, গ্রসে পরিষিড, সাপ্তাহিক ছুটির দিন, ফ্লেক্সিবিল/নাইট শফিট ফ্ল্যাগ ইত্যাদি পরিচালনা করে।

`bool get _isEdit`

`widget.existing` নাল না হলে `true`।

`static TimeOfDay _parse(String? value, int fallbackHour)`

ব্র্যাকএন্ড থেকে আসা `HH:mm:ss` ফরম্যাটে স্ট্রিং পার্স করে `TimeOfDay` বানায়; পার্স করা না গেলে `fallbackHour`-কে ঘণ্টা ধরে মিনিট শূন্য সহ ডিফল্ট রিটার্ন করে — যাত্রে মধ্যরাত্রে বদলে যুক্তিসিঙগত অফসি আওয়ার দেখায়।

`static String _iso(TimeOfDay time)`

`TimeOfDay` কে `HH:mm:00` ফরম্যাটে স্ট্রিং এ রূপান্তর করে, কারণ ব্র্যাকএন্ডের `LocalTime` সেকেন্ডসহ ফরম্যাট চায়।

`void dispose()`

`_name`, `_grace`, `_offDays`, `_description`, `_notes` কন্ট্রোলার `dispose` করে।

`Future<void> _pickTime({required bool start})`

`showTimePicker` দিয়ে শুরু বা শেষ সময় বাছাই করার ডায়ালগ দেখায়; `start` ফ্ল্যাগ অনুযায়ী `_start` বা `_end` স্টেটে আপডেট করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে `ShiftRequest` তৈরি করে (সময়গুলো `_iso()` দিয়ে ফরম্যাট করে), এডিট মোডে `updateShift` কল করে `hrShiftsProvider.notifier.apply()`, অথবা `createShift` কল করে লিস্ট রিফ্রেশ করে। ফলাফল অনুযায়ী শিট বন্ধ বা এরর দেখায়।

`Widget build(BuildContext context)`

শুরু ও শেষ মিনিট থেকে মোট কাজের সময় হিসাব করে — শেষ সময় শুরুর আগে হলে সটেকে "overnight" (মধ্যরাত পার হওয়া) শফিট ধরে ২৪ ঘণ্টা যোগ করে সংশোধন করে। নাম, ধরন, শুরু/শেষ সময় বাটন, হিসাবকৃত ঘণ্টা টেক্সট, গ্রসে পরিষিড, সাপ্তাহিক অফ ডে, ফ্লেক্সিবিল/নাইট শফিট সুইচ এবং বিবরণ/নোট ফিল্ড রেন্ডার করে।

ক্লাস/উইজটে: _LetterSheet

নতুন HR চিঠি তৈরির `ConsumerStatefulWidget`, কোনো কনস্ট্রাক্টর প্যারামিটার নেই কারণ চিঠি সবসময় নতুন করে তৈরি হয়।

`_LetterSheetState createState()`

স্টেটে অবজেক্ট তৈরি স্ট্যান্ডার্ড override।

ক্লাস: `_LetterSheetState`

চিঠি ধরন, ইস্যু-তারিখ, সংশ্লিষ্ট কর্মচারী, রফোরেন্স নম্বর, স্বাক্ষরকারী ও চিঠি মূল বিষয়বস্তু পরচালনা করে।

`void dispose()`

`_content`, `_reference`, `_signedBy` কন্ট্রোলার dispose করে।

`Future<void> _pickEmployee()`

`EmployeePicker.show()` দিয়ে একজন কর্মচারী বাছাইয়ের UI দেখায়; নির্বাচন হলে `_employeeId` ও `_employeeName` সেট করে এবং আগের এরর ক্লিয়ার করে।

`Future<void> _draft()`

`hrPolicyRepositoryProvider.draftLetter()` কল করে বাছাইকৃত চিঠি ধরন ও কর্মচারীর ভিত্তিতে AI দিয়ে চিঠি মূল বিষয়বস্তু লিখিয়ে নেয় এবং `_content` কন্ট্রোলারে বসায়। ব্যর্থ হলে এরর মেসেজ সেট করে; `_drafting` দিয়ে লোডিং স্টেটে দেখায়।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, ইস্যু-তারিখ না থাকলে এরর দেখায়। `hrPolicyRepositoryProvider.createLetter()` কল করে `HrLetterRequest` পাঠায় (ধরন, তারিখ, বিষয়বস্তু, কর্মচারী আইডি, রফোরেন্স, স্বাক্ষরকারী সহ), তারপর `lettersProvider.notifier.refresh()` করে। সফল হলে শিট বন্ধ করে "Drafted. Issue it when it is ready." বার্তা দেখায়।

`Widget build(BuildContext context)`

চিঠি ধরন (dropdown), সম্পর্কিত ব্যক্তি (InkWell দিয়ে `EmployeePicker` খোল), ইস্যু-তারিখ, রফোরেন্স ও স্বাক্ষরকারী (Row-এ পাশাপাশি), এবং মূল বিষয়বস্তুর মাল্টিলাইন টেক্সট ফিল্ড রেন্ডার করে — AI অনুমতি থাকলে বিষয়বস্তু ফিল্ডে ড্রাফট বাটন দেখায়।

ক্লাস/উইজটে: `_InstructionDialog`

একলাইনের নির্দেশনা (instruction) নেওয়ার জন্য ছোট `AlertDialog`-ভিত্তিক `StatefulWidget`, যা `_HolidaySheet._draft()` থেকে ব্যবহৃত হয়। `title` ও `hint` কনস্ট্রাক্টর প্যারামিটার নেয়।

`_InstructionDialogState createState()`

স্টেটে অবজেক্ট তৈরি স্ট্যান্ডার্ড override।

ক্লাস: `_InstructionDialogState`

টেক্সট ইনপুট ও বাটনের এনাবল/ডিজিবেল স্টেটে পরচালনা করে।

`void dispose()`

`_controller` dispose করে।

Widget build(BuildContext context)

AlertDialog রন্ডার করে যাত একটি TextField (হন্ট সহ), "Never mind" (বাতলি) ও "Look it up" (ইনপুট খালি থাকলে ডজিবেল) বাটন থাকে। "Look it up" চাপলে ইনপুট টক্সট নযি ডায়ালগ বন্ধ করে রটার্নন করে।

ক্লাস/উইজটে: _PolicyDocumentSheet

এনটাইটলেমেন্ট থেকে পলসি ডকুমেন্ট AI দযি ড্রাফট করানোর ConsumerStatefulWidget, কনো কনস্ট্রাক্টর প্যারামটার নই।

_PolicyDocumentSheetState createState()

স্টটে অবজেক্ট তরৈরি স্ট্যান্ডার্ড override।

ক্লাস: _PolicyDocumentSheetState

রমিোট-ওয়ার্ক ফ্ল্যাগ, অতিরিক্ত প্রসঙ্গ টক্সট, এবং ড্রাফট করা ডকুমেন্টের ফলাফল ধরে রাখে।

void dispose()

_context কন্ট্রোলার dispose করে।

Future<void> _draft()

hrPolicyRepositoryProvider.draftPolicyDocument() কল করে remoteWorkAllowed ও additionalContext পাঠযি AI-জনেরটেডে ডকুমেন্ট আনে এবং _document স্টটে বসায়। ব্যর্থ হলে এরর মসেজে স্টে করে; _drafting লোডিং স্টটে নযিন্ত্রণ করে।

Widget build(BuildContext context)

FormSheetFrame-এ একটি ব্যাখ্যামূলক টক্সট, "রমিোট ওয়ার্ক অনুমোদতি" সুইচ, অতিরিক্ত প্রসঙ্গে টক্সট ফল্ড এবং (ড্রাফট হয়ে থাকলে) SelectableText দযি ফলাফল ডকুমেন্ট দখোয়। বাটনের লবেলে প্রথমবার "Draft it" এবং পরে "Draft it again" হয়।

ফাইল: lib/features/itam/asset_detail_screen.dart

সংক্ষিপ্ত ববিরণ: একটি নির্দিষ্ট হার্ডওয়্যার অ্যাসটে বসিতারতি পর্দা — কোথায় আছে, কার কাছে আছে, এবং তার উপর করা যায় এমন অ্যাকশন (assign, unassign, maintenance, edit, dispose, delete)। এখানে AssetDetailScreen মূল স্ক্রনি এবং _Header, _Actions, _Specs তনিট সহায়ক প্রাইভটে উইজটে রযছে।

ক্লাস/উইজটে: AssetDetailScreen

নির্দিষ্ট একটি Asset নযি তার বসিতারতি তথ্য ও কার্যক্রমের বাটন দখোনো ConsumerStatefulWidget।

static void open(BuildContext context, {required Asset asset})

MaterialPageRoute ব্যবহার করে AssetDetailScreen-এ নভেগিটে করে — নভেগেশন লজকি এক জায়গায় রাখার জন্য স্ট্যাটকি হলেপার।

`_AssetDetailScreenState createState()`

স্টেটে অবজেক্ট তৈরি স্ট্যান্ডার্ড override।

ক্লাস: `_AssetDetailScreenState`

বর্তমান অ্যাসটে লোকাল কপি (`_asset`) এবং ব্যস্ততার (`_busy`) স্টেটে রাখে; সব অ্যাকশন মথড এখানে সংজ্ঞায়িত।

`Future<void> _run(Future<Asset> Function() action, String success, String failure)`

একটি সাধারণ (generic) হেল্পার যা যেকোনো অ্যাসটে-আপডেটেং action (assign, unassign ইত্যাদি) কে wrap করে: `_busy` true করে, action চালায়, ফলাফলের নতুন `Asset` দিয়ে `assetsProvider.notifier.apply()` ও লোকাল `_asset` স্টেটে আপডেট করে, সফলতার/ব্যর্থতার সন্ধ্যাকবার দেখায়। `ApiException` হলে তার নজিস্ব মসেজে দেখায়, অন্য কোনো এক্সপেশনে দেওয়া `failure` মসেজে দেখায়। শেষে `_busy` false করে।

`Future<void> _assign()`

`EmployeePicker.show()` দিয়ে একজন ব্যক্তি বাছাই করায়, তারপর `_run()` এর মাধ্যমে `itamRepositoryProvider.assignAsset()` কল করে সেই ব্যক্তির কাছে অ্যাসটে বরাদ্দ করে।

`Future<void> _unassign()`

নশ্চতিকরণ ডায়ালগ দেখায় (বর্তমান হোল্ডারের নাম উল্লেখ করে), নশ্চিতি হলে `_run()` এর মাধ্যমে `unassignAsset()` কল করে অ্যাসটেটি স্টকে ফেরত আনে।

`Future<void> _toggleMaintenance()`

বর্তমান `isUnderMaintenance` অবস্থার বিপরীত মান নিয়ে `_run()` এর মাধ্যমে `setMaintenance()` কল করে — মরোমতে পাঠায় বা সার্বভসি ফেরিয়ে আনে, সেই অনুযায়ী বার্তা দেখায়।

`Future<void> _edit()`

`showEditAssetSheet()` খুলে এডটি শিট দেখায়; শিট থেকে আপডেটেড `Asset` ফেরি এলে লোকাল `_asset` স্টেটে আপডেট করে (তালিকা ইতিমধ্যে শিট থেকে আপডেট হয়ে গেছে ধরে নেয়)।

`Future<void> _dispose()`

`askForText()` দিয়ে কারণ জিজ্ঞাসা করে (আবশ্যিক, destructive হিসেবে চহিনতি), তারপর `_run()` এর মাধ্যমে `disposeAsset()` কল করে অ্যাসটেটি রজিস্টারে "disposed" হিসেবে চহিনতি করে — এটি ডিলিটিরে থেকে ভিন্ন, কারণ ডসিপোজড অ্যাসটে অডটিরে জন্য রজিস্টারে থেকে যায়।

`Future<void> _delete()`

নশ্চতিকরণ ডায়ালগ দেখায়, নশ্চিতি হলে `assetsProvider.notifier.delete()` কল করে অ্যাসটে মুছে ফেলে এবং সফল হলে নভেগিটের পপ করে স্ক্রনি থেকে বেরিয়ে যায়। ব্যর্থ হলে এরর মসেজে সন্ধ্যাকবার দেখায়।

`Widget build(BuildContext context)`

পারমশিন (`hardwareUpdate`, `hardwareDelete`) চকে করে অ্যাপবারে পপআপ মেনু (Edit/Write off/Delete) দেখায়। বডতি `_Header`, ওয়ারন্টসিতরকতা ব্যানার (ময়োদোতীর্ণ/শীঘ্রই শেষ হচ্ছে অনুযায়ী ভিন্ন টেক্সট), `_Actions` (আপডেটে পারমশিন থাকলে ও অ্যাসটে actionable হলে), ডসিপোজড হলে তথ্য-ব্যানার, এবং `_Specs` সেকশন সাজায়।

ক্লাস/উইজটে: _Header

অ্যাসটেরে নাম, মকে/মডলে, স্ট্যাটাস চপি, আইডেন্টিফায়ার এবং বর্তমান হোল্ডার দেখানো একটি StatelessWidget। asset কনস্ট্রাক্টর প্যারামটার নিয়ে।

Widget build(BuildContext context)

AppCard এর ভিতরে নাম/মকে-মডলে, StatusChip, (থাকলে) আইডেন্টিফায়ার আইকন-সহ, একটি ডিভাইডার, এবং assign স্ট্যাটাস অনুযায়ী আইকন ও টেক্সট ("With X since date" বা "Not assigned to anyone") সাজায়।

ক্লাস/উইজটে: _Actions

অ্যাসাইন/টকে-ব্যাক/মইনটেন্যান্স বাটনগুলোর সারি (Wrap) দেখানো StatelessWidget। asset, onAssign, onUnassign, onMaintenance কনস্ট্রাক্টর প্যারামটার নিয়ে।

Widget build(BuildContext context)

Wrap উইজটে শর্তসাপেক্ষে বাটন দেখায়: অ্যাসটে available হলে "Assign" (FilledButton), assigned হলে "Take back" (OutlinedButton), এবং সবসময় "Send for repair"/"Back in service" (isUnderMaintenance অনুযায়ী আইকন-লবেলে বদলায়)।

ক্লাস/উইজটে: _Specs

অ্যাসটেরে প্রযুক্তিগত বিবরণ (স্পেসিফিকেশন) সারণী আকারে দেখানো StatelessWidget। asset কনস্ট্রাক্টর প্যারামটার নিয়ে।

Widget build(BuildContext context)

একটা রেকর্ড-টাইপ লিস্ট (label, value, icon) তৈরি করে (ক্যাটাগরি, সিরিয়াল নম্বর, OS, প্রসেসর, RAM, স্টোরেজ, IP, MAC, ক্রয়-তারিখ, ওয়ারেন্ট-শেষ-তারিখ) যখন থেকে খালি মান বাদ দেয়। যদি কোনো সারি না থাকে এবং নোটও খালি থাকে, SizedBox.shrink() রিটার্ন করে। নাহলে SectionHeader সহ AppCard-এ প্রতটি সারি আইকন-লবেলে-ভ্যালু আকারে ডিভাইডার সহ দেখায়, এবং শেষে (থাকলে) নোটে জন্ম আলাদা AppCard দেখায়।

ফাইল: lib/features/itam/asset_form_sheet.dart

সংক্ষিপ্ত বিবরণ: নতুন হার্ডওয়ার অ্যাসটে যোগ করা বা বর্তমান অ্যাসটে এডিট করার জন্য একটি বটম-শিট ফর্ম প্রদান করে। এডিট শিট সারভারের আপডেটেড রেকর্ড ফিরিয়ে দেয় যাতে ডিটাইল স্ক্রিনি নজরে কপি সিঙ্ক করতে পারে।

উপ-লভেলে ফাংশন

Future<void> showNewAssetSheet(BuildContext context)

রজিস্টারে নতুন মেশিন যোগ করার বটম-শিট খোলে (_AssetFormSheet কোনো existing ছাড়া)।

Future<Asset?> showEditAssetSheet(BuildContext context, Asset asset)

বর্ধমান একটি অ্যাসেটে এডিট করার বটম-শিট খোলে। শিট বাতলি হলে null রিটার্ন করে, সবে হলে সারভারের আপডেটেডে Asset রিটার্ন করে।

ক্লাস/উইজটে: _AssetFormSheet

অ্যাসেটে তৈরি/এডিট ফর্মের ConsumerStatefulWidget। ঐচ্ছকি existing (Asset?) কনস্ট্রাক্টর প্যারামিটার নেই।

_AssetFormSheetState createState()

স্টেটে অবজেক্ট তৈরি স্ট্যান্ডার্ড override।

ক্লাস: _AssetFormSheetState

নাম, ক্যাটাগরি, ট্যাগ, সিরিয়াল, ব্র্যান্ড, মডেল, OS, প্রসেসর, RAM, স্টোরেজ, IP, MAC, নোট — এই ১৩টি টেক্সট কন্ট্রোলার এবং ক্রয়/ওয়ারেন্ট তারিখ স্টেটে পরিচালনা করে।

bool get _isEdit

widget.existing নাল না হলে true।

void initState()

সুপার কল করার পর widget.existing থেকে (থাকলে) প্রতটি টেক্সট কন্ট্রোলার ও তারিখ (Fmt.parse দিয়ে) ইনশিয়ালাইজ করে, না থাকলে খালি মান দিয়ে শুরু করে।

void dispose()

সবগুলো (১৩টি) টেক্সট কন্ট্রোলার dispose করে।

AssetRequest _buildRequest()

ফর্মের সব কন্ট্রোলার ও তারিখ স্টেটে থেকে একটি AssetRequest অবজেক্ট তৈরি করে রিটার্ন করে; তারিখগুলো Fmt.isoDate() দিয়ে ফর্ম্যাট করা হয় (নাল হলে নাল থাকে)।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে, _buildRequest() দিয়ে রকিওয়েস্ট বানিয়ে assetsProvider.notifier এর updateItem() (এডিট মোডে, ফলাফল নভিগেটর পপ করে ফেরত পাঠায়) বা create() (নতুন মোডে) কল করে।

ApiException ধরে ডুপ্লিকেট সিরিয়াল/ট্যাগ এরর মেসেজে যমেন আসে তমেনই দেখায়; অন্য ব্যর্থতায় সাধারণ এরর মেসেজে দেখায়।

Widget build(BuildContext context)

একটি Form-এ শিরোনাম, (থাকলে) এরর ব্যানার, নাম/ক্যাটাগরি/অ্যাসেট-ট্যাগ/সিরিয়াল/ব্র্যান্ড/মডেল টেক্সট ফিল্ড, ক্রয়-তারিখ ও ওয়ারেন্ট-তারিখ (DateField, নর্দর্ষিট রঞ্জে সহ), একটি "Specification" সেকশন হেডার, OS/প্রসেসর/RAM/স্টোরেজ/IP/MAC ফিল্ড, নোট ফিল্ড এবং সাবমিট বাটন (LoadingButton) সাজায়।

ফাইল: lib/features/itam/asset_history_screen.dart

সংক্ষিপ্ত ববিরণ: কোন কর্মচারী কোন অ্যাসটে কবে থেকে কবে পর্যন্ত ব্যবহার করেছে তার ইতিহাস তালিকা দেখায়, এবং CSV ফাইল থেকে বাল্ক অ্যাসটে ইমপোর্ট করার একটি বটম-শাট প্রদান করে।

ক্লাস/উইজটে: AssetHistoryScreen

অ্যাসটে হ্যান্ড-ওভার/রটার্ন ইতিহাসের তালিকা দেখানো `ConsumerWidget` — প্রতিটি সারি একটি "spell" (কেউ যতদিন কিছু ধরে রেখেছিল), ইভেন্ট নয়। `returnedAt` নাল মানের এখনও তার কাছেই আছে।

`static void open(BuildContext context)`

`MaterialPageRoute` দিয়ে `AssetHistoryScreen`-এ নভিগেটে করে।

`Widget build(BuildContext context, WidgetRef ref)`

`ConfigList<AssetHistoryEntry>` ব্যবহার করে `assetHistoryProvider` থেকে ডটো লোড করে দেখায়; খালি/এর অবস্থার বার্তা কনফিগার করে। প্রতিটি এন্ট্রির জন্য `AppCard`-এ আইকন (এখনো বাইরে থাকলে বনাম ফরমের এসছে অনুযায়ী ভিন্ন), অ্যাসটের নাম, কর্মচারীর নাম, তারিখ পরিসীমা, (থাকলে) কন্ডিশন ও নোট দেখায়।

টপ-লেভেলে ফাংশন

`Future<void> showAssetImportSheet(BuildContext context)`

CSV থেকে অ্যাসটে ইমপোর্ট করার বটম-শাট (`_AssetImportSheet`) খোলে।

ক্লাস/উইজটে: _AssetImportSheet

CSV ইমপোর্টের UI ধারণকারী `ConsumerStatefulWidget`, কোনো কনস্ট্রাক্টর প্যারামিটার নেই। ব্যাকএন্ডের টেমপ্লেটে ডাউনলোড ফোনে অর্থবহ নয় বলে সেই সুবিধা এখনো নেই — শুধু ফাইলের প্রয়োজনীয়তা বর্ণনা করা হয়।

`_AssetImportSheetState createState()`

স্টেটে অবজেক্ট তৈরি স্ট্যান্ডার্ড override।

ক্লাস: _AssetImportSheetState

ইমপোর্টের ফলাফল (`_result`), এর ও ব্যস্ততার স্টেটে পরিচালনা করে।

`Future<void> _pick()`

`pickAttachment()` দিয়ে ফাইল বাছাই করায়, তারপর `itamRepositoryProvider.importAssets()` কল করে ফাইলটি আপলোড ও প্রসেস করে। ফলাফল `_result`-এ বসায় এবং `assetsProvider.notifier.refresh()` কল করে যাতে নতুন তৈরি অ্যাসটেগুলো তালিকায় দেখা যায়। ব্যর্থতায় এর মैसेজ স্টেট করে।

`Widget build(BuildContext context)`

শরীোনাম ও ব্যাখ্যা টেক্সট, (থাকলে) এর ব্যানার, এবং ফলাফল থাকলে "X of Y imported" সারাংশ ও প্রতিটি ব্যর্থতার সারি জন্য সারি-নম্বর ও কারণ তালিকাবদ্ধ করে (`ListView`, `shrinkWrap`)। নচি "Choose a file"/"Import another" লেবেলের `LoadingButton` থাকে।

ফাইল: lib/features/itam/itam_models.dart

সংক্ষিপ্ত ববিরণ: ITAM (IT Asset Management) মডউিলরে সব ডটো মডলে, পারমশিন কনস্ট্যান্ট এবং রকি়োয়স্ট/রসেপন্স ক্লাস সংজ্ঞায়িত করে — হার্ডওয়্যার অ্যাসটে, সফটওয়্যার লাইসেন্স, অফবোর্ডিং চকেলস্ট এবং অ্যাসটে হস্ট্রি কনো উইজটে নই, শুধু প্লাইন Dart ডটো ক্লাস ও fromJson/toJson কনভার্টার।

টপ-লেভেলে abstract ক্লাস/enum-সদৃশ কনস্ট্যান্ট হোল্ডার

abstract final class ItamPermissions

ITAM মডউিলরে সব পারমশিন কী স্ট্রিং হিসেবে ধারণ করে: hardwareView, assetView (হার্ডওয়্যার তালিকা দেখার জন্য যেকোনো একটি যথেষ্ট), hardwareUpdate (assign/return/maintenance/edit সবকছির জন্য একই পারমশিন), hardwareCreate, hardwareDelete, softwareView, softwareCreate, softwareAssign (সটি বরাদ্দরে জন্য আলাদা), offboardingView, offboardingCreate, offboardingManage (ধাপ টকি করার জন্য, তরৈ/ডলিটিরে থকে আলাদা), assignmentView।

abstract final class AssetStatus

অ্যাসটেরে সম্ভাব্য স্ট্যাটাস স্ট্রিং ধারণ করে: available, assigned, underMaintenance, disposed, এবং all (ফল্টার-অর্ডারে, disposed সবার শেষে কারণ সটে একটি আর্কাইভ)।

ক্লাস: Asset

কোম্পানরি একটি হার্ডওয়্যার অ্যাসটে প্রতিনিধিত্ব করে — নাম, স্ট্যাটাস, ক্যাটাগরি, ট্যাগ, সরিযাল, ব্র্যান্ড/মডলে, অ্যাসাইনমেন্ট তথ্য, ক্রয়/ওয়ারেন্ট তারিখ, প্রযুক্তিগত স্পেসিফিকেশন ও নোট ধারণ করে। সব ফিল্ড id, name, status ছাড়া ঐচ্ছকি।

bool get isAssigned

status == AssetStatus.assigned হলে true।

bool get isAvailable

status == AssetStatus.available হলে true।

bool get isUnderMaintenance

status == AssetStatus.underMaintenance হলে true।

bool get isDisposed

status == AssetStatus.disposed হলে true।

bool get isActionable

!isDisposed — ডসিপোজড অ্যাসটে কোনো অ্যাকশন (assign/return/maintenance) প্রযোজ্য নয় বলে ডিটাইল স্ক্রনি কোনো বাটন দেখায় না।

String? get makeModel

brand ও model কে দিয়ে জোড়া দেয়, যে অংশ খালি নয় শুধু সটোই রাখতে; দুটোই খালি হলে null রটার্ন করে।

String? get identifier

প্রথম assetTag (ট্রমি করা, খালি না হলে) ফরেনত দেয়, না থাকলে serialNumber ফরেনত দেয় — কসেরে গায় লেখো ট্যাগ, যা একজন ধারক পড়তে পারে।

`bool get warrantyExpiringSoon`

`warrantyExpiry` পার্স করে বর্তমান তারখের থেকে ৩০ দিন বা তার কম বাকি থাকলে `true` রিটার্ন করে (ম্যেডোতীরণ হলেও `true` হবে, কারণ পার্থক্য ঋণাত্মক হলেও `<= 30` শর্ত পূরণ হয়)।

`factory Asset.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে `Asset` অবজেক্ট তৈরি করে, প্রতিটি ফিল্ডের জন্য নরিপদ ডফিল্ট (নাল বা ফলব্যাক ভ্যালু) ব্যবহার করে।

ক্লাস: SoftwareLicense

একটি সফটওয়্যার লাইসেন্স ও তার সটি ব্যবহারের অবস্থা প্রতিনিধিত্ব করে।

`bool get hasSeatsFree`

`seatsAvailable > 0` হলে `true`।

`bool get isFull`

`totalSeatsLicensed > 0 && seatsUsed >= totalSeatsLicensed` — সব সটি ব্যবহৃত হলে `true`।

`bool get isOverAllocated`

`totalSeatsLicensed > 0 && seatsUsed > totalSeatsLicensed` — কোনো সটির চেয়ে বেশি ব্যবহৃত হলে `true` (ব্যাকএন্ড এটি অনুমতি দিয়ে)।

`double get seatFraction`

`seatsUsed / totalSeatsLicensed` কে ০-১ এর মধ্যে ক্ল্যাম্প করে রিটার্ন করে (সটি বার প্রগ্রসেরে জন্য); `totalSeatsLicensed <= 0` হলে ০।

`String get seatsLabel`

"X of Y seats" ফরম্যাটে টেক্সট রিটার্ন করে।

`factory SoftwareLicense.fromJson(Map<String, dynamic> json)`

JSON থেকে `SoftwareLicense` অবজেক্ট তৈরি করে, নরিপদ ডফিল্ট মান সহ।

ক্লাস: OffboardingStep

একটি অফবোর্ডিং চকেলস্টিরে একটি ধাপ (যেমন "Hardware collected") প্রতিনিধিত্ব করে — `key`, `label`, `done`, `path` (PATCH endpoint সগেমনেট), ঐচ্ছকি `date`, `notes`, `by` ধারণ করে। এই ক্লাসের কোনো মেথড নেই, শুধু কনস্ট্রাক্টর ও ফিল্ড।

ক্লাস: OffboardingChecklist

একজন কর্মচারীর প্রস্থানের সময় করণীয় পাঁচটি ধাপের সম্পূর্ণ চকেলস্টি প্রতিনিধিত্ব করে (ব্যাকএন্ডে পাঁচটি বুলিয়ান কলাম হিসেবে সংরক্ষিত, এখানে একটি লিস্টে রূপান্তরিত)।

`String get personLabel`

`employeeName` থাকলে ও খালি না হলে সেট ট্রিম করে রিটার্ন করে, নাহলে "Employee #{id}" রিটার্ন করে।

List<OffboardingStep> get steps

পাঁচটি ধাপ (hardware, licenses, access, data, interview) — প্রতিটি ঠিক করা ক্রম, path, done, date, notes, (হার্ডওয়্যারের ক্ষেত্রে) by সহ একটি লিস্ট তৈরি করে রিটার্ন করে।

int get stepsDone

steps এর মধ্যে যেগুলো done == true তার গণনা রিটার্ন করে।

bool get isOverdue

completed হলে false; নাহলে targetCompletionDate পার্স করে বর্তমান সময় তার পরে হলে true রিটার্ন করে (টারগেট তারিখ না থাকলে false)।

factory OffboardingChecklist.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড (পাঁচ ধাপের প্রতিটির সংশ্লিষ্ট ফ্ল্যাগ/তারিখ/নোট সহ) নিরিপদ ডফিল্টসহ পার্স করে OffboardingChecklist তৈরি করে।

ক্লাস: AssetRequest

POST /hr/assets ও PUT /hr/assets/{id} উভয়ের জন্য ব্যবহৃত রকিওয়েস্ট বডি ক্লাস। মন্তব্যে উল্লেখ আছে যে আপডেট আসলে sparse patch (শুধু নাল-না-হওয়া ফিল্ড আপডেট হয়), এবং description কখনো পাঠানো হয় না কারণ ব্যাকএন্ড সার্ভিস description ও notes উভয়কেই একই কলামে লেখে, ফলে দুটো একসাথে পাঠালে race condition হয়।

factory AssetRequest.from(Asset asset)

একটি বিদ্যমান Asset থেকে AssetRequest তৈরি করে — সব মালিতি ফিল্ড কপি করে (description নই কারণ Asset-এও নই, শুধু notes)।

Map<String, dynamic> toJson()

প্রতিটি স্ট্রিং ফিল্ডকে অভ্যন্তরীণ clean() হলেপার (ট্রিম করে, খালি হলে নাল) দিয়ে পরিশ্কার করে, শুধু নাল-না-হওয়া মান JSON ম্যাপে অন্তর্ভুক্ত করে। name সবসময় পাঠানো হয় (ট্রিম করে); description ইচ্ছাকৃতভাবে বাদ দেওয়া হয়।

টপ-লেভেলে কনস্ট্যান্ট

const licenseTypes

সফটওয়্যার লাইসেন্সের ধরন (value, label) রকেডের লিস্ট — SUBSCRIPTION, PERPETUAL, TRIAL, OPEN_SOURCE।

const licenseRenewalTypes

লাইসেন্স নবায়নের ধরন (value, label) লিস্ট — ANNUAL, MONTHLY, BIENNIAL, PERPETUAL।

ক্লাস: SoftwareLicenseRequest

POST /v1/itam/software-এর জন্য রকিওয়েস্ট বডি। মন্তব্যে বলা হয়েছে যে ৯টি ফিল্ড ব্যাকএন্ডে @NotNull/@NotBlank, তাই প্রায় কিছুই ঐচ্ছিক নয়।

Map<String, dynamic> toJson()

প্রতিটি ঐচ্ছকি স্ট্রিং ফল্ড `clean()` হলেপার দিয়ে পরষ্কার করে JSON বানায। উল্লেখযোগ্য: `licenseKey` ট্রমি করা হয় না কারণ এটিভিন্ডেরে থেকে যেমন এসছে তমেনই কপ করা প্রয়োজন। বাধ্যতামূলক ফল্ডগুলো (`licenseKey`, `softwareName`, `publisher`, ইত্যাদি) সরাসরি অন্তর্ভুক্ত হয়, ঐচ্ছকগুলো (`version`, `nextRenewalDate`, `renewalCost`) শর্তসাপেক্ষে।

ক্লাস: OffboardingChecklistRequest

POST /v1/company/offboarding/checklist-এর রকি়োস্টে বডি— শুধু `employeeId` আবশ্যকি, ঐচ্ছকি `notes`।

Map<String, dynamic> toJson()

`employeeId` ও (ট্রমি করার পর খালি না হলে) `notes` ধারণকারী JSON ম্যাপ রটার্ন করে।

ক্লাস: AssetHistoryEntry

একটি অ্যাসটে একজন ব্যক্তরি কাছে যতদনি ছিল তার একটি "spell" (দফা) প্রতিনিধিত্ব করে। `returnedAt` নাল মানে এখনও তার কাছে আছে; `condition` (গলে যখন) ও `conditionOnReturn` (ফরিল যখন) আলাদাভাবে রকের্ড হয়।

bool get stillOut

`returnedAt == null` হলে `true`।

factory AssetHistoryEntry.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফল্ড নরিপদ ডফিল্টসহ পার্স করে AssetHistoryEntry তরৈকিরে।

ক্লাস: AssetImportResult

একটি CSV ইমপোর্টরে সারাংশ ফলাফল — মোট সারি, সফল, ব্যর্থ সংখ্যা এবং প্রতিটি ব্যর্থ সাররি কারণ (`errors`)।

factory AssetImportResult.fromJson(Map<String, dynamic> json)

JSON থেকে সংখ্যাগুলো এবং `errors` লস্ট (প্রতিটি এন্টরি AssetImportRowError.fromJson দিয়ে পার্স) বানিয়ে AssetImportResult রটার্ন করে।

ক্লাস: AssetImportRowError

CSV-এর একটি নিরুদৃষ্ট সারি (হিডোরসহ গণনা) কনে ব্যর্থ হয়েছে তা ধারণ করে — `row` ও `reason`।

factory AssetImportRowError.fromJson(Map<String, dynamic> json)

JSON থেকে `row` (int) ও `reason` (String) পার্স করে অবজেক্ট তরৈকিরে।

ক্লাস: abstract final class LicenceStatus

ব্যাকএন্ডের LicenseStatus enum-এর সমতুল্য স্ট্রিং কনস্ট্যান্ট: active, expiringSoon, expired, suspended, revoked, এবং all লিস্ট। মন্তব্যে ব্যাখ্যা করা আছে যে এটি SoftwareLicense-এর expiringSoon/expired ফ্ল্যাগ থেকে ভিন্ন — সগেলো প্রতাবার তারখি থেকে হিসাব করা হয়, এই স্ট্যাটাস সংরক্ষণ এবং SUSPENDED/REVOKED-ও ধারণ করতে পারে।

ফাইল: lib/features/itam/itam_repository.dart

সংক্ষিপ্ত বিবরণ: ITAM মডেলের সব API কল (হার্ডওয়্যার, সফটওয়্যার, অফবোর্ডিং, অ্যাসেট হিস্ট্রি, CSV ইমপোর্ট) ধারণকারী রপিসিটরিক্লাস, এবং তার সাথে সংশ্লিষ্ট Riverpod প্রোভাইডার/কন্ট্রোলার। মন্তব্যে বলা আছে তিনটি এন্ডপয়েন্ট-ফ্যামলি ভিন্ন ভিন্ন পথ প্যাটার্নে থাকে (/hr/assets, /v1/itam/software, /v1/company/offboarding/checklist) — শুধু একটির পথে "itam" শব্দটি আছে।

ক্লাস: ItamRepository

API ক্লায়েন্ট ব্যবহার করে ITAM-সংক্রান্ত সব HTTP কল সম্পাদন করে। কনস্ট্রাক্টরে ApiClient নেয়।

Future<PagedResponse<Asset>> assets({int page = 0, int size = 20})

GET /hr/assets কল করে পেজিনিটেডে অ্যাসেট তালিকা আনে।

Future<Asset> createAsset(AssetRequest request)

POST /hr/assets কল করে নতুন অ্যাসেটে তৈরি করে ও রেসপন্স Asset হিসেবে পার্স করে।

Future<Asset> updateAsset(int id, AssetRequest request)

PUT /hr/assets/{id} কল করে অ্যাসেটে আপডেট করে — মন্তব্য অনুযায়ী এটি প্রকৃতপক্ষে sparse patch (শুধু নাল-না-হওয়া ফিল্ড লেখা হয়)।

Future<void> deleteAsset(int id)

DELETE /hr/assets/{id} কল করে সফট-ডেলিট করে। _api.deleteText() ব্যবহার করা হয় কারণ এন্ডপয়েন্টটি প্লাইন টেক্সট ("Deleted successfully") রিটার্ন করে, JSON নয়। অ্যাসেটে বর্তমান assigned থাকলে ব্যাকএন্ড এটি প্রত্যাখ্যান করে।

Future<Asset> assignAsset(int id, int employeeId)

PATCH /hr/assets/{id}/assign/{employeeId} কল করে অ্যাসেটে বরাদ্দ করে — employeeId পথ সগেমেন্ট হিসেবে পাঠানো হয়, বডি/কোয়েরিতে নয়।

Future<List<Asset>> assetsForEmployee(int employeeId)

GET /hr/assets/employee/{employeeId} কল করে (একটি বিড় পেজে, size 50) সেই কর্মচারীর কাছে থাকা সব অ্যাসেটে ফিরিয়ে দেয়।

Future<Asset> disposeAsset(int id, {String? reason})

PATCH /hr/assets/{id}/dispose কল করে (ঐচ্ছকি reason কোয়েরি-প্যারামিটার হিসেবে URL-এনকোড করে যোগ করে) অ্যাসেটকে চরিতরে "disposed" চিহ্নিত করে — ডেলিটিবে বপিরীতে এটি অডিটরে জন্ম রেজিস্টারে থেকে যায়।

Future<Asset> unassignAsset(int id)

PATCH /hr/assets/{id}/unassign কল করে অ্যাসটে ফরেন্ত নয়।

Future<Asset> setMaintenance(int id, bool underMaintenance)

PATCH /hr/assets/{id}/maintenance?underMaintenance=... কল করে (কোয়রে-প্যারামটির হসিবে, বডি হসিবে নয়) মরোমতে পাঠায় বা ফরিযি আনে।

Future<PagedResponse<SoftwareLicense>> licenses({String? status, int page = 0, int size = 20})

status নাল হল GET /v1/itam/software, নাহলে GET /v1/itam/software/status/{status} কল করে — স্ট্যাটাস-ফল্টার একটি পৃথক পাথ, কোয়রে-স্ট্রিং নয়।

Future<SoftwareLicense> createLicense(SoftwareLicenseRequest request)

POST /v1/itam/software কল করে নতুন লাইসেন্স নবিন্ধন করে।

Future<void> assignSeat(int licenseld, int employeeeld)

POST /v1/itam/software/{licenseld}/assign-seat?employeeeld=... কল করে একটি সিটি বরাদ্দ করে।

Future<void> releaseSeat(int licenseld, int employeeeld)

POST /v1/itam/software/{licenseld}/release-seat?employeeeld=... কল করে একটি সিটি ছড়ে দেয়।

Future<List<OffboardingChecklist>> checklists()

GET /v1/company/offboarding/checklist কল করে (এক বড় পেজে, size 50) সব অফবোর্ডিং চকেলস্টিট আনে; এন্ডপয়ন্টে Page<...> রটার্ন করে বলে পেজেড ভাবে পড়া হয়, বয়োর লস্টিট হসিবে নয়।

Future<OffboardingChecklist> createChecklist(OffboardingChecklistRequest request)

POST /v1/company/offboarding/checklist কল করে একজন কর্মচারীর জন্য নতুন চকেলস্টিট থোলে (পাঁচটি ধাপ শুরুতে অসম্পূর্ণ)।

Future<OffboardingChecklist> checklistForEmployee(int employeeeld)

GET /v1/company/offboarding/checklist/employee/{employeeeld} কল করে সেই কর্মচারীর (প্রতি কর্মচারী সর্বোচ্চ একটি) চকেলস্টিট আনে।

Future<OffboardingChecklist> checklist(int id)

GET /v1/company/offboarding/checklist/{id} কল করে নরিদষ্টিট আইডরি চকেলস্টিট আনে।

Future<List<OffboardingChecklist>> pendingChecklists()

GET /v1/company/offboarding/checklist/pending কল করে (বয়োর লস্টিট, পেজেড নয়, checklists()-এর থেকে ভিন্ন) এখনো থোলা চকেলস্টিটগুলো আনে।

Future<OffboardingChecklist> deleteChecklist(int id)

DELETE /v1/company/offboarding/checklist/{id} কল করে একটি চকেলস্টিট মুছে ফলে — অস্বাভাবিকভাবে এই ডলিটি এন্ডপয়ন্টে মুছে ফলো রকেডটাই রসেপন্স হসিবে ফরেন্ত দেয়।

Future<OffboardingChecklist> completeStep(int checklistId, String stepPath, {String? notes})

PATCH /v1/company/offboarding/checklist/{checklistId}/{stepPath} কল করে (ঐচ্ছকি notes বডতি) একটি ধাপ সম্পন্ন হসিবে চহ্নিতি করে। এন্ডপয়ন্টে খালি বডি (ResponseEntity<Void>) ফরেন্ত দেয় বলে সাড়া পার্স না করে checklist(checklistId) কল করে পুরো চকেলস্টিট পুনরায় পড়ে। কোনো un-tick (ধাপ বাতলিরে) এন্ডপয়ন্টে নাই — তাই UI-তে দুই-দকি টেগল দেওয়া চলবে না।

`Future<List<AssetHistoryEntry>> assetHistory((int? assetId, int? employeeId))`
assetId থাকলে `/hr/asset-history/asset/{assetId}`, employeeId থাকলে
`/hr/asset-history/employee/{employeeId}`, নাহলে `/hr/asset-history` — এই তিনটির একটি পাথে GET কল
করে (এক বড় পজে, size 50) সংশ্লিষ্ট হ্যান্ড-ওভার/রটার্ন ইতিহাস আনে।

`Future<AssetImportResult> importAssets(String filePath, String fileName)`
POST `/itam/asset-import` কল করে (মাল্টিপার্ট ফাইল আপলোড, `_api.postFile`) একটি CSV ফাইল
আপলোড ও প্রসেস করে, ফলাফল `AssetImportResult` হিসেবে পার্স করে রটার্ন করে।

টপ-লেভেলে প্রোভাইডার

`final itamRepositoryProvider`
`ItamRepository`-এর একটি সিঙ্গেলেটন ইনস্ট্যান্স তৈরি করে, `apiClientProvider` থেকে `ApiClient` নিয়ে।

ক্লাস: `AssetsController` (**extends** `AsyncNotifier<PagedState<Asset>>` **with** `PagedLoader<Asset>`)

হার্ডওয়ার অ্যাসটে তালিকার পজেনিটেডে স্টেটে পরিচালনা করে।

`Future<PagedState<Asset>> build()`
`currentUserProvider` watch করে (লগইন ইউজার বদলালে রিবিল্ড হওয়ার জন্য) এবং `loadFirstPage()` কল
করে প্রথম পজে লোড করে।

`Future<PagedResponse<Asset>> fetchPage(int page)`
`itamRepositoryProvider.assets(page: page)` কল করে — `PagedLoader` মক্সিনিরে জন্য প্রয়োজনীয়
`override`।

`void apply(Asset updated)`
`replaceItem()` দিয়ে তালিকায় থাকা মালিতি আইডির অ্যাসটেটিকে `updated` দিয়ে প্রতস্থাপন করে, পুরো
তালিকা রিলোড না করে — যাতো তালিকায় স্ক্রল-পজিশন হারিয়ে না যায়।

`Future<Asset> create(AssetRequest request)`
`itamRepositoryProvider.createAsset()` কল করে নতুন অ্যাসটে তৈরি করে, তারপর `refresh()` করে তালিকা
রিলোড করে এবং তৈরি হওয়া অ্যাসটে রটার্ন করে।

`Future<Asset> updateItem(int id, AssetRequest request)`
`itamRepositoryProvider.updateAsset()` কল করে, `apply()` দিয়ে তালিকায় আপডেটে প্রতফিলতি করে,
সার্বভারের রটার্ন করা অ্যাসটে ফেরত দেয় — যাতো ডিটাইল স্ক্রিনি (যা প্রোভাইডার watch করে না)
নজিস্ব কপি আপডেটে করতে পারে।

`Future<void> delete(int id)`
`itamRepositoryProvider.deleteAsset()` কল করে, সফল হলেই `removeItem()` দিয়ে তালিকা থেকে সরায়
(ব্যাকএন্ড এখনো-অ্যাসাইনড অ্যাসটে মুছতে প্রত্যাখ্যান করতে পারে বলে সাফল্য নশ্চিতি হওয়ার
পরই সরানো হয়)।

টপ-লেভেলে প্রোভাইডার

final assetsProvider

AssetsController-এর AsyncNotifierProvider।

ক্লাস: LicensesController (**extends** AsyncNotifier<PagedState<SoftwareLicense>> **with** PagedLoader<SoftwareLicense>)

সফটওয়্যার লাইসেন্স রজিস্টারের পজেনিটেডে স্টেটে পরিচালনা করে।

Future<PagedState<SoftwareLicense>> build()

currentUserProvider ও licenceStatusFilterProvider উভয়ই watch করে (ফিল্টার বদলালে রিলোড হওয়ার জন্য), loadFirstPage() কল করে।

Future<PagedResponse<SoftwareLicense>> fetchPage(int page)

itemRepositoryProvider.licenses() কল করে বর্তমান ফিল্টার-স্ট্যাটাস ও পজে দয়ি।

Future<SoftwareLicense> create(SoftwareLicenseRequest request)

itemRepositoryProvider.createLicense() কল করে, refresh() করে তালিকা রিলোড করে, তরৈহওয়া লাইসেন্স রটরিন করে।

void apply(SoftwareLicense updated)

replaceItem() দয়িে মলিতি আইডরি লাইসেন্স প্রতস্থাপন করে — এডটি করলে তালকায় অবস্থান বদলায় না।

ক্লাস: LicenceStatusFilterController (**extends** Notifier<String?>)

লাইসেন্স রজিস্টার কোন স্ট্যাটাস দেখাচ্ছে তা নয়িন্ত্রণ করে (নাল মানে সবগুলো)।

String? build()

প্রাথমকি স্টেটে null রটরিন করে।

void set(String? status)

নতুন status আগরেটির থকে ভনিন হলই স্টেটে আপডটে করে (অপ্রয়োজনীয় রবিল্ড এডাত)।

টপ-লেভেলে প্রোভাইডার

final licenceStatusFilterProvider

LicenceStatusFilterController-এর NotifierProvider।

final licensesProvider

LicensesController-এর AsyncNotifierProvider।

ক্লাস: OffboardingController (**extends** AsyncNotifier<List<OffboardingChecklist>>)

অফবোর্ডিং চকেলস্টিরে তালিকা (বয়ের লস্টি, পজেং ছাড়া) পরচালনা করে।

Future<List<OffboardingChecklist>> build()

currentUserProvider watch করে itamRepositoryProvider.checklists() কল করে প্রাথমিক তালিকা আনে।

Future<void> refresh()

AsyncValue.guard() দিয়ে নিরাপদে checklists() পুনরায় কল করে স্টেটে আপডেট করে।

Future<OffboardingChecklist> create(OffboardingChecklistRequest request)

itamRepositoryProvider.createChecklist() কল করে, refresh() করে, তরৈ হওয়া চকেলস্টি রটাইন করে।

void apply(OffboardingChecklist updated)

বর্তমান তালিকার (state.value নাল না হলে) মালিতি আইডির চকেলস্টিকে updated দিয়ে প্রতস্থাপন করে একটি নতুন লস্টি বানিয়ে স্টেটে বসায় — কোনো ধাপ সম্পন্ন হওয়ার পর সারভারের নতুন ভার্সন প্রতফিলতি করতে ব্যবহৃত হয়।

উপ-লভেলে প্রোভাইডার

final offboardingProvider

OffboardingController-এর AsyncNotifierProvider।

final pendingOffboardingProvider

FutureProvider.autoDispose — এখনো থালা (অসম্পূর্ণ) চকেলস্টিগুলো
itamRepositoryProvider.pendingChecklists() থেকে আনে।

final offboardingForEmployeeProvider

FutureProvider.autoDispose.family<OffboardingChecklist, int> — একজন নির্দিষ্ট কর্মচারীর (employeeId প্রারামটির) চকেলস্টি checklistForEmployee() থেকে আনে।

final assetHistoryProvider

FutureProvider.autoDispose — কোম্পানির সব অ্যাসেটে হিস্ট্রি এন্ট্রি
itamRepositoryProvider.assetHistory() থেকে আনে।

final assetsForEmployeeProvider

FutureProvider.autoDispose.family<List<Asset>, int> — একজন নির্দিষ্ট কর্মচারী (employeeId) বর্তমানে কী কী ধরে আছে তা assetsForEmployee() থেকে আনে।

ফাইল: lib/features/itam/itam_screen.dart

সংক্ষিপ্ত ববিরণ: IT অ্যাসেটে মডউলের প্রধান স্ক্রিনি — হার্ডওয়ার, সফটওয়ার লাইসেন্স এবং অফবোর্ডিং, এই তিনটি ট্যাবে বিভক্ত। প্রতিটি ট্যাব নিজস্ব পারমশিনের উপর নির্ভরশীল এবং পারমশিন না থাকলে ট্যাবটি একবারেই দেখা যায় না।

ক্লাস/উইজটে: ItamScreen

`ConsumerWidget`, IT অ্যাসটে মডউলরে রুট স্ক্রিনি। ব্যবহারকারীর পারমিশন অনুযায়ী Hardware/Software/Offboarding — কোন কোন ট্যাব দেখাবে তা ঠিকি করে, প্রতিটি ট্যাবের নিজস্ব "নতুন তৈরিকরো" অ্যাকশন (FAB) থাকে যা সক্রিয় ট্যাবের সাথে বদলায়, এবং কোনো পারমিশন না থাকলে "Not available to you" খালি স্টেটে দেখায়।

`ItamScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর। ঐচ্ছকি `initialTabLabel` প্যারামিটার দিয়ে নির্দিষ্ট করা যায় কোন ট্যাবে স্ক্রিনিটি শুরুতে খুলবে (যেমন অন্য কোনো স্ক্রিনি থেকে "Software" ট্যাবে সরাসরি নিভেগিটে করা)।

`Widget build(BuildContext context, WidgetRef ref)`

পারমিশন কন্ট্রোলার লোড না হওয়া পর্যন্ত `Loader` দেখায়। এরপর `hardwareView/assetView, softwareView, offboardingView` পারমিশন চকে করে কোন ট্যাব দেখাবে তা নির্ণয় করে; কোনোটিই না থাকলে খালি-স্টেটে দেখায়। প্রতিটি ট্যাবের জন্য পৃথক তৈরিকরার অ্যাকশন ম্যাপ (`createOn`) বানায়, যা লবেলে দিয়ে কী করা হবে (নতুন অ্যাসটে/লাইসেন্স/অফবোর্ডিং শুরু) নির্ধারণ করে।

`DefaultTabController` ও `AnimatedBuilder` দিয়ে সক্রিয় ট্যাব অনুসারে `AppBar`, `FloatingActionButton` এবং `TabBarView` রেন্ডার করে। হার্ডওয়্যার ট্যাবে অতিরিক্ত পপআপ মেনু থাকে যেখানে ইতিহাস দেখা এবং CSV ইমপোর্টের অপশন থাকে।

ক্লাস/উইজটে: _HardwareTab

`ConsumerWidget`, হার্ডওয়্যার তালিকা দেখানোর জন্য `PagedList<Asset>` ব্যবহার করে।

`Widget build(BuildContext context, WidgetRef ref)`

`assetsProvider` থেকে পজেড অ্যাসটে লিস্ট রফ্রেশ/লোডমোর সহ দেখায়; প্রতিটি আইটেমে `_AssetRow` দিয়ে রেন্ডার হয়। কোনো হার্ডওয়্যার না থাকলে "No hardware yet" খালি স্টেটে দেখায়।

ক্লাস/উইজটে: _AssetRow

`StatelessWidget`, একটি হার্ডওয়্যার অ্যাসটেরে কার্ড-আকৃতির সারি — নাম, মডেল, স্ট্যাটাস চপি, বর্তমান হোল্ডারের নাম এবং শনাক্তকারী নম্বর দেখায়।

`Widget build(BuildContext context)`

অ্যাসটেরে নাম ও মডেল দেখায়, ডানপাশে `StatusChip` দেখায়। নচিরে সারিতে অ্যাসাইনড/আনঅ্যাসাইনড অবস্থা অনুযায়ী আইকন ও হোল্ডারের নাম দেখায় — অ্যাসাইনড থাকলেও নাম না থাকলে "Holder not recorded" লেখে, যাতে তথ্যের ঘাটতি লুকানো না হয়। ট্যাপ করলে `AssetDetailScreen.open` কল হয়।

ক্লাস/উইজটে: _SoftwareTab

`ConsumerWidget`, লাইসেন্স তালিকা স্ট্যাটাস ফিল্টারসহ দেখায়।

`Widget build(BuildContext context, WidgetRef ref)`

`licenceStatusFilterProvider` থেকে বর্তমান স্ট্যাটাস ফিল্টার পড়ে `FilterBar` দেখায়, এবং নচি `PagedList<SoftwareLicense>` দিয়ে `_LicenseCard` আইটেমসহ তালিকা রেন্ডার করে।

ক্লাস/উইজটে: _RenewalsButton

`StatelessWidget`, `AppBar`-এর একটি আইকন বাটন যা রনিউয়াল স্ক্রিনি খোলে।

`Widget build(BuildContext context)`

`IconButton` রেন্ডার করে, চাপলে `LicenceRenewalsScreen.open(context)` কল করে।

ক্লাস/উইজটে: _LicenseCard

`ConsumerStatefulWidget`, একটি সফটওয়্যার লাইসেন্সের কার্ড — নাম, প্রকাশক, স্ট্যাটাস, সটি ব্যবহারের বার, ময়োদ শেষ/ওভার-অ্যালোকেশন সতর্কতা এবং সটি অ্যাসাইন করার বাটন দেখায়।

`ConsumerState<_LicenseCard> createState()`

`_LicenseCardState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: _LicenseCardState

`_LicenseCard`-এর স্টেটে ক্লাস। সটি অ্যাসাইন করার প্রক্রিয়া এবং লোডিং স্টেটে পরিচালনা করে।

`Future<void> _assignSeat()`

`EmployeePicker` দিয়ে একজন কর্মী বাছাই করায়, তারপর `itamRepositoryProvider.assignSeat()` কল করে সটি অ্যাসাইন করে। যহেতু API রেসপন্সে কিছু ফেরত দেয় না, তাই `licensesProvider.notifier.refresh()` কল করে সটি-সংখ্যা পুনরায় লোড করে। সফল/ব্যর্থ উভয় ক্ষেত্রে `SnackBar` দেখায় এবং `_busy` ফ্ল্যাগ সটে/আনসটে করে।

`Widget build(BuildContext context)`

লাইসেন্সের নাম, প্রকাশক, স্ট্যাটাস চপি, `_SeatBar` রেন্ডার করে। ময়োদ শেষ বা শীঘ্রই শেষ হলে `MessageBanner` (error/warning) দেখায়; ওভার-অ্যালোকেশনে হলে সতর্কতা দেখায়। `softwareAssign` পারমিশন থাকলে সটি অ্যাসাইন বাটন দেখায় (ফাঁকা সটি না থাকলে নষ্ট করে)। ট্যাপ করলে `SoftwareDetailScreen.open` কল হয়।

ক্লাস/উইজটে: _SeatBar

`StatelessWidget`, একটি লাইসেন্সের সটি ব্যবহারের প্রগ্রেস বার — ব্যবহৃত/মোট সটি, ফ্রি সটি সংখ্যা এবং রঙ (danger/warning/success) দিয়ে অবস্থা বোঝায়।

`Widget build(BuildContext context)`

`license.isOverAllocated`, `isFull` অনুযায়ী রঙ নির্ধারণ করে, তারপর `LinearProgressIndicator` ও লবেলে টেক্সট রেন্ডার করে।

ক্লাস/উইজটে: _OffboardingTab

`ConsumerWidget`, অফবোর্ডিং চেকলিস্টের তালিকা — "শুধু বাক থাকা" নাকি "সব" তা টগল করা যায়।

`Widget build(BuildContext context, WidgetRef ref)`

`offboardingOpenOnlyProvider` অনুযায়ী `pendingOffboardingProvider` বা `offboardingProvider` থেকে ডটো নেয়। ফলাফল খালি হলে খালি-স্টেটে দেখায়। তালিকাকে সাজায় — অসম্পূর্ণ প্রথম, তার মধ্যে ওভারডিউ সবার আগে, তারপর সম্পূর্ণতার শতাংশ অনুযায়ী। `RefreshIndicator` দিয়ে পুল-টু-রিফ্রেশ সাপোর্ট করে, যা উভয় প্রোভাইডার রিফ্রেশ করে।

ক্লাস/উইজটে: _ChecklistRow

`StatelessWidget`, একটি অফবোর্ডিং চেকলিস্টের সারাংশ কার্ড — ব্যক্তির নাম, স্ট্যাটাস চপি (Done/Overdue), ধাপের প্রগ্রেস বার ও লক্ষ্য/সম্পন্ন তারিখ দেখায়।

`Widget build(BuildContext context)`

কার্ডটি ট্যাপ করলে `OffboardingDetailScreen.open` কল হয়। ধাপের সংখ্যা অনুযায়ী প্রগ্রেস বার এবং তারিখ (ওভারডিউ হলে লাল রঙে) দেখায়।

ক্লাস: OffboardingOpenOnlyController

Notifier<bool> — অফবোর্ডিং ট্যাব "শুধু খোলা" নাকি "সব" দেখাচ্ছে তার স্টেটে রাখতে; ডিফল্টভাবে true (শুধু খোলা)।

bool build()

প্রাথমিক স্টেটে true রিটার্ন করে।

void set(bool openOnly)

স্টেটে আপডেট করে।

ক্লাস/উইজটে: _OffboardingScope

ConsumerWidget, "Still open" বনাম "All of them" বছে নেওয়ার FilterBar।

Widget build(BuildContext context, WidgetRef ref)

বর্তমান offboardingOpenOnlyProvider স্টেটে অনুযায়ী নির্বাচনী অপশন দেখায় এবং পরবর্তীতে OffboardingOpenOnlyController.set কল করে।

ফাইল: lib/features/itam/license_form_sheet.dart

সংক্ষিপ্ত বিবরণ: সফটওয়্যার লাইসেন্স যোগ বা সম্পাদনা করার জন্য বটম শীট ফর্ম। ভন্ডর পাসওয়ার্ড হ্যাশ সংগ্রহ বা প্রেরণ করে না, কারণ রসেসপন্সে তা ফেরত আসে না।

টপ-লেভেলে ফাংশন: showNewLicenseSheet

Future<void> showNewLicenseSheet(BuildContext context, {SoftwareLicense? existing})

একটি মডাল বটম শীট খোলে যা _LicenseFormSheet দেখায়। existing দিলে সম্পাদনা মোডে খোলে, না দিলে নতুন লাইসেন্স তৈরি মোডে খোলে।

ক্লাস/উইজটে: _LicenseFormSheet

ConsumerStatefulWidget, লাইসেন্স তৈরি/সম্পাদনার ফর্মের ধারক উইজটে।

ConsumerState<_LicenseFormSheet> createState()

_LicenseFormSheetState ইনস্ট্যান্স তৈরি করে।

ক্লাস: _LicenseFormSheetState

লাইসেন্স ফর্মের পুরো লজিক — টেক্সট কন্ট্রোলার, ভ্যালিডেশন, সাবমিশন — এখানে থাকে।

bool get _isEdit

widget.existing != null কনি তা রিটার্ন করে, অর্থাৎ ফর্মটি সম্পাদনা মোডে আছে কনি তা নির্দেশ করে।

void dispose()

সব TextEditingController (সফটওয়্যার নাম, প্রকাশক, লাইসেন্স-কী, ভার্সন, সটি, খরচ, রনিউয়াল খরচ) ডিসপোজ করে এবং super.dispose() কল করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে; ইতিমধ্যে সাবমিট হচ্ছে থাকলে ফিরে যায়। ময়োদ শেষের তারখি কনোর তারখিরে আগে হলে এরর দেখায়। `SoftwareLicenseRequest` তৈরি করে — যদি এডিট মোড হয় তাহলে `softwareRepositoryProvider.update()` কল করে এবং `licensesProvider`-এ ফলাফল প্রয়োগ (apply) করে ও `licenceProvider`-কে ইনভ্যালিডেট করে; নতুন হলে `licensesProvider.notifier.create()` কল করে। সফল হলে শাট বন্ধ করে `SnackBar` দেখায়; ব্যর্থ হলে এরর বার্তা `_error` স্টেটে সেট করে।

Widget build(BuildContext context)

পুরো ফর্ম UI রেন্ডার করে — সফটওয়্যার নাম, প্রকাশক, লাইসেন্স-কী, ভার্সন, লাইসেন্স টাইপ ড্রপডাউন, সটি সংখ্যা, খরচ, কনোর/ময়োদ শেষের তারখি (`DateField`), এবং রনিডিয়াল সেকশন (রনিডিয়াল টাইপ, পরবর্তী রনিডিয়াল তারখি, রনিডিয়াল খরচ — যদি `PERPETUAL` না হয়)। শেষে `LoadingButton` দিয়ে সাবমিট বাটন দেখায়।

static String? Function(String?) _required(String message)

একটি ভ্যালিডেটর ফাংশন তৈরি করে যা খালি বা শুধু স্পেস থাকা ইনপুটের জন্য দেওয়া `message` রিটার্ন করে, নাহলে `null`।

ফাইল: lib/features/itam/offboarding_detail_screen.dart

সংক্ষিপ্ত ববিরণ: একজন কর্মীর অফবোর্ডিং চকেলসিটে বসিতারতি স্ক্রিনি — পাঁচটি ধাপের অবস্থা, নোট, এবং সেই কর্মীর কাছে থাকা সফটওয়্যার সটির তালিকা দেখায়। ধাপগুলো একমুখী — একবার সম্পন্ন হলে ফিরিয়ে নেওয়া যায় না।

ক্লাস/উইজটে: OffboardingDetailScreen

`ConsumerStatefulWidget`, একটি নির্দিষ্ট অফবোর্ডিং চকেলসিট প্রদর্শনকারী স্ক্রিনি।

`OffboardingDetailScreen({super.key, required this.checklist})`

কনস্ট্রাক্টর, প্রদর্শনযোগ্য `checklist` নেয়।

`static void open(BuildContext context, {required OffboardingChecklist checklist})`

`MaterialPageRoute` দিয়ে নতুন পজে পুশ করে `OffboardingDetailScreen` খোলে।

`ConsumerState<OffboardingDetailScreen> createState()`

`_OffboardingDetailScreenState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: _OffboardingDetailScreenState

চকেলসিটের স্টেটে এবং ধাপ সম্পন্ন/চকেলসিট মুছে ফেলার লজিক পরিচালনা করে।

Future<void> _delete()

`confirmAction` দিয়ে নিশ্চিতকরণ চায়, তারপর `itamRepositoryProvider.deleteChecklist()` কল করে সম্পূর্ণ চকেলসিট মুছে ফেলে। এটি শুধুমাত্র ভুলবশত চকেলসিট খোলা হলে ব্যবহারযোগ্য, কারণ এতে কোনো ধাপ ফিরিয়ে আনা যায় না। সফল হলে `pendingOffboardingProvider` ইনভ্যালিডেট করে, `offboardingProvider` রফির্শে করে এবং আগের স্ক্রিনি ফিরে যায়।

Future<void> _complete(OffboardingStep step)

_NotesDialog দেখিয়ে ঐচ্ছকি নোট চায় (ডায়ালগ বাতলি করলে null আসে, যা কোনও পরিবর্তন করে না)। এরপর itamRepositoryProvider.completeStep() কল করে ধাপটি সম্পূর্ণ হসিবে চহিনতি করে, offboardingProvider-এ ফলাফল প্রয়োগ করে এবং স্থানীয় _checklist স্টেটে আপডেটে করে।

Widget build(BuildContext context)

offboardingManage পারমশিন চকে করে canManage নরিধারণ করে। _Summary, ধাপগুলোর তালিকা (_StepRow), _LicencesHeld এবং সামগ্রিকি নোট (যদি থাকে) দেখায়। ম্যানজে পারমশিন না থাকলে তথ্যমূলক ব্যানার দেখায়। AppBar-এর পপআপ মনেতে "Delete the checklist" অপশন থাকে (শুধু canManage থাকলে)।

ক্লাস/উইজেটে: _Summary

StatelessWidget, চকেলসিটরে সামগ্রিকি অগ্রগতি— কতটি ধাপ সম্পূর্ণ, প্রগ্রসে বার, শেষে দনি ও লক্ষ্য/সম্পূর্ণ তারখি দেখায়।

Widget build(BuildContext context)

সম্পূর্ণ হলে "Everything is done" নাহলে "X of Y steps done" দেখায়, স্ট্যাটাস চপি (Completed/Overdue), প্রগ্রসে বার এবং _Line উইজেটে দয়ি তারখিরে তথ্য দেখায়।

ক্লাস/উইজেটে: _Line

StatelessWidget, একটি আইকন-সহ তথ্য লাইন — সাধারণত তারখি দেখানোর জন্য ব্যবহৃত।

Widget build(BuildContext context)

icon ও text একটি সারতি দেখায়, tone দেওয়া থাকলে সেই রঙে (যেমন ওভারডুই হলে danger রঙে) ও বোল্ড ফন্টে দেখায়।

ক্লাস/উইজেটে: _StepRow

StatelessWidget, একটি অফবোর্ডিং ধাপের সারি— সম্পূর্ণ/অসম্পূর্ণ আইকন, লবেলে, তারখি+কার-দ্বারা, নোট এবং "Mark done" বাটন দেখায়।

Widget build(BuildContext context)

ধাপ সম্পূর্ণ হলে চকে আইকন ও ধূসর টেক্সট, নাহলে খালি বৃত্ত আইকন দেখায়। busy হলে লোডিং ইন্ডিকিটের, নাহলে (যদি canManage ও ধাপ অসম্পূর্ণ হয়) "Mark done" বাটন দেখায় যা onComplete কল করে।

ক্লাস/উইজেটে: _NotesDialog

StatefulWidget, ধাপ সম্পূর্ণ করার সময় ঐচ্ছকি নোট নেওয়ার ডায়ালগ। সার্ভার-সাইডেও নোট ঐচ্ছকি, তাই "Skip" একটি বিধে পছন্দ, কাজে পরিত্যাগ নয়।

State<_NotesDialog> createState()

_NotesDialogState ইনস্ট্যান্স তরৈকিরে।

ক্লাস: _NotesDialogState

নোট টেক্সট ইনপুট নয়ন্তরণ করে।

void dispose()

_controller ডিসপোজ করে এবং super.dispose() কল করে।

Widget build(BuildContext context)

AlertDialog রন্ডার করে — একটি সতর্কতা ব্যানার (এটি আনডু করা যাবে না), একটি TextField (সর্বোচ্চ ২ লাইন, ৩০০ ক্যারেক্টার), এবং Cancel/Mark done বাটন। "Mark done" চাপলে টেক্সট রিটার্ন করে ডায়ালগ বন্ধ করে; "Cancel" চাপলে null রিটার্ন করে।

ক্লাস/উইজটে: _LicencesHeld

ConsumerWidget, একজন কর্মীর কাছে থাকা সফটওয়্যার সটিরে তালিকা — লাইসেন্স রজিস্টার থেকে সরাসরি পড়ে, চকেলস্ট থেকে নয়, কারণ চকেলস্ট শুধু "revoke হয়েছে" রেকর্ড করে, কী revoke করতে হবে তা নয়।

Widget build(BuildContext context, WidgetRef ref)

licencesHeldProvider(employeeId) থেকে ডটো নেয়। লোডিং/এরর/খালি/ডটো — চার অবস্থার জন্য যথাক্রমে লোডার, সতর্কতা ব্যানার (এরর হলে "খালি" বলে দেখানো হয় না, যাতায়ে ভুলভাবে ধাপ টকি করা না হয়), "কোনো সটি নাই" বার্তা, বা সটিগুলোর তালিকা দেখায়।

ফাইল: lib/features/itam/software_detail_screen.dart

সংক্ষিপ্ত বিবরণ: একটি নির্দিষ্ট সফটওয়্যার লাইসেন্সের বিস্তারিত স্ক্রিনি — সটি ব্যবহার, ম্যোদ, কারা সটি ব্যবহার করছে তা দেখায় এবং সটি অ্যাসাইন/রলিজি করার সুবিধা দেয়। এছাড়াও এই ফাইলে SoftwareRepository, লাইসেন্স-সম্পর্কিত প্রোভাইডার এবং রনিডিয়াল স্ক্রিনি রয়েছে।

ক্লাস: LicenceSeat

একটি লাইসেন্সের একটি সটি ক ধারণ করছে তার তথ্য বহনকারী ডটো ক্লাস (id, employeeId, employeeName, softwareName, assignedAt)।

LicenceSeat({required this.id, required this.employeeId, this.employeeName, this.softwareName, this.assignedAt})

কনস্ট্রাক্টর, সব ফিল্ড নেয়।

factory LicenceSeat.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে LicenceSeat অবজেক্ট তৈরি করে, সংখ্যাগত id/employeeId num?-কে int-এ রূপান্তর করে, না পলে 0 ডিফল্ট দেয়।

ক্লাস: SoftwareRepository

সফটওয়্যার লাইসেন্স রজিস্টারের নটেওয়ার্ক লয়োর — তালিকা স্ক্রিনি যা কভার করে না তার বাকি অংশ (একক লাইসেন্স, সটি হোল্ডার, রনিডিয়াল/এক্সপায়ার) হ্যান্ডলে করে।

SoftwareRepository(this._api)

কনস্ট্রাক্টর, ApiClient নেয়।

Future<SoftwareLicense> licence(int id)

GET /v1/itam/software/{id} কল করে একক লাইসেন্স ফেরত দেয়।

`Future<List<SoftwareLicense>> expiring()`

`GET /v1/itam/software/expiring` কল করে শীঘ্রই ময়োদ শেষে হবে এমন লাইসেন্সের তালিকা ফেরত দিয়ে (কতটা "শীঘ্রই" তা ব্যাকএন্ড ঠিকি করে)।

`Future<List<SoftwareLicense>> expired()`

`GET /v1/itam/software/expired` কল করে ইতিমধ্যে ময়োদ শেষে হওয়া লাইসেন্সের তালিকা ফেরত দিয়ে।

`Future<SoftwareLicense> update(int id, SoftwareLicenseRequest request)`

`PATCH /v1/itam/software/{id}` কল করে লাইসেন্স আপডেট করে। ভন্ডের পাসওয়ার্ড হ্যাশ পাঠায় না — রসেপন্সে তা ফেরত না আসায় এডিট ফর্মের তা সডি করার উপায় নাই।

`Future<void> delete(int id)`

`DELETE /v1/itam/software/{id}` কল করে লাইসেন্স রজিস্টার থেকে মুছে ফেলে।

`Future<List<LicenceSeat>> seats(int id)`

`GET /v1/itam/software/{id}/seats` কল করে বর্তমানে কারা সডি ধারণ করছে তার তালিকা ফেরত দিয়ে।

`Future<List<LicenceSeat>> forEmployee(int employeeId)`

`GET /v1/itam/software/employee/{employeeId}` কল করে একজন কর্মী কী কী ধারণ করছে তার তালিকা ফেরত দিয়ে; অফবোর্ডিং এডিট পিডে জানে কী revoke করতে হবে।

উপ-লভেলে প্রোভাইডার

- `softwareRepositoryProvider: SoftwareRepository`-এর `Provider`। - `licenceProvider: id` দিয়ে একক লাইসেন্স আনার `FutureProvider.family`। - `licencesHeldProvider: employeeId` দিয়ে একজন কর্মীর সডি আনার `FutureProvider.family`, অফবোর্ডিং চকেলস্ট্রে জন্ম ব্যবহৃত। - `licenceSeatsProvider: id` দিয়ে একটা লাইসেন্সের সডি হোল্ডার আনার `FutureProvider.family`। - `expiringLicencesProvider: শীঘ্রই ময়োদ শেষে হবে এমন লাইসেন্সের FutureProvider`। - `expiredLicencesProvider: ময়োদ শেষে হওয়া লাইসেন্সের FutureProvider`।

ক্লাস/উইজটে: SoftwareDetailScreen

`ConsumerStatefulWidget`, একটা লাইসেন্সের বসিতারতি স্ক্রিনি — খরচ, ময়োদ, সডি ব্যবহার, এবং সডি হোল্ডারদের তালিকা দেখায়।

`SoftwareDetailScreen({super.key, required this.licence})`

কনস্ট্রাক্টর, প্রদর্শনযোগ্য প্রাথমিক `licence` নেয়।

`static void open(BuildContext context, {required SoftwareLicense licence})`

`MaterialPageRoute` দিয়ে নতুন পেজে পুশ করে `SoftwareDetailScreen` খোলে।

`ConsumerState<SoftwareDetailScreen> createState()`

`_SoftwareDetailScreenState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: _SoftwareDetailScreenState

লাইসেন্স বসিতারতি স্ক্রিনির স্টেটে — সডি অ্যাসাইন/রলিজি, লাইসেন্স মুছে ফেলে।

Future<void> _act(Future<void> Function(ItamRepository repo) action, String success, String failure)

একটি সাধারণ হলেপার — `_busy` স্টেট করে, দেওয়া `action` চালায়, সফল হলে `licenceSeatsProvider` ও `licenceProvider` ইনভ্যালিডেট করে (কারণ সটি ও লাইসেন্সের কাউন্ট উভয়ই বদলায়) এবং `success/failure` বার্তা `SnackBar`-এ দেখায়।

Future<void> _delete(SoftwareLicense licence)

`confirmAction` দিয়ে নিশ্চিতকরণ চায় — বার্তায় জানায় কতজন সটি ব্যবহার করছে তারা সটি হারাবে। নিশ্চিত হলে `softwareRepositoryProvider.delete()` কল করে, `licensesProvider` রিফ্রেশ করে এবং আগের স্ক্রিনে ফিরে যায়।

Future<void> _assign()

`EmployeePicker` দিয়ে একজন কর্মী বাছাই করায়, তারপর `_act` দিয়ে `itamRepository.assignSeat()` কল করে সটি অ্যাসাইন করে।

Future<void> _release(LicenceSeat seat)

`confirmAction` দিয়ে নিশ্চিতকরণ চায়, তারপর `_act` দিয়ে `itamRepository.releaseSeat()` কল করে সটি মুক্ত করে।

Widget build(BuildContext context)

`licenceProvider(widget.licence.id)`-এর নতুন মান (থাকলে) ব্যবহার করে, নাহলে সডি হিসেবে পাওয়া `widget.licence` ব্যবহার করে। `softwareAssign` ও `softwareCreate` পারমিশন চেক করে যথাক্রমে সটি ম্যানজেনেন্ট ও এডটি/ডেলিটি এক্সেসে নির্ধারণ করে। দুটি `StatCard` (সটি ব্যবহার, ম্যোদ) দেখায়, তারপর লাইসেন্সের বিবরণ কার্ড এবং সটি হোল্ডারদের তালিকা (`ConfigRow`, প্রতিটি পাশে "Take the seat back" অ্যাকশন যদি `canManage` থাকে) দেখায়। ফাঁকা সটি থাকলে FAB দিয়ে "Assign a seat" দেখায়।

ক্লাস/উইজটে: LicenceRenewalsScreen

`ConsumerWidget`, দুটি ট্যাব ("Running out", "Expired") সহ রনিউয়াল স্ক্রিন — রজিস্টার ব্রাউজ করার উপায় নয়, বরং একটি নির্দিষ্ট দিনে কটে যা করে তার জন্য পৃথক স্ক্রিন।

static void open(BuildContext context)

`MaterialPageRoute` দিয়ে `LicenceRenewalsScreen` পুশ করে।

Widget build(BuildContext context, WidgetRef ref)

`DefaultTabController`(length: 2) দিয়ে দুটি `_LicenceList` (expiring, expired প্রোভাইডার দিয়ে) দেখায়।

ক্লাস/উইজটে: _LicenceList

`ConsumerWidget`, একটি প্রদত্ত প্রোভাইডার থেকে লাইসেন্স তালিকা দেখানোর সাধারণ উইজটে (Running out বা Expired ট্যাবের জন্য ব্যবহৃত)।

Widget build(BuildContext context, WidgetRef ref)

`ConfigList<SoftwareLicense>` দিয়ে দেওয়া `provider`-এর তালিকা দেখায়, প্রতিটি সারিতে নাম, প্রকাশক, সটি সংখ্যা, ম্যোদ তারিখ ও বাকি/অতিবাহতি দিন দেখায় (ম্যোদ শেষে হলে ধূসর/inactive হিসেবে)। `onEdit` চাপলে `SoftwareDetailScreen.open` কল হয়।

ফাইল: lib/features/itam/start_offboarding_sheet.dart

সংক্ষিপ্ত ববিরণ: নতুন অফবোর্ডিং চকেলস্টিট শুরু করার জন্য ছোট বটম শিট — শুধুমাত্র কর্মী বাছাই করা প্রয়োজন, বাকি প্যাঁচটি ধাপ পরে করা হয়।

টপ-লেভেলে ফাংশন: showStartOffboardingSheet

`Future<void> showStartOffboardingSheet(BuildContext context)`

একটি মিডাল বটম শিট খোলে যা `_StartOffboardingSheet` দেখায়।

ক্লাস/উইজটে: _StartOffboardingSheet

`ConsumerStatefulWidget`, অফবোর্ডিং শুরু করার ফর্মের ধারক উইজটে।

`ConsumerState<_StartOffboardingSheet> createState()`

`_StartOffboardingSheetState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: _StartOffboardingSheetState

কর্মী বাছাই, নোট এবং সাবমিশন লজিক পরিচালনা করে।

`void dispose()`

`_notes` কন্ট্রোলার ডিসপোজ করে এবং `super.dispose()` কল করে।

`Future<void> _pickEmployee()`

`EmployeePicker.show()` দিয়ে "Who is leaving?" শিরোনামে কর্মী বাছাই করায় এবং ফলাফল `_employee` স্টেটে সেটে করে।

`Future<void> _submit()`

কোনো কর্মী বাছাই না করা থাকলে এরর দেখায়। নাহলে `offboardingProvider.notifier.create()` কল করে `OffboardingChecklistRequest` পাঠায় (`employeeId`, `notes`)। সফল হলে শিট বন্ধ করে "Offboarding started" বার্তা দেখায়; আগের থেকেই সেই ব্যক্তির জন্য চকেলস্টিট খোলা থাকলে `ApiException`-এর মैसेজের এরর দেখায়।

`Widget build(BuildContext context)`

টাইটলে, এরর ব্যানার (থাকলে), কর্মী বাছাই করার `InputDecorator` (উপরে `_pickEmployee`), ঐচ্ছিক নোট `TextField`, এবং "Start offboarding" `LoadingButton` রেন্ডার করে।

ফাইল: lib/features/kb/kb_authoring_screen.dart

সংক্ষিপ্ত ববিরণ: নলজে বসের লেখক-স্ক্রিনি — খসড়া, প্রকাশিত এবং সংরক্ষণাগারভুক্ত সব ধরনের আর্টিকলে দেখানো, লেখা, প্রকাশ ও সংরক্ষণাগারভুক্ত করার সুবিধা দেয়। পাঠক-স্ক্রিনি থেকে আলাদা রাখা হয়েছে কারণ পাঠক শুধু প্রকাশিত আর্টিকলে চায়, লেখক খসড়াও দেখতে চায়।

ক্লাস: KbStatusFilterController

`Notifier<String?>` — অথরিং তালিকা কোন স্ট্যাটাসের আর্টিকলে দেখাচ্ছে তার স্টেটে রাখে; `null` মানের সব।

String? build()

প্রাথমিক স্টেটে null রিটার্ন করে (অর্থাৎ সব আর্টিকলে)।

void set(String? status)

স্টেটে আপডেট করে।

টপ-লেভেলে প্রোভাইডার

- kbStatusFilterProvider: KbStatusFilterController-এর NotifierProvider। - kbAllArticlesProvider: বর্তমান স্ট্যাটাস ফিল্টার অনুযায়ী সব আর্টিকলে (৫০টি পর্যন্ত) আনার FutureProvider.dispose, kbRepositoryProvider.allArticles() কল করে ও পজেড রেসপন্সের content রিটার্ন করে।

ক্লাস/উইজটে: KbAuthoringScreen

ConsumerWidget, নলজে বসে আর্টিকলে ম্যানেজমেন্টের প্রধান স্ক্রিনি।

Widget build(BuildContext context, WidgetRef ref)

create ও update পারমিশন চেক করে canCreate/canEdit নির্ধারণ করে। canCreate থাকলে FAB ("Write one") দেখায় যা showArticleSheet খোলে। ConfigList<KbArticle> দিয়ে তালিকা দেখায়, শীর্ষে FilterBar (All/Drafts/Published/Archived) থাকে। প্রতিটি সারি _ArticleRow দিয়ে রেন্ডার হয়।

ক্লাস/উইজটে: _ArticleRow

ConsumerStatefulWidget, একটি আর্টিকলের সারি — শিরোনাম, ক্যাটাগরি, দৃশ্যমানতা, ভিউ সংখ্যা, প্রকাশের তারিখ, এবং পাবলিশ/আর্কাইভ অ্যাকশন দেখায়।

ConsumerState<_ArticleRow> createState()

_ArticleRowState ইনস্ট্যান্স তৈরি করে।

ক্লাস: _ArticleRowState

আর্টিকলের অবস্থা পরিবর্তনের (পাবলিশ/আর্কাইভ) লজিক পরিচালনা করে।

Future<void> _run(Future<KbArticle> Function(KbRepository) action, String done)

একটি সাধারণ হলেপার — _busy স্টেট করে দেওয়া action চালায়, সফল হলে kbAllArticlesProvider ও kbArticlesProvider (পাঠক-তালিকা, যা শুধু প্রকাশিত আর্টিকলে দেখায় এবং এই অ্যাকশনে বদলে যেতে পারে) উভয়ই ইনভ্যালিডেট করে, এবং done বার্তা SnackBar-এ দেখায়।

Future<void> _archive()

confirmAction দিয়ে নিশ্চিতকরণ চায় (আর্টিকলেটি আর খুঁজে পাওয়া যাবে না, কিন্তু কিছুই মোছা হবে না এবং পুনরায় প্রকাশ করলে ফিরে আসবে)। নিশ্চিতি হলে _run দিয়ে repo.archive(article.id) কল করে।

Widget build(BuildContext context)

আর্টিকলের তথ্য (ক্যাটাগরি, ক্যাচনেট-দৃশ্যমানতা, ভিউ সংখ্যা, প্রকাশের তারিখ) একটি facts তালিকায় জড়ো করে ConfigRow-এ দেখায়। canEdit থাকলে এডটি (showArticleSheet) সক্রিয় থাকে। স্ট্যাটাস অনুযায়ী "Publish" ও/অথবা "Archive" (destructive) অ্যাকশন দেখায়।

টপ-লেভেলে ফাংশন: showArticleSheet

Future<void> showArticleSheet(BuildContext context, {KbArticle? article})

একটি মিডাল বটম শিট খোলে যা `_ArticleSheet` দেখায়। `article` দিলে এডটি মোডে খোলে; এডটি মোডে ফর্ম-না-দেখানো ফিল্ডসহ সমস্ত ফিল্ড আর্টিকলে থাকে সেডি করা হয়, কারণ আপডেট এন্ডপয়েন্ট পাঠানো ডটো দিয়ে সম্পূর্ণ ওভাররাইট করে, মার্জ করে না।

ক্লাস/উইজটে: _ArticleSheet

ConsumerStatefulWidget, আর্টিকলে লেখা/সম্পাদনার ফর্মের ধারক উইজটে।

ConsumerState<_ArticleSheet> createState()

_ArticleSheetState ইনস্ট্যান্স তৈরি করে।

ক্লাস: _ArticleSheetState

আর্টিকলে ফর্মের ইনপুট কন্ট্রোলার, ভ্যালিডেশন ও সাবমিশন লজিক পরিচালনা করে।

void dispose()

_title, _summary, _content, _keywords কন্ট্রোলার ডিসপোজ করে এবং `super.dispose()` কল করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে। এডটি মোডে হলে `KbArticleRequest.from(existing)` দিয়ে বিদ্যমান আর্টিকলের সব ফিল্ড দিয়ে শুরু করে (ক্যাটাগরিও সম্পর্কিত সার্ভিসের মতো ফর্ম-না-দেখানো ফিল্ড হারিয়ে যাওয়া এড়াতে), তারপর `copyWith()` দিয়ে ফর্মের ফিল্ড (title, content, clientVisible, summary, keywords) ওভাররাইট করে; নতুন হলে খালি `KbArticleRequest` দিয়ে শুরু করে। এরপর `repo.create()` বা `repo.update()` কল করে, সংশ্লিষ্ট প্রোভাইডার (`kbAllArticlesProvider`, `kbArticlesProvider`, প্রয়োজ্য হলে `kbArticleProvider`) ইনভ্যালিডেট করে এবং শিট বন্ধ করে।

Widget build(BuildContext context)

`FormSheetFrame` ব্যবহার করে ফর্মের কাঠামো (শিরোনাম, এরর, সাবমিট বাটন) তৈরি করে। ফিল্ডসমূহ: শিরোনাম (আবশ্যিক), সারাংশ (ঐচ্ছিক, ২ লাইন), মূল লেখা (আবশ্যিক, ৬-১০ লাইন), কীওয়ার্ড (ঐচ্ছিক), এবং "Clients can read it" সুইচ। নতুন আর্টিকলের ক্ষেত্রে নচি একটি নোট দেখায় যে এটি খসড়া হিসেবে সংরক্ষিত হবে।

done

ফাইল: lib/features/kb/kb_models.dart

সংক্ষিপ্ত বিবরণ: নলজে বেস (Knowledge Base) ফিচারের মডেল ফাইল। এখানে পারমিশন কোড, আর্টিকলের স্ট্যাটাস, `KbArticle` (আর্টিকলে পড়ার মডেল) এবং `KbArticleRequest` (আর্টিকলে লেখা/এডিট করার মডেল) সংজ্ঞায়িত করা হয়েছে।

ক্লাস: KbPermissions

নলজে বসে সংক্রান্ত পারমিশন কোডগুলোর একটি abstract final ক্লাস — কোনো ইনস্ট্যান্স তৈরি করা যায় না, শুধু স্ট্যাটিক কনস্ট্যান্ট রাখার জন্য ব্যবহৃত। তিনটি কনস্ট্যান্ট আছে: `view` (`KNOWLEDGE_BASE_VIEW`) — স্টাফদের জন্য লিস্ট দেখার গটে, ক্লায়েন্ট ইউজারের ক্ষেত্রে এই চকে এড়িয়ে যাওয়া হয় (তারা পোর্টাল থেকে পাবলিশড ও ক্লায়েন্ট-ভিজিবল আর্টিকলে ব্রাউজ করতে পারে); `create` (`KNOWLEDGE_BASE_CREATE`); এবং `update` (`KNOWLEDGE_BASE_UPDATE`) — এডটি, পাবলিশ ও আর্কাইভ তিনটিই এই একই কোড দিয়ে নিয়ন্ত্রণ করা হয়, আলাদা কোনো পাবলিশ পারমিশন নেই।

ক্লাস: KbArticleStatus

আর্টিকলের সম্ভাব্য স্ট্যাটাস মানগুলোর জন্য abstract final ধ্রুবক ক্লাস — `draft` (`DRAFT`), `published` (`PUBLISHED`), `archived` (`ARCHIVED`)।

ক্লাস: KbArticle

একটি নলজে বসে আর্টিকলের ডটো মডেল। `content` ফিল্ডটি প্লইন টেক্সট হিসেবে সংরক্ষণ এবং রেন্ডার করা হয় — মার্কআপ হিসেবে পার্স করা হয় না, কারণ ওয়েব এডটির কোনো নির্দিষ্ট ফরম্যাট নিশ্চিত করা নেই, এবং ফোনে ভুল অনুমান করলে ট্যাগ দেখাবে টেক্সটের বদলে। `id`, `title`, `status`, `viewCount`, `helpfulCount` আবশ্যিক ফিল্ড; বাকগুলো (`summary`, `content`, `keywords`, `categoryId/Name`, `relatedServiceId/Name`, `authorName`, `publishedAt`) ঐচ্ছিক। `clientVisible` ডিফল্ট `false`।

`KbArticle({required this.id, required this.title, ...})`

কনস্ট্রাক্টর যা সব ফিল্ড সরাসরি সেট করে; `clientVisible` এর ডিফল্ট মান `false`।

`bool get isPublished`

`status` মান `KbArticleStatus.published` এর সমান কিনা তা যাচাই করে একটি বুলিয়ান রিটার্ন করে।

`String? get preview`

লিস্টে টাইটলের নিচের লাইনটি তৈরি করে। প্রথমে `summary` ট্রিম করে দেখে, নন-এম্পটি হলে তা রিটার্ন করে; না থাকলে `content` থেকে হোয়াইটস্পেস একত্র করে ১৬০ ক্যারেক্টার পর্যন্ত কটে এলপিসিসি (...) যোগ করে রিটার্ন করে। কোনোটিই না থাকলে `null` রিটার্ন করে।

`List<String> get keywordList`

`keywords` স্ট্রিং (কমা দিয়ে আলাদা করা) থেকে ট্রিম করা, নন-এম্পটি কিওয়ার্ডের একটি অপরিবর্তনীয় (`growable: false`) লিস্ট তৈরি করে; ডিফল্ট স্ক্রিনিং চাপি হিসেবে দেখানোর জন্য ব্যবহৃত হয়।

`factory KbArticle.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে `KbArticle` তৈরি করে। প্রতিটি ফিল্ডের জন্য null-safe কাস্টিং এবং ডিফল্ট মান (যেমন `id` না থাকলে ০, `title` না থাকলে 'Untitled article', `status` না থাকলে `DRAFT`) ব্যবহার করে।

ক্লাস: KbArticleRequest

আর্টিকলে লেখা বা রি-রাইট করার জন্য রকিওয়েস্ট মডেল। নামে "PATCH" হলো

`KbArticleServiceImpl.update` প্রত্যাশিত ফিল্ড null-চকে ছাড়াই ওভাররাইট করে দেয় — তাই বাদ দেওয়া কোনো কী থাকলে যা আগে ছিল তা মুছে যায়, এবং `clientVisible` জাভা প্রমিটিভি বলে অনুপস্থিতি থাকলে `false` হিসেবে যায় ও নিশ্চিতভাবে আর্টিকলেট ক্লায়েন্টদের কাছ থেকে লুকিয়ে ফেলে। তাই এই মডেলে কোনো ফিল্ড ঐচ্ছিক নয় — এডটি ফর্ম সবসময় আগে পুরো আর্টিকলে লোড করে এবং সম্পূর্ণটাই ফেরত পাঠায়। স্ট্যাটাস এখানে নেই — পাবলিশ/আর্কাইভের জন্য আলাদা এন্ডপয়েন্ট আছে, এডটি কখনো স্ট্যাটাস পরিবর্তন করে না।

`KbArticleRequest({required this.title, required this.content, required this.clientVisible, this.summary, this.keywords, this.categoryId, this.relatedServiceId})`

কনস্ট্রাক্টর যা title, content, clientVisible আবশ্যিক হসিবে এবং বাকি ফিল্ড ঐচ্ছকি হসিবে নেয়ে।

`factory KbArticleRequest.from(KbArticle article)`

একটি বিদ্যমান `KbArticle` থেকে রকি়োস্ট তৈরি করে, যাতে ফর্ম না-দেখানো ফিল্ডগুলো অপরিবর্তিত অবস্থায় ফেরত পাঠানো যায়, মুছে না যায়।

`KbArticleRequest copyWith({...})`

বর্তমান রকি়োস্ট থেকে নির্বাচিত ফিল্ড পরিবর্তন করে নতুন `KbArticleRequest` ইনস্ট্যান্স তৈরি করে; অনুল্লখিত ফিল্ডগুলো আগের মান ধরে রাখবে।

`Map<String, dynamic> toJson()`

সব ফিল্ডকে একটি JSON ম্যাপে সিরিয়লাইজ করে (title, content, clientVisible, summary, keywords, categoryId, relatedServiceId)।

ফাইল: `lib/features/kb/kb_repository.dart`

সংক্ষিপ্ত বিবরণ: নলজে বসে আর্টিকিলের জন্য API রপিয়োটরি এবং Riverpod প্রোভাইডার/কন্ট্রোলারসমূহ। পড়া (ব্রাউজ, সার্চ), লেখা (তৈরি, আপডেট, পাবলিশ, আর্কাইভ) এবং "helpful" মার্ক করার কার্যকারিতা আছে।

ক্লাস: `KbRepository`

নলজে বসে সমস্ত API কল এই ক্লাসে কেন্দ্রীভূত। মূলত ফোনে এটি ব্যবহৃত হয় কাস্টমার অপেক্ষা করার সময় কিছু খুঁজে বের করতে এবং তা সহায়ক ছিল কনি তা জানাতে। লেখার অংশ (তৈরি/আপডেট) `KNOWLEDGE_BASE_CREATE/_UPDATE` পারমিশন দ্বারা গটেডে, যা বেশিরভাগ স্টাফের কাছে নেই।

`KbRepository(this._api)`

কনস্ট্রাক্টর, একটি `ApiClient` ইনস্ট্যান্স গ্রহণ করে।

`Future<PagedResponse<KbArticle>> articles({String? keyword, int page = 0, int size = 20})`

শুধুমাত্র পাবলিশড আর্টিকিলে ফেরত দেয়। ড্রাফট এবং আর্কাইভড আর্টিকিলে একই এন্ডপয়েন্টে ভিন্ন স্ট্যাটাস পাস করে পাওয়া সম্ভব হলেও এখানে ইচ্ছাকৃতভাবে বাদ দেওয়া হয়েছে — ড্রাফট কাস্টমারকে পড়ে শোনানোর মতো কিছু নয়, আর আর্কাইভড আর্টিকিলে ইচ্ছাকৃতভাবেই ভুল।

`Future<KbArticle> article(int id)`

একটি নির্দিষ্ট আইডি পূর্ণাঙ্গ আর্টিকিলে আনবে। লিস্ট এন্ডপয়েন্টও একই শেপে ফেরত দেয়, কিন্তু এই কলটিই সার্ভার-সাইডে ভিউ কাউন্ট বাড়ায়।

`Future<PagedResponse<KbArticle>> allArticles({String? keyword, String? status, int page = 0, int size = 20})`

যেকোনো স্ট্যাটাসের সব আর্টিকিলে আনবে — যারা আর্টিকিলে লেখে তাদের জন্য, যারা পড়ে তাদের জন্য নয়। `articles()` ইচ্ছাকৃতভাবে শুধু পাবলিশড দেখায়; এটি তার বপিরীত ভিউ।

`Future<KbArticle> create(KbArticleRequest request)`

নতুন আর্টিকিলে তৈরি করে POST রকি়োস্টের মাধ্যমে।

Future<KbArticle> update(int id, KbArticleRequest request)

বর্ধমান আর্টিকলে আপডেট করে (PATCH এন্ডপয়েন্টে হলেও প্রকৃতপক্ষে পুরো রিপ্লসেমন্ট — বস্তুতঃ `KbArticleRequest` ডকুমেন্টেশনে)।

Future<KbArticle> publish(int id)

আর্টিকলে পাবলিশ করে; এটি পাবলিশের সময়ও রেকর্ড (স্ট্রাম্প) করে। কোনো "অনপাবলিশ" নই — আর্কাইভ করাই একমাত্র ফরমে আসার পথ, এবং এটি স্ট্রাম্প রেখে দেয়।

Future<KbArticle> archive(int id)

আর্টিকলকে আর্কাইভ স্ট্যাটাসে নেয়।

Future<KbArticle> markHelpful(int id)

আর্টিকলেটি সহায়ক ছিল বলে চিহ্নিত করে। এর কোনো বপিরীত ("unhelpful") নই এবং একবার করলে ফরমে নেওয়ার উপায় নই, তাই `UI`। প্রতিভিজিটি এটি একবারই অফার করে।

টপ-লেভেলে ভ্যারিয়েবল: `kbRepositoryProvider`

`Provider<KbRepository>` — `ApiClientProvider` থেকে `ApiClient` নিয়ে `KbRepository` ইনস্ট্যান্স সরবরাহ করে।

ক্লাস: `KbSearchController`

`Notifier<String?>` — সার্চ টার্মের স্টেটে রাখে। খালি মানে "সব পাবলিশড আর্টিকলে দেখাও"।

`String? build()`

প্রাথমিক স্টেটে `null` রটার্ন করে (কোনো সার্চ ফিল্টার নই)।

`void set(String? keyword)`

প্রদত্ত কীওয়ার্ড ট্রিম করে; ট্রিম করার পর খালি/`null` হলে স্টেটে `null` করে, অন্যথায় সেই ট্রিম করা মান স্টেটে বসায়। মান অপরিবর্তিত হলে কিছু করে না (early return)।

টপ-লেভেলে ভ্যারিয়েবল: `kbSearchProvider`

`NotifierProvider<KbSearchController, String?>` — সার্চ কীওয়ার্ড স্টেটে এক্সপোজ করে।

ক্লাস: `KbArticlesController`

`AsyncNotifier<PagedState<KbArticle>>` যা `PagedLoader<KbArticle>` মক্সইন ব্যবহার করে — পজেনিটেডে আর্টিকলে লিস্ট পরিচালনা করে।

`Future<PagedState<KbArticle>> build()`

`currentUserProvider` ও `kbSearchProvider` ওয়াচ করে (যেকোনো পরিবর্তনে রিলিড ট্রিগার হয়) এবং প্রথম পজে লোড করে।

`Future<PagedResponse<KbArticle>> fetchPage(int page)`

`kbRepositoryProvider` থেকে বর্তমান সার্চ কীওয়ার্ড সহ নির্দিষ্ট পজে আনবে (override, `PagedLoader` মক্সইনের প্রয়োজনীয়তা পূরণ করে)।

`void apply(KbArticle updated)`

আইডি মিলিয়ে লিস্টের একটি আর্টিকলে রো প্রতিস্থাপন করে — `helpful` কাউন্ট পরিবর্তনের পর লিস্টকে সাম্প্রতিক অবস্থার সাথে সামঞ্জস্যপূর্ণ রাখে।

টপ-লভেলে ভ্যারিয়েবল: kbArticlesProvider

`AsyncNotifierProvider<KbArticlesController, PagedState<KbArticle>>` — আর্টিকলে লস্ট কন্ট্রোলার এক্সপোজ করবে।

টপ-লভেলে ভ্যারিয়েবল: kbArticleProvider

`FutureProvider.autoDispose.family<KbArticle, int>` — নরদিশিট আইডরি একক আর্টিকলে আনে, প্রতিটি আইডরি জন্য পৃথক ইনস্ট্যান্স (family) হিসাবে কাজ করে।

ফাইল: lib/features/kb/kb_screen.dart

সংক্ষিপ্ত বিবরণ: নলজে বসে দেখানোর UI — লস্ট স্ক্রিনি (সার্চসহ) এবং একটি আর্টিকলে বিস্তারিত স্ক্রিনি। ফোনে কাস্টমার অপেক্ষা করার সময় দ্রুত কিছু লুকআপ করার জন্য ডিজাইন করা।

ক্লাস/উইজটে: KbScreen

`ConsumerWidget` — নলজে বসে প্রধান লস্ট স্ক্রিনি। পারমিশন লোড না হওয়া পর্যন্ত লোডার দেখায়; `KbPermissions.view` না থাকলে "Not available to you" এম্পটি স্টেটে দেখায়; অন্যথায় সার্চ ফিল্ড ও পেজিনিটেডে আর্টিকলে লস্ট রেন্ডার করে।

`Widget build(BuildContext context, WidgetRef ref)`

পারমিশন কন্ট্রোলার ওয়াচ করে। পারমিশন লোড না-হওয়া অবস্থায় `Loader` দেখায়; `view` পারমিশন না থাকলে লক আইকনসহ `EmptyState` দেখায়; থাকলে `_SearchField` এবং `PagedListview<KbArticle>` (রিফ্রেশে, লোড-মোর, খালি/এরর স্টেটে, আইটেমে বন্ডার হিসাবে `_ArticleCard` সহ) দেখায়।

ক্লাস/উইজটে: _SearchField

`ConsumerWidget` — একটি প্যাডে `AppSearchField` যা `kbSearchProvider`-এ টাইপ করা মান পাঠায়।

`Widget build(BuildContext context, WidgetRef ref)`

সার্চ ফিল্ড রেন্ডার করে; `onChanged`-এ টাইপ করা মান খালি হলে `null` (ফিল্টার ক্লিয়ার করত), নাহলে সরাসরি `kbSearchProvider.notifier.set()`-এ পাঠায়।

ক্লাস/উইজটে: _ArticleCard

`StatelessWidget` — লস্টে একটি আর্টিকলে কার্ড রো দেখায়: টাইটলে, প্রভিউ টেক্সট, ক্যাটাগরি, helpful কাউন্ট এবং `client-visible` হলে একটি স্ট্যাটাস চিপ। ট্যাপ করলে `KbArticleScreen` খোলে।

`Widget build(BuildContext context)`

`InkWell`-এ মোড়ানো `AppCard` রটরন করে যাত টাইটলে, (থাকলে) প্রভিউ টেক্সট ম্যাক্স ২ লাইনে, এবং ক্যাটাগরি/helpful কাউন্ট/client-visible চিপ সম্বলিত একটি রো থাকে। ট্যাপে `KbArticleScreen.open` কল করে।

ক্লাস/উইজটে: KbArticleScreen

`ConsumerStatefulWidget` — একটি একক আর্টিকলে বিস্তারিত স্ক্রিনি, ফোনে করার মতো একটিই কাজ (helpful মার্ক করা) সহ।

```
static void open(BuildContext context, {required KbArticle article})
```

MaterialPageRoute দিয়ে KbArticleScreen পুশ করে নভেগেটে করে।

```
ConsumerState<KbArticleScreen> createState()
```

_KbArticleScreenState ইনস্ট্যান্স তৈরি করে (override)।

ক্লাস: _KbArticleScreenState

KbArticleScreen-এর স্টেটে ক্লাস। _saidHelpful (প্রতিভিজিটি একবারই সহায়ক মার্ক করার অনুমতি—সার্ভার-সাইডে ফেরত নেওয়ার উপায় না থাকায়) এবং _busy (সাবমশিন চলাকালীন) ফ্ল্যাগ রাখে।

```
Future<void> _markHelpful(KbArticle article)
```

_busy স্টেটে করে kbRepository.markHelpful কল করে, সফল হলে kbArticlesProvider.notifier.apply() দিয়ে লিস্ট আপডেটে করে, _saidHelpful স্টেটে করে এবং স্ন্যাকবার দেখায়। ApiException এবং অন্য সাধারণ ত্রুটির জন্য পৃথক স্ন্যাকবার বার্তা দেখায়; finally-তে _busy রিসেটে করে।

```
Widget build(BuildContext context)
```

kbArticleProvider(widget.article.id) ওয়াচ করে; লিস্ট থেকে পাঠানো সারাংশ আর্টিকলে (widget.article) সাথে সাথে দেখায়, পূরণ ডটো এলে তা দিয়ে প্রতস্থাপতি হয়। টাইটলে, লেখক/প্রকাশের তারিখ, কীওয়ার্ড চপি, লোডিং/এরর/কনটেন্ট অবস্থা অনুযায়ী বর্ডা (SelectableText), এবং নচি _busy/_saidHelpful/সাধারণ অবস্থার উপর ভিত্তি করে "This helped" বোতাম বা কনফার্মেশন দেখায়।

ফাইল: lib/features/leave/apply_leave_sheet.dart

সংক্ষিপ্ত ববিরণ: কর্মচারীর নজিরে জন্য ছুটির আবেদন করার বটম শিট। ছুটির ধরন, তারিখ পরিসীমা এবং কারণ নিয়ে ফর্ম, ও সাবমশিন লজিক।

উপ-লভেলে ফাংশন: Future<void> showApplyLeaveSheet(BuildContext context, WidgetRef ref)

showModalBottomSheet কল করে _ApplyLeaveSheet বটম শিট প্রদর্শন করে।

ক্লাস/উইজটে: _ApplyLeaveSheet

ConsumerStatefulWidget — ছুটির আবেদন ফর্মের রুট উইজটে, কোনো নির্মাণকালীন প্যারামিটার নেই।

```
ConsumerState<_ApplyLeaveSheet> createState()
```

_ApplyLeaveSheetState ইনস্ট্যান্স রিটার্ন করে (override)।

ক্লাস: _ApplyLeaveSheetState

ফর্ম স্টেটে: _reason (কারণ টেক্সট কন্ট্রোলার), _type (ডিফল্ট leaveTypes.first), _range (তারিখ পরিসীমা), _submitting ও _error।

```
void dispose()
```

_reason কন্ট্রোলার dispose করে (override)।

`int? get _calendarDays`

নর্বাচতি `_range` থেকে ইনক্লুসভি দিনের সংখ্যা গণনা করে (`end - start + 1`)। কোনো পরসীমা না থাকলে `null`। এটি শুধুই একটি আনুমানিক হিসাব — সারভার সপ্তাহান্ত ও সরকারি ছুটি বিদ্যে পুনরায় গণনা করে, তাই এটিকে "যত দিন কাটা যাব" হিসেবে না দেখিয়ে "ক্যালেন্ডার দিন" লবেলে দেওয়া হয়েছে।

`Future<void> _pickRange()`

`showDateRangePicker` দেখায় (এক বছর পছিন্দ থেকে পরের বছরে ৩১ ডিসেম্বর পর্যন্ত সীমা — পছিন্দ ব্যাকডেটেড অসুস্থতার ছুটি, সামনে পরের বছরে ছুটি বুক করার জন্য), নর্বাচতি পরসীমা `_range`-এ স্টেট করে।

`Future<void> _submit()`

`_range` না থাকলে এরর দেখিয়ে থামে। অন্যথায় `_submitting=true` স্টেট করে `leaveControllerProvider.notifier.apply()` কল করে `LeaveRequestPayload` (`leaveType`, ISO স্টার্ট/এন্ড ডেট, `reason`) সহ। সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়; `ApiException`/সাধারণ ত্রুটির জন্য `_error` স্টেট করে; `finally`-তে `_submitting` রিসেট করে।

`Widget build(BuildContext context)`

কীবোর্ডের উপরে শিট রাখতে `viewInsets` অনুসারে প্যাডিং দেয়। টাইটলে, এরর ব্যানার (থাকলে), লভি টাইপ ড্রপডাউন, তারখি নর্বাচনরে `InkWell+InputDecorator`, নর্বাচতি হল ক্যালেন্ডার দিন সংখ্যার নোট, কারণ টেক্সটফিল্ড, এবং সাবমিট `LoadingButton` রেন্ডার করে।

ফাইল: `lib/features/leave/entitlements_screen.dart`

সংক্ষিপ্ত বিবরণ: ছুটির এনটাইটলেমেন্ট (কে কতটা পায়) এবং নয়িম (পলসি) দেখানোর দুই-ট্যাব স্ক্রিনি। HR পলসি ঠিকি করে এবং তারপর ব্যালেন্স গ্র্যান্ট করে, তাই দুটোই এক স্ক্রিনি।

ক্লাস: `EntitlementYearController`

`Notifier<int>` — এনটাইটলেমেন্ট স্ক্রিনি বর্তমানে কোন বছর দেখানো হচ্ছে তা ট্র্যাক করে।

`int build()`

প্রাথমিক স্টেটে হিসেবে বর্তমান বছর (`DateTime.now().year`) রিটার্ন করে।

`void set(int year)`

স্টেটে নতুন বছর স্টেট করে।

টপ-লভেলে ভ্যারিয়েবল: `entitlementYearProvider`

`NotifierProvider<EntitlementYearController, int> |`

টপ-লভেলে ভ্যারিয়েবল: `allBalancesProvider`

`FutureProvider.autoDispose<List<LeaveBalance>>` — নর্বাচতি বছরে সকল কর্মচারীর ব্যালেন্স `leaveRepositoryProvider.allBalances(year: ...)` থেকে আনে ও তার `content` রিটার্ন করে।

ক্লাস/উইজটে: `EntitlementsScreen`

`StatelessWidget` — দুই-ট্যাব (Who has what / The rules) স্ক্রিনিরে রুট, `DefaultTabController` দিয়ে মোড়ানো।

Widget build(BuildContext context)

AppBar-এ TabBar সহ Scaffold রটাইরন করে, বডতি TabBarView দিয়ে _BalancesTab ও _PoliciesTab দেখায়।

ক্লাস/উইজটে: _BalancesTab

ConsumerWidget — "কার কী আছে" ট্যাব: বছর ফিল্টার সহ ব্যালেন্স লস্ট, এবং পারমিশন থাকলে "Grant days" ফ্লোটিং অ্যাকশন বাটন।

Widget build(BuildContext context, WidgetRef ref)

বর্তমান বছর এবং LeavePermissions.balanceCreate/balanceUpdate-এর যেকোনো একটি থাকা কনি (canManage) নরিণয় করে। ConfigList<LeaveBalance> (বছর FilterBar হডোরসহ, গত/এই/পররে বছর অপশন) দেখায়, প্রতটি আইটেমে _BalanceRow দিয়ে রন্ডার করে। canManage হলে নচি-ডানে FloatingActionButton.extended দেখায় যা showGrantLeaveBalanceSheet খুলে ফরোর পর লস্ট ইনভ্যালডিটে করে।

ক্লাস/উইজটে: _BalanceRow

ConsumerStatefulWidget — একটি ক্রমচারীর একটি ব্যালেন্স রো, ম্যানজে পারমিশন থাকলে এডটি/রমিভ অ্যাকশনসহ।

ConsumerState<_BalanceRow> createState()

_BalanceRowState রটাইরন করে (override)।

ক্লাস: _BalanceRowState

_busy ফ্ল্যাগ রাখে মুছে ফলোর (delete) সময়।

Future<void> _delete()

confirmAction দিয়ে নশ্চতিকরণ চায় ("Remove this entitlement?")। কনফার্ম না হলে বা আনমাউন্টেডে হলে থামে। নাহলে _busy=true করে leaveRepository.deleteBalance() কল করে, সফল হলে allBalancesProvider ইনভ্যালডিটে করে সন্যাকবার দেখায়; ত্রুটিতে যথায়থ বার্তা দেখায়; finally-তে _busy রসিটে করে।

Widget build(BuildContext context)

ConfigRow রটাইরন করে — টাইটলে ক্রমচারীর নাম/আইডি, সাবটাইটলে লভি টাইপ ও বাক/মোট দিনের হিসাব, active = অবশষ্টি দিন > ০, canManage অনুযায়ী এডটি (showEditEntitlementSheet) ও রমিভ (_delete) অ্যাকশন।

টপ-লভেলে ফাংশন: Future<void> showEditEntitlementSheet(BuildContext context, {required LeaveBalance balance})

_EditEntitlementSheet বটম শটি দেখায় একটি এনটাইটলেমেন্ট বদলানোর জন্য। এটি মোট বরাদ্দ (কত পাবে) বদলায়, বাককিত আছে তা নয় — ব্যবহৃত ও পন্ডং দিন রকি়োয়স্ট থকে আসে, কোনো এন্ডপয়ন্ট সরাসর সিগেলো এডটি করে না।

ক্লাস/উইজটে: _EditEntitlementSheet

ConsumerStatefulWidget — একটি নরিদষ্টি LeaveBalance-এর মোট দিন সংখ্যা এডটি করার ফর্ম শটি।

ConsumerState<_EditEntitlementSheet> createState()

_EditEntitlementSheetState রটাইরন করে (override)।

ক্লাস: `_EditEntitlementSheetState`

`_formKey`, `_days` (বর্তমান `entitledDays` দিয়ে প্র-ফিলি করা টেক্সট কন্ট্রোলার), `_error`, `_busy` স্টেটে রাখা।

`void dispose()`

`_days` কন্ট্রোলার `dispose` করে (override)।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে; `employeeId` না থাকলে এরর দেখিয়ে থামে। `_busy=true` করে `leaveRepository.updateBalance()` কল করে নতুন `LeaveBalanceRequest` (একই `employeeId/leaveType/year`, নতুন `totalDays`) সহ। সফল হলে `allBalancesProvider` ইনভ্যালিডেট করে শাট বন্ধ করে; ত্রুটিতে `_error` স্টেটে করে; `finally`-তে `_busy` রিসেটে করে।

`Widget build(BuildContext context)`

`FormSheetFrame` রিটার্ন করে — কর্মচারী/লভি টাইপ/বছরের সারসংক্ষেপে, দনিসংখ্যার `TextFormField` (নন-নগেটেভি পূরণসংখ্যা ভ্যালিডেটেরসহ), এবং ব্যবহৃত/পেন্ডিং দিনের তথ্যসূচক নোট (এগুলো এখানে এডটি করা যায় না তা উল্লেখসহ)।

ক্লাস/উইজটে: `_PoliciesTab`

`ConsumerWidget` — "নয়িম" ট্যাব: শুধুমাত্র সক্রিয় (in force) পলিসিগুলোর তালিকা। বন্ধ করা পলিসি এখনো থাকে কন্ট্রোলার গ্র্যান্ট করে না, আর এই ট্যাব দেখায় "মানুষ এখন কী পাওয়ার যোগ্য" — "কখনো লখো হয়েছে কী" নয় (সেটা HR নয়িম স্ক্রিনি থাকে)।

`Widget build(BuildContext context, WidgetRef ref)`

`activeLeavePoliciesProvider` ওয়াচ করে `ConfigList<LeavePolicy>` রিটার্ন করে; প্রতিটি আইটেমে লভি টাইপ ও বার্ষিকি এনটাইটলেমেন্ট দিন, এবং কর্মসংস্থান ধরন/paid-unpaid/অ্যাপ্রুভাল প্রয়োজন/ক্যারি-ফরওয়ার্ড/প্রয়োজ্য মাসের তথ্য বুলেট-বিন্দু আকারে দেখায়।

ফাইল: `lib/features/leave/grant_leave_balance_sheet.dart`

সংক্ষিপ্ত বিবরণ: HR-এর জন্য কোনো কর্মচারীকে নির্দিষ্ট বছরের জন্য ছুটির এনটাইটলেমেন্ট গ্র্যান্ট করার বটম শাট। এটি সিলেফ-সার্ভিস নয়, `LEAVE_BALANCE_CREATE` পারমিশন দ্বারা গটেডে — ছুটি অ্যাপ্রুভ করা এবং দিন গ্র্যান্ট করা আলাদা কাজ বলে আলাদা পারমিশন কোড।

টপ-লেভেলে ফাংশন: `Future<void> showGrantLeaveBalanceSheet(BuildContext context)`

`showModalBottomSheet` দিয়ে `_GrantLeaveBalanceSheet` প্রদর্শন করে।

ক্লাস/উইজটে: `_GrantLeaveBalanceSheet`

`ConsumerStatefulWidget` — ব্যালেন্স গ্র্যান্ট ফর্মের রুট উইজটে।

`ConsumerState<_GrantLeaveBalanceSheet> createState()`

`_GrantLeaveBalanceSheetState` রিটার্ন করে (override)।

ক্লাস: _GrantLeaveBalanceSheetState

_formKey, _days কন্ট্রোলার, _employee (নর্বিচাতি Person), _leaveType (ডিফল্ট allLeaveTypes.first), _year (ডিফল্ট বর্তমান বছর), _submitting, _error স্টেটে রাখা।

void dispose()

_days কন্ট্রোলার dispose করে (override)।

Future<void> _pickEmployee()

EmployeePicker.show() দিয়ে একজন কর্মচারী বেছে নেওয়ার UI দেখায়; নর্বিচাতি হলো _employee স্টেটে করে।

Future<void> _submit()

ইতিমধ্যে সাবমিট চলাকালীন হলে থামে (_submitting চকে)। ফর্ম ভ্যালিডেট করে; _employee না থাকলে এরর দেখিয়ে থামে। ফোকাস আনফোকাস করে _submitting=true স্টেটে করে

leaveRepository.createBalance() কল করে নতুন LeaveBalanceRequest (employeeId, leaveType, year, totalDays) সহ। সফল হলে leaveControllerProvider ইনভ্যালিডেট করে (যাতে ব্যক্তির নজিরে ব্যালেন্স ভিডি আপডেটে হয়), শিট বন্ধ করে সফলতার সন্ধ্যাকবার দেখায়। ApiException (যেমন "একই বছরে একই টাইপের ব্যালেন্স ইতিমধ্যে আছে" জাতীয় কনফ্লিক্ট বার্তা) সরাসরি দেখায়; সাধারণ ত্রুটিতে জনেরেকি বার্তা; finally-তে _submitting রিসেটে করে।

Widget build(BuildContext context)

Form-এ মোডানো SingleChildScrollView রটাইন করে (কীবোর্ড ইনস্টে-অনুসারী প্যাডিং সহ)। টাইটলে, এরর ব্যানার (থাকলে), কর্মচারী নর্বিচাচনরে InkWell+InputDecorator, লভি টাইপ ড্রপডাউন, বছর ড্রপডাউন (গত/এই/পরবর্তী বছর), মোট দিনের TextFormField (নন-নেগেটিভ পূর্ণসংখ্যা ভ্যালিডেটরসহ), এবং সাবমিট LoadingButton রেন্ডার করে।

done

ফাইল: lib/features/leave/leave_models.dart

সংক্ষিপ্ত ববিরণ: লভি (ছোট) মডউলরে ডটো মডলে, স্ট্যাটাস/পারমশিন কনস্ট্যান্ট এবং রকোয়স্ট/রসপন্স JSON ম্যাপিং এই ফাইলে সংজ্ঞায়িত। এখানে ছুটির আবদেন, রভিডি এবং ব্যালেন্স (entitlement) সম্ভব্রকতি সব ডটো টাইপ আছে।

টপ-লভেলে কনস্ট্যান্ট: leaveTypes

``dart const leaveTypes = <String>['ANNUAL', 'SICK', 'CASUAL', 'MATERNITY', 'PATERNITY', 'UNPAID', 'COMPENSATORY']` একজন কর্মী ছুটির জন্য আবদেন করার সময় যে ধরনগুলো বেছে নতি পারে তার তালিকা। ব্যাকএন্ডরে LeaveType` এনামরে চয়ে ইচ্ছাকৃতভাবে ছোট — EMERGENCY এবং OTHER ব্যাকএন্ডে থাকলেও এগুলো এমন কিছু নয় যা কডে ছুটি বুক করার সময় লস্ট থকে বেছে নেয়।

টপ-লভেলে কনস্ট্যান্ট: allLeaveTypes

``dart const allLeaveTypes = <String>[...leaveTypes, 'EMERGENCY', 'OTHER']` ব্যাকএন্ডরে জানা প্রতটি ধরন, যা HR-কে কারও বপিরীতে ব্যালেন্স কনফগার করার সময় ব্যবহার করা হয়। এমন ধরনের জন্যও ব্যালেন্স থাকতে পারে যা কডে নজিরে আবদেন করে না।

ক্লাস: LeaveStatus

abstract final class — একটি ছুটির আবদেন যার চারটি অবস্থার একটিতে থাকতে পারে তার স্ট্যাটিক কনস্ট্যান্ট ধারণ করে। - pending = 'PENDING' - approved = 'APPROVED' - rejected = 'REJECTED' - cancelled = 'CANCELLED' - all — উপরে চারটির একটি তালিকা।

ক্লাস: LeavePermissions

abstract final class — লভি মডেল সম্পর্কিত পারমিশন কোডগুলোর স্ট্যাটিক কনস্ট্যান্ট। - view = 'LEAVE_VIEW' — কডি পড়ার অনুমতি - approve = 'LEAVE_APPROVE' — একটি আবদেনের সিদ্ধান্ত নেওয়ার অনুমতি ব্যাকএন্ড শুধু এই কোডটাই PATCH /hr/leaves/{id}/review-তে চেক করে — অ্যাপ্রুভ এবং রজিক্ট দুটোই একই এন্ডপয়েন্ট দিয়ে যায়। তাই LEAVE_REJECT কাউকে কডিতে ঢুকতে দেবে কিন্তু সিদ্ধান্ত নতি দেবে না, যার কারণে স্ক্রিনি লস্ট এবং বাটনগুলোকে ভিন্ন কোডে গটে করে। - reject = 'LEAVE_REJECT' — শুধু আবদেন বাতিল করার অনুমতি থাকা রভিউয়ারদের কাছে থাকে। ওয়েব অ্যাপ অনুযায়ী কডি দেখার জন্য যথেষ্ট, কিন্তু রভিউ কল সফল হওয়ার জন্য যথেষ্ট নয়। - balanceCreate = 'LEAVE_BALANCE_CREATE' — কারও এনটাইটলেমেন্ট কনফিগার করার আলাদা কোড — ছুটিনিষে সিদ্ধান্ত নেওয়া আর দনি বরাদ্দ করা ভিন্ন কাজ। - balanceView = 'LEAVE_BALANCE_VIEW', balanceUpdate = 'LEAVE_BALANCE_UPDATE', balanceDelete = 'LEAVE_BALANCE_DELETE' — একটি এনটাইটলেমেন্ট পড়া, পরিবর্তন করা এবং মুছে ফেলার জন্য প্রতিটি নজিস্ব কোড, কারণ দনি বরাদ্দ করা আর একটি সিংখ্যা সংশোধন করা একই কাজ নয়।

ক্লাস: ReviewLeaveRequest

PATCH /hr/leaves/{id}/review এন্ডপয়েন্টের বডি তৈরি করে — একটি ছুটির আবদেন অ্যাপ্রুভ বা রজিক্ট করার পলিগোড।

ReviewLeaveRequest.approve()

কনস্ট নামযুক্ত কনস্ট্রাক্টর যা status কে LeaveStatus.approved এবং rejectionReason কে null সেট করে।

ReviewLeaveRequest.reject(String reason)

কনস্ট নামযুক্ত কনস্ট্রাক্টর যা status কে LeaveStatus.rejected এবং rejectionReason কে দেওয়া কারণে সেট করে। কারণ ঐচ্ছিক নয় — ব্যাকএন্ড খালি কারণ প্রত্যাখ্যান করে, এবং ব্যাখ্যাহীন প্রত্যাখ্যান কর্মীকে আবার আবদেন করা উচিত কিনা সে সম্পর্কে কিছু বলে না।

Map<String, dynamic> toJson()

status সবসময় অন্তর্ভুক্ত করে এবং rejectionReason তখনই অন্তর্ভুক্ত করে যখন এটি null নয় এবং ট্রিম করার পর খালি নয়। মানটি.trim() করে পাঠানো হয়।

ক্লাস: LeaveRequest

একটি একক ছুটির আবদেনের ডোমেন মডেল — ব্যক্তিগত তালিকা এবং রভিউয়ারের কডি উভয়ই এই টাইপ ব্যবহার করে।

ফিল্ডসমূহ: id, leaveType, startDate, endDate, totalDays (double), status, employeeId, employeeName (nullable — ব্যক্তিগত তালিকায় অনুপস্থিতি থাকে কারণ আপনজানেন আপনাকে, রভিউয়ারের কডিতে উপস্থিতি থাকে), reason, rejectionReason, reviewedAt, reviewedByName, createdAt।

`bool get canCancel`

`status == LeaveStatus.pending` হলে `true` রিটার্ন করে। শুধু একটি এমন আবেদন প্রত্যাহার করা যায় যার ওপর কটে এখনও কোনো সন্ধান নেই — একবার HR অ্যাপ্রুভ বা রজিক্ট করলে রেকর্ডটি তাদের হয়ে যায়।

`factory LeaveRequest.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে একটি `LeaveRequest` তৈরি করে। প্রতিটি সংখ্যা/স্ট্রিং ফিল্ডের জন্য নরিপদ ডিফল্ট আছে (যেমন `id` না থাকলে `0`, `status` না থাকলে `LeaveStatus.pending`), যাত অসম্পূর্ণ ডটোতেও পার্সিং ভঙে না পড়ে।

ক্লাস: LeaveRequestPayload

`POST /hr/leaves` এন্ডপয়েন্টে ছুটির আবেদন জমা দেওয়ার পলোড। ফিল্ডসমূহ: `leaveType`, `startDate`, `endDate`, `reason` (ঐচ্ছিক)।

`Map<String, dynamic> toJson()`

`leaveType`, `startDate`, `endDate` সবসময় অন্তর্ভুক্ত করে; `reason` ট্রিম করার পর খালি না হলেই অন্তর্ভুক্ত করে।

ক্লাস: LeaveBalance

একজন কর্মীর একটি নির্দিষ্ট বছরে একটি ছুটির ধরনের এনটাইটলেমেন্ট (কত দিন বরাদ্দ, কত ব্যবহৃত, কত অমীমাংসিত, কত বাকি) উপস্থাপন করে।

ফিল্ডসমূহ: `id`, `leaveType`, `year`, `entitledDays`, `usedDays`, `pendingDays`, `remainingDays`, `employeeId` (nullable), `employeeName` (nullable)।

`double get consumedFraction`

`entitledDays <= 0` হলে `0` রিটার্ন করে; নাহলে $(usedDays + pendingDays) / entitledDays$ হিসাব করে `0` ও `1`-এর মধ্যে ক্ল্যাম্প করে রিটার্ন করে — প্রোগ্রসে বারে ব্যবহারের জন্য।

`factory LeaveBalance.fromJson(Map<String, dynamic> json)`

JSON থেকে `LeaveBalance` তৈরি করে। `remainingDays` সারভার থেকে যতবে আসে ততবেই নেওয়া হয় $(\max(0, entitled - used - pending))$ হিসাবে সারভার-সাইডে গণনা করা — এখানে পুনরায় গণনা না করার কারণ হলো দুটো সংখ্যা যাত কখনো ভিন্ন না হয়ে যায় এবং কর্মী ও HR একই সংখ্যা দেখে। `year` না থাকলে বর্তমান বছর ডিফল্ট হিসাবে ব্যবহৃত হয়।

ক্লাস: LeaveBalanceRequest

`POST /hr/leave-balances` এন্ডপয়েন্টের পলোড — কাউকে একটি এনটাইটলেমেন্ট বরাদ্দ করার জন্য। চারটি ফিল্ডই ব্যাকএন্ডে `@NotNull` এবং এন্ডপয়েন্টে `@Valid`, তাই এখানে কোনো ঐচ্ছিক ফিল্ড নেই — একটি ব্যালেন্স মানে একজন কর্মী, একটি ধরন, একটি বছর ও একটি সংখ্যা, নয়তো কিছুই নয়। `totalDays` সারভার-সাইডে পূর্ণসংখ্যা (`Integer`), যখনে ব্যালেন্স ফেরত পড়ার সময় ভগ্নাংশযুক্ত হতে পারে।

ফিল্ডসমূহ: `employeeId`, `leaveType`, `year`, `totalDays` (`int`)।

`Map<String, dynamic> toJson()`

চারটি ফিল্ডকেই সরাসরি ম্যাপে রূপান্তর করে।

ফাইল: lib/features/leave/leave_repository.dart

সংক্ষিপ্ত ববিরণ: লভি মডউলরে নটেওয়ার্ক লয়োর (LeaveRepository) এবং তার ওপর ভিত্তিকরে গড়ে তোলা Riverpod কন্ট্রোলার/প্রোভাইডারগুলো — ব্যক্তিগিত ছুটির তালিকা, ব্যালেন্স, এবং রিভিউয়াররে কডি পরচালনা করে।

ক্লাস: LeaveRepository

ApiClient মুড়িয়ে লভি ও ব্যালেন্স সংক্রান্ত সব HTTP কল উন্মুক্ত করে।

LeaveRepository(this._api)

কনস্ট্রাক্টর — একটি ApiClient ইনস্ট্যান্স গ্রহণ করে।

Future<PagedResponse<LeaveRequest>> myRequests({int page = 0, int size = 20})

GET /hr/leaves/my কল করে সাইন-ইন করা ব্যবহারকারীর নজিরে ছুটির আবেদনগুলো পৃষ্ঠাবিন্যস্ত আকারে ফেরত দেয়।

Future<LeaveRequest> apply(LeaveRequestPayload payload)

POST /hr/leaves কল করে একটি নতুন ছুটির আবেদন জমা দেয় এবং তৈরি হওয়া LeaveRequest পার্স করে ফেরত দেয়।

Future<void> cancel(int id)

PATCH /hr/leaves/{id}/cancel কল করে একটি আবেদন প্রত্যাহার করে। রেসপন্স text/plain, তাই কিছু পার্স করা হয় না।

Future<PagedResponse<LeaveRequest>> allRequests({String? status, int page = 0, int size = 20})

GET /hr/leaves কল করে প্রতটি কর্মীর আবেদন ফেরত দেয় — এটি রিভিউয়াররে কডি। status কয়েরে প্যারামিটার দিয়ে ফিল্টার করা যায়।

Future<LeaveRequest> review(int id, ReviewLeaveRequest request)

PATCH /hr/leaves/{id}/review কল করে একটি আবেদন অ্যাপ্রুভ বা রজিক্ট করে। শুধু PENDING অবস্থার একটি আবেদনই রিভিউ করা যায় — ব্যাকএন্ড অন্য কিছু প্রত্যাখ্যান করে, তাই কলারকে সন্ধানত নেওয়া আবেদনের ওপর এটি অফার করা উচিত নয়।

Future<List<LeaveBalance>> employeeBalances(int employeeId, {int? year})

GET /hr/leaves/balances/employee/{employeeId} কল করে একজন নির্দিষ্ট কর্মীর ব্যালেন্স ফেরত দেয় — একটি আবেদন রিভিউ করার সময় প্রসঙ্গ হিসেবে ব্যবহারের জন্য। এই এন্ডপয়েন্টটি *leaves* কন্ট্রোলাররে অংশ, myBalances-এর মতো আলাদা leave-balances কন্ট্রোলাররে নয় — দুটি ভিন্ন এন্ডপয়েন্ট যাদের নাম ভিন্নান্তকিরভাবে একই রকম। ApiException ধরে isForbidden বা isNotFound হলে খালি তালিকা ফেরত দেয় (LEAVE_VIEW না থাকলেও কডি কাজ করে, শুধু প্রসঙ্গ ছাড়া), নাহলে ব্যতিক্রম পুনরায় ছুঁড়ে দেয়।

Future<List<LeaveBalance>> myBalances({int? year})

GET /hr/leave-balances/my কল করে কলাররে নজিরে ব্যালেন্স ফেরত দেয়। বনি-স্কোপ তালিকা এন্ডপয়েন্ট প্রতটি কর্মীর ব্যালেন্স ফেরত দেয়, যা একজন সাধারণ কর্মীর দখো উচিত নয়। যে কর্মীর জন্য HR এখনও ব্যালেন্স কনফিগার করেনি সে 403 বা খালি তালিকা পায়, এবং কোনোটাই দেখানোর মতো ত্রুটি নয় — ApiException ধরে isForbidden/isNotFound হলে খালি তালিকা ফেরত দেয়।

Future<LeaveBalance> createBalance(LeaveBalanceRequest request)

POST /hr/leave-balances কল করে কাউকে একটি বছরে জন্য এনটাইটলেমেন্ট বরাদ্দ করে। একই কর্মী, ধরন ও বছরে জন্য দ্বিতীয় ব্যালেন্স তৈরি করা ব্যাকএন্ড দ্বারা প্রত্যাখ্যাত হয় (মার্জ করা হয় না), তাই কলারের সাথে বার্তাটি সাধারণ ব্যর্থতার পরবর্ত্তে দেখানো উচিত।

Future<LeaveBalance> updateBalance(int id, LeaveBalanceRequest request)

PUT /hr/leave-balances/{id} কল করে একটি এনটাইটলেমেন্ট পরবর্ত্তন করে। এটি একটি সম্পূর্ণ প্রতিস্থাপন, প্যাচ নয় — যা পাঠানো হয় তা থেকেই ধরন, বছর ও দনিসংখ্যা নির্ধারিত হয়। শুধু কর্মী ফলিডিটি সুরক্ষিত, এবং স্টোও শুধু ভুলবশত অন্য ব্যক্তিতে পরবর্ত্তিত হওয়ার বন্দিধে। এটি এনটাইটলেমেন্ট সেট করে, বাকি কিত আছে তা নয় — ব্যবহৃত ও অমীমাংসিত দনিসংখ্যা আবেদন থেকে উদ্ভূত এবং এখানে সম্পাদনা করা যায় না।

Future<void> deleteBalance(int id)

DELETE /hr/leave-balances/{id} কল করে একটি ব্যালেন্স মুছে ফেলে।

Future<PagedResponse<LeaveBalance>> allBalances({int? year, int page = 0, int size = 50})

GET /hr/leave-balances কল করে একটি বছরে জন্য সবার ব্যালেন্স পৃষ্ঠাবিন্যস্ত আকারে ফেরত দেয়।

প্রোভাইডার: leaveRepositoryProvider

Provider<LeaveRepository> — apiClientProvider-এর ওপর নির্ভর করে একটি LeaveRepository ইনস্ট্যান্স তৈরি করে।

ক্লাস: LeaveState

@immutable — লভি স্ক্রিনিং ব্যক্তিগত-দৃশ্য (personal view) ধারণ করে: আবেদনসমূহ, ব্যালেন্স, পজেনিশন অবস্থা এবং কর্মী রেকর্ড আছে কনি।

ফলিডসমূহ: requests, balances, page, totalPages, loadingMore, hasEmployeeRecord।

bool get hasMore

page + 1 < totalPages — আরও পৃষ্ঠা লোড করার বাকি আছে কনি তা বলে।

double? get totalAvailable

balances.isEmpty হল null রিটার্ন করে (কোনো ব্যালেন্স কনফিগার করা নেই বোঝাতে — এটি একটি ড্যাশ, শূন্য নয়, কারণ শূন্য বোঝাতে "সব ব্যবহার হয়ে গেছে")। নাহলে সব ধরনের remainingDays যোগ করে মোট উপলব্ধ দনিসংখ্যা রিটার্ন করে।

LeaveState copyWith({...})

একটি নতুন LeaveState তৈরি করে, প্রতিটি প্র্যারামিটার null হলে বর্তমান মান বজায় রাখে।

ক্লাস: LeaveController

AsyncNotifier<LeaveState> — ব্যক্তিগত ছুটির তালিকা ও ব্যালেন্স লোড, রফ্রেশ, পজেনিশন এবং আবেদন/বাতিলের কাজ পরিচালনা করে।

Future<LeaveState> build()

currentUserProvider watch করে যাত সাইন-আউট করে অন্য কারও হিসাবে সাইন-ইন করলে এটি পুনর্নির্মিত হয় (এই প্রোভাইডার অটো-ডিসপোজ নয়, তাই এই watch ছাড়া এটি নতুন ব্যবহারকারীকে আগের ব্যক্তির ডটো দেখাতে থাকতো)। এরপর _load() কল করে।

`Future<LeaveState> _load()`

প্রথমত `myEmployeeProvider` থেকে কর্মী রেকর্ড আছে কিনা যাচাই করে — কারণ `/hr/leaves/my` অনুপস্থিতি কর্মী রেকর্ডের উত্তর দিয়ে 400 দিয়ে, খালি পৃষ্ঠা দিয়ে নয়; এটি না করলে একটি সম্পূর্ণ স্বাভাবিক পরিস্থিতি মূল ট্যাবে লাল এরর ব্যানারে পরণিত হবে। কর্মী না থাকলে `hasEmployeeRecord: false` সহ একটি খালি `LeaveState` রিটার্ন করে। থাকলে `myRequests()` ও `myBalances()` উভয় কল একসাথে শুরু করে, প্রতিটি ওপর `.ignore()` কল করে (কারণ কোনো একটি আগে ব্যর্থ হলে ফাংশনটি অন্যটির `await`-এ পৌঁছানোর আগেই রিটার্ন করতে পারে, এবং যে কলরে ওপর কডে আর অপেক্ষা করছে না তার নিজস্ব প্রত্যাখ্যান একটি ইন্ডলে-না-করা ব্যতক্রিম হিসেবে রিপোর্ট হতে পারে যেদণ্ডি কলার ইতিমধ্যে ব্যর্থতা দেখে ফলেছে), তারপর উভয়ের ফলাফল একত্র করে `LeaveState` তৈরি করে।

`Future<void> refresh()`

`AsyncValue.guard(_load)` ব্যবহার করে অবস্থা পুনরায় লোড করে।

`Future<void> loadMore()`

বর্তমান অবস্থা `null` হলে, বা আরও পৃষ্ঠা না থাকলে, বা ইতিমধ্যে লোড হচ্ছে হলে কিছু করে না। নাহলে `loadingMore: true` সেট করে পরবর্তী পৃষ্ঠা আনে, ফলাফলগুলো বর্ধমান তালিকার সাথে যুক্ত করে, এবং ব্যর্থ হলে `loadingMore` রিসিটে করে ব্যতক্রিম পুনরায় ছুঁড়ে দিয়ে।

`Future<void> apply(LeaveRequestPayload payload)`

`LeaveRepository.apply` কল করে, তারপর `refresh()` করে। সারভার `totalDays` (সপ্তাহান্ত ও ছুটির দিন বাদ দিয়ে) এবং প্রাথমিক অবস্থা নির্ধারণ করে, তাই নতুন সারটি স্থানীয়ভাবে ফর্ম থেকে একত্র না করে পুনরায় পড়া হয়।

`Future<void> cancel(int id)`

`LeaveRepository.cancel` কল করে, তারপর `refresh()` করে।

প্রোভাইডার: `leaveControllerProvider`

`AsyncNotifierProvider<LeaveController, LeaveState> |`

ক্লাস: `LeaveQueueFilterController`

`Notifier<String?>` — রিভিউয়ারের কডি কোন স্ট্যাটাসে ফিল্টার করা আছে তা ধারণ করে।

`String? build()`

`LeaveStatus.pending` রিটার্ন করে ডিফল্ট হিসেবে, কারণ পেন্ডিং হলো একমাত্র স্ট্যাটাস যার ওপর কডে কিছু করতে পারে।

`void set(String? status)`

নতুন স্ট্যাটাস বর্তমান অবস্থার সমান না হলে অবস্থা আপডেট করে।

প্রোভাইডার: `leaveQueueFilterProvider`

`NotifierProvider<LeaveQueueFilterController, String?> |`

ক্লাস: `LeaveQueueController`

`AsyncNotifier<PagedState<LeaveRequest>> with PagedLoader<LeaveRequest>` — যে কডে ছুটি রিভিউ করে তার জন্য সবার আবদেনের পৃষ্ঠাবিন্যস্ত কডি পরিচালনা করে।

`Future<PagedState<LeaveRequest>> build()`

`currentUserProvider` ও `leaveQueueFilterProvider` watch করে, তারপর `loadFirstPage()` কল করে (মক্সিনি থেকে)।

`Future<PagedResponse<LeaveRequest>> fetchPage(int page)`

`PagedLoader` মক্সিনিরে override — বর্তমান ফিল্টার স্ট্যাটাস সহ `LeaveRepository.allRequests` কল করে।

`Future<void> review(int id, ReviewLeaveRequest request)`

`LeaveRepository.review` কল করে একটি আবদেনের সন্ধান নেয়। ফিল্টার `pending` হলে, সন্ধান নেওয়া আবদেনটি এখন আর `pending` নয় বলে `removeItem` দিয়ে তালিকা থেকে সরিয়ে দেয়; নাহলে `refresh()` করে। এরপর `employeeBalancesProvider` অকার্যকর (`invalidate`) করে এবং `leaveControllerProvider`-এর `refresh()` কল করে, কারণ অ্যাপ্রুভ করলে দনিসংখ্যা কর্মীর `pending` ব্যালেন্স থেকে `used`-এ চলে যায়, ফলে ব্যালেন্সসমূহ (রভিউয়ার নিজের তালিকা দেখলে তারও) পুরনো হয়ে যায়।

পুরোভাইডার: `leaveQueueProvider`

`AsyncNotifierProvider<LeaveQueueController, PagedState<LeaveRequest>> |`

পুরোভাইডার: `employeeBalancesProvider`

`FutureProvider.autoDispose.family<List<LeaveBalance>, int>` — একজন কর্মীর ব্যালেন্স দেয়, একটি আবদেন বচিার করার জন্য প্রসঙ্গ হিসেবে। এমপ্লয়ি আইডি দিয়ে কী করা এবং অটো-ডিসপোজ করা হয়েছে যাতো রভিউয়ার বিভিন্ন মানুষের কডি দেখার সময় প্রতটিরি জন্য ডটো ধরে রাখা কোনো লাভ ছাড়াই ধীরে ধীরে মমের লিকি না হয়ে ওঠে।

ফাইল: `lib/features/leave/leave_approvals_tab.dart`

সংক্ষিপ্ত ববিরণ: রভিউয়ারের ছুটি অনুমোদন কডি UI — স্ট্যাটাস ফিল্টার, প্রতটি আবদেনের কার্ড, এবং অ্যাপ্রুভ/রজিক্ট করার সময় প্রদর্শতি বটম শটি। এই ট্যাবটি লিভি স্ক্রিনিরে ভতিরই থাকে, আলাদা অ্যাডমনি সেকশনে নয়, কারণ কারও ছুটি অনুমোদন করা এবং নিজেরে ছুটি বুক করা একই বিষয়।

ক্লাস/উইজটে: `LeaveApprovalsTab`

`ConsumerWidget` — পুরো রভিউয়ার কডি ট্যাব: একটি স্ট্যাটাস ফিল্টার স্ট্রপি এবং একটি পৃষ্ঠাবন্যিস্ত তালিকা।

`Widget build(BuildContext context, WidgetRef ref)`

`leaveQueueProvider` এর নোটিফায়ার পড়ে ও `leaveQueueFilterProvider` watch করে। `_StatusFilter` এবং একটি `PagedList<LeaveRequest>` ধারণকারী `Column` তরৈকিরে, যা রফ্রিশে/লোড-মোর হ্যান্ডল করে এবং ফিল্টার `pending` কনি অনুযায়ী ভিন্ন খালি-অবস্থা বার্তা দেখায়।

ক্লাস/উইজটে: `_StatusFilter`

`ConsumerWidget (private)` — পাঁচটি অনুভূমিক চপি (`pending`, `approved`, `rejected`, `cancelled`, `all`) দেখায় কডি ফিল্টার করার জন্য।

Widget build(BuildContext context, WidgetRef ref)

বর্তমান নর্বিচাতি ফলিটার watch করে একটি অনুভূমিক `ListView.separated` রেন্ডার করে; প্রতিটি চিপে ট্যাপ করলে `leaveQueueFilterProvider.notifier.set(status)` কল হয়। নর্বিচাতি চপি ব্র্যান্ড রঙে হাইলাইট হয়।

ক্লাস/উইজটে: `_QueueCard`

`ConsumerStatefulWidget` (private) — কডিতে একটি একক ছুটির আবদেন কার্ড আকারে দেখায়, অ্যাপ্রুভ/রজিক্ট অ্যাকশন সহ।

`_QueueCard({required this.request})`

কনস্ট্রাক্টর — একটি `LeaveRequest` গ্রহণ করে।

`ConsumerState<_QueueCard> createState()`

`_QueueCardState`-এর একটি ইনস্ট্যান্স তৈরি করে।

ক্লাস: `_QueueCardState`

`ConsumerState<_QueueCard>` (private) — কার্ডের বাস্তব UI ও ব্যবসায়িক যুক্তি (অ্যাপ্রুভ/রজিক্ট ডায়ালগ, বাটন) ধারণ করে; `_busy` ফ্ল্যাগ দিয়ে ইন-ফ্লাইট রিভিউ কল ট্র্যাক করে।

`Future<void> _decide({required bool approve})`

অ্যাপ্রুভের ক্ষেত্রে `_confirmApproval()` দেখায় ও নিশ্চিতকরণ পলে `ReviewLeaveRequest.approve()` তৈরি করে; রজিক্টের ক্ষেত্রে `_askRejectionReason()` দেখায় এবং একটি কারণ পলে

`ReviewLeaveRequest.reject(reason)` তৈরি করে। তারপর `_busy = true` সেট করে

`leaveQueueProvider.notifier.review(...)` কল করে; সফল হলে সন্মতিক্রম দেখায়, `ApiException` হলে তার বার্তা দেখায়, অন্য যেকোনো ব্যতিক্রমে একটি জিনেরেকি বার্তা দেখায়, এবং সবশেষে `_busy = false` করে।

`Future<bool?> _confirmApproval()`

একটি `showModalBottomSheet<bool>` দেখায় যাতো কর্মীর নাম, দনিসংখ্যা, ছুটির ধরন, তারিখ, এবং `_BalanceContext` (কর্মীর অবশিষ্ট ব্যালেন্স দেখানোর জন্য) থাকে, সাথে একটি "Approve" বাটন যা `Navigator.pop(sheetContext, true)` করে। এটিই সেই প্রশ্ন যা একজন রিভিউয়ার আসলে জিজ্ঞাসা করে — "এরা কি এই দিনগুলো নতি সক্ষম?" — এবং এটিই একমাত্র প্রসঙ্গ যা তালিকা নজি বহন করতে পারে না।

`Future<String?> _askRejectionReason()`

একটি `showModalBottomSheet<String>` দেখায় যাতো একটি বিহু-লাইন টেক্সট ফর্ম ফল্ড থাকে যেখানে কারণ লেখা যায় ("Reject" বাটনে ভ্যালিডেট করে খালি হলে এগোতে দেয় না — ব্যাকএন্ড খালিকারণ প্রত্যাখ্যান করে বলে এখানেই ধরে রাউন্ড ট্রপি ও কাঁচা এরর মেসেজে বাঁচানো হয়)। বৈধ হলে `Navigator.pop(sheetContext, controller.text.trim())` করে; শিট বন্ধ হলে `controller.dispose()` কল হয়।

`static String _dayLabel(double days)`

দনিসংখ্যা যদি পূর্ণসংখ্যা হয় তবে "N day(s)" ফরম্যাটে, নাহলে এক দশমিক স্থান পর্যন্ত "N.N days" ফরম্যাটে ফেরত দেয়।

Widget build(BuildContext context)

`permissionControllerProvider` watch করে `canDecide` নির্ধারণ করে (আবদেনটি pending কনি এবং ব্যবহারকারীর `LeavePermissions.approve` আছে কনি — দুটো ভিন্ন কোড ইচ্ছাকৃতভাবে, কারণ কডি দেখা ও তাতে কাজ করা ভিন্ন অধিকার, এবং ব্যাকএন্ড রভিউ এন্ডপয়েন্টে শুধুই `LEAVE_APPROVE` গ্রহণ করে যেকোনো সিদ্ধান্তের জন্য)। কর্মীর নাম, ধরন, দনিসংখ্যা, স্ট্যাটাস চপি, তারখি, কারণ (থাকলে), রজিকেশন কারণ ব্যানার (রজিক্টেডে হল), রভিউকারীর নাম (থাকলে), এবং `canDecide` হলো অ্যাপ্রুভ/রজিক্টেট বাটন সহ একটি `AppCard` তৈরি করে।

ক্লাস/উইজটে: `_BalanceContext`

`ConsumerWidget` (private) — একটি নির্দিষ্ট কর্মীর একটি নির্দিষ্ট ছুটির ধরনের জন্য অবশিষ্ট দনিসংখ্যা তথ্য প্যানলে দেখায়।

`_BalanceContext({required this.employeeId, required this.leaveType, required this.requestedDays})`

কনস্ট্রাক্টর।

Widget build(BuildContext context, WidgetRef ref)

`employeeBalancesProvider(employeeId)` watch করে। লোডিং অবস্থায় একটি `Loader`, এরর হলে খালি বক্স (অনুপস্থিতি প্রসঙ্গ সিদ্ধান্ত আটকানোর কারণ নয় — রভিউয়ার হয়তো অ্যাপরে সাহায্য ছাড়াই উত্তরটি জানেন) দেখায়। ডটো এলে মিলে যাওয়া `leaveType`-এর ব্যালেন্স খোঁজে; না পলে খালি বক্স দেখায়। পলে, অনুরোধকৃত দনিসংখ্যা অবশিষ্ট দনিসংখ্যার বশে কনি (`short`) দেখে সতর্কতা (`warning`) বা তথ্য (`info`) স্টাইলরে একটি বিবর্তা বক্স দেখায় — pending দনিসংখ্যা ইতিমধ্যে এই আবদেনটি অন্তর্ভুক্ত করে বলে অ্যাপ্রুভ করলে আবার বয়োগ হয় না, গুরুত্বপূর্ণ বিষয় হলো এন্টাইটলেমেন্ট এটি সম্পূর্ণভাবে কভার করে কনি।

`static String _n(double v)`

একটি দ্বিগুণ (`double`) সংখ্যাকে, পূর্ণসংখ্যা হলে পূর্ণসংখ্যা স্ট্রিং হিসেবে, নাহলে এক দশমিক স্থান সহ স্ট্রিং হিসেবে ফরম্যাট করে।

ফাইল: `lib/features/leave/leave_screen.dart`

সংক্ষিপ্ত বিবরণ: লভি ফচারের মূল স্ক্রিনি — তিনটি সম্ভাব্য ট্যাব (`My requests`, `Balances`, `To approve`) সহ একটি `TabBar/TabBarView` তৈরি করে এবং প্রসঙ্গ-নির্ভর ফ্লোটিং অ্যাকশন বাটন (আবদেন করা বা দনি বরাদ্দ করা) দেখায়।

ক্লাস/উইজটে: `LeaveScreen`

`ConsumerWidget` — লভি মডউলের প্রবশেদ্বার স্ক্রিনি, যা `DefaultTabController` দিয়ে ট্যাব পরিচালনা করে।

`LeaveScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর — ঐচ্ছকি `initialTabLabel` গ্রহণ করে, যা কোন ট্যাব দিয়ে শুরু হবে তা নির্ধারণ করে।

Widget build(BuildContext context, WidgetRef ref)

leaveControllerProvider ও permissionControllerProvider watch করে। isReviewer নির্ধারণ করে LeavePermissions.approve অথবা LeavePermissions.reject থাকা অনুযায়ী (যেকোনো একটি কীওড কাউকে কডি দেখতে দেয়, ওয়বে অ্যাপের সাথে মলি রয়েছে — বাটন দেখাবে কনি সটে প্রতিকার্ড ভিত্তিতে নির্ধারণিত হয়, কারণ রভিউ এন্ডপয়েন্ট নজি শুধু LEAVE_APPROVE গ্রহণ করে)। isReviewer হলে তৃতীয় ট্যাব "To approve" যুক্ত করে একটি ট্যাব তালিকা তৈরি করে (প্রতিটি ট্যাব নজি ডেটো লোড করে, যাতে ব্যক্তিগত তালিকা ব্যর্থ হলেও কডি ব্যর্থ না হয়)। canApply নির্ধারণ করে কর্মী রেকর্ড আছে কনি তার ওপর ভিত্তি করে (কর্মী রেকর্ড না থাকলে বাটন দেখানো হয় না, নথি করা হয় না, কারণ সার্ভার-সাইডে প্রত্যাখ্যাত হওয়া অনুরোধই একমাত্র উৎপাদিত ফলাফল হতো)। canGrant নির্ধারণ করে LeavePermissions.balanceCreate থাকা অনুযায়ী। AnimatedBuilder দিয়ে বর্তমান সক্রিয় ট্যাব ট্র্যাক করে "Balances" ট্যাবে থাকলে ও canGrant থাকলে "Grant days" FAB দেখায়, নাহলে canApply থাকলে "Apply" FAB দেখায়।

ক্লাস/উইজটে: _MyRequestsTab

ConsumerWidget (private) — সাইন-ইন করা ব্যবহারকারীর নজি আবদেনসমূহের ট্যাব।

Widget build(BuildContext context, WidgetRef ref)

leaveControllerProvider-কে .when() দিয়ে হ্যান্ডল করে — লোডিং, এরর (রিট্রাই সহ), এবং ডেটো এলে _Requests উইজটে দেখায়।

ক্লাস/উইজটে: _BalancesTab

ConsumerWidget (private) — কর্মীর ব্যালেন্সসমূহের ট্যাব।

Widget build(BuildContext context, WidgetRef ref)

leaveControllerProvider-কে .when() দিয়ে হ্যান্ডল করে — লোডিং, এরর, এবং ডেটো এলে _Balances উইজটে দেখায়।

ক্লাস/উইজটে: _Requests

ConsumerWidget (private) — আবদেনের তালিকা রেন্ডার করে, খালি অবস্থা, প্রত্যাহার নিশ্চিতকরণ, এবং লোড-মোর হ্যান্ডল করে।

_Requests({required this.state})

কনস্ট্রাক্টর — LeaveState গ্রহণ করে।

Future<void> _cancel(BuildContext context, WidgetRef ref, LeaveRequest request)

একটি AlertDialog দিয়ে প্রত্যাহার নিশ্চিত করে; নিশ্চিত হলে

leaveControllerProvider.notifier.cancel(request.id) কল করে ও সন্ধ্যাকবার দেখায়, ব্যর্থ হলে

ApiException-এর বার্তা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

আবদেন তালিকা খালি হলে, hasEmployeeRecord অনুযায়ী দুই ধরনের EmptyState (কোনো আবদেন নই বনাম কোনো কর্মী রেকর্ড নই) দেখায়। খালি হলে ListView.separated দেখায় যখন প্রতিটি _RequestCard এবং একটি "Load older requests" বাটন (আরও পৃষ্ঠা থাকলে) থাকে। RefreshIndicator দিয়ে পুল-টু-রিফ্রেশ সমর্থিত।

ক্লাস/উইজটে: _RequestCard

`StatelessWidget` (private) — একটি একক ব্যক্তিগত ছুটির আবদেনের কার্ড, ঐচ্ছকি "Withdraw" বাটন সহ।

`_RequestCard({required this.request, this.onCancel})`

কনস্ট্রাক্টর।

`Widget build(BuildContext context)`

ছুটির ধরন, স্ট্যাটাস চপি, তারখিসীমা ও দনিসংখ্যা, কারণ (থাকলে), রজিকেশন কারণ ব্যানার (রজিক্টেডে হলো), রভিডিকারী তথ্য (থাকলে), এবং `onCancel` নন-নাল হলো একটি "Withdraw" টেক্সট বাটন সহ একটি `AppCard` তৈরিকরে।

ক্লাস/উইজটে: _Balances

`ConsumerWidget` (private) — ব্যালেন্স তালিকা রেন্ডার করে, বা খালিহলে একটি `EmptyState`।

`_Balances({required this.balances})`

কনস্ট্রাক্টর।

`Widget build(BuildContext context, WidgetRef ref)`

`balances` খালিহলে "No balances configured" `EmptyState` দেখায়। নাহলে `RefreshIndicator` মোড়ানো `ListView.separated` দেখায় যখনে প্রতিটি `_BalanceCard` থাকে।

ক্লাস/উইজটে: _BalanceCard

`StatelessWidget` (private) — একটি একক ছুটির ধরনের ব্যালেন্স কার্ড, প্রোগ্রসে বার সহ।

`_BalanceCard({required this.balance})`

কনস্ট্রাক্টর।

`static String _n(double v)`

দ্বিগুণ (double) সংখ্যাকে পূর্ণসংখ্যা হলো পূর্ণসংখ্যা হিসেবে, নাহলে এক দশমিক স্থান সহ ফরম্যাট করে।

`Widget build(BuildContext context)`

ছুটির ধরন, বছর, বড় করে অবশিষ্ট দনিসংখ্যা, একটি `LinearProgressIndicator` (`balance.consumedFraction` মান দিয়ে), এবং তিনটি `_Figure` (Entitled, Used, Pending) সারসিহ একটি `AppCard` তৈরিকরে।

ক্লাস/উইজটে: _Figure

`StatelessWidget` (private) — একটি লেবেল/মান জোড়া দেখানো ছোট কলাম উইজটে।

`_Figure({required this.label, required this.value})`

কনস্ট্রাক্টর।

`Widget build(BuildContext context)`

মান (বেড়, বোল্ড) এবং লেবেল (ছোট, ম্লান) সহ একটি `Expanded` কলাম তৈরিকরে।

ফাইল: lib/features/payables/payables_models.dart

সংক্ষিপ্ত ববিরণ: পয়েবেলস (অ্যাকাউন্টস পয়েবেল) মডউলরে ডটো মডলে — ভন্ডের (বকিরতো) এবং ভন্ডের বলি সম্পর্কতি টাইপ, স্ট্যাটাস কনস্ট্যান্ট, পারমশিন কোড এবং JSON ম্যাপিং। একজন ভন্ডের হলো এমন কটে যার কাছ থেকে কোম্পানি কনে; একটি বলি হলো তাদরে থেকে একটি চালান, যা অনুমোদতি হয় এবং তারপর কসিততি পরশিোধ করা হয় যতক্ষণ না কিছু বাকি থাকে।

ক্লাস: PayablesPermissions

abstract final class — ভন্ডের ও বলি সম্পর্কতি পারমশিন কোডরে স্ট্যাটিকি কনস্ট্যান্ট। - vendorView = 'VENDOR_VIEW' - vendorCreate = 'VENDOR_CREATE' - vendorUpdate = 'VENDOR_UPDATE' — active/inactive টগলও কভার করে, কারণ সার্ভিসি উভয়রে জন্যই একই কোড চকে করে। - vendorDelete = 'VENDOR_DELETE' - billView = 'VENDOR_BILL_VIEW' - billCreate = 'VENDOR_BILL_CREATE' - billApprove = 'VENDOR_BILL_APPROVE' - billPayment = 'VENDOR_BILL_PAYMENT' — VENDOR_BILL_PAY নয়, এনামে যভোবে লখো আছে সভোবই। - billCancel = 'VENDOR_BILL_CANCEL'

ক্লাস: BillStatus

abstract final class — একটি বলিরে সম্ভাব্য অবস্থার স্ট্যাটিকি কনস্ট্যান্ট, ব্যাকএন্ডরে VendorBillStatus-এর প্রতফিলন। - draft = 'DRAFT', approved = 'APPROVED', partiallyPaid = 'PARTIALLY_PAID', overdue = 'OVERDUE', paid = 'PAID', cancelled = 'CANCELLED' - all — উপরে ছয়টি তালিকা। - settled = {paid, cancelled} — এই দুটি একটিতে থাকলে আর কিছু বাকি নই। - payableFrom = {approved, partiallyPaid, overdue} — এই তিনটি অবস্থায় একটি পমেন্ট রকের্ড করা যায়; OVERDUE ও PARTIALLY_PAID উভয়ই একটি অনুমোদতি বলি যে অবস্থার মধ্য দিয়ে যায়, তাই তিনটিই যোগ্য — সার্ভিসি "approved"-কে শথিলিভাবে চকে করে, শুধু আক্সরকি DRAFT/APPROVED জোড়া নয়।

ক্লাস: Vendor

একজন বকিরতার ডোমইন মডলে।

ফল্ডিসমূহ: id, name, active, contactPerson, email, phone, taxId, address, paymentTerms, notes, outstandingBalance (nullable — সব বলি জুড়ে তাকে যা এখনও দেওয়া বাকি)।

bool get owesMoney

(outstandingBalance ?? 0) > 0 রটার্ন করে।

factory Vendor.fromJson(Map<String, dynamic> json)

JSON থেকে Vendor তরৈকিরে, প্রতটি ফল্ডরে জন্য নরিপদ ডফিল্ট সহ (active না থাকলে true)।

ক্লাস: VendorRequest

POST ও PUT /company/finance/vendors-এর পলোড। name @NotBlank; বাকিগুলো দরৈঘ্য সীমা সহ ঐচ্ছকি — contact ও email ১৫০, phone ৫০, tax id ও payment terms ১০০, address ৫০০। এই সীমাগুলো একটি 400 থেকে আবশ্কার না করে ফর্মই প্রয়োগ করা হয়।

ফল্ডিসমূহ: name, contactPerson, email, phone, taxId, address, paymentTerms, notes।

factory VendorRequest.from(Vendor vendor)

একটি বিদ্যমান Vendor থেকে তার সব ফল্ড কপিকিরে একটি VendorRequest তরৈকিরে (এডিটি ফর্ম প্র-ফিলি করার জন্য উপযোগী)।

Map<String, dynamic> toJson()

name.trim() সহ একটি ম্যাপ তৈরি করে; বাকি প্রতিটি ফিল্ড একটি অভ্যন্তরীণ clean() হেল্পার দিয়ে ট্রিম করে, ফাঁকা হলে null করে পাঠায়। ফিল্ডগুলো স্পষ্ট null হিসেবে পাঠানো হয়, বাদ দেওয়া হয় না — কারণ আপডেট রকিওয়েস্ট থেকে প্রতিটি ফিল্ড অ্যাসাইন করে, তাই অনুপস্থিতি কী যত্নেই হোক সঠিকভাবে দেয়, আর স্পষ্ট থাকা মান কী বোঝানো হচ্ছে তা বলা।

ক্লাস: VendorBill

একটি ভেন্ডরের একটি বিলের ডোমেন মডেল।

ফিল্ডসমূহ: id, billNumber, status, subtotal, taxAmount, totalAmount, paidAmount, balanceAmount, vendorId (nullable), vendorName, vendorReference, billDate, dueDate, description, expenseAccountName, createdBy, approvedBy।

bool get isSettled

BillStatus.settled.contains(status) রিটার্ন করে।

bool get canApprove

status == BillStatus.draft রিটার্ন করে।

bool get canPay

BillStatus.payableFrom.contains(status) && balanceAmount > 0 রিটার্ন করে।

bool get canCancel

status != BillStatus.cancelled && paidAmount <= 0 রিটার্ন করে। কোনো কিছু পরিশোধ হয়ে গেলে বাতলিকরণ প্রত্যাখ্যাত হয় — ব্যাকএন্ড এমনটাই বলে, এবং স্ক্রিনিও জিজ্ঞাসা করার আগে সটাই বলে।

double? get paidShare

totalAmount <= 0 হলে null রিটার্ন করে; নাহলে (paidAmount / totalAmount) কে 0.0 ও 1.0-এর মধ্যে ক্ল্যাম্প করে রিটার্ন করে — কতটুকু নষ্পত্তি হয়েছে তা বোঝাতে।

factory VendorBill.fromJson(Map<String, dynamic> json)

JSON থেকে VendorBill তৈরি করে, প্রতিটি সংখ্যা/স্ট্রিং ফিল্ডের জন্য নরিপদ ডিফল্ট সহ (status না থাকলে BillStatus.draft)।

ক্লাস: VendorBillRequest

POST /company/finance/vendor-bills-এর পলোড। চারটি @NotNull ফিল্ড — vendor, উভয় তারিখ, এবং subtotal — এবং subtotal @DecimalMin("0.01"), তাই শূন্য মূল্যের বিল প্রত্যাখ্যাত হয়। কোনো আপডেট এন্ডপয়েন্ট নেই — একটি বিল প্রবশে করানো হয়, অন্য কডে অনুমোদন করে, এবং পরিশোধ করা হয়; ভুল একটি বাতলি করে পুনরায় প্রবশে করানো হয়, যা লজোর ইতিমধ্যে রেকর্ড হওয়া কছির জন্য সঠিক পদ্ধতি। এক্সপেন্স অ্যাকাউন্টটি অবশ্যই expense-টাইপ অ্যাকাউন্ট হতে হবে — সার্বভিসি অন্য যেকোনো কিছু প্রত্যাখ্যান করে অ্যাকাউন্ট ও তার টাইপ উল্লেখ করে একটি বার্তা সহ।

ফিল্ডসমূহ: vendorId, billDate (yyyy-MM-dd), dueDate, subtotal, taxAmount (ডিফল্ট 0), vendorReference, description, expenseAccountId।

double get total

subtotal + taxAmount রিটার্ন করে।

Map<String, dynamic> toJson()

vendorId, billDate, dueDate, subtotal, taxAmount সবসময় অন্তর্ভুক্ত করে; vendorReference, description ট্রিম করার পর অ-খালিহলে অন্তর্ভুক্ত করে; expenseAccountId নন-নাল হলে অন্তর্ভুক্ত করে।

ফাইল: lib/features/payables/payables_repository.dart

সংক্ষিপ্ত ববিরণ: অ্যাকাউন্টস পয়েবেল মডউিলরে নটেওয়ার্ক লয়োর (PayablesRepository) এবং তার ওপর ভিত্তি করে ভেন্ডর ও বিলি তালকির জন্য Riverpod কন্ট্রোলার/প্রোভাইডারগুলো।

ক্লাস: PayablesRepository

ApiClient মুড়িয়ে ভেন্ডর ও বিলি সংক্রান্ত সব HTTP কল উন্মুক্ত করে।

PayablesRepository(this._api)

কনস্ট্রাক্টর — একটি ApiClient ইনস্ট্যান্স গ্রহণ করে।

Future<PagedResponse<Vendor>> vendors({String? search, int page = 0, int size = 50})

GET /company/finance/vendors কল করে ভেন্ডররে পৃষ্ঠাবিন্যস্ত তালিকা ফরেত দিয়ে, ঐচ্ছকি search কোয়েরি প্যারামটার সহ (খালি না হলেই পাঠানো হয়)।

Future<List<Vendor>> activeVendors()

GET /company/finance/vendors/active কল করে যারা এখনও ব্যবসা করছে তাদের একটি বিয়োর তালিকা ফরেত দিয়ে — একটি বিলি ফর্ম যা থেকে বাছাই করে।

Future<Vendor> createVendor(VendorRequest request)

POST /company/finance/vendors কল করে একটি নতুন ভেন্ডর তৈরি করে।

Future<Vendor> updateVendor(int id, VendorRequest request)

PUT /company/finance/vendors/{id} কল করে একটি ভিন্ডর আপডেট করে।

Future<Vendor> toggleVendor(int id)

PATCH /company/finance/vendors/{id}/toggle কল করে ভেন্ডররে active/inactive অবস্থা টগল করে।

Future<void> deleteVendor(int id)

DELETE /company/finance/vendors/{id} কল করে একটি ভিন্ডর মুছে ফলে।

Future<PagedResponse<VendorBill>> bills({String? status, int? vendorId, int page = 0, int size = 30})

GET /company/finance/vendor-bills কল করে বিলির পৃষ্ঠাবিন্যস্ত তালিকা ফরেত দিয়ে। উভয় ফিল্টার (status, vendorId) এখানে কোয়েরি প্যারামটার হিসেবে পাঠানো হয় — ফাইন্যান্স মডউিলরে বশেরিভাগ জায়গার বপিরীতে যখনে সেগুলো পাথ সেগমেন্ট, এবং অস্বাভাবিকভাবে উভয়ই একসাথে প্রযোজ্য হতে পারে।

Future<VendorBill> createBill(VendorBillRequest request)

POST /company/finance/vendor-bills কল করে একটি নতুন বিলি তৈরি করে।

Future<VendorBill> approveBill(int id)

POST /company/finance/vendor-bills/{id}/approve কল করে বিলটি সিইসাবুদ করে। এটি অবশ্যই যবে ব্যবহারকারী বিলটি প্রবশে করযিছে তর থকে ভন্নি একজন ব্যবহারকারী হতে হবে — জার্নাল এন্ট্রি যভোবে কর্তব্য পৃথকীকরণ (segregation of duties) প্রয়োগ করে সেই একই রকম, একই ধরনের বার্তা সহ: "You entered this bill - a different user must approve it"।

Future<VendorBill> payBill(int id, double amount)

POST /company/finance/vendor-bills/{id}/pay?amount=\$amount কল করে বলিরে বপিরীতে একটি পমেন্টে রকেৰ্ড করে। পরমাণটি একটি কৌয়েরে প্যারামটার, এবং অবশ্যিট ব্যালেন্স অতক্ৰম করলে তা প্রত্যাখ্যাত হয় — বার্তায় ব্যালেন্স নাম উল্লেখ থাকে, তাই এটি যিমন লখে তমেনই দেখানোর যোগ্য।

Future<VendorBill> cancelBill(int id)

POST /company/finance/vendor-bills/{id}/cancel কল করে একটি বিলি বাতলি করে। কোনো কিছু পরশিোধ হয়ে গেলে এটি প্রত্যাখ্যাত হয়।

Future<Ageing> apAgeing({String? asOfDate})

GET /company/finance/vendor-bills/ap-ageing কল করে কতটা বকযো, কত দরেতিে বভিক্ত করে ফরেত দিয়ে। রসিভিবেলস এজিং রপিোর্টরে মতোই একই আকৃত ব্যবহার করে, তাই সেই মডলেই পুনরায় ব্যবহার করা হয় — লাইন ফলিডগুলো শুধু ইনভয়সে/ক্লায়েন্টরে পরবির্তে বলি/ভন্ডের সম্পর্কে হওয়াতে আলাদা, যা Ageing উভয়ভাবেই পড়তে পারে।

প্রোভাইডার: payablesRepositoryProvider

Provider<PayablesRepository> — apiClientProvider-এর ওপর নির্ভর করে একটি PayablesRepository ইনস্ট্যান্স তৈরিকরে।

ক্লাস: VendorSearchController

Notifier<String> — টাইপ করা ভন্ডের অনুসন্ধান শব্দ ধারণ করে, যাতে তালকি কন্ট্রোলার এর ওপর ভিত্তি করে পুনর্নির্মিত হতে পারে টেক্সট ফলিড রকৌয়েস্টেরে মালকিনা না নযিে।

String build()

একটি খালি স্ট্রিং রটার্ন করে ডফিল্ট হিসেবে।

void set(String value)

মানটি ট্রমি করে; ট্রমি করা মান বর্তমান অবস্থার সমান না হলে অবস্থা আপডেট করে।

প্রোভাইডার: vendorSearchProvider

NotifierProvider<VendorSearchController, String>।

ক্লাস: VendorsController

AsyncNotifier<List<Vendor>> — অনুসন্ধান শব্দ অনুযায়ী ফলিটার করা ভন্ডের তালকি পরচালনা করে।

Future<List<Vendor>> build()

currentUserProvider ও vendorSearchProvider watch করে, _load() কল করে।

`Future<List<Vendor>> _load()`

`vendorSearchProvider`-এর বর্তমান মান পড়ে, খালি হিলে `null` হিসেবে পাঠায়, এবং `PayablesRepository.vendors` কল করে ফলাফলকে `content` রিটার্ন করে।

`Future<void> refresh()`

`AsyncValue.guard(_load)` দিয়ে তালিকা পুনরায় লোড করে।

`void apply(Vendor updated)`

বর্তমান তালিকায় থাকা একই `id`-র ভেন্ডরকে `updated` দিয়ে প্রতিস্থাপন করে স্থানীয়ভাবে অবস্থা আপডেট করে (নতুন করে সার্ভার কল না করে)।

`void remove(int id)`

বর্তমান তালিকা থেকে দেওয়া `id`-র ভেন্ডরটি সরিয়ে অবস্থা আপডেট করে।

প্রোভাইডার: `vendorsProvider`

`AsyncNotifierProvider<VendorsController, List<Vendor>> |`

প্রোভাইডার: `activeVendorsProvider`

`FutureProvider<List<Vendor>>` — যাদের বপিরীতে একটি নতুন বলি তোলা যায় তাদের ভেন্ডর তালিকা।

ক্লাস: `BillStatusFilterController`

`Notifier<String?>` — বলির তালিকা কোন স্ট্যাটাসে সংকুচিত তা ধারণ করে; `null` মানে সব।

`String? build()`

`null` রিটার্ন করে ডিফল্ট হিসেবে।

`void set(String? status)`

নতুন স্ট্যাটাস বর্তমান অবস্থার সমান না হলে অবস্থা আপডেট করে।

প্রোভাইডার: `billStatusFilterProvider`

`NotifierProvider<BillStatusFilterController, String?> |`

ক্লাস: `BillsController`

`AsyncNotifier<List<VendorBill>>` — ফিল্টার করা বলি তালিকা পর্যালোচনা করে।

`Future<List<VendorBill>> build()`

`currentUserProvider` ও `billStatusFilterProvider` watch করে, `_load()` কল করে।

`Future<List<VendorBill>> _load()`

`billStatusFilterProvider`-এর বর্তমান মান দিয়ে `PayablesRepository.bills` কল করে ফলাফলকে `content` রিটার্ন করে।

`Future<void> refresh()`

`AsyncValue.guard(_load)` দিয়ে তালিকা পুনরায় লোড করে।

`void apply(VendorBill updated)`

বর্তমান তালিকায় থাকা একই `id`-র বলিকে `updated` দিয়ে প্রতিস্থাপন করে স্থানীয়ভাবে অবস্থা আপডেট করে।

প্রোভাইডার: billsProvider

AsyncNotifierProvider<BillsController, List<VendorBill>> |

প্রোভাইডার: apAgeingProvider

FutureProvider.autoDispose<Ageing> — আজকের হিসেবে কী বক্যো আছে তা দিয়ে।

done

ফাইল: lib/features/payables/payables_screen.dart

সংক্ষিপ্ত বিবরণ: কোম্পানির কাছে কত টাকা পাওনা (Accounts Payable) দেখানোর স্ক্রিনি। এতে তিনটি ট্যাব থাকে — Bills (ভেন্ডার বিল), Ageing (বক্যোর বয়স রিপোর্ট) এবং Vendors (ভেন্ডার তালিকা), প্রতিটি পারমিশন অনুযায়ী দেখানো বা লুকানো হয়।

ক্লাস/উইজটে: PayablesScreen

ConsumerWidget, যটি পারমিশন অনুযায়ী উপলব্ধ ট্যাবগুলোর তালিকা তৈরি করে, প্রতিটি ট্যাবের জন্য একটি ট্রিচ্চকি "create" অ্যাকশন সেট করে (Floating Action Button-এর মাধ্যমে), এবং DefaultTabController দিয়ে ট্যাবগুলো দেখায়।

PayablesScreen({super.key, this.initialTabLabel})

কনস্ট্রাক্টর। initialTabLabel দিয়ে নির্দিষ্ট নামের ট্যাব দিয়ে স্ক্রিনি শুরু করা যায় (যেমন অন্য স্ক্রিনি থেকে সরাসরি "Vendors" ট্যাবে নিয়ে আসা)।

Widget build(BuildContext context, WidgetRef ref)

প্রথমে পারমিশন লোড না হলে Loader দেখায়। এরপর PayablesPermissions.billView/vendorView অনুযায়ী ট্যাব-তালিকা (label, view, create-কলব্যাক) তৈরি করে। কোনো ট্যাব উপলব্ধ না হলে "Not available to you" বার্তাসহ EmptyState দেখায়। initialTabLabel অনুযায়ী প্রাথমিক ইনডেক্স নির্ণয় করে, তারপর DefaultTabController ও AnimatedBuilder দিয়ে সলিকেটেড ট্যাবের create কলব্যাক থাকলে একটি FloatingActionButton.extended ("New") দেখায়, আর TabBar/TabBarView দিয়ে ট্যাবগুলো রেন্ডার করে।

ক্লাস/উইজটে: _BillsTab

ConsumerStatefulWidget — ভেন্ডার বিলের তালিকা দেখায়, approve/pay/cancel অ্যাকশন সহ।

ক্লাস/উইজটে: _BillsTabState

বিল-সংক্রান্ত অ্যাকশনগুলোর ব্যস্ততা (_busyId) ট্র্যাক করে এবং সংশ্লিষ্ট রিপোর্টের কিল চালায়।

Future<void> _act(VendorBill bill, Future<VendorBill> Function(PayablesRepository repo) action, String success, String failure)

একটি সাধারণ হেল্পার যা _busyId সেট করে, দেওয়া action চালায়, ফলাফল দিয়ে billsProvider আপডেট করে, apAgeingProvider ইনভ্যালিডেট করে ও ভেন্ডার তালিকা রিফ্রেশ করে (কারণ পমেন্ট/অ্যাপ্রুভাল উভয়ই কী বক্যো তা বদলে দেয়), সাফল্যে/ব্যর্থতায় SnackBar দেখায়। ApiException-এর বার্তা যেমন আছে তেমনই দেখানো হয় কারণ সেগুলো অর্থবহ (যেমন অন্য ব্যবহারকারীর অনুমোদন প্রয়োজন)।

Future<void> _pay(VendorBill bill)

askForPayment ডায়ালগ দেখিয়ে পমেন্ট পরমাণ নয়ে, তারপর _act দিয়ে repo.payBill কল করে।

`Future<void> _cancel(VendorBill bill)`

`confirmAction` দিয়ে নশ্চিতিকরণ নিয়ে (বিলি বাতলি হলো তা আর ফেরত আনা যায় না বলে সতর্ক করে), তারপর `_act` দিয়ে `repo.cancelBill` কল করে।

`Widget build(BuildContext context)`

পারমশিন ও ফিল্টার (`billStatusFilterProvider`) পড়ে, `ConfigList<VendorBill>` দিয়ে বিলির তালিকা দেখায়, একটা `FilterBar` হডোর দিয়ে স্ট্যাটাস অনুযায়ী ফিল্টার করার সুযোগ দেয়, এবং প্রতিটি বিলির জন্য `_BillCard` তৈরিকরে যেখানে `canApprove/canPay/canCancel` ও সংশ্লিষ্ট পারমশিন অনুযায়ী কলব্যাক পাস করা হয়।

ক্লাস/উইজটে: `_BillCard`

`StatelessWidget` — একটা বিলির কার্ড দেখায়: ভেন্ডরের নাম, বিলি নম্বর/রফোরেন্স/ডিউ-ডেট, মোট পরিমাণ, স্ট্যাটাস চপি, এবং পেমেন্ট প্রগ্রসে বার (কতটা পরিশোধ হয়েছে)। ব্যস্ত থাকলে স্পিনার, নাহলে অ্যাকশন মেনু (`PopupMenuButton`) দেখায়।

`Widget build(BuildContext context)`

`onApprove/onPay/onCancel` অনুযায়ী `RowAction` তালিকা তৈরিকরে একটা `PopupMenuButton` বসায়। বিলি আংশিক পরিশোধিত হলে (`paidAmount > 0`) একটা `LinearProgressIndicator` ও "কত পরিশোধ, কত বাকি" টেক্সট দেখায়।

ক্লাস/উইজটে: `_AgeingTab`

`ConsumerWidget` — Accounts Payable ageing রিপোর্ট (`apAgeingProvider`) লোড করে `RefreshIndicator`-এ মোড়া এবং লোড/এরর/ডেটো অবস্থা অনুযায়ী `_Ageing` উইজটে দেখায়।

`Widget build(BuildContext context, WidgetRef ref)`

`apAgeingProvider` watch করে, রফ্রেশে সটে ইনভ্যালিডিটে করে, `async.when` দিয়ে `Loader/ErrorState/_Ageing` রেন্ডার করে।

ক্লাস/উইজটে: `_Ageing`

`StatelessWidget` — মোট বক্যো, ওভারডিউ পরিমাণ, বয়স-বাকটে (`bucket`) ভিত্তিক ব্রকেডাউন এবং প্রতিটি বক্যো বিলির লাইন-আইটেমে দেখায়।

`Widget build(BuildContext context)`

দুটা `StatCard` (`Owed out, Overdue` — ওভারডিউ থাকলে `bos.danger` টোন) দেখায়, তারপর প্রতিটি `bucket`-এর জন্য লবেলে, পরিমাণ ও একটা `LinearProgressIndicator` (Current বাকটে ব্র্যান্ড রঙে, বাকিগুলো `danger` রঙে) দেখায়। `report.lines` খালি না হলে প্রতিটি বিলির কাউন্টারপার্টি, রফোরেন্স/ডিউ-ডেট/দনি-লটে ও ব্যালেন্স দেখায় (ওভারডিউ হলে `danger` রঙে)। শেষে "As at" তারিখ দেখায়।

ক্লাস/উইজটে: `_VendorsTab`

`ConsumerStatefulWidget` — ভেন্ডরদের তালিকা (সার্চসহ) দেখায়।

ক্লাস/উইজটে: `_VendorsTabState`

টগল/ডিলিট অ্যাকশনের ব্যস্ততা ট্র্যাক করে।

`Future<void> _toggle(Vendor vendor)`

`repo.toggleVendor` কল করে ভেন্ডরকে `active/inactive` স্ট্যাটাস পাল্টায়, `vendorsProvider` আপডেট করে ও `activeVendorsProvider` ইনভ্যালিডেট করে। ব্যর্থ হলে এরর দেখায়।

`Future<void> _delete(Vendor vendor)`

`confirmAction` দিয়ে নিশ্চিতকরণ নেয় — ভেন্ডরকে কাছাকাছি বকসে থাকলে বিশেষ সতর্কবার্তা দেখায় (রেটিয়ার করার পরামর্শ দিয়ে), তারপর `repo.deleteVendor` কল করে, তালিকা থেকে সরায় ও `activeVendorsProvider` ইনভ্যালিডেট করে।

`Widget build(BuildContext context)`

`AppSearchField` দিয়ে সার্চ ইনপুট নেয় (`vendorSearchProvider`-এ সেট করে) — এটি ইচ্ছাকৃতভাবে `ConfigList`-এর হডোরের বাইরে রাখা, কারণ খালি ফলাফলে সার্চবক্সটি হিরিয়ে যাওয়া উচিত নয়। তারপর `ConfigList<Vendor>` দিয়ে ভেন্ডর তালিকা দেখায়, প্রতিটি জন্য `ConfigRow` তৈরি করে যাতে নাম, একটি স্ট্যাটাস, যোগাযোগ তথ্য, বকসে থাকলে তার পরিমাণ এবং এডটি/টগল/রমিউ অ্যাকশন (পারমিশন অনুযায়ী) দেখানো হয়।

ফাইল: `lib/features/payables/payables_sheets.dart`

সংক্ষিপ্ত বিবরণ: ভেন্ডর তৈরি/সম্পাদনা, নতুন বিল তৈরি এবং একটি বিলের বিপরীতে পেমেন্ট রেকর্ড করার জন্য বটম-শিট/ডায়ালগ ফর্মগুলো এই ফাইলে আছে।

টপ-লেভেলে ফাংশন: `Future<void> showVendorSheet(BuildContext context, {Vendor? existing})`

ভেন্ডর তৈরি বা সম্পাদনার জন্য `_VendorSheet` মডাল বটম-শিট খোলবে। `existing` দেওয়া থাকলে এডটি মোড, না থাকলে নতুন ভেন্ডর তৈরি মোড।

ক্লাস/উইজটে: `_VendorSheet`

`ConsumerStatefulWidget` — নাম, যোগাযোগ ব্যক্তি, ইমেইল, ফোন, ট্যাক্স আইডি, ঠিকানা, পেমেন্ট শর্ত ও নোট দেওয়ার ফর্ম।

ক্লাস/উইজটে: `_VendorSheetState`

ফর্মে টেক্সট কন্ট্রোলারগুলো রাখা ও সাবমিট লজিক চালায়।

`bool get _isEdit`

`widget.existing` নন-নাল হলে সত্য — এডটি মোডে আছে কিনা তা বলে।

`void dispose()`

সব `TextEditingController` (name, contact, email, phone, taxId, address, terms, notes) ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেশন পাস করলে, `VendorRequest` তৈরি করে — এডটি মোডে `repo.updateVendor` কল করে ও `vendorsProvider`-এ ফলাফল অ্যাপ্লাই করে; নতুন হলে `repo.createVendor` কল করে ও তালিকা রিফ্রেশ করে। উভয় ক্ষেত্রে `activeVendorsProvider` ইনভ্যালিডেট করে, শিট বন্ধ করে ও সাফল্য বার্তা দেখায়। এরর হলে `_error` সেট করে ফর্মে দেখায়।

String? Function(String?) _max(int limit)

একটি ভ্যালিডেটর-জেনারেটর — ট্রিম করা ইনপুটের দৈর্ঘ্য `limit`-এর বেশি হলে এরর মैसेজ দেয়।
ব্যাকএন্ডের ফিল্ড-লংথ লিমিটের সাথে মিলিয়ে আগভাগে ভ্যালিডেট করে, যাত 400 এরর এড়ানো যায়।

Widget build(BuildContext context)

`FormSheetFrame`-এর ভেতরে নাম (required), যোগাযোগ ব্যক্তি, ইমেইল, ফোন+ট্যাক্স-আইডি (এক সারতি), পমেন্ট শর্ত, ঠিকানা ও নোট ফিল্ড সাজায়, প্রতিটিতে যথাযথ `_max` ভ্যালিডেটর যুক্ত করে।

টপ-লেভেলে ফাংশন: Future<void> showBillSheet(BuildContext context)

নতুন ভেন্ডর বলি তৈরি জন্য `_BillSheet` মডাল বটম-শিট খোলে।

ক্লাস/উইজটে: `_BillSheet`

`ConsumerStatefulWidget` — ভেন্ডর নির্বাচন, বলির তারিখ/ডাউ-ডেট, সাবটোটাল/ট্যাক্স, খরচ-অ্যাকাউন্ট, রফোরেন্স ও বিবরণ নেওয়ার ফর্ম।

ক্লাস/উইজটে: `_BillSheetState`

ফর্মের কন্ট্রোলার ও নির্বাচতি ভেন্ডর/অ্যাকাউন্ট/তারিখ স্টেটে রাখে।

void dispose()

`_subtotal`, `_tax`, `_reference`, `_description` কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ভেন্ডর নির্বাচতি না থাকলে বা তারিখ না থাকলে এরর দেখিয়ে থামে। অন্যথায় `VendorBillRequest` তৈরি করে `repo.createBill` কল করে, `billsProvider` রিফ্রেশ করে ও `apAgeingProvider` ইনভ্যালিডেট করে, শিট বন্ধ করে এবং "Somebody else has to approve it" বার্তা দেখায়। এরর হলে যথাযথ বার্তা দেখায় (যেমন non-expense অ্যাকাউন্টের ক্ষেত্রে ব্যাকএন্ডের নামসহ এরর)।

Widget build(BuildContext context)

সক্রিয় ভেন্ডর ও শুধুমাত্র `EXPENSE` টাইপের অ্যাকাউন্ট কারণ বলি শুধু খরচ-অ্যাকাউন্টের বিপরীতে পোস্ট করা যায়) লোড করে ড্রপডাউনে দেখায়। বলি/ডাউ ডেটের জন্য দুটি `DateField`, পরমাণ/ট্যাক্সের জন্য দুটি `TextFormField` (পরমাণ শূন্যের বেশি হতে হবে, ট্যাক্স শূন্য বা বেশি) দেখায়, subtotal থাকলে মোট দেখায়। খরচ-অ্যাকাউন্ট ঐচ্ছকি ড্রপডাউনে, তারপর রফোরেন্স ও বিবরণ ফিল্ড, শেষে একটি তথ্য-ব্যানার যে বলি একবার এন্টার করার পর এডিট করা যায় না — ভুল হলে বাতলি করে পুনরায় এন্টার করতে হবে।

টপ-লেভেলে ফাংশন: Future<double?> askForPayment(BuildContext context, {required VendorBill bill})

একটি বলির বিপরীতে পমেন্ট নেওয়ার ডায়ালগ দেখায়। ডিফল্টভাবে পুরো বক্যো ব্যালেন্স পূরণ করা থাকে (যেহেতু সাধারণত পুরোটাই পরিশোধ হয়) এবং ব্যালেন্সের বেশি ইনপুট করা হলে ভ্যালিডেশন আটকায (ব্যাকএন্ডও একই ন্যম মানবে, ব্যালেন্স নাম উল্লেখ করে)। ইনপুট বন্ধ হলে ব্যবহারকারীর দেওয়া `double` পরমাণ রটার্ন করে, বাতলি করলে `null`।

ফাইল: lib/features/payroll/payroll_models.dart

সংক্ষিপ্ত ববিরণ: মাসকি পরেোল রান, ড্যাশবোর্ড, স্টেটিংস ও বাল্ক-জেনারেটে ফলাফলরে জন্য মডলে ও ধ্রুবক সংজ্ঞায়িত করে। কর্মচারীর নজিরে প-স্লিপের অংশ payslips মডুলে আলাদাভাবে থাকে; এই ফাইলটি প্রশাসকের দিকটি — যে রান প-স্লিপ তৈরিকরে এবং যে স্টেটিংস হিসাবরে নয়িম ঠকি করে।

enum/ধ্রুবক শ্রেণি: RunStatus

abstract final class — PayrollRun.RunStatus-এর মরির। পরেোল রান কোন কোন স্ট্যাটাসে থাকতে পারে (draft, calculated, pendingApproval, approved, paid, rejected, cancelled) এবং কোন অ্যাকশন কোন স্ট্যাটাস থেকে সম্ভব তা সংজ্ঞায়িত করে। settled স্টেটে paid/cancelled আছে — এগুলোয় পৌঁছালে রানের আর কিছু ঘটবে না। recalculateFrom, submitFrom, approveFrom, rejectFrom, cancelFrom, payFrom — প্রতিটি ব্যাকএন্ডরে PayrollRunService.requireStatus-এর নয়িম হুবহু কপি, যাতে স্ক্রিনি শুধু সেই অ্যাকশনগুলোই দেখায় যগুলো আসলে কাজ করবে।

টপ-লেভেলে ধ্রুবক: perDayBases

(value, label, hint) রেকর্ডরে তালিকা — মাসকি বতেন থেকে একদিনরে বতেন কীভাবে বের করা হয় (CALENDAR_DAYS, FIXED_30, FIXED_26, ACTUAL_WORKING_DAYS) তার মরির, প্রতিটির সাথে ব্যাখ্যামূলক হিন্টি।

টপ-লেভেলে ধ্রুবক: salaryBases

['BASIC', 'GROSS'] — কোন বতেন-ভিত্তিতে শতাংশ/কর্তন গণনা হবে তার মরির (SalaryBase)।

ক্লাস/উইজটে: PayrollRun

একটি মাসরে পুরো কোম্পানির পরেোল রান প্রতিনিধিত্ব করে — রান নম্বর, মাস/বছর, স্ট্যাটাস, মোট কর্মচারী, গ্রস/কর্তন/নেটে এবং তারখিসংক্রান্ত ঐচ্ছকি ফল্ড।

PayrollRun({...})

কনস্ট্রাক্টর — সব প্রয়োজনীয় ও ঐচ্ছকি ফল্ড নিয়ে।

bool get isSettled

status যদি RunStatus.settled-এর মধ্যে থাকে (paid বা cancelled) তবে সত্য।

bool get hasLines

totalEmployees > 0 কনি — লাইন-বহীন রান সাবমিট করা যায় না, তাই ব্যাকএন্ডরে এরর মেসেজেরে আগাই স্ক্রিনি এটি জানায়।

bool can(Set<String> from)

বর্তমান status দেওয়া স্টেটে আছে কনি যাচাই করে — কোনো অ্যাকশন বর্তমান অবস্থায় সম্ভব কনি তা যাচাইয়ের সাধারণ পদ্ধতি।

factory PayrollRun.fromJson(Map<String, dynamic> json)

JSON থেকে PayrollRun তৈরিকরে; সব সংখ্যাসূচক ফল্ডে null-safe ফলব্যাক আছে।

ক্লাস/উইজটে: PayrollDashboard

GET /hr/payroll-dashboard-এর উত্তর — একটি মাসরে সারসংক্ষেপে ও একটি ছোট ট্রেন্ড, যেখানে শূন্য একটি বৈধ মান (রান-না-হওয়া মাসকে কোনো লাইন থাকে না)।

PayrollDashboard({...})

কনস্ট্রাক্টর — মাস, বছর, মোট কর্মচারী, পরোল-কাউন্ট, পরিশোধিত-কর্মচারী, পেন্ডিং-কাউন্ট, গ্রস/নেট/কর্তন এবং ঐচ্ছিক রান-স্ট্যাটাস/নম্বর/পরবর্তী-পে-ডেটে ও ট্রেন্ড নিয়ে।

double? get paidShare

`payrollCount <= 0` হলে `null` (পরিশোধ করার কিছু নেই বোঝাতে — শূন্য-বার হিসেবে ভুল বোঝানো এড়াতে), নাহলে `employeesPaid / payrollCount` ক্ল্যাম্প করা 0.0–1.0 রঞ্জে।

int get missing

`totalEmployees` থেকে `payrollCount` বাদ দিয়ে বের করে এমন লোক যাদের এই মাসে কোনো পরোল লাইনই নেই — এটি `pending` থেকে আলাদা, কারণ `pending` মানে লাইন আছে কিন্তু অপরিশোধিত।

factory PayrollDashboard.fromJson(Map<String, dynamic> json)

JSON থেকে ড্যাশবোর্ড তৈরি করে, `trend` লিস্টের প্রতিটি আইটেমকে `PayrollTrendPoint.fromJson` দিয়ে পার্স করে।

ক্লাস/উইজটে: PayrollTrendPoint

একটি মাস/বছরে নেট-পরিশোধিত পরিমাণ ধারণ করে।

PayrollTrendPoint({required this.month, required this.year, required this.netPaid})

কনস্ট্রাক্টর।

factory PayrollTrendPoint.fromJson(Map<String, dynamic> json)

JSON থেকে তৈরি করে, null-safe ফলব্যাক সহ।

ক্লাস/উইজটে: PayrollSettings

পরোল হিসাব করার সময় যসেব নয়িম অনুসরণ করা হয় তা ধারণ করে — `perDayBasis`, `absenceDeductionBase`, ওভারটাইম নয়িম ও বিভিন্ন ভাতা/কর্তনের শতাংশ।

PayrollSettings({...})

কনস্ট্রাক্টর — সব ফিল্ড আবশ্যিক।

double get allowanceTotal

`houseRent`, `medical`, `transport`, `food` শতাংশের যোগফল — এগুলো মূল বতেনের ভাগ এবং একসাথে ১০০-এর কম হওয়ার কথা, তবে ব্যাকএন্ডে মোট যাচাই করে না, তাই ফর্মের এটি শুধু সতর্কবার্তা হিসেবে ব্যবহৃত হয়।

factory PayrollSettings.fromJson(Map<String, dynamic> json)

JSON থেকে স্টেটিস তৈরি করে, প্রতিটি সংখ্যাসূচক ফিল্ডের জন্য একটি লোকাল `num_` হলেপার ব্যবহার করে ডিফল্ট মান সহ পার্স করে।

ক্লাস/উইজটে: PayrollSettingsRequest

`PUT /hr/payroll-settings`-এর জন্য রকিওয়েস্ট বডি — কোডবসের বরিল একটি আপডেট যা সম্পূর্ণভাবে null-guarded (ব্যাকএন্ডে প্রতিটি ফিল্ড এসাইনের আগে চেক করা হয়), তবে ফর্ম সবসময় সব ফিল্ড পাঠায় কারণ ফর্মের সব ফিল্ড দৃশ্যমান।

PayrollSettingsRequest({...})

কনস্ট্রাক্টর — `PayrollSettings`-এর মতো একই ফিল্ডসেটে নিয়ে।

Map<String, dynamic> toJson()

সব ফিল্ডকে একটি Map-এ রূপান্তর করে API-তে পাঠানোর জন্য।

ক্লাস/উইজটে: BulkPayrollResult

POST /hr/payroll/generate-এর ফলাফল — তনিট নামের তালিকা: যাদের জন্য নতুন লাইন তৈরি হয়েছে, যাদের আগে থেকেই ছিল, এবং যাদের বতেন-কাঠামো নই বলে বাদ পড়ছে। শেষেটিসবচেয়ে গুরুত্বপূর্ণ কারণ এই মানুষগুলো এই মাসে বতেন পাবে না।

BulkPayrollResult({this.created = const [], this.alreadyExisted = const [], this.noSalaryStructure = const []})

কনস্ট্রাক্টর — ডিফল্ট খালি তালিকা সহ।

bool get hasProblems

noSalaryStructure তালিকা খালি না হলে সত্য — যার মান নেওয়া কঠিন কিছু আছে।

String get summary

সব তালিকা খালি হলে "কিছু জনের টে করার নই" বার্তা দেয়; নাহলে created/alreadyExisted/noSalaryStructure-এর সংখ্যা জুড়ে একটি এক-লাইন সারাংশ তৈরি করে।

factory BulkPayrollResult.fromJson(Map<String, dynamic> json)

JSON থেকে তৈরি করে, created, skippedAlreadyExists, skippedNoSalaryStructure কী থেকে নামের তালিকা বের করে (একটি লোকাল names হলে পার দিবে)।

ফাইল: lib/features/payroll/payroll_repository.dart

সংক্ষিপ্ত বিবরণ: পরেোল সংক্রান্ত সব API কল এখানে — রান (মাসিক ব্যাচ) এবং লাইন (একজন কর্মচারীর এন্ট্রি) উভয়ের জন্য। এতে ড্যাশবোর্ড, স্টেটিংস, স্যালারি শিট এবং এক্সপোর্ট (CSV/PDF) সংক্রান্ত ফাংশনও আছে, পাশাপাশি Riverpod প্রোভাইডার/কন্ট্রোলার।

ক্লাস/উইজটে: PayrollRepository

পরেোল-সংক্রান্ত সব HTTP কলরে একক শ্রেণি রান-এন্ডপয়েন্ট /hr/payroll-runs-এ এবং লাইন-এন্ডপয়েন্ট /hr/payroll-এ থাকবে; উভয়ই একই ফাইলে কারণ স্ক্রিনি নিয়মিত দুটোর মধ্যে চলাচল করে।

PayrollRepository(this._api)

কনস্ট্রাক্টর — একটি ApiClient নেয়।

Future<List<PayrollRun>> runs()

সব পরেোল রানের বয়ের তালিকা আনে (পেজে নয়, কারণ প্রতি মাসে একটিই রান এবং ইতিহাস ছোট)।

Future<PayrollRun> createRun(int month, int year, String? remarks)

নির্দিষ্ট মাস/বছরে জন্য একটি রান খোলে; একই পরিষিড়ে আগের থেকে রান থাকলে ব্যাকএন্ড তা নাম-সহ প্রত্যাখ্যান করে। remarks ট্রিম করে খালি না হলে পাঠায়।

Future<PayrollRun> recalculate(int id)

_runAction ব্যবহার করে রান পুনঃগণনার অ্যাকশন কল করে।

`Future<PayrollRun> submit(int id)`

`_runAction` দিয়ে রান অনুমোদনের জন্য সাবমিট করে।

`Future<PayrollRun> approve(int id)`

`_runAction` দিয়ে রান অনুমোদন করে।

`Future<PayrollRun> cancel(int id)`

`_runAction` দিয়ে রান বাতলি করে।

`Future<PayrollRun> reject(int id, String reason)`

রান প্রত্যাখ্যান করে — `reason` একটি রিকোয়েস্ট বডি হিসেবে যায় (এই কন্ট্রোলার `@RequestBody RejectRequest(String reason)` নিয়ে, অ্যাপরে অন্যান্য রজিক্ট-এন্ডপয়েন্টের মতো `@RequestParam` নয়)।

`Future<PayrollRun> pay(int id, {required String paymentMethod, String? referencePrefix, String? paymentDate})`

রানের সব অপারেশিওন লাইন পরিশোধ করে। `referencePrefix` ঐচ্ছিক — না দিলে ব্যাকএন্ড রান-নম্বর ব্যবহার করে ও প্রতিলি লাইনে তিনি-অঙ্করে সক্রিয়নেস যোগ করে।

`Future<PayrollRun> _runAction(int id, String action)`

প্রাইভেট হলেপার — `$_runs/$id/$action` পাথে POST করে ও `PayrollRun` রিটার্ন করে; `recalculate/submit/approve/cancel` সবাই এটি ব্যবহার করে।

`Future<PagedResponse<Payslip>> forPeriod(int month, int year, {int page = 0, int size = 50})`

একটি নির্দিষ্ট মাসের সবার পরের লাইন পেজ-ভিত্তিকভাবে আনবে।

`Future<BulkPayrollResult> generate(int month, int year)`

বতেন-কাঠামো থাকা প্রতিলি কর্মচারীর জন্য একটি ড্রাফট লাইন তৈরি করে (বাল্ক-জেনারেট)।

`month/year` উভয়ই কয়ে-প্যারামিটার হিসেবে যায়; কাঠামো-বহীনদরে বাদ দিয়ে নাম-সহ ফলাফলে জানানো হয়।

`Future<Payslip> approveLine(int id)`

একটি পরের লাইন অনুমোদন করে (PATCH)।

`Future<Payslip> payLine(int id, {String? paymentReference, String? paymentMethod})`

একটি লাইনকে পরিশোধিত হিসেবে চিহ্নিত করে। দুটি প্যারামিটারই ঐচ্ছিক কয়ে-প্যারামিটার হিসেবে ম্যানুয়ালি এনকোড করে জোড়া হয়।

`Future<void> deleteLine(int id)`

একটি পরের লাইন মুছে দেয়। এই এন্ডপয়েন্ট `ResponseEntity<String>` ঘোষণা করে (`deleteText`) — অর্থাৎ JSON নয়, একটি বাক্য উত্তর দেয়।

`Future<String> disbursementCsv(int month, int year)`

নির্দিষ্ট মাসের ব্যাংক ডিসবার্সমেন্ট ফাইল আনবে — এন্ডপয়েন্ট `text/csv` ঘোষণা করে, তাই এটি বাইট হিসেবে ফেচ করে অস্থায়ী ফাইলে সেভ করা হয়, `_download` হলেপার ব্যবহার করে।

Future<SalarySheet> salarySheet(int month, int year)

একটি মাসের পুরো কোম্পানির বর্তমান সরাসরি (live) হিসাব করে আনে — এটিকোনো নির্দিষ্ট রান নয়, বরং বর্তমান কার্যক্রম ও উপস্থিতির উপর ভিত্তি করে তখনকার মুহূর্তের হিসাব; যে সারণিতে payrollId আছে সেটি ইতিমধ্যে রান হয়েছে, বাকিগুলো এখনো প্রক্ষেপণ (projection) মাত্র।

Future<String> salarySheetPdf(int month, int year)

_download ব্যবহার করে মাসের স্যালারি-শিট PDF হিসেবে নামিয়ে ফাইল-পাথ রিটার্ন করে।

Future<String> _download(String path, int month, int year, String name, String extension)

প্রাইভেট হলেপার — api.getBytes দিয়ে বাইট আনে, getTemporaryDirectory()-এ একটি নামযুক্ত ফাইল লখে (নাম-বছর-মাস.এক্সটেনশন) ও পাথ রিটার্ন করে — একই প্যাটার্ন যা পে-স্লিপি ডাউনলোডও ব্যবহার করে।

Future<PayrollDashboard> dashboard({int? month, int? year})

একটি মাসের সারসংক্ষেপে আনে; month/year না দিলে ব্যাকএন্ড বর্তমান মাসের জন্য উত্তর দেয়।

Future<PayrollSettings> settings()

বর্তমান পেরোল সেটিংস আনে।

Future<PayrollSettings> updateSettings(PayrollSettingsRequest request)

সেটিংস আপডেট করে (PUT) — ব্যাকএন্ডে সব ফল্ড null-guarded, তাই কী বাদ দিলে তা অপরিবর্তিত থাকে; কনফর্ম সবসময় সব ফল্ড পাঠায়।

টপ-লেভেলে প্রোভাইডার: payrollRepositoryProvider

Provider<PayrollRepository> — apiClientProvider থেকে PayrollRepository ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: PayrollPeriodController

Notifier<({int month, int year})> — ড্যাশবোর্ড, রান ও লাইন — সব পেরোল স্ক্রিনি কোন মাস দেখছে তা কন্ট্রোলিংভাবে নিয়ন্ত্রণ করে, যাতে মাস পাল্টালে তিনিটাই একসাথে বদলায়।

(({int month, int year}) build()

বর্তমান তারখির মাস/বছর দিয়ে প্রাথমিক স্টেটে তৈরি করে।

void set(int month, int year)

নতুন মাস/বছর একই হলে কিছু করে না, নাহলে স্টেটে আপডেট করে।

void shift(int months)

DateTime ব্যবহার করে একবারে একমাস (বা যেকোনো নির্দিষ্ট সংখ্যক মাস) এগিয়ে/পছিয়ে যায়, বছর ওভারফ্লো নজি নজি সামলায়।

টপ-লেভেলে প্রোভাইডার: payrollPeriodProvider

NotifierProvider<PayrollPeriodController, ({int month, int year})> |

ক্লাস/উইজটে: PayrollRunsController

AsyncNotifier<List<PayrollRun>> — সব পেরোল রানের তালিকা লোড ও ব্যবস্থাপনা করে।

`Future<List<PayrollRun>> build()`

`currentUserProvider` watch করে (রিলিগইনে রফ্রিশে হওয়ার জন্য) ও `repo.runs()` থেকে তালিকা লোড করে।

`Future<void> refresh()`

`AsyncValue.guard` দিয়ে তালিকা পুনরায় লোড করে স্টেটে আপডেট করে।

`void apply(PayrollRun updated)`

বর্তমান তালিকায় থাকা একটি রানকে নতুন `updated` মান দিয়ে প্রতস্থাপন করে (আইডমিলিটি);
বর্তমান তালিকা `null` হলে কিছু করে না।

টপ-লেভেলে প্রোভাইডার: `payrollRunsProvider`

`AsyncNotifierProvider<PayrollRunsController, List<PayrollRun>> |`

টপ-লেভেলে প্রোভাইডার: `payrollDashboardProvider`

`FutureProvider.autoDispose<PayrollDashboard>` — বর্তমান নির্বাচতি পরিষিডরে
(`payrollPeriodProvider` watch করে) ড্যাশবোর্ড আনে।

টপ-লেভেলে প্রোভাইডার: `payrollLinesProvider`

`FutureProvider.autoDispose<List<Payslip>>` — নির্বাচতি পরিষিডরে সবার পরেোল লাইন আনে,
`forPeriod` কল করে `page.content` রিটার্ন করে।

টপ-লেভেলে প্রোভাইডার: `payrollSettingsProvider`

`FutureProvider<PayrollSettings>` — `currentUserProvider` watch করে এবং `repo.settings()` কল করে
পরেোল সটেংস আনে।

ফাইল: `lib/features/payroll/payroll_screen.dart`

সংক্ষিপ্ত ববিরণ: পরেোল চালানোর মূল স্ক্রিনি — একটি একক স্ক্রিনি (ট্যাব-বার নয়, কারণ পরেোল
একটি ধারাবাহিক প্রক্রিয়া): মাস নির্বাচন করুন, বর্তমান অবস্থা দেখুন, লাইন জনোরটে করুন,
অনুমোদনের জন্য পাঠান, পরিশোধ করুন। উপরে মাস-পিকার নচি্রে সবকিছু নিয়ন্ত্রণ করে।

ক্লাস/উইজেটে: `PayrollScreen`

`ConsumerWidget` — পারমিশন-চকে, মাস-পিকার, মাসের ড্যাশবোর্ড সারাংশ, রান-সেকশন ও
লাইন-সেকশন একত্রে দেখায়। অ্যাপবারে স্যালার-শিটি, এক্সপোর্ট-মেনু ও (process পারমিশন থাকলে)
পরেোল-সটেংস আইকন থাকে।

`Widget build(BuildContext context, WidgetRef ref)`

পারমিশন লোড না হলে `Loader` দেখায়। `view/process/approve` — কোনোটিই না থাকলে "Not available
to you" বার্তা দেখায় (নজিরে পে-স্লিপি `Payslips` স্ক্রিনি আছে বলে জানায়)। নির্বাচতি পরিষিড ও
ড্যাশবোর্ড watch করে, `RefreshIndicator`-এ `_MonthPicker`, ড্যাশবোর্ড ডটো (`_Month`), `_RunSection`
ও `_LinesSection` সাজায়।

ক্লাস/উইজটে: _ExportMenu

ConsumerStatefulWidget — পরেোল য়ে দুটি ফাইল তরৈকিরে (ব্যাংক ডিসিবার্সমেন্ট CSV ও মাসরে স্যালার-শিট PDF) তার এক্সপোর্ট মেনু। উভয়ই বাইট আকারে আসে বলে অস্থায়ী ফাইলে লেখা হয় ও ডভাইসে কনো অ্যাপ দয়ি থোলা হয় — পে-স্লপি ডাউনলোডরে মতোই।

ক্লাস/উইজটে: _ExportMenuState

এক্সপোর্ট-বাটনরে ব্য়স্ততা (**_busy**) ট্র্যাক করে।

Future<void> _open(Future<String> Function(PayrollRepository repo, int month, int year) fetch, String failure)

দেওয়া **fetch** ফাংশন দয়ি ফাইল-পাথ আনে, **OpenFile.open** দয়ি থোলার চেষ্টা করে; ব্য়র্থ হলে "ডভাইসে কনো অ্যাপ খুলতে পারছে না" বার্তা দেখায়, **ApiException** বা অন্য এরর হলে যথাযথ বার্তা দেখায়।

Widget build(BuildContext context)

ব্য়স্ত থাকলে স্পনিার, নাহলে **PopupMenuButton** দেখায় যখনে "Salary sheet (PDF)" ও "Bank disbursement (CSV)" অপশন থাকে, নরিবাচনে **_open** কল করে যথাক্রমে **salarySheetPdf/disbursementCsv**।

ক্লাস/উইজটে: _MonthPicker

ConsumerWidget — বর্তমান দেখা মাস দেখায়, একমাস পছিনে/সামনে সরার বাটন সহ; বর্তমান মাস অতক্রিম করে সামনে যাওয়া নষিদ্ধ (কারণ ভবষ্যতরে মাসরে পরেোল অস্তুতিবহীন)।

Widget build(BuildContext context, WidgetRef ref)

period বনাম আজকরে তারখি তুলনা করে **atLatest** নরিণয় করে, দুটি **IconButton** (আগরে/পররে মাস) ও মাঝে মাস-বছরে টেক্সট দেখায়; পররে-মাস বাটন সর্বশেষে মাসে থাকলে নষিক্রয়ি থাকে।

ক্লাস/উইজটে: _Month

StatelessWidget — একটি মাসরে ড্যাশবোর্ড সারাংশ (নেটে, রানে থাকা কর্মচারী সংখ্যা, গ্রস, কর্তন, পরশিোধ-প্রগ্রসে ও মাসকি ট্রেন্ড) দেখায়।

Widget build(BuildContext context)

দুটি সিরতি চারটি **StatCard** (Net to pay, On the run — মসিং কর্মচারী থাকলে সতর্ক-টোন, Gross, Deductions) দেখায়। **paidShare** নন-নাল হলে একটি প্রগ্রসে কার্ড দেখায় (পরশিোধতি সংখ্যা, পরবর্তী-পে-ডেটে, প্রগ্রসেবার এবং মসিং কর্মচারী থাকলে সতর্কবার্তা)। **dashboard.trend**-এর দর্ঘ্ঘ ১-এর বশেইলে **_Trend** উইজটে দেখায়।

ক্লাস/উইজটে: _Trend

StatelessWidget — গত কয়কে মাসরে নেটে-পরশিোধতি পরমাণ বার-চার্ট আকারে দেখায় (লাইন-চার্ট নয়, কারণ পয়ন্টে সংখ্যা কম এবং সরু স্ক্রিনি বার বশে পাঠযোগ্য)।

Widget build(BuildContext context)

সব পয়ন্টরে মধ্যে সর্বোচ্চ মান (**peak**) বরে করে, প্রতিটি পয়ন্টরে জন্য একটি **Container** বার আঁকে (শূন্য মাসেও একটি ছোট স্লভির দেখায়, যাত "শূন্য পরশিোধ" আর "মাসই নহে" গুলিয়ে না যায়) এবং নচি তনি-অক্ষরে মাসরে নাম দেখায়। উচ্চতা **scaledStripHeight** দয়ি টেক্সট-স্কেলিং অনুযায়ী মাপা হয়।

ক্লাস/উইজটে: `_RunSection`

`ConsumerWidget` — নির্বাচতি মাসের রান খুঁজে বের করে এবং তা অনুযায়ী "কোনো রান নই" কার্ড বা `_RunCard` দেখায়।

Widget build(BuildContext context, WidgetRef ref)

`payrollRunsProvider`-এর তালিকা থেকে `payMonth/payYear` মিলিয়ে বর্তমান রান খুঁজে বের করে। লোড হচ্ছে ও ডটো নই হলো `Loader` দেখায়। রান না থাকলে ব্যাখ্যা-টেক্সট ও (process পারমিশন থাকলে) "Open a run" বাটন দেখায় যা `showOpenRunSheet` খোলো। রান থাকলে `_RunCard` রেন্ডার করে।

ক্লাস/উইজটে: `_RunCard`

`ConsumerStatefulWidget` — একটি নির্দিষ্ট রানের বস্টিতারতি ও তার উপর সম্ভাব্য অ্যাকশন (recalculate, submit, approve, reject, pay, cancel) দেখায়।

ক্লাস/উইজটে: `_RunCardState`

রান-অ্যাকশনের ব্যস্ততা (`_busy`) ট্র্যাক করে।

Future<void> `_act`(Future<PayrollRun> Function(PayrollRepository repo) action, String success, String failure)

`action` চালিয়ে ফলাফল দিয়ে `payrollRunsProvider`-এ `apply` করে, ড্যাশবোর্ড ও লাইন প্রোভাইডার ইনভ্যালিডিটে করে (কারণ টোটাল ও পরিশোধ-সংখ্যা উভয়ই রানের সাথে বদলায়), সাফল্য/ব্যর্থতা `SnackBar`-এ দেখায়। `ApiException`-এর মসেজে যখন আছে তখনই দেখানো হয় (যখন "Cannot submit a PAID payroll run")।

Widget build(BuildContext context)

রান নম্বর, স্ট্যাটাস-চপি, কর্মচারী-সংখ্যা/নটে/পরিশোধ-তারিখ, রমির্কস (থাকলে), রজিকেশন-কারণ (রজিক্টেডে হলো) এবং লাইন-না-থাকলে হন্ট-টেক্সট দেখায়। ব্যস্ত থাকলে স্পিনার, নাহলে `Wrap` দিয়ে পারমিশন ও `run.can(...)` অনুযায়ী উপলব্ধ অ্যাকশন বাটনগুলো (`_RunAction`) সাজায় — Recalculate, Send for approval, Approve, Reject (কারণ জিজ্ঞাসা করে), Pay everybody (`showPayRunSheet` খোলো), Cancel (নশ্চতিকরণসহ)।

ক্লাস/উইজটে: `_RunAction`

`StatelessWidget` — একটি সাধারণ `OutlinedButton.icon`, ধ্বংসাত্মক (destructive) হলো danger রঙে দেখায়।

Widget build(BuildContext context)

আইকন ও লবেলসহ একটি `OutlinedButton.icon` তৈরি করে, `destructive` ফ্ল্যাগ অনুযায়ী রঙ নির্ধারণ করে।

ক্লাস/উইজটে: `_LinesSection`

`ConsumerStatefulWidget` — মাসের সব পরেোল লাইন এবং "Generate" বাটন দেখায় যা বাল্ক-জেনারেটে প্রক্রিয়া চালায়।

ক্লাস/উইজটে: `_LinesSectionState`

জেনারেটে-বাটনের ব্যস্ততা (`_generating`) ট্র্যাক করে।

Future<void> _generate()

repo.generate(month, year) কল করে, লাইন ও ড্যাশবোর্ড প্রোভাইডার ইনভ্যালিডেট করে ও রান-তালিকা রিফ্রেশ করে। ফলাফলে সমস্যা (hasProblems) থাকলে _GenerateResultDialog দেখায় (কারণ বতেন-কাঠামোহীনরাই একমাত্র কার্যকর অংশ, সন্ধ্যাকবার হারিয়ে যাওয়া উচিত নয়), নাহলে সারাংশ-সন্ধ্যাকবার দেখায়।

Widget build(BuildContext context)

SectionHeader (Generate বাটনসহ, process পারমিশন থাকলে) দেখায়, তারপর payrollLinesProvider-এর when দিয়ে Loader/ErrorState/(খালি হিলে EmptyState)/অথবা প্রতিলিাইনরে জন্য _LineRow তালিকা রেন্ডার করে।

enum/ধ্রুবক শ্রুণে: PayrollLineStatus

abstract final class — ব্যাকএন্ডরে PayrollStatus-এর মরির: draft, approved, paid, cancelled। একটি লাইন অনুমোদিত হওয়ার পরই পরিশোধ করা যায়, এবং পরিশোধ হয়ে গেলে আর কিছু ঘটবে না — এ কারণেই সারিতে ভিন্ন ভিন্ন অবস্থায় ভিন্ন অ্যাকশন দেখানো হয়।

ক্লাস/উইজটে: _LineRow

ConsumerStatefulWidget — একটি কর্মচারীর পরে লাইন কার্ড আকারে দেখায়, approve/pay/delete অ্যাকশনসহ।

ক্লাস/উইজটে: _LineRowState

এই লাইনরে অ্যাকশনের ব্যস্ততা (_busy) ট্র্যাক করে।

Future<void> _run(Future<void> Function(PayrollRepository) action, String done)

action চালিয়ে লাইন ও ড্যাশবোর্ড প্রোভাইডার ইনভ্যালিডেট করে, সাফল্য/ব্যর্থতা Snackbar-এ দেখায়।

Future<void> _pay()

pickOne দিয়ে পমেন্ট-মথড (Bank transfer/Cash/Cheque) বেছে নতি বলে, তারপর askForText দিয়ে রফোরেন্স নিয়ে (ঐচ্ছিক, পরে বদলানো যায় না বলে সতর্ক করে), এবং _run দিয়ে repo.payLine কল করে।

Future<void> _delete()

confirmAction দিয়ে নিশ্চিতকরণ নিয়ে (মুছে ফেললে পুনরায় জনোরেটে করলে তখনকার কাঠামো/উপস্থিতি অনুযায়ী নতুন লাইন তৈরি হবে বলে জানায়), তারপর _run দিয়ে repo.deleteLine কল করে।

Widget build(BuildContext context)

পারমিশন ও লাইন-স্ট্যাটাস অনুযায়ী PopupMenuEntry তালিকা (approve/pay/delete) তৈরি করে। কর্মচারীর নাম, বসেকি-বতেন, অনুপস্থিতি-দনি (থাকলে), নটে-বতেন ও স্ট্যাটাস-চপি দেখায়; ব্যস্ত থাকলে স্পিনার, নাহলে PopupMenuButton দেখায় যা নির্বাচন অনুযায়ী approveLine/_pay/_delete কল করে।

ক্লাস/উইজটে: _GenerateResultDialog

StatelessWidget — বাল্ক-জেনারেটরে পর কাকে বাদ দেওয়া হয়েছে এবং কনে তা গুরুত্বপূর্ণ তা দেখানো ডায়ালগ।

Widget build(BuildContext context)

result.summary দেখায়, তারপর সতর্ক-রঙে ব্যাখ্যা দেয় যে বতেন-কাঠামোহীনরা এই মাসে বতেন পাবে না, এবং result.noSalaryStructure-এর প্রতিটি নাম তালিকাভুক্ত করে। "Got it" বাটনে ডায়ালগ বন্ধ হয়।

ফাইল: lib/features/payroll/payroll_sheets.dart

সংক্ষিপ্ত বিবরণ: পরেোল রান খোলা, রান পরিশোধ করা এবং পরেোল-স্টেটস সম্পাদনার জন্য তিনটি বটম-শিট ফর্ম এই ফাইলে আছে।

টপ-লেভেলে ফাংশন: Future<void> showOpenRunSheet(BuildContext context, {required int month, required int year})

মাসিক রান খোলার জন্য _OpenRunSheet মডাল বটম-শিট খোলে। মাস/বছর এখানে বেছে নেওয়া যায় না — বাইরের স্ক্রিনি থেকে আসে, কারণ প্রতিপরিষিডে ঠিক একটিই রান হতে পারে এবং শিটে ভেতর থেকে অন্য পরিষিড বাছাই করলে স্ক্রিনি ভিন্ন কিছু দেখানো শুরু করত।

ক্লাস/উইজটে: _OpenRunSheet

ConsumerStatefulWidget — নির্দিষ্ট মাস/বছরের জন্য একটি নতুন রান খোলার ফর্ম, শুধু ঐচ্ছিক রমার্কস ফিল্ড নিয়ে।

ক্লাস/উইজটে: _OpenRunSheetState

রমার্কস কন্ট্রোলার ও সাবমিশন-স্টেটে রাখা।

void dispose()

_remarks কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

repo.createRun(month, year, remarks) কল করে, রান-তালিকা রিফ্রেশ করে ও ড্যাশবোর্ড ইনভ্যালিডিটে করে, শিট বন্ধ করে "Run opened" বার্তা দেখায়। এই পরিষিডে আগে থেকেই রান থাকলে ব্যাকগ্রাউন্ডে এরর-বার্তা (রানের নাম-সহ) যমেন আছে তমেনই দেখায়।

Widget build(BuildContext context)

FormSheetFrame-এর ভেতরে রান কী তা ব্যাখ্যা করা টেক্সট ও একটি ঐচ্ছিক মাল্টি-লাইন রমার্কস ফিল্ড দেখায়।

টপ-লেভেলে ফাংশন: Future<void> showPayRunSheet(BuildContext context, {required PayrollRun run})

রানের সব অপরিশোধিত লাইন পরিশোধ করার _PayRunSheet মডাল বটম-শিট খোলে — স্ক্রিনে একমাত্র অপরিস্রবীণ ধাপ, তাই কী ঘটতে যাচ্ছে (কতজন, কত টাকা) আগাই বলে দেয়।

ক্লাস/উইজটে: _PayRunSheet

ConsumerStatefulWidget — পমেন্ট-মথড, পমেন্ট-তারিখ ও ঐচ্ছিক রফোরেন্স-প্রফিক্স নেওয়ার ফর্ম।

ক্লাস/উইজটে: _PayRunSheetState

রফোরেন্স-প্রফিক্স কন্ট্রোলার, নির্বাচতি মথেড ও পমেন্ট-তারখি স্টেটে রাখো।

void dispose()

_prefix কন্ট্রোলার ডিসপোজ করো।

Future<void> _submit()

repo.pay(runId, paymentMethod, referencePrefix, paymentDate) কল করে, ফলাফল দিয়ে payrollRunsProvider-এ apply করে, ড্যাশবোর্ড ও লাইন প্রোভাইডার ইনভ্যালিডিটে করে, শিট বন্ধ করে "paid" বার্তা দেখায়।

Widget build(BuildContext context)

একটি MessageBanner.info দিয়ে কতজন/কত টাকা পরিশোধ হবে ও এটি অপ্রবর্তনীয় তা জানায়, তারপর পমেন্ট-মথেড ড্রপডাউন, DateField (পমেন্ট-তারখি) ও রফোরেন্স-প্রফিক্স ফিল্ড (হেল্পার-টেক্সটে বলা থাকে যে না দিলে রান-নম্বর ব্যবহৃত হবে) দেখায়, শেষে জানায় যে ইতিমধ্যে পরিশোধিত/বাতিল লাইন বাদ যাবে।

উপ-লভেলে ফাংশন: Future<void> showPayrollSettingsSheet(BuildContext context)

পরেোল-হিসাবের ন্যূনতম সম্পাদনার _SettingsSheet মডাল বটম-শিট খোলো।

ক্লাস/উইজটে: _SettingsSheet

ConsumerStatefulWidget — perDayBasis, absence-base, ওভারটাইম ও প্রতিটি ভিত্তি/কর্তনের শতাংশ সম্পাদনার ফর্ম। এটি কীভাবে একমাত্র আপডেট যা ব্যাকএন্ডে সম্পূর্ণ null-guarded, তবু ফর্ম সবসময় সব ফিল্ড পাঠায় কারণ সবগুলোই দৃশ্যমান।

ক্লাস/উইজটে: _SettingsSheetState

সব স্টেইস-ফিল্ডের কন্ট্রোলার ও নির্বাচতি ড্রপডাউন-মান রাখো; _seeded ফ্ল্যাগ দিয়ে সারভারের উত্তর একবারই ফর্মে বসানো হয়েছে কনি ট্র্যাক করো।

void _seed(PayrollSettings settings)

সারভার থেকে পাওয়া settings-এর মান দিয়ে সব কন্ট্রোলার ও ড্রপডাউন-স্টেটে একবার পূরণ করে, _seeded = true সেট করে।

static String _plain(double value)

একটি সংখ্যাকে টেক্সট-ফিল্ডের উপযোগী প্লেন স্ট্রিং-এ রূপান্তর করে (পূরণসংখ্যা হলে দশমকি ছাড়া) — Fmt.money জাতীয় ফর্ম্যাটার ব্যবহার করা হয় না কারণ সেগুলোর সম্ভল/সাফক্স পার্স করে ফরেনো যায় না।

void dispose()

সব কন্ট্রোলার (multiplier, hours, houseRent, medical, transport, food, providentFund, tax) ডিসপোজ করো।

double _read(TextEditingController controller, double fallback)

কন্ট্রোলারের টেক্সট পার্স করে double রিটার্ন করে, পার্স ব্যর্থ হলে fallback দেয়।

Future<void> _submit()

ফর্মেরে সব মান দিয়ে PayrollSettingsRequest তৈরিকরে repo.updateSettings কল করে, payrollSettingsProvider ইনভ্যালিডেটে করে, শিট বন্ধ করে "Saved" বার্তা দেখায়। রঞ্জে-বহিষ্ঠত মান হলে ব্যাকএন্ডেরে নামসহ এরর-বার্তা (যেমন "Overtime multiplier must be between 1 and 5") যেন আছে তেমনই দেখায়।

String? _percent(String? value)

একটি শতাংশ-ভ্যালিডেটের — সংখ্যা পার্স না হলে বা 0–100 রঞ্জে বাইরে হলে এরর-মসেজে দেয়।

Widget build(BuildContext context)

সেটিংস লোড না হলে ও seed না হলে ErrorState/Loader দেখায়; সেটিংস এলে _seed কল করে একবার ফর্ম পূরণ করে। এরপর FormSheetFrame-এর ভেতরে: "A day's pay" বিভাগে perDayBasis ড্রপডাউন ও তার হিট-টেক্সট, absence-base ড্রপডাউন; "Overtime" বিভাগে একটি SwitchListTile এবং সক্রিয় থাকলে multiplier (1–5)/hours (1–24) ফিল্ড ও overtime-base ড্রপডাউন; "Allowances" বিভাগে houseRent/medical/transport/food শতাংশ ফিল্ড (মোট ১০০-এর বেশি হলে সতর্কবার্তা, কারণ ব্যাকএন্ড প্রতিটি আলাদাভাবে চেক করে কিন্তু মোট নয়); "Deductions" বিভাগে providentFund ও tax শতাংশ ফিল্ড — সাজায়।

ক্লাস/উইজটে: _Label

StatelessWidget — একটি সিকেশন-লেবেল টেক্সট, ছোট/মিডিউম/বোল্ড স্টাইলে।

Widget build(BuildContext context)

_Label-এর টেক্সট নর্দিষ্ট স্টাইলে (মিডিউম রঙ, ছোট ফন্ট, বোল্ড, লটার-স্পেসিং) প্যাডিং সহ দেখায়।

ফাইল: lib/features/payroll/salary_sheet_models.dart

সংক্ষিপ্ত ববিরণ: পুরো কোম্পানির এক মাসের বতেনের হিসাব লাইভ দেখানোর জন্য মডেল। এখানে কছুই স্টোর করা হয় না — অ্যাটেন্ডেন্স ঠিকি হলে এই হিসাবও বদলে যায়; পরে "রান" করাটাই এই ফিগারগুলো ফ্রজি করে দেওয়ার আলাদা কাজ।

ক্লাস: SalarySheet

পুরো মাসের বতেন-শটিকে প্রতিনিধিত্ব করে — কোন মাস/বছরের জন্য, কতজন কর্মীর সারি (rows), আর কলাম-টোটাল (totals)। এছাড়া প্রতিদিনের হার কীভাবে নির্ধারিত হয়েছে তা বোঝানোর জন্য perDayBasis ও perDayDivisor, এবং ওভারটাইম সক্রিয় কনি ও তার মাল্টিপ্লায়ার রাখা।

SalarySheet({required payMonth, required payYear, required rows, required totals, perDayBasis, perDayDivisor = 0, overtimeEnabled = false, overtimeMultiplier})

কনস্ট্রাক্টর — সব ফিল্ড সরাসরি সেট করে দেয়। totals রসেপন্স থেকেই নেওয়া হয়, লোকাল যোগ করে বানানো হয় না, যাতে ফুটার আর শিটে হিসাব কখনও আলাদা না হয়ে যায়।

factory SalarySheet.fromJson(Map<String, dynamic> json)

সারভার রসেপন্স থেকে SalarySheet তৈরিকরে। rows লিস্টের প্রতিটি আইটেমকে Map<String, dynamic> হিসেবে ফিল্টার করে SalarySheetRow.fromJson দিয়ে ম্যাপ করে অপ্রবর্তনীয় (growable: false) লিস্ট বানায়; totals পুরো json-কেই SalarySheetTotals.fromJson-এ পাঠিয়ে বের করে।

সংখ্যাগত/বুলিয়ান ফিল্ডগুলোতে null-নিরপদ ডিফল্ট বসানো হয় (যেমন মাস/বছর 0, overtimeEnabled false)।

ক্লাস: SalarySheetTotals

শাটিরে ফুটার — গ্রস আয়, মোট কর্তন, নটে প্রদায়ে, বসেকি, ওভারটাইম পমেন্ট, বোনাস, ট্যাক্স, প্রভিডেন্ট ফান্ড, অনুপস্থিতি দনি ও তার কর্তন। রোগুলোর পাশে একই রসেপন্স বডি থেকে এগুলো পড়া হয়, আলাদা নস্টেডে অবজেক্ট থেকে নয়।

SalarySheetTotals({required grossEarnings, required deductions, required netPayable, required basic, required overtimePayment, required bonus, required tax, required providentFund, required absentDays, required absentDeduction})

কনস্ট্রাক্টর — সব টোটাল ফিল্ড সটে করে।

factory SalarySheetTotals.fromJson(Map<String, dynamic> json)

json থেকে টোটালগুলো বরে করে। ভতেরে একটি লোকাল হলেপার ফাংশন number(String key) সংজ্ঞায়িত করা হয়, যা json[key]-কে num? হিসেবে পড়ে toDouble() করে, না পলে 0 ফরত দায়ে — এভাবে totalGrossEarnings, totalDeductions, totalNetPayable ইত্যাদিকী নাম থেকে মান বরে করা হয়। totalAbsentDays আলাদাভাবে int হিসেবে পার্স করা হয়।

ক্লাস: SalarySheetRow

বতেন-শাটিকে একজন কর্মীর একটি লাইন। বসেকি, গ্রস আয়, মোট কর্তন, নটে প্রদায়ে, অনুপস্থিতি দনি বাধ্যতামূলক; বাকসিব (হাউস রেন্ট, মেডিকলে, ট্রান্সপোর্ট, ফুড, স্পেশাল, ওভারটাইম, বোনাস, অন্যান্য আয়/কর্তন, ট্যাক্স, প্রভিডেন্ট ফান্ড ইত্যাদি) ঐচ্ছকি ডফিল্ট-সহ ফিল্ড। কর্মীর পরচিতি (আইডি, নম্বর, নাম, পদবি, বিভাগ), পরেোল আইডি, পমেন্ট স্ট্যাটাস/মথেড এবং ফগিাররে উৎস (source) রাখা।

SalarySheetRow({required basic, required grossEarnings, required totalDeductions, required netPayable, required absentDays, employeeId, employeeNumber, employeeName, position, department, houseRent = 0, medical = 0, transport = 0, food = 0, special = 0, overtimeHours = 0, overtimePayment = 0, otherEarnings = 0, otherDeductions = 0, bonus = 0, absentDeduction = 0, tax = 0, providentFund = 0, note, payrollId, paymentStatus, paymentMethod, source})

কনস্ট্রাক্টর — একজন কর্মীর সাররি সব ফিল্ড সটে করে দায়ে। otherEarnings/otherDeductions কর্মীর স্ট্রাকচারে অতিরিক্ত আইটেমে (লোন পরশিোধ, ইন্টারনেটে ভাতা ইত্যাদি) বহন করে, পরেোল ফরজি করার সময় যভাবে ফরজি হয় সভাবেই।

bool get isRun

payrollId নাল না হলে true — অরথাৎ এই মাসরে জন্য এই কর্মীর পরেোল আসলই "রান" হয়ছে কনি, নাকি এটা এখনও একটি প্রক্সপেণ (projection) মাত্র তা বলে দায়ে।

String get personLabel

কর্মীর প্রদর্শনযোগ্য নাম ফরত দায়ে — employeeName, না থাকলে employeeNumber, তাও না থাকলে Employee #<id বা ">"> ফরম্যাটে ফলব্যাক করে।

factory SalarySheetRow.fromJson(Map<String, dynamic> json)

JSON থেকে একটি সারি বানায়। ভতেররে লোকাল number(String key) হলেপার প্রতটি সংখ্যাসূচক ফিল্ডরে (basic, grossEarnings, houseRent, medical ইত্যাদি) জন্য null-নরিপদ double কনভার্সন করে; employeeId, absentDays, payrollId int হিসেবে পার্স হয়; বাকি স্ট্রিং ফিল্ড সরাসরি as String? দায়ে নেওয়া হয়।

ফাইল: lib/features/payroll/salary_sheet_screen.dart

সংক্ষিপ্ত ববিরণ: যে মাসের দিকে পরেরো স্ক্রিনি তাকিয়ে আছে তার লাইভ বতেন-শটি দেখানোর UI। রান-করা সারিও প্রক্ষেপণ (projection) সারির মধ্যে স্পষ্ট পার্থক্য দেখায় এবং প্রতিটি সারি ট্যাপ করলে বিস্তারিত ব্রেকেডাউন খোলে।

টপ-লেভেলে প্রোভাইডার: salarySheetProvider

`FutureProvider.autoDispose<SalarySheet>` — বর্তমানে নির্বাচতি পরেরো পরিষিড (`payrollPeriodProvider`) পড়ে, তারপর `payrollRepositoryProvider`-এর `salarySheet(month, year)` কল করে শটি আনে।

ক্লাস/উইজটে: SalarySheetScreen

`ConsumerWidget` যা মাস অনুযায়ী পুরো বতেন-শটি প্রদর্শন করে — টোটাল স্ট্যাট কার্ড, প্রতিদিনের হারের ব্যাখ্যা এবং কর্মী-ভিত্তিক সারিগুলোর তালিকা। ডটো সবসময় লাইভ হিসাব করা, কোথাও থেকে সংরক্ষিত পড়া নয়।

`static void open(BuildContext context)`

`MaterialPageRoute` দিয়ে `SalarySheetScreen`-কে নভিগেশন স্ট্যাককে পুশ করে, স্ক্রিনিটি খোলার প্রমতি উপায় হিসেবে কাজ করে।

`Widget build(BuildContext context, WidgetRef ref)`

`payrollPeriodProvider` ও `salarySheetProvider` watch করে UI বানায়। `AppBar`-এ বর্তমান মাস/বছর দেখায়। `RefreshIndicator` দিয়ে pull-to-refresh সমর্থন করে যা `salarySheetProvider` ইনভ্যালিডেট করে। `async.when` দিয়ে লোডিং/এর/ডটো তিনি অবস্থা হ্যান্ডল করে — ডটো অবস্থায় নটে-টু-পে, মোট কর্মী, গ্রস ও কর্তনের `StatCard`, প্রতিদিনের হারের ব্যাখ্যা টেক্সট (যদি `perDayBasis` থাকে), এবং প্রতিটি সারির জন্য `_Row` উইজটে রেন্ডার করে; সারি খালি থাকলে একটি ব্যাখ্যামূলক `AppCard` দেখায়।

ক্লাস: _Row (StatefulWidget)

একজন কর্মীর একটি সারি — নাম, পদবি/বিভাগ, নটে পে এবং রান/প্রক্ষেপণ স্ট্যাটাস সংক্ষিপ্তভাবে দেখায়, ট্যাপ করলে বিস্তারিত খোলে।

ক্লাস: _RowState

`_Row`-এর স্টেটে — সারিখোলা আছে কিনা (`_open`) ট্র্যাক করে।

`Widget build(BuildContext context)`

সারির সংক্ষিপ্ত অংশ (নাম, পদবি/বিভাগ/অনুপস্থিতি, নটে পে, স্ট্যাটাস) সবসময় দেখায়; `_open` true হলে ডিভাইডারে পর বেসিক, হাউস রেন্ট, মডেকিলে, ট্রান্সপোর্ট, ফুড, স্পেশাল, ওভারটাইম, বোনাস, অন্যান্য আয় (শূন্য না হলে প্রতিটি শির্তসাপক্ষে) দিয়ে গ্রস, তারপর অনুপস্থিতি/ট্যাক্স/প্রভিডেন্ট ফান্ড/অন্যান্য কর্তন দিয়ে নটে পর্যন্ত `_Line` উইজটে একটি বিস্তারিত তালিকা দেখায়; নোট ও উৎস (source) থাকলে তাও দেখায়। ট্যাপে `setState`-এর মাধ্যমে `_open` টগল হয়।

ক্লাস: _Line (StatelessWidget)

একটি লেবেল-মূল্য জোড়া সারিতে দেখায়; `bold` true হলে ভারী ফন্ট ব্যবহার হয়, ঋণাত্মক পরিমাণ লাল রঙে দেখায়।

Widget build(BuildContext context)

label ও Fmt.money(amount) পাশাপাশি একটি Row-এ রেন্ডার করে; স্টাইল bold ও পরিমাণ ঋণাত্মক কনি তার উপর নরিভর করে বদলায়।

ফাইল: lib/features/payslips/create_payroll_sheet.dart

সংক্ষিপ্ত ববিরণ: হাতে করে একটি পিরোল রকের্ড তরৈকিরার বটম শটি — মাসকি ব্যাচ রান নয়, বরং সইে ব্যতকিরম যখনে কনো নতুন কর্মী বা সংশোধনরে জন্য একজনরে রকের্ড ম্যানুয়াল বিনাত হয। ফর্মে এগারোট নিয়, মাত্র কয়কেট মূল ফগির রাখা হযছে, বাকগিলো ফাঁকা রাখলে ব্যাকএন্ড কর্মীর বতেন-স্ট্রাকচার থেকে নেযে।

টপ-লভেলে ফাংশন: Future<void> showCreatePayrollSheet(BuildContext context)

showModalBottomSheet দিয়ে _CreatePayrollSheet উইজটেকে স্ক্রল-নয়িন্ত্রতি (isScrollControlled: true) মোডাল বটম শটি হসিবে থোলো।

ক্লাস: _CreatePayrollSheet (ConsumerStatefulWidget)

পিরোল রকের্ড তরৈরি ফর্মরে কনটেইনার উইজটে; কনো নজিস্ব লজকি নই, শুধু স্টটে তরৈকিরে।

ক্লাস: _CreatePayrollSheetState

ফর্মরে সব ইনপুট কন্ট্রোলার (বসেকি, হাউস রেন্ট, মডেকিলে, ট্রান্সপোর্ট, বোনাস, কর্তন, নোট), নরিবাচতি কর্মী, মাস/বছর, এবং সাবমশিনরে অবস্থা (_submitting, _error) ম্যানজে করে।

static DateTime _previousMonth()

বর্তমান মাসরে ঠকি আগরে মাসরে DateTime ফরেত দিয়ে — ডফিল্ট মাস/বছর হসিবে ব্যবহৃত হয, কারণ পিরোল সবসময় শেষে-হওয়া একটি পিরিয়ডরে জন্য রান করা হয।

void dispose()

সব TextEditingController (বসেকি, হাউস রেন্ট, মডেকিলে, ট্রান্সপোর্ট, বোনাস, কর্তন, নোট) ডসিপোজ করে, তারপর super.dispose() কল করে।

Future<void> _pickEmployee()

EmployeePicker.show দিয়ে "Run payroll for" শরিোনামে কর্মী বাছাইয়ের ডায়ালগ থোলো; কর্মী নরিবাচতি হলে setState দিয়ে _employee সটে করে।

Future<void> _submit()

ফর্ম সাবমটি করে। ইতমিধ্যে সাবমটি চলমান থাকলে বা ফর্ম-ভ্যালিডেশন ব্যর্থ হলে থমে যায; _employee নাল থাকলে এরর দেখোয়। ফোকাস সরিয়ে _submitting true করে, তারপর payslipRepositoryProvider.create() কল করে CreatePayrollRequest পাঠায় (টেক্সট ফিল্ডগুলো double.tryParse দিয়ে পার্স করে, খালি হলে নাল যায)। সফল হলে myPayslipsProvider ইনভ্যালিডিটে করে (কারণ সাইন-ইন করা ব্যবহারকারীও এই রকের্ডরে বযিয় হতে পারনে), শটি বন্ধ করে সাফল্যরে সন্ধ্যাকবার দেখোয়। ApiException (যমেন "already exists" বার্তা) ক্যাচ করে _error-এ দেখোয়, অন্য কনো এরর হলে জনেরেকি বার্তা দেখোয়; finally-তে _submitting false করে।

Widget build(BuildContext context)

পুরো ফর্ম UI তৈরি করে — কর্মী নির্বাচক (InkWell + InputDecorator), মাস/বছর ড্রপডাউন, এবং বসেকি/হাউস রেন্ট/মডেকিলে/ট্রান্সপোর্ট/বোনাস/কর্তন-এর জন্য জোড়ায়-জোড়ায় TextFormField, একটি নোট ফিল্ড, ও একটি LoadingButton। কীবোর্ড ভিউ-ইনস্টেটের জন্য Padding দিয়ে র‍্যাপ করা এবং SingleChildScrollView-এ মোড়ানো, যাতে কীবোর্ড খোলা অবস্থায়ও ব্যবহারযোগ্য থাকে।

static String? _money(String? value)

আর্থিক ইনপুট ভ্যালিডেট করে — ফাঁকা হলে বৈধ (নাল রটার্ন, কারণ ফাঁকা মান স্ট্রাকচার থেকে নেওয়া হবে); সংখ্যা পার্স করা না গেলে বা ঋণাত্মক হলে সংশ্লিষ্ট এরর বার্তা ফেরত দেয়।

ফাইল: lib/features/payslips/payslip_models.dart

সংক্ষিপ্ত বিবরণ: একজন কর্মীর এক মাসের পরে রেকর্ডের মডেল। পরে রান চালানোর সময় ফিগারগুলো (বলিবেল ও ওভারটাইম হার-সহ) রেকর্ডে ফরজি হয়ে যায়, ফলে কর্মীর বতেন-স্ট্রাকচার পরে বদলে গেলেও পসেলপি পুরনো, সঠিক তথ্যের প্রতিনিধিত্ব করে।

ক্লাস: Payslip

একটি সম্পূর্ণ পসেলপি রেকর্ড — মৌলিক তথ্য (id, মাস, বছর, বসেকি স্যালারি, নটে স্যালারি, স্ট্যাটাস, কর্মী আইডি/নাম), সব ঐচ্ছিক ভাতা (হাউস রেন্ট, মডেকিলে, ট্রান্সপোর্ট, ফুড, স্পেশাল, বোনাস, বলিবেল আওয়ার/পে, ওভারটাইম আওয়ার/পে, অন্যান্য আয়), সব ঐচ্ছিক কর্তন (ট্যাক্স, ইনস্যুরেন্স, প্রভিডেন্ট ফান্ড, অ্যাটনেডেন্স, সাধারণ কর্তন, অন্যান্য কর্তন), অনুপস্থিতি দনি, এবং পেমেন্ট মেটোডটো (রেফারেন্স, মেথড, তারিখ, নোট, তৈরি সময়)।

Payslip({required id, required payMonth, required payYear, required basicSalary, required netSalary, required status, required employeeId, required employeeName, houseRent, medicalAllowance, transportAllowance, foodAllowance, specialAllowance, bonus, billableHours, billablePay, overtimeHours, overtimePay, otherEarnings, taxDeduction, insuranceDeduction, providentFundDeduction, attendanceDeduction, deductions, otherDeductions, absentDays, paymentReference, paymentMethod, paidAt, notes, createdAt})

কনস্ট্রাক্টর — সব ফিল্ড সরাসরি সেট করে।

double get gross

বসেকি স্যালারি সাথে সব ঐচ্ছিক আয় (হাউস রেন্ট, মডেকিলে, ট্রান্সপোর্ট, ফুড, স্পেশাল, বোনাস, বলিবেল পে, ওভারটাইম পে, অন্যান্য আয়) যোগ করে মোট গ্রস আয় বের করে — যাই ক্রমে PDF-এর বাম কলামে টোটাল হয় ঠিক সেই ক্রমে।

double get totalDeductions

সব ঐচ্ছিক কর্তন (ট্যাক্স, প্রভিডেন্ট ফান্ড, ইনস্যুরেন্স, অ্যাটনেডেন্স, সাধারণ কর্তন, অন্যান্য কর্তন) যোগ করে মোট কর্তন বের করে — PDF-এর ডান কলামের প্রতিলিপি।

bool get canDownload

স্ট্যাটাস 'DRAFT' না হলে true। DRAFT অনুমোদিত হয়নি বলে ফিগার এখনও বদলাতে পারে, তাই ডাউনলোডযোগ্য ডকুমেন্ট দেওয়া হলে সেটা পরে দেওয়া প্রকৃত টাকার সাথে না-মিলতে পারে।

List<({String label, double amount})> get earnings

আয়রে লাইনগুলোর তালিকা রকের্ড-টাইপ হিসেবে ফেরত দেয় — বেসিকি স্যালারিসি বসময় থাকে, বাকগুলো (হাউস রেন্ট থেকে অন্যান্য আয় পর্যন্ত) শুধু শূন্য না হলে যোগ হয়, যাতে UI-তে অপ্ৰাসংগিক শূন্য লাইন না দেখায়।

List<({String label, double amount})> get deductionLines

কর্তনের লাইনগুলোর একই ধরনের ফিল্টার করা তালিকা ফেরত দেয় — ট্যাক্স, প্রভিডেন্ট ফান্ড, ইনস্যুরেন্স, অ্যাটেনেডেন্স, কর্তন, অন্যান্য কর্তন — শুধু শূন্য নয় এমনগুলো।

factory Payslip.fromJson(Map<String, dynamic> json)

JSON থেকে Payslip তৈরিকরে। ভেতরে লোকাল হলেপার d(String key) num?-কে double?-এ কনভার্ট করে (নালযোগ্য, ডফিল্ট বসায় না)। id, payMonth, payYear, basicSalary, netSalary, employeeId, employeeName, status বাধ্যতামূলক ফিল্ডে যুক্তসিঙত ডফিল্ট (0, 1, বর্তমান বছর, 'DRAFT', খালি স্ট্রিং) বসায়; বাকসি ফিল্ড ঐচ্ছিক হিসেবে পার্স করে।

টপ-লেভেলে ক্লাস: PayrollPermissions

abstract final class — পরোল-সংক্রান্ত পারমিশন কোডের ধ্রুবক ধারক।

static const view

'PAYROLL_VIEW' — রান, ড্যাশবোর্ড ও সবার লাইন পড়ার অনুমতি

static const process

'PAYROLL_PROCESS' — পরোল রান চালানো ও একক রকের্ড তৈরিকরার অনুমতি, এবং রান খোলা/পুনর্গণনা/সাবমিট/বাতলি করারও অনুমতি এই একই কোডে কভার হয়।

static const approve

'PAYROLL_APPROVE' — রান সাইন-অফ, প্রত্যাখ্যান বা পরিশোধ করার অনুমতি ইচ্ছাকৃতভাবে process থেকে আলাদা কোড — যে প্রস্তুত করে সে যেনে নজি অনুমোদনও না করে, এবং এই বিভাজন ব্যাকএন্ড কার্যকর করে।

ক্লাস: CreatePayrollRequest

POST /hr/payroll-এ পাঠানোর অনুরোধ মডেলে — হাতে একটি মাসিক রকের্ড তৈরিকরে। employeeId, payMonth, payYear বাধ্যতামূলক; বাকসি ফগিার (বেসিকি, হাউস রেন্ট, মেডিকিলে, ট্রান্সপোর্ট, বোনাস, কর্তন, নোট) ঐচ্ছিক — বাদ দিলে ব্যাকএন্ড কর্মীর বতেন-স্ট্রাকচার থেকে ভেরাট করে, শূন্য হিসেবে ধরে না। নটে কখনও পাঠানো হয় না কারণ ব্যাকএন্ড অ্যাটেনেডেন্স, ওভারটাইম, বলিবেল আওয়ার, লোন পরিশোধ ও অতিরিক্ত উপাদান থেকে সটো বরে করে।

CreatePayrollRequest({required employeeId, required payMonth, required payYear, basicSalary, houseRent, medicalAllowance, transportAllowance, bonus, deductions, notes})

কনস্ট্রাক্টর — সব ফিল্ড সটে করে।

Map<String, dynamic> toJson()

রকি়োসেট বডিতৈরিকরে। notes-কে ট্রমি করে; employeeId/payMonth/payYear সবসময় অন্তর্ভুক্ত হয়; বাকি প্রতটি ফিল্ড শুধুমাত্র নাল না হলে (এবং নোটে কষত্রে ট্রমি করার পর খালি না হলে) ম্যাপে যুক্ত হয় — যাতে বাদ দেওয়া ফিল্ড JSON-এ একদমই না থাকে ("ব্যবহার করো স্ট্রাকচার" বোঝাতে, "শূন্য" নয়)।

ফাইল: lib/features/payslips/payslip_repository.dart

সংক্ষিপ্ত ববিরণ: পস্লেপি-সংক্রান্ত API কল (তালিকা আনা, একক রকের্ড তরৈ, PDF ডাউনলোড) মোড়ানো রপিজিটরি, এবং সাইন-ইন করা ব্যবহারকারীর নজিস্ব পস্লেপি আনার প্রোভাইডার।

ক্লাস: PayslipRepository

পরেোল/পস্লেপি এন্ডপয়ন্টগুলোর জন্য `ApiClient`-ভিত্তিক রূপার; বসে পাথ `/hr/payroll`।

`PayslipRepository(this._api)`

কনস্ট্রাক্টর — একটি `ApiClient` ইনস্ট্যান্স ইনজেক্ট করে।

`Future<PagedResponse<Payslip>> forEmployee(int employeeId, {int page = 0, int size = 24})`

`GET /hr/payroll/employee/{employeeId}` কল করে নরিদষ্টি কর্মীর পস্লেপিগুলোর পৃষ্ঠাবভিক্ত ফলাফল আনে, `Payslip.fromJson` দিয়ে প্রতটি এন্টরি পার্স করে।

`Future<Payslip> create(CreatePayrollRequest request)`

`POST /hr/payroll`-এ `request.toJson()` পাঠিয়ে একটি নতুন পরেোল রকের্ড হাতে তরৈকিরে ও তরৈহিওয়া `Payslip` ফরেত দিয়ে। `PAYROLL_PROCESS` পারমশিন লাগে; একই কর্মী ও পরিষিডরে জন্য রকের্ড আগে থেকে থাকলে প্রত্যাখ্যাত হয়, এবং সেই বার্তা ফরেত আসে।

`Future<String> downloadPdf(Payslip payslip)`

`GET /hr/payroll/{id}/payslip` থেকে PDF বাইট আনে, অস্থায়ী ডরিক্টরতি `payslip-<year>-<month>.pdf` নামে সেভে করে ও ফাইলরে পাথ ফরেত দিয়ে। এন্ডপয়ন্টটি নজিই স্কোপ করা — `PAYROLL_VIEW` থাকলে যেকোরো, নয়তো শুধু নজিরেটা ডাউনলোড হয়, তাই এখানে অতিরিক্ত গার্ড লাগে না। ফাইলটি স্থায়ী স্টোরজে না গিয়ে সরাসরি প্ল্যাটফর্ম ভিয়ারে হস্তান্তরে জন্য টেম্প ডরিক্টরতি যায়।

টপ-লেভেলে প্রোভাইডার: payslipRepositoryProvider

`Provider<PayslipRepository>` — `apiClientProvider` থেকে `ApiClient` নিয়ে `PayslipRepository` ইনস্ট্যান্স তরৈকিরে।

টপ-লেভেলে প্রোভাইডার: myPayslipsProvider

`FutureProvider<List<Payslip>>` — সাইন-ইন করা ব্যবহারকারীর নজিস্ব পস্লেপি, সবচেয়ে নতুনটি আগে। প্রথমে `myEmployeeProvider` থেকে কর্মী রকের্ড আনে (কারণ এন্ডপয়ন্ট কর্মী আইডি দিয়ে কী করা, ব্যবহারকারী আইডি দিয়ে নয়, আর JWT-তে থাকে ব্যবহারকারীর আইডি); কর্মী রকের্ড না থাকলে "খালি তালিকা" না দেখিয়ে ব্যাখ্যাসহ `ApiException` ছোঁড়ে। এরপর `forEmployee` কল করে পৃষ্ঠা আনে এবং বছর তারপর মাস অনুযায়ী অবরোহী ক্রমে সাজায়।

ফাইল: lib/features/payslips/payslips_screen.dart

সংক্ষিপ্ত ববিরণ: ব্যবহারকারীর নজিস্ব পস্লেপি তালিকা ও বসিতারতি দেখার স্ক্রনি।

`PAYROLL_PROCESS` পারমশিনধারীদের জন্য একটি মিসিড রকের্ড হাতে তরৈকিরার বাটনও এখানে যুক্ত করা আছে, যদণ্ডি স্ক্রনিটি মূলত ব্যক্তগিত।

ক্লাস/উইজটে: PayslipsScreen (ConsumerWidget)

"My payslips" তালিকা স্ক্রিনি — লোডিং/এরর/খালি/ডিটো অবস্থা হ্যান্ডল করে, এবং পারমিশন থাকলে একটি ফ্লোটিং অ্যাকশন বাটন দেখায়।

Widget build(BuildContext context, WidgetRef ref)

myPayslipsProvider watch করে এবং permissionControllerProvider থেকে PayrollPermissions.process পারমিশন আছে কিনা চকে করে। পারমিশন থাকলে showCreatePayrollSheet খোলার FloatingActionButton.extended দেখায় (অ্যাপরে একমাত্র পরেোল সারফসে বলে অ্যাক্সেসেরে ভিত্তিতে দেখানো, কিন্তু পারমিশন ছাড়া সম্পূর্ণ লুকানো থাকে)। ডিটো এলে খালিহলে EmptyState, নয়তো RefreshIndicator-সহ ListView.separated দিয়ে _PayslipCard লস্টি রেন্ডার করে।

ক্লাস: _PayslipCard (StatelessWidget)

তালিকার প্রতিটি পস্লেপিরে সংক্ষিপ্ত কার্ড — মাস/বছর, স্ট্যাটাস চপি, নটে পে দেখায় এবং ট্যাপে PayslipDetailScreen-এ নিয়ে যায়।

Widget build(BuildContext context)

AppCard ব্যবহার করে মাস/বছর, StatusChip, নটে পে ও একটি শিভেরন আইকন সাজিয়ে দেখায়; onTap-এ MaterialPageRoute দিয়ে PayslipDetailScreen পুশ করে।

ক্লাস/উইজটে: PayslipDetailScreen (ConsumerStatefulWidget)

একটি নির্দিষ্ট পস্লেপিরে সম্পূর্ণ বিবরণ দেখানোর স্ক্রিনি — নটে পে, স্ট্যাটাস, আয়/কর্তনের ব্রেকেডাউন, অনুপস্থিতি নোটিশ, নোট এবং PDF ডাউনলোড/ওপনে বাটন।

ক্লাস: _PayslipDetailScreenState

PayslipDetailScreen-এর স্টেটে; PDF ডাউনলোডেরে প্রগ্রসে (_downloading) ট্র্যাক করে।

Future<void> _openPdf()

_downloading true করে payslipRepositoryProvider.downloadPdf() কল করে PDF নামায়, তারপর OpenFile.open(path) দিয়ে ফাইলটি সিস্টেমে ভিয়ারে খোলার চেষ্টা করে। ফলাফল সফল না হলে "কোনো অ্যাপ PDF খুলতে পারছে না" জাতীয় সন্ধ্যাকবার দেখায় (সাধারণত PDF ভিয়ার ইনস্টল না থাকার লক্ষণ)। ApiException ক্যাচ করে তার বার্তা দেখায়, অন্য এরর হলে জনেরেকি বার্তা দেখায়; finally-তে _downloading false করে।

Widget build(BuildContext context)

পস্লেপিরে সব তথ্য সাজিয়ে দেখায়: শীর্ষে নটে পে ও স্ট্যাটাস কার্ড (পরিশোধ হয়ে থাকলে তারখি ও পমেন্টে মখেড), তারপর _Lines দিয়ে আয় (p.earnings, টোটাল p.gross) ও কর্তন (p.deductionLines, টোটাল p.totalDeductions, negative: true) বিভাগ — কর্তন খালি থাকলে "কিছু কাটা হয়নি" বার্তা। অনুপস্থিতি দনি থাকলে তথ্য ব্যানার, নোট থাকলে নোট সেকশন। শেষে p.canDownload true হলে "Open PDF payslip" বাটন (_openPdf কল করে), নয়তো DRAFT-এর ব্যাখ্যাসহ তথ্য ব্যানার দেখায়, কারণ অনুমোদনের আগে ফগিার বদলাতে পারে বলে ডকুমেন্ট এখনই দেওয়া হয় না।

ক্লাস: _Lines (StatelessWidget)

লবেলে-মূল্য জোড়ার একটি তালিকা এবং তার নচি একটি টোটাল লাইন দেখানোর পুনর্ব্যবহারযোগ্য উইজটে — আয় ও কর্তন উভয় সেকশনে ব্যবহৃত।

Widget build(BuildContext context)

AppCard-এর ভেতরে প্রতিটি line-এর লবেলে ও Fmt.money(amount) সারতি সারতি দেখায়, তারপর একটি Divider, তারপর totalLabel ও মোট পরিমাণ (negative true হলে "-" প্রফিক্স ও লাল রঙে, নয়তো সবুজে) দেখায়।

done

ফাইল: lib/features/performance/performance_models.dart

সংক্ষিপ্ত বিবরণ: পারফরম্যান্স রিভিউ (কর্মী মূল্যায়ন) মডেলের ডটো মডেলে, পারমিশন কোড, স্টেজে কনস্ট্যান্ট এবং জেনেরেট করা রিভিউ ইভেন্ট লজিক এখানে সংজ্ঞায়িত। এতে রিভিউ, KPI (উদ্দেশ্যমূলক পরিমাপ) এবং সংযুক্তি (attachment)-এর জন্য আলাদা আলাদা মডেল ক্লাস আছে।

ক্লাস: PerformancePermissions

পারফরম্যান্স মডেলের তিনটি পারমিশন কোড ধরে রাখা abstract final class। view মানে "অন্যের রিভিউ দেখতে পারা" — নিজের রিভিউ দেখার জন্য এই পারমিশন লাগে না, কারণ ব্যাকগ্রাউন্ডে guardOwnReviewAccess নিজের রেকর্ড সবসময় অনুমতি দেয়; তাই অ্যাপ শুধু টিম-লিস্টে এই চেক বসায়, ব্যক্তিগত ট্যাবে নয়। update স্কোর সম্পাদনা, স্টেজে অগ্রসর করা ও চূড়ান্ত করার জন্য, আর create নতুন রিভিউ খোলার জন্য — এই দুটি আলাদা কোড।

static const view

'PERFORMANCE_VIEW' স্ট্রিং ধারণ করে। টিম ট্যাব এবং টিমের রিভিউ তালিকা দেখানোর গটে হিসেবে ব্যবহৃত হয়।

static const update

'PERFORMANCE_UPDATE' স্ট্রিং। স্কোর এডটি, স্টেজে অ্যাডভান্স এবং ফাইনলাইজ করার অ্যাকশন বাটনগুলো দেখানোর শর্ত।

static const create

'PERFORMANCE_CREATE' স্ট্রিং। নতুন রিভিউ শুরু করার ফ্লোটিং অ্যাকশন বাটন দেখানোর শর্ত।

ক্লাস: PerformanceStage

রিভিউ যে ক্রমানুসারে স্টেজগুলোর মধ্য দিয়ে যায়, তার কনস্ট্যান্ট ও লবেল-ম্যাপ ধরে রাখা abstract final class। পাঁচটি স্টেজ: সেলফ-অ্যাসেসমেন্ট, ম্যানেজার রিভিউ, এইচআর অ্যাপ্রুভাল, ফাইনাল অ্যাপ্রুভাল এবং কমপ্লিটেড।

static const selfAssessment / managerReview / hrApproval / finalApproval / completed

প্রতিটি স্টেজে ব্যাকগ্রাউন্ড-ম্যাচিং স্ট্রিং কোড (যেমন 'SELF_ASSESSMENT')।

static const ordered

পাঁচটি স্টেজ কোডের ক্রমবর্ধক লিস্ট — stageIndex এবং _StageBar উইজেটে এই তালিকায় বর্তমান স্টেজের অবস্থান বের করতে ব্যবহার করে।

static const labels

প্রতিটি স্টেজ কোড থেকে মানুষের পড়ার-উপযোগী লবেলে ম্যাপ (যেমন 'SELF_ASSESSMENT' 'Self assessment')।

ক্লাস: ReviewScore

একটি স্কোর করা মাত্রা (dimension) — লবেলে ও ঐচ্ছিক পূর্ণসংখ্যা মান ধারণকারী সাধারণ ইমিউটেবল ডটো ক্লাস।

`const ReviewScore(this.label, this.value)`

পজিশনাল কনস্ট্রাক্টর, লবেলে স্ট্রিং ও `int?` মান নিয়ে।

ক্লাস: PerformanceReview

একটি সম্পূর্ণ পারফরম্যান্স রিভিউ রেকর্ড উপস্থাপন করে — কর্মী তথ্য, নয়টি স্কোর করা যোগ্যতা, লিখিত অংশ, সুপারশিসমূহ এবং AI-উৎপন্ন সারাংশসহ। এটি স্ক্রিনি-লভেলে প্রদর্শনের জন্য বেশ কিছু ডেরাইভড গটোর সরবরাহ করে।

`String get personLabel`

`employeeName` খালি না হলে ট্রিম করে ফেরত দেয়, নাহলে `'Employee # $employeeId'` ফরম্যাটে ফলব্যাক করে।

`String get stageLabel`

`PerformanceStage.labels` ম্যাপ থেকে বর্তমান স্টেজে লবেলে খোঁজ; না পলে আন্ডারস্কোরকে স্পেসে বদলে কাঁচা কডেড ফেরত দেয়।

`int get stageIndex`

`PerformanceStage.ordered`-এ বর্তমান স্টেজে ইনডেক্স খোঁজ; না পলে ০ ফেরত দেয়। `_StageBar` এটি দিয়ে কতগুলো ডট পূরণ করবে তা ঠিকি করে।

`bool get isComplete`

স্টেজে `'COMPLETED'`-এর সমান কনি তা যাচাই করে।

`bool get canAdvance`

রিভিউ ফাইনলাইজড নয় এবং সম্পূর্ণও হয়নি — এমন হলে সত্য; স্টেজে অগ্রসর করার বাটন দেখানোর শর্ত।

`bool get canFinalise`

ফাইনলাইজড নয় কিন্তু সম্পূর্ণ (শেষ স্টেজে পৌঁছেছে) — এমন হলে সত্য; ফাইনলাইজ বাটনের শর্ত।

`String? get period`

`reviewPeriodStart` ও `reviewPeriodEnd` উভয়ই `null` হলে `null`; নাহলে `'$start $end'` ফরম্যাটে (অনুপস্থিতি অংশ '?' দিয়ে) ফেরত দেয়।

`List<ReviewScore> get scores`

নয়টি স্কোর করা মাত্রা (Work quality, Productivity, Communication ইত্যাদি) ফর্মের ক্রম অনুযায়ী `ReviewScore` অবজেক্টেরে লিস্ট বানায়। প্রতিটি ঐচ্ছিক (nullable)।

`List<ReviewScore> get filledScores`

`scores`-এর মধ্যে থেকে শুধু যোগ্যতার `value != null`, সগুলো ফিল্টার করে একটি গ্রোয়েবল-নয় লিস্ট ফেরত দেয়।

`bool get hasScores`

`filledScores` খালি নয় কনি তা বলে; ডিটাইল স্ক্রিনি স্কোর সেকশন দেখাবে কনি তার শর্ত।

List<({String title, String body})> get narrative

লখিত ছয়টি সিকেশন (Strengths, Areas to improve, Goals, Training, Recognition, Comments) একত্র করে; ফাঁকা বা ট্রিমি করলে খালি হয়ে যায় এমন সিকেশন বাদ দিয়ে শুধু non-empty শিরোনাম-বডি রেকর্ডের লিস্ট ফেরত দেয়, যাতে খালি ইডেং দেখা না যায়।

factory PerformanceReview.fromJson(Map<String, dynamic> json)

API থেকে আসা JSON ম্যাপকে PerformanceReview ইনস্ট্যান্সে রূপান্তর করে। প্রতিটি সংখ্যাসূচক ফিল্ড num? থেকে int/double-এ নরিপদে কাস্ট করা হয়, আর অনুপস্থিতি id/stage/finalised-এর জন্য যথাক্রমে 0, selfAssessment, ও false ডিফল্ট বসে।

টপ-লভেলে কনস্ট্যান্ট: reviewCompetencies

({String key, String label}) রেকর্ডের একটি লিস্ট, নয়টি যোগ্যতার DTO-ফিল্ড-নাম ও প্রদর্শন-লবেলে জোড়া ধরে রাখে (যেমন scoreWorkQuality 'Quality of work')। প্রতিটি DTO-তে ১-১০ স্কেলেরে হলও সারভার-সাইড ভ্যালিডেশন নই বলে রেঞ্জ-নিয়ন্ত্রণে দায়িত্ব ফর্মেরে।

ক্লাস: PerformanceReviewRequest

POST /hr/performance (তৈরি) এবং PATCH /hr/performance/{id} (আপডেট) — উভয়ের জন্য একই রকিয়োস্ট ক্লাস, কারণ ব্যাকএন্ড একই DTO ব্যবহার করে। আপডেট পাথ প্রকৃত স্পার্স-প্যাচ: প্রতিটি ফিল্ড null-চকে করে বসানো হয়, তাই বাদ দেওয়া কী মান "অপরিবর্তিত রাখো"। employeeId ও পরিচিৎ শুধু create-এ অর্থবহ। overallScore কখনো পাঠানো হয় না — সারভার প্রতাবির লখার সময় উপস্থিতি যোগ্যতাগুলোর গড় করে পুনর্গণনা করে।

factory PerformanceReviewRequest.from(PerformanceReview review)

একটি বিদ্যমান PerformanceReview থেকে এডটি-ফর্ম প্র-ফিলি করার জন্য একটি রকিয়োস্ট বানায় — নয়টি স্কোর ও সমস্ত লখিত/সুপারিশ ফিল্ড কপিকরে।

Map<String, dynamic> toJson()

রকিয়োস্ট বডি বানায়। ভতরে একটি লোকাল clean() হলেপার স্ট্রিং ট্রিমি করে খালিহলে null বানায়। শুধু non-null মান-সহ কী-গুলো ম্যাপে যুক্ত হয় (স্পার্স প্যাচ আচরণ বজায় রাখতে); স্কোরের ক্ষত্রে un-scored যোগ্যতা বাদ দেওয়া হয়, null হিসেবে পাঠানো হয় না।

ক্লাস: PerformanceKpis

রভিউর ময়োদকালরে জন্য উদ্দেশ্যমূলক পরিসংখ্যান (attendance, leave, task, project ইত্যাদি)। রভিউতে কছুই স্টোর করা থাকে না — প্রতাবির রভি-এর সময় অন্যান্য মডিল থেকে পুনর্গণনা করা হয়, তাই তথ্য সবসময় হালনাগাদ। কছু ফিল্ড null-যোগ্য এবং null মান "শূন্য" নয়, বরং "রেকর্ড নই"।

bool get hasAnything

workingDaysRecorded, tasksCompleted, projectsCompleted, বা leaveDaysTaken-এর যকোনোটি ০-এর বেশিহলে সত্য — এটি যাচাই করে যে ময়োদে দেখানোর মতো কছু আছে কনি।

factory PerformanceKpis.fromJson(Map<String, dynamic> json)

JSON থেকে ইনস্ট্যান্স বানায়। ভতরে একটি লোকাল count() হলেপার পূর্ণসংখ্যা ফিল্ডগুলোকে নরিপদে পার্স করে (অনুপস্থিতি হল ০)। attendancePercent ও customerSatisfaction double?-এ কাস্ট হয়, ডিফল্ট ছাড়া — তাই সত্যিকারের null থেকে যায়।

ক্লাস: ReviewAttachment

রভিউর সাথে সংযুক্ত একটি ফাইলকে প্রতিনিধিত্ব করে — ফাইলের নাম, URL, টাইপ, আকার, লেবেল ও কী আপলোড করেছে তার তথ্যসহ।

factory ReviewAttachment.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে ইনস্ট্যান্স বানায়; id ডিফল্ট ০, fileName/fileName ডিফল্ট খালি স্ট্রিং, বাকি ফিল্ড nullable।

ক্লাস: ReviewAttachmentRequest

একটি রভিউর বর্ণনায় ইতিমধ্যে আপলোড হওয়া ফাইল রেকর্ড করার রকিউয়েস্ট। ফাইলের বাইট নজরে এখানে থাকে না — সেগুলো আগেই POST /upload-এ যায়; এই ক্লাস শুধু ফলাফল-URL রেকর্ড করে।

Map<String, dynamic> toJson()

fileName ও fileName বাধ্যতামূলকভাবে অন্তর্ভুক্ত করে; fileType, fileSizeBytes null না হলে যুক্ত হয়; label ট্রিম করে non-empty হলে যুক্ত হয়।

ফাইল: lib/features/performance/performance_repository.dart

সংক্ষিপ্ত ববিরণ: পারফরম্যান্স রভিউ API-এর সাথে যোগাযোগকারী রপিজিটরি ক্লাস এবং Riverpod প্রোভাইডার/কন্ট্রোলার এখানে সংজ্ঞায়িত। এতে রভিউ তৈরি/আপডেট/অ্যাডভান্স/ফাইনলাইজ, KPI আনা, সংযুক্ত ব্যবস্থাপনা এবং টিম বনাম ব্যক্তিগত রভিউ তালিকার জন্য পৃথক স্টেট ম্যানেজমেন্ট রয়েছে।

ক্লাস: PerformanceRepository

/hr/performance এন্ডপয়েন্টের সাথে সরাসরি যোগাযোগকারী ক্লাস। এতে স্কোর, লিখিত অংশ ও সুপারিশ লেখার সমস্ত সুবিধা ওয়েব অ্যাপ থেকে ফিচার-প্যারামিটারের জন্য আনা হয়েছে, তবে সংযুক্ত-আপলোড (ফাইল-পিকার প্রয়োজন) এবং goals ফিল্ড (হাতে JSON অ্যারে বানানো লাগে) — এই দুটি এখনো অনুপস্থিত।

PerformanceRepository(this._api)

কনস্ট্রাক্টর, একটি ApiClient ইনজেক্ট করে।

Future<PagedResponse<PerformanceReview>> all({int page = 0, int size = 20})

/hr/performance-এ পজেড GET কল করে সব রভিউর একটি পাতা ফেরত দেয়।

Future<List<PerformanceReview>> forEmployee(int employeeId)

একজন কর্মীর সব রভিউ আনে। PERFORMANCE_VIEW ছাড়াই অ্যাক্সেসযোগ্য যদি employeeId কলকারীর নজরে হয় — ব্যাকএন্ডের guardOwnReviewAccess ইচ্ছাকৃতভাবে এটি অনুমোদন করে। এন্ডপয়েন্ট Page<...> ফেরত দেয় বলে ভুলবশত bare list হিসেবে পড়লে ক্র্যাশ হতো; তাই এখানে পজে হিসেবে (page 0, size 50, প্যাজনিশেন ছাড়াই) পড়ে শুধু .content ফেরত দেওয়া হয়।

Future<PerformanceReview> review(int id)

একক রভিউ আইডি দিয়ে GET করে।

Future<PerformanceReview> create(PerformanceReviewRequest request)

নতুন রিভিউ খোলে (POST)। রিভিউয়ার সেশন থেকে নির্ধারিত হয়, রকিওয়েস্ট বডি থেকে নয় — সাইন-ইন করা ইউজারের কর্মী রেকর্ড না থাকলে সার্ভার "Employee profile not found" ত্রুটি দিয়ে।

Future<PerformanceReview> update(int id, PerformanceReviewRequest request)

স্কোর ও লিখিত অংশ এডিট করে (PATCH)। রিভিউ ফাইনাইজড হলে সার্ভার "Cannot edit a finalised review" বলে প্রত্যাখ্যান করে; তাই UI ফাইনাইজড রিভিউতে এডিট অ্যাকশন লুকিয়ে রাখে।

Future<PerformanceReview> advance(int id)

রিভিউকে পরবর্তী স্টেজে সরায় (POST .../advance)। কোন স্টেজে যাবে তা ব্যাকএন্ড ঠিকি করে — লাফ দেওয়া বা পছন্দে যাওয়ার উপায় নেই, তাই UI একটি বাটনই দেয়।

Future<PerformanceKpis> kpis(int employeeId, String from, String to)

একটি মিয়োদরে উদ্দেশ্যমূলক পরিসংখ্যান আনে। from/to উভয়ই আবশ্যিক কৌয়ের-প্যারামিটার, কোনো ডিফল্ট উইন্ডো নেই।

Future<PerformanceReview> summarise(int id)

.../summary এন্ডপয়েন্ট GET করে — এই কলটিই ব্যাকএন্ডকে AI-সারাংশ বানাতে বাধ্য করে, ফরেক আসে যা যা সার্ভার নির্ণয় করতে পরেছে তা পূরণ করা একই রিভিউ।

Future<List<ReviewAttachment>> attachments(int id)

একটি রিভিউর সব সংযুক্তি তালিকা আনে।

Future<ReviewAttachment> addAttachment(int id, ReviewAttachmentRequest request)

আগেই আপলোড হওয়া একটি ফাইল রিভিউর বপিরীতে রেকর্ড করে (POST)।

Future<void> deleteAttachment(int id, int attachmentId)

একটি সংযুক্তি মুছে ফেলে; সার্ভার প্লাইন-টেক্সট উত্তর দেয় বলে _api.deleteText ব্যবহার হয়।

Future<PerformanceReview> finalise(int id)

রিভিউ চূড়ান্তভাবে সাইন-অফ করে (PATCH .../finalise) — অপরিবর্তনীয়।

টপ-লেভেলে প্রোভাইডার: performanceRepositoryProvider

ApiClient থেকে PerformanceRepository তৈরি করা একটি সাধারণ Riverpod Provider।

ক্লাস: TeamReviewsController

সবার রিভিউ (টমি ট্যাব) — PERFORMANCE_VIEW প্রয়োজন।

AsyncNotifier<PagedState<PerformanceReview>> এক্সটেন্ড করে ও

PagedLoader<PerformanceReview> মক্সইন করে পজেনিশেন লজিকি পায়।

Future<PagedState<PerformanceReview>> build()

currentUserProvider ওয়াচ করে (রিলগনিং রফ্রেশে ঘটাতো) এবং loadFirstPage() চালিয়ে প্রথম পজে লোড করে।

Future<PagedResponse<PerformanceReview>> fetchPage(int page)

PagedLoader মক্সইনের জন্য একটি পজে আনার ওভাররাইড —

performanceRepositoryProvider.all(page: page) কল করে।

```
void apply(PerformanceReview updated)
```

স্টেটে ভেতর আইডি মিলিয়ে একটি রিভিউকে হালনাগাদ সংস্করণ দিয়ে প্রতস্থাপন করে।

```
Future<PerformanceReview> create(PerformanceReviewRequest request)
```

রপিজটির `create` কল করে, তারপর তালিকা রিফ্রেশ করে এবং `myReviewsProvider`-কে ইনভ্যালিডেট করে (কারণ যাকে নিয়ে রিভিউ, সে হয়তো নজিরে তালিকা দেখেছে)।

```
Future<PerformanceReview> updateItem(int id, PerformanceReviewRequest request)
```

রপিজটির `update` কল করে, `apply()` দিয়ে লোকাল স্টেটে হালনাগাদ করে ও `myReviewsProvider` ইনভ্যালিডেট করে। সারভারের সংস্করণ ফেরত দিয়ে যাতে ডিটাইল স্ক্রিনি পুনর্গণতি `overallScore` গ্রহণ করতে পারে — ক্লায়েন্টের নজিরে হিসাব করার দরকার নেই।

উপ-লেভেলে প্রোভাইডার: `teamReviewsProvider`

`TeamReviewsController`-এর `AsyncNotifierProvider`।

উপ-লেভেলে প্রোভাইডার: `myReviewsProvider`

সাইন-ইন করা ব্যক্তির নজিরে রিভিউর `FutureProvider<List<PerformanceReview>>`। প্রথমে `myEmployeeProvider` থেকে কর্মী রেকর্ড আনবে; কর্মী রেকর্ড না থাকলে (যেমন কোম্পানির মালিকের) খালি লিস্ট ফেরত দিয়ে, ত্রুটি না ছুঁড়ে — স্ক্রিনি এটি ব্যাখ্যা করে দেখায়।

উপ-লেভেলে প্রোভাইডার: `reviewKpisProvider`

`((int employeeId, String from, String to))` কী দিয়ে ফ্যামলি করা `FutureProvider.autoDispose` — একজন কর্মীর একটি নির্দিষ্ট সময়কাল পরিসংখ্যান আনবে। তিনটি ফিল্ড দিয়েই কী করা কারণ এন্ডপয়েন্ট তিনটিই আবশ্যিক করে এবং ভিন্ন উইন্ডোতে ফলাফল ভিন্ন হয়।

উপ-লেভেলে প্রোভাইডার: `reviewAttachmentsProvider`

রিভিউ আইডি দিয়ে ফ্যামলি করা `FutureProvider.autoDispose` — একটি রিভিউর সংযুক্ত তালিকা আনবে।

ফাইল: `lib/features/performance/performance_screen.dart`

সংক্ষিপ্ত ববিরণ: পারফরম্যান্স রিভিউ মডউলরে প্রধান স্ক্রিনি — দুটি ট্যাব ("My reviews" ও "Team") এবং প্রতিটি রিভিউর একটি সারি (row) উইজেটে, স্কোর-পলি ও স্টেজ-বার সহ। ব্যক্তিগত ট্যাব কখনো গটে করা হয় না, শুধু টমি ট্যাব `PERFORMANCE_VIEW` পারমিশন চায়।

ক্লাস: `PerformanceScreen`

পুরো পারফরম্যান্স স্ক্রিনির রুট উইজেটে — `TabBar`, `FloatingActionButton` (নতুন রিভিউ) এবং ট্যাব-ভিউ নিয়ন্ত্রণ করে। `ConsumerWidget`।

```
Widget build(BuildContext context, WidgetRef ref)
```

`PermissionControllerProvider` থেকে `canSeeTeam` ও `canCreate` নির্ণয় করে। `canSeeTeam` সত্য হলে 'Team' ট্যাব তালিকায় যুক্ত হয়, নাহলে শুধু 'My reviews' দেখায় এবং `TabBar` উহ্য থাকে। `canCreate` সত্য হলে উভয় ট্যাবের ওপর একটি ভাগাভাগি FAB দেখায়, যটা নতুন রিভিউ শীট খোলে।

ক্লাস: `_MyReviewsTab`

নজিরে রভিউ দখানোর ট্যাব; `myReviewsProvider` ওয়াচ করে। `ConsumerWidget`।

`Widget build(BuildContext context, WidgetRef ref)`

`RefreshIndicator` দিয়ে মোড়ানো — টানলে `myReviewsProvider` ইনভ্যালিডেট হয়।

লোডিং/ত্রুটি/খালি/ডিটো — চারটি অবস্থাই `async.when` দিয়ে হ্যান্ডল করে। খালি অবস্থায় `EmptyState` দখানো হয়, যা "রভিউ নেই" ও "কর্মী রেকর্ড নেই" — উভয় পরিস্থিতিতেই একইভাবে কভার করে, কারণ পাঠকের দৃষ্টিতে দুটোই একই রকম দখায়।

ক্লাস: `_TeamTab`

টমিরে সব রভিউ দখানোর ট্যাব; `teamReviewsProvider` ব্যবহার করে সাধারণ `PagedListView` উইজেটে মধ্যমে পেজিনেটেডে তালিকা দখায়। `ConsumerWidget`।

`Widget build(BuildContext context, WidgetRef ref)`

`PagedListView<PerformanceReview>`-কে রফিরশে/লোড-মোর কলব্যাক ও খালি/ত্রুটি বার্তার সাথে কনফিগার করে রটার্ন করে; প্রতিটি আইটেমে `_ReviewRow` দিয়ে রেন্ডার হয়।

ক্লাস: `_ReviewRow`

একটি রভিউর তালিকা-আইটেমে কার্ড — নাম/পরিষিড, রভিউয়ার বা স্টজে, স্কোর-পলি ও স্টজে-বার দখায়, ট্যাপ করলে ডিটাইল স্ক্রিনি খোলে। `StatelessWidget`।

`Widget build(BuildContext context)`

`showPerson` সত্য হলে ব্যক্তির নাম উপরে ও পরিষিড নচে দখায় (টমি ট্যাবের জন্য); মথিা হলে পরিষিড উপরে ও রভিউয়ার/স্টজে নচে দখায় (নজিরে ট্যাবের জন্য, নাম দখানোর দরকার নেই)।

`overallScore` উপস্থিতি থাকলে `_ScorePill` দখায়। `InkWell`-এর মধ্যমে ট্যাপে `ReviewDetailScreen.open` কল হয়।

ক্লাস: `_ScorePill`

সামগ্রিক স্কোর একটি রঙনি পলি দখানোর উইজেটে — একটি গ্রডিয়েন্টের বদলে তনিটি ব্যান্ড (৮+ সবুজ/সাফল্য, ৫+ ব্র্যান্ড-কালি, নচে হলুদ/সতর্কতা) ব্যবহার করা হয়, কারণ স্কোর একটি রায হিসেবে পড়া হয় আর দশটিশিডের চয়ে তনিটি টোন বশে বিলে। `StatelessWidget`।

`Widget build(BuildContext context)`

স্কোরের মান অনুযায়ী `tone` রঙ নির্ধারণ করে এবং সেই রঙের হালকা ব্যাকগ্রাউন্ডে এক-দশমিক-স্থান পর্যন্ত স্কোর টেক্সট দখায়।

ক্লাস: `_StageBar`

রভিউ কোথায় পৌঁছেছে তা পাঁচটি বিন্দু (ডেট) হিসেবে দখানো উইজেটে, চপি নয় — কারণ স্টজে একটি ক্রমের মধ্য অবস্থান, চপি সটো হারিয়ে ফলে। `StatelessWidget`।

`Widget build(BuildContext context)`

`PerformanceStage.ordered`-এর প্রতিটি স্টজেই জন্য একটি ডিট এবং (প্রথমটি ছাড়া) মাঝে একটি সংযোগকারী রখো আঁকে; বর্তমান স্টজে পর্যন্ত সবকিছু ব্র্যান্ড-রঙে, বাকটি হালকা ব্রডার-রঙে হাইলাইট হয়। শেষে ফাইনলাইজড হলে 'Finalised' (সাফল্য-রঙে), নাহলে স্টজে লবেলে (সকেন্ডার-রঙে) টেক্সট দখায়।

ফাইল: lib/features/performance/review_detail_screen.dart

সংক্ষিপ্ত ববিরণ: একটি একক পারফরম্যান্স রিভিউর সম্পূর্ণ বিস্তারিত স্ক্রিনি — হডোর, স্টজে-অ্যাকশন বাটন, স্কোর, গোল-প্রগ্রেসে, লিখিত সেকশন, AI সারাংশ, KPI, সংযুক্তি এবং ইতিহাস দেখায়। এডিট, স্টজে-অ্যাডভান্স, ফাইনলাইজ ও সারাংশ-তরৈর অ্যাকশনও এখান থেকেই ট্রিগার হয়।

ক্লাস: ReviewDetailScreen

রিভিউ ডিটেইল স্ক্রিনিরে এন্ট্রি পিয়ন্টে উইজেটে। `ConsumerStatefulWidget`

`static void open(BuildContext context, {required PerformanceReview review})`

একটি `MaterialPageRoute` দিয়ে `ReviewDetailScreen`-কে পুশ করে নেভিগেটে করে।

`ConsumerState<ReviewDetailScreen> createState()`

`_ReviewDetailScreenState` স্টেটে অবজেক্ট তরৈকিরে।

ক্লাস: _ReviewDetailScreenState

স্ক্রিনিরে ভেতরে স্টেটে — লোকাল কপি `_review` (এডিট/অ্যাকশনের পর আপডেটে হয়) ও ব্যস্ততা (`_busy`) ফ্ল্যাগ পরিচালনা করে।

`Future<void> _edit(PerformanceReview review)`

`showEditReviewSheet` খোলে; ফলাফল `null` না হলে ও উইজেটে এখনো মাউন্টেডে হলে `_review`-কে সারভারের হালনাগাদ সংস্করণ দিয়ে সটে করে (সটে শীট নজিহে তালিকাকে জানায়; এই স্ক্রিনি শুধু নজিরে কপি রাখতে)।

`Future<void> _run(Future<PerformanceReview> Function() action, String success)`

স্টজে-অ্যাডভান্স, ফাইনলাইজ ও সারাংশ-তরৈ — এই তিনটি অ্যাকশনের জন্য সাধারণ রানার। `_busy` সটে করে, অ্যাকশন চালায়, `teamReviewsProvider`-এ ফলাফল প্রয়োগ করে (`apply`), `myReviewsProvider` ইনভ্যালিডেটে করে, লোকাল স্টেটে আপডেটে করে এবং সফলতা/ত্রুটি বার্তা সন্ধ্যাকবারে দেখায়।

`ApiException` ও অন্যান্য ব্যতিক্রম আলাদাভাবে ধরা হয়।

`Future<void> _finalise()`

প্রথমত একটি নিশ্চিতকরণ ডায়ালগ দেখায় ("Finalise this review?", অপরবর্তীতীয়তার সতর্কবার্তাসহ)। নিশ্চিত হলে `_run` দিয়ে `finalise` রপিজিটরিকল চালায়।

`Widget build(BuildContext context)`

পুরো স্ক্রিনি লে-আউট করে: `_Header`, শর্তসাপেক্ষে অ্যাডভান্স/ফাইনলাইজ বাটন (`canUpdate` ও `canAdvance/canFinalise` দুটোই সত্য হলে), `_Scores` (স্কোর থাকলে), `_Goals` (গোল-শতাংশ থাকলে), `narrative`-এর প্রতটি সেকশন, AI-সারাংশ বিভাগ (না থাকলে "ড্রাফট করুন" বাটন, থাকলে লবেলেসহ প্রদর্শন), `_Kpis`, `_Attachments` ও `_Trail`। এডিট আইকন শুধু `canUpdate` সত্য ও রিভিউ ফাইনলাইজড না হলে অ্যাপবারে দেখায়।

ক্লাস: _Kpis

রিভিউর ম্যোদরে উদ্দেশ্যমূলক পরিসংখ্যান দেখানো সেকশন — অ্যাটনেড্যান্স, দিন উপস্থিতি/অনুপস্থিতি, দেরি, ছুটি, কাজ/প্রজেক্ট সম্পন্ন ও ক্লায়েন্ট রটেই। `ConsumerWidget`

Widget build(BuildContext context, WidgetRef ref)

employeeId, from, to — এই তিনটির যেকোনোটি null হলে কছিই রেন্ডার করে না (SizedBox.shrink())। নাহলে reviewKpisProvider ওয়াচ করে এবং লোডিং/ত্রুটি/খালি/ডটো অবস্থায় যথাযথ UI দেখায়; প্রতিটি পরিসংখ্যান _Kpi সারি দিয়ে দেখানো হয়, শূন্য-মানের কছি ফল্ড (absent days, late arrivals) কবেল ০-এর বশেহিলে দেখানো হয়।

ক্লাস: _Kpi

একটি একক লবেলে-মান জোড়া সারি হিসেবে দেখানো উইজটে। StatelessWidget।

Widget build(BuildContext context)

বাম দিকে লবেলে, ডান দিকে মান — মান null হলে Fmt.dash ('-') দেখায় এবং হালকা রঙে, নাহলে সাধারণ টেক্সট রঙে বোল্ড দেখায়।

ক্লাস: _Attachments

রভিউর সাথে সংযুক্ত ফাইলগুলোর তালিকা, যোগ ও অপসারণের সুবিধাসহ। ConsumerStatefulWidget।

ConsumerState<_Attachments> createState()

_AttachmentsState তৈরি করে।

ক্লাস: _AttachmentsState

_Attachments-এর স্টেটে — ব্যস্ততা ফ্ল্যাগ ও ফাইল-যোগ/অপসারণের লজিক ধারণ করে।

Future<void> _add()

pickAttachment দিয়ে ব্যবহারকারীর কাছ থেকে ফাইল বেছে নেয়। তারপর দুই ধাপে কাজ করে (ব্যাকএন্ডের ইচ্ছা অনুযায়ী): প্রথমে apiClientProvider.uploadDocument দিয়ে সাধারণ আপলোড এন্ডপয়েন্টে ফাইল পাঠায়, তারপর যে URL ফেরত আসে সেটাই addAttachment দিয়ে রভিউর বপিরীতে রেকর্ড করে। সফল/ব্যর্থ হলে সন্ধ্যাকবার দেখায় এবং reviewAttachmentsProvider ইনভ্যালিডেট করে।

Future<void> _remove(ReviewAttachment attachment)

confirmAction দিয়ে নিশ্চিতকরণ চায়, তারপর deleteAttachment কল করে এবং সফল হলে প্রোভাইডার ইনভ্যালিডেট করে।

Widget build(BuildContext context)

SectionHeader-এ 'Attach' বাটন (শুধু editable && !_busy হলে) এবং AppCard-এর ভেতর ব্যস্ত/লোডিং/ত্রুটি/খালি/ডটো অবস্থার জন্য যথাযথ UI দেখায়। প্রতিটি সংযুক্তি সারিতে আইকন, নাম/লবেলে, আপলোডকারীর নাম (থাকলে) ও অপসারণ বাটন (editable হলে) থাকে।

ক্লাস: _Header

রভিউর শীর্ষ কার্ড — পরিষিড, রভিউয়ার, সামগ্রিক স্কোর, স্টেজে চপি, পারফরম্যান্স লভেলে ও প্রমোশন সুপারিশ/প্রস্তুতি দেখায়। StatelessWidget।

Widget build(BuildContext context)

বহির্নির্ conditional অংশ সাজায়: পরিষিড/রভিউয়ার (বাম), overallScore থাকলে বড় সংখ্যায় ডানে; নচি StatusChip (ফাইনালাইজড হলে 'COMPLETED'/'Finalised', নাহলে স্টেজে কোড/লবেলে) ও পারফরম্যান্স লভেলে; সবশেষে প্রমোশন-সংক্রান্ত তথ্য একটি ডিভাইডারের নচি (promotionRecommendation বা promotionReadiness যেকোনোট থাকলে)।

ক্লাস: _Scores

নয়টি ভিউ স্কোরের প্রগ্রেস-বার আকারে প্রদর্শন। `StatelessWidget`।

`Widget build(BuildContext context)`

`review.filledScores` লুপ করে প্রতিটি জন্য লবেলে, `LinearProgressIndicator` (মান/১০, ক্ল্যাম্প করা ০-১) ও সংখ্যাসূচক মান — একটি সিরিজে দেখায়।

ক্লাস: _Goals

গোল-সম্পন্নতার শতাংশ দেখানো একটি সাধারণ কার্ড। `StatelessWidget`।

`Widget build(BuildContext context)`

লবেলে ও শতাংশ পাশাপাশি দেখায়; রঙ শতাংশের ব্যান্ড অনুযায়ী (৮০+ সবুজ, ৫০+ ব্র্যান্ড-কালি, নীচে হলুদ) নির্ধারণিত হয়।

ক্লাস: _Trail

রভিউর অডিও ইতিহাস — শুরুর তারিখ-সময়। `StatelessWidget`।

`Widget build(BuildContext context)`

`createdAt` null হলে কিছুই রেন্ডার করে না। নাহলে একটি `SectionHeader` ও `AppCard`-এ "Started ..." টেক্সট (`Fmt.dateTime` দিয়ে ফরম্যাট করা) দেখায়।

ফাইল: lib/features/performance/review_form_sheet.dart

সংক্ষিপ্ত ববিরণ: নতুন রভিউ খোলা বা বদ্যমান রভিউ সম্পাদনার জন্য একটি মডাল বটম-শীট ফর্ম। কর্মী নির্বাচন, পরিষিড, নয়টি স্কোর, লিখিত সেকশন এবং সুপারিশগুলো একটিমাত্র শীটে সাজানো।

টপ-লেভেলে ফাংশন: showNewReviewSheet

`dart Future<void> showNewReviewSheet(BuildContext context)` `নতুন রভিউ খোলার মডাল বটম-শীট` দেখায় (`_ReviewFormSheet` বদ্যমান রভিউ ছাড়া)।

টপ-লেভেলে ফাংশন: showEditReviewSheet

`dart Future<PerformanceReview?> showEditReviewSheet(BuildContext context, PerformanceReview review)` `বদ্যমান রভিউ সম্পাদনার শীট` দেখায়। ব্যবহারকারী বাতলি করলে null ফেরত দেয়; সফল হলে সার্ভারের হালনাগাদ সংস্করণ ফেরত দেয়, কারণ ডিটেইল স্ক্রিনি নজিরে কপি রাখা এবং `overallScore` সার্ভার-সাইড পুনর্গণতি — তাই নজিরে হিসাব করার দরকার নেই।

ক্লাস: _ReviewFormSheet

ফর্ম শীটের রুট উইজেট। `existing` null হলে "নতুন" মোড, নাহলে "এডিট" মোড।

`ConsumerStatefulWidget`।

`ConsumerState<_ReviewFormSheet> createState()`

`_ReviewFormSheetState` তৈরি করে।

ক্লাস: `_ReviewFormSheetState`

ফর্মের সম্পূর্ণ স্টেটে — আউটটেক্সট কন্ট্রোলার, নয়টি স্কোরের ম্যাপ, কর্মী নির্বাচন, পরিষিড তারিখ, গোল-শতাংশ ও সাবমিট/ত্রুটি স্টেটে পরিচালনা করে।

`bool get _isEdit`

`widget.existing != null` কনি তা বলে — পুরো ফর্মের আচরণ (কোন ফিল্ড দেখাবে, কী বাটন লবেলে, কোন রপিজিটরিকল) এটার ওপর নির্ভর করে।

`void initState()`

`widget.existing` থেকে (বা নতুন হলে খালি `PerformanceReviewRequest` থেকে) একটি "seed" রকিওয়েস্ট বানায় এবং সব কন্ট্রোলার/স্কোর/গোল-শতাংশ প্র-ফিলি করে। নতুন রভিউর জন্য ডফিল্ট পরিষিড হিসেবে সদ্য শেষ হওয়া কোয়ারটার (তনি মাস) স্টেট করে, কারণ বশেরিভাগ রভিউ আসলে স্টো নথিই হয়।

`void dispose()`

আউটটি `TextEditingController` disposed করে।

`Future<void> _pickEmployee()`

`EmployeePicker.show` দিয়ে কর্মী বাছাই ডায়ালগ খোল; নির্বাচতি হলে `_employee` স্টেট করে।

`PerformanceReviewRequest _buildRequest()`

বর্তমান ফর্ম-স্টেটে থেকে একটি `PerformanceReviewRequest` বানায়। এডটি মোডে `employeeId`/পরিষিড ফিল্ডগুলো ইচ্ছাকৃতভাবে null রাখা হয় (আপডেটে পাথ এগুলো উপেক্ষা করে); নতুন মোডে সেগুলো ফর্মের মান থেকে নেওয়া হয়।

`Future<void> _submit()`

ফর্ম ভ্যালিডেশন চালায়; নতুন-মোডে অতিরিক্তভাবে কর্মী নির্বাচতি হয়েছে কনি, পরিষিড স্টেট করা আছে কনি এবং শেষে তারিখ শুরুর আগে নয় কনি — এসব ম্যানুয়ালি যাচাই করে। ফোকাস সরিয়ে সাবমিট-অবস্থা স্টেট করে, তারপর `teamReviewsProvider`-এর `create` বা `updateItem` কল করে (মোড অনুযায়ী)। সাফল্যে শীট বন্ধ করে ও সন্ধ্যাকবার দেখায় (এডটি মোডে আপডেটে হওয়া রভিউসহ পপ করে); `ApiException`-এর বার্তা (যেমন "Employee profile not found") সরাসরি দেখায়, অন্য ব্যতক্রমে একটি জনেরেকি বার্তা।

`Widget build(BuildContext context)`

পুরো ফর্ম লে-আউট করে: শিরোনাম ('New review'/'Edit review'), ত্রুটি ব্যানার (থাকলে), (শুধু নতুন-মোডে) কর্মী-পিকার ও দুটি `DateField`, স্কোর সেকেশন (`reviewCompetencies` লুপ করে `_ScoreRow`), লখিতি-অংশে চারটি `TextFormField` (Strengths, Areas for improvement, Goals, Comments), সুপারিশ সেকেশনের চারটি ঐচ্ছকি ফিল্ড ও একটি গোল-শতাংশ ড্রপডাউন, এবং শেষে `LoadingButton`।

ক্লাস: `_ScoreRow`

একটি যোগ্যতার জন্য দশ-পয়েন্ট স্কেলের ড্রপডাউন সারি স্লাইডার বা সংখ্যা-ফিল্ডের বদলে ড্রপডাউন বছে নেওয়া হয়েছে কারণ রঞ্জে নরিদষ্টি, "স্কোর করা হয়নি" অবস্থাটিধিরে রাখা দরকার, আর স্লাইডার স্টো প্রকাশ করতে পারে না। `StatelessWidget`।

`Widget build(BuildContext context)`

বাম দকি লবেলে, ডান দকি একটি `DropdownButtonFormField<int?>` — যেখানে '—' (null) সহ ১ থেকে ১০ পর্যন্ত অপশন থাকে। `enabled` মথিয়া হলে `onChanged` null স্টেট হয়ে ড্রপডাউন নথিক্রয়ি হয়।

ফাইল: lib/features/platform/context_models.dart

সংক্ষিপ্ত ববিরণ: সাপোর্ট স্টাফের "কনটেক্সট সুইচ" ফিচারের রোল-কনস্ট্যান্ট ও ডটো মডলে — সাপোর্ট এজেন্ট কখন কোন গ্রাহক কোম্পানি দেখেছিল তার একটি জিবাবদহিমূলক রেকর্ড, যা কোনও প্রকৃত অ্যাক্সেসে দেয় না।

টপ-লেভেলে কনস্ট্যান্ট: contextSwitchRoles

SupportContextSwitchController-এর write এন্ডপয়েন্টে কে কনটেক্সট সুইচ শুরু/শেষ করতে পারে তার রোল-তালিকা: SUPER_ADMIN, SUPPORT_MANAGER, SUPPORT_AGENT। লক্ষণীয়ভাবে SYSTEM_ADMIN অনুপস্থিতি — ইমপারসোনেশন ফিচার সটোকে অনুমতি দিয়ে কিন্তু এটি দিয়ে না, তাই দুটি ফিচার মিলি থাকলেও রোল-তালিকা শেয়ার করা যায় না।

টপ-লেভেলে কনস্ট্যান্ট: contextSwitchReviewRoles

লাইভ বোর্ড, একজন এজেন্টের ইতিহাস এবং একক রেকর্ড দেখার জন্য রোল-তালিকা: SUPER_ADMIN, SUPPORT_MANAGER। একজন সাপোর্ট এজেন্ট নজিরে সুইচ ঘোষণা করতে পারলেও কারো (এমনকি নজিরেও) ইতিহাস পড়তে পারে না — তাই এই তালিকা contextSwitchRoles-এর চয়ে সংকীর্ণ।

ক্লাস: SupportContextSwitch

একটি রেকর্ড যে সাপোর্ট স্টাফের কটে একজন গ্রাহকের কোম্পানি দেখেছিল। গুরুত্বপূর্ণভাবে, এটি কোনও অ্যাক্সেসই দেয় না — ইমপারসোনেশনের বিপরীতে এখানে কোনও টোকেনে নই, এজেন্টের নিজস্ব সেশন অপরিবর্তিত থাকে এবং টেন্যান্ট এন্ডপয়েন্ট তখনও তাকে প্রত্যাখ্যান করে। এটি একজন কী নিয়ে কাজ করছিল তার একটি ঘোষণা, আর সারাটি জিবাবদহিতির প্রমাণ।

String get companyLabel

viewedCompanyName খালি না হলে ট্রিম করে ফেরত দেয়, নাহলে 'Company #\${viewedCompanyId}'-এ ফলব্যাক করে।

String get agentLabel

supportAgentName খালি না হলে ট্রিম করে ফেরত দেয়, নাহলে 'Agent #\${supportAgentId}'-এ ফলব্যাক করে।

bool get hasPurpose

purpose ট্রিম করার পর খালি নয় কনি তা বলে।

Duration? get elapsed

সুইচটিকিতকষণ চলছে (বা চলছে) তা গণনা করে। switchedInTime পার্স করা না গেলে null। সক্রিয় (stillActive) হলে শেষে সময় হিসেবে DateTime.now() ব্যবহার হয়, নাহলে switchedOutTime পার্স করার চেষ্টা হয় (পার্স না হলে null)। নগেটেভি স্প্যান হলে Duration.zero ফেরত দেয়, যাতো বকিত টাইমস্ট্যাম্প ঋণাত্মক সময় না দেখায়।

String? get elapsedLabel

elapsed-কে কম্প্যাক্ট মানব-পাঠযোগ্য ফরম্যাটে রূপান্তর করে: এক মিনিটের কম হলে 'just now', এক ঘণ্টার কম হলে '\${minutes}m', এক দিনের কম হলে '\${hours}h \${minutes}m' (মিনিট শূন্য হলে শুধু ঘণ্টা), নাহলে '\${days}d'।

`bool get looksForgotten`

সুইচ এখনো সক্রিয় এবং ৮ ঘণ্টা বা তার বেশি সময় ধরে চলছে কনি তা বলে — এত দীর্ঘ সময় ধরে খোলা থাকা একটি সুইচ প্রায় নশ্চিতিভাবেই কটে বন্ধ করতে ভুলে গেছে, চলমান কাজ নয়; আর একটি বাসি "বর্তমানে দেখেছে" সার্নি-থাকার চেষ্টা খারাপ, কারণ সটোকে বশ্বাস করা হয়।

`factory SupportContextSwitch.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে ইনস্ট্যান্স বানায়। `id`, `supportAgentId`, `viewedCompanyId` ডিফল্ট ০, `stillActive` ডিফল্ট `false`; বাকি সব ফিল্ড nullable স্ট্রিং হিসেবে সরাসরি কাস্ট হয়। `ipAddress` ও `userAgent` উভয়ই সার্নি-সাইড রকি়োস্ট থেকে বের করা হয়, ক্লায়েন্ট কখনো পাঠায় না — কারণ রকেডটির উদ্দেশ্যই জবাবদহিতি নশ্চিতি করা, তাই এটি এমন কিছু হওয়া চলবে না যা কটে নজি লেখিত পারে।

ফাইল: `lib/features/platform/context_repository.dart`

সংক্ষিপ্ত ববিরণ: সাপোর্ট এজেন্টরা কাস্টমারের অ্যাকাউন্টের ভেতরে কাজ করার সময় যে "context switch" রকেড রাখা হয়, তার জন্য রপিজিটিও Riverpod প্রোভাইডার/কন্ট্রোলারগুলো এখনে সংজ্ঞায়িত। এন্ডপয়ন্ট প্রফিক্স `/support/context-switches`, যা টকিটি এন্ডপয়ন্টের `/v1/support/...` থেকে আলাদা।

ক্লাস: `ContextSwitchRepository`

সাপোর্ট কন্টেকস্ট সুইচের API কল মোডানো একটি ক্লাস — নতুন সুইচ শুরু করা, শেষ করা, নজিরে/অন্যের সক্রিয় সুইচ দেখা, সবার সক্রিয় সুইচের তালিকা এবং একজন এজেন্টের ইতিহাস পড়া।

`ContextSwitchRepository(this._api)`

কনস্ট্রাক্টর, `ApiClient` ইনস্ট্যান্স ইনজেক্ট করে।

`Future<SupportContextSwitch> start({required int companyId, String? purpose})`

নরিদষ্টি `companyId`-র ভেতরে বর্তমান এজেন্ট এখন কাজ করছে বলে ঘোষণা করে। `purpose` ট্রমি করে খালি হলে বডতি পাঠানো হয় না (ইম্পারসোনেশনের মতো বাধ্যতামূলক নয়)। এজেন্ট আইডি সেশন থেকে নেওয়া হয়, বড থেকে নয়। নতুন সুইচ শুরু করলে সার্নির নজি থেকে আগেরটি বন্ধ করে দেয় (এক এজেন্টের একবারে একটিই সক্রিয় সুইচ থাকতে পারে)।

`Future<void> end(int id)`

`id` অনুযায়ী একটি সক্রিয় সুইচ শেষ (বন্ধ) করে।

`Future<SupportContextSwitch?> activeForAgent(int agentId)`

নরিদষ্টি এজেন্টের বর্তমান সক্রিয় সুইচ ফেরত দেয়, না থাকলে `null`। ব্যাকএন্ড "কোনো সুইচ নই" অবস্থায় ৪০৪ (`ResourceNotFoundException`) দেয়, যা এখানে ধরে `null`-এ রূপান্তর করা হয় — কারণ বেশিরভাগ সময় কোনো সুইচ না থাকাই স্বাভাবিক, একে এরর হিসেবে দেখানো ভুল হবে।

`Future<SupportContextSwitch> byId(int id)`

`id` দিয়ে একটি নরিদষ্টি সুইচ রকেড আনবে। শুধু সুপার অ্যাডমিন ও সাপোর্ট ম্যানেজারদের জন্য (এজেন্ট নজিরেটা `activeForAgent` দিয়ে দেখতে পারে, কন্টি আরবটিরার সুইচ `byId`-এ দেখতে পারে না)।

`Future<List<SupportContextSwitch>> activeEverywhere()`

এই মুহূর্তে যত এজেন্ট যেকোনো কোম্পানির ভেতরে সক্রিয়ভাবে কাজ করছে, তাদের সবার তালিকা আনবে। শুধু ম্যানেজার ও তদূর্ধ্ব রোলের জন্য।

```
Future<PagedResponse<SupportContextSwitch>> historyForAgent(int agentId, {int page = 0, int size = 20})
```

একজন এজেন্টের অতীত সুইচগুলোর পজেনিটেডে ইতিহাস আনবে। শুধু ম্যানজোর ও তদূর্ধ্ব রোলারে জন্ম — এজেন্ট নজিরেটাও দেখতে পারে না।

টপ-লেভেলে ফাংশন/প্রোভাইডার

```
final contextSwitchRepositoryProvider = Provider<ContextSwitchRepository>(...)
```

ApiClient থেকে ContextSwitchRepository তৈরি প্রোভাইডার।

ক্লাস: MyContextSwitchController extends AsyncNotifier<SupportContextSwitch?>

সাইন-ইন করা এজেন্টের নজিরে খোলা সুইচ (থাকলে) ধরে রাখা স্টেটে কন্ট্রোলার।

```
Future<SupportContextSwitch?> build()
```

বর্তমান ইউজার না থাকলে বা contextSwitchRoles না থাকলে সরাসরি null ফেরত দেয় (৪০৩ এড়াতে UI-তেও এবং এখানও গটে করা)। নাহলে activeForAgent কল করে।

```
Future<void> refresh()
```

স্টেটে আবার লোডিংয়ে সেটে করে build() পুনরায় চালায়।

```
Future<void> start({required int companyId, String? purpose})
```

রপিজিটরির start কল করে, স্টেটে আপডেটে করে, এবং activeContextSwitchesProvider ইনভ্যালিডেটে করে — কারণ সারভার-সাইডে আগের সুইচও বন্ধ হয়ে যায়, ফলে বোর্ড দুই জায়গায় ভুল হয়ে যায়।

```
Future<void> end()
```

বর্তমান সুইচ থাকলে সেটেশিষে করে, স্টেটে null করে এবং activeContextSwitchesProvider ইনভ্যালিডেটে করে।

টপ-লেভেলে ফাংশন/প্রোভাইডার

```
final myContextSwitchProvider = AsyncNotifierProvider<MyContextSwitchController, SupportContextSwitch?>(...)
```

MyContextSwitchController-এর প্রোভাইডার।

```
final activeContextSwitchesProvider = FutureProvider<List<SupportContextSwitch>>(...)
```

সব এজেন্টের লাইভ বোর্ড — ইউজার না থাকলে বা contextSwitchReviewRoles না থাকলে খালি তালিকা ফেরত দেয়, নাহলে activeEverywhere() কল করে।

ক্লাস: AgentContextHistoryController extends

AsyncNotifier<PagedState<SupportContextSwitch>> with
PagedLoader<SupportContextSwitch>

একজন নরদিশ্ট এজেন্টের সুইচ-ইতিহাস পজে-বাই-পজে লোড করার কন্ট্রোলার (agentId-কীড ফ্যামলি প্রোভাইডার দিয়ে ব্যবহৃত)।

```
AgentContextHistoryController(this.agentId)
```

কনস্ট্রাক্টর — Riverpod 3-এ ফ্যামলি আরগুমেন্ট কনস্ট্রাক্টরে আসে, build()-এ নয়, তাই agentId এখানে ধরে রাখা হয় যাতা fetchPage প্রতটি পজে পড়তে পারে।

Future<PagedState<SupportContextSwitch>> build()

currentUserProvider ওয়াচ করে (রোল পরিবর্তনে রিবিল্ড হওয়ার জন্য) এবং loadFirstPage() কল করে।

Future<PagedResponse<SupportContextSwitch>> fetchPage(int page)

নর্দিষ্ট agentId-র জন্য historyForAgent কল করে একটি পিজে আনবে।

উপ-লভেলে প্রোভাইডার

final agentContextHistoryProvider =

AsyncNotifierProvider.family<AgentContextHistoryController,

PagedState<SupportContextSwitch>, int>(...)

agentId দিয়ে কীড AgentContextHistoryController-এর ফ্যামলি প্রোভাইডার।

ফাইল: lib/features/platform/context_screen.dart

সংক্ষিপ্ত ববিরণ: সাপোর্ট কনটেক্সট সুইচ ট্যাবের UI — নিজের বর্তমান কনটেক্সট দেখা/শুরু করা/শেষ করা, এবং যাদের রিভিউ অধিকার আছে তাদের জন্য কের অ্যাকাউন্টে কাজ করছে তার লাইভ বোর্ড ও প্রতিটি এজেন্টের ইতিহাস। কনটেক্সট সুইচ ও ইম্পারসোনেশনের পার্থক্য স্ক্রিনে স্পষ্টভাবে বলা আছে — এটিকোনো টোকেনে দিয়ে না, শুধু কাজের ঘোষণা/জবাবদহিতার রেকর্ড রাখবে।

ক্লাস/উইজটে: ContextSwitchTab

মূল ট্যাব উইজটে। ইউজারের রোল অনুযায়ী "Your context" (নিজের সুইচ) এবং/অথবা "Working now" (লাইভ বোর্ড) সেকশন দেখায়; কোনো অধিকার না থাকলে "Not available to you" এমপ্টি স্টেটে দেখায়।

Widget build(BuildContext context, WidgetRef ref)

বর্তমান ইউজারের contextSwitchRoles/contextSwitchReviewRoles অনুযায়ী canSwitch/canReview নরিণয় করে, কোনোটিই না থাকলে লক-করা এমপ্টি স্টেটে দেখায়। নাহলে RefreshIndicator-এর ভেতরে ইনফো ব্যানার, শর্তসাপেক্ষে _MyContextCard এবং _ActiveBoard সেকশন সাজায়।

ক্লাস/উইজটে: _MyContextCard (ConsumerStatefulWidget)

বর্তমান এজেন্টের নিজের খোলা সুইচ (থাকলে) এবং শুরু/শেষ করার নথি়ন্তরণ দেখায়।

createState()

_MyContextCardState তরৈকিরে।

ক্লাস: _MyContextCardState

_MyContextCard-এর স্টেটে, একটি _busy ফ্ল্যাগ দিয়ে লোডিং/একশন-ব্লকিং হ্যান্ডল করে।

Future<void> _guard(Future<void> Function() action, String failure)

_busy স্টেটে করে অ্যাকশন চালায়; ApiException হলে এর মসেজে, অন্য এরর হলে ডিফল্ট failure মসেজে স্ন্যাকবারে দেখায়; শেষে _busy আনস্টেটে করে (mounted চকে সহ)।

Future<void> _start()

_StartSheet বটম-শটি খুলে কোম্পানিও পারপাস বাছাই করা়; ফলাফল থাকলে _guard-এর ভেতরে myContextSwitchProvider.notifier.start কল করে।

Future<void> _end()

_guard-এর ভেতরে myContextSwitchProvider.notifier.end কল করে বর্তমান সুইচ শেষ করে।

Widget build(BuildContext context)

myContextSwitchProvider ওয়াচ করে AppCard-এর ভেতরে লোডিং/এরর/ডটো স্টেটে রেন্ডার করে। ডটো null হলে "Not working in any company" ও শুরু-করার বাটন দেখায়; ডটো থাকলে কোম্পানির নাম, "ACTIVE" চপি, পারপাস, শুরুর সময়/এলাপসড লবেলে, দীর্ঘ সময় খোলা থাকলে সতর্কতা ব্যানার, এবং End/Switch company বাটন দেখায়।

ক্লাস/উইজটে: _StartSheet (ConsumerStatefulWidget)

নতুন কনটেক্সট সুইচের জন্য কোম্পানি ও পারপাস বাছাই করার বটম-শিট।

createState()

_StartSheetState তৈরি করে।

ক্লাস: _StartSheetState

_StartSheet-এর স্টেটে — পারপাস ও ফিল্টার টেক্সট কন্ট্রোলার এবং বাছাই করা কোম্পানি ধরে রাখে।

void initState()

_filter কন্ট্রোলারে লসিনোর যোগ করে যাতে টাইপ করলে তালিকা রফিল্টার হয়ে রিবিল্ড হয়।

void dispose()

_purpose ও _filter কন্ট্রোলার ডিসপোজ করে।

Widget build(BuildContext context)

companiesProvider ওয়াচ করে (এখনো লোড না হলে প্রথম পজে লোড করায়), ফিল্টার টেক্সট অনুযায়ী কোম্পানি নাম/সাবডোমেইনে ম্যাচ করে তালিকা ফিল্টার করে। DraggableScrollableSheet-এ পারপাস টেক্সটফিল্ড, সার্চ টেক্সটফিল্ড, কোম্পানি তালিকা (ListTile নরিবাচনযোগ্য) এবং নচি "Start on ..." বাটন দেখায় যা (companyId, purpose) রেকর্ড নিয়ে পপ করে।

ক্লাস/উইজটে: _ActiveBoard (ConsumerWidget)

activeContextSwitchesProvider থেকে সবার সক্রিয় সুইচের তালিকা এনে প্রতটিরি জন্য _ActiveRow সাজায়; খালি হলে "Nobody is in a customer account" এমপ্টি স্টেটে দেখায়।

Widget build(BuildContext context, WidgetRef ref)

প্রোভাইডার ওয়াচ করে লোডিং/এরর/ডটো হ্যান্ডল করে; ডটো থাকলে প্রতটি এন্ট্রির জন্য প্যাডিং সহ _ActiveRow তৈরি করে কলামে সাজায়।

ক্লাস/উইজটে: _ActiveRow (ConsumerWidget)

একজন এজেন্ট বর্তমানে কোন কোম্পানিতে কাজ করছে তার একটি সারি — ট্যাপ করলে সেই এজেন্টের ইতিহাস স্ক্রিনি যায়।

Widget build(BuildContext context, WidgetRef ref)

InkWell-এর ভেতরে AppCard — এজেন্টের অ্যাভাটার/ইনশিয়াল, নাম, "in {company}", দীর্ঘসময় খোলা থাকলে "OVERDUE" চপি (নাহলে এলাপসড লবেলে), এবং পারপাস থাকলে তা দেখায়। ট্যাপে

AgentContextHistoryScreen.open কল করে।

`static String _initials(String name)`

নামের শব্দগুলো থেকে ইনিশিয়াল বের করে — একটি শব্দ না থাকলে ?, একটি হলে তার প্রথম অক্ষর, একাধিক হলে প্রথম ও শেষ শব্দে প্রথম অক্ষর যুক্ত করে বড় হাতের অক্ষরে ফেরত দেয়।

ক্লাস/উইজটে: AgentContextHistoryScreen (ConsumerWidget)

একজন নির্দিষ্ট এজেন্টের অতীত কনটেক্সট সুইচগুলোর পজেনিটেডে তালিকা দেখানোর স্ক্রিনি — শুধু রিভিউ-রোলারদ্বারা জন্ম।

`AgentContextHistoryScreen({super.key, required this.agentId, required this.agentName})`

কনস্ট্রাক্টর, দেখানোর এজেন্ট আইডি ও নাম নেয়।

`static void open(BuildContext context, {required int agentId, required String agentName})`

`MaterialPageRoute` দিয়ে এই স্ক্রিনি পুশ করে নভিগেটে করে।

`Widget build(BuildContext context, WidgetRef ref)`

বর্তমান ইউজারের `contextSwitchReviewRoles` না থাকলে "Not available to you" এমপ্টি স্টেটে সহ Scaffold ফেরত দেয়। নাহলে `agentContextHistoryProvider(agentId)` ওয়াচ করে `PagedListView` দিয়ে ইতিহাসের সারিগুলো (`_HistoryRow`) রেন্ডার করে, `refresh/loadMore` হুক করে।

ক্লাস/উইজটে: _HistoryRow (StatelessWidget)

একটি অতীত (বা এখনো সক্রিয়) কনটেক্সট সুইচের সংক্ষিপ্ত সারি — ট্যাপে বস্টিয়ারতি রকেড স্ক্রিনি যায়।

`Widget build(BuildContext context)`

`AppCard`-এ কন্ট্রোলারের নাম, সক্রিয় হলে "ACTIVE" চিহ্ন, পারাপাস, শুরুর সময় (বা এখনো সক্রিয় হলে শুরুর সময়), এলাপসড লবেলে, এবং IP ঠিকানা থাকলে তা দেখায়। ট্যাপে

`ContextSwitchDetailScreen.open(context, id: entry.id)` কল করে।

ফাইল: lib/features/platform/context_switch_detail.dart

সংক্ষিপ্ত বিবরণ: একটি নির্দিষ্ট কনটেক্সট সুইচ রকেডের সম্পূর্ণ বস্টিয়ারতি দেখানোর স্ক্রিনি — ক, কোন কন্ট্রোলারে, কনে, কোথা থেকে, কতক্ষণ, এবং এখনো সক্রিয় কিনা। এই প্রোডাক্টের সবচেয়ে সংবদনশীল কাজের অ্যাক্সেস-রকেড বলে এটি সারাংশ না করে পূর্ণ বিবরণে দেখানো হয়।

টপ-লেভেলে প্রোভাইডার

`final contextSwitchProvider = FutureProvider.autoDispose.family<SupportContextSwitch, int>(...)`

`id` দিয়ে কীড, একটি নির্দিষ্ট সুইচ রকেড আনার জন্য `ContextSwitchRepository.byId` কল করা `autoDispose` প্রোভাইডার।

ক্লাস/উইজটে: ContextSwitchDetailScreen (ConsumerWidget)

একটি সুইচ রকেডের ফুল-ডিটেইল স্ক্রিনি।

`ContextSwitchDetailScreen({super.key, required this.id})`

কনস্ট্রাক্টর — দেখানোর রকেড আইডি নেয়।

static void open(BuildContext context, {required int id})

MaterialPageRoute দিয়ে এই স্ক্রিনি পুশ করে।

Widget build(BuildContext context, WidgetRef ref)

contextSwitchProvider(id) ওয়াচ করে লোডিং/এরর/ডটো হ্যান্ডল করে। ডটো থাকলে: সুইচ এখনো সক্রিয় ও দীর্ঘসময় খোলা থাকলে সতর্কতা ব্যানার (নাহলে সাধারণ ইনফো ব্যানার), তারপর কোম্পানির নাম/এজেন্টের নাম/পারপাস সম্বলিত কার্ড, "When" সেকশনে (`_Fact` দিয়ে) ঢোকান সময়, বের হওয়ার সময় (বা "Still in there"), এলাপসড লবেলে, এবং IP/ইউজার-এজেন্ট থাকলে "From" সেকশনে সগুলো দেখায়।

ক্লাস/উইজটে: `_Fact (StatelessWidget)`

একটি লবেলে-ভ্যালু জোড়া সারি হিসেবে দেখানোর ছোট হেল্পার উইজটে (যেমন "Went in: ...")।

`_Fact(this.label, this.value)`

কনস্ট্রাক্টর — লবেলে ও মান নেয়।

Widget build(BuildContext context)

ফক্সড-প্রস্থ লবেলে কলাম ও `SelectableText`-এ মান (কপি-যোগ্য, যেহেতু IP/ব্রাউজার স্ট্রিং কপি করার প্রয়োজন হতে পারে) সাজিয়ে রো রেন্ডার করে।

ফাইল: `lib/features/platform/location_models.dart`

সংক্ষিপ্ত বিবরণ: দেশের প্রশাসনিক হয়ারার্কি (Division/District/Upazila/Police Station জাতীয়) মডেল করার ডটো ক্লাস, রকিওয়েস্ট মডেল এবং প্রতিটি দেশের নিজস্ব লভেলে-নামের টবেলি। কোনো ফ্ল্যাটার/UI কোড নেই — বিশুদ্ধ ডটো ও হেল্পার ফাংশন।

ক্লাস: `GeoNode`

হয়ারার্কির একটি নোড উপস্থাপন করে — `type COUNTRY` (শুধু পঠনযোগ্য, তৈরি/আপডেট/মুছা যায় না) অথবা এর নচিরে `LEVEL1..LEVEL4` হতে পারে। `code` (ISO দেশ কোড, যেমন "BD") শুধু `COUNTRY` সারতি সেটে থাকে, তার নচি সবসময় `null`।

`const GeoNode({required this.id, required this.name, required this.type, this.code})`

কনস্ট্রাক্টর।

`bool get isCountry`

`type == 'COUNTRY'` কনি তা যাচাই করে।

`factory GeoNode.fromJson(Map<String, dynamic> json)`

JSON থেকে `GeoNode` তৈরি করে — সব ফিল্ডের জন্য নরিপদ ডফিল্ট (id: 0, name/type: খালি স্ট্রিং, code: null) দিয়ে।

ক্লাস: `LocationCreateRequest`

নতুন লোকেশন নোড তৈরি (POST /locations) রকিওয়েস্ট বডি LEVEL1-এর জন্য `countryId` লাগে, LEVEL2-4-এর জন্য `parentId` — কখনো দুটোই নয়। Angular-এর নিজস্ব স্ক্রিনি LEVEL1-এর জন্যও ভুলভাবে `parentId` পাঠায় (যা ব্যাকএন্ড প্রত্যাখ্যান করে); এই ক্লাস সেই ভুল করে না।

```
const LocationCreateRequest({required this.name, required this.type, this.countryId, this.parentId})
```

কনস্ট্রাক্টর।

```
Map<String, dynamic> toJson()
```

নাম ট্রিম করে, type, এবং countryId/parentId যটো null নয় শুধু সটো JSON বডিতে অন্তর্ভুক্ত করে।

ক্লাস: LocationUpdateRequest

লোকেশন নোড রনিমে করার (PUT /locations/{id}) রকিওয়েস্ট বডি ব্যাকএন্ড re-parent করার জন্য parentIdও গ্রহণ করে, কনিন্তু এই ক্লাস শুধু নাম বদলানোর সুবিধা দিয়ে।

```
const LocationUpdateRequest({required this.name})
```

কনস্ট্রাক্টর।

```
Map<String, dynamic> toJson()
```

ট্রিম করা নাম-সহ JSON বডি ফিরেত দিয়ে।

টপ-লভেলে কনস্ট্যান্ট/ফাংশন

```
const Map<String, List<String>> _hierarchyLabels
```

প্রতিটি দেশে ISO কোড অনুযায়ী তার চার-লভেলে হায়ারার্কি নামের তালিকা (যেমন BD ['Division', 'District', 'Upazila / Municipality', 'Police Station'])। ওয়েবে অ্যাপের globalHierarchyLabels থেকে পোর্ট করা।

```
String hierarchyLabel(String? countryCode, int level)
```

countryCode ও level (১-৪) অনুযায়ী মানুষের পড়ার উপযোগী লভেলে-নাম ফেরেত দিয়ে; টবেলি না থাকা দেশে কোড বা কোনো দেশ-প্রসঙ্গ না থাকলে "Level N" ফলব্যাক দিয়ে।

```
const _levelTypes = ['LEVEL1', 'LEVEL2', 'LEVEL3', 'LEVEL4']
```

লভেলে-টাইপ স্ট্রিংগুলোর তালিকা।

```
String locationTypeForDepth(int depth)
```

প্রদত্ত depth (০ = দেশ) থেকে এক লভেলে গভীরের নোডের LocationType স্ট্রিং ফেরেত দিয়ে (depth ০-৩ এর মধ্যে ক্ল্যাম্প করে)।

ফাইল: lib/features/platform/location_repository.dart

সংক্ষিপ্ত ববিরণ: গ্লোবাল দেশ/ডভিশন/জেলো রফোরেন্স হায়ারার্কি জন্য রপিজিটরিও Riverpod প্রোভাইডার। পড়া (GET) কোনো পারমিশন ছাড়াই থোলা, কনিন্তু লখো (create/update/delete) শুধু প্ল্যাটফর্ম স্টাফের জন্য।

ক্লাস: LocationRepository

লোকেশন হায়ারার্কি CRUD API কলগুলো মোডানো ক্লাস।

```
LocationRepository(this._api)
```

কনস্ট্রাক্টর, ApiClient ইনজেক্ট করে।

`Future<List<GeoNode>> countries()`

সব দেশের তালিকা আনবে।

`Future<List<GeoNode>> divisionsForCountry(int countryId)`

একটি দেশের LEVEL1 ডিভিশন তালিকা আনবে। দেশ আইডিও লোকেশন আইডি ভিন্ন স্কোপে হওয়ায় (কলিশন সম্ভব) এই মেথডই একমাত্র উপায় দেশের নিজস্ব টপ-লেভেলে আনার — `children`-এর মাধ্যমে নয়।

`Future<List<GeoNode>> children(int parentId)`

একটি LEVEL1-3 প্রায়ন্টের LEVEL2-4 সন্তান নোডগুলো আনবে। এখানে দেশ আইডি দেওয়া উচিত নয়।

`Future<GeoNode> create(LocationCreateRequest request)`

নতুন লোকেশন নোড তৈরি করে।

`Future<GeoNode> update(int id, LocationUpdateRequest request)`

নর্দিষ্ট নোড আপডেট (রনিমে) করে।

`Future<void> delete(int id)`

নর্দিষ্ট নোড মুছে ফেলে।

টপ-লেভেলে প্রোভাইডার

`final locationRepositoryProvider = Provider<LocationRepository>(...)`

`ApiClient` থেকে `LocationRepository` তৈরি প্রোভাইডার।

`final countriesProvider = FutureProvider<List<GeoNode>>(...)`

সব দেশের তালিকা আনার প্রোভাইডার।

`final locationChildrenProvider = FutureProvider.family.autoDispose<List<GeoNode>, ({int parentId, bool parentIsCountry})>(...)`

একটি নোডের সন্তানদের জন্য ফ্যামলি প্রোভাইডার — `parentIsCountry` অনুযায়ী `divisionsForCountry` বা `children` কল করে বেছে নেয়। প্রতিটি প্রায়ন্টের জন্য আলাদা (ফ্যামলি) হওয়ায় একটি শাখায় ড্রলি-ইন করলে অন্য ইতিমধ্যে লোড হওয়া শাখা ইনভ্যালিডিটে হয় না।

ফাইল: `lib/features/platform/locations_screen.dart`

সংক্ষিপ্ত ববিরণ: দেশ/ডিভিশন/জেলো রফোরন্স হায়ারার্ক ব্রাউজ ও সম্পাদনার স্ক্রিনি — পড়া সবার জন্য উন্মুক্ত, সম্পাদনা শুধু প্ল্যাটফর্ম স্টাফের জন্য। মোবাইলে Angular-এর পাঁচ-কলাম লেআউটের বদলে একবারে একটি শাখায় ড্রলি-ডাউন করার পদ্ধতি ব্যবহার করা হয়েছে।

ক্লাস/উইজটে: `LocationsScreen (ConsumerStatefulWidget)`

লোকেশন হায়ারার্ক ব্রাউজ করার মূল স্ক্রিনি।

`createState()`

`_LocationsScreenState` তৈরি করে।

ক্লাস: `_LocationsScreenState`

ড্রলি-ডাউন ট্রাইল (`_path`) ও সংশ্লিষ্ট CRUD অ্যাকশনগুলো ধারণ করা স্টেটে ক্লাস।

`void _drillInto(GeoNode node)`

`node.type LEVEL4` না হলে স্টেটিকে `_path`-এ যোগ করে ভেতরে ঢোকে (`LEVEL4` হলে কিছু করে না, কারণ এটি সর্বনম্ন লভেলে)।

`bool _canPop(BuildContext context)`

`_path` খালি নয় কনি যাচাই করে — অর্থাৎ পপ (উপরে যাওয়া) সম্ভব কনি।

`void _up()`

`_path` থেকে শেষে নোড সরিয়ে এক ধাপ উপরে যায়।

`Future<void> _addChild()`

বর্তমান লভেলে এক ধাপ নচি নতুন নোড তৈরিকরে — নাম প্রম্পট করে, তারপর `LocationCreateRequest` তৈরিকরে (`level == 1` হলে `countryId`, নাহলে `_path.last.id`-কে `parentId` হিসেবে ব্যবহার করে) `create` কল করে, সফল/ব্যর্থ হলে সন্মাকবার দেখায় এবং লোকাল লিস্ট ইন্ড্যালডিটে করে। শুধু দেশের ভেতরে ঢোকার পরই ডাকা হয়, তাই `_path` কখনো খালি থাকে না এখানো।

`Future<void> _rename(GeoNode node)`

নাম প্রম্পট করে (আগের নাম প্র-ফিল্ড করে), নতুন নাম আলাদা হলে `update` কল করে ও লিস্ট রফ্রেশ করে।

`Future<void> _delete(GeoNode node)`

কনফার্মেশন ডায়ালগ দেখায় (নিচের সব কিছু চরিতরে মুছে যাবে বলে সতর্ক করে), নিশ্চিতি হলে `delete` কল করে, সন্মাকবার দেখায় ও লিস্ট রফ্রেশ করে।

`void _invalidateCurrentLevel()`

`_path` খালি হলে `countriesProvider` ইন্ড্যালডিটে করে, নাহলে বর্তমান প্যারেন্টের জন্য `locationChildrenProvider` ইন্ড্যালডিটে করে।

`Future<String?> _promptForName(BuildContext context, {required String title, String? initial})`

একটি `AlertDialog` দিয়ে টেক্সট ইনপুট প্রম্পট দেখায়, `Cancel/Save` দিয়ে ফলাফল (ট্রমি করা নাম বা `null` খালি হলে) ফেরত দেয়।

`Widget build(BuildContext context)`

বর্তমান ইউজারের `platformUserRoles` দিয়ে `canManage` নির্ণয় করে; `_path` খালি হলে `countriesProvider`, নাহলে `locationChildrenProvider` ওয়াচ করে। `PopScope` দিয়ে ব্যাক-বাটনকে ড্রলি-আপে ম্যাপ করে। `Scaffold`-এ `AppBar` (বর্তমান নোডের নাম, উপরে যাওয়ার আইকন), `FAB` (দেশে নজি তৈরিকরা যায় না বলে শুধু কখনো দেশের ভেতরে ঢুকলে এবং `LEVEL4`-এর আগ পর্যন্ত দেখানো হয়), এবং `RefreshIndicator`-এর ভেতরে লোডিং/এরর/খালি/তালিকা স্টেটে রেন্ডার করে। প্রতিটি নোড কার্ডে নাম, ম্যানেজ-অধিকার থাকলে `Rename/Delete` পপআপ মেনু (দেশে ছাড়া), অথবা লফি না হলে `chevron` আইকন দেখায়।

ফাইল: lib/features/platform/plans_screen.dart

সংক্ষিপ্ত ববিরণ: সাবস্ক্রিপশন প্ল্যানগুলোর তালিকা ও এডটি স্ক্রিনি — যখনে সব প্ল্যান (বন্ধ করা প্ল্যানসহ) দেখানো হয়, নতুন প্ল্যান তৈরি করা যায়, এবং বদ্যমান প্ল্যান এডটি/টগল করা যায়। `code` একবার তৈরিহলে আর বদলানো যায় না, কারণ কোম্পানিও সাবস্ক্রিপশন হিস্ট্রি এই স্ট্রিং রফোরেন্স করে।

উপ-লভেলে প্রোভাইডার: allPlansProvider

`FutureProvider.autoDispose<List<SubscriptionPlanOption>>` — `activeOnly: false` দিয়ে সব প্ল্যান (বন্ধ থাকা সহ) লোড করে। `subscriptionPlansProvider`-এর থেকে আলাদা, কারণ স্টেশিউ সক্রিয় প্ল্যান আনে (কোম্পানিকে নতুন প্ল্যানের সরানোর পকারেরে জন্য), কিন্তু এই স্ক্রিনি বন্ধ প্ল্যান আবার চালু করার সুযোগও দিতে হয়।

ক্লাস/উইজটে: PlansScreen

`ConsumerWidget`, প্ল্যান তালিকা দেখানোর স্ক্রিনি। উপরে অ্যাপবার, নচে "New plan" ফ্লোটিং বাটন এবং `ConfigList` দিয়ে প্ল্যানগুলোর তালিকা রেন্ডার করে।

`static void open(BuildContext context)`

`MaterialPageRoute` দিয়ে `PlansScreen` পুশ করে নেভিগেটে করে।

`Widget build(BuildContext context, WidgetRef ref)`

`allPlansProvider` ওয়াচ করে `ConfigList<SubscriptionPlanOption>` বানায়। FAB চাপলে `showNewSubscriptionPlanSheet` খোলে এবং সফল হলে `allPlansProvider` ও `subscriptionPlansProvider` উভয়ই ইনভ্যালিডেটে করে যাতে দুই জায়গার তালিকাই আপডেটে হয়। খালি অবস্থায়, এরর অবস্থায় আইকন/টেক্সট দেখায় এবং প্রতটি আইটেমেরে জন্য `_PlanRow` বানায়।

ক্লাস/উইজটে: _PlanRow

`ConsumerStatefulWidget` — একটি প্ল্যান সারি দেখায়, টগল/এডটি অ্যাকশনসহ।

`ConsumerState<_PlanRow> createState()`

`_PlanRowState` তৈরি করে।

ক্লাস: _PlanRowState

`_PlanRow`-এর স্টেটে, টগল অ্যাকশনেরে বজি-স্টেটে নয়ন্ত্রণ করে।

`Future<void> _toggle()`

প্ল্যানের `id` না থাকলে কিছু করে না। থাকলে বজি-স্টেটে স্টে করে `togglePlan` কল করে, তারপর দুটো প্রোভাইডার ইনভ্যালিডেটে করে এবং সফল/ব্যর্থ সন্ধ্যাকবার দেখায় — সফল হলে প্ল্যান আগে সক্রিয় ছিল কনি তার ওপর ভিত্তি করে ভিন্ন বার্তা দেখায়।

`Widget build(BuildContext context)`

`ConfigRow` বানায় — টাইটলে, সাবটাইটলে কোড/দাম/বলিং সাইকেলে জোড়া লাগানো (· দিয়ে), সক্রিয়তা, বজি ফ্ল্যাগ এবং `id` না থাকলে এডটি/টগল কলব্যাক নাল করে দেয়।

টপ-লভেলে ফাংশন: `Future<void> showEditPlanSheet(BuildContext context, {required SubscriptionPlanOption plan})`

`_EditPlanSheet` বটম শাটিকে দেখায় প্ল্যান এডিট করার জন্য। কোড দেখানো হয় কিন্তু এডিটিযোগ্য নয়, কারণ কোম্পানি সাবস্ক্রিপশন হিস্ট্রি সহি হিস্ট্রি রিফরেন্স করে এবং ব্যাকএন্ড বদলাতে দেয় না।

ক্লাস/উইজটে: `_EditPlanSheet`

`ConsumerStatefulWidget` — প্ল্যানের নাম, দাম, বলিং সাইকেলে ও বিবরণ এডিট করার ফর্ম শাট।

`ConsumerState<_EditPlanSheet> createState()`

`_EditPlanSheetState` তৈরি করে।

ক্লাস: `_EditPlanSheetState`

`_EditPlanSheet`-এর স্টেটে, ফর্ম কন্ট্রোলার, বলিং সাইকেলে ও এরর/বজি স্টেটে রাখো।

`void dispose()`

`_name`, `_description`, `_price` কন্ট্রোলার ডিসপোজ করে, তারপর `super.dispose()` কল করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, বজি-স্টেটে সেটে করে `updatePlan` কল করে (কোড পাঠানো হয় কিন্তু রপিজিটরিস্ট্রিতে বাদ দেওয়া হয়)। সফল হলে দুটো প্রোভাইডার ইনভ্যালিডেট করে শাট বন্ধ করে; ব্যর্থ হলে এরর টেক্সট দেখায়।

`Widget build(BuildContext context)`

`FormSheetFrame` বানায় — শুধু-পাঠযোগ্য "Code" ফিল্ড, নাম, দাম+বলিং সাইকেলে সারি, বিবরণ, এবং একটা নোট যে দাম বদলালে বর্তমান কোম্পানির বলিং বদলায় না, শুধু ভবিষ্যতে এই প্লানে আসা কোম্পানির জন্য প্রযোজ্য হয়।

ফাইল: `lib/features/platform/platform_create_sheets.dart`

সংক্ষিপ্ত বিবরণ: প্ল্যাটফর্ম স্টাফ তৈরি এবং নতুন সাবস্ক্রিপশন প্ল্যান তৈরির দুটি বটম-শাট ফর্ম এই ফাইলে আছে।

টপ-লভেলে ফাংশন: `Future<void> showNewPlatformUserSheet(BuildContext context)`

`_NewPlatformUserSheet` বটম শাটিকে দেখায়, প্ল্যাটফর্ম স্টাফ যোগ করার ফর্ম।

টপ-লভেলে ফাংশন: `Future<void> showNewSubscriptionPlanSheet(BuildContext context)`

`_NewSubscriptionPlanSheet` বটম শাটিকে দেখায়, নতুন সাবস্ক্রিপশন প্ল্যান তৈরি ফর্ম।

ক্লাস/উইজটে: `_NewPlatformUserSheet`

`ConsumerStatefulWidget` — নতুন প্ল্যাটফর্ম স্টাফ ইউজার যোগ করার ফর্ম শাট।

`ConsumerState<_NewPlatformUserSheet> createState()`

`_NewPlatformUserSheetState` তৈরি করে।

ক্লাস: `_NewPlatformUserSheetState`

`_NewPlatformUserSheet`-এর স্টেটে। নাম, ইমেইল, ফোন, পাসওয়ার্ড/কনফার্ম পাসওয়ার্ড কন্ট্রোলার, পাসওয়ার্ড দেখানো/লুকানো টগল, রোল নির্বাচন এবং সাবমিট/এরর স্টেটে রাখা।

`void dispose()`

সব টেক্সট কন্ট্রোলার (`_firstName`, `_lastName`, `_email`, `_phone`, `_password`, `_confirmPassword`) ডিসপোজ করে সুপার কল করে।

`Future<void> _submit()`

ইতিমধ্যে সাবমিট হচ্ছে বা ভ্যালিডেশন ফলে করলে ফিরে যায়। নাহলে ফোকাস সরিয়ে বজি-স্টেটে সেটে করে `platformUsersProvider.notifier.create` কল করে `CreatePlatformUserRequest` দিয়ে। সফল হলে শিট বন্ধ করে সন্ধ্যাকবার দেখায়; ব্যর্থ হলে সারভার-বার্তা বা জনেরেকি এরর দেখায়।

`Widget build(BuildContext context)`

স্ক্রলযোগ্য ফর্ম বানায় — প্রথম নাম, শেষ নাম, ইমেইল (সাইন-ইন হলেপার টেক্সটসহ), অস্থায়ী পাসওয়ার্ড (ন্যূনতম ৮ অক্ষর, চোখ-আইকন দিয়ে দেখানো/লুকানো টগল), কনফার্ম পাসওয়ার্ড (মলি যাচাই), রোল ড্রপডাউন (`assignablePlatformRoles` থেকে), ঐচ্ছকি ফোন, এবং লোডিং বাটন।

`static String? Function(String?) _required(String message)`

একটি ভ্যালিডেটর ফাংশন রিটার্ন করে যা ফাঁকা/নাল মান পলে `message` দেয়, নাহলে নাল।

ক্লাস/উইজটে: `_NewSubscriptionPlanSheet`

`ConsumerStatefulWidget` — নতুন সাবস্ক্রিপশন প্ল্যান তৈরি ফর্ম শিট।

`ConsumerState<_NewSubscriptionPlanSheet> createState()`

`_NewSubscriptionPlanSheetState` তৈরি করে।

ক্লাস: `_NewSubscriptionPlanSheetState`

`_NewSubscriptionPlanSheet`-এর স্টেটে। কোড, নাম, দাম, বিবরণ কন্ট্রোলার, বলিৎ সাইকলে নির্বাচন এবং সাবমিট/এরর স্টেটে রাখা।

`void dispose()`

`_code`, `_name`, `_price`, `_description` কন্ট্রোলার ডিসপোজ করে সুপার কল করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, বজি-স্টেটে সেটে করে `platformRepositoryProvider.createPlan` কল করে `CreateSubscriptionPlanRequest` দিয়ে। সফল হলে `subscriptionPlansProvider` ইনভ্যালিডেট করে (কারণ কোম্পানির প্ল্যান-পিকার এই লিস্ট পড়ে), শিট বন্ধ করে সন্ধ্যাকবার দেখায়। ব্যর্থ হলে ডুপ্লিকেট-কোড বার্তা বা জনেরেকি এরর দেখায়।

`Widget build(BuildContext context)`

স্ক্রলযোগ্য ফর্ম বানায় — নাম, কোড (আপার-কসে, ইউনিকি হতে হবে), বলিৎ সাইকলে ড্রপডাউন (`billingCycles` থেকে), দাম (ঋণাত্মক নষিধে), ঐচ্ছকি বিবরণ, এবং লোডিং বাটন।

ফাইল: lib/features/platform/platform_edit_sheets.dart

সংক্ষিপ্ত ববিরণ: প্ল্যাটফর্মেরে নজিস্ব খরচ ফাইল করার শটি এবং প্ল্যাটফর্ম স্টাফ এডটি করার শটি — এই দুই ধরনেরে ফর্ম এই ফাইলে আছে।

টপ-লেভেলে ফাংশন: `Future<void> showPlatformExpenseSheet(BuildContext context)`

`_PlatformExpenseSheet` বটম শটি দেখোয়, প্ল্যাটফর্মেরে বই-এ খরচ ফাইল করার ফর্ম। কোম্পানি-খরচেরে একই DTO ব্যবহার করে (`finance` মডেল থকে); এডটিং দেওয়া হয়নি কারণ `PlatformExpensesTab`-এ ব্যাখ্যা করা আছে।

ক্লাস/উইজটে: `_PlatformExpenseSheet`

`ConsumerStatefulWidget` — প্ল্যাটফর্ম খরচ ফাইল করার ফর্ম শটি।

`ConsumerState<_PlatformExpenseSheet> createState()`

`_PlatformExpenseSheetState` তৈরি করে।

ক্লাস: `_PlatformExpenseSheetState`

`_PlatformExpenseSheet`-এর স্টেটে। ববিরণ, পরমাণ, ভেন্ডর, ক্যাটাগরি, নোট কন্ট্রোলার, তারিখ ও সাবমিট/এরর স্টেটে রাখে।

`void dispose()`

`_description`, `_amount`, `_vendor`, `_category`, `_notes` কন্ট্রোলার ডিসপোজ করে সুপার কল করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে; তারিখ নাল হলে এরর দেখিয়ে ফিরে যায়। নাহলে বজি-স্টেটে সেটে করে `platformRepositoryProvider.createPlatformExpense` কল করে `CreateExpenseRequest` দিয়ে, তারপর `platformExpensesProvider.notifier.refresh()` করে। সফল হলে শটি বন্ধ করে সন্ধ্যাকবার দেখোয়; ব্যর্থ হলে এরর দেখোয়।

`Widget build(BuildContext context)`

`FormSheetFrame` বানায় — ববিরণ, পরমাণ (শূন্যেরে বশেহিত হব), `DateField` (আজকেরে পর তারিখ নষিধে, ক্লয়ার-বাটন নহে), ঐচ্ছকি ভেন্ডর, ক্যাটাগরি, নোট।

টপ-লেভেলে ফাংশন: `Future<void> showEditPlatformUserSheet(BuildContext context, {required PlatformUser user})`

`_EditPlatformUserSheet` বটম শটি দেখোয়, প্ল্যাটফর্ম স্টাফ এডটি করার ফর্ম। নাম, ইমেইল ও রোল আবশ্যকি — ব্যাকএন্ড হাতে-হাতে চকে করে ফাঁকা মান প্রত্যাখ্যান করে, তাই ফর্ম বদ্যমান মান দিয়ে শুরু করে এখানই ভ্যালিডেট করে।

ক্লাস/উইজটে: `_EditPlatformUserSheet`

`ConsumerStatefulWidget` — বদ্যমান প্ল্যাটফর্ম স্টাফ এডটি করার ফর্ম শটি।

`ConsumerState<_EditPlatformUserSheet> createState()`

`_EditPlatformUserSheetState` তৈরি করে।

ক্লাস: `_EditPlatformUserSheetState`

`_EditPlatformUserSheet`-এর স্টেটে। ইউজারের বর্তমান মান দিয়ে প্র-ফিলি করা কন্ট্রোলার, রোল, পাসওয়ার্ড কন্ট্রোলার (ফাঁকা রাখলে লগইন অপরিবর্তিত থাকে) এবং সাবমিট/এর স্টেটে রাখে।

`void dispose()`

`_firstName`, `_lastName`, `_email`, `_phone`, `_password` কন্ট্রোলার ডিসপোজ করে সুপার কল করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, বজিস্টেটে সেটে করে `platformUsersProvider.notifier.updateItem` কল করে `UpdatePlatformUserRequest` দিয়ে। পাসওয়ার্ড ফিল্ড ফাঁকা কনি তা দিয়ে ঠিকি করে বার্তায় পাসওয়ার্ড বদলানোর উল্লেখ যোগ করবে কনি। ব্যর্থ হলে "Email already exists" বা "Invalid platform role selected"-এর মতো সারভার-বার্তা দেখায়।

`Widget build(BuildContext context)`

`FormSheetFrame` বানায় — প্রথম/শেষ নামের সারি, ইমেইল, ঐচ্ছিক ফোন, রোল ড্রপডাউন, এবং একটি "Password" শিরোনামের নিচে নতুন পাসওয়ার্ড ফিল্ড (ফাঁকা রাখলে বর্তমান পাসওয়ার্ড বজায় থাকে, টাইপ করলে ন্যূনতম ৮ অক্ষর যাচাই হয়)।

ফাইল: `lib/features/platform/platform_expenses_tab.dart`

সংক্ষিপ্ত বিবরণ: প্ল্যাটফর্মের নিজস্ব খরচের ট্যাব — একটি কীম্পানির নয়, অপারেটরের নিজের বই। ফিল্টার, অনুমোদন/প্রত্যাখ্যান/পরিশোধ-চলিতকরণ/মুছে ফেলা অ্যাকশন এবং তালিকা প্রদর্শন এখানে আছে। এডিটিং ইচ্ছাকৃতভাবে বাদ দেওয়া হয়েছে কারণ PATCH এন্ডপয়েন্ট এখানে থেকে ব্যবহার করলে খরচকে তার অ্যাকাউন্ট থেকে নিশ্চিন্দে বর্জিত করবে দিতে পারে।

ক্লাস: `PlatformExpenseFilterController`

`Notifier<String?>` — খরচের তালিকা কোন স্ট্যাটাসে ফিল্টার করা আছে তা রাখে। নাল মান সব দেখানো হচ্ছে।

`String? build()`

প্রাথমিক অবস্থা নাল রিটার্ন করে।

`void set(String? value)`

নতুন মান আগেরটির সমান না হলে স্টেটে আপডেট করে।

টপ-লেভেলে কনস্ট্যান্ট: `mineFilter`

'MINE' — এটি `ExpenseStatus` নয়, বরং `/my-expenses` এন্ডপয়েন্টের জন্য একটি ছিদ্র-ফিল্টার মান।

ক্লাস: `PlatformExpensesController`

`AsyncNotifier<List<Expense>>` — প্ল্যাটফর্ম খরচের তালিকা লোড ও রিফ্রেশ করে, নির্বাচনী ফিল্টার অনুযায়ী।

`Future<List<Expense>> build()`

`currentUserProvider` ও `platformExpenseFilterProvider` ওয়াচ করে `_load()` কল করে।

`Future<List<Expense>> _load()`

বর্তমান ফিল্টার পড়ে; ফিল্টার `mineFilter` হলে `mineOnly: true` পাঠায়, নাহলে ফিল্টারকে `status` হিসেবে পাঠায়। `platformRepositoryProvider.platformExpenses` কল করে পজেরে `content` রটাবন করে।

`Future<void> refresh()`

`AsyncValue.guard` দিয়ে `_load()` পুনরায় চালিয়ে স্টেটে আপডেট করে।

`void remove(int id)`

বর্তমান তালিকা থেকে দেওয়া `id`-এর সারবিাদ দিয়ে নতুন স্টেটে সেট করে (কিছু না থাকলে চুপচাপ ফিরে যায়)।

ক্লাস/উইজটে: PlatformExpensesTab

`ConsumerStatefulWidget` — প্ল্যাটফর্ম খরচের ট্যাবের মূল উইজটে।

`ConsumerState<PlatformExpensesTab> createState()`

`_PlatformExpensesTabState` তৈরি করে।

ক্লাস: _PlatformExpensesTabState

`_PlatformExpensesTab`-এর স্টেটে। কোন খরচের আইডি বর্তমানে অ্যাকশন-বজিতি `_busyId`-এ রাখা।

`Future<void> _run(int id, Future<void> Function() action, String success, String failure)`

বজি-স্টেটে সেট করে `action()` চালায়। সফল হলে তালিকা রফ্রেশ করে সাকসেসে সন্ধ্যাকবার দেখায় কারণ প্রতিটি অ্যাকশন খালি রিসেপন্স দেয়, তাই প্যাচ না করে পুরো তালিকা রলিড করা হয় এবং স্ট্যাটাস বদলের ফলে সারি অন্য ফিল্টারে সরে যায়। ব্যর্থ হলে সারভার-বার্তা বা `failure` টেক্সট দেখায়; শেষে বজি-স্টেটে রসিটে করে।

`Future<void> _approve(Expense expense)`

`askForText` দিয়ে আবশ্যিক নোট চায় (এন্ডপয়েন্ট নোট ছাড়া অনুমোদন প্রত্যাখ্যান করে)। নোট পলে `_run` দিয়ে `approvePlatformExpense` কল করে।

`Future<void> _reject(Expense expense)`

`askForText` দিয়ে আবশ্যিক কারণ চায় (ডেস্ট্রাক্টিভি স্টাইলে, যা যনি ফাইল করছেন তাকে দেখানো হয়)। কারণ পলে `_run` দিয়ে `rejectPlatformExpense` কল করে।

`Future<void> _markPaid(Expense expense)`

`_MarkPaidDialog` ডায়ালগ দেখিয়ে পদ্ধতি/রিফরেন্স নিয়ে (উভয়ই ঐচ্ছিক)। ফলাফল পলে `_run` দিয়ে `markPlatformExpensePaid` কল করে।

`Future<void> _delete(Expense expense)`

নিশ্চিতকরণ ডায়ালগ দেখায় (মুছে ফেললে কোনো ট্রাইল থাকে না, প্রত্যাখ্যান করলে ট্রাইল থাকে বলে সতর্ক করে)। নিশ্চিতি হলে বজি-স্টেটে সেট করে `deletePlatformExpense` কল করে, সফল হলে `PlatformExpensesController.remove` দিয়ে তালিকা থেকে বাদ দেয় এবং সন্ধ্যাকবার দেখায়; ব্যর্থ হলে এরর দেখায়।

`Widget build(BuildContext context)`

বর্তমান ফিল্টার ও `canDelete` (ইউজারের রোল `platformExpenseDeleteRoles`-এর মধ্যে কনি) নির্ণয় করে। `ConfigList<Expense>` বানায়, `FilterBar` হডারসহ (সব/মাইন/প্রতিটি স্ট্যাটাস) এবং প্রতিটি আইটেমের জন্য `_ExpenseRow` তৈরি করে যথাযথ কলব্যাক দিয়ে।

ক্লাস/উইজটে: `_ExpenseRow`

`StatelessWidget` — একটি খরচের কার্ড দেখায়, শিরোনাম/পরিমাণ, উপ-তথ্য (নম্বর, তারিখ, ভেন্ডর, ফাইলকারী), স্ট্যাটাস চিপ এবং পপআপ মেনুতে প্রয়োজ্য অ্যাকশন।

`Widget build(BuildContext context)`

খরচের স্ট্যাটাস অনুযায়ী কোন অ্যাকশন দেখানো হবে তা নির্ধারণ করে — পেন্ডিং হলে অনুমোদন/প্রত্যাখ্যান, অনুমোদিত হলে পরিশোধ-চলিতকরণ, এবং `onDelete` থাকলে মুছে ফেলো। `AppCard`-এ শিরোনাম, পরিমাণ, উপ-তথ্য সারি, স্ট্যাটাস চিপ, এবং বজি হলে স্পনির নাহলে (অ্যাকশন থাকলে) `PopupMenuButton` দেখায়।

ক্লাস/উইজটে: `_MarkPaidDialog`

`StatefulWidget` — পরিশোধের পদ্ধতি ও রফোরেন্স নেওয়ার ডায়ালগ। উভয় ফিল্ড ঐচ্ছিক, তাই খালি অবস্থাতেও নিশ্চিতি করা যায়।

`State<_MarkPaidDialog> createState()`

`_MarkPaidDialogState` তৈরি করে।

ক্লাস: `_MarkPaidDialogState`

`_MarkPaidDialog`-এর স্টেটে, `_method` ও `_reference` টেক্সট কন্ট্রোলার রাখো।

`void dispose()`

`_method`, `_reference` কন্ট্রোলার ডিসপোজ করে সুপার কল করে।

`Widget build(BuildContext context)`

`AlertDialog` বানাও — "How it was paid" ও "Reference" টেক্সটফিল্ড, "Never mind" (বাতলি) এবং "Mark as paid" (দুই ফিল্ডের মান রেকর্ড রূপে পপ করে) বাটন।

ফাইল: `lib/features/platform/platform_models.dart`

সংক্ষিপ্ত বিবরণ: প্ল্যাটফর্ম-স্তরের (ক্রেস-টেন্যান্ট) মডেলে, DTO ও রোল-তালিকা কনস্ট্রাক্ট রাখো — কোম্পানি, ফচার ফ্ল্যাগ, প্ল্যাটফর্ম স্টাফ ইউজার, সাবস্ক্রিপশন প্ল্যান এবং সংশ্লিষ্ট রকিওয়েস্ট DTO।

টপ-লেভেলে কনস্ট্রাক্ট: `assignablePlatformRoles`

যাদের প্ল্যাটফর্ম স্টাফ হিসেবে নিয়োগ দেওয়া যায় এমন রোলের তালিকা (`SUPER_ADMIN`, `SYSTEM_ADMIN`, `SUPPORT_AGENT`, `SUPPORT_MANAGER`, `MARKETING_MANAGER`, `PLATFORM_ACCOUNTANT`, `SALES_MANAGER`)। `auth` মডেলে `platformRoles`-এর মরি।

টপ-লেভেলে কনস্ট্রাক্ট: `platformUserRoles`

প্ল্যাটফর্ম স্টাফের ডিরেক্টরি পিড়ার অনুমতি রাখা রোল (`SUPER_ADMIN`, `SYSTEM_ADMIN`) — `PlatformUserController`-এর ক্লাস-স্তরের গটে।

টপ-লেভেলে কনস্ট্যান্ট: platformExpenseRoles

প্ল্যাটফর্মের নজিস্ব বই দেখার/পরচালনার অনুমতি রাখা রোল (PLATFORM_ACCOUNTANT, SUPER_ADMIN)।

টপ-লেভেলে কনস্ট্যান্ট: platformExpenseDeleteRoles

খরচ সরাসরি মুছে ফেলার অনুমতি রাখা রোল — শুধু SUPER_ADMIN।

ক্লাস: UpdatePlatformUserRequest

PATCH /platform/users/{id}-এর DTO। পাসওয়ার্ড ছাড়া সব ফিল্ড আবশ্যিক; পাসওয়ার্ড ফাঁকা/নাল হলে বর্তমান লগইন অপরবর্তী থাকবে। রোল অবশ্যই assignablePlatformRoles-এর একটি হতে হবে।

কনস্ট্রাক্টর UpdatePlatformUserRequest({required firstName, required lastName, required email, required role, phone, password})

সব ফিল্ড ইনশিয়ালাইজ করে।

factory UpdatePlatformUserRequest.from(PlatformUser user)

একটি বর্তমান PlatformUser থেকে রকিওয়েস্ট বানায় (পাসওয়ার্ড ছাড়া, বাকি সব ফিল্ড কপি করে)।

Map<String, dynamic> toJson()

নাম/ইমেইল/ফোন ট্রিম করে JSON বানায়; পাসওয়ার্ড ট্রিম করা হয় না (হোয়াইটস্পেস পাসওয়ার্ডের অংশ) এবং খালি/নাল হলে পুরোপুরি বাদ দেওয়া হয় — এটাই ব্যাকএন্ডকে জানায় পাসওয়ার্ড রসিটে না করতে।

টপ-লেভেলে কনস্ট্যান্ট: companyStatusRoles

তনেন্ট সাসপেন্ড/র-অ্যাক্টিভিটে/ডঅ্যাক্টিভিটে করার অনুমতি রাখা রোল (SUPER_ADMIN, SYSTEM_ADMIN, PLATFORM_ACCOUNTANT)।

টপ-লেভেলে কনস্ট্যান্ট: companyPlanRoles

তনেন্টকে অন্য প্লানে সরানোর অনুমতি রাখা রোল (SUPER_ADMIN, SYSTEM_ADMIN, SALES_MANAGER, PLATFORM_ACCOUNTANT)।

টপ-লেভেলে কনস্ট্যান্ট: impersonationRoles

ইমপারসোনেশন সেশন শুরু করার অনুমতি রাখা রোল (SUPER_ADMIN, SYSTEM_ADMIN, SUPPORT_AGENT, SUPPORT_MANAGER)।

টপ-লেভেলে কনস্ট্যান্ট: impersonationHistoryRoles

ইমপারসোনেশন অডিট ট্রাইল দেখার অনুমতি রাখা রোল (SUPER_ADMIN, SYSTEM_ADMIN, SUPPORT_MANAGER)।

ক্লাস: CompanyStatus

abstract final class — কোম্পানির স্ট্যাটাস কোডের কনস্ট্যান্ট রাখবে।

স্ট্যাটিক কনস্ট্যান্ট: pendingVerification, trial, active, suspended, deactivated

প্রতিটি স্ট্যাটাস স্ট্রিং কোড।

স্ট্যাটিক কনস্ট্যান্ট: all

উপরে পাঁচটি স্ট্যাটাসের তালিকা।

স্ট্যাটিক কনস্ট্যান্ট: live

{trial, active} — এখনো লেনদেন করা তনোন্টের স্ট্যাটাস স্টে।

ক্লাস: Company

একটি তনোন্টের মডেল — নাম, সাবডোমেইন, স্ট্যাটাস, সাবস্ক্রিপশন প্ল্যান, ট্রায়াল-ম্যোদ ইত্যাদি সহ।

কনস্ট্রাক্টর Company({...})

সব ফিল্ড ইনশিয়ালাইজ করে (id, companyName, subdomain, status, subscriptionPlan, trialExpired, createdAt আবশ্যিক; বাকি ঐচ্ছিক)।

bool get isLive

স্ট্যাটাস CompanyStatus.live-এর মধ্যে আছে কিনা রটার্ন করে।

bool get isSuspended

স্ট্যাটাস suspended কিনা রটার্ন করে।

bool get isDeactivated

স্ট্যাটাস deactivated কিনা রটার্ন করে।

bool get needsAttention

ট্রায়ালে আছে ও ম্যোদ শেষ হয়ে গেছে কিনা রটার্ন করে (সেলসের ফলো-আপ প্রয়োজন)।

int? get daysUntilExpiry

subscriptionEnd নাল বা পার্স-অযোগ্য হলে নাল রটার্ন করে; নাহলে আজকের তারিখ থেকে ম্যোদ শেষের তারিখ পর্যন্ত দিনের সংখ্যা (ম্যোদ পার হলে ঋণাত্মক) রটার্ন করে।

bool get expiringSoon

লাইভ অবস্থায় থাকা ও ম্যোদ ৩০ দিনের মধ্যে (০ থেকে ৩০) শেষ হচ্ছে কিনা রটার্ন করে।

factory Company.fromJson(Map<String, dynamic> json)

JSON থেকে Company অবজেক্ট বানায়, প্রতিটি ফিল্ডের জন্য যুক্তসিঙিত ডফিল্ট মান দিয়ে।

ক্লাস: FeatureFlag

একটি প্ল্যাটফর্ম-ওয়াইড ফচার সুইচের মডেল — id, flagKey, enabled, description, updatedAt

কনস্ট্রাক্টর FeatureFlag({required id, required flagKey, required enabled, description, updatedAt})

সব ফিল্ড ইনশিয়ালাইজ করে।

String get label

flagKey (যেমন AI_ASSISTANT_ENABLED) কে সেন্টেন্স-কসে লবেলে রূপান্তর করে (যেমন "AI assistant enabled")। ছোট (৩ অক্ষর বা কম) শব্দ আপারকসে রাখা হয় যাত অ্যাক্রোনিমি (AI, SMS, API, PDF) নষ্ট না হয়; বাকগুলো লোয়ারকসে করা হয়, শুধু প্রথম শব্দটি অ্যাক্রোনিমি না হলে ক্যাপিটাইজ করা হয়।

FeatureFlag toggled()

enabled উল্টায় নতুন FeatureFlag অবজেক্ট রিটার্ন করে, বাকি ফিল্ড অপরিবর্তিত।

factory FeatureFlag.fromJson(Map<String, dynamic> json)

JSON থেকে FeatureFlag বানায়, ডিফল্ট মান সহ।

ক্লাস: PlatformUser

একজন প্ল্যাটফর্ম স্টাফ সদস্যের মডেলে — নাম, ইমেইল, রোল, সক্রিয়তা, ইমেইল যাচাই-অবস্থা ইত্যাদি।

কনস্ট্রাক্টর PlatformUser({required id, required firstName, required lastName, required email, required role, required active, required emailVerified, required createdAt, phone})

সব ফিল্ড ইনশিয়ালাইজ করে।

String get fullName

প্রথম ও শেষ নাম জোড়া লাগিয়ে ট্রিম করে রিটার্ন করে।

String get initials

নাম থেকে ইনশিয়াল বানায় — উভয় নাম ফাঁকা হলে ইমেইলের প্রথম অক্ষর (বা ?); একটি ফাঁকা হলে অন্যটির প্রথম অক্ষর; দুটোই থাকলে দুই নামের প্রথম অক্ষর জোড়া লাগিয়ে আপারকেসে করে।

bool get isUnusable

active মথিয়া বা emailVerified মথিয়া হলে সত্য রিটার্ন করে — অ্যাকাউন্ট আছে কিন্তু এখনো ব্যবহারযোগ্য নয়।

factory PlatformUser.fromJson(Map<String, dynamic> json)

JSON থেকে PlatformUser বানায়, ডিফল্ট মান সহ।

ক্লাস: SubscriptionPlanOption

একটি সাবস্ক্রিপশন প্ল্যানের মডেলে — যেটোতে কোম্পানি সরানো যায়। প্ল্যানের ডটো সার্ভার থেকে আসে, কোডে হার্ডকোড করা নেই।

কনস্ট্রাক্টর SubscriptionPlanOption({required key, required name, id, description, active = true, price, billingCycle})

সব ফিল্ড ইনশিয়ালাইজ করে।

factory SubscriptionPlanOption.fromJson(Map<String, dynamic> json)

JSON থেকে প্ল্যান বানায়। key নির্ণয়ে code, তারপর planKey, তারপর key, তারপর সবশেষে ফলব্যাক হিসেবে name — এটি ভুল হলেও ফাঁকা প্ল্যানের চেষ্টা ভালো। বাকি ফিল্ড ডিফল্ট সহ পার্স করা হয়।

টপ-লেভেলে কনস্ট্যান্ট: billingCycles

সাবস্ক্রিপশন প্ল্যান যে বিলিং সাইকেলে চলতে পারে তার তালিকা — MONTHLY, QUARTERLY, YEARLY, ONE_TIME।

ক্লাস: CreatePlatformUserRequest

POST /platform/users-এর DTO — নতুন প্ল্যাটফর্ম স্টাফ ও তাদের লগইন তৈরি করে। রোল অবশ্যই assignablePlatformRoles-এর একটি হতে হবে।

কনস্ট্রাক্টর CreatePlatformUserRequest({required firstName, required lastName, required email, required password, required role, phone})

সব ফিল্ড ইনশিয়ালাইজ করে।

Map<String, dynamic> toJson()

নাম/ইমেইল/ফোন ট্রমি করে JSON বানায়ে; পাসওয়ার্ড ট্রমি করা হয় না (হোয়াইটস্পেসে অংশ)।

ক্লাস: CreateSubscriptionPlanRequest

POST /subscription-plans-এর DTO — কোড প্ল্যানের পরচয়, সারভার-সাইডে আপার-কসে করে ইউনিকিনসে চকে করা হয়।

কনস্ট্রাক্টর CreateSubscriptionPlanRequest({required code, required name, required billingCycle, required price, description})

সব ফিল্ড ইনশিয়ালাইজ করে।

Map<String, dynamic> toJson()

কোড ট্রমি করে আপারকসে করে, নাম ট্রমি করে, ববিরণ ট্রমি করে খালি না হলে যোগ করে; বলিং সাইকেলে ও দাম যথার্থভাবে যোগ করে JSON বানায়ে।

ফাইল: lib/features/platform/platform_repository.dart

সংক্ষিপ্ত ববিরণ: SaaS অপারেটরেরে নিজস্ব কনসোলরে রপিজিটরি ও প্রোভাইডার — কোম্পানি, ফচার ফ্ল্যাগ, প্ল্যাটফর্ম স্টাফ, সাবস্ক্রিপশন প্ল্যান এবং প্ল্যাটফর্মেরে নিজস্ব খরচ পরচালনা করে। ইমপারসোনেশন-অ্যাকশন ও কাস্টম-রোল স্ক্রিনি ইচ্ছাকৃতভাবে এখানে নেই।

ক্লাস: PlatformRepository

ক্রস-টেন্যান্ট প্ল্যাটফর্ম কনসোলরে সব API কল ধারণ করে। শুধু প্ল্যাটফর্ম স্টাফ এতে পৌঁছাতে পারে; ব্যাকএন্ড আবার প্রতিটি কলে রোল যাচাই করে।

কনস্ট্রাক্টর PlatformRepository(this._api)

ApiClient ইনজেক্ট করে।

Future<PagedResponse<Company>> companies({status, plan, keyword, page = 0, size = 20})
/companies-এ পজেড GET কল করে, স্ট্যাটাস/প্ল্যান/কীওয়ার্ড কোয়েরি-প্যারামিটার হিসেবে পাঠায়।

Future<Company> changeStatus(int id, String status)

তনোন্টেরে স্ট্যাটাস বদলায় — status কোয়েরি-প্যারামিটার হিসেবে পাঠানো হয় (বডি হিসেবে নয়, কারণ কন্ট্রোলার @RequestParam ঘোষণা করে; বডি পাঠালে চুপচাপ উপেক্ষা হতো)। PATCH
/companies/{id}/status?status=... কল করে Company রটার্ন করে।

Future<Company> changePlan(int id, String plan, {amountPaid, transactionRef})

তনোন্টকে অন্য প্লানে সরায়। amountPaid ও transactionRef ঐচ্ছকি, সাবস্ক্রিপশন হিস্ট্রিতে রেকর্ড হয় — ছাড়া পাঠালে প্ল্যাটফর্মেরে রভেনিউ সংখ্যায় প্রতফিলতি হয় না। কোয়েরি-স্ট্রিং জোড়া লাগিয়ে PATCH কল করে Company রটার্ন করে।

`Future<List<SubscriptionPlanOption>> plans({bool activeOnly = true})`

`/subscription-plans`-এ GET কল করে (activeOnly কয়েরেসিহ), `SubscriptionPlanOption` লিস্টে পার্স করে খালিকী প্ল্যান বাদ দিয়ে। `ApiException` ধরা পড়লে খালি লিস্ট রটার্ন করে — প্ল্যান ছাড়া প্ল্যান-অ্যাকশন কবেল অফার করা হয় না।

`Future<PagedResponse<ImpersonationAuditEntry>> impersonationHistory({companyId, page = 0, size = 20})`

`/platform-admin/impersonate/history`-এ পজেড GET কল করে। কন্ট্রোলারের বাকি অংশের চয়ে সংকীর্ণ পড়ার-অনুমতি — একজন সাপোর্ট এজেন্ট সেশন খুলতে পারলেও সবার হিস্ট্রি রিভিডি করতে পারে না।

`Future<List<FeatureFlag>> flags()`

`/feature-flags`-এ GET কল করে, `FeatureFlag` লিস্টে পার্স করে `flagKey` অনুসারে সর্ট করে রটার্ন করে।

`Future<FeatureFlag> toggleFlag(String key)`

ফ্ল্যাগের `key` দিয়ে (id দিয়ে নয়) `$_flags/$key/toggle`-এ PATCH কল করে টগল করা ফ্ল্যাগ রটার্ন করে।

`Future<SubscriptionPlanOption> updatePlan(int id, CreateSubscriptionPlanRequest request)`

প্ল্যান বদলায়। কোড বদলানো যায় না এবং পাঠানো হয় না (`request.toJson().remove('code')`) — কোম্পানিও সাবস্ক্রিপশন হিস্ট্রি উভয়েই কোডটি রিফোরেন্স করে, বদলালে সেগুলো এতমি হয়ে যাবে। PATCH `/subscription-plans/{id}` কল করে।

`Future<SubscriptionPlanOption> togglePlan(int id)`

প্ল্যানকে মেনু থেকে সরায় বা ফেরত আনে (`/subscription-plans/{id}/toggle-active`-এ PATCH); ইতিমধ্যে সেই প্ল্যানে থাকা কোম্পানি প্রভাবিত হয় না।

`Future<PlatformUser> createUser(CreatePlatformUserRequest request)`

POST `_users` করে প্ল্যাটফর্ম স্টাফ ও তার লগইন তৈরিকরে। ডুপ্লিকেটে ইমেইল "Email already exists" এবং অবধৈ রোল "Invalid platform role selected" বার্তা ফেরত দিয়ে।

`Future<SubscriptionPlanOption> createPlan(CreateSubscriptionPlanRequest request)`

POST `/subscription-plans` করে নতুন সাবস্ক্রিপশন প্ল্যান তৈরিকরে; কোড ইতিমধ্যে ব্যবহৃত হলে প্রত্যাখ্যাত হয়।

`Future<PagedResponse<PlatformUser>> users({page = 0, size = 20})`

`_users`-এ পজেড GET কল করে প্ল্যাটফর্ম স্টাফের তালিকা আনে।

`Future<PlatformUser> updateUser(int id, UpdatePlatformUserRequest request)`

`$_users/{id}`-এ PATCH কল করে প্ল্যাটফর্ম স্টাফ সদস্যের তথ্য বদলায়। পাসওয়ার্ড ও ফোন ছাড়া সব ফিল্ড আবশ্যিক; ফাঁকা পাসওয়ার্ড লগইন অপরিবর্তিত রাখা।

`Future<PagedResponse<Expense>> platformExpenses({status, vendor, mineOnly = false, page = 0, size = 20})`

প্ল্যাটফর্মের নিজস্ব খরচের তালিকা আনে। `status` ও `vendor` উভয়েই পাথ-সেগমেন্ট (কয়েরে-প্যারামিটার নয়), একবারে একটিই প্রযোজ্য হতে পারে — তাই কলার সর্বোচ্চ একটাই পাঠায়। `switch` এক্সপ্রেশন দিয়ে পাথ নির্ণয় করে (স্ট্যাটাস > ভেন্ডর > মাইন-ওনলি > সবগুলো) এবং পজেড GET কল করে।

Future<Expense> createPlatformExpense(CreateExpenseRequest request)

\$_platformExpenses-এ POST করে প্ল্যাটফর্মের খাতায় নতুন খরচ ফাইল করে।

Future<void> approvePlatformExpense(int id, String notes)

\$_platformExpenses/\$id/approve-এ POST করে, notes আবশ্যিক কয়েরে-প্যারামটার হিসেবে পাঠায় (কন্ট্রোলার @RequestParam String notes নন-অপশনাল ঘোষণা করে, তাই ফাঁকা হলে 400 এরর হয়)।

Future<void> rejectPlatformExpense(int id, String reason)

\$_platformExpenses/\$id/reject-এ POST করে, reason একইভাবে আবশ্যিক কয়েরে-প্যারামটার হিসেবে পাঠায়।

Future<void> markPlatformExpensePaid(int id, {method, reference})

রহিমবার্সমেন্ট রেকর্ড করে। method ও reference উভয়ই ঐচ্ছিক — খালি বা নাল না হলে কয়েরে-স্ট্রিং-এ যোগ হয়। POST \$_platformExpenses/\$id/mark-as-paid\$query কল করে।

Future<void> deletePlatformExpense(int id)

\$_platformExpenses/\$id-এ DELETE কল করে খরচ সম্পূর্ণ মুছে ফেলে। শুধু SUPER_ADMIN — দেখুন platformExpenseDeleteRoles।

টপ-লেভেলে প্রোভাইডার: platformRepositoryProvider

Provider<PlatformRepository> — apiClientProvider থেকে ApiClient নিয়ে PlatformRepository ইনস্ট্যান্স বানায়।

ক্লাস: CompanyFilterController

Notifier<String?> — কোম্পানি তালিকা কোন স্ট্যাটাসে ফিল্টার করা তা রাখবে; নাল মান সবগুলো।

String? build()

প্রাথমিক অবস্থা নাল রিটার্ন করে।

void set(String? status)

নতুন মান আগেরটির সমান না হলে স্টেটে আপডেট করে।

টপ-লেভেলে প্রোভাইডার: companyFilterProvider

NotifierProvider<CompanyFilterController, String?>।

ক্লাস: CompanySearchController

Notifier<String> — তনোনট তালিকায় ফ্রন্ট-টেক্সট সার্চের কীওয়ার্ড রাখে।

String build()

প্রাথমিক অবস্থা খালি স্ট্রিং রিটার্ন করে।

void set(String keyword)

ট্রমি করা কীওয়ার্ড আগেরটির সমান না হলে স্টেটে আপডেট করে।

টপ-লেভেলে প্রোভাইডার: companySearchProvider

NotifierProvider<CompanySearchController, String>।

ক্লাস: CompaniesController

`AsyncNotifier<PagedState<Company>>` (`PagedLoader<Company>` মক্সিনি সহ) — তনোন্ট তালিকা পজেনিশেন সহ লোড করে এবং স্ট্যাটাস/প্ল্যান পরিবর্তন প্রয়োগ করে।

`Future<PagedState<Company>>` `build()`

`currentUserProvider`, `companyFilterProvider`, `companySearchProvider` ওয়াচ করে প্রথম পজে লোড করে।

`Future<PagedResponse<Company>>` `fetchPage(int page)`

বর্তমান ফিল্টার ও কীওয়ার্ড দিয়ে `platformRepositoryProvider.companies` কল করে নব্বির্দষ্ট পজে আনে।

`Future<void>` `setStatus(int id, String status)`

`changeStatus` কল করে ফলাফল `_applyOrRefresh`-এ পাস করে — তালিকায় ঘুরে বড়োনো অ্যাডমিনি যনে জায়গা না হারায়, তাই পুরো তালিকা রলিড না করে সারবিদলানো হয়।

`Future<void>` `setPlan(int id, String plan, {amountPaid, transactionRef})`

`changePlan` কল করে ফলাফল `_applyOrRefresh`-এ পাস করে।

`void` `_applyOrRefresh(Company updated)`

বর্তমান ফিল্টার সক্রিয় থাকলে এবং নতুন স্ট্যাটাস ফিল্টারের বাইরে চলে গেলে সারিতালিকা থেকে বাদ দেয়; নাহলে সারিজায়গায় আপডেট করে।

টপ-লভেলে প্রোভাইডার: companiesProvider

`AsyncNotifierProvider<CompaniesController, PagedState<Company>>` |

টপ-লভেলে প্রোভাইডার: subscriptionPlansProvider

`FutureProvider<List<SubscriptionPlanOption>>` — `currentUserProvider` ওয়াচ করে `platformRepositoryProvider.plans()` (শুধু সক্রিয় প্ল্যান) রটার্ন করে।

ক্লাস: FeatureFlagsController

`AsyncNotifier<List<FeatureFlag>>` — ফচার ফ্ল্যাগের তালিকা লোড, রফ্রেশ ও অপটমিস্টিকি টগল করে।

`Future<List<FeatureFlag>>` `build()`

`currentUserProvider` ওয়াচ করে `platformRepositoryProvider.flags()` রটার্ন করে।

`Future<void>` `refresh()`

`AsyncValue.guard` দিয়ে `flags()` পুনরায় চালিয়ে স্টেটে আপডেট করে।

`Future<void>` `toggle(String key)`

প্রথমে তালিকায় নব্বির্দষ্ট ফ্ল্যাগের অবস্থান খুঁজে অপটমিস্টিকিভাবে টগল করে দেখায় টাচ করলে সুইচ না নড়লে খারাপ লাগে। তারপর `toggleFlag` কল করে সারভারের নশ্চিতি মান দিয়ে প্রতস্থাপন করে; ব্যর্থ হলে আগের স্টেটে ফরিয়ে এনে এরর রথ্রো করে।

টপ-লভেলে প্রোভাইডার: featureFlagsProvider

`AsyncNotifierProvider<FeatureFlagsController, List<FeatureFlag>>` |

ক্লাস: PlatformUsersController

AsyncNotifier<PagedState<PlatformUser>> (PagedLoader<PlatformUser> মক্‌সিনি সহ) — প্ল্যাটফর্ম স্টাফ তালিকা পজেনিশেন, তৈরিও এডটি করে।

Future<PagedState<PlatformUser>> build()

currentUserProvider ওয়াচ করে প্রথম পজে লোড করে।

Future<PagedResponse<PlatformUser>> fetchPage(int page)

platformRepositoryProvider.users(page: page) কল করে।

Future<PlatformUser> create(CreatePlatformUserRequest request)

createUser কল করে, সফল হলে refresh() করে এবং তৈরি হওয়া ইউজার রটাইন করে।

Future<void> updateItem(int id, UpdatePlatformUserRequest request)

updateUser কল করে ফলাফল দিয়ে তালিকার সংশ্লিষ্ট সারি প্রতিস্থাপন করে।

টপ-লেভেলে প্রোভাইডার: platformUsersProvider

AsyncNotifierProvider<PlatformUsersController, PagedState<PlatformUser>> |

ক্লাস: ImpersonationHistoryController

AsyncNotifier<PagedState<ImpersonationAuditEntry>> (PagedLoader<ImpersonationAuditEntry> মক্‌সিনি সহ) — ইমপারসোনেশন অডিট হিস্ট্রি পজেনিশেন সহ লোড করে।

Future<PagedState<ImpersonationAuditEntry>> build()

currentUserProvider ওয়াচ করে প্রথম পজে লোড করে।

Future<PagedResponse<ImpersonationAuditEntry>> fetchPage(int page)

platformRepositoryProvider.impersonationHistory(page: page) কল করে।

টপ-লেভেলে প্রোভাইডার: impersonationHistoryProvider

AsyncNotifierProvider<ImpersonationHistoryController, PagedState<ImpersonationAuditEntry>> |

ফাইল: lib/features/platform/platform_screen.dart

সংক্ষিপ্ত ববিরণ: এটি Zuhoo প্ল্যাটফর্ম স্টাফদের অপারটের কনসোল — কোম্পানি (টেন্যান্ট), ফচার ফ্ল্যাগ, স্টাফ ডিরেক্টরি, কনটেক্সট সুইচ এবং ইমপারসোনেশন অডিট ট্রাইল একসাথে দেখানো হয়। রোল-ভিত্তিকি গটেং করা হয়েছে যাতো যার যতটুকু অ্যাক্সেসে আছে ঠিকি ততটুকুই ট্রাব ও অ্যাকশন দেখানো হয়, যদও প্রকৃত নরিপত্তা যাচাই ব্যাকএন্ডই হয়।

ক্লাস/উইজেট: PlatformScreen

মূল স্ক্রিনি, যা ইউজারের রোল অনুযায়ী কোন কোন ট্রাব (Companies, Flags, Staff, Context, Audit, Expenses) দেখানো হবে তা নরিধারণ করে এবং DefaultTabController দিয়ে সেগুলো সাজায়।

প্ল্যাটফর্ম-স্টাফ না হলে "Not available to you" দেখায়।

PlatformScreen({super.key, this.initialTabLabel})

কনস্ট্রাক্টর, যা ঐচ্ছকি initialTabLabel নেয় — কোন নরিদষ্টি ট্রাব খুলে স্ক্রিনি প্রবশে করানোর জন্য ব্যবহার হয়।

Widget build(BuildContext context, WidgetRef ref)

প্রথমত ইউজার প্ল্যাটফর্ম-স্টাফ কনি যাচাই করে; না হলে একটি লিক করা `EmptyState` দেখায়। এরপর `canAudit`, `canContext`, `canStaff`, `canExpenses` — চারটি পৃথক রোল-গেটে হিসাব করে, কারণ ব্যাকএন্ডে প্রতিটি এন্ডপয়েন্টের জন্য আলাদা রোল সেটে প্রযোজ্য। এই গেটগুলো অনুযায়ী `labels` লিস্ট এবং `createOn` ম্যাপ (কোন ট্যাবে কোন FAB অ্যাকশন থাকবে) তৈরি করে। শেষে `TabBar` ও `TabBarView` সহ `Scaffold` রিটার্ন করে, যখনে সক্রিয় ট্যাব অনুযায়ী ফ্লোটিং অ্যাকশন বাটন পরিবর্তিত হয়।

ক্লাস/উইজটে: `_CompaniesTab`

"Companies" ট্যাবের বডি — সার্চ, স্ট্যাটাস ফিল্টার এবং পেজড কোম্পানি লিস্ট একত্রে দেখায়।

Widget build(BuildContext context, WidgetRef ref)

`companiesProvider` থেকে কন্ট্রোলার নথি `_CompanySearch`, `_StatusFilter` এবং `PagedList<Company>` সাজায়, যখনে প্রতিটি আইটেমে `_CompanyCard` দিয়ে রেন্ডার হয়।

ক্লাস/উইজটে: `_CompanySearch`

কোম্পানি অনুসন্ধানের জন্য একটি সার্চ ইনপুট ফিল্ড দেখায়।

Widget build(BuildContext context, WidgetRef ref)

`AppSearchField` রেন্ডার করে, যার `onChanged` কলব্যাক `companySearchProvider` নোটিফায়ারে টাইপ করা মান পাঠায়।

ক্লাস/উইজটে: `_StatusFilter`

কোম্পানির স্ট্যাটাস অনুযায়ী ফিল্টার করার জন্য অনুভূমিক চপি-স্ক্রল দেখায় (All + প্রতিটি `CompanyStatus`)।

Widget build(BuildContext context, WidgetRef ref)

`companyFilterProvider` থেকে নির্বাচতি স্ট্যাটাস পড়ে এবং `ListView.separated` দিয়ে চপিগুলো আঁকে; ট্যাপ করলে `companyFilterProvider.notifier.set(status)` কল হয়, নির্বাচতি চপি ব্র্যান্ড কালারে হাইলাইট হয়।

ক্লাস/উইজটে: `_CompanyCard` / `_CompanyCardState`

একটি একক কোম্পানির কার্ড, যখনে নাম, সাবডোমেইন, প্ল্যান, ময়োদ-সংক্রান্ত সতর্কতা এবং রোল অনুযায়ী অ্যাকশন বাটন (Access, Suspend/Reactivate/Deactivate, Change plan) দেখানো হয়।

Future<void> `_setStatus(String status)`

কোম্পানির স্ট্যাটাস পরিবর্তন করে। স্ট্যাটাস `suspended` বা `deactivated` হলে প্রথমত নিশ্চিতকরণ ডায়ালগ দেখায়, কারণ এটি গ্রাহকের অ্যাক্সেস বন্ধ করে দেয়। নিশ্চিত হলে

`companiesProvider.notifier.setStatus` কল করে, সফল/ব্যর্থ হলে সন্ধ্যাকবার দেখায়, এবং সবসময় `_busy` ফ্ল্যাগ রিসেট করে।

Future<void> `_accessCompany()`

এই কোম্পানির ভেতরে একটি ইমপারসোনেশন সেশন শুরু করে। প্রথমত `_ReasonDialog` দেখিয়ে কারণ সংগ্রহ করে (যা অডটি লগে রেকর্ড হয়), তারপর `authControllerProvider.notifier.startImpersonation` কল করে এবং সফল হলে `Routes.home`-এ নভিগেটে করে, কারণ এই স্ক্রিনি প্ল্যাটফর্ম-স্টাফদের জন্য গেটেড এবং ইমপারসোনেশনের পর ইউজার আর প্ল্যাটফর্ম-স্টাফ হিসেবে কাজ করবে না।

Future<void> _changePlan()

একটি বিটম শাট দৈখ্যে যখনে উপলব্ধ সাবস্ক্রিপশন প্ল্যানগুলোর তালিকা থাকে (subscriptionPlansProvider থেকে)। নির্বাচতি প্ল্যান আগরেটার চয়ে আলাদা হলে companiesProvider.notifier.setPlan কল করে এবং ফলাফল সন্ধ্যাকবারে জানায়। প্ল্যান ফাঁকা থাকলে কছিই করে না।

Widget build(BuildContext context)

কোম্পানির নাম, সাবডোমেইন, স্ট্যাটাস চপি, প্ল্যান, মালকিরে নাম এবং ময়োদ-সংক্রান্ত সতর্কতা (needsAttention/expiringSoon) দৈখ্যে। তনিটি পৃথক রোল-গটে (canImpersonate, canSetStatus, canSetPlan) হিসাব করে কোন অ্যাকশন বাটন দৈখ্যে তা ঠকি করে, এবং _busy হলে লোডার দৈখ্যে।

ক্লাস/উইজটে: _ReasonDialog / _ReasonDialogState

ইমপারসোনেশন শুরু করার আগে বাধ্যতামূলক কারণ সংগ্রহেরে ডায়ালগ। ন্যূনতম ৮ অক্ষরের বৈধতা ক্লায়েন্ট-সাইডে প্রয়োগ করে, কারণ ব্যাকএন্ড শুধু ফাঁকা-নয় এমন চকে করে।

void initState()

_controller-এ একটি লিসিনোর যোগ করে যাতে টাইপ করার সাথে সাথে setState কল হয় এবং বৈধতা রয়িলে-টাইমে আপডেটে হয়।

void dispose()

_controller ডিসপোজ করে।

Widget build(BuildContext context)

একটি সতর্কতা ব্যানার, টেক্সটফিল্ড (কারণেরে জন্য) এবং প্রয়োজনে সহায়ক বার্তা দৈখ্যে। "Access company" বাটন তখনই সক্রিয় হয় যখন কারণ ন্যূনতম দৈর্ঘ্যেরে শর্ত পূরণ করে।

ক্লাস/উইজটে: _Action

একটি সাধারণ পুনঃব্যবহারযোগ্য আউটলাইনড আইকন-বাটন যা লবেলে, আইকন, রঙ এবং ট্যাপ কলব্যাক নিয়ে।

Widget build(BuildContext context)

নির্দিষ্ট রঙের OutlinedButton.icon রেন্ডার করে।

ক্লাস/উইজটে: _FlagsTab

প্ল্যাটফর্মেরে ফচার ফ্ল্যাগগুলোর তালিকা দৈখ্যে এবং রোল অনুযায়ী টগল করার অনুমতি নিয়ন্ত্রণ করে।

Widget build(BuildContext context, WidgetRef ref)

featureFlagsProvider-এর AsyncValue হ্যান্ডল করে (loading/error/data)। ফাঁকা হলে EmptyState দৈখ্যে। SUPER_ADMIN/SYSTEM_ADMIN রোল থাকলে canToggle = true হয় এবং একটি তথ্যভিত্তিক ব্যানার দৈখ্যে যে পরবর্তন সব কোম্পানির জন্য তাৎক্ষণিক প্রয়োজ্য হবে; না থাকলে রডি-অনলিবার্তা দৈখ্যে। প্রতিটি ফ্ল্যাগ _FlagRow দিয়ে রেন্ডার হয়।

ক্লাস/উইজটে: _FlagRow

একটি একক ফচার ফ্ল্যাগেরে সুইচ-লিস্ট-টাইল।

Widget build(BuildContext context, WidgetRef ref)

SwitchListTile রন্ডার করে; canToggle মথিযা হলে onChanged নাল রাখা হয় (দৃশ্যত নষিক্রয়ি সুইচ)। টগল করলে featureFlagsProvider.notifier.toggle কল হয় এবং ত্রুটিহলে সন্যাকবারে দেখানো হয়।

ক্লাস/উইজটে: _StaffTab

প্ল্যাটফর্ম স্টাফদরে ডরিকেটরি — প্রতটি স্টাফরে নাম, ইমইল, রোল চপি এবং একটি এডিট বাটন সহ।

Widget build(BuildContext context, WidgetRef ref)

platformUsersProvider থেকে পজেড লস্টি তরৈকিরে; প্রতটি ইউজাররে জন্য অ্যাভাটার, নাম/ইমইল, "Email not verified"/"Deactivated" ব্যাজ (যদি isUnusable), রোল চপি, এবং showEditPlatformUserSheet খোলার জন্য এডিট আইকন বাটন দেখায়।

ক্লাস/উইজটে: _AuditTab

ইমপারসোনেশন সেশনের অডিট ট্রইল দেখানোর ট্যাব।

Widget build(BuildContext context, WidgetRef ref)

impersonationHistoryProvider থেকে পজেড লস্টি রন্ডার করে, প্রতটি এন্টরি _AuditRow দিয়ে দেখানো হয়।

ক্লাস/উইজটে: _AuditRow

একটি একক ইমপারসোনেশন লগ এন্টরির কার্ড।

Widget build(BuildContext context)

কোম্পানির নাম, "Not ended" চপি (যদি openEnded), স্টাফরে নাম, কারণ (না থাকলে "No reason recorded." ইটালিকি টেক্সটে), এবং সময়সীমা (শুরু ও শেষ, বা শুধু শুরু যদি এখনো চলমান) দেখায়।

ফাইল: lib/features/portal/edit_portal_profile_sheet.dart

সংক্ষিপ্ত ববিরণ: পোর্টাল ক্লায়ন্ট নজিরে প্রোফাইলরে সীমতি অংশ (কোম্পানি, ইন্ডাস্ট্রি, ওয়েবসাইট, বলিং ঠিকানা) সম্পাদনা করার জন্য একটি বটম শাটি। নাম, ইমইল বা অ্যাকাউন্ট স্ট্যাটাস এখান থেকে পরবর্তনযোগ্য নয়।

টপ-লেভেলে ফাংশন

Future<void> showEditPortalProfileSheet(BuildContext context, ClientProfile profile)

showModalBottomSheet কল করে _EditPortalProfileSheet উইজটেকে বটম শাটি হিসেবে খোলে, বর্তমান profile পাস করে।

ক্লাস/উইজটে: _EditPortalProfileSheet / _EditPortalProfileSheetState

প্রোফাইল সম্পাদনার ফর্ম — কোম্পানির নাম, ইন্ডাস্ট্রি, ওয়েবসাইট এবং বলিং ঠিকানার জন্য টেক্সট ফিল্ড সহ।

void initState()

widget.profile-এর বদ্যমান মান দিয়ে চারটি TextEditingController (_company, _industry, _website, _billingAddress) ইনশিয়ালাইজ করে।

`void dispose()`

চারট কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, ফোকাস সরিয়ে দেয়, `_submitting` স্টেট করে এবং

`portalRepositoryProvider().updateProfile` কল করে `UpdateMyProfileRequest` পাঠায়। সফল হলে `clientProfileProvider` ইনভ্যালিডেট করে (কারণ পুরো প্রোফাইল একাধিক স্ক্রিনি ব্যবহৃত হয়), শাট বন্ধ করে এবং সন্ধ্যাকবার দেখায়। ব্যর্থ হলে `_error` স্টেট করে।

`Widget build(BuildContext context)`

কীবোর্ড-সচেতন প্যাডিং সহ একটি স্ক্রলযোগ্য ফর্ম রেন্ডার করে — শিরোনাম, ত্রুটি বি্যানার (থাকলে), চারটি `TextFormField`, একটি সহায়ক তথ্য নোট (নাম/ইমেইল পরবর্তনরে জন্য অ্যাকাউন্ট ম্যানেজারকে জানাতে বলে), এবং একটি `LoadingButton` (Save details)।

ফাইল: `lib/features/portal/portal_billing_screen.dart`

সংক্ষিপ্ত বিবরণ: পোর্টাল ক্লায়েন্টদেরে বলিং স্ক্রিনি, যেখানে Invoices ও Payments দুটি ট্যাব আছে। ইনভয়সেরে বসিতারতি, বকযো/পরশিোধতি অবস্থা এবং PDF খোলার সুবিধা দেয়।

ক্লাস/উইজটে: `PortalBillingScreen`

বলিং স্ক্রিনিরে মূল কার্ঠামো, দুটি ট্যাব (Invoices, Payments) সহ `Scaffold`।

`Widget build(BuildContext context, WidgetRef ref)`

`DefaultTabController(length: 2)` এর ভেতরে একটি `AppBar` (টাইটলে "Billing", `TabBar`) ও `TabBarView` তৈরি করে যেখানে `_InvoicesTab` এবং `_ReceiptsTab` রয়েছে।

ক্লাস/উইজটে: `_InvoicesTab`

ক্লায়েন্টের ইনভয়সেগুলোর পেজেড তালিকা দেখায়।

`Widget build(BuildContext context, WidgetRef ref)`

`clientInvoicesProvider` থেকে কন্ট্রোলার নথি `PagedListView<Invoice>` রেন্ডার করে, প্রতিটি আইটেমে `_InvoiceCard`।

ক্লাস/উইজটে: `_InvoiceCard / _InvoiceCardState`

একটি একক ইনভয়সেরে কার্ড — নম্বর, বিবরণ, স্ট্যাটাস চপি, বকযো/মোট পরমাণ, ডিউ ডটে এবং PDF খোলার বাটন।

`Future<void> _open()`

`_downloading` স্টেট করে, `portalRepositoryProvider().invoicePdf(invoice)` দিয়ে PDF-এর লোকাল পথ আনবে, তারপর `OpenFile.open(path)` দিয়ে ফাইলটি খোলবে। কোনো অ্যাপ PDF খুলতে না পারলে সন্ধ্যাকবারে জানায়; ত্রুটি হিলেও তাই।

`Widget build(BuildContext context)`

ইনভয়সে নম্বর, বিবরণ (থাকলে), স্ট্যাটাস চপি (`isOverdue` হলে `overdue` দেখায়, নাহলে আসল স্ট্যাটাস), বকযো/মোট পরমাণ ও "due"/"paid in full" লবেলে, ডিউ ডটে, আংশিক পরশিোধের নোট (`isPartlyPaid`), এবং "Open PDF" বাটন (বা `_downloading` হলে লোডার) দেখায়।

ক্লাস/উইজটে: _ReceiptsTab

ক্লায়েন্টের পমেন্ট রসিটিরে পজেড তালিকা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

clientReceiptsProvider থেকে কন্ট্রোলার নথি PagedList<PaymentReceipt> রন্ডার করে; প্রতিটি রসিটি একটি চেকমার্ক আইকন, পরিমাণ, তারিখ+ পমেন্ট মেথড, সম্প্রতি ইনভয়েস নম্বর, স্ট্যাটাস চিহ্ন এবং রসিটি নম্বর সহ AppCard-এ দেখানো হয়।

ফাইল: lib/features/portal/portal_home_screen.dart

সংক্ষিপ্ত বিবরণ: পোর্টাল ক্লায়েন্টের হোম/ড্যাশবোর্ড স্ক্রিন — অভিবাদন, সারসংক্ষেপে পরিসংখ্যান, সক্রিয় সাবস্ক্রিপশন এবং সাম্প্রতিক রকিউয়েস্ট দেখায়। এটি স্টাফদের ড্যাশবোর্ড থেকে ইচ্ছাকৃতভাবে সম্পূর্ণ আলাদা স্ক্রিন হিসেবে ডিজাইন করা হয়েছে।

ক্লাস/উইজটে: PortalHomeScreen

মূল হোম স্ক্রিন, যা প্রোফাইল ও ইউজার তথ্য পড়ে সাবউইজটেগুলো সাজায় এবং পুল-টু-রিফ্রেশে সমর্থন করে।

Widget build(BuildContext context, WidgetRef ref)

clientProfileProvider ও currentUserProvider পড়ে RefreshIndicator-এর onRefresh-এ clientSummaryProvider, clientProfileProvider, clientSubscriptionsProvider ইনভ্যালিডেট করে এবং myRequestsProvider.notifier.refresh() অ্যাওয়ার্ড করে। ListView-এ _Greeting, _Summary, _Subscriptions ও _RecentRequests সাজায়।

ক্লাস/উইজটে: _Greeting

সময় অনুযায়ী অভিবাদন বার্তা (সকাল/বকিলে/সন্ধ্যা), ক্লায়েন্টের নাম, অ্যাকাউন্ট ম্যানেজারের নাম (থাকলে) এবং অ্যাভাটার দেখায়।

Widget build(BuildContext context)

বর্তমান ঘন্টা (DateTime.now().hour) অনুযায়ী "Good morning/afternoon/evening" নির্ধারণ করে এবং নাম, অভিবাদন, ম্যানেজার-কন্ট্রাক্ট লাইন ও Avatar সহ একটি Row রন্ডার করে।

ক্লাস/উইজটে: _Summary

চারটি পরিসংখ্যান কার্ড (Open requests, Completed, Unpaid invoices, Outstanding) দেখানো একটি গ্রিড।

Widget build(BuildContext context, WidgetRef ref)

clientSummaryProvider থেকে সারাংশ পড়ে চারটি StatCard তৈরি করে (দুটিতে ট্যাপ করলে যথাক্রমে requests/billing বুটে যায়)। টেক্সট স্কলে অনুযায়ী গ্রিড আইটেমের উচ্চতা (extent) সামঞ্জস্য করে, এবং GridView.builder-এ ২-কলামে সাজায়।

ক্লাস/উইজটে: _Subscriptions

ক্লায়েন্টের সক্রিয় প্ল্যান/সাবস্ক্রিপশনের সেকশন, খালি থাকলে কিছুই দেখায় না।

Widget build(BuildContext context, WidgetRef ref)

clientSubscriptionsProvider পড়ে; ফাঁকা/নাল হলে `SizedBox.shrink()` রিটার্ন করে (কারণ একবারে-এক-জব কোনো ক্লায়েন্টের কোনো প্ল্যান থাকে না, যা স্বাভাবিক)। নাহলে "Your plan" শিরোনাম সহ প্রতটি সাবস্ক্রিপশনের জন্য `_SubscriptionCard` দেখায়।

ক্লাস/উইজটে: `_SubscriptionCard`

একটি একক সাবস্ক্রিপশন প্ল্যানের কার্ড — নাম, স্ট্যাটাস, ব্লিং সাইকেল, কতো ব্যবহারের প্রোগ্রসে বার এবং পরবর্তী নবায়ন তারিখ।

Widget build(BuildContext context)

প্ল্যানের নাম/আইডি, স্ট্যাটাস চাপি, ব্লিং সাইকেল (থাকলে), `quotaUsedFraction` না-নাল হলে "Requests used" প্রোগ্রসে বার (১০০%+ হলে বপিদজনক রঙ), এবং `nextBillingDate` থাকলে নবায়নের তারিখ + অটো-রনিডি বন্ধ থাকার নোট দেখায়।

ক্লাস/উইজটে: `_RecentRequests`

সাম্প্রতিক ৩টি রিকোয়েস্টের তালিকা এবং "See all" লিংক।

Widget build(BuildContext context, WidgetRef ref)

myRequestsProvider থেকে আইটেমে পড়ে; ফাঁকা/নাল হলে `SizedBox.shrink()`। নাহলে `SectionHeader` (See all বাটন সহ) এবং প্রথম সর্বোচ্চ ৩টি রিকোয়েস্টের জন্য `RequestCard` দেখায়, ট্যাপে `openRequestDetail` কল হয়।

ফাইল: `lib/features/portal/portal_models.dart`

সংক্ষিপ্ত বিবরণ: পোর্টাল সংশ্লিষ্ট ডটো মডেলগুলো — ক্লায়েন্ট সারসংক্ষেপে, ইনভয়েস ও তার আইটেমে, পেমেন্ট রসিডি, প্যাকেজে সাবস্ক্রিপশন, ক্লায়েন্ট প্রোফাইল এবং প্রোফাইল-আপডেট রিকোয়েস্ট — সবগুলোতে JSON (de)সিরিয়লাইজেশন লজকি আছে।

ক্লাস: `ClientSummary`

ক্লায়েন্টের ড্যাশবোর্ডের হডেলাইন কাউন্ট (`GET /dashboard/client-summary`) ধারণ করে — পেন্ডিং, ইন-প্রোগ্রসে, সম্পূর্ণ রিকোয়েস্ট এবং অপারশোধিত ইনভয়েসের হিসাব।

`ClientSummary({this.pendingRequests = 0, this.inProgressRequests = 0, this.completedRequests = 0, this.unpaidInvoices = 0, this.outstandingInvoiceAmount = 0})`

সব ফিল্ডের জন্য ডিফল্ট মান-সহ কনস্ট্রাক্টর।

`int get openRequests`

`pendingRequests` ও `inProgressRequests`-এর যোগফল রিটার্ন করে।

`bool get owesMoney`

`unpaidInvoices > 0` কনি তা রিটার্ন করে।

`factory ClientSummary.fromJson(Map<String, dynamic> json)`

JSON থেকে সংখ্যাসূচক ফিল্ডগুলো নরিপদভাবে (নাল-সফে) পার্স করে `ClientSummary` তৈরি করে।

ক্লাস: InvoiceStatus (abstract final — enum-সদৃশ ধ্রুবক)

ব্র্যাকএন্ডের InvoiceStatus জাভা এনামেরে হুবহু প্রতরূপ — মান ভুল হলে সারভার 500 রটার্ন করে বলে সঠিকি বানান বজায় রাখা জরুরি।

স্ট্যাটিক ফিল্ডসমূহ

draft, issued, partiallyPaid, paid, overdue, cancelled, voided, refunded — প্রতটি ব্র্যাকএন্ড এনাম মানেরে স্ট্রিং ধ্রুবক।

static const all

সব স্ট্যাটাস মানেরে তালিকা।

static const settled

যসেব স্ট্যাটাসে আর কিছু সংগ্রহেরে অপেক্ষা নাই (paid, cancelled, voided, refunded) তার সেটে।

ক্লাস: InvoiceItem

ইনভয়সেরে একটি লাইন-আইটেম — বিবরণ, পরিমাণ, ইউনিট মূল্য।

double get amount

lineTotal থাকলে তা রটার্ন করে, নাহলে quantity * unitPrice হিসাব করে।

Map<String, dynamic> toJson()

বিবরণ ট্রান্সফর্ম করে এবং নোটস ফাঁকা না হলে তা যোগ করে একটি JSON ম্যাপ তৈরি করে।

factory InvoiceItem.fromJson(Map<String, dynamic> json)

JSON থেকে InvoiceItem তৈরি করে, নাল-সেফ ডিফল্ট মান সহ।

ক্লাস: Invoice

একটি সম্পূর্ণ ইনভয়স — মোট, বকযো, পরিশোধিত পরিমাণ, স্ট্যাটাস, আইটেম তালিকা ইত্যাদি।

bool get isSettled

স্ট্যাটাস InvoiceStatus.settled সেটে আছে কিনা যাচাই করে।

bool get isDraft

স্ট্যাটাস draft কিনা যাচাই করে — শুধু ড্রাফট ইনভয়সেই সম্পাদনা/মুছে ফেলা যায়।

bool get isPartlyPaid

paidAmount > 0 এবং balanceAmount > 0 উভয়ই সত্য কিনা যাচাই করে।

bool get isOverdue

নিষ্পত্তি হয়ে গেলে বা বকযো না থাকলে false। সারভার-স্ট্যাটাস overdue হলে true। নাহলে dueDate পার্স করে আজকের তারখের সাথে তুলনা করে নির্ধারণ করে — যাত নাইটলজিব রলিবেলে করার আগাই ক্ল্যাক্সনেট সময়মতো সতর্কতা দেখতে পায়ে।

factory Invoice.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড (আর্থিক মান, তারখ, আইটেম লিস্ট) নিরাপদভাবে পার্স করে Invoice অবজেক্ট তৈরি করে; ভেতরের d(key) হলে পার্স সংখ্যাসূচক ফিল্ড পার্স করে।

ক্লাস: PaymentReceipt

একটি পেমেন্ট রসিট — পরমাণ, তারিখ, স্ট্যাটাস, সম্পর্কিত ইনভয়েস নম্বর ইত্যাদি

factory PaymentReceipt.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড নাল-সফেভাবে পার্স করে PaymentReceipt তৈরি করে।

ক্লাস: PackageSubscription

একটি প্যাকেজ/প্ল্যান সাবস্ক্রিপশন — কোটা ব্যবহার, ব্লিৎ সাইকেল, অটো-রিনিউ ইত্যাদি

bool get isActive

স্ট্যাটাস 'ACTIVE' কনি যাচাই করে।

double? get quotaUsedFraction

requestQuota নাল বা 0 হলে নাল রিটার্ন করে (আনমিটার্ড প্ল্যানের জন্য বার দেখানো ভুল বার্তা দিয়ে বল); নাহলে requestsUsed / quota-কে ০-১ এর মধ্যে কল্যাম্প করে রিটার্ন করে।

factory PackageSubscription.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে PackageSubscription তৈরি করে।

ক্লাস: ClientProfile

ক্লায়েন্টের নজিস্ব অ্যাকাউন্ট রকের্ড (GET /clients/me) — যোগাযোগের তথ্য, কোম্পানি তথ্য, অ্যাকাউন্ট ম্যানেজার, লাইফটাইম ভ্যালু ইত্যাদি

String get contactName

firstName ও lastName-এর ফাঁকা-নয় অংশগুলো জোড়া দিয়ে ট্রিমি করে রিটার্ন করে।

String get headline

কোম্পানির নাম থাকলে তা রিটার্ন করে; নাহলে contactName, তাও ফাঁকা হলে ইমেইল, একবোরই কিছু না থাকলে 'Your account'।

String get initials

headline স্পেসে দিয়ে ভেঙে প্রথম শব্দ ও শেষ শব্দের প্রথম অক্ষর (আপারকসে) জোড়া দিয়ে রিটার্ন করে; একটি শব্দ থাকলে শুধু তার প্রথম অক্ষর, কিছু না থাকলে '?'।

factory ClientProfile.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড নাল-সফেভাবে পার্স করে ClientProfile তৈরি করে (স্ট্যাটাসের ডফিল্ট 'ACTIVE')।

ক্লাস: UpdateMyProfileRequest

PATCH /clients/me-এর জন্য রকি়োস্ট বডি — সত্যকারের স্পার্স-প্যাচ, কারণ ব্যাকএন্ড প্রতিটি ফিল্ড নাল-চকে করে এবং বাদ দেওয়া কী অপরিবর্তিত থাকে।

factory UpdateMyProfileRequest.from(ClientProfile profile)

একটি বিদ্যমান ClientProfile থেকে তার চারটি সম্পাদনাযোগ্য ফিল্ড কপি করে একটি UpdateMyProfileRequest তৈরি করে।

Map<String, dynamic> toJson()

ভেতরের clean(value) হলেপার দিয়ে প্রতিটি ফিল্ড ট্রিমি করে; ট্রিমি করার পর ফাঁকা না হলেই সটে JSON ম্যাপে অন্তর্ভুক্ত করে, যাতে ফাঁকা মান পাঠিয়ে সার্ভার-সাইড ফিল্ড ভুলবশত কল্যার না হয়ে যায়।

ফাইল: lib/features/portal/portal_profile_screen.dart

সংক্ষিপ্ত ববিরণ: পোর্টাল ক্লায়েন্টের "Account" স্ক্রিনি — নিজস্ব প্রোফাইল তথ্য, স্টেটিংস (থিম, পাসওয়ার্ড পরিবর্তন) এবং সাইন-আউট অপশন দেখায়। প্রোফাইলের বেশিরভাগ ফিল্ড কবেল দেখার জন্য; সম্পাদনা `edit_portal_profile_sheet.dart`-এর মাধ্যমে হয়।

ক্লাস/উইজটে: PortalProfileScreen

মূল অ্যাকাউন্ট স্ক্রিনি — প্রোফাইল কার্ড, তথ্য তালিকা, স্টেটিংস সেকশন ও সাইন-আউট বাটন সাজায়।

Widget build(BuildContext context, WidgetRef ref)

`clientProfileProvider` ও `currentUserProvider` পড়ে। অ্যাপবারে প্রোফাইল লোড হলে এডটি আইকন বাটন দেখায় (যা `showEditPortalProfileSheet` খোলে)। `RefreshIndicator`-এ প্রোফাইল ইনভ্যালিডেটে করে। বডতি অ্যাক্সেস/নাম/ইমেইল/স্ট্যাটাস-চাপি কার্ড, `async.when` দিয়ে লোডিং/এরর/ডটো অবস্থা (`_Facts`), স্টেটিংস সেকশন (`_Settings`) এবং `_SignOut` বাটন সাজায়।

ক্লাস/উইজটে: _Facts

প্রোফাইলের বসিতারতি তথ্য (Contact, Email, Phone, Industry, Website, Billing address, Account manager, Client since, Requests to date) সারি আকারে দেখায়।

Widget build(BuildContext context)

`profile`-এর ভিত্তিন ফিল্ড থেকে লবেল-ভ্যালু জোড়া তৈরি করে, ফাঁকা/নাল মানগুলো ফিল্টার করে বাদ দেয়। সব ফাঁকা হলে `SizedBox.shrink()` রটার্ন করে; নাহলে প্রতটি সারি বিভাজক-সহ একটি `AppCard`-এ দেখায় এবং শেষে "To change any of this, message your account team." নোট দেখায় (কারণ ক্লায়েন্ট রেকর্ড সম্পাদনা অ্যাকাউন্ট টিমের কাজ)।

ক্লাস/উইজটে: _Settings

থিম/অ্যাপয়ারেন্স এবং পাসওয়ার্ড পরিবর্তনের লিংক-টাইল দেখায়।

Widget build(BuildContext context, WidgetRef ref)

`themeControllerProvider` পড়ে দুটি টাইল তৈরি করে — "Appearance" (বর্তমান থিম মোড ও অ্যাকসেন্ট লবেল সহ, `Routes.appearance`-এ যায়) এবং "Change password" (`Routes.changePassword`-এ যায়)। প্রতটি টাইল একটি `AppCard`-এর মধ্যে বিভাজক-সহ `ListTile` হিসেবে রেন্ডার হয়।

static String _modeLabel(ThemeMode mode)

`ThemeMode`-কে "Light"/"Dark"/"System" স্ট্রিং লবেলে রূপান্তর করে (একটি `switch` এক্সপ্রেশন দিয়ে)।

ক্লাস/উইজটে: _SignOut

সাইন-আউট বাটন, নিশ্চিতকরণ ডায়ালগসহ।

Widget build(BuildContext context, WidgetRef ref)

একটি বিপিড-রঙের `OutlinedButton` রেন্ডার করে; ট্যাপ করলে একটি `AlertDialog` ("Sign out?") দেখায়, নিশ্চিতি হলে `authControllerProvider.notifier.logout()` কল করে।

ফাইল: lib/features/portal/portal_repository.dart

সংক্ষিপ্ত ববিরণ: ক্লায়েন্ট পোর্টালরে সব ডটো-অ্যাক্সেসে একজায়গায় ধরে রাখা — সব API কল `/me`-স্কোপড, অর্থাৎ ব্যাকএন্ড কলারের নজিরে Client রেকর্ড অনুযায়ী উত্তর দেয়, কোনো id পাঠাতে হয় না। এই ফাইলে `PortalRepository` ক্লাস এবং তার সাথে সংশ্লিষ্ট বিভিন্ন Riverpod প্রোভাইডার ও কন্ট্রোলার সংজ্ঞায়িত আছে।

ক্লাস/উইজটে: PortalRepository

এই ক্লাসটি ক্লায়েন্ট পোর্টালরে জন্য ড্যাশবোর্ড সামারি, প্রোফাইল, ইনভয়েস, রসিটি, প্যাকেজ সাবস্ক্রিপশন এবং সাপোর্ট টিকিট সম্পর্কিত সব HTTP অনুরোধ পরিচালনা করে। কনস্ট্রাক্টরে একটি `ApiClient` ইনজেক্ট করা হয় এবং কয়েকটি স্ট্যাটিক এন্ডপয়েন্ট কনস্ট্যান্ট (`_invoices`, `_receipts`, `_packages`, `_tickets`) ব্যবহার করে path তৈরি করা হয়।

`PortalRepository(this._api)`

কনস্ট্রাক্টর — একটি `ApiClient` ইনস্ট্যান্স নিয়ে `_api` ফিল্ডে সংরক্ষণ করে। এটাই একমাত্র নির্ভরতা যা এই রপিজিটরির সব মথোডে ব্যবহৃত হয়।

`Future<ClientSummary> summary()`

`/dashboard/client-summary` এন্ডপয়েন্ট থেকে GET রকোয়েস্ট করে ড্যাশবোর্ড সামারি ডটো আনে এবং JSON থেকে `ClientSummary` অবজেক্টে রূপান্তর করে রিটার্ন করে।

`Future<ClientProfile> profile()`

`/clients/me` থেকে GET করে বর্তমান লগইন করা ক্লায়েন্টের প্রোফাইল আনে এবং `ClientProfile.fromJson` দিয়ে পার্স করে।

`Future<ClientProfile> updateProfile(UpdateMyProfileRequest request)`

`/clients/me`-তে PATCH রকোয়েস্ট পাঠিয়ে ক্লায়েন্টের প্রোফাইল আপডেট করে। এটি স্টাফ-সাইড ক্লায়েন্ট আপডেট থেকে ইচ্ছাকৃতভাবে সংকীর্ণ — শুধু কোম্পানির তথ্য ও ঠিকানা পরিবর্তনযোগ্য; নাম, ইমেইল ও অ্যাকাউন্ট স্ট্যাটাস এখান থেকে বদলানো যায় না, কারণ সেগুলো সেই কোম্পানির অধিকারে থাকা উচিত যারা সম্পর্কটির মালিক — নইলে ক্লায়েন্ট নজিহে বন্ধ করা অ্যাকাউন্ট পুনরায় খুলে ফেলেতে পারত।

`Future<PagedResponse<Invoice>> invoices({int page = 0, int size = 20})`

`$_invoices/me` (অর্থাৎ `/company/finance/invoices/me`) থেকে পজেনিটেডে ইনভয়েসে তালিকা আনে, `Invoice.fromJson` পার্সার ব্যবহার করে।

`Future<PagedResponse<PaymentReceipt>> receipts({int page = 0, int size = 20})`

`$_receipts/me` থেকে পজেনিটেডে পেমেন্ট রসিটি তালিকা আনে।

`Future<String> invoicePdf(Invoice invoice)`

`$_invoices/${invoice.id}/pdf` থেকে বাইট আকারে PDF ডাউনলোড করে, ফাইলের নাম থেকে অনরিপদ ক্যারেক্টার সরিয়ে (regex দিয়ে) একটি নিরিপদ ফাইলনেমে তৈরি করে এবং প্ল্যাটফর্মের temp ডিরেক্টরিতে সংরক্ষণ করে ফাইলের পাথ রিটার্ন করে। স্থায়ী স্টোরেজের বদলে temp ব্যবহার করা হয়েছে কারণ ফাইলটি সরাসরি প্ল্যাটফর্ম ভাউয়ারে দেওয়া হয় এবং একটি ইনভয়েসে অ্যাপ স্টোরেজে স্থায়ীভাবে রাখার মতো জনিসি নয়।

`Future<PagedResponse<PackageSubscription>> subscriptions({int page = 0, int size = 20})`

`/packages/subscriptions/my` থেকে পজেনিটেডে প্যাকেজ সাবস্ক্রিপশন তালিকা আনে।

Future<PagedResponse<SupportTicket>> tickets({int page = 0, int size = 20})

\$_tickets/client/my (অর্থাৎ /v1/support/tickets/client/my) থেকে ক্লায়েন্টের নজিরে রাইজ করা সাপোর্ট টিকিটগুলোর পজেনিটেডে তালিকা আনবে। এটি স্টাফ-ফসেং /my-tickets এন্ডপয়েন্ট থেকে ভিন্ন — একই টবেলি কন্ট্রোল ভিন্ন অডিয়েন্স রজেলভ করে (/client/my কলারের Client row, /my-tickets স্টাফ ইউজার রজেলভ করে)।

Future<List<SupportMessage>> ticketMessages(int ticketId)

/v1/support/messages/client/ticket/\$ticketId থেকে একটি নির্দিষ্ট টিকিটের বার্তাগুলোর লিস্ট আনবে, Map<String, dynamic> টাইপের এন্ট্রি ফিল্টার করে SupportMessage অবজেক্টে ম্যাপ করে এবং createdAt অনুযায়ী ক্রমানুসারে সাজায়। এই এন্ডপয়েন্ট কনস্ট্রাকশনগতভাবেই শুধু বাহ্যিক (external) বার্তা রিটার্ন করে — স্টাফ-ইন্টারনাল নোট কখনো ফেরত আসবে না।

Future<SupportMessage> replyToTicket(int ticketId, String message, {String? attachmentUrl, String? attachmentFileName})

/v1/support/messages/client-এ POST করে একটি ক্লায়েন্ট রিপ্লাই পাঠায়, মেসেজটিকে trim করে এবং ঐচ্ছিক অ্যাটাচমেন্ট URL/ফাইলনামে যোগ করে। ব্যাকএন্ড এখানে isInternal জোর করে false সেট করে, ফলে একজন ক্লায়েন্ট ভুলবশতও এমন নোট লিখতে পারে না যা শুধু স্টাফ দখতে পাবে।

Future<SupportTicket> raiseTicket(CreateTicketRequest request)

\$_tickets/client-এ POST করে নতুন একটি সাপোর্ট টিকিট তৈরি করে এবং তৈরি হওয়া SupportTicket রিটার্ন করে।

টপ-লেভেলে ফাংশন/প্রোভাইডার

final portalRepositoryProvider = Provider<PortalRepository>(...)

apiClientProvider থেকে ApiClient নিয়ে একটি PortalRepository ইনস্ট্যান্স তৈরি করে সরবরাহ করে।

final clientSummaryProvider = FutureProvider<ClientSummary>(...)

currentUserProvider ওয়াচ করে (যাতে ইউজার বদলালে রিফ্রেশ হয়) এবং PortalRepository.summary() কল করে ড্যাশবোর্ড সামারি সরবরাহ করে।

final clientProfileProvider = FutureProvider<ClientProfile>(...)

একইভাবে currentUserProvider ওয়াচ করে PortalRepository.profile() কল করে ক্লায়েন্ট প্রোফাইল সরবরাহ করে।

final clientSubscriptionsProvider = FutureProvider<List<PackageSubscription>>(...)

subscriptions() কল করে প্যাকেজ সাবস্ক্রিপশনের content লিস্ট রিটার্ন করে; ApiException ধরা পড়লে খালি লিস্ট রিটার্ন করে, কারণ যে ক্লায়েন্ট একবারে একটি করে সাবস্ক্রিপশন কনে তার কোনো সাবস্ক্রিপশন না থাকাটা কোনো এরর নয়, প্যানলে দেখানোর মতো ঘটনা নয়।

ক্লাস: ClientInvoicesController

AsyncNotifier<PagedState<Invoice>> এক্সটেন্ড করে এবং PagedLoader<Invoice> মক্সইন ব্যবহার করে ইনভয়েসের পজেনিটেডে লোডিং স্টেটে পরিচালনা করে।

Future<PagedState<Invoice>> build()

currentUserProvider ওয়াচ করে এবং loadFirstPage() কল করে প্রথম পজে লোড করে।

`Future<PagedResponse<Invoice>> fetchPage(int page)`

`PagedLoader` মক্সইনরে প্রয়োজনীয় override — `portalRepositoryProvider.invoices(page: page)` কল করে নির্দিষ্ট পেজে আনবে।

`final clientInvoicesProvider = AsyncNotifierProvider<ClientInvoicesController, PagedState<Invoice>> (...)`

`ClientInvoicesController` সরবরাহকারী প্রোভাইডার।

ক্লাস: ClientReceiptsController

`AsyncNotifier<PagedState<PaymentReceipt>>` এক্সটেন্ড করে ও `PagedLoader<PaymentReceipt>` মক্সইন ব্যবহার করে রসিডিং পেজেনিটেডে স্টেটে পরিচালনা করে।

`Future<PagedState<PaymentReceipt>> build()`

`currentUserProvider` ওয়াচ করে ও `loadFirstPage()` কল করে।

`Future<PagedResponse<PaymentReceipt>> fetchPage(int page)`

`portalRepositoryProvider.receipts(page: page)` কল করে নির্দিষ্ট পেজে আনবে।

`final clientReceiptsProvider = AsyncNotifierProvider<ClientReceiptsController, PagedState<PaymentReceipt>> (...)`

`ClientReceiptsController` সরবরাহকারী প্রোভাইডার।

ক্লাস: ClientTicketsController

`AsyncNotifier<PagedState<SupportTicket>>` এক্সটেন্ড করে ও `PagedLoader<SupportTicket>` মক্সইন ব্যবহার করে টিকিটিং পেজেনিটেডে স্টেটে পরিচালনা করে, পাশাপাশি নতুন টিকিটি রইজ করার মধ্যে দিয়ে।

`Future<PagedState<SupportTicket>> build()`

`currentUserProvider` ওয়াচ করে ও `loadFirstPage()` কল করে।

`Future<PagedResponse<SupportTicket>> fetchPage(int page)`

`portalRepositoryProvider.tickets(page: page)` কল করে নির্দিষ্ট পেজে আনবে।

`Future<SupportTicket> raise(CreateTicketRequest request)`

`portalRepositoryProvider.raiseTicket(request)` কল করে নতুন টিকিটি তৈরি করে, এরপর `refresh()` কল করে তালিকা পুনরায় লোড করে, এবং তৈরি হওয়া টিকিটি রিটার্ন করে।

`final clientTicketsProvider = AsyncNotifierProvider<ClientTicketsController, PagedState<SupportTicket>> (...)`

`ClientTicketsController` সরবরাহকারী প্রোভাইডার।

`final clientTicketMessagesProvider =`

`FutureProvider.autoDispose.family<List<SupportMessage>, int> (...)`

টিকিটি id অনুযায়ী `ticketMessages(id)` কল করে সেই টিকিটিং বার্তার লিস্ট সরবরাহ করে; `autoDispose` ব্যবহার হওয়ায় স্ক্রিনি ছাড়লে স্টেটে মুক্ত হয়ে যায়।

ফাইল: lib/features/portal/portal_requests_screen.dart

সংক্ষিপ্ত ববিরণ: ক্লায়েন্ট পোর্টাল ব্য়বহারকারীর নজিরে সার্ভিস রকি়োস্টের তালিকা দেখানোর স্ক্রিনি। এটি স্টাফ মডউলরে সাথে কার্ড, ডিইল স্ক্রিনি ও রইজ শটি উইজেটে শয়োর করে, যহেতু ব্য়াকএন্ড ইতমিধ্যই ক্লায়েন্ট-শপেড ভডি রটার্নন করে।

ক্লাস/উইজেটে: PortalRequestsScreen

ConsumerWidget — /service-requests/my এন্ডপয়ন্টে থকে ব্য়াক করা **myRequestsProvider** ব্য়বহার করে ক্লায়েন্টের নজিরে সার্ভিস রকি়োস্টগুলো একটি পজেনিটেডে লস্টে দেখায়, নতুন রকি়োস্ট তরির জন্য় একটি ফ্লোটিং অ্যাকশন বাটন প্রদান করে।

Widget build(BuildContext context, WidgetRef ref)

থমি টোকনে ও **myRequestsProvider** কন্ট্রোলার নযি একটি **Scaffold** তরৈকিরে — অ্যাপবারে "My requests" শরিোনাম, একটি "Request" ফ্লোটিং অ্যাকশন বাটন যা **showRaiseRequestSheet** খোলে, এবং বডতি **PagedListview<ServiceRequest>** যা রফ্রিশে, লোড-মোর, খাল-স্টেটে মসেজে ও এরর মসেজে হ্যান্ডল করে। প্রতটি আইটেমকে **RequestCard** দযি রন্ডার করে, ট্যাপ করলে **openRequestDetail** কল করে ডিইল স্ক্রিনি নযি যায়।

ফাইল: lib/features/portal/portal_tickets_screen.dart

সংক্ষিপ্ত ববিরণ: ক্লায়েন্ট পোর্টালরে "Help" বিভাগ — ক্লায়েন্টের সাপোর্ট টকিটি তালিকা, টকিটি ডিইল/চ্যাট থ্রডে এবং নতুন টকিটি রইজ করার শটি এই একই ফাইলে সংজ্ঞায়তি।

ক্লাস/উইজেটে: PortalTicketsScreen

ConsumerWidget — **clientTicketsProvider** ব্য়বহার করে ক্লায়েন্টের নজিরে সাপোর্ট টকিটিগুলোর তালিকা **PagedListview<SupportTicket>** দযি দেখায়, প্রতটি টকিটি **TicketCard** দযি রন্ডার হয় এবং ট্যাপ করলে **PortalTicketDetailScreen**-এ নভেগিটে করে। একটি "Ask" ফ্লোটিং অ্যাকশন বাটন নতুন টকিটি রইজ করার শটি খোলে।

Widget build(BuildContext context, WidgetRef ref)

থমি টোকনে ও **clientTicketsProvider** কন্ট্রোলার নযি **Scaffold** তরৈকিরে — অ্যাপবার শরিোনাম "Help", FAB **_raise** কল করে, বডতি পজেনিটেডে টকিটি লস্ট।

static Future<void> _raise(BuildContext context, WidgetRef ref)

_AskSheet মোডাল বটম শটি খুলে **CreateTicketRequest** ফলাফল সংগ্রহ করে; ফলাফল null হলে বা কন্টেক্সট আনমাউন্টেডে হলে থমে যায়। অন্যথায় **clientTicketsProvider.notifier.raise(request)** কল করে টকিটি পাঠায় এবং সফল/ব্য়র্থ হওয়ার উপর ভিত্তি করে সন্ধ্যাকবার দেখায় (**ApiException**-এর মসেজে বা জনেরেকি এরর মসেজে)।

ক্লাস/উইজেটে: PortalTicketDetailScreen

ConsumerStatefulWidget — একটি নিরদিষ্ট **SupportTicket** নযি সেই টকিটিরে বসিতারতি তথ্য় (শরিোনাম, ববিরণ, স্ট্যাটাস, রজেোলউশন নোট) এবং বার্তার চ্যাট থ্রডে দেখায়, রপ্লাই পাঠানো ও অ্যাটাচমেন্ট আপলোডরে সুবধিসহ।

`ConsumerState<PortalTicketDetailScreen> createState()`

`_PortalTicketDetailScreenState` ইনস্ট্যান্স তৈরি করে রিটার্ন করে।

ক্লাস: `_PortalTicketDetailScreenState`

`ConsumerState<PortalTicketDetailScreen>` — কম্পোজার টেক্সট ফিল্ড, পাঠানো/অ্যাটাচিং স্টেটে এবং অ্যাটাচমেন্ট URL/নাম নিয়ন্ত্রণ করে।

`void dispose()`

`_composer` কন্ট্রোলার ডিসপোজ করে এবং সুপারক্লাসের `dispose()` কল করে।

`Future<void> _pickAttachment()`

`pickAttachment(context)` কল করে একটি ফাইল বেছে নেয়; বেছে না নলি বা আনমাউন্টেডে হলে থামে। বেছে নলি `_attaching true` সেটে করে, ফাইলের নাম দেখায়, `apiClientProvider.uploadDocument` দিয়ে আপলোড করে এবং সফল হলে `_attachmentUrl` সেটে করে। ব্যর্থ হলে অ্যাটাচমেন্ট নাম রসিটে করে এরর সন্যাকবার দেখায়, শেষে `_attaching false` করে।

`void _removeAttachment()`

`_attachmentName` ও `_attachmentUrl` উভয়কে null করে সেটে করে বাছাই করা অ্যাটাচমেন্ট বাতলি করে।

`Future<void> _send()`

কম্পোজার থেকে trim করা টেক্সট নেয়; খালি হলে বা ইতিমধ্যে পাঠানো/অ্যাটাচিং চলছে থাকলে থামে। `_sending true` সেটে করে `portalRepositoryProvider.replyToTicket` কল করে বার্তা ও অ্যাটাচমেন্ট পাঠায়, সফল হলে কম্পোজার খালি করে ও `clientTicketMessagesProvider(ticketId)` ইনভ্যালিডেটে করে বার্তা তালিকা রিফ্রেশ করে। ব্যর্থ হলে এরর সন্যাকবার দেখায়, শেষে `_sending false` করে।

`Widget build(BuildContext context)`

থিমি টোকেন, টিকিটি, বার্তার async স্টেটে ও বর্তমান ইউজার নিয়ে `Scaffold` তৈরি করে — অ্যাপবারে টিকিটি নম্বর, বডিতে `RefreshIndicator`-এ মোড়ানো `ListView`, যখনে থাকে: টিকিটির শিরোনাম/ববিরণ/স্ট্যাটাস চপি/তৈরি সময় দেখানো একটি `AppCard`, ঐচ্ছকি রজেলডিশন নোট ব্যানার, এবং বার্তার `ChatThread` (লোডিং/এরর/ডটো স্টেটে হ্যান্ডল করে) যা রপ্লাই কম্পোজার, অ্যাটাচমেন্ট বাটন এবং টিকিটি বন্ধ থাকলে ডজিবেল নোটসি দেখায়।

ক্লাস/উইজটে: `_AskSheet`

`StatefulWidget` — নতুন সাপোর্ট টিকিটি রইজ করার জন্য একটি বিটম শাট ফর্ম, যখনে বমিয়, জরুরতি (প্রায়েরটি) ও বসিতারতি ববিরণ ইনপুট নেওয়া হয়।

`State<_AskSheet> createState()`

`_AskSheetState` ইনস্ট্যান্স রিটার্ন করে।

ক্লাস: `_AskSheetState`

`State<_AskSheet>` — ফর্ম কী, শিরোনাম ও ববিরণের `TextEditingController`, এবং নরিবাচতি প্রায়েরটি (`_priority`, ডফিল্ট 'MEDIUM') নিয়ন্ত্রণ করে।

`void dispose()`

`_title` ও `_description` কন্ট্রোলার ডিসপোজ করে এবং সুপারক্লাসের `dispose()` কল করে।

Widget build(BuildContext context)

থমি টোকেনে নথি একটি স্ক্রলযোগ্য ফর্ম তৈরি করে — "Ask for help" শিরোনাম, "This goes to your account team." বার্তা (ক্লায়েন্ট Zuhoo-র সাথে নয়, তার নথিগত ফার্মের সাথে কথা বলছে তা স্পষ্ট করার জন্য), বিষয় ইনপুট, প্রায়েরটি ড্রপডাউন (ticketPriorities থেকে জেনারেট করা), বিবরণ ইনপুট, এবং "Send" বাটন যা ফর্ম ভ্যালিডেট করার পর CreateTicketRequest তৈরি করে Navigator.pop-এর মাধ্যমে ফলাফল রিটার্ন করে।

ফাইল: lib/features/profile/appearance_screen.dart

সংক্ষিপ্ত বিবরণ: থমি মোড (লাইট/ডার্ক/সিস্টেম) এবং অ্যাকসেন্ট কালার বছে নেওয়ার স্ক্রিনি। উভয় স্টেটিং ট্যাপ করার সাথে সাথে তাৎক্ষণিকভাবে প্রয়োগ হয়, Save বাটন ছাড়াই — কারণ প্রভিডি-ই হলো পুরো অ্যাপ, তাই ভ্যালিডেট করার কিছু নেই।

ক্লাস/উইজটে: AppearanceScreen

ConsumerWidget — themeControllerProvider থেকে বর্তমান থমি স্টেটিংস পড়ে এবং তিনটি বিভাগ দেখায়: থমি মোড নির্বাচন, অ্যাকসেন্ট কালার নির্বাচন, ও একটি প্রভিডি প্যানলে।

Widget build(BuildContext context, WidgetRef ref)

থমি টোকেনে, বর্তমান স্টেটিংস ও কন্ট্রোলার নথি Scaffold তৈরি করে। "Theme" সেকশনে তিনটি _ModeTile (Light, Dark, System) দেখায়, প্রতিটি ট্যাপে controller.setMode(...) কল করে। "Accent colour" সেকশনে AppAccent.values-এর জন্য _AccentSwatch উইজটে Wrap দেখায়, প্রতিটি ট্যাপে controller.setAccent(...) কল করে, সাথে একটি বিখ্যামূলক টেক্সট। "Preview" সেকশনে _Preview উইজটে দেখায়।

ক্লাস/উইজটে: _ModeTile

StatelessWidget — একটি থমি মোড অপশন (আইকন, শিরোনাম, সাবটাইটলে) প্রদর্শনকারী ListTile, নির্বাচন হলে হাইলাইট ও চকে আইকন দেখায়।

Widget build(BuildContext context)

থমি টোকেনে নথি একটি ListTile তৈরি করে যা onTap কলব্যাক চালায়, নির্বাচন অবস্থায় selectedTileColor, বোল্ড টাইটলে ও check_circle_rounded আইকন দেখায়, অন্যথায় ফাঁকা বৃত্ত আইকন দেখায়।

ক্লাস/উইজটে: _AccentSwatch

StatelessWidget — একটি বৃত্তাকার রঙের সোয়াচ যা একটি নির্দিষ্ট AppAccent প্রতিনিধিত্ব করে, ট্যাপযোগ্য এবং নির্বাচন হলে বর্ডার ও চকেমার্ক দেখায়।

Widget build(BuildContext context)

থমি টোকেনে ও ব্রাইটনে অনুযায়ী অ্যাকসেন্টের ফিলি কালার (accent.fillFor(bos.brightness)) নথি একটি Semantics-মোডানো InkWell তৈরি করে — বৃত্তাকার কনটেনারের রঙ, নির্বাচন হলে বর্ডার ও চকে আইকন, নথি অ্যাকসেন্টের লবেলে টেক্সট।

ক্লাস/উইজটে: _Preview

`StatelessWidget` — অ্যাকসেন্ট কালার যসেব কম্পোনেন্টে প্রভাব ফলে (আইকন, প্রোগ্রেস বার, বাটন) তার একটি ক্সুদ্র নমুনা দেখায়, যাতে সাতটি রঙনি বৃত্তের চয়ে বাস্তবসম্মত কছির ভিত্তিতে সন্ধানত নেওয়া যায়।

`Widget build(BuildContext context)`

থমি টোকেনে নিয়ে একটি `AppCard` তৈরি করে যাতে থাকে: একটি আইকনসহ স্যাম্পল রো (শিরোনাম, সাবটেক্সট, একটি `StatusChip('APPROVED')`), একটি `LinearProgressIndicator` (মান 0.62, ব্র্যান্ড কালার), এবং পাশাপাশি "Primary" (`ElevatedButton`) ও "Secondary" (`OutlinedButton`) বাটন।

ফাইল: lib/features/profile/change_email_screen.dart

সংক্ষিপ্ত বিবরণ: ব্যবহারকারীর ইমেইল পরিবর্তন ফর্ম স্ক্রিনি। বর্তমান পাসওয়ার্ড দিয়ে নিশ্চিত করার পর ইমেইল বদলায়, এবং সফল হলে সেশন সাইন-আউট করে দেয় কারণ পুরনো JWT টোকেনে পুরনো ইমেইলের জন্য ইস্যু করা।

ক্লাস/উইজটে: ChangeEmailScreen

`ConsumerStatefulWidget` — ইমেইল পরিবর্তন ফর্মের এন্ট্রি পয়েন্ট, স্টেট `_ChangeEmailScreenState`-এ রাখা।

`ConsumerState<ChangeEmailScreen> createState()`
`_ChangeEmailScreenState` ইনস্ট্যান্স তৈরি করে রিটার্ন করে।

ক্লাস: _ChangeEmailScreenState

`ConsumerState<ChangeEmailScreen>` — ফর্ম কী, নতুন ইমেইল ও পাসওয়ার্ডের কন্ট্রোলার, পাসওয়ার্ড দেখানো/না-দেখানো টগল, স্টেট ও এরর মैसेজ পরিচালনা করে।

`void dispose()`

`_newEmail` ও `_password` কন্ট্রোলার ডিসপোজ করে এবং সুপারক্লাসের `dispose()` কল করে।

`Future<void> _submit()`

ইতিমধ্যে সবে চলছে থাকলে বা ফর্ম ভ্যালিডি না হলে থামে। ফোকাস সরিয়ে `_saving` true ও `_error` null স্টেট করে, `authRepositoryProvider.changeEmail(ChangeEmailRequest(...))` কল করে নতুন ইমেইল ও বর্তমান পাসওয়ার্ড পাঠায়। সফল হলে সন্ধ্যাকবারে "Email changed. Please sign in again." দেখায় এবং `authControllerProvider.notifier.clearSession()` কল করে সেশন সাফ করে — কারণ JWT পুরনো ইমেইলের জন্য ইস্যু করা, তাই পরবর্তী রকিওয়েস্টে এমনতিও ব্যর্থ হতো, এখানে সাইন-আউট করে তা স্পষ্টভাবে দেখানো হয়। ব্যর্থ হলে `ApiException`-এর মैसेজ বা জনেরেকি এরর মैसेজ `_error`-এ স্টেট করে, শেষে `_saving` false করে।

`Widget build(BuildContext context)`

থমি টোকেন ও বর্তমান ইউজারের ইমেইল নিয়ে `Scaffold` তৈরি করে — এরর থাকলে

`MessageBanner.error`, সবসময় একটি ইনফো ব্যানার (সাইন-আউট হয়ে যাওয়ার সতর্কতা), বর্তমান ইমেইল দেখানোর ডিজিবেলড ফল্ড (থাকলে), নতুন ইমেইল ইনপুট (ভ্যালিডেশন: খালি নয়, বৈধ @ ফরম্যাট, বর্তমান ইমেইলের সমান নয়), বর্তমান পাসওয়ার্ড ইনপুট (দেখানো/লুকানো টগলসহ, ভ্যালিডেশন: খালি নয়), এবং `LoadingButton` যা `_submit` কল করে।

ফাইল: lib/features/profile/change_password_screen.dart

সংক্ষিপ্ত ববিরণ: ব্যবহারকারীর পাসওয়ার্ড পরিবর্তনের ফর্ম স্ক্রিনি। সফল হলে ব্যাকএন্ড সব রফ্রিশে টোকেনে বাতলি করে দেয়, তাই এখানেও সফল পরিবর্তনের পর স্থানীয় সেশন সাইন-আউট করে দেওয়া হয়।

ক্লাস/উইজটে: ChangePasswordScreen

ConsumerStatefulWidget — পাসওয়ার্ড পরিবর্তনের ফর্মে এন্ট্রি পয়েন্ট, স্টেটে `_ChangePasswordScreenState`-এ রাখা।

ConsumerState<ChangePasswordScreen> createState()
`_ChangePasswordScreenState` ইনস্ট্যান্স তৈরি করে রটার্ন করে।

ক্লাস: _ChangePasswordScreenState

ConsumerState<ChangePasswordScreen> — ফর্ম কী, বর্তমান/নতুন/নশ্চিতিকরণ পাসওয়ার্ডের তিনটি কন্ট্রোলার, দেখানো/না-দেখানো টগল, সত্বেই স্টেটে ও এরর মেসেজে পরিচালনা করে।

void dispose()

`_current`, `_next` ও `_confirm` কন্ট্রোলার ডিসপোজ করে এবং সুপারক্লাসের `dispose()` কল করে।

Future<void> _submit()

ইতিমধ্যে সত্বে চলছে থাকলে বা ফর্ম ভ্যালিডি না হলে থামে। ফোকাস সরিয়ে `_saving` true ও `_error` null স্টেট করে, `authRepositoryProvider.changePassword(ChangePasswordRequest(...))` কল করে বর্তমান, নতুন ও নশ্চিতিকরণ পাসওয়ার্ড পাঠায়। সফল হলে সন্ধ্যাকবারে "Password changed. Please sign in again." দেখায় এবং `authControllerProvider.notifier.clearSession()` কল করে সেশন সাফ করে — কারণ ব্যাকএন্ড সফল পরিবর্তনে সব রফ্রিশে টোকেনে বাতলি করে দেয়, ফলে ডিভাইসে থাকা টোকেনে এমনতিই মৃত; সাইন-আউট এটি এখনই স্পষ্ট করে, পরবর্তী রকি়োয়স্টে বভিরান্তকির ব্যর্থতা হসিবে নয়। ব্যর্থ হলে `ApiException`-এর মেসেজে বা জনেরেকি এরর মেসেজে `_error`-এ স্টেট করে, শেষে `_saving` false করে।

Widget build(BuildContext context)

থমি টোকেনে নযি `Scaffold` তৈরি করে — এরর থাকলে `MessageBanner.error`, সবসময় একটি ইনফো ব্যানার (সাইন-আউট হয়ে যাওয়ার সতর্কতা), বর্তমান পাসওয়ার্ড ইনপুট (ভ্যালিডেশন: খালি নয়), নতুন পাসওয়ার্ড ইনপুট (ভ্যালিডেশন: কমপক্ষে ৮ ক্যারেক্টার, বর্তমান পাসওয়ার্ডের সমান নয়), নশ্চিতিকরণ পাসওয়ার্ড ইনপুট (ভ্যালিডেশন: নতুন পাসওয়ার্ডের সাথে মলিত হবে), এবং `LoadingButton` যা `_submit` কল করে। তিনটি ফিল্ড একটি একক `_show` টগলের মাধ্যমে একসাথে দেখানো/লুকানো নযিন্ত্রতি হয়।

ফাইল: lib/features/profile/edit_profile_screen.dart

সংক্ষিপ্ত ববিরণ: কর্মচারী নজি যেসেব ফলিড পরিবর্তন করতে পারে (ফোন, ইমারজনেস কন্টাক্ট, জন্ডার) তার জন্য এডটি ফর্ম স্ক্রিনি। নাম, বিভাগ, পদবি, শফিট বা বতেনরে মতো ফলিডগুলো এইচআর-এর নযিন্ত্রণে বলে এখানে দেখানো হয় না। শুধু পরিবর্ততি ফলিডগুলো PATCH রকি়োয়স্টে পাঠানো হয়।

ক্লাস/উইজটে: EditProfileScreen

ConsumerWidget — myEmployeeProvider থেকে কর্মচারীর রেকর্ড লোড করে; লোডিং/এরর/খালি-রেকর্ড অবস্থা সামলে _Form-কে রেকর্ডসহ রেন্ডার করে।

Widget build(BuildContext context, WidgetRef ref)

থিমি টোকেনে ও myEmployeeProvider-এর AsyncValue পড়ে Scaffold তৈরি করে। employee.when দিয়ে তিনটি অবস্থা হ্যান্ডল করে: লোডিং-এ Loader, এরর-এ ErrorState (রেট্রাই বাটনসহ myEmployeeProvider ইনভ্যালিডেট করে), আর ডটো null হলে "No employee record" দেখানো EmptyState, নয়তো _Form(employee: record)।

ক্লাস/উইজটে: _Form

ConsumerStatefulWidget, যা প্রদত্ত Employee-কে ফর্মের প্রাথমিক মান হিসেবে নেয়।

ConsumerState<_Form> createState()

_FormState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _FormState

পাঁচটি TextEditingController (ফোন, ওয়ার্ক ফোন, ইমার্জেন্সি কন্টাক্ট নাম/ফোন/সম্পর্ক), জেন্ডার নির্বাচন এবং সভে-স্টেটে রাখা; শুধু বদলানো ফিল্ডগুলো নিয়ে প্যাচ তৈরি করে সার্ভারে পাঠায়।

void initState()

widget.employee-এর বর্তমান মান দিয়ে সব কন্ট্রোলার ইনশিয়ালাইজ করে (null হলে খালি স্ট্রিং)। _gender-কে বৈধ genderOptions-এর মধ্যে থাকলেই সেট করে, নয়তো null রাখে।

void dispose()

পাঁচটি TextEditingController ডিসপোজ করে, তারপর সুপারক্লাসের dispose কল করে।

SelfUpdateEmployeeRequest _buildPatch()

মূল রেকর্ডের সাথে বর্তমান কন্ট্রোলার/জেন্ডার মান তুলনা করে শুধু পরিবর্তিত ফিল্ডগুলো নিয়ে SelfUpdateEmployeeRequest তৈরি করে। ভেতরের changed হলেপার ফাংশন ট্রিম করা টেক্সট আসল মানের সাথে মিলিয়ে দেখে, না মিলিলে নতুন মান রিটার্ন করে নয়তো null। উদ্দেশ্য: স্ক্রিনি খোলা থাকা অবস্থায় এইচআর অন্য কিছু বদলালে সেটা ওভাররাইট না হওয়া।

Future<void> _save()

ফর্ম ভ্যালিডেট করে, প্যাচ খালি হলে সরাসরি পিপ করে বেরিয়ে যায়। নয়তো ফোকাস সরিয়ে _saving true করে employeeRepositoryProvider.updateMe(patch) কল করে। সফল হলে authControllerProvider.notifier.setAvatar দিয়ে ক্যাশড অ্যাভাটার রিফ্রেশ করে (কারণ এই এন্ডপয়েন্ট সার্ভার-সাইডে ছবির সঙ্কেত করে যা AuthController-এর মধ্যে দিয়ে যায় না), myEmployeeProvider ইনভ্যালিডেট করে, সন্ধ্যাকবার দেখিয়ে স্ক্রিনি পপ করে। ApiException বা অন্য এরর হলে _error সেট করে; finally-তে _saving false করে।

Widget build(BuildContext context)

স্ক্রলযোগ্য Form তৈরি করে: এরর ব্যানার (থাকলে), "Contact" সেকশন (ব্যক্তিগত ফোন, ওয়ার্ক ফোন, জেন্ডার ড্রপডাউন), "Emergency contact" সেকশন (নাম, ফোন, সম্পর্ক) এবং "Save changes" LoadingButton, যা _saving অনুসারে লোডিং দেখায় ও _save কল করে।

static String? _optionalPhone(String? value)

ফোন নম্বর ভ্যালিডেটর। খালি হলে বৈধ (ফল্ডিগুলো ঐচ্ছিক)। ৬ অক্ষরের কম হলে "too short" এরর, আর `^[0-9+\\-\\s()]+` রজেক্সে না মিলিলে অনুমোদিত অক্ষরের বার্তা দিয়ে।

ফাইল: lib/features/profile/employee_models.dart

সংক্ষিপ্ত ববিরণ: GET /api/employees/me-এর জন্য Employee মডেলে এবং PATCH /api/employees/me-এর জন্য SelfUpdateEmployeeRequest। ব্যাকএন্ডের পূর্ণ EmployeeResponse অনেকে বড় হলেও এখানে শুধু মোবাইল অ্যাপে দেখানো ফল্ডিগুলো মডেলে করা হয়েছে।

ক্লাস/উইজটে: Employee

সাইন-ইন করা ব্যবহারকারীর কর্মচারী রেকর্ডকে প্রতিনিধিত্ব করে — ব্যক্তিগত তথ্য, কর্মসংস্থান তথ্য, ইমার্জেন্সি কন্টাক্ট ইত্যাদি ধারণ করে (অপরিবর্তনীয়/const কনস্ট্রাক্টর)।

String get fullName

firstName ও lastName জুড়ে ট্রিম করে পূর্ণ নাম ফেরত দেয়।

String get roleLabel

customRoleName ?? designationName ?? jobTitle ?? 'Employee' — সব null হলে "Employee" ফলব্যাক দেখায়, কারণ এখানে রেকর্ড থাকা মানই একজন কর্মচারী।

String? get imageUrl

Env.resolveImageUrl(image) কল করে সম্পূর্ণ ইমেজ URL রিটার্ন করে।

String get initials

নাম-পদবী থেকে ইনিশিয়াল বানায়। দুটোই খালি হলে ইমেইলের প্রথম অক্ষর (বা ইমেইলও খালি হলে '?'), একটি খালি হলে অন্যটির প্রথম অক্ষর, নয়তো উভয়ের প্রথম অক্ষর জুড়ে আপারকসে রিটার্ন করে।

factory Employee.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে Employee তৈরি করে; সংখ্যাসূচক ফল্ডি num?.toInt() ব্যবহার হয়, image ফল্ডি image অথবা ফলব্যাক হিসেবে profileImageUrl কী থেকে নেয়, অনুপস্থিতি ফল্ডি যেখাযখ ডিফল্ট (খালি স্ট্রিং, 0, true) বসে।

ক্লাস/উইজটে: SelfUpdateEmployeeRequest

PATCH /api/employees/me-এর বডি — শুধু কর্মচারী নিজের বদলাতে পারা ফল্ডিগুলো (ফোন, ওয়ার্ক ফোন, জন্ম তারিখ, ইমার্জেন্সি কন্টাক্ট, প্রোফাইল ইমেজ URL) ধারণ করে।

Map<String, dynamic> toJson()

শুধু non-null ফল্ডিগুলো ম্যাপে যোগ করে (if (field != null) কন্ডিশনাল এন্ট্রি ব্যবহার করে) — অনুপস্থিতি কী পাঠায়, explicit null নয়, কারণ ব্যাকএন্ডে কোনো ফল্ডি ক্লিয়ার করার উপায় নেই।

উপ-লভেলে কনস্ট্যান্ট

const genderOptions = <String>['MALE', 'FEMALE', 'OTHER'] — বৈধ জন্ম তারিখ মানের তালিকা, ফর্মের ড্রপডাউন ও ভ্যালিডেশনে ব্যবহৃত হয়।

ফাইল: lib/features/profile/employee_repository.dart

সংক্ষিপ্ত ববিরণ: /employees/me এন্ডপয়েন্টেরে জন্য রপিজিটরি (GET ও PATCH) এবং সাইন-ইন করা ব্যবহারকারীর কর্মচারী রেকর্ড শয়ের করার জন্য myEmployeeProvider। কর্মচারী রেকর্ড না থাকা (যেমন কোম্পানির মালিক) একটি স্বাভাবিক null অবস্থা হিসেবে হ্যান্ডল করা হয়, এরর নয়।

ক্লাস/উইজটে: EmployeeRepository

ApiClient-এর মাধ্যমে কর্মচারী নজি-রেকর্ড GET ও PATCH করার দুটি মিথোড সরবরাহ করে।

Future<Employee> me()

GET /employees/me কল করে JSON থেকে Employee পার্স করে রিটার্ন করে।

Future<Employee> updateMe(SelfUpdateEmployeeRequest request)

request.toJson()-কে বডি হিসেবে ব্যবহার করে PATCH /employees/me কল করে, ফলাফল থেকে হালনাগাদকৃত Employee পার্স করে রিটার্ন করে।

টপ-লেভেলে প্রোভাইডার/ফাংশন

employeeRepositoryProvider — ApiClient দিয়ে EmployeeRepository-এর একটি Provider তৈরি করে।

myEmployeeProvider (FutureProvider<Employee?>) — currentUserProvider ওয়াচ করে (ব্যবহারকারী বদলালে রিবিল্ড হওয়ার জন্য), EmployeeRepository.me() কল করে; ApiException এ isNotFound true হলে null রিটার্ন করে (রেকর্ড-বাহীন ব্যবহারকারীর জন্য স্বাভাবিক কেস), অন্য এরর হলে rethrow করে।

ফাইল: lib/features/profile/profile_screen.dart

সংক্ষিপ্ত ববিরণ: "Me" ট্যাবেরে মূল স্ক্রিনি — প্রোফাইল ছবি/নাম/স্ট্যাটাস, কর্মসংস্থান বিস্তারিত, স্টেটিংস তালিকা এবং সাইন-আউট বাটন প্রদর্শন করে। রিফ্রেশ করলে কর্মচারী রেকর্ড ও ব্যবহারকারী প্রোফাইল দুটোই পুনরায় লোড হয়।

ক্লাস/উইজটে: ProfileScreen

ConsumerWidget — বর্তমান ব্যবহারকারী ও কর্মচারী রেকর্ড ওয়াচ করে পুরো প্রোফাইল স্ক্রিনি কম্পোজ করে।

Widget build(BuildContext context, WidgetRef ref)

RefreshIndicator-সহ ListView তৈরি করে: প্রোফাইল কার্ড (ছবি, নাম, ইমেইল, স্ট্যাটাস চপি — কর্মচারী মান না থাকলে ব্যবহারকারীর মান ফলব্যাক), employee.when দিয়ে লোডিং/এরর/_NoEmployeeRecord/_Details, "Settings" সেকশন হডোর, _SettingsList ও _SignOutButton। রিফ্রেশে myEmployeeProvider ইনভ্যালিডিটে এবং authControllerProvider.notifier.refreshProfile() কল হয়।

ক্লাস/উইজটে: _NoEmployeeRecord

StatelessWidget — যখন সাইন-ইন করা অ্যাকাউন্টেরে employees টেবিলে কোনো সারানিই (সাধারণত কোম্পানির মালিক), তখন একটি তথ্যমূলক ব্যানার দেখায়।

Widget build(BuildContext context)

"This account is not linked to an employee record..." বার্তাসহ `AppCard`-এ মোড়ানো `MessageBanner.info` রটাইরন করে।

ক্লাস/উইজটে: `_Details`

`StatelessWidget` — একটি `Employee` রেকর্ড নিয়ে কর্মসংস্থান-সংক্রান্ত ফলিডগুলো লবেলে-ভ্যালু সারাইসিবে প্রদর্শন করে।

Widget build(BuildContext context)

রেকর্ড নাম্বার, বিভাগ, পদবি, রিপোর্টিং ম্যানেজার, শফিট, যোগদানের তারিখ, ওয়ার্ক ফোন, ব্যক্তিগত ফোন ও ইমারজেন্সি কন্টাক্ট — এই আইটেমগুলোর একটি রেকর্ড-লিস্ট বানিয়ে খালি মানগুলো ফলিটার করে ফেলে দেয়। সব খালি হলে `SizedBox.shrink()` রটাইরন করে। নয়তো "Employment" `SectionHeader` (এডিট বাটনসহ, যা `Routes.editProfile`-এ নভেগিটে করে) এবং প্রতটিসারি জন্য আইকন-লবেলে-ভ্যালু দেখানো `AppCard` রটাইরন করে (মাঝে ডিভাইডার সহ)।

ক্লাস/উইজটে: `_SettingsList`

`ConsumerWidget` — অ্যাপের বিভিন্ন সেকশনের (পারফরম্যান্স, হায়ারিং, আইটি অ্যাসেস্টে, কর্মচারী তালিকা, অ্যাপথারনেস, পেস্‌সলপি, সারভিস রকিওয়েস্ট, সআরএম, ফাইন্যান্স, সাপোর্ট, নোটিফিকেশন, পাসওয়ার্ড পরিবর্তন) নভেগেশন তালিকা দেখায়; প্ল্যাটফর্ম কনসোল শুধু প্ল্যাটফর্ম স্টাফদের জন্য দেখায়।

Widget build(BuildContext context, WidgetRef ref)

`isPlatformStaff` চেক করে শর্তসাপেক্ষে "Platform console" টাইল যোগ করে টাইলের তালিকা বানায় (প্রতটি টাইলে লবেলে, ঐচ্ছকি ট্রেলিং টেক্সট, আইকন, রুট পাথ)। "Appearance" টাইলের ট্রেলিং টেক্সট বর্তমান থিম মোড ও অ্যাক্সেন্ট লবেলে দেখায়। `AppCard`-এর মধ্যে `ListTile`-এর কলাম রেন্ডার করে, প্রতটিতে ট্যাপে `context.push(tiles[i].path)` কল হয়।

static String `_modeLabel(ThemeMode mode)`

`ThemeMode`-কে 'Light', 'Dark' বা 'System' — এই মানুষ-পাঠযোগ্য লবেলে রূপান্তর করে।

ক্লাস/উইজটে: `_SignOutButton`

`ConsumerWidget` — নশ্চিতিকরণ ডায়ালগসহ সাইন-আউট বাটন দেখায়।

Widget build(BuildContext context, WidgetRef ref)

`OutlinedButton.icon` রটাইরন করে; ট্যাপে একটি "Sign out?" `AlertDialog` দেখায় ("Stay" বা "Sign out" অপশনসহ)। নশ্চিতি হলে `authControllerProvider.notifier.logout()` কল করে — নিজেকে কোনো নভেগেশন করে না, কারণ সেশন ক্লিয়ার হলে রাউটারের রডাইরেক্ট লজিকই গন্তব্য ঠিকি করে।

ক্লাস/উইজটে: `_ChangeablePhoto`

`ConsumerStatefulWidget` — প্রোফাইল ছবি দেখায় এবং `canChange` true হলে ট্যাপে ছবি বিদলানোর সুবিধা দেয়।

`ConsumerState<_ChangeablePhoto> createState()`

`_ChangeablePhotoState` ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: `_ChangeablePhotoState`

ছবি আপলোড ও প্রোফাইলে সেভে করার দুই-ধাপের প্রক্রিয়া পরিচালনা করে।

Future<void> _change()

pickAttachment দিয়ে একটি ফাইল বাছাই করা; বাতলি হলে ফেরত যায়। নয়তো _busy true করে
apiClientProvider.uploadAvatar দিয়ে ফাইল আপলোড করে URL পাঠায়, তারপর
employeeRepositoryProvider.updateMe দিয়ে সেই URL কর্মচারী রেকর্ডে সেভ করে। সফল হলে
myEmployeeProvider ইনভ্যালিডেট করে ও authControllerProvider.notifier.refreshProfile() কল করে
(কারণ ছবি User অ্যাকাউন্টেও লেখা হয়) এবং সন্ধ্যাকবার দেখায়। ApiException-এ সারভারের বার্তা
দেখায়, অন্য এরর হলে জেনেরিক বার্তা দেখায়; finally-তে _busy false করে।

Widget build(BuildContext context)

Avatar উইজেটে তৈরি করে। canChange false হলে শুধু অ্যাভাটার রিটার্ন করে। নয়তো
GestureDetector-এ মোড়ানো Stack রিটার্ন করে — _busy true হলে অ্যাভাটারের উপর একটি ছোট
CircularProgressIndicator, নয়তো একটি ক্র্যামরো-আইকন ব্যাজ দেখায়।

ফাইল: lib/features/receivables/receivables_models.dart

সংক্ষিপ্ত ববিরণ: ইনভয়েস রইজ করার পরে প্রক্রিয়া — পমেন্ট রসিটি, ক্রেডিট নোট, রফান্ড
এবং ইনভয়েসের নজিস্ব সন্ড/ক্যান্সলে অ্যাকশনের সাথে সম্পর্কিত পারমিশন কোড, স্ট্যাটাস
এনাম ও মডলে ক্লাস ধারণ করে।

টপ-লেভেলে ক্লাস: ReceivablesPermissions

abstract final class — রসিভিবেলস-সংক্রান্ত সব পারমিশন কোড কনস্ট্যান্ট হিসেবে রাখা:
invoiceView, invoiceSend, invoicePayment, invoiceCancel, invoiceRefund (প্রসেস ও রজিক্ট দুটোই
একই কোড), creditNote, receiptView, receiptCreate, receiptConfirm (কনফার্ম, ডিপোজিট, রভার্স
তনিটরিই একই কোড), receiptDelete।

টপ-লেভেলে ক্লাস: PaymentStatus

abstract final class — PaymentStatus-এর সম্ভাব্য মান (pending, confirmed, deposited, failed,
reversed) এবং তাদের একটি সিম্বলিক্যালিক্যালি all ধারণ করে।

টপ-লেভেলে ক্লাস: RefundStatus

abstract final class — রফান্ড স্ট্যাটাসের মান (requested, approved, rejected, processed), all তালিকা
এবং settled সেটে (যেখানে আর কিছু ঘটবে না — rejected ও processed) ধারণ করে।

ক্লাস/উইজেটে: PaymentReceipt

কোনো ইনভয়েসের বিপরীতে গৃহীত অর্থ প্রতিনিধিত্ব করে — রসিটি নম্বর, পরিমাণ, স্ট্যাটাস,
সংশ্লিষ্ট ইনভয়েস/ক্লায়েন্ট তথ্য, পমেন্ট মেথড, রভার্সাল তথ্য ইত্যাদি ধারণ করে।

bool get canConfirm

স্ট্যাটাস pending হলে true — একটি রসিটি pending অবস্থা থেকে শুরু হয়ে কনফার্ম হয়।

bool get canDeposit

স্ট্যাটাস confirmed হলে true — কনফার্ম হওয়ার পরই ব্যাংকে জমা দেওয়া যায়।

`bool get canReverse`

স্ট্যাটাস `confirmed`, `deposited` বা `pending` হলে `true` — রিভার্স করলে এটি বাতলি হয়ে যায়; ইতিমধ্যে রিভার্সড বা ফাইলড রসিটি পুনরায় রিভার্স করা যায় না।

`factory PaymentReceipt.fromJson(Map<String, dynamic> json)`

JSON থেকে `PaymentReceipt` পার্স করে; সংখ্যাসূচক/`String` ফিল্ডে যথাযথ ফলব্যাক ডিফল্ট (0, খালি স্ট্রিং, `PaymentStatus.pending`) ব্যবহার করে।

ক্লাস/উইজটে: Refund

ক্লায়েন্টকে ফেরত যাওয়া অর্থ প্রতিনিধিত্ব করে (সাধারণত বাতলি হওয়া সার্ভিস রকিওয়েস্টের বিপরীতে রইজড, এখানে শুধু প্রসেস/রজিক্ট করা হয়)।

`bool get isSettled`

স্ট্যাটাস `RefundStatus.settled` স্টেটে থাকলে `true`।

`bool get canDecide`

`!isSettled` — স্টেট না হওয়া যেকোনো রফান্ডে প্রসেস বা রজিক্ট দুটো অ্যাকশনই খোলা থাকে।

`factory Refund.fromJson(Map<String, dynamic> json)`

JSON থেকে `Refund` পার্স করে; ডিফল্ট স্ট্যাটাস `RefundStatus.requested`।

ক্লাস/উইজটে: CreditNote

কোনো ইনভয়েসের পাওনা কমানোর ডকুমেন্ট (অর্থ স্থানান্তর ছাড়াই) প্রতিনিধিত্ব করে।

`factory CreditNote.fromJson(Map<String, dynamic> json)`

JSON থেকে `CreditNote` পার্স করে সংশ্লিষ্ট ফিল্ড ম্যাপ করে।

ক্লাস/উইজটে: CreditNoteRequest

`POST /company/finance/invoices/credit-notes`-এর বডি — ইনভয়েস আইডি, পরিমাণ (ন্যূনতম ০.০১) ও ঐচ্ছিক কারণ ধারণ করে। কোনো আপডেট/ডিলিট এন্ডপয়েন্ট নেই — একটি ক্রেডিট নোট রেকর্ডের ডকুমেন্ট, সংশোধন মানে নতুন একটি ইস্যু করা।

`Map<String, dynamic> toJson()`

`clientId` ও `amount` সবসময় অন্তর্ভুক্ত করে; `reason` ট্রিমি করে খালি না হলেই যোগ করে।

ফাইল: `lib/features/receivables/receivables_repository.dart`

সংক্ষিপ্ত ববিরণ: ইনভয়েস সেন্ড/ক্যান্সলে, ক্রেডিট নোট, রফান্ড এবং পমেন্ট রসিটি নিয়ে কাজ করা রপিজিটরি ও সংশ্লিষ্ট Riverpod প্রোভাইডার/কন্ট্রোলার। ইনভয়েস তরৈলিস্টিং/এডিট ফাইন্যান্স মডুলে থাকলেও, এখানে শুধু ইনভয়েস রইজ হওয়ার পরে অ্যাকশনগুলো আছে।

ক্লাস/উইজটে: ReceivablesRepository

`ApiClient`-কে মুড়ে ইনভয়েস-পরিবর্তী সব কার্যক্রমে (সেন্ড, ক্যান্সলে, ক্রেডিট নোট, রফান্ড, পমেন্ট রসিটি) API কল সরবরাহ করে।

Future<void> sendInvoice(int id)

POST `$_invoices/$id/send` কল করে ক্লায়েন্টকে ইনভয়েস ইমেইল করে; খালি রিসেপন্স।

Future<void> cancelInvoice(int id)

POST `$_invoices/$id/cancel` কল করে; খালি রিসেপন্স।

Future<PagedResponse<CreditNote>> creditNotes({int? invoiceId, int page = 0, int size = 30})

`$_invoices/credit-notes` থেকে পজেনিটেডে CreditNote তালিকা আনে, ঐচ্ছকি invoiceId কুয়েরি প্যারামিটার দিয়ে ফিল্টার করা যায়।

Future<CreditNote> issueCreditNote(CreditNoteRequest request)

request.toJson() বড দিয়ে POST `$_invoices/credit-notes` কল করে, ফলাফল থেকে নতুন CreditNote পার্স করে রিটার্ন করে।

Future<PagedResponse<Refund>> refunds({String? status, int page = 0, int size = 30})

`$_invoices/refunds` থেকে পজেনিটেডে Refund তালিকা আনে, ঐচ্ছকি status কুয়েরি প্যারামিটার দিয়ে ফিল্টার করা যায়।

Future<void> processRefund(int id)

POST `$_invoices/refunds/$id/process` কল করে টাকা ফেরত পাঠায়; খালি রিসেপন্স।

Future<void> rejectRefund(int id, String? reason)

কারণ ট্রমি করে খালি না হলে ?reason=... কুয়েরি প্যারামিটার হিসেবে URL-এনকোড করে যুক্ত করে, তারপর POST `$_invoices/refunds/$id/reject$query` কল করে রিফান্ড প্রত্যাখ্যান করে।

Future<PagedResponse<PaymentReceipt>> receipts({int page = 0, int size = 30})

`$_receipts` থেকে পজেনিটেডে PaymentReceipt তালিকা আনে।

Future<void> confirmReceipt(int id)

POST `$_receipts/$id/confirm` কল করে টাকা ক্লয়ার হয়েছে বলে চহিনতি করে; খালি রিসেপন্স, তাই কলার পুনরায় ডটো পড়ে।

Future<void> depositReceipt(int id, String bank)

কোন ব্যাংকে টাকা জমা হয়েছে তা রকের্ড করে — bank একটি আবশ্যকি (required) কুয়েরি প্যারামিটার হিসেবে ট্রমি ও URL-এনকোড করে POST `$_receipts/$id/deposit?bank=...` কল করে (খালি হলে ব্যাকএন্ড 400 রিটার্ন করবে)।

Future<void> reverseReceipt(int id, String? reason)

পমেন্ট বাতলি করে (যেমন বাউন্সড চকে বা ভুল ইনভয়েসে প্রয়োগ করা পমেন্ট); কারণ ট্রমি করে খালি না হলে কুয়েরি প্যারামিটার হিসেবে যুক্ত করে POST `$_receipts/$id/reverse$query` কল করে।

Future<void> deleteReceipt(int id)

DELETE `$_receipts/$id` কল করে রসিটি মুছে ফেলে।

টপ-লভেলে প্রোভাইডার/ফাংশন

receivablesRepositoryProvider — ApiClient দিয়ে ReceivablesRepository-এর একটি Provider তৈরি করে।

ক্লাস/উইজটে: ReceiptsController

`AsyncNotifier<List<PaymentReceipt>>` — পমেন্ট রসিটিরে তালিকা লোড, রফ্রেশ ও লোকাল একটা এন্ট্রিসিরানোর কাজ পরিচালনা করে।

`Future<List<PaymentReceipt>> build()`

`currentUserProvider` ওয়াচ করে (ব্যবহারকারী বদলালে রিবিল্ড হওয়ার জন্য) এবং `_load()` কল করে রিটার্ন করে।

`Future<List<PaymentReceipt>> _load()`

`receivablesRepositoryProvider.receipts()` কল করে পাওয়া পজেবের `content` রিটার্ন করে।

`Future<void> refresh()`

`AsyncValue.guard(_load)` দিয়ে স্টেটে পুনরায় লোড করে, এরর হলে সেটোও স্টেটে ধরে রাখে।

`void remove(int id)`

বর্তমান স্টেটে থেকে (থাকলে) নির্দিষ্ট `id`-এর রসিটি বাদ দিয়ে নতুন `AsyncValue.data` সেট করে — সার্বভার কল ছাড়াই লোকাল লিস্ট আপডেট করে।

টপ-লেভেলে প্রোভাইডার/ফাংশন

`receiptsProvider` — `ReceiptsController`-এর `AsyncNotifierProvider`।

ক্লাস/উইজটে: RefundFilterController

`Notifier<String?>` — রফান্ড তালিকার স্ট্যাটাস ফিল্টার (`null` মানে সব) ধারণ করে।

`String? build()`

প্রাথমিক অবস্থা `null` রিটার্ন করে (কোনো ফিল্টার নেই)।

`void set(String? status)`

নতুন স্ট্যাটাস আগের থেকে ভিন্ন হলেই স্টেটে আপডেট করে (একই মান হলে কিছু করে না)।

টপ-লেভেলে প্রোভাইডার/ফাংশন

`refundFilterProvider` — `RefundFilterController`-এর `NotifierProvider`।

ক্লাস/উইজটে: RefundsController

`AsyncNotifier<List<Refund>>` — বর্তমান ব্যবহারকারী ও ফিল্টার অনুসারে রফান্ড তালিকা লোড ও রফ্রেশ করে।

`Future<List<Refund>> build()`

`currentUserProvider` ও `refundFilterProvider` উভয়ই ওয়াচ করে (যেকোনো একটা বদলালে রিবিল্ড হয়) এবং `_load()` কল করে রিটার্ন করে।

`Future<List<Refund>> _load()`

`refundFilterProvider`-এর বর্তমান মান স্ট্যাটাস হিসেবে দিয়ে `receivablesRepositoryProvider.refunds(status: ...)` কল করে পজেবের `content` রিটার্ন করে।

`Future<void> refresh()`

`AsyncValue.guard(_load)` দিয়ে স্টেটে পুনরায় লোড করে।

টপ-লভেলে প্রোভাইডার/ফাংশন

refundsProvider — RefundsController-এর AsyncNotifierProvider।

ক্লাস/উইজটে: CreditNotesController

AsyncNotifier<List<CreditNote>> — ক্রেডিট নোটের তালিকা লোড ও রিফ্রেশ করে।

Future<List<CreditNote>> build()

currentUserProvider ওয়াচ করে এবং _load() কল করে রিটার্ন করে।

Future<List<CreditNote>> _load()

receivablesRepositoryProvider.creditNotes() কল করে পজেবল content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে স্টেটে পুনরায় লোড করে।

টপ-লভেলে প্রোভাইডার/ফাংশন

creditNotesProvider — CreditNotesController-এর AsyncNotifierProvider।

ফাইল: lib/features/receivables/receivables_screen.dart

সংক্ষিপ্ত বিবরণ: ইনভয়েসের পরবর্তী ধাপ — রসিপিট (টাকা আসা), রফিন্ড (টাকা ফেরত) এবং ক্রেডিট নোট (টাকা না নড়ে দায় কমানো) — এই তিনটির জন্য ট্যাবযুক্ত স্ক্রিনি। পারমিশন অনুযায়ী ট্যাব দেখানো/লুকানো হয় এবং প্রতিটি ট্যাবের নিজস্ব অ্যাকশন (কনফার্ম, ডিপ্লোজিটি, রভার্স, ডলিটি, প্রসেস, রজিক্ট) হ্যান্ডলে করে।

ক্লাস/উইজটে: ReceivablesScreen

ConsumerWidget। ব্যবহারকারীর পারমিশন অনুযায়ী কোন কোন ট্যাব (Receipts, Refunds, Credit notes) দেখানো হবে তা নির্ধারণ করে, DefaultTabController দিয়ে ট্যাব-ভিউ তৈরি করে, এবং সক্রিয় ট্যাবের create কলব্যাক থাকলে ফ্লোটিং অ্যাকশন বাটন দেখায়।

ReceivablesScreen({super.key, this.initialTabLabel})

কনস্ট্রাক্টর। ঐচ্ছকি initialTabLabel দিয়ে নির্দিষ্ট লবেলের ট্যাব খোলা অবস্থায় স্ক্রিনি শুরু হতে পারে।

Widget build(BuildContext context, WidgetRef ref)

পারমিশন লোড না হলে লোডার দেখায়। permissions.has(...) দিয়ে receiptView ও invoiceView চকে করে কোন ট্যাব যোগ হবে তা ঠিকি করে; কোন ট্যাব না থাকলে "Not available to you" এম্পটি-স্টেটে দেখায়। initialTabLabel মিলিয়ে সূচনা-ইনডেক্স বের করে, এবং AnimatedBuilder দিয়ে ট্যাব পরিবর্তনের সাথে সাথে ফ্লোটিং বাটন আপডেট করে।

ক্লাস/উইজটে: _ReceiptsTab / _ReceiptsTabState

ConsumerStatefulWidget/স্টেটে। রসিপিট তালিকা দেখায় এবং কনফার্ম, ডিপ্লোজিটি (ব্যাংকে জমা), রভার্স ও ডলিটি অ্যাকশন হ্যান্ডলে করে। _busyId দিয়ে কোন রো বর্তমানে প্রসেসিং তা ট্র্যাক করা হয়।

Future<void> _act(int id, Future<void> Function(ReceivablesRepository repo) action, String success, String failure)

সাধারণ অ্যাকশন-রানার। `_busyId` স্টেট করে, রপিজিটরিতে অ্যাকশন চালায়, সফল হলে `receiptsProvider` রফ্রেশ করে ও সাকসেসে সন্ধ্যাকবার দেখায়; `ApiException` হলে তার মসেজে, অন্য এরর হলে জনেরেকি `failure` মসেজে দেখায়। প্রতটি রসিপিট অ্যাকশন খালবিডি রিটার্ন করে বলে প্যাচ না করে পুরো তালিকা রলিড করা হয়।

Future<void> _deposit(PaymentReceipt receipt)

ব্যাংক নাম চয়ে একটি টিকেস্ট প্রম্পট দেখায় (আবশ্যক ফলিড, কারণ এন্ডপয়ন্টে এটি বাধ্যতামূলক); মান পলে `repo.depositReceipt` কল করে `_act` দিয়ে।

Future<void> _reverse(PaymentReceipt receipt)

ঐচ্ছকি কারণ চয়ে প্রম্পট দেখায় (বাউন্সড পমেন্ট বা ভুল ইনভয়সে প্রয়োগ করা রসিপিটের জন্য), তারপর `repo.reverseReceipt` কল করে; রভার্স করলে যা প্রয়োজ্য ছিল তা আবার বকয়ো হয়ে যায়।

Future<void> _delete(PaymentReceipt receipt)

নশ্চতিকরণ ডায়ালগ দেখায় (স্থায়ী ডলিটি, রভার্সের বপিরীতে রকের্ড রাখে না), তারপর `deleteReceipt` কল করে সফল হলে `receiptsProvider.notifier.remove` দিয়ে লোকাল সিরিয়ে দিয়ে।

Widget build(BuildContext context)

`canConfirm` ও `canDelete` পারমিশন চকে করে `ConfigList<PaymentReceipt>` রনেডার করে, প্রতটি আইটমেরে জন্য `_ReceiptCard` তরৈকিরে সংশ্লিষ্ট কলব্যাকসহ (রসিপিটের নজিস্ব `canConfirm/canDeposit/canReverse` ফল্যাগ দিয়েও গটে করা)।

ক্লাস/উইজটে: `_ReceiptCard`

`StatelessWidget`। একটি রসিপিটের কার্ড — ক্লায়েন্ট নাম/রসিপিট নম্বর, পমেন্ট মথেড, পরমাণ, স্ট্যাটাস চপি, তারখি, ব্যাংক তথ্য এবং রভার্সাল কারণ দেখায়; একটি পিপআপ মনেতে সব সক্রিয় অ্যাকশন তালিকাভুক্ত করে।

Widget build(BuildContext context)

ননুল কলব্যাকগুলো থেকে `RowAction` তালিকা বানায় (কনফার্ম, ডিপিজিটি, রভার্স, ডলিটি — শেষে দুটো `destructive`)। ক্লায়েন্ট নাম থাকলে সার্বটাইটলে রসিপিট নম্বর পুনরাবৃত্তি এড়ানো হয় (একলাইনের জায়গা বাঁচাতে)। `busy true` হলে স্পিনার, নাহলে অ্যাকশন থাকলে `PopupMenuButton` দেখায়।

ফাইল: `lib/features/receivables/receivables_sheets.dart`

সংক্ষিপ্ত ববিরণ: একটি ইনভয়সের বপিরীতে ক্রেডিট নোট ইস্যু করার বটম শটি। ক্রেডিট নোট ইস্যুর পরে এডিট করার উপায় নেই — সংশোধন করতে হলে নতুন একটি ইস্যু করতে হয়।

Future<void> showCreditNoteSheet(BuildContext context)

টপ-লেভেলে ফাংশন। `_CreditNoteSheet` মডাল বটম শটি হিসেবে দেখায়।

ক্লাস/উইজটে: `_CreditNoteSheet / _CreditNoteSheetState`

`ConsumerStatefulWidget`/স্টেটে। ইনভয়সে বাছাই, পরমাণ ও কারণ নিয়ে ক্রেডিট নোট রকি়োস্টে জমা দেয়।

`void dispose()`

`_amount` ও `_reason` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে; `_invoiceId` নাল হলে এরর দেখায়। `issueCreditNote` কল করে, তারপর `creditNotesProvider` ও `invoicesProvider` (কারণ ইনভয়েসের বকযো বদলে গেছে) রিফ্রেশ করে, সফল হলে শাট বন্ধ করে স্ন্যাকবার দেখায়। `ApiException` বা অন্য এরর হলে `_error` সটে করে।

`Widget build(BuildContext context)`

শুধু ইতিবাচক ব্যালেন্স সহ ইনভয়েসগুলো (`balanceAmount > 0`) ড্রপডাউনে দেখায় — সম্পূর্ণ পরিশোধিত ইনভয়েসকে ক্রেডিট করলে ঋণাত্মক হয়ে যেতে। কোনো ইনভয়েস না থাকলে তথ্যবাহিতা দেখায়। পরমাণ ফিল্ডের ভ্যালিডেটর শূন্যের বশেই হওয়া বাধ্যতামূলক করে। এন্টার করা পরমাণ বকয়ের চয়ে বশেই হলে (ব্যাকএন্ড এটা চেকে করে না) সতর্কবার্তা দেখায়, কন্ট্রোল জমা আটকাই না। `FormSheetFrame` দিয়ে সাবমিট/এরর UI গঠিত হয়।

ফাইল: `lib/features/recruitment/application_detail_screen.dart`

সংক্ষিপ্ত বিবরণ: একজন আবেদনকারীর বিস্তারিত স্ক্রিন — কে, পাইপলাইনে কোথায়, এবং এ পর্যন্ত কী ঘটছে। ইন্টারভিউ ও অফার এই স্ক্রিন থেকেই দেখানো ও পরিচালনা করা হয়, কারণ এগুলো আবেদনের সাথে যুক্ত, আলাদাভাবে নয়।

ক্লাস/উইজটে: `ApplicationDetailScreen / _ApplicationDetailScreenState`

`ConsumerStatefulWidget`/স্টেটে। আবেদন লোড, স্টেজে পরিবর্তন, স্কোরিং, ট্যালেন্ট পুলে যোগ করা এবং ডলিটি — এই সবকিছু পরিচালনা করে।

`ApplicationDetailScreen({super.key, required this.id})`

কনস্ট্রাক্টর, আবেদনের `id` নেয়।

`static void open(BuildContext context, {required int id})`

`MaterialPageRoute` দিয়ে এই স্ক্রিন পুশ করে।

`void initState()`

`_load()` কল করে শুরুতে আবেদন লোড করে।

`Future<void> _load()`

`recruitmentRepositoryProvider.application(id)` কল করে `_application` সটে করে; ব্যর্থ হলে `_error` সটে করে।

`Future<void> _run(Future<JobApplication> Function() action, String success)`

সাধারণ অ্যাকশন-রনার — `_busy` সটে করে, অ্যাকশন চালায়, সফল হলে

`applicationsProvider.notifier.apply` দিয়ে তালিকার কপি আপডেট করে ও লোকাল `_application` সটে করে, স্ন্যাকবার দেখায়; এরর হলে `ApiException` মসেজে বা জনেরেকি মসেজে দেখায়।

`Future<void> _moveStage()`

`_StagePicker` বটম শাট দেখিয়ে নতুন স্ট্যাটাস বাছাই করায়, তারপর `setApplicationStatus` কল করে `_run` দিয়ে।

Future<void> _score()

_ScoreSheet বটম শাট দিখেই স্কোর ম্যাপ পায়, তারপর evaluate(...) কল করে পাঁচটি স্কোর (education, experience, technicalSkills, interview, communication) পাঠায়।

Future<void> _addToPool()

_pooling সটে করে poolApplication কল করে; আবদেনের যোগাযোগ তথ্য থেকেই পুল এন্ট্রি তৈরি হয়, তাই ব্যবহারকারীকে কিছু পূরণ করতে হয় না, শুধু সর্দিধানত নতি হয়।

Future<void> _delete(JobApplication application)

নিশ্চিতকরণ ডায়ালগ দেখায় (এটি সারভার-সাইড সফট-ডিলিট, কন্ট্রোল অ্যাপে ফরোনার কোনও উপায় নেই), তারপর deleteApplication কল করে সফল হলে তালিকা থেকে সরিয়ে স্ক্রিনি বন্ধ করে।

Widget build(BuildContext context)

এরর/লোডিং স্টেটে হ্যান্ডলে করে। হডোর, কন্টাক্ট অ্যাকশন, (টার্মিনাল না হলে) মুভ-স্টেজ/স্কোর বাটন, হায়ার্ড ব্যানার, ট্যালেন্ট-পুল বাটন, স্কোর, ইন্টারভিউ সেকশন, অফার সেকশন এবং নোট — এসব ListView এ সাজায়।

ক্লাস/উইজটে: _Header

StatelessWidget। আবদেনকারীর নাম, পদবি, স্ট্যাটাস চপি, সোর্স ও ATS স্কোর দেখায়।

Widget build(BuildContext context)

একটি AppCard এ নাম/পদ/চপি সাজায়; সোর্স বা ATS স্কোর থাকলে একটি ডিভাইডারের নচি দেখায়।

ক্লাস/উইজটে: _Contact

StatelessWidget। ফোন, ইমেইল, সম্মিলিত অ্যাকশন বাটন দেখায়।

Future<void> _open(BuildContext context, Uri uri, String failure)

launchUrl দিয়ে URI খোলে; ব্যর্থ হলে বা এক্সপেশন এলে ব্যর্থতার মসেজে সন্ধ্যাকবারে দেখায়।

Widget build(BuildContext context)

ফোন/ইমেইল/সম্মিলিত থাকলে সংশ্লিষ্ট _Action বাটন তৈরি করে tel:/mailto:/সরাসরি URI দিয়ে। কোনোটাই না থাকলে তথ্যবাহ্যতা দেখায়।

ক্লাস/উইজটে: _Action

StatelessWidget। একটি আইকন-লেবেলে যুক্ত OutlinedButton — যোগাযোগ কার্যক্রমের সাধারণ বন্ডিং ব্লক।

Widget build(BuildContext context)

icon, label, onTap দিয়ে বাটন রেন্ডার করে, ন্যূনতম উচ্চতা ৪৪।

ক্লাস/উইজটে: _Scores

StatelessWidget। পাঁচটি ম্যানুয়াল স্কোর (education, experience, technical skills, interview, communication) এবং সামগ্রিক স্কোর দেখায়।

Widget build(BuildContext context)

null না থাকা স্কোরগুলো ফিল্টার করে সারি হিসেবে দেখায়; overallScore থাকলে হডোরের ট্রাইলিংয়ে দেখায়।

ক্লাস/উইজটে: _Interviews

`ConsumerWidget`। আবদেনরে ইন্টারভিউ তালিকা দেখায় এবং নতুন ইন্টারভিউ শিডিউল করার অপশন দেয়।

`Future<void> _schedule(BuildContext context, WidgetRef ref)`

`showScheduleInterviewSheet` দেখায়; শিডিউল হলে `applicationInterviewsProvider` ইনভ্যালিডেট করে এবং `onScheduled()` কল করে (কারণ প্রথম রাউন্ড শিডিউল হলে আবদেন এগিয়ে যায়, তাই হডোরও stale হয়ে যায়)।

`Widget build(BuildContext context, WidgetRef ref)`

`applicationInterviewsProvider` থেকে ডটো লোড/এরর/ফাঁকা/তালিকা অবস্থা হ্যান্ডলে করে; `canSchedule` হলে হডোরে "Schedule" বাটন এবং প্রতিটি ইন্টারভিউয়ের জন্য `InterviewCard` দেখায়।

ক্লাস/উইজটে: _Offers / _OffersState

`ConsumerStatefulWidget`/স্টেটে। আবদেনরে অফার তালিকা দেখায় এবং অফার ড্রাফট, পাঠানো, গ্রহণ/প্রত্যাখ্যান রকেড, প্রত্যাহার — এসব অ্যাকশন পরিচালনা করে।

`Future<void> _draft()`

`showCreateOfferSheet` দেখায়; তৈরি হলে `applicationOffersProvider` ইনভ্যালিডেট করে ও `onDrafted()` কল করে (ড্রাফট করলে আবদেন `OFFER_PENDING`-এ যায়)।

`Future<void> _transition(JobOffer offer, String action, String success)`

`_busyOffer` স্টেটে করে `offerTransition(offer.id, action)` কল করে (`send/accept/decline/withdraw`), সফল হলে তালিকা ইনভ্যালিডেট করে সফলতা মেসেজে দেখায়, এরর হ্যান্ডলে করে।

`Widget build(BuildContext context)`

হডোরে "Draft" বাটন শুধু তখনই দেখায় যখন কোনো লাইভ অফার নেই (একরে বশে অফার ব্যাকএন্ড প্রত্যাখ্যান করে)। প্রতিটি অফার কার্ডে পদবি, স্ট্যাটাস চপি, গ্রস স্যালারি, জিনেং ডেট, ম্যোদোত্তীর্ণ সত্বকতা, প্রত্যাখ্যানের কারণ, এবং (অফার স্টেলে না হলে)

`Send/Accepted/Declined/Withdraw` বাটন দেখায় — অফারের অবস্থা (`draft/sent`) অনুযায়ী কোনগুলো দেখানো হবে তা নির্ধারণিত হয়।

ক্লাস/উইজটে: _StagePicker

`StatelessWidget`। আবদেনটি পরিবর্তী কোন স্ট্যাটাসে যতে পারে তার তালিকা দেখানো বটম শিট।

`Widget build(BuildContext context)`

`application.nextStatuses` থেকে প্রতিটি অপশনের জন্য `ListTile` দেখায়, ট্যাপ করলে `Navigator.pop(context, status)` করে বেছে নেওয়া স্ট্যাটাস ফেরত দেয়।

ক্লাস/উইজটে: _ScoreSheet / _ScoreSheetState

`StatefulWidget`/স্টেটে। পাঁচটি (সবই ঐচ্ছিক) ম্যানুয়াল স্কোর স্লাইডারসহ একটি `DraggableScrollableSheet`; ব্যাকএন্ড যা উপস্থিতি আছে তার গড় করে, তাই অর্ধপূর্ণ শিটও বৈধ জমা।

`Widget build(BuildContext context)`

প্রতিটি স্কোরের জন্য `Slider` (০-১০, স্লাইডার শূন্যে থাকলে মান `null`) ও লবেলে দেখায়। "Save scores" বাটনে ননুল স্কোরগুলো নিয়ে একটি ম্যাপ `Navigator.pop` দিয়ে ফেরত দেয়।

ফাইল: lib/features/recruitment/candidate_detail_screen.dart

সংক্ষিপ্ত ববিরণ: একজন ব্যক্তির প্রোফাইল — তিনি এখন পর্যন্ত যত চাকরিতে আবেদন করছেন সব জুড়ে যা সত্য থাকে (JobApplication হলো প্রতি-পোস্টিং পাইপলাইন স্টেটে, আর এই স্ক্রিনি যা পোস্টিং নির্বিশেষে অপরিবর্তিত)।

ক্লাস/উইজটে: CandidateDetailScreen / _CandidateDetailScreenState

ConsumerStatefulWidget/স্টেটে। প্রার্থীর তথ্য লোড, এডটি এবং ডলিটি পরিচালনা করে।

CandidateDetailScreen({super.key, required this.id})

কনস্ট্রাক্টর, প্রার্থীর id নেয়।

static void open(BuildContext context, {required int id})

MaterialPageRoute দিয়ে স্ক্রিনি পুশ করে।

void initState()

_load() কল করে।

Future<void> _load()

recruitmentRepositoryProvider.candidate(id) কল করে _candidate স্টেটে করে, ব্যর্থ হলে _error স্টেটে করে।

Future<void> _edit(Candidate candidate)

showEditCandidateSheet দেখায়; ফলাফল পলে লোকাল _candidate আপডেটে করে (তালিকা ইতিমধ্যে শাটের মধ্যেই আপডেটে হয়ে গেছে, এই স্ক্রিনি নিজেরে কপি রাখতে)।

Future<void> _delete(Candidate candidate)

নিশ্চিতকরণ ডায়ালগ দেখায় (আবেদনগুলো মুছে যায় না, শুধু প্রার্থী তালিকা থেকে বাদ যায়), candidatesProvider.notifier.delete কল করে সফল হলে স্ক্রিনি বন্ধ করে।

Widget build(BuildContext context)

এরর/লোডিং হ্যান্ডলে করে; পারমিশন অনুযায়ী এডটি/ডলিটি পপআপ মেনু দেখায়; হডোর, কন্টাক্ট, আবেদন তালিকা ListView-এ সাজায়।

ক্লাস/উইজটে: _Header

StatelessWidget। প্রার্থীর নাম, বর্তমান পদবি, স্কলি, আবেদন সংখ্যা ও সোর্স দেখায়।

Widget build(BuildContext context)

AppCard-এ নাম/পদবি/স্কলি এবং একটি ডিভাইডারের নচে আবেদন-সংখ্যা ও সোর্স দেখায়।

ক্লাস/উইজটে: _Contact

StatelessWidget। ফোন, ইমেইল, সন্নিবিষ্ট অ্যাকশন বাটন দেখায়

(application_detail_screen.dart-এর সমতুল্য, তবে Candidate মডেলের উপর কাজ করে)।

Future<void> _open(BuildContext context, Uri uri, String failure)

launchUrl কল করে, ব্যর্থতায় সন্যাকবার দেখায়।

Widget build(BuildContext context)

ফোন/ইমেইল/সন্নিবিষ্ট থাকলে _Action বাটন তৈরি করে; কোনোটাই না থাকলে তথ্যবাহ্য দেখায়।

ক্লাস/উইজটে: `_Action`

`StatelessWidget`। আইকন-লবেলযুক্ত `OutlinedButton`।

`Widget build(BuildContext context)`

`icon`, `label`, `onTap` দিয়ে বাটন রেন্ডার কর।

ক্লাস/উইজটে: `_Applications`

`ConsumerWidget`। এই প্রার্থীর সব আবদেনের তালিকা দেখায়, প্রতিটি ট্যাপ করলে

`ApplicationDetailScreen`-এ যায়।

`Widget build(BuildContext context, WidgetRef ref)`

`candidateApplicationsProvider(candidateId)` থেকে লোডিং/এরর/ফাঁকা/তালিকা অবস্থা হ্যান্ডলে করে; প্রতিটি আবদেনের জন্য পদবী ও স্ট্যাটাস চপিয়ুক্ত ট্যাপযোগ্য কার্ড দেখায়।

ফাইল: `lib/features/recruitment/candidate_form_sheet.dart`

সংক্ষিপ্ত বিবরণ: প্রার্থী এডিট করা, ট্যালেন্ট পুলে যোগ করা এবং পুল এন্ট্রি এডিট করা — তিনটি কাজের জন্য একই ফর্ম শেয়ার করা বটম শিট, কারণ একজন ব্যক্তির তথ্য যখনই ফাইল করা হোক না কেন একই — কিন্তু রকিওয়েস্ট বডি ভিন্ন (ব্যাকএন্ডে দুটি ভিন্ন DTO একই নামে

`CandidateRequest`)।

`enum _Mode { editCandidate, addToPool, editPoolEntry }`

শিটটি কী এডিট করছে তা নির্দেশ করে; তিনটি মোডের ভিত্তিতে ফর্ম কোন এন্ডপয়েন্টে কী বডি পাঠাবে তা নির্ধারণিত হয়।

`extension on _Mode { bool get isPool }`

`_Mode` এর উপর এক্সটেনশন। `editCandidate` ছাড়া অন্য যেকোনো মোডে `true` — অর্থাৎ এটি ট্যালেন্ট-পুল-সম্পর্কিত মোড কনি তা বলে।

`Future<Candidate?> showEditCandidateSheet(BuildContext context, Candidate candidate)`

টপ-লভেলে ফাংশন। প্রার্থীর নিজস্ব রেকর্ড এডিট করার শিট দেখায়, সার্ভারের ফরেক-দওয়া ভার্সন রটার্ন করে (বা বাতলি হল `null`) — ডিটাইল স্ক্রিনি নিজের কপিরাখে বলে এটি দরকার।

`Future<void> showAddToTalentPoolSheet(BuildContext context)`

টপ-লভেলে ফাংশন। কখনো আবদেন করেনি এমন কাউকে (রেফারলে বা ফানলে বাইরের যোগ্য প্রার্থী) ট্যালেন্ট পুলে যোগ করার শিট দেখায়।

`Future<void> showEditTalentPoolSheet(BuildContext context, TalentPoolCandidate entry)`

টপ-লভেলে ফাংশন। একটি বিদ্যমান পুল-এন্ট্রি এডিট করার শিট দেখায়।

ক্লাস/উইজটে: `_CandidateFormSheet / _CandidateFormSheetState`

`ConsumerStatefulWidget`/স্টেটে। মোড অনুযায়ী নাম, ইমেইল, ফোন, বর্তমান পদবী/কাঙ্ক্ষিত ভূমিকা, স্কলি, সোর্স/কারণ/রেটিং, লিংক ও নোট নিয়ে একটি ফর্ম দেখায় এবং সংশ্লিষ্ট রকিওয়েস্ট বডি বানিয়ে জমা দেয়।

`void initState()`

`widget.initial` (candidate seed) বা `widget.pool` (pool seed) — যেকোনো একটি সীটে থাকে — থেকে সব টেক্সট কন্ট্রোলার এবং `_source/_reason/_rating` প্রাথমিক মান নির্ধারণ করে।

`void dispose()`

সব টেক্সট কন্ট্রোলার (name, email, phone, currentTitle, skills, resumeUrl, linkedIn, portfolio, notes) ডিসপোজ করে।

`String get _title`

মোড অনুযায়ী শিরোনাম রিটার্ন করে ("Edit candidate" / "Add to talent pool" / "Edit pool entry")।

`String get _action`

মোড অনুযায়ী সাবমিট-বাটনের লবেলে রিটার্ন করে।

`String get _done`

মোড অনুযায়ী সাফল্যের সন্ধ্যাকবার মসেজে রিটার্ন করে।

`CandidateRequest _candidateRequest()`

টেক্সট কন্ট্রোলার ও `_source` থেকে `CandidateRequest` তৈরি করে — শুধু `editCandidate` মোডে ব্যবহৃত হয়।

`TalentPoolRequest _poolRequest()`

টেক্সট কন্ট্রোলার, `_rating`, `_reason` থেকে `TalentPoolRequest` তৈরি করে — `_currentTitle` কন্ট্রোলারের মান এখানে `desiredRole` হিসেবে ব্যবহৃত হয় (একই ইনপুট বক্স, ভিন্ন অর্থ)।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, মোড অনুযায়ী `switch` করে `candidatesProvider.notifier.updateItem` / `talentPoolProvider.notifier.add` / `talentPoolProvider.notifier.updateItem` কল করে; সফল হলে শিট বন্ধ করে (`editCandidate` মোডে আপডেটেড প্রার্থী `pop` এর সাথে ফেরত দেয়) ও `_done` মসেজে দেখায়; এরর হ্যান্ডলে করে `_error` সটে করে।

`Widget build(BuildContext context)`

`_source` বর্তমান তালিকার বাইরে থাকলে সটেকে ড্রপডাউন অপশনে যোগ করে। নাম (আবশ্যিক), ইমেল (পুল মোডে আবশ্যিক, নাহলে ঐচ্ছিক — কারণ ব্যাকএন্ড খালি ইমেলকে `.trim()` করে ৪০০ রিটার্ন করে), ফোন, বর্তমান পদবী/কাঙ্ক্ষিত ভূমিকা, স্কলি ফল্ড দেখায়। পুল না হলে সোর্স ড্রপডাউন, পুল হলে কারণ ও রটেই (১-৫, ব্যাকএন্ড রঞ্জে বাইরে প্রত্যাখ্যান করে বলে ড্রপডাউন ব্যবহৃত, ফ্রি নাম্বার ফল্ড নয়) দেখায়। এরপর লিংক সেকশন (CV, LinkedIn, এবং পুল না হলে Portfolio) এবং নোট দেখায়। শেষে `LoadingButton` দিয়ে সাবমিট করে।

ফাইল: `lib/features/recruitment/career_page_screen.dart`

সংক্ষিপ্ত বিবরণ: কোম্পানির পাবলিক ক্যারিয়ার্স পেজে দেখা ও এডটি করার স্ক্রিনি, এর রপিজিটরি/মডলে/প্রোভাইডারসহ। পেজটিকোম্পানিপ্রোফাইলের অংশ নয়, বরং নিজস্ব স্টেইস রেকর্ড — স্লাগ ক্লায়েন্ট পোর্টাল সাবডোমেইন থেকে আলাদা একটি পাবলিক ঠিকানা।

ক্লাস/উইজটে: `CareerPageSettings`

ক্যারিয়ার্স পেজের স্টেইস মডলে — স্লাগ, published ফ্ল্যাগ, headline, about, brandColor।

```
const CareerPageSettings({required this.slug, required this.published, this.headline, this.about, this.brandColor})
```

কনস্ট্রাক্টর।

```
factory CareerPageSettings.fromJson(Map<String, dynamic> json)
```

JSON থেকে অবজেক্ট তৈরি করে; `slug` ও `published` না থাকলে যথাক্রমে খালি স্ট্রিং ও `false` ডিফল্ট নিয়ে।

```
Map<String, dynamic> toJson()
```

সব ফিল্ড (নাল-চকে ছাড়াই) একটি ম্যাপে রূপান্তর করে — PUT এন্ডপয়েন্ট যা রিটার্ন করে সেই একই আকৃতি নিয়ে, তাই সম্ভবে সবসময় পুরোটাই পাঠায়।

```
CareerPageSettings copyWith({...})
```

নর্দিশ্টি ফিল্ড পরিবর্তন করে নতুন `CareerPageSettings` তৈরি করে, বাকগুলো অপরিবর্তিত রাখা।

ক্লাস/উইজটে: CareerPageRepository

ক্যারিয়ার্স পেজে স্টেইস আনা ও সম্ভবে করার জন্য API রপিজিটরি।

```
CareerPageRepository(this._api)
```

কনস্ট্রাক্টর, `ApiClient` নিয়ে।

```
Future<CareerPageSettings> settings()
```

GET `/hr/career-page` কল করে স্টেইস আনা; পড়াই এটি তৈরি করে দেয় যদি না থাকে, কোম্পানির নাম দিয়ে সডি করে — তাই এমন কোনো কোম্পানিই যার পেজে নাই।

```
Future<CareerPageSettings> save(CareerPageSettings settings)
```

PUT `/hr/career-page` কল করে স্টেইস সম্ভবে করে। সম্ভবে জন্য `JOB_POSTING_CREATE` লাগে, পড়ার জন্য শুধু `JOB_POSTING_VIEW` — একজন রকিউটার এডিট করতে না পারলেও পেজটি দেখতে পারে।

টপ-লেভেলে প্রোভাইডার

`careerPageRepositoryProvider` (`Provider<CareerPageRepository>`) এবং `careerPageProvider` (`FutureProvider.autoDispose<CareerPageSettings>`) — যথাক্রমে রপিজিটরি ইনস্ট্যান্স এবং স্টেইস ফেচ করার জন্য।

ক্লাস/উইজটে: CareerPageScreen

`ConsumerWidget`। ক্যারিয়ার্স পেজের প্রভিডি, `published/unpublished` ব্যানার, ঠিকানা কপি বাটন, এডিট বাটন এবং বর্তমানে খোলা চাকরির তালিকা দেখায়।

```
static void open(BuildContext context)
```

`MaterialPageRoute` দিয়ে স্ক্রিনি পুশ করে।

```
Widget build(BuildContext context, WidgetRef ref)
```

`canEdit` (`JOB_POSTING_CREATE`) পারমিশন চকে করে। লোডিং/এরর/ডটো হ্যান্ডলে করে; `published` কনি অনুযায়ী সতর্কতা বা তথ্য ব্যানার দেখায়, `headline/about`, স্লাগ ও কপি-বাটন, (`canEdit` হলো) এডিট বাটন, এবং `_OpenJobs` সেকশন দেখায়।

ক্লাস/উইজটে: _OpenJobs

`ConsumerWidget`। বর্তমানে বজ্র্গপতি চাকরির তালিকা দেখায় — নজিস্ব এন্ডপয়েন্ট থেকে (পুরো তালিকা ফলিটার করে নয়), কারণ কোনটি "খোলা" তা ব্যাকএন্ডের সিদ্ধান্ত (published, not closed, closing date পার হয়নি) — এই লজিক এখানে ডুপ্লিকেট করলে দুটো জায়গা ভিন্নমত পোষণ করতে পারত।

`Widget build(BuildContext context, WidgetRef ref)`

`openJobsProvider` থেকে লোডিং/এরর/ফাঁকা/তালিকা হ্যান্ডলে করে। `canRefer` (applicationCreate পারমিশন) থাকলে প্রতটি চাকরির পাশে একটি "Put somebody forward" বাটন দেখায় যা `showReferCandidateSheet` খোলে — পজে এডটি করার পারমিশন থেকে আলাদা একটি এনটাইটলেমেন্ট।

`Future<void> showCareerPageSheet(BuildContext context, {required CareerPageSettings settings})`

টপ-লেভেলে ফাংশন। `_CareerPageSheet` মডাল বটম শিট দেখায়।

ক্লাস/উইজটে: _CareerPageSheet / _CareerPageSheetState

`ConsumerStatefulWidget`/স্টেটে। ক্যারিয়ার্স পজেরে স্লাগ, headline, about ও published স্ট্যাটাস এডটি করার ফর্ম।

`void dispose()`

`_slug`, `_headline`, `_about` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, `careerPageRepositoryProvider.save` কল করে (স্লাগ ট্রিম ও লোয়ারকেসে, headline ও about ট্রিম করে), সফল হলে `careerPageProvider` ইনভ্যালিডেট করে শিট বন্ধ করে। `ApiException` থেকে আসা দুটি নির্দিষ্ট প্রত্যাখ্যান-বার্তা (ভুল প্যাটার্নের স্লাগ, বা অন্য কোম্পানি ইতিমধ্যে নেওয়া স্লাগ) সরাসরি দেখানো হয়।

`Widget build(BuildContext context)`

`FormSheetFrame` ব্যবহার করে: স্লাগ ফিল্ড (রেজেক্স `^[a-z0-9](?:[a-z0-9-]{1,58}[a-z0-9])?$` দিয়ে ব্যাকএন্ডের নিম্ন আগভাগে যাচাই করে — লোয়ারকেসে অক্ষর, সংখ্যা ও হাইফেন, শুরু/শেষে হাইফেন নয়), headline, about (মাল্টিলাইন), এবং published টগলে জন্য `SwitchListTile.adaptive` (বন্ধ করলে পজে নামিয়ে নেওয়া হয়, কিন্তু খোলা চাকরিলো খোলাই থাকে, শুধু বজ্র্গপতি হওয়া বন্ধ হয়)।

done

ফাইল: lib/features/recruitment/create_offer_sheet.dart

সংক্ষিপ্ত বিবরণ: একটি জিব অ্যাপ্লিকেশনের বিপরীতে অফার তৈরি (draft) জন্য বটম শিট। এটি অফারের মূল তথ্য (জিব টাইটলে, বতেন, মেয়াদ, যোগদানের তারিখ) এবং ঐচ্ছিক ব্রেকেডাউন (basic, house rent, medical, transport) সংগ্রহ করে `OfferRequest` তৈরি করে ব্যাকএন্ডে পাঠায়।

টপ-লেভেলে ফাংশন

Future<bool> showCreateOfferSheet(BuildContext context, {required int applicationId, String? suggestedJobTitle})

একটি মডাল বটম শিট (`_CreateOfferSheet`) খোলে যখন `applicationId` এবং ঐচ্ছিক `suggestedJobTitle` (পোস্টিং-এর জব টাইটলে, ডফিল্ট মান হিসেবে দেখানো হয়) পাস করা হয়। শিট থেকে

`Navigator.pop(true)` এলে অফার তৈরি হয়েছে বলে ধরা হয় এবং `true` রিটার্ন করে; বন্ধ করে দিলে `false`।
ক্যালার এই রিটার্ন মান দিয়ে অফার লিস্ট ও অ্যাপ্লিকেশন রিফ্রেশ করে, কারণ অফার তৈরি হলে অ্যাপ্লিকেশনের স্ট্যাটাস `OFFER_PENDING`-এ চলে যায়।

ক্লাস/উইজটে: `_CreateOfferSheet`

`ConsumerStatefulWidget`, বটম শিটে কনফিগারেশন ধারণ করে — `applicationId` (যে অ্যাপ্লিকেশনে বপিরীতে অফার হবে) এবং `suggestedJobTitle`।

`ConsumerState<_CreateOfferSheet> createState()`

`_CreateOfferSheetState` ইনস্ট্যান্স তৈরি করে ফর্মের স্টেটে পরিচালনার জন্য।

ক্লাস: `_CreateOfferSheetState`

ফর্মের সব `TextEditingController` (জব টাইটলে, গ্রস স্যালারি, `basic`, `house rent`, `medical`, `transport`, `notes`), তারিখ স্টেটে (`_joiningDate`, `_expiryDate`), এবং সাবমিশন/এর স্টেটে রাখা ও পরিচালনা করে।
ডফিল্টভাবে `_expiryDate` আজকে তারিখ থেকে ১৪ দিন পরে সেটে থাকে, কারণ ব্যাকএন্ড খালি মিয়াদ গ্রহণ করে না।

`void dispose()`

সব `TextEditingController` (`_jobTitle`, `_gross`, `_basic`, `_houseRent`, `_medical`, `_transport`, `_notes`) `dispose` করে মেমোরি লিক এড়াই, তারপর `super.dispose()` কল করে।

Future<void> `_submit()`

ইতিমধ্যে সাবমিট হচ্ছে বা ফর্ম ভ্যালিড না হলে কিছু করে না। ভ্যালিড হলে ফোকাস সরিয়ে `_submitting` `true` করে, `recruitmentRepositoryProvider.createOffer` কল করে `OfferRequest` পাঠায় (মিয়াদ ও যোগদানের তারিখ `Fmt.isoDate` দিয়ে ফর্ম্যাট করা হয়, গ্রস স্যালারি `double.parse`, বাকি ব্রেকেডাউন ফিল্ড `double.tryParse` দিয়ে ঐচ্ছিক)। সফল হলে শিট বন্ধ করে `true` রিটার্ন করে এবং সন্ধ্যাকবার দেখায়; `ApiException` এলে সার্ভারের এরর মেসেজ দেখায় (যেমন "already has an active offer" বা "application is closed"), অন্য এরর হলে জনৈকি মেসেজ দেখায়; শেষে `_submitting` `false` করে।

`Widget build(BuildContext context)`

`Form` এর মধ্যে সব ইনপুট ফিল্ড সাজায় — জব টাইটলে (আবশ্যিক), গ্রস স্যালারি (আবশ্যিক, ধনাত্মক সংখ্যা), `DateField` দিয়ে মিয়াদ (`clearable: false`) ও যোগদানের তারিখ (ঐচ্ছিক), এবং "Breakdown" সেকশনে `basic/house rent/medical/transport` ও নোটস। শেষে `LoadingButton` দিয়ে সাবমিট বাটন।

`static String? _money(String? value)`

ব্রেকেডাউন ফিল্ডগুলোর (ঐচ্ছিক মান ফিল্ড) ভ্যালিডিটর। খালি হলে বৈধ (`null` রিটার্ন), সংখ্যা পার্স করতে না পারলে "Enter a number." এবং ঋণাত্মক হলে "Cannot be negative." রিটার্ন করে।

ফাইল: lib/features/recruitment/interview_feedback_sheet.dart

সংক্ষিপ্ত ববিরণ: একটি নিরীধারতি ইন্টারভিউ কার্ড ও তার সাথে সম্পর্কতি কাজ (ফডিবিয়াক দেওয়া, বাতলি করা) দেখোয়, এবং ফডিবিয়াক সংগ্রহরে জন্য় একটি বিটম শটি রাখে। ইন্টারভিউর ফলাফল সারাংশ, "এখনো ফডিবিয়াক দেওয়া হয়নি" সতর্কতা এবং অ্যাকশন বাটনও পরচিলনা করে।

ক্লাস/উইজটে: InterviewCard

একটি Interview অবজেক্টকে কার্ড আকারে রেন্ডার করে, ঐচ্ছকিভাবে আবদেনকারীর নাম দেখোয় (showApplicant) এবং স্ট্যাটাস পরবির্তনের কলব্যাক (onChanged) নেয়।

ConsumerState<InterviewCard> createState()
_InterviewCardState তরৈকিরে।

ক্লাস: _InterviewCardState

ইন্টারভিউ কার্ডরে বজি-স্টেটে (_busy) ও ফডিবিয়াক/বাতলিরে কার্যক্রম পরচিলনা করে।

Future<void> _feedback()

_FeedbackSheet মডাল বিটম শটি খোলে, ব্যবহারকারীর দেওয়া _FeedbackResult পলে _run কল করে recruitmentRepositoryProvider.submitFeedback চালায়, rating/strengths/concerns/recommendation/noShow পাঠয়ি। বাতলি করা হলে বা widget আনমাউন্ট হলে কছি করে না।

Future<void> _cancel()

AlertDialog দয়িে নশ্চিতিকরণ চায় ("Cancel this interview?", জানায় অ্যাপ কাউকে নোটিফাই করে না)। নশ্চিতি হলে _run দয়িে cancelInterview কল করে।

Future<void> _run(Future<Interview> Function() action, String success)

সাধারণ হলেপার — _busy true করে action() চালায়, সফল হলে upcomingInterviewsProvider.notifier.apply(updated) কল করে লস্টি আপডেটে করে, widget.onChanged কলব্যাক ডাকে এবং সফলতার সন্য়াকবার দেখোয়। ApiException এলে সার্তভাররে মসেজে, অন্য়থায় জনেরেকি এরর মসেজে দেখোয়; শেষে _busy false করে।

Widget build(BuildContext context)

ইন্টারভিউর শরিনোম (আবদেনকারীর নাম বা রাউন্ড), সাবটাইটলে, স্ট্যাটাস চপি (আজকরে হলে "Today" দেখোয়), সময়সূচিতিথ্য (তারথি, সময়কাল, মোড), awaitingFeedback হলে সতর্কবার্তা, hasFeedback হলে _FeedbackSummary, এবং permission (applicationUpdate) থাকলে ও স্ট্যাটাস isScheduled হলে Feedback/Move/Cancel বাটন দেখোয়। "Move" বাটন showScheduleInterviewSheet কে বদিয়মান ইন্টারভিউসহ কল করে (রশিডিউল)।

ক্লাস/উইজটে: _FeedbackSummary

ইতিমধ্যে দেওয়া ফডিবিয়াকরে (rating, recommendation, strengths, concerns) সংক্ষিপ্ত প্রদর্শন করে একটি স্টাইল করা কনটেইনারে।

Widget build(BuildContext context)

rating থাকলে তারকা আইকন ও সংখ্যা, recommendation থাকলে লবেলে দেখোয়; strengths/concerns খালি না হলে সেগুলো টেক্সট হিসেবে দেখোয়।

ক্লাস: `_FeedbackResult`

ফিডব্যাক শাটিকে ফরমের আসা ডটো বহনকারী সরল ডটো ক্লাস — `rating`, `strengths`, `concerns`, `recommendation`, `noShow`।

টপ-লেভেলে কনস্ট্যান্ট

`const _recommendations`

ব্যাকগ্রাউন্ডে গ্রহণযোগ্য সুপারিশগুলোর তালিকা, সবচেয়ে শক্তিশালী থেকে দুর্বল ক্রমে: `STRONG_HIRE`, `HIRE`, `NEUTRAL`, `NO_HIRE`, `STRONG_NO_HIRE`।

ক্লাস/উইজটে: `_FeedbackSheet`

একটি ইন্টারভিউর জন্য ফিডব্যাক ফর্ম ধারণকারী `StatefulWidget`; `interview` প্যারামিটার নিয়ে প্রদর্শনের জন্য (আবেদনকারীর নাম)।

`State<_FeedbackSheet> createState()`

`_FeedbackSheetState` তৈরি করে।

ক্লাস: `_FeedbackSheetState`

ফিডব্যাক ফর্মের কন্ট্রোলার (`_strengths`, `_concerns`) ও স্টেটে (`_rating`, `_recommendation`, `_noShow`) ধারণ করে।

`void dispose()`

`_strengths` ও `_concerns` কন্ট্রোলার `dispose` করে।

`Widget build(BuildContext context)`

`DraggableScrollableSheet` এর মধ্যে ফর্ম দেখায়: "They did not turn up" সুইচ (no-show হলে স্কোরিং সেকশন লুকানো হয়), ১-৫ তারকা রেটিং সলিক্টর, রকিমেন্ডেশন চপি (`ChoiceChip`, একই অপশন আবার সলিক্ট করলে ডিসলিক্ট হয়), `strengths/concerns` টেক্সট ফিল্ড, এবং সাবমিট বাটন যা `Navigator.pop` এর মাধ্যমে `_FeedbackResult` ফরমে দিয়ে (no-show হলে `rating/strengths/concerns/recommendation` সব null পাঠায়)।

ফাইল: `lib/features/recruitment/job_posting_form_sheet.dart`

সংক্ষিপ্ত বিবরণ: নতুন জব পোস্টিং তৈরি বা বর্তমান পোস্টিং সম্পাদনার জন্য বটম শাট ফর্ম। স্কলিস লিস্ট, শিক্ষাগত যোগ্যতার সীমা এবং রকিউটার এসাইনমেন্ট ইচ্ছাকৃতভাবে এখানে নেই — সগোলো ওয়েব অ্যাপ থেকে যায়।

টপ-লেভেলে ফাংশন

`Future<void> showNewJobPostingSheet(BuildContext context, {JobPosting? existing})`

`_JobPostingFormSheet` মডাল বটম শাট খোলে। `existing` দিলে এডটি মোড, না দিলে নতুন পোস্টিং তৈরি মোড।

ক্লাস/উইজটে: `_JobPostingFormSheet`

`ConsumerStatefulWidget`, ঐচ্ছকি `existing` জব পোস্টিং ধারণ করে যা সম্পাদনার জন্য প্র-ফিলি করতে ব্যবহৃত হয়।

`ConsumerState<_JobPostingFormSheet> createState()`

`_JobPostingFormSheetState` তৈরি করে।

ক্লাস: `_JobPostingFormSheetState`

সকল টেক্সট কন্ট্রোলার (`title`, `jobTitle`, `location`, `vacancies`, `salaryMin`, `salaryMax`, `description`, `requirements`, `responsibilities`), `employmentType`, `deadline`, `remote` ফ্ল্যাগ ও সাবমিশন/এরর স্টেটে পরিচালনা করে।

`static String _plain(num? value)`

একটি সংখ্যাকে টেক্সট ফিল্ডে দেখানোর জন্য প্লাইন স্ট্রিং-এ রূপান্তর করে (কোনো মুদ্রা চহিন ছাড়া, যাতে আবার পার্স করা যায়)। মান পূর্ণসংখ্যা হলে দশমকি ছাড়া দেখায়, অন্যথায় যমেন আছে তমেন দেখায়। `null` হলে খালি স্ট্রিং রিটার্ন করে।

`bool get _isEdit`

`widget.existing` non-null কনি তা যাচাই করে বোঝায় ফর্মটি এডিট মোডে আছে কনি।

`void dispose()`

সব টেক্সট কন্ট্রোলার (`_title`, `_jobTitle`, `_location`, `_vacancies`, `_salaryMin`, `_salaryMax`, `_description`, `_requirements`, `_responsibilities`) `dispose` করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেশনের পর `JobPostingRequest` তৈরি করে সব ফিল্ড দিয়ে। `_isEdit` true হলে `jobsProvider.notifier.updateItem` কল করে, না হলে `create` কল করে। সফল হলে শটি বন্ধ করে যথাযথ সন্ধ্যাকবার মসেজে দেখায় ("Posting updated." বা "Saved as a draft. Publish it when ready."); `ApiException` বা সাধারণ এরর হ্যান্ডল করে।

`Widget build(BuildContext context)`

ফর্মে সব ফিল্ড সাজায়: টাইটলে (আবশ্যক), জব টাইটলে, এমপ্লয়মেন্ট টাইপ ড্রপডাউন (`employmentTypes` থেকে, `directory_models.dart` থেকে আমদানকিত), লোকেশন, রমিোট সুইচ, ভ্যাকান্সি সংখ্যা (কমপক্ষে ১), স্যালারি রেঞ্জ (max, min-এর কম হলে এরর), ডেডলাইন `DateField` (শুধু ভবিষ্যতের তারখি, কারণ অতীত ডেডলাইন প্রকাশের সাথে সাথেই পোস্টিং বন্ধ করে দেবে), বর্ণনা/দায়িত্ব/প্রয়োজনীয়তা টেক্সট ফিল্ড, এবং সাবমিট বাটন।

`static String? _money(String? value)`

স্যালারি ফিল্ডের ভ্যালিডেটর — খালি হলে বধৈ, পার্স ব্যর্থ হলে "Enter a number.", ঋণাত্মক হলে "Cannot be negative."।

ফাইল: `lib/features/recruitment/recruitment_kpi_screen.dart`

সংক্ষিপ্ত ববিরণ: নযিোগ প্রক্রিয়ার KPI/রপিোর্ট স্ক্রনি — খোলা পদ, প্রার্থী, আবদেন, নযিোগের সংখ্যা, বিভিন্ন হারের মটেরকি এবং ফানলে/সোর্স ব্রকেডাউন দেখায়। প্রতি-জব ও প্রতি-রিক্রুটার তুলনা টবেলি ইচ্ছাকৃতভাবে বাদ দেওয়া হয়েছে কারণ সেগুলো ফোনে কার্যকর নয়।

টপ-লভেলে প্রোভাইডার

final recruitmentKpisProvider

FutureProvider.autoDispose<RecruitmentKpis> — recruitmentRepositoryProvider.kpis() কল করে ডটো লোড করে।

ক্লাস/উইজটে: RecruitmentKpiScreen

ConsumerWidget, KPI স্ক্রিনিং মূল স্ক্র্যাফোল্ড।

static void open(BuildContext context)

MaterialPageRoute দিয়ে RecruitmentKpiScreen পুশ করে নেভিগেটে করে।

Widget build(BuildContext context, WidgetRef ref)

recruitmentKpisProvider ওয়াচ করে; AppBar ("Hiring") ও RefreshIndicator সহ Scaffold তৈরিকরে। লোডিং/এরর/ডটো স্টেটে অনুযায়ী Loader, ErrorState, বা _Body দেখায়। রিফ্রেশে প্রোভাইডার invalidate করে।

ক্লাস/উইজটে: _Body

KPI ডটোর মূল বডি লিআউট — kpis অবজেক্ট নিয়ে।

Widget build(BuildContext context)

ListView এ স্ট্যাট কার্ড (open positions, hired this month/total, candidates, applications), "How long it takes" সেকশনে টাইম-টু-হায়ার/ফলি মটেরিকি, "Conversion" সেকশনে বিভিন্ন হার, এবং ফানলে ও সোর্স ব্রকেডাউন থাকলে _Bars উইজটে দিয়ে বার চার্ট সাজায়।

static String? _rate(double? value)

value null হলে null রিটার্ন করে, না হলে Fmt.percent দিয়ে ফরম্যাট করে। ব্যাকএন্ড এই মানগুলো ইতিমধ্যে শতাংশ আকারে পাঠায়, ভগ্নাংশ নয়।

ক্লাস/উইজটে: _Metric

একটিলবেলে-মান জোড়া সারি হিসাবে দেখায়; value null হলে ড্যাশ (—) দেখায় (অর্থাৎ মান অজানা, শূন্য নয়)।

Widget build(BuildContext context)

লবেলে বামে, মান ডানে — Row এ সাজায়; null মান হলে মডিউটে রঙে দেখায়।

ক্লাস/উইজটে: _Bars

KpiCount লিস্ট থেকে অনুভূমিক বার চার্ট আঁকে, প্রতিটিবার সর্বোচ্চ মানের (peak) সাপেক্ষে আঁকা হয় — কারণ ফানলের গুরুত্বপূর্ণ বিষয় হলো ধাপগুলোর মধ্যে তুলনা, নরিঙ্কুশ সংখ্যা নয়।

Widget build(BuildContext context)

প্রতিটি সারিতে লবেলে, সংখ্যা এবং LinearProgressIndicator দিয়ে আনুপাতিকি বার আঁকে (peak শূন্য হলে ০ ভ্যালু ব্যবহার করে বিভাজন এড়ায়)।

ফাইল: lib/features/recruitment/recruitment_models.dart

সংক্ষিপ্ত ববিরণ: নযি়োগ (recruitment) মডউিলরে সব ডোমইন মডলে, স্ট্য়াটাস কনস্ট্য়ান্ট, পারমশিন কোড এবং রকি়োস্ট DTO। জব পোস্টিং, অ্যাপ্লিকেশন, ইন্টারভিউ, অফার, ক্যান্ডিডেট, ট্যালেন্ট পুল ও KPI ডটোর গঠন এখানে সংজ্ঞায়িত।

ক্লাস: RecruitmentPermissions

abstract final class — নযি়োগ মডউিলরে পারমশিন কোডগুলো ধুবক হিসেবে রাখবে, ব্যাকএন্ড সার্ভিসরে নাম অনুযায়ী নামকরণ করা।

স্ট্য়াটিক ফিল্ডসমূহ: **reportView**, **jobView**, **jobUpdate** (publish/close/reassign সব একই পারমশিন), **applicationView**, **applicationCreate** (রেফারলে/নতুন অ্যাপ্লিকেশন তৈরি), **applicationUpdate** (স্ট্য়াটাস পরিবর্তন, স্কোরিং, ইন্টারভিউ/অফার কার্যক্রম), **jobCreate**, **applicationDelete** (অ্যাপ্লিকেশন ও ক্যান্ডিডেট মোছার জন্য একই কোড), **hire** (**EMPLOYEE_CREATE**, অ্যাপে এই অ্যাকশন নই কনিতু নামকরণ করা আছে)।

টপ-লেভেলে কনস্ট্য়ান্ট

const interviewRounds

ইন্টারভিউ রাউন্ডরে ক্রম: **SCREENING**, **TECHNICAL**, **HR**, **FINAL**।

const interviewModes

ইন্টারভিউ হওয়ার মাধ্যম: **VIDEO** (ব্যাকএন্ডরে ডিফল্ট), **ONSITE**, **PHONE**।

const applicationSources

প্রার্থী কোথা থেকে এসছে তার তালিকা — **CAREER_PAGE**, **LINKEDIN**, **FACEBOOK**, **JOB_PORTAL**, **EMPLOYEE_REFERRAL**, **AGENCY**, **DIRECT**, **OTHER**।

ক্লাস: JobPostingStatus

abstract final class — পোস্টিং স্ট্য়াটাসরে ধুবক: **draft**, **open**, **closed**, **onHold**।

ক্লাস: ApplicationStatus

abstract final class — অ্যাপ্লিকেশন পাইপলাইনরে সব স্ট্য়াটাস ধুবক (**applied** থেকে **hired/rejected/withdrawn** পর্যন্ত), সাথে **pipeline** (ফিল্টার ও পরবর্তী ধাপ হিসেবে অফার করা ক্রম) ও **terminal** (**hired**, **rejected**, **withdrawn**, **offerRejected** — যোগেলোর জন্য কডে আর অপেক্ষা করছে না) সটে।

ক্লাস: JobPosting

একটি জব পোস্টিং প্রতিনিধিত্ব করে — **id**, **title**, **status**, **vacancies** এবং একগুচ্ছ ঐচ্ছকি ফিল্ড (**jobTitle**, **location**, **employmentType**, **departmentName**, **deadline**, **remote**, **assignedRecruiterName**, **description**, **requirements**, **responsibilities**, **salaryMin/Max**)।

bool get isDraft

status == JobPostingStatus.draft কনি যাচাই করে।

bool get isOpen

status == JobPostingStatus.open কনি যাচাই করে।

`bool get isClosed`

`status == JobPostingStatus.closed` কনি যাচাই করে।

`bool get canPublish`

ড্রাফট বা on-hold অবস্থায় থাকলে `true` — এগুলো থেকেই প্রকাশ করা যায়।

`bool get canClose`

বন্ধ না থাকলে `true (!isClosed)`।

`bool get deadlinePassed`

পোস্টিং খোলা থাকা অবস্থায় তার ডেডলাইন পার হয়ে গেছে কনি যাচাই করে — অর্থাৎ ক্যারিয়ার পজে লাইভ আছে কনিতু থাকা উচিত না এমন অবস্থা শনাক্ত করে।

`String? get whereLabel`

রিমোট হলে "Remote" বা "Remote · location" রিটার্ন করে, না হলে শুধু location রিটার্ন করে।

`factory JobPosting.fromJson(Map<String, dynamic> json)`

JSON থেকে `JobPosting` তৈরি করে। title না থাকলে `jobTitle` বা "Untitled posting" ফলব্যাক ব্যবহার করে। সব সংখ্যাসূচক ফিল্ড null-safe কনভার্সন দিয়ে পার্স করে।

ক্লাস: JobApplication

একটি নির্দিষ্ট জবের বিপরীতে একজন প্রার্থীর আবেদন প্রতিনিধিত্ব করে — id, status, candidate তথ্য, jobPosting তথ্য, source, notes, resume/linkedin/portfolio URL, পাঁচটি ম্যানুয়াল স্কোর, সারভার-সাইড গড় করা `overallScore` এবং CV পার্সিং থেকে আসা `atsScore`।

`String get personLabel`

candidateName থাকলে ট্রিম করে দেখায়, না হলে "Candidate #id" ফরম্যাটে দেখায়।

`bool get isTerminal`

`ApplicationStatus.terminal` স্টেটে status আছে কনি যাচাই করে।

`bool get isHired`

`status == ApplicationStatus.hired` কনি যাচাই করে।

`bool get hasScores`

পাঁচটি স্কোরের যেকোনো একটি non-null হলে `true`।

`List<String> get nextStatuses`

পরবর্তী সম্ভাব্য স্ট্যাটাসের তালিকা রিটার্ন করে — terminal হলে খালি লিস্ট, না হলে বর্তমান স্ট্যাটাস বাদে পাইপলাইনে বাকি স্ট্যাটাস প্লাস rejected ও withdrawn।

`bool get canPoolCandidate`

স্ট্যাটাস rejected, withdrawn বা offerRejected হলে `true` — `TalentPoolController.fromApplication`-এর একই যাচাই মেলোয়।

`factory JobApplication.fromJson(Map<String, dynamic> json)`

JSON থেকে `JobApplication` তৈরি করে, সব ফিল্ড null-safe পার্সিং দিয়ে।

ক্লাস: InterviewStatus

abstract final class — scheduled, completed, cancelled, noShow ধ্রুবক।

ক্লাস: Interview

একটি নির্ধারিত ইন্টারভিউ প্রতিনিধিত্ব করে — id, jobId, status, প্রার্থীর নাম, jobTitle, round, scheduledAt, durationMinutes, mode, meetingLink, interviewerName, rating, strengths, concerns, recommendation।

bool get isScheduled

status == InterviewStatus.scheduled কনি যাচাই করে।

bool get hasFeedback

rating বা recommendation non-null হলে true।

DateTime? get when

scheduledAt কে DateTime-এ পার্স করে (ব্যর্থ হলে null)।

bool get awaitingFeedback

এখনো scheduled অবস্থায় আছে কিন্ত সময় পার হয়ে গেছে কনি যাচাই করে — অর্থাৎ কটে ফিডব্যাক দিতে বাকি আছে।

bool get isToday

ইন্টারভিউর তারিখ আজকের তারিখের সাথে (বেছর, মাস, দিন) মিলে কনি যাচাই করে।

factory Interview.fromJson(Map<String, dynamic> json)

JSON থেকে Interview তৈরি করে, সব ফিল্ড null-safe পার্সিং দিয়ে।

ক্লাস: OfferStatus

abstract final class — draft, sent, accepted, declined, withdrawn ধ্রুবক।

ক্লাস: JobOffer

একটি নিয়োগ অফার প্রতিনিধিত্ব করে — id, jobId, status, expired (সাবতার-সাইড গণনা করা), applicantName, jobPostingTitle, offeredJobTitle, joiningDate, expiryDate, grossSalary, sentAt, decidedAt, declineReason।

bool get isDraft

status == OfferStatus.draft কনি যাচাই করে।

bool get isSent

status == OfferStatus.sent কনি যাচাই করে।

bool get isPending

sent ও এখনো expired না হলে true — উত্তরে অপেক্ষায় আছে এমন অফার।

bool get isSettled

status accepted, declined বা withdrawn হলে true।

String get displayStatus

expired ও sent হলে "EXPIRED" রিটার্ন করে, না হলে সাধারণ status রিটার্ন করে — কারণ শুধু stored status দেখালে expired অফার "Sent" হিসেবেই দেখাত।

factory `JobOffer.fromJson(Map<String, dynamic> json)`

JSON থেকে `JobOffer` তৈরি করে, সব ফিল্ড null-safe পার্সিং দিয়ে।

ক্লাস: Candidate

একজন ব্যক্তি, যেকোনো একটি নির্দিষ্ট অ্যাপ্লিকেশন থেকে স্বাধীন — id, name, applicationCount, email, phone, resumeUrl, linkedInUrl, portfolioUrl, currentTitle, skills, source।

factory `Candidate.fromJson(Map<String, dynamic> json)`

JSON থেকে `Candidate` তৈরি করে; name না থাকলে "Unnamed candidate" ব্যবহার করে।

ক্লাস: TalentPoolReason

abstract final class — ট্যালেন্ট পুলে যুক্ত হওয়ার কারণের ধরবক (`futureFit`, `declinedOffer`, `noVacancy`, `withdrew`, `referral`, `other`) এবং all তালিকা যা প্রচলিত ব্যবহার অনুযায়ী সাজানো।

টপ-লেভেলে কনস্ট্যান্ট

const `talentPoolReasons`

`TalentPoolReason.all`-এর শর্টহ্যান্ড।

ক্লাস: TalentPoolCandidate

ভবিষ্যতের জন্য "পার্ক করা" একজন প্রার্থী — id, name, email, phone, resumeUrl, linkedInUrl, desiredRole, skills, rating, reason, notes, sourceJobTitle। `sourceJobTitle` শুধু তখনই সেট হয় যখন এটি "add from application" অ্যাকশনের মাধ্যমে এসেছে।

factory `TalentPoolCandidate.fromJson(Map<String, dynamic> json)`

JSON থেকে `TalentPoolCandidate` তৈরি করে; name না থাকলে "Unnamed candidate" ব্যবহার করে।

ক্লাস: JobPostingRequest

POST `/recruitment/jobs`-এর বডি — title, jobTitle, location, employmentType, vacancies, salaryMin/Max, deadline, remote, description, requirements, responsibilities ধারণ করে।

`Map<String, dynamic> toJson()`

একটি লোকাল `clean` হলে পার দিয়ে স্ট্রিং ট্রিম করে খালি হলে null বানায়। title ও remote সবসময় পাঠায় (remote explicit ভাবে পাঠানো হয় কারণ "not remote" একটি বাস্তব উত্তর, অনুপস্থিতি নয়); বাকি ফিল্ড শুধু non-null/non-empty হলে পাঠায়।

ক্লাস: InterviewRequest

POST `/recruitment/interviews`-এর বডি — jobId, scheduledAt (আবশ্যিক, zone ছাড়া wall-clock স্ট্রিং), interviewerId (আবশ্যিক), round, durationMinutes, mode, meetingLink ধারণ করে।

`Map<String, dynamic> toJson()`

আবশ্যিক তিনটি ফিল্ড সবসময় পাঠায়; round/durationMinutes/mode non-null হলে পাঠায়; meetingLink ট্রিম করে খালি হলে পাঠায়।

ক্লাস: OfferRequest

POST /recruitment/offers-এর বডি— jobId, offeredJobTitle, expiryDate, grossSalary (তিনিটাই আবশ্যিক, যদিও DTO তা বলে না — কন্ট্রোলার ম্যানুয়ালি যাচাই করে), joiningDate, basicSalary, houseRent, medicalAllowance, transportAllowance, notes ধারণ করে।

Map<String, dynamic> toJson()

আবশ্যিক চারটি ফিল্ড সবসময় পাঠায় (offeredJobTitle ট্রিম করা); বাকি ফিল্ড non-null হলে পাঠায়, notes ট্রিম করে খালি না হলে পাঠায়।

ক্লাস: CandidateRequest

PUT /recruitment/candidates/{id}-এর বডি— name (আবশ্যিক), email, phone, currentTitle, skills, source, resumeUrl, linkedInUrl, portfolioUrl, notes। প্রায় সব ফিল্ড আনকন্ডিশনালি স্টে হয় সার্ভার-সাইডে (name/email/source ছাড়া), তাই খালি করলে ফিল্ড ক্লিয়ার হয়ে যায়।

factory CandidateRequest.from(Candidate candidate)

একটি বিদ্যমান Candidate থেকে রকিওয়েস্ট তৈরি করে, এডটি ফর্ম সডি করার জন্য।

Map<String, dynamic> toJson()

name সবসময় পাঠায়; email/source শুধু non-null হলে পাঠায় (গার্ডেডে সার্ভার-সাইড); phone, currentTitle, skills, resumeUrl, linkedInUrl, portfolioUrl, notes আনকন্ডিশনালি পাঠায় (trim করে, খালি হলে null) — কারণ সার্ভার এগুলো unconditionally assign করে।

ক্লাস: TalentPoolRequest

POST/PUT /recruitment/talent-pool-এর বডি— name, email (দুটোই আবশ্যিক), phone, resumeUrl, linkedInUrl, desiredRole, skills, rating, reason, notes। CandidateRequest-এর থেকে আলাদা shape যদিও ব্যাকএন্ডে দুটোরই ক্লাস নাম CandidateRequest।

factory TalentPoolRequest.from(TalentPoolCandidate entry)

বিদ্যমান TalentPoolCandidate থেকে রকিওয়েস্ট তৈরি করে, এডটি ফর্ম সডি করার জন্য (email null হলে খালি স্ট্রিং ব্যবহার করে)।

Map<String, dynamic> toJson()

name ও email সবসময় ট্রিম করে পাঠায়; reason non-null হলে পাঠায়; phone, resumeUrl, linkedInUrl, desiredRole, skills, rating, notes আনকন্ডিশনালি পাঠায় (apply) এগুলো unconditionally assign করে বলে।

ক্লাস: RecruitmentKpis

GET /recruitment/kpis-এর রসেপন্স — openPositions, totalCandidates, totalApplications, hiresThisMonth, hiresTotal (আবশ্যিক), এবং ঐচ্ছিক rate/average ফিল্ড (avgTimeToHireDays, avgTimeToFillDays, applicationToInterviewRate, interviewToHireRate, offerAcceptanceRate, avgAtsMatchScore), এবং funnel/sourceBreakdown (KpiCount লিস্ট)। প্রতিটি rate nullable — null মানে "অজানা", শূন্য নয়।

factory RecruitmentKpis.fromJson(Map<String, dynamic> json)

JSON থেকে RecruitmentKpis তৈরি করে। ভেতরে একটি লোকাল counts হলেপার ফাংশন সংজ্ঞায়িত করে যা একটি key ও labelField নিয়ে KpiCount লিস্ট তৈরি করে funnel ও sourceBreakdown উভয়ই জন্য পুনর্ব্যবহার করা হয়।

ক্লাস: KpiCount

একটি লিবেলেযুক্ত সংখ্যা — funnel ও source breakdown-এর জন্য ব্যবহৃত। ফিল্ড: `label`, `count`।

ক্লাস: JobApplicationRequest

স্টাফ-সাইড থেকে একটি জব প্রার্থীকে পাইপলাইনে যুক্ত করার (রেফারলে বা ইমইলে আসা CV) রকিওয়েস্ট — `applicantName` (আবশ্যিক), `applicantEmail`, `applicantPhone`, `resumeUrl` (আপলোড থেকে আসা URL, CV নজিরে নয়), `linkedinUrl`, `portfolioUrl`, `coverLetter`, `source`, `referredByEmployeeId` (সারভার-সাইড কলারের নজিস্ব `employee id` থেকে সেট হয়, অন্য কারো নামে দেওয়া যায় না)।

`Map<String, dynamic> toJson()`

`applicantName` সবসময় ট্রিম করে পাঠায়; বাকি সব ফিল্ড ট্রিম করে non-empty হলে (বা non-null হলে `source/referredByEmployeeId`-এর ক্ষেত্রে) পাঠায়।

ফাইল: lib/features/recruitment/recruitment_repository.dart

সংক্ষিপ্ত বিবরণ: নথিগত মডেলের সব API কল (postings, applications, interviews, offers, candidates, talent pool) ধারণকারী রপিজিটরিক্লাস, এবং সংশ্লিষ্ট Riverpod প্রোভাইডার ও কন্ট্রোলার।
হায়ারিং (hire), ক্যারিয়ার পজে এবং রিপোর্ট টবেলি ইচ্ছাকৃতভাবে অনুপস্থিতি।

ক্লাস: RecruitmentRepository

`ApiClient` মুড়িয়ে নথিগত-সম্পর্কিত সব HTTP এন্ডপয়েন্ট কল করে।

`RecruitmentRepository(this._api)`

কনস্ট্রাক্টর, `ApiClient` ইনজেক্ট করে।

`Future<PagedResponse<JobPosting>> jobs({int page = 0, int size = 20})`

পজেনিটেডে জব পোস্টিং লিস্ট আনে `/recruitment/jobs` থেকে।

`Future<List<JobPosting>> openJobs()`

`/recruitment/jobs/open` থেকে শুধু বর্তমানে বজ্র্গাপতি জবের তালিকা আনে — এটি একটি আলাদা এন্ডপয়েন্ট, তালিকার উপর ফিল্টার নয়, যাত "খোলা" এর সংজ্ঞা ব্যাকএন্ডেই থাকে এবং দুই জায়গায় দ্বিমিত না হয়।

`Future<JobApplication> apply(int jobPostingId, JobApplicationRequest request)`

`$_jobs/$jobPostingId/apply`-এ POST করে একটি অ্যাপ্লিকেশন রেকর্ড করে (স্টাফ-সাইড রেফারলে/ইমইল CV)।

`Future<JobPosting> createJob(JobPostingRequest request)`

`/recruitment/jobs`-এ POST করে নতুন পোস্টিং তৈরি করে (DRAFT হিসেবে আসে)।

`Future<JobPosting> updateJob(int id, JobPostingRequest request)`

`$_jobs/$id`-এ PUT করে পোস্টিং সম্পাদনা করে।

`Future<JobPosting> assignRecruiter(int id, int recruiterId)`

`$_jobs/$id/assign-recruiter`-এ PATCH করে, `recruiterId` বডিতে পাঠায় (পাথ বা কোয়েরি প্যারামিটার নয়)।

Future<Interview> rescheduleInterview(int id, InterviewRequest request)

\$_interviews/\$id-এ PUT করে একটি ইন্টারভিউ পুনঃনির্ধারণ করে — শিডিউল করার একই DTO ব্যবহার করে।

Future<RecruitmentKpis> kpis({String? from, String? to})

/recruitment/kpis-এ GET করে, ঐচ্ছকি from/to তারিখ কোয়রে প্যারামিটার হিসেবে পাঠায়; না দিলে ব্যাকএন্ড নিজস্ব উইন্ডো বেছে নেয়।

Future<JobPosting> publishJob(int id)

\$_jobs/\$id/publish-এ PATCH করে পোস্টিং প্রকাশ করে।

Future<JobPosting> closeJob(int id)

\$_jobs/\$id/close-এ PATCH করে পোস্টিং বন্ধ করে।

Future<PagedResponse<JobApplication>> applications({String? status, int? jobPostingId, int page = 0, int size = 20})

পজেনিটেডে অ্যাপ্লিকেশন লিস্ট আনবে। jobPostingId দেওয়া থাকলে \$_jobs/\$jobPostingId/applications এন্ডপয়েন্ট ব্যবহার করে, না হলে \$_base/applications — দুটো ভিন্ন এন্ডপয়েন্ট, একটর উপর ফিল্টার নয়।

Future<JobApplication> application(int id)

\$_base/applications/\$id-এ GET করে একটি নির্দিষ্ট অ্যাপ্লিকেশন আনবে।

Future<JobApplication> setApplicationStatus(int id, String status)

\$_base/applications/\$id/status-এ PATCH করে অ্যাপ্লিকেশনের স্ট্যাটাস পরিবর্তন করে।

Future<JobApplication> evaluate(int id, {int? education, int? experience, int? technicalSkills, int? interview, int? communication})

\$_base/applications/\$id/evaluate-এ PATCH করে ঐচ্ছকি স্কোরগুলো পাঠায়; ব্যাকএন্ড উপস্থিতি স্কোরগুলোর গড় করে overallScore গণনা করে।

Future<PagedResponse<Interview>> interviews({String? status, int page = 0, int size = 20})

পজেনিটেডে ইন্টারভিউ লিস্ট আনবে /recruitment/interviews থেকে। status দিলে ascending (আসন্নগুলো আগে), না দিলে descending (নতুনগুলো আগে) সাজানো — এই আচরণ রপিজিটরিতেই সংজ্ঞায়িত।

Future<Interview> scheduleInterview(InterviewRequest request)

/recruitment/interviews-এ POST করে ইন্টারভিউ বুক করে। ইতিমধ্যে hired/rejected/withdrawn অ্যাপ্লিকেশনের জন্য প্রত্যাখ্যাত হয়; প্রথম রাউন্ড শিডিউল করলে অ্যাপ্লিকেশন INTERVIEW_SCHEDULED-এ চলে যায়।

Future<List<Interview>> interviewsForApplication(int applicationId)

\$_interviews/application/\$applicationId-এ GET করে একটি অ্যাপ্লিকেশনের সব ইন্টারভিউ আনবে।

Future<Interview> submitFeedback(int id, {int? rating, String? strengths, String? concerns, String? recommendation, bool noShow = false})

\$_interviews/\$id/feedback-এ PATCH করে ফিডব্যাক জমা দেয়; strengths/concerns ট্রিম করে non-empty হলেই পাঠায়।

Future<Interview> cancelInterview(int id)

\$_interviews/\$id/cancel-এ PATCH করে ইন্টারভিউ বাতলি করে।

Future<List<JobOffer>> offersForApplication(int applicationId)

\$_offers/application/\$applicationId-এ GET করে একটি অ্যাপ্লিকেশনের সব অফার আনবে।

Future<JobOffer> offerTransition(int id, String action, {String? reason})

\$_offers/\$id/\$action-এ PATCH করে অফারের স্ট্যাটাস ট্রানজিশন করায় (send, accept, decline ইত্যাদি অ্যাকশন প্যারামিটার দ্বিধা), ঐচ্ছিক reason ট্রান্সমিট করে পাঠায়।

Future<JobOffer> createOffer(OfferRequest request)

/recruitment/offers-এ POST করে অফার তৈরি করে (unsent অবস্থায়, DRAFT); সৃষ্টি করলে অ্যাপ্লিকেশন OFFER_PENDING-এ যায়। ইতিমধ্যে live অফার থাকলে প্রতিরোধিত হয়।

Future<PagedResponse<Candidate>> candidates({String? q, int page = 0, int size = 20})

পজেনিটেড ক্যান্ডিডেট লিস্ট আনবে /recruitment/candidates থেকে, q সার্চ ক্রয়ের হিসাবে পাঠায়।

Future<Candidate> candidate(int id)

\$_candidates/\$id-এ GET করে একটি নির্দিষ্ট ক্যান্ডিডেট আনবে।

Future<List<JobApplication>> candidateApplications(int id)

\$_candidates/\$id/applications-এ GET করে একজন ক্যান্ডিডেটের সব অ্যাপ্লিকেশন আনবে।

Future<Candidate> updateCandidate(int id, CandidateRequest request)

\$_candidates/\$id-এ PUT করে ক্যান্ডিডেট আপডেট করে।

Future<void> deleteCandidate(int id)

\$_candidates/\$id-এ DELETE করে ক্যান্ডিডেট মুছে দেয় — APPLICATION_DELETE পারমিশন প্রয়োজন, যা এডিট করার APPLICATION_UPDATE থেকে ভিন্ন।

Future<void> deleteApplication(int id)

\$_base/applications/\$id-এ deleteText (সাধারণ delete নয়) কল করে মুছে দেয় — কারণ এন্ডপয়েন্ট ResponseEntity<String> রিটার্ন করে, JSON নয়।

Future<PagedResponse<TalentPoolCandidate>> talentPool({String? keyword, int page = 0, int size = 20})

পজেনিটেড ট্যালেন্ট পুল লিস্ট আনবে /recruitment/talent-pool থেকে, keyword সার্চ প্যারামিটার হিসাবে পাঠায়।

Future<TalentPoolCandidate> poolApplication(int applicationId, {String? reason, String? notes})

\$_talentPool/from-application/\$applicationId-এ POST করে একটি বিদ্যমান অ্যাপ্লিকেশনকে এক ক্লিকে ট্যালেন্ট পুলে যুক্ত করে।

Future<TalentPoolCandidate> addToTalentPool(TalentPoolRequest request)

/recruitment/talent-pool-এ POST করে হাতে-তৈরি একটি এন্ট্রি যুক্ত করে।

Future<TalentPoolCandidate> updateTalentPoolEntry(int id, TalentPoolRequest request)

\$_talentPool/\$id-এ PUT করে বিদ্যমান ট্যালেন্ট পুল এন্ট্রি সম্পাদনা করে।

Future<void> removeFromTalentPool(int id)
\$_talentPool/\$id-এ DELETE করে এন্ট্রি সরিয়ে দেয়।

টপ-লেভেলে প্রোভাইডার

final recruitmentRepositoryProvider

Provider<RecruitmentRepository> — apiClientProvider থেকে ApiClient নিয়ে RecruitmentRepository তৈরি করে।

ক্লাস: JobsController

AsyncNotifier<PagedState<JobPosting>> with PagedLoader<JobPosting> — জব পোস্টিং লিস্টের পজেনিটেডে স্টেটে পরিচালনা করে।

Future<PagedState<JobPosting>> build()

currentUserProvider ওয়াচ করে (ইউজার পাল্টালে রিফ্রেশে), প্রথম পজে লোড করে।

Future<PagedResponse<JobPosting>> fetchPage(int page)

recruitmentRepositoryProvider.jobs(page: page) কল করে একটি পজে আনবে।

void apply(JobPosting updated)

id মিলিয়ে লিস্টে আপডেটেড পোস্টিং প্রতিস্থাপন করে।

Future<JobPosting> create(JobPostingRequest request)

নতুন পোস্টিং তৈরি করে এবং পুরো লিস্ট রিফ্রেশে করে (ইনসার্ট নয়, কারণ নতুন পোস্টিং কোথায় বসবে তা সার্ভার ঠিকি করে)।

Future<JobPosting> updateItem(int id, JobPostingRequest request)

পোস্টিং আপডেট করে এবং apply দিয়ে জায়গায় প্রতিস্থাপন করে (পজিশন পরিবর্তন হয় না)।

Future<JobPosting> assignRecruiter(int id, int recruiterId)

পোস্টিং-এ রক্‌রুটার এসাইন করে এবং apply দিয়ে লিস্ট আপডেট করে।

টপ-লেভেলে প্রোভাইডার

final jobsProvider

AsyncNotifierProvider<JobsController, PagedState<JobPosting>> |

ক্লাস: ApplicationFilterController

Notifier<String?> — অ্যাপ্লিকেশন লিস্ট কোন পাইপলাইন স্টেজে সংকীর্ণ করা আছে তা রাখবে; null মানে সব।

String? build()

প্রাথমিক স্টেটে null রিটার্ন করে।

void set(String? status)

স্টেটে পরিবর্তন করে, অপরিবর্তিত হলে কিছু করে না।

উপ-লভেলে প্রোভাইডার

final applicationFilterProvider

NotifierProvider<ApplicationFilterController, String?> |

ক্লাস: ApplicationsController

AsyncNotifier<PagedState<JobApplication>> with PagedLoader<JobApplication> — অ্যাপ্লিকেশন লিস্টের পজেনিটেডে স্টেটে পরিচালনা করে।

Future<PagedState<JobApplication>> build()

currentUserProvider ও applicationFilterProvider ওয়াচ করে, প্রথম পজে লোড করে।

Future<PagedResponse<JobApplication>> fetchPage(int page)

বর্তমান ফিল্টার সহ recruitmentRepositoryProvider.applications কল করে।

void apply(JobApplication updated)

ফিল্টার সটে থাকলে ও আপডেটেডে স্ট্যাটাস ফিল্টারের সাথে না মিললে লিস্ট থেকে সরিয়ে দিয়ে (কারণ স্ট্যাটাস পরিবর্তন সারটিকি দেখা স্টেজে থেকে বাইরে নিয়ে যেতে পারে); না হলে জায়গায় প্রতিস্থাপন করে।

উপ-লভেলে প্রোভাইডার

final applicationsProvider

AsyncNotifierProvider<ApplicationsController, PagedState<JobApplication>> |

ক্লাস: UpcomingInterviewsController

AsyncNotifier<PagedState<Interview>> with PagedLoader<Interview> — এখনো ঘটনোত্রিমন ইন্টারভিউ (soonest first) পরিচালনা করে।

Future<PagedState<Interview>> build()

currentUserProvider ওয়াচ করে, প্রথম পজে লোড করে।

Future<PagedResponse<Interview>> fetchPage(int page)

status: InterviewStatus.scheduled দিয়ে recruitmentRepositoryProvider.interviews কল করে।

void apply(Interview updated)

আপডেটেডের স্ট্যাটাস আর scheduled না থাকলে লিস্ট থেকে সরিয়ে দিয়ে (ফডিব্যাক/বাতলি উভয়ই SCHEDULED থেকে সরিয়ে দিয়ে); না হলে জায়গায় প্রতিস্থাপন করে।

উপ-লভেলে প্রোভাইডার

final upcomingInterviewsProvider

AsyncNotifierProvider<UpcomingInterviewsController, PagedState<Interview>> |

final applicationInterviewsProvider

FutureProvider.autoDispose.family<List<Interview>, int> — নির্দিষ্ট applicationId-এর ইন্টারভিউ তালিকা আনবে।

final applicationOffersProvider

FutureProvider.autoDispose.family<List<JobOffer>, int> — নর্দিশ্টি applicationId-এর অফার তালিকা আনবে।

ক্লাস: CandidateSearchController

Notifier<String?> — ক্যান্ডিডেট সার্চ কীওয়ার্ড রাখে।

String? build()

প্রাথমিক স্টেটে null।

void set(String? keyword)

কীওয়ার্ড ট্রিম করে, খালি হলে null বানায়; অপরিবর্তিত হলে কিছু করে না।

টপ-লেভেলে প্রোভাইডার

final candidateSearchProvider

NotifierProvider<CandidateSearchController, String?>।

ক্লাস: CandidatesController

AsyncNotifier<PagedState<Candidate>> with PagedLoader<Candidate> — ক্যান্ডিডেট লিস্টের পজেনিটেডে স্টেটে পরিচালনা করে।

Future<PagedState<Candidate>> build()

currentUserProvider ও candidateSearchProvider ওয়াচ করে, প্রথম পজে লোড করে।

Future<PagedResponse<Candidate>> fetchPage(int page)

বর্তমান সার্চ কীওয়ার্ড দিয়ে recruitmentRepositoryProvider.candidates কল করে।

Future<Candidate> updateItem(int id, CandidateRequest request)

ক্যান্ডিডেট আপডেট করে, লিস্টে প্রতিস্থাপন করে এবং সার্ভারের আপডেটেড ভার্সন রিটার্ন করে — যাত্রে ডিটাইল স্ক্রিনি (যা প্রোভাইডার ওয়াচ না করে নিজস্ব স্টেটে ফেচ করে) দ্বিতীয় রাউন্ড ট্রপি ছাড়াই তা গ্রহণ করতে পারে।

Future<void> delete(int id)

ক্যান্ডিডেট মুছে দেয় এবং লিস্ট থেকে সরিয়ে দেয়।

টপ-লেভেলে প্রোভাইডার

final candidatesProvider

AsyncNotifierProvider<CandidatesController, PagedState<Candidate>>।

final candidateApplicationsProvider

FutureProvider.autoDispose.family<List<JobApplication>, int> — নর্দিশ্টি candidateId-এর সব অ্যাপ্লিকেশন আনবে।

ক্লাস: TalentPoolSearchController

Notifier<String?> — ট্যালেন্ট পুল সার্চ কীওয়ার্ড রাখে।

String? build()
প্রাথমিক স্টেটে null।

void set(String? keyword)
কীওয়ার্ড ট্রিম করে, খালি হলে null বানায়; অপরিবর্তিত হলে কিছু করে না।

উপ-লভেলে প্রোভাইডার

final talentPoolSearchProvider
NotifierProvider<TalentPoolSearchController, String?>।

ক্লাস: TalentPoolListController

AsyncNotifier<PagedState<TalentPoolCandidate>> with PagedLoader<TalentPoolCandidate> —
ট্যালেন্ট পুল লিস্টের পেজিনেটেড স্টেটে পরিচালনা করে।

Future<PagedState<TalentPoolCandidate>> build()
currentUserProvider ও talentPoolSearchProvider ওয়াচ করে, প্রথম পেজ লোড করে।

Future<PagedResponse<TalentPoolCandidate>> fetchPage(int page)
বর্তমান কীওয়ার্ড দিয়ে recruitmentRepositoryProvider.talentPool কল করে।

void removeCandidate(int id)
id মিলিয়ে লিস্ট থেকে একটি এন্ট্রি সরিয়ে দেয়।

Future<TalentPoolCandidate> add(TalentPoolRequest request)
নতুন এন্ট্রি যুক্ত করে এবং পুরো লিস্ট রিফ্রেশ করে।

Future<TalentPoolCandidate> updateItem(int id, TalentPoolRequest request)
বর্তমান এন্ট্রি আপডেট করে এবং লিস্টে প্রতিস্থাপন করে।

উপ-লভেলে প্রোভাইডার

final talentPoolProvider
AsyncNotifierProvider<TalentPoolListController, PagedState<TalentPoolCandidate>>।

final openJobsProvider
FutureProvider.autoDispose<List<JobPosting>> — বর্তমানে বজ্রপতি জবগুলোর তালিকা আনবে
recruitmentRepositoryProvider.openJobs() কল করে।

ফাইল: lib/features/recruitment/recruitment_screen.dart

সংক্ষিপ্ত বিবরণ: নিয়োগ (Hiring) মডুলের প্রধান স্ক্রিনি, যেখানে Jobs, Applicants, Interviews, Candidates এবং Talent pool — এই পাঁচটি ট্যাব রয়েছে। প্রতিটি ট্যাব আলাদা পারমিশনের উপর ভিত্তি করে দেখানো বা লুকানো হয়, এবং দুটি ট্যাবে ফ্লোটিং অ্যাকশন বাটনের মাধ্যমে নতুন কিছু তৈরি সুযোগ থাকে।

ক্লাস/উইজটে: RecruitmentScreen

এটি একটি `ConsumerWidget`, যা `permissionControllerProvider` দেখে কোন কোন ট্যাব দেখানো হবে তা নির্ধারণ করে। Job postings এবং applications-এর পারমিশন আলাদা, তাই কটে শুধু postings ম্যানেজ করতে পারে আবার কটে শুধু candidates রিভিউ করতে পারে। Interviews, Candidates এবং Talent pool — এই তিনটি ট্যাব applications পারমিশনের সাথে গটে করা, কারণ এগুলো একই পাইপলাইনের ভিন্ন ভিউ। যদিকোনো ট্যাব দেখানোর অনুমতি না থাকে তাহলে একটি `EmptyState` দেখানো হয়। `AppBar`-এ Careers page ও KPI স্ক্রিনি যাওয়ার শর্টকাট আইকন আছে, এবং স্ক্রিনি ট্যাব অনুযায়ী FAB (Jobs-এ "New posting", Talent pool-এ "Add to pool") দেখা যায়।

Widget build(BuildContext context, WidgetRef ref)

পারমিশন লোড না হলে একটি `Loader` দেখায়। এরপর `canJobs` ও `canApplications` অনুযায়ী দৃশ্যমান ট্যাবের তালিকা (tabs) তৈরি করে; সবগুলো অনুমতি না থাকলে `EmptyState` রিটার্ন করে। `createOn` ম্যাপে ট্যাবের লেবেলে অনুযায়ী কোন ট্যাবে creation অ্যাকশন থাকবে তা নির্ধারণিত হয় (লেবেলে দিচ্ছে কী করা হয়, কারণ ট্যাবের ইনডেক্স ব্যবহারকারীভেদে পরিবর্তনশীল)। `initialTabLabel` থাকলে সেই ট্যাব থেকে শুরু হয়, নাহলে ০ নম্বর ইনডেক্স থেকে। `DefaultTabController` ও `AnimatedBuilder` দিয়ে স্ক্রিনি ট্যাব অনুযায়ী `AppBar`, `TabBar` এবং শর্তসাপেক্ষ `FloatingActionButton` সহ `Scaffold` তৈরি করে।

ক্লাস/উইজটে: _JobsTab

Job postings-এর পজেড লিস্ট দেখায়। `jobsProvider` থেকে ডটো নিয়ে `PagedListView<JobPosting>`-এর মাধ্যমে রিফ্রেশ ও লোড-মোর সাপোর্ট করে, প্রতিটি আইটেমের জন্য `_JobCard` ব্যবহার করে।

Widget build(BuildContext context, WidgetRef ref)

`ref.watch(jobsProvider)`-এর অ্যাসিঙ্ক স্টেটে, রিফ্রেশ/লোড-মোর কলব্যাক, খালি ও এরর অবস্থার মসেজে সহ `PagedListView` রিটার্ন করে।

ক্লাস/উইজটে: _JobCard

একটি `ConsumerStatefulWidget`, যা একটি single job posting-এর কার্ড দেখায় এবং স্টেটের উপর edit/assign recruiter/close/publish অ্যাকশন চালাতে পারে।

`ConsumerState<_JobCard> createState()`

`_JobCardState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: _JobCardState

`_JobCard`-এর স্টেটে, যা `_busy` ফ্ল্যাগ দিয়ে চলমান নটেওয়ার্ক অ্যাকশন ট্র্যাক করে এবং সাফল্য/ব্যর্থতার সন্যাকবার দেখায়।

`Future<void> _run(Future<JobPosting> Function() action, String success)`

একটি জিনেরেকি হলেপার — `_busy` স্টেট করে `action()` চালায়, সফল হলে `jobsProvider.notifier.apply()` দিয়ে আপডেটেড জব প্রয়োগ করে এবং সাফল্যের বার্তা দেখায়; `ApiException` বা অন্য এরর ধরা পড়লে যথাযথ বার্তা দেখায়; শেষে `_busy` false করে।

`Future<void> _assignRecruiter()`

`EmployeePicker.show()` দিয়ে একজন এমপ্লয়ি বাছাই করানো হয়, নির্বাচনিত হলে

`jobsProvider.notifier.assignRecruiter(jobId, personId)` কল করে রিক্রুটার নিয়োগ করা হয়। ব্যাকএন্ড আইডি বিডতি নেয় (পাথ সগেমেন্টে নয়) — এ কারণে রপিজটিরতি এটি রিপায় করা আছে। সফল/ব্যর্থ উভয় ক্ষেত্রেই সন্যাকবার দেখানো হয়।

Future<void> _close()

একটি নিশ্চিতকরণ ডায়ালগ দেখায় যা জানায় যে পোস্টিং বন্ধ হলে নতুন আবেদন নেওয়া বন্ধ হবে এবং careers page থেকে সরে যাবে, কিন্তু বর্তমান আবেদনগুলো অক্ষত থাকবে। নিশ্চিতি হলে `_run()`-এর মাধ্যমে `closeJob()` কল করে।

Widget build(BuildContext context)

`canUpdate` পারমিশন চেক করে `RowAction`-এর তালিকা (Edit, Assign/Change recruiter, Close) তৈরি করে, যা একটি `PopupMenuButton`-এ দেখানো হয়। কার্ডে জব-শিরোনাম, ডিপার্টমেন্ট, `StatusChip`, মতো তথ্য (location, employment type, vacancies), ডেডলাইন পরেয়ে গেলে সতর্কবার্তা, এবং `canPublish` হলে একটি Publish/Reopen বাটন দেখানো হয়।

ক্লাস/উইজটে: `_Meta`

আইকন ও টেক্সট নিয়ে একটি ছোট মটোডটো row (যেমন location, vacancy count) রেন্ডার করার জন্য ব্যবহৃত সাধারণ `StatelessWidget`।

Widget build(BuildContext context)

একটি আইকন ও ছোট টেক্সট সহ একটি `Row` রিটার্ন করে।

ক্লাস/উইজটে: `_ApplicationsTab`

আবেদনসমূহের তালিকা দেখায়, স্টজে ফিল্টার (`_StageFilter`) এবং পজেড লিস্ট (`_ApplicationRow`) সহ।

Widget build(BuildContext context, WidgetRef ref)

উপরে `_StageFilter` ও নিচে `PagedList<JobApplication>` সহ একটি `Column` রিটার্ন করে।

ক্লাস/উইজটে: `_StageFilter`

আবেদন-পাইপলাইনরে প্রতটি স্টজে অনুযায়ী অনুভূমিক ফিল্টার চপিরে তালিকা দেখায়, যাতো ব্যবহারকারী একটি নির্দিষ্ট স্টজে আবেদন ফিল্টার করতে পারে।

Widget build(BuildContext context, WidgetRef ref)

`applicationFilterProvider` থেকে বর্তমান নির্বাচতি স্টজে পড়ে, "All" সহ সব স্টজে `_FilterChip` একটি অনুভূমিক `ListView.separated`-এ দেখায়; ট্যাপ করলে একই স্টজে হলে ফিল্টার সরিয়ে দেয়, নাহলে সেটে করে।

ক্লাস/উইজটে: `_FilterChip`

একটি সিলিকেটবেল চপি, যা সক্রিয় থাকলে ব্র্যান্ড কালারে হাইলাইট হয়।

Widget build(BuildContext context)

`active` অনুযায়ী রং পরিবর্তনকারী একটি `InkWell`-মোড়ানো `Container` রিটার্ন করে।

ক্লাস/উইজটে: `_ApplicationRow`

একটি একক আবেদনের কার্ড, যা ট্যাপ করলে `ApplicationDetailScreen`-এ নিয়ে যায়।

Widget build(BuildContext context)

আবেদনকারীর নাম, জব টাইটলে, `StatusChip`, এবং থাকলে overall score ও ATS/CV ম্যাচ স্কোর দেখায়।

ক্লাস/উইজটে: _InterviewsTab

আসন্ন ইন্টারভিউয়ের পজেড লিস্ট দেখায়, প্রতটি সারিতে InterviewCard ব্যবহার করে (আবদেনকারীর নামসহ)।

Widget build(BuildContext context, WidgetRef ref)

upcomingInterviewsProvider-এর ডটো নিয়ে PagedListView<Interview> রটরন করে।

ক্লাস/উইজটে: _CandidatesTab

সব candidate-এর সার্চযোগ্য পজেড লিস্ট দেখায়, প্রতটি candidate-এর নাম, বর্তমান পদবি এবং কতগুলো আবদেন করেছে তা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

উপরে _SearchField (যা candidateSearchProvider-এ লখে) ও নচি PagedListView<Candidate> রন্ডার করে; ট্যাপ করলে CandidateDetailScreen-এ যায়।

ক্লাস/উইজটে: _TalentPoolTab

Talent pool-এর candidate তালিকা দেখায়, সার্চসহ, এবং applicationUpdate পারমশিন অনুযায়ী _PoolCard-এ এডটি/রমিউ অ্যাকশন সক্রিয় হয়।

Widget build(BuildContext context, WidgetRef ref)

canUpdate পারমশিন চকে করে, _SearchField ও PagedListView<TalentPoolCandidate> রটরন করে।

ক্লাস/উইজটে: _PoolCard

Talent pool-এর একটি candidate-এর কার্ড; কল করা, এডটি করা এবং pool থেকে সরানোর অ্যাকশন সমর্থন করে।

ConsumerState<_PoolCard> createState()

_PoolCardState তরৈকিরে।

ক্লাস: _PoolCardState

_PoolCard-এর স্টেটে, _busy ফ্ল্যাগ দয়ি চলমান অ্যাকশন ট্র্যাক করে।

Future<void> _call(BuildContext context, String phone)

tel: স্কমিসহ launchUrl কল করে ফোন অ্যাপ খোলার চেষ্টা করে; ব্যর্থ হলে বা এক্সপেশন ধরা পড়লে "কল করার মতো অ্যাপ নই" বার্তা দেখায়।

Future<void> _remove()

একটি নিশ্চিতকরণ ডায়ালগ দেখায় (এটি জানায় যে এটি candidate-এর কোনো আবদেনকে প্রভাবিত করবে না), নিশ্চিতি হলে removeFromTalentPool() কল করে এবং

talentPoolProvider.notifier.removeCandidate() দয়ি লোকাল স্টেটে আপডেট করে।

Widget build(BuildContext context)

candidate-এর নাম, চাহদির পদবি, রটেং, উৎস জব, কারণ ও নোট দেখায়; _busy হলে Loader, নাহলে ফোন নম্বর থাকলে Call বাটন এবং canUpdate হলে Edit ও Remove বাটন দেখায়।

ক্লাস/উইজটে: _SearchField

Candidates ও Talent pool ট্যাবে ব্যবহৃত একটি সাধারণ প্যাডডে সার্চ ফিল্ড wrapper।

Widget build(BuildContext context)
AppSearchField-কে প্যাডটিং দিয়ে মুড়িয়ে রিটার্ন করো।

ফাইল: lib/features/recruitment/refer_candidate_sheet.dart

সংক্ষিপ্ত ববিরণ: একটি নির্দিষ্ট জব পোস্টিং-এর জন্য কাউকে রেফার করার (বা ইমেইলে আসা CV পাইপলাইনে যোগ করার) বটম শিট। এটি পাবলিক ক্যারিয়ার্স ফর্মের স্টাফ-সাইড সমতুল্য।

টপ-লেভেলে ফাংশন

Future<void> showReferCandidateSheet(BuildContext context, {required JobPosting job})
প্রদত্ত job-এর জন্য _ReferSheet মডাল বটম শিট হিসেবে দেখায় (isScrollControlled: true)।

ক্লাস/উইজটে: _ReferSheet

রেফারলে ফর্মের প্রায়নেন্ট ConsumerStatefulWidget, যা job প্রারামিটার নিয়ে।

ConsumerState<_ReferSheet> createState()
_ReferSheetState তৈরি করে।

ক্লাস: _ReferSheetState

নাম, ইমেইল, ফোন, লিংকডইন, নোট-এর জন্য TextEditingController, একটি source ড্রপডাউন, এবং resume আপলোড স্টেটে (_resumeUrl, _resumeName) ধারণ করে।

void dispose()

সব TextEditingController (_name, _email, _phone, _linkedIn, _notes) ডিসপোজ করে।

Future<void> _attachResume()

pickAttachment() দিয়ে ফাইল বাছাই করায়, তারপর apiClientProvider.uploadDocument() দিয়ে সরাসরি আপলোড করে। আবদেন শুধু একটি URL বহন করে, ফাইল নজির নেয় — তাই আগে আপলোড করা হয়, যাতে আপলোড ব্যর্থ হলে অসম্পূর্ণ আবদেন তৈরি না হয়। সফল হলে _resumeUrl/_resumeName সেটে করে; ব্যর্থ হলে _error সেটে করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে; ইমেইল ও ফোন দুটোই খালি থাকলে কাস্টম এরর দেখায় (অন্যত একটা যোগাযোগের মাধ্যম দরকার)। তারপর recruitmentRepositoryProvider.apply() কল করে JobApplicationRequest পাঠায়, openJobsProvider ইনভ্যালিডেট করে, এবং সফল হলে শিট বন্ধ করে দেয়। ApiException বা সাধারণ এরর হ্যান্ডল করে _error সেটে করে।

Widget build(BuildContext context)

FormSheetFrame-এর ভেতরে জব-টাইটলে, নাম/ইমেইল/ফোন/লিংকডইন ফিল্ড, একটি সোর্স ড্রপডাউন (applicationSources), resume সংযুক্তির বাটন (আপলোড চলাকালে Loader), নোট ফিল্ড, এবং একটি সিতরুকবার্তা ("রেফারলে সবসময় বর্তমান ব্যবহারকারীর নামে রেকর্ড হয়") দেখায়।

ফাইল: lib/features/recruitment/schedule_interview_sheet.dart

সংক্ষিপ্ত ববিরণ: কোনো একটিআবদেনরে বপিরীতে ইন্টারভাউ বুক করার বা বদিযমান ইন্টারভাউ পুনঃনরিধারণ (reschedule) করার বটম শাটি। শডিডিউল করা ও রশিডিডিউল করা একই DTO ও একই ফর্ম ব্যবহার করে, শুধু বদিযমান মান দয়িে প্র-ফিলি করা থাকে।

টপ-লভেলে ফাংশন

Future<bool> showScheduleInterviewSheet(BuildContext context, {required int applicationId, Interview? existing})

_ScheduleInterviewSheet মডাল বটম শাটি হসিবে দখেয় এবং bool রটার্ন করে — সত্য মানে একটি ইন্টারভাউ শডিডিউল/রশিডিডিউল হয়ছে, যা কলারকে ইন্টারভাউ তালকা ও আবদেন উভয়ই রফি়রশে করতবে বলে (প্রথম রাউন্ড শডিডিউল হলে আবদেন INTERVIEW_SCHEDULED-এ চলে যায়)।

ক্লাস/উইজেটে: _ScheduleInterviewSheet

ফর্মরে প্যারেন্ট widget, যা applicationId ও ঐচ্ছকি existing ইন্টারভাউ নয়ে।

ConsumerState<_ScheduleInterviewSheet> createState()

_ScheduleInterviewSheetState তরৈ করো।

ক্লাস: _ScheduleInterviewSheetState

ডউরিশন ও মটিং-লংক কন্ট্রোলার, রাউন্ড/মোড/তারখি/সময়/ইন্টারভাউয়াররে স্টেটে ধারণ করে, existing থকে প্র-ফিলি করা।

bool get _isEdit

widget.existing != null কনি তা রটার্ন করে — অর্থাৎ এটি রশিডিডিউল মোড কনি।

TimeOfDay? get _existingTime

widget.existing.scheduledAt-কে পার্স করে TimeOfDay-তে রূপান্তর করে; থাকলে সটোই প্রাথমকি সময়।

void dispose()

_duration ও _meetingLink কন্ট্রোলার ডসিপোজ করে।

Future<void> _pickTime()

showTimePicker() দখেয় এবং নরিবাচতি হলে _time সটে করে।

Future<void> _pickInterviewer()

EmployeePicker.show() দয়িে একজন ইন্টারভাউয়ার বাছাই করায। রো থকে ইন্টারভাউয়ার সডি করা যায় না কারণ রোতে শুধু নাম থাকে, আইডি নয় — ব্যাকএন্ডরে আইডি দরকার, তাই রশিডিডিউল করার সময়ও নতুন করে জজিঞাসা করা হয়।

String _scheduledAt(DateTime date, TimeOfDay time)

তারখি ও সময়কে LocalDateTime ফর্ম্যাটে (zone/Z ছাড়া wall-clock স্ট্রিং, যমেন 2026-08-30T10:00:00) রূপান্তর করে। ইনস্ট্যান্ট পার্ঠালে ব্যাকএন্ড সটোকে লোকাল টাইম হসিবে পড়ে নতিো, যা প্রতটি ইন্টারভাউকে বুককারী টাইমজোন অফসটে অনুযায়ী নীরবে সরয়িে দতি।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে; `_date/_time/_interviewer` কোনোটাই null থাকলে (এগুলো টেক্সট ফিল্ড না হওয়ায় ফর্ম ভ্যালিডেটের ধরতে পারে না) নজিৎ এরর দেখায়। `InterviewRequest` তৈরিকরে `_isEdit` অনুযায়ী `rescheduleInterview()` অথবা `scheduleInterview()` কল করে। সফল হলে `Navigator.pop(true)` করে ও সন্ধ্যাকবার দেখায়; `ApiException` (যেমন "আবেদনটি বিন্ধ") বা সাধারণ এরর ধরে `_error` সটে করে।

Widget build(BuildContext context)

একটি স্ক্রলবেল `Form` তৈরিকরে — `Round` ড্রপডাউন, `DateField` (এক সপ্তাহ পূর্ববর্তী তারিখ পর্যন্ত অনুমোদিত, যাতে পছন্দে ইন্টারভিউ রকেড করা যায়), সময় বাছাইয়ের `InkWell+InputDecorator`, ইন্টারভিউয়ার বাছাইয়ের অনুরূপ উপাদান, `Mode` ড্রপডাউন, দৈর্ঘ্য (মিনিট) ফিল্ড (পজিটিভ সংখ্যা ভ্যালিডেশনসহ), `mode != 'ONSITE'` হলে ঐচ্ছকি মটিং-লিংক ফিল্ড, এবং সবশেষে `LoadingButton`।

ফাইল: lib/features/reports/report_models.dart

সংক্ষিপ্ত বিবরণ: ছয়টি আর্থিক রিপোর্টের (Profit & Loss, Balance Sheet, Trial Balance, Cash Flow, Ageing, Account Ledger) জন্য enum ও ডটো মডলে ক্লাস, যগুলো লজোর থেকে ডেরাইভ করা এবং শুধু পঠনযোগ্য (read-only)।

enum: ReportPeriodKind

রিপোর্ট কোন ধরনের সময়কাল চায় তা নির্দেশ করে — `asAt` (একটি নির্দিষ্ট তারিখে সন্ধ্যাপশট) অথবা `between` (দুই তারিখের মধ্যবর্তী রেঞ্জ)। এই পার্থক্যই ঠিক করে স্ক্রিনি রিপোর্ট চালানোর আগে কী জিজ্ঞাসা করবে।

enum: FinancialReport

ছয়টি রিপোর্টের প্রতিনিধিত্বকারী এনাম, প্রতিটির জন্য একটি লিবেলে, API পাথ সগেমেন্ট, `ReportPeriodKind` এবং একটি বিখ্যামূলক টেক্সট (`blurb`) সংরক্ষণ করে — মান: `profitLoss`, `balanceSheet`, `trialBalance`, `cashFlow`, `ageing`, `accountLedger`।

bool get needsAccount

শুধুমাত্র `accountLedger`-এর জন্য true — এটাই একমাত্র রিপোর্ট যেটির একটি অ্যাকাউন্টও দরকার।

ক্লাস: ProfitLoss

একটি নির্দিষ্ট সময়ে রাজস্ব বনাম খরচের সারসংক্ষেপ — মোবাইলে অ্যাকাউন্ট-ভিত্তিক ব্রকেডাউন না দেখিয়ে শুধু টোটাল দেখায় (ওয়েবে অ্যাকাউন্ট-ভিত্তিক টবেলি দেখানো হয়, যা ফোনের প্রদায় অর্থবহ নয়)।

bool get isProfit

`netProfit >= 0` কনি রটার্ন করে।

double? get margin

`netProfit / totalRevenue` রটার্ন করে, তবে `totalRevenue <= 0` হলে null — কারণ শূন্য রাজস্বে মার্জিনি শূন্য নয়, অসংজ্ঞায়িত।

factory ProfitLoss.fromJson(Map<String, dynamic> json)

JSON থেকে ProfitLoss অবজেক্ট তৈরি করে, সংখ্যাসূচক ফিল্ড না থাকলে ডিফল্ট ০ ব্যবহার করে।

ক্লাস: BalanceSheet

একটি নির্দিষ্ট তারিখে কোম্পানির সম্পদ ও দায়ের অবস্থা প্রত্যাফলিত করে।

bool balanced (ফিল্ড, ডকুমেন্টেড)

সম্পদ = দায় + ইকুইটি সমান হওয়া উচিত; না হলে দুটি সাধারণ কারণ থাকতে পারে — কোথাও ভ্রাসাম্যহীন এন্ট্রি পোস্ট করা, বা একটি ফিসিকাল ইয়ার এখনও ক্লোজ না হওয়া (ফলে নেটে ইনকাম retained earnings-এ যোগ হয়নি)। স্ক্রিনি উভয় কারণই বলে দেয়, কারণ কোনটি ঘটছে তা জানা প্রয়োজনীয় পদক্ষেপে নির্ধারণে সাহায্য করে।

factory BalanceSheet.fromJson(Map<String, dynamic> json)

JSON থেকে BalanceSheet অবজেক্ট বানায়; balanced ডিফল্ট true।

ক্লাস: TrialBalance

প্রতিটি অ্যাকাউন্টের ব্যালেন্স এবং উভয় দিকের যোগফল ধারণ করে।

bool get balances

totalDebit ও totalCredit-এর পার্থক্য 0.005-এর কম কনি যাচাই করে (এক পাইসা পর্যন্ত সহনশীলতা) — ডটোবসের ডেসিমিলে সংখ্যায় সঠিক float তুলনা করলে ভুল পার্থক্য রিপোর্ট হতে পারে বলে এই টলারেন্স ব্যবহৃত হয়েছে।

double get difference

totalDebit - totalCredit রিটার্ন করে।

factory TrialBalance.fromJson(Map<String, dynamic> json)

JSON থেকে TrialBalance তৈরি করে, accounts তালিকার প্রতিটি এন্ট্রি TrialBalanceLine.fromJson দিয়ে পার্স করে।

ক্লাস: TrialBalanceLine

Trial balance-এর একটি একক অ্যাকাউন্ট সারি — debit/credit ব্যালেন্স, অ্যাকাউন্ট আইডি/কোড/নাম ধারণ করে।

factory TrialBalanceLine.fromJson(Map<String, dynamic> json)

JSON থেকে TrialBalanceLine অবজেক্ট তৈরি করে।

ক্লাস: CashFlow

একটি সময়কালে ক্যাটাগরি অনুযায়ী টাকা ইন ও আউটের সারসংক্ষেপে, ওপেনিং/ক্লোজিং ব্যালেন্স ও নেটে পরিবর্তনসহ।

factory CashFlow.fromJson(Map<String, dynamic> json)

JSON থেকে CashFlow তৈরি করে, lines-এর প্রতিটি এন্ট্রি CashFlowLine.fromJson দিয়ে পার্স করে।

ক্লাস: CashFlowLine

একটি ক্যাটাগরির inflow/outflow ধারণ করে।

double get net

inflow - outflow রিটার্ন করে।

factory CashFlowLine.fromJson(Map<String, dynamic> json)

JSON থেকে CashFlowLine তৈরি করে।

ক্লাস: Ageing

বক্যো (outstanding) ইনভয়েস/বিলি কতটা দেরিতে আছে তা বয়স-অনুযায়ী বাকটে (current, 1-30, 31-60, 61-90, 90+) ভাগ করে দেখায়।

double current (ফিল্ড, ডকুমেন্টেডে)

এখনও ডিউ হয়নি এমন পরমাণ।

List<({String label, double amount})> get buckets

পাঁচটা বাকটে নরমালিস্ট ক্রমে (Current, 1-30, 31-60, 61-90, 90+) named record হিসেবে রিটার্ন করে — পজিশনাল না করে নামাঙ্কিত করা হয়েছে, কারণ দুটির ক্রম ভুল হলে তা দৃশ্যমান হবে না।

double get overdue

days1to30 + days31to60 + days61to90 + over90 — সময়সীমা পার হয়ে যাওয়া সবকিছুর যোগফল, যাটো আসলে গুরুত্বপূর্ণ সংখ্যা।

factory Ageing.fromJson(Map<String, dynamic> json)

JSON থেকে Ageing তৈরি করে, lines-এর প্রতিটি এন্ট্রি AgeingLine.fromJson দিয়ে পার্স করে।

ক্লাস: AgeingLine

একটি ageing রিপোর্টের একক বক্যো ডকুমেন্ট — receivables ও payables উভয় দিকের জন্য একই ক্লাস ব্যবহৃত হয় ভিন্ন ভিন্ন JSON কী (invoiceId/billId, invoiceNumber/billNumber, clientName/vendorName) থেকে ম্যাপ করে। ফিল্ডগুলো তাদের অর্থ অনুযায়ী নামাঙ্কিত (documentId, reference, counterparty), কোন রিপোর্ট থেকে এসেছে তার উপর ভিত্তি করে নয়।

int daysOverdue (ফিল্ড, ডকুমেন্টেডে)

শূন্য বা ঋণাত্মক হলে বোঝায় এখনও ডিউ হয়নি।

int? documentId (ফিল্ড, ডকুমেন্টেডে)

ইনভয়েস অথবা বিলি-এর আইডি।

String? reference (ফিল্ড, ডকুমেন্টেডে)

এর নম্বর (ইনভয়েস/বিলি নম্বর)।

String? counterparty (ফিল্ড, ডকুমেন্টেডে)

যে ক্লায়েন্ট এটি বিক্যো রেখেছে, অথবা যে ভেন্ডরকে এটি বিক্যো দেওয়া আছে।

bool get isOverdue

daysOverdue > 0 কনি রিটার্ন করে।

factory AgeingLine.fromJson(Map<String, dynamic> json)

JSON থেকে AgeingLine তৈরি করে, invoiceId/billId, invoiceNumber/billNumber, clientName/vendorName — যে কোনো একটি পাওয়া গেলে সেটি ব্যবহার করে ম্যাপ করে।

ক্লাস: AccountLedgerReport

একটি নির্দিষ্ট অ্যাকাউন্টের একটি সময়কালের সব পোস্টিং, ওপেনিং ও ক্লোজিং ব্যালেন্সসহ।

List<Map<String, dynamic>> entries (ফিল্ড, ডকুমেন্টেডে)

General ledger যশেপে রটরন করে সেই একই শপে — তাই মডউলরে নজিস্ব LedgerLine-ই এগুলো পার্স করতে পারে।

double get movement

closingBalance - openingBalance রটরন করে।

factory AccountLedgerReport.fromJson(Map<String, dynamic> json)

JSON থেকে AccountLedgerReport তরৈকিরে, entries-এর প্রতটি Map<String, dynamic> এন্টরি সরাসরি রাখে (পার্স না করে)।

ফাইল: lib/features/reports/report_repository.dart

সংক্ষিপ্ত ববিরণ: ছয়টি আরথকি রপিোর্টরে জন্য API কল সম্পাদনকারী রপিোর্জটির, এবং একটি Riverpod প্রোভাইডার যা রপিোর্ট-কুয়রে অনুযায়ী ফলাফল ক্যাশ করে। প্রতটি রপিোর্ট একটি সাধারণ GET রকি়োস্টে মাত্র, তাই আলাদা কন্ট্রোলাররে প্রয়োজন নহে।

ক্লাস: ReportRepository

ApiClient ব্র্যাপ করে /company/finance/reports বসে পাথরে নচি ছয়টি রপিোর্ট এন্ডপয়ন্ট কল করে।

Future<ProfitLoss> profitLoss(String startDate, String endDate)

/profit-loss এন্ডপয়ন্টে startDate/endDate কুয়রে প্যারামটিরসহ GET করে ProfitLoss.fromJson দিয়ে পার্স করে রটরন করে।

Future<BalanceSheet> balanceSheet(String asOfDate)

/balance-sheet এন্ডপয়ন্টে asOfDate প্যারামটিরসহ GET করে BalanceSheet.fromJson-এ পার্স করে।

Future<TrialBalance> trialBalance(String asOfDate)

/trial-balance এন্ডপয়ন্টে asOfDate প্যারামটিরসহ GET করে TrialBalance.fromJson-এ পার্স করে।

Future<CashFlow> cashFlow(String startDate, String endDate)

/cash-flow এন্ডপয়ন্টে startDate/endDateসহ GET করে CashFlow.fromJson-এ পার্স করে।

Future<Ageing> ageing(String asOfDate)

/ageing এন্ডপয়ন্টে asOfDateসহ GET করে Ageing.fromJson-এ পার্স করে।

Future<AccountLedgerReport> accountLedger(int accountId, String startDate, String endDate)

/ledger এন্ডপয়ন্টে accountId, startDate, endDate কুয়রে প্যারামটিরসহ GET করে AccountLedgerReport.fromJson-এ পার্স করে।

টপ-লভেলে প্রোভাইডার

reportRepositoryProvider — apiClientProvider থেকে ApiClient নিয়ে ReportRepository তরৈকিরে দেয়।

ক্লাস: ReportQuery

একটি রিপোর্টের অনুরোধ কী চয়েছেলি তার একটি ভিউ-টাইপ প্রতিনিধিত্ব — কোন রিপোর্ট, কোন সময়সীমা, এবং প্রয়োজ্য ক্ষেত্রে কোন অ্যাকাউন্ট। এটি একটি ভিউ টাইপ যাতো প্রোভাইডার এর উপর কী করে ক্যাশ করতে পারে — এর যেকোনো অংশ পরবর্তন হলে স্টেট একটি ভিন্ন রিপোর্ট বলে গণ্য হয়, একই জনিসি আবার চাইলে পূর্বের উত্তর পুনরায় ব্যবহৃত হয়।

String get asAt

end-কে as-at রিপোর্টের তারখি হিসেবে রিটার্ন করে (as-at রিপোর্টে start উপক্ষেতি হয়)।

bool operator ==(Object other)

report, start, end, accountId — সবগুলো ফিল্ড সমান কনি তুলনা করে সমতা নির্ধারণ করে।

int get hashCode

report, start, end, accountId-এর উপর ভিত্তি করে হ্যাশ কোড রিটার্ন করে।

টপ-লেভেলে প্রোভাইডার

reportProvider — FutureProvider.autoDispose.family<Object, ReportQuery>; query.report-এর উপর switch করে ছয়টি রিপোর্টের মিথেডের একটিকে কল করে ফলাফল Object টাইপে (untyped) রিটার্ন করে, কারণ প্রতটি রিপোর্টেরে নজিস্ব শপে আছে এবং স্ক্রিনি নজিই কোনো রিপোর্ট চাওয়া হয়েছে তার উপর সুইচ করে; এটিকে টাইপ করে আলাদা করলে ছয়টি প্রায়-একই প্রোভাইডার তৈরি হতো।

টপ-লেভেলে ফাংশন

ReportQuery defaultQuery(FinancialReport report, {DateTime? now})

একটি যুক্তিসিঙগত ডিফল্ট সময়সীমা তৈরি করে — চলতি মাসেরে শুরু থেকে আজ পর্যন্ত (start/end), as-at রিপোর্টেরে ক্ষেত্রে end-ই ব্যবহৃত হয়।

ফাইল: lib/features/reports/report_screen.dart

সংক্ষিপ্ত ববিরণ: ছয়টি আর্থিক রিপোর্টের একটি একক স্ক্রিনি, যখনে ব্যবহারকারী একটি রিপোর্ট বছে নেয়, সময়সীমা স্টেট করে এবং রিপোর্ট চালায়। প্রতটি রিপোর্টেরে জন্য নজিস্ব ভিজ্যুয়ালাইজেশন উইজটে আছে।

ক্লাস/উইজটে: ReportsScreen

মূল স্ক্রিনি, একটি ConsumerStatefulWidget, যা নির্বাচতি রিপোর্ট, কুয়েরি এবং "চালু হওয়া" (running) কুয়েরি স্টেটে ধারণ করে।

ConsumerState<ReportsScreen> createState()

_ReportsScreenState তৈরি করে।

ক্লাস: _ReportsScreenState

_report (নির্বাচতি রিপোর্ট), _query (বর্তমান সময়সীমা/অ্যাকাউন্ট) এবং _running (যা রিপোর্ট আসলে চালানো হয়েছে) স্টেটে রাখো। _running null হয় যতক্ষণ না রিপোর্ট প্রকৃতপক্ষে চাওয়া হয় — সময়সীমা পরবর্তন করলে নীরবে ব্যয়বহুল কছি পুনরায় চলা উচতি নয়, তাই চালানো স্পষ্টভাবে করতে হয়।

`void _pick(FinancialReport report)`

`_report` পরিবর্তন করে, কনিন্তু বর্তমান `_query.start/_query.end/_query.accountId` বজায় রেখে নতুন `ReportQuery` তৈরি করে (একই সময়সীমায় দুটি রিপোর্ট তুলনাকারী কাউকে আবার তারখি টাইপ করতে হবে না), এবং `_running` রসিটে করে।

`Widget build(BuildContext context)`

পারমশিন লোড না হলে `Loader` দেখায়। `AccountingPermissions.ledgerView` না থাকলে `EmptyState` দেখায় (কারণ সব রিপোর্ট জনোরলে লজোর পড়ে)। অন্যথায় একটি অনুভূমিক `ChoiceChip` তালিকা (ছয়টি রিপোর্ট), নির্বাচতি রিপোর্টের `blurb`, `_Period` উইজেটে এবং `_running != null` হলে `_Result` উইজেটে দেখায়।

ক্লাস/উইজেটে: `_Period`

রিপোর্ট কীসরে জন্য চাওয়া হচ্ছে তার ইনপুট ফর্ম (অ্যাকাউন্ট নির্বাচন, তারখি/তারখি-রঞ্জে) এবং "Run it" বাটন দেখায়।

`Widget build(BuildContext context, WidgetRef ref)`

`report.needsAccount` হলে `accountsProvider` থেকে অ্যাকাউন্ট তালিকা এনে একটি `DropDownButtonFormField<int>` দেখায় (শুধু পোস্ট করা যায় এমন অ্যাকাউন্ট)। `report.periodKind`-এর উপর ভিত্তি করে হয় একক "As at" `DateField` অথবা "From"/"To" জোড়া `DateField` দেখায়। `ready` (অ্যাকাউন্ট বাছাই লাগলে বাছাই হয়েছে কনি) অনুযায়ী `LoadingButton`-এর "Run it" সক্রিয়/নিষ্ক্রিয় করে; না-প্রস্তুত হলে একটি সহায়ক বার্তা দেখায়।

ক্লাস/উইজেটে: `_Result`

নির্বাচতি `ReportQuery`-এর ফলাফল দেখায়, রিপোর্টের টাইপ অনুযায়ী উপযুক্ত ভিউতে রূপান্তরতি করে।

`Widget build(BuildContext context, WidgetRef ref)`

`reportProvider(query)`-কে `watch` করে `.when()` দিয়ে `loading/error/data` হ্যান্ডল করে। `error`-এ `retry`-সহ `ErrorState` দেখায় (`retry reportProvider(query)` ইনভ্যালডিটে করে)। `data` এলে `switch` দিয়ে ডটোর রানটাইম টাইপ অনুযায়ী `_ProfitLossView`, `_BalanceSheetView`, `_TrialBalanceView`, `_CashFlowView`, `_AgeingView` অথবা `_AccountLedgerView`-এর একটি রিটার্ন করে; কোনোটির সাথে না মিললে (অপ্রাপ্য কসে) একটি "Nothing to show" `EmptyState` দেখায়।

ক্লাস/উইজেটে: `_ProfitLossView`

Profit & Loss রিপোর্টের ভিজ্যুয়াল উপস্থাপনা — রাজস্ব ও খরচের `StatCard` জোড়া, নটে প্রফিট/লস কার্ড (মার্জিনিসহ) এবং সময়সীমার footnote দেখায়।

`Widget build(BuildContext context)`

Revenue ও Expense-এর `StatCard`, তারপর net profit/loss-এর `StatCard` (মার্জনি suffix, লস হলে danger tone সহ আইকন), এবং `_Footnote`-এ সময়সীমা দেখায়।

ক্লাস/উইজেটে: `_BalanceSheetView`

Balance Sheet রিপোর্টের ভিজ্যুয়াল, ভারসাম্যহীন হলে সতর্কবার্তাসহ।

Widget build(BuildContext context)

!report.balanced হলে একটি MessageBanner.warning দেখায় যা কারণ ব্যাখ্যা করে। এরপর একটি AppCard-এ Assets, Liabilities, Equity এবং Liabilities+Equity সমষ্টি সারি(_Row) দেখায়, শেষে as-of তারখিरे _Footnote।

ক্লাস/উইজটে: _TrialBalanceView

Trial Balance রপোর্টের টবেলি-জাতীয় ভিউ, প্রতটি অ্যাকাউন্টের debit/credit এবং টোটাল দেখায়।

Widget build(BuildContext context)

!report.balances হলে একটি MessageBanner.error দেখায়। তারপর একটি হিডার রো (Account/Debit/Credit), প্রতটি অ্যাকাউন্টের সারি, এবং সবশেষে Totals সারি দেখায়; শেষে as-of তারখিरे _Footnote।

ক্লাস/উইজটে: _CashFlowView

Cash Flow রপোর্টের ভিজ্যুয়াল — In/Out StatCard, opening/net change/closing balance কার্ড, এবং থাকলে ক্যাটাগরি-ভিত্তিক তালিকা।

Widget build(BuildContext context)

In/Out StatCard জোড়া, AppCard-এ opening balance/net change/closing balance (_Row), report.lines অথবা থাকলে "By category" SectionHeader ও প্রতটি ক্যাটাগরির net (নেগেটিভ হলে danger রঙে) দেখায়; শেষে সময়সীমার _Footnote।

ক্লাস/উইজটে: _AgeingView

Ageing রপোর্টের ভিজ্যুয়াল — outstanding/overdue StatCard, বয়স-বাকটে প্রগ্রসে বার এবং ঐচ্ছিক ইনভয়েস তালিকা।

Widget build(BuildContext context)

Outstanding ও Overdue StatCard (overdue > 0 হলে warning tone), তারপর প্রতটি বাকটের জন্য লবেল/পরিমাণ ও LinearProgressIndicator (মোট-এর বপিরীতে অনুপাত, যাত বাকটগুলো তুলনাযোগ্য হয়, প্রতটি নিজেরটাই ভর না ফলে; "Current" ব্র্যান্ড রঙে, বাকগুলো warning রঙে) দেখায়। report.lines অথবা থাকলে "Invoices" SectionHeader ও প্রতটি লাইনের রফোরেন্স, counterparty, দেরি/ডিডি তথ্য এবং ব্যালেন্স দেখায়; শেষে as-of _Footnote।

ক্লাস/উইজটে: _AccountLedgerView

Account Ledger রপোর্টের ভিজ্যুয়াল — অ্যাকাউন্টের কোড/নাম, opening/movement/closing ব্যালেন্স, এবং postings-এর সংখ্যাসহ footnote।

Widget build(BuildContext context)

অ্যাকাউন্ট কোড ও নাম দেখায়, তারপর _Row-এ Opening/Movement/Closing, এরপর কতগুলো posting ও সময়সীমার _Footnote, এবং শেষে একটি বার্তা যা জানায় যে প্রকৃত postings Books স্ক্রিনিরে Ledger-এ (এই অ্যাকাউন্ট ফলিটার করা অবস্থায়) দেখা যাবে।

ক্লাস/উইজটে: _Row

রপোর্টের ভেতরে একটি লবেল ও মুদ্রা-পরিমাণ পাশাপাশি দেখানোর জন্য পুনঃব্যবহারযোগ্য সারি; bold হলে ভারী ফন্ট।

Widget build(BuildContext context)

লবেলে ও Fmt.money(amount) সহ একটি Row রটাইরন করে; পরমাণ খাণাত্মক হলে danger রঙে দেখায়, bold হলে ফন্ট-ওয়েটে ভারী করে।

ক্লাস/উইজটে: _Footnote

রপিোর্টরে নচি ছোট, muted রঙে ব্যাখ্যামূলক টেক্সট (যেমন সময়সীমা বা as-of তারখি) দেখানোর জন্য ব্যবহৃত।

Widget build(BuildContext context)

muted রঙে ছোট ফন্টরে একটি Text রটাইরন করে।

ফাইল: lib/features/requests/approvals_tab.dart

সংক্ষিপ্ত বিবরণ: এই ফাইলটি ওয়ার্কফলো স্টেজে অ্যাপ্রুভালরে ট্যাব দেখায়, যেখানে ব্যবহারকারীর সন্ধিন্তরে অপেক্ষায় থাকা সব স্টেজে তালিকাভুক্ত হয়। প্রতিটি অ্যাপ্রুভাল কার্ডে অনুমোদন বা প্রত্যাখ্যান করার বাটন থাকে, এবং প্রত্যাখ্যানের ক্ষেত্রে কারণ লেখা বাধ্যতামূলক।

ক্লাস/উইজটে: ApprovalsTab

এটি একটি ConsumerWidget যা approvalsProvider থেকে পজেনিটেডে অ্যাপ্রুভাল তালিকা টেনে এনে PagedListView এর মাধ্যমে প্রদর্শন করে; খালি অবস্থা, রিফ্রেশ, লোড-মোর এবং এরর মসেজের কনফিগারেশন এখানে সেট করা হয়।

Widget build(BuildContext context, WidgetRef ref)

approvalsProvider.notifier থেকে কন্ট্রোলার নিয়ে PagedListView<StageApproval> রটাইরন করে। onRefresh কন্ট্রোলারের refresh মেথডে বাঁধা, onLoadMore একটি guardListAction রূপারের মধ্য দিয়ে loadMore কল করে। খালি অবস্থায় "Nothing waiting on you" শিরোনাম ও একটি বার্তা দেখানো হয়, এবং প্রতিটি আইটেমে _ApprovalCard দিয়ে বন্ডি করা হয়।

ক্লাস/উইজটে: _ApprovalCard

একটি ConsumerStatefulWidget যা একটিমাত্র StageApproval কে কার্ড আকারে দেখায় এবং অনুমোদন/প্রত্যাখ্যান করার ইন্টারঅ্যাকশন পরিচালনা করে।

_ApprovalCardState createState()

_ApprovalCardState এর একটি নতুন ইনস্ট্যান্স তৈরি করে রটাইরন করে; স্ট্যান্ডার্ড Flutter স্টেটে ফ্যাক্টরি মেথড।

ক্লাস: _ApprovalCardState

_ApprovalCard এর স্টেটে ম্যানেজ করে — একটি _busy ফ্ল্যাগ রাখে যা সন্ধিন্ত নওয়ার সময় UI-তে স্পনির দেখানোর জন্য ব্যবহৃত হয়, এবং সন্ধিন্ত-সংক্রান্ত API কল, নোট চাওয়ার বটম শিট, ও কার্ডের ভিজ্যুয়াল লেআউট এখানে থাকে।

`Future<void> _decide({required bool approved})`

অনুমোদন বা প্রত্যাখ্যানের সিদ্ধান্ত কার্যকর করে। প্রত্যাখ্যানের ক্ষেত্রে `_askForReason()` দিয়ে বাধ্যতামূলক কারণ চাওয়া হয়, আর অনুমোদনের ক্ষেত্রে `_askForOptionalNote()` দিয়ে ঐচ্ছকি নোট; শর্তি বাতলি করলে (null রটার্ন) পুরো অ্যাকশনও বাতলি হয়ে যায়। এরপর `approvalsProvider.notifier.decide()` কল করে ফলাফল অনুযায়ী স্ন্যাকবার দেখায় এবং `ApiException` বা অন্য এরর আলাদাভাবে হ্যান্ডেল করে। শেষে `_busy` false করে দেয়, তবে শুধু যদি উইজেটটি এখনও `mounted` থাকে।

`Future<String?> _askForReason()`

প্রত্যাখ্যানের কারণ জানতে `_promptForNotes` কে `required: true` সহ কল করে একটি বিটম শর্তি দেখায়, যার শরিনোম "Why are you rejecting this?"।

`Future<String?> _askForOptionalNote()`

অনুমোদনের সময় ঐচ্ছকি নোট নতি `_promptForNotes` কে `required: false` সহ কল করে, শরিনোম "Approve this stage?"।

`Future<String?> _promptForNotes({required String title, required String hint, required String confirmLabel, required bool required})`

একটি সাধারণ মোডাল বিটম শর্তি তৈরি করে যাতে একটি `TextFormField` (মন্তব্য/কারণ লেখার জন্য) ও একটি কনফার্ম বাটন থাকে। `required true` হলে খালি টেক্সট ভ্যালিডেশন এরর দেখায়। ব্যবহারকারী কনফার্ম করলে ট্রিম করা টেক্সট নিয়ে শর্তি পপ করে, আর শর্তি বন্ধ হলে কন্ট্রোলার ডিসপোজ হয় যায়।

`Widget build(BuildContext context)`

`AppCard` আকারে অ্যাপ্রুভাল কার্ড রেন্ডার করে — শরিনোম, স্ট্যাটাস চপি, স্টেজ/রোল তথ্য, "Raised by" ও সময়ের তথ্য দেখায়। ট্যাপ করলে `openRequestDetail` কল হয়ে রকি়োস্ট ডিটাইল স্ক্রিনিং যায়। `_busy true` হলে `Loader` দেখায়, নাহলে "Reject" ও "Approve" বাটন দুটি সিরতি দেখায়, যগুলো `_decide()` কল করে।

ফাইল: `lib/features/requests/edit_request_sheet.dart`

সংক্ষিপ্ত বিবরণ: একটি রকি়োস্টের শরিনোম, বিবরণ, প্রায়েরটি ও সম্মত মূল্য এডিট করার বিটম শর্তি। শুধু এই কয়েকটি বেসিক ফিল্ড এখানে এডিটযোগ্য — স্টেজ পরিবর্তন বা অ্যাসাইনমেন্ট রকি়োস্ট ডিটাইল স্ক্রিনিং থেকেই করতে হয়।

উপ-লভেলে ফাংশন

`Future<ServiceRequest?> showEditRequestSheet(BuildContext context, ServiceRequest request)`
`_EditRequestSheet` কে একটি `isScrollControlled` মোডাল বিটম শর্তি দেখায় এবং সারভার থেকে ফরি আসা আপডেটেড `ServiceRequest` (বা বাতলি হলে null) রটার্ন করে।

ক্লাস/উইজেট: `_EditRequestSheet`

একটি `ConsumerStatefulWidget` যা এডিট করার জন্য প্রাথমিক `ServiceRequest` ডটো নিয়ে ফর্ম তৈরি করে।

`ConsumerState<_EditRequestSheet> createState()`
`_EditRequestSheetState` এর নতুন ইনস্ট্যান্স রিটার্ন করে।

ক্লাস: `_EditRequestSheetState`

ফর্ম কন্ট্রোলার (`_title`, `_description`, `_price`), প্রায়েরটি স্টেটে, সাবমিশন স্ট্যাটাস ও এরর মैसेজে ম্যানেজ করে; বর্তমান রকিওয়েস্ট ডটো দিয়ে ফল্ডগুলো প্রফিলি করা থাকে।

`void dispose()`

`_title`, `_description`, ও `_price` — এই তিনটি `TextEditingController` ডিসপোজ করে মেরলিকি প্রতরোধ করে, তারপর সুপারক্লাসের `dispose()` কল করে।

`Future<void> _submit()`

ফর্ম ভ্যালডিটে করে (ইতিমধ্যে সাবমিট হচ্ছে থাকলে বা ভ্যালডিশন ফলে করলে থমে যায়), ফোকাস সরিয়ে `_submitting` true করে, তারপর `requestRepositoryProvider.update()` কল করে

`UpdateServiceRequestRequest` পাঠায় (শিরোনাম, বিবরণ, প্রায়েরটি, ও পার্স করা মূল্য)। সফল হলে নভিগেটর পপ করে আপডেটেড রকিওয়েস্ট রিটার্ন করে ও সন্ধ্যাকবার দেখায়; ব্যর্থ হলে `_error` স্টেটে করে। শেষে `_submitting` false করে দেয়।

`Widget build(BuildContext context)`

একটি স্ক্রলযোগ্য ফর্ম তৈরি করে — শিরোনাম "Edit request", এরর ব্যানার (থাকলে), শিরোনাম/বিবরণ টেক্সট ফিল্ড, প্রায়েরটি ড্রপডাউন (বর্তমান মান তালিকায় না থাকলে তা যোগ করে দেয়), মূল্য ফিল্ড (নেগেটিভি বা অসংখ্য মান ভ্যালডিটে করে), স্টেজ/অ্যাসাইনমেন্ট পরিবর্তন এখানে হয় না — এই মর্মে একটি তথ্য নোট, এবং সবশেষে `LoadingButton` যা `_submit` কল করে।

ফাইল: `lib/features/requests/new_package_sheet.dart`

সংক্ষিপ্ত বিবরণ: কোম্পানির সার্ভিস ক্যাটালগে নতুন প্যাকেজে যোগ করার বটম শিট। শুধু তৈরি করাই সমর্থিত — এডিট বা রিটার্ন করার কাজ ওয়েব অ্যাপে করতে হয়।

টপ-লেভেলে ফাংশন

`Future<void> showNewPackageSheet(BuildContext context)`

`_NewPackageSheet` কে একটি `isScrollControlled` মোডাল বটম শিটে দেখায়।

ক্লাস/উইজটে: `_NewPackageSheet`

একটি `ConsumerStatefulWidget` যা প্যাকেজে তৈরি ফর্ম ধারণ করে (নাম, ব্লিইং সাইকেল, মূল্য, ডিসকাউন্ট, কোটা, ডেলিভারি দিন, বিবরণ, এবং অন্তর্ভুক্ত সার্ভিসসমূহ)।

`ConsumerState<_NewPackageSheet> createState()`

`_NewPackageSheetState` এর নতুন ইনস্ট্যান্স রিটার্ন করে।

ক্লাস: `_NewPackageSheetState`

সব টেক্সট কন্ট্রোলার, নির্বাচনী ব্লিইং সাইকেল, নির্বাচনী সার্ভিস আইডির স্টেট (`_serviceIds`), সাবমিশন স্ট্যাটাস ও এরর ম্যানেজ করে।

`void dispose()`

`_name`, `_description`, `_price`, `_discount`, `_quota`, `_deliveryDays` — এই ছয়টি কন্ট্রোলার ডিসপোজ করে, তারপর সুপারক্লাসের `dispose()` কল করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, `_submitting` true করে, তারপর `requestRepositoryProvider.createPackage()` কল করে `CreateServicePackageRequest` পাঠায় (নাম, ব্লিইং সাইকেলে, ববিরণ, প্যাকজে মূল্য, ডিসকাউন্ট শতাংশ, রকি়োয়স্ট কোটা, ডলেভিারি দিনি, নরিবাচতি সারভিস আইডিসিমূহ)। সফল হলে শটি পপ করে "added to the catalogue" স্ন্যাকবার দেখায়; ব্যর্থ হলে `_error` সটে করে; শেষে `_submitting` false করে।

`Widget build(BuildContext context)`

`catalogServiceProvider` ওয়াচ করে ফর্ম ব্লিড করে — নাম, ব্লিইং সাইকেলে ড্রপডাউন, মূল্য/ডিসকাউন্ট (এক সারতি), কোটা/ডলেভিারি দিনি (আরকে সারতি), ববিরণ, এবং ক্যাটালগ থেকে টানা সারভিস তালিকা `CheckboxListTile` হিসেবে দেখায়। ক্যাটালগ লোডিং/এরর/খালি অবস্থার জন্য আলাদা বার্তা আছে। সবশেষে `LoadingButton` যা `_submit` কল করে।

`static String? _money(String? value)`

মূল্য ফিল্ডের ভ্যালিডেটর — খালি হলে বই (ঐচ্ছকি), সংখ্যায় পার্স না হলে "Enter a number." রটার্নন করে, নেগেটিভি হলে "Cannot be negative." রটার্নন করে, অন্যথায় null (বই)।

ফাইল: `lib/features/requests/raise_request_sheet.dart`

সংক্ষিপ্ত ববিরণ: ক্লায়েন্ট বা পোর্টাল ব্যবহারকারীর জন্য নতুন সারভিস রকি়োয়স্ট তরৈকিয়ার বটম শটি, যখনে ক্যাটালগ থেকে সারভিস বাছাই করে শরিোনাম, প্রায়েরটি ও বসিতারতি লখো যায়।

টপ-লেভেলে ফাংশন

`Future<void> showRaiseRequestSheet(BuildContext context)`

`_RaiseRequestSheet` কে একটি `isScrollControlled` মোডাল বটম শটি দেখায়।

ক্লাস/উইজটে: `_RaiseRequestSheet`

একটি `ConsumerStatefulWidget` যা রকি়োয়স্ট তরৈরি ফর্ম ধারণ করে।

`ConsumerState<_RaiseRequestSheet> createState()`

`_RaiseRequestSheetState` এর নতুন ইনস্ট্যান্স রটার্নন করে।

ক্লাস: `_RaiseRequestSheetState`

নরিবাচতি `CatalogService`, প্রায়েরটি (ডিফল্ট 'NORMAL'), টাইটলে/ববিরণ কন্ট্রোলার, সাবমশিন স্ট্যাটাস ও এরর ম্যানজে করে।

`void dispose()`

`_title` ও `_description` কন্ট্রোলার ডিসপোজ করে, তারপর সুপারক্লাসের `dispose()` কল করে।

`void _pickService(CatalogService? service)`

নির্বাচতি সার্ভিস স্টেট করে; সার্ভিসের নিজস্ব ডিফল্ট প্রায়োরিটি বৈধ হলে (তালকীয় থাকলে) স্টেটিকে `_priority` তে স্টেট করে দেয়, যাত সাধারণ ক্ষেত্রে ব্যবহারকারীকে আলাদা করে ভাবতে না হয় — তবে ফলিউটি এডিটিযোগ্যই থাকে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, একটি সার্ভিস নির্বাচতি আছে কিনা যাচাই করে (না থাকলে এরর দেখায় ও থমে যায়), তারপর `myRequestsProvider.notifier.create()` কল করে `CreateServiceRequest` পাঠায় (শিরোনাম, `hubServiceId`, ববিরণ, প্রায়োরিটি)। সফল হলে শিটি পপ করে "Request submitted." সন্ধ্যাকবার দেখায়; ব্যর্থ হলে `_error` স্টেট করে; শেষে `_submitting` false করে।

`Widget build(BuildContext context)`

`catalogServiceProvider` ওয়াচ করে — লোডিং/এরর/খালি অবস্থা হ্যান্ডল করে, সার্ভিস ড্রপডাউন দেখায় (নির্বাচতি হলে `_ServiceHint` দেখায়), শিরোনাম ফলিউ, প্রায়োরিটি ড্রপডাউন, ঐচ্ছকি ববিরণ ফলিউ, এবং পমেন্টে ছাড়াই সাবমিট হব জানিয়ে একটি তথ্য ব্যানার দেখায়। সবশেষে `LoadingButton` যা `_submit` কল করে।

ক্লাস/উইজটে: `_ServiceHint`

একটি `StatelessWidget` যা নির্বাচতি ক্যাটালগ সার্ভিস সম্পর্কে ক্যাটালগ থেকে পাওয়া তথ্য দেখায় — ক্যাটেগরি, মূল্য, আনুমানিকি দিন, ববিরণ, এবং কন্টেনশন/ডকুমেন্ট প্রয়োজন কিনা তার নোট।

`Widget build(BuildContext context)`

সার্ভিসের ক্যাটেগরিনাম, মূল্য (ও প্রাইস টাইপ), আনুমানিকি দিন — এই তথ্যগুলো `facts` তালকীয় জড়ো করে একটি লাইনে দেখায়; ববিরণ থাকলে তা দেখায়; এবং সার্ভিসের

`requiresQuotation/requiresDocuments` true হলে সে সংক্রান্ত একটি তথ্যসূচক নোট দেখায়, সব একটি হালকা রঙের কন্টইনারে।

ফাইল: `lib/features/requests/rate_request_sheet.dart`

সংক্ষিপ্ত ববিরণ: সম্পন্ন হওয়া রকি়োস্টে রটেং দেওয়ার বটম শিটি। শুধু পোর্টাল ক্লায়েন্টকে তার নিজের সম্পন্ন রকি়োস্টে এই অপশন দেখানো হয়; ব্যাকএন্ড অন্য সব ক্ষেত্রে প্রত্যাখ্যান করে এবং সেই কারণটি বার্তা হিসেবে দেখানো হয়।

উপ-লভেলে ফাংশন

`Future<bool> showRateRequestSheet(BuildContext context, {required ServiceRequest request})`

`_RateRequestSheet` কে একটি মোডাল বটম শিটি দেখায় ও ফলাফলের জন্য অপেক্ষা করে। রটেং সফলভাবে জমা হলে `true` রটার্ন করে, বাতলি বা `null` হলে `false` রটার্ন করে।

ক্লাস/উইজটে: `_RateRequestSheet`

একটি `ConsumerStatefulWidget` যা রটেং দেওয়ার ফর্ম (স্টার রটেং + মন্তব্য) ধারণ করে।

`ConsumerState<_RateRequestSheet> createState()`

`_RateRequestSheetState` এর নতুন ইনস্ট্যান্স রটার্ন করে।

ক্লাস: `_RateRequestSheetState`

মন্তব্য কন্ট্রোলার, নরিবাচতি রটেং (`_rating`, শুরুতে `null` — যাত্রে একটি প্রি-ফিল্ড স্কোর ব্যবহারকারীর প্রকৃত পছন্দ বলে বহিরান্তা তরৈ কিরে), সাবমশিন স্ট্যাটাস ও এরর ম্যানজে করে।

`void dispose()`

`_comment` কন্ট্রোলার ডসিপোজ করে, তারপর সুপারক্লাসরে `dispose()` কল করে।

`Future<void> _submit()`

রটেং নরিবাচতি না থাকলে এরর দেখেয়ে থামে। নাহলে `_submitting` `true` করে

`requestRepositoryProvider.submitReview()` কল করে `SubmitReviewRequest` পাঠায় (`serviceRequestId`, রটেং, মন্তব্য)। সফল হলে নভেগিটের `true` দিয়ে পপ করে ধন্যবাদ স্ন্যাকবার দেখেয়ে; `ApiException` (যেমন শুধু সমপন্ন রকি়োস্টেই রভিডি দেওয়া যায়, বা নজিরে রকি়োস্টেই দেওয়া যায় — এই দুই ধরনরে এরর মসেজে) বা অন্য এরর হলে `_error` সটে করে; শেষে `_submitting` `false` করে দেয়ে।

`Widget build(BuildContext context)`

শরি়োনাম "How did it go?", সারভসিরে নাম, এরর ব্যানার (থাকলে), ১ থেকে ৫ স্টাররে একটি সারি (নরিবাচতি রটেং পরযন্ত ভরাট তারা দেখেয়ে, ট্যাপ করলে `_rating` সটে হয়), রটেং অনুযায়ী একটি লবেলে টেক্সট, ঐচ্ছকি মন্তব্য ফিল্ড, এবং `LoadingButton` যা `_submit` কল করে — এইসব উপাদান নয়ে ফর্ম বন্ডি করে।

`static String _label(int rating)`

রটেং সংখ্যাকে (১-৫) একটি বরণনামূলক লবেলে রূপান্তর করে — `switch` এক্সপ্রেশন দিয়ে ১='Poor', ২='Not great', ৩='Fine', ৪='Good', এবং ডফিল্ট (৫)='Excellent' রটার্ন করে।

ফাইল: `lib/features/requests/request_card.dart`

সংক্ষপ্ত ববিরণ: যেকোনো রকি়োস্ট তালকায় ব্যবহৃত একটি সাধারণ কার্ড উইজেটে, যা শরি়োনাম, স্ট্যাটাস, প্রায়েরটি, SLA অবস্থা, টাস্ক প্রগ্রসে ও কটেশন সংক্রান্ত তথ্য দেখেয়ে।

ক্লাস/উইজেটে: `RequestCard`

একটি `StatelessWidget` যা একটি `ServiceRequest` কে সংক্ষপ্ত কার্ড আকারে উপস্থাপন করে; বহির্ন রকি়োস্ট তালকায় (মাই রকি়োস্ট, অ্যাসাইনড ইত্যাদি) এই একই কার্ড পুনরায় ব্যবহৃত হয়।

`Widget build(BuildContext context)`

`AppCard` এর ভতরে শরি়োনাম ও `StatusChip` (এক সারতি), হাব সারভসিরে নাম (থাকলে), প্রায়েরটি ডিট ও লবেলে, তরৈ হওয়ার আপক্ষকি সময়, অ্যাসাইনড কর্মীর নাম (থাকলে, ডানে সারবিদ্ধ) দেখেয়ে। রকি়োস্টের `slaBreach` `true` হলে একটি সিতরকতা সারি দেখেয়ে (SLA সময়সীমা নরিধারণ সারভার নজিই করে, তাই এটি পুনরায় হসিব না করে সরাসরি দেখানো হয়)। `taskProgress` না-`null` হলে একটি প্রগ্রসে বার ও "সমপন্ন/মোট টাস্ক" টেক্সট দেখেয়ে। `quotationAwaitsDecision` `true` হলে কটেশনের পরমাণসহ একটি তথ্য ব্যানার দেখেয়ে।

ক্লাস/উইজেটে: `_PriorityDot`

একটি `StatelessWidget` যা প্রায়েরটিকে একটি ছোট রঙনি বৃত্ত হসিবে দেখেয়ে — আলাদা পলি/চপি ব্যবহার না করে, কারণ স্ট্যাটাস চপি ইতমিধ্যই সবচেয়ে বেশি দৃষ্টি আকর্ষণ করে এবং দুটি বিয়াজ থাকলে একনজরে পড়া কঠনি হয়ে যায়।

Widget build(BuildContext context)

প্রায়োরিটি অনুযায়ী রঙ নির্ধারণ করে (switch এক্সপ্রেশন: URGENTdanger, HIGHwarning, NORMALinfo, ডিফল্টmuted) এবং সেই রঙের একটি চিহ্ন পিক্সেলে বৃত্তাকার Container রটারণ করে।

done

ফাইল: lib/features/requests/request_controllers.dart

সংক্ষিপ্ত ববিরণ: সার্ভিস রকি়োস্ট সম্পর্কিত সব Riverpod controller ও provider এখানে আছে — নজিরে রইজ করা রকি়োস্ট, নজিরে কাছে অ্যাসাইন করা রকি়োস্ট, পেন্ডিং অ্যাপ্রুভাল, একটি নির্দিষ্ট রকি়োস্টের ডিটেইল/কমেন্ট/ডকুমেন্ট/হিস্টরি, এবং raise-a-request শাটরে জন্ম সক্রিয় ক্যাটালগ। প্রতিটি controller PagedLoader মক্সিনি ব্যবহার করে পেজেনিটেডে ডটো লোড করে।

ক্লাস/উইজটে: MyRequestsController

AsyncNotifier<PagedState<ServiceRequest>> এক্সটেন্ড করে এবং PagedLoader<ServiceRequest> মক্সিনি যোগ করে — বর্তমান ইউজার যসেব রকি়োস্ট রইজ করেছে তার পেজেনিটেডে তালিকা পরিচালনা করে, এবং নতুন রকি়োস্ট তৈরি বা বাতলি করার পর তালিকা রফ্রেশ করে।

Future<PagedState<ServiceRequest>> build()

currentUserProvider ওয়াচ করে যাত সাইন-ইন/সাইন-আউটে কন্ট্রোলার পুনর্নির্মিত হয় — একই ডিভাইসে দ্বিতীয় অ্যাকাউন্ট যনে প্রথম অ্যাকাউন্টের রকি়োস্ট উত্তরাধিকার সূত্রে না পায়। এরপর loadFirstPage() কল করে প্রথম পেজে লোড করে।

Future<PagedResponse<ServiceRequest>> fetchPage(int page)

requestRepositoryProvider-এর mine(page: page) মেথড কল করে নির্দিষ্ট পেজের ডটো আনে। PagedLoader মক্সিনির প্রয়োজনীয় override।

Future<ServiceRequest> create(CreateServiceRequest request)

রপিজিটরির create কল করে নতুন রকি়োস্ট রইজ করে, তারপর refresh() কল করে পুরো তালিকা পুনরায় লোড করে। রসেপন্স থেকে স্থানীয়ভাবে সরাসরি সামনে না বসিয়ে রলিড করা হয়, কারণ ব্যাকএন্ড ক্রয়িটের সময় status, SLA deadline এবং সম্ভবত workflow stage নির্ধারণ করে — লোকাল তৈরি একটি রো সসেব দখোতে পারবে না পরবর্তী রফ্রেশে পর্যন্ত।

Future<void> cancel(int id)

রপিজিটরির cancel(id) কল করে রকি়োস্টটি বাতলি করে, তারপর refresh() কল করে তালিকা আপডেট করে।

টপ-লভেলে: myRequestsProvider

AsyncNotifierProvider<MyRequestsController, PagedState<ServiceRequest>> — MyRequestsController-এর প্রোভাইডার ইনস্ট্যান্স।

ক্লাস/উইজটে: AssignedRequestsController

AsyncNotifier<PagedState<ServiceRequest>> এক্সটেন্ড করে ও PagedLoader<ServiceRequest> মক্সিনি নেয় — বর্তমান ইউজারের কাছে কাজ করার জন্ম অ্যাসাইন করা রকি়োস্টের পেজেনিটেডে তালিকা পরিচালনা করে।

`Future<PagedState<ServiceRequest>> build()`

`currentUserProvider` ওয়াচ করে (সাইন-ইন/আউটে রবিল্ড) এবং `loadFirstPage()` কল করে প্রথম পজে লোড করে।

`Future<PagedResponse<ServiceRequest>> fetchPage(int page)`

`requestRepositoryProvider`-এর `assignedToMe(page: page)` কল করে নির্দিষ্ট পজে আনে।

টপ-লেভেল: `assignedRequestsProvider`

`AsyncNotifierProvider<AssignedRequestsController, PagedState<ServiceRequest>> —`

`AssignedRequestsController`-এর প্রোভাইডার।

ক্লাস/উইজটে: `ApprovalsController`

`AsyncNotifier<PagedState<StageApproval>>` এক্সটেন্ড করে ও `PagedLoader<StageApproval>` মক্সিনি নেয়ে — বর্তমান ইউজারের সিদ্ধান্তের অপেক্ষায় থাকা workflow stage-গুলোর তালিকা পরিচালনা করে।

`Future<PagedState<StageApproval>> build()`

`currentUserProvider` ওয়াচ করে ও `loadFirstPage()` কল করে প্রথম পজে লোড করে।

`Future<PagedResponse<StageApproval>> fetchPage(int page)`

`requestRepositoryProvider`-এর `pendingApprovals(page: page)` কল করে ডটো আনে।

`Future<void> decide(int id, {required bool approved, String? notes})`

একটি অ্যাপ্রোভাল অনুমোদন বা প্রত্যাখ্যান করে — `approved true` হলে `repo.approve(id, notes: notes)`, নাহলে `repo.reject(id, notes ?? "")` কল করে। এরপর তালিকা থেকে সঙ্গে সঙ্গে আইটেমটি সরিয়ে ফেলে (`removeItem`) কারণ এটি `*pending*`-দরে তালিকা — সিদ্ধান্ত নেওয়া একটি রিও আর এতে থাকার কথা না; সার্বভারের সাথে পরবর্তী রিফ্রেশে মলিয়ে নেওয়া হয়।

টপ-লেভেল: `approvalsProvider`

`AsyncNotifierProvider<ApprovalsController, PagedState<StageApproval>> —`

`ApprovalsController`-এর প্রোভাইডার।

টপ-লেভেল: `requestDetailProvider`

`FutureProvider.autoDispose.family<ServiceRequest, int> —` নির্দিষ্ট আইডি দিয়ে একটি রিকোয়েস্ট আনে (`requestRepositoryProvider.byId(id)`)। অটো-ডিসপোজড, কারণ `status/assignee/comment count` বদলাতে থাকে দেখার বাইরে থাকলে, এবং লম্বা সেশনে প্রতিটি খোলা রিকোয়েস্ট জমতে দেয় না।

টপ-লেভেল: `requestCommentsProvider`

`FutureProvider.autoDispose.family<List<RequestComment>, int> —`

`requestRepositoryProvider.comments(id)` থেকে একটি পজে আনে এবং `createdAt` অনুযায়ী পুরনো থেকে নতুন ক্রমে সাজায়, কারণ একটি কথোপকথন উপর থেকে নিচে পড়া হয় এবং সর্বশেষে রিপ্লাই যখনে পাঠক যাচ্ছে।

টপ-লেভেল: `requestDocumentsProvider`

`FutureProvider.autoDispose.family<List<RequestDocument>, int> —`

`requestRepositoryProvider.documents(id)` থেকে একটি রিকোয়েস্টের ডকুমেন্ট তালিকা আনে।

টপ-লভেলে: requestHistoryProvider

`FutureProvider.autoDispose.family<List<RequestStatusChange>, int> — requestRepositoryProvider.history(id)` থেকে হিস্ট্রি আনবে এবং `changedAt` অনুযায়ী নতুন থেকে পুরনো ক্রমে সাজায়, কারণ ফোনরে স্ক্রিনি বর্তমান অবস্থাই সবার আগে দেখানো উচিত।

টপ-লভেলে: catalogServiceProvider

`FutureProvider<List<CatalogService>> — raise-a-request` শিটের জন্য স্ক্রিনি ক্যাটালগ আনবে। `currentUserProvider` ওয়াচ করে, কারণ ক্যাটালগ একটি স্ট্রেন্থান্ট*-এর সার্ভিসের তালিকা — সাইন-আউট করে অন্য কোম্পানির কারও হিসাবে সাইন-ইন করলে আগের কোম্পানির সার্ভিস দেখানো উচিত না, আর ব্যাকএন্ড সেই আইডগুলো প্রত্যাখ্যান করবে যা একটি ভিঙা অ্যাপের মতো দেখাবে।

ফাইল: lib/features/requests/request_detail_screen.dart

সংক্ষিপ্ত ববিরণ: একটি সার্ভিস রকি়োস্টের বস্তিতারতি স্ক্রিনি — হডোর, ফ্যাক্টস, বরণনা, quotation, ডকুমেন্ট, কমেন্ট এবং হিস্ট্রি টাইমলাইন দেখায়। এই স্ক্রিনি ক্লায়েন্ট ও স্টাফ উভয়ই দেখতে পায়, ফচার-গটেডে অ্যাকশন সহ (rate, edit, withdraw, workflow-এ যাওয়া)।

টপ-লভেলে ফাংশন: void openRequestDetail(BuildContext context, int id)

একটি রকি়োস্ট খোলে — `MaterialPageRoute` দিয়ে `RequestDetailScreen` পুশ করে, path দিয়ে রুট না করে, যাতে যেখান থেকে খোলা হয়েছিল সেই তালিকা তার স্ক্রল পজিশনসহ ঠিকি জায়গায় থাকে।

ক্লাস/উইজটে: RequestDetailScreen

`ConsumerWidget` — একটি নির্দিষ্ট আইডের রকি়োস্ট ডিটাইল দেখানোর মূল স্ক্রিনি। `AppBar`-এ ভূমিকা-ভিত্তিক অ্যাকশন (workflow, rate, edit, withdraw) দেখায় এবং বডতি হডোর, ফ্যাক্টস, বরণনা, quotation, ডকুমেন্ট, কমেন্ট ও টাইমলাইন সেকশন সাজায়।

`Widget build(BuildContext context, WidgetRef ref)`

`requestDetailProvider(id)` ওয়াচ করে ডটো আনবে এবং `canRate` (ক্লায়েন্ট, COMPLETED স্ট্যাটাস), `isStaff` (নন-ক্লায়েন্ট), `canEdit` (ওপনে রকি়োস্ট এবং assign/approve/close অনুমতির যেকোনো একটি আছে) নির্ণয় করে। `AppBar`-এর অ্যাকশনগুলো এসব শর্তে গটেডে: workflow বাটন শুধু স্টাফের জন্য, rate বাটন শুধু ক্লায়েন্টের জন্য যার রকি়োস্ট শেষ হয়েছে, edit বাটন অনুমতিভিত্তিক, এবং withdraw বাটন `canCancel` হলে দেখায়। বডি `async.when` দিয়ে loading/error/data অবস্থা হ্যান্ডল করে এবং data অবস্থায় `RefreshIndicator`-এর ভেতর detail, documents, comments ও history প্রোভাইডার ইনভ্যালিডেট করে রফ্রেশ করে।

`Future<void> _withdraw(BuildContext context, WidgetRef ref, ServiceRequest request)`

নশিচতিকরণ ডায়ালগ দেখায় ("Withdraw this request?"), নশিচতি হলে

`myRequestsProvider.notifier.cancel(request.id)` কল করে, `requestDetailProvider` ইনভ্যালিডেট করে, একটি স্ন্যাকবার দেখায় এবং স্ক্রিনি থেকে ফরি যায় (`navigator.pop()`)। ব্যর্থ হলে (`ApiException`) এরর মেসেজে স্ন্যাকবার দেখায়।

ক্লাস/উইজটে: _Header

রকি়োস্টের শিরোনাম, স্ট্যাটাস চপি, সার্ভিসের নাম, SLA breach ব্যানার এবং টাস্ক progress বার দেখানো একটি `StatelessWidget`।

Widget build(BuildContext context)

AppCard-এর ভেতরে title ও StatusChip সারতি, তারপর ঐচ্ছকভাবে hubServiceName, SLA breach থাকলে MessageBanner.error, এবং taskProgress থাকলে "X of Y done" টেক্সট সহ একটি LinearProgressIndicator রেন্ডার করে।

ক্লাস/উইজটে: _Facts

রকি়োস্টের বসিতারতি ফিল্ড (priority, raised date, SLA deadline, assignee, client, package, price, filing reference, completed date, resubmit count) একটি key-value তালিকায় দেখানো StatelessWidget।

Widget build(BuildContext context)

একটরিকের্ড-টাইপ তালিকা তৈরিকরে সবগুলো সম্ভাব্য ফ্যাক্টরো ধারণ করে, নাল/খালি value বাদ দেয়, এবং বাকি রোগুলো AppCard-এর মধ্যে label-value সারি হিসেবে Divider দিয়ে আলাদা করে রেন্ডার করে।

ক্লাস/উইজটে: _Quotation

রকি়োস্টের উপর থাকা quotation দেখানো read-only StatelessWidget। অ্যাকসপেট/রজিক্ট এখানে করা যায় না, কারণ সসেব ইনভয়সে ও পমেন্টে শুরু করে যা billing flow-এর অংশ।

Widget build(BuildContext context)

AppCard-এ quotation amount, StatusChip, valid-until তারখি, নোটস এবং quotationAwaitsDecision হলে একটি ইনফো ব্যানার দেখায় যা বলে যে অ্যাকসপেট/রজিক্ট করা এখন ওয়বে অ্যাপ থেকে করতে হবে।

ক্লাস/উইজটে: _Documents

রকি়োস্টে সংযুক্ত ডকুমেন্টগুলোর তালিকা এবং নতুন ডকুমেন্ট আপলোডের সুবিধা দেওয়া ConsumerStatefulWidget।

ConsumerState<_Documents> createState()

_DocumentsState ইনস্ট্যান্স তৈরিকরে।

ক্লাস/উইজটে: _DocumentsState

_Documents-এর স্টেটে ক্লাস — আপলোড হচ্ছে কনি তার বুলিয়ান ফ্ল্যাগ রাখে এবং ফাইল যোগ/খোলার লজিকি পরিচালনা করে।

Future<void> _add()

pickAttachment দিয়ে একটি ফাইল বেছে নেয়, তারপর apiClientProvider.uploadDocument দিয়ে আপলোড করে, requestRepositoryProvider.addDocument দিয়ে রেকর্ড করে, এবং requestDocumentsProvider ইনভ্যালিডিটে করে তালিকা রফ্রেশে করে। ব্যর্থতায় স্ন্যাকবারে এরর মেসেজে দেখায়।

Future<void> _open(String url)

url_launcher-এর launchUrl দিয়ে বাইরের অ্যাপে ডকুমেন্ট URL খোলে (LaunchMode.externalApplication); ব্যর্থ হলে স্ন্যাকবারে মেসেজে দেখায়।

Widget build(BuildContext context)

requestDocumentsProvider(widget.id) ওয়াচ করে; আপলোড চলাকালীন লোডিং ইন্ডিকেটর, নাহলে "Add" বাটন দেখায়। ডটো এলো খালি হলে info ব্যানার, নাহলে প্রতিটি ডকুমেন্টের জন্য একটি ListTile দেখায় যা ট্যাপে _open কল করে।

ক্লাস/উইজটে: _Comments

রকিওয়েস্টের কমেন্ট থ্রুডে এবং নতুন কমেন্ট লখোর ইনপুট বক্স দেখানো

ConsumerStatefulWidget।

ConsumerState<_Comments> createState()

_CommentsState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _CommentsState

_Comments-এর স্টেটে — একটি TextEditingController এবং _sending বুলিয়ান ফ্ল্যাগ রাখা।

void dispose()

_controller ডিসপোজ করে; সুপারক্লাসের dispose() কল করে।

Future<void> _send()

টেক্সটফিল্ডের বিষয়বস্তু ট্রিম করে, খালি বা ইতিমধ্যে পাঠানো চলছে হলে ফিরে যায়, নাহলে

requestRepositoryProvider.addComment কল করে, কন্ট্রোলার কলয়ার করে এবং

requestCommentsProvider ইনভ্যালিডেট করে। ব্যর্থতায় এরর দেখায়।

Widget build(BuildContext context)

requestCommentsProvider(widget.id) ওয়াচ করে — লোডিং/এরর/খালি অবস্থার জন্য টেক্সট দেখায়, নাহলে প্রতিটি কমেন্টের জন্য _Comment উইজটে রেন্ডার করে; নচি একটি TextField এবং পাঠানোর IconButton.filled থাকে।

ক্লাস/উইজটে: _Comment

একটি একক কমেন্ট রো দেখানো StatelessWidget — অ্যাভাটার, লেখকের নাম, সময়, INTERNAL ট্যাগ (যদি প্রযোজ্য) এবং কন্টেন্ট।

Widget build(BuildContext context)

একটি Avatar (ইনশিয়ালস সহ), লেখকের নাম, Fmt.relative দিয়ে সময়, comment.isInternal হলে একটি StatusChip('INTERNAL', ...), এবং নচি কমেন্টের বিষয়বস্তু সাজিয়ে রেন্ডার করে। ইন্টারনাল নোট লবেলে করা থাকে যাতো কেউ ভুল করে ক্ল্যায়েন্টকে ফোনে পড়ে না শোনায।

static String _initials(String? name)

নাম থেকে ইনশিয়ালস তৈরি করে — শব্দে ভাগ করে, খালি হলে '?', একটি শব্দ হলে প্রথম অক্ষর, একাধিক শব্দ হলে প্রথম ও শেষ শব্দের প্রথম অক্ষর যুক্ত করে আপারকসে ফেরত দেয়।

ক্লাস/উইজটে: _Timeline

রকিওয়েস্টের status history দেখানো ConsumerWidget।

Widget build(BuildContext context, WidgetRef ref)

requestHistoryProvider(id) ওয়াচ করে; হিস্ট্রি নাল বা খালি হলে SizedBox.shrink() ফেরত দেয় (একজন কর্মীর হিস্ট্রি পড়ার অনুমতি না থাকলে এটা এরর হিসেবে না দেখিয়ে সেকশনটাই বাদ দেয়)। নাহলে

AppCard-এ প্রতিটি পরিবর্তনের জন্য _TimelineRow রেন্ডার করে।

ক্লাস/উইজটে: `_TimelineRow`

একটি একক status change এন্ট্রি — একটি vertical "rail" (রখো ও ডট) সহ দেখানো `StatelessWidget`, যা পরিবর্তনের ক্রম দৃশ্যমান করে।

Widget build(BuildContext context)

`IntrinsicHeight`-এর ভেতর একটি রিলে (উপরে/নিচে লাইন, মাঝে গোলাকার ডট রঙনি `statusColors` থেকে) এবং পাশে old/new status টেক্সট, আপেক্ষিক সময়, পরিবর্তনকারীর নাম ও reason (যদি থাকে) রেন্ডার করে।

ফাইল: `lib/features/requests/request_models.dart`

সংক্ষিপ্ত ববিরণ: সার্ভিস রকি়োস্ট মডেলের কোর DTO ও কনস্ট্রাক্টগুলো — permission code, status enum, `ServiceRequest`, `CatalogService`, request তৈরি/আপডেট DTO, কমেন্ট, স্ট্যাটাস-চেঞ্জ, অ্যাপ্রুভাল, ডকুমেন্ট, প্যাকজে এবং রিভিউ মডেল। এসব ক্লায়েন্ট-সাইড থেকে দেখা রকি়োস্টের প্রতিনিধিত্ব করে।

ক্লাস: `RequestPermissions`

abstract final class — ব্যাকএন্ডের নিজস্ব permission কোড স্ট্রিং ধারণ করা প্লাইন কনস্ট্রাক্টের সংগ্রহ, `GET /api/users/permissions`-কে সোর্স-অব-ট্রুথ ধরে এনাম না বানিয়ে।

স্ট্যাটিক ফলিড: `view`, `approve`, `assign`, `close`, `editAny` (`assign/approve/close`-এর তালিকা — যেকোনো একটি থাকলেই যথেষ্ট), `packageView`, `reviewView`। কোনো মথেন্ড নেই।

ক্লাস: `RequestStatus`

abstract final class — রকি়োস্টের সম্ভাব্য স্ট্যাটাস কোডগুলো ধারণ করে।

স্ট্যাটিক ফলিড: `pending`, `quotationPending`, `assigned`, `inProgress`, `waitingClient`, `underReview`, `completed`, `rejected`, `cancelled`, `resubmitted`, এবং `closed` (`completed/rejected/cancelled`-এর স্টে, যার পরে কিছু ঘটবে না)।

static bool `isOpen(String status)`

প্রদত্ত status যদি `closed` স্টে না থাকে তাহলে `true` ফেরত দেয় — অর্থাৎ রকি়োস্টটি এখনো সক্রিয়/খোলা কনি তা নির্ণয় করে।

টপ-লেভেল: `const requestPriorities = <String>['LOW', 'NORMAL', 'HIGH', 'URGENT']`

সার্ভিস রকি়োস্টের সম্ভাব্য অগ্রাধিকার মানের তালিকা।

ক্লাস: `ServiceRequest`

একটি সার্ভিস রকি়োস্ট (`ServiceRequestResponse`-এর প্রতিলিপি)। quotation ডটো সরাসরি এই রেকর্ডে থাকে, কারণ standalone QuotationController সার্ভার-সাইড থেকে সরানো হয়েছে।

কনস্ট্রাক্টর: `id`, `title`, `status`, `priority`, `createdAt` আবশ্যিক; বাকসিবি (`description`, `agreedPrice`, `slaDeadline`, `slaBreach`, `assignedAt`, `completedAt`, `clientName`, `hubServiceId/Name`, `assignedEmployeeId/Name`, `taskCount`, `completedTaskCount`, `packageName`, `resubmitCount`, `permanentlyClosed`, `invoiceId`, `govRefNumber/Type`,

quotationAmount/Currency/Notes/ValidUntil/Status, aiSummary, updatedAt) ঐচ্ছকি।

bool get isOpen

RequestStatus.isOpen(status) ফেরত দেয়।

bool get canCancel

isOpen && !permanentlyClosed — শুধু একটিখোলা এবং স্থায়ীভাবে বন্ধ-না-হওয়া রকি়োয়স্টে প্রত্যাহারযোগ্য।

bool get hasQuotation

quotationAmount != null ফেরত দেয়।

bool get quotationAwaitsDecision

কো্টেশন আছে এবং (quotationStatus নাল অথবা 'PENDING') হলে true — যে কো্টেশনের এখনো ক্লায়েন্টের সিদ্ধান্ত বাকি।

double? get taskProgress

taskCount <= 0 হলে null (কোনো টাস্ক ভাঙা হয়নি সিন্টো বোঝাতে, ০% অগ্রগতিনিয়), নাহলে completedTaskCount / taskCount (০ থেকে ১-এর মধ্যে ক্ল্যাম্প করা) ফেরত দেয়।

factory ServiceRequest.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে একটি ServiceRequest তৈরি করে, প্রতিটি ফিল্ডের জন্য null-safe কাস্টিং ও ডিফল্ট মান প্রয়োগ করে (helper ফাংশন d ডাবল কনভার্ট করে, i ইন্টজিয়ার কনভার্ট করে)।

ক্লাস: CatalogService

কোম্পানির ক্যাটালগের একটি সার্ভিস, রকি়োয়স্ট রইজ করার সময় ব্যবহৃত হয়।

কনস্ট্রাক্টর: id, name আবশ্যক; description, price, priceType, currency, estimatedDays, defaultPriority, categoryName, requiresQuotation, requiresDocuments ঐচ্ছকি/ডিফল্টসহ।

factory CatalogService.fromJson(Map<String, dynamic> json)

JSON থেকে CatalogService তৈরি করে, প্রতিটি ফিল্ডে null-safe কাস্টিং প্রয়োগ করে।

ক্লাস: CreateServiceRequest

POST /api/service-requests-এর জন্য পলোড। paymentChoice সবসময় PAY_LATER পাঠানো হয় — PAY_NOW ইনভয়েস ও গটেওয়ে রডাইরেক্ট URL ফেরত দেয়, যা ফোনে অসম্পূর্ণ চেকআউট দেখানোর চেষ্টা খারাপ, তাই সিন্টো এড়ানো হয়েছে।

কনস্ট্রাক্টর: title, hubServiceId আবশ্যক; description, priority ঐচ্ছকি।

Map<String, dynamic> toJson()

title ও description ট্রিম করে, খালি বর্ণনা বাদ দেয়, priority থাকলে যোগ করে, এবং সবসময় 'paymentChoice': 'PAY_LATER' যুক্ত করে JSON ম্যাপ তৈরি করে।

ক্লাস: RequestComment

একটি কমেন্ট। ভিজিবিলাটি (PUBLIC/INTERNAL) তৈরির সময় বাদ দেওয়া হয় যাতো ব্যাকএন্ড নজিই role-aware ডিফল্ট প্রয়োগ করতে পারে।

কনস্ট্রাক্টর: id, content, createdAt আবশ্যক; visibility, authorName, authorId, attachmentUrl ঐচ্ছকি।

bool get isInternal

visibility == 'INTERNAL' ফরেনত দায়ে।

factory RequestComment.fromJson(Map<String, dynamic> json)

JSON থকে RequestComment তরৈ কিরে, null-safe কাস্টহিসহ।

ক্লাস: RequestStatusChange

হস্ট্রি টাইমলাইনরে একটি এন্ট্রি — status পরবির্তনরে রকের্ড।

কনস্ট্রাক্টর: id, newStatus, changedAt আবশ্যক; oldStatus, reason, changedByName ঐচ্ছকি।

factory RequestStatusChange.fromJson(Map<String, dynamic> json)

JSON থকে RequestStatusChange তরৈ কিরে, null-safe কাস্টহিসহ।

ক্লাস: StageApproval

কারো সদিধান্তরে অপকেষায় থাকা একটি workflow stage।

কনস্ট্রাক্টর: id, status, serviceRequestId, createdAt আবশ্যক; serviceRequestTitle, workflowStageName, stageOrder, approverRole, requestedByName, decidedByName, decisionNotes, decidedAt ঐচ্ছকি।

bool get isPending

status == 'PENDING' ফরেনত দায়ে।

factory StageApproval.fromJson(Map<String, dynamic> json)

JSON থকে StageApproval তরৈ কিরে, null-safe কাস্টহিসহ।

ক্লাস: RequestDocument

একটি রকি়োয়স্টে সংযুক্ত ফাইল — POST /upload-এর ফলাফল যা POST /service-requests/{id}/documents-এ রকের্ড করা হয়।

কনস্ট্রাক্টর: id, fileName, fileUrl আবশ্যক; uploadedByName, createdAt ঐচ্ছকি।

factory RequestDocument.fromJson(Map<String, dynamic> json)

JSON থকে RequestDocument তরৈ কিরে; fileName না থাকলে 'Attachment' ডফিল্ট ব্যবহার করে।

ক্লাস: UpdateServiceRequestRequest

PATCH /service-requests/{id}-এর জন্ম একটি সিত্যকাররে স্পার্স প্যাচ — প্রতটি ফিল্ড null-চকে করে বসানো হয় যাত বাদ দেওয়া কী স্পর্শ না করে। রকি়োয়স্ট বন্ধ হয়ে গেলে প্রত্যাখ্যাত হয়।

কনস্ট্রাক্টর: সব ফিল্ড (title, description, priority, agreedPrice) ঐচ্ছকি।

factory UpdateServiceRequestRequest.from(ServiceRequest request)

একটি বিদ্যমান ServiceRequest-এর বর্তমান মান দায়ে একটি UpdateServiceRequestRequest তরৈ কিরে, এডটি শটিরে প্র-ফিলিরে জন্ম।

Map<String, dynamic> toJson()

ভতেরে একটি লোকাল clean হলেপার ব্যবহার করে ট্রমি করা/খালি স্ট্রিং নালাে রূপান্তর করে, এবং কবেল অ-নাল, অ-খালি মান JSON-এ যোগ করে।

টপ-লভেলে: `const packageBillingCycles = <String>['MONTHLY', 'QUARTERLY', 'YEARLY', 'ONE_TIME']`

সার্ভিস প্র্যাকজের সম্ভাব্য বলিং সাইকেলের তালিকা।

ক্লাস: `CreateServicePackageRequest`

`POST /packages` ও `PUT /packages/{id}`-এর জন্য পলোড — ক্যাটালগ সার্ভিসের একটি বান্ডলে একটি সাইকেলে বক্রি হয়।

কনস্ট্রাক্টর: `name`, `billingCycle` আবশ্যিক; বাকি (`description`, `packagePrice`, `discountPercent`, `requestQuota`, `deliveryDays`, `serviceIds`, `featured`, `popular`) ঐচ্ছিক/ডফিল্টসহ।

`Map<String, dynamic> toJson()`

`packagePrice`, `discountPercent`, `featured`, `popular` — এই চারটি ফিল্ড সবসময় (unconditionally) পাঠায়, কারণ ব্যাকএন্ড সেগুলোকে null-check ছাড়াই পড়ে (নাল মান override সাফ করা বা ডফিল্ট)। বাকি ফিল্ড শুধু থাকলে যোগ করে।

ক্লাস: `SubmitReviewRequest`

`POST /reviews`-এর পলোড — একটি পিওরটাল ক্লায়েন্ট তাদের রহেজ করা রকিওয়েস্টের রটেং দিচ্ছে। আবার পোস্ট করলে আগের রটেং প্রতস্থাপতি হয়।

কনস্ট্রাক্টর: `serviceRequestId`, `rating` আবশ্যিক; `comment` ঐচ্ছিক।

`Map<String, dynamic> toJson()`

`comment` ট্রমি করে, খালি হলে বাদ দিয়ে, বাকিসব যোগ করে JSON তৈরিকরে।

ক্লাস: `ServiceReview`

একটি ফিরি আসা রটেং রকের্ড।

কনস্ট্রাক্টর: `id`, `rating`, `published` আবশ্যিক; `comment`, `hubServiceId`, `hubServiceName`, `clientName`, `serviceRequestId`, `createdAt` ঐচ্ছিক।

`factory ServiceReview.fromJson(Map<String, dynamic> json)`

JSON থেকে `ServiceReview` তৈরিকরে, null-safe কাস্টিংসহ।

ফাইল: `lib/features/requests/request_repository.dart`

সংক্ষিপ্ত ববিরণ: সার্ভিস রকিওয়েস্ট, তার workflow, tasks, quotation এবং রভিউ-সম্পর্কিত সমস্ত API কল এই একটি রিপিজিটরিক্লাসে কেন্দ্রীভূত। প্রতিটি মেথড `ApiClient` রূপায় করে HTTP কল করে এবং JSON রসপন্সকে মডেলে অবজেক্টে রূপান্তর করে।

ক্লাস/উইজটে: `RequestRepository`

কনস্ট্রাক্টর একটি `ApiClient` নেয়। `_base = '/service-requests'` এবং `_approvals = '/approvals'` — দুটি বেসে পাথ কনস্ট্যান্ট হিসেবে সংরক্ষিত।

Future<PagedResponse<ServiceRequest>> mine({int page = 0, int size = 20})

GET `$_base/my` কল করে বর্তমান ইউজার পোর্টাল ক্লায়েন্ট হিসেবে রইজ করা রকি়োস্টেট আনে।
ব্যাকএন্ড ইউজার আইডি দিয়ে Client রো খোঁজে এবং স্টাফ অ্যাকাউন্টের জন্য 400 "Client profile not found" ফেরত দেয়; তাই 400, forbidden বা not-found ধরনের ব্যর্থতা ক্যাচ করে একটি খালি PagedResponse ফেরত দেয়, বাকি এরর re-throw করে।

Future<PagedResponse<ServiceRequest>> assignedToMe({int page = 0, int size = 20})

GET `$_base/assigned-to-me` কল করে বর্তমান ইউজারের কাছে অ্যাসাইন করা রকি়োস্টেট আনে।

Future<ServiceRequest> byId(int id)

GET `$_base/$id` কল করে একটি নির্দিষ্ট রকি়োস্টেট আনে এবং `ServiceRequest.fromJson` দিয়ে পার্স করে।

Future<ServiceRequest> create(CreateServiceRequest request)

POST `$_base` কল করে একটি নতুন রকি়োস্টেট তৈরি করে।

Future<ServiceRequest> update(int id, UpdateServiceRequest request)

PATCH `$_base/$id` কল করে রকি়োস্টেটের বসিতারতি (status নয়) সংশোধন করে। স্ট্যাটাস পরিবর্তনের নজিস্ব এন্ডপয়েন্ট আছে যা হিস্ট্রিতে রেকর্ড রাখে; এই মথড দিয়ে সেটা পাশ কাটানো হলে হিস্ট্রি এন্ট্রি তৈরি হতো না। রকি়োস্টেট বন্ধ হয়ে গেলে প্রত্যাখ্যাত হয়।

Future<void> cancel(int id)

PATCH `$_base/$id/cancel` কল করে (text/plain রসেপন্স, `patchText` ব্যবহার করে) রকি়োস্টেট বাতিল করে।

Future<PagedResponse<RequestComment>> comments(int id, {int page = 0, int size = 30})

GET `$_base/$id/comments` কল করে কমেন্টের একটি পিজে আনে।

Future<RequestComment> addComment(int id, String content)

POST `$_base/$id/comments` কল করে একটি কমেন্ট যোগ করে। visibility ইচ্ছাকৃতভাবে পাঠানো হয় না — ব্যাকএন্ড role-aware ডফিল্ট (ক্লায়েন্টের কমেন্ট PUBLIC, স্টাফের INTERNAL) প্রয়োগ করে, ভুল অনুমান করলে ইন্টারনাল নোট ফাঁস হতে পারত বা রপ্লাই লুকিয়ে যেতে।

Future<List<RequestStatusChange>> history(int id)

GET `$_base/$id/history` কল করে স্ট্যাটাস টাইমলাইন আনে। ঐচ্ছকি — যে কর্মীর হিস্ট্রি পিড়ার পারমিশন নেই, তার জন্যও বাকি ডিটাইল স্ক্রিনি যেনে কাজ করে, তাই forbidden/not-found ক্যাচ করে খালি তালিকা ফেরত দেয়।

Future<List<RequestDocument>> documents(int requestId)

GET `$_base/$requestId/documents` কল করে সংযুক্ত ডকুমেন্টের তালিকা আনে।

Future<RequestDocument> addDocument(int requestId, {required String fileName, required String fileUrl})

POST `$_base/$requestId/documents` কল করে একটি ফাইলের URL রকি়োস্টেটের বর্নুদ্ধে রেকর্ড করে (ফাইলটি নিজি আগে `ApiClient.uploadDocument` দিয়ে আপলোড করা থাকতে হয়)।

Future<List<CatalogService>> activeServices()

GET `/services/active` কল করে raise-a-request শিটিরে জন্য সক্রিয় ক্যাটালগ আনে।

Future<void> createPackage(CreateServicePackageRequest request)

POST /packages কল করে কোম্পানির ক্যাটালগে একটি প্যাকেজ যোগ করে। শুধু role-gated (COMPANY_OWNER বা EMPLOYEE) — নির্দিষ্ট SERVICE_PACKAGE_CREATE কোড নই, তাই এই ফ্যামিলির একমাত্র permission VIEW কাছাকাছি প্রক্সি হিসেবে ব্যবহার করা হয়।

Future<ServiceReview> submitReview(SubmitReviewRequest request)

POST /reviews কল করে একটি সম্পূর্ণ রকিওয়েস্টের রটেং জমা দেয়। শুধু ক্লায়েন্ট এবং শুধু নিজের রকিওয়েস্টের জন্য। আবার পোস্ট করলে আগের রটেং প্রতস্থাপিত হয়।

Future<PagedResponse<StageApproval>> pendingApprovals({int page = 0, int size = 20})

GET \$_approvals/pending কল করে সন্ধিন্তরে অপেক্ষায় থাকা অ্যাপ্রুভালের পজে আনে।

Future<void> approve(int id, {String? notes})

POST \$_approvals/\$id/approve কল করে অনুমোদন করে। নোট না থাকলে স্টো null হিসেবে না পাঠিয়ে key-টাই বাদ দেওয়া হয়, যাতে ব্যাকএন্ড "not provided" ও "provided as empty" আলাদা করতে পারে।

Future<List<StageApproval>> approvalsForRequest(int requestId)

GET \$_approvals/request/\$requestId কল করে একটি নির্দিষ্ট রকিওয়েস্টের বন্দিধে রেইজ করা সব অ্যাপ্রুভাল আনে (যেকোনো অবস্থায়)। ক্লায়েন্টও এটি দেখতে পায় — তারা জানার অধিকার রাখে যে তাদের রকিওয়েস্ট সাইন-অফের অপেক্ষায় আছে।

Future<void> reject(int id, String notes)

POST \$_approvals/\$id/reject কল করে প্রত্যাখ্যান করে। notes বাধ্যতামূলক — ব্যাকএন্ড এটি দাবি করে, কারণ ব্যাখ্যাহীন প্রত্যাখ্যান কারো কাজে আসে না।

Future<ServiceRequest> summary(int id)

GET \$_base/\$id/summary কল করে রকিওয়েস্টের স্টাফ-ভাউ আনে — byId-এর মতো একই আকারের কনিতু computed (AI সারাংশ ও task count পূরণ করে), শুধু স্টাফের জন্য।

Future<String> draftReply(int id, String roughNotes)

POST \$_base/\$id/draft-reply কল করে অসংগঠিত নোট থেকে একটি খসড়া রপ্লাই তৈরি করে। এটি শুধু খসড়া তৈরি করে, পাঠায় না — মানুষকে পড়ে, সম্পাদনা করে কমেন্ট হিসেবে পোস্ট করতে হয়।

Future<ServiceRequest> changeStatus(int id, String status, {String? reason})

PATCH \$_base/\$id/status কল করে রকিওয়েস্টকে ভিন্ন স্ট্যাটাসে সরায়, হিস্ট্রির জন্য একটি কারণসহ।

Future<ServiceRequest> assignTo(int id, int employeeId)

PATCH \$_base/\$id/assign/\$employeeId কল করে রকিওয়েস্টটি একজন কর্মীর হাতে তুলে দেয় (id পাথে থাকে, বডিতে নয়)।

Future<ServiceRequest> advanceStage(int id)

POST \$_base/\$id/advance-stage কল করে বর্তমান workflow stage সম্পন্ন হিসেবে চহ্নিত করে পরের স্টেজে যায়। পরের স্টেজে সাইন-অফ চাইলে প্রত্যাখ্যাত হয় এবং একই প্রত্যাখ্যানটি অ্যাপ্রুভাল রকিওয়েস্ট তৈরি করে — তাই একটি গটেডে স্টেজে প্রথম প্রচেষ্টা ব্যর্থ হওয়াটাই প্রত্যাশিত।

Future<StageProgress> stageProgress(int id)

GET \$_base/\$id/stage-progress কল করে রকিওয়েস্টটি workflow-এ কতদূর পৌঁছেছে তা আনে। ক্লায়েন্ট ও স্টাফ উভয়ের জন্য উন্মুক্ত।

Future<List<RequestTask>> tasks(int id)

GET `$_base/$id/tasks` কল করে একটি বিয়ের তালিকা (পজে নয়) আনবে।

Future<RequestTask> addTask(int id, TaskRequest request)

POST `$_base/$id/tasks` কল করে একটি নতুন টাস্ক যোগ করে।

Future<RequestTask> updateTask(int id, int taskId, TaskRequest request)

PATCH `$_base/$id/tasks/$taskId` কল করে একটি টাস্ক আপডেট করে।

Future<void> deleteTask(int id, int taskId)

DELETE `$_base/$id/tasks/$taskId` কল করে (plain text রেসপন্স, `deleteText` ব্যবহার করে) একটি টাস্ক মুছে দেয়।

Future<ServiceRequest> submitQuotation(int id, QuotationRequest request)

POST `$_base/$id/quotation` কল করে একটি দাম কোট করে। এটি রিকোয়েস্টকে QUOTATION_PENDING-এ ফলে এবং ক্লায়েন্টকে নোটিফাই করে — এটি নিরব সম্পাদনা নয়।

Future<ServiceRequest> acceptQuotation(int id)

POST `$_base/$id/quotation/accept` কল করে কোটেশন গ্রহণ করে; সাইড-এফেক্ট হিসেবে সম্মত মূল্যের একটি ইনভয়েস তৈরি হয়। শুধু পেন্ডিং কোটেশন গ্রহণ করা যায়; ম্যোদোত্তীর্ণটি প্রত্যাখ্যাত হয় (ব্যাকএন্ড আউটে EXPIRED-এ ফলপি করে)।

Future<ServiceRequest> rejectQuotation(int id, String reason)

POST `$_base/$id/quotation/reject` কল করে কোটেশন প্রত্যাখ্যান করে, একটি কারণ সহ।

Future<PagedResponse<ServiceReview>> reviews({int page = 0, int size = 20})

GET `$_reviews` কল করে এই কোম্পানির জন্য দেওয়া সব রিভিউ আনবে।

Future<PagedResponse<ServiceReview>> reviewsForService(int hubServiceId, {int page = 0, int size = 20})

GET `$_reviews/service/$hubServiceId` কল করে একটি নির্দিষ্ট সার্ভিসের রিভিউ আনবে — ক্যাটালগে একটি সার্ভিসের নচি যা দেখানো হয়, ক্লায়েন্টের জন্যও উন্মুক্ত।

Future<double?> averageRating()

GET `$_reviews/average-rating` কল করে সবকিছু মিলিয়ে কোম্পানির গড় রেটিং আনবে; কিছু রেট করা না থাকলে null (ভুল বোঝানো শূন্য নয়)।

Future<double?> averageRatingForService(int hubServiceId)

GET `$_reviews/service/$hubServiceId/average-rating` কল করে একটি নির্দিষ্ট সার্ভিসের গড় রেটিং আনবে।

Future<ServiceReview> review(int id)

GET `$_reviews/$id` কল করে একটি নির্দিষ্ট রিভিউ আনবে।

Future<void> deleteReview(int id)

DELETE `$_reviews/$id` কল করে একটি রিভিউ স্থায়ীভাবে মুছে দেয়। এটি লুকিয়ে রাখার (`published` ফ্ল্যাগ) কোনো বকিল্প নেই — কোনো এন্ডপয়েন্ট সেই ফ্ল্যাগ সেট করে না।

টপ-লভেলে: requestRepositoryProvider

Provider<RequestRepository> — apiClientProvider থেকে ApiClient নিয়ে একটি RequestRepository ইনস্ট্যান্স তৈরি করে।

ফাইল: lib/features/requests/request_workflow_models.dart

সংক্ষিপ্ত ববিরণ: একটি রিকোয়েস্টের পছন্দে workflow-সম্পর্কিত মডেল — task, stage progress এবং quotation। এটি একটি আলাদা ফাইল কারণ request_models.dart ক্লায়েন্টের দৃষ্টিতে রিকোয়েস্ট থেকে ভিন্ন — এখানকার সবকিছু স্টাফ-সাইড, রিকোয়েস্টকে সরানোর যন্ত্রপাতি।

ক্লাস: TaskStatus

abstract final class — একটি টাস্কের সম্ভাব্য স্ট্যাটাস কোডগুলো।

স্ট্যাটিক ফিল্ড: pending, inProgress, completed, blocked, cancelled, all (সবগুলোর তালিকা), settled (completed/cancelled-এর সেটে — এসবের পরে কিছু ঘটবে না)।

static String label(String status)

switch এক্সপ্রেশন দিয়ে প্রতিটি স্ট্যাটাসের মানবপাঠ্য লেবেল ফেরত দেয় ('Not started', 'In progress', 'Done', 'Blocked', 'Cancelled'); অচেনা মান হলে সেটি নিজস্ব ফেরত দেয়।

ক্লাস: QuotationStatus

abstract final class — একটি কোটেশনের সম্ভাব্য স্ট্যাটাস কোডগুলো।

স্ট্যাটিক ফিল্ড: pending, accepted, rejected, expired।

static String label(String status)

switch এক্সপ্রেশন দিয়ে প্রতিটি স্ট্যাটাসের মানবপাঠ্য লেবেল ফেরত দেয় ('Awaiting a decision', 'Accepted', 'Rejected', 'Expired'); অচেনা মান হলে সেটি নিজস্ব ফেরত দেয়।

ক্লাস: RequestTask

একটি রিকোয়েস্টের অধীনে একটি কাজ (TaskResponse-এর প্রতিফলন)।

কনস্ট্রাক্টর: id, title, status, priority আবশ্যিক; description, dueDate, slaDeadline, completedAt, assignedEmployeeId/Name/Role, estimatedHours, createdByName, workflowStageId/Name, createdAt ঐচ্ছিক। dueDate একটি সাধারণ তারিখ (মোমেন্ট নয়) — ব্যাকএন্ডে LocalDateTime থেকে LocalDate-এ পরিবর্তিত হয়েছে, slaDeadline তার পাশে সত্যিকারের একটি মোমেন্ট।

bool get isSettled

TaskStatus.settled.contains(status) ফেরত দেয়।

factory RequestTask.fromJson(Map<String, dynamic> json)

JSON থেকে RequestTask তৈরি করে, null-safe কাস্টমিংসহ।

ক্লাস: TaskRequest

একটি টাস্ক তৈরি বা পরিবর্তনের পলোড। ব্যাকএন্ডে TaskServiceImpl.updateTask প্রতিটি ফিল্ড null-চেক করে, তাই এই পলোড আংশিক ভাবে key বাদ দিতে পারে।

কনস্ট্রাক্টর: সব ফিল্ড (title, description, status, priority, dueDate, assignedEmployeeId, estimatedHours, workflowStageId) ঐচ্ছক। workflowStageId কেবল create-এ মান্য হয় — UpdateTaskRequest-এ এমন ফিল্ড নেই, তাই তৈরির পরে একটি টাস্ককে স্টেজে বদল করা যায় না। assignedEmployeeId: null মান্নে "unchanged" — কাউকে অ্যানঅ্যাসাইন করার কোনো উপায় নেই, শুধু অন্য কারো কাছে সরানো যায়।

Map<String, dynamic> toJson()

কেবল অ-নাল ফিল্ডগুলো JSON-এ যোগ করে (আংশিক প্যাচ তৈরি করে)।

ক্লাস: StageStep

একটি রিকোয়েস্ট যে workflow-এর মধ্য দিয়ে যাচ্ছে তার একটি ধাপ।

কনস্ট্রাক্টর: stageId, name, stageOrder, completed, current আবশ্যক; slaHours, requiresApproval, approvalStatus, requiresPayment, paymentPercent ঐচ্ছক/ডফল্টসহ। approvalStatus শুধু সাইন-অফের প্রয়োজন এমন স্টেজে এবং কাউ চাওয়ার পরই সেটে থাকে; একটি গিটেডে স্টেজে অগ্রসর হলে সাইড-এফেক্ট হিসেবে অ্যাপ্রুভাল রিকোয়েস্ট তৈরি হয় এবং তারপর প্রত্যাখ্যাত হয়, তাই প্রথম প্রচেষ্টা সবসময় ব্যর্থ হয়।

factory StageStep.fromJson(Map<String, dynamic> json)

JSON থেকে StageStep তৈরি করে, null-safe কাস্টিংসহ।

ক্লাস: StageProgress

একটি রিকোয়েস্টের সম্পূর্ণ workflow অগ্রগতি — কতগুলো স্টেজে, কতটা সম্পন্ন এবং প্রতটি স্টেজেরে বসিতারতি।

কনস্ট্রাক্টর: serviceRequestId, currentStage, totalStages, stages আবশ্যক। currentStage একটি ইনডেক্স নয়, সম্পন্ন স্টেজের একটি সংখ্যা — পাঁচটির মধ্যে তিনটি সম্পন্ন হলে এটি 3 এবং বর্তমান কাজ চলা স্টেজে stages[3]।

bool get finished

totalStages > 0 && currentStage >= totalStages ফেরত দেয়।

double get fraction

totalStages == 0 হলে 0, নাহলে currentStage / totalStages ফেরত দেয় — progress bar-এর জন্য একটি ভগ্নাংশ।

StageStep? get inFlight

stages-এর মধ্যে current == true এমন প্রথম স্টেজে খুঁজে ফেরত দেয়; না পলে currentStage < stages.length হলে stages[currentStage] ফেরত দেয়, নাহলে null (সব স্টেজে শেষে হলে)।

factory StageProgress.fromJson(Map<String, dynamic> json)

JSON থেকে StageProgress তৈরি করে, stages তালিকাটি StageStep.fromJson দিয়ে ম্যাপ করে।

ক্লাস: QuotationRequest

একটি রিকোয়েস্টের জন্য দাম কোট করার পলোড। currency ছাড়া প্রতটি ফিল্ড কোনো null-guard ছাড়াই সরাসরি কের্ডে লেখা হয়, তাই বাদ দেওয়া amount/note/expiry সাফ হয়ে যায় এবং স্ট্যাটাস জোর করে PENDING-এ ফেরানো হয়। পুনরায় কোট করা মান্নে একটি নতুন কোট, সম্পাদনা নয়।

কনস্ট্রাক্টর: `amount`, `currency` আবশ্যিক; `notes`, `validUntil` ঐচ্ছিক। `validUntil` একটি `LocalDateTime`, তাই খালি `yyyy-MM-dd` প্রত্যাখ্যাত হয় — শটি একটি তারিখ নিয়ে এবং সেই দিনের শেষে মিনিটে সেট করে দেয়।

`Map<String, dynamic> toJson()`

সবগুলো ফিল্ড (এমনকি নাল হলও) সবসময় যোগ করে JSON তৈরি করে — কোনো conditional অন্তর্ভুক্তি নেই।

ফাইল: `lib/features/requests/request_workflow_screen.dart`

সংক্ষিপ্ত ববিরণ: স্টাফের "back office" স্ক্রিনি যখনে একটি রিকোয়েস্ট workflow-এর মধ্য দিয়ে সরানো, টাসকে ভাগ করা, হস্তান্তর করা এবং দাম কোট করা হয়। ক্লায়েন্ট এই স্ক্রিনি দেখে না — এর প্রতিটি এন্ডপয়েন্ট `hasAnyRole('COMPANY_OWNER', 'EMPLOYEE')`।

টপ-লেভেল: `stageProgressProvider`

`FutureProvider.autoDispose.family<StageProgress, int> —`

`requestRepositoryProvider.stageProgress(id)` কল করে একটি রিকোয়েস্টের workflow অগ্রগতি আনবে।

টপ-লেভেল: `requestApprovalsProvider`

`FutureProvider.autoDispose.family<List<StageApproval>, int> —`

`requestRepositoryProvider.approvalsForRequest(id)` কল করে একটি রিকোয়েস্টের বর্ধিত রইজ করা সাইন-অফ আনবে।

টপ-লেভেল: `requestTasksProvider`

`FutureProvider.autoDispose.family<List<RequestTask>, int> — requestRepositoryProvider.tasks(id)`

কল করে একটি রিকোয়েস্টের অধীনের কাজ আনবে।

ক্লাস/উইজটে: `RequestWorkflowScreen`

`ConsumerWidget` — একটি রিকোয়েস্টের workflow ব্যবস্থাপনার মূল স্ক্রিনি। `_WhoAndWhat`, `_Stages`, `_Quotation`, `_Tasks` সেকশনগুলো `ListView`-এ সাজায়, এবং `AppBar`-এ "Draft a reply" বাটন (রিকোয়েস্ট খোলা থাকলে) দেখায়।

`static void open(BuildContext context, {required int id})`

`MaterialPageRoute` দিয়ে `RequestWorkflowScreen` পুশ করে।

`Widget build(BuildContext context, WidgetRef ref)`

`requestDetailProvider(id)` ওয়াচ করে ডেটা আনবে। `async.when` দিয়ে `loading/error/data` হ্যান্ডল করে; `data` অবস্থায় `RefreshIndicator`-এর ভেতর `detail`, `stageProgress`, `approvals` ও `tasks` প্রোভাইডার ইনভ্যালিডেট করে রিফ্রেশ করে এবং চারটি সেকশন উইজটে রেন্ডার করে।

ক্লাস/উইজটে: `_WhoAndWhat`

রিকোয়েস্টের শরীোনাম, স্ট্যাটাস, বর্তমান assignee এবং "Hand it over" / "Change status" অ্যাকশন দেখানো `ConsumerStatefulWidget`।

ConsumerState<_WhoAndWhat> createState()

_WhoAndWhatState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _WhoAndWhatState

_WhoAndWhat-এর স্টেটে — _busy বুলিয়ান ফ্ল্যাগ রাখা এবং assign/status-change অ্যাকশনের লজিক ধারণ করে।

Future<void> _run(Future<ServiceRequest> Function(RequestRepository) action, String done)

_busy = true স্টেট করে, প্রদত্ত অ্যাকশন কল করে, detail ও stageProgress প্রভাইডার ইনভ্যালিডিটে করে, সাফল্যে সন্যাকবার দেখায়। ApiException বা সাধারণ ব্যর্থতায় এরর মেসেজ দেখায়, শেষে _busy = false করে।

Future<void> _handOver()

EmployeePicker.show দিয়ে একজন কর্মী বেছে নিয়ে, তারপর _run কল করে repo.assignTo(id, person.id) চালায়।

Future<void> _changeStatus()

pickOne দিয়ে _settableStatuses-এর মধ্য থেকে একটি নতুন স্ট্যাটাস বেছে নতি বলে, তারপর askForText দিয়ে একটি ঐচ্ছিক কারণ জিজ্ঞাসে করে, এবং _run কল করে repo.changeStatus(id, status, reason: reason) চালায়।

Widget build(BuildContext context)

রকি়োয়েস্টের শিরোনাম ও StatusChip, assignee-এর নাম (কাউকে না দিলে সতর্কতা রঙে "Nobody has it yet"), এবং রকি়োয়েস্ট খোলা থাকলে LoadingButton ("Hand it over") ও OutlinedButton ("Change status") রেন্ডার করে।

টপ-লেভেলে: const _settableStatuses

হাত স্টেট করা যায় এমন স্ট্যাটাসের তালিকা (pending, assigned, inProgress, waitingClient, underReview, completed, rejected)। QUOTATION_PENDING, RESUBMITTED, CANCELLED বাদ দেওয়া হয়েছে কারণ এসব যথাক্রমে কোট করা, ক্লায়েন্ট পুনরায় জমা দেওয়া বা প্রত্যাহারের নজিস্ব এন্ডপয়েন্ট দিয়ে স্টেট হয়।

ক্লাস/উইজটে: _Stages

একটি রকি়োয়েস্টের workflow স্টেজগুলোর অগ্রগতি ও "Finish [stage]" অ্যাকশন দেখানো

ConsumerStatefulWidget।

ConsumerState<_Stages> createState()

_StagesState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _StagesState

_Stages-এর স্টেটে — _busy ফ্ল্যাগ রাখা এবং স্টেজে অগ্রসর করার লজিক ধারণ করে।

Future<void> _advance(StageStep next)

confirmAction দিয়ে নশ্চিতিকরণ চাষ (গটেডে স্টজে হলে ভিন্ন মসেজে দেখায়), নশ্চিতি হলে requestRepositoryProvider.advanceStage(id) কল করে, detail/stageProgress/approvals প্রোভাইডার ইনভ্যালিডেট করে। সাফল্যে "Moved on." সন্ধ্যাকবাব। ApiException ব্য়ব্হতায (যা একটি গটেডে স্টজে প্রত্যাশতি) stageProgress ও approvals ইনভ্যালিডেট করে সেই এরর মসেজে দেখায়, কারণ সটোই সদ্য তরৈইওয়া অ্যাপ্ৰুভাল রকি়োস্টে নরিদশে করে।

Widget build(BuildContext context)

stageProgressProvider(requestId) ওয়াচ করে; স্টজে খালি হলে ব্যাখ্যামূলক টকেস্ট দেখায়, নাহলে progress bar, প্রতটি স্টজে রে জন্য _StageRow, _Approvals উইজেটে, এবং রকি়োস্টে খোলা ও next স্টজে থাকলে একটি "Finish [name]" LoadingButton রনেডার করে।

ক্লাস/উইজেটে: _Approvals

একটি রকি়োস্টে বরিদ্ধে রেইজ করা সাইন-অফরে সংক্ষপ্ত তালকা দেখানো ConsumerWidget। সিদ্ধান্ত নেওয়া হয় আলাদা approvals স্ক্রিনি — এটি শুধু অবস্থা জানায়।

Widget build(BuildContext context, WidgetRef ref)

requestApprovalsProvider(requestId)-এর মান ওয়াচ করে; নাল বা খালি হলে SizedBox.shrink() ফরেত দিয়ে। নাহলে প্রতটি অ্যাপ্ৰুভালে রে জন্য স্টজে রে নাম, রোল/অনুরোধকারী/সিদ্ধান্তকারী নাম ও আপেক্ষকি সময়, ঐচ্ছকি decision notes, এবং একটি StatusChip রনেডার করে।

ক্লাস/উইজেটে: _StageRow

একটি একক workflow স্টজে রো — আইকন, নাম এবং ফুটনোট (SLA hours, approval status, payment info) দেখানো StatelessWidget।

Widget build(BuildContext context)

স্টজে রে completed/current/অন্য অবস্থা অনুযায়ী একটি আইকন ও রঙ নরিণয় করে, একটি notes তালকা তরৈকিরে (SLA hours, approval অবস্থা, payment ডিটেইল), এবং সগেলো স্টজে রে নামে সাথে রনেডার করে।

ক্লাস/উইজেটে: showQuotationSheet (টপ-লভেলে ফাংশন)

Future<void> showQuotationSheet(BuildContext context, {required ServiceRequest request}) — showModalBottomSheet দিয়ে _QuotationSheet খোলো। প্রতবার সবগুলো ফল্ড পাঠায় কারণ ব্যাকএন্ড নাল-চকে ছাড়াই সরাসরি রিকের্ডে লখে, তাই একটি নোট বাদ দেওয়া re-quote পুরনো নোটটি নীরবে মুছে দতি।

ক্লাস/উইজেটে: _Quotation

রকি়োস্টে রে quotation তথ্য এবং staff-side অ্যাকশন (accept/turn down/quote-again) দেখানো ConsumerStatefulWidget।

ConsumerState<_Quotation> createState()

_QuotationState ইনস্ট্যান্স তরৈকিরে।

ক্লাস/উইজেটে: _QuotationState

_Quotation-এর স্টেটে — _busy ফল্যাগ রাখে এবং accept/reject/re-quote অ্যাকশনের লজকি ধারণ করে।

Future<void> _run(Future<ServiceRequest> Function(RequestRepository) action, String done)
_busy = true সটে করে, প্রদত্ত অ্যাকশন কল করে, requestDetailProvider ইনভ্যালিডেটে করে, সাফল্যে
সন্ধ্যাকবার দেখায়; ব্যর্থতায় এরর মসেজে দেখায়; শেষে _busy = false করে।

Future<void> _reject()

askForText দিয়ে (required, destructive) একটি প্রত্যাখ্যানের কারণ চায়, তারপর _run কল করে
repo.rejectQuotation(id, reason) চালায়।

Widget build(BuildContext context)

currentUserProvider ওয়াচ করে ভূমিকা নির্ণয় করে; canDecide = quotation PENDING এবং ইউজার
CLIENT বা COMPANY_OWNER (কর্মী নজিরে কোট নজিরে অ্যাকসপ্ট করতে পারে না)। কোট না
থাকলে ব্যাখ্যামূলক টেক্সট, থাকলে amount/currency/valid-until/notes দেখায়, ময়োদোতীর্ণ হলে
সতর্কতা ব্যানার। canDecide হলে Accept/Turn down বাটন, এবং রকি়োস্টে খোলা ও অ্যাকসপ্টেডে
না হলে "Quote (again) for this" বাটন দেখায় যা showQuotationSheet খোলে।

ক্লাস/উইজটে: _QuotationSheet

দাম কোট করার ফর্ম শিট — ConsumerStatefulWidget।

ConsumerState<_QuotationSheet> createState()

_QuotationSheetState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _QuotationSheetState

_QuotationSheet-এর স্টেটে — amount/currency/notes-এর জন্য TextEditingController, একটি
_validUntil তারিখ, ফর্ম কী, এরর ও busy ফ্ল্যাগ রাখতে; বদ্যমান কোটেশনের মান দিয়ে প্র-ফিলি করা।

void dispose()

_amount, _currency, _notes কন্ট্রোলার ডিসপোজ করে; সুপাররে dispose() কল করে।

Future<void> _submit()

ফর্ম ভ্যালিডেটে করে, requestRepositoryProvider.submitQuotation কল করে একটি QuotationRequest
পাঠায় (validUntil-কে T23:59:00 যোগ করে দিনের শেষে মুহূর্তে রূপান্তর করে), সফল হলে
requestDetailProvider ইনভ্যালিডেটে করে শিট বন্ধ করে। ব্যর্থতায় এরর মসেজে সটে করে।

Widget build(BuildContext context)

FormSheetFrame-এর ভেতর amount/currency ইনপুট (ভ্যালিডেশনসহ), DateField, notes টেক্সটফিল্ড
এবং একটি সতর্কতা টেক্সট রেন্ডার করে যা ব্যাখ্যা করে যে পাঠানো মাত্র রকি়োস্টে পুনরায়
সদ্বিন্তরে অপেক্ষায় ফরিতে যায় এবং ক্লায়েন্টকে জানানো হয়।

ক্লাস/উইজটে: _Tasks

রকি়োস্টের অধীনের কাজের তালিকা এবং নতুন যোগ করার বাটন দেখানো ConsumerWidget।

Widget build(BuildContext context, WidgetRef ref)

requestTasksProvider(request.id) ওয়াচ করে; রকি়োস্টে খোলা থাকলে "Add" বাটন দেখায় যা
showTaskSheet খোলে। তালিকা খালি হলে ব্যাখ্যামূলক টেক্সট, নাহলে প্রতিটি টাস্কের জন্য _TaskRow
রেন্ডার করে।

ক্লাস/উইজটে: _TaskRow

একটি একক টাস্ক রো — আইকন, শিরোনাম, ফ্যাক্টস (assignee, due date, hours, stage) এবং একটি পপআপ মেনু (status/assign/edit/delete) দেখানো ConsumerStatefulWidget।

ConsumerState<_TaskRow> createState()

_TaskRowState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _TaskRowState

_TaskRow-এর স্টেটে — _busy ফ্ল্যাগ রাখা এবং status/assign/delete অ্যাকশনের লজিক ধারণ করে।

Future<void> _run(Future<void> Function(RequestRepository) action, String done)

_busy = true সেট করে, প্রদত্ত অ্যাকশন কল করে, requestTasksProvider ও requestDetailProvider ইনভ্যালিডেট করে, সাফল্যে সন্ধ্যাকবার দেখায়; ব্যর্থতায় এরর দেখায়; শেষে _busy = false করে।

Future<void> _setStatus()

pickOne দিয়ে TaskStatus.all-এর মধ্য থেকে একটি নতুন স্ট্যাটাস বেছে নতি বলে, তারপর _run কল করে repo.updateTask(...) দিয়ে স্ট্যাটাস আপডেট করে।

Future<void> _handOver()

EmployeePicker.show দিয়ে একজন কর্মী বেছে নেয়, তারপর _run কল করে repo.updateTask(...) দিয়ে assignedEmployeeId আপডেট করে।

Future<void> _delete()

confirmAction দিয়ে নিশ্চিতকরণ চায় ("এটা স্থায়ীভাবে চলে যাবে, বাতলি করলে রেকর্ড থাকবে"), নিশ্চিতি হলে _run কল করে repo.deleteTask(...) চালায়।

Widget build(BuildContext context)

টাস্কের স্ট্যাটাস অনুযায়ী আইকন ও রঙ নির্ণয় করে, facts তালিকা (assignee, due date, hours, stage) তৈরি করে, শিরোনাম ও ফ্যাক্টস রেন্ডার করে; _busy হলে লোডিং ইন্ডিকেটর, নাহলে widget.editable হলে একটি PopupMenuButton (status/assign/edit/delete অপশনসহ) দেখায়।

ক্লাস/উইজটে: showTaskSheet (টপ-লভেলে ফাংশন)

Future<void> showTaskSheet(BuildContext context, {required int requestId, RequestTask? task}) — showModalBottomSheet দিয়ে _TaskSheet খোলো। task নাল হলে নতুন টাস্ক তৈরি হয়, নাহলে বর্তমান টাস্ক এডাট হয়। এই সম্ভে পথটি শিখু যা বদলছে তা পাঠাতে পারে, কারণ updateTask প্রতিটি ফলিড null-চকে করে।

ক্লাস/উইজটে: _TaskSheet

একটি টাস্ক তৈরি বা এডাট করার ফর্ম শিট — ConsumerStatefulWidget।

ConsumerState<_TaskSheet> createState()

_TaskSheetState ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: _TaskSheetState

_TaskSheet-এর স্টেটে — title/description/hours-এর জন্য TextEditingController, priority ও dueDate স্টেটে, ফর্ম কী, এরর ও busy ফ্ল্যাগ রাখা; বর্তমান টাস্কের মান দিয়ে প্র-ফলি করা।

`void dispose()`

`_title`, `_description`, `_hours` কন্ট্রোলার ডিসপোজ কর; সুপাররে `dispose()` কল কর।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে, একটি `TaskRequest` তৈরি করে, `widget.task` নাল হলে `repo.addTask` নাহলে `repo.updateTask` কল করে। সফল হলে `requestTasksProvider` ও `requestDetailProvider` ইনভ্যালিডেট করে শিট বন্ধ করে। ব্যর্থতায় এরর মैसेজে স্টেট করে।

`Widget build(BuildContext context)`

`FormSheetFrame`-এর ভেতর `title`, `description`, `priority` ড্রপডাউন, `due date` (`DateField`) ও `estimated hours` ইনপুট রেন্ডার করে; এডিট মোডে একটি নোট দেখায় যে `assignee` ও স্ট্যাটাস এখান থেকে না, টাস্ক মনে থেকে বদলাতে হয়।

ক্লাস/উইজটে: `showDraftReplySheet` (টপ-লেভেলে ফাংশন)

`Future<void> showDraftReplySheet(BuildContext context, {required int id})` —

`showModalBottomSheet` দিয়ে `_DraftReplySheet` খোল। এআই-সহায়তায় ব্লক নোট থেকে খসড়া রপ্লাই তৈরি করে; শুধু খসড়া তৈরি করে, নজি থেকে ক্ল্যানেটকে কিছু পাঠায় না।

ক্লাস/উইজটে: `_DraftReplySheet`

ব্লক নোট থেকে একটি খসড়া রপ্লাই তৈরি ও কমেন্ট হিসেবে পোস্ট করার ফর্ম শিট —

`ConsumerStatefulWidget`।

`ConsumerState<_DraftReplySheet> createState()`

`_DraftReplySheetState` ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: `_DraftReplySheetState`

`_DraftReplySheet`-এর স্টেটে — একটি `_notes` কন্ট্রোলার, `_draft` (জেনারেটে হওয়া টেক্সট), ফর্ম কী, এরর ও `busy` ফ্ল্যাগ রাখে।

`void dispose()`

`_notes` কন্ট্রোলার ডিসপোজ কর; সুপাররে `dispose()` কল কর।

`Future<void> _draftIt()`

ফর্ম ভ্যালিডেট করে, `requestRepositoryProvider.draftReply(id, notes)` কল করে জেনারেটে হওয়া রপ্লাই `_draft`-এ সংরক্ষণ করে। ব্যর্থতায় এরর মैसेজে স্টেট করে।

`Future<void> _post()`

`requestRepositoryProvider.addComment(id, _draft!)` কল করে খসড়াটিকে কমেন্ট হিসেবে পোস্ট করে, `requestCommentsProvider` ইনভ্যালিডেট করে শিট বন্ধ করে। ব্যর্থতায় এরর মैसेজে স্টেট করে।

`Widget build(BuildContext context)`

`FormSheetFrame`-এর ভেতর ব্লক নোটে একটি টেক্সটফিল্ড রেন্ডার করে; বাটনের অ্যাকশন `_draft == null` হলে `_draftIt`, নাহলে `_post`। খসড়া তৈরি হলে স্টেটে একটি `SelectableText`-এ দেখায় সহ একটি সতর্কতা যে পোস্ট করার আগে পড়তে হবে, কারণ পোস্ট করলে স্টেট ব্যবহারকারীর নামে কমেন্ট হিসেবে যুক্ত হয়ে যাবে।

ফাইল: lib/features/requests/requests_screen.dart

সংক্ষিপ্ত ববিরণ: সারভিস রকি়োয়স্টের প্রধান স্ক্রিনি — পারমিশন অনুযায়ী "Assigned", "Raised by me" (শুধু ক্লায়েন্ট) এবং "Approvals" (শুধু অ্যাপ্রুভাল-পারমিশনধারী) ট্যাব দেখায়। এতে নতুন প্যাকজে তৈরিও নতুন রকি়োয়স্ট রইজ করার অ্যাকশনও আছে।

ক্লাস/উইজটে: RequestsScreen

`ConsumerWidget`, `initialTabLabel` নিয়ে নির্দিষ্ট ট্যাবে খোলা যায়। পারমিশন লোড না হওয়া পর্যন্ত লোডার দেখায় (খালি স্টে মানই "না" নয়, বরং "এখনও জানা নাই" — তাই ভুলভাবে অ্যাকসেসে প্রত্যাখ্যান দেখায় না), `view` পারমিশন না থাকলে লক করা এম্পটি-স্টেটে দেখায়। ব্যাকএন্ড যেকোন পারমিশন যাচাই করে স্টোর সাথে মিলিয়ে গটে করা হয়েছে, যাতে দুটো কখনো ভিন্ন সন্ধিধান্তে না যায়। "Raised by me" ট্যাবটি শুধু ক্লায়েন্টদের জন্য দেখানো হয় কারণ সংশ্লিষ্ট এন্ডপয়েন্ট স্টাফ অ্যাকাউন্টে 400 এরর দিয়ে। অ্যাপবারে প্যাকজে-পারমিশনধারীদের জন্য "New package" আইকন বাটন এবং সবার জন্য "New" ফ্লোটিং অ্যাকশন বাটন (রকি়োয়স্ট রইজ করার শিট খোলে) থাকে।

`RequestsScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর; ঐচ্ছকি `initialTabLabel` দিয়ে নির্দিষ্ট ট্যাব লবেলে দিয়ে স্ক্রিনি খোলা যায় (যেমন ডিপি-লিংক থাকে)।

`Widget build(BuildContext context, WidgetRef ref)`

পারমিশন কন্ট্রোলার ও বর্তমান ইউজার watch করে `view/approve` পারমিশন ও `isClient` নির্ণয় করে। এরপর উপযুক্ত ট্যাবের তালিকা (`Assigned`, শর্তসাপেক্ষে `Raised by me/Approvals`) তৈরি করে, `initialTabLabel` মিলিয়ে প্রাথমিক ইনডেক্স ঠিকি করে, এবং `DefaultTabController` সহ `Scaffold` রটাইন্ করতে যাতে অ্যাপবার, ট্যাববার, প্যাকজে-বাটন ও "New" FAB থাকে। প্রতিটি অবস্থার (লোডিং/আনঅথরাইজড/সাধারণ) জন্য আলাদা `Scaffold` রটাইন্ করা হয়।

ক্লাস/উইজটে: _MyRequestsTab

`ConsumerWidget`; ক্লায়েন্ট নজি যেসব রকি়োয়স্ট রইজ করেছে তার পজেড লিস্ট দেখায়।

`Widget build(BuildContext context, WidgetRef ref)`

`myRequestsProvider` থেকে কন্ট্রোলার নিয়ে `PagedList<ServiceRequest>` রন্ডার করে — রফ্রেশ, লোড-মোর (গার্ডেড), খালি-অবস্থা ও এরর মেসেজে স্টে করে, এবং প্রতিটি আইটেমে ট্যাপ করলে `openRequestDetail` কল করে বসিতারতি স্ক্রিনি নিয়ে যায়।

ক্লাস/উইজটে: _AssignedTab

`ConsumerWidget`; বর্তমান ইউজারকে অ্যাসাইন করা রকি়োয়স্টগুলোর পজেড লিস্ট দেখায় — স্টাফদের জন্য এটাই ডিফল্ট/প্রথম ট্যাব।

`Widget build(BuildContext context, WidgetRef ref)`

`assignedRequestsProvider` থেকে কন্ট্রোলার নিয়ে `PagedList<ServiceRequest>` রন্ডার করে, একই ধরনের রফ্রেশ/লোড-মোর/এম্পটি/এরর কনফিগারেশন সহ, এবং ট্যাপে `openRequestDetail` কল করে।

ফাইল: lib/features/requests/reviews_screen.dart

সংক্ষিপ্ত ববিরণ: ক্লায়েন্টেরা সম্পন্ন হওয়া রকি়োয়েস্টে যে রভিউ/রটেং দিচ্ছে তার তালিকা ও গড় রটেং দেখানোর স্ক্রিনি। রভিউ শুধু পড়া ও ডলিটি করা যায় — সম্পাদনা করা যায় না, কারণ এন্ডপয়ন্টে শুধুই ক্লায়েন্টেরে নিজেরে রকি়োয়েস্টেরে সাথে কী-করা।

ক্লাস/উইজটে: ReviewServiceFilterController

Notifier<int?>; রভিউ লস্টিকে কোন হাব-সার্ভিসে সীমাবদ্ধ করা হয়েছে তা রাখতে (null মানেরে সবগুলো)।

int? build()

প্রাথমিক অবস্থা null (কোনো ফিল্টার না) রটার্ন করে।

void set(int? hubServiceId)

স্টেটে পরিবর্তন করে নতুন hubServiceId (বা null) সেটে করে, যা ফিল্টার প্রোভাইডারদেরে রফ্রেশ করে।

টপ-লেভেলে প্রোভাইডার

- reviewServiceFilterProvider: ReviewServiceFilterController-এর NotifierProvider। - reviewsProvider: FutureProvider.autoDispose<List<ServiceReview>> — বর্তমান ফিল্টার অনুযায়ী repo.reviews(size: 50) বা repo.reviewsForService(...) কল করে সর্বোচ্চ ৫০টি রভিউ আনবে। - averageRatingProvider: FutureProvider.autoDispose<double?> — বর্তমান ফিল্টার অনুযায়ী repo.averageRating() বা repo.averageRatingForService(...) কল করে গড় রটেং আনবে।

ক্লাস/উইজটে: ReviewsScreen

ConsumerWidget; পুরো রভিউ স্ক্রিনেরে প্রধান কাঠামো — গড় রটেং হডোর, সার্ভিস-ফিল্টার বার এবং ConfigList-ভিত্তিক রভিউ কার্ড তালিকা দেখায়।

Widget build(BuildContext context, WidgetRef ref)

ক্যাটালগ সার্ভিস তালিকা ও নির্বাচতি ফিল্টার watch করে; ConfigList<ServiceReview>-এর মধ্যে হডোর হিসেবে _Average উইজটে ও (সার্ভিস থাকলে) FilterBar বসায়, যেখানে ফিল্টার-ভ্যালু স্ট্রেং হিসেবে পাস হয় ও ফিরে int.tryParse দিয়ে পার্স হয়। onRefresh-এ রভিউ ও গড়-রটেং উভয় প্রোভাইডার ইনভ্যালিডেটে করে।

ক্লাস/উইজটে: _Average

ConsumerWidget; বর্তমানে দেখানো তালিকার গড় রটেং একটি কার্ডে দেখায়।

Widget build(BuildContext context, WidgetRef ref)

averageRatingProvider watch করে লোডিং/এরর/ডটো তনি অবস্থা হ্যান্ডল করে — ডটো থাকলে সংখ্যাসূচক গড় (এক দশমিক স্থান), _Stars ও ব্যাখ্যামূলক টেক্সট দেখায়; null হলে "কিছু রটে করা হয়নি" বার্তা দেখায়।

ক্লাস/উইজটে: _Stars

StatelessWidget; একটি পূর্ণসংখ্যা রটেং (১-৫) কে ৫টি তারা আইকনেরে সার হিসেবে রেন্ডার করে।

Widget build(BuildContext context)

১ থেকে ৫ পর্যন্ত লুপ করে প্রতিটি ইনডেক্স rating-এর চেয়ে ছোট বা সমান হলে ভেরাট তারা, নাহলে ফাঁকা তারা আইকন দেখায়, রঙ ভেরাট অবস্থায় bos.warning আর নাহলে bos.muted।

ক্লাস/উইজটে: _ReviewCard

ConsumerStatefulWidget; একটি একক রিভিউ (তারা, সার্ভিস নাম, কমেন্ট, ক্লায়েন্ট নাম, সময়, ডলিটি মনে) দেখায়।

ক্লাস/উইজটে: _ReviewCardState

ConsumerState<_ReviewCard>; ডলিটি অ্যাকশনের বজি-স্টেটে ও লজিক রাখা।

Future<void> _delete()

confirmAction দিয়ে নিশ্চিতকরণ ডায়ালগ দেখায় (রিভিউ চরিতরে মুছে যায় এবং যতক্ষণ থাকে ততক্ষণ গড়ে যুক্ত থাকে সে সতর্কবার্তা সহ); নিশ্চিতি হলে `_busy = true` স্টেটে করে `deleteReview` কল করে, সফল হলে `reviewsProvider` ও `averageRatingProvider` ইনভ্যালিডিটে করে সন্ধ্যাকবার দেখায়; `ApiException` বা অন্য এরর হলে যথাক্রমে সার্ভার-মসেজে বা জনেরেকি এরর সন্ধ্যাকবার দেখায়; শেষে `_busy = false`।

Widget build(BuildContext context)

AppCard-এর ভেতর তারা রটেং, সার্ভিস নাম, বজি-হলে স্পনির নাহলে ডলিটি `PopupMenuButton`, কমেন্ট (থাকলে), এবং ক্লায়েন্ট নাম/সময়/"not shown publicly" (unpublished হলে) দিয়ে গঠিত ফুটার-লাইন দেখায়।

ফাইল: lib/features/salary/salary_models.dart

সংক্ষিপ্ত বিবরণ: বতেন কাঠামো (salary structure), লোন/অ্যাডভান্স এবং বতেন-উপাদান (component) সংক্রান্ত সব মডেলে ক্লাস, রকি-স্টেট DTO ও পারমিশন কোড এখানে সংজ্ঞায়িত। কোনো মডেল নই এমন সাধারণ ডটো-ক্লাস ও কনস্ট্রাক্ট তালিকাও আছে।

টপ-লেভেলে/ক্লাস: SalaryPermissions

abstract final class; বতেন-সংক্রান্ত তিনটি পারমিশন কোড কনস্ট্রাক্ট রাখা — `view` (`SALARY_STRUCTURE_VIEW`), `create` (`SALARY_STRUCTURE_CREATE`, স্ট্রাকচার ও কম্পোনেন্ট উভয়ই তৈরি/এডিট করার ক্ষমতা), `delete` (`SALARY_STRUCTURE_DELETE`)। কোনো মডেল নই।

ক্লাস/উইজটে: SalaryStructure

একজন কর্মীর একটি নির্দিষ্ট সময়ের জন্য বতেন কাঠামো (গ্রস, বেসিক, বিভিন্ন এলাউন্স ও ডিডাকশন) ধারণ করে। `effectiveTo` স্টেট থাকলে বোঝায় এটি ইতিহাসিক/সুপারসডিডে রেকর্ড।

bool get isCurrent

`effectiveTo == null` হলে `true` রিটার্ন করে — অর্থাৎ এটিই বর্তমান কার্যকর কাঠামো, একমাত্র যখন এডিট করা যায়।

double get allowances

হাউস রেন্ট, মডেলিং, ট্রান্সপোর্ট, ফুড ও স্পেশাল এলাউন্স (প্রতিটি null হলে ০ ধরে) যোগ করে মোট এলাউন্স রিটার্ন করে।

double get deductions

প্রভিডেন্ট ফান্ড ও ট্যাক্স ডিডাকশন (null হলে ০) যোগ করে মোট ডিডাকশন রিটার্ন করে।

double get unallocated

গ্রস থেকে বেসেকি ও মোট এলাউন্স বাদ দিয়ে অবশিষ্ট (হাতে-লখো মান পরস্পরবিরোধী হলে দেখানোর জন্য) রিটার্ন করে।

bool get addsUp

unallocated-এর পরম মান ০.০১-এর কম হলে true — অর্থাৎ, হিসাব মিলে যায় কনি।

factory SalaryStructure.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে সব ফিল্ড পার্স করে SalaryStructure তৈরি করে; সংখ্যাসূচক ফিল্ডে num?.toInt()/toDouble() ও ?? 0/?? 0 ব্যবহার করে ফলব্যাংক দিয়ে, বাকি ফিল্ড সরাসরি as String? কাস্ট করে।

ক্লাস/উইজটে: SalaryStructureRequest

স্ট্রাকচার তৈরি/আপডেট করার POST/PUT বডি। সব সাতটি এলাউন্স/ডিকশন ফিল্ড নন-নালবেল (ডিফল্ট ০), কারণ সারভার অনুপস্থিতি মানকে শূন্য হিসেবে ধরে নেয়, তাই আংশিক পাঠানো মান বাকিগুলো মুছে যাওয়া।

factory SalaryStructureRequest.from(SalaryStructure structure)

একটি বিদ্যমান SalaryStructure থেকে ফর্ম প্র-ফিলি করার জন্য রকিওয়েস্ট অবজেক্ট তৈরি করে, সব ঐচ্ছিক সংখ্যাসূচক ফিল্ড ?? 0 দিয়ে ডিফল্ট করে।

Map<String, dynamic> toJson()

সব ফিল্ড (কর্মী আইডি, কার্যকর তারিখ, গ্রস, বেসেকি, সাতটি এলাউন্স/ডিকশন সবসময় সবগুলো) ম্যাপে রাখে; notes ট্রিম করে খালি হলে যুক্ত করে।

টপ-লেভেল/enum: loanTypes

const List<String> — ['LOAN', 'ADVANCE'], লোন বনাম অ্যাডভান্সের ধরন নির্দেশ করে।

ক্লাস/উইজটে: LoanStatus

abstract final class; লোনের স্ট্যাটাস কনস্ট্যান্ট রাখে — active (ACTIVE), closed (CLOSED), cancelled (CANCELLED)। কোনো মথেন্ড নেই।

ক্লাস/উইজটে: LoanAdvance

একটি লোন বা অ্যাডভান্স রেকর্ড — মূলধন, মাসিক কিস্তি, অবশিষ্ট ব্যালেন্স ও স্ট্যাটাস ধারণ করে।

bool get isActive

status == LoanStatus.active হলে true।

double get repaid

principalAmount - remainingBalance — এ পর্যন্ত ফেরত পাওয়া অর্থ।

double? get repaidShare

মূলধন ০ বা কম হলে null; নাহলে repaid / principalAmountকে ০ থেকে ১-এর মধ্যে ক্ল্যাম্প করে রিটার্ন করে — প্রগ্রসে বার আঁকার জন্য।

int? get monthsRemaining

লোন সক্রয় না থাকলে বা মাসিক কিস্তি ০ বা কম হলে null; নাহলে remainingBalance / monthlyInstallment-কে উপরে দিকে রাউন্ড করে বাকমাসের সংখ্যা রটাইন করে (শেষ কিস্তি সাধারণত ছোট হয় বলে ceil ব্যবহার)।

factory LoanAdvance.fromJson(Map<String, dynamic> json)

JSON থেকে সব ফিল্ড পার্স করে LoanAdvance তৈরি করে, অনুপস্থিতি type/status-এর জন্য যথাক্রমে 'LOAN'/LoanStatus.active ডিফল্ট রাখে।

ক্লাস/উইজটে: CreateLoanRequest

লোন তৈরি POST বডি কোনো আপডেট এন্ডপয়েন্ট নই — ভুল হলে লোন বাতলি করে নতুন করে এন্ট্রি করতে হয়।

Map<String, dynamic> toJson()

অভ্যন্তরীণ clean(String?) হলেপার দিয়ে reason/notes ট্রিমি করে খালি হলে বাদ দেয়; আবশ্যিক ফিল্ড (employeeId, type, principalAmount, disbursedDate, monthlyInstallment) সবসময় পাঠায়।

ক্লাস/উইজটে: LoanRepayment

একটি নির্দিষ্ট মাসের পরে লোন থেকে আদায়কৃত একটি কিস্তি রিকের্ড।

factory LoanRepayment.fromJson(Map<String, dynamic> json)

JSON থেকে id, amount, payMonth, payYear, paidDate, balanceAfter পার্স করে LoanRepayment তৈরি করে।

টপ-লেভেলে কনস্ট্যান্ট: componentTypes, calculationTypes

componentTypes: ['EARNING', 'DEDUCTION', 'EMPLOYER_CONTRIBUTION'] — কম্পোনেন্টের ধরন।

calculationTypes: ['FIXED', 'PERCENTAGE'] — কীভাবে হিসাব হয়।

ক্লাস/উইজটে: SalaryComponent

একটি পুনঃব্যবহারযোগ্য বতেন লাইন-আইটেম (বোনাস, লভে, এমপ্লয়ার কন্ট্রিবিউশন ইত্যাদি)।

bool get isEarning

type == 'EARNING' হলে true।

factory SalaryComponent.fromJson(Map<String, dynamic> json)

JSON থেকে id, name, type (ডিফল্ট 'EARNING'), calculationType (ডিফল্ট 'FIXED'), taxable (ডিফল্ট true), active (ডিফল্ট true), sortOrder পার্স করে অবজেক্ট তৈরি করে।

ক্লাস/উইজটে: SalaryComponentRequest

কম্পোনেন্ট তৈরি/আপডেট করার POST/PUT বডি — এন্ডপয়েন্ট পুরো SalaryComponent এন্টিটি নিয়ে বলে কোম্পানি আইডি ব্যতীত বাকি সব ফিল্ড পাঠানো হয়, আপডেটে সব ফিল্ড শর্তহীনভাবে ওভাররাইট হয়।

factory SalaryComponentRequest.from(SalaryComponent component)

বর্তমান কম্পোনেন্ট থেকে সব ফিল্ড কপি করে রিকোয়েস্ট অবজেক্ট তৈরি করে।

Map<String, dynamic> toJson()

name ট্রিম করে এবং type, calculationType, taxable, active, sortOrder সহ একটি ম্যাপ রিটার্ন করে।

ফাইল: lib/features/salary/salary_repository.dart

সংক্ষিপ্ত ববিরণ: বতেন কাঠামো, লোন/অ্যাডভান্স, বতেন-উপাদান ক্যাটালগ, স্ট্রাকচার টেমপ্লেটে ও এক্সট্রা-কম্পোনেন্টের জন্য সব API কল এবং সংশ্লিষ্ট Riverpod প্রোভাইডার/কন্ট্রোলার এখানে সংজ্ঞায়িত।

ক্লাস/উইজটে: SalaryRepository

ApiClient মোডানো — বতেন সংক্রান্ত সব HTTP এন্ডপয়েন্ট কল করার একক জায়গা। তনিট বিসে পাথ কনস্ট্যান্ট (_structures, _loans, _components) হিসিবে রাখা।

Future<PagedResponse<SalaryStructure>> structures({int page = 0, int size = 30})

GET /hr/salary-structures পজেনিটেডে কল করে SalaryStructure-এর পজেড রসেপন্স রিটার্ন করে।

Future<List<SalaryStructure>> historyFor(int employeeId)

GET /hr/salary-structures/employee/{id}/history কল করে একজন কর্মীর সব (নতুন-থেকে-পুরনো ক্রমে) স্ট্রাকচার একটা লিস্ট হিসিবে রিটার্ন করে; whereType দিয়ে ম্যাপ-টাইপ আইটেমে ফিল্টার করে পার্স করে।

Future<SalaryStructure> createStructure(SalaryStructureRequest request)

POST /hr/salary-structures কল করে নতুন স্ট্রাকচার তৈরিকরে; আগরে বদ্যমান স্ট্রাকচার থাকলে তা সুপারসডি হয় (ব্যাকএন্ড লজিক)।

Future<SalaryStructure> updateStructure(int id, SalaryStructureRequest request)

PUT /hr/salary-structures/{id} কল করে বর্তমান স্ট্রাকচার এডিট করে; সুপারসডিডে স্ট্রাকচারে কল করলে ব্যাকএন্ড প্রত্যাখ্যান করে।

Future<void> deleteStructure(int id)

DELETE /hr/salary-structures/{id} কল করে; এন্ডপয়েন্ট প্লাইন-টেক্সট রসেপন্স দেয় বলে deleteText ব্যবহৃত হয়।

Future<List<LoanAdvance>> loans()

GET /hr/loans কল করে কোম্পানির সব বকয়ো লোন/অ্যাডভান্সের বয়ের লিস্ট রিটার্ন করে।

Future<LoanAdvance> createLoan(CreateLoanRequest request)

POST /hr/loans কল করে নতুন লোন/অ্যাডভান্স তৈরিকরে।

Future<List<LoanRepayment>> repayments(int id)

GET /hr/loans/{id}/repayments কল করে একটা লোনের বিপরীতে পরেোল থেকে আদায়কৃত সব কিস্তির লিস্ট আনে।

Future<LoanAdvance> cancelLoan(int id)

POST /hr/loans/{id}/cancel কল করে লোনের ভবিষ্যৎ আদায় বন্ধ করে (কোনো এডিট এন্ডপয়েন্ট নই, তাই বাতলি-ও-পুনঃএন্ট্রি পদ্ধতি)।

Future<List<SalaryComponent>> components()

GET /hr/salary-components কল করে সব কম্পোনেন্ট আনবে।

Future<SalaryComponent> createComponent(SalaryComponentRequest request)

POST /hr/salary-components কল করে নতুন কম্পোনেন্ট তৈরি করে।

Future<SalaryComponent> updateComponent(int id, SalaryComponentRequest request)

PUT /hr/salary-components/{id} কল করে বর্তমান কম্পোনেন্ট আপডেট করে।

Future<List<SalaryTemplate>> templates()

GET /hr/salary-components/templates কল করে গ্রুডেভিত্তিক বতেন-রসেপি (টেমপ্লেট) লিস্ট আনবে; নতুন কোম্পানিতে প্রথম রডিরে স্ট্যান্ডার্ড ক্যাটালগ সডি হয়, তাই খালি রিজোল্ট মানকে কডে মুছে ফেলেছে।

Future<SalaryTemplate> createTemplate(SalaryTemplateRequest request)

POST /hr/salary-components/templates কল করে নতুন টেমপ্লেট তৈরি করে।

Future<SalaryTemplate> updateTemplate(int id, SalaryTemplateRequest request)

PUT /hr/salary-components/templates/{id} কল করে টেমপ্লেট আপডেট করে; এটি প্রকৃত PUT — প্রতিটি ফিল্ড পাঠানো মান দিয়ে ওভাররাইট হয়, অনুপস্থিতি মান শূন্য লেখা হয়।

Future<void> deleteTemplate(int id)

DELETE /hr/salary-components/templates/{id} কল করে টেমপ্লেট মুছে দেয়।

Future<SalaryBreakdown> breakdown(int templateId, double gross)

GET /hr/salary-components/templates/{templateId}/breakdown?gross=... কল করে একটি নির্দিষ্ট গ্রস অঙ্কে টেমপ্লেটটিকে ভাঙে দেয় তা আনবে; gross একটি আবশ্যিক কুয়েরি প্যারামিটার।

Future<List<StructureExtra>> extras(int structureId)

GET /hr/salary-components/structure/{structureId} কল করে একটি স্ট্রাকচারে যুক্ত অতিরিক্ত কম্পোনেন্টের লিস্ট আনবে।

Future<List<StructureExtra>> setExtras(int structureId, List<StructureExtraLine> lines)

PUT /hr/salary-components/structure/{structureId} কল করে একটি স্ট্রাকচারের এক্সট্রা কম্পোনেন্টসমূহ সম্পূর্ণভাবে প্রতিস্থাপন করে — ব্যাকএন্ড বর্তমানগুলো মুছে ঠিক এই লিস্টটি লেখে, তাই টিকে থাকা উচিত এমন প্রতিটি লাইন এতে থাকতে হবে।

টপ-লেভেলে প্রোভাইডার: salaryRepositoryProvider

Provider<SalaryRepository> — apiClientProvider watch করে SalaryRepository ইনস্ট্যান্স তৈরি করে দেয়।

ক্লাস/উইজটে: SalaryStructuresController

AsyncNotifier<List<SalaryStructure>>; স্ট্রাকচার তালিকার স্টেটে ও রিফ্রেশ/আপডেট লজিক পরিচালনা করে।

Future<List<SalaryStructure>> build()

currentUserProvider watch করে (ইউজার পরিবর্তনে রিবিল্ড হওয়ার জন্য) load() কল করে প্রাথমিক ডটো রিটার্ন করে।

`Future<List<SalaryStructure>> _load()`

রপিয়োজটিরথিকে `structures()` কল করে পজেরে `content` রটার্ন করে।

`Future<void> refresh()`

`AsyncValue.guard(_load)` দয়ি পুনরায় ডটো আনে ও স্টটে আপডটে করে।

`void apply(SalaryStructure updated)`

বর্তমান লস্টে `updated.id` মলো রো থাকলে সটেকি প্রতস্থাপন করে নতুন `AsyncValue.data` সটে করে। তালকা `null` হলে কছুই করে না। (মন্তব্য: `create`-এর পর এটি বিবহার করা হয় না, কারণ সুপারসডিডে আগরে রো-ও বদলে যায় বলে পুরো তালকা রলিড করা হয়।)

`void remove(int id)`

বর্তমান লস্ট থেকে দেওয়া `id`-এর রো বাদ দয়ি নতুন `AsyncValue.data` সটে করে।

টপ-লভেলে প্রোভাইডার: `salaryStructuresProvider`

`AsyncNotifierProvider<SalaryStructuresController, List<SalaryStructure>> |`

ক্লাস/উইজটে: `LoansController`

`AsyncNotifier<List<LoanAdvance>>`; লোন তালকার স্টটে পরচালনা করে।

`Future<List<LoanAdvance>> build()`

`currentUserProvider` watch করে রপিয়োজটির `loans()` কল করে প্রাথমিক লস্ট রটার্ন করে।

`Future<void> refresh()`

রপিয়োজটির `loans()` পুনরায় কল করে `AsyncValue.guard` দয়ি স্টটে আপডটে করে।

`void apply(LoanAdvance updated)`

বর্তমান লস্টে মলি যাওয়া `id`-এর লোন রো প্রতস্থাপন করে নতুন স্টটে সটে করে (যেমন `cancel`-এর পর)।

টপ-লভেলে প্রোভাইডার: `loansProvider`

`AsyncNotifierProvider<LoansController, List<LoanAdvance>> |`

ক্লাস/উইজটে: `SalaryComponentsController`

`AsyncNotifier<List<SalaryComponent>>`; কম্পোনেন্ট ক্যাটালগরে স্টটে পরচালনা করে।

`Future<List<SalaryComponent>> build()`

`currentUserProvider` watch করে রপিয়োজটির `components()` কল করে প্রাথমিক লস্ট রটার্ন করে।

`Future<void> refresh()`

রপিয়োজটির `components()` পুনরায় কল করে স্টটে আপডটে করে।

`void apply(SalaryComponent updated)`

বর্তমান লস্টে মলি যাওয়া `id`-এর কম্পোনেন্ট রো প্রতস্থাপন করে নতুন স্টটে সটে করে।

টপ-লভেলে প্রোভাইডার: `salaryComponentsProvider`

`AsyncNotifierProvider<SalaryComponentsController, List<SalaryComponent>> |`

টপ-লভেলে প্রোভাইডার: structureHistoryProvider

`FutureProvider.autoDispose.family<List<SalaryStructure>, int>` — একজন কর্মীর (employeeId দিয়ে) সব স্ট্রাকচারের হিস্ট্রি ডিউইল স্ক্রিনিংর জন্য আনবে।

টপ-লভেলে প্রোভাইডার: repaymentsProvider

`FutureProvider.autoDispose.family<List<LoanRepayment>, int>` — একটি নির্দিষ্ট লোনের (loanId দিয়ে) আদায়কৃত কিস্তির লিস্ট আনবে।

টপ-লভেলে প্রোভাইডার: salaryTemplatesProvider

`FutureProvider.autoDispose<List<SalaryTemplate>>` — `currentUserProvider` watch করে বতেন-রসেপি টেমপ্লেটে তালিকা আনবে।

টপ-লভেলে প্রোভাইডার: structureExtrasProvider

`FutureProvider.autoDispose.family<List<StructureExtra>, int>` — একটি নির্দিষ্ট স্ট্রাকচারের (structureId দিয়ে) এক্সট্রা-কম্পোনেন্ট আনবে।

ফাইল: lib/features/salary/salary_screen.dart

সংক্ষিপ্ত ববিরণ: বতেন-সংক্রান্ত মূল স্ক্রিনিং — চারটি ট্যাব (Structures, Loans, Components, Templates) নিয়ে গঠিত, প্রতিটির নিজস্ব ডেটা তালিকা, এডিট/ডিলিট/ক্যান্সলে অ্যাকশন এবং প্রক্বেসাপট-নব্বির "New" ফ্লোটিং বাটন থাকবে।

ক্লাস/উইজটে: SalaryScreen

`ConsumerWidget`; পারমিশন-গেটেডে প্রধান স্ক্রিনিং — লোড না হওয়া পর্যন্ত লোডার, view/create কনোটেই না থাকলে লক করা এম্পটি-স্টেটে, নাহলে চার-ট্যাব `DefaultTabController` UI দেখায়। বর্তমান ট্যাব অনুযায়ী FAB-এর অ্যাকশন বদলায় (`AnimatedBuilder` দিয়ে ট্যাব-কন্ট্রোলার watch করে)।

`SalaryScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর; ঐচ্ছক `initialTabLabel` দিয়ে নির্দিষ্ট ট্যাব দিয়ে শুরু করা যায়।

`Widget build(BuildContext context, WidgetRef ref)`

পারমিশন watch করে view/create যাচাই করে; `canWrite` (create পারমিশন) অনুযায়ী প্রতিটি ট্যাবে `create` কলব্যাক (নাল হলে FAB লুকাই) সেটে করে চার-ট্যাবে রেকর্ড-তালিকা বানায়ে; `initialTabLabel` মিলিয়ে প্রাথমিক ইনডেক্স ঠিক করে; `Builder+AnimatedBuilder` দিয়ে সক্রিয় ট্যাবে `create` কলব্যাক অনুযায়ী FAB আপডেট করে `Scaffold` রিটার্ন করে।

ক্লাস/উইজটে: _StructuresTab

`ConsumerStatefulWidget`; বতেন কাঠামোর তালিকা ট্যাব।

ক্লাস/উইজটে: _StructuresTabState

`ConsumerState<_StructuresTab>`; ডিলিট-বজি স্টেটে ও ডিলিট লজিক রাখবে।

Future<void> _delete(SalaryStructure row)

confirmAction দিয়ে নশ্চিতিকরণ চায় — বার্তা রো isCurrent কনি তার উপর নর্ভর করে বদলায় (বর্তমান কাঠামো মুছলে কর্মী পরোল-রান বাদ পড়বে, ঐতিহাসিকি হলে শুধু ইতিহাস হারায়)। নশ্চিতি হলে _busyId সটে করে deleteStructure কল করে, সফল হলে কন্ট্রোলার থেকে remove করে ও structureHistoryProvider ইনভ্যালিডেটে করে সন্ধ্যাকবার দেখায়; এরর হ্যান্ডলিং ApiException/জেনেরিকি দুই ভাগে; শেষে _busyId = null।

Widget build(BuildContext context)

পারমশিন (create/delete) watch করে ConfigList<SalaryStructure> রনেডার করে — প্রতটিরি-তে নাম, isCurrent অনুযায়ী active/inactive স্টাইল, গ্রস/বসেকি/কার্যকর-তারখি সাবটাইটলে, addsUp না হলে "figures disagree" ট্রাইলিং লবেলে দেখায়; শুধু বর্তমান কাঠামোতে এডটি বাটন সক্রিয় থাকে (কারণ ব্যাকএন্ড অন্যগুলোতে প্রত্যাখ্যান করে); অ্যাকশন হিসিবে History (সবসময়), "Supersede with new pay" (canWrite), Remove (canDelete) থাকে।

ক্লাস/উইজটে: StructureHistoryScreen

ConsumerWidget; একজন কর্মীর সব (নতুন-থেকে-পুরনো) স্ট্রাকচার হিস্ট্রি দেখানোর পূরণ স্ক্রিনি।

StructureHistoryScreen({super.key, required this.employeeId, required this.name})

কনস্ট্রাক্টর — কর্মী আইডিও নাম নিয়ে।

static void open(BuildContext context, {required int employeeId, required String name})

MaterialPageRoute দিয়ে StructureHistoryScreen পুশ করে নভেগিটে করার সুবিধাজনক স্ট্যাটিকি মথেড।

Widget build(BuildContext context, WidgetRef ref)

structureHistoryProvider(employeeId) watch করে ConfigList<SalaryStructure> রনেডার করে, প্রতটি আইটেমে _StructureCard দিয়ে দেখায়; খালি হলে "No pay on file" বার্তা দেখায়।

ক্লাস/উইজটে: _StructureCard

StatelessWidget; হিস্ট্রি লিস্টে একটা স্ট্রাকচারে বসিতারতি কার্ড — তারখি-রঞ্জে, "In force" চপি, এক্সট্রা-কম্পোনেন্ট বাটন, গ্রস/বসেকি/এলাউন্স/ডডিকশন/নেটে লাইন, হিসাব না মলিলে সতর্কবার্তা, নোট।

Widget build(BuildContext context)

AppCard-এর ভতের তারখি-রঞ্জে শরিনোম (একক তারখি যদি এখনও কার্যকর, নাহলে "from — to"), কার্যকর হলে StatusChip('ACTIVE', label: 'In force'), StructureExtrasScreen.open থোলার আইকন বাটন, এরপর _Line দিয়ে Gross/Basic/(শর্তসাপেক্ষ) Allowances/Deductions/Net সারি, addsUp মথিয়া হলে অসামঞ্জস্য বার্তা, এবং নোট (থাকলে) সাজিয়ে দেখায়।

ক্লাস/উইজটে: _Line

StatelessWidget; একটা লবেলে-ও-অর্থ জোড়া সারি হিসিবে দেখায় (যেমন "Gross: ট...")।

_Line(this.label, this.amount, {this.bold = false})

কনস্ট্রাক্টর — লবেলে, অর্থমূল্য ও ঐচ্ছকি বোল্ড ফ্ল্যাগ নিয়ে।

Widget build(BuildContext context)

একটা Row-এ বাম পাশে expand হওয়া লবেলে, ডান পাশে Fmt.money(amount) দেখায়; bold সত্য হলে ভারী ফন্ট-ওয়েটে ব্যবহার করে।

ক্লাস/উইজটে: _LoansTab

ConsumerStatefulWidget; লোন/অ্যাডভান্স তালিকা ট্যাব।

ক্লাস/উইজটে: _LoansTabState

ConsumerState<_LoansTab>; ক্যান্সলে-বজিস্টিটে ও ক্যান্সলে লজকি রাখা।

Future<void> _cancel(LoanAdvance loan)

confirmAction দিয়ে নিশ্চিতকরণ চায় — বাকি ব্যালেন্স থাকলে সতর্ক করে যে এটি বাকি নিওয়া বন্ধ করে, ব্যালেন্স মওকুফ বা আদায় করে না; নই হলে শুধু রকেট কলোজ হয় বলে। নিশ্চিত হলে _busyId স্টে করে cancelLoan কল করে, সফল হলে loansProvider কন্ট্রোলারে apply করে সন্ধ্যাকবার দেখায়; এরর হ্যান্ডলিং একইভাবে দুই ভাগে; শেষে _busyId = null।

Widget build(BuildContext context)

create পারমিশন watch করে ConfigList<LoanAdvance> রেন্ডার করে, প্রতটি আইটেমে _LoanCard-এ দেখায়, ক্যান্সলে কলব্যাক শুধু canWrite && loan.isActive হলে পাস করে।

ক্লাস/উইজটে: _LoanCard

ConsumerWidget; একটি লোন/অ্যাডভান্স রকেটের কার্ড — নাম, ধরন, মূলধন/কিস্তি, স্ট্যাটাস চপি, প্রগ্রসে বার, "Repayments"/"Stop recovering" মেনু, বাকি-ব্যালেন্স ও ময়োদ তথ্য।

_LoanCard({required this.loan, required this.busy, this.onCancel})

কনস্ট্রাক্টর — লোন ডটো, বজি-ফ্ল্যাগ ও ঐচ্ছকি ক্যান্সলে কলব্যাক নিয়ে।

Widget build(BuildContext context, WidgetRef ref)

loan.repaidShare ও loan.monthsRemaining নিয়ে AppCard সাজায় — নাম/ধরন/অঙ্ক, StatusChip, বজি হলে স্পিনার নাহলে PopupMenuButton (Repayments সবসময়, Stop recovering যদি onCancel != null), repaid থাকলে LinearProgressIndicator, এবং শেষে বাকি-ব্যালেন্স/ময়োদ/বতিরণ-তারিখ যুক্ত ফুটার-লাইন দেখায়।

ক্লাস/উইজটে: RepaymentsScreen

ConsumerWidget; একটি নির্দিষ্ট লোনের বপিরীতে পরেোল থেকে প্রকৃত আদায়কৃত কিস্তির তালিকা দেখানোর পূরণ স্ক্রিনি।

RepaymentsScreen({super.key, required this.loan})

কনস্ট্রাক্টর — সম্পূর্ণ LoanAdvance অবজেক্ট নিয়ে (শরিনো নামে নাম দেখানোর জন্য)।

static void open(BuildContext context, {required LoanAdvance loan})

MaterialPageRoute দিয়ে RepaymentsScreen পুশ করে নভিগেটে করে।

Widget build(BuildContext context, WidgetRef ref)

repaymentsProvider(loan.id) watch করে ConfigList<LoanRepayment> রেন্ডার করে; প্রতটি রো-তে মাস/বছর (বা তারিখ, উভয় না থাকলে) শরিনো নাম, বাকি-থাকা-ব্যালেন্স সাবটাইটলে ও কিস্তির অর্থমূল্য ট্রাইলিং-লেবেলে হিসাবে দেখায়।

ক্লাস/উইজটে: _ComponentsTab

ConsumerWidget; বতেন-উপাদান ক্যাটালগ তালিকা ট্যাব।

Widget build(BuildContext context, WidgetRef ref)

create পারমিশন watch করে ConfigList<SalaryComponent> রেন্ডার করে — প্রতিটি রো নাম, active/inactive স্টাইল, ধরন/হিসাব-পদ্ধতি/taxable-না-হলে সাবটাইটলে হিসাবে দেখায়; canWrite হলে এডটি বাটনে showComponentSheet(context, existing: row) কল করে।

ফাইল: lib/features/salary/salary_sheets.dart

সংক্ষিপ্ত বিবরণ: বতেন কাঠামো, লোন/অ্যাডভান্স ও বতেন-উপাদান তৈরি/সম্পাদনার জন্য তিনটি মডাল বটম-শিট ফর্ম এবং তাদের সহায়ক উইজেট এখানে সংজ্ঞায়িত।

টপ-লেভেলে ফাংশন: Future<void> showStructureSheet(BuildContext context, {SalaryStructure? existing, int? employeeId, String? employeeName})

বতেন কাঠামো সটে/এডটি করার মডাল বটম-শিট খোলে। existing দিলে সটে এডটি করে শুধু বর্তমান কাঠামোতে অনুমোদিত; employeeId দিলে ঐ কর্মীর জন্য নতুন কাঠামো তৈরি করে যা আগেরটি সুপারসডি করে; দুটোই না দিলে ফর্মে প্রথমে কর্মী বেছে নতিবে।

ক্লাস/উইজেট: _StructureSheet

ConsumerStatefulWidget; বতেন কাঠামো তৈরি/এডটি ফর্মের কন্ট্রোলার — existing/employeeId/employeeName ঐচ্ছকি প্যারামিটার নিয়ে।

ক্লাস/উইজেট: _StructureSheetState

ConsumerState<_StructureSheet>; ফর্মের সব টেক্সট কন্ট্রোলার, তারখি, কর্মী-নির্বাচন ও সাবমিট-লজিক পরিচালনা করে।

bool get _isEdit

widget.existing != null হলে true — ফর্ম এডটি নাকি তৈরিমোডে আছে তা নির্দেশ করে।

static TextEditingController _money(double? value)

একটিনিম্ন-সফে প্লেন-নাম্বার টেক্সট কন্ট্রোলার তৈরি করে (মুদ্রা-চহ্ন ছাড়া, যাতো পার্স-ব্যাক করা যায়); মান null বা 0 হলে খালি স্ট্রিং, পূর্ণসংখ্যা হলে দশমিক ছাড়া, নাহলে সরাসরি স্ট্রিং রূপান্তর ব্যবহার করে।

void dispose()

সব দশটি টেক্সট কন্ট্রোলার (_gross থেকে _notes পর্যন্ত) লুপ করে dispose() কল করে, তারপর super.dispose() কল করে।

double _read(TextEditingController controller)

কন্ট্রোলারের টেক্সট ট্রিম করে double.tryParse করে, ব্যর্থ হলে 0 রটার্ন করে।

Future<void> _pickEmployee()

EmployeePicker.show দিয়ে কর্মী বাছাই ডায়ালগ খোলে; নির্বাচতি হলে _employeeId/_employeeName সটে করে ও _error ক্লয়ার করে setState কল করে।

Future<void> _submit()

ইতিমধ্যে সাবমিট চলছে বা ফর্ম ভ্যালিডেশন ব্যর্থ হলে ফিরে যায়; employeeId/effectiveFrom না থাকলে সংশ্লিষ্ট এরর দেখায়; নাহলে সব দশটি ফিল্ড পড়ে SalaryStructureRequest বানায়, _isEdit অনুযায়ী updateStructure (কন্ট্রোলারে apply) বা createStructure+পূরণ refresh কল করে (কারণ তরৈকিরলে আগের রো-ও বদলে যায়), structureHistoryProvider ইনভ্যালিডেট করে, শাট বন্ধ করে সফলতার সন্ধ্যাকবার দেখায়; ApiException/জেনেরিক এরর ধরে _error সেটে করে; শেষে _submitting = false।

String? _required(String? value)

মান পার্সবেল সংখ্যা না হলে "An amount is required." রিটার্ন করে; ০ বা কম হলে "More than zero." রিটার্ন করে (গ্রস/বসেকিরে জন্য @DecimalMin("0.01") মলিয়ি); নাহলে null (ভ্যালিডি)।

String? _optional(String? value)

খালি হলে null (গ্রহণযোগ্য); পার্স করতে না পারলে "A number." রিটার্ন করে; ঋণাত্মক হলে "Zero or more." রিটার্ন করে; নাহলে null।

Widget build(BuildContext context)

গ্রস ও বরাদ্দকৃত (বসেকি+সব এলাউন্স) মান হিসাব করে পার্থক্য বের করে; FormSheetFrame-এর মধ্যে (এডিট হলে) কর্মীর নাম বা (তরৈ হলে) কর্মী-বাছাই InkWell+InputDecorator, DateField, গ্রস/বসেকি জোড়া, _Label('Allowances') ও দুটি _MoneyPair, স্পশোল-এলাউন্স ফিল্ড, (অমলি থাকলে) সতর্কবার্তা, _Label('Deductions') ও _MoneyPair, নোট ফিল্ড এবং "খালি বিক্স মান শূন্য" ব্যাখ্যা সাজিয়ে দেখায়।

ক্লাস/উইজটে: _MoneyPair

StatelessWidget; দুটি অর্থ-ইনপুট ফিল্ড পাশাপাশি দেখানোর পুনঃব্যবহারযোগ্য উইজটে — স্ট্রাকচার ফর্ম চারবার একই প্যাটার্নে ব্যবহৃত হয়, শুধু লবেলে ভিন্ন।

_MoneyPair({required this.left, required this.leftLabel, required this.right, required this.rightLabel, required this.validator, required this.onChanged})

কনস্ট্রাক্টর — দুই কন্ট্রোলার, দুই লবেলে, একটি শিয়ার্ড ভ্যালিডিটের ও পরিবর্তন-কলব্যাক নিয়ে।

Widget build(BuildContext context)

একটি Row-এ দুটি Expanded TextFormField (প্রতটিরি নিজস্ব লবেলে, ভ্যালিডিটের, ও onChanged-এ onChanged() কল) সাজায়।

টপ-লভেলে ফাংশন: Future<void> showLoanSheet(BuildContext context)

লোন/অ্যাডভান্স রেকর্ড করার মডাল বটম-শাট (_LoanSheet) খোল।

ক্লাস/উইজটে: _LoanSheet

ConsumerStatefulWidget; নতুন লোন/অ্যাডভান্স রেকর্ড করার ফর্মের কন্টইনার (কোনো এডিট মোড নই)।

ক্লাস/উইজটে: _LoanSheetState

ConsumerState<_LoanSheet>; ফর্মের কন্ট্রোলার, ধরন (_type), বতিরণ-তারখি, কর্মী-নির্বাচন ও সাবমিট-লজকি রাখ।

void dispose()

_principal, _installment, _reason, _notes চারটি কন্ট্রোলার dispose() করে super.dispose() কল করে।

Future<void> _pickEmployee()

EmployeePicker.show দিয়ে কর্মী বাছাই করে _employeeId/_employeeName সটে করে ও _error ক্লিয়ার করে।

Future<void> _submit()

ইতিমধ্যে সাবমিট চলছে বা ভ্যালিডেশন ব্যর্থ হলে ফিরে যায়; employeeId/disbursedDate অনুপস্থিতি হলে সংশ্লিষ্ট এরর দেখায়; নাহলে CreateLoanRequest বানিয়ে createLoan কল করে, সফল হলে loansProvider কন্ট্রোলার refresh করে, শিট বন্ধ করে সফলতার সন্ধ্যাকবার দেখায়; এরর হ্যান্ডলিং ApiException/জেনেরিক দুই ভাগে; শেষে _submitting = false।

String? _money(String? value)

মান পার্সবেল না হলে "An amount is required." রিটার্ন করে; ০ বা কম হলে "More than zero." (@DecimalMin("0.01") মিলিয়ে) রিটার্ন করে; নাহলে null।

Widget build(BuildContext context)

মূলধন ও কিস্তি থেকে মাসের সংখ্যা হিসাব করে (উভয় ধনাত্মক হলে); FormSheetFrame-এর মধ্যে কর্মী-বাছাই, ধরন DropdownButtonFormField (loanTypes থেকে), অঙ্ক ও মাসিক-কিস্তি জোড়া, (হিসাবযোগ্য হলে) মাসের সংখ্যা বার্তা, বতিরণ-তারিখ DateField (lastDate: DateTime.now()), কারণ ও নোট ফিল্ড, এবং "এডিট করা যায় না" তথ্য-ব্যানার সাজিয়ে দেখায়।

টপ-লেভেলে ফাংশন: Future<void> showComponentSheet(BuildContext context, {SalaryComponent? existing})

বতেন-উপাদান তৈরি/এডিট করার মডাল বটম-শিট (_ComponentSheet) খোলো।

ক্লাস/উইজটে: _ComponentSheet

ConsumerStatefulWidget; কম্পোনেন্ট তৈরি/এডিট ফর্মের কন্টইনার — ঐচ্ছকি existing নয়।

ক্লাস/উইজটে: _ComponentSheetState

ConsumerState<_ComponentSheet>; ফর্মের নাম, সর্ব-অর্ডার, ধরন, হিসাব-পদ্ধতি, taxable/active সুইচ ও সাবমিট-লজিক পরিচালনা করে।

bool get _isEdit

widget.existing != null হলে true।

void dispose()

_name ও _sortOrder দুটি কন্ট্রোলার dispose() করে super.dispose() কল করে।

Future<void> _submit()

ইতিমধ্যে সাবমিট চলছে বা ভ্যালিডেশন ব্যর্থ হলে ফিরে যায়; SalaryComponentRequest বানায়ে (sortOrder পার্স ব্যর্থ হলে ০); _isEdit অনুযায়ী updateComponent (কন্ট্রোলারে apply) বা createComponent+refresh কল করে; শিট বন্ধ করে সফলতার সন্ধ্যাকবার দেখায়; ApiException/জেনেরিক এরর _error-এ ধরে; শেষে _submitting = false।

Widget build(BuildContext context)

FormSheetFrame-এর মধ্যে নাম ফিল্ড (আবশ্যিকি ভ্যালিডেশন), ধরন DropdownButtonFormField (componentTypes থেকে), হিসাব-পদ্ধতি DropdownButtonFormField (calculationTypes থেকে, লেবেলে "A fixed amount"/"A percentage"), প্যাজেশিন/সর্ব-অর্ডার ফিল্ড (খালি-অনুমোদিত, পূর্ণসংখ্যা ভ্যালিডেশন), এবং "Taxable"/"In use" দুটি SwitchListTile সাজিয়ে দেখায়।

ক্লাস/উইজটে: _Label

`StatelessWidget`; ফর্ম সেকশনরে ছোট আপারকসে-স্টাইল সাব-শরিনোম (যেমন "Allowances", "Deductions") দেখানোর জন্য পুনঃব্যবহারযোগ্য উইজটে।

`_Label(this.text)`

কনস্ট্রাক্টর — দেখানোর টেক্সট নিয়ে।

`Widget build(BuildContext context)`

`Padding`-এর ভেতরে `bos.muted` রঙ, ছোট ফন্ট-সাইজ, semi-bold ও letter-spacing সহ `Text` উইজটে রিটার্ন করে।

ফাইল: lib/features/salary/structure_extras_screen.dart

সংক্ষিপ্ত বিবরণ: একজন কর্মীর স্যালারি স্ট্রাকচারে বেসিক রেসপিরি বাইরে বাড়তি যিসেব কম্পোনেন্ট (এক্সট্রা) জোড়া লাগানো হয়, সেগুলো দেখা, যোগ করা ও মুছে ফেলার স্ক্রিনি। সভে করলে পুরো তালিকাটাই পুনরায় লেখা হয় (হোল-লিস্ট রিপ্লিসে), কবেল যোগ করা হয় না।

ক্লাস/উইজটে: StructureExtrasScreen

`ConsumerStatefulWidget`, যা একটি নির্দিষ্ট `structureId`-এর এক্সট্রা কম্পোনেন্টের তালিকা দেখায় এবং সম্পাদনার সুযোগ দেয়। `subtitle` প্যারামিটারে স্ট্রাকচারটিকে কোন সময়কাল কভার করে তা মানুষ-বোঝা টেক্সট আকারে দেখানো হয়।

`const StructureExtrasScreen({super.key, required this.structureId, required this.subtitle})`

কনস্ট্রাক্টর যা `structureId` (পার্মানেন্ট আইডি) ও `subtitle` (প্রদর্শনযোগ্য বিবরণ) গ্রহণ করে উইজটে তৈরি করে।

`static void open(BuildContext context, {required int structureId, required String subtitle})`

`MaterialPageRoute` ব্যবহার করে `StructureExtrasScreen`-কে নভিগেশন স্ট্যাকে পুশ করে খুলে দেয়। কলার সাইড থেকে সরাসরি `Navigator` ব্যবহারের বদলে এই স্ট্যাটিকি হলেপার ব্যবহার করা হয়।

`ConsumerState<StructureExtrasScreen> createState()`

`_StructureExtrasScreenState` তৈরি করে রিটার্ন করে — Riverpod-চালিত স্টেটফুল উইজটের আবশ্যিক ওভাররাইড।

ক্লাস/উইজটে: _StructureExtrasScreenState

এই স্ক্রিনিরে প্রকৃত স্টেটে ও লজিকি ধারণ করে — সারভার থেকে পাওয়া তালিকার একটি "ওয়ার্কিং কপি" (`_lines`) মেমোরিতে রাখে, ব্যবহারকারীর করা প্রত্যাখ্যান/যোগ সন্ধানে প্রত্যাফলিত করে, এবং সভে চাপলে গোটা তালিকাটি সারভারে পাঠায়।

`Future<void> _save()`

`_lines`-এ থাকা সম্পূর্ণ তালিকা `salaryRepositoryProvider.setExtras(structureId, lines)` কল করে সারভারে পাঠায়। সফল হলে `structureExtrasProvider` ইনভ্যালিডেট করে, `_dirty` ফ্ল্যাগ মার্ক করে এবং একটি সন্ধ্যাকবার দেখায়। ব্যর্থ হলে `ApiException`-এর বার্তা বা একটি জিনেরেকি ভুল বার্তা `_error`-এ সেট করে। শুরুতে ও শেষে `_busy` ফ্ল্যাগ টগল করে লোডিং অবস্থা নিয়ন্ত্রণ করে।

Future<void> _add(List<SalaryComponent> catalogue)

ক্যাটালগ থেকে যেকোনো সক্রিয় কম্পোনেন্ট এখনও তালিকাভুক্ত নই, সেগুলো ফিল্টার করে; কোনোটিই বাকনি থাকলে একটি সন্ধ্যাকবার দেখিয়ে থেমে যায়। অন্যথায় pickOne দিয়ে কম্পোনেন্ট বছে নতি বলে, তারপর askForText দিয়ে পরিমাণ জিজ্ঞাসে করে; পরিমাণটি সংখ্যায় রূপান্তর ব্যর্থ হলে _error স্টে করে ফিরে আসে, নাহলে নতুন StructureExtraLine তৈরি করে _lines-এ যোগ করে এবং _dirty সত্য করে।

void _remove(int componentId)

প্রদত্ত componentId-সহ লাইনটি বাদ দিয়ে _lines-এর একটি নতুন তালিকা তৈরি করে এবং _dirty সত্য করে।

Widget build(BuildContext context)

পারমিশন (SalaryPermissions.create) পড়ে লেখার অনুমতি আছে কিনা যাচাই করে, structureExtrasProvider থেকে সংরক্ষিত তালিকা ও salaryComponentsProvider থেকে ক্যাটালগ ওয়াচ করে। প্রথমবার সংরক্ষিত তালিকা এলে স্টেটিকে _lines-এ কপি করে (রিবিল্ডে যাতে অসংরক্ষিত পরিবর্তন হারিয়ে না যায়), একটি স্থানীয় nameFor ফাংশন দিয়ে কম্পোনেন্টের নাম বের করে, এবং একটি AppCard-ভিত্তিক তালিকা, "যোগ করুন" বোতাম ও "সভে" বোতামসহ পুরো স্ক্রিনের UI নির্মাণ করে।

ফাইল: lib/features/salary/template_models.dart

সংক্ষিপ্ত বিবরণ: স্যালারি টেমপ্লেটে (গ্রুডে-ভিত্তিক রেসেপি), সেগুলো তৈরি/সম্পাদনার রকিওয়েস্টে, একটি নির্দিষ্ট গ্রুপের বর্ণিত ব্রেকেডাউন, এবং একজন কর্মীর স্ট্রাকচারে বাড়তি জোড়া কম্পোনেন্টের মডেলে ক্লাসগুলো এখানে সংজ্ঞায়িত। কোনো JSON রূপান্তর ব্যর্থ হলে ডিফল্ট মান (০, খালি স্ট্রিং ইত্যাদি) ব্যবহৃত হয়।

ক্লাস/উইজটে: SalaryTemplate

একটি পুনর্ব্যবহারযোগ্য বতেন-রেসেপি, যার নাম "Software Engineer Grade A"। এটি প্রয়োগ করলে এর সংখ্যাগুলো কর্মীর নিজস্ব স্ট্রাকচারে স্ট্রাম্প হয়ে যায়, কিন্তু টেমপ্লেটে নিজস্ব অপরিবর্তিত থাকে।

```
const SalaryTemplate({required this.id, required this.structureName, required
this.basicPercentage, required this.hraPercentage, required this.medicalAmount, required
this.transportAmount, required this.internetAmount, required this.mobileAmount, required
this.mealAmount, required this.active, this.defaultGross})
```

সব ফিল্ড ধারণ করে টেমপ্লেটে অবজেক্ট তৈরির কনস্ট্রাক্টর; defaultGross ঐচ্ছিক, বাকসি আবশ্যিক।

factory SalaryTemplate.fromJson(Map<String, dynamic> json)

JSON ম্যাপ থেকে SalaryTemplate তৈরি করে। ভেতরে হলেপার number(key) প্রতিটি সংখ্যাসূচক ফিল্ডকে num? থেকে double-এ রূপান্তর করে, না থাকলে ০ ধরে।

ক্লাস/উইজটে: SalaryTemplateRequest

টেমপ্লেট তৈরি বা সম্পাদনার জন্য পাঠানো রকিওয়েস্ট বডি। সৃষ্টি ও সম্পাদনা একই এন্ডপয়েন্ট ও শেপে শেয়ার করে। active ছাড়া বাকসি ফিল্ড আবশ্যিক, কারণ ব্যাকএন্ড প্রতিটি ফিল্ড অ্যাসাইন করে এবং অনুপস্থিতিটিকে শূন্যে পরিণত করে।


```
const SalaryTemplateRequest({required this.structureName, required this.basicPercentage,
required this.hraPercentage, required this.medicalAmount, required this.transportAmount,
required this.internetAmount, required this.mobileAmount, required this.mealAmount,
this.defaultGross, this.active})
```

রকি়োয়েস্ট অবজেক্ট বানানোর কনস্ট্রাক্টর।

```
factory SalaryTemplateRequest.from(SalaryTemplate template)
```

একটি বিদ্যমান `SalaryTemplate` থেকে সমতুল্য রকি়োয়েস্ট অবজেক্ট বানায় — এডটি ফর্ম প্র-ফিলি করার জন্য কাজে লাগে।

```
Map<String, dynamic> toJson()
```

রকি়োয়েস্ট অবজেক্টের সব ফিল্ডকে একটি JSON ম্যাপে রূপান্তর করে; `active` কেবল তখনই অন্তর্ভুক্ত হয় যখন এটিনাল না হয়।

ক্লাস/উইজটে: SalaryBreakdown

একটি নির্দিষ্ট গ্রস অঙ্কে টেমপ্লেটটি আসলে কী কী দিকে বিভক্ত হবে তার ফলাফল ধারণ করে। হিসাবটি সারভার-সাইডে করা হয় যাতে UI ও ব্যাকএন্ডের হিসাব কখনো না মেলার ঝুঁকি না থাকে।

```
const SalaryBreakdown(this.lines)
```

নাম-থেকে-অঙ্ককে একটি `Map<String, double>` ধারণ করে কনস্ট্রাক্ট হয়।

```
double get gross
```

`lines` থেকে 'grossSalary' কী-এর মান ফেরত দেয়, না থাকলে 0।

```
double get total
```

`grossSalary` বাদে বাকসিব এন্ট্রির যোগফল হিসাব করে ফেরত দেয়।

```
double get overrun
```

`total` ও `gross`-এর পার্থক্য বের করে; পার্থক্য ০.০০৫-এর বেশি হলে সটোই ফেরত দেয়, নাহলে 0। কারণ ব্যাকএন্ড ইন্টারনেটে ও মোবাইল অ্যালাউন্স গ্রস থেকে বাদ না দিয়েই স্পেশাল অ্যালাউন্স হিসাব করে, ফলে অংশগুলোর যোগফল গ্রসের চেয়ে বেশি হতে পারে।

```
factory SalaryBreakdown.fromJson(Map<String, dynamic> json)
```

JSON ম্যাপেরে যেসব এন্ট্রির মান সংখ্যা (`num`), শুধু সেগুলো নিয়ে `double`-এ রূপান্তর করে একটি `SalaryBreakdown` বানায়।

ক্লাস/উইজটে: StructureExtra

একজন কর্মীর নিজস্ব স্ট্রাকচারে রেসেপিরি উপরে বাড়তি জোড়া লাগানো একটি কম্পোনেন্ট, যত্নে এটি সারভার থেকে ফেরত আসে (আইডি, নাম, টাইপসহ)।

```
const StructureExtra({required this.id, required this.componentId, required this.amount,
this.componentName, this.type})
```

এক্সট্রা কম্পোনেন্ট অবজেক্ট তৈরির কনস্ট্রাক্টর।

```
factory StructureExtra.fromJson(Map<String, dynamic> json)
```

JSON ম্যাপ থেকে `StructureExtra` তৈরিকরে, প্রতিটি সংখ্যাসূচক ফিল্ডের জন্য ডিফল্ট মান ব্যবহার করে।

ক্লাস/উইজটে: StructureExtraLine

একটি স্ট্রাকচারে এক্সট্রা সেট করার (setExtras) জন্য পাঠানো একটি লাইন — কেবল `componentId` ও `amount`। এই এন্ডপয়েন্ট হোল-লিস্ট রপ্লিসে করে, অ্যাপনেড নয়।

```
const StructureExtraLine({required this.componentId, required this.amount})
```

কনস্ট্রাক্টর।

```
factory StructureExtraLine.from(StructureExtra extra)
```

একটি `StructureExtra`-কে হালকা `StructureExtraLine`-এ রূপান্তর করে — সার্বভারে পাঠানোর সময় অতিরিক্ত মটোডটো (নাম, টাইপ) বাদ দেওয়া হয়।

```
Map<String, dynamic> toJson()
```

`componentId` ও `amount`-কে একটি JSON ম্যাপে রূপান্তর করে।

ফাইল: lib/features/salary/template_tab.dart

সংক্ষিপ্ত ববিরণ: স্যালারি মিডউইলরে "টেমপ্লেট" ট্যাব — গ্রুডে-রসেপিরি তালিকা, নতুন/সম্পাদনা ফর্ম শাটি, ব্রকেডাউন হিসাব করার শাটি এবং প্রতটিসাররি অ্যাকশন (এডটি, ডলিটি) নযি গঠতি।

ক্লাস/উইজটে: TemplatesTab

`ConsumerWidget`, যা `salaryTemplatesProvider` থেকে টেমপ্লেটের তালিকা `ConfigList` উইজটে দেখায়, প্রতটিসাররি `_TemplateRow` দযিে বল্ডি করে।

```
Widget build(BuildContext context, WidgetRef ref)
```

পারমশিন কন্ট্রোলার থেকে `canWrite` ও `canDelete` নরিণয় করে, `ConfigList<SalaryTemplate>`-কে `async` স্টটে, রফির্শে কলব্যাক, খালি/এরর মসেজে ও `itemBuilder` দযিে কনফগিার করে রটার্ন করে।

ক্লাস/উইজটে: _TemplateRow

একটি একক টেমপ্লেটকে `ConfigRow` হিসেবে দেখায় — নাম, গ্রস/বসেকি/এইচআরএ সারাংশ, এডটি-অ্যাকসসে অনুযায়ী এডটি বোতাম, এবং "ব্রকেডাউন দেখুন" ও (অনুমতি থাকলে) "ডলিটি" অ্যাকশন।

```
ConsumerState<_TemplateRow> createState()
```

`_TemplateRowState` তরৈ কিরে রটার্ন করে।

ক্লাস/উইজটে: _TemplateRowState

সাররি স্থানীয় "ব্যস্ত" (busy) অবস্থা রাখে এবং ডলিটি অ্যাকশনের লজকি ধারণ করে।

```
Future<void> _delete()
```

`confirmAction` দযিে নশ্চতিকরণ নযে বোর্তায় বলে যে বদিষমান স্ট্রাকচারগুলো নজিদেের সংখ্যা ধরে রাখবে, শুধু রসেপিটাই মুছে যাবে। নশ্চতি হলে `salaryRepositoryProvider.deleteTemplate(id)` কল করে, `salaryTemplatesProvider` ইনভ্যালডিটে করে, ও সফলতা/ব্যর্থতার স্ন্যাকবার দেখায়।

Widget build(BuildContext context)

টমপ্লেটের তথ্য থেকে facts তালিকা (গ্রস, বসেসি %, এইচআরএ %) বানিয়ে ConfigRow উইজটে রিটার্ন করে, যাতে টাইটলে, সাবটাইটলে, সক্রিয়তা, বজি স্টেটে, এডটি হ্যান্ডলার ও অ্যাকশন তালিকা থাকে।

টপ-লেভেলে ফাংশন: Future<void> showTemplateSheet(BuildContext context, {SalaryTemplate? template})

একটি মডাল বটম শিট খুলে _TemplateSheet দেখায়। template নাল হলে "নতুন টমপ্লেটে" মডাল, নাহলে "এডটি টমপ্লেটে" মডাল।

ক্লাস/উইজটে: _TemplateSheet

টমপ্লেটে তৈরি বা সম্পাদনার ফর্ম শিট। প্রতিটি সিংখ্যাসূচক ফিল্ড স্পর্শ না করা হলেও সবে করার সময় পাঠানো হয়, কারণ ব্যাকএন্ড অনুপস্থিতি ফিল্ডকে শূন্য ধরে নেয়।

ConsumerState<_TemplateSheet> createState()

_TemplateSheetState তৈরি করে রিটার্ন করে।

ক্লাস/উইজটে: _TemplateSheetState

ফর্মে সব TextEditingController (নাম, গ্রস, বসেসি %, এইচআরএ %, মডেকিলে, ট্রান্সপোর্ট, ইন্টারনেট, মোবাইল, মলি), active সুইচ, এরর ও বজি স্টেটে ধারণ করে; সাবমিশন লজিক পরিচালনা করে।

static TextEditingController _money(double? value)

একটি nullable ডাবল থেকে TextEditingController তৈরি করে — নাল হলে খালি টেক্সট, নাহলে Fmt.plain(value) দিয়ে ফর্ম্যাট করা টেক্সট।

void dispose()

সব টেক্সট কন্ট্রোলার (_name, _gross, _basic, ... _meal) লুপ করে ডিসপোজ করে, তারপর সুপারক্লাসের dispose() কল করে।

double _read(TextEditingController controller)

কন্ট্রোলারের টেক্সট ট্রিম করে double-এ পার্স করে, ব্যর্থ হলে 0 ফেরত দেয়।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে, ব্যর্থ হলে থমে যায়। বৈধ হলে সব ফিল্ড থেকে SalaryTemplateRequest বানায়, তারপর widget.template নাল হলে createTemplate নাহলে updateTemplate কল করে। সফল হলে প্রোভাইডার ইনভ্যালিডেট করে শিট বন্ধ করে; ব্যর্থ হলে _error স্টেটে করে।

Widget build(BuildContext context)

FormSheetFrame-এর ভেতরে নাম, গ্রস, বসেসি/এইচআরএ %, ফক্সড অ্যালাউন্স গ্রুপ (_AmountRow দুইবার), মলি, ও "ইন ইউজ" সুইচসহ পুরো ফর্ম UI ব্লিড করে।

static String? _percentage(String? value)

মানকে double-এ পার্স করার চেষ্টা করে; পার্স ব্যর্থ হলে বা মান ০-১০০ সীমার বাইরে হলে সংশ্লিষ্ট ভুল বার্তা ফেরত দেয়, নাহলে null (বৈধ)।

ক্লাস/উইজটে: _AmountRow

দুইটি TextFormField-কে পাশাপাশি একটি Row-এ সাজানোর জন্য ছোট রাইউজবেল উইজটে (যেমন মডেকিলে ও ট্রান্সপোর্ট একসাথে)।

Widget build(BuildContext context)

left/leftLabel ও right/rightLabel দিয়ে দুইটি Expanded TextFormField পাশাপাশি রিন্‌ডার করে।

উপ-লভেলে ফাংশন: Future<void> showBreakdownSheet(BuildContext context, {required SalaryTemplate template})

একটি মডাল বটম শিট খুলে _BreakdownSheet দেখায়, যা একটি প্রদত্ত গ্রসের বিপরীতে টেমপ্লেটের কী কী দিকে ভাগ হবে তা দেখায়।

ক্লাস/উইজটে: _BreakdownSheet

একটি স্টেটফুল শিট যখন ব্যবহারকারী গ্রস পরিবর্তন করে "Work it out" চপে ব্রেকেডাউন দেখতে পারে।

ConsumerState<_BreakdownSheet> createState()

_BreakdownSheetState তৈরি করে রিটার্ন করে।

ক্লাস/উইজটে: _BreakdownSheetState

গ্রস ইনপুট কন্ট্রোলার, বর্তমান _breakdown ফলাফল, এরর ও বজি স্টেট ধারণ করে।

void initState()

সুপারক্লাসের initState() কল করার পর, টেমপ্লেটের defaultGross থাকলে তৎক্ষণাৎ _work() কল করে প্রাথমিক ব্রেকেডাউন হিসাব করে।

void dispose()

_gross কন্ট্রোলার ডিসপোজ করে, তারপর সুপারক্লাসের dispose() কল করে।

Future<void> _work()

গ্রস টেক্সট পার্স করে; নাল বা শূন্য-বা-কম হলে _error স্টেট করে থেমে যায়। বৈধ হলে salaryRepositoryProvider.breakdown(templateId, gross) কল করে ফলাফল _breakdown-এ স্টেট করে; ব্যর্থ হলে ApiException-এর বার্তা বা জনেরেকি ভুল বার্তা দেখায়।

Widget build(BuildContext context)

টেমপ্লেটের নাম, গ্রস ইনপুট ফিল্ড ও "Work it out" বোতাম, এরর ব্যানার (থাকলে), এবং ব্রেকেডাউনের প্রতিটি লাইন (grossSalary বোল্ড করে) দেখায়। overrun > 0 হলে একটি সিতরুতা ব্যানার দেখায় যা ব্যাখ্যা করে কেনে অংশগুলোর যোগফল গ্রসের চেয়ে বেশি।

উপ-লভেলে ফাংশন: String _readable(String key)

ব্রেকেডাউনের camelCase কী (যেমন grossSalary)-কে মানুষ-পড়ার-উপযোগী টেক্সটে রূপান্তর করে — ছোট-থেকে-বড় অক্ষরে রূপান্তর করে বিন্দুতে স্পেসে বসিয়ে, প্রথম অক্ষর বড় করে বাকটি ছোট করে। Fmt.label ব্যবহার করা হয়নি কারণ স্টেট SCREAMING_SNAKE কী-এর জন্য তৈরি।

ফাইল: lib/features/search/search_models.dart

সংক্ষিপ্ত বিবরণ: গ্লোবাল সার্চের ফলাফল, প্রতিটি ফলাফলের ধরন (লিড, ক্লায়েন্ট, অপরচুনটি ইত্যাদি) ও সেই ধরনগুলোর লবেল/আইকন/পারমশিন ম্যাপিং সংজ্ঞায়িত করে।

ক্লাস/উইজটে: SearchHit

সার্চে পাওয়া একটি একক ফলাফল — সাতটি ধরনের যেকোনো একটি হতে পারে, `type` ফিল্ড দিয়ে আলাদা করা হয়।

`const SearchHit({required this.type, required this.id, required this.title, this.subtitle, this.link})`
একটি সার্চ হিট অবজেক্ট তৈরির কনস্ট্রাক্টর।

`factory SearchHit.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে `SearchHit` তৈরি করে, না থাকা ফিল্ডের জন্য ডিফল্ট মান ('UNKNOWN', 0, 'Untitled') ব্যবহার করে।

ক্লাস/উইজটে: SearchResults

একটি সার্চ কোয়েরির সম্পূর্ণ ফলাফল সেট — কোয়েরিটেক্সট, হিটের তালিকা, এবং ব্যাকএন্ড-গণতি মোট মিল সংখ্যা।

`const SearchResults({required this.query, required this.hits, required this.totalMatches})`
কনস্ট্রাক্টর।

`factory SearchResults.fromJson(Map<String, dynamic> json)`

JSON ম্যাপের 'results' লিস্ট থেকে প্রতিটি ম্যাপকে `SearchHit.fromJson` দিয়ে রূপান্তর করে `hits` তালিকা বানায়; `query` ও `totalMatches`-ও পার্স করে।

ক্লাস/উইজটে: SearchKind

প্রতিটি ফলাফল-ধরনের প্রদর্শন লবেলে, আইকন ও তা দেখানোর জন্য প্রয়োজনীয় পারমিশন ধারণ করে। ব্যাকএন্ড কোনো পারমিশন-চেক না করাই কোম্পানি অনুযায়ী ফিল্টার করে ফলাফল পাঠায়, তাই এই ম্যাপিং ব্যবহার করে অ্যাপ নজি ফলাফল ফিল্টার করে।

`const SearchKind({required this.label, required this.icon, required this.permission})`

কনস্ট্রাক্টর; `permission` নাল হতে পারে যদিও ধরনের নজিস্ব কোনো পারমিশন না থাকে।

`static SearchKind of(String type)`

`_kinds` ম্যাপে প্রদত্ত `type` খুঁজে স্টেরি `SearchKind` ফেরত দেয়; না পলে একটি ডিফল্ট `SearchKind` ('Result', বৃত্ত আইকন, `permission: null`) ফেরত দেয় — অজানা ধরনকে গটে করা যায় না বলে কলাররা নাল পারমিশনকে "লুকাও" হিসেবে ব্যাখ্যা করে (শুধু পরিচিতি ধরনের ক্ষেত্রে "দখোও" হিসেবে নয়)।

`static bool isKnown(String type)`

`_kinds` ম্যাপে প্রদত্ত `type`-এর কী আছে কিনা যাচাই করে বুলিয়ান ফেরত দেয়।

ফাইল: lib/features/search/search_repository.dart

সংক্ষিপ্ত বিবরণ: একটি একক কোয়েরি দিয়ে লিড, ক্লায়েন্ট, অপরচুনটি, সার্ভিস রকোয়েস্ট, টকিটে, ইনভয়েস ও রফিন্ড — সাত ধরনের রেকর্ড একসাথে খোঁজার রপিজিটরি ও সংশ্লিষ্ট Riverpod প্রোভাইডার/কন্ট্রোলার।

ক্লাস/উইজটে: SearchRepository

GET /search এন্ডপয়েন্টে HTTP কল করে কাঁচা সার্চ ফলাফল ফেরত আনে। এআই অ্যাসিস্ট্যান্টের প্রশ্ন-উত্তর সার্চ (POST /search/ask) এখানে অন্তর্ভুক্ত নয়, কারণ স্টেরি জন্ম আলাদা AI_CHAT পারমিশন লাগে।

SearchRepository(this._api)

ApiClient ইনজেক্ট করে রপিজিটরি তৈরি করে।

Future<SearchResults> search(String query)

/search এন্ডপয়েন্টে q কয়েরে প্যারামিটার সহ GET কল করে, ফলাফল JSON-ক SearchResults.fromJson দিয়ে প্যার্স করে ফেরত দেয়।

ক্লাস/উইজটে: SearchQueryController

Notifier<String> — বর্তমানে খোঁজা টার্মের স্টেট ধারণ করে। খালি স্ট্রিং মানে স্ক্রিনি নষিক্রয়ি।

String build()

প্রাথমিক স্টেট হিসেবে খালি স্ট্রিং ফেরত দেয়।

void set(String value)

প্রদত্ত value ট্রমি করে; ট্রমি করা মান বর্তমান স্টেটের সমান হলে কছির করে না (অপ্রয়োজনীয় রবিলিড এডাত), নাহলে স্টেট আপডেট করে।

টপ-লভেলে প্রোভাইডার/ভেরিফেল

- searchRepositoryProvider: SearchRepository-এর Provider, apiClientProvider থেকে ApiClient নযি তৈরি করে। - searchQueryProvider: SearchQueryController-এর

NotifierProvider<SearchQueryController, String> | - searchResultsProvider:

FutureProvider<SearchResults?> — বর্তমান কয়েরেপিডে; দুই অক্ষরের কম হলে null ফেরত দেয় (কারণ এক অক্ষর ডাটাবেসের বশেরিভাগ মেলে অথচ কছিরই বলে না)। পারমিশন কন্ট্রোলার পডে, রপিজিটরি থেকে কাঁচা ফলাফল আনে, তারপর প্রতিটি ইটিকে SearchKind.isKnown ও ব্যবহারকারীর পারমিশন দিয়ে ফিল্টার করে একটি নতুন SearchResults (একই totalMatches-সহ) ফেরত দেয়। ফিল্টারিং এখানে করা হয় যাত অ্যাপের প্রতিটি consumer একই তালিকা পায়।

ফাইল: lib/features/search/search_screen.dart

সংক্ষিপ্ত ববিরণ: এক বাক্স থেকে সবকছির খোঁজার স্ক্রিনি — সার্চ ফলিড, ধরন-অনুযায়ী গ্রুপ করা ফলাফল তালিকা, এবং প্রতিটি ফলাফলে ট্যাপ করলে সংশ্লিষ্ট ডিটাইল স্ক্রিনি নেভিগিটে করার লজকি।

টপ-লভেলে ফাংশন: void openSearchHit(BuildContext context, SearchHit hit)

একটি সার্চ হিট (অথবা এআই উত্তরের একই আকৃতির সোর্স) কী নরিদশে করে সেখানে নেভিগিটে করে।

hit.type অনুযায়ী সুইচ করে: LEAD হলে openLead, OPPORTUNITY হলে openOpportunity,

SERVICE_REQUEST হলে openRequestDetail, CLIENT হলে ClientDetailScreen পুশ করে, TICKET হলে

সাপোর্ট মডউলে যায় (নরিদশিট থ্রডে অজানা বলে), এবং INVOICE/REFUND হলে ফাইন্যান্স রুটে যায়।

ক্লাস/উইজটে: SearchScreen

ConsumerStatefulWidget — সার্চ স্ক্রিনিং এর এন্ট্রি পয়েন্ট।

ConsumerState<SearchScreen> createState()

_SearchScreenState তৈরি করে রিটার্ন করে।

ক্লাস/উইজটে: _SearchScreenState

সার্চ ফিল্ড ও ফলাফলের UI অবস্থা ধারণ করে; নজিরে কোনো লোকাল ফিল্ড নেই, সব Riverpod প্রোভাইডার থেকে পড়ে।

Widget build(BuildContext context)

searchQueryProvider ও searchResultsProvider ওয়াচ করে। একটি AppSearchField দেখায় (৪০০ মলিসিকেন্ড ডিবাউন্স সহ, কারণ কোয়েরিটি সার্ভার-সাইডে সার্চ রিপোর্জটিতে ছড়িয়ে যায়) যা টাইপ করলে searchQueryProvider.notifier.set কল করে। কোয়েরি দুই অক্ষরে কম হলে একটি নির্দেশনামূলক EmptyState দেখায়; নাহলে async.when দিয়ে লোডিং/এরর/ডাটা তনি অবস্থা পরিচালনা করে — ডাটা খালি হলে totalMatches > 0 অনুযায়ী দুই রকম বার্তা দেখায় (পারামশিনের কারণে লুকানো বনাম সত্যিই কিছু না থাকা), আর ফলাফল থাকলে _Results উইজটে দেখায়।

ক্লাস/উইজটে: _Results

StatelessWidget — সার্চ ফলাফলকে ধরন অনুযায়ী গ্রুপ করে হডার ও তালিকা আকারে দেখায়।

Widget build(BuildContext context)

results.hits-কে hit.type অনুযায়ী একটি Map<String, List<SearchHit>>-এ গ্রুপ করে (সার্ভার যত ক্রমে ফেরত দেয় সেই ক্রমে বজায় রেখে)। প্রতিটি গ্রুপের জন্য আইকন-লেবেল-সংখ্যাসহ একটি হডার এবং প্রতিটি ইটিমে জন্য একটি ট্যাপযোগ্য AppCard (শিরোনাম, সাবটাইটলে, ও একটি শিভেরন আইকন) রেন্ডার করে; ট্যাপ করলে onTap(hit) কল হয়।

ফাইল: lib/features/support/chat_thread.dart

সংক্ষিপ্ত বিবরণ: একটি সাপোর্ট টিকিটের কথোপকথন বাবল আকারে দেখানোর জন্য প্রজেন্টেশনাল উইজটে। এটি নিজের কোনো সিস্টেম না — বার্তাগুলো কোন এন্ডপয়েন্ট থেকে এসেছে বা কোন ইন্টারনাল, তা উপরে স্ক্রিনিং ঠিক করে দেয়; এই উইজটে শুধু যা দেওয়া হয় তা রেন্ডার করে এবং কম্পোজার ব্যবহার করে টেক্সট পাঠায়।

ক্লাস/উইজটে: ChatThread

StatelessWidget। বার্তার তালিকা, বর্তমান ব্যবহারকারীর আইডি (কোন পাশে বাবল বসবে তা নির্ধারণের জন্য), একটি TextEditingController, পাঠানোর কলব্যাক এবং অ্যাটাচমেন্ট-সম্পর্কিত ইচ্ছা প্যারামিটার নিয়ে। currentUserId নাল হলে (প্রোফাইল এখনো লোড হচ্ছে) সব বার্তা "অন্য পক্ষের" হিসেবে দেখানো হয় — সংক্ষিপ্তভাবে ভুল কন্ট্রোল নথি পড়তে। disabled সত্য হলে (টিকিট বন্ধ) কম্পোজার বন্ধ হয়ে একটি নোটিশ দেখায়। onAttach নাল হলে পোপআপ বোতাম সম্পূর্ণ লুকানো থাকে।

Widget build(BuildContext context)

একটি `Column` তৈরি করে: প্রথম বার্তার তালিকা (খালি হলে "No messages yet" বার্তা), তারপর একটি ডিভাইডার, তারপর হয় "conversation is closed" নোটিশ (যদি `disabled`) অথবা একটি কম্পোজার সেকশন। কম্পোজারে পেন্ডিং অ্যাটাচমেন্ট বা আপলোড-চলছে ইনডিকেটর, একটি টেক্সট ফিল্ড এবং পাঠানোর বোতাম (লোডিং স্পিনার সহ যখন `sending` সত্য) থাকে। প্রতিটি বার্তার জন্য `_Bubble` উইজেটে তৈরি করে, `mine` ফ্ল্যাগ পাঠিয়ে দেয়।

ক্লাস/উইজেট: `_Bubble` (প্রাইভেট)

`StatelessWidget`। একটি একক `SupportMessage` কে চ্যাট বাবল হিসেবে রেন্ডার করে — নিজের বার্তা হলে ব্র্যান্ড রঙের ফিল ও সাদা টেক্সট ডানদিকে, অন্যেরটা হলে নডিট্রাল সারফেস ও অ্যাভাটার বামদিকে।

Widget build(BuildContext context)

বাবলের পটভূমি ও অগ্রভাগ রং (`mine` অনুযায়ী), প্রেরকের নাম, বার্তার টেক্সট, অ্যাটাচমেন্ট থাকলে তার আইকন ও ফাইলনাম, এবং "Internal" / "Resolution" লবেলে (প্রযোজ্য হলে) ও আপেক্ষিক সময় দেখায়। `Internal` লবেলেটি ইচ্ছাকৃতভাবে স্টাইলের বদলে টেক্সট হিসেবে দেখানো হয়েছে, কারণ ক্লায়েন্টের কাছে পৌঁছানো একটি ইন্টারনাল নোট গোপনীয়তার লঙ্ঘন — পড়ার সময় এটাই একমাত্র প্রতিক্রিয়া।

static String `_initials(String? name)`

নাম থেকে ইনিশিয়াল বের করে অ্যাভাটারে দেখানোর জন্য। নাম খালি/নাল হলে '?', একটি শব্দ হলে তার প্রথম অক্ষর, একাধিক শব্দ হলে প্রথম ও শেষ শব্দের প্রথম অক্ষর জোড়া লাগিয়ে বড় হাতের অক্ষরে ফেরত দেয়।

ফাইল: `lib/features/support/edit_ticket_sheet.dart`

সংক্ষিপ্ত বিবরণ: একটি বিদ্যমান সাপোর্ট টিকিট এডিট করার জন্য মোডাল বটম শিট। শিরোনাম, বিবরণ ও প্রায়োরিটি পরিবর্তন করে সার্ভারে পাঠায় এবং আপডেট হওয়া টিকিট ফেরত দেয়।

টপ-লেভেলে ফাংশন

`Future<SupportTicket?> showEditTicketSheet(BuildContext context, SupportTicket ticket)`
`showModalBottomSheet` কল করে `_EditTicketSheet` দেখায় এবং সার্ভার থেকে পাওয়া আপডেটেড `SupportTicket` (অথবা বাতিল হলে `null`) রিটার্ন করে।

ক্লাস/উইজেট: `_EditTicketSheet` (প্রাইভেট)

`ConsumerStatefulWidget`, যা এডিট করার জন্য একটি `SupportTicket` নেয়। এর স্টেটে ক্লাস `_EditTicketSheetState` মূল ফর্ম লজিক ধারণ করে।

`void dispose()`

`_title` এবং `_description` কন্ট্রোলার ডিসপোজ করে মেরিলিকি এড়ায়।

Future<void> _submit()

ফর্ম ভ্যালিডেশন চেক করে, ফোকাস সরিয়ে দিয়ে লোডিং স্টেটে স্টেট করে, `supportRepositoryProvider`-এর `update()` মেথড কল করে (নতুন শিরোনাম, বিবরণ ও প্রায়েরটি সহ)। সফল হলে শিটটি বন্ধ করে আপডেট হওয়া টিকিট পপ করে এবং একটি সাকসেস সন্ধ্যাকবার দেখায়। `ApiException` হলে সারভারের বার্তা দেখায়, অন্য ব্যতিক্রমে একটি জিনেরেকি বার্তা দেখায়; শেষে `_submitting` ফলস করে দেয়।

Widget build(BuildContext context)

একটি স্ক্রলযোগ্য ফর্ম তৈরি করে যাতে টিকিট নম্বর, ত্রুটি বিবরণ (থাকলে), শিরোনাম ও বিবরণের টেক্সট ফিল্ড (উভয়ই বাধ্যতামূলক ভ্যালিডেটর সহ), প্রায়েরটি ড্রপডাউন এবং একটি "Save changes" লোডিং বোতাম থাকে। ড্রপডাউনের অপশন তালিকায় টিকিটে বর্তমান প্রায়েরটি না থাকলে সটেকে শুরুতে যোগ করে দেয়, যাতে অচেনা মান হারিয়ে না যায়।

ফাইল: `lib/features/support/internal_notes_screen.dart`

সংক্ষিপ্ত বিবরণ: একটি টিকিটে ইন্টারনাল নোটগুলো (এজেন্টরা একে অপরকে যা বলছে, যা রকিওয়েস্টার কখনো দেখে না) আলাদা স্ক্রিনি দেখায়, এডিট ও ডিলিট করার সুবিধাসহ। ইন্টারনাল নোটকে সাধারণ কথোপকথনের ফিল্টার না বানিয়ে আলাদা স্ক্রিনি করা হয়েছে যাতে ভুলবশত কন্সলেন্টের কাছে দেখানোর ঝুঁকি থাকে।

টপ-লেভেলে প্রোভাইডার

`internalNotesProvider`

`FutureProvider.autoDispose.family<List<SupportMessage>, int>` — টিকিট আইডি দিয়ে `supportQueueRepositoryProvider.internalNotes(ticketId)` কল করে ইন্টারনাল নোটের তালিকা আনে।

ক্লাস/উইজটে: `InternalNotesScreen`

`ConsumerStatefulWidget`, যখন `ticket` প্রারামিটার হিসেবে দেওয়া হয়।

`static void open(BuildContext context, {required SupportTicket ticket})`

`MaterialPageRoute` দিয়ে `InternalNotesScreen`-এ নভিগেটে করে।

ক্লাস: `_InternalNotesScreenState` (প্রাইভেট)

স্ক্রিনিং স্টেটে, যখন `_busyId` দেখায় কোন নোট বর্তমানে এডিট/ডিলিট হচ্ছে।

Future<void> _edit(SupportMessage note)

`askForText` ডায়ালগ দেখিয়ে নতুন টেক্সট নেয় (লেখক ও সময় অপরিবর্তিত থাকে বলে জানিয়ে)। টেক্সট অপরিবর্তিত থাকলে বা নাল হলে কিছু করে না। অন্যথায় `editMessage` কল করে, `internalNotesProvider` ইনভ্যালিডেট করে ও সাকসেস সন্ধ্যাকবার দেখায়। `ApiException` হলে (যেমন অন্যের নোট এডিট করার চেষ্টায় প্রত্যাখ্যান) সারভারের বার্তা দেখায়।

Future<void> _delete(SupportMessage note)

`confirmAction` দিয়ে নিশ্চিতকরণ চায় (শুধু সাপোর্ট ম্যানেজার বা কোম্পানি মালিক এটা করতে পারবে বলে সতর্ক করে), নিশ্চিত হলে `deleteMessage` কল করে, প্রোভাইডার ইনভ্যালিডেট করে ও ফলাফল সন্ধ্যাকবার দেখায়।

Widget build(BuildContext context)

Scaffold তৈরিকরে যার AppBar-এ শিরোনাম "Internal notes" ও নচি টেকিটি নম্বর দেখায়। বডতি ConfigList<SupportMessage> ব্যবহার করে internalNotesProvider থেকে তালিকা দেখায় (খালি/এরর স্টেটে মসেজেসহ), প্রতিটি নোটে জন্য _NoteCard তৈরিকরে। onEdit শুধু তখনই দেওয়া হয় যখন নোটটি বর্তমান ব্যবহারকারীর নজিরে লখো — এটি ব্যাকএন্ডের নয়িমের সাথে মলিয়ি অপ্ৰয়োজনীয় 403 এড়াই।

ক্লাস/উইজটে: _NoteCard (প্ৰাইভেট)

StatelessWidget। একটি নোট, ব্যস্ত-স্টেটে এবং ঐচ্ছকি এডটি/ডলিটি কলব্যাক নয়ে।

Widget build(BuildContext context)

একটি AppBar-এ প্রেরকের নাম, আপেক্ষিক সময়, ব্যস্ত থাকলে স্পনির বা এডটি/ডলিটি অপশনসহ PopupMenuButton, নোটে টেক্সট এবং অ্যাটাচমেন্ট ফাইলনাম (থাকলে) দেখায়।

ফাইল: lib/features/support/my_agent_card.dart

সংক্ষিপ্ত ববিরণ: সাইন-ইন করা ব্যবহারকারীর নজিস্ব এজেন্ট রকের্ড (যদি থাকে) দেখানো একটি কার্ড — তার স্ট্যাটাস, নতুন টেকিটি নেওয়ার সুইচ এবং লোড সংক্রান্ত তথ্য। এজেন্ট না হলে কার্ড কিছুই দেখায় না।

টপ-লেভেলে প্রোভাইডার

myAgentProvider

FutureProvider.autoDispose<SupportAgent?> — বর্তমান ব্যবহারকারীর আইডি দিয়ে agentForUser কল করে। ApiException (404, এজেন্ট না হলে) ধরে null রটার্ন করে — এটাকে ব্যর্থতা নয়, স্বাভাবিক অবস্থা হিসেবে গণ্য করা হয়।

ক্লাস/উইজটে: MyAgentCard

ConsumerStatefulWidget, প্রারামিটারবহীন।

ক্লাস: _MyAgentCardState (প্ৰাইভেট)

Future<void> _run(Future<void> Function(SupportAdminRepository) action, String done)

একটি সাধারণ helper যা _busy স্টেট করে, দেওয়া action চালায়, myAgentProvider ইনভ্যালিডিটে করে ও সফল বার্তা দেখায়। ApiException ও অন্যান্য ব্যতিক্রম আলাদাভাবে ধরে যথাযথ সন্ধ্যাকবার দেখায়; শেষে _busy ফলস করে।

Future<void> _setStatus(SupportAgent agent)

pickOne ডায়ালগ দিয়ে agentStatuses থেকে একটি নতুন স্ট্যাটাস বাছাই করতে দেয়। পরবর্তন না হলে কিছু করে না; অন্যথায় _run দিয়ে setAgentStatus কল করে একটি নিশ্চিতকরণ বার্তা দেখায়।

Widget build(BuildContext context)

myAgentProvider-এর মান পড়ে; null হলে (এজেন্ট নয়) `SizeBox.shrink()` রিটার্ন করে। অন্যথায় একটি `AppCard`-এ এজেন্টের নাম, স্ট্যাটাস চপি (ট্যাপ করলে `_setStatus()`), সর্বোচ্চ একযোগে-টিকিটি সংখ্যা/মোট হ্যান্ডেলড/সন্মুখী স্ক্রোর দিয়ে গঠিত তথ্য-লাইন, এবং "Taking new tickets" সুইচ (যা `setAcceptingTickets` কল করে) দেখায়। সুইচের সাবটাইটলে ব্যাখ্যা করে যে বন্ধ করলে শুধু নতুন টিকিটি রাউটিং বন্ধ হয়, ইতিমধ্যে অ্যাসাইন করা কাজ থেকে যায়।

ফাইল: lib/features/support/new_ticket_sheet.dart

সংক্ষিপ্ত বিবরণ: নতুন সাপোর্ট টিকিটি তৈরি জন্য মোডাল বটম শিট, যা সবসময় Zuhoo সাপোর্ট টিমের কাছে যায় (কোম্পানির নিজস্ব ক্লায়েন্ট থ্রুডে থেকে আলাদা)।

টপ-লেভেলে ফাংশন

Future<void> showNewTicketSheet(BuildContext context)

showModalBottomSheet দিয়ে `_NewTicketSheet` দেখায়।

ক্লাস/উইজটে: _NewTicketSheet (প্রাইভেট)

ConsumerStatefulWidget, প্যারামিটারহীন।

ক্লাস: _NewTicketSheetState (প্রাইভেট)

`_priority` ডিফল্ট 'MEDIUM'।

void dispose()

`_title` ও `_description` কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে, লোডিং স্টেটে সেট করে, `myTicketsProvider.notifier.create()` কল করে নতুন `CreateTicketRequest` (শিরোনাম, বিবরণ, প্রায়েরটি) পাঠায়। সফল হলে শিট বন্ধ করে "Ticket raised." সন্ধ্যাকবার দেখায়। `ApiException`-এ সারভার বার্তা, অন্যথায় জনৈকি বার্তা দেখিয়ে `_submitting` রসিটে করে।

Widget build(BuildContext context)

স্ক্রলযোগ্য ফর্ম "Report a problem" শিরোনাম, একটি নোট যে এটি Zuhoo সাপোর্ট টিমের কাছে যাবে, ত্রুটি বিবরণ (থাকলে), শিরোনামের টেক্সট ফিল্ড, প্রায়েরটি ড্রপডাউন (`ticketPriorities` থেকে), বিবরণের মাল্টি-লাইন টেক্সট ফিল্ড (উভয়ই বাধ্যতামূলক ভ্যালিডেটরসহ) এবং "Raise ticket" লোডিং বোতাম দেখায়।

ফাইল: lib/features/support/support_controllers.dart

সংক্ষিপ্ত বিবরণ: সাপোর্ট মডুলের বিভিন্ন Riverpod কন্ট্রোলার ও প্রোভাইডার একত্রে রাখা ফাইল — ব্যবহারকারীর নিজের টিকিটি, কোম্পানির ক্লায়েন্ট-টিকিটি ইনবক্স, একক টিকিটি বস্তিত্ব ও কথোপকথনের বার্তা।

ক্লাস: MyTicketsController

AsyncNotifier<PagedState<SupportTicket>> যা PagedLoader<SupportTicket> মক্সইন ব্যবহার করে। ব্যবহারকারী নজি যেসব টিকিট BusinessOS-কে রইজ করেছে তার পজেড তালিকা পরচালনা করে।

Future<PagedState<SupportTicket>> build()

currentUserProvider ওয়াচ করে (ব্যবহারকারী বদলালে রফ্রেশ হয়) এবং loadFirstPage() কল করে প্রথম পাতা লোড করে।

Future<PagedResponse<SupportTicket>> fetchPage(int page)

supportRepositoryProvider.myTickets(page: page) কল করে একটি নির্দিষ্ট পাতা আনবে। PagedLoader মক্সইন প্রয়োজনীয় ওভাররাইড।

Future<SupportTicket> create(CreateTicketRequest request)

supportRepositoryProvider.create(request) দিয়ে নতুন টিকিট তৈরি করে, তারপর refresh() কল করে তালিকা রলোড করে ও তৈরি হওয়া টিকিট ফেরত দেয়। টিকিট নম্বর, SLA ডেডলাইন ও শুরুর স্ট্যাটাস ব্যাকএন্ড ঠিক করে বলে নতুন সারটি ফর্ম থেকে বানানোর বদলে রফ্রেশ করে সার্ভার থেকে পড়া হয়।

টপ-লেভেলে প্রোভাইডার

myTicketsProvider

AsyncNotifierProvider<MyTicketsController, PagedState<SupportTicket>> |

ক্লাস: ClientTicketStatusController

Notifier<String?>। ক্লায়েন্ট-চ্যাট ইনবক্স কোন স্ট্যাটাসে ফিল্টার করা আছে তা ধরে রাখবে; null মান সবগুলো।

String? build()

প্রাথমিক অবস্থা null (সব স্ট্যাটাস) রটার্ন করে।

void set(String? status)

নতুন স্ট্যাটাস আগেরটির সমান না হলে স্টেটে আপডেট করে।

টপ-লেভেলে প্রোভাইডার

clientTicketStatusProvider

NotifierProvider<ClientTicketStatusController, String?> |

ক্লাস: ClientTicketsController

AsyncNotifier<PagedState<SupportTicket>> যা PagedLoader<SupportTicket> মক্সইন ব্যবহার করে। এই কোম্পানির নিজস্ব ক্লায়েন্টদের টিকিটে স্ট্যাফ ইনবক্স পরচালনা করে।

Future<PagedState<SupportTicket>> build()

currentUserProvider ওয়াচ করে এবং clientTicketStatusProvider ওয়াচ করে (রডি নয়) — কারণ স্ট্যাটাস বাছাই করাই ইনবক্স রলোড করার কারণ; ব্যাকএন্ড এটিকে কোয়েরি প্যারামিটারের বদলে পাথ দিয়ে আলাদা করে।

Future<PagedResponse<SupportTicket>> fetchPage(int page)

supportRepositoryProvider.clientTickets(status: ..., page: page) কল করে, বর্তমান নির্বাচতি স্ট্যাটাস (clientTicketStatusProvider থেকে রডি করে) পাঠিয়ে।

টপ-লেভেলে প্রোভাইডার

clientTicketsProvider

AsyncNotifierProvider<ClientTicketsController, PagedState<SupportTicket>> |

ticketDetailProvider

FutureProvider.autoDispose.family<SupportTicket, int> — একক টিকিটে বসিতারিতি আনে, প্রতাবার খোলার সময় পুনরায় পড়া হয় কারণ স্ট্যাটাস, অ্যাসাইনিং SLA পছিনে বদলে যেতে পারে। ব্যবহারকারী ক্লায়েন্ট হল myClientTicket(id) (নেজি্রে অ্যাকাউন্টে সীমাবদ্ধ), অন্যথায় byId(id) (স্টাফ রোলরে জন্ম গটেডে, CLIENT বাদ) কল করে — কারণ GET /tickets/{id} ক্লায়েন্টদরে বাদ দেয়।

enum: ThreadKind

platform এবং clientChat — একটিকিথোপকথন থ্রেডে কোন পাশরে তা নরিদশে করে। দুটিএন্ডপয়ন্টে একে অপরে বদলে ব্যবহারযোগ্য নয়: ক্লায়েন্ট থ্রেডে শুধু বহরিগত (external-only), প্ল্যাটফর্ম থ্রেডে ব্যবহারকারী যা কিছু দেখার অনুমতি রাখে সবই থাকে। ভুল ধরনটি পাঠালে হয় অর্ধেক কথোপকথন লুকিয়ে যায়, নয়তো ক্লায়েন্টরে প্রতাপিক্ষকে একটাইন্টারনাল নোট দেখানো হয়ে যায়।

typedef: ThreadKey

((int ticketId, ThreadKind kind)) — ticketMessagesProvider-এর জন্ম কী রকের্ড টাইপ।

টপ-লেভেলে প্রোভাইডার

ticketMessagesProvider

FutureProvider.autoDispose.family<List<SupportMessage>, ThreadKey> — key.kind অনুযায়ী switch করে হয় repo.messages(key.ticketId) (প্ল্যাটফর্ম থ্রেডে) অথবা repo.clientChatMessages(key.ticketId) (ক্লায়েন্ট চ্যাট) কল করে বার্তার তালিকা আনে।

ফাইল: lib/features/support/support_models.dart

সংক্ষিপ্ত ববিরণ: Support মডউিলরে ডটো মডলে, পারমশিন/রোল কনস্ট্যান্ট, টিকিটে স্ট্যাটাস ও প্রায়েরটি সংক্রান্ত enum-এর মতো ক্লাস, এবং টিকিটে তরৈ/আপডটে রকি়োয়স্টে DTO ধারণ করে। এখানে SupportTicket ও SupportMessage — দুটি মূল ডোমইন এনটিটি — JSON থেকে ডিসিরিয়লাইজ করার লজিকিসহ সংজ্ঞায়িত।

টপ-লেভেলে ক্লাস/কনস্ট্যান্ট: SupportPermissions

এই মডউিল যসেব পারমশিন কোডরে উপর নরিভর করে তার abstract final class। messageView (SUPPORT_MESSAGE_VIEW) ও ticketView (TICKET_VIEW) — দুটি স্ট্যাটিক কনস্ট্যান্ট স্ট্রিং ধারণ করে।

টপ-লেভেলে কনস্ট্যান্ট: ticketEditRoles

<String> লস্ট — টিকিটে এডটি করার অনুমতপ্রাপ্ত রোলগুলোর তালিকা: SUPPORT_MANAGER, SUPPORT_AGENT, COMPANY_OWNER, SUPER_ADMIN, SYSTEM_ADMIN। টিকিটে এডটি/ডলিটি এই মডউিলে permission-code দয়ি়ে নয়, বরং রোল দয়ি়ে গটে করা হয় — কারণ ব্যাকএন্ডে কোনো TICKET_UPDATE পারমশিন কোড নই, তাই অ্যাপটি কন্ট্রোলাররে @PreAuthorize তালিকা অনুসরণ করে।

টপ-লভেলে কনস্ট্যান্ট: ticketDeleteRoles

ticketEditRoles-এর চেয়ে সংকীর্ণ তালিকা — COMPANY_OWNER, SUPER_ADMIN, SYSTEM_ADMIN।
একজন এজেন্ট টিকিটে এডটি করতে পারলেও ডিলিট করতে পারে না।

ক্লাস: TicketType

Abstract final class — টিকিটে কোন দিকে চলছে তা নির্দেশ করে। platform (PLATFORM_SUPPORT) মানে এই কোম্পানি BusinessOS-এর কাছে সাহায্য চাইছে, আর customer (CUSTOMER_SUPPORT) মানে এই কোম্পানির নিজস্ব পোর্টাল ক্লায়েন্ট তাদের কাছে সাহায্য চাইছে। লস্টি এন্ডপয়েন্টগুলো সার্ভার-সাইডে টাইপ অনুযায়ী ফিল্টার করা, তাই ভুল টাইপ কল করলে সম্পূর্ণ ভিন্ন কথোপকথন ফেরত আসবে।

ক্লাস: TicketStatus

Abstract final class — টিকিটের সম্ভাব্য স্ট্যাটাসসমূহ ধারণ করে।

static const isNew, open, inProgress, waiting, onHold, resolved, closed

প্রতিটি একটি স্ট্যাটাস স্ট্রিং কনস্ট্যান্ট (NEW, OPEN, IN_PROGRESS, WAITING, ON_HOLD, RESOLVED, CLOSED)।

static const all

উপরে সাতটি স্ট্যাটাসের সম্পূর্ণ তালিকা, ফিল্টার অপশন তৈরির জন্য ব্যবহৃত।

static const settled

{resolved, closed} — যসেব স্ট্যাটাসে টিকিটের উপর আর কারও কিছু করার নেই তার সেট।

static bool isOpen(String status)

দেওয়া স্ট্যাটাসটি settled সেটে না থাকলে true রিটার্ন করে — অর্থাৎ টিকিটটি এখনো খোলা কনি তা যাচাই করে।

টপ-লভেলে কনস্ট্যান্ট: ticketPriorities

<String>['LOW', 'MEDIUM', 'HIGH', 'CRITICAL'] — সাপোর্ট টিকিটের প্রায়েরটি স্কেলে। সার্ভিস রকিওয়েস্টের প্রায়েরটি স্কেলে (NORMAL/URGENT) থেকে ভিন্ন, তাই আলাদা কনস্ট্যান্ট হিসেবে রাখা হয়েছে যাতো কোনোটাই অন্যটার সাথে গুলিয়ে না যায়।

ক্লাস: SupportTicket

একটি সাপোর্ট টিকিটের সম্পূর্ণ ডটো মডেলে — id, নম্বর, শিরোনাম, স্ট্যাটাস, প্রায়েরটি থেকে শুরু করে অ্যাসাইনমেন্ট, SLA ডেডলাইন, রেজোলিউশন, সন্তুষ্ট রিটেং এবং এস্কেলেশন তথ্য পর্যন্ত সবকিছু ধারণ করে। সব ফিল্ড final, ক্লাসটি ইমডিটবেল।

const SupportTicket({...})

কনস্ট্রাক্টর — id, ticketNumber, title, status, priority, createdAt আবশ্যিক; বাকসি ঐচ্ছকি প্যারামিটার যমেন assignedToAgentName, slaBreached (ডিফল্ট false), escalationLevel (ডিফল্ট 0) ইত্যাদিনিয়ে।

bool get isOpen

TicketStatus.isOpen(status) কল করে টিকিটটি খোলা কনি যাচাই করে।

`bool get isResolved`

`status == TicketStatus.resolved` — টিকিটেট রিজেলভড কনি যাচাই করে।

`bool get isClosed`

`status == TicketStatus.closed` — টিকিটেট ক্লোজড কনি যাচাই করে।

`bool get isEscalated`

`escalationLevel > 0` — টিকিটেট প্রথম স্তরের বাইরে এস্কেলেটে করা হয়েছে কনি তা জানায়। এটি একটি সাধারণ খোলা টিকিটে আর এমন টিকিটে যা ইতিমধ্যে উর্ধ্বতন কারো কাছে ঠলে দেওয়া হয়েছে তার মধ্যে পার্থক্য তুলে ধরে।

`bool get isUnassigned`

`assignedToAgentName` ও `assignedEmployeeName` — দুটোই `null` হলে `true`, অর্থাৎ টিকিটেট এখনো কউ হাতে নেই।

`String? get handlerName`

`assignedToAgentName ?? assignedEmployeeName` — ব্যাকএন্ড যে ফিল্ডে হ্যান্ডলারের নাম ভরছে সেটাই ফেরত দেয়।

`factory SupportTicket.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে একটি `SupportTicket` তৈরি করে। প্রতিটি ফিল্ড নরিপদ টাইপ কাস্টিং ও ডফিল্ট মান ব্যবহার করে পার্স করা হয় (যেমন `id` না থাকলে `0`, `status` না থাকলে `TicketStatus.isNew`, `priority` না থাকলে `'MEDIUM'`)।

ক্লাস: SupportMessage

টিকিটে কথোপকথনের একটি বার্তা প্রতিনিধিত্ব করে — অভ্যন্তরীণ নোট বা ক্লায়ন্টের সাথে বনিময় করা বার্তা, উভয়ের জন্যই ব্যবহৃত।

`const SupportMessage({...})`

কনস্ট্রাক্টর — `id`, `ticketId`, `message`, `createdAt`, `sentById` আবশ্যিক; `isInternal` ও `isResolution` ডফিল্ট `false`।

`bool get hasAttachment`

`attachmentUrl` না-`null` এবং খালি না হলে `true` রিটার্ন করে।

`factory SupportMessage.fromJson(Map<String, dynamic> json)`

JSON ম্যাপ থেকে একটি `SupportMessage` তৈরি করে, প্রতিটি ফিল্ডের জন্য নরিপদ টাইপ কাস্টিং ও ডফিল্ট মান সহ।

ক্লাস: CreateTicketRequest

`POST /api/v1/support/tickets`-এ পাঠানোর জন্য প্ল্যাটফর্ম-সাপোর্ট টিকিটে তৈরি রকোয়েস্ট বডি — এই কোম্পানি BusinessOS-এর কাছে সাহায্য চাইছে এমন টিকিটে জন্য।

`const CreateTicketRequest({...})`

কনস্ট্রাক্টর — `title`, `description` আবশ্যিক; `priority` ডফিল্ট `'MEDIUM'`; `categoryId` ঐচ্ছক।

`Map<String, dynamic> toJson()`

`title` ও `description` ট্রমি করে, `priority` সহ একটি JSON ম্যাপ তৈরি করে; `categoryId` শুধু তখনই যোগ হয় যখন এটি `null` নয়।

ক্লাস: UpdateTicketRequest

PATCH /v1/support/tickets/{id}-এ পাঠানোর রকিওয়েস্ট বডি ব্যাকএন্ডে create DTO-ই পুনরায় ব্যবহৃত হয় এবং এটি @Valid, তাই title, description, priority — তিনটিই আবশ্যিক এবং সবসময় নশ্বরভাবে সেট হয়। ফলে এটি কার্যত একটি প্যাচ নয় বরং একটি সম্পূর্ণ প্রতস্থাপন — ফর্মটি বদ্যমান টিকেটে থাকে সেডি হয়ে পুরোটা ফরেক পাঠায়। ব্যতকিরম শুধু categoryId — এটি null-চকে করা হয়, তাই এটি সেটে না করলে টিকেটে বদ্যমান ক্যাটাগরি অক্শত থাকে।

```
const UpdateTicketRequest({...})
```

কনস্ট্রাক্টর — title, description, priority আবশ্যিক; categoryId ঐচ্ছক।

```
factory UpdateTicketRequest.from(SupportTicket ticket)
```

একটি বদ্যমান SupportTicket থেকে UpdateTicketRequest তরৈকিরে, title, description (ডফিল্ট খালি স্ট্রিং), ও priority কপি কিরে — categoryId সেটে করা হয় না।

```
Map<String, dynamic> toJson()
```

title ও description ট্রমি করে, priority সহ JSON ম্যাপ তরৈকিরে; categoryId কবেল null না হলে যোগ হয়।

ফাইল: lib/features/support/support_queue_screen.dart

সংক্শপিত ববিরণ: সাপোর্ট ডস্করে (এজেন্ট/ম্যানজোররে) দৃষ্টকিওণ থেকে টিকেটে কডি পরচালনার রপিওজটিরি, প্রোভাইডার ও UI স্ক্রনি ধারণ করে — কার কাছে টিকেটে অ্যাসাইন করা, রঅ্যাসাইন, এস্কেলেটে, প্রথম রসেপন্স রকেরড ও সন্তুষ্টিরকেরড করার মতো কাজ। এটি support_repository.dart-এর "রকিওয়েস্টাররে" ভডিয়রে বপিরীতে "ডস্করে" ভডি প্রতনিধিত্ব করে।

ক্লাস: SupportQueueRepository

সাপোর্ট ডস্করে কডি ও তাদের উপর পরচালতি ক্রয়িা নয়িে কাজ করা রপিওজটিরি ক্লাস।

ApiClient-এর মাধ্যমে /v1/support/tickets ও /v1/support/messages এন্ডপয়নেটে কল করে।

```
SupportQueueRepository(this._api)
```

কনস্ট্রাক্টর — একটি ApiClient ইনস্ট্যান্স নয়িে ও সংরক্শণ করে।

```
Future<PagedResponse<SupportTicket>> assignedToMe({int page = 0, int size = 30})
```

লগ-ইনকৃত এজেন্টকে অ্যাসাইন করা টিকেটেগুলোর পৃষ্ঠাবন্যিস্ত তালকি আনে (SUPPORT_AGENT-এর জন্য; ম্যানজোর পুরো বোর্ড দখেনে)।

```
Future<PagedResponse<SupportTicket>> byStatus(String status, {int page = 0, int size = 30})
```

নরিদষ্টি একটি স্ট্যাটাসরে টিকেটে গোটা ডস্ক জুড়ে আনে।

```
Future<PagedResponse<SupportTicket>> slaBreached()
```

SLA-ডেডলাইন অতকিরান্ন্ত টিকেটেগুলো আনে (পেজে ০, সাইজ ১০০)। এন্ডপয়নেট ResponseEntity<?> ঘোষণা করে বলে getPaged উভয় ধরনের রসেপন্স শপে (পেজেড/বয়োর লস্টি) পড়তে সক্শম।

```
Future<PagedResponse<SupportTicket>> criticalOpen()
```

করটিক্যাল প্রায়োরটিরি এবং এখনো খোলা টিকেটেগুলো আনে (পেজে ০, সাইজ ১০০)।

Future<SupportTicket> byNumber(String number)

টিকিটে নম্বর দিয়ে (ইমেইলে যমেনটা পাওয়া যায়) নরিদষ্টি একটি টিকিটে খুঁজে বের করে; নম্বরটি URL-এনকোড করে GET কল করে।

Future<void> assign(int id, int agentId)

একটি টিকিটে একজন এজেন্টের উপর অর্পণ করে; **agentId** কোয়েরি প্যারামিটার হিসেবে পাঠানো হয়, রসেপন্স খালি।

Future<void> reassign(int id, int newAgentId, String reason)

টিকিটে অন্য একজন এজেন্টের কাছে সরায়; **newAgentId** ও **reason** (ট্রিমি করে URL-এনকোড করা) উভয়ই আবশ্যিক — খালি কারণ ৪০০ এর তরৈকিববে।

Future<void> escalate(int id, String reason)

টিকিটেটি এস্কেলেটে করে; কারণ আবশ্যিক এবং URL-এনকোড করা হয়ে পাঠানো হয়।

Future<void> recordFirstResponse(int id)

প্রথম-রসেপন্স ঘড়ি বন্ধ করে। বাস্তবে আইডেমপোটেন্ট — ব্যাকএন্ড শুধু প্রথমটাই রকের্ড করে।

Future<void> recordSatisfaction(int id, int rating, String feedback)

রকি়োস্টারের রায় রকের্ড করে; **rating** ও **feedback** উভয়ই আবশ্যিক কোয়েরি প্যারামিটার হিসেবে পাঠানো হয়।

Future<List<SupportMessage>> internalNotes(int ticketId)

এজেন্টরা একে অপরকে যসেব নোট রাখতে, যা রকি়োস্টার কখনো দেখে না — সেই তালিকা আনতে ও **SupportMessage** অবজেক্টে ম্যাপ করে।

Future<SupportMessage> editMessage(int id, String message)

একটি বার্তার শব্দচয়ন সংশোধন করে (নেজিরে লেখা বা এজেন্ট হিসেবে); PATCH কল করে আপডেটেড **SupportMessage** ফেরত দেয়।

Future<void> deleteMessage(int id)

একটি বার্তা মুছে ফেলে; শুধু সাপোর্ট ম্যানেজার বা কোম্পানিমালিক এই কাজ করতে পারেন।

টপ-লেভেলে প্রোভাইডার: supportQueueRepositoryProvider

Provider<SupportQueueRepository> — **apiClientProvider** থেকে **ApiClient** নিয়ে একটি **SupportQueueRepository** ইনস্ট্যান্স সরবরাহ করে।

enum: SupportQueue

কোন কডিটি দেখানো হচ্ছে তা নরিদশে করে — **mine**, **breached**, **critical**, **unassigned**। প্রতিটি ভ্যারিয়েন্টের একটি **label** ও **blurb** (বর্ণনামূলক টেক্সট) থাকে।

const SupportQueue(this.label, this.blurb)

enum কনস্ট্রাক্টর — লেবেল ও ব্লার্ব স্ট্রিং সেট করে।

ক্লাস: SupportQueueController

Notifier<SupportQueue> — বর্তমানে নরিবাচতি কডি-এর স্টেটে পরিচালনা করে।

SupportQueue build()

প্রাথমিক স্টেটে SupportQueue.mine রিটার্ন করে।

void set(SupportQueue queue)

নতুন কডি নির্বাচন হলে স্টেটে পরিবর্তন করে; একই কডি হলে কিছু করে না।

উপ-লভেলে প্রোভাইডার: supportQueueProvider

NotifierProvider<SupportQueueController, SupportQueue> — বর্তমান কডি সলিকেশন এক্সপোজ করে।

ক্লাস: SupportQueueListController

AsyncNotifier<List<SupportTicket>> — নির্বাচন কডি অনুযায়ী টিকেটে তালিকা লোড করে।

Future<List<SupportTicket>> build()

currentUserProvider ও supportQueueProvider watch করে, তারপর _load() কল করে প্রাথমিক তালিকা তৈরি করে।

Future<List<SupportTicket>> _load()

বর্তমানে নির্বাচন SupportQueue-এর উপর ভিত্তি করে switch expression দিয়ে সংশ্লিষ্ট রপিকোজটির মাঝে কল করে (assignedToMe, slaBreached, criticalOpen, বা byStatus(TicketStatus.isNew) — "Unassigned"-এর নিজস্ব কোনো এন্ডপয়েন্ট নেই, এটি NEW স্ট্যাটাসেরই প্রতিনিধিত্ব করে) এবং page.content রিটার্ন করে।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে তালিকা পুনরায় লোড করে স্টেটে আপডেট করে।

উপ-লভেলে প্রোভাইডার: supportQueueListProvider

AsyncNotifierProvider<SupportQueueListController, List<SupportTicket>> — নির্বাচন কডির টিকেটে তালিকা এক্সপোজ করে।

উপ-লভেলে প্রোভাইডার: assignableAgentsProvider

FutureProvider<List<SupportAgent>> — supportAdminRepositoryProvider থেকে এজেন্ট তালিকা এনে, যারা isTakingWork তাদের ফিল্টার করে রিটার্ন করে — একটা টিকেটে যাদের কাছে হস্তান্তর করা যতে পারে।

ক্লাস/উইজটে: SupportQueueScreen

ConsumerWidget — সাপোর্ট ডেস্কের নিজস্ব কডিগুলো প্রদর্শন করে।

Widget build(BuildContext context, WidgetRef ref)

বর্তমান ব্যবহারকারীর SUPPORT_AGENT বা supportAgentAdminRoles-এর মধ্যে কোনো রোল না থাকলে "Not available to you" স্টেটে দেখায়। অন্যথায় একটা Scaffold তৈরি করে যাতে থাকে: টিকেটে-নম্বর দিয়ে খোঁজার সার্চ আইকন, MyAgentCard, একটা FilterBar (কডি পাল্টানোর জন্য), এবং ConfigList দিয়ে বর্তমান কডির টিকেটে তালিকা _QueueRow আইটেমে হিসাবে রেন্ডার করে।

ক্লাস/উইজটে: _QueueRow

ConsumerStatefulWidget — কডিতে একটা একক টিকেটে সারা প্রদর্শন করে, সাথে দ্রুত ক্রিয়া মেনে।

ConsumerState<_QueueRow> createState()

_QueueRowState ইনস্ট্যান্স তৈরি করে।

ক্লাস: _QueueRowState

ConsumerState<_QueueRow> — _QueueRow-এর স্টেটে ও ক্রিয়া হ্যান্ডলিং লজিক ধারণ করে; _busy ফ্ল্যাগ দিয়ে ক্রিয়া চলাকালীন লোডিং ইন্ডিকটর দেখায়।

Future<void> _act(Future<void> Function(SupportQueueRepository repo) action, String success, String failure)

একটি ক্রিয়া (যেমন assign, escalate) সম্পাদনের সাধারণ wrapper — _busy স্টেট করে, ক্রিয়াটি চালায়, তারপর supportQueueListProvider রিফ্রেশ করে (কারণ প্রতিটি ক্রিয়ার রেসপন্স খালিও অধিকাংশ টকিটকে অন্য কডিটে সরিয়ে দেয়) এবং সাফল্য/ব্যর্থতার সন্ধ্যাকবার দেখায়।

Future<SupportAgent?> _pickAgent()

assignableAgentsProvider থেকে এজেন্ট তালিকা নিয়ে, খালি থাকলে একটি সন্ধ্যাকবার দেখিয়ে null রিটার্ন করে; নয়তো pickOne ডায়ালগ দিয়ে ব্যবহারকারীকে একজন এজেন্ট বাছাই করতে দেয় ও নির্বাচন SupportAgent রিটার্ন করে।

Future<void> _assign()

_pickAgent() দিয়ে এজেন্ট বাছাই করে repo.assign কল করতে _act ব্যবহার করে।

Future<void> _reassign()

এজেন্ট বাছাই করে, askForText দিয়ে আবশ্যিক একটি কারণ চেয়ে নেয়, তারপর repo.reassign কল করতে _act ব্যবহার করে।

Future<void> _satisfaction()

pickOne দিয়ে রেটিং (৫ থেকে ১) এবং askForText দিয়ে ফিডব্যাক টেক্সট চেয়ে নিয়ে repo.recordSatisfaction কল করে — উভয় মান আবশ্যিক কোয়রে প্যারামিটার হওয়ায় কোনোটো বিবাদ দেওয়া যায় না।

Future<void> _escalate()

askForText (destructive স্টাইলে) দিয়ে আবশ্যিক কারণ চেয়ে নিয়ে repo.escalate কল করে।

Widget build(BuildContext context)

টকিটের শিরোনাম, স্ট্যাটাস চিহ্ন, একটি পিপআপ মেনু (assign/reassign, first-response, internal notes, satisfaction — শুধু settled অবস্থায়, ও escalate) এবং টকিটে নম্বর/প্রায়োরিটি/হ্যান্ডলার/সময় সংবলিত একটি সারাংশ লাইন রেন্ডার করে। _busy হলে প্রগ্রসে ইন্ডিকটর দেখায়।

টপ-লেভেলে ফাংশন: Future<void> showFindTicketDialog(BuildContext context, WidgetRef ref)

askForText দিয়ে ব্যবহারকারীর কাছে একটি টকিটে নম্বর চায়, তারপর

SupportQueueRepository.byNumber কল করে ফলাফল একটি সন্ধ্যাকবার দেখায় (নম্বর, শিরোনাম, স্ট্যাটাস), অথবা এরর/ব্যর্থতার বার্তা দেখায়।

ফাইল: lib/features/support/support_repository.dart

সংক্ষিপ্ত ববিরণ: সাপোর্ট মডউলরে মূল রপি়োজিটরি, যা "রকি়োয়স্টাররে" দৃষ্টকি়োণ থকে টকিটে (নজিরে রইজ করা এবং ক্লায়নেটরে টকিটে) ও বার্তা ম্যানজে করে — তরৈ, দখো, এডটি, রপ্লাই, resolve/close/reopen। এন্ডপয়নেটগুলো /v1/support/...-এর অধিনে, যা বাকি API থকে ভিন্ন একটা ভার্সনড প্রফিক্স।

ক্লাস: SupportRepository

সাপোর্ট টকিটে ও বার্তা নযি API কল করার রপি়োজিটরি।

`SupportRepository(this._api)`

কনস্ট্রাক্টর — `ApiClient` সংরক্ষণ করে।

`Future<PagedResponse<SupportTicket>> myTickets({int page = 0, int size = 20})`

বর্তমান ব্যবহারকারী BusinessOS-কে রইজ করা নজিরে টকিটগুলোর পৃষ্ঠাবন্যিস্ত তালিকা আনে।

`Future<SupportTicket> byId(int id)`

আইডি দযি়ে একটা টকিটে আনে (GET /v1/support/tickets/{id})।

`Future<SupportTicket> create(CreateTicketRequest request)`

একটা নতুন টকিটে তরৈকিরে; `request.toJson()` পাঠযি়ে তরৈ ইওয়া `SupportTicket` ফরেত দযে।

`Future<SupportTicket> update(int id, UpdateTicketRequest request)`

টকিটরে শরি়োনাং, বরণনা বা প্রায়োরটি সংশোধন করে (PATCH); রোল-গটেডে, এবং যহেতু পুরো বডি প্রতবার পাঠানো হয় তাই এটি প্রকৃতপক্ষে প্যাচ নয় বরং প্রতস্থাপন।

`Future<void> delete(int id)`

একটা টকিটে সম্পূর্ণভাবে সরযি়ে দযে (DELETE, ২০৪ নো-বডি); এডটিরে চযে সংকীরণ রোল প্রযোজন।

`Future<PagedResponse<SupportTicket>> clientTickets({String? status, int page = 0, int size = 20})`

স্টাফ-সাইড "Client Chat" ইনবক্স আনে — `myTickets`-এর ফলিটার নয়, বরং সম্পূর্ণ ভিন্ন একটা এন্ডপয়নেট, কারণ ব্যাকএন্ড `ticketType` দযি়ে দুইটা আলাদা করে; `status` দওয়া থাকলে স্ট্যাটাস-নরিদষ্টি এন্ডপয়নেট কল করে।

`Future<SupportTicket> myClientTicket(int id)`

সাইন-ইনকৃত ক্লায়নেটরে নজিস্ব একটা টকিটে আনে (`hasRole('CLIENT')`); স্টাফ একই টকিটে `byId`-এর মাধ্যমে পড়ে — দুটা ভিন্ন এন্ডপয়নেট কারণ স্কোপিং ভিন্ন।

`Future<List<SupportMessage>> messages(int ticketId)`

একটা টকিটরে সম্পূর্ণ থরডে (ব্যবহারকারী যতটুকু দখোর অনুমতপ্রাপ্ত) আনে ও `_oldestFirst` দযি়ে সাজযি়ে দযে।

`Future<List<SupportMessage>> clientChatMessages(int ticketId)`

শুধু ক্লায়নেট দখতে পারে এমন বার্তাগুলো আনে (external-only এন্ডপয়নেট); ইচ্ছাকৃতভাবে ইন্টারনাল নোট বাদ দওয়া হয় যাতো ফোন থকে রপ্লাই করা স্টাফরে প্রাইভটে মন্তব্য ভুলবশত ক্লায়নেটকে দখোনো না হয়।

`Future<SupportMessage> reply({required int ticketId, required String message, required bool internal, String? attachmentUrl, String? attachmentFileName})`

একটি রিপ্লাই পোস্ট করে; `isInternal` স্পষ্টভাবে পাঠানো হয় (ব্যাকএন্ডের ডফিল্টের উপর নির্ভর না করে), কারণ দুই কলার (ইন্টারনাল নোট বনাম ক্লায়েন্ট চ্যাট) বপিরীত জনিসি চায়।

`Future<void> resolve(int id, String notes)`

টিকিটেটি `resolve` করে; `notes` URL-এনকোড করা কয়েরে প্যারামটির হিসিবে পাঠানো হয় (কন্ট্রোলার `@RequestParam` ব্যবহার করে, তাই বডিতে পাঠালে নীরবে কছিই হয় না)।

`Future<void> close(int id)`

টিকিটেটি `close` করে।

`Future<void> reopen(int id, String reason)`

টিকিটেটি `reopen` করে; `reason` URL-এনকোড করা কয়েরে প্যারামটির হিসিবে পাঠানো হয়।

`static List<SupportMessage> _oldestFirst(List<SupportMessage> messages)`

বার্তার তালিকা `createdAt` অনুযায়ী পুরাতন-থেকে-নতুন ক্রমে সাজায়, যাত কথোপকথন উপর-থেকে-নিচে পড়া যায় এবং কম্পোজার সবচেয়ে নতুন বার্তার নিচে অবস্থান করে।

টপ-লভেলে প্রোভাইডার: `supportRepositoryProvider`

`Provider<SupportRepository>` — `ApiClientProvider` থেকে `ApiClient` নিয়ে `SupportRepository`-এর একটি ইনস্ট্যান্স সরবরাহ করে।

ফাইল: `lib/features/support/support_screen.dart`

সংক্ষিপ্ত ববিরণ: সাপোর্ট মডউলরে প্রধান স্ক্রনি — "My tickets" এবং (পারমশিন থাকলে) "Client chat" — দুটি ট্যাবে বিভক্ত। নতুন টিকিটে রইজ করার ফ্লোটিং অ্যাকশন বাটনও এখানে।

ক্লাস/উইজেট: `SupportScreen`

`ConsumerWidget` — সাপোর্ট মডউলরে রুট স্ক্রনি, ট্যাবগুলোর মধ্যে ন্যাভগেশন ও পারমশিন-ভিত্তিক ট্যাব-দৃশ্যমানতা নিয়ন্ত্রণ করে।

`const SupportScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর — ঐচ্ছকি `initialTabLabel` নেয়, যা দ্যি়ে নির্দিষ্ট একটি ট্যাব দ্যি়ে স্ক্রনি খোলা যায়।

`Widget build(BuildContext context, WidgetRef ref)`

পারমশিন এখনো লোড না হলে (`!permissions.loaded && permissions.codes.isEmpty`) একটি `Loader` দেখায়। `SupportPermissions.messageView` থাকলে "Client chat" ট্যাব যোগ করে (যেকেউ নিজের টিকিটে রইজ করতে পারে বলে "My tickets" ট্যাবের কোনো পারমশিন লাগে না)। `initialTabLabel` অনুযায়ী প্রাথমিক ট্যাব ইনডেক্স নির্ণয় করে, তারপর `DefaultTabController` ও `Scaffold` তৈরি করে যাত `TabBar/TabBarView` (একাধিক ট্যাব থাকলে) এবং নতুন টিকিটে তৈরি `FloatingActionButton.extended` থাকে।

ক্লাস/উইজেট: `_MyTicketsTab`

`ConsumerWidget` — ব্যবহারকারীর নিজের রইজ করা টিকিটে তালিকা প্রদর্শন করে।

Widget build(BuildContext context, WidgetRef ref)

myTicketsProvider watch করে PagedList<SupportTicket> রেন্ডার করে, প্রতিটি আইটেমে TicketCard হিসেবে দেখায়, ট্যাপে openTicket(context, ticket.id, ThreadKind.platform) কল করে।

ক্লাস/উইজটে: _ClientChatTab

ConsumerWidget — এই কোম্পানির নিজস্ব ক্লায়েন্টদের পোর্টাল থেকে রইজ করা টিকিটে ইনবক্স প্রদর্শন করে।

Widget build(BuildContext context, WidgetRef ref)

clientTicketStatusProvider অনুযায়ী একটি FilterBar (স্ট্যাটাস ফিল্টার) দেখায়, তারপর clientTicketsProvider watch করে PagedList<SupportTicket> রেন্ডার করে (showRaisedBy: true সহ TicketCard), ট্যাপে openTicket(context, ticket.id, ThreadKind.clientChat) কল করে।

ফাইল: lib/features/support/ticket_card.dart

সংক্ষিপ্ত বিবরণ: টিকিটে তালিকা (My tickets ও Client chat উভয় ট্যাবে) ব্যবহৃত পুনরায়-ব্যবহারযোগ্য একটি কার্ড উইজটে, যা একটি SupportTicket-এর সারাংশ (শিরোনাম, স্ট্যাটাস, প্রায়োরিটি, সময়, ফ্ল্যাগ) প্রদর্শন করে।

ক্লাস/উইজটে: TicketCard

StatelessWidget — একটি টিকিটে সংক্ষিপ্ত তথ্য প্রদর্শনকারী কার্ড।

const TicketCard({super.key, required this.ticket, this.onTap, this.showRaisedBy = false})

কনস্ট্রাক্টর — একটি SupportTicket, ঐচ্ছিক ট্যাপ কলব্যাক, এবং showRaisedBy ফ্ল্যাগ (ক্লায়েন্ট-চ্যাট ইনবক্সে কী টিকিটে রইজ করেছে তা দেখানোর জন্য — "My tickets" তালিকায় এটি অপ্ৰয়োজনীয়, কারণ উত্তর সবসময় পাঠকই) নিয়ে।

Widget build(BuildContext context)

AppBar তৈরি করে যাত থাকে: শিরোনাম ও StatusChip, টিকিটে নম্বর ও ক্যাটাগরির নাম, প্রায়োরিটি ডিউ ও লবেলে, তৈরি আপেক্ষিক সময়, এবং (showRaisedBy অনুযায়ী) রইজকারীর নাম বা হ্যান্ডলারের নাম। SLA breach, escalation, বা unassigned অবস্থা (শুধু খোলা টিকিটে ক্ষেত্রে) সংশ্লিষ্ট _Flag উইজটে প্রদর্শন করে।

ক্লাস/উইজটে: _Flag

StatelessWidget — একটি আইকন ও লবেলসহ একটি ছোট সতর্কতা/অবস্থা ফ্ল্যাগ প্রদর্শন করে (যেমন "SLA breached", "Escalated L1", "Unassigned")।

const _Flag({required this.icon, required this.label, required this.colour})

কনস্ট্রাক্টর — আইকন, লবেলে টেক্সট, রং নিয়ে।

Widget build(BuildContext context)

আইকন ও লবেলেযুক্ত একটি সারি (Row) রেন্ডার করে, উল্লিখিত রঙে স্টাইল করা।

ক্লাস/উইজটে: _PriorityDot

StatelessWidget — প্রায়োরিটির রঙ নির্দেশক একটি ছোট বৃত্ত।

`const _PriorityDot({required this.priority})`

কনস্ট্রাক্টর — প্রায়োরিটি স্ট্রিং নষে।

`Widget build(BuildContext context)`

প্রায়োরিটি অনুযায়ী রঙ নির্ধারণ করে (**CRITICAL** danger, **HIGH** warning, **MEDIUM** info, অন্যথায় muted) এবং একটি চ্যুট বৃত্তাকার **Container** রেন্ডার করে।

ফাইল: `lib/features/support/ticket_detail_screen.dart`

সংক্ষিপ্ত ববিরণ: একটি একক টকিটে বসিতারতি স্ক্রিনি — শিরোনাম, তথ্য, রেজোলিউশন নোট, কথোপকথন থ্রেডে এবং resolve/close/reopen/edit/delete ক্রিয়াগুলো প্রদর্শন করে। **ThreadKind** (প্ল্যাটফর্ম বনাম ক্লায়েন্ট-চ্যাট) অনুযায়ী আচরণ পাল্টায়।

টপ-লেভেলে ফাংশন: `void openTicket(BuildContext context, int id, ThreadKind kind)`

TicketDetailScreen-কে একটি **MaterialPageRoute**-এর মাধ্যমে নভিগেটে করে ওপেন করে, নির্দিষ্ট টকিটে আইডি ও থ্রেডে-কাইন্ড সহ।

ক্লাস/উইজটে: **TicketDetailScreen**

ConsumerStatefulWidget — একটি টকিটে সম্পূর্ণ বসিতারতি ভিউ ও তার কথোপকথন প্রদর্শন করে।

`const TicketDetailScreen({super.key, required this.id, required this.kind})`

কনস্ট্রাক্টর — টকিটে আইডি ও **ThreadKind** নষে (কোন মসেজে এন্ডপয়েন্ট ব্যবহৃত হবে এবং রপ্লাই ইন্টারনাল কনি তা নির্ধারণ করে)।

`ConsumerState<TicketDetailScreen> createState()`

`_TicketDetailScreenState` ইনস্ট্যান্স তৈরি করে।

ক্লাস: `_TicketDetailScreenState`

ConsumerState<TicketDetailScreen> — স্ক্রিনিরে সব স্টেটে (কম্পোজার টেক্সট, sending/acting ফ্ল্যাগ, অ্যাটাচমেন্ট) ও ক্রিয়া-হ্যান্ডলিং লজিকি ধারণ করে।

`ThreadKey get _threadKey`

(`ticketId: widget.id, kind: widget.kind`) — বর্তমান থ্রেডের কী তৈরি করে, প্রোভাইডার লুকআপে ব্যবহৃত।

`void dispose()`

`_composer` টেক্সট কন্ট্রোলার dispose করে।

`Future<void> _pickAttachment()`

`pickAttachment` দিয়ে একটি ফাইল বাছাই করায়, তারপর `apiClientProvider.uploadDocument` দিয়ে সেটি আপলোড করে; সফল হলে `_attachmentUrl` সেটে করে, ব্যর্থ হলে এরর সন্ধ্যাকবার দেখিয়ে অ্যাটাচমেন্ট রসিটে করে।

`void _removeAttachment()`

`_attachmentName` ও `_attachmentUrl` উভয়ই null করে সেটে করে, পন্ডিং অ্যাটাচমেন্ট বাতলি করে।

Future<void> _send()

কম্পোজারের টেক্সট খালি না থাকলে ও sending/attaching চলাকালীন না হলে, `supportRepositoryProvider.reply` কল করে বার্তা পাঠায় (`internal: false` — কারণ ক্লায়েন্ট-চ্যাট থ্রুডে থেকে পাঠানো সবকিছুই এক্সটারনাল; ইন্টারনাল নোট আলাদা একটি ওয়বে ফিচার)। সফল হলে কম্পোজার ও অ্যাটাচমেন্ট ক্লিয়ার করে এবং `ticketMessagesProvider` ও `ticketDetailProvider` invalidate করে।

Future<void> _act(String label, Future<void> Function(SupportRepository repo) action)

`resolve/close/reopen`-এর মতো একটি ক্রিয়া সম্পাদনের সাধারণ wrapper — `_acting` স্টেট করে, ক্রিয়াটি চালায়, তারপর `ticketDetailProvider`, `ticketMessagesProvider`, `myTicketsProvider`, `clientTicketsProvider` invalidate করে (কারণ উভয় তালিকায় এই টিকেটে থাকতে পারে এবং তার স্ট্যাটাস মাত্র পরিবর্তিত হয়েছে) ও সাফল্য/ব্যর্থতার সন্ধ্যাকবার দেখায়।

Future<void> _edit(SupportTicket ticket)

`showEditTicketSheet` খুলে সম্পাদনার শীট দেখায়; আপডেট হলে `ticketDetailProvider` invalidate করে।

Future<void> _delete(SupportTicket ticket)

একটি নিশ্চিতকরণ ডায়ালগ দেখায় ("টিকেটে ও তার পুরো কথোপকথন সবার জন্য অদৃশ্য হয়ে যাবে"); নিশ্চিতি হলে `supportRepositoryProvider.delete` কল করে, সফল হলে স্ক্রিনি থেকে বেরিয়ে যায় ও সন্ধ্যাকবার দেখায়।

Widget build(BuildContext context)

`ticketDetailProvider(widget.id)` watch করে; বর্তমান ব্যবহারকারীর `ticketEditRoles/ticketDeleteRoles`-এর মধ্যে রোল থাকলে অ্যাপবারে `_TicketMenu` দেখায়। বডিতে লোডিং/এরর/ডটো অবস্থা হ্যান্ডল করে; ডটো থাকলে `_Header`, `_Facts`, (থাকলে) `রেজোলিউশন নোট` কার্ড, `_Conversation`, ও `_Actions` — এই সব ক্রমানুসারে একটি `ListView`-এ রেন্ডার করে, `RefreshIndicator` সহ।

ক্লাস/উইজটে: _Header

`StatelessWidget` — টিকেটের শিরোনাম, স্ট্যাটাস চপি, বর্ণনা এবং SLA/এস্কেলেশন ব্যানার প্রদর্শন করে।

const _Header({required this.ticket})

কনস্ট্রাক্টর।

Widget build(BuildContext context)

একটি `AppCard`-এ শিরোনাম ও `StatusChip`, বর্ণনা (থাকলে), SLA breach হলে একটি এরর ব্যানার, এবং এস্কেলেটেড হলে একটি ইনফো ব্যানার (এস্কেলেশন লভেলে ও তারখিসহ) রেন্ডার করে।

ক্লাস/উইজটে: _Facts

`StatelessWidget` — টিকেটের বিস্তারিত তথ্যের একটি লিবেল/মান তালিকা প্রদর্শন করে।

const _Facts({required this.ticket})

কনস্ট্রাক্টর।

প্রায়োরিটি, ক্যাটাগরি, রইজকারী, রইজরে তারিখ, হ্যান্ডলার, সোর্স, প্রথম-রসেপন্স ডেডলাইন, রজোলিউশন ডেডলাইন, কলোজড তারিখ, ও সনতুষ্টিরিটেং — এই রকেডগুলোর একটি তালিকা তৈরি করে, খালি/null মান বাদ দিয়ে ফিল্টিং করে, তারপর একটি AppCard-এ ডিভাইডার সহ সারাহিসিবে রনুডার করে।

ConsumerWidget — টকিটেরে কথোপকথন থ্রুডে ও রপ্‌লাই কম্পোজার প্রদর্শন করে।

কনস্ট্রাক্টর — থ্রেডে কী, কম্পোজার কন্ট্রোলার, পাঠানোর কলব্যাক, ও অ্যাটাচমেন্ট-সম্পর্কিত স্টেট/কলব্যাক নষে।

`ticketMessagesProvider(threadKey)` watch করে; `loading/error/data` হ্যান্ডল করে; ডটো থাকলে `ChatThread` রন্ডার করে, যা `disabled: !ticket.isOpen` ক্লোজড/রজেলভড টকিটে কথোপকথন লক থাকে, উপযুক্ত নোটসিসহ এবং থরডে-কাইনড অনুযায়ী পলসেহোল্ডার টকেসট দেখায়।

StatelessWidget — টকিটের বর্তমান স্ট্যাটাস অনুযায়ী সম্ভাব্য ক্রিয়া বাটন (resolve/close/reopen) প্রদর্শন করে।

কনস্ট্রাক্টর।

busy হল একটি **Loader** দেখায়। টকিতে ক্লোজড হল শুধু "Reopen" বাটন; রজেলভড হল "Reopen" ও "Close" — দুটি বাটন; অন্যথায় (খোলা) শুধু "Mark resolved" বাটন দেখায়। কোনো অবস্থাতই ভুল/অপরয়োজ্য করণী অফার করা হয় না।

(title, label, hint, confirm) রকেরড — resolve করবার জন্য পরমপট টকেসট ধারণ করে।

(title, label, hint, confirm) রকেরড — reopen ক্রিয়ার জন্য পরম্পট টক্‌সট ধারণ করে।

`askForText` দাখিলে একটি টেক্সট চাখে নখে; ফলাফল অ-null হলে `then` কলব্যাক কল করে।

```
static Future<String?> _askForText(BuildContext context, ({...}) prompt)
```

একটি `showModalBottomSheet` খুলে একটি ফর্ম দেখায় যাতায়ে থাকে শিরোনাম, একটি বিহু-লাইন `TextFormField` (খালি রাখলে ভ্যালিডেশন এরর — "This will be read by whoever picks the ticket up next"), ও একটি নিশ্চিতকরণ বাটন। নিশ্চিত হলে টিকিট করা টেক্সট রিটার্ন করে। ব্যাকএন্ড কোয়েরি প্যারামিটার হিসেবে খালি নোটও গ্রহণ করবে, তবে ইউআই স্তরে বাধ্যতামূলক করা হয়েছে যাতায়ে অডিটযোগ্য রজেক্টলিডেশন নিশ্চিত হয়।

ক্লাস/উইজটে: `_TicketMenu`

`StatelessWidget` — অ্যাপবারের ওভারফ্লো মেনুতে `Edit` ও `Delete` অপশন প্রদর্শন করে। একটি পৃথক উইজটে হিসেবে রাখা হয়েছে যাতায়ে কলব্যাকগুলো একটি `non-nullable SupportTicket` কলোজ করতে পারে।

```
const _TicketMenu({required this.ticket, required this.canEdit, required this.canDelete, required this.enabled, required this.onEdit, required this.onDelete})
```

কনস্ট্রাক্টর।

`Widget build(BuildContext context)`

একটি `PopupMenuButton` রেন্ডার করে; `canEdit/canDelete` অনুযায়ী "Edit ticket" ও "Delete" আইটেমে শর্তসাপেক্ষে দেখায়; নির্বাচনে যথাক্রমে `onEdit(ticket)` বা `onDelete(ticket)` কল করে।

ফাইল: `lib/features/support_admin/audit_detail_screen.dart`

সংক্ষিপ্ত বিবরণ: সাপোর্ট অডিট ট্রাইলরে একটি এন্ট্রির সম্পূর্ণ বিস্তারিত দেখানোর স্ক্রিনি, এবং একই স্ক্রিনিকে পুনর্ব্যবহার করে একটি রিসোর্স বা একজন ব্যবহারকারীর সাথে সম্পর্কিত সব এন্ট্রি দেখানোর স্ক্রিনি। তালিকার রো-তে শুধু সারাংশ থাকে; এখানে পরিবর্তন, IP, ইউজার-এজেন্ট এবং সম্পর্কিত ট্রাইলে যাওয়ার দুইটি পথ দেখানো হয়।

ক্লাস/উইজটে: `AuditDetailScreen`

`ConsumerWidget`, একটি নির্দিষ্ট `id`-এর অডিট এন্ট্রি লোড করে দেখায়। কনটেক্সট-সুইচ করে করা অ্যাকশন হলে সতর্কবার্তা দেখায়, "Details" কার্ডে রিসোর্স টাইপ/আইডি/আইপি/ইউজার-এজেন্ট দেখায়, `changes` থাকলে কাঁচা টেক্সট আকারে দেখায় (পার্স না করে, কারণ এর শেপে মডেল অনুযায়ী আলাদা), এবং নীচে দুটি বাটন দেয় — "এই রেকর্ডে যা যা ঘটছে" ও "এই ব্যক্তি যা যা করছে"।

```
const AuditDetailScreen({super.key, required this.id})
```

কনস্ট্রাক্টর, যেটি দেখানোর মতো এন্ট্রির `id` নেয়।

```
static void open(BuildContext context, {required int id})
```

`MaterialPageRoute` দিয়ে `AuditDetailScreen`-কে নতুন রাউট হিসেবে পুশ করে — কল করার জায়গা থেকে সরাসরি নিভেগিশন কোড লেখার দরকার নেই।

`Widget build(BuildContext context, WidgetRef ref)`

`auditEntryProvider(id)` ওয়াচ করে `async.when` দিয়ে লোডিং/এরর/ডটো তিন অবস্থা হ্যান্ডেল করে। ডটো এলে অ্যাকশন টাইপ, বিবরণ, কার দ্বারা ও কখন করা হয়েছে তা একটি কার্ডে দেখায়; কনটেক্সট-সুইচ কোম্পানির নাম থাকলে সতর্কবার্তা ব্যানার দেখায়। এরপর "Details" সেকশনে `_Fact` উইজটে দিয়ে রিসোর্স টাইপ, রিসোর্স আইডি, আইপি ও ইউজার-এজেন্ট সারবিধভাবে দেখায়, `changes` থাকলে সটিকে সলিক্বেল মনোস্পেসে টেক্সটে দেখায়, এবং শেষে `_RelatedButton` দিয়ে দুটি সম্পর্কিত-ট্রাইল স্ক্রিনি যাওয়ার বাটন দেখায়।

ক্লাস/উইজটে: _Fact

একটি লেবেল-ভ্যালু জোড়া সারবিদ্ধভাবে দেখানোর জন্য ছোট প্রাইভেট `StatelessWidget` — বাম দিকে ৬৪-পিক্সলে প্রশস্ত লেবেল, ডানে সলিকেটবেল ভ্যালু।

```
const _Fact(this.label, this.value)
```

পজিশনাল প্যারামিটার দিয়ে লেবেল ও ভ্যালু সেট করা কনস্ট্রাক্টর।

```
Widget build(BuildContext context)
```

একটি `Row`-এ ফক্সিড-প্রস্থ লেবেল ও `Expanded` সলিকেটবেল টেক্সট রেন্ডার করে, নচি প্যাডিং সহ।

ক্লাস/উইজটে: _RelatedButton

একটি পূর্ণ-প্রস্থ `OutlinedButton` `icon` মোডানো প্রাইভেট উইজটে, যা সম্পর্কিত-ট্রাইল স্ক্রিনে যাওয়ার বাটন হিসেবে ব্যবহৃত হয়।

```
const _RelatedButton({required this.label, required this.icon, required this.onPressed})
```

লেবেল, আইকন এবং প্রেস-হ্যান্ডলার নিয়ে কনস্ট্রাক্টর।

```
Widget build(BuildContext context)
```

`SizedBox(width: double.infinity)`-এর ভেতরে আইকন ও লেবেলসহ `OutlinedButton` তৈরি করে।

ক্লাস/উইজটে: RelatedTrailScreen

`ConsumerWidget`, যা একটি নির্দিষ্ট রিসোর্স বা ব্যক্তির সাথে সম্পর্কিত সব অডিট এন্ট্রি দেখায়। আইডি ও ফ্ল্যাগের বদলে সরাসরি `FutureProvider<List<SupportAuditEntry>>` নেয়, কারণ দুটলিকআপ ওয়্যারে ভিন্ন শেপ ফেরত দেয় এবং ভিন্নভাবে পজেড হয় — তাই প্রতিটি নিজস্ব রপিজিটরি মিথেড রাখা।

```
const RelatedTrailScreen({super.key, required this.title, required this.provider})
```

স্ক্রিনের শিরোনাম এবং ডটো আনার প্রোভাইডার নিয়ে কনস্ট্রাক্টর।

```
static void open(BuildContext context, {required String title, required  
FutureProvider<List<SupportAuditEntry>> provider})
```

`MaterialPageRoute` দিয়ে `RelatedTrailScreen`-কে টাইটলে ও প্রোভাইডারসহ পুশ করে।

```
Widget build(BuildContext context, WidgetRef ref)
```

দেওয়া `provider` ওয়াচ করে লোডিং/এরর/খালি/ডটো অবস্থা হ্যান্ডল করে। খালি হলে `EmptyState` দেখায়; ডটো থাকলে `ListView.builder` দিয়ে প্রতিটি এন্ট্রির অ্যাকশন টাইপ, ববিরণ (২ লাইন পর্যন্ত), করনওয়ালার নাম, আপেক্ষিক সময় এবং কনটেক্সট-সুইচ কোম্পানি (থাকলে) দেখায়; ট্যাপ করলে `AuditDetailScreen.open`-এ নিয়ে যায়।

ফাইল: lib/features/support_admin/support_admin_models.dart

সংক্ষিপ্ত ববিরণ: সাপোর্ট ডেস্ক পরিচালনার মডেলে — এজেন্ট, ক্যাটাগরি, SLA পলিসি এবং অডিট এন্ট্রি, এবং প্রতিটির জন্য `create/update` রকিওয়েস্ট ক্লাস। এই মডেলে পারমিশনের বদলে রোল-ভিত্তিক অ্যাক্সেস কন্ট্রোল ব্যবহৃত হয়, এবং প্রতিটি সারফেসের রোল সেট আলাদা — ফাইল শীর্ষে থাকা `constant`-গুলো প্রতিটি সারভার-সাইড `@PreAuthorize`-এর হুবহু প্রতিফলন।

টপ-লেভেলে কনস্ট্যান্ট: - `supportAgentAdminRoles` — এজেন্ট তরৈ/তালিকা/সম্পাদনা/স্ট্যাটাস সেট করার রোল (`SUPER_ADMIN`, `SUPPORT_MANAGER`)। - `acceptingTicketsRoles` — নিজের বা অধীনস্থ এজেন্টের অ্যাজাইলবেলিটি বিদলানোর রোল (`SUPPORT_AGENT`, `SUPPORT_MANAGER`)। - `supportCategoryAdminRoles` — ক্যাটাগরি লিখার রোল (`SUPER_ADMIN`, `SYSTEM_ADMIN`, `SUPPORT_MANAGER`)। - `supportCategoryViewRoles` — ক্যাটাগরি পড়ার রোল, ডেস্কে থাকা যেকোনো। - `slaAdminRoles` — SLA পলিসি লিখার রোল, শুধু প্ল্যাটফর্ম (`SUPER_ADMIN`, `SYSTEM_ADMIN`)। - `slaViewRoles` — SLA পলিসি পড়ার রোল। - `supportAuditRoles` — অডিট ট্রাইল দেখার রোল; লক্ষণীয় যে `SUPER_ADMIN` এতেনই, শুধু `SUPPORT_MANAGER` ও `COMPANY_OWNER`। - `availableAgentsFilter` — 'AVAILABLE', প্রকৃত `SupportAgentStatus` নয়, বরং `/available` এন্ডপয়েন্টে পৌঁছানোর একটি সাউন্ড-স্ট্যাটাস ফিল্টার, যা "কে এখনই টিকিট নতি পারবে" বোঝায়। - `agentStatuses` — `SupportAgentStatus`-এর প্রত্যাফলন: `ACTIVE`, `ON_BREAK`, `OFFLINE`, `VACATION`, `INACTIVE`।

ক্লাস/উইজটে: SupportAgent

সাপোর্ট ডেস্কে কর্মরত একজন এজেন্টের ডটো মডেলে — স্ট্যাটাস, টিকিট-গ্রহণ ক্ষমতা, সর্বোচ্চ কনকারেন্ট টিকিট সংখ্যা, পরিসংখ্যান ও যোগাযোগ তথ্য বহন করে।

`const SupportAgent({...})`

`id`, `status`, `acceptingTickets`, `maxConcurrentTickets`, `totalTicketsHandled` আবশ্যিক এবং বাকি ঐচ্ছিক ফিল্ড (`userId`, `fullName`, `email`, `department`, `specialization`, `notes`, গড় সাড়াদান/সমাধান সময়, সন্তুষ্টি স্কোর, সর্বশেষ সক্রিয় সময়) সহ কনস্ট্রাক্টর।

`String get displayName`

`fullName` অ-খালি থাকলে সেটি ফিরে দেয়, নাহলে `email`, তাও না থাকলে 'Agent \$id'।

`bool get isTakingWork`

এজেন্ট এখন কাজ নতি পারবে কনি — শুধু `status == 'ACTIVE'` যথেষ্ট নয়, `acceptingTickets` সত্যও হতে হবে, কারণ একজন এজেন্ট `ACTIVE` থেকেও নতুন টিকিট বন্ধ রাখতে পারেন।

`factory SupportAgent.fromJson(Map<String, dynamic> json)`

JSON থেকে অবজেক্ট তরৈ করে, প্রতটি সংখ্যাসূচক/বুলিয়ান ফিল্ডে নাল-সেফ ডিফল্ট বসায়। `fullName json['fullName']` অথবা ব্যাকআপ হিসেবে `json['userName']` থেকে আসে।

ক্লাস/উইজটে: SupportAgentRequest

POST ও PATCH `/v1/support/agents`-এ পাঠানো রকি়েস্ট বডি PATCH সব ফিল্ড শর্তহীনভাবে সেট করে দেয় — `department`, `specialization`, `maxConcurrentTickets`, `notes` সবই রকি়েস্ট থেকে সরাসরি বসে, কোনো নাল-চকে ছাড়াই। বিশেষভাবে বপিজ্জনক হলো `maxConcurrentTickets` — এটি জাভার প্রমিটিভি `int` যার ডিফল্ট মান ১০, তাই কী বাদ পড়লে জ্যাকসন শূন্য না রেখে জাভার ডিফল্ট ১০ রাখে এবং এজেন্টের লমিটি নশিব্দে ১০ হয়ে যায়। তাই এডিটি ফর্ম প্রতটি ফিল্ড এজেন্ট থেকে সেডি করে সবকিছু আবার পাঠায়। `userId` শুধু `create`-এর সময় অর্থবহ; PATCH সেটো উপেক্ষা করে।

`const SupportAgentRequest({required this.maxConcurrentTickets, this.userId, this.department, this.specialization, this.status, this.notes})`

আবশ্যিক `maxConcurrentTickets` এবং ঐচ্ছিক বাকি ফিল্ড নয়ো কনস্ট্রাক্টর।

`factory SupportAgentRequest.from(SupportAgent agent)`

বদ্যমান একটি `SupportAgent` থেকে রকি়েস্ট তরৈ করে — `maxConcurrentTickets`, `department`, `specialization`, `notes` কপি করে; `userId` ও `status` বাদ রাখে (এডিটি এগুলোর দরকার নই)।

Map<String, dynamic> toJson()

অভ্যন্তরীণ `clean` হলেপার ফাংশন দিয়ে স্ট্রিং ট্রিম করে খালি হলে `null` বানায়। `maxConcurrentTickets` সবসময় পাঠায়; `department`, `specialization`, `notes` সবসময় (নাল হলেও) পাঠায় কারণ সার্ভার-সাইডে শর্তহীন অ্যাসাইনমেন্ট, তাই `null` পাঠানোই মুছে ফেলোর একমাত্র উপায়; `userId` ও `status` শুধু নন-নাল হলে পাঠায়।

ক্লাস/উইজটে: SupportCategory

একটা টিকিট যি ক্যাটাগরির অধীনে ফাইল করা হয় তার মডলে — নাম, সক্রিয়তা, ববিরণ, আইকন বহন করে।

```
const SupportCategory({required this.id, required this.name, required this.active,
this.description, this.icon})
```

কনস্ট্রাক্টর।

```
factory SupportCategory.fromJson(Map<String, dynamic> json)
```

JSON থেকে তৈরি করে; ব্যাকএন্ড ফিল্ডের নাম `categoryName`, `name` নয় — তাই এখানে ম্যাপিং করা হয়। `active` ডিফল্ট `true`।

ক্লাস/উইজটে: SupportCategoryRequest

POST ও PATCH `/support/categories`-এর রকোয়েস্ট বডি। PATCH `name`, `description`, `icon` শর্তহীনভাবে সেটে করে, তাই তিনটিই সবসময় পাঠানো হয়। `active` স্পর্শ করে না — তার নজিস্ব আলাদা এন্ডপয়েন্টে আছে, তাই এই রকোয়েস্টে `active` নাই।

```
const SupportCategoryRequest({required this.name, this.description, this.icon})
```

কনস্ট্রাক্টর।

```
Map<String, dynamic> toJson()
```

`clean` হলেপার দিয়ে ট্রিম করে; `categoryName`, `description`, `icon` সবসময় (দরকারমতো নাল সহ) পাঠায়।

ক্লাস/উইজটে: SlaPolicy

প্রতিটি প্রায়োরিটির জন্য ডেস্ক কী প্রতশ্রুতি দিয়ে তার মডলে — প্রথম সাড়াদান ও সমাধানের সময়সীমা, বজিনসে-আওয়ার সীমাবদ্ধতা, সক্রিয়তা বহন করে।

```
const SlaPolicy({...})
```

`id`, `name`, `priority`, `firstResponseTimeHours`, `resolutionTimeHours`, `businessHoursOnly`, `active` আবশ্যিক এবং `description`, `notes` ঐচ্ছিক।

```
factory SlaPolicy.fromJson(Map<String, dynamic> json)
```

JSON থেকে তৈরি করে; `name` আসে `json['policyName']` থেকে, `priority` আসে `json['applicablePriority']` থেকে, ডিফল্ট `'MEDIUM'`।

ক্লাস/উইজটে: SlaPolicyRequest

POST ও PATCH `/support/sla-policies`-এর রকোয়েস্ট বডি। PATCH-যেসব ফিল্ড স্পর্শ করে সব শর্তহীনভাবে সেটে হয়, এবং দুটা ফিল্ড — ঘণ্টার সংখ্যা — প্রমিটিভি `int`, তাই বাদ পড়লে শূন্য হিসেবে আসত (অর্থাৎ তাৎক্ষণিক সাড়ার প্রতশ্রুতি) — তাই সবগুলো সবসময় পাঠানো হয়। `notes` ব্যতিক্রম: `create` এটি সংরক্ষণ করে, `update` কখনো পড়ে না — তাই এডটি ফর্ম এটি দেখানো হয় না।


```
const SlaPolicyRequest({required this.name, required this.priority, required
this.firstResponseTimeHours, required this.resolutionTimeHours, required
this.businessHoursOnly, required this.active, this.description, this.notes})
কনস্ট্রাক্টর।
```

```
Map<String, dynamic> toJson()
```

```
policyName, applicablePriority, firstResponseTimeHours, resolutionTimeHours, businessHoursOnly,
active সবসময় পাঠায়; description ট্রমি করে খালি হিলে নাল; notes শুধু ট্রমি করে অ-খালি হিলে পাঠায়।
```

ক্লাস/উইজটে: SupportAuditEntry

সাপোর্ট ডেস্কেরে অডিট ট্রাইলরে একটিলাইন — কে, কী, কখন, কোথা থেকে, কী পরিবর্তন হলো তা ধারণ করে।

```
const SupportAuditEntry({required this.id, required this.actionType, this.actionByUserName,
this.description, this.resourceType, this.resourceId, this.contextSwitchToCompanyName,
this.createdAt, this.actionById, this.changes, this.ipAddress, this.userAgent})
```

কনস্ট্রাক্টর, যেখানে contextSwitchToCompanyName (কোম্পানিতে কনটেক্সট-সুইচ করে করা হলে সবচেয়ে গুরুত্বপূর্ণ তথ্য), actionByUserId ("এই ব্যক্তি যা করেছে" লুকআপের জন্য প্রয়োজন) এবং changes (মডিউলভদে ভিন্নরূপ ফর-ফর্ম JSON, তাই পার্স না করে যেমন এসছে তেমন দেখানো হয়) উল্লেখযোগ্য।

```
factory SupportAuditEntry.fromJson(Map<String, dynamic> json)
```

JSON থেকে অবজেক্ট তৈরি করে, প্রতিটি ফিল্ডে যথাযথ নাল-সফে কনভারশন সহ।

ক্লাস/উইজটে: AuditFilter

অডিট তালিকা কীভাবে সংকীর্ণ করা হয় তার মডেল। ব্যাকএন্ডে প্রতিটি ফিল্টারের জন্য আলাদা এন্ডপয়েন্ট আছে (একটি এন্ডপয়েন্টে ঐচ্ছিক প্যারামিটার নয়), তাই ফিল্টারের পছন্দই URL নির্ধারণ করে। স্ক্রিনিংর জন্য একটাই জেনিসি ধরে রাখার সুবিধার্থে এটি একটি অবজেক্টে বাঁধা।

```
const AuditFilter({this.actionType, this.start, this.end})
```

কনস্ট্রাক্টর; actionType ফর-টেক্সট অ্যাকশন-টাইপ স্ট্রিং (এনাম নয়, কারণ সেটা থকেই আসে); start/end উভয়ে বা কোনোটিই না, yyyy-MM-dd ফরম্যাটে যা @DateTimeFormat(ISO.DATE)-এর প্রত্যাশা।

```
bool get isEmpty
```

actionType == null && start == null হলে সত্য — কোনো ফিল্টার সক্রিয় নই।

```
bool get hasDateRange
```

start ও end উভয়ই নন-নাল হলে সত্য।

ফাইল: lib/features/support_admin/support_admin_repository.dart

সংক্ষিপ্ত বিবরণ: সাপোর্ট ডেস্কের জন্য HTTP রপ্যোজিটরি এবং Riverpod প্রোভাইডার/কন্ট্রোলার — এজেন্ট, ক্যাটাগরি, SLA পলিসিও অডিট ট্রাইলরে জন্য। লক্ষণীয় যে এজেন্ট /v1/support/... প্রফিক্সে আর ক্যাটাগরি/SLA/অডিট পলইন /support/... প্রফিক্সে থাকে — এটি ব্যাকএন্ডের গঠন, টাইপো নয়।

ক্লাস/উইজটে: SupportAdminRepository

সব সাপোর্ট-অ্যাডমিনি HTTP কল কন্ড্রীভূত করা রপিওজটির ক্লাস, **ApiClient** মুড়ে।

SupportAdminRepository(this._api)

ApiClient ইনজেক্ট করা কনস্ট্রাক্টর।

Future<PagedResponse<SupportAgent>> agents({String? status, int page = 0, int size = 30})

এজেন্টদের পুরো তালিকা বা একটি স্ট্যাটাসে সংকীর্ণ তালিকা আনবে। **status** একটি পাথ-সেগমেন্ট, কোয়েরি-প্যারামিটার নয় — **status == null** হলে বেসে পাথ, **availableAgentsFilter** হলে **/available**, নাহলে **/status/\$status**। **/available** একটি বিয়ের লিস্ট ফরমের দিয়ে (পেজ নয়), কিন্তু **getPaged** উভয় শেপেই পড়তে পারে।

Future<SupportAgent> createAgent(SupportAgentRequest request)

প্ল্যাটফর্ম ব্যবহারকারীকে এজেন্ট বানায়। **userId** আবশ্যিক, কিন্তু ব্যাকএন্ড শুধু **plain** **IllegalArgumentException** দিয়ে ভ্যালিডেট করে, তাই ফর্মটাজিমা দ্বয়ের আগাই **userId** আছে কনি যাচাই করে।

Future<SupportAgent> updateAgent(int id, SupportAgentRequest request)

পুরো রেকর্ড PATCH করে পাঠায় — দেখুন **SupportAgentRequest**-এর নোট, বাদ পড়া ফিল্ড অস্পৃশ্য থাকে না।

Future<void> setAgentStatus(int id, String status)

কোয়েরি-প্যারামিটার দিয়ে এজেন্টের স্ট্যাটাস বদলায়; রসেপন্স খালি, তাই পার্স করার কিছু নেই।

Future<void> setAcceptingTickets(int id, bool accepting)

কোয়েরি-প্যারামিটার দিয়ে নতুন টিকিট নেওয়া চালু/বন্ধ করে; রসেপন্স খালি।

Future<PagedResponse<SupportCategory>> categories({int page = 0, int size = 50})

ক্যাটাগরি পেজেড তালিকা আনবে।

Future<SupportCategory> createCategory(SupportCategoryRequest request)

নতুন ক্যাটাগরি তৈরি করে।

Future<SupportCategory> updateCategory(int id, SupportCategoryRequest request)

বর্তমান ক্যাটাগরি PATCH করে।

Future<void> setCategoryActive(int id, bool active)

কোয়েরি-প্যারামিটার দিয়ে ক্যাটাগরি সক্রিয়/নিস্ক্রিয় (রেটিয়ার বা রসিটোর) করে; রসেপন্স খালি।

Future<void> deleteCategory(int id)

ক্যাটাগরি স্থায়ীভাবে মুছে দেয়।

Future<PagedResponse<SlaPolicy>> slaPolicies({String? priority, int page = 0, int size = 50})

SLA পলিসি তালিকা আনবে; এজেন্ট ফিল্টারের মতোই — **priority** পাথ-সেগমেন্ট, এবং সেই এন্ডপয়েন্ট বিয়ের লিস্ট ফরমের দিয়ে যা **getPaged** সহ্য করে।

Future<SlaPolicy> createSlaPolicy(SlaPolicyRequest request)

নতুন SLA পলিসি তৈরি করে।

`Future<SlaPolicy> updateSlaPolicy(int id, SlaPolicyRequest request)`

বর্তমান SLA পলিসি PATCH করে।

`Future<void> setSlaActive(int id, bool active)`

কোয়েরি-প্যারামিটার দিয়ে পলিসি সক্রিয়/নসক্রিয় করে।

`Future<void> deleteSlaPolicy(int id)`

SLA পলিসি স্থায়ীভাবে মুছে দেয়।

`Future<PagedResponse<SupportAuditEntry>> auditLog({AuditFilter filter = const AuditFilter(), int page = 0, int size = 25})`

ফিল্টার অনুযায়ী পাতা নির্ধারণ করে অডিট ট্রাইল আনবে — `actionType` থাকলে `/action/{encoded}`, `hasDateRange` থাকলে `/date-range` সহ `start/end` কোয়েরি-প্যারামিটার, নাহলে বেসে পাতা।

`Future<SupportAuditEntry> auditEntry(int id)`

একটি নির্দিষ্ট অডিট এন্ট্রির সম্পূর্ণ তথ্য আনবে (পরিবর্তন, আইপি, ব্রাউজারসহ)।

`Future<List<SupportAuditEntry>> auditForResource(int resourceId)`

একটি নির্দিষ্ট রিসোর্সে যা ঘটছে তার সব এন্ট্রি বিয়ের লিস্ট হিসেবে আনবে — পজে নয়, কারণ একক রিসোর্সের ট্রাইল স্বভাবতই সংক্ষিপ্ত।

`Future<PagedResponse<SupportAuditEntry>> auditForUser(int userId, {int page = 0, int size = 25})`

একজন ব্যক্তি যা করছে তার এন্ট্রি পিজেড আকারে আনবে (রিসোর্স-লুকআপ থেকে ভিন্ন)।

`Future<List<SlaPolicy>> activeSlaPolicies()`

বর্তমানে বলবৎ SLA পলিসি (সব লকো পলিসি নয়) বিয়ের লিস্ট হিসেবে আনবে।

`Future<SupportAgent> agentForUser(int userId)`

একটি ইউজার অ্যাকাউন্টের পছন্দে এজেন্ট রেকর্ড থাকলে স্টেট আনবে; এজেন্ট না হলে 404 আসবে, যা কলার "এজেন্ট নয়" হিসেবে ধরে, ব্যর্থতা হিসেবে নয়।

টপ-লেভেলে প্রোভাইডার

`supportAdminRepositoryProvider` — `SupportAdminRepository`-এর `Provider`।

ক্লাস/উইজটে: AgentStatusFilterController

এজেন্ট তালিকা কোন স্ট্যাটাসে সংকীর্ণ তা ধরে রাখা `Notifier<String?> | null` মানবে সবাই।

`String? build()`

প্রাথমিক অবস্থা `null` (কোনো ফিল্টার নেই) ফেরত দেয়।

`void set(String? status)`

নতুন স্ট্যাটাস ফিল্টার স্টেট করে; আগেরটির সমান হলে কিছু করে না (অপ্রয়োজনীয় রিবিল্ড এড়ায়)।

`agentStatusFilterProvider` — উপরের কন্ট্রোলারের `NotifierProvider`।

ক্লাস/উইজটে: SupportAgentsController

এজেন্ট তালিকা লোড ও রফির্শে পরিচালনা করা `AsyncNotifier<List<SupportAgent>> |`

`Future<List<SupportAgent>> build()`

`currentUserProvider` ও `agentStatusFilterProvider` ওয়াচ করে (ফিল্টার বদলালে পুরো কন্ট্রোলার রিবিল্ড হয়) এবং `_load()` কল করে।

`Future<List<SupportAgent>> _load()`

বর্তমান ফিল্টার-স্ট্যাটাস দিয়ে রপিজিটরি থেকে এজেন্ট আনে এবং `content` ফরম্যাট দিয়ে।

`Future<void> refresh()`

`AsyncValue.guard` দিয়ে `_load` আবার চালিয়ে স্টেটে আপডেট করে (পুল-টু-রিফ্রেশ)।

`void apply(SupportAgent updated)`

বর্তমান তালিকায় নরমাল আইডি এজেন্টকে নতুন ভার্সন দিয়ে প্রতিস্থাপন করে (কোনো রেসপন্স-বডি না থাকা এন্ডপয়েন্টের পর অপটিমিস্টিক আপডেটের জন্য ব্যবহৃত)।

`supportAgentsProvider` — উপরে কন্ট্রোলারের `AsyncNotifierProvider`।

ক্লাস/উইজটে: `SupportCategoriesController`

ক্যাটাগরি তালিকা লোড, রিফ্রেশ ও ইন-পুলসে আপডেট পরিচালনা করা

`AsyncNotifier<List<SupportCategory>>`।

`Future<List<SupportCategory>> build()`

`currentUserProvider` ওয়াচ করে `_load()` কল করে।

`Future<List<SupportCategory>> _load()`

রপিজিটরি থেকে ক্যাটাগরি পিজেসে `content` আনে।

`Future<void> refresh()`

`_load` পুনরায় চালিয়ে স্টেটে রিফ্রেশ করে।

`void apply(SupportCategory updated)`

নরমাল আইডি সারি প্রতিস্থাপন করে।

`void remove(int id)`

নরমাল আইডি সারি তালিকা থেকে বাদ দিয়ে।

`supportCategoriesProvider` — উপরে কন্ট্রোলারের `AsyncNotifierProvider`।

ক্লাস/উইজটে: `SlaPriorityFilterController`

SLA তালিকা কোন প্রায়োরিটিতে সংকীর্ণ তা ধরে রাখা `Notifier<String?>`।

`String? build()`

প্রাথমিক `null` ফরম্যাট দিয়ে।

`void set(String? priority)`

নতুন প্রায়োরিটি ফিল্টার সেট করে, অপরিসীম হলে কিছু করে না।

`slaPriorityFilterProvider` — উপরে কন্ট্রোলারের `NotifierProvider`।

ক্লাস/উইজটে: SlaPoliciesController

SLA পলিসি তালিকা লোড/রফ্রেশ/আপডেট/মুছে ফেলো পরচালনা করা
`AsyncNotifier<List<SlaPolicy>> I`

`Future<List<SlaPolicy>> build()`

`currentUserProvider` ও `slaPriorityFilterProvider` ওয়াচ করে `_load()` কল করে।

`Future<List<SlaPolicy>> _load()`

ফিল্টার-প্রায়োরিটি একটি সিন্থনিলে ("শুধু বলবৎ") বহন করতে পারে, যা আসলে একটা আলাদা এন্ডপয়েন্ট বোঝায়, প্রায়োরিটি নয় — তাই `ticketPriorities`-এ নই এমন ফিল্টার-মান এড়িয়ে `priority: null` ব্যবহার করে, নাহলে সরাসরি পাঠালে 400 এরর হতো।

`Future<void> refresh()`

`_load` পুনরায় চালিয়ে স্টেটে রফ্রেশ করে।

`void apply(SlaPolicy updated)`

নরিদর্শিত আইডির সারি প্রতিস্থাপন করে।

`void remove(int id)`

নরিদর্শিত আইডির সারি বিদ্যে।

`slaPoliciesProvider` — উপরে কন্ট্রোলারের `AsyncNotifierProvider`।

ক্লাস/উইজটে: AuditFilterController

বর্তমানে বলবৎ অডিট ফিল্টার ধরে রাখা `Notifier<AuditFilter> I`

`AuditFilter build()`

প্রাথমিক অবস্থায় খালি `AuditFilter` ফেরত দেয়।

`void set(AuditFilter filter)`

নতুন ফিল্টার সেট করে।

`void clear()`

ফিল্টার খালতি রসিটে করে।

`auditFilterProvider` — উপরে কন্ট্রোলারের `NotifierProvider`।

ক্লাস/উইজটে: SupportAuditController

অডিট তালিকা লোড ও রফ্রেশ পরচালনা করা `AsyncNotifier<List<SupportAuditEntry>> I`

`Future<List<SupportAuditEntry>> build()`

`currentUserProvider` ও `auditFilterProvider` ওয়াচ করে `_load()` কল করে।

`Future<List<SupportAuditEntry>> _load()`

বর্তমান ফিল্টার দিয়ে রপিজিটরি থেকে অডিট লগ পজেরে `content` আনবে।

`Future<void> refresh()`

`_load` পুনরায় চালিয়ে স্টেটে রফ্রেশ করে।

`supportAuditProvider` — উপরে কন্ট্রোলারের `AsyncNotifierProvider`।

টপ-লভেলে প্রোভাইডারসমূহ (বাকি)

- `auditEntryProvider` — একটি `id`-এর জন্য `auditEntry` কল করা
`FutureProvider.autoDispose.family<SupportAuditEntry, int> |` - `auditForResourceProvider` —
`resourceId`-এর জন্য `auditForResource` কল করা
`FutureProvider.autoDispose.family<List<SupportAuditEntry>, int> |` - `auditForUserProvider` —
`userId`-এর জন্য `auditForUser` কল করে `content` ফেরত দেয়
`FutureProvider.autoDispose.family<List<SupportAuditEntry>, int> |` - `activeSlaPoliciesProvider` —
`activeSlaPolicies` কল করা `FutureProvider.autoDispose<List<SlaPolicy>> |`

ফাইল: `lib/features/support_admin/support_admin_screen.dart`

সংক্ষিপ্ত বিবরণ: সাপোর্ট ডেস্ক পরিচালনার মূল স্ক্রিনি — এজেন্ট, ক্যাটাগরি, SLA ও ট্রাইল, চারটি ট্যাবে বিভক্ত, প্রতিটি ট্যাব ব্যবহারকারীর রোল অনুযায়ী দেখানো/লুকানো হয়। রোল-গেটেডে, পারমিশন-গেটেডে নয়।

ক্লাস/উইজটে: `SupportAdminScreen`

`ConsumerWidget`, ব্যবহারকারীর রোল অনুযায়ী উপলব্ধ ট্যাব গণনা করে, একটিও না থাকলে "Not available to you" দেখায়, নাহলে `DefaultTabController` দিয়ে ট্যাবড স্ক্রিনি এবং প্রতিটি ট্যাবের নিজস্ব ফ্লোটিং অ্যাকশন বাটন (যদি সেই ট্যাবে "create" থাকে) দেখায়।

`const SupportAdminScreen({super.key, this.initialTabLabel})`

কনস্ট্রাক্টর; `initialTabLabel` দিলে সেই লবেলের ট্যাব দিয়ে শুরু হয়।

`Widget build(BuildContext context, WidgetRef ref)`

`currentUserProvider` থেকে ব্যবহারকারী নিয়ে `can(roles)` হলেপার দিয়ে চেক করে কোন কোন ট্যাব (`Agents/Categories/SLA/Trail`) দেখানো যাবে, প্রতিটির `view` ও ঐচ্ছিক `create` কলব্যাক সহ একটি রেকর্ড-টাইপ তালিকা বানায়। ট্যাব খালি থাকলে লক-আইকনসহ `EmptyState` দেখায়। নাহলে `initialTabLabel` থেকে শুরুর ইনডেক্স বের করে `DefaultTabController` দিয়ে `AnimatedBuilder` ব্যবহার করে সক্রিয় ট্যাব অনুযায়ী FAB দেখায়/লুকায়ে, এবং `TabBarView`-এ সব ভিউ রাখে।

ক্লাস/উইজটে: `_AgentsTab`

এজেন্ট তালিকার ট্যাব, `ConsumerStatefulWidget`।

`ConsumerState<_AgentsTab> createState()`

`_AgentsTabState` ইনস্ট্যান্স তৈরি করে।

ক্লাস/উইজটে: `_AgentsTabState`

এজেন্ট তালিকার স্টেটে — স্ট্যাটাস/অ্যাক্সপেট্টিং টগল ও ব্যস্ততা-ইনডিকটর পরিচালনা করে।

`Future<void> _run(SupportAgent agent, Future<void> Function() action, SupportAgent optimistic, String failure)`

জেনেরিক হলেপার: `_busyId` সেটে করে অ্যাকশন চালায়; স্ট্যাটাস/অ্যাবাইলবিলিটি এন্ডপয়েন্ট কোনো রসেপন্স-বডি ফেরত দেয় না বলে সফল হলে `optimistic` কপি দিয়ে

`supportAgentsProvider.notifier.apply` কল করে; `ApiException` বা অন্য এরর হলে সন্ধ্যাকবারে বার্তা দেখায়; শেষে `_busyId` রসিটে করে।

Future<void> _setStatus(SupportAgent agent)

pickOne ডায়ালগ দ্বিধে নতুন স্ট্যাটাস বছে নয়ো, একই স্ট্যাটাস বা বাতলি হলো কছি না করা, নাহলে repo.setAgentStatus কল করে _run দ্বিধে আপডটে করা।

Future<void> _toggleAccepting(SupportAgent agent)

acceptingTickets উল্টো repo.setAcceptingTickets কল করে _run দ্বিধে প্রতফিলতি করে।

Widget build(BuildContext context)

স্ট্যাটাস-ফিল্টার ও acceptingTicketsRoles অনুযায়ী canAccept বরে করে; ConfigList<SupportAgent> রন্ডার করে যখনে FilterBar-এ "Everyone", "Free now", প্রতটি স্ট্যাটাস দখোয় এবং প্রতটি সারতি এজন্টে নাম, সক্রিয়তা, সাবটাইটলে (স্ট্যাটাস, গ্রহণক্ষমতা, বিভাগ), হ্যান্ডলে-করা টকিটি সংখ্যা এবং "Change status" ও (যোগ্য হলো) "Stop/Take new tickets" অ্যাকশন দখোয়।

টপ-লভেলে ফাংশন: SupportAgent _copyWithStatus(SupportAgent agent, {String? status, bool? accepting})

স্ট্যাটাস বা অ্যাক্সপেটিং-ফল্গা পরবিত্তন করে একটি নতুন SupportAgent অবজেক্ট তরৈকিরে, বাকি সব ফিল্ড অপবিত্ততি রখে — কারণ স্ট্যাটাস/অ্যাক্সপেটিং এন্ডপয়ন্টে রসেপন্স-বডি ফিরেত দয়ে না, তাই যা রকি়োয়স্ট করা হয়ছেলি তা থকেই সারা পুনরনরিমাণ করা হয়, শুধু কলটি সফল হওয়ার পরই।

ক্লাস/উইজটে: _CategoriesTab

ক্যাটাগরিতালিকার ট্যাব, ConsumerStatefulWidget।

ConsumerState<_CategoriesTab> createState()

_CategoriesTabState ইন্সট্যান্স তরৈকিরে।

ক্লাস/উইজটে: _CategoriesTabState

ক্যাটাগরি টগল/মোছার স্টেটে পরচালনা করে।

Future<void> _toggle(SupportCategory row)

setCategoryActive কল করে active উল্টো দয়ে ও ফলাফল দ্বিধে supportCategoriesProvider.notifier.apply কল করে (রসেপন্স-বডি না থাকায় ম্যানুয়াল পুনরনরিমতি অবজেক্ট দ্বিধে); এরর হলো স্ন্যাকবার দখোয়।

Future<void> _delete(SupportCategory row)

confirmAction ডায়ালগ দ্বিধে নশ্চিতি করে (বেলে যে বদ্যমান টকিটি ক্যাটাগরি ধরে রাখবে, রটিয়ার করাই বরং নরিপদ), নশ্চিতি হলো deleteCategory কল করে remove দ্বিধে তালিকা থকে বাদ দয়ে ও সাফল্য/ব্যর্থতা স্ন্যাকবার দখোয়।

Widget build(BuildContext context)

supportCategoryAdminRoles অনুযায়ী canWrite বরে করে ConfigList<SupportCategory> রন্ডার করে, প্রতটি সারতি নাম, সক্রিয়তা, ববিরণ এবং (লেখোর অনুমতি থাকলে) এডিট, টগল, "Remove" অ্যাকশন দখোয়।

ক্লাস/উইজটে: _SlaTab

SLA পলসিতালিকার ট্যাব, ConsumerStatefulWidget।

ConsumerState<_SlaTab> createState()

_SlaTabState ইন্সট্যান্স তরৈকিরে।

টপ-লভেলে কনস্ট্যান্ট: `_inForce`

'in-force' — শুধু বর্তমানে বলবৎ পলসিদিখোনার ফিল্টার-মান। প্রায়োরটি নিয়, তাই সংঘর্ষ হয় না, কারণ ব্যাকএন্ডের প্রায়োরটিগুলো সবসময় SCREAMING_SNAKE কসে থাকে।

ক্লাস/উইজটে: `_SlaTabState`

SLA পলসিটিগল/মোছার স্টেটে পরচালনা করে।

`Future<void> _toggle(SlaPolicy row)`

`setSlaActive` কল করে `active` উল্টে দিয়ে ফলাফল দিয়ে `apply` কল করে; এরর হলে স্ন্যাকবার দেখায়।

`Future<void> _delete(SlaPolicy row)`

`confirmAction` দিয়ে নিশ্চিতি করে (বলে যে ওই প্রায়োরটির টিকিটি অন্য পলসিতি বা কোনোটিকেই পড়বে না), নিশ্চিতি হলে `deleteSlaPolicy` কল করে `remove` দিয়ে তালিকা থেকে বাদ দেয়।

`Widget build(BuildContext context)`

প্রায়োরটি-ফিল্টার ও `slaAdminRoles` অনুযায়ী `canWrite` বের করে; `filter == _inForce` হলে `activeSlaPoliciesProvider` নাহলে `slaPoliciesProvider` ওয়াচ করে; `onRefresh`-এ উভয় প্রোভাইডার রিফ্রেশ/ইনভ্যালিডেট করে; `FilterBar`-এ "All priorities", "In force", প্রতটি প্রায়োরটি দেখায়; প্রতটি সারতি নাম, সক্রিয়তা, সাবটাইটলে (প্রায়োরটি, সাড়াদান/সমাধান সময়, বজিনসে-আওয়ার) এবং এডিট/টগল/"Remove" অ্যাকশন দেখায়।

ক্লাস/উইজটে: `_AuditTab`

অডিট ট্রাইল তালিকার ট্যাব, `ConsumerWidget`।

`Widget build(BuildContext context, WidgetRef ref)`

`auditFilterProvider` ও `supportAuditProvider` ওয়াচ করে `ConfigList<SupportAuditEntry>` রেন্ডার করে; হডোর বর্তমান ফিল্টারের বিবরণ (সবকছি/নির্দিষ্ট অ্যাকশন টাইপ/তারিখ-পরসির) এবং "Filter" বাটন দেখায় যা `showAuditFilterSheet` খোলে; প্রতটি সারতি `_AuditRow` ব্যবহার করে।

ক্লাস/উইজটে: `_AuditRow`

একটি অডিট-এন্ট্রির সারাংশ-সারি, `StatelessWidget`।

`const _AuditRow({required this.entry})`

এন্ট্রি নিয়ো কনস্ট্রাক্টর।

`Widget build(BuildContext context)`

`AppCard`-এ ট্যাপ করলে `AuditDetailScreen.open`-এ নিয়ে যায়; অ্যাকশন টাইপ, আপেক্ষিক সময়, বিবরণ (থাকলে), করনওয়ালার নাম + রসিোর্স টাইপ/আইডি, এবং কনটেক্সট-সুইচ কোম্পানি (থাকলে সতর্ক আইকনসহ) দেখায়।

ফাইল: `lib/features/support_admin/support_admin_sheets.dart`

সংক্ষিপ্ত বিবরণ: সাপোর্ট ডেস্কে সব তরৈ/সম্পাদনা বটম-শিট ফর্ম — এজেন্ট, ক্যাটাগরি, SLA পলসি এবং অডিট ফিল্টার শিট। প্রতটি ফর্ম `FormSheetFrame` ব্যবহার করে একই প্যাটার্নে জমা দেয়, এরর দেখায়, প্রাসংগিক প্রোভাইডার আপডেট করে।

টপ-লভেলে ফাংশন: Future<void> showSupportAgentSheet(BuildContext context, {SupportAgent? existing})

_AgentSheet মডাল বটম-শিট দখোয়; existing দলি এডটি মোড, না দলি ক্রয়িটে মোড।

ক্লাস/উইজটে: _AgentSheet

এজেন্ট তরৈসিম্পাদনার ফর্ম উইজটে, ConsumerStatefulWidget।

const _AgentSheet({this.existing})

কনস্ট্রাক্টর।

ConsumerState<_AgentSheet> createState()

_AgentSheetState তরৈকিরে।

ক্লাস/উইজটে: _AgentSheetState

এজেন্ট ফর্মের স্টেটে — টেক্সট কন্ট্রোলার, ইউজার নির্বাচন, স্ট্যাটাস, জমাদান ও এরর পরচালনা করে।

bool get _isEdit

widget.existing != null কনি।

void dispose()

_department, _specialization, _notes, _maxTickets কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে; ক্রয়িটে মোডে ব্যবহারকারী বাছাই করা না থাকলে এরর দখোয়;

SupportAgentRequest বানয়ি এডটি হল updateAgent + apply, নাহলে createAgent + refresh কল করে; সফল হলে শিট বন্ধ করে স্ন্যাকবার দখোয়; ব্যর্থ হলে এরর মেসেজে স্টেট করে।

Widget build(BuildContext context)

FormSheetFrame-এর মধ্যে ক্রয়িটে মোডে _UserField ও শুরুর স্ট্যাটাস ড্রপডাউন, তারপর বিভাগ, বিশেষত্ব, "Tickets at once" (সংখ্যা ভ্যালিডেশনসহ, ব্যাকএন্ডের @Min(1) মেলানো), নোটস ফিল্ড দখোয়; এডটি মোডে অতিরিক্ত নোট দখোয় যে স্ট্যাটাস/অ্যাবাইলিটি এখান থেকে বদলানো যায় না।

ক্লাস/উইজটে: _UserField

কোন প্ল্যাটফর্ম ইউজার এজেন্ট হবে তা বছে নেয়ার ড্রপডাউন, ConsumerWidget। একজন ইউজার শুধু একবারই এজেন্ট হতে পারে — দ্বিতীয়বার চেষ্টা করলে ব্যাকএন্ড "User is already a support agent" দয়ি প্রত্যাখ্যান করে।

const _UserField({required this.value, required this.onChange})

বর্তমান নির্বাচতি ইউজার ও পরিবর্তন-কলব্যাক নয়ো কনস্ট্রাক্টর।

Widget build(BuildContext context, WidgetRef ref)

platformUsersProvider ওয়াচ করে লোডিং/এরর/খালি/ডটো হ্যান্ডল করে; ডটো থাকলে প্রতিটি ইউজারের নাম ও ইমেইল দেখয়ি DropdownButtonFormField<int> রেন্ডার করে, নির্বাচন হলে সংশ্লিষ্ট PlatformUser অবজেক্ট onChange-এ পাঠায়; ভ্যালিডেটের নিশ্চিতি করে কডে বাছাই করা হয়ছে।

টপ-লেভেলে ফাংশন: Future<void> showSupportCategorySheet(BuildContext context, {SupportCategory? existing})

_CategorySheet মডাল বটম-শিট দেখায়।

ক্লাস/উইজটে: _CategorySheet

ক্যাটাগরি তৈরি/সম্পাদনার ফর্ম, ConsumerStatefulWidget।

const _CategorySheet({this.existing})

কনস্ট্রাক্টর।

ConsumerState<_CategorySheet> createState()

_CategorySheetState তৈরি করে।

ক্লাস/উইজটে: _CategorySheetState

ক্যাটাগরি ফর্মের স্টেটে পরিচালনা করে।

bool get _isEdit

এডিট মোড কনি।

void dispose()

_name, _description কন্ট্রোলার ডিসপোজ করে।

Future<void> _submit()

ফর্ম ভ্যালিডেট করে; SupportCategoryRequest বানায় যখন icon বদলিমান রেকর্ড থেকে অপরিবর্তিতভাবে বহন করা হয় (অ্যাপ আইকন বাছাই করে না, কিন্তু ফিল্ড শর্তহীনভাবে সেটে হয়, তাই বাদ দিলে ওয়বে সেটে করা আইকন মুছে যতে); এডিট হলে updateCategory + apply, নাহলে createCategory + refresh; সফল/ব্যর্থ হ্যান্ডলিং।

Widget build(BuildContext context)

FormSheetFrame-এ নাম (আবশ্যিকি ভ্যালিডেশন) ও বিবরণ ফিল্ড দেখায়।

টপ-লেভেলে ফাংশন: Future<void> showSlaPolicySheet(BuildContext context, {SlaPolicy? existing})

_SlaSheet মডাল বটম-শিট দেখায়।

ক্লাস/উইজটে: _SlaSheet

SLA পলিসি তৈরি/সম্পাদনার ফর্ম, ConsumerStatefulWidget।

const _SlaSheet({this.existing})

কনস্ট্রাক্টর।

ConsumerState<_SlaSheet> createState()

_SlaSheetState তৈরি করে।

ক্লাস/উইজটে: _SlaSheetState

SLA পলিসি ফর্মের স্টেটে পরিচালনা করে — নাম, প্রায়েরটি, সময়সীমা, বজিনসে-আওয়ার, সক্রিয়তা এবং জমাদান।

`bool get_isEdit`

এডটি মোড কনি।

`void dispose()`

`_name, _description, _notes, _firstResponse, _resolution` কন্ট্রোলার ডিসপোজ করে।

`Future<void> _submit()`

ফর্ম ভ্যালিডেট করে; `SlaPolicyRequest` বানায়, যখন `notes` শুধু ক্রয়িটে মোডে পাঠানো হয় (এডটি `null`, কারণ আপডেট এন্ডপয়েন্ট এটি কখনো পড়ে না); এডটি হলে `updateSlaPolicy + apply`, নাহলে `createSlaPolicy + refresh`; সফল/ব্যর্থ হ্যান্ডলিং।

`String? _hours(String? value)`

ঘণ্টার সংখ্যার ভ্যালিডিটর — পার্স করতে না পারলে বা ১-এর কম হলে এরর মেসেজে ফেরত দেয়, নাহলে `null`।

`Widget build(BuildContext context)`

`FormSheetFrame`-এ নাম, প্রায়োরিটি ড্রপডাউন, প্রথম-সাদা ও সমাধান-সময়ের সংখ্যা ফিল্ড (পাশাপাশি সিরতি, `_hours` ভ্যালিডেশন সহ), বিবরণ, (শুধু ক্রয়িটে মোডে) নোটস ফিল্ড, এবং "Business hours only" ও "In force" সুইচ দেখায়।

উপ-লভেলে ফাংশন: `Future<void> showAuditFilterSheet(BuildContext context)`

`_AuditFilterSheet` মডাল বটম-শিট দেখায়, যা অডিট ট্রাইল সংকীর্ণ করে। ব্যাকএন্ডে প্রতিটি ফিল্টারের নজিস্ব এন্ডপয়েন্ট থাকায় অ্যাকশন-টাইপ ও তারিখ-পরিসর একসাথে ব্যবহার করা যায় না — একটা বাছাই করলে অন্যটা মুছে যায়, এবং শিটটিসিই কথা স্পষ্ট বলে দেয়।

ক্লাস/উইজেট: `_AuditFilterSheet`

অডিট ফিল্টার ফর্ম, `ConsumerStatefulWidget`।

`ConsumerState<_AuditFilterSheet> createState()`

`_AuditFilterSheetState` তৈরি করে।

ক্লাস/উইজেট: `_AuditFilterSheetState`

ফিল্টার ফর্মের স্টেটে — অ্যাকশন-টাইপ টেক্সট এবং শুরু/শেষ তারিখ পর্যালোচনা করে।

`void dispose()`

`_actionType` কন্ট্রোলার ডিসপোজ করে।

`void _apply()`

অ্যাকশন-টাইপ খালি থাকলে সেট দিয়ে ফিল্টার সেট করে; নাহলে দুই তারিখ উভয়ই থাকলে তারিখ-পরিসর দিয়ে সেট করে; নাহলে ফিল্টার ক্লিয়ার করে; তারপর শিট বন্ধ করে।

`Widget build(BuildContext context)`

`FormSheetFrame`-এ অ্যাকশন-টাইপ টেক্সটফিল্ড, ব্যাখ্যামূলক টেক্সট (তারিখ-পরিসর বনাম অ্যাকশন টাইপের পারস্পরিক বর্জন সম্পর্কে), শুরু/শেষ `DateField`, শুধু একটি তারিখ দিয়ে থাকলে সতর্কবার্তা, এবং "Show everything" বাটন (যা সরাসরি ফিল্টার ক্লিয়ার করে শিট বন্ধ করে) দেখায়।

ফাইল: lib/features/workflow/workflow_models.dart

সংক্ষিপ্ত ববিরণ: ওয়ার্কফ্লো টেমপ্লেটে — একটি সার্বভাসি রকি়োয়স্টে য়ে ধারাবাহিক ধাপ দ্বি়ে এগোয় তার মডলে, স্টজে, রকি়োয়স্ট ক্লাস এবং AI-সাজশেন মডলে। এই ফাইলে HTTP কল বা প্রোভাইডার নই, শুধু ডটো মডলে ও পারমশিন কনস্ট্যান্ট।

ক্লাস/উইজটে: WorkflowPermissions

abstract final class, ওয়ার্কফ্লো টেমপ্লেটে পারমশিন স্ট্রিং ধরে রাখে: view (WORKFLOW_VIEW), create (WORKFLOW_CREATE), update (WORKFLOW_UPDATE), delete (WORKFLOW_DELETE)। এটি ইনস্ট্যান্স-বহীন — শুধু স্ট্যাটিক কনস্ট্যান্ট নমেস্পেসে হিসেবে কাজ করে।

ক্লাস/উইজটে: WorkflowTemplate

একটি ওয়ার্কফ্লো টেমপ্লেটে ও তার স্টজেগুলোর মডলে।

const WorkflowTemplate({required this.id, required this.name, required this.version, required this.active, this.description, this.stages = const []})

কনস্ট্রাক্টর; version টেমপ্লেটে বা তার যেকোনো স্টজে যেকোনো এডটি ব্র্যাকেন্ড দ্বারা বাড়ানো হয়, এবং একমাত্র পরবর্তনরে পর দুটো নিহলে অভিন্ন টেমপ্লেটকে আলাদা করে চেনার উপায়।

int? get totalDays

প্রতিটি স্টজে estimatedDays যোগ করে মোট সময় গণনা করে; কোনো স্টজে খালি থাকলে (stages.isEmpty) বা কোনো স্টজে estimatedDays নাল থাকলে পুরো ফলাফল null — কারণ আংশিক যোগফল সত্যিকারের মোট হিসেবে ভুল বার্তা দি়ে।

factory WorkflowTemplate.fromJson(Map<String, dynamic> json)

JSON থেকে তৈরিকরে; stages তালিকা WorkflowStage.fromJson দ্বি়ে ম্যাপ করে stageOrder অনুযায়ী সাজিয়ে (sort) রাখে।

ক্লাস/উইজটে: WorkflowStage

একটি টেমপ্লেটে একটি ধাপের মডলে — নাম, ক্রম, অনুমোদন/পমেন্ট প্রয়োজনীয়তা, আনুমানিক সময়, SLA ও দায়িত্বপ্রাপ্ত রোল বহন করে।

const WorkflowStage({required this.id, required this.name, required this.stageOrder, required this.requiresApproval, required this.requiresPayment, this.description, this.estimatedDays, this.slaHours, this.assigneeRole, this.paymentPercent})

কনস্ট্রাক্টর।

factory WorkflowStage.fromJson(Map<String, dynamic> json)

JSON থেকে অবজেক্ট তৈরিকরে, প্রতিটি সংখ্যাসূচক/বুলিয়ান ফিল্ডে নাল-সফে ডিফল্ট বসিয়ে (stageOrder ডিফল্ট 1)।

ক্লাস/উইজটে: WorkflowTemplateRequest

POST ও PUT /workflows-এর রকি়োয়স্ট বডি। name সার্বভার-সাইডে @NotBlank ও সর্বোচ্চ ১৫০ ক্যারেক্টার; description সর্বোচ্চ ৫০০। আপডটে নাম শর্তহীনভাবে সেটে করে কনিতু ববিরণ নাল-গার্ড করা, তাই খালকিরা ববিরণ মুছে যায় না, রখে দেয়া হয় — ফর্মও এটাই বলে দেয়।

const WorkflowTemplateRequest({required this.name, this.description})

কনস্ট্রাক্টর।

Map<String, dynamic> toJson()

name ট্রিম করে সবসময় পাঠায়; description ট্রিম করে অ-খালি হলে পাঠায়, নাহলে কী-টাই বাদ দিয়ে (নাল পাঠায় না, যহেতু খালি বিবরণ রয়েছে দিয়ে হয়)।

ক্লাস/উইজটে: WorkflowStageRequest

POST ও PUT /workflows/{templateId}/stages-এর রকোয়েস্ট বডি আপডেটে order, estimatedDays, slaHours, উভয় ফ্ল্যাগ, assigneeRole এবং paymentPercent শর্তহীনভাবে সেটে করে — শুধু বিবরণ নাল-গার্ড করা। তাই এডিট করা স্টেজে থেকে সেডি করা সবকিছু প্রতিটি সিনে পাঠানো হয়। ভ্যালিডেশন: name @NotBlank ও সর্বোচ্চ ১০০ ক্যারেক্টার, stageOrder @NotNull ও @Min(1), এবং estimatedDays, slaHours, paymentPercent প্রতিটি দিয়ে হলে @Min(1) — তাই এই তিনটির যেকোনোটি হয় বাস্তব সংখ্যা নয়তো অনুপস্থিতি, কখনো শূন্য নয়। একই টেমপ্লেটের মধ্যে stage order অনন্য হতে হয়; পুনরাবৃত্তি হলে "Stage order N is already taken" দিয়ে প্রত্যাখ্যাত হয়।

```
const WorkflowStageRequest({required this.name, required this.stageOrder, required  
this.requiresApproval, required this.requiresPayment, this.description, this.estimatedDays,  
this.slaHours, this.assigneeRole, this.paymentPercent})
```

কনস্ট্রাক্টর।

```
factory WorkflowStageRequest.from(WorkflowStage stage)
```

বর্তমান WorkflowStage থেকে সব ফিল্ড কপি করে রকোয়েস্ট তৈরি করে।

Map<String, dynamic> toJson()

name ট্রিম করে পাঠায়; stageOrder, requiresApproval, requiresPayment, estimatedDays, slaHours সবসময় (দরকারমতো নাল সহ) পাঠায়; assigneeRole ট্রিম করে খালি হলে নাল; paymentPercent শুধু requiresPayment সত্য হলে পাঠায় (নাহলে নাল, কারণ এটি শুধু পেমেন্ট প্রয়োজন হলেই অর্থবহ এবং সার্ভার-সাইডে @Min(1)); description ট্রিম করে অ-খালি হলে পাঠায়।

ক্লাস/উইজটে: WorkflowSuggestion

AI-সহায়ক একটি নির্দিষ্ট লক্ষ্যের জন্য যা প্রস্তাব করে তার মডেল। কিছুই সংরক্ষিত হয় না — সাজেশন ফরি আসে যাতো কডে এটি থেকে বানাতো পারে; স্ক্রিনি তাই টেমপ্লেটে তৈরি প্রস্তাব দিয়ে এবং স্টেজগুলো ইচ্ছাকৃতভাবে যোগ করার জন্য রয়েছে দিয়ে।

```
const WorkflowSuggestion({required this.name, required this.stages, this.suggestion})
```

কনস্ট্রাক্টর; suggestion মডেলের যুক্তরি এক বাক্যের ব্যাখ্যা, যখন দিয়ে হয়।

```
factory WorkflowSuggestion.fromJson(Map<String, dynamic> json)
```

JSON থেকে তৈরি করে; stages তালিকা SuggestedStage.fromJson দিয়ে ম্যাপ করে।

ক্লাস/উইজটে: SuggestedStage

একটি প্রস্তাবিত স্টেজের মডেল — নাম, অনুমোদন প্রয়োজন কনি, এবং উদ্দেশ্য বহন করে।

```
const SuggestedStage({required this.name, required this.needsApproval, this.purpose})
```

কনস্ট্রাক্টর।

```
factory SuggestedStage.fromJson(Map<String, dynamic> json)
```

JSON থেকে অবজেক্ট তৈরি করে, নাল-সেফ ডিফল্টসহ।

done

ফাইল: lib/features/workflow/workflow_repository.dart

সংক্ষিপ্ত ববিরণ: ওয়ার্কফ্লো টেমপ্লেটে ও তার স্টজে নথি ব্যাকএন্ডের সাথে যোগাযোগ করার রপিওজিটরিক্লাস, এবং সটোকে ঘরিতে থাকা Riverpod প্রোভাইডার ও কন্ট্রোলারগুলো। একটি টেমপ্লেটে হলো কয়েকটি স্টজে ছোট অর্ডারড লিস্ট (নাম, অর্ডার, দুইকেটি ফ্ল্যাগ) — তাই এটি মোবাইলও এডটি করা যায়, যখনে সারভিসের ফর্ম-ফল্ড ডিজাইনার শুধু ওয়বেই সীমাবদ্ধ। ফাইলে অ্যাকটভি/সব টেমপ্লেটে লোড, তরৈ/আপডেট/টগল/মুছে ফলো, স্টজে CRUD, এবং AI সাজেশন — সবকছির API কল রয়েছে।

ক্লাস: WorkflowRepository

`ApiClient` মুড়িয়ে `/workflows` এন্ডপয়ন্টগুলোর সাথে সমস্ত HTTP যোগাযোগ সামলায় — টেমপ্লেটে তালিকাভুক্তি, একক টেমপ্লেটে পড়া, তরৈ/আপডেট/টগল/মোছা, প্রতিটি টেমপ্লেটের স্টজে যোগ/আপডেট/মোছা, এবং একটি লিক্ষ্য (goal) থেকে AI-চালতি স্টজে সাজেশন আনা।

`WorkflowRepository(this._api)`

কনস্ট্রাক্টর — একটি `ApiClient` ইনস্ট্যান্স গ্রহণ করে সংরক্ষণ করে, যা প্রতিটি মিথেডে HTTP কল করতে ব্যবহৃত হয়।

`Future<List<WorkflowTemplate>> activeWorkflows()`

`/workflows/active` থেকে শুধু "ইন ফোর্স" (চালু) থাকা টেমপ্লেটগুলো আনে — যগুলোতে নতুন সারভিস রকি়োস্ট রাখা যায়। বন্ধ (টগল-অফ) করা একটি ওয়ার্কফ্লো তখনও তার মধ্যে চলমান পুরনো রকি়োস্টগুলো নয়ন্তরণ করে, তাই এটি পূর্ণ তালিকার চয়ে সংকীর্ণ এবং আলাদা এন্ডপয়ন্ট। JSON তালিকা থেকে `Map<String, dynamic>` টাইপের আইটেমগুলো ফলিটার করে

`WorkflowTemplate.fromJson` দিয়ে ম্যাপ করে অপরিবর্তনীয় (growable: false) লিস্টে রূপান্তর করে।

`Future<PagedResponse<WorkflowTemplate>> templates({int page = 0, int size = 30})`

`_base (/workflows)` থেকে পেজিনেটেড সব টেমপ্লেটে আনে, `_api.getPaged` ব্যবহার করে

`WorkflowTemplate.fromJson` কে পার্সার হিসেবে পাস করে। ডফিল্টভাবে প্রথম পেজে, ৩০টি আইটেমে আনে।

`Future<WorkflowTemplate> template(int id)`

একটি নির্দিষ্ট আইডি টেমপ্লেটে তার সব স্টজেসহ আনে। তালিকা এন্ডপয়ন্টও স্টজে ফরত দেয়, কিন্তু প্রতিটি স্টজে পরিবর্তনের পরে এটিকে আবার পড়া হয় — কারণ ব্যাকএন্ড প্রতিটি স্টজে পরিবর্তনে টেমপ্লেটের ভার্সন বাড়িয়ে দেয়, এবং নতুন যোগ করা স্টজের আইডিকিবেল সারভারের উত্তর থেকেই জানা যায়।

`Future<WorkflowTemplate> create(WorkflowTemplateRequest request)`

`_base` এ POST করে নতুন একটি ওয়ার্কফ্লো টেমপ্লেটে তরৈ করে এবং সারভারের উত্তর থেকে পার্স করা `WorkflowTemplate` ফরত দেয়।

`Future<WorkflowTemplate> update(int id, WorkflowTemplateRequest request)`

`$_base/$id` এ PUT করে একটি বিদ্যমান টেমপ্লেটে আপডেট করে এবং হালনাগাদ করা টেমপ্লেটে ফরত দেয়।

`Future<WorkflowTemplate> toggle(int id)`

`$_base/$id/toggle` এ PATCH করে একটি টেমপ্লেটের সক্রিয়/নষ্ক্রিয় (active/inactive) অবস্থা পাল্টায় এবং হালনাগাদ টেমপ্লেটে ফরত দেয়।

Future<void> delete(int id)

\$_base/\$id এ deleteText কল করে টেমপ্লেটে মুছে ফেলে। মথেডের নাম delete হলও ভেতরে deleteText ব্যবহৃত হয়েছে কারণ ব্যাকএন্ড এন্ডপয়ন্টটি ResponseEntity<String> ঘোষণা করে এবং JSON নয়, একটি বাক্য দিয়ে উত্তর দেয়।

Future<WorkflowStage> addStage(int templateId, WorkflowStageRequest request)

\$_base/\$templateId/stages এ POST করে একটি নির্দিষ্ট টেমপ্লেটে নতুন স্টেজে যোগ করে এবং তৈরি হওয়া WorkflowStage ফেরত দেয়।

Future<WorkflowStage> updateStage(int templateId, int stageId, WorkflowStageRequest request)

\$_base/\$templateId/stages/\$stageId এ PUT করে একটি স্টেজের সম্পূর্ণ ডটো (পুরো স্টেজে অবজেক্ট, শুধু পরিবর্তিত ফিল্ড নয়) পাঠিয়ে আপডেট করে এবং হালনাগাদ স্টেজে ফেরত দেয়।

Future<void> deleteStage(int templateId, int stageId)

\$_base/\$templateId/stages/\$stageId এ deleteText কল করে একটি স্টেজে মুছে ফেলে।

Future<WorkflowSuggestion> suggest(String goal)

\$_base/suggest এ POST করে একটি ট্রিম করা লক্ষ্য (goal) বাক্য পাঠায় এবং AI-প্রস্তাবিত স্টেজে একটি সিটে ফেরত পায়। এই কলট কিছই সংরক্ষণ (save) করে না — শুধু প্রস্তাব দেয়।

টপ-লেভেলে প্রোভাইডার/ফাংশন

final workflowRepositoryProvider = Provider<WorkflowRepository>(...)

apiClientProvider থেকে ApiClient নিয়ে একটি WorkflowRepository ইনস্ট্যান্স তৈরি করে দেয়।

ক্লাস: WorkflowsController extends AsyncNotifier<List<WorkflowTemplate>>

সব ওয়ার্কফলো টেমপ্লেটের তালিকার অবস্থা পরিচালনা করে — লোড, রিফ্রেশ, এবং একক আইটেম আপডেট/মোছা লোকালি প্রত্যাফলিত করার সুবিধাসহ।

Future<List<WorkflowTemplate>> build()

currentUserProvider ওয়াচ করে (যাতে ব্যবহারকারী বদলালে রিলোড হয়) এবং _load() কল করে প্রাথমিক তালিকা ফেরত দেয়।

Future<List<WorkflowTemplate>> _load()

রপিজিটরি থেকে templates() কল করে প্রথম পজের content ফেরত দেয় (প্রাইভেটে হলেপার)।

Future<void> refresh()

AsyncValue.guard(_load) দিয়ে তালিকা পুনরায় লোড করে state আপডেট করে, ব্যর্থ হলে error state ধরে রাখে।

void apply(WorkflowTemplate updated)

বর্তমান তালিকায় একই আইডির টেমপ্লেটকে updated দিয়ে প্রতিস্থাপন করে — পুরো তালিকা আবার লোড না করাই স্থানীয়ভাবে আপডেট প্রত্যাফলিত করার জন্য। state null থাকলে কিছুই করে না।

void remove(int id)

বর্তমান তালিকা থেকে নির্দিষ্ট আইডির টেমপ্লেটটি বাদ দেয়। state null থাকলে কিছুই করে না।

টপ-লভেলে প্রোভাইডার

```
final workflowsProvider = AsyncNotifierProvider<WorkflowsController,  
List<WorkflowTemplate>> (...)
```

`WorkflowsController` কে অ্যাপের বাকি অংশে ব্যবহারের জন্য উন্মুক্ত করে।

```
final workflowDetailProvider = FutureProvider.autoDispose.family<WorkflowTemplate, int> (...)
```

একটি নির্দিষ্ট আইডির টেমপ্লেটে (তার স্টেজসহ) আনবে এবং প্রতিটি স্টেজে পরিবর্তনের পর পুনরায় পড়া হয়। `autoDispose` ব্যবহৃত হয়েছে কারণ এটি শুধু সংশ্লিষ্ট স্ক্রিনি খোলা থাকা অবস্থাতেই প্রাসঙ্গিক — স্ক্রিনি বন্ধ হওয়া মাত্র এর ধারণ করা ভারসন পুরনো হয়ে যায়।

```
final activeWorkflowsProvider = FutureProvider.autoDispose<List<WorkflowTemplate>> (...)
```

রপিজিটরির `activeWorkflows()` কল করে নতুন রকিওয়েস্ট রাখার যোগ্য ওয়ার্কফ্লোগুলোর তালিকা ফেরত দেয়।

ফাইল: `lib/features/workflow/workflow_screen.dart`

সংক্ষিপ্ত ববিরণ: ওয়ার্কফ্লো টেমপ্লেটের তালিকা এবং একক টেমপ্লেটের স্টেজ-তালিকা দেখানোর দুটি স্ক্রিনি এবং সহায়ক উইজেটে ধারণ করে। টেমপ্লেটে তালিকা মোবাইল-এ এডটি করা যায় কারণ এটি সংক্ষিপ্ত অর্ডারড লিস্ট, তবে প্রতিটি স্টেজের ভেতরের টাইপড ফিল্ড ডিজাইনার ওয়েবেই থাকে।

ক্লাস: `WorkflowScreen` extends `ConsumerStatefulWidget`

ওয়ার্কফ্লো টেমপ্লেটের প্রধান তালিকা স্ক্রিনি — পারমিশন-চকে, "সব/ইন-ফোর্স" ফিল্টার, প্রতিটি সারিতে টগল/এডটি/স্টেজ/মোছার অ্যাকশন প্রদর্শন করে।

```
const WorkflowScreen({super.key})
```

কনস্ট্রাক্টর, কোনো প্যারামিটার প্রয়োজন হয় না।

```
ConsumerState<WorkflowScreen> createState()
```

```
_WorkflowScreenState ইনস্ট্যান্স তৈরি করে (override)।
```

ক্লাস: `_WorkflowScreenState` extends `ConsumerState<WorkflowScreen>`

`WorkflowScreen`-এর স্টেটে — কোন সারি বর্তমানে "busy" (কোনো অ্যাকশন চলছে) তা ট্র্যাক করে এবং টগল/মোছার লজিক পরিচালনা করে।

```
Future<void> _toggle(WorkflowTemplate row)
```

নির্দিষ্ট টেমপ্লেটের সক্রিয়/নিষ্ক্রিয় অবস্থা টগল করে; বাটনটি টগল হওয়ার সময় `_busyId` স্টেট করে UI-তে লোডিং দেখায়। সফল হলে `WorkflowsController.apply` দিয়ে তালিকা আপডেট করে; `ApiException` বা অন্য যেকোনো ব্যর্থতায় সন্ধ্যাকবার দেখায়; শেষে (finally) `_busyId` রিসিটে করে, তবে শুধু widget এখনো mounted থাকলে।

```
Future<void> _delete(WorkflowTemplate row)
```

একটি নিশ্চিতকরণ ডায়ালগ দেখায় ("Never mind" বনাম "Remove") — মুছেলে যে রকিওয়েস্টগুলো ইতিমধ্যে এই ওয়ার্কফ্লোতে চলছে তারা তাদের বর্তমান স্টেজে থেকে যাবে এমন সতর্কবার্তাসহ। নিশ্চিত হলে রপিজিটর থেকে `delete` কল করে, `WorkflowsController.remove` দিয়ে তালিকা থেকে সরায়, এবং সাফল্য/ব্যর্থতার সন্ধ্যাকবার দেখায়।

Widget build(BuildContext context)

পারমশিন লোড না হলে লোডার, `WorkflowPermissions.view` না থাকলে একটি "Not available to you" `EmptyState` দেখায়। অন্যথায় `ConfigList<WorkflowTemplate>` দিয়ে তালিকা রেন্ডার করে — `inForceOnly` ফ্ল্যাগ অনুযায়ী `activeWorkflowsProvider` বা `workflowsProvider` থেকে ডটো নিয়ে, প্রতিটি সারিতে নাম, স্টেজ সংখ্যা, মোট দিন ও ভার্সন দেখায়, এবং শীর্ষে "New" ফ্লোটিং বাটন (তৈরির অনুমতি থাকলে) ও একটি ফিল্টারবার থাকে।

ক্লাস: `WorkflowStagesScreen` extends `ConsumerWidget`

একটি নির্দিষ্ট ওয়ার্কফ্লো টেমপ্লেটে স্টেজগুলো ক্রমানুসারে দেখানোর স্ক্রিনি।

`const WorkflowStagesScreen({super.key, required this.template})`

কনস্ট্রাক্টর — একটি `WorkflowTemplate` (যেটি থেকে স্ক্রিনিটি খোলা হয়েছিল) প্রয়োজন।

`static void open(BuildContext context, {required WorkflowTemplate template})`

`MaterialPageRoute` দিয়ে `WorkflowStagesScreen` পুশ করে নেভিগেটে করে।

Widget build(BuildContext context, WidgetRef ref)

`workflowDetailProvider(template.id)` ওয়াচ করে সাম্প্রতিকি টেমপ্লেটে তথ্য আনবে; নটেওয়ার্ক প্রতিক্রিয়া না আসা পর্যন্ত পাস করা `template` কে ফলব্যাক হিসেবে ব্যবহার করে যাবে। ফাঁকা না দেখায়। পরবর্তী স্টেজে অর্ডার নম্বর (`nextOrder`) হিসাব করে, স্টেজ যোগ করার ফ্লোটিং বাটন দেখায় (অনুমতি থাকলে), এবং `error/empty/তালিকা অবস্থাভেদে` যথাক্রমে `ErrorState`, `EmptyState`, অথবা `ListView.builder` দিয়ে `_StageCard` সারি রেন্ডার করে।

ক্লাস: `_StageCard` extends `StatelessWidget`

একটি একক স্টেজকে অর্ডার-নম্বর বৃত্ত, সংযোগকারী লাইন (রেল), নাম, বিবরণ, ফ্যাক্ট (দিন/SLA/রোল/অনুমোদন/পমেন্ট) এবং এডটি/মোছার পপআপ মেনুসহ প্রদর্শন করে।

`const _StageCard({required this.stage, required this.isLast, this.onEdit, this.onDelete})`

কনস্ট্রাক্টর — স্টেজে ডটো, এটি শেষে স্টেজে কনি (সংযোগ-লাইন আঁকার জন্য), এবং ঐচ্ছিক এডটি/মোছা কলব্যাক নিয়ে।

Widget build(BuildContext context)

স্টেজে বিভিন্ন তথ্য (দিন, SLA ঘণ্টা, নথিকৃত রোল, অনুমোদন প্রয়োজন কনি, পমেন্ট প্রয়োজন কনি ও তার শতাংশ) থেকে একটি `facts` লিস্ট বানিয়ে জোড়া লাগায়, বাম পাশে অর্ডার-নম্বর বৃত্ত ও শেষে না হলে একটি উল্লম্ব সংযোগ-লাইন আঁকে, ডান পাশে `AppCard`-এ নাম/বিবরণ/ফ্যাক্ট এবং `edit/delete` থাকলে একটি `PopupMenuButton` দেখায়।

ক্লাস: `WorkflowScopeController` extends `Notifier<bool>`

তালিকাটি শুধু "ইন ফোর্স" ওয়ার্কফ্লো দেখাচ্ছে কনি তার বুলিয়ান অবস্থা ধরে রাখে।

`bool build()`

প্রাথমিক অবস্থা `false` (অর্থাৎ সব ওয়ার্কফ্লো দেখানো) ফেরত দেয়।

`void set(bool inForceOnly)`

`state` কে দেওয়া মান দিয়ে সেটে করে — `FilterBar`-এ নির্বাচন বদলালে কল হয়।

টপ-লভেলে প্রোভাইডার

`final workflowScopeProvider = NotifierProvider<WorkflowScopeController, bool>(...)`

`WorkflowScopeController` কে অ্যাপারে বাকি অংশে ব্যবহারের জন্য উন্মুক্ত করে।

ফাইল: `lib/features/workflow/workflow_sheets.dart`

সংক্ষিপ্ত বিবরণ: ওয়ার্কশিট টেমপ্লেটে ও স্টেজে তৈরি/এডিট করার জন্য বটম শীট ফর্ম, AI-চালিত সাজেশন ডায়ালগ, এবং একটি স্টেজে মোছার নশ্চতিকরণ ফাংশন ধারণ করে।

টপ-লভেলে ফাংশন

`Future<void> showWorkflowSheet(BuildContext context, {WorkflowTemplate? existing})`

`showModalBottomSheet` দিয়ে `_WorkflowSheet` (নতুন টেমপ্লেটে তৈরি বা `existing` থাকলে এডিটের জন্য) খোলে, `isScrollControlled: true` সহ যাতে কবির্ড খোলার সময় শীট পুরোপুরি স্ক্রল করা যায়।

ক্লাস: `_WorkflowSheet` extends `ConsumerStatefulWidget`

টেমপ্লেটে তৈরি/এডিট করার বটম-শীট উইজটে; নাম ও বিবরণের ফর্ম-ফিল্ড এবং AI-সাজেশন বাটন ধারণ করে।

`const _WorkflowSheet({this.existing})`

কনস্ট্রাক্টর — ঐচ্ছকি `existing` টেমপ্লেটে (এডিট মোডের জন্য) নয়।

`ConsumerState<_WorkflowSheet> createState()`

`_WorkflowSheetState` তৈরি করে (override)।

ক্লাস: `_WorkflowSheetState` extends `ConsumerState<_WorkflowSheet>`

`_WorkflowSheet`-এর স্টেটে — টেক্সট কন্ট্রোলার, সাবমিট/সাজেস্ট-চলাকালীন লোডিং ফ্ল্যাগ এবং error বার্তা পরিচালনা করে।

`bool get _isEdit`

`widget.existing` null না হলে `true` — এই সেশনটি তৈরী নাকি এডিট তা নির্ধারণ করে।

`void dispose()`

`_name` ও `_description` টেক্সট কন্ট্রোলার `dispose` করে, তারপর `super.dispose()` কল করে।

`Future<void> _suggest()`

`_GoalDialog` দেখিয়ে ব্যবহারকারীর কাছ থেকে একটি লক্ষ্য (goal) নেয়; খালি/বাতলি করা হলে কিছু করে না। অন্যথায় রপিজিটর `suggest()` কল করে, প্রাপ্ত সাজেশনের নাম ও প্রস্তাবিত স্টেজগুলোর তালিকা (নাম, উদ্দেশ্য, অনুমোদন লাগবে কিনা) ফর্মের নাম ও বিবরণ ফিল্ডে বসিয়ে দেয়। স্টেজগুলো এখানে সরাসরি তৈরি করা যায় না — এন্ডপয়েন্টটি কিছুই সংরক্ষণ করে না এবং প্রতিটি স্টেজে আলাদা কল — তাই সেগুলো পরে স্ক্রিনি একে একে যোগ করার জন্য শুধু বিবরণে তালিকাভুক্ত করা হয়।

Future<void> _submit()

ইতিমধ্যে সাবমিট চলছে বা ফর্ম ভ্যালিডেশন ব্যর্থ হলে ফিরে যায়। `WorkflowTemplateRequest` বানিয়ে এডটি মোডে `repo.update` (এবং তালিকা/ডাটাইল আপডেট), নতুন মোডে `repo.create` (এবং তালিকা রিফ্রেশ) কল করে। সফল হলে শিট বন্ধ করে একটি স্ন্যাকবার দেখায়; `ApiException` (যেমন ডুপ্লিকেট নাম) বা অন্য ব্যর্থতায় error state স্টেট করে; শেষে সাবমিটিং ফ্ল্যাগ রিসেট করে।

Widget build(BuildContext context)

`FormSheetFrame` ব্যবহার করে একটি ফর্ম বানায় — নাম ফিল্ড (এআই চ্যাট পারমিশন থাকলে ও এডটি মোড না হলে একটি "Draft one from a goal" সাজেশন-আইকন সহ) এবং ববিরণ ফিল্ড। এডটি মোডে একটি সিতরকবারতা দেখায় যে খালি বিবিরণ পুরনোটাকই রেখে দেবে, কারণ এন্ডপয়েন্ট খালি মান উপেক্ষা করে।

ক্লাস: `_GoalDialog` extends `StatefulWidget`

"এটাকী অর্জন করবে?" — ব্যবহারকারীর কাছ থেকে এক লাইনে লক্ষ্য জিজ্ঞাসা করার ডায়ালগ, AI স্টেজে-সাজেশনের ইনপুট হিসেবে।

const `_GoalDialog`()

কনস্ট্রাক্টর, কোনো প্রারামিটার নেই না।

State<`_GoalDialog`> createState()

`_GoalDialogState` তৈরি করে (override)।

ক্লাস: `_GoalDialogState` extends `State<_GoalDialog>`

`_GoalDialog`-এর স্টেটে — একটি টেক্সট কন্ট্রোলার এবং "Draft it" বাটন সক্রিয়/নিস্ক্রিয় করার লজিক ধারণ করে।

void dispose()

`_controller` dispose করে, তারপর `super.dispose()` কল করে।

Widget build(BuildContext context)

একটি `AlertDialog` রেন্ডার করে — ব্যাখ্যামূলক টেক্সট, একটি মাল্টিলাইন `TextField` (উদাহরণ হিট সহ), এবং "Never mind"/"Draft it" বাটন। "Draft it" শুধুমাত্র টেক্সট খালি না থাকলে সক্রিয় থাকে এবং চাপলে টাইপ করা টেক্সটসহ ডায়ালগ পপ করে।

টপ-লেভেলে ফাংশন

Future<void> showStageSheet(BuildContext context, {required int templateId, required int nextOrder, WorkflowStage? existing})

showModalBottomSheet দিয়ে `_StageSheet` (নতুন স্টেজে তৈরি বা `existing` থাকলে এডটিরে জন্ম) খোলে।

ক্লাস: `_StageSheet` extends `ConsumerStatefulWidget`

একটি স্টেজে তৈরি/এডটি করার বটম-শিট — নাম, অবস্থান (order), দিন, SLA, নথিকৃত রোল, ববিরণ, অনুমোদন/পমেন্ট সুইচ ও পমেন্ট শতাংশ ফিল্ড ধারণ করে।

const `_StageSheet`({required this.templateId, required this.nextOrder, this.existing})

কনস্ট্রাক্টর — টেমপ্লেটে আইডি, পরবর্তী উপলব্ধ অর্ডার-নম্বর (নতুন স্টেজে যাত্রে বদ্যমান কোনো অর্ডারের সাথে সংঘর্ষ না করে), এবং ঐচ্ছক `existing` স্টেজে (এডটি মোডের জন্ম) নেই।

ConsumerState<_StageSheet> createState()

_StageSheetState তরৈকিরে (override)।

ক্লাস: _StageSheetState extends ConsumerState<_StageSheet>

_StageSheet-এর স্টেটে — সাতটি টিক্সট কন্ট্রোলার, দুটি সুইচ বুলিয়ান, সাবমিটিং ফ্ল্যাগ ও error বার্তা পরচালনা করে।

bool get _isEdit

widget.existing null না হলে true।

void dispose()

_name, _description, _order, _days, _sla, _role, _percent — সাতটি কন্ট্রোলার dispose করে, তারপর super.dispose() কল করে।

Future<void> _submit()

ইতিমধ্যে সাবমিট চলছে বা ফর্ম ভ্যালিডেশন ব্যর্থ হলে ফিরে যায়। ফর্মের ফিল্ডগুলো থেকে (সংখ্যাসূচক ফিল্ডে int.tryParse ব্যবহার করে) একটি WorkflowStageRequest বানিয়ে এডটি মোডে repo.updateStage, নতুন মোডে repo.addStage কল করে। সাফল্যে workflowDetailProvider ইনভ্যালিডেট করে (প্যাচ না করে পুনরায় পড়ে — কারণ প্রতিটি স্টেজে পরিবর্তন টেমপ্লেটে ভাঙ্গন বাড়ে এবং নতুন স্টেজে আইডিসার্ভারের উত্তর থেকেই জানা যায়) এবং তালিকা রিফ্রেশ করে, শিট বন্ধ করে সন্ধ্যাকবার দেখায়। ব্যর্থতায় ApiException বার্তা (যেমন "Stage order N is already taken") বা জনৈকি error দেখায়।

String? _atLeastOne(String? value)

দিন/SLA/পমেন্ট-শতাংশ ফিল্ডের যৌথ ভ্যালিডেটর — খালি হলে বৈধ (না-স্টে অর্থ), সংখ্যা না হলে "A whole number." এবং ১-এর কম হলে "At least one, or leave it empty." error দেখায়। ব্যাকএন্ডে এই তিনটি ঐচ্ছিক সংখ্যায় @Min(1) থাকায় শূন্য প্রত্যাখ্যাত হয়, তাই "স্টে নয়" বোঝাতে ফিল্ড খালি রাখাই একমাত্র উপায়।

Widget build(BuildContext context)

FormSheetFrame দিয়ে ফর্ম বানায় — নাম, অবস্থান (পজিশন-ভ্যালিডেশনসহ), দিন/SLA (পাশাপাশি দুই কলামে), নিযুক্ত রোল, বিবরণ, "Needs approval" ও "Needs payment" SwitchListTile, এবং পমেন্ট প্রয়োজন হলে শর্তসাপেক্ষে পমেন্ট-শতাংশ ফিল্ড দেখায়।

টপ-লেভেলে ফাংশন

Future<void> deleteStage(BuildContext context, WidgetRef ref, {required int templateId, required WorkflowStage stage})

একটি স্টেজে মুছে ফেলার নিশ্চিতকরণ ডায়ালগ দেখায় (ইতিমধ্যে চলমান রকি়োস্টগুলো তাদের বর্তমান স্টেজেই থাকবে এই সতর্কবার্তাসহ)। নিশ্চিতি হলে রপি়োজিটরির deleteStage কল করে, workflowDetailProvider ইনভ্যালিডেট করে, তালিকা রিফ্রেশ করে এবং সাফল্য/ব্যর্থতার সন্ধ্যাকবার দেখায়।

ফাইল: lib/main.dart

সংক্ষিপ্ত ববিরণ: অ্যাপলিকেশনের এন্ট্রি পয়েন্ট — Firebase (শুধু Android-এ), `SharedPreferences`, এবং ব্যাকগ্রাউন্ড পুশ-মসেজে হ্যান্ডলার স্টেআপ করার পর `ZuhooApp` কে `ProviderScope`-এর ভেতরে রান করে।

টপ-লেভেলে ফাংশন

`Future<void> main()`

`WidgetsFlutterBinding.ensureInitialized()` কল করে ফলাটার বাইন্ডিং প্রস্তুত করে। কোল্ড-স্টার্ট দ্রুততর করতে `Firebase.initializeApp()` (শুধু non-web Android-এ) এবং `SharedPreferences.getInstance()` — এই দুটি `Future.wait` দিয়ে সমান্তরালে চালায়। Android হলো

`FirebaseMessaging.onBackgroundMessage` রজিস্টার করে — এটি `runApp`-এর আগে টপ-লেভেলেই করতে হয়, কারণ প্ল্যাটফর্ম UI ছাড়াই আসা মসেজের জন্য একটি ফ্রেশে আইসোলটে রলিঞ্চ করতে এই রফোরেন্স পড়ে। শেষে প্রাপ্ত `SharedPreferences` ইনস্ট্যান্স দিয়ে `sharedPreferencesProvider` ওভাররাইড করে `ProviderScope`-এর ভেতরে `ZuhooApp` রান করে।

ফাইল: lib/shared/paged_controller.dart

সংক্ষিপ্ত ববিরণ: স্প্রিং ব্যাকএন্ডের `Page<T>` রটাইন করা যেকোনো এন্ডপয়েন্টের জন্য পুনঃব্যবহারযোগ্য "লোড/রিফ্রেশ/আরও লোড করো" লজিক — যাতে প্রতিটি তালিকা স্ক্রিনিং একই বেশি কুড়িলাইন কোড বারবার লিখে সূক্ষ্ম এজ-কসে ভুল করতে না হয়। মূল দুটি নিয়ম: একটি বিয়র্থ "load more" স্ক্রিনিং থাকা পুরনো পজেগুলো মুছে ফেলেবে না, এবং একটরি লোড চলাকালীন দ্বিতীয়বার ট্যাপ করলে কছুই হবে না।

ক্লাস: `PagedState<T>` (@immutable)

একটি পিজেনিটেডে তালিকার এক স্ক্রিনিংর সমান ডটো এবং সে কোন পজে পর্যন্ত লোড হয়েছে তা ধরে রাখে।

`const PagedState({this.items = const [], this.page = 0, this.totalPages = 0, this.loadingMore = false})`

কনস্ট্রাক্টর — সবগুলো ফিল্ডের ডিফল্ট মান শূন্য/খালি অবস্থা।

`bool get hasMore`

`page + 1 < totalPages` — আরও পজে বাকি আছে কিনা তা জানায়।

`bool get isEmpty`

`items.isEmpty` — তালিকা খালি কিনা তা জানায়।

`PagedState<T> copyWith({List<T>? items, int? page, int? totalPages, bool? loadingMore})`

প্রদত্ত ফিল্ডগুলো (যদি দেওয়া হয়) পরিবর্তন করে একটি নতুন `PagedState` অবজেক্ট তৈরি করে, বাকিগুলো অপরিবর্তিত রাখে।

মক্সিনি: `PagedLoader<T>` on `AsyncNotifier<PagedState<T>>`

যেকোনো পিজেনিটেডে `AsyncNotifier`-এ মিশ্রিত করার জন্য — লোড-ফার্স্ট-পজে, রিফ্রেশ, লোড-মোর, এবং একক আইটেম প্রতস্থাপন/অপসারণের প্রস্তুত মথড দিয়ে।

`Future<PagedResponse<T>> fetchPage(int page)`

একটি অ্যাবস্ট্রাক্ট মেথড — যেকোনো সাবক্লাসকে শুধু এটুকু বাস্তবায়ন করতে হয়: নির্দিষ্ট পজে নম্বরের ডটো কীভাবে আনতে হয়।

`Future<PagedState<T>> loadFirstPage()`

`fetchPage(0)` কল করে প্রথম পজে আনে এবং তার `content`, `currentPage`, `totalPages` দিয়ে একটি নতুন `PagedState<T>` বানিয়ে ফেরত দেয়।

`Future<void> refresh()`

`AsyncValue.guard(loadFirstPage)` দিয়ে প্রথম পজে থেকে পুরো state পুনরায় লোড করে।

`Future<void> loadMore()`

বর্তমান state null, আর কোনো পজে বাকি না থাকলে, অথবা ইতিমধ্যে একটি লোড চলমান থাকলে কিছুই করে না। অন্যথায় `loadingMore: true` স্টেট করে পরের পজে আনে এবং সফল হলে বর্তমান আইটেমের সাথে নতুন আইটেম জুড়ে দেয়, পজে/টোটাল-পজে আপডেট করে ও `loadingMore` false করে। ব্যর্থ হলে স্ক্রিনিং থাকা পুরনো আইটেমগুলো অক্ষত রেখে শুধু `loadingMore` false করে এবং exception পুনরায় ছুড়ে দেয় (rethrow), যাতে কলার নিজের ব্যর্থতা রিপোর্ট করতে পারে অথচ ইতিমধ্যে স্ক্রল করা কয়েকটি পজে হারিয়ে না যায়।

`void replaceItem(bool Function(T) matches, T updated)`

`matches` শর্ত মলে এমন প্রথম আইটেমকে `updated` দিয়ে প্রতিস্থাপন করে — এমন কোনো অ্যাকশনের জন্য যা হালনাগাদ সারাটি ফেরত দেয় এবং পুরো তালিকা রিলোড করার প্রয়োজন হয় না। মিলি না পালে বা state null হলে কিছুই করে না।

`void removeItem(bool Function(T) matches)`

`matches` শর্ত মলে এমন সব আইটেমে তালিকা থেকে বাদ দেয় — যখন একটি অনুমোদন সন্ধিান্ত নেওয়া হলে স্টেট "pending" তালিকা থেকে সরানো। state null হলে কিছুই করে না।

ফাইল: `lib/shared/util/formatters.dart`

সংক্ষিপ্ত ববিরণ: ব্যাকএন্ড যসেব কাঁচা স্ট্রিং/সংখ্যা পাঠায় (yyyy-MM-dd তারিখ, HH:mm:ss সময়, ISO টাইমস্ট্যাম্প — সবই স্ট্রিং হিসেবে, কখনো সংখ্যা নয়) সেগুলোকে বিভিন্ন প্রদর্শনযোগ্য বাংলা/ইংরেজি ফর্ম্যাটে রূপান্তর করার স্ট্যাটিক হেল্পার ক্লাস। প্রতিটি হেল্পার null বা malformed ইনপুটও নরিপদে একটি মান ফেরত দেয়, যাতে একটি খারাপ সাররি কারণে পুরো লিস্ট স্ক্রিনিং ক্র্যাশ না করে।

ক্লাস: `Fmt`

সব মেথড ও ফলিড `static` — ইনস্ট্যান্স তৈরি করা যায় না (`Fmt._()` প্রাইভেট কনস্ট্রাক্টর দিয়ে আটকানো)। তারিখ, সময়, মুদ্রা, শতাংশ, ঘণ্টা/মিনিট, আপেক্ষিক সময়, ও এনাম-লেবেল ফর্ম্যাটিং সরবরাহ করে।

`Fmt._()`

প্রাইভেট কনস্ট্রাক্টর — এই ক্লাসের কোনো ইনস্ট্যান্স তৈরি করা যাবে না, শুধু স্ট্যাটিক মেম্বার ব্যবহারের জন্য।

`static DateTime? parse(String? value)`

`value` null বা খালি হিলে null ফেরত দেয়; অন্যথায় `DateTime.TryParse` দিয়ে পার্স করার চেষ্টা করে (ব্যর্থ হলে null)। বাকসিঁব ফরম্যাটিং মথেডরে ভিত্তি

`static String date(String? value)`

`d MMM yyyy` ফরম্যাটে তারখি ফেরত দেয় (যমেন "5 Jan 2026"); পার্স ব্যর্থ হলে `dash`।

`static String dateShort(String? value)`

`d MMM` ফরম্যাটে সংক্ষিপ্ত তারখি (বছর ছাড়া) ফেরত দেয়; পার্স ব্যর্থ হলে `dash`।

`static String dayDate(String? value)`

`EEE, d MMM` ফরম্যাটে বার-সহ তারখি (যমেন "Wed, 5 Jan") ফেরত দেয়; পার্স ব্যর্থ হলে `dash`।

`static String dateTime(String? value)`

একটি ISO টাইমস্ট্যাম্পকে তারখি ও সময় একসাথে ("5 Jan 2026, 9:05 AM") দেখায় — অডিট ট্রইলরে মতো রকের্ডরে জন্য যখনে "ঠকি কখন" গুরুত্বপূর্ণ; পার্স ব্যর্থ হলে `dash`।

`static String time(String? value)`

শুধু সময়ে অংশ (`h:mm a` ফরম্যাটে) ফেরত দেয়; পার্স ব্যর্থ হলে `dash`।

`static String monthYear(int month, int year)`

`MMMM yyyy` ফরম্যাটে মাস-বছর (যমেন "January 2026") ফেরত দেয়।

`static String monthName(int month)`

শুধু মাসরে নাম (`MMMM`) ফেরত দেয়, ২০০০ সালরে একটি ডিআমি `DateTime` ব্যবহার করে।

`static String clock(String? value)`

`HH:mm:ss` ফরম্যাটে অ্যাটনেডনেস-এন্ডপয়ন্ট স্ট্রিং (যমেন "09:05:00") কে "9:05 AM"-এর মতো দেখায়। : দিয়ে স্পলটি করে ঘণ্টা-মিনিট বরে করে; খালি মান, অপব্যাপ্ত অংশ বা পার্স ব্যর্থতায় `dash` ফেরত দেয়।

`static String wallClockNow([DateTime? now])`

চকে-ইন এন্ডপয়ন্টরে প্রত্যাশতি ওয়াল-ক্লক স্ট্রিং (`HH:mm:ss`, স্থানীয় সময়, UTC বা ইনস্ট্যান্ট নয়) তরৈকিরে — কারণ ব্যাকএন্ড এটকি শফিটরে শুরুর সময়ে (যেটোও একটি স্থানীয় ওয়াল-ক্লক) সাথে তুলনা করে। প্যারামটির না দিলে `DateTime.now()` ব্যবহার করে, প্রতটি অংশ দুই-অঙ্কে প্যাড করে।

`static String isoDate(DateTime date)`

`yyyy-MM-dd` ফরম্যাটে তারখি তরৈকিরে — প্রতটি তারখি প্যারামটির ও পলোড এই আকারই প্রত্যাশা করে।

`static String money(num? value)`

টাকা চহিন (ট) ও দুই দশমকি স্থানসহ মুদ্রা ফরম্যাট করে; `value` null হলে `dash`।

`static String plain(num value)`

একটি সংখ্যাকে গ্রুপিং, মুদ্রা-চহিন বা অহতুক "." ছাড়া বয়ের স্ট্রিং হিসাবে ফেরত দেয় — `money` পড়ার জন্য, কনিতু এটি ফর্ম-ফল্ডে ফরিযি দেওয়ার জন্য (এডিট করার সময়)। `_trim` কল করে।

`static String percent(num? value)`

"12%" বা "12.5%" — ট্রইলিং জরিো ছাড়া একটি শিতাংশ ফেরত দেয়; `value` null হলে `dash`।

`static String hours(num? value)`

"Nh" ফরম্যাটে ঘণ্টা ফরমত দেয়; `value` null হলে `dash`।

`static String minutes(num? value)`

মোট মিনিটকে গোল করে (`round()`) নিয়ে এক ঘণ্টার কম হলে "45 min", তার বেশি হলে "1h 15m" (মিনিট শূন্য হলে শুধু "1h") ফরম্যাটে ফরমত দেয়। শূন্য বা ঋণাত্মক হলে খালি স্ট্রিং ফরমত দেয় (`dash` নয়)।

`static String relative(String? value)`

আপেক্ষিক সময় দেখায় — "2 days ago" / "in 3 days" / "Yesterday" / "Tomorrow" / "Xh ago" / "Xm ago" / "Just now" — নোটিফিকেশন তালিকা ও ছুটির হিসাবের জন্য। পার্থক্য ৭ দিনের বেশি হলে পূর্ণ তারিখে ফরমত দেয় (`_date.format`)। পার্স ব্যর্থ হলে `dash`।

`static String label(String? value)`

ব্যাকএন্ডের `SCREAMING_SNAKE_CASE` এনাম মানকে "Screaming Snake"-এর মতো মানুষের পড়ার উপযোগী শরীফোনা-কসে রূপান্তর করে — `_` দিয়ে স্পলিট করে প্রতিটি শব্দকে প্রথম অক্ষর বড় ও বাকি ছোট করে জোড়া লাগায়। `value` null বা খালি হলে `dash`।

`static String _trim(num value)`

একটি প্রাইভেট হেল্পার — সংখ্যাটি পূর্ণসংখ্যা (রাউন্ড করলে অপরিবর্তিত) হলে দশমিক ছাড়া দেখায়, নয়তো দুই দশমিক স্থান পর্যন্ত (`toStringAsFixed(2)`) দেখায়। `plain` ও `percent` উভয়ই এটি ব্যবহার করে।

ফাইল: `lib/shared/widgets/attachment_picker.dart`

সংক্ষিপ্ত বিবরণ: ক্যামেরা, গ্যালারি বা ডিভাইস স্টোরেজ থেকে একটি অ্যাটাচমেন্ট বেছে নেওয়ার জন্য একটি বিটম শিট ও হেল্পার ফাংশন সরবরাহ করে। ফলাফলকে একটি অভিন্ন `PickedAttachment` আকারে ফিরিয়ে দেয়, যাতে কলার সোর্স নির্বিশেষে একইভাবে কাজ করতে পারে।

ক্লাস: `PickedAttachment`

ক্যামেরা, গ্যালারি বা ফাইল—যেখান থেকেই বেছে নেওয়া হোক না কেন—একটি অ্যাটাচমেন্টের ফাইল-পাথ ও নাম ধরে রাখার জন্য একটি সাধারণ, অপরিবর্তনীয় (`immutable`) ডটো ক্লাস। কনস্ট্রাক্টরে `path` ও `name` দুটোই আবশ্যিক প্যারামিটার হিসেবে নেয়।

`PickedAttachment({required this.path, required this.name})`

`const` কনস্ট্রাক্টর যা `path` (ফাইল সিস্টেমে পাথ) এবং `name` (প্রদর্শনযোগ্য ফাইলনাম) সেট করে দেয়। কোনো লজিক নেই, শুধু ফিল্ড অ্যাসাইনমেন্ট।

enum: `_AttachmentSource`

একটি প্রাইভেট enum যা `camera`, `gallery`, `file`—এই তিনটি সম্ভাব্য উৎসের মধ্যে বিটম শিট থেকে ব্যবহারকারীর পছন্দ কোনটি চিহ্নিত করতে ব্যবহৃত হয়।

টপ-লভেলে ফাংশন: pickAttachment

Future<PickedAttachment?> pickAttachment(BuildContext context) async

একটি মডেল বটম শিট (`_SourceSheet`) দেখায় যখন ব্যবহারকারী ক্যামেরা, গ্যালারি বা ফাইলের মধ্যে বেছে নিতে পারে। শিট থেকে কোনো সোর্স ফেরত না এলে বা কনটেক্সট আনমাউন্ট হলে `null` রিটার্ন করে। সোর্স অনুযায়ী `ImagePicker` বা `FilePicker.pickFile()` কল করে, এবং ব্যবহারকারী কিছু না বাছলে (`image/file null` হলে) আবার `null` ফেরত দেয়; নয়তো ফলাফলকে `PickedAttachment`-এ মুড়ে ফেরত দেয়। ক্যামেরা ও গ্যালারি উভয় ক্ষেত্রেই `imageQuality: 85` দিয়ে ছবিসংকুচতি করা হয়।

ক্লাস/উইজটে: _SourceSheet

প্রাইভেট `StatelessWidget` যা বটম শিটে UI আঁকে—একটি শিরোনাম এবং তিনটি `ListTile` (ছবিতোলা, ছবি বাছাই, ফাইল বাছাই)। প্রতিটি টাইলে ট্যাপ করলে সংশ্লিষ্ট `_AttachmentSource` মান দিয়ে `Navigator.pop` কল হয়।

Widget build(BuildContext context)

থমি থেকে `bos` টোকেনে নিয়ে একটি `SafeArea`-মডানে `Column` তৈরি করে: শীর্ষে "Add an attachment" শিরোনাম, তারপর তিনটি অপশনের `ListTile`, প্রতিটির `onTap`-এ সংশ্লিষ্ট enum মান দিয়ে শিট বন্ধ করা হয়। মথেকে কোনো প্রারামিটার লজিক নেই, শুধু widget tree রিটার্ন করে।

ফাইল: lib/shared/widgets/brand_mark.dart

সংক্ষিপ্ত বিবরণ: Zuhoo-এর প্রকৃত ব্র্যান্ড লোগো (Angular ওয়েব অ্যাপের সঙ্গে অভিন্ন অ্যাসেটে থেকে নেওয়া) স্প্ল্যাশ, লগইন ও ড্রয়ার হডারে দেখানোর জন্য একটি রিইউজবেল উইজটে। থমি (হালকা/গাঢ়) অনুযায়ী সঠিক ব্যাকগ্রাউন্ড ভ্যারিয়েন্ট বেছে নেয় এবং অ্যাসেটে লোড ব্যর্থ হলে ফলব্যাক আইকন দেখায়।

ক্লাস/উইজটে: BrandMark

`StatelessWidget` যা একটি নির্দিষ্ট আকারের (ডিফল্ট ৬৪) গোলাকার-কোণায়ুক্ত লোগো ছবি আঁকে। কনস্ট্রাক্টরে শুধু ঐচ্ছিক `size` প্রারামিটার নেয়।

Widget build(BuildContext context)

থমির `bos.isDark` অনুযায়ী `assets/brand/logo-dark-bg.png` বা `assets/brand/logo-light-bg.png` বেছে নেয়, যাতে হালকা ব্যাকগ্রাউন্ডের লোগো গাঢ় থমি বা উল্টোটা ফ্যাকাশে/গাঢ় বাক্স হিসেবে দৃশ্যমান না হয়। `ClipRRect` দিয়ে কোণা গোল করে (`size * 0.28` ব্যাসার্ধ) এবং `Image.asset` দিয়ে ছবি দেখায় (`BoxFit.cover`)। ছবিলোড ব্যর্থ হলে `errorBuilder` একটি ব্র্যান্ড-রঙের `Container`-এ সাদা `Icons.grid_view_rounded` আইকন দেখায়, যাতে অ্যাসেটে অনুপস্থিতি থাকলেও স্প্ল্যাশ/লগইন স্ক্রিনি ভেঙে না পড়ে।

ফাইল: lib/shared/widgets/config_list.dart

সংক্ষিপ্ত বিবরণ: কনফিগারেশন-ধরনের তালিকা (সেটআপ, ক্যাটালগ ইত্যাদি) স্ক্রিনিগুলোর জন্য সাধারণ চেসেসি—একটি সারি উইজটে, ফিল্টার চপি বার, এবং লোডিং/এরর/খালি/ডিটো অবস্থা পরচালনাকারী তালিকা মডক—সরবরাহ করে।

ক্লাস: RowAction

একটি `ConfigRow`-এর পপআপ মনুতে এডটি ও রটিয়ার/রসিটোরের বাইরে একটি অতিরিক্ত এন্ট্রি বর্ণনা করে। `label`, `onSelected` কলব্যাক এবং ঐচ্ছিক `destructive` ফ্ল্যাগ (`true` হলে বপিদ রঙে আঁকা হয়) ধরে রাখবে।

`RowAction({required this.label, required this.onSelected, this.destructive = false})`

`const` কনস্ট্রাক্টর যা তনিটি ফিল্ড সটে করে; `destructive` ডফিল্টভাবে `false`।

ক্লাস/উইজটে: ConfigRow

`StatelessWidget`—একটি কিনফগারশেন তালকার একক সারি আঁকে: শরিনোম, ঐচ্ছিক সাবটাইটলে, ঐচ্ছিক ট্রাইলিং লবেলে, নষ্টক্রয়ি হলে একটি "Retired" (বা কাস্টম) স্ট্যাটাস চপি, এবং একটি ঐচ্ছিক পপআপ মনু (এডটি/টগল/অতিরিক্ত অ্যাকশন) অথবা `busy` হলে একটি স্পিনার।

`Widget build(BuildContext context)`

`bool` থমি টোকেনে নয়ি, `hasMenu` নষ্টধারণ করে (`onEdit`, `onToggle` বা `actions`-এর যেকোনো একটি থাকলে `true`)। একটি `AppCard`-এর ভতেরে `Row` আঁকে: বাঁ পাশে শরিনোম+সাবটাইটলে+(নষ্টক্রয়ি হলে) `StatusChip`, ডান পাশে `busy` হলে ছোট `CircularProgressIndicator`, নয়তো `hasMenu` হলে `PopupMenuButton<int>`। মনু আইটেমগুলোকে `negative` ইনডেক্স (`-1`=এডটি, `-2`=টগল) ও `non-negative` ইনডেক্স (কাস্টম `actions`) দয়ি ম্যাপ করে সংঘর্ষ এডায়; `onSelected` কলব্যাকে সংশ্লিষ্ট হ্যান্ডলার কল করে।

ক্লাস/উইজটে: FilterBar

`StatelessWidget`—একটি অনুভূমিকভাবে স্ক্রলযোগ্য `ChoiceChip` সারি দেখায়, যখনে একটি বিকল্প সর্বদা নষ্টবাচতি থাকে (প্রথাগতভাবে প্রথমটি `null` মান বহন করে "সব")।

`Widget build(BuildContext context)`

একটি নষ্টদষ্ট-উচ্চতার (`scaledStripHeight(context, 34)`), যা ব্যবহারকারীর টেক্সট-স্কেলে অনুসরণ করে) `SizedBox`-এর ভতের `ListView.separated` তরৈকিরে, প্রতিটি বিকল্পেরে জন্য একটি `ChoiceChip` যার `selected` অবস্থা `option.value == selected` তুলনার ভিত্তিতে নষ্টধারতি এবং ট্যাপে `onSelected(option.value)` কল করে।

ক্লাস/উইজটে: ConfigList<T>

জনেরেকি `StatelessWidget` যা একবারে সম্পূর্ণ লোড হওয়া তালকার (পজেনিটেডে নয়) জন্য `AsyncValue<List<T>>` মোডায়। `RefreshIndicator`, লোডিং/এরর/খালি/ডটো অবস্থা এবং একটি ঐচ্ছিক `header` উইজটে (যা খালি অবস্থাতও দেখানো হয়) সামলায়।

`Widget build(BuildContext context)`

`RefreshIndicator`-এর ভতের `async.when` কল করে: `loading` অবস্থায় `Loader()`, `error` অবস্থায় `ErrorState` (যদি এরর `ApiException` হয় তবে তার বার্তা, নয়তো `errorMessage`) এবং `onRetry`: `onRefresh` দেখায়। `data` অবস্থায়, সারি খালি হলে একটি স্ক্রলযোগ্য `ListView` তরৈকিরে যাত `header` (থাকলে) ও একটি `EmptyState` থাকে, নচি ফ্লোটিং বাটনেরে জন্য জায়গা রাখতে `bottom: 90` প্যাডিং দিয়ে। সারি খালি না হলে `ListView.builder` তরৈকিরে যখনে `header` থাকলে ইনডেক্স ০-এ সটে বসে এবং বাকি ইনডেক্সে `itemBuilder(context, rows[...])` কল হয়।

ফাইল: lib/shared/widgets/date_field.dart

সংক্ষিপ্ত ববিরণ: ফর্মের ভেতরে বসানোর জন্য একটি ট্যাপযোগ্য তারিখ-ইনপুট উইজটে, যা পাশের TextFormField-গুলোর মতোই লবেলে, বর্ডার ও উচ্চতা বজায় রাখে (InputDecorator ব্যবহার করে, কারণ Flutter-এ কোনো নটেভি ফর্ম-স্টাইল ডেই ইনপুট নাই)।

ক্লাস/উইজটে: DateField

StatelessWidget—একটি InputDecorator-স্টাইল ফিল্ড দেখায় যাতে একটি তারিখ বা "Not set"-জাতীয় খালি-টেক্সট থাকে; ট্যাপে showDatePicker খোলে এবং ফলাফলকে onChanged-এর মাধ্যমে ফেরত পাঠায়। clearable true ও মান সেট থাকলে একটি ক্লিয়ার () বাটনও দেখায়।

Future<void> _pick(BuildContext context) async

প্রাইভেট হলো—showDatePicker কল করে, initialDate হিসেবে বর্তমান value (বা এখনকার সময়) এবং firstDate/lastDate হিসেবে ক্যালারের দেওয়া মান বা ডিফল্ট ৫ বছরের রেঞ্জ (আজ থেকে ±৫ বছর) ব্যবহার করে। ব্যবহারকারী একটি তারিখ বেছে নলে onChanged(picked) কল করে।

Widget build(BuildContext context)

একটি InkWell তৈরি করে যা enabled হলে _pick ট্রিগার করে; ভেতরে InputDecorator থাকে যার prefixIcon হলো icon, এবং suffixIcon হলো একটি ক্লিয়ার IconButton—কিন্তু কেবল তখনই যখন value অ-null এবং clearable true (খালি ফিল্ডে স্থায়ী ক্লিয়ার বাটন দেখানো এড়াত)। ভেতরে Text-এ value থাকলে Fmt.date(Fmt.isoDate(value!)) দিয়ে ফর্ম্যাট করা তারিখ, নয়তো emptyText দেখায়, এবং রঙ value-এর উপস্থিতি অনুযায়ী bos.muted/bos.text হয়।

ফাইল: lib/shared/widgets/employee_picker.dart

সংক্ষিপ্ত ববিরণ: কোম্পানি ডিরেক্টরিকে একজন সহকর্মী বেছে নেওয়ার জন্য একটি সার্চযোগ্য বটম শিট (যেমন—ল্যাপটপ হস্তান্তর, লাইসেন্স সী বরাদ্দ, ইন্টারভিউ নির্ধারণের সময়)। People স্ক্রিনিং প্রোভাইডার ব্যবহার না করে সরাসরি directoryRepositoryProvider কল করে, কারণ এখানে এনটাইটলেমেন্টের যুক্তি ভিন্ন।

ক্লাস/উইজটে: EmployeePicker

ConsumerStatefulWidget—একটি title নিয়ে একটি সার্চ-সহ কর্মচারী তালিকা বটম শিট হিসেবে দেখায়।

EmployeePicker({super.key, required this.title})

কনস্ট্রাক্টর, শুধু title সেট করে।

static Future<Person?> show(BuildContext context, {required String title})

showModalBottomSheet<Person> কল করে EmployeePicker উইজটে দেখানোর সুবিধাজনক স্ট্যাটিক হলো (isScrollControlled: true সহ)। ব্যবহারকারীর বাছাই করা Person বা বাতলি হলে null রিটার্ন করে।

ConsumerState<EmployeePicker> createState()

_EmployeePickerState-এর একটি ইনস্ট্যান্স তৈরি করে; স্ট্যান্ডার্ড override।

ক্লাস: `_EmployeePickerState`

`EmployeePicker`-এর প্রাইভেট স্টেট ক্লাস। সার্চ টেক্সট কন্ট্রোলার, একটি ডিবাউন্স টাইমার, লোড হওয়া ফলাফল, লোডিং/এর অবস্থা রাখা।

`void initState()`

`super.initState()` কল করে পর তাৎক্ষণিকভাবে `_load()` কল করে প্রাথমিক তালিকা লোড করে।

`void dispose()`

ডিবাউন্স টাইমার বাতলি করে এবং সার্চ কন্ট্রোলার ডিসপোজ করে, তারপর `super.dispose()` কল করে—মেমরালিকি এড়াতে।

`void _onChanged(String value)`

সার্চ টেক্সটফিল্ডের `onChanged` হ্যান্ডলার। আগের ডিবাউন্স টাইমার বাতলি করে এবং একটি নতুন ৩৫০ মিলিসেকেন্ডের টাইমার সেট করে যা শেষে `_load` কল করে, যাতে প্রতিটি কি-স্ট্রোক নটেওয়ার্ক কল না হয়।

`Future<void> _load() async`

`_loading` true ও `_error` null সেট করে, তারপর `directoryRepositoryProvider`-এর `people()` মেথড কল করে (সার্চ টার্ম ট্রিম করা, `size: 30`)। প্রাক্তন কর্মচারীদের (`!p.isFormer`) ফিল্টার করে বাদ দেয়।

`ApiException` হলে তার `message` দেখায়; অন্য কোনো এরর হলে একটি জেনেরিক বার্তা দেখায়। প্রতিটি `await`-এর পর `mounted` চেক করে অনরিপদ `setState` এড়ায়।

`Widget build(BuildContext context)`

একটি `DraggableScrollableSheet` (initial 0.8, max 0.95) তৈরি করে যার মধ্যে একটি হ্যান্ডলে-বার, `title`, একটি সার্চ `TextField` (`autofocus: true`, `onChanged: _onChanged`), এবং নচি একটি `Builder`-মোড়ানো ফলাফল অংশ থাকে যা `_loading/_error/খালি/ListView.builder`—চারটি অবস্থার একটি দেখায়। প্রতিটি `ListTile`-এ ট্যাপ করলে `Navigator.pop(context, person)` কল করে বাছাই করা ব্যক্তিকে ফিরিয়ে দেয়।

কীবোর্ডের নচি চাপা পড়া এড়াতে পুরো `Column`-কে

`MediaQuery.viewInsetsOf(context).bottom`-সমান প্যাডিং দিয়ে মোড়ানো হয়েছে।

ফাইল: `lib/shared/widgets/form_sheet.dart`

সংক্ষিপ্ত ববিরণ: অ্যাপের সব এডটিং বটম শাটের জন্য অভিন্ন কাঠামো (শিরোনাম, ঐচ্ছকি এরর ব্যানার, ফিল্ডের কলাম, সাবমিট বাটন) সরবরাহ করে, যাতে প্রতিটি ফিচারে আলাদা করে লেখার দরকার না পড়ে।

ক্লাস/উইজটে: `FormSheetFrame`

`StatelessWidget`—একটি `Form`-মোড়ানো, কীবোর্ড-সচতেন, স্ক্রলযোগ্য বটম-শাট ফ্রমে তৈরি করে। শিরোনাম, ঐচ্ছকি এরর `MessageBanner`, কলার-সরবরাহকৃত `children` ফিল্ড এবং একটি `LoadingButton` (সাবমিট) নেয়।

FormSheetFrame({super.key, required this.title, required this.formKey, required this.error, required this.onDismissError, required this.action, required this.submitting, required this.onSubmit, required this.children})

কনস্ট্রাক্টর—সবগুলো প্যারামিটার আবশ্যিক (কেবল কী ছাড়া) এবং সরাসরি ফিল্ডে অ্যাসাইন হয়; কোনো ডিফল্ট মান নেই।

Widget build(BuildContext context)

পুরো কন্টেন্টকে `MediaQuery.viewInsetsOf(context).bottom`-সমান নচিরে প্যাডিং দিয়ে মোড়ায় (কীবোর্ড ওঠার সময় কন্টেন্ট উপরে তোলার জন্য), তারপর একটি `SingleChildScrollView` (ফিল্ড বশে হলে স্ক্রল করার জন্য) এর ভেতর `Form(key: formKey)` ও একটি `Column` তৈরি করে: শিরোনাম টেক্সট, `error non-null` হলে `MessageBanner.error(error!, onDismiss: onDismissError)`, তারপর `children`, এবং সবশেষে একটি `LoadingButton` (`label: action`, `loading: submitting`, `icon: Icons.check_rounded`, `onPressed: onSubmit`)।

done

ফাইল: lib/shared/widgets/paged_list_view.dart

সংক্ষিপ্ত ববিরণ: `PagedState<T>`-কে পুল-টু-রিফ্রেশ ও "load older" ফুটার-সহ একটি লিস্ট হিসেবে রেন্ডার করার জন্য জনৈকি উইজটে। এখানে লোডিং, এরর, খালি এবং ডটো-সহ-আরও-আছে — এই চারটি অবস্থা একই নথিমে আঁকা হয়, যাতে প্রতিটি স্ক্রিনি আলাদা করে নিজের বাস্তবায়ন না করে কোনো একটি অবস্থা ভুল না বানায়। এছাড়া একটি হিল্পার ফাংশন `guardListAction` আছে যা "load older"-এর মতো ব্যর্থ হতে-পারা অ্যাকশনকে পুরো লিস্ট না ভেঙে সন্ধ্যাকবারে রিপোর্ট করে।

ক্লাস/উইজটে: `PagedListView<T>`

`StatelessWidget`, `AsyncValue<PagedState<T>>` নেয় এবং তার ভিত্তিতে চারটি ভিন্ন UI অবস্থা দেখায়: লোডিং (স্কলেটিন লিস্ট), এরর (`ErrorState` + রটাই), খালি (রিফ্রেশযোগ্য `EmptyState`), এবং ডটো-সহ লিস্ট (আইটেমগুলো `itemBuilder` দিয়ে বন্ডি করা হয়, শেষে `hasMore` হলে `Loader` অথবা "Load older" বাটন)। কনস্ট্রাক্টরে `async`, `itemBuilder`, `onRefresh`, `onLoadMore`, `emptyIcon`, `emptyTitle` আবশ্যিক; `emptyMessage`, `errorMessage`, `moreLabel`, `padding`-এর ডিফল্ট মান আছে।

`PagedListView`({super.key, required this.async, required this.itemBuilder, required this.onRefresh, required this.onLoadMore, required this.emptyIcon, required this.emptyTitle, this.emptyMessage, this.errorMessage = 'Could not load this list.', this.moreLabel = 'Load older', this.padding = const EdgeInsets.fromLTRB(16, 16, 16, 90)})

কনস্ট কনস্ট্রাক্টর। `async` হলো লোড অবস্থার `AsyncValue<PagedState<T>>`, `itemBuilder` প্রতিটি আইটেমের উইজটে বানায়, `onRefresh/onLoadMore` যথাক্রমে পুল-টু-রিফ্রেশ ও পরবর্তী পজে আনার `async` কলব্যাক। `emptyIcon/emptyTitle/emptyMessage` খালি অবস্থার UI নির্ধারণ করে, `errorMessage` ফলব্যাক এরর টেক্সট (যদি এররটি `ApiException` না হয়), `moreLabel` "আরও আনুন" বাটনের লবেলে, এবং `padding` লিস্টের চারপাশে প্যাডিং।

Widget build(BuildContext context)

`async.when` দিয়ে তিনটি শাখায় ভাগ করে UI বানায়। `loading` অবস্থায় স্ক্রল-নষিক্রয়ি একটি `ListView`-এ `SkeletonList()` দেখায়, যাতে জায়গাটাই তমিধ্য প্রায় সঠিক আকারে সংরক্ষিত থাকে এবং ডটো এলে স্ক্রিনি লাফায় না। `error` অবস্থায় `ErrorState` দেখায়, বার্তা হিসেবে `ApiException.message` অথবা `errorMessage` ব্যবহার করে, এবং `onRetry` হিসেবে `onRefresh` পাস করে। `data` অবস্থায় `state.isEmpty` হলে `RefreshIndicator`-এর ভেতরে সবসময়-স্ক্রলযোগ্য `ListView`-এ `EmptyState` দেখানো হয় (কারণ খালি তালিকা প্রায়ই সঙ্কীর্ণ না হওয়ার কারণে হয়, তাই পুল-টু-রিফ্রেশে রাখা জরুরি); অন্যথায় `ListView.separated` দিয়ে আইটেমগুলো এবং প্রয়োজনে শেষে একটি লোডিং/"Load older" এন্ট্রি রিন্ডার করা হয়।

টপ-লেভেলে ফাংশন: guardListAction

`Future<void> guardListAction(BuildContext context, Future<void> Function() action)` একটি লস্ট-সম্পর্কিত অ্যাকশন (যেমন `onLoadMore`) চালায় এবং ব্যর্থ হলে পুরো স্ক্রিনি এরর-এ পাল্টে না দিয়ে `ScaffoldMessenger`-এর মাধ্যমে সন্ধ্যাকবারে রিপোর্ট করে। `action` চালানোর আগে `ScaffoldMessenger.of(context)` সংরক্ষণ করা হয় (`async` গ্যাপ পার হওয়ার পরে `context` ব্যবহারের ঝুঁকি এড়াতে)। `ApiException` ধরা পড়লে তার `message` দেখানো হয়, অন্য যেকোনো এক্সসপেশনে একটি জনৈকি "Could not load any more." বার্তা দেখানো হয়।

ফাইল: lib/shared/widgets/primitives.dart

সংক্ষিপ্ত ববিরণ: পুরো অ্যাপ জুড়ে বারবার ব্যবহৃত ছোট উইজেটগুলোর সংগ্রহ — Angular ওয়েব অ্যাপের `Loader`, `EmptyState`, ব্যানার ব্লক এবং Bootstrap স্ট্যাটাস ব্যাজের ফ্লাটার প্রতরূপ। এতে বাটন, ব্যানার, স্ট্যাটাস চপি, সেকশন হেডার, কার্ড, লোডার, খালি/এরর অবস্থা এবং অ্যাকশন অন্তর্ভুক্ত।

টপ-লেভেলে ফাংশন: scaledStripHeight

`double scaledStripHeight(BuildContext context, double base)` একটি নির্দিষ্ট-উচ্চতার হরাইজন্টাল স্ট্রিপি (যেমন ফলিটার চপিরে সারি) সিস্টেমে টেক্সট স্কেলে বড় করা থাকলে যেন লেখোটি ক্লপি না হয়, সজেন্য এর উচ্চতা বৃদ্ধি করে। `MediaQuery.textScalerOf(context).scale(1)` থেকে স্কেলে বরে করে, (`scale - 1`)-কে ০ থেকে ১.৫ এর মধ্যে ক্ল্যাম্প করে এবং `base * 0.5` দিয়ে গুণ করে `base`-এর সাথে যোগ করে চূড়ান্ত উচ্চতা রিটার্ন করে। `home_screen.dart`-এর স্ট্যাট গ্রিডের একই যুক্তি এখানে যেকোনো বেস উচ্চতার জন্য সাধারণীকৃত হয়েছে।

ক্লাস/উইজেট: LoadingButton

`StatelessWidget`, কাজ চলাকালীন লবেলের জায়গায় স্পনির দেখানো এবং আরও ট্যাপ প্রত্যাখ্যান করা একটি ফুল-উইথ সাবমিট বাটন। ওয়েব অ্যাপের `<button [disabled]="loading"> <spinner/> </button>` প্যাটার্নের সমতুল্য।

`LoadingButton({super.key, required this.label, required this.loading, required this.onPressed, this.icon})`

কনস্ট্রাক্টর। `label` বাটনের টেক্সট, `loading true` হলে বাটন নষিক্রয়ি হয়ে স্পনির দেখায়, `onPressed` ট্যাপ কলব্যাক (`nullable`), এবং ঐচ্ছকি `icon` লবেলের আগে দেখানো হয়।

Widget build(BuildContext context)

ElevatedButton তৈরিকরে যার onPressed, loading true থাকলে null (ফলে বাটন নষ্টকরায়), নাহলে onPressed | loading true হলে একটি ২০x২০ সাদা CircularProgressIndicator দেখায়; নাহলে ঐচ্ছকি icon (থাকলে) ও label টেক্সট সহ একটি Row দেখায়।

ক্লাস/উইজটে: MessageBanner

StatelessWidget, চারটি সমিন্টিকি টোনে (danger/success/warning/info) মধ্য একটিতে রঙনি করা মসেজে ব্লক। চারটি প্রায়-অভিন উইজটে বদলে একটিমাত্র উইজটে, কারণ পরবর্তনশীল অংশ শুধু টোনে জোড়া ও আইকন।

MessageBanner._({super.key, required this.message, required this.icon, required this.tone, this.onDismiss})

প্রাইভেট বেসে কনস্ট্রাক্টর, যা নামযুক্ত কনস্ট্রাক্টরগুলো ভেতরে ব্যবহার করে; message, icon, tone, ও ঐচ্ছকি onDismiss সেট করে।

MessageBanner.error(String message, {Key? key, VoidCallback? onDismiss})

BannerTone.danger টোন ও Icons.error_outline_rounded আইকন সহ _ কনস্ট্রাক্টরকে ডেলিগেটে করে একটি এরর ব্যানার বানায়।

MessageBanner.success(String message, {Key? key, VoidCallback? onDismiss})

BannerTone.success টোন ও Icons.check_circle_outline_rounded আইকন সহ সাকসেস ব্যানার বানায়।

MessageBanner.warning(String message, {Key? key, VoidCallback? onDismiss})

BannerTone.warning টোন ও Icons.warning_amber_rounded আইকন সহ ওয়ার্নিং ব্যানার বানায়; কছি ভুল হয়েছে এমন .error-এর বদলে, বিবেচনা করার মতো একটি পরিস্থিতির জন্য ব্যবহার্য।

MessageBanner.info(String message, {Key? key, VoidCallback? onDismiss})

BannerTone.info টোন ও Icons.info_outline_rounded আইকন সহ তথ্যমূলক ব্যানার বানায়।

Widget build(BuildContext context)

tone-এর ভিত্তিতে switch দিয়ে (fg, bg) রঙের জোড়া নির্বাচন করে (Bos.danger/dangerSoft, success/successSoft, warning/warningSoft, info/infoSoft)। একটি রাউন্ডেড, বর্ডারযুক্ত Container-এর ভেতরে আইকন, বার্তা টেক্সট, এবং onDismiss দেওয়া থাকলে একটি ৪৪dp ট্যাপ-বক্সসহ কলোজ IconButton (নাহলে সমান স্পেসিং রাখতে একটি ফাঁকা SizedBox) দেখায়।

enum: BannerTone

{ danger, warning, success, info } — একটি MessageBanner কোন সমিন্টিকি রঙের জোড়ায় আঁকা হবে তা নির্ধারণ করে।

ক্লাস/উইজটে: StatusChip

StatelessWidget, ব্যাকগ্রাউন্ড স্ট্যাটাস enum-এর জন্য একটি রঙনি পলি দেখায়। রঙ আসে BosPalette.statusColors থেকে, যাতো একই স্ট্যাটাস সর্বত্র একইভাবে দেখায়।

StatusChip(this.status, {super.key, this.label, this.dense = false})

কনস্ট্রাক্টর। status হলো ব্যাকগ্রাউন্ড থেকে আসা raw স্ট্যাটাস স্ট্রিং, ঐচ্ছকি label কাস্টম প্রদর্শন-টেক্সট (না দিলে _prettyfy(status) ব্যবহৃত হয়), dense true হলে ছোট প্যাডিং/ফন্ট ব্যবহৃত হয়।

Widget build(BuildContext context)

Theme.of(context).bos.statusColors(status) থেকে রঙ নিয়ে একটি রাউন্ডেডে, বর্ডারযুক্ত Container-এ label ?? _prettify(status) টেক্সট দেখায়; dense অনুযায়ী প্যাডিং ও ফন্ট সাইজ বদলায়।

static String _prettify(String status)

status-কে _ দিয়ে ভাগ করে প্রতটি শব্দে প্রথম অক্ষর বড় হাতের ও বাকটি ছোট হাতের করে স্পেস দিয়ে জোড়া দেয় (যেমন "IN_PROGRESS" "In Progress"), খালি অংশ বাদ দিয়ে।

ক্লাস/উইজটে: SectionHeader

StatelessWidget, ওয়েব অ্যাপের মাল্টি-সেকশন ফর্মের সেকশন হেডারের মতো একটি ছোট আপারকসে হেডিং ও নচি একটি ব্লক লাইন দেখায়।

SectionHeader(this.title, {super.key, this.icon, this.trailing})

কনস্ট্রাক্টর। title হেডিং টেক্সট, ঐচ্ছকি icon টাইটলে আগে, ঐচ্ছকি trailing উইজটে ডান পাশে।

Widget build(BuildContext context)

একটি Row-এ (ঐচ্ছকি icon থাকলে সটে প্রথম), তারপর আপারকসে রূপান্তরিত title (মডিউলে রঙ, বোল্ড, লটোর-স্পেসিং সহ), এবং শেষে ?trailing (null-aware spread) রেন্ডার করে; নচি একটি প্যাডিং রাখা।

ক্লাস/উইজটে: AppCard

StatelessWidget, সর্বত্র কার্ডের বদলে ব্যবহৃত একটি বর্ডারযুক্ত সারফেস। কাঁচা Card-এর বদলে একটি উইজটে হিসেবে রাখা হয়েছে যাত প্যাডিং ও বর্ডার সব স্ক্রিনি জুড়ে অভিন্ন থাকে।

AppCard({super.key, required this.child, this.padding = const EdgeInsets.all(16), this.onTap, this.color})

কনস্ট্রাক্টর। child কার্ডের বিষয়বস্তু padding ডিফল্ট ১৬ চারপাশে, ঐচ্ছকি onTap কার্ডকে ট্যাপযোগ্য বানায়, ঐচ্ছকি color ব্যাকগ্রাউন্ড ওভাররাইড করে (ডিফল্ট bos.bgCard)।

Widget build(BuildContext context)

Material InkWell (onTap-সহ) Ink (রাউন্ডেডে বর্ডার-সহ) প্যাডেডে child — এই স্তরবিন্যাসে কার্ডটি বানায়, যাত রপিল ইফেক্ট ও বর্ডার একসাথে কাজ করে।

ক্লাস/উইজটে: Loader

StatelessWidget, একটি কার্ড বা পুরো পেজের ভেতরে ব্যবহারযোগ্য কন্ড্রীভূত স্পনিার।

Loader({super.key, this.message, this.padding = 32})

কনস্ট্রাক্টর। ঐচ্ছকি message স্পনিারের নচি দেখানো টেক্সট, padding উল্লম্ব প্যাডিং (ডিফল্ট ৩২)।

Widget build(BuildContext context)

একটি Column-এ ২৬x২৬ আকারের ব্র্যান্ড-রঙের CircularProgressIndicator, এবং message থাকলে তার নচি মডিউলে রঙে টেক্সট দেখায়।

ক্লাস/উইজটে: EmptyState

StatelessWidget, লস্টে কিছু না থাকলে যা দেখানো হয় — এরর থেকে আলাদা, কারণ এরর মানে ডটো আনাই যায়না।

`EmptyState({super.key, required this.icon, required this.title, this.message, this.action})`

কনস্ট্রাক্টর। `icon` ও `title` আবশ্যিক, ঐচ্ছিক `message` (বিস্তারিত ব্যাখ্যা) এবং ঐচ্ছিক `action` (যেমন একটি বাটন)।

`Widget build(BuildContext context)`

একটি বৃত্তাকার নডিট্রাল-ব্রাউজিং কন্টইনারে `icon`, তারপর বোল্ড `title`, ঐচ্ছিক `message`, এবং ঐচ্ছিক `action` — সব কন্ট্রোল সাজানো একটি `Column`-এ রেন্ডার করে।

ক্লাস/উইজটে: `ErrorState`

`StatelessWidget`, ফেচ ব্যর্থ হলে দেখানো একটি রিট্রাই-সহ ব্যর্থতা অবস্থা।

`ErrorState({super.key, required this.message, this.onRetry})`

কনস্ট্রাক্টর। `message` আবশ্যিক এরর টেক্সট, ঐচ্ছিক `onRetry` কলব্যাক থাকলে একটি "Try again" বাটন দেখায়।

`Widget build(BuildContext context)`

একটি `Column`-এ ক্লাউড-অফ আইকন, `message` টেক্সট, এবং `onRetry` দেওয়া থাকলে একটি রিফ্রেশ-আইকন সহ `OutlinedButton` (`icon` "Try again") দেখায়।

ক্লাস/উইজটে: `Avatar`

`StatelessWidget`, একটি বৃত্তাকার অ্যাভাটার যা ইনশিয়াল ফলব্যাক করে। ব্যাকগ্রাউন্ড হোস্টের বপিরীতে রজিলভ করা প্রয়োজন এমন প্যাথ থেকে ছবি সার্ভ করে, এবং ভাড়া বা অনুপস্থিতি ছবি হিডারে ধূসর গর্ত রেখে যেতে পারে না।

`Avatar({super.key, required this.initials, this.imageUrl, this.size = 44})`

কনস্ট্রাক্টর। `initials` ফলব্যাক টেক্সট, ঐচ্ছিক `imageUrl`, `size` আবর্তনের ব্যাস (ডিফল্ট ৪৪)।

`Widget build(BuildContext context)`

প্রথমে একটি `fallback` উইজটে (ব্র্যান্ড-সফট ব্যাকগ্রাউন্ডে ইনশিয়াল টেক্সট) বানায়। `imageUrl` null/খালি হলে সরাসরি `fallback` রিটার্ন করে; নাহলে `ClipOval`-এর ভেতরে `Image.network` দেখায়, যার `errorBuilder` ও `loadingBuilder` (progress না-শেষ হওয়া অবস্থায়) উভয়ই `fallback`-এ ফিরে যায়।

ফাইল: `lib/shared/widgets/prompts.dart`

সংক্ষিপ্ত বিবরণ: যেসব অ্যাকশন ফিরিয়ে নেওয়া যায় না, সেগুলোর আগে দেখানো দুটি ডায়ালগ ফাংশন এবং একটি অপশন-পিকার শিট। ডিসমিস করলে সবসময় `null` রিটার্ন হয়, এবং `null` কখনোই "হ্যাঁ" নয়।

টপ-লেভেলে ফাংশন: `confirmAction`

`Future<bool> confirmAction(BuildContext context, {required String title, required String message, required String action, bool destructive = true})` এগিয়ে যাওয়া উচিত কনি জিজ্ঞাসে করে একটি

`AlertDialog` দেখায়, যাকে "Never mind" (false) ও `action`-লেবেলযুক্ত (true) দুটি বাটন থাকে; `destructive` true হলে সম্মত বাটনটি বিপিড-রঙে দেখানো হয়। ডায়ালগ `dismiss` হলে (বাটন ছাড়া) `confirmed` null হয়, যা ?? false-এর মাধ্যমে false-এ রূপান্তরিত হয়ে রিটার্ন হয়। `action` প্রারামিটারে "OK" না লিখে কী ঘটবে তা নাম দেওয়া উচিত (যেমন "Remove"), যাতে উপরের বাক্য না পড়লেও বাটনটি বোধগম্য থাকে।

টপ-লেভেলে ফাংশন: askForText

`Future<String?> askForText(BuildContext context, {required String title, required String message, required String label, required String action, bool required = false, bool destructive = false, String? initialValue})` একটি লাইনরে টেক্সট চ্যেয়ে তারপর এগিয়ে যাওয়া উচিত কনি জিজ্ঞাসে করে।

`TextEditingController` দিয়ে একটি `TextField`-সহ `AlertDialog` দেখায়; `StatefulBuilder` দিয়ে টেক্সট পরিবর্তনের সাথে `filled` অবস্থা ট্র্যাক করা হয়, যাতে `required` `true` ও ফিল্ড খালি থাকলে সম্মতি বাটন নষ্টকর য়ি থাকে। ডায়ালগ `dismiss` হলে `null` রটার্ন হয় (বাতলি), আর সম্মতি দিলে টাইপ করা টেক্সট (এমনকি খালি স্ট্রিং, যখন `required` `false`) রটার্ন হয় — এই দুটি ফলাফল ইচ্ছাকৃতভাবে আলাদা রাখা হয়েছে। শেষে `controller.dispose()` কল করা হয়।

টপ-লেভেলে ফাংশন: pickOne

`Future<String?> pickOne(BuildContext context, {required List<({String value, String label})> options, String? current})` অল্প কয়েকটি বিকল্প থেকে একটি মান বেছে নেওয়ার জন্য ড্রপডাউনের বদলে একটি `showModalBottomSheet` দেখায়। প্রতিটি `option`-এর জন্য একটি `ListTile` বানায়, যার `title` হলো `option.label` এবং `option.value == current` হলে একটি চেক-আইকন `trailing`-এ দেখানো হয়; ট্যাপ করলে `option.value` দিয়ে শিট বন্ধ হয়ে রটার্ন হয়। স্ট্যাটাস পরিবর্তনের মতো জায়গায় ব্যবহারযোগ্য, যেখানে বিকল্প কম, প্রতিটির একটি পাঠযোগ্য লবেলে প্রয়োজন, এবং বর্তমান মানটি দৃশ্যমানভাবে বর্তমান হিসেবে দেখানো দরকার।

ফাইল: lib/shared/widgets/search_field.dart

সংক্ষিপ্ত বিবরণ: একটি লিস্টের উপরে বসা সার্চ বক্স। ডিবাউন্সড, কারণ এই অ্যাপ যা কিছু সার্চ করে তা সার্ভার রাউন্ড-ট্রপি, এবং প্রতিটি কি-স্ট্রোককে একটি করে রকিওয়েস্ট পাঠালে অনেকগুলো স্টলে রকিওয়েস্ট তৈরি হয় যা ক্রম এলোমেলো করে ভুল ফলাফল দেখাতে পারে। ক্লিয়ার বাটন ফিল্ডেরই অংশ, যাতে ফোনে (যেখানে `Escape` কী নেই) আনফিল্টার্ড লিস্টে ফরোর কোনো পথ না থাকা একটি ফাঁদ না হয়ে যায়।

ক্লাস/উইজটে: AppSearchField

`StatefulWidget`, একটি ডিবাউন্সড, ক্লিয়ারযোগ্য সার্চ `TextField`। `onChanged` কলব্যাক ডিবাউন্স পার হওয়ার পরে ট্রিম করা ক্যোরেসিহ ডাকা হয়, এবং ফিল্ড ক্লিয়ার হলে তাৎক্ষণিকভাবে খালি স্ট্রিং সহ ডাকা হয়।

`AppSearchField({super.key, required this.onChanged, this.hint = 'Search', this.initial, this.debounce = const Duration(milliseconds: 350), this.autofocus = false})`

কনস্ট্রাক্টর। `onChanged` আবশ্যিক কলব্যাক, `hint` প্লসেহোল্ডার টেক্সট, `initial` প্রি-ফিলি করা মান, `debounce` ডিলি ডিউরেশন (ডিফল্ট ৩৫০ms), `autofocus` ফিল্ড লোড হওয়ার সাথে সাথে ফোকাস করবে কনি।

`State<AppSearchField> createState()`

`_AppSearchFieldState`-এর একটি ইনস্ট্যান্স রটার্ন করে।

ক্লাস: _AppSearchFieldState

`State<AppSearchField>`-এর প্রাইভেটে বাস্তবায়ন, যা টেক্সট কন্ট্রোলার, ডিবাউন্স টাইমার এবং ক্লিয়ার-বাটন দৃশ্যমানতার অবস্থা পরিচালনা করে।

`void initState()`

`super.initState()` কল করার পর `_hasText`-কে `(widget.initial ?? "").isEmpty` দিয়ে ইনশিয়ালাইজ করে, যাতে প্র-ফলি করা মান থাকলে ক্লিয়ার বাটন শুরু থেকেই দৃশ্যমান হয়।

`void dispose()`

`_timer` ক্যান্সলে করে এবং `_controller.dispose()` কল করে, তারপর `super.dispose()` ডাকে — মমেরলিকি ও স্টেটে-আফটার-ডিসপোজ এরর এড়াতে।

`void _onChanged(String raw)`

প্রতটি কিস্ট্রোকে ডাকা হয়। `raw.isEmpty`-এর ভিত্তিতে `_hasText` পরিবর্তিত হলেই `setState` কল করে (যাতে প্রতটি কিস্ট্রোকে অহতুক রবিল্ড না হয়)। বদ্যমান `_timer` ক্যান্সলে করে একটি নতুন `Timer` শুরু করে যা `widget.debounce` পরে `widget.onChanged(raw.trim())` কল করবে।

`void _clear()`

টাইমার ক্যান্সলে করে, কন্ট্রোলার খালি করে, `_hasText`-কে `false` সেট করে এবং তাৎক্ষণিকভাবে (ডেবিউন্স ছাড়াই) `widget.onChanged("")` কল করে — কারণ ক্লিয়ার করা একটি স্পষ্ট কাজ, যা অপেক্ষা করার দরকার নেই।

`Widget build(BuildContext context)`

একটি `TextField` বানায় যার `prefixIcon` সার্চ-আইকন, এবং `_hasText` `true` হলে `suffixIcon`-এ একটি ক্লিয়ার `IconButton` (৪০x৪০ ট্যাপ-বক্স সহ) দেখায়। `onChanged` `_onChanged`; `onSubmitted`-এ পনুডিং টাইমার ক্যান্সলে করে সরাসরি `widget.onChanged(value.trim())` কল করে (সাবমিট করলে বাকি ডেবিউন্স অপেক্ষা করতে হয় না)। ডেকোরেশনে `bos` থিমি টোকনে ব্যবহার করে বর্ডার, ফলি-কালার ইত্যাদি সেট করে।

ফাইল: `lib/shared/widgets/skeleton.dart`

সংক্ষিপ্ত বিবরণ: যে কন্টেন্ট আসছে তার আকারের প্লসেহোল্ডার ব্লক। স্পনিার বলে "কিছু একটা ঘটছে"; স্কেলটিন বলে "একটি লিস্ট আসছে, এবং প্রতটি সারা প্রায় এখানে বসবে" — যা ডটো আসার সময় স্ক্রিনি লাফানো বন্ধ করে। কোনো শিমির প্যাকজে ব্যবহার করা হয়নি; একটি `AnimationController` দিয়ে গ্র্যাডিয়েন্ট স্লাইড করানো হয়েছে।

ক্লাস/উইজেট: `Skeleton`

`StatefulWidget`, একটি একক প্লসেহোল্ডার ব্লক (লাইন বা বৃত্ত), যার উপর দিয়ে একটি হালকা হাইলাইট গ্র্যাডিয়েন্ট বাম থেকে ডানে স্লাইড করে, যদি না ব্যবহারকারী অ্যানিমেশন নষ্টক্রয়ি করে রেখে থাকেন।

`Skeleton({super.key, this.width, this.height = 14, this.radius = 6})`

কনস্ট্রাক্টর। `width` `null` মান প্যারেন্টের পুরো প্রস্থ; নর্দিষ্ট মান দলে (যেমন লবেলের মতো) একটি সংক্ষিপ্ত লাইন বানানো যায়। `height` ও `radius`-এর ডফিল্ট মান যথাক্রমে ১৪ ও ৬।

`factory Skeleton.circle({Key? key, double size = 40})`

`width`, `height` উভয়ই `size` এবং `radius`-কে `size / 2` সেট করে একটি বৃত্তাকার `Skeleton` বানায় — অ্যাভাটারের জায়গায় ব্যবহারের জন্য।

`State<Skeleton> createState()`

`_SkeletonState`-এর একটি ইনস্ট্যান্স রটার্ন করে।

ক্লাস: _SkeletonState

State<Skeleton> with SingleTickerProviderStateMixin — অ্যানিমেশন কন্ট্রোলার পরচালনা করে এবং MediaQuery.disableAnimationsOf অনুযায়ী চালু/বন্ধ করে।

void didChangeDependencies()

super.didChangeDependencies() কল করার পর MediaQuery.disableAnimationsOf(context) থেকে still পড়ে। যদি অবস্থা অপরিবর্তিত থাকে ও কন্ট্রোলারের চলমান অবস্থা ইতিমধ্যে সঠিক হয়, তবে কিছু করে না। নাহলে _still আপডেট করে এবং still হলে _controller.stop(), নাহলে (এখনো অ্যানিমিটে না করলে) _controller.repeat() কল করে। এটি গুরুত্বপূর্ণ কারণ একটি চলমান কন্ট্রোলার প্রতিটিকে একটি ফ্রেমে শডিউল করে, তাই থামানো না হলে পুরো অ্যাপ 60fps-এ রেন্ডার হতে থাকবে এমন কিছু অ্যানিমিটে করার জন্য যা ইচ্ছাকৃতভাবে স্থির রাখা হয়েছে।

void dispose()

_controller.dispose() কল করে, তারপর super.dispose()।

Widget build(BuildContext context)

থিম থেকে base/highlight রঙ নির্বাচন করে (ডার্ক মোডে ভিন্নি জোড়া)। box নামে একটি রাউন্ডে DecoratedBox বানায় নির্দিষ্ট width/height সহ। _still true হলে সরাসরি box রিটার্ন করে (কোনো অ্যানিমেশন ছাড়া)। নাহলে AnimatedBuilder দিয়ে _controller.value-এর ভিত্তিতে একটি LinearGradient-চালিত ShaderMask দিয়ে box-এর উপর একটি হালকা ব্যান্ড বাম প্রান্তের বাইরে থেকে ডান প্রান্তের বাইরে পর্যন্ত স্লাইড করায়, যাতে ব্লকটুকখানো মাঝ-হাইলাইটে শুরু বা শেষ না হয়।

ক্লাস/উইজটে: SkeletonList

StatelessWidget, এই অ্যাপের কার্ড-লিস্টের আকারের একগুচ্ছ Skeleton সারি (টাইটলে লাইন, ছোট সাবটাইটলে, ঐচ্ছিক লডিং অ্যাভিটার)। সারিগুলো ইচ্ছাকৃতভাবে ভিন্নি প্রস্থের, যাতে চোখ এটিকে "কন্টেন্ট আসছে" হিসেবে চিনতে পারে, একটি প্রোগ্রেস-এলমিন্টের মতো না দেখায়।

SkeletonList((super.key, this.rows = 5, this.hasLeading = true, this.padding = const EdgeInsets.all(16)))

কনস্ট্রাক্টর। rows কতগুলো প্লসেহোল্ডার-কার্ড দেখানো হবে (ডিফল্ট ৫), hasLeading প্রতিটি কার্ডে একটি বৃত্তাকার লডিং স্কলেটিন থাকবে কিনা, padding প্রতিটি কার্ডের ভেতরের প্যাডিং।

Widget build(BuildContext context)

widths নামে একটি স্থির (const) তালিকা [0.55, 0.42, 0.62, 0.38, 0.5] সংজ্ঞায়িত করে — এলোমেলো না করে সাইকেলে করা হয়, যাতে রবিলিডে প্লসেহোল্ডারের প্রস্থ পাল্টে না যায়। rows সংখ্যক বর্ডারযুক্ত Container বানায়, প্রতিটিতে ঐচ্ছিক Skeleton.circle (হাসপাতাল) এবং একটি Column-এ দুটি FractionallySizedBox-মোড়া Skeleton লাইন (widths[i % widths.length] ও widths[(i + 2) % widths.length] * 0.8 প্রস্থের) থাকে।

ফাইল: lib/shared/widgets/stat_card.dart

সংক্ষিপ্ত ববিরণ: একক হডেলাইন সংখ্যা দেখানোর কার্ড, যমেন হোম স্ক্রিনিরে স্ট্যাট গ্রুডিে ব্যবহৃত হয়। এর মূল বশেষিট্য হলো value nullable এবং null সবসময় এম-ড্যাশ হিসেবে রেন্ডার হয়, কখনো শূন্য হিসেবে নয়।

ক্লাস/উইজটে: StatCard

`StatelessWidget`, একটি আইকন-টাইল, একটি মান (ঐচ্ছিক প্রত্যয়সহ) এবং একটি লিবেলে প্রদর্শন করে। "আমাদের কাছে এই রডিং নই" এবং "রডিং শূন্য" ভিন্ন অর্থ বহন করে (যেমন কারো নজিরে হাজারি বা ছুটির ব্যালেন্স দেখার সময়), তাই `null`-কে চুপচাপ 0-এ রূপান্তর করা ভাল তথ্য দিয়ে — এই উইজটে সেই পার্থক্য বজায় রাখে।

`StatCard({super.key, required this.label, required this.value, required this.icon, this.suffix, this.tone, this.onTap})`

কনস্ট্রাক্টর। `label` ও `icon` আবশ্যিক, `value` nullable (null হলে `em dash` দেখাবে), ঐচ্ছিক `suffix` (যেমন একক প্রত্যয়, শুধুমাত্র `value` non-null হলে দেখানো হয়), ঐচ্ছিক `tone` আইকন-টাইলের অ্যাকসেন্ট রঙ (ডিফল্ট ব্র্যান্ড), ঐচ্ছিক `onTap` কার্ডকে ট্যাপযোগ্য বানায়।

`Widget build(BuildContext context)`

`accent`-কে `tone ?? bos.brandInk` হিসেবে নির্ধারণ করে, `hasValue`-কে `value != null` হিসেবে গণনা করে।

`Material InkWell` (`onTap`-সহ) `Ink` (বর্ডারযুক্ত) কার্ডামের ভেতরে একটি `Column` রেন্ডার করে:

প্রথমে একটি রাউন্ডেডে অ্যাকসেন্ট-ব্যাকগ্রাউন্ড আইকন-টাইল, তারপর `value ?? Fmt.dash` (এক লাইন, ইলিপসিসি-সহ, non-null হলে সাধারণ টেক্সট-রঙ নাহলে মডিটেডে) ও পাশে `suffix` (যদি `hasValue` ও `suffix != null`), এবং সবশেষে সর্বোচ্চ দুই লাইনের `label` টেক্সট মডিটেডে রঙে।

`done`